

IA-32 Intel® Architecture Software Developer's Manual

Volume 3: System Programming Guide

NOTE: The *IA-32 Intel Architecture Developer's Manual* consists of three books: *Basic Architecture*, Order Number 245470-012; *Instruction Set Reference Manual*, Order Number 245471-012; and the *System Programming Guide*, Order Number 245472-012. Please refer to all three volumes when evaluating your design needs.

INFORMATION IN THIS DOCUMENT IS PROVIDED IN CONNECTION WITH INTEL PRODUCTS. NO LICENSE, EXPRESS OR IMPLIED, BY ESTOPPEL OR OTHERWISE, TO ANY INTELLECTUAL PROPERTY RIGHTS IS GRANTED BY THIS DOCUMENT. EXCEPT AS PROVIDED IN INTEL'S TERMS AND CONDITIONS OF SALE FOR SUCH PRODUCTS, INTEL ASSUMES NO LIABILITY WHATSOEVER, AND INTEL DISCLAIMS ANY EXPRESS OR IMPLIED WARRANTY, RELATING TO SALE AND/OR USE OF INTEL PRODUCTS INCLUDING LIABILITY OR WARRANTIES RELATING TO FITNESS FOR A PARTICULAR PURPOSE, MERCHANTABILITY, OR INFRINGEMENT OF ANY PATENT, COPYRIGHT OR OTHER INTELLECTUAL PROPERTY RIGHT. INTEL PRODUCTS ARE NOT INTENDED FOR USE IN MEDICAL, LIFE SAVING, OR LIFE SUSTAINING APPLICATIONS.

Intel may make changes to specifications and product descriptions at any time, without notice.

Designers must not rely on the absence or characteristics of any features or instructions marked "reserved" or "undefined." Intel reserves these for future definition and shall have no responsibility whatsoever for conflicts or incompatibilities arising from future changes to them.

The Intel® IA-32 architecture processors (e.g., Pentium® 4 and Pentium III processors) may contain design defects or errors known as errata. Current characterized errata are available on request.

Intel, Intel386, Intel486, Pentium, Intel Xeon, Intel NetBurst, Intel SpeedStep, MMX, Celeron, and Itanium are trademarks or registered trademarks of Intel Corporation or its subsidiaries in the United States and other countries.

Contact your local Intel sales office or your distributor to obtain the latest specifications and before placing your product order.

Copies of documents which have an ordering number and are referenced in this document, or other Intel literature, may be obtained from:

Intel Corporation
P.O. Box 5937
Denver, CO 80217-9808

or call 1-800-548-4725
or visit Intel's website at <http://www.intel.com>

Copyright © 1997 - 2003 Intel Corporation

CONTENTS

PAGE

CHAPTER 1

ABOUT THIS MANUAL

1.1.	IA-32 PROCESSORS COVERED IN THIS MANUAL	1-1
1.2.	OVERVIEW OF THE <i>IA-32 INTEL ARCHITECTURE SOFTWARE DEVELOPER'S MANUAL, VOLUME 3: SYSTEM PROGRAMMING GUIDE</i>	1-2
1.3.	NOTATIONAL CONVENTIONS	1-4
1.3.1.	Bit and Byte Order	1-4
1.3.2.	Reserved Bits and Software Compatibility	1-4
1.3.3.	Instruction Operands	1-5
1.3.4.	Hexadecimal and Binary Numbers	1-6
1.3.5.	Segmented Addressing	1-6
1.3.6.	Exceptions	1-6
1.4.	RELATED LITERATURE	1-7

CHAPTER 2

SYSTEM ARCHITECTURE OVERVIEW

2.1.	OVERVIEW OF THE SYSTEM-LEVEL ARCHITECTURE	2-1
2.1.1.	Global and Local Descriptor Tables	2-3
2.1.2.	System Segments, Segment Descriptors, and Gates	2-3
2.1.3.	Task-State Segments and Task Gates	2-4
2.1.4.	Interrupt and Exception Handling	2-4
2.1.5.	Memory Management	2-5
2.1.6.	System Registers	2-5
2.1.7.	Other System Resources	2-6
2.2.	MODES OF OPERATION	2-6
2.3.	SYSTEM FLAGS AND FIELDS IN THE EFLAGS REGISTER	2-7
2.4.	MEMORY-MANAGEMENT REGISTERS	2-10
2.4.1.	Global Descriptor Table Register (GDTR)	2-10
2.4.2.	Local Descriptor Table Register (LDTR)	2-11
2.4.3.	IDTR Interrupt Descriptor Table Register	2-11
2.4.4.	Task Register (TR)	2-11
2.5.	CONTROL REGISTERS	2-12
2.5.1.	CPUID Qualification of Control Register Flags	2-18
2.6.	SYSTEM INSTRUCTION SUMMARY	2-18
2.6.1.	Loading and Storing System Registers	2-20
2.6.2.	Verifying of Access Privileges	2-21
2.6.3.	Loading and Storing Debug Registers	2-21
2.6.4.	Invalidating Caches and TLBs	2-21
2.6.5.	Controlling the Processor	2-22
2.6.6.	Reading Performance-Monitoring and Time-Stamp Counters	2-22
2.6.7.	Reading and Writing Model-Specific Registers	2-23

CHAPTER 3

PROTECTED-MODE MEMORY MANAGEMENT

3.1.	MEMORY MANAGEMENT OVERVIEW	3-1
3.2.	USING SEGMENTS	3-3
3.2.1.	Basic Flat Model	3-3
3.2.2.	Protected Flat Model	3-3

	PAGE
3.2.3. Multi-Segment Model	3-5
3.2.4. Paging and Segmentation	3-6
3.3. PHYSICAL ADDRESS SPACE	3-6
3.4. LOGICAL AND LINEAR ADDRESSES	3-6
3.4.1. Segment Selectors	3-7
3.4.2. Segment Registers	3-8
3.4.3. Segment Descriptors	3-9
3.4.3.1. Code- and Data-Segment Descriptor Types	3-13
3.5. SYSTEM DESCRIPTOR TYPES	3-14
3.5.1. Segment Descriptor Tables	3-16
3.6. PAGING (VIRTUAL MEMORY) OVERVIEW	3-17
3.6.1. Paging Options	3-18
3.6.2. Page Tables and Directories	3-19
3.7. PAGE TRANSLATION USING 32-BIT PHYSICAL ADDRESSING	3-20
3.7.1. Linear Address Translation (4-KByte Pages)	3-20
3.7.2. Linear Address Translation (4-MByte Pages)	3-21
3.7.3. Mixing 4-KByte and 4-MByte Pages	3-22
3.7.4. Memory Aliasing	3-23
3.7.5. Base Address of the Page Directory	3-23
3.7.6. Page-Directory and Page-Table Entries	3-23
3.7.7. Not Present Page-Directory and Page-Table Entries	3-28
3.8. 36-BIT PHYSICAL ADDRESSING USING THE PAE PAGING MECHANISM	3-28
3.8.1. Linear Address Translation With PAE Enabled (4-KByte Pages)	3-29
3.8.2. Linear Address Translation With PAE Enabled (2-MByte Pages)	3-30
3.8.3. Accessing the Full Extended Physical Address Space With the Extended Page-Table Structure	3-31
3.8.4. Page-Directory and Page-Table Entries With Extended Addressing Enabled	3-31
3.9. 36-BIT PHYSICAL ADDRESSING USING THE PSE-36 PAGING MECHANISM	3-34
3.10. MAPPING SEGMENTS TO PAGES	3-36
3.11. TRANSLATION LOOKASIDE BUFFERS (TLBS)	3-37

CHAPTER 4 PROTECTION

4.1. ENABLING AND DISABLING SEGMENT AND PAGE PROTECTION	4-2
4.2. FIELDS AND FLAGS USED FOR SEGMENT-LEVEL AND PAGE-LEVEL PROTECTION	4-2
4.3. LIMIT CHECKING	4-5
4.4. TYPE CHECKING	4-6
4.4.1. Null Segment Selector Checking	4-7
4.5. PRIVILEGE LEVELS	4-7
4.6. PRIVILEGE LEVEL CHECKING WHEN ACCESSING DATA SEGMENTS	4-10
4.6.1. Accessing Data in Code Segments	4-12
4.7. PRIVILEGE LEVEL CHECKING WHEN LOADING THE SS REGISTER	4-12
4.8. PRIVILEGE LEVEL CHECKING WHEN TRANSFERRING PROGRAM CONTROL BETWEEN CODE SEGMENTS	4-12
4.8.1. Direct Calls or Jumps to Code Segments	4-13
4.8.1.1. Accessing Nonconforming Code Segments	4-14
4.8.1.2. Accessing Conforming Code Segments	4-15
4.8.2. Gate Descriptors	4-16
4.8.3. Call Gates	4-17

	PAGE
4.8.4. Accessing a Code Segment Through a Call Gate	4-18
4.8.5. Stack Switching	4-21
4.8.6. Returning from a Called Procedure	4-24
4.8.7. Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions	4-25
4.9. PRIVILEGED INSTRUCTIONS	4-26
4.10. POINTER VALIDATION	4-27
4.10.1. Checking Access Rights (LAR Instruction)	4-27
4.10.2. Checking Read/Write Rights (VERR and VERW Instructions)	4-28
4.10.3. Checking That the Pointer Offset Is Within Limits (LSL Instruction)	4-29
4.10.4. Checking Caller Access Privileges (ARPL Instruction)	4-29
4.10.5. Checking Alignment	4-31
4.11. PAGE-LEVEL PROTECTION	4-31
4.11.1. Page-Protection Flags	4-32
4.11.2. Restricting Addressable Domain	4-32
4.11.3. Page Type	4-33
4.11.4. Combining Protection of Both Levels of Page Tables	4-33
4.11.5. Overrides to Page Protection	4-33
4.12. COMBINING PAGE AND SEGMENT PROTECTION	4-34

CHAPTER 5

INTERRUPT AND EXCEPTION HANDLING

5.1. INTERRUPT AND EXCEPTION OVERVIEW	5-1
5.2. EXCEPTION AND INTERRUPT VECTORS	5-2
5.3. SOURCES OF INTERRUPTS	5-2
5.3.1. External Interrupts	5-2
5.3.2. Maskable Hardware Interrupts	5-4
5.3.3. Software-Generated Interrupts	5-4
5.4. SOURCES OF EXCEPTIONS	5-5
5.4.1. Program-Error Exceptions	5-5
5.4.2. Software-Generated Exceptions	5-5
5.4.3. Machine-Check Exceptions	5-5
5.5. EXCEPTION CLASSIFICATIONS	5-6
5.6. PROGRAM OR TASK RESTART	5-6
5.7. NONMASKABLE INTERRUPT (NMI)	5-8
5.7.1. Handling Multiple NMIs	5-8
5.8. ENABLING AND DISABLING INTERRUPTS	5-8
5.8.1. Masking Maskable Hardware Interrupts	5-8
5.8.2. Masking Instruction Breakpoints	5-9
5.8.3. Masking Exceptions and Interrupts When Switching Stacks	5-10
5.9. PRIORITY AMONG SIMULTANEOUS EXCEPTIONS AND INTERRUPTS	5-10
5.10. INTERRUPT DESCRIPTOR TABLE (IDT)	5-11
5.11. IDT DESCRIPTORS	5-12
5.12. EXCEPTION AND INTERRUPT HANDLING	5-14
5.12.1. Exception- or Interrupt-Handler Procedures	5-14
5.12.1.1. Protection of Exception- and Interrupt-Handler Procedures	5-16
5.12.1.2. Flag Usage By Exception- or Interrupt-Handler Procedure	5-17
5.12.2. Interrupt Tasks	5-18
5.13. ERROR CODE	5-20
5.14. EXCEPTION AND INTERRUPT REFERENCE	5-21
Interrupt 0—Divide Error Exception (#DE)	5-22
Interrupt 1—Debug Exception (#DB)	5-23

Interrupt 2—NMI Interrupt	5-23
Interrupt 3—Breakpoint Exception (#BP)	5-24
Interrupt 4—Overflow Exception (#OF)	5-25
Interrupt 5—BOUND Range Exceeded Exception (#BR)	5-26
Interrupt 6—Invalid Opcode Exception (#UD)	5-27
Interrupt 7—Device Not Available Exception (#NM)	5-29
Interrupt 8—Double Fault Exception (#DF)	5-31
Interrupt 9—Coprocesor Segment Overrun	5-33
Interrupt 10—Invalid TSS Exception (#TS)	5-34
Interrupt 11—Segment Not Present (#NP)	5-36
Interrupt 12—Stack Fault Exception (#SS)	5-38
Interrupt 13—General Protection Exception (#GP)	5-40
Interrupt 14—Page-Fault Exception (#PF)	5-43
Interrupt 16—x87 FPU Floating-Point Error (#MF)	5-47
Interrupt 17—Alignment Check Exception (#AC)	5-49
Interrupt 18—Machine-Check Exception (#MC)	5-51
Interrupt 19—SIMD Floating-Point Exception (#XF)	5-53
Interrupts 32 to 255—User Defined Interrupts.	5-56

CHAPTER 6**TASK MANAGEMENT**

6.1. TASK MANAGEMENT OVERVIEW.	6-1
6.1.1. Task Structure	6-1
6.1.2. Task State	6-2
6.1.3. Executing a Task.	6-3
6.2. TASK MANAGEMENT DATA STRUCTURES.	6-4
6.2.1. Task-State Segment (TSS)	6-4
6.2.2. TSS Descriptor	6-7
6.2.3. Task Register	6-8
6.2.4. Task-Gate Descriptor	6-9
6.3. TASK SWITCHING	6-12
6.4. TASK LINKING.	6-16
6.4.1. Use of Busy Flag To Prevent Recursive Task Switching	6-17
6.4.2. Modifying Task Linkages.	6-18
6.5. TASK ADDRESS SPACE	6-18
6.5.1. Mapping Tasks to the Linear and Physical Address Spaces.	6-19
6.5.2. Task Logical Address Space.	6-20
6.6. 16-BIT TASK-STATE SEGMENT (TSS)	6-21

CHAPTER 7**MULTIPLE-PROCESSOR MANAGEMENT**

7.1. LOCKED ATOMIC OPERATIONS.	7-2
7.1.1. Guaranteed Atomic Operations.	7-3
7.1.2. Bus Locking.	7-3
7.1.2.1. Automatic Locking	7-4
7.1.2.2. Software Controlled Bus Locking	7-4
7.1.3. Handling Self- and Cross-Modifying Code	7-6
7.1.4. Effects of a LOCK Operation on Internal Processor Caches.	7-7
7.2. MEMORY ORDERING.	7-7
7.2.1. Memory Ordering in the Pentium® and Intel486™ Processors.	7-8

	PAGE
7.2.2. Memory Ordering Pentium 4, Intel® Xeon™, and P6 Family Processors	7-8
7.2.3. Out-of-Order Stores For String Operations in Pentium 4, Intel Xeon, and P6 Family Processors	7-10
7.2.4. Strengthening or Weakening the Memory Ordering Model	7-11
7.3. PROPAGATION OF PAGE TABLE AND PAGE DIRECTORY ENTRY CHANGES TO MULTIPLE PROCESSORS	7-13
7.4. SERIALIZING INSTRUCTIONS	7-14
7.5. MULTIPLE-PROCESSOR (MP) INITIALIZATION	7-15
7.5.1. BSP and AP Processors	7-16
7.5.2. MP Initialization Protocol Requirements and Restrictions for Intel Xeon Processors	7-16
7.5.3. MP Initialization Protocol Algorithm for the Intel Xeon Processors	7-17
7.5.4. MP Initialization Example	7-18
7.5.4.1. Typical BSP Initialization Sequence	7-19
7.5.4.2. Typical AP Initialization Sequence	7-21
7.5.5. Identifying the Processors in an MP System	7-22
7.6. HYPER-THREADING TECHNOLOGY	7-23
7.6.1. Intel Hyper-Threading Technology Architecture	7-24
7.6.1.1. State of the Logical Processors	7-24
7.6.1.2. APIC Functionality	7-25
7.6.1.3. Memory Type Range Registers (MTRR)	7-26
7.6.1.4. Page Attribute Table (PAT)	7-26
7.6.1.5. Machine Check Architecture	7-26
7.6.1.6. Debug Registers and Extensions	7-26
7.6.1.7. Performance Monitoring Counters	7-27
7.6.1.8. IA32_MISC_ENABLE MSr	7-27
7.6.1.9. Memory Ordering	7-27
7.6.1.10. Serializing Instructions	7-27
7.6.1.11. MICROCODE UPDATE Resources	7-27
7.6.1.12. Self Modifying Code	7-28
7.6.2. Implementation-Specific HT Technology Facilities for the Intel Xeon Processor MP	7-28
7.6.2.1. Processor Caches	7-28
7.6.2.2. Processor Translation Lookaside Buffers (TLBs)	7-29
7.6.2.3. Thermal Monitor	7-29
7.6.2.4. External Signal Compatibility	7-29
7.6.3. Detecting Hyper-Threading Technology	7-30
7.6.4. Initializing IA-32 Processors With Hyper-Threading Technology	7-30
7.6.5. Executing Multiple Threads on an IA-32 Processor With Hyper-Threading Technology	7-31
7.6.6. Handling Interrupts on an IA-32 Processor With Hyper-Threading Technology	7-31
7.6.7. Management of Idle and Blocked Conditions	7-33
7.6.7.1. HLT Instruction	7-33
7.6.7.2. PAUSE Instruction	7-33
7.6.8. Identifying Logical Processors in an MP System	7-33
7.6.9. Required Operating System Support	7-39
7.6.9.1. Use the PAUSE Instruction in Spin-Wait Loops	7-39
7.6.9.2. Halt Idle Logical Processors	7-40
7.6.9.3. Guidelines for Scheduling Threads On Multiple Logical Processors	7-40
7.6.9.4. Eliminate Execution-Based Timing Loops	7-40
7.6.9.5. Place Locks and Semaphores in Aligned, 128-Byte Blocks of Memory	7-41

CHAPTER 8**ADVANCED PROGRAMMABLE
INTERRUPT CONTROLLER (APIC)**

8.1.	LOCAL AND I/O APIC OVERVIEW	8-1
8.2.	SYSTEM BUS VS. APIC BUS	8-5
8.3.	RELATIONSHIP BETWEEN THE INTEL 82489DX EXTERNAL APIC, THE APIC, AND THE XAPIC	8-5
8.4.	LOCAL APIC	8-6
8.4.1.	The Local APIC Block Diagram	8-6
8.4.2.	Presence of the Local APIC	8-9
8.4.3.	Enabling or Disabling the Local APIC	8-10
8.4.4.	Local APIC Status and Location	8-11
8.4.5.	Relocating the Local APIC Registers	8-11
8.4.6.	Local APIC ID	8-11
8.4.7.	Local APIC State	8-12
8.4.7.1.	Local APIC State After Power-Up or Reset	8-12
8.4.7.2.	Local APIC State After It Has Been Software Disabled	8-13
8.4.7.3.	Local APIC State After an INIT Reset ("Wait-for-SIPI" State).	8-13
8.4.7.4.	Local APIC State After It Receives an INIT-Deassert IPI	8-14
8.4.8.	Local APIC Version Register	8-14
8.5.	HANDLING LOCAL INTERRUPTS	8-15
8.5.1.	Local Vector Table	8-15
8.5.2.	Valid Interrupt Vectors	8-18
8.5.3.	Error Handling	8-18
8.5.4.	APIC Timer	8-20
8.5.5.	Local Interrupt Acceptance	8-21
8.6.	ISSUING INTERPROCESSOR INTERRUPTS	8-21
8.6.1.	Interrupt Command Register (ICR)	8-21
8.6.2.	Determining IPI Destination	8-27
8.6.2.1.	Physical Destination Mode	8-27
8.6.2.2.	Logical Destination Mode	8-28
8.6.2.3.	Broadcast/Self Delivery Mode	8-29
8.6.2.4.	Lowest Priority Delivery Mode	8-29
8.6.3.	IPI Delivery and Acceptance	8-31
8.7.	SYSTEM AND APIC BUS ARBITRATION	8-31
8.8.	HANDLING INTERRUPTS	8-31
8.8.1.	Interrupt Handling with the Pentium 4 and Intel Xeon Processors	8-32
8.8.2.	Interrupt Handling with the P6 Family and Pentium Processors	8-32
8.8.3.	Interrupt, Task, and Processor Priority	8-34
8.8.3.1.	Task and Processor Priorities	8-35
8.8.4.	Interrupt Acceptance for Fixed Interrupts	8-36
8.8.5.	Signaling Interrupt Servicing Completion	8-37
8.9.	SPURIOUS INTERRUPT	8-38
8.10.	APIC BUS MESSAGE PASSING MECHANISM AND PROTOCOL (P6 FAMILY AND PENTIUM PROCESSORS ONLY)	8-39
8.10.1.	Bus Message Formats	8-40
8.11.	MESSAGE SIGNALLED INTERRUPTS	8-40
8.11.1.	Message Address Register Format	8-40
8.11.2.	Message Data Register Format	8-42

CHAPTER 9
PROCESSOR MANAGEMENT AND INITIALIZATION

9.1.	INITIALIZATION OVERVIEW	9-1
9.1.1.	Processor State After Reset	9-2
9.1.2.	Processor Built-In Self-Test (BIST)	9-2
9.1.3.	Model and Stepping Information	9-5
9.1.4.	First Instruction Executed	9-6
9.2.	X87 FPU INITIALIZATION	9-6
9.2.1.	Configuring the x87 FPU Environment	9-6
9.2.2.	Setting the Processor for x87 FPU Software Emulation	9-7
9.3.	CACHE ENABLING	9-8
9.4.	MODEL-SPECIFIC REGISTERS (MSRS)	9-8
9.5.	MEMORY TYPE RANGE REGISTERS (MTRRS)	9-9
9.6.	SSE AND SSE2 EXTENSIONS INITIALIZATION	9-9
9.7.	SOFTWARE INITIALIZATION FOR REAL-ADDRESS MODE OPERATION	9-10
9.7.1.	Real-Address Mode IDT	9-10
9.7.2.	NMI Interrupt Handling	9-10
9.8.	SOFTWARE INITIALIZATION FOR PROTECTED-MODE OPERATION	9-11
9.8.1.	Protected-Mode System Data Structures	9-12
9.8.2.	Initializing Protected-Mode Exceptions and Interrupts	9-12
9.8.3.	Initializing Paging	9-13
9.8.4.	Initializing Multitasking	9-13
9.9.	MODE SWITCHING	9-14
9.9.1.	Switching to Protected Mode	9-14
9.9.2.	Switching Back to Real-Address Mode	9-15
9.10.	INITIALIZATION AND MODE SWITCHING EXAMPLE	9-16
9.10.1.	Assembler Usage	9-19
9.10.2.	STARTUP.ASM Listing	9-20
9.10.3.	MAIN.ASM Source Code	9-29
9.10.4.	Supporting Files	9-30
9.11.	MICROCODE UPDATE FACILITIES	9-32
9.11.1.	Microcode Update	9-32
9.11.2.	Microcode Update Loader	9-35
9.11.2.1.	Update Loading Procedure	9-36
9.11.2.2.	Hard Resets in Update Loading	9-37
9.11.2.3.	Update in a Multiprocessor System	9-37
9.11.2.4.	Update Loader Enhancements	9-37
9.11.3.	Update Signature and Verification	9-37
9.11.3.1.	Determining the Signature	9-38
9.11.3.2.	Authenticating the Update	9-38
9.11.4.	Pentium 4, Intel Xeon, and P6 Family Processor Microcode Update Specifications	9-39
9.11.4.1.	Responsibilities of the BIOS	9-39
9.11.4.2.	Responsibilities of the Calling Program	9-40
9.11.4.3.	Microcode Update Functions	9-43
9.11.4.4.	INT 15H-based Interface	9-43
9.11.4.5.	Function 00H—Presence Test	9-44
9.11.4.6.	Function 01H—Write Microcode Update Data	9-44
9.11.4.7.	Function 02H—Microcode Update Control	9-48
9.11.4.8.	Function 03H—Read Microcode Update Data	9-49
9.11.4.9.	Return Codes	9-50

CHAPTER 10**MEMORY CACHE CONTROL**

10.1.	INTERNAL CACHES, TLBS, AND BUFFERS	10-1
10.2.	CACHING TERMINOLOGY	10-4
10.3.	METHODS OF CACHING AVAILABLE	10-5
10.3.1.	Buffering of Write Combining Memory Locations	10-7
10.3.2.	Choosing a Memory Type	10-9
10.4.	CACHE CONTROL PROTOCOL	10-9
10.5.	CACHE CONTROL	10-10
10.5.1.	Cache Control Registers and Bits	10-11
10.5.2.	Precedence of Cache Controls	10-15
10.5.2.1.	Selecting Memory Types for Pentium Pro and Pentium II Processors	10-16
10.5.2.2.	Selecting Memory Types for Pentium 4, Intel Xeon, and Pentium III Processors	10-17
10.5.2.3.	Writing Values Across Pages with Different Memory Types	10-18
10.5.3.	Preventing Caching	10-18
10.5.4.	Disabling and Enabling the L3 Cache	10-19
10.5.5.	Cache Management Instructions	10-19
10.5.6.	L1 Data Cache Context Mode	10-20
10.5.6.1.	Adaptive Mode	10-20
10.5.6.2.	Shared Mode	10-21
10.6.	SELF-MODIFYING CODE	10-21
10.7.	IMPLICIT CACHING (PENTIUM 4, INTEL XEON, AND P6 FAMILY PROCESSORS)	10-21
10.8.	EXPLICIT CACHING	10-22
10.9.	INVALIDATING THE TRANSLATION LOOKASIDE BUFFERS (TLBS)	10-22
10.10.	STORE BUFFER	10-23
10.11.	MEMORY TYPE RANGE REGISTERS (MTRRS)	10-24
10.11.1.	MTRR Feature Identification	10-26
10.11.2.	Setting Memory Ranges with MTRRs	10-27
10.11.2.1.	IA32_MTRR_DEF_TYPE MSR	10-27
10.11.2.2.	Fixed Range MTRRs	10-28
10.11.2.3.	Variable Range MTRRs	10-28
10.11.3.	Example Base and Mask Calculations	10-31
10.11.4.	Range Size and Alignment Requirement	10-32
10.11.4.1.	MTRR Precedences	10-32
10.11.5.	MTRR Initialization	10-33
10.11.6.	Remapping Memory Types	10-34
10.11.7.	MTRR Maintenance Programming Interface	10-34
10.11.7.1.	MemTypeGet() Function	10-34
10.11.7.2.	MemTypeSet() Function	10-36
10.11.8.	MTRR Considerations in MP Systems	10-37
10.11.9.	Large Page Size Considerations	10-39
10.12.	PAGE ATTRIBUTE TABLE (PAT)	10-39
10.12.1.	Detecting Support for the PAT Feature	10-40
10.12.2.	IA32_CR_PAT MSR	10-40
10.12.3.	Selecting a Memory Type from the PAT	10-41
10.12.4.	Programming the PAT	10-42
10.12.5.	PAT Compatibility with Earlier IA-32 Processors	10-43

CHAPTER 11
INTEL® MMX™ TECHNOLOGY SYSTEM PROGRAMMING

11.1.	EMULATION OF THE MMX INSTRUCTION SET	11-1
11.2.	THE MMX STATE AND MMX REGISTER ALIASING	11-1
11.2.1.	Effect of MMX, x87 FPU, FXSAVE, and FXRSTOR Instructions on the x87 FPU Tag Word	11-3
11.3.	SAVING AND RESTORING THE MMX STATE AND REGISTERS	11-4
11.4.	SAVING MMX STATE ON TASK OR CONTEXT SWITCHES	11-5
11.5.	EXCEPTIONS THAT CAN OCCUR WHEN EXECUTING MMX INSTRUCTIONS	11-5
11.5.1.	Effect of MMX Instructions on Pending x87 Floating-Point Exceptions	11-6
11.6.	DEBUGGING MMX CODE	11-6

CHAPTER 12
SSE AND SSE2 SYSTEM PROGRAMMING

12.1.	PROVIDING OPERATING SYSTEM SUPPORT FOR THE SSE AND SSE2 EXTENSIONS	12-1
12.1.1.	General Guidelines for Adding Support to an Operating System for the SSE and SSE2 Extensions	12-1
12.1.2.	Checking for SSE and SSE2 Support	12-2
12.1.3.	Checking for Support for the FXSAVE and FXRSTOR Instructions	12-2
12.1.4.	Initialization of the SSE and SSE2 Extensions	12-2
12.1.5.	Providing Non-Numeric Exception Handlers for Exceptions Generated by the SSE and SSE2 Instructions	12-3
12.1.6.	Providing an Handler for the SIMD Floating-Point Exception (#XF)	12-5
12.1.6.1.	Numeric Error flag and IGNNE#	12-6
12.2.	EMULATION OF THE SSE AND SSE2 EXTENSIONS	12-6
12.3.	SAVING AND RESTORING THE SSE AND SSE2 STATE	12-6
12.4.	SAVING SSE AND SSE2 STATE ON TASK OR CONTEXT SWITCHES	12-7
12.5.	DESIGNING OPERATING SYSTEM FACILITIES FOR AUTOMATICALLY SAVING X87 FPU, MMX, SSE, AND SSE2 STATE ON TASK OR CONTEXT SWITCHES	12-7
12.5.1.	Using the TS Flag to Control the Saving of the x87 FPU, MMX, SSE, and SSE2 State	12-8

CHAPTER 13
SYSTEM MANAGEMENT

13.1.	SYSTEM MANAGEMENT MODE OVERVIEW	13-1
13.2.	SYSTEM MANAGEMENT INTERRUPT (SMI)	13-2
13.3.	SWITCHING BETWEEN SMM AND THE OTHER PROCESSOR OPERATING MODES	13-2
13.3.1.	Entering SMM	13-3
13.3.2.	Exiting From SMM	13-3
13.4.	SMRAM	13-4
13.4.1.	SMRAM State Save Map	13-5
13.4.2.	SMRAM Caching	13-8
13.5.	SMI HANDLER EXECUTION ENVIRONMENT	13-9
13.6.	EXCEPTIONS AND INTERRUPTS WITHIN SMM	13-10
13.7.	NMI HANDLING WHILE IN SMM	13-12
13.8.	SAVING THE X87 FPU STATE WHILE IN SMM	13-12
13.9.	SMM REVISION IDENTIFIER	13-13
13.10.	AUTO HALT RESTART	13-14
13.10.1.	Executing the HLT Instruction in SMM	13-15

	PAGE
13.11. SMBASE RELOCATION	13-15
13.11.1. Relocating SMRAM to an Address Above 1 MByte.	13-15
13.12. I/O INSTRUCTION RESTART	13-16
13.12.1. Back-to-Back SMI Interrupts When I/O Instruction Restart Is Being Used ...	13-17
13.13. SMM MULTIPLE-PROCESSOR CONSIDERATIONS	13-17
13.14. ENHANCED INTEL SPEEDSTEP® TECHNOLOGY	13-18
13.14.1. Software Interface For Initiating Performance State Transitions	13-18
13.15. THERMAL MONITORING AND PROTECTION	13-19
13.15.1. Catastrophic Shutdown Detector	13-20
13.15.2. Thermal Monitor	13-20
13.15.2.1. Thermal Monitor 1	13-20
13.15.2.2. Thermal Monitor 2	13-20
13.15.2.3. Performance State Transitions and Thermal Monitoring	13-21
13.15.2.4. Thermal Status Information	13-21
13.15.3. Software Controlled Clock Modulation	13-23
13.15.4. Detection of Thermal Monitor and Software Controlled Clock Modulation Facilities	13-24

CHAPTER 14

MACHINE-CHECK ARCHITECTURE

14.1. MACHINE-CHECK EXCEPTIONS AND ARCHITECTURE	14-1
14.2. COMPATIBILITY WITH PENTIUM PROCESSOR	14-1
14.3. MACHINE-CHECK MSRS	14-2
14.3.1. Machine-Check Global Control MSRs	14-2
14.3.1.1. IA32_MCG_CAP MSR (Pentium 4 and Intel Xeon Processors)	14-3
14.3.1.2. MCG_CAP MSR (P6 Family Processors)	14-4
14.3.1.3. IA32_MCG_STATUS MSR	14-4
14.3.1.4. IA32_MCG_CTL MSR	14-5
14.3.1.5. IA32_MCG_MISC MSR	14-5
14.3.2. Error-Reporting Register Banks	14-5
14.3.2.1. IA32_MCi_CTL MSRs	14-5
14.3.2.2. IA32_MCi_STATUS MSRs	14-6
14.3.2.3. IA32_MCi_ADDR MSRs	14-8
14.3.2.4. IA32_MCi_MISC MSRs	14-8
14.3.2.5. IA32_MCG Extended Machine Check State MSRs	14-9
14.3.3. Mapping of the Pentium Processor Machine-Check Errors to the Machine-Check Architecture	14-10
14.4. MACHINE-CHECK AVAILABILITY	14-10
14.5. MACHINE-CHECK INITIALIZATION	14-10
14.6. INTERPRETING THE MCA ERROR CODES	14-12
14.6.1. Simple Error Codes	14-12
14.6.2. Compound Error Codes	14-13
14.6.3. Example Machine-Check Error Codes Interpretation	14-15
14.7. GUIDELINES FOR WRITING MACHINE-CHECK SOFTWARE	14-15
14.7.1. Machine-Check Exception Handler	14-16
14.7.2. Pentium Processor Machine-Check Exception Handling	14-18
14.7.3. Logging Correctable Machine-Check Errors	14-18

CHAPTER 15

DEBUGGING AND PERFORMANCE MONITORING

15.1. OVERVIEW OF THE DEBUGGING SUPPORT FACILITIES	15-1
15.2. DEBUG REGISTERS	15-2

	PAGE
15.2.1. Debug Address Registers (DR0-DR3)	15-3
15.2.2. Debug Registers DR4 and DR5	15-4
15.2.3. Debug Status Register (DR6)	15-4
15.2.4. Debug Control Register (DR7)	15-5
15.2.5. Breakpoint Field Recognition	15-6
15.3. DEBUG EXCEPTIONS	15-7
15.3.1. Debug Exception (#DB)—Interrupt Vector 1	15-7
15.3.1.1. Instruction-Breakpoint Exception Condition	15-8
15.3.1.2. Data Memory and I/O Breakpoint Exception Conditions	15-9
15.3.1.3. General-Detect Exception Condition	15-9
15.3.1.4. Single-Step Exception Condition	15-10
15.3.1.5. Task-Switch Exception Condition	15-10
15.3.2. Breakpoint Exception (#BP)—Interrupt Vector 3	15-10
15.4. LAST BRANCH RECORDING OVERVIEW	15-11
15.5. LAST BRANCH, INTERRUPT, AND EXCEPTION RECORDING (PENTIUM 4 AND INTEL XEON PROCESSORS)	15-11
15.5.1. IA32_DEBUGCTL MSR (Pentium 4 and Intel Xeon Processors)	15-12
15.5.2. LBR Stack (Pentium 4 and Intel Xeon Processors)	15-13
15.5.3. Monitoring Branches, Exceptions, and Interrupts (Pentium 4 and Intel Xeon Processors)	15-15
15.5.4. Single-Stepping on Branches, Exceptions, and Interrupts	15-16
15.5.5. Branch Trace Messages	15-16
15.5.6. Last Exception Records (Pentium 4 and Intel Xeon Processors)	15-16
15.5.7. Branch Trace Store (BTS)	15-16
15.5.7.1. Detection of the BTS Facilities	15-17
15.5.7.2. Setting Up the DS Save Area	15-17
15.5.7.3. Setting Up the BTS Buffer	15-18
15.5.7.4. Writing the DS Interrupt Service Routine	15-19
15.6. LAST BRANCH, INTERRUPT, AND EXCEPTION RECORDING (P6 FAMILY PROCESSORS)	15-19
15.6.1. DebugCtlMSR Register (P6 Family Processors)	15-20
15.6.2. Last Branch and Last Exception MSRs (P6 Family Processors)	15-22
15.6.3. Monitoring Branches, Exceptions, and Interrupts (P6 Family Processors)	15-22
15.7. TIME-STAMP COUNTER	15-23
15.8. PERFORMANCE MONITORING OVERVIEW	15-24
15.9. PERFORMANCE MONITORING (PENTIUM 4 AND INTEL XEON PROCESSORS)	15-24
15.9.1. ESCR MSRs	15-28
15.9.2. Performance Counters	15-30
15.9.3. CCCR MSRs	15-32
15.9.4. Debug Store (DS) Mechanism	15-34
15.9.5. DS Save Area	15-34
15.9.6. Programming the Performance Counters for Non-Retirement Events	15-37
15.9.6.1. Selecting Events to Count	15-38
15.9.6.2. Filtering Events	15-41
15.9.6.3. Starting Event Counting	15-42
15.9.6.4. Reading a Performance Counter's Count	15-43
15.9.6.5. Halting Event Counting	15-43
15.9.6.6. Cascading Counters	15-43
15.9.6.7. EXTENDED CASCADING	15-44
15.9.6.8. EXTENDED CASCADING	15-45
15.9.6.9. Generating an Interrupt on Overflow	15-46

	PAGE
15.9.6.10. Counter Usage Guideline	15-47
15.9.7. At-Retirement Counting	15-47
15.9.7.1. Using At-Retirement Counting	15-48
15.9.7.2. Tagging Mechanism for Front_end_event	15-49
15.9.7.3. Tagging Mechanism For Execution_event	15-49
15.9.7.4. Tagging Mechanism For Replay_event	15-50
15.9.8. Precise Event-Based Sampling (PEBS)	15-50
15.9.8.1. Detection of the Availability of the PEBS Facilities	15-51
15.9.8.2. Setting Up the DS Save Area	15-51
15.9.8.3. Setting Up the PEBS Buffer	15-51
15.9.8.4. Writing a PEBS Interrupt Service Routine	15-52
15.9.8.5. Other DS Mechanism Implications	15-52
15.9.9. Counting Clocks	15-52
15.9.10. Operating System Implications	15-54
15.10. PERFORMANCE MONITORING AND HYPER-THREADING TECHNOLOGY	15-55
15.10.1. ESCR MSRs	15-55
15.10.2. CCCR MSRs	15-57
15.10.3. IA32_PEBS_ENABLE MSR	15-59
15.10.4. Performance Monitoring Events	15-60
15.11. PERFORMANCE MONITORING (P6 FAMILY PROCESSOR)	15-61
15.11.1. PerfEvtSel0 and PerfEvtSel1 MSRs	15-62
15.11.2. PerfCtr0 and PerfCtr1 MSRs	15-63
15.11.3. Starting and Stopping the Performance-Monitoring Counters	15-64
15.11.4. Event and Time-Stamp Monitoring Software	15-64
15.11.5. Monitoring Counter Overflow	15-65
15.12. PERFORMANCE MONITORING (PENTIUM PROCESSORS)	15-65
15.12.1. Control and Event Select Register (CESR)	15-66
15.12.2. Use of the Performance-Monitoring Pins	15-67
15.12.3. Events Counted	15-68

CHAPTER 16

8086 EMULATION

16.1. REAL-ADDRESS MODE	16-1
16.1.1. Address Translation in Real-Address Mode	16-3
16.1.2. Registers Supported in Real-Address Mode	16-4
16.1.3. Instructions Supported in Real-Address Mode	16-4
16.1.4. Interrupt and Exception Handling	16-6
16.2. VIRTUAL-8086 MODE	16-7
16.2.1. Enabling Virtual-8086 Mode	16-9
16.2.2. Structure of a Virtual-8086 Task	16-9
16.2.3. Paging of Virtual-8086 Tasks	16-10
16.2.4. Protection within a Virtual-8086 Task	16-11
16.2.5. Entering Virtual-8086 Mode	16-11
16.2.6. Leaving Virtual-8086 Mode	16-13
16.2.7. Sensitive Instructions	16-14
16.2.8. Virtual-8086 Mode I/O	16-14
16.2.8.1. I/O-Port-Mapped I/O	16-14
16.2.8.2. Memory-Mapped I/O	16-15
16.2.8.3. Special I/O Buffers	16-15
16.3. INTERRUPT AND EXCEPTION HANDLING IN VIRTUAL-8086 MODE	16-15
16.3.1. Class 1—Hardware Interrupt and Exception Handling in Virtual-8086 Mode	16-17

	PAGE
16.3.1.1. Handling an Interrupt or Exception Through a Protected-Mode Trap or Interrupt Gate	16-17
16.3.1.2. Handling an Interrupt or Exception With an 8086 Program Interrupt or Exception Handler	16-19
16.3.1.3. Handling an Interrupt or Exception Through a Task Gate	16-20
16.3.2. Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism.	16-20
16.3.3. Class 3—Software Interrupt Handling in Virtual-8086 Mode	16-23
16.3.3.1. Method 1: Software Interrupt Handling	16-25
16.3.3.2. Methods 2 and 3: Software Interrupt Handling	16-26
16.3.3.3. Method 4: Software Interrupt Handling	16-26
16.3.3.4. Method 5: Software Interrupt Handling	16-26
16.3.3.5. Method 6: Software Interrupt Handling	16-27
16.4. PROTECTED-MODE VIRTUAL INTERRUPTS	16-28

CHAPTER 17

MIXING 16-BIT AND 32-BIT CODE

17.1. DEFINING 16-BIT AND 32-BIT PROGRAM MODULES	17-2
17.2. MIXING 16-BIT AND 32-BIT OPERATIONS WITHIN A CODE SEGMENT.	17-2
17.3. SHARING DATA AMONG MIXED-SIZE CODE SEGMENTS	17-3
17.4. TRANSFERRING CONTROL AMONG MIXED-SIZE CODE SEGMENTS	17-4
17.4.1. Code-Segment Pointer Size	17-5
17.4.2. Stack Management for Control Transfer	17-5
17.4.2.1. Controlling the Operand-Size Attribute For a Call.	17-7
17.4.2.2. Passing Parameters With a Gate	17-7
17.4.3. Interrupt Control Transfers.	17-8
17.4.4. Parameter Translation	17-8
17.4.5. Writing Interface Procedures	17-8

CHAPTER 18

IA-32 COMPATIBILITY

18.1. IA-32 PROCESSOR FAMILIES AND CATEGORIES.	18-1
18.2. RESERVED BITS.	18-2
18.3. ENABLING NEW FUNCTIONS AND MODES	18-2
18.4. DETECTING THE PRESENCE OF NEW FEATURES THROUGH SOFTWARE	18-2
18.5. INTEL MMX TECHNOLOGY	18-3
18.6. STREAMING SIMD EXTENSIONS (SSE)	18-3
18.7. STREAMING SIMD EXTENSIONS 2 (SSE2).	18-3
18.8. HYPER-THREADING TECHNOLOGY.	18-4
18.9. NEW INSTRUCTIONS IN THE PENTIUM AND LATER IA-32 PROCESSORS	18-4
18.9.1. Instructions Added Prior to the Pentium Processor.	18-4
18.10. OBSOLETE INSTRUCTIONS	18-6
18.11. UNDEFINED OPCODES	18-6
18.12. NEW FLAGS IN THE EFLAGS REGISTER.	18-6
18.12.1. Using EFLAGS Flags to Distinguish Between 32-Bit IA-32 Processors	18-6
18.13. STACK OPERATIONS.	18-7
18.13.1. PUSH SP.	18-7
18.13.2. EFLAGS Pushed on the Stack	18-7
18.14. X87 FPU.	18-8
18.14.1. Control Register CR0 Flags.	18-8
18.14.2. x87 FPU Status Word	18-9
18.14.2.1. Condition Code Flags (C0 through C3).	18-9

	PAGE
18.14.2.2. Stack Fault Flag	18-9
18.14.3. x87 FPU Control Word	18-9
18.14.4. x87 FPU Tag Word	18-10
18.14.5. Data Types	18-10
18.14.5.1. NaNs	18-10
18.14.5.2. Pseudo-zero, Pseudo-NaN, Pseudo-infinity, and Unnormal Formats	18-11
18.14.6. Floating-Point Exceptions	18-11
18.14.6.1. Denormal Operand Exception (#D)	18-11
18.14.6.2. Numeric Overflow Exception (#O)	18-11
18.14.6.3. Numeric Underflow Exception (#U)	18-12
18.14.6.4. Exception Precedence	18-12
18.14.6.5. CS and EIP For FPU Exceptions	18-13
18.14.6.6. FPU Error Signals	18-13
18.14.6.7. Assertion of the FERR# Pin	18-13
18.14.6.8. Invalid Operation Exception On Denormals	18-13
18.14.6.9. Alignment Check Exceptions (#AC)	18-14
18.14.6.10. Segment Not Present Exception During FLDENV	18-14
18.14.6.11. Device Not Available Exception (#NM)	18-14
18.14.6.12. Coprocessor Segment Overrun Exception	18-14
18.14.6.13. General Protection Exception (#GP)	18-14
18.14.6.14. Floating-Point Error Exception (#MF)	18-14
18.14.7. Changes to Floating-Point Instructions	18-15
18.14.7.1. FDIV, FPREM, and FSQRT Instructions	18-15
18.14.7.2. FSCALE Instruction	18-15
18.14.7.3. FPREM1 Instruction	18-15
18.14.7.4. FPREM Instruction	18-15
18.14.7.5. FUCOM, FUCOMP, and FUCOMPP Instructions	18-15
18.14.7.6. FPTAN Instruction	18-16
18.14.7.7. Stack Overflow	18-16
18.14.7.8. FSIN, FCOS, and FSINCOS Instructions	18-16
18.14.7.9. FPATAN Instruction	18-16
18.14.7.10. F2XM1 Instruction	18-16
18.14.7.11. FLD Instruction	18-16
18.14.7.12. FXTRACT Instruction	18-17
18.14.7.13. Load Constant Instructions	18-17
18.14.7.14. FSETPM Instruction	18-17
18.14.7.15. FXAM Instruction	18-17
18.14.7.16. FSAVE and FSTENV Instructions	18-18
18.14.8. Transcendental Instructions	18-18
18.14.9. Obsolete Instructions	18-18
18.14.10. WAIT/FWAIT Prefix Differences	18-18
18.14.11. Operands Split Across Segments and/or Pages	18-18
18.14.12. FPU Instruction Synchronization	18-19
18.15. SERIALIZING INSTRUCTIONS	18-19
18.16. FPU AND MATH COPROCESSOR INITIALIZATION	18-19
18.16.1. Intel 387 and Intel 287 Math Coprocessor Initialization	18-19
18.16.2. Intel486 SX Processor and Intel 487 SX Math Coprocessor Initialization	18-20
18.17. CONTROL REGISTERS	18-21
18.18. MEMORY MANAGEMENT FACILITIES	18-23
18.18.1. New Memory Management Control Flags	18-23
18.18.1.1. Physical Memory Addressing Extension	18-23
18.18.1.2. Global Pages	18-23

	PAGE
18.18.1.3. Larger Page Sizes	18-23
18.18.2. CD and NW Cache Control Flags	18-23
18.18.3. Descriptor Types and Contents	18-24
18.18.4. Changes in Segment Descriptor Loads	18-24
18.19. DEBUG FACILITIES	18-24
18.19.1. Differences in Debug Register DR6	18-24
18.19.2. Differences in Debug Register DR7	18-24
18.19.3. Debug Registers DR4 and DR5	18-25
18.19.4. Recognition of Breakpoints	18-25
18.20. TEST REGISTERS	18-25
18.21. EXCEPTIONS AND/OR EXCEPTION CONDITIONS	18-25
18.21.1. Machine-Check Architecture	18-27
18.21.2. Priority OF Exceptions	18-27
18.22. INTERRUPTS	18-27
18.22.1. Interrupt Propagation Delay	18-27
18.22.2. NMI Interrupts	18-28
18.22.3. IDT Limit	18-28
18.23. ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)	18-28
18.23.1. Software Visible Differences Between the Local APIC and the 82489DX	18-28
18.23.2. New Features Incorporated in the Local APIC for the P6 Family and Pentium Processors	18-29
18.23.3. New Features Incorporated in the Local APIC of the Pentium 4 and Intel Xeon Processors	18-29
18.24. TASK SWITCHING AND TSS	18-30
18.24.1. P6 Family and Pentium Processor TSS	18-30
18.24.2. TSS Selector Writes	18-30
18.24.3. Order of Reads/Writes to the TSS	18-30
18.24.4. Using A 16-Bit TSS with 32-Bit Constructs	18-30
18.24.5. Differences in I/O Map Base Addresses	18-30
18.25. CACHE MANAGEMENT	18-31
18.25.1. Self-Modifying Code with Cache Enabled	18-32
18.25.2. Disabling the L3 Cache	18-33
18.26. PAGING	18-33
18.26.1. Large Pages	18-33
18.26.2. PCD and PWT Flags	18-33
18.26.3. Enabling and Disabling Paging	18-34
18.27. STACK OPERATIONS	18-34
18.27.1. Selector Pushes and Pops	18-34
18.27.2. Error Code Pushes	18-35
18.27.3. Fault Handling Effects on the Stack	18-35
18.27.4. Interlevel RET/IRET From a 16-Bit Interrupt or Call Gate	18-35
18.28. MIXING 16- AND 32-BIT SEGMENTS	18-35
18.29. SEGMENT AND ADDRESS WRAPAROUND	18-36
18.29.1. Segment Wraparound	18-36
18.30. STORE BUFFERS AND MEMORY ORDERING	18-37
18.31. BUS LOCKING	18-38
18.32. BUS HOLD	18-39
18.33. MODEL-SPECIFIC EXTENSIONS TO THE IA-32	18-39
18.33.1. Model-Specific Registers	18-39
18.33.2. RDMSR and WRMSR Instructions	18-40
18.33.3. Memory Type Range Registers	18-40
18.33.4. Machine-Check Exception and Architecture	18-40

18.33.5.	Performance-Monitoring Counters	18-41
18.34.	TWO WAYS TO RUN INTEL 286 PROCESSOR TASKS	18-41

APPENDIX A**PERFORMANCE-MONITORING EVENTS**

A.1.	PENTIUM 4 AND INTEL XEON PROCESSOR PERFORMANCE-MONITORING EVENTS	A-1
A.2.	PERFORMANCE MONITORING EVENTS FOR INTEL® PENTIUM® M PROCESSORS	A-42
A.3.	P6 FAMILY PROCESSOR PERFORMANCE-MONITORING EVENTS	A-45
A.4.	PENTIUM PROCESSOR PERFORMANCE-MONITORING EVENTS	A-60

APPENDIX B**MODEL-SPECIFIC REGISTERS (MSRS)**

B.1.	MSRS IN THE PENTIUM 4 AND INTEL XEON PROCESSORS	B-1
B.2.	MSRS IN THE PENTIUM M PROCESSOR	B-20
B.3.	MSRS IN THE P6 FAMILY PROCESSORS	B-27
B.4.	MSRS IN PENTIUM PROCESSORS	B-36
B.5.	ARCHITECTURAL MSRS	B-37

APPENDIX C**MP INITIALIZATION FOR P6 FAMILY PROCESSORS**

C.1.	OVERVIEW OF THE MP INITIALIZATION PROCESS FOR P6 FAMILY PROCESSORS	C-1
C.2.	MP INITIALIZATION PROTOCOL ALGORITHM	C-2
C.2.1.	Error Detection and Handling During the MP Initialization Protocol	C-4

APPENDIX D**PROGRAMMING THE LINT0 AND LINT1 INPUTS**

D.1.	CONSTANTS	D-1
D.2.	LINT[0:1] PINS PROGRAMMING PROCEDURE	D-1

APPENDIX E**INTERPRETING MACHINE-CHECK
ERROR CODES****APPENDIX F****APIC BUS MESSAGE FORMATS**

F.1.	BUS MESSAGE FORMATS	F-1
F.2.	EOI MESSAGE	F-1
F.2.1.	Short Message	F-2
F.2.2.	Non-focused Lowest Priority Message	F-3
F.2.3.	APIC Bus Status Cycles	F-5

FIGURES

	PAGE
Figure 1-1. Bit and Byte Order	1-5
Figure 2-1. IA-32 System-Level Registers and Data Structures	2-2
Figure 2-2. Transitions Among the Processor's Operating Modes	2-7
Figure 2-3. System Flags in the EFLAGS Register	2-8
Figure 2-4. Memory Management Registers	2-10
Figure 2-5. Control Registers	2-12
Figure 3-1. Segmentation and Paging	3-2
Figure 3-2. Flat Model	3-4
Figure 3-3. Protected Flat Model	3-4
Figure 3-4. Multi-Segment Model	3-5
Figure 3-5. Logical Address to Linear Address Translation	3-7
Figure 3-6. Segment Selector	3-8
Figure 3-7. Segment Registers	3-9
Figure 3-8. Segment Descriptor	3-10
Figure 3-9. Segment Descriptor When Segment-Present Flag Is Clear	3-12
Figure 3-10. Global and Local Descriptor Tables	3-16
Figure 3-11. Pseudo-Descriptor Format	3-17
Figure 3-12. Linear Address Translation (4-KByte Pages)	3-21
Figure 3-13. Linear Address Translation (4-MByte Pages)	3-22
Figure 3-14. Format of Page-Directory and Page-Table Entries for 4-KByte Pages and 32-Bit Physical Addresses	3-24
Figure 3-15. Format of Page-Directory Entries for 4-MByte Pages and 32-Bit Addresses	3-25
Figure 3-16. Format of a Page-Table or Page-Directory Entry for a Not-Present Page	3-28
Figure 3-17. Register CR3 Format When the Physical Address Extension is Enabled	3-29
Figure 3-18. Linear Address Translation With PAE Enabled (4-KByte Pages)	3-30
Figure 3-19. Linear Address Translation With PAE Enabled (2-MByte Pages)	3-31
Figure 3-20. Format of Page-Directory-Pointer-Table, Page-Directory, and Page-Table Entries for 4-KByte Pages with PAE Enabled	3-32
Figure 3-21. Format of Page-Directory-Pointer-Table and Page-Directory Entries for 2-MByte Pages with PAE Enabled	3-33
Figure 3-22. Linear Address Translation (4-MByte Pages)	3-35
Figure 3-23. Format of Page-Directory Entries for 4-MByte Pages and 36-Bit Physical Addresses	3-35
Figure 3-24. Memory Management Convention That Assigns a Page Table to Each Segment	3-36
Figure 4-1. Descriptor Fields Used for Protection	4-4
Figure 4-2. Protection Rings	4-8
Figure 4-3. Privilege Check for Data Access	4-10
Figure 4-4. Examples of Accessing Data Segments From Various Privilege Levels	4-11
Figure 4-5. Privilege Check for Control Transfer Without Using a Gate	4-13
Figure 4-6. Examples of Accessing Conforming and Nonconforming Code Segments From Various Privilege Levels	4-15
Figure 4-7. Call-Gate Descriptor	4-17
Figure 4-8. Call-Gate Mechanism	4-19
Figure 4-9. Privilege Check for Control Transfer with Call Gate	4-19

	PAGE
Figure 4-10. Example of Accessing Call Gates At Various Privilege Levels.	4-21
Figure 4-11. Stack Switching During an Interprivilege-Level Call	4-23
Figure 4-12. Use of RPL to Weaken Privilege Level of Called Procedure	4-30
Figure 5-1. Relationship of the IDTR and IDT.	5-12
Figure 5-2. IDT Gate Descriptors	5-13
Figure 5-3. Interrupt Procedure Call	5-15
Figure 5-4. Stack Usage on Transfers to Interrupt and Exception-Handling Routines	5-16
Figure 5-5. Interrupt Task Switch	5-19
Figure 5-6. Error Code	5-20
Figure 5-7. Page-Fault Error Code	5-44
Figure 6-1. Structure of a Task	6-2
Figure 6-2. 32-Bit Task-State Segment (TSS)	6-5
Figure 6-3. TSS Descriptor	6-8
Figure 6-4. Task Register	6-10
Figure 6-5. Task-Gate Descriptor	6-10
Figure 6-6. Task Gates Referencing the Same Task	6-12
Figure 6-7. Nested Tasks	6-16
Figure 6-8. Overlapping Linear-to-Physical Mappings	6-20
Figure 6-9. 16-Bit TSS Format	6-22
Figure 7-1. Example of Write Ordering in Multiple-Processor Systems	7-10
Figure 7-2. Interpretation of APIC ID in MP Systems	7-23
Figure 7-3. IA-32 Processor with Intel Hyper-Threading Technology using Two Logical Processors	7-24
Figure 7-4. Local APICs and I/O APIC When IA-32 Processors with Hyper-Threading Technology Are Used in MP Systems	7-32
Figure 7-5. Interpretation of the APIC ID.	7-34
Figure 8-1. Relationship of Local APIC and I/O APIC In Single-Processor Systems	8-3
Figure 8-2. Local APICs and I/O APIC When Intel Xeon Processors Are Used in Multiple-Processor Systems	8-4
Figure 8-3. Local APICs and I/O APIC When P6 Family Processors Are Used in Multiple-Processor Systems	8-4
Figure 8-4. Local APIC Structure	8-7
Figure 8-5. IA32_APIC_BASE MSR	8-10
Figure 8-6. Local APIC ID Register.	8-12
Figure 8-7. Local APIC Version Register	8-14
Figure 8-8. Local Vector Table (LVT)	8-16
Figure 8-9. Error Status Register (ESR)	8-19
Figure 8-10. Divide Configuration Register.	8-20
Figure 8-11. Initial Count and Current Count Registers	8-20
Figure 8-12. Interrupt Command Register (ICR).	8-22
Figure 8-13. Logical Destination Register (LDR)	8-28
Figure 8-14. Destination Format Register (DFR)	8-28
Figure 8-15. Arbitration Priority Register (APR)	8-30
Figure 8-16. Interrupt Acceptance Flow Chart for the Local APIC (Pentium 4 and Intel Xeon Processors)	8-32
Figure 8-17. Interrupt Acceptance Flow Chart for the Local APIC (P6 Family and Pentium Processors)	8-33
Figure 8-18. Task Priority Register (TPR).	8-35
Figure 8-19. Processor Priority Register (PPR)	8-35
Figure 8-20. IRR, ISR and TMR Registers	8-36
Figure 8-21. EOI Register	8-37

	PAGE
Figure 8-22. Spurious-Interrupt Vector Register (SVR)	8-39
Figure 8-23. Layout of the MSI Message Address Register	8-41
Figure 8-24. Layout of the MSI Message Data Register	8-42
Figure 9-1. Contents of CR0 Register after Reset	9-5
Figure 9-2. Version Information in the EDX Register after Reset	9-5
Figure 9-3. Processor State After Reset	9-18
Figure 9-4. Constructing Temporary GDT and Switching to Protected Mode (Lines 162-172 of List File)	9-27
Figure 9-5. Moving the GDT, IDT and TSS from ROM to RAM (Lines 196-261 of List File)	9-28
Figure 9-6. Task Switching (Lines 282-296 of List File)	9-29
Figure 9-7. Integrating Processor Specific Updates	9-32
Figure 9-8. Format of the Microcode Update Data Block	9-35
Figure 9-9. Write Operation Flow Chart	9-47
Figure 10-1. Cache Structure of the Pentium 4 and Intel Xeon Processors	10-1
Figure 10-2. Cache-Control Registers and Bits Available in IA-32 Processors.	10-12
Figure 10-3. Mapping Physical Memory With MTRRs.	10-25
Figure 10-4. IA32_MTRRCAP Register	10-26
Figure 10-5. IA32_MTRR_DEF_TYPE MSR.	10-27
Figure 10-6. IA32_MTRR_PHYSBASEn and IA32_MTRR_PHYSMASKn Variable-Range Register Pair	10-30
Figure 10-7. IA32_CR_PAT MSR	10-40
Figure 11-1. Mapping of MMX Registers to Floating-Point Registers	11-2
Figure 11-2. Mapping of MMX Registers to x87 FPU Data Register Stack.	11-7
Figure 12-1. Example of Saving the x87 FPU, MMX, SSE, and SSE2 State During an Operating-System Controlled Task Switch	12-9
Figure 13-1. SMRAM Usage	13-5
Figure 13-2. SMM Revision Identifier	13-13
Figure 13-3. Auto HALT Restart Field	13-14
Figure 13-4. SMBASE Relocation Field	13-15
Figure 13-5. I/O Instruction Restart Field	13-16
Figure 13-6. Processor Modulation Through Stop-Clock Mechanism.	13-19
Figure 13-7. MSR_THERM2_CTL Register	13-21
Figure 13-8. IA32_THERM_STATUS MSR.	13-22
Figure 13-9. IA32_THERM_INTERRUPT MSR	13-22
Figure 13-10. IA32_THERM_CONTROL MSR	13-23
Figure 14-1. Machine-Check MSRs	14-2
Figure 14-2. IA32_MCG_CAP Register	14-3
Figure 14-3. MCG_CAP Register	14-4
Figure 14-4. IA32_MCG_STATUS Register	14-4
Figure 14-5. IA32_MCi_CTL Register	14-6
Figure 14-6. IA32_MCi_STATUS Register	14-6
Figure 14-7. IA32_MCi_ADDR MSR	14-8
Figure 15-1. Debug Registers	15-3
Figure 15-2. IA32_DEBUGCTL MSR (Pentium 4 and Intel Xeon Processors)	15-13
Figure 15-3. LBR MSR Stack Structure	15-14
Figure 15-4. MSR_LASTBRANCH_TOS MSR Layout	15-14
Figure 15-5. LBR MSR Branch Record Layout	15-15
Figure 15-6. DebugCtlMSR Register (P6 Family Processors)	15-21
Figure 15-7. Event Selection Control Register (ESCR) for Pentium 4 and Intel Xeon processors without HT Technology support	15-29
Figure 15-8. Performance Counter (Pentium 4 and Intel Xeon Processors)	15-31

	PAGE
Figure 15-9. Counter Configuration Control Register (CCCR)	15-33
Figure 15-10. DS Save Area.	15-36
Figure 15-11. Branch Trace Record Format.	15-37
Figure 15-12. PEBS Record Format.	15-38
Figure 15-13. Event Example	15-39
Figure 15-14. Effects of Edge Filtering	15-42
Figure 15-15. Event Selection Control Register (ESCR) for the Pentium 4 processor, Intel Xeon processor and Intel Xeon processor MP with HT Technology support	15-56
Figure 15-16. Counter Configuration Control Register (CCCR)	15-58
Figure 15-17. PerfEvtSel0 and PerfEvtSel1 MSRs.	15-62
Figure 15-18. CESR MSR (Pentium Processor Only).	15-66
Figure 16-1. Real-Address Mode Address Translation.	16-4
Figure 16-2. Interrupt Vector Table in Real-Address Mode	16-7
Figure 16-3. Entering and Leaving Virtual-8086 Mode	16-12
Figure 16-4. Privilege Level 0 Stack After Interrupt or Exception in Virtual-8086 Mode	16-18
Figure 16-5. Software Interrupt Redirection Bit Map in TSS.	16-25
Figure 17-1. Stack after Far 16- and 32-Bit Calls	17-6
Figure 18-1. I/O Map Base Address Differences	18-31
Figure C-1. MP System With Multiple Pentium III Processors.	C-3

TABLES

	PAGE
Table 2-1.	Action Taken By x87 FPU Instructions for Different Combinations of EM, MP and TS
	2-15
Table 2-2.	Summary of System Instructions
	2-19
Table 3-1.	Code- and Data-Segment Types
	3-13
Table 3-2.	System-Segment and Gate-Descriptor Types
	3-15
Table 3-3.	Page Sizes and Physical Address Sizes
	3-20
Table 4-1.	Privilege Check Rules for Call Gates
	4-20
Table 4-2.	Combined Page-Directory and Page-Table Protection.
	4-34
Table 5-1.	Protected-Mode Exceptions and Interrupts
	5-3
Table 5-2.	Priority Among Simultaneous Exceptions and Interrupts
	5-10
Table 5-3.	Interrupt and Exception Classes.
	5-31
Table 5-4.	Conditions for Generating a Double Fault
	5-32
Table 5-5.	Invalid TSS Conditions
	5-34
Table 5-6.	Alignment Requirements by Data Type
	5-49
Table 5-7.	SIMD Floating-Point Exceptions Priority.
	5-54
Table 6-1.	Exception Conditions Checked During a Task Switch
	6-15
Table 6-2.	Effect of a Task Switch on Busy Flag, NT Flag, Previous Task Link Field, and TS Flag
	6-17
Table 7-1.	Initial APIC IDs for the Logical Processors in a System that has Four MP-Type Intel Xeon Processors with Hyper-Threading Technology
	7-34
Table 8-1.	Local APIC Register Address Map
	8-8
Table 8-2.	Valid Combinations for the Pentium 4 and Intel Xeon Processors' Local xAPIC Interrupt Command Register
	8-25
Table 8-3.	Valid Combinations for the P6 Family Processors' Local APIC Interrupt Command Register
	8-26
Table 9-1.	32-Bit IA-32 Processor States Following Power-up, Reset, or INIT
	9-3
Table 9-2.	Recommended Settings of EM and MP Flags on IA-32 processors.
	9-7
Table 9-3.	Software Emulation Settings of EM, MP, and NE Flags.
	9-8
Table 9-4.	Main Initialization Steps in STARTUP.ASM Source Listing
	9-18
Table 9-5.	Relationship Between BLD Item and ASM Source File
	9-31
Table 9-6.	Processor MSR Register Components.
	9-33
Table 9-7.	Microcode Update Encoding Format
	9-34
Table 9-8.	Microcode Update Functions
	9-43
Table 9-9.	Parameters for the Presence Test
	9-44
Table 9-10.	Parameters for the Write Update Data Function.
	9-45
Table 9-11.	Parameters for the Control Update Sub-function
	9-48
Table 9-12.	Mnemonic Values
	9-48
Table 9-13.	Parameters for the Read Microcode Update Data Function.
	9-49
Table 9-14.	Return Code Definitions
	9-50
Table 10-1.	Characteristics of the Caches, TLBs, Store Buffer, and Write Combining Buffer in IA-32 processors
	10-2
Table 10-2.	Memory Types and Their Properties
	10-6
Table 10-3.	Methods of Caching Available in Pentium 4, Intel Xeon, P6 Family, and Pentium Processors.
	10-7
Table 10-4.	MESI Cache Line States.
	10-10
Table 10-5.	Cache Operating Modes.
	10-13
Table 10-6.	Effective Page-Level Memory Type for Pentium Pro and Pentium II Processors*
	10-16

Table 10-7.	Effective Page-Level Memory Types for Pentium III, Pentium 4, and Intel Xeon Processors.	10-17
Table 10-8.	Memory Types That Can Be Encoded in MTRRs.	10-25
Table 10-9.	Address Mapping for Fixed-Range MTRRs	10-29
Table 10-10.	Memory Types That Can Be Encoded With PAT	10-41
Table 10-11.	Selection of PAT Entries with PAT, PCD, and PWT Flags.	10-41
Table 10-12.	Memory Type Setting of PAT Entries Following a Power-up or Reset.	10-42
Table 11-1.	Action Taken By MMX Instructions for Different Combinations of EM, MP and TS	11-1
Table 11-2.	Effects of MMX Instructions on x87 FPU State.	11-3
Table 11-3.	Effect of the MMX, x87 FPU, and FXSAVE/FXRSTOR Instructions on the x87 FPU Tag Word.	11-4
Table 12-1.	Action Taken for Combinations of OSFXSR, OSXMMEXCPT, SSE, SSE2, EM, MP, and TS1	12-3
Table 13-1.	SMRAM State Save Map	13-5
Table 13-2.	Processor Register Initialization in SMM	13-9
Table 13-3.	Auto HALT Restart Flag Values	13-14
Table 13-4.	I/O Instruction Restart Field Values	13-17
Table 13-5.	On-Demand Clock Modulation Duty Cycle Field Encoding	13-24
Table 14-1.	Extended Machine-Check State MSRs	14-9
Table 14-2.	Simple Error Codes	14-13
Table 14-3.	General Forms of Compound Error Codes.	14-13
Table 14-4.	Encoding for TT (Transaction Type) Sub-Field.	14-14
Table 14-5.	Level Encoding for LL (Memory Hierarchy Level) Sub-Field	14-14
Table 14-6.	Encoding of Request (RRRR) Sub-Field	14-14
Table 14-7.	Encodings of PP, T, and II Sub-Fields	14-15
Table 15-1.	Breakpointing Examples.	15-7
Table 15-2.	Debug Exception Conditions	15-8
Table 15-3.	IA32_DEBUGCTL MSR Flag Encodings	15-18
Table 15-4.	Performance Counter MSRs and Associated CCCR and ESCR MSRs (Pentium 4 and Intel Xeon Processors)	15-25
Table 15-5.	CCR Names and Bit Positions	15-45
Table 15-6.	Effect of Logical Processor and CPL Qualification for Logical-Processor-Specific (TS) Events.	15-60
Table 15-7.	Effect of logical processor and CPL qualification for non-logical-processor-specific (TI) events	15-61
Table 16-1.	Real-Address Mode Exceptions and Interrupts	16-8
Table 16-2.	Software Interrupt Handling Methods While in Virtual-8086 Mode.	16-24
Table 17-1.	Characteristics of 16-Bit and 32-Bit Program Modules.	17-1
Table 18-1.	New Instruction in the Pentium Processor and Later IA-32 Processors	18-4
Table 18-2.	Recommended Values of the EM, MP, and NE Flags for Intel486 SX Microprocessor/Intel 487 SX Math Coprocessor System.	18-20
Table 18-3.	EM and MP Flag Interpretation.	18-20
Table A-1.	Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting.	A-2
Table A-2.	Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting	A-28
Table A-3.	List of Metrics Available for Front_end Tagging (For Front_end Event Only)	A-34
Table A-4.	List of Metrics Available for Execution Tagging (For Execution Event only)	A-35

	PAGE
Table A-5. List of Metrics Available for ReplayTagging (For Replay Event only)	A-36
Table A-6. Event Mask Qualification for Logical Processors	A-37
Table A-7. Performance Monitoring Events on Intel® Pentium® M Processors	A-42
Table A-8. Performance Monitoring Events Modified on Intel® Pentium® M Processors	A-44
Table A-9. Events That Can Be Counted with the P6 Family Performance- Monitoring Counters	A-45
Table A-10. Events That Can Be Counted with the Pentium Processor Performance- Monitoring Counters	A-60
Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors	B-1
Table B-2. MSRs in Pentium M Processors	B-20
Table B-3. MSRs in the P6 Family Processors	B-27
Table B-4. MSRs in the Pentium Processor	B-36
Table B-5. IA-32 Architectural MSRs	B-37
Table C-1. Boot Phase IPI Message Format	C-2
Table E-1. Encoding of the MCI_STATUS Register for External Bus Errors	E-1
Table F-1. EOI Message (14 Cycles)	F-1
Table F-2. Short Message (21 Cycles)	F-2
Table F-3. Non-Focused Lowest Priority Message (34 Cycles)	F-3
Table F-4. APIC Bus Status Cycles Interpretation	F-5

1

About This Manual

CHAPTER 1

ABOUT THIS MANUAL

The *IA-32 Intel[®] Architecture Software Developer's Manual, Volume 3: System Programming Guide* (Order Number 245472), is part of a three-volume set that describes the architecture and programming environment of all IA-32 Intel Architecture processors. The other two volumes in this set are:

- The *IA-32 Intel Architecture Software Developer's Manual, Volume 1: Basic Architecture* (Order Number 245470)
- The *IA-32 Intel Architecture Software Developer's Manual, Volume 2: Instruction Set Reference* (Order Number 245471).

The *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, describes the basic architecture and programming environment of an IA-32 processor; the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*, describes the instruction set of the processor and the opcode structure. These two volumes are aimed at application programmers who are writing programs to run under existing operating systems or executives. The *IA-32 Intel Architecture Software Developer's Manual, Volume 3*, describes the operating-system support environment of an IA-32 processor, including memory management, protection, task management, interrupt and exception handling, and system management mode. It also provides IA-32 processor compatibility information. This volume is aimed at operating-system and BIOS designers and programmers.

1.1. IA-32 PROCESSORS COVERED IN THIS MANUAL

This manual includes information pertaining primarily to the most recent IA-32 processors, which include the Pentium[®] processors, the P6 family processors, the Pentium 4 processors, and the Intel[®] Xeon[™] processors. The P6 family processors are those IA-32 processors based on the P6 family micro-architecture, which include the Pentium Pro, Pentium II, and Pentium III processors. The Pentium 4 and Intel Xeon processors are based on the Intel[®] NetBurst[™] micro-architecture.

1.2. OVERVIEW OF THE IA-32 INTEL ARCHITECTURE SOFTWARE DEVELOPER'S MANUAL, VOLUME 3: SYSTEM PROGRAMMING GUIDE

The contents of this manual are as follows:

Chapter 1 — About This Manual. Gives an overview of all three volumes of the *IA-32 Intel Architecture Software Developer's Manual*. It also describes the notational conventions in these manuals and lists related Intel manuals and documentation of interest to programmers and hardware designers.

Chapter 2 — System Architecture Overview. Describes the modes of operation of an IA-32 processor and the mechanisms provided in the IA-32 architecture to support operating systems and executives, including the system-oriented registers and data structures and the system-oriented instructions. The steps necessary for switching between real-address and protected modes are also identified.

Chapter 3 — Protected-Mode Memory Management. Describes the data structures, registers, and instructions that support segmentation and paging and explains how they can be used to implement a “flat” (unsegmented) memory model or a segmented memory model.

Chapter 4 — Protection. Describes the support for page and segment protection provided in the IA-32 architecture. This chapter also explains the implementation of privilege rules, stack switching, pointer validation, user and supervisor modes.

Chapter 5 — Interrupt and Exception Handling. Describes the basic interrupt mechanisms defined in the IA-32 architecture, shows how interrupts and exceptions relate to protection, and describes how the architecture handles each exception type. Reference information for each IA-32 exception is given at the end of this chapter.

Chapter 6 — Task Management. Describes the mechanisms the IA-32 architecture provides to support multitasking and inter-task protection.

Chapter 7 — Multiple-Processor Management. Describes the instructions and flags that support multiple processors with shared memory, memory ordering, and Hyper-Threading Technology.

Chapter 8 — Advanced Programmable Interrupt Controller (APIC). Describes the programming interface to the local APIC and gives an overview of the interface between the local APIC and the I/O APIC.

Chapter 9 — Processor Management and Initialization. Defines the state of an IA-32 processor after reset initialization. This chapter also explains how to set up an IA-32 processor for real-address mode operation and protected- mode operation, and how to switch between modes.

Chapter 10 — Memory Cache Control. Describes the general concept of caching and the caching mechanisms supported by the IA-32 architecture. This chapter also describes the memory type range registers (MTRRs) and how they can be used to map memory types of physical memory. Information on using the new cache control and memory streaming instructions introduced with the Pentium III, Pentium 4, and Intel Xeon processors is also given.

Chapter 11 — Intel® MMX™ Technology System Programming. Describes those aspects of the Intel MMX technology that must be handled and considered at the system programming level, including task switching, exception handling, and compatibility with existing system environments. The Intel MMX technology was introduced into the IA-32 architecture with the Pentium processor.

Chapter 12 — SSE and SSE2 System Programming. Describes those aspects of SSE and SSE2 extensions that must be handled and considered at the system programming level, including task switching, exception handling, and compatibility with existing system environments.

Chapter 13 — System Management. Describes the IA-32 architecture's system management mode (SMM) and the thermal monitoring facilities.

Chapter 14 — Machine-Check Architecture. Describes the machine-check architecture.

Chapter 15 — Debugging and Performance Monitoring. Describes the debugging registers and other debug mechanism provided in the IA-32 architecture. This chapter also describes the time-stamp counter and the performance-monitoring counters.

Chapter 16 — 8086 Emulation. Describes the real-address and virtual-8086 modes of the IA-32 architecture.

Chapter 17 — Mixing 16-Bit and 32-Bit Code. Describes how to mix 16-bit and 32-bit code modules within the same program or task.

Chapter 18 — IA-32 Architecture Compatibility. Describes architectural compatibility among the IA-32 processors, which include the Intel 286, Intel386™, Intel486™, Pentium, P6 family, Pentium 4, and Intel Xeon processors. The P6 family includes the Pentium Pro, Pentium II, and Pentium III processors. The differences among the 32-bit IA-32 processors are also described throughout the three volumes of the *IA-32 Software Developer's Manual*, as relevant to particular features of the architecture. This chapter provides a collection of all the relevant compatibility information for all IA-32 processors and also describes the basic differences with respect to the 16-bit IA-32 processors (the Intel 8086 and Intel 286 processors).

Appendix A — Performance-Monitoring Events. Lists the events that can be counted with the performance-monitoring counters and the codes used to select these events. Both Pentium processor and P6 family processor events are described.

Appendix B — Model Specific Registers (MSRs). Lists the MSRs available in the Pentium processors, the P6 family processors, and the Pentium 4 and Intel Xeon processors and describes their functions.

Appendix C — MP Initialization For P6 Family Processors. Gives an example of how to use of the MP protocol to boot P6 family processors in a MP system.

Appendix D — Programming the LINT0 and LINT1 Inputs. Gives an example of how to program the LINT0 and LINT1 pins for specific interrupt vectors.

Appendix E — Interpreting Machine-Check Error Codes. Gives an example of how to interpret the error codes for a machine-check error that occurred on a P6 family processor.

Appendix F — APIC Bus Message Formats. Describes the message formats for messages transmitted on the APIC bus for P6 family and Pentium processors.

1.3. NOTATIONAL CONVENTIONS

This manual uses specific notation for data-structure formats, for symbolic representation of instructions, and for hexadecimal and binary numbers. A review of this notation makes the manual easier to read.

1.3.1. Bit and Byte Order

In illustrations of data structures in memory, smaller addresses appear toward the bottom of the figure; addresses increase toward the top. Bit positions are numbered from right to left. The numerical value of a set bit is equal to two raised to the power of the bit position. IA-32 processors are “little endian” machines; this means the bytes of a word are numbered starting from the least significant byte. Figure 1-1 illustrates these conventions.

1.3.2. Reserved Bits and Software Compatibility

In many register and memory layout descriptions, certain bits are marked as **reserved**. When bits are marked as reserved, it is essential for compatibility with future processors that software treat these bits as having a future, though unknown, effect. The behavior of reserved bits should be regarded as not only undefined, but unpredictable. Software should follow these guidelines in dealing with reserved bits:

- Do not depend on the states of any reserved bits when testing the values of registers which contain such bits. Mask out the reserved bits before testing.
- Do not depend on the states of any reserved bits when storing to memory or to a register.
- Do not depend on the ability to retain information written into any reserved bits.
- When loading a register, always load the reserved bits with the values indicated in the documentation, if any, or reload them with values previously read from the same register.

NOTE

Avoid any software dependence upon the state of reserved bits in IA-32 registers. Depending upon the values of reserved register bits will make software dependent upon the unspecified manner in which the processor handles these bits. Programs that depend upon reserved values risk incompatibility with future processors.

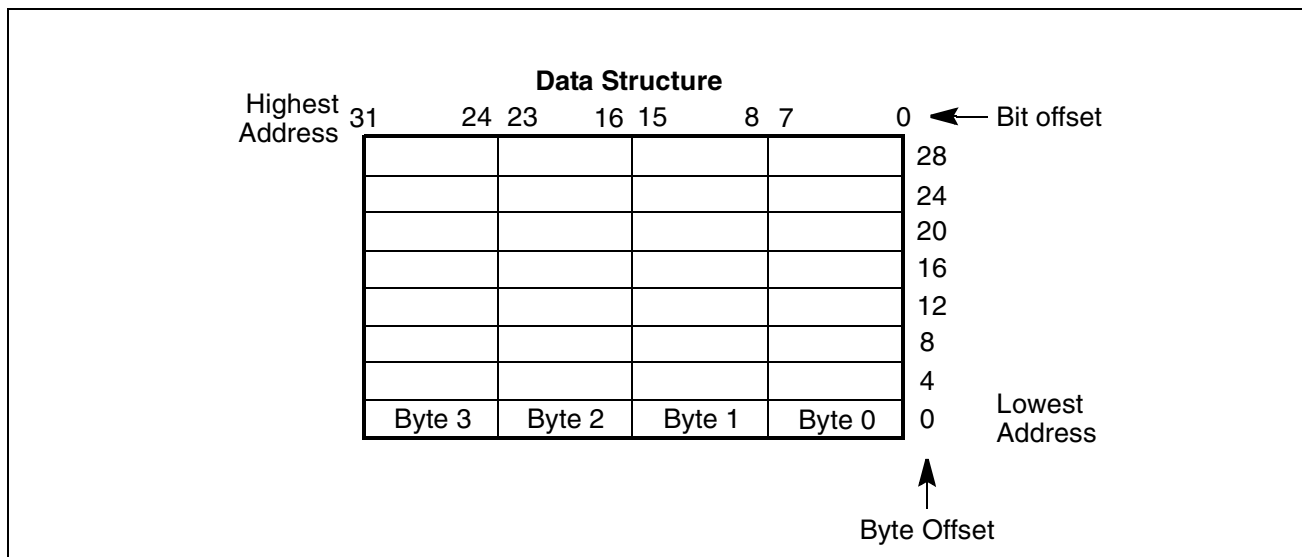


Figure 1-1. Bit and Byte Order

1.3.3. Instruction Operands

When instructions are represented symbolically, a subset of the IA-32 assembly language is used. In this subset, an instruction has the following format:

label: mnemonic argument1, argument2, argument3

where:

- A **label** is an identifier which is followed by a colon.
- A **mnemonic** is a reserved name for a class of instruction opcodes which have the same function.
- The operands **argument1**, **argument2**, and **argument3** are optional. There may be from zero to three operands, depending on the opcode. When present, they take the form of either literals or identifiers for data items. Operand identifiers are either reserved names of registers or are assumed to be assigned to data items declared in another part of the program (which may not be shown in the example).

When two operands are present in an arithmetic or logical instruction, the right operand is the source and the left operand is the destination.

For example:

LOADREG: MOV EAX, SUBTOTAL

In this example LOADREG is a label, MOV is the mnemonic identifier of an opcode, EAX is the destination operand, and SUBTOTAL is the source operand. Some assembly languages put the source and destination in reverse order.

1.3.4. Hexadecimal and Binary Numbers

Base 16 (hexadecimal) numbers are represented by a string of hexadecimal digits followed by the character H (for example, F82EH). A hexadecimal digit is a character from the following set: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, and F.

Base 2 (binary) numbers are represented by a string of 1s and 0s, sometimes followed by the character B (for example, 1010B). The “B” designation is only used in situations where confusion as to the type of number might arise.

1.3.5. Segmented Addressing

The processor uses byte addressing. This means memory is organized and accessed as a sequence of bytes. Whether one or more bytes are being accessed, a byte address is used to locate the byte or bytes memory. The range of memory that can be addressed is called an **address space**.

The processor also supports segmented addressing. This is a form of addressing where a program may have many independent address spaces, called **segments**. For example, a program can keep its code (instructions) and stack in separate segments. Code addresses would always refer to the code space, and stack addresses would always refer to the stack space. The following notation is used to specify a byte address within a segment:

Segment-register:Byte-address

For example, the following segment address identifies the byte at address FF79H in the segment pointed to by the DS register:

DS:FF79H

The following segment address identifies an instruction address in the code segment. The CS register points to the code segment and the EIP register contains the address of the instruction.

CS:EIP

1.3.6. Exceptions

An exception is an event that typically occurs when an instruction causes an error. For example, an attempt to divide by zero generates an exception. However, some exceptions, such as break-points, occur under other conditions. Some types of exceptions may provide error codes. An error code reports additional information about the error. An example of the notation used to show an exception and error code is shown below.

#PF(fault code)

This example refers to a page-fault exception under conditions where an error code naming a type of fault is reported. Under some conditions, exceptions which produce error codes may not be able to report an accurate code. In this case, the error code is zero, as shown below for a general-protection exception.

#GP(0)

See Chapter 5, *Interrupt and Exception Handling*, for a list of exception mnemonics and their descriptions.

1.4. RELATED LITERATURE

Literature related to IA-32 processors is listed on-line at the following Intel web site:

<http://developer.intel.com/design/processors/>

Some of the documents listed at this web site can be viewed on-line; others can be ordered on-line. The literature available is listed by Intel processor and then by the following literature types: applications notes, data sheets, manuals, papers, and specification updates. The following literature may be of interest:

- Data Sheet for a particular Intel IA-32 processor.
- Specification Update for a particular Intel IA-32 processor.
- AP-485, *Intel Processor Identification and the CPUID Instruction*, Order Number 241618.
- *Intel® Pentium® 4 and Intel® Xeon™ Processor Optimization Reference Manual*, Order Number 248966.

2

System Architecture Overview

CHAPTER 2

SYSTEM ARCHITECTURE OVERVIEW

The IA-32 architecture (beginning with the Intel386 processor family) provides extensive support for operating-system and system-development software. This support is part of the IA-32 system-level architecture and includes features to assist in the following operations:

- Memory management
- Protection of software modules
- Multitasking
- Exception and interrupt handling
- Multiprocessing
- Cache management
- Hardware resource and power management
- Debugging and performance monitoring

This chapter provides a brief overview of the IA-32 system-level architecture; a detailed description of each part of this architecture given in the following chapters. This chapter also describes the system registers that are used to set up and control the processor at the system level and gives a brief overview of the processor's system-level (operating system) instructions.

Many of the features of the IA-32 system-level architectural are used only by system programmers. Application programmers may need to read this chapter, and the following chapters which describe the use of these features, in order to understand the hardware facilities used by system programmers to create a reliable and secure environment for application programs.

NOTE

This overview and most of the subsequent chapters of this book focus on the “native” or protected-mode operation of the IA-32 architecture. As described in Chapter 9, *Processor Management and Initialization*, all IA-32 processors enter real-address mode following a power-up or reset. Software must then initiate a switch from real-address mode to protected mode.

2.1. OVERVIEW OF THE SYSTEM-LEVEL ARCHITECTURE

The IA-32 system-level architecture consists of a set of registers, data structures, and instructions designed to support basic system-level operations such as memory management, interrupt and exception handling, task management, and control of multiple processors (multiprocessing). Figure 2-1 provides a generalized summary of the system registers and data structures.

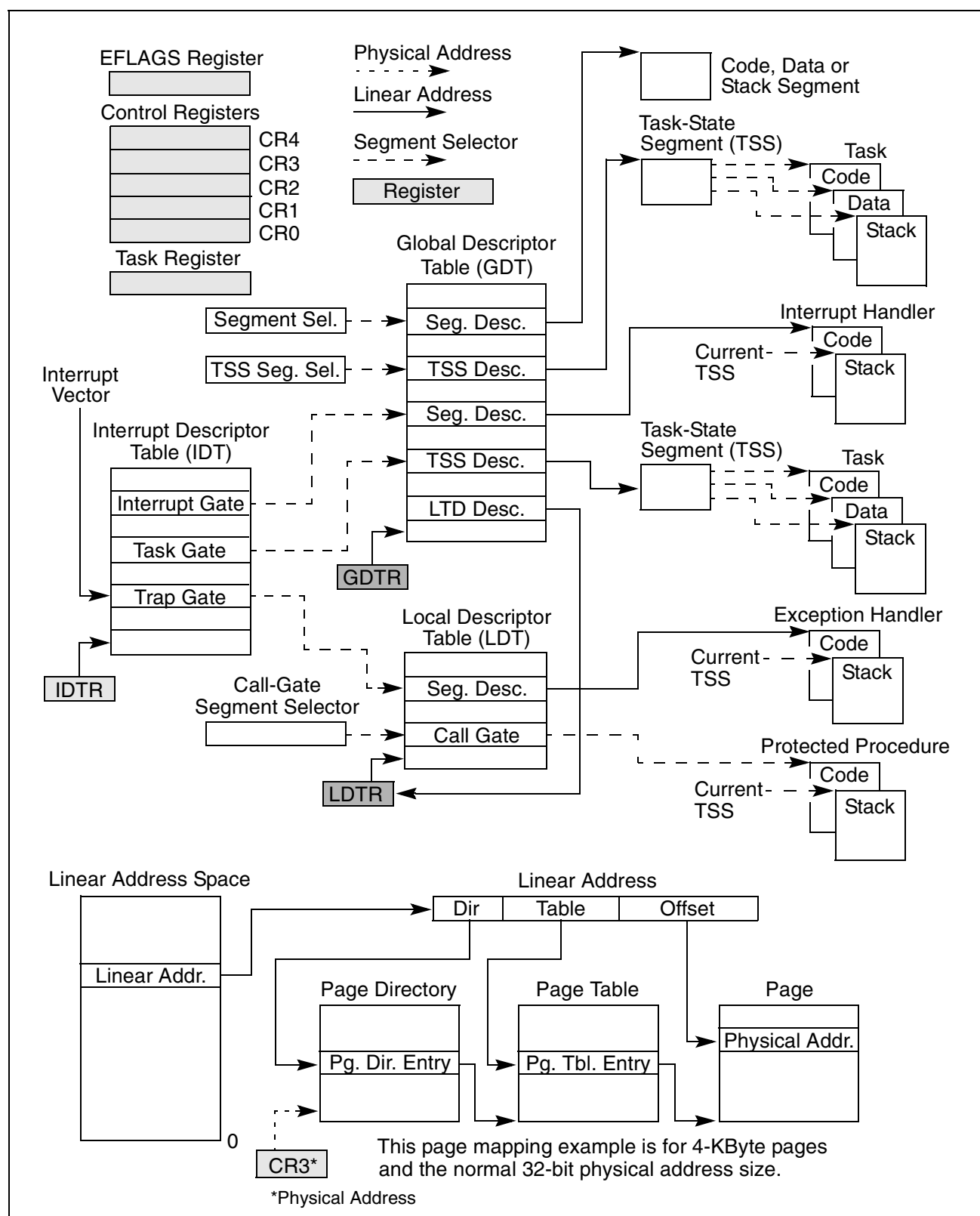


Figure 2-1. IA-32 System-Level Registers and Data Structures

2.1.1. Global and Local Descriptor Tables

When operating in protected mode, all memory accesses pass through either the global descriptor table (GDT) or the (optional) local descriptor table (LDT), shown in Figure 2-1. These tables contain entries called segment descriptors. A segment descriptor provides the base address of a segment and access rights, type, and usage information. Each segment descriptor has a segment selector associated with it. The segment selector provides an index into the GDT or LDT (to its associated segment descriptor), a global/local flag (that determines whether the segment selector points to the GDT or the LDT), and access rights information.

To access a byte in a segment, both a segment selector and an offset must be supplied. The segment selector provides access to the segment descriptor for the segment (in the GDT or LDT). From the segment descriptor, the processor obtains the base address of the segment in the linear address space. The offset then provides the location of the byte relative to the base address. This mechanism can be used to access any valid code, data, or stack segment in the GDT or LDT, provided the segment is accessible from the current privilege level (CPL) at which the processor is operating. (The CPL is defined as the protection level of the currently executing code segment.)

In Figure 2-1 the solid arrows indicate a linear address, the dashed lines indicate a segment selector, and the dotted arrows indicate a physical address. For simplicity, many of the segment selectors are shown as direct pointers to a segment. However, the actual path from a segment selector to its associated segment is always through the GDT or LDT.

The linear address of the base of the GDT is contained in the GDT register (GDTR); the linear address of the LDT is contained in the LDT register (LDTR).

2.1.2. System Segments, Segment Descriptors, and Gates

Besides the code, data, and stack segments that make up the execution environment of a program or procedure, the system architecture also defines two system segments: the task-state segment (TSS) and the LDT. (The GDT is not considered a segment because it is not accessed by means of a segment selector and segment descriptor.) Each of these segment types has a segment descriptor defined for it.

The system architecture also defines a set of special descriptors called gates (the call gate, interrupt gate, trap gate, and task gate) that provide protected gateways to system procedures and handlers that operate at different privilege levels than application programs and procedures. For example, a CALL to a call gate provides access to a procedure in a code segment that is at the same or numerically lower privilege level (more privileged) than the current code segment. To access a procedure through a call gate, the calling procedure¹ must supply the selector of the call gate. The processor then performs an access rights check on the call gate, comparing the CPL with the privilege level of the call gate and the destination code segment pointed to by the call gate. If access to the destination code segment is allowed, the processor gets the segment selector for the destination code segment and an offset into that code segment from the call gate.

1. The word “procedure” is commonly used in this document as a general term for a logical unit or block of code (such as a program, procedure, function, or routine).

If the call requires a change in privilege level, the processor also switches to the stack for that privilege level. (The segment selector for the new stack is obtained from the TSS for the currently running task.) Gates also facilitate transitions between 16-bit and 32-bit code segments, and vice versa.

2.1.3. Task-State Segments and Task Gates

The TSS (see Figure 2-1) defines the state of the execution environment for a task. It includes the state of the general-purpose registers, the segment registers, the EFLAGS register, the EIP register, and segment selectors and stack pointers for three stack segments (one stack each for privilege levels 0, 1, and 2). It also includes the segment selector for the LDT associated with the task and the page-table base address.

All program execution in protected mode happens within the context of a task, called the current task. The segment selector for the TSS for the current task is stored in the task register. The simplest method of switching to a task is to make a call or jump to the task. Here, the segment selector for the TSS of the new task is given in the CALL or JMP instruction. In switching tasks, the processor performs the following actions:

1. Stores the state of the current task in the current TSS.
2. Loads the task register with the segment selector for the new task.
3. Accesses the new TSS through a segment descriptor in the GDT.
4. Loads the state of the new task from the new TSS into the general-purpose registers, the segment registers, the LDTR, control register CR3 (page-table base address), the EFLAGS register, and the EIP register.
5. Begins execution of the new task.

A task can also be accessed through a task gate. A task gate is similar to a call gate, except that it provides access (through a segment selector) to a TSS rather than a code segment.

2.1.4. Interrupt and Exception Handling

External interrupts, software interrupts, and exceptions are handled through the interrupt descriptor table (IDT), see Figure 2-1. The IDT contains a collection of gate descriptors, which provide access to interrupt and exception handlers. Like the GDT, the IDT is not a segment. The linear address of the base of the IDT is contained in the IDT register (IDTR).

The gate descriptors in the IDT can be of the interrupt-, trap-, or task-gate type. To access an interrupt or exception handler, the processor must first receive an interrupt vector (interrupt number) from internal hardware, an external interrupt controller, or from software by means of an INT, INTO, INT 3, or BOUND instruction. The interrupt vector provides an index into the IDT to a gate descriptor. If the selected gate descriptor is an interrupt gate or a trap gate, the associated handler procedure is accessed in a manner very similar to calling a procedure through a call gate. If the descriptor is a task gate, the handler is accessed through a task switch.

2.1.5. Memory Management

The system architecture supports either direct physical addressing of memory or virtual memory (through paging). When physical addressing is used, a linear address is treated as a physical address. When paging is used, all the code, data, stack, and system segments and the GDT and IDT can be paged, with only the most recently accessed pages being held in physical memory.

The location of pages (or page frames as they are sometimes called in the IA-32 architecture) in physical memory is contained in two types of system data structures (a page directory and a set of page tables), both of which reside in physical memory (see Figure 2-1). An entry in a page directory contains the physical address of the base of a page table, access rights, and memory management information. An entry in a page table contains the physical address of a page frame, access rights, and memory management information. The base physical address of the page directory is contained in control register CR3.

To use this paging mechanism, a linear address is broken into three parts, providing separate offsets into the page directory, the page table, and the page frame.

A system can have a single page directory or several. For example, each task can have its own page directory.

2.1.6. System Registers

To assist in initializing the processor and controlling system operations, the system architecture provides system flags in the EFLAGS register and several system registers:

- The system flags and IOPL field in the EFLAGS register control task and mode switching, interrupt handling, instruction tracing, and access rights. See Section 2.3., “System Flags and Fields in the EFLAGS Register”, for a description of these flags.
- The control registers (CR0, CR2, CR3, and CR4) contain a variety of flags and data fields for controlling system-level operations. Other flags in these registers are used to indicate support for specific processor capabilities within the operating system or executive. See Section 2.5., “Control Registers”, for a description of these flags.
- The debug registers (not shown in Figure 2-1) allow the setting of breakpoints for use in debugging programs and systems software. See Chapter 15, *Debugging and Performance Monitoring*, for a description of these registers.
- The GDTR, LDTR, and IDTR registers contain the linear addresses and sizes (limits) of their respective tables. See Section 2.4., “Memory-Management Registers”, for a description of these registers.
- The task register contains the linear address and size of the TSS for the current task. See Section 2.4., “Memory-Management Registers”, for a description of this register.
- Model-specific registers (not shown in Figure 2-1).

The model-specific registers (MSRs) are a group of registers available primarily to operating-system or executive procedures (that is, code running at privilege level 0). These registers control items such as the debug extensions, the performance-monitoring counters, the machine-check architecture, and the memory type ranges (MTRRs).

The number and functions of these registers varies among the different members of the IA-32 processor families. Section 9.4., “Model-Specific Registers (MSRs)”, for more information about the MSRs and Appendix B, *Model-Specific Registers (MSRs)*, for a complete list of the MSRs.

Most systems restrict access to all system registers (other than the EFLAGS register) by application programs. Systems can be designed, however, where all programs and procedures run at the most privileged level (privilege level 0), in which case application programs are allowed to modify the system registers.

2.1.7. Other System Resources

Besides the system registers and data structures described in the previous sections, the system architecture provides the following additional resources:

- Operating system instructions (see Section 2.6., “System Instruction Summary”).
- Performance-monitoring counters (not shown in Figure 2-1).
- Internal caches and buffers (not shown in Figure 2-1).

The performance-monitoring counters are event counters that can be programmed to count processor events such as the number of instructions decoded, the number of interrupts received, or the number of cache loads. See Section 15.8., “Performance Monitoring Overview”, for more information about these counters.

The processor provides several internal caches and buffers. The caches are used to store both data and instructions. The buffers are used to store things like decoded addresses to system and application segments and write operations waiting to be performed. See Chapter 10, *Memory Cache Control*, for a detailed discussion of the processor’s caches and buffers.

2.2. MODES OF OPERATION

The IA-32 architecture supports three operating modes and one quasi-operating mode:

- **Protected mode.** This is the native operating mode of the processor. In this mode all instructions and architectural features are available, providing the highest performance and capability. This is the recommended mode for all new applications and operating systems.
- **Real-address mode.** This operating mode provides the programming environment of the Intel 8086 processor, with a few extensions (such as the ability to switch to protected or system management mode).
- **System management mode (SMM).** The system management mode (SMM) is a standard architectural feature in all IA-32 processors, beginning with the Intel386 SL processor. This mode provides an operating system or executive with a transparent mechanism for implementing power management and OEM differentiation features. SMM is entered through activation of an external system interrupt pin (SMI#), which generates a system management interrupt (SMI). In SMM, the processor switches to a separate address space

while saving the context of the currently running program or task. SMM-specific code may then be executed transparently. Upon returning from SMM, the processor is placed back into its state prior to the SMI.

- **Virtual-8086 mode.** In protected mode, the processor supports a quasi-operating mode known as **virtual-8086 mode**. This mode allows the processor execute 8086 software in a protected, multitasking environment.

Figure 2-2 shows how the processor moves among these operating modes.

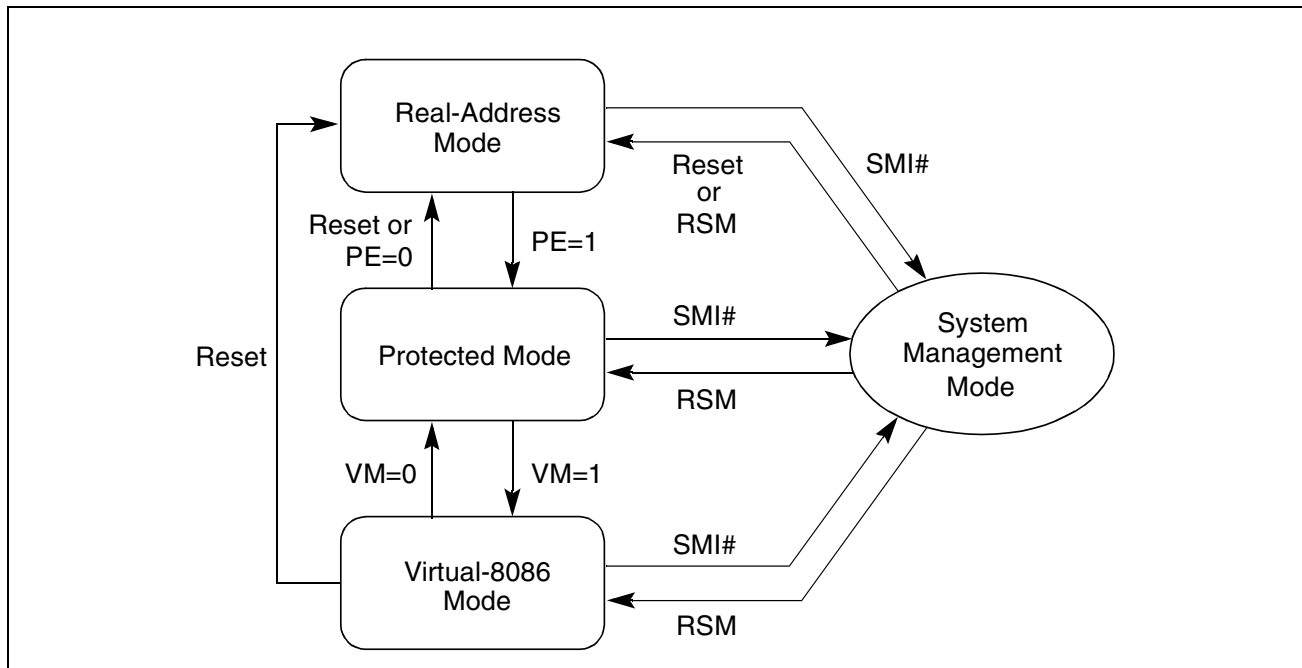


Figure 2-2. Transitions Among the Processor's Operating Modes

The processor is placed in real-address mode following power-up or a reset. Thereafter, the PE flag in control register CR0 controls whether the processor is operating in real-address or protected mode (see Section 2.5., “Control Registers”). See Section 9.9., “Mode Switching”, for detailed information on switching between real-address mode and protected mode.

The VM flag in the EFLAGS register determines whether the processor is operating in protected mode or virtual-8086 mode. Transitions between protected mode and virtual-8086 mode are generally carried out as part of a task switch or a return from an interrupt or exception handler (see Section 16.2.5., “Entering Virtual-8086 Mode”).

The processor switches to SMM whenever it receives an SMI while the processor is in real-address, protected, or virtual-8086 modes. Upon execution of the RSM instruction, the processor always returns to the mode it was in when the SMI occurred.

2.3. SYSTEM FLAGS AND FIELDS IN THE EFLAGS REGISTER

The system flags and IOPL field of the EFLAGS register control I/O, maskable hardware interrupts, debugging, task switching, and the virtual-8086 mode (see Figure 2-3). Only privileged code (typically operating system or executive code) should be allowed to modify these bits.

The functions of the system flags and IOPL are as follows:

TF **Trap (bit 8).** Set to enable single-step mode for debugging; clear to disable single-step mode. In single-step mode, the processor generates a debug exception after each instruction, which allows the execution state of a program to be inspected after each instruction. If an application program sets the TF flag using a POPF, POPFD, or IRET instruction, a debug exception is generated after the instruction that follows the POPF, POPFD, or IRET instruction.

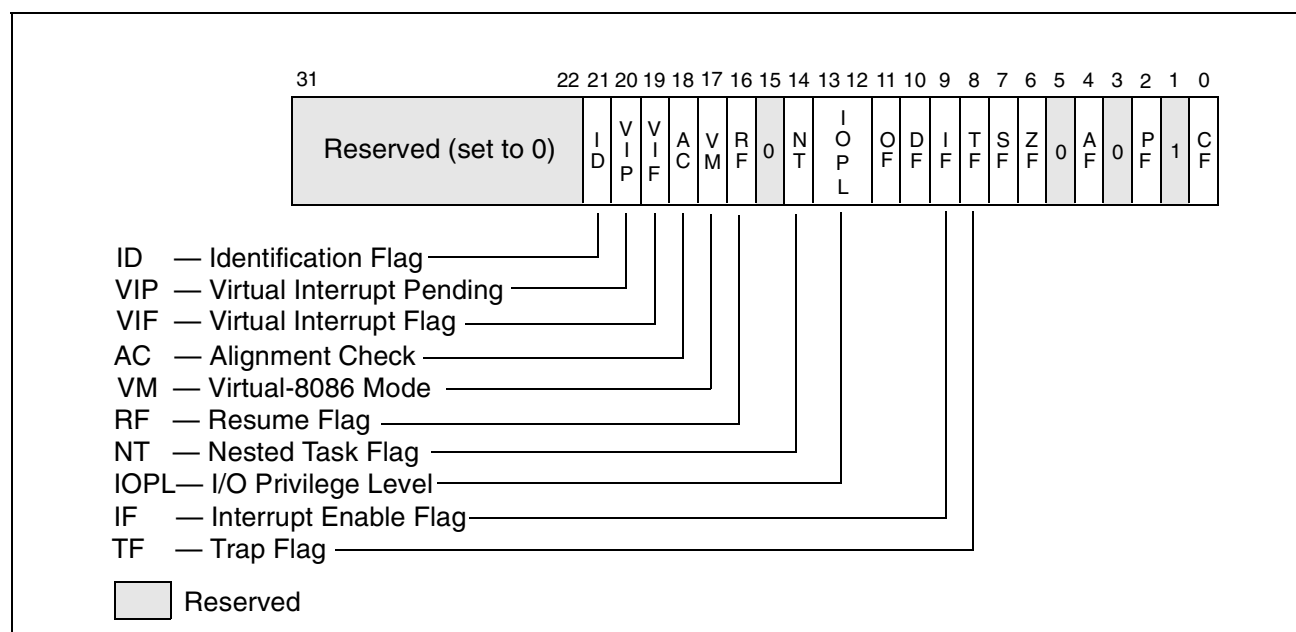


Figure 2-3. System Flags in the EFLAGS Register

IF **Interrupt enable (bit 9).** Controls the response of the processor to maskable hardware interrupt requests (see Section 5.3.2., “Maskable Hardware Interrupts”). Set to respond to maskable hardware interrupts; cleared to inhibit maskable hardware interrupts. The IF flag does not affect the generation of exceptions or nonmaskable interrupts (NMI interrupts). The CPL, IOPL, and the state of the VME flag in control register CR4 determine whether the IF flag can be modified by the CLI, STI, POPF, POPFD, and IRET instructions.

IOPL **I/O privilege level field (bits 12 and 13).** Indicates the I/O privilege level (IOPL) of the currently running program or task. The CPL of the currently running program or task must be less than or equal to the IOPL to access the I/O address space. This field can only be modified by the POPF and IRET instructions when operating at a CPL of 0. See Chapter 12, *Input/Output*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for more information on the relationship of the IOPL to I/O operations.

The IOPL is also one of the mechanisms that controls the modification of the IF flag and the handling of interrupts in virtual-8086 mode when the virtual mode extensions are in effect (the VME flag in control register CR4 is set).

NT Nested task (bit 14). Controls the chaining of interrupted and called tasks. The processor sets this flag on calls to a task initiated with a CALL instruction, an interrupt, or an exception. It examines and modifies this flag on returns from a task initiated with the IRET instruction. The flag can be explicitly set or cleared with the POPF/POPF instructions; however, changing to the state of this flag can generate unexpected exceptions in application programs. See Section 6.4., “Task Linking”, for more information on nested tasks.

RF Resume (bit 16). Controls the processor’s response to instruction-breakpoint conditions. When set, this flag temporarily disables debug exceptions (#DE) from being generated for instruction breakpoints; although, other exception conditions can cause an exception to be generated. When clear, instruction breakpoints will generate debug exceptions.

The primary function of the RF flag is to allow the restarting of an instruction following a debug exception that was caused by an instruction breakpoint condition. Here, debugger software must set this flag in the EFLAGS image on the stack just prior to returning to the interrupted program with the IRETD instruction, to prevent the instruction breakpoint from causing another debug exception. The processor then automatically clears this flag after the instruction returned to has been successfully executed, enabling instruction breakpoint faults again.

See Section 15.3.1.1., “Instruction-Breakpoint Exception Condition”, for more information on the use of this flag.

VM Virtual-8086 mode (bit 17). Set to enable virtual-8086 mode; clear to return to protected mode. See Section 16.2.1., “Enabling Virtual-8086 Mode”, for a detailed description of the use of this flag to switch to virtual-8086 mode.

AC Alignment check (bit 18). Set this flag and the AM flag in control register CR0 to enable alignment checking of memory references; clear the AC flag and/or the AM flag to disable alignment checking. An alignment-check exception is generated when reference is made to an unaligned operand, such as a word at an odd byte address or a doubleword at an address which is not an integral multiple of four. Alignment-check exceptions are generated only in user mode (privilege level 3). Memory references that default to privilege level 0, such as segment descriptor loads, do not generate this exception even when caused by instructions executed in user-mode.

The alignment-check exception can be used to check alignment of data. This is useful when exchanging data with other processors, which require all data to be aligned. The alignment-check exception can also be used by interpreters to flag some pointers as special by misaligning the pointer. This eliminates overhead of checking each pointer and only handles the special pointer when used.

VIF Virtual Interrupt (bit 19). Contains a virtual image of the IF flag. This flag is used in conjunction with the VIP flag. The processor only recognizes the VIF flag when either the VME flag or the PVI flag in control register CR4 is set and the IOPL is less than 3. (The VME flag enables the virtual-8086 mode extensions; the PVI flag enables the protected-mode virtual interrupts.)

See Section 16.3.3.5., “Method 6: Software Interrupt Handling”, and Section 16.4., “Protected-Mode Virtual Interrupts”, for detailed information about the use of this flag.

- VIP Virtual interrupt pending (bit 20).** Set by software to indicate that an interrupt is pending; cleared to indicate that no interrupt is pending. This flag is used in conjunction with the VIF flag. The processor reads this flag but never modifies it. The processor only recognizes the VIP flag when either the VME flag or the PVI flag in control register CR4 is set and the IOPL is less than 3. (The VME flag enables the virtual-8086 mode extensions; the PVI flag enables the protected-mode virtual interrupts.) See Section 16.3.3.5., “Method 6: Software Interrupt Handling”, and Section 16.4., “Protected-Mode Virtual Interrupts”, for detailed information about the use of this flag.
- ID Identification (bit 21).** The ability of a program or procedure to set or clear this flag indicates support for the CPUID instruction.

2.4. MEMORY-MANAGEMENT REGISTERS

The processor provides four memory-management registers (GDTR, LDTR, IDTR, and TR) that specify the locations of the data structures which control segmented memory management (see Figure 2-4). Special instructions are provided for loading and storing these registers.

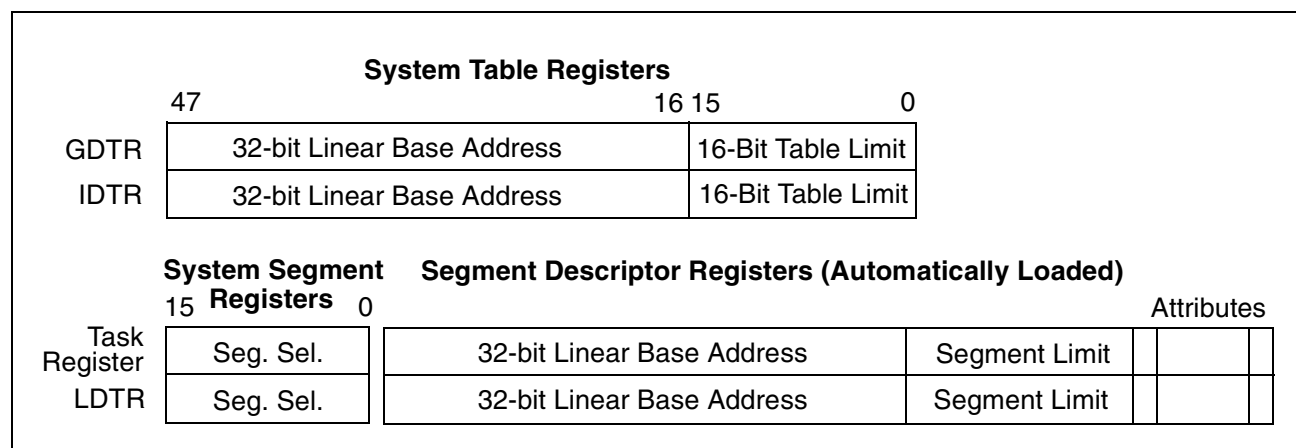


Figure 2-4. Memory Management Registers

2.4.1. Global Descriptor Table Register (GDTR)

The GDTR register holds the 32-bit base address and 16-bit table limit for the GDT. The base address specifies the linear address of byte 0 of the GDT; the table limit specifies the number of bytes in the table. The LGDT and SGDT instructions load and store the GDTR register, respectively. On power up or reset of the processor, the base address is set to the default value of 0 and the limit is set to FFFFH. A new base address must be loaded into the GDTR as part of the processor initialization process for protected-mode operation. See Section 3.5.1., “Segment Descriptor Tables”, for more information on the base address and limit fields.

2.4.2. Local Descriptor Table Register (LDTR)

The LDTR register holds the 16-bit segment selector, 32-bit base address, 16-bit segment limit, and descriptor attributes for the LDT. The base address specifies the linear address of byte 0 of the LDT segment; the segment limit specifies the number of bytes in the segment. See Section 3.5.1., “Segment Descriptor Tables”, for more information on the base address and limit fields.

The LLDT and SLDT instructions load and store the segment selector part of the LDTR register, respectively. The segment that contains the LDT must have a segment descriptor in the GDT. When the LLDT instruction loads a segment selector in the LDTR, the base address, limit, and descriptor attributes from the LDT descriptor are automatically loaded into the LDTR.

When a task switch occurs, the LDTR is automatically loaded with the segment selector and descriptor for the LDT for the new task. The contents of the LDTR are not automatically saved prior to writing the new LDT information into the register.

On power up or reset of the processor, the segment selector and base address are set to the default value of 0 and the limit is set to FFFFH.

2.4.3. IDTR Interrupt Descriptor Table Register

The IDTR register holds the 32-bit base address and 16-bit table limit for the IDT. The base address specifies the linear address of byte 0 of the IDT; the table limit specifies the number of bytes in the table. The LIDT and SIDT instructions load and store the IDTR register, respectively. On power up or reset of the processor, the base address is set to the default value of 0 and the limit is set to FFFFH. The base address and limit in the register can then be changed as part of the processor initialization process. See Section 5.10., “Interrupt Descriptor Table (IDT)”, for more information on the base address and limit fields.

2.4.4. Task Register (TR)

The task register holds the 16-bit segment selector, 32-bit base address, 16-bit segment limit, and descriptor attributes for the TSS of the current task. It references a TSS descriptor in the GDT. The base address specifies the linear address of byte 0 of the TSS; the segment limit specifies the number of bytes in the TSS. (See Section 6.2.3., “Task Register”, for more information about the task register.)

The LTR and STR instructions load and store the segment selector part of the task register, respectively. When the LTR instruction loads a segment selector in the task register, the base address, limit, and descriptor attributes from the TSS descriptor are automatically loaded into the task register. On power up or reset of the processor, the base address is set to the default value of 0 and the limit is set to FFFFH.

When a task switch occurs, the task register is automatically loaded with the segment selector and descriptor for the TSS for the new task. The contents of the task register are not automatically saved prior to writing the new TSS information into the register.

2.5. CONTROL REGISTERS

The control registers (CR0, CR1, CR2, CR3, and CR4, see Figure 2-5) determine operating mode of the processor and the characteristics of the currently executing task, as described below:

- CR0—Contains system control flags that control operating mode and states of the processor.
- CR1—Reserved.
- CR2—Contains the page-fault linear address (the linear address that caused a page fault).

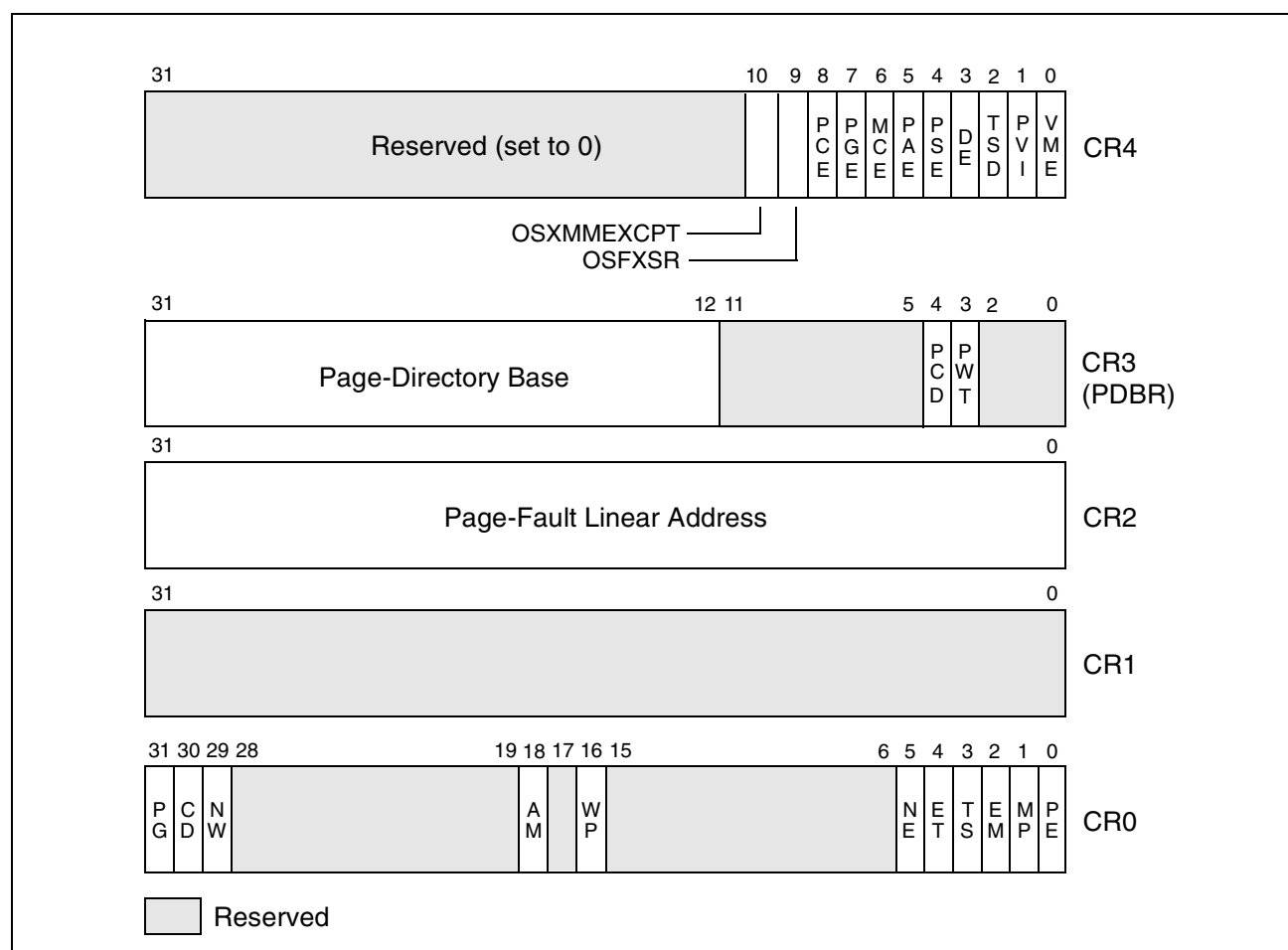


Figure 2-5. Control Registers

- CR3—Contains the physical address of the base of the page directory and two flags (PCD and PWT). This register is also known as the page-directory base register (PDBR). Only the 20 most-significant bits of the page-directory base address are specified; the lower 12 bits of the address are assumed to be 0. The page directory must thus be aligned to a page (4-KByte) boundary. The PCD and PWT flags control caching of the page directory in the processor's internal data caches (they do not control TLB caching of page-directory information).

When using the physical address extension, the CR3 register contains the base address of the page-directory-pointer table (see Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”).

- **CR4**—Contains a group of flags that enable several architectural extensions, and indicate operating system or executive support for specific processor capabilities.

The control registers can be read and loaded (or modified) using the move-to-or-from-control-registers forms of the MOV instruction. In protected mode, the MOV instructions allow the control registers to be read or loaded (at privilege level 0 only). This restriction means that application programs or operating-system procedures (running at privilege levels 1, 2, or 3) are prevented from reading or loading the control registers.

When loading the control register, reserved bits should always be set to the values previously read.

The functions of the flags in the control registers are as follows:

- | | |
|-----------|--|
| PG | Paging (bit 31 of CR0). Enables paging when set; disables paging when clear. When paging is disabled, all linear addresses are treated as physical addresses. The PG flag has no effect if the PE flag (bit 0 of register CR0) is not also set; in fact, setting the PG flag when the PE flag is clear causes a general-protection exception (#GP) to be generated. See Section 3.6., “Paging (Virtual Memory) Overview”, for a detailed description of the processor’s paging mechanism. |
| CD | Cache Disable (bit 30 of CR0). When the CD and NW flags are clear, caching of memory locations for the whole of physical memory in the processor’s internal (and external) caches is enabled. When the CD flag is set, caching is restricted as described in Table 10-5. To prevent the processor from accessing and updating its caches, the CD flag must be set and the caches must be invalidated so that no cache hits can occur (see Section 10.5.3., “Preventing Caching”). See Section 10.5., “Cache Control”, for a detailed description of the additional restrictions that can be placed on the caching of selected pages or regions of memory. |
| NW | Not Write-through (bit 29 of CR0). When the NW and CD flags are clear, write-back (for Pentium 4, Intel Xeon, P6 family, and Pentium processors) or write-through (for Intel486 processors) is enabled for writes that hit the cache and invalidation cycles are enabled. See Table 10-5 for detailed information about the affect of the NW flag on caching for other settings of the CD and NW flags. |
| AM | Alignment Mask (bit 18 of CR0). Enables automatic alignment checking when set; disables alignment checking when clear. Alignment checking is performed only when the AM flag is set, the AC flag in the EFLAGS register is set, the CPL is 3, and the processor is operating in either protected or virtual-8086 mode. |
| WP | Write Protect (bit 16 of CR0). Inhibits supervisor-level procedures from writing into user-level read-only pages when set; allows supervisor-level procedures to write into user-level read-only pages when clear. This flag facilitates implementation of the copy-on-write method of creating a new process (forking) used by operating systems such as UNIX*. |

- NE Numeric Error (bit 5 of CR0).** Enables the native (internal) mechanism for reporting x87 FPU errors when set; enables the PC-style x87 FPU error reporting mechanism when clear. When the NE flag is clear and the IGNNE# input is asserted, x87 FPU errors are ignored. When the NE flag is clear and the IGNNE# input is deasserted, an unmasked x87 FPU error causes the processor to assert the FERR# pin to generate an external interrupt and to stop instruction execution immediately before executing the next waiting floating-point instruction or WAIT/FWAIT instruction. The FERR# pin is intended to drive an input to an external interrupt controller (the FERR# pin emulates the ERROR# pin of the Intel 287 and Intel 387 DX math coprocessors). The NE flag, IGNNE# pin, and FERR# pin are used with external logic to implement PC-style error reporting. (See “Software Exception Handling” in Chapter 8, and Appendix D in the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, for more information about x87 FPU error reporting and for detailed information on when the FERR# pin is asserted, which is implementation dependent.)
- ET Extension Type (bit 4 of CR0).** Reserved in the Pentium 4, Intel Xeon, P6 family, and Pentium processors. (In the Pentium 4, Intel Xeon, and P6 family processors, this flag is hardcoded to 1.) In the Intel386 and Intel486 processors, this flag indicates support of Intel 387 DX math coprocessor instructions when set.
- TS Task Switched (bit 3 of CR0).** Allows the saving of the x87 FPU, MMX, SSE, and SSE2 context on a task switch to be delayed until an x87 FPU, MMX, SSE, or SSE2 instruction is actually executed by the new task. The processor sets this flag on every task switch and tests it when executing x87 FPU, MMX, SSE, and SSE2 instructions.
- If the TS flag is set and the EM flag (bit 2 of CR0) is clear, a device-not-available exception (#NM) is raised prior to the execution of any x87 FPU, MMX, SSE, and SSE2 instruction, with the exception of the PAUSE, PREFETCHh, SFENCE, LFENCE, MFENCE, MOVNTI, and CLFLUSH instructions. (See the paragraph below for the special case of the WAIT/FWAIT instructions.)
 - If the TS flag is set and the MP flag (bit 1 of CR0) and EM flag are clear, an #NM exception is not raised prior to the execution of an x87 FPU WAIT/FWAIT instruction.
 - If the EM flag is set, the setting of the TS flag has no affect on the execution of the x87 FPU, MMX, SSE, and SSE2 instructions.

Table 2-1 shows the actions taken when the processor encounters an x87 FPU instruction based on the settings of the TS, EM, and MP flags. Tables 11-1 and 12-1 show the actions taken when the processor encounters an MMX and or an SSE or SSE2 instruction, respectively.

The processor does not automatically save the context of the x87 FPU, XMM, and MXCSR registers on a task switch. Instead it sets the TS flag, which causes the processor to raise an #NM exception whenever it encounters an x87 FPU, MMX, SSE, or SSE2 instruction in the instruction stream for the new task (with the exception of the instructions listed above).

The fault handler for the #NM exception can then be used to clear the TS flag (with the CLTS instruction) and save the context of the x87 FPU, XMM, and MXCSR registers. If the task never encounters an x87 FPU, MMX, SSE, or SSE2 instruction, the x87 FPU, MMX, SSE, and SSE2 context is never saved.

Table 2-1. Action Taken By x87 FPU Instructions for Different Combinations of EM, MP and TS

CR0 Flags			x87 FPU Instruction Type	
EM	MP	TS	Floating-Point	WAIT/FWAIT
0	0	0	Execute	Execute.
0	0	1	#NM Exception	Execute.
0	1	0	Execute	Execute.
0	1	1	#NM Exception	#NM exception.
1	0	0	#NM Exception	Execute.
1	0	1	#NM Exception	Execute.
1	1	0	#NM Exception	Execute.
1	1	1	#NM Exception	#NM exception.

EM Emulation (bit 2 of CR0). Indicates that the processor does not have an internal or external x87 FPU when set; indicates an x87 FPU is present when clear. This flag also affects the execution of MMX, SSE, and SSE2 instructions.

When the EM flag is set, execution of an x87 FPU instruction generates a device-not-available exception (#NM). This flag must be set when the processor does not have an internal x87 FPU or is not connected to an external math coprocessor. Setting this flag forces all floating-point instructions to be handled by software emulation. Table 9-2 shows the recommended setting of this flag, depending on the IA-32 processor and x87 FPU or math coprocessor present in the system. Table 2-1 shows the interaction of the EM, MP, and TS flags.

Also, when the EM flag is set, execution of an MMX instruction causes an invalid-opcode exception (#UD) to be generated (see Table 11-1). Thus, if an IA-32 processor incorporates MMX technology, the EM flag must be set to 0 to enable execution of MMX instructions.

Similarly for the SSE and SSE2 extensions, when the EM flag is set, execution of most SSE and SSE2 instructions causes an invalid opcode exception (#UD) to be generated (see Table 12-1). Thus, if an IA-32 processor incorporates the SSE and/or SSE2 extensions, the EM flag must be set to 0 to enable execution of these extensions. Those SSE and SSE2 instructions that are not affected by the EM flag are the PAUSE, PREFETCHh, SFENCE, LFENCE, MFENCE, MOVNTI, and CLFLUSH instructions.

- MP Monitor Coprocessor (bit 1 of CR0).** Controls the interaction of the WAIT (or FWAIT) instruction with the TS flag (bit 3 of CR0). If the MP flag is set, a WAIT instruction generates a device-not-available exception (#NM) if the TS flag is set. If the MP flag is clear, the WAIT instruction ignores the setting of the TS flag. Table 9-2 shows the recommended setting of this flag, depending on the IA-32 processor and x87 FPU or math coprocessor present in the system. Table 2-1 shows the interaction of the MP, EM, and TS flags.
- PE Protection Enable (bit 0 of CR0).** Enables protected mode when set; enables real-address mode when clear. This flag does not enable paging directly. It only enables segment-level protection. To enable paging, both the PE and PG flags must be set. See Section 9.9., “Mode Switching”, for information using the PE flag to switch between real and protected mode.
- PCD Page-level Cache Disable (bit 4 of CR3).** Controls caching of the current page directory. When the PCD flag is set, caching of the page-directory is prevented; when the flag is clear, the page-directory can be cached. This flag affects only the processor’s internal caches (both L1 and L2, when present). The processor ignores this flag if paging is not used (the PG flag in register CR0 is clear) or the CD (cache disable) flag in CR0 is set. See Chapter 10, *Memory Cache Control*, for more information about the use of this flag. See Section 3.7.6., “Page-Directory and Page-Table Entries”, for a description of a companion PCD flag in the page-directory and page-table entries.
- PWT Page-level Writes Transparent (bit 3 of CR3).** Controls the write-through or write-back caching policy of the current page directory. When the PWT flag is set, write-through caching is enabled; when the flag is clear, write-back caching is enabled. This flag affects only the internal caches (both L1 and L2, when present). The processor ignores this flag if paging is not used (the PG flag in register CR0 is clear) or the CD (cache disable) flag in CR0 is set. See Section 10.5., “Cache Control”, for more information about the use of this flag. See Section 3.7.6., “Page-Directory and Page-Table Entries”, for a description of a companion PCD flag in the page-directory and page-table entries.
- VME Virtual-8086 Mode Extensions (bit 0 of CR4).** Enables interrupt- and exception-handling extensions in virtual-8086 mode when set; disables the extensions when clear. Use of the virtual mode extensions can improve the performance of virtual-8086 applications by eliminating the overhead of calling the virtual-8086 monitor to handle interrupts and exceptions that occur while executing an 8086 program and, instead, redirecting the interrupts and exceptions back to the 8086 program’s handlers. It also provides hardware support for a virtual interrupt flag (VIF) to improve reliability of running 8086 programs in multitasking and multiple-processor environments. See Section 16.3., “Interrupt and Exception Handling in Virtual-8086 Mode”, for detailed information about the use of this feature.
- PVI Protected-Mode Virtual Interrupts (bit 1 of CR4).** Enables hardware support for a virtual interrupt flag (VIF) in protected mode when set; disables the VIF flag in protected mode when clear. See Section 16.4., “Protected-Mode Virtual Interrupts”, for detailed information about the use of this feature.
- TSD Time Stamp Disable (bit 2 of CR4).** Restricts the execution of the RDTSC instruction to procedures running at privilege level 0 when set; allows RDTSC instruction to be executed at any privilege level when clear.

- DE Debugging Extensions (bit 3 of CR4).** References to debug registers DR4 and DR5 cause an undefined opcode (#UD) exception to be generated when set; when clear, processor aliases references to registers DR4 and DR5 for compatibility with software written to run on earlier IA-32 processors. See Section 15.2.2., “Debug Registers DR4 and DR5”, for more information on the function of this flag.
- PSE Page Size Extensions (bit 4 of CR4).** Enables 4-MByte pages when set; restricts pages to 4 KBytes when clear. See Section 3.6.1., “Paging Options”, for more information about the use of this flag.
- PAE Physical Address Extension (bit 5 of CR4).** Enables paging mechanism to reference 36-bit physical addresses when set; restricts physical addresses to 32 bits when clear. See Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”, for more information about the physical address extension.
- MCE Machine-Check Enable (bit 6 of CR4).** Enables the machine-check exception when set; disables the machine-check exception when clear. See Chapter 14, *Machine-Check Architecture*, for more information about the machine-check exception and machine-check architecture.
- PGE Page Global Enable (bit 7 of CR4).** (Introduced in the P6 family processors.) Enables the global page feature when set; disables the global page feature when clear. The global page feature allows frequently used or shared pages to be marked as global to all users (done with the global flag, bit 8, in a page-directory or page-table entry). Global pages are not flushed from the translation-lookaside buffer (TLB) on a task switch or a write to register CR3.
- When enabling the global page feature, paging must be enabled (by setting the PG flag in control register CR0) before the PGE flag is set. Reversing this sequence may affect program correctness, and processor performance will be impacted. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for more information on the use of this bit.
- PCE Performance-Monitoring Counter Enable (bit 8 of CR4).** Enables execution of the RDPMC instruction for programs or procedures running at any protection level when set; RDPMC instruction can be executed only at protection level 0 when clear.

OSFXSR

Operating System Support for FXSAVE and FXRSTOR instructions (bit 9 of CR4). When set, this flag preforms the following functions: (1) indicates to software that the operating system supports the use of the FXSAVE and FXRSTOR instructions, (2) enables the FXSAVE and FXRSTOR instructions to save and restore the contents of the XMM and MXCSR registers along with the contents of the x87 FPU and MMX registers, and (3) enables the processor to execute any of the SSE and SSE2 instructions, with the exception of the PAUSE, PREFETCHh, SFENCE, LFENCE, MFENCE, MOVNTI, and CLFLUSH instructions.

If this flag is clear, the FXSAVE and FXRSTOR instructions will save and restore the contents of the x87 FPU and MMX instructions, but they may not save and restore the contents of the XMM and MXCSR registers.

Also, if this flag is clear, the processor will generate an invalid opcode exception (#UD) whenever it attempts to execute any of the SSE and SSE2 instructions, with the exception of the PAUSE, PREFETCHH, SFENCE, LFENCE, MFENCE, MOVNTI, and CLFLUSH instructions. The operating system or executive must explicitly set this flag.

NOTE

The CPUID feature flags FXSR, SSE, and SSE2 (bits 24, 25, and 26) indicate availability of the FXSAVE/FXRESTOR instructions, the SSE extensions, and the SSE2 extensions, respectively, on a particular IA-32 processor. The OSFXSR bit provides operating system software with a means of enabling these features and indicating that the operating system supports the features.

OSXMMEXCPT

Operating System Support for Unmasked SIMD Floating-Point Exceptions (bit 10 of CR4). Indicates that the operating system supports the handling of unmasked SIMD floating-point exceptions through an exception handler that is invoked when a SIMD floating-point exception (#XF) is generated. SIMD floating-point exceptions are only generated by SSE and SSE2 SIMD floating-point instructions. The operating system or executive must explicitly set this flag. If this flag is not set, the processor will generate an invalid opcode exception (#UD) whenever it detects an unmasked SIMD floating-point exception.

2.5.1. CPUID Qualification of Control Register Flags

The VME, PVI, TSD, DE, PSE, PAE, MCE, PGE, PCE, OSFXSR, and OSXMMEXCPT flags in control register CR4 are model specific. All of these flags (except the PCE flag) can be qualified with the CPUID instruction to determine if they are implemented on the processor before they are used.

2.6. SYSTEM INSTRUCTION SUMMARY

The system instructions handle system-level functions such as loading system registers, managing the cache, managing interrupts, or setting up the debug registers. Many of these instructions can be executed only by operating-system or executive procedures (that is, procedures running at privilege level 0). Others can be executed at any privilege level and are thus available to application programs. Table 2-2 lists the system instructions and indicates whether they are available and useful for application programs. These instructions are described in detail in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*.

Table 2-2. Summary of System Instructions

Instruction	Description	Useful to Application?	Protected from Application?
LLDT	Load LDT Register	No	Yes
SLDT	Store LDT Register	No	No
LGDT	Load GDT Register	No	Yes
SGDT	Store GDT Register	No	No
LTR	Load Task Register	No	Yes
STR	Store Task Register	No	No
LIDT	Load IDT Register	No	Yes
SIDT	Store IDT Register	No	No
MOV CR _n	Load and store control registers	No	Yes
SMSW	Store MSW	Yes	No
LMSW	Load MSW	No	Yes
CLTS	Clear TS flag in CR0	No	Yes
ARPL	Adjust RPL	Yes ¹	No
LAR	Load Access Rights	Yes	No
LSL	Load Segment Limit	Yes	No
VERR	Verify for Reading	Yes	No
VERW	Verify for Writing	Yes	No
MOV DB _n	Load and store debug registers	No	Yes
INVD	Invalidate cache, no writeback	No	Yes
WBINVD	Invalidate cache, with writeback	No	Yes
INVLPG	Invalidate TLB entry	No	Yes
HLT	Halt Processor	No	Yes
LOCK (Prefix)	Bus Lock	Yes	No
RSM	Return from system management mode	No	Yes
RDMSR ³	Read Model-Specific Registers	No	Yes
WRMSR ³	Write Model-Specific Registers	No	Yes
RDPMC ⁴	Read Performance-Monitoring Counter	Yes	Yes ²
RDTSC ³	Read Time-Stamp Counter	Yes	Yes ²

NOTES:

1. Useful to application programs running at a CPL of 1 or 2.
2. The TSD and PCE flags in control register CR4 control access to these instructions by application programs running at a CPL of 3.
3. These instructions were introduced into the IA-32 Architecture with the Pentium processor.
4. This instruction was introduced into the IA-32 Architecture with the Pentium Pro processor and the Pentium® processor with MMX™ technology.

2.6.1. Loading and Storing System Registers

The GDTR, LDTR, IDTR, and TR registers each have a load and store instruction for loading data into and storing data from the register:

LGDT (Load GDTR Register)	Loads the GDT base address and limit from memory into the GDTR register.
SGDT (Store GDTR Register)	Stores the GDT base address and limit from the GDTR register into memory.
LIDT (Load IDTR Register)	Loads the IDT base address and limit from memory into the IDTR register.
SIDT (Store IDTR Register)	Stores the IDT base address and limit from the IDTR register into memory.
LLDT (Load LDT Register)	Loads the LDT segment selector and segment descriptor from memory into the LDTR. (The segment selector operand can also be located in a general-purpose register.)
SLDT (Store LDT Register)	Stores the LDT segment selector from the LDTR register into memory or a general-purpose register.
LTR (Load Task Register)	Loads segment selector and segment descriptor for a TSS from memory into the task register. (The segment selector operand can also be located in a general-purpose register.)
STR (Store Task Register)	Stores the segment selector for the current task TSS from the task register into memory or a general-purpose register.

The LMSW (load machine status word) and SMSW (store machine status word) instructions operate on bits 0 through 15 of control register CR0. These instructions are provided for compatibility with the 16-bit Intel 286 processor. Programs written to run on 32-bit IA-32 processors should not use these instructions. Instead, they should access the control register CR0 using the MOV instruction.

The CLTS (clear TS flag in CR0) instruction is provided for use in handling a device-not-available exception (#NM) that occurs when the processor attempts to execute a floating-point instruction when the TS flag is set. This instruction allows the TS flag to be cleared after the x87 FPU context has been saved, preventing further #NM exceptions. See Section 2.5., “Control Registers”, for more information about the TS flag.

The control registers (CR0, CR1, CR2, CR3, and CR4) are loaded with the MOV instruction. This instruction can load a control register from a general-purpose register or store the contents of the control register in a general-purpose register.

2.6.2. Verifying of Access Privileges

The processor provides several instructions for examining segment selectors and segment descriptors to determine if access to their associated segments is allowed. These instructions duplicate some of the automatic access rights and type checking done by the processor, thus allowing operating-system or executive software to prevent exceptions from being generated.

The ARPL (adjust RPL) instruction adjusts the RPL (requestor privilege level) of a segment selector to match that of the program or procedure that supplied the segment selector. See Section 4.10.4., “Checking Caller Access Privileges (ARPL Instruction)”, for a detailed explanation of the function and use of this instruction.

The LAR (load access rights) instruction verifies the accessibility of a specified segment and loads the access rights information from the segment’s segment descriptor into a general-purpose register. Software can then examine the access rights to determine if the segment type is compatible with its intended use. See Section 4.10.1., “Checking Access Rights (LAR Instruction)”, for a detailed explanation of the function and use of this instruction.

The LSL (load segment limit) instruction verifies the accessibility of a specified segment and loads the segment limit from the segment’s segment descriptor into a general-purpose register. Software can then compare the segment limit with an offset into the segment to determine whether the offset lies within the segment. See Section 4.10.3., “Checking That the Pointer Offset Is Within Limits (LSL Instruction)”, for a detailed explanation of the function and use of this instruction.

The VERR (verify for reading) and VERW (verify for writing) instructions verify if a selected segment is readable or writable, respectively, at the CPL. See Section 4.10.2., “Checking Read/Write Rights (VERR and VERW Instructions)”, for a detailed explanation of the function and use of this instruction.

2.6.3. Loading and Storing Debug Registers

The internal debugging facilities in the processor are controlled by a set of 8 debug registers (DR0 through DR7). The MOV instruction allows setup data to be loaded into and stored from these registers.

2.6.4. Invalidating Caches and TLBs

The processor provides several instructions for use in explicitly invalidating its caches and TLB entries. The INVD (invalidate cache with no writeback) instruction invalidates all data and instruction entries in the internal caches and sends a signal to the external caches indicating that they should be invalidated also.

The WBINVD (invalidate cache with writeback) instruction performs the same function as the INVD instruction, except that it writes back any modified lines in its internal caches to memory before it invalidates the caches. After invalidating the internal caches, it signals the external caches to write back modified data and invalidate their contents.

The INVLPG (invalidate TLB entry) instruction invalidates (flushes) the TLB entry for a specified page.

2.6.5. Controlling the Processor

The HLT (halt processor) instruction stops the processor until an enabled interrupt (such as NMI or SMI, which are normally enabled), a debug exception, the BINIT# signal, the INIT# signal, or the RESET# signal is received. The processor generates a special bus cycle to indicate that the halt mode has been entered. Hardware may respond to this signal in a number of ways. An indicator light on the front panel may be turned on. An NMI interrupt for recording diagnostic information may be generated. Reset initialization may be invoked (note that the BINIT# pin was introduced with the Pentium Pro processor). If any non-wake events are pending during shutdown, they will be handled after the wake event from shutdown is processed (for example, A20M# interrupts).

The LOCK prefix invokes a locked (atomic) read-modify-write operation when modifying a memory operand. This mechanism is used to allow reliable communications between processors in multiprocessor systems. In the Pentium processor and earlier IA-32 processors, the LOCK prefix causes the processor to assert the LOCK# signal during the instruction, which always causes an explicit bus lock to occur. In the Pentium 4, Intel Xeon, and P6 family processors, the locking operation is handled with either a cache lock or bus lock. If a memory access is cacheable and affects only a single cache line, a cache lock is invoked and the system bus and the actual memory location in system memory are not locked during the operation. Here, other Pentium 4, Intel Xeon, or P6 family processors on the bus write-back any modified data and invalidate their caches as necessary to maintain system memory coherency. If the memory access is not cacheable and/or it crosses a cache line boundary, the processor's LOCK# signal is asserted and the processor does not respond to requests for bus control during the locked operation.

The RSM (return from SMM) instruction restores the processor (from a context dump) to the state it was in prior to an system management mode (SMM) interrupt.

2.6.6. Reading Performance-Monitoring and Time-Stamp Counters

The RDPMC (read performance-monitoring counter) and RDTSC (read time-stamp counter) instructions allow an application program to read the processor's performance-monitoring and time-stamp counters, respectively.

The Pentium 4 and Intel Xeon processors have 18 40-bit performance-monitoring counters and the P6 family processors have 2 40-bit counters. These counters can be used to record either the occurrence of events or the duration of events. The events that can be monitored are model specific and include the number of instructions decoded, number of interrupts received, of number of cache loads. Each counter can be set up to monitor a different event, using the system instruction WRMSR to set up values in the one of the 45 ESCR and one of the 18 CCCR MSRs (for Pentium 4 and Intel Xeon processors) or in either the PerfEvtSel0 or the PerfEvtSel1 MSR

(for the P6 family processors). The RDPMC instruction loads the current count from a counter into the EDX:EAX registers.

The time-stamp counter is a model-specific 64-bit counter that is reset to zero each time the processor is reset. If not reset, the counter will increment $\sim 6.3 \times 10^{15}$ times per year when the processor is operating at a clock rate of 200 MHz. At this clock frequency, it would take over 2000 years for the counter to wrap around. The RDTSC instruction loads the current count of the time-stamp counter into the EDX:EAX registers.

See Section 15.8., “Performance Monitoring Overview”, and Section 15.7., “Time-Stamp Counter”, for more information about the performance monitoring and time-stamp counters.

The RDTSC instruction was introduced into the IA-32 architecture with the Pentium processor. The RDPMC instruction was introduced into the IA-32 architecture with the Pentium Pro processor and the Pentium processor with MMX technology. Earlier Pentium processors have two performance-monitoring counters, but they can be read only with the RDMSR instruction, and only at privilege level 0.

2.6.7. Reading and Writing Model-Specific Registers

The RDMSR (read model-specific register) and WRMSR (write model-specific register) allow the processor’s 64-bit model-specific registers (MSRs) to be read and written to, respectively. The MSR to be read or written to is specified by the value in the ECX register. The RDMSR instruction reads the value from the specified MSR into the EDX:EAX registers; the WRMSR writes the value in the EDX:EAX registers into the specified MSR. See Section 9.4., “Model-Specific Registers (MSRs)”, for more information about the MSRs.

The RDMSR and WRMSR instructions were introduced into the IA-32 architecture with the Pentium processor.

3

Protected-Mode Memory Management

CHAPTER 3

PROTECTED-MODE MEMORY MANAGEMENT

This chapter describes the IA-32 architecture's protected-mode memory management facilities, including the physical memory requirements, the segmentation mechanism, and the paging mechanism. See Chapter 4, *Protection*, for a description of the processor's protection mechanism. See Chapter 16, *8086 Emulation*, for a description of memory addressing protection in real-address and virtual-8086 modes.

3.1. MEMORY MANAGEMENT OVERVIEW

The memory management facilities of the IA-32 architecture are divided into two parts: segmentation and paging. Segmentation provides a mechanism of isolating individual code, data, and stack modules so that multiple programs (or tasks) can run on the same processor without interfering with one another. Paging provides a mechanism for implementing a conventional demand-paged, virtual-memory system where sections of a program's execution environment are mapped into physical memory as needed. Paging can also be used to provide isolation between multiple tasks. When operating in protected mode, some form of segmentation must be used. **There is no mode bit to disable segmentation.** The use of paging, however, is optional.

These two mechanisms (segmentation and paging) can be configured to support simple single-program (or single-task) systems, multitasking systems, or multiple-processor systems that used shared memory.

As shown in Figure 3-1, segmentation provides a mechanism for dividing the processor's addressable memory space (called the **linear address space**) into smaller protected address spaces called **segments**. Segments can be used to hold the code, data, and stack for a program or to hold system data structures (such as a TSS or LDT). If more than one program (or task) is running on a processor, each program can be assigned its own set of segments. The processor then enforces the boundaries between these segments and insures that one program does not interfere with the execution of another program by writing into the other program's segments. The segmentation mechanism also allows typing of segments so that the operations that may be performed on a particular type of segment can be restricted.

All the segments in a system are contained in the processor's linear address space. To locate a byte in a particular segment, a **logical address** (also called a **far pointer**) must be provided. A logical address consists of a **segment selector and an offset**. The segment selector is a unique identifier for a segment. Among other things it provides an offset into a descriptor table (such as the global descriptor table, **GDT**) to a data structure called a **segment descriptor**. Each segment has a segment descriptor, which specifies the size of the segment, the access rights and privilege level for the segment, the segment type, and the location of the first byte of the segment in the linear address space (called the **base address** of the segment). The offset part of the logical address is added to the base address for the segment to locate a byte within the segment. The base address plus the offset thus forms a **linear address** in the processor's linear address space.

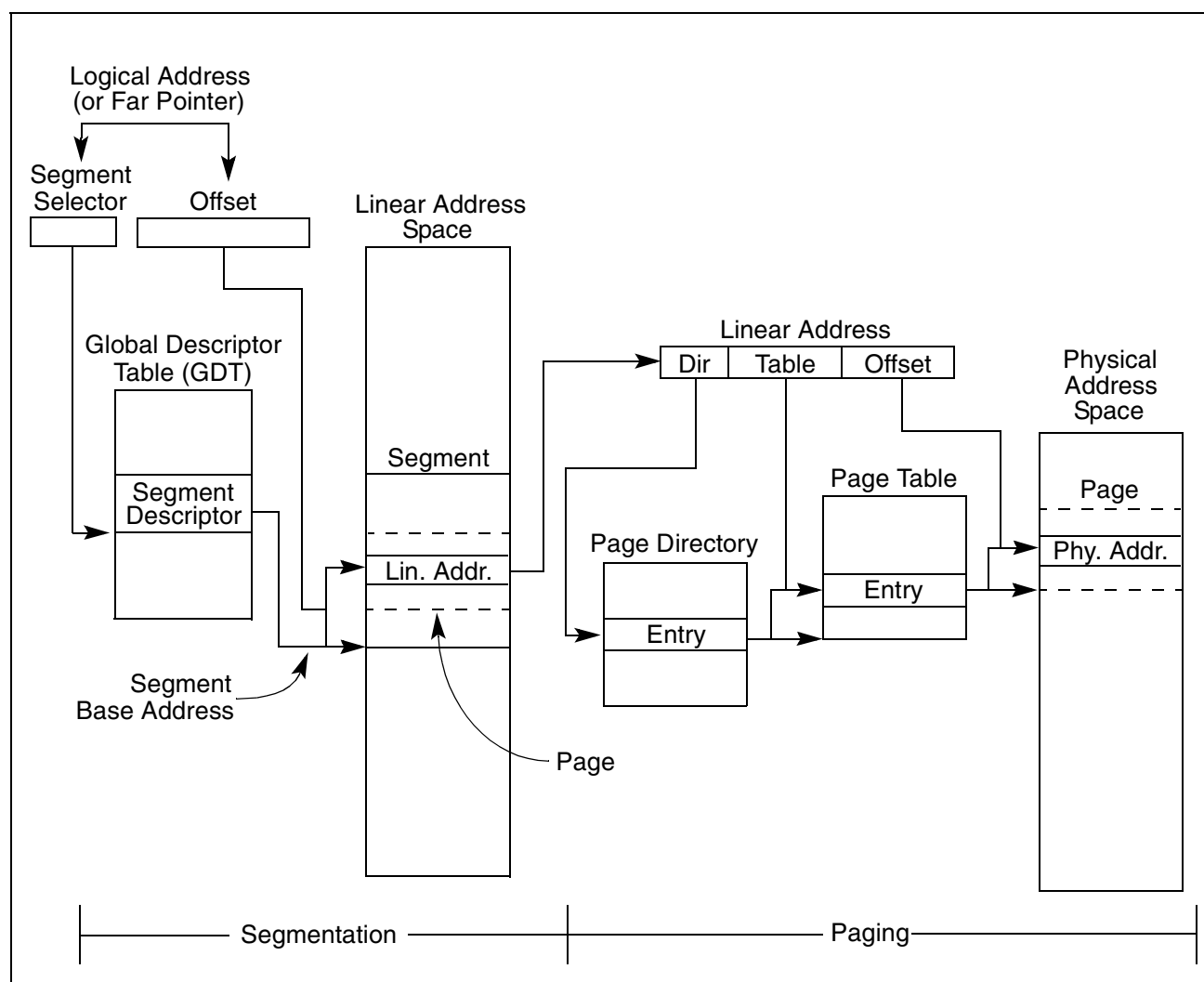


Figure 3-1. Segmentation and Paging

If paging is not used, the linear address space of the processor is mapped directly into the physical address space of processor. The physical address space is defined as the range of addresses that the processor can generate on its address bus.

Because multitasking computing systems commonly define a linear address space much larger than it is economically feasible to contain all at once in physical memory, some method of “virtualizing” the linear address space is needed. This virtualization of the linear address space is handled through the processor’s paging mechanism.

Paging supports a “virtual memory” environment where a large linear address space is simulated with a small amount of physical memory (RAM and ROM) and some disk storage. When using paging, each segment is divided into pages (typically 4 KBytes each in size), which are stored either in physical memory or on the disk. The operating system or executive maintains a page directory and a set of page tables to keep track of the pages. When a program (or task) attempts to access an address location in the linear address space, the processor uses the page directory and page tables to translate the linear address into a physical address and then performs the requested operation (read or write) on the memory location.

If the page being accessed is not currently in physical memory, the processor interrupts execution of the program (by generating a page-fault exception). The operating system or executive then reads the page into physical memory from the disk and continues executing the program.

When paging is implemented properly in the operating-system or executive, the swapping of pages between physical memory and the disk is transparent to the correct execution of a program. Even programs written for 16-bit IA-32 processors can be paged (transparently) when they are run in virtual-8086 mode.

3.2. USING SEGMENTS

The segmentation mechanism supported by the IA-32 architecture can be used to implement a wide variety of system designs. These designs range from flat models that make only minimal use of segmentation to protect programs to multi-segmented models that employ segmentation to create a robust operating environment in which multiple programs and tasks can be executed reliably.

The following sections give several examples of how segmentation can be employed in a system to improve memory management performance and reliability.

3.2.1. Basic Flat Model

The simplest memory model for a system is the basic “flat model,” in which the operating system and application programs have access to a continuous, unsegmented address space. To the greatest extent possible, this basic flat model hides the segmentation mechanism of the architecture from both the system designer and the application programmer.

To implement a basic flat memory model with the IA-32 architecture, at least two segment descriptors must be created, one for referencing a code segment and one for referencing a data segment (see Figure 3-2). Both of these segments, however, are mapped to the entire linear address space: that is, both segment descriptors have the same base address value of 0 and the same segment limit of 4 GBytes. By setting the segment limit to 4 GBytes, the segmentation mechanism is kept from generating exceptions for out of limit memory references, even if no physical memory resides at a particular address. ROM (EPROM) is generally located at the top of the physical address space, because the processor begins execution at FFFF_FFF0H. RAM (DRAM) is placed at the bottom of the address space because the initial base address for the DS data segment after reset initialization is 0.

3.2.2. Protected Flat Model

The protected flat model is similar to the basic flat model, except the segment limits are set to include only the range of addresses for which physical memory actually exists (see Figure 3-3). A general-protection exception (#GP) is then generated on any attempt to access nonexistent memory. This model provides a minimum level of hardware protection against some kinds of program bugs.

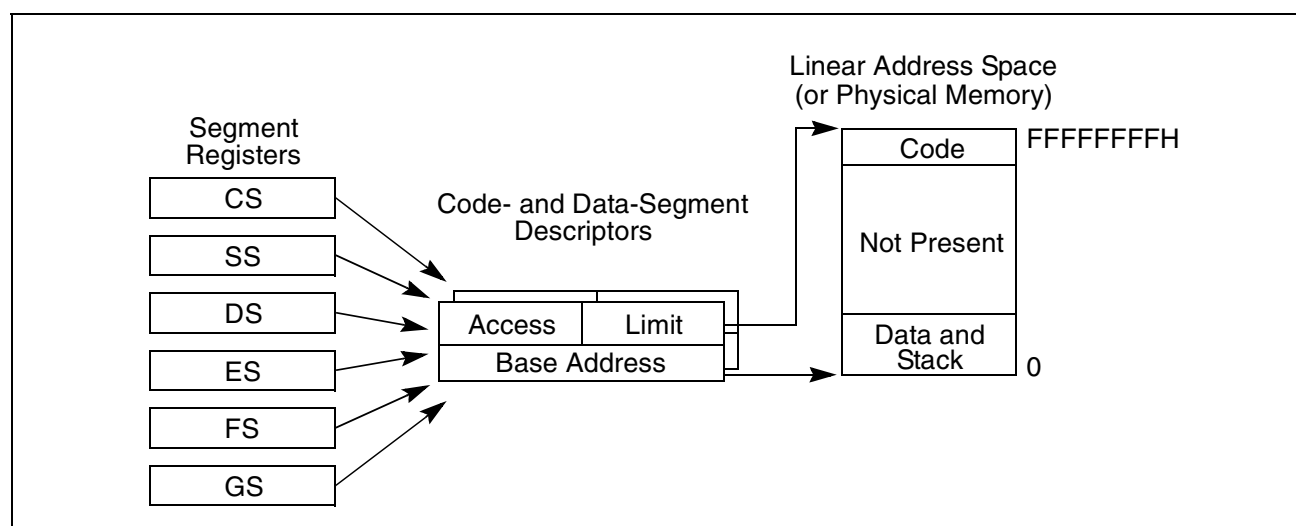


Figure 3-2. Flat Model

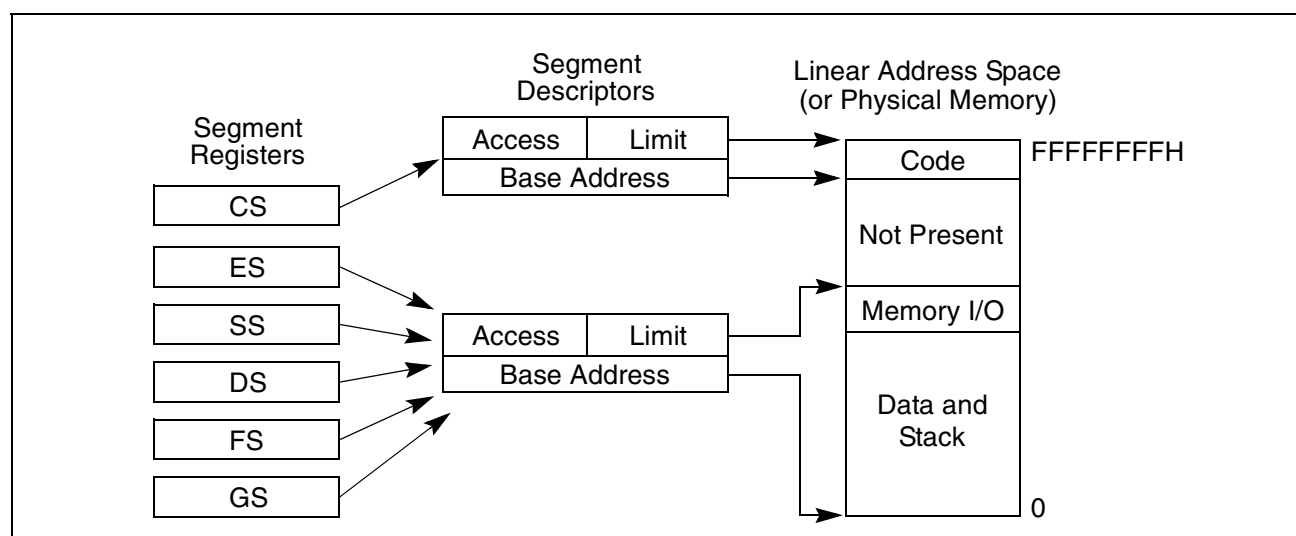


Figure 3-3. Protected Flat Model

More complexity can be added to this protected flat model to provide more protection. For example, for the paging mechanism to provide isolation between user and supervisor code and data, four segments need to be defined: code and data segments at privilege level 3 for the user, and code and data segments at privilege level 0 for the supervisor. Usually these segments all overlay each other and start at address 0 in the linear address space. This flat segmentation model along with a simple paging structure can protect the operating system from applications, and by adding a separate paging structure for each task or process, it can also protect applications from each other. Similar designs are used by several popular multitasking operating systems.

3.2.3. Multi-Segment Model

A multi-segment model (such as the one shown in Figure 3-4) uses the full capabilities of the segmentation mechanism to provide hardware enforced protection of code, data structures, and programs and tasks. Here, each program (or task) is given its own table of segment descriptors and its own segments. The segments can be completely private to their assigned programs or shared among programs. Access to all segments and to the execution environments of individual programs running on the system is controlled by hardware.

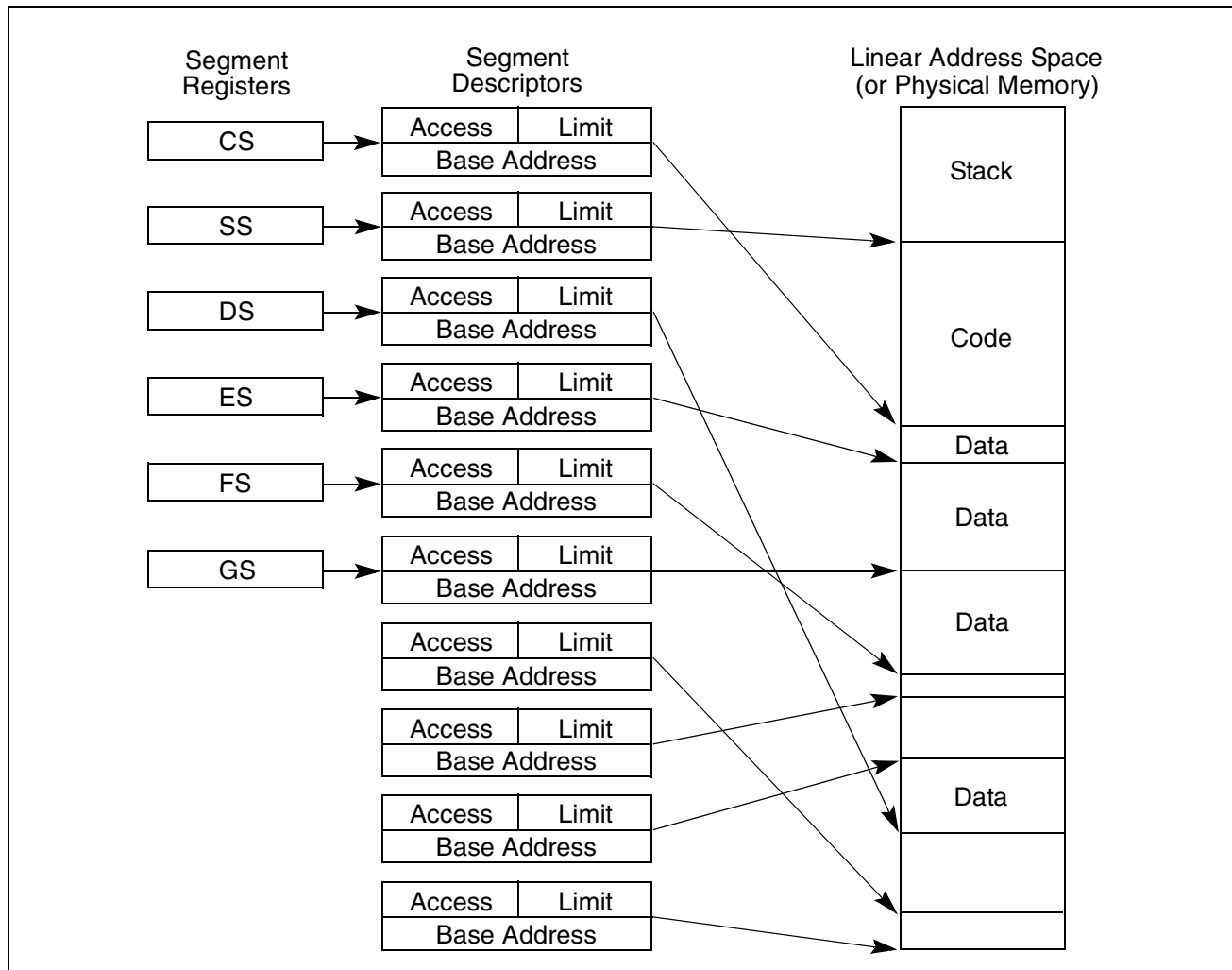


Figure 3-4. Multi-Segment Model

Access checks can be used to protect not only against referencing an address outside the limit of a segment, but also against performing disallowed operations in certain segments. For example, since code segments are designated as read-only segments, hardware can be used to prevent writes into code segments. The access rights information created for segments can also be used to set up protection rings or levels. Protection levels can be used to protect operating-system procedures from unauthorized access by application programs.

3.2.4. Paging and Segmentation

Paging can be used with any of the segmentation models described in Figures 3-2, 3-3, and 3-4. The processor's paging mechanism divides the linear address space (into which segments are mapped) into pages (as shown in Figure 3-1). These linear-address-space pages are then mapped to pages in the physical address space. The paging mechanism offers several page-level protection facilities that can be used with or instead of the segment-protection facilities. For example, it lets read-write protection be enforced on a page-by-page basis. The paging mechanism also provides two-level user-supervisor protection that can also be specified on a page-by-page basis.

3.3. PHYSICAL ADDRESS SPACE

In protected mode, the IA-32 architecture provides a normal physical address space of 4 GBytes (2^{32} bytes). This is the address space that the processor can address on its address bus. This address space is flat (unsegmented), with addresses ranging continuously from 0 to FFFFFFFFH. This physical address space can be mapped to read-write memory, read-only memory, and memory mapped I/O. The memory mapping facilities described in this chapter can be used to divide this physical memory up into segments and/or pages.

(Introduced in the Pentium Pro processor.) The IA-32 architecture also supports an extension of the physical address space to 2^{36} bytes (64 GBytes), with a maximum physical address of FFFFFFFFHH. This extension is invoked in either of two ways:

- Using the physical address extension (PAE) flag, located in bit 5 of control register CR4.
- Using the 36-bit page size extension (PSE-36) feature (introduced in the Pentium III processors).

(See Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism” and Section 3.9., “36-Bit Physical Addressing Using the PSE-36 Paging Mechanism” for more information about 36-bit physical addressing.)

3.4. LOGICAL AND LINEAR ADDRESSES

At the system-architecture level in protected mode, the processor uses two stages of address translation to arrive at a physical address: logical-address translation and linear address space paging.

Even with the minimum use of segments, every byte in the processor's address space is accessed with a logical address. A logical address consists of a 16-bit segment selector and a 32-bit offset (see Figure 3-5). The segment selector identifies the segment the byte is located in and the offset specifies the location of the byte in the segment relative to the base address of the segment.

The processor translates every logical address into a linear address. A linear address is a 32-bit address in the processor's linear address space. Like the physical address space, the linear address space is a flat (unsegmented), 2^{32} -byte address space, with addresses ranging from 0 to FFFFFFFFH. The linear address space contains all the segments and system tables defined for a system.

To translate a logical address into a linear address, the processor does the following:

1. Uses the offset in the segment selector to locate the segment descriptor for the segment in the GDT or LDT and reads it into the processor. (This step is needed only when a new segment selector is loaded into a segment register.)
2. Examines the segment descriptor to check the access rights and range of the segment to insure that the segment is accessible and that the offset is within the limits of the segment.
3. Adds the base address of the segment from the segment descriptor to the offset to form a linear address.

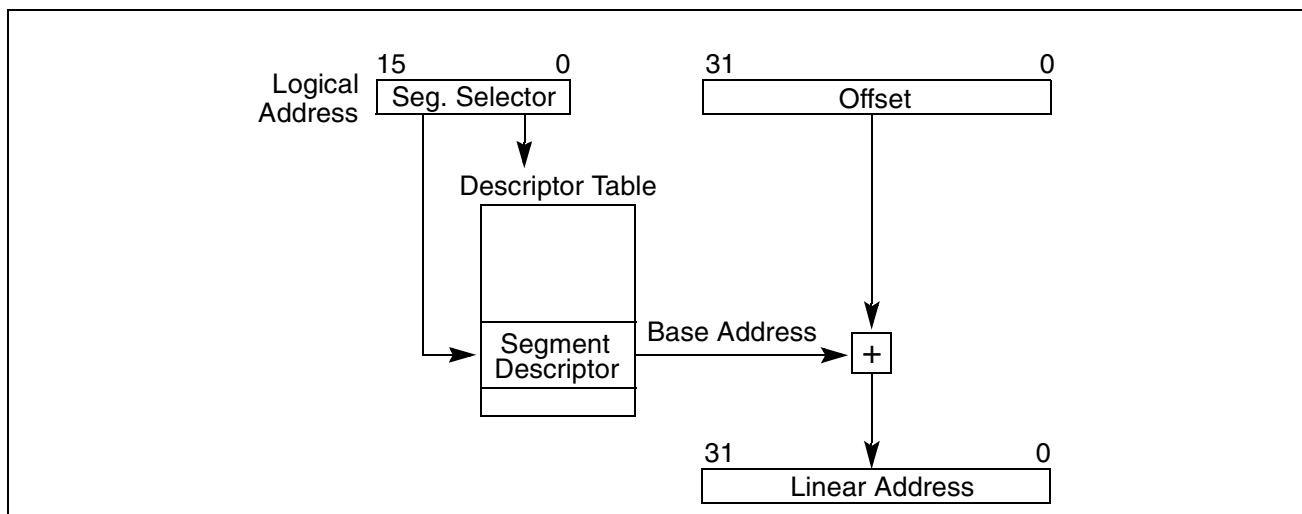


Figure 3-5. Logical Address to Linear Address Translation

If paging is not used, the processor maps the linear address directly to a physical address (that is, the linear address goes out on the processor's address bus). If the linear address space is paged, a second level of address translation is used to translate the linear address into a physical address. Page translation is described in Section 3.6., "Paging (Virtual Memory) Overview".

3.4.1. Segment Selectors

A segment selector is a 16-bit identifier for a segment (see Figure 3-6). It does not point directly to the segment, but instead points to the segment descriptor that defines the segment. A segment selector contains the following items:

Index (Bits 3 through 15). Selects one of 8192 descriptors in the GDT or LDT. The processor multiplies the index value by 8 (the number of bytes in a segment descriptor) and adds the result to the base address of the GDT or LDT (from the GDTR or LDTR register, respectively).

TI (table indicator) flag

(Bit 2). Specifies the descriptor table to use: clearing this flag selects the GDT; setting this flag selects the current LDT.

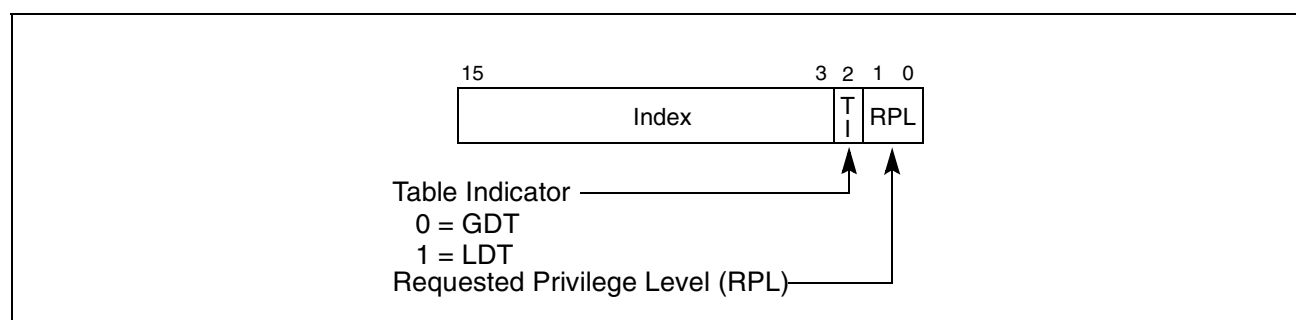


Figure 3-6. Segment Selector

Requested Privilege Level (RPL)

(Bits 0 and 1). Specifies the privilege level of the selector. The privilege level can range from 0 to 3, with 0 being the most privileged level. See Section 4.5., “Privilege Levels”, for a description of the relationship of the RPL to the CPL of the executing program (or task) and the descriptor privilege level (DPL) of the descriptor the segment selector points to.

The first entry of the GDT is not used by the processor. A segment selector that points to this entry of the GDT (that is, a segment selector with an index of 0 and the TI flag set to 0) is used as a “null segment selector.” The processor does not generate an exception when a segment register (other than the CS or SS registers) is loaded with a null selector. It does, however, generate an exception when a segment register holding a null selector is used to access memory. A null selector can be used to initialize unused segment registers. Loading the CS or SS register with a null segment selector causes a general-protection exception (#GP) to be generated.

Segment selectors are visible to application programs as part of a pointer variable, but the values of selectors are usually assigned or modified by link editors or linking loaders, not application programs.

3.4.2. Segment Registers

To reduce address translation time and coding complexity, the processor provides registers for holding up to 6 segment selectors (see Figure 3-7). Each of these segment registers support a specific kind of memory reference (code, stack, or data). For virtually any kind of program execution to take place, at least the code-segment (CS), data-segment (DS), and stack-segment (SS) registers must be loaded with valid segment selectors. The processor also provides three additional data-segment registers (ES, FS, and GS), which can be used to make additional data segments available to the currently executing program (or task).

For a program to access a segment, the segment selector for the segment must have been loaded in one of the segment registers. So, although a system can define thousands of segments, only 6 can be available for immediate use. Other segments can be made available by loading their segment selectors into these registers during program execution.

Visible Part		Hidden Part	
Segment Selector		Base Address, Limit, Access Information	
			CS
			SS
			DS
			ES
			FS
			GS

Figure 3-7. Segment Registers

Every segment register has a “visible” part and a “hidden” part. (The hidden part is sometimes referred to as a “descriptor cache” or a “shadow register.”) When a segment selector is loaded into the visible part of a segment register, the processor also loads the hidden part of the segment register with the base address, segment limit, and access control information from the segment descriptor pointed to by the segment selector. The information cached in the segment register (visible and hidden) allows the processor to translate addresses without taking extra bus cycles to read the base address and limit from the segment descriptor. In systems in which multiple processors have access to the same descriptor tables, it is the responsibility of software to reload the segment registers when the descriptor tables are modified. If this is not done, an old segment descriptor cached in a segment register might be used after its memory-resident version has been modified.

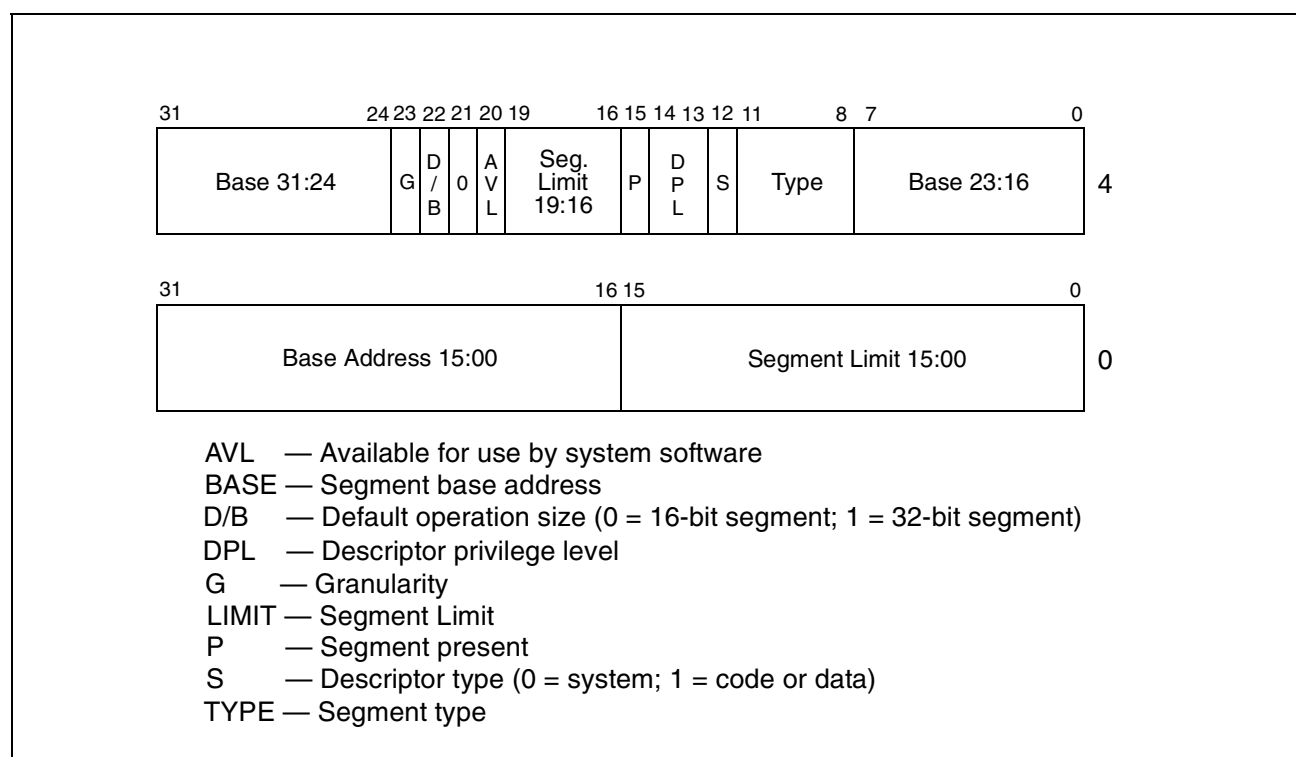
Two kinds of load instructions are provided for loading the segment registers:

1. Direct load instructions such as the MOV, POP, LDS, LES, LSS, LGS, and LFS instructions. These instructions explicitly reference the segment registers.
2. Implied load instructions such as the far pointer versions of the CALL, JMP, and RET instructions, the SYSENTER and SYSEXIT instructions, and the IRET, INT n , INTO and INT3 instructions. These instructions change the contents of the CS register (and sometimes other segment registers) as an incidental part of their operation.

The MOV instruction can also be used to store visible part of a segment register in a general-purpose register.

3.4.3. Segment Descriptors

A segment descriptor is a data structure in a GDT or LDT that provides the processor with the size and location of a segment, as well as access control and status information. Segment descriptors are typically created by compilers, linkers, loaders, or the operating system or executive, but not application programs. Figure 3-8 illustrates the general descriptor format for all types of segment descriptors.

**Figure 3-8. Segment Descriptor**

The flags and fields in a segment descriptor are as follows:

Segment limit field

Specifies the size of the segment. The processor puts together the two segment limit fields to form a 20-bit value. The processor interprets the segment limit in one of two ways, depending on the setting of the G (granularity) flag:

- If the granularity flag is clear, the segment size can range from 1 byte to 1 MByte, in byte increments.
- If the granularity flag is set, the segment size can range from 4 KBytes to 4 GBytes, in 4-KByte increments.

The processor uses the segment limit in two different ways, depending on whether the segment is an expand-up or an expand-down segment. See Section 3.4.3.1., “Code- and Data-Segment Descriptor Types”, for more information about segment types. For expand-up segments, the offset in a logical address can range from 0 to the segment limit. Offsets greater than the segment limit generate general-protection exceptions (#GP). For expand-down segments, the segment limit has the reverse function; the offset can range from the segment limit to FFFFFFFFH or FFFFH, depending on the setting of the B flag. Offsets less than the segment limit generate general-protection exceptions. Decreasing the value in the segment limit field for an expand-down segment allocates new memory at the bottom of the segment's address space, rather than at the top. IA-32 architecture stacks always grow downwards, making this mechanism convenient for expandable stacks.

Base address fields

Defines the location of byte 0 of the segment within the 4-GByte linear address space. The processor puts together the three base address fields to form a single 32-bit value. Segment base addresses should be aligned to 16-byte boundaries. Although 16-byte alignment is not required, this alignment allows programs to maximize performance by aligning code and data on 16-byte boundaries.

Type field

Indicates the segment or gate type and specifies the kinds of access that can be made to the segment and the direction of growth. The interpretation of this field depends on whether the descriptor type flag specifies an application (code or data) descriptor or a system descriptor. The encoding of the type field is different for code, data, and system descriptors (see Figure 4-1). See Section 3.4.3.1., “Code- and Data-Segment Descriptor Types”, for a description of how this field is used to specify code and data-segment types.

S (descriptor type) flag

Specifies whether the segment descriptor is for a system segment (S flag is clear) or a code or data segment (S flag is set).

DPL (descriptor privilege level) field

Specifies the privilege level of the segment. The privilege level can range from 0 to 3, with 0 being the most privileged level. The DPL is used to control access to the segment. See Section 4.5., “Privilege Levels”, for a description of the relationship of the DPL to the CPL of the executing code segment and the RPL of a segment selector.

P (segment-present) flag

Indicates whether the segment is present in memory (set) or not present (clear). If this flag is clear, the processor generates a segment-not-present exception (#NP) when a segment selector that points to the segment descriptor is loaded into a segment register. Memory management software can use this flag to control which segments are actually loaded into physical memory at a given time. It offers a control in addition to paging for managing virtual memory.

Figure 3-9 shows the format of a segment descriptor when the segment-present flag is clear. When this flag is clear, the operating system or executive is free to use the locations marked “Available” to store its own data, such as information regarding the whereabouts of the missing segment.

D/B (default operation size/default stack pointer size and/or upper bound) flag

Performs different functions depending on whether the segment descriptor is an executable code segment, an expand-down data segment, or a stack segment. (This flag should always be set to 1 for 32-bit code and data segments and to 0 for 16-bit code and data segments.)

- **Executable code segment.** The flag is called the D flag and it indicates the default length for effective addresses and operands referenced by instructions in the segment. If the flag is set, 32-bit addresses and 32-bit or 8-bit operands are assumed; if it is clear, 16-bit addresses and 16-bit or 8-bit operands are assumed.

The instruction prefix 66H can be used to select an operand size other than the default, and the prefix 67H can be used select an address size other than the default.

- **Stack segment (data segment pointed to by the SS register).** The flag is called the B (big) flag and it specifies the size of the stack pointer used for implicit stack operations (such as pushes, pops, and calls). If the flag is set, a 32-bit stack pointer is used, which is stored in the 32-bit ESP register; if the flag is clear, a 16-bit stack pointer is used, which is stored in the 16-bit SP register. If the stack segment is set up to be an expand-down data segment (described in the next paragraph), the B flag also specifies the upper bound of the stack segment.
- **Expand-down data segment.** The flag is called the B flag and it specifies the upper bound of the segment. If the flag is set, the upper bound is FFFFFFFFH (4 GBytes); if the flag is clear, the upper bound is FFFFH (64 KBytes).

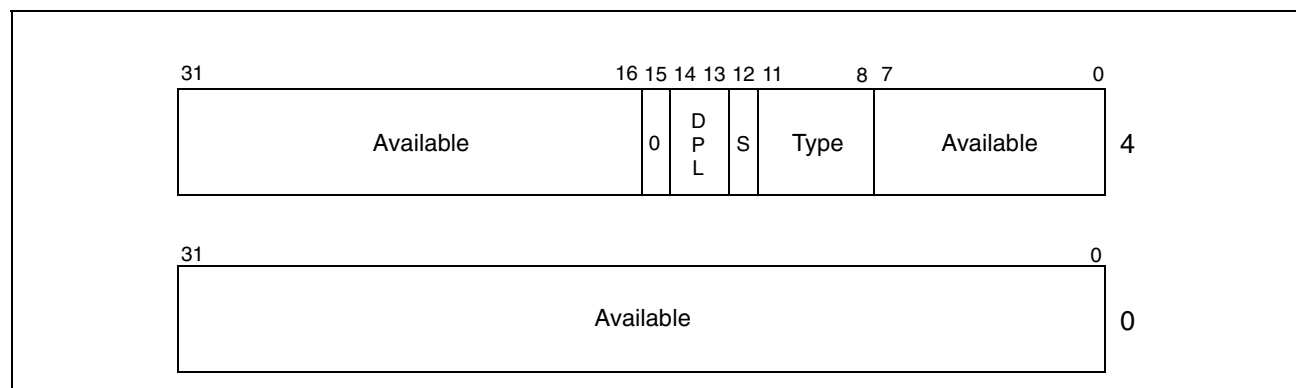


Figure 3-9. Segment Descriptor When Segment-Present Flag Is Clear

G (granularity) flag

Determines the scaling of the segment limit field. When the granularity flag is clear, the segment limit is interpreted in byte units; when flag is set, the segment limit is interpreted in 4-KByte units. (This flag does not affect the granularity of the base address; it is always byte granular.) When the granularity flag is set, the twelve least significant bits of an offset are not tested when checking the offset against the segment limit. For example, when the granularity flag is set, a limit of 0 results in valid offsets from 0 to 4095.

Available and reserved bits

Bit 20 of the second doubleword of the segment descriptor is available for use by system software; bit 21 is reserved and should always be set to 0.

3.4.3.1. CODE- AND DATA-SEGMENT DESCRIPTOR TYPES

When the S (descriptor type) flag in a segment descriptor is set, the descriptor is for either a code or a data segment. The highest order bit of the type field (bit 11 of the second double word of the segment descriptor) then determines whether the descriptor is for a data segment (clear) or a code segment (set).

For data segments, the three low-order bits of the type field (bits 8, 9, and 10) are interpreted as accessed (A), write-enable (W), and expansion-direction (E). See Table 3-1 for a description of the encoding of the bits in the type field for code and data segments. Data segments can be read-only or read/write segments, depending on the setting of the write-enable bit.

Table 3-1. Code- and Data-Segment Types

Type Field					Descriptor Type	Description
Decimal	11	10 E	9 W	8 A		
0	0	0	0	0	Data	Read-Only
1	0	0	0	1	Data	Read-Only, accessed
2	0	0	1	0	Data	Read/Write
3	0	0	1	1	Data	Read/Write, accessed
4	0	1	0	0	Data	Read-Only, expand-down
5	0	1	0	1	Data	Read-Only, expand-down, accessed
6	0	1	1	0	Data	Read/Write, expand-down
7	0	1	1	1	Data	Read/Write, expand-down, accessed
		C	R	A		
8	1	0	0	0	Code	Execute-Only
9	1	0	0	1	Code	Execute-Only, accessed
10	1	0	1	0	Code	Execute/Read
11	1	0	1	1	Code	Execute/Read, accessed
12	1	1	0	0	Code	Execute-Only, conforming
13	1	1	0	1	Code	Execute-Only, conforming, accessed
14	1	1	1	0	Code	Execute/Read-Only, conforming
15	1	1	1	1	Code	Execute/Read-Only, conforming, accessed

Stack segments are data segments which must be read/write segments. Loading the SS register with a segment selector for a nonwritable data segment generates a general-protection exception (#GP). If the size of a stack segment needs to be changed dynamically, the stack segment can be an expand-down data segment (expansion-direction flag set). Here, dynamically changing the segment limit causes stack space to be added to the bottom of the stack. If the size of a stack segment is intended to remain static, the stack segment may be either an expand-up or expand-down type.

The accessed bit indicates whether the segment has been accessed since the last time the operating-system or executive cleared the bit. The processor sets this bit whenever it loads a segment selector for the segment into a segment register, assuming that the type of memory that contains the segment descriptor supports processor writes. The bit remains set until explicitly cleared. This bit can be used both for virtual memory management and for debugging.

For code segments, the three low-order bits of the type field are interpreted as accessed (A), read enable (R), and conforming (C). Code segments can be execute-only or execute/read, depending on the setting of the read-enable bit. An execute/read segment might be used when constants or other static data have been placed with instruction code in a ROM. Here, data can be read from the code segment either by using an instruction with a CS override prefix or by loading a segment selector for the code segment in a data-segment register (the DS, ES, FS, or GS registers). In protected mode, code segments are not writable.

Code segments can be either conforming or nonconforming. A transfer of execution into a more-privileged conforming segment allows execution to continue at the current privilege level. A transfer into a nonconforming segment at a different privilege level results in a general-protection exception (#GP), unless a call gate or task gate is used (see Section 4.8.1., “Direct Calls or Jumps to Code Segments”, for more information on conforming and nonconforming code segments). System utilities that do not access protected facilities and handlers for some types of exceptions (such as, divide error or overflow) may be loaded in conforming code segments. Utilities that need to be protected from less privileged programs and procedures should be placed in nonconforming code segments.

NOTE

Execution cannot be transferred by a call or a jump to a less-privileged (numerically higher privilege level) code segment, regardless of whether the target segment is a conforming or nonconforming code segment. Attempting such an execution transfer will result in a general-protection exception.

All data segments are nonconforming, meaning that they cannot be accessed by less privileged programs or procedures (code executing at numerically high privilege levels). Unlike code segments, however, data segments can be accessed by more privileged programs or procedures (code executing at numerically lower privilege levels) without using a special access gate.

If the segment descriptors in the GDT or an LDT are placed in ROM, the processor can enter an indefinite loop if software or the processor attempts to update (write to) the ROM-based segment descriptors. To prevent this problem, set the accessed bits for all segment descriptors that are placed in a ROM. Also, remove any operating-system or executive code that attempts to modify segment descriptors that are located in ROM.

3.5. SYSTEM DESCRIPTOR TYPES

When the S (descriptor type) flag in a segment descriptor is clear, the descriptor type is a system descriptor. The processor recognizes the following types of system descriptors:

- Local descriptor-table (LDT) segment descriptor.
- Task-state segment (TSS) descriptor.
- Call-gate descriptor.

- Interrupt-gate descriptor.
- Trap-gate descriptor.
- Task-gate descriptor.

These descriptor types fall into two categories: system-segment descriptors and gate descriptors. System-segment descriptors point to system segments (LDT and TSS segments). Gate descriptors are in themselves “gates,” which hold pointers to procedure entry points in code segments (call, interrupt, and trap gates) or which hold segment selectors for TSS’s (task gates). Table 3-2 shows the encoding of the type field for system-segment descriptors and gate descriptors.

Table 3-2. System-Segment and Gate-Descriptor Types

Type Field					Description
Decimal	11	10	9	8	
0	0	0	0	0	Reserved
1	0	0	0	1	16-Bit TSS (Available)
2	0	0	1	0	LDT
3	0	0	1	1	16-Bit TSS (Busy)
4	0	1	0	0	16-Bit Call Gate
5	0	1	0	1	Task Gate
6	0	1	1	0	16-Bit Interrupt Gate
7	0	1	1	1	16-Bit Trap Gate
8	1	0	0	0	Reserved
9	1	0	0	1	32-Bit TSS (Available)
10	1	0	1	0	Reserved
11	1	0	1	1	32-Bit TSS (Busy)
12	1	1	0	0	32-Bit Call Gate
13	1	1	0	1	Reserved
14	1	1	1	0	32-Bit Interrupt Gate
15	1	1	1	1	32-Bit Trap Gate

For more information on the system-segment descriptors, see Section 3.5.1., “Segment Descriptor Tables”, and Section 6.2.2., “TSS Descriptor”; for more information on the gate descriptors, see Section 4.8.3., “Call Gates”, Section 5.11., “IDT Descriptors”, and Section 6.2.4., “Task-Gate Descriptor”.

3.5.1. Segment Descriptor Tables

A segment descriptor table is an array of segment descriptors (see Figure 3-10). A descriptor table is variable in length and can contain up to 8192 (2^{13}) 8-byte descriptors. There are two kinds of descriptor tables:

- The global descriptor table (GDT)
- The local descriptor tables (LDT)

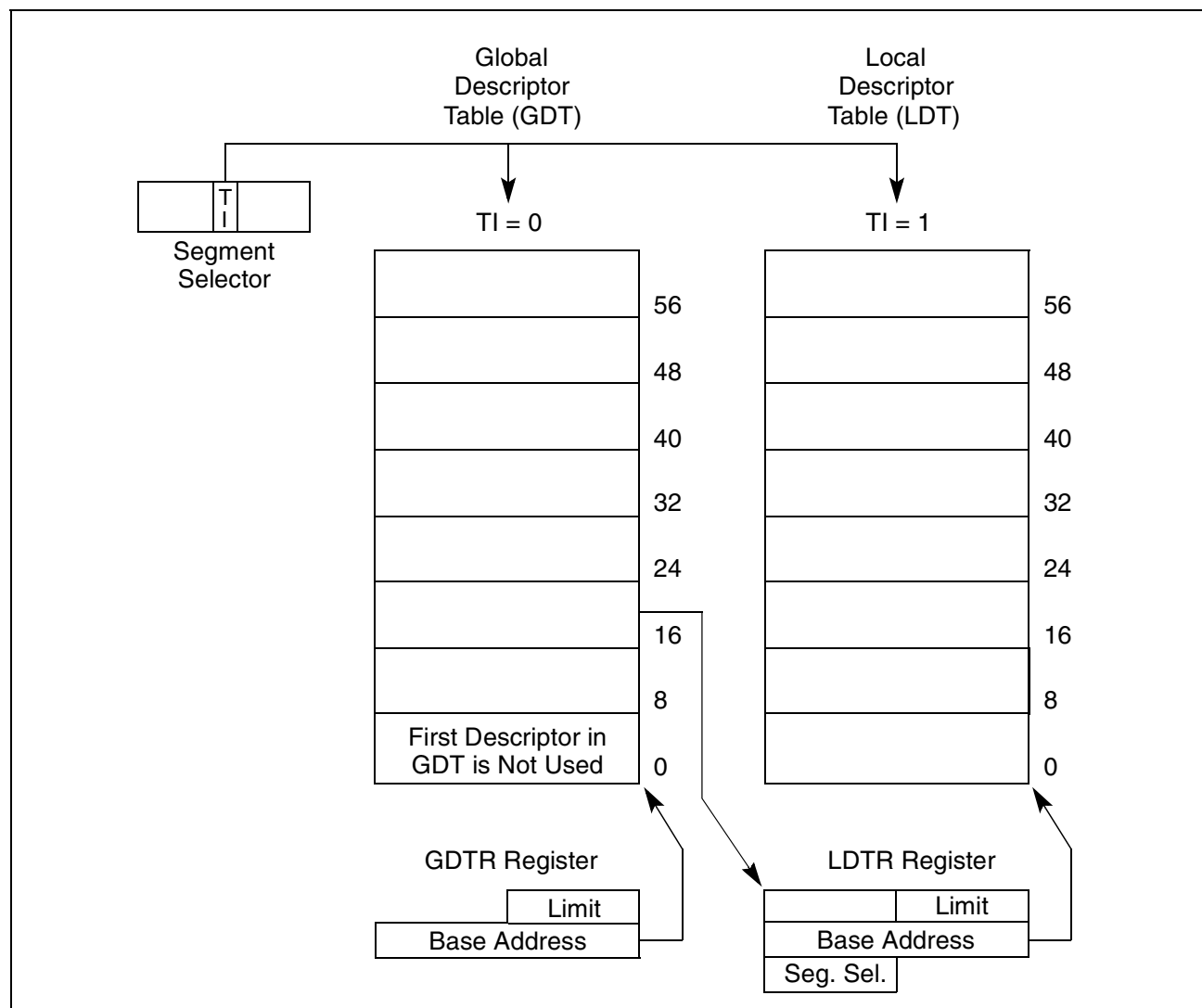


Figure 3-10. Global and Local Descriptor Tables

Each system must have one GDT defined, which may be used for all programs and tasks in the system. Optionally, one or more LDTs can be defined. For example, an LDT can be defined for each separate task being run, or some or all tasks can share the same LDT.

The GDT is not a segment itself; instead, it is a data structure in the linear address space. The base linear address and limit of the GDT must be loaded into the GDTR register (see Section 2.4., “Memory-Management Registers”). The base addresses of the GDT should be aligned on

an eight-byte boundary to yield the best processor performance. The limit value for the GDT is expressed in bytes. As with segments, the limit value is added to the base address to get the address of the last valid byte. A limit value of 0 results in exactly one valid byte. Because segment descriptors are always 8 bytes long, the GDT limit should always be one less than an integral multiple of eight (that is, $8N - 1$).

The first descriptor in the GDT is not used by the processor. A segment selector to this “null descriptor” does not generate an exception when loaded into a data-segment register (DS, ES, FS, or GS), but it always generates a general-protection exception (#GP) when an attempt is made to access memory using the descriptor. By initializing the segment registers with this segment selector, accidental reference to unused segment registers can be guaranteed to generate an exception.

The LDT is located in a system segment of the LDT type. The GDT must contain a segment descriptor for the LDT segment. If the system supports multiple LDTs, each must have a separate segment selector and segment descriptor in the GDT. The segment descriptor for an LDT can be located anywhere in the GDT. See Section 3.5., “System Descriptor Types”, information on the LDT segment-descriptor type.

An LDT is accessed with its segment selector. To eliminate address translations when accessing the LDT, the segment selector, base linear address, limit, and access rights of the LDT are stored in the LDTR register (see Section 2.4., “Memory-Management Registers”).

When the GDTR register is stored (using the SGDT instruction), a 48-bit “pseudo-descriptor” is stored in memory (see Figure 3-11). To avoid alignment check faults in user mode (privilege level 3), the pseudo-descriptor should be located at an odd word address (that is, address MOD 4 is equal to 2). This causes the processor to store an aligned word, followed by an aligned doubleword. User-mode programs normally do not store pseudo-descriptors, but the possibility of generating an alignment check fault can be avoided by aligning pseudo-descriptors in this way. The same alignment should be used when storing the IDTR register using the SIDT instruction. When storing the LDTR or task register (using the SLTR or STR instruction, respectively), the pseudo-descriptor should be located at a doubleword address (that is, address MOD 4 is equal to 0).

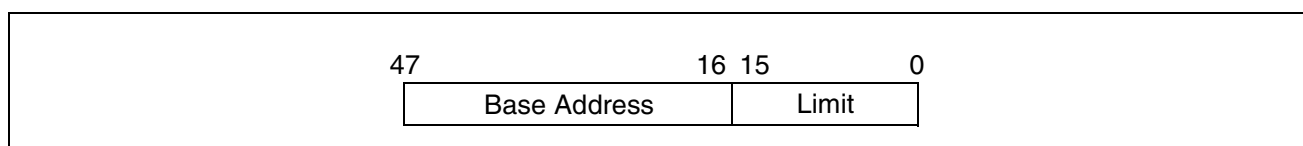


Figure 3-11. Pseudo-Descriptor Format

3.6. PAGING (VIRTUAL MEMORY) OVERVIEW

When operating in protected mode, the IA-32 architecture permits the linear address space to be mapped directly into a large physical memory (for example, 4 GBytes of RAM) or indirectly (using paging) into a smaller physical memory and disk storage. This latter method of mapping the linear address space is commonly referred to as virtual memory or demand-paged virtual memory.

When paging is used, the processor divides the linear address space into fixed-size pages (of 4 KBytes, 2 MBytes, or 4 MBytes in length) that can be mapped into physical memory and/or disk storage. When a program (or task) references a logical address in memory, the processor translates the address into a linear address and then uses its paging mechanism to translate the linear address into a corresponding physical address.

If the page containing the linear address is not currently in physical memory, the processor generates a page-fault exception (#PF). The exception handler for the page-fault exception typically directs the operating system or executive to load the page from disk storage into physical memory (perhaps writing a different page from physical memory out to disk in the process). When the page has been loaded in physical memory, a return from the exception handler causes the instruction that generated the exception to be restarted. The information that the processor uses to map linear addresses into the physical address space and to generate page-fault exceptions (when necessary) is contained in page directories and page tables stored in memory.

Paging is different from segmentation through its use of fixed-size pages. Unlike segments, which usually are the same size as the code or data structures they hold, pages have a fixed size. If segmentation is the only form of address translation used, a data structure present in physical memory will have all of its parts in memory. If paging is used, a data structure can be partly in memory and partly in disk storage.

To minimize the number of bus cycles required for address translation, the most recently accessed page-directory and page-table entries are cached in the processor in devices called translation lookaside buffers (TLBs). The TLBs satisfy most requests for reading the current page directory and page tables without requiring a bus cycle. Extra bus cycles occur only when the TLBs do not contain a page-table entry, which typically happens when a page has not been accessed for a long time. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for more information on the TLBs.

3.6.1. Paging Options

Paging is controlled by three flags in the processor’s control registers:

- **PG (paging) flag.** Bit 31 of CR0 (available in all IA-32 processors beginning with the Intel386 processor).
- **PSE (page size extensions) flag.** Bit 4 of CR4 (introduced in the Pentium processor).
- **PAE (physical address extension) flag.** Bit 5 of CR4 (introduced in the Pentium Pro processors).

The PG flag enables the page-translation mechanism. The operating system or executive usually sets this flag during processor initialization. The PG flag must be set if the processor’s page-translation mechanism is to be used to implement a demand-paged virtual memory system or if the operating system is designed to run more than one program (or task) in virtual-8086 mode.

The PSE flag enables large page sizes: 4-MByte pages or 2-MByte pages (when the PAE flag is set). When the PSE flag is clear, the more common page length of 4 KBytes is used. See Section 3.7.2., “Linear Address Translation (4-MByte Pages)”, Section 3.8.2., “Linear Address Translation With PAE Enabled (2-MByte Pages)”, and Section 3.9., “36-Bit Physical Addressing Using the PSE-36 Paging Mechanism” for more information about the use of the PSE flag.

The PAE flag provides a method of extending physical addresses to 36 bits. This physical address extension can only be used when paging is enabled. It relies on an additional page directory pointer table that is used along with page directories and page tables to reference physical addresses above FFFFFFFFH. See Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”, for more information about extending physical addresses using the PAE flag.

The 36-bit page size extension (PSE-36) feature provides an alternate method of extending physical addressing to 36 bits. This paging mechanism uses the page size extension mode (enabled with the PSE flag) and modified page directory entries to reference physical addresses above FFFFFFFFH. The PSE-36 feature flag (bit 17 in the EDX register when the CPUID instruction is executed with a source operand of 1) indicates the availability of this addressing mechanism. See Section 3.9., “36-Bit Physical Addressing Using the PSE-36 Paging Mechanism”, for more information about the PSE-36 physical address extension and page size extension mechanism.

3.6.2. Page Tables and Directories

The information that the processor uses to translate linear addresses into physical addresses (when paging is enabled) is contained in four data structures:

- **Page directory**—An array of 32-bit page-directory entries (PDEs) contained in a 4-KByte page. Up to 1024 page-directory entries can be held in a page directory.
- **Page table**—An array of 32-bit page-table entries (PTEs) contained in a 4-KByte page. Up to 1024 page-table entries can be held in a page table. (Page tables are not used for 2-MByte or 4-MByte pages. These page sizes are mapped directly from one or more page-directory entries.)
- **Page**—A 4-KByte, 2-MByte, or 4-MByte flat address space.
- **Page-Directory-Pointer Table**—An array of four 64-bit entries, each of which points to a page directory. This data structure is only used when the physical address extension is enabled (see Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”).

These tables provide access to either 4-KByte or 4-MByte pages when normal 32-bit physical addressing is being used and to either 4-KByte or 2-MByte pages or 4-MByte pages only when extended (36-bit) physical addressing is being used. Table 3-3 shows the page size and physical address size obtained from various settings of the paging control flags and the PSE-36 CPUID feature flag. Each page-directory entry contains a PS (page size) flag that specifies whether the entry points to a page table whose entries in turn point to 4-KByte pages (PS set to 0) or whether the page-directory entry points directly to a 4-MByte (PSE and PS set to 1) or 2-MByte page (PAE and PS set to 1).

3.7. PAGE TRANSLATION USING 32-BIT PHYSICAL ADDRESSING

The following sections describe the IA-32 architecture's page translation mechanism when using 32-bit physical addresses and a maximum physical address space of 4 Gbytes. Section 3.8., "36-Bit Physical Addressing Using the PAE Paging Mechanism" and Section 3.9., "36-Bit Physical Addressing Using the PSE-36 Paging Mechanism" describe extensions to this page translation mechanism to support 36-bit physical addresses and a maximum physical address space of 64 Gbytes.

Table 3-3. Page Sizes and Physical Address Sizes

PG Flag, CR0	PAE Flag, CR4	PSE Flag, CR4	PS Flag, PDE	PSE-36 CPUID Feature Flag	Page Size	Physical Address Size
0	X	X	X	X	—	Paging Disabled
1	0	0	X	X	4 KBytes	32 Bits
1	0	1	0	X	4 KBytes	32 Bits
1	0	1	1	0	4 MBytes	32 Bits
1	0	1	1	1	4 MBytes	36 Bits
1	1	X	0	X	4 KBytes	36 Bits
1	1	X	1	X	2 MBytes	36 Bits

3.7.1. Linear Address Translation (4-KByte Pages)

Figure 3-12 shows the page directory and page-table hierarchy when mapping linear addresses to 4-KByte pages. The entries in the page directory point to page tables, and the entries in a page table point to pages in physical memory. This paging method can be used to address up to 2^{20} pages, which spans a linear address space of 2^{32} bytes (4 GBytes).

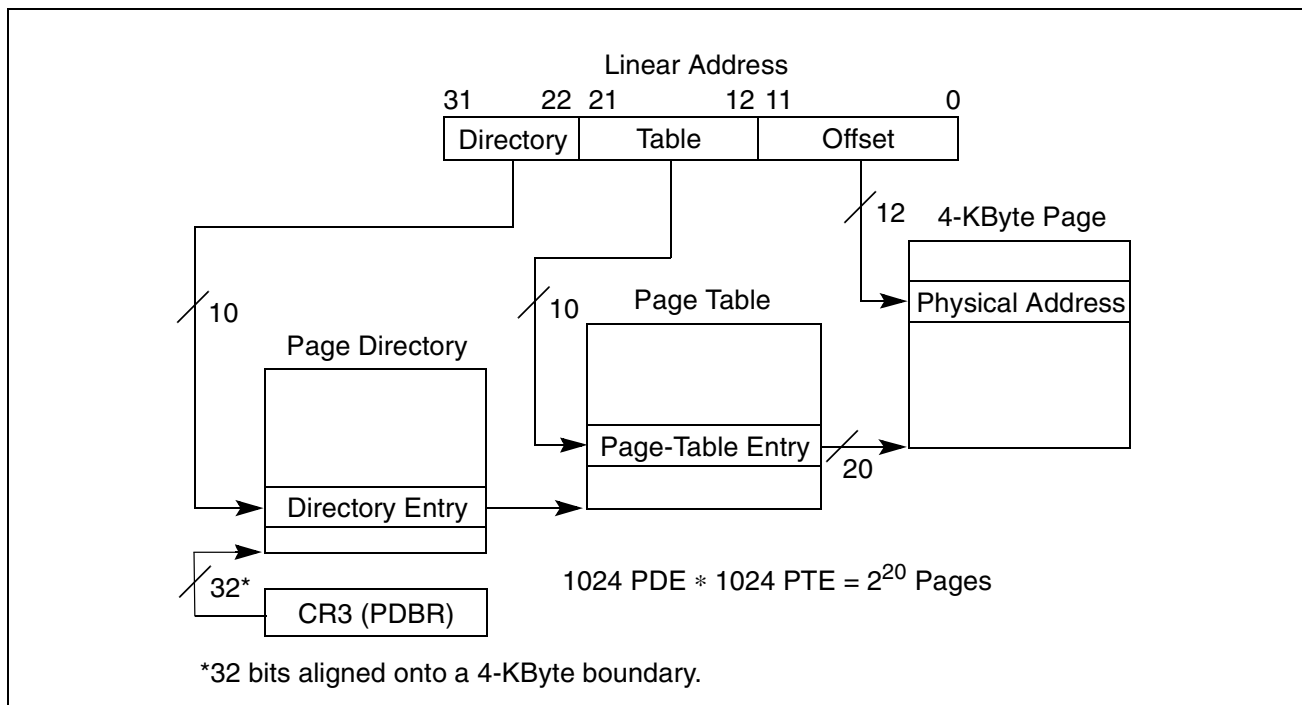


Figure 3-12. Linear Address Translation (4-KByte Pages)

To select the various table entries, the linear address is divided into three sections:

- Page-directory entry—Bits 22 through 31 provide an offset to an entry in the page directory. The selected entry provides the base physical address of a page table.
- Page-table entry—Bits 12 through 21 of the linear address provide an offset to an entry in the selected page table. This entry provides the base physical address of a page in physical memory.
- Page offset—Bits 0 through 11 provides an offset to a physical address in the page.

Memory management software has the option of using one page directory for all programs and tasks, one page directory for each task, or some combination of the two.

3.7.2. Linear Address Translation (4-MByte Pages)

Figure 3-12 shows how a page directory can be used to map linear addresses to 4-MByte pages. The entries in the page directory point to 4-MByte pages in physical memory. This paging method can be used to map up to 1024 pages into a 4-GByte linear address space.

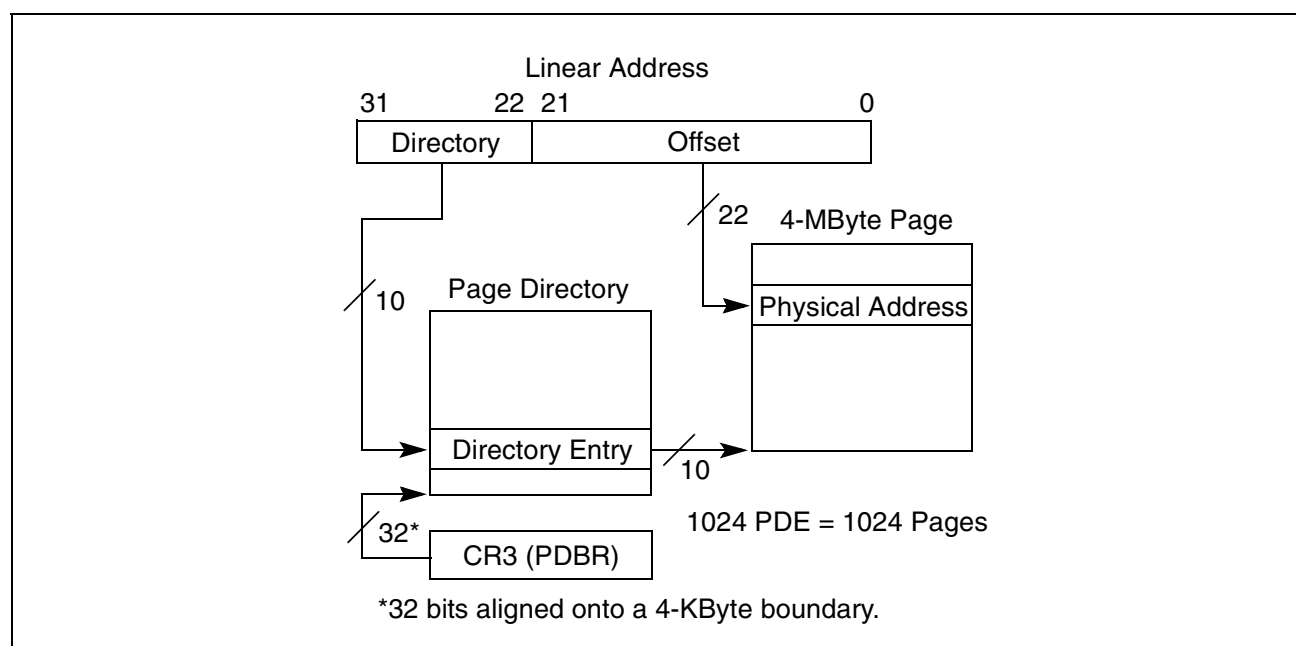


Figure 3-13. Linear Address Translation (4-MByte Pages)

The 4-MByte page size is selected by setting the PSE flag in control register CR4 and setting the page size (PS) flag in a page-directory entry (see Figure 3-14). With these flags set, the linear address is divided into two sections:

- Page directory entry—Bits 22 through 31 provide an offset to an entry in the page directory. The selected entry provides the base physical address of a 4-MByte page.
- Page offset—Bits 0 through 21 provides an offset to a physical address in the page.

NOTE

(For the Pentium processor only.) When enabling or disabling large page sizes, the TLBs must be invalidated (flushed) after the PSE flag in control register CR4 has been set or cleared. Otherwise, incorrect page translation might occur due to the processor using outdated page translation information stored in the TLBs. See Section 10.9., “Invalidating the Translation Lookaside Buffers (TLBs)”, for information on how to invalidate the TLBs.

3.7.3. Mixing 4-KByte and 4-MByte Pages

When the PSE flag in CR4 is set, both 4-MByte pages and page tables for 4-KByte pages can be accessed from the same page directory. If the PSE flag is clear, only page tables for 4-KByte pages can be accessed (regardless of the setting of the PS flag in a page-directory entry).

A typical example of mixing 4-KByte and 4-MByte pages is to place the operating system or executive’s kernel in a large page to reduce TLB misses and thus improve overall system performance.

The processor maintains 4-MByte page entries and 4-KByte page entries in separate TLBs. So, placing often used code such as the kernel in a large page, frees up 4-KByte-page TLB entries for application programs and tasks.

3.7.4. Memory Aliasing

The IA-32 architecture permits memory aliasing by allowing two page-directory entries to point to a common page-table entry. Software that needs to implement memory aliasing in this manner must manage the consistency of the accessed and dirty bits in the page-directory and page-table entries. Allowing the accessed and dirty bits for the two page-directory entries to become inconsistent may lead to a processor deadlock.

3.7.5. Base Address of the Page Directory

The physical address of the current page directory is stored in the CR3 register (also called the page directory base register or PDBR). (See Figure 2-5 and Section 2.5., “Control Registers”, for more information on the PDBR.) If paging is to be used, the PDBR must be loaded as part of the processor initialization process (prior to enabling paging). The PDBR can then be changed either explicitly by loading a new value in CR3 with a MOV instruction or implicitly as part of a task switch. (See Section 6.2.1., “Task-State Segment (TSS)”, for a description of how the contents of the CR3 register is set for a task.)

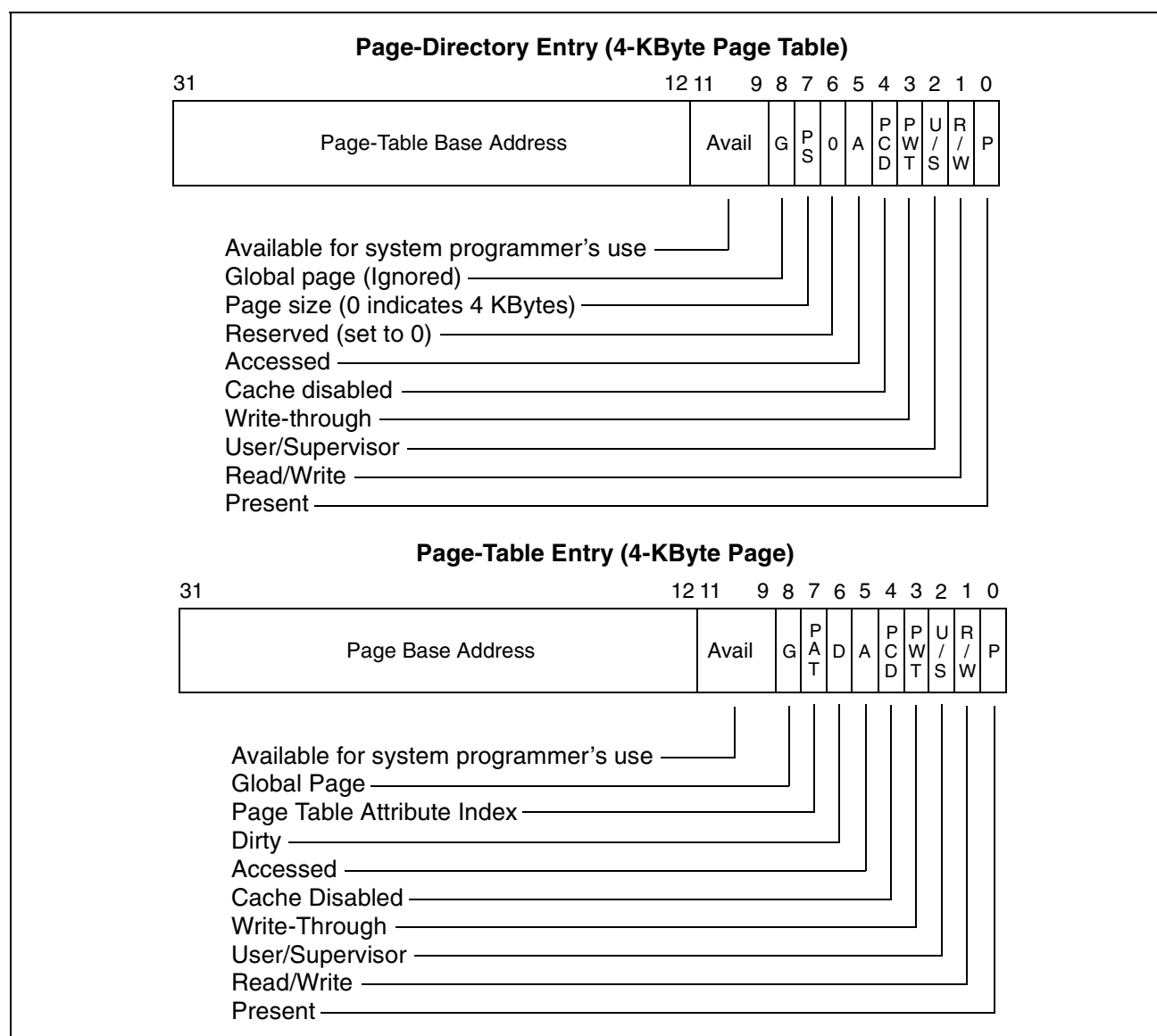
There is no present flag in the PDBR for the page directory. The page directory may be not-present (paged out of physical memory) while its associated task is suspended, but the operating system must ensure that the page directory indicated by the PDBR image in a task's TSS is present in physical memory before the task is dispatched. The page directory must also remain in memory as long as the task is active.

3.7.6. Page-Directory and Page-Table Entries

Figure 3-14 shows the format for the page-directory and page-table entries when 4-KByte pages and 32-bit physical addresses are being used. Figure 3-15 shows the format for the page-directory entries when 4-MByte pages and 32-bit physical addresses are being used. The functions of the flags and fields in the entries in Figures 3-14 and 3-15 are as follows:

Page base address, bits 12 through 32

(Page-table entries for 4-KByte pages.) Specifies the physical address of the first byte of a 4-KByte page. The bits in this field are interpreted as the 20 most-significant bits of the physical address, which forces pages to be aligned on 4-KByte boundaries.



(Page-directory entries for 4-KByte page tables.) Specifies the physical address of the first byte of a page table. The bits in this field are interpreted as the 20 most-significant bits of the physical address, which forces page tables to be aligned on 4-KByte boundaries.

(Page-directory entries for 4-MByte pages.) Specifies the physical address of the first byte of a 4-MByte page. Only bits 22 through 31 of this field are used (and bits 12 through 21 are reserved and must be set to 0, for IA-32 processors through the Pentium II processor). The base address bits are interpreted as the 10 most-significant bits of the physical address, which forces 4-MByte pages to be aligned on 4-MByte boundaries.

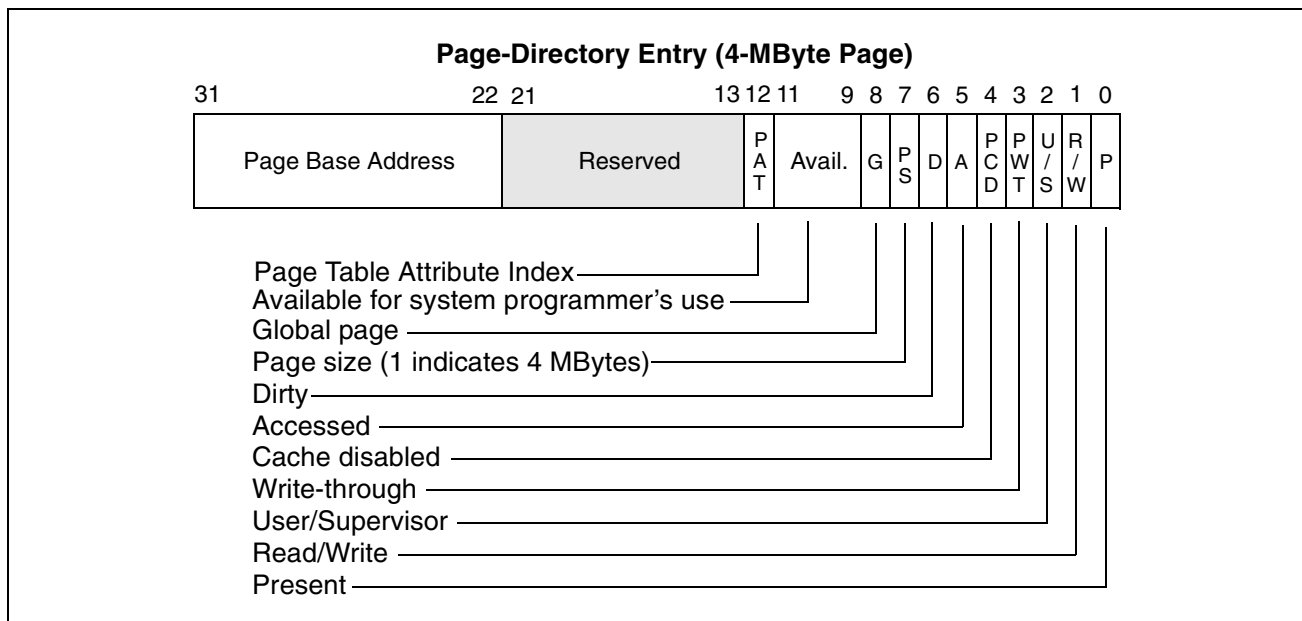


Figure 3-15. Format of Page-Directory Entries for 4-MByte Pages and 32-Bit Addresses

Present (P) flag, bit 0

Indicates whether the page or page table being pointed to by the entry is currently loaded in physical memory. When the flag is set, the page is in physical memory and address translation is carried out. When the flag is clear, the page is not in memory and, if the processor attempts to access the page, it generates a page-fault exception (#PF).

The processor does not set or clear this flag; it is up to the operating system or executive to maintain the state of the flag.

If the processor generates a page-fault exception, the operating system generally needs to carry out the following operations:

1. Copy the page from disk storage into physical memory.
2. Load the page address into the page-table or page-directory entry and set its present flag. Other flags, such as the dirty and accessed flags, may also be set at this time.
3. Invalidate the current page-table entry in the TLB (see Section 3.11., "Translation Lookaside Buffers (TLBs)", for a discussion of TLBs and how to invalidate them).
4. Return from the page-fault handler to restart the interrupted program (or task).

Read/write (R/W) flag, bit 1

Specifies the read-write privileges for a page or group of pages (in the case of a page-directory entry that points to a page table). When this flag is clear, the page is read only; when the flag is set, the page can be read and written into.

This flag interacts with the U/S flag and the WP flag in register CR0. See Section 4.11., “Page-Level Protection”, and Table 4-2 for a detailed discussion of the use of these flags.

User/supervisor (U/S) flag, bit 2

Specifies the user-supervisor privileges for a page or group of pages (in the case of a page-directory entry that points to a page table). When this flag is clear, the page is assigned the supervisor privilege level; when the flag is set, the page is assigned the user privilege level. This flag interacts with the R/W flag and the WP flag in register CR0. See Section 4.11., “Page-Level Protection”, and Table 4-2 for a detail discussion of the use of these flags.

Page-level write-through (PWT) flag, bit 3

Controls the write-through or write-back caching policy of individual pages or page tables. When the PWT flag is set, write-through caching is enabled for the associated page or page table; when the flag is clear, write-back caching is enabled for the associated page or page table. The processor ignores this flag if the CD (cache disable) flag in CR0 is set. See Section 10.5., “Cache Control”, for more information about the use of this flag. See Section 2.5., “Control Registers”, for a description of a companion PWT flag in control register CR3.

Page-level cache disable (PCD) flag, bit 4

Controls the caching of individual pages or page tables. When the PCD flag is set, caching of the associated page or page table is prevented; when the flag is clear, the page or page table can be cached. This flag permits caching to be disabled for pages that contain memory-mapped I/O ports or that do not provide a performance benefit when cached. The processor ignores this flag (assumes it is set) if the CD (cache disable) flag in CR0 is set. See Chapter 10, *Memory Cache Control*, for more information about the use of this flag. See Section 2.5., “Control Registers”, for a description of a companion PCD flag in control register CR3.

Accessed (A) flag, bit 5

Indicates whether a page or page table has been accessed (read from or written to) when set. Memory management software typically clears this flag when a page or page table is initially loaded into physical memory. The processor then sets this flag the first time a page or page table is accessed. This flag is a “sticky” flag, meaning that once set, the processor does not implicitly clear it. Only software can clear this flag.

The accessed and dirty flags are provided for use by memory management software to manage the transfer of pages and page tables into and out of physical memory.

Dirty (D) flag, bit 6

Indicates whether a page has been written to when set. (This flag is not used in page-directory entries that point to page tables.) Memory management software typically clears this flag when a page is initially loaded into physical memory. The processor then sets this flag the first time a page is accessed for a write operation.

This flag is “sticky,” meaning that once set, the processor does not implicitly clear it. Only software can clear this flag. The dirty and accessed flags are provided for use by memory management software to manage the transfer of pages and page tables into and out of physical memory.

Page size (PS) flag, bit 7 page-directory entries for 4-KByte pages

Determines the page size. When this flag is clear, the page size is 4 KBytes and the page-directory entry points to a page table. When the flag is set, the page size is 4 MBytes for normal 32-bit addressing (and 2 MBytes if extended physical addressing is enabled) and the page-directory entry points to a page. If the page-directory entry points to a page table, all the pages associated with that page table will be 4-KByte pages.

Page attribute table index (PAT) flag, bit 7 in page-table entries for 4-KByte pages and bit 12 in page-directory entries for 4-MByte pages

(Introduced in the Pentium III processor.) Selects PAT entry. For processors that support the page attribute table (PAT), this flag is used along with the PCD and PWT flags to select an entry in the PAT, which in turn selects the memory type for the page (see Section 10.12., “Page Attribute Table (PAT)”). For processors that do not support the PAT, this bit is reserved and should be set to 0.

Global (G) flag, bit 8

(Introduced in the Pentium Pro processor.) Indicates a global page when set. When a page is marked global and the page global enable (PGE) flag in register CR4 is set, the page-table or page-directory entry for the page is not invalidated in the TLB when register CR3 is loaded or a task switch occurs. This flag is provided to prevent frequently used pages (such as pages that contain kernel or other operating system or executive code) from being flushed from the TLB. Only software can set or clear this flag. For page-directory entries that point to page tables, this flag is ignored and the global characteristics of a page are set in the page-table entries. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for more information about the use of this flag. (This bit is reserved in Pentium and earlier IA-32 processors.)

Reserved and available-to-software bits

For all IA-32 processors. Bits 9, 10, and 11 are available for use by software. (When the present bit is clear, bits 1 through 31 are available to software—see Figure 3-16.) In a page-directory entry that points to a page table, bit 6 is reserved and should be set to 0. When the PSE and PAE flags in control register CR4 are set, the processor generates a page fault if reserved bits are not set to 0.

For Pentium II and earlier processors. Bit 7 in a page-table entry is reserved and should be set to 0. For a page-directory entry for a 4-MByte page, bits 12 through 21 are reserved and must be set to 0.

For Pentium III and later processors. For a page-directory entry for a 4-MByte page, bits 13 through 21 are reserved and must be set to 0.

3.7.7. Not Present Page-Directory and Page-Table Entries

When the present flag is clear for a page-table or page-directory entry, the operating system or executive may use the rest of the entry for storage of information such as the location of the page in the disk storage system (see Figure 3-16).

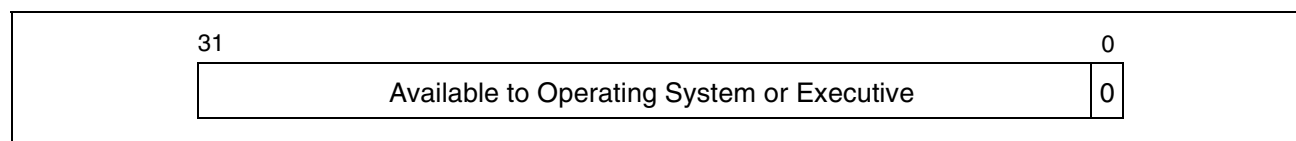


Figure 3-16. Format of a Page-Table or Page-Directory Entry for a Not-Present Page

3.8. 36-BIT PHYSICAL ADDRESSING USING THE PAE PAGING MECHANISM

The PAE paging mechanism and support for 36-bit physical addressing were introduced into the IA-32 architecture in the Pentium Pro processors. Implementation of this feature in an IA-32 processor is indicated with CPUID feature flag PAE (bit 6 in the EDX register when the source operand for the CPUID instruction is 2). The physical address extension (PAE) flag in register CR4 enables the PAE mechanism and extends physical addresses from 32 bits to 36 bits. Here, the processor provides 4 additional address line pins to accommodate the additional address bits. To use this option, the following flags must be set:

- PG flag (bit 31) in control register CR0—Enables paging
- PAE flag (bit 5) in control register CR4 are set—Enables the PAE paging mechanism.

When the PAE paging mechanism is enabled, the processor supports two sizes of pages: 4-KByte and 2-MByte. As with 32-bit addressing, both page sizes can be addressed within the same set of paging tables (that is, a page-directory entry can point to either a 2-MByte page or a page table that in turn points to 4-KByte pages). To support the 36-bit physical addresses, the following changes are made to the paging data structures:

- The paging table entries are increased to 64 bits to accommodate 36-bit base physical addresses. Each 4-KByte page directory and page table can thus have up to 512 entries.
- A new table, called the page-directory-pointer table, is added to the linear-address translation hierarchy. This table has 4 entries of 64-bits each, and it lies above the page directory in the hierarchy. With the physical address extension mechanism enabled, the processor supports up to 4 page directories.
- The 20-bit page-directory base address field in register CR3 (PDPR) is replaced with a 27-bit page-directory-pointer-table base address field (see Figure 3-17). (In this case, register CR3 is called the PDPTR.) This field provides the 27 most-significant bits of the physical address of the first byte of the page-directory-pointer table, which forces the table to be located on a 32-byte boundary.
- Linear address translation is changed to allow mapping 32-bit linear addresses into the larger physical address space.

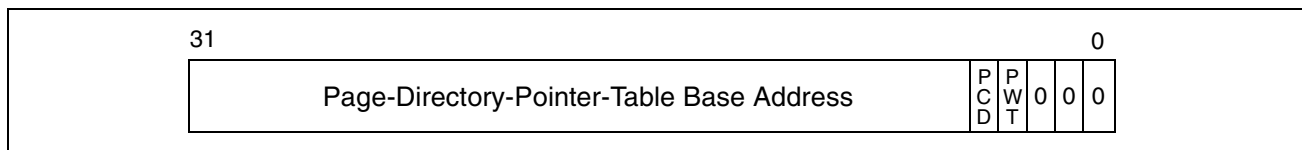


Figure 3-17. Register CR3 Format When the Physical Address Extension is Enabled

3.8.1. Linear Address Translation With PAE Enabled (4-KByte Pages)

Figure 3-18 shows the page-directory-pointer, page-directory, and page-table hierarchy when mapping linear addresses to 4-KByte pages when the PAE paging mechanism enabled. This paging method can be used to address up to 2^{20} pages, which spans a linear address space of 2^{32} bytes (4 GBytes).

To select the various table entries, the linear address is divided into three sections:

- Page-directory-pointer-table entry—Bits 30 and 31 provide an offset to one of the 4 entries in the page-directory-pointer table. The selected entry provides the base physical address of a page directory.
- Page-directory entry—Bits 21 through 29 provide an offset to an entry in the selected page directory. The selected entry provides the base physical address of a page table.
- Page-table entry—Bits 12 through 20 provide an offset to an entry in the selected page table. This entry provides the base physical address of a page in physical memory.
- Page offset—Bits 0 through 11 provide an offset to a physical address in the page.

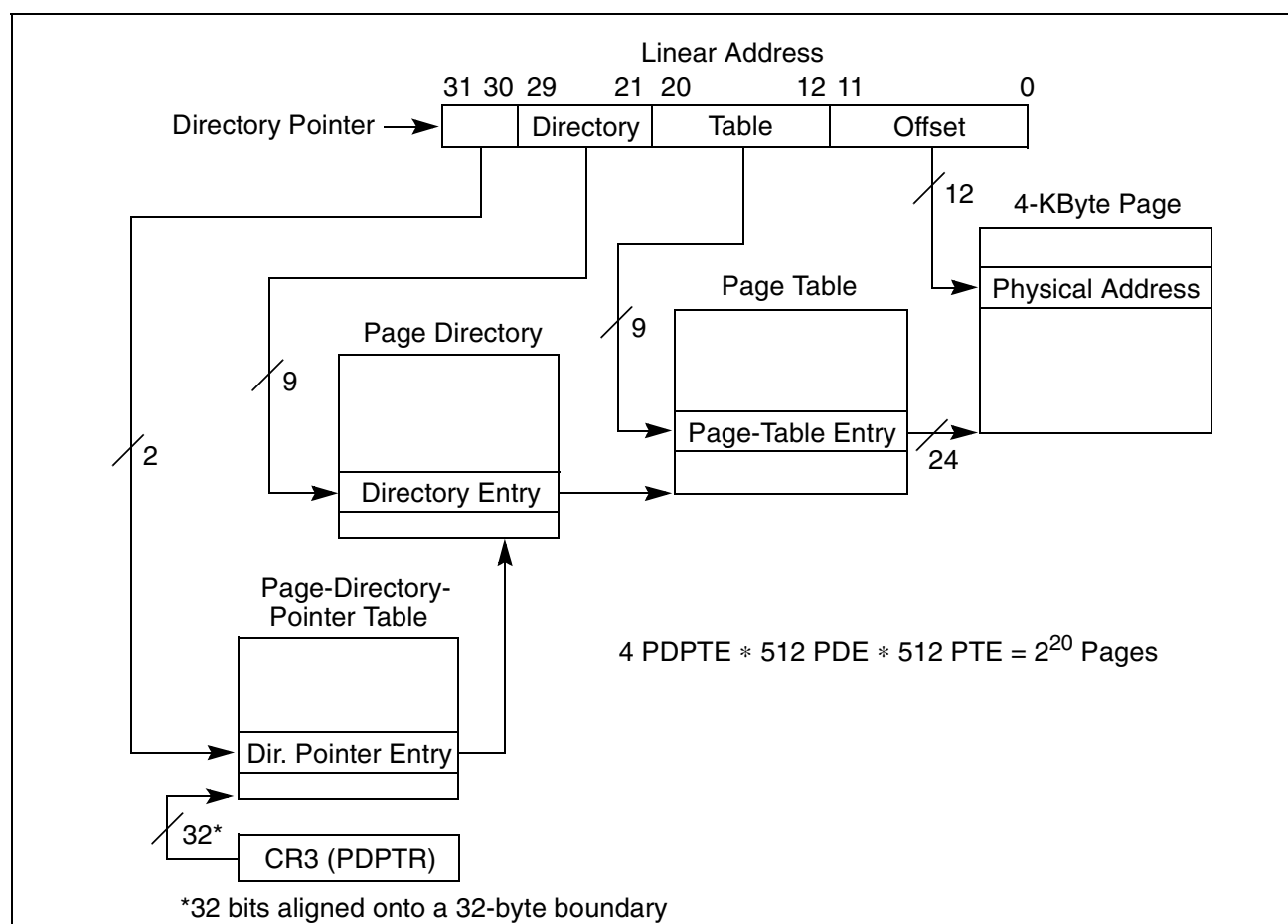


Figure 3-18. Linear Address Translation With PAE Enabled (4-KByte Pages)

3.8.2. Linear Address Translation With PAE Enabled (2-MByte Pages)

Figure 3-19 shows how a page-directory-pointer table and page directories can be used to map linear addresses to 2-MByte pages when the PAE paging mechanism enabled. This paging method can be used to map up to 2048 pages (4 page-directory-pointer-table entries times 512 page-directory entries) into a 4-GByte linear address space.

When PAE is enabled, the 2-MByte page size is selected by setting the page size (PS) flag in a page-directory entry (see Figure 3-14). (As shown in Table 3-3, the PSE flag in control register CR4 has no effect on the page size when PAE is enabled.) With the PS flag set, the linear address is divided into three sections:

- **Page-directory-pointer-table entry**—Bits 30 and 31 provide an offset to an entry in the page-directory-pointer table. The selected entry provides the base physical address of a page directory.
- **Page-directory entry**—Bits 21 through 29 provide an offset to an entry in the page directory. The selected entry provides the base physical address of a 2-MByte page.
- **Page offset**—Bits 0 through 20 provides an offset to a physical address in the page.

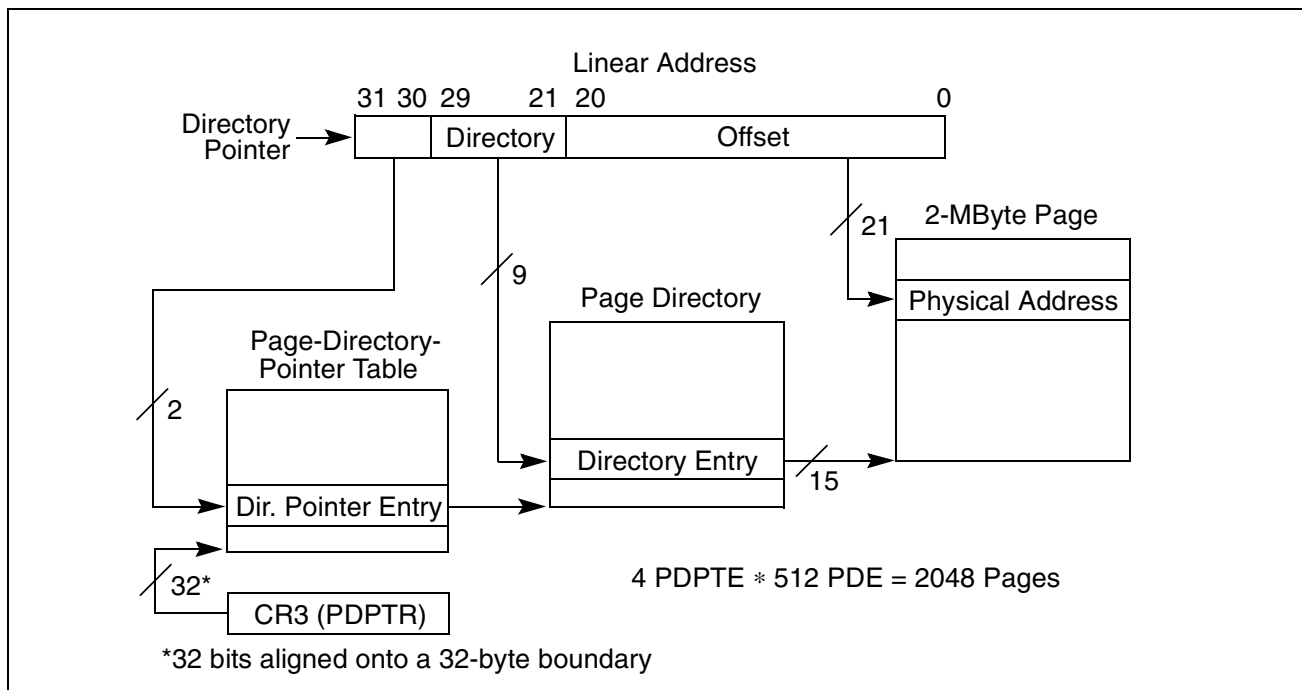


Figure 3-19. Linear Address Translation With PAE Enabled (2-MByte Pages)

3.8.3. Accessing the Full Extended Physical Address Space With the Extended Page-Table Structure

The page-table structure described in the previous two sections allows up to 4 GBytes of the 64 GByte extended physical address space to be addressed at one time. Additional 4-GByte sections of physical memory can be addressed in either of two way:

- Change the pointer in register CR3 to point to another page-directory-pointer table, which in turn points to another set of page directories and page tables.
- Change entries in the page-directory-pointer table to point to other page directories, which in turn point to other sets of page tables.

3.8.4. Page-Directory and Page-Table Entries With Extended Addressing Enabled

Figure 3-20 shows the format for the page-directory-pointer-table, page-directory, and page-table entries when 4-KByte pages and 36-bit extended physical addresses are being used. Figure 3-21 shows the format for the page-directory-pointer-table and page-directory entries when 2-MByte pages and 36-bit extended physical addresses are being used. The functions of the flags in these entries are the same as described in Section 3.7.6., “Page-Directory and Page-Table Entries”. The major differences in these entries are as follows:

- A page-directory-pointer-table entry is added.
- The size of the entries are increased from 32 bits to 64 bits.

- The maximum number of entries in a page directory or page table is 512.
- The base physical address field in each entry is extended to 24 bits.

NOTE

Current IA-32 processors that implement the PAE mechanism use uncached accesses when loading page-directory-pointer table entries. This behavior is model specific and not architectural. Future IA-32 processors may cache page-directory-pointer table entries.

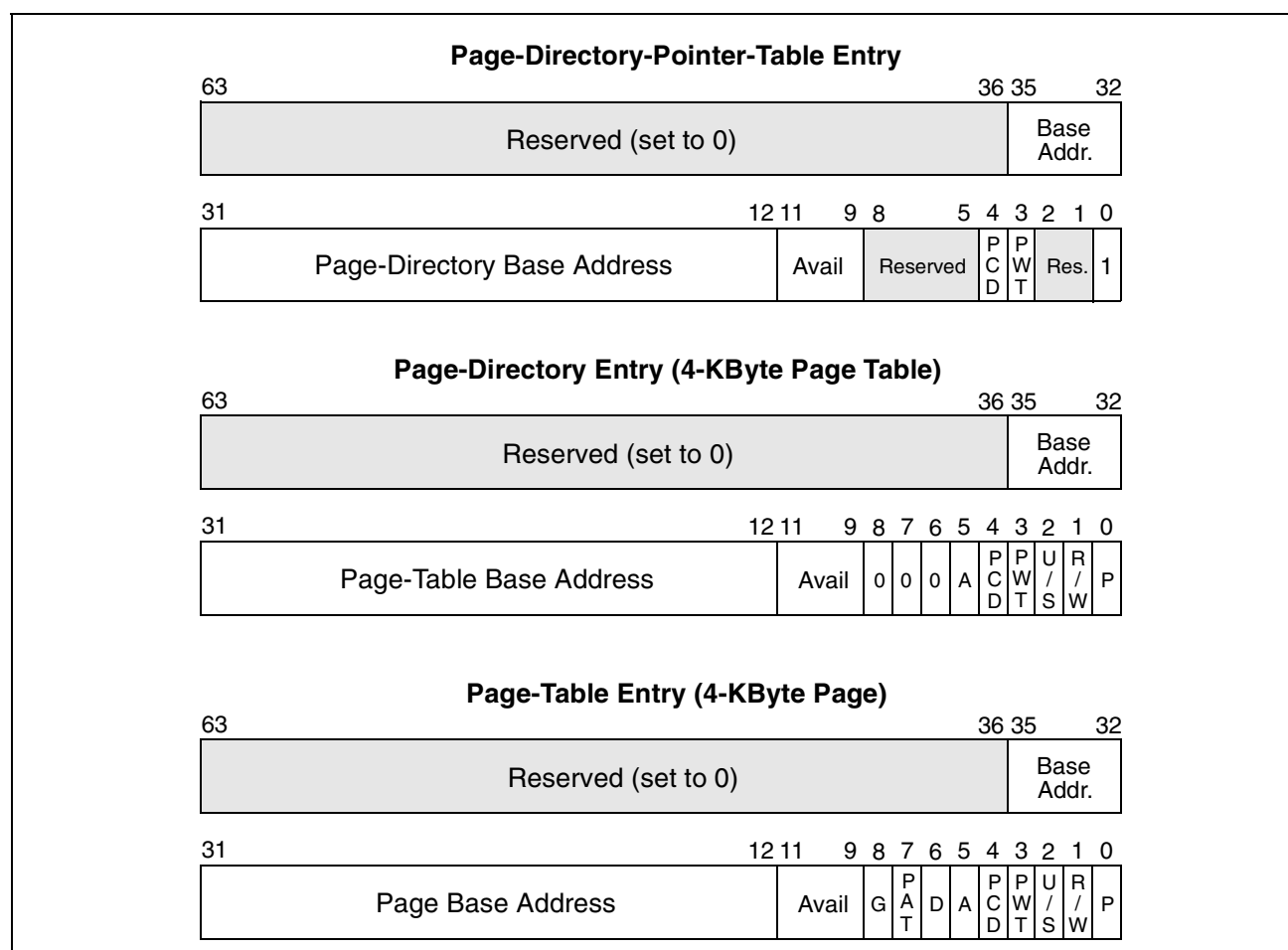


Figure 3-20. Format of Page-Directory-Pointer-Table, Page-Directory, and Page-Table Entries for 4-KByte Pages with PAE Enabled

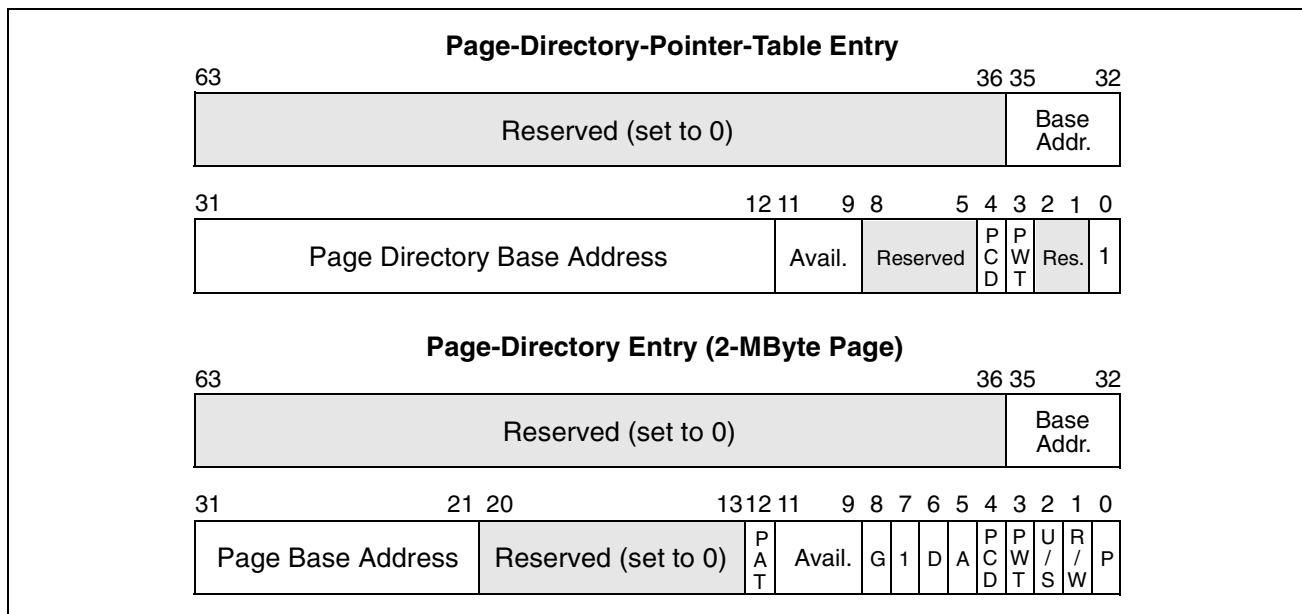


Figure 3-21. Format of Page-Directory-Pointer-Table and Page-Directory Entries for 2-MByte Pages with PAE Enabled

The base physical address in an entry specifies the following, depending on the type of entry:

- Page-directory-pointer-table entry—the physical address of the first byte of a 4-KByte page directory.
- Page-directory entry—the physical address of the first byte of a 4-KByte page table or a 2-MByte page.
- Page-table entry—the physical address of the first byte of a 4-KByte page.

For all table entries (except for page-directory entries that point to 2-MByte pages), the bits in the page base address are interpreted as the 24 most-significant bits of a 36-bit physical address, which forces page tables and pages to be aligned on 4-KByte boundaries. When a page-directory entry points to a 2-MByte page, the base address is interpreted as the 15 most-significant bits of a 36-bit physical address, which forces pages to be aligned on 2-MByte boundaries.

The present flag (bit 0) in all page-directory-pointer-table entries must be set to 1 anytime extended physical addressing mode is enabled; that is, whenever the PAE flag (bit 5 in register CR4) and the PG flag (bit 31 in register CR0) are set. If the P flag is not set in all 4 page-directory-pointer-table entries in the page-directory-pointer table when extended physical addressing is enabled, a general-protection exception (#GP) is generated.

The page size (PS) flag (bit 7) in a page-directory entry determines if the entry points to a page table or a 2-MByte page. When this flag is clear, the entry points to a page table; when the flag is set, the entry points to a 2-MByte page. This flag allows 4-KByte and 2-MByte pages to be mixed within one set of paging tables.

Access (A) and dirty (D) flags (bits 5 and 6) are provided for table entries that point to pages.

Bits 9, 10, and 11 in all the table entries for the physical address extension are available for use by software. (When the present flag is clear, bits 1 through 63 are available to software.) All bits in Figure 3-14 that are marked reserved or 0 should be set to 0 by software and not accessed by software. When the PSE and/or PAE flags in control register CR4 are set, the processor generates a page fault (#PF) if reserved bits in page-directory and page-table entries are not set to 0, and it generates a general-protection exception (#GP) if reserved bits in a page-directory-pointer-table entry are not set to 0.

3.9. 36-BIT PHYSICAL ADDRESSING USING THE PSE-36 PAGING MECHANISM

The PSE-36 paging mechanism provides an alternate method (from the PAE mechanism) of extending physical memory addressing to 36 bits. This mechanism uses the page size extension (PSE) mode and a modified page-directory table to map 4-MByte pages into a 64-Gbyte physical address space. As with the PAE mechanism, the processor provides 4 additional address line pins to accommodate the additional address bits.

The PSE-36 mechanism was introduced into the IA-32 architecture with the Pentium III processors. The availability of this feature is indicated with the PSE-36 feature bit (bit 17 of the EDX register when the CUID instruction is executed with a source operand of 1).

As is shown in Table 3-3, the following flags must be set or cleared to enable the PSE-36 paging mechanism:

- PSE-36 CUID feature flag—When set, it indicates the availability of the PSE-36 paging mechanism on the IA-32 processor on which the CUID instruction is executed.
- PG flag (bit 31) in register CR0—Set to 1 to enable paging.
- PAE flag (bit 5) in control register CR4—Clear to 0 to disable the PAE paging mechanism.
- PSE flag (bit 4) in control register CR4 and the PS flag in PDE— Set to 1 to enable the page size extension for 4-Mbyte pages.
- Or the PSE flag (bit 4) in control register CR4— Set to 1 and the PS flag (bit 7) in PDE— Set to 0 to enable 4-KByte pages with 32-bit addressing (below 4 GBytes).

Figure 3-22 shows how the expanded page directory entry can be used to map a 32-bit linear address to a 36-bit physical address. Here, the linear address is divided into two sections:

- Page directory entry—Bits 22 through 35 provide an offset to an entry in the page directory. The selected entry provides the 14 most significant bits of a 36-bit address, which locates the base physical address of a 4-MByte page.
- Page offset—Bits 0 through 21 provides an offset to a physical address in the page.

This paging method can be used to map up to 1024 pages into a 64-GByte physical address space.

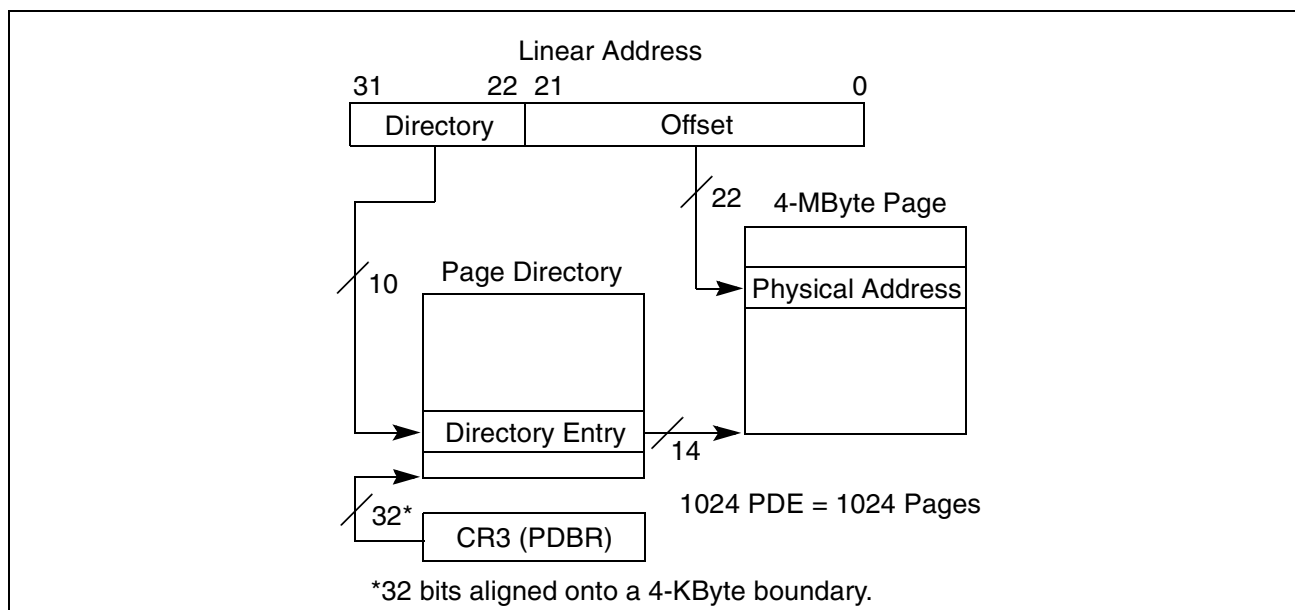


Figure 3-22. Linear Address Translation (4-MByte Pages)

Figure 3-23 shows the format for the page-directory entries when 4-MByte pages and 36-bit physical addresses are being used. Section 3.7.6., “Page-Directory and Page-Table Entries” describes the functions of the flags and fields in bits 0 through 11.

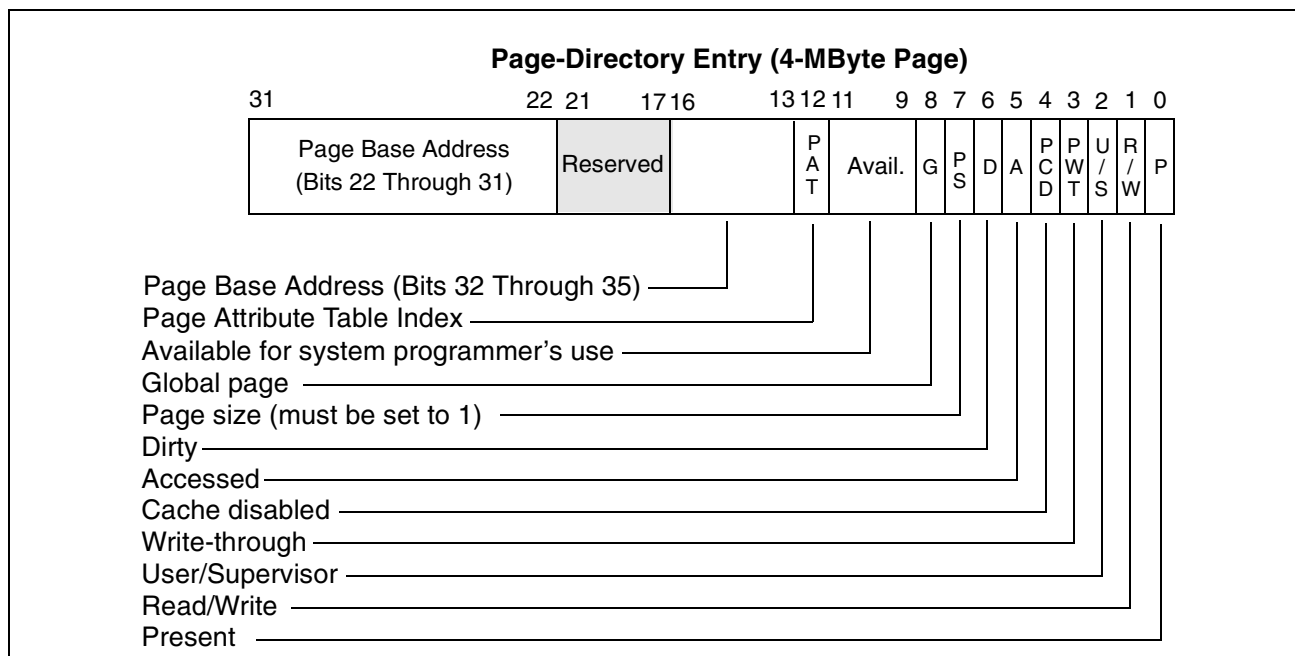


Figure 3-23. Format of Page-Directory Entries for 4-MByte Pages and 36-Bit Physical Addresses

3.10. MAPPING SEGMENTS TO PAGES

The segmentation and paging mechanisms provide in the IA-32 architecture support a wide variety of approaches to memory management. When segmentation and paging is combined, segments can be mapped to pages in several ways. To implement a flat (unsegmented) addressing environment, for example, all the code, data, and stack modules can be mapped to one or more large segments (up to 4-GBytes) that share same range of linear addresses (see Figure 3-2). Here, segments are essentially invisible to applications and the operating-system or executive. If paging is used, the paging mechanism can map a single linear address space (contained in a single segment) into virtual memory. Or, each program (or task) can have its own large linear address space (contained in its own segment), which is mapped into virtual memory through its own page directory and set of page tables.

Segments can be smaller than the size of a page. If one of these segments is placed in a page which is not shared with another segment, the extra memory is wasted. For example, a small data structure, such as a 1-byte semaphore, occupies 4K bytes if it is placed in a page by itself. If many semaphores are used, it is more efficient to pack them into a single page.

The IA-32 architecture does not enforce correspondence between the boundaries of pages and segments. A page can contain the end of one segment and the beginning of another. Likewise, a segment can contain the end of one page and the beginning of another.

Memory-management software may be simpler and more efficient if it enforces some alignment between page and segment boundaries. For example, if a segment which can fit in one page is placed in two pages, there may be twice as much paging overhead to support access to that segment.

One approach to combining paging and segmentation that simplifies memory-management software is to give each segment its own page table, as shown in Figure 3-24. This convention gives the segment a single entry in the page directory which provides the access control information for paging the entire segment.

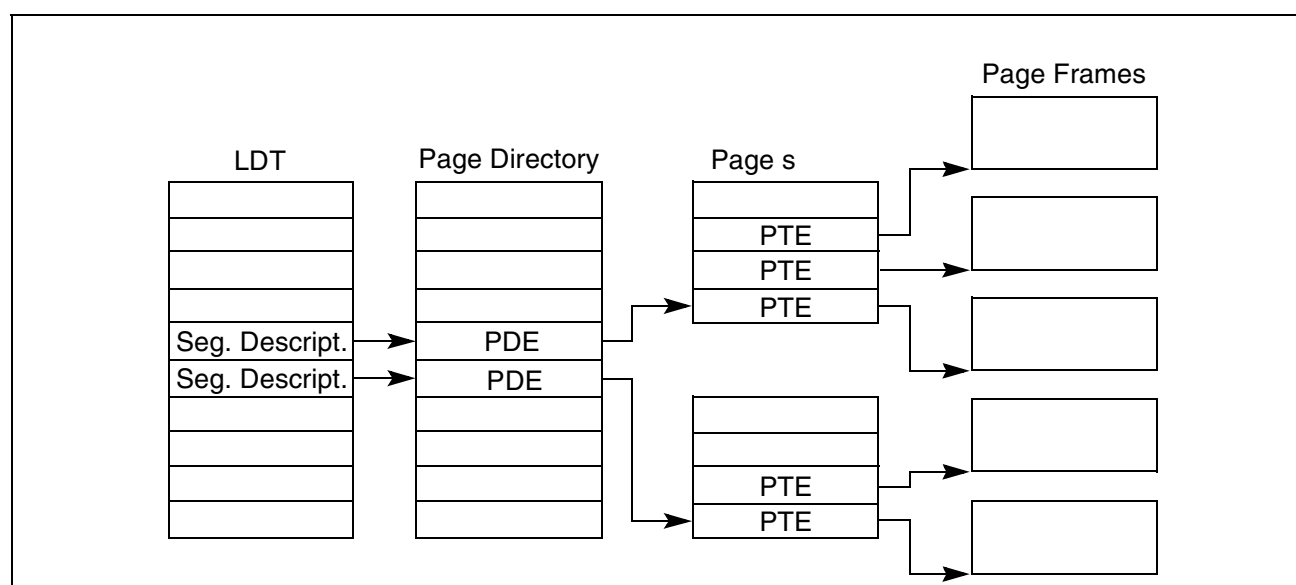


Figure 3-24. Memory Management Convention That Assigns a Page Table to Each Segment

3.11. TRANSLATION LOOKASIDE BUFFERS (TLBS)

The processor stores the most recently used page-directory and page-table entries in on-chip caches called translation lookaside buffers or TLBs. The P6 family and Pentium processors have separate TLBs for the data and instruction caches. Also, the P6 family processors maintain separate TLBs for 4-KByte and 4-MByte page sizes. The CPUID instruction can be used to determine the sizes of the TLBs provided in the P6 family and Pentium processors.

Most paging is performed using the contents of the TLBs. Bus cycles to the page directory and page tables in memory are performed only when the TLBs do not contain the translation information for a requested page.

The TLBs are inaccessible to application programs and tasks (privilege level greater than 0); that is, they cannot invalidate TLBs. Only, operating system or executive procedures running at privilege level of 0 can invalidate TLBs or selected TLB entries. Whenever a page-directory or page-table entry is changed (including when the present flag is set to zero), the operating-system must immediately invalidate the corresponding entry in the TLB so that it can be updated the next time the entry is referenced.

All of the (non-global) TLBs are automatically invalidated any time the CR3 register is loaded (unless the G flag for a page or page-table entry is set, as describe later in this section). The CR3 register can be loaded in either of two ways:

- Explicitly, using the MOV instruction, for example:

```
MOV CR3, EAX
```

where the EAX register contains an appropriate page-directory base address.

- Implicitly by executing a task switch, which automatically changes the contents of the CR3 register.

The INVLPG instruction is provided to invalidate a specific page-table entry in the TLB. Normally, this instruction invalidates only an individual TLB entry; however, in some cases, it may invalidate more than the selected entry and may even invalidate all of the TLBs. This instruction ignores the setting of the G flag in a page-directory or page-table entry (see following paragraph).

(Introduced in the Pentium Pro processor.) The page global enable (PGE) flag in register CR4 and the global (G) flag of a page-directory or page-table entry (bit 8) can be used to prevent frequently used pages from being automatically invalidated in the TLBs on a task switch or a load of register CR3. (See Section 3.7.6., “Page-Directory and Page-Table Entries”, for more information about the global flag.) When the processor loads a page-directory or page-table entry for a global page into a TLB, the entry will remain in the TLB indefinitely. The only ways to deterministically invalidate global page entries are as follows:

- Clear the PGE flag and then invalidate the TLBs.

- Execute the INVLPG instruction to invalidate individual page-directory or page-table entries in the TLBs.

For additional information about invalidation of the TLBs, see Section 10.9., “Invalidating the Translation Lookaside Buffers (TLBs)”.

Protection

CHAPTER 4 PROTECTION

In protected mode, the IA-32 architecture provides a protection mechanism that operates at both the segment level and the page level. This protection mechanism provides the ability to limit access to certain segments or pages based on privilege levels (four privilege levels for segments and two privilege levels for pages). For example, critical operating-system code and data can be protected by placing them in more privileged segments than those that contain applications code. The processor's protection mechanism will then prevent application code from accessing the operating-system code and data in any but a controlled, defined manner.

Segment and page protection can be used at all stages of software development to assist in localizing and detecting design problems and bugs. It can also be incorporated into end-products to offer added robustness to operating systems, utilities software, and applications software.

When the protection mechanism is used, each memory reference is checked to verify that it satisfies various protection checks. All checks are made before the memory cycle is started; any violation results in an exception. Because checks are performed in parallel with address translation, there is no performance penalty. The protection checks that are performed fall into the following categories:

- Limit checks.
- Type checks.
- Privilege level checks.
- Restriction of addressable domain.
- Restriction of procedure entry-points.
- Restriction of instruction set.

All protection violation results in an exception being generated. See Chapter 5, *Interrupt and Exception Handling*, for an explanation of the exception mechanism. This chapter describes the protection mechanism and the violations which lead to exceptions.

The following sections describe the protection mechanism available in protected mode. See Chapter 16, *8086 Emulation*, for information on protection in real-address and virtual-8086 mode.

4.1. ENABLING AND DISABLING SEGMENT AND PAGE PROTECTION

Setting the PE flag in register CR0 causes the processor to switch to protected mode, which in turn enables the segment-protection mechanism. Once in protected mode, there is no control bit for turning the protection mechanism on or off. The part of the segment-protection mechanism that is based on privilege levels can essentially be disabled while still in protected mode by assigning a privilege level of 0 (most privileged) to all segment selectors and segment descriptors. This action disables the privilege level protection barriers between segments, but other protection checks such as limit checking and type checking are still carried out.

Page-level protection is automatically enabled when paging is enabled (by setting the PG flag in register CR0). Here again there is no mode bit for turning off page-level protection once paging is enabled. However, page-level protection can be disabled by performing the following operations:

- Clear the WP flag in control register CR0.
- Set the read/write (R/W) and user/supervisor (U/S) flags for each page-directory and page-table entry.

This action makes each page a writable, user page, which in effect disables page-level protection.

4.2. FIELDS AND FLAGS USED FOR SEGMENT-LEVEL AND PAGE-LEVEL PROTECTION

The processor's protection mechanism uses the following fields and flags in the system data structures to control access to segments and pages:

- Descriptor type (S) flag—(Bit 12 in the second doubleword of a segment descriptor.) Determines if the segment descriptor is for a system segment or a code or data segment.
- Type field—(Bits 8 through 11 in the second doubleword of a segment descriptor.) Determines the type of code, data, or system segment.
- Limit field—(Bits 0 through 15 of the first doubleword and bits 16 through 19 of the second doubleword of a segment descriptor.) Determines the size of the segment, along with the G flag and E flag (for data segments).
- G flag—(Bit 23 in the second doubleword of a segment descriptor.) Determines the size of the segment, along with the limit field and E flag (for data segments).
- E flag—(Bit 10 in the second doubleword of a data-segment descriptor.) Determines the size of the segment, along with the limit field and G flag.
- Descriptor privilege level (DPL) field—(Bits 13 and 14 in the second doubleword of a segment descriptor.) Determines the privilege level of the segment.
- Requested privilege level (RPL) field. (Bits 0 and 1 of any segment selector.) Specifies the requested privilege level of a segment selector.

- Current privilege level (CPL) field. (Bits 0 and 1 of the CS segment register.) Indicates the privilege level of the currently executing program or procedure. The term current privilege level (CPL) refers to the setting of this field.
- User/supervisor (U/S) flag. (Bit 2 of a page-directory or page-table entry.) Determines the type of page: user or supervisor.
- Read/write (R/W) flag. (Bit 1 of a page-directory or page-table entry.) Determines the type of access allowed to a page: read only or read-write.

Figure 4-1 shows the location of the various fields and flags in the data, code, and system-segment descriptors; Figure 3-6 shows the location of the RPL (or CPL) field in a segment selector (or the CS register); and Figure 3-14 shows the location of the U/S and R/W flags in the page-directory and page-table entries.

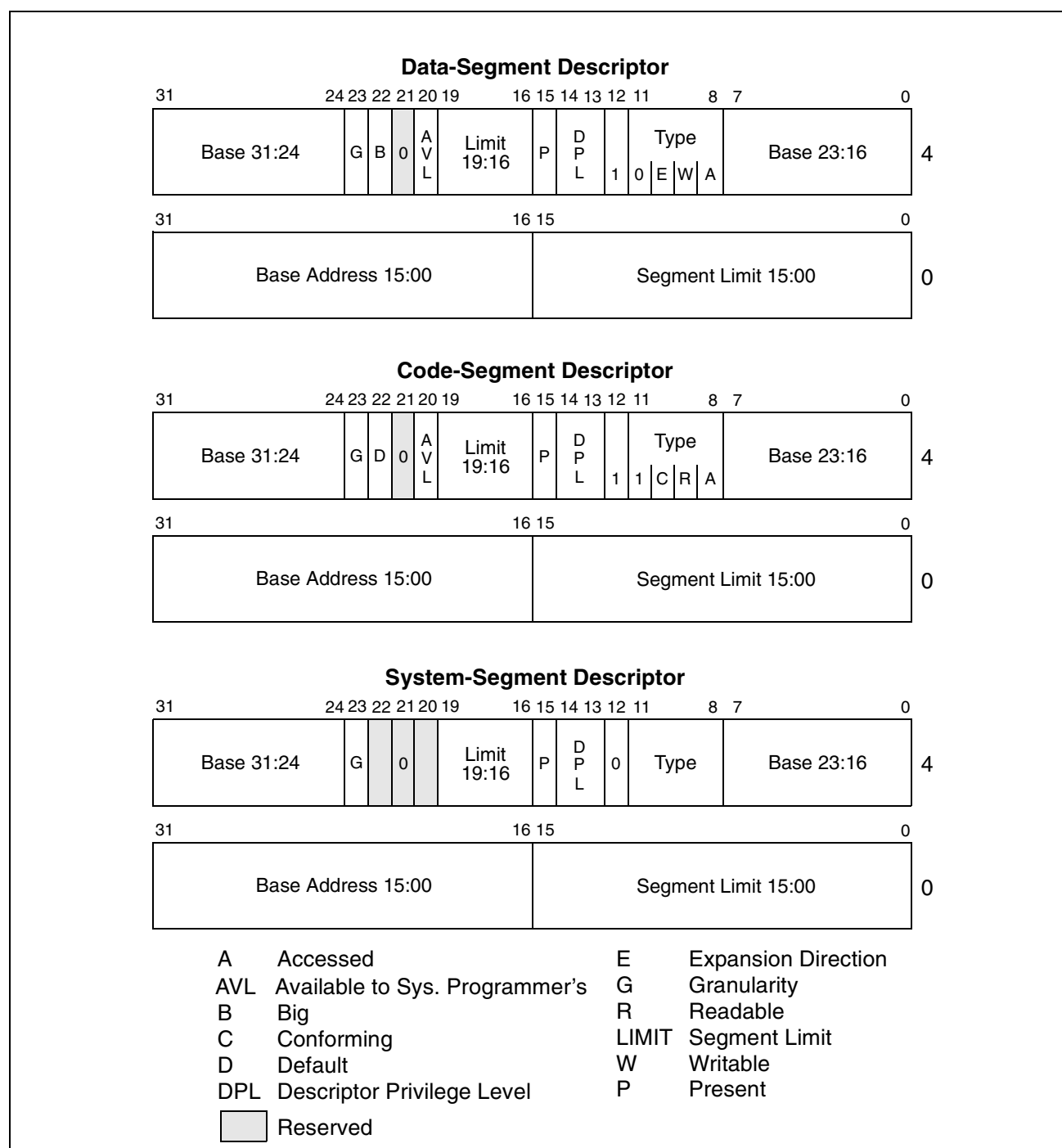


Figure 4-1. Descriptor Fields Used for Protection

Many different styles of protection schemes can be implemented with these fields and flags. When the operating system creates a descriptor, it places values in these fields and flags in keeping with the particular protection style chosen for an operating system or executive. Application program do not generally access or modify these fields and flags.

The following sections describe how the processor uses these fields and flags to perform the various categories of checks described in the introduction to this chapter.

4.3. LIMIT CHECKING

The limit field of a segment descriptor prevents programs or procedures from addressing memory locations outside the segment. The effective value of the limit depends on the setting of the G (granularity) flag (see Figure 4-1). For data segments, the limit also depends on the E (expansion direction) flag and the B (default stack pointer size and/or upper bound) flag. The E flag is one of the bits in the type field when the segment descriptor is for a data-segment type.

When the G flag is clear (byte granularity), the effective limit is the value of the 20-bit limit field in the segment descriptor. Here, the limit ranges from 0 to FFFFFH (1 MByte). When the G flag is set (4-KByte page granularity), the processor scales the value in the limit field by a factor of 2^{12} (4 KBytes). In this case, the effective limit ranges from FFFH (4 KBytes) to FFFFFFFFH (4 GBytes). Note that when scaling is used (G flag is set), the lower 12 bits of a segment offset (address) are not checked against the limit; for example, note that if the segment limit is 0, offsets 0 through FFFH are still valid.

For all types of segments except expand-down data segments, the effective limit is the last address that is allowed to be accessed in the segment, which is one less than the size, in bytes, of the segment. The processor causes a general-protection exception any time an attempt is made to access the following addresses in a segment:

- A byte at an offset greater than the effective limit
- A word at an offset greater than the (effective-limit – 1)
- A doubleword at an offset greater than the (effective-limit – 3)
- A quadword at an offset greater than the (effective-limit – 7)

For expand-down data segments, the segment limit has the same function but is interpreted differently. Here, the effective limit specifies the last address that is not allowed to be accessed within the segment; the range of valid offsets is from (effective-limit + 1) to FFFFFFFFH if the B flag is set and from (effective-limit + 1) to FFFFH if the B flag is clear. An expand-down segment has maximum size when the segment limit is 0.

Limit checking catches programming errors such as runaway code, runaway subscripts, and invalid pointer calculations. These errors are detected when they occur, so identification of the cause is easier. Without limit checking, these errors could overwrite code or data in another segment.

In addition to checking segment limits, the processor also checks descriptor table limits. The GDTR and IDTR registers contain 16-bit limit values that the processor uses to prevent programs from selecting a segment descriptors outside the respective descriptor tables. The LDTR and task registers contain 32-bit segment limit value (read from the segment descriptors for the current LDT and TSS, respectively). The processor uses these segment limits to prevent accesses beyond the bounds of the current LDT and TSS. See Section 3.5.1., “Segment Descriptor Tables”, for more information on the GDT and LDT limit fields; see Section 5.10., “Interrupt Descriptor Table (IDT)”, for more information on the IDT limit field; and see Section 6.2.3., “Task Register”, for more information on the TSS segment limit field.

4.4. TYPE CHECKING

Segment descriptors contain type information in two places:

- The S (descriptor type) flag.
- The type field.

The processor uses this information to detect programming errors that result in an attempt to use a segment or gate in an incorrect or unintended manner.

The S flag indicates whether a descriptor is a system type or a code or data type. The type field provides 4 additional bits for use in defining various types of code, data, and system descriptors. Table 3-1 shows the encoding of the type field for code and data descriptors; Table 3-2 shows the encoding of the field for system descriptors.

The processor examines type information at various times while operating on segment selectors and segment descriptors. The following list gives examples of typical operations where type checking is performed. This list is not exhaustive.

- **When a segment selector is loaded into a segment register.** Certain segment registers can contain only certain descriptor types, for example:
 - The CS register only can be loaded with a selector for a code segment.
 - Segment selectors for code segments that are not readable or for system segments cannot be loaded into data-segment registers (DS, ES, FS, and GS).
 - Only segment selectors of writable data segments can be loaded into the SS register.
- **When a segment selector is loaded into the LDTR or task register.**
 - The LDTR can only be loaded with a selector for an LDT.
 - The task register can only be loaded with a segment selector for a TSS.
- **When instructions access segments whose descriptors are already loaded into segment registers.** Certain segments can be used by instructions only in certain predefined ways, for example:
 - No instruction may write into an executable segment.
 - No instruction may write into a data segment if it is not writable.
 - No instruction may read an executable segment unless the readable flag is set.
- **When an instruction operand contains a segment selector.** Certain instructions can access segments or gates of only a particular type, for example:
 - A far CALL or far JMP instruction can only access a segment descriptor for a conforming code segment, nonconforming code segment, call gate, task gate, or TSS.
 - The LLDT instruction must reference a segment descriptor for an LDT.
 - The LTR instruction must reference a segment descriptor for a TSS.

- The LAR instruction must reference a segment or gate descriptor for an LDT, TSS, call gate, task gate, code segment, or data segment.
- The LSL instruction must reference a segment descriptor for a LDT, TSS, code segment, or data segment.
- IDT entries must be interrupt, trap, or task gates.
- **During certain internal operations.** For example:
 - On a far call or far jump (executed with a far CALL or far JMP instruction), the processor determines the type of control transfer to be carried out (call or jump to another code segment, a call or jump through a gate, or a task switch) by checking the type field in the segment (or gate) descriptor pointed to by the segment (or gate) selector given as an operand in the CALL or JMP instruction. If the descriptor type is for a code segment or call gate, a call or jump to another code segment is indicated; if the descriptor type is for a TSS or task gate, a task switch is indicated.
 - On a call or jump through a call gate (or on an interrupt- or exception-handler call through a trap or interrupt gate), the processor automatically checks that the segment descriptor being pointed to by the gate is for a code segment.
 - On a call or jump to a new task through a task gate (or on an interrupt- or exception-handler call to a new task through a task gate), the processor automatically checks that the segment descriptor being pointed to by the task gate is for a TSS.
 - On a call or jump to a new task by a direct reference to a TSS, the processor automatically checks that the segment descriptor being pointed to by the CALL or JMP instruction is for a TSS.
 - On return from a nested task (initiated by an IRET instruction), the processor checks that the previous task link field in the current TSS points to a TSS.

4.4.1. Null Segment Selector Checking

Attempting to load a null segment selector (see Section 3.4.1., “Segment Selectors”) into the CS or SS segment register generates a general-protection exception (#GP). A null segment selector can be loaded into the DS, ES, FS, or GS register, but any attempt to access a segment through one of these registers when it is loaded with a null segment selector results in a #GP exception being generated. Loading unused data-segment registers with a null segment selector is a useful method of detecting accesses to unused segment registers and/or preventing unwanted accesses to data segments.

4.5. PRIVILEGE LEVELS

The processor’s segment-protection mechanism recognizes 4 privilege levels, numbered from 0 to 3. The greater numbers mean lesser privileges. Figure 4-2 shows how these levels of privilege can be interpreted as rings of protection.

The center (reserved for the most privileged code, data, and stacks) is used for the segments containing the critical software, usually the kernel of an operating system. Outer rings are used for less critical software. (Systems that use only 2 of the 4 possible privilege levels should use levels 0 and 3.)

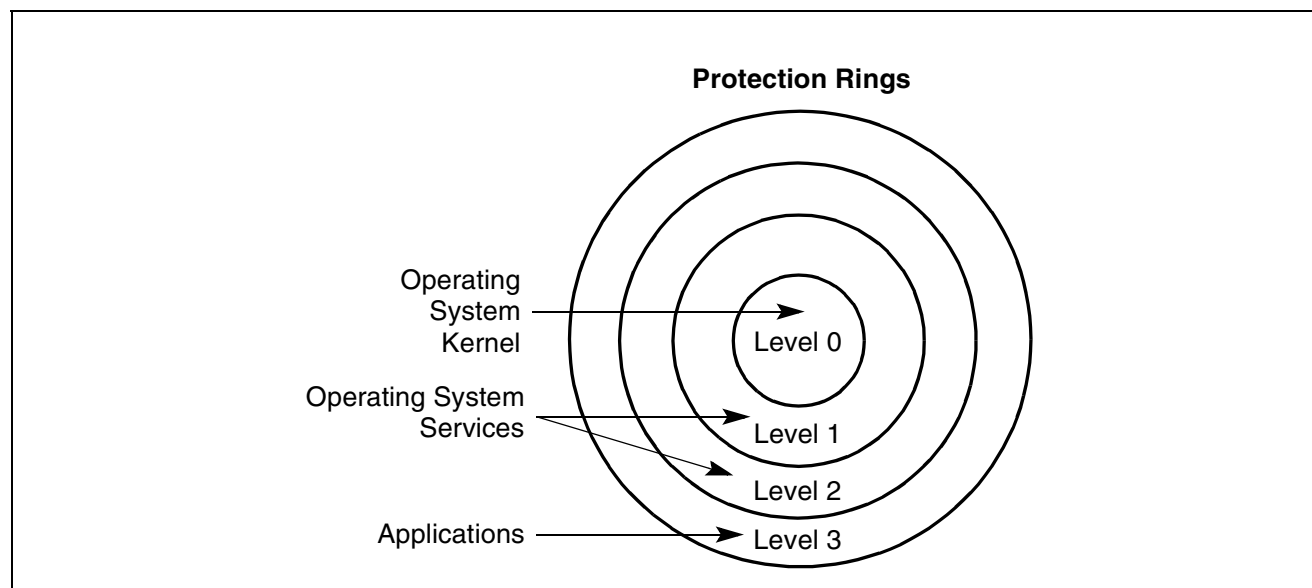


Figure 4-2. Protection Rings

The processor uses privilege levels to prevent a program or task operating at a lesser privilege level from accessing a segment with a greater privilege, except under controlled situations. When the processor detects a privilege level violation, it generates a general-protection exception (#GP).

To carry out privilege-level checks between code segments and data segments, the processor recognizes the following three types of privilege levels:

- **Current privilege level (CPL).** The CPL is the privilege level of the currently executing program or task. It is stored in bits 0 and 1 of the CS and SS segment registers. Normally, the CPL is equal to the privilege level of the code segment from which instructions are being fetched. The processor changes the CPL when program control is transferred to a code segment with a different privilege level. The CPL is treated slightly differently when accessing conforming code segments. Conforming code segments can be accessed from any privilege level that is equal to or numerically greater (less privileged) than the DPL of the conforming code segment. Also, the CPL is not changed when the processor accesses a conforming code segment that has a different privilege level than the CPL.

- **Descriptor privilege level (DPL).** The DPL is the privilege level of a segment or gate. It is stored in the DPL field of the segment or gate descriptor for the segment or gate. When the currently executing code segment attempts to access a segment or gate, the DPL of the segment or gate is compared to the CPL and RPL of the segment or gate selector (as described later in this section). The DPL is interpreted differently, depending on the type of segment or gate being accessed:
 - **Data segment.** The DPL indicates the numerically highest privilege level that a program or task can have to be allowed to access the segment. For example, if the DPL of a data segment is 1, only programs running at a CPL of 0 or 1 can access the segment.
 - **Nonconforming code segment (without using a call gate).** The DPL indicates the privilege level that a program or task must be at to access the segment. For example, if the DPL of a nonconforming code segment is 0, only programs running at a CPL of 0 can access the segment.
 - **Call gate.** The DPL indicates the numerically highest privilege level that the currently executing program or task can be at and still be able to access the call gate. (This is the same access rule as for a data segment.)
 - **Conforming code segment and nonconforming code segment accessed through a call gate.** The DPL indicates the numerically lowest privilege level that a program or task can have to be allowed to access the segment. For example, if the DPL of a conforming code segment is 2, programs running at a CPL of 0 or 1 cannot access the segment.
 - **TSS.** The DPL indicates the numerically highest privilege level that the currently executing program or task can be at and still be able to access the TSS. (This is the same access rule as for a data segment.)
- **Requested privilege level (RPL).** The RPL is an override privilege level that is assigned to segment selectors. It is stored in bits 0 and 1 of the segment selector. The processor checks the RPL along with the CPL to determine if access to a segment is allowed. Even if the program or task requesting access to a segment has sufficient privilege to access the segment, access is denied if the RPL is not of sufficient privilege level. That is, if the RPL of a segment selector is numerically greater than the CPL, the RPL overrides the CPL, and vice versa. The RPL can be used to insure that privileged code does not access a segment on behalf of an application program unless the program itself has access privileges for that segment. See Section 4.10.4., “Checking Caller Access Privileges (ARPL Instruction)” for a detailed description of the purpose and typical use of the RPL.

Privilege levels are checked when the segment selector of a segment descriptor is loaded into a segment register. The checks used for data access differ from those used for transfers of program control among code segments; therefore, the two kinds of accesses are considered separately in the following sections.

4.6. PRIVILEGE LEVEL CHECKING WHEN ACCESSING DATA SEGMENTS

To access operands in a data segment, the segment selector for the data segment must be loaded into the data-segment registers (DS, ES, FS, or GS) or into the stack-segment register (SS). (Segment registers can be loaded with the MOV, POP, LDS, LES, LFS, LGS, and LSS instructions.) Before the processor loads a segment selector into a segment register, it performs a privilege check (see Figure 4-3) by comparing the privilege levels of the currently running program or task (the CPL), the RPL of the segment selector, and the DPL of the segment's segment descriptor. The processor loads the segment selector into the segment register if the DPL is numerically greater than or equal to both the CPL and the RPL. Otherwise, a general-protection fault is generated and the segment register is not loaded.

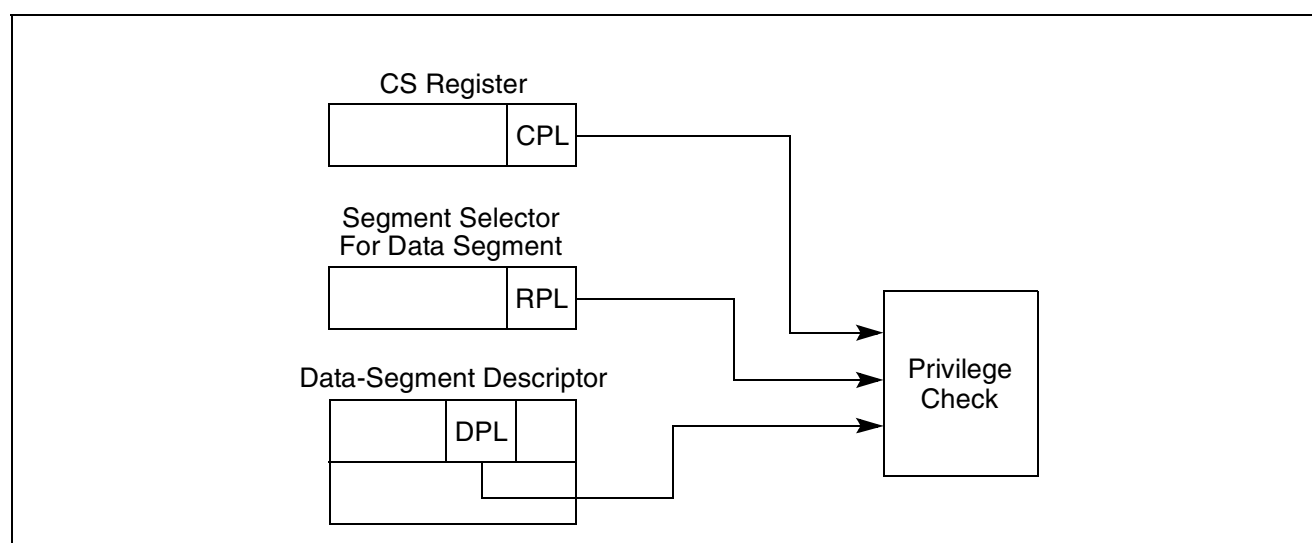


Figure 4-3. Privilege Check for Data Access

Figure 4-4 shows four procedures (located in codes segments A, B, C, and D), each running at different privilege levels and each attempting to access the same data segment.

- The procedure in code segment A is able to access data segment E using segment selector E1, because the CPL of code segment A and the RPL of segment selector E1 are equal to the DPL of data segment E.
- The procedure in code segment B is able to access data segment E using segment selector E2, because the CPL of code segment A and the RPL of segment selector E2 are both numerically lower than (more privileged) than the DPL of data segment E. A code segment B procedure can also access data segment E using segment selector E1.
- The procedure in code segment C is not able to access data segment E using segment selector E3 (dotted line), because the CPL of code segment C and the RPL of segment selector E3 are both numerically greater than (less privileged) than the DPL of data segment E. Even if a code segment C procedure were to use segment selector E1 or E2, such that the RPL would be acceptable, it still could not access data segment E because its CPL is not privileged enough.

- The procedure in code segment D should be able to access data segment E because code segment D's CPL is numerically less than the DPL of data segment E. However, the RPL of segment selector E3 (which the code segment D procedure is using to access data segment E) is numerically greater than the DPL of data segment E, so access is not allowed. If the code segment D procedure were to use segment selector E1 or E2 to access the data segment, access would be allowed.

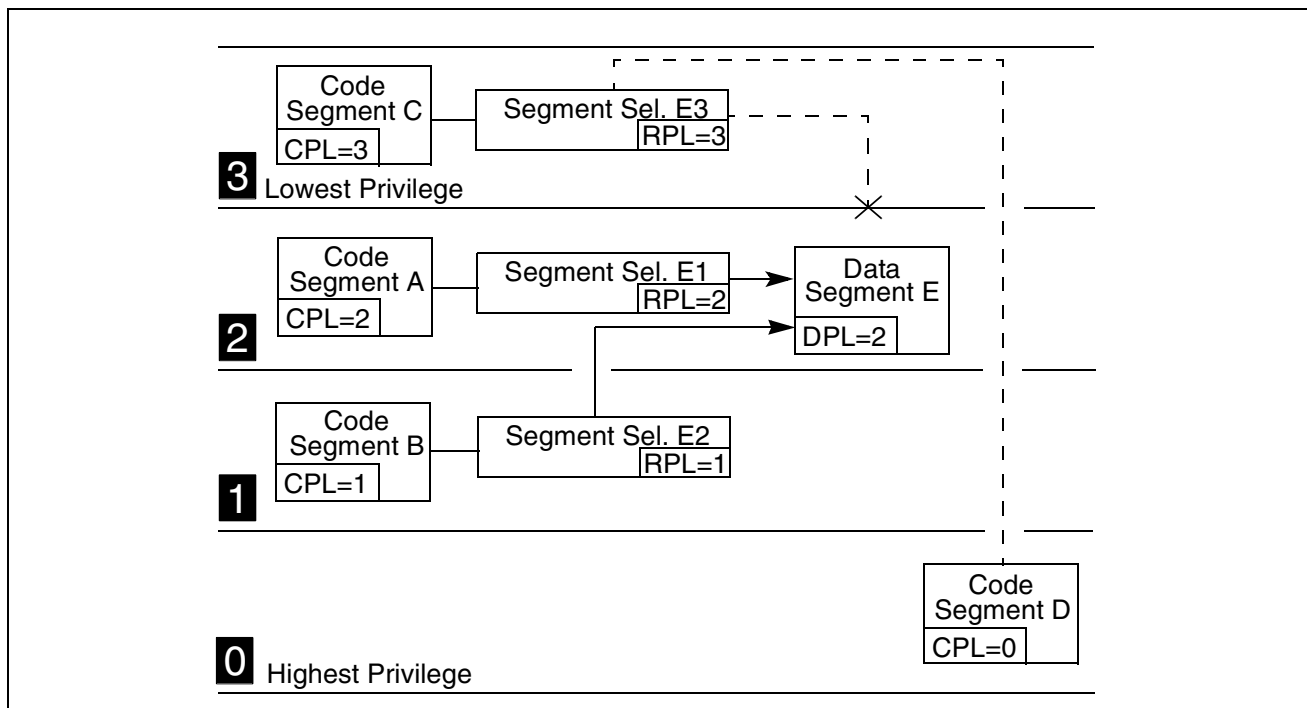


Figure 4-4. Examples of Accessing Data Segments From Various Privilege Levels

As demonstrated in the previous examples, the addressable domain of a program or task varies as its CPL changes. When the CPL is 0, data segments at all privilege levels are accessible; when the CPL is 1, only data segments at privilege levels 1 through 3 are accessible; when the CPL is 3, only data segments at privilege level 3 are accessible.

The RPL of a segment selector can always override the addressable domain of a program or task. When properly used, RPLs can prevent problems caused by accidental (or intentional) use of segment selectors for privileged data segments by less privileged programs or procedures.

It is important to note that the RPL of a segment selector for a data segment is under software control. For example, an application program running at a CPL of 3 can set the RPL for a data-segment selector to 0. With the RPL set to 0, only the CPL checks, not the RPL checks, will provide protection against deliberate, direct attempts to violate privilege-level security for the data segment. To prevent these types of privilege-level-check violations, a program or procedure can check access privileges whenever it receives a data-segment selector from another procedure (see Section 4.10.4., “Checking Caller Access Privileges (ARPL Instruction)”).

4.6.1. Accessing Data in Code Segments

In some instances it may be desirable to access data structures that are contained in a code segment. The following methods of accessing data in code segments are possible:

- Load a data-segment register with a segment selector for a nonconforming, readable, code segment.
- Load a data-segment register with a segment selector for a conforming, readable, code segment.
- Use a code-segment override prefix (CS) to read a readable, code segment whose selector is already loaded in the CS register.

The same rules for accessing data segments apply to method 1. Method 2 is always valid because the privilege level of a conforming code segment is effectively the same as the CPL, regardless of its DPL. Method 3 is always valid because the DPL of the code segment selected by the CS register is the same as the CPL.

4.7. PRIVILEGE LEVEL CHECKING WHEN LOADING THE SS REGISTER

Privilege level checking also occurs when the SS register is loaded with the segment selector for a stack segment. Here all privilege levels related to the stack segment must match the CPL; that is, the CPL, the RPL of the stack-segment selector, and the DPL of the stack-segment descriptor must be the same. If the RPL and DPL are not equal to the CPL, a general-protection exception (#GP) is generated.

4.8. PRIVILEGE LEVEL CHECKING WHEN TRANSFERRING PROGRAM CONTROL BETWEEN CODE SEGMENTS

To transfer program control from one code segment to another, the segment selector for the destination code segment must be loaded into the code-segment register (CS). As part of this loading process, the processor examines the segment descriptor for the destination code segment and performs various limit, type, and privilege checks. If these checks are successful, the CS register is loaded, program control is transferred to the new code segment, and program execution begins at the instruction pointed to by the EIP register.

Program control transfers are carried out with the JMP, CALL, RET, SYSENTER, SYSEXIT, INT *n*, and IRET instructions, as well as by the exception and interrupt mechanisms. Exceptions, interrupts, and the IRET instruction are special cases discussed in Chapter 5, *Interrupt and Exception Handling*. This chapter discusses only the JMP, CALL, RET, SYSENTER, and SYSEXIT instructions.

A JMP or CALL instruction can reference another code segment in any of four ways:

- The target operand contains the segment selector for the target code segment.
- The target operand points to a call-gate descriptor, which contains the segment selector for the target code segment.
- The target operand points to a TSS, which contains the segment selector for the target code segment.
- The target operand points to a task gate, which points to a TSS, which in turn contains the segment selector for the target code segment.

The following sections describe first two types of references. See Section 6.3., “Task Switching”, for information on transferring program control through a task gate and/or TSS.

The SYSENTER and SYSEXIT instructions are special instructions for making fast calls to and returns from operating system or executive procedures. These instructions are discussed briefly in Section 4.8.7., “Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions”.

4.8.1. Direct Calls or Jumps to Code Segments

The near forms of the JMP, CALL, and RET instructions transfer program control within the current code segment, so privilege-level checks are not performed. The far forms of the JMP, CALL, and RET instructions transfer control to other code segments, so the processor does perform privilege-level checks.

When transferring program control to another code segment without going through a call gate, the processor examines four kinds of privilege level and type information (see Figure 4-5):

- The CPL. (Here, the CPL is the privilege level of the calling code segment; that is, the code segment that contains the procedure that is making the call or jump.)

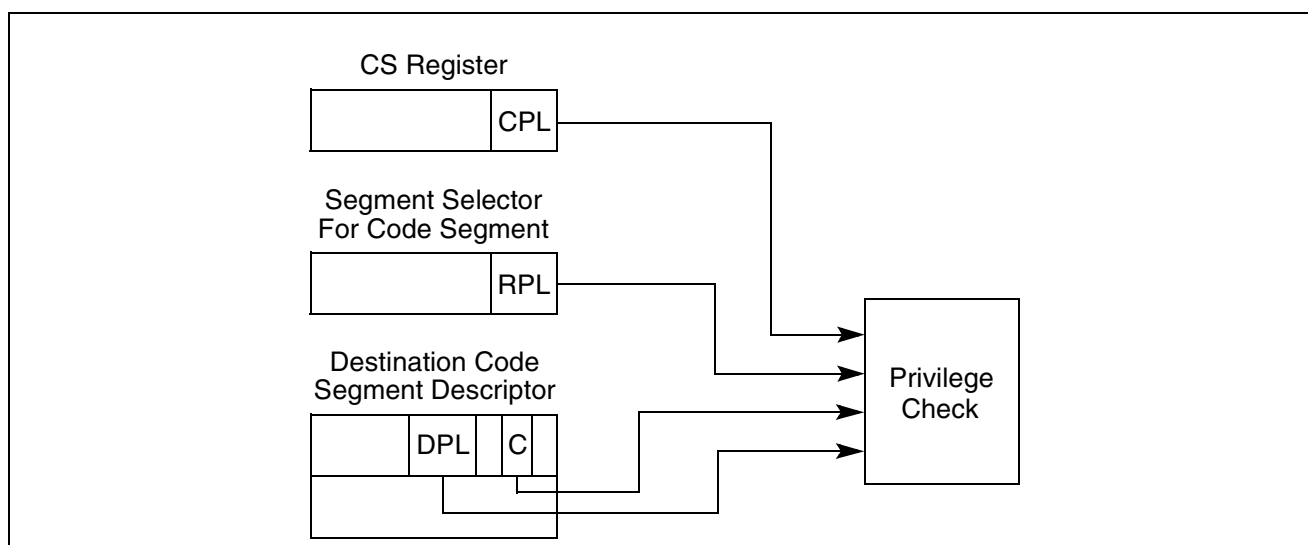


Figure 4-5. Privilege Check for Control Transfer Without Using a Gate

- The DPL of the segment descriptor for the destination code segment that contains the called procedure.
- The RPL of the segment selector of the destination code segment.
- The conforming (C) flag in the segment descriptor for the destination code segment, which determines whether the segment is a conforming (C flag is set) or nonconforming (C flag is clear) code segment. (See Section 3.4.3.1., “Code- and Data-Segment Descriptor Types”, for more information about this flag.)

The rules that the processor uses to check the CPL, RPL, and DPL depends on the setting of the C flag, as described in the following sections.

4.8.1.1. ACCESSING NONCONFORMING CODE SEGMENTS

When accessing nonconforming code segments, the CPL of the calling procedure must be equal to the DPL of the destination code segment; otherwise, the processor generates a general-protection exception (#GP).

For example, in Figure 4-6, code segment C is a nonconforming code segment. Therefore, a procedure in code segment A can call a procedure in code segment C (using segment selector C1), because they are at the same privilege level (the CPL of code segment A is equal to the DPL of code segment C). However, a procedure in code segment B cannot call a procedure in code segment C (using segment selector C2 or C1), because the two code segments are at different privilege levels.

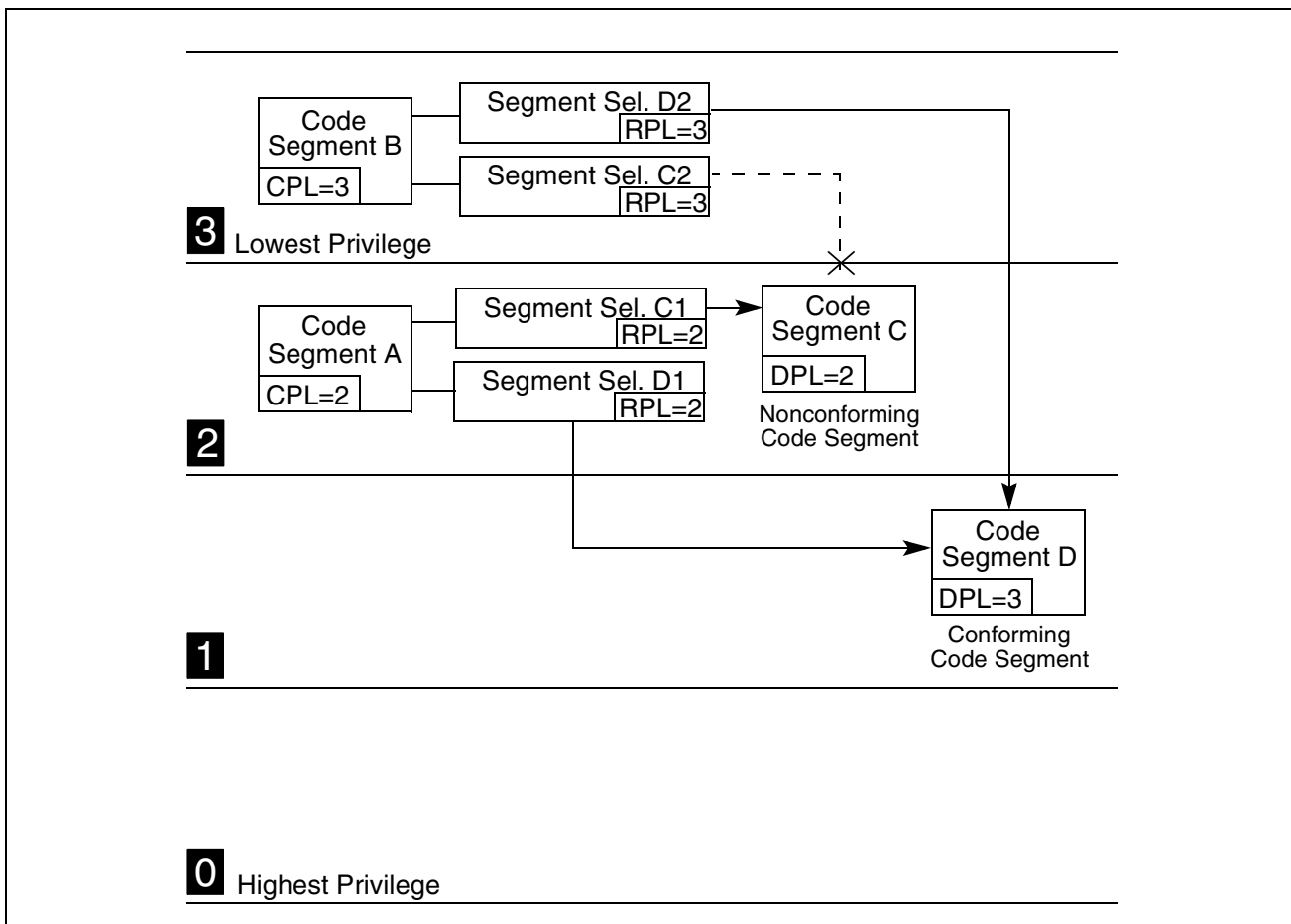


Figure 4-6. Examples of Accessing Conforming and Nonconforming Code Segments From Various Privilege Levels

The RPL of the segment selector that points to a nonconforming code segment has a limited effect on the privilege check. The RPL must be numerically less than or equal to the CPL of the calling procedure for a successful control transfer to occur. So, in the example in Figure 4-6, the RPLs of segment selectors C1 and C2 could legally be set to 0, 1, or 2, but not to 3.

When the segment selector of a nonconforming code segment is loaded into the CS register, the privilege level field is not changed; that is, it remains at the CPL (which is the privilege level of the calling procedure). This is true, even if the RPL of the segment selector is different from the CPL.

4.8.1.2. ACCESSING CONFORMING CODE SEGMENTS

When accessing conforming code segments, the CPL of the calling procedure may be numerically equal to or greater than (less privileged) the DPL of the destination code segment; the processor generates a general-protection exception (#GP) only if the CPL is less than the DPL. (The segment selector RPL for the destination code segment is not checked if the segment is a conforming code segment.)

In the example in Figure 4-6, code segment D is a conforming code segment. Therefore, calling procedures in both code segment A and B can access code segment D (using either segment selector D1 or D2, respectively), because they both have CPLs that are greater than or equal to the DPL of the conforming code segment. **For conforming code segments, the DPL represents the numerically lowest privilege level that a calling procedure may be at to successfully make a call to the code segment.**

(Note that segments selectors D1 and D2 are identical except for their respective RPLs. But since RPLs are not checked when accessing conforming code segments, the two segment selectors are essentially interchangeable.)

When program control is transferred to a conforming code segment, the CPL does not change, even if the DPL of the destination code segment is less than the CPL. This situation is the only one where the CPL may be different from the DPL of the current code segment. Also, since the CPL does not change, no stack switch occurs.

Conforming segments are used for code modules such as math libraries and exception handlers, which support applications but do not require access to protected system facilities. These modules are part of the operating system or executive software, but they can be executed at numerically higher privilege levels (less privileged levels). Keeping the CPL at the level of a calling code segment when switching to a conforming code segment prevents an application program from accessing nonconforming code segments while at the privilege level (DPL) of a conforming code segment and thus prevents it from accessing more privileged data.

Most code segments are nonconforming. For these segments, program control can be transferred only to code segments at the same level of privilege, unless the transfer is carried out through a call gate, as described in the following sections.

4.8.2. Gate Descriptors

To provide controlled access to code segments with different privilege levels, the processor provides special set of descriptors called gate descriptors. There are four kinds of gate descriptors:

- Call gates
- Trap gates
- Interrupt gates
- Task gates

Task gates are used for task switching and are discussed in Chapter 6, *Task Management*. Trap and interrupt gates are special kinds of call gates used for calling exception and interrupt handlers. They are described in Chapter 5, *Interrupt and Exception Handling*. This chapter is concerned only with call gates.

4.8.3. Call Gates

Call gates facilitate controlled transfers of program control between different privilege levels. They are typically used only in operating systems or executives that use the privilege-level protection mechanism. Call gates are also useful for transferring program control between 16-bit and 32-bit code segments, as described in Section 17.4., “Transferring Control Among Mixed-Size Code Segments”.

Figure 4-7 shows the format of a call-gate descriptor. A call-gate descriptor may reside in the GDT or in an LDT, but not in the interrupt descriptor table (IDT). It performs six functions:

- It specifies the code segment to be accessed.
- It defines an entry point for a procedure in the specified code segment.
- It specifies the privilege level required for a caller trying to access the procedure.

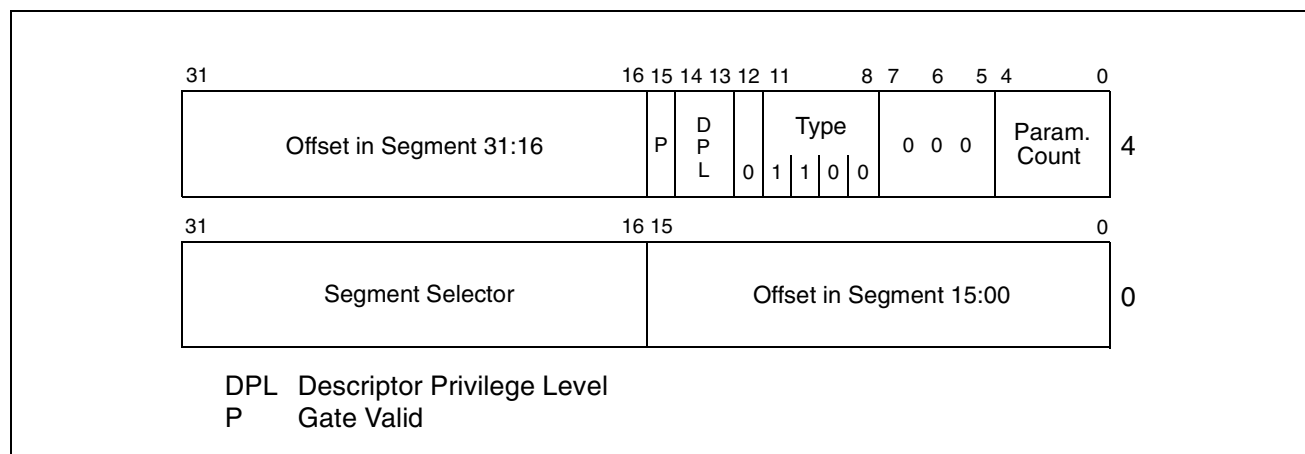


Figure 4-7. Call-Gate Descriptor

- If a stack switch occurs, it specifies the number of optional parameters to be copied between stacks.
- It defines the size of values to be pushed onto the target stack: 16-bit gates force 16-bit pushes and 32-bit gates force 32-bit pushes.
- It specifies whether the call-gate descriptor is valid.

The segment selector field in a call gate specifies the code segment to be accessed. The offset field specifies the entry point in the code segment. This entry point is generally to the first instruction of a specific procedure. The DPL field indicates the privilege level of the call gate, which in turn is the privilege level required to access the selected procedure through the gate. The P flag indicates whether the call-gate descriptor is valid. (The presence of the code segment to which the gate points is indicated by the P flag in the code segment’s descriptor.) The parameter count field indicates the number of parameters to copy from the calling procedures stack to the new stack if a stack switch occurs (see Section 4.8.5., “Stack Switching”). The parameter count specifies the number of words for 16-bit call gates and doublewords for 32-bit call gates.

Note that the P flag in a gate descriptor is normally always set to 1. If it is set to 0, a not present (#NP) exception is generated when a program attempts to access the descriptor. The operating system can use the P flag for special purposes. For example, it could be used to track the number of times the gate is used. Here, the P flag is initially set to 0 causing a trap to the not-present exception handler. The exception handler then increments a counter and sets the P flag to 1, so that on returning from the handler, the gate descriptor will be valid.

4.8.4. Accessing a Code Segment Through a Call Gate

To access a call gate, a far pointer to the gate is provided as a target operand in a CALL or JMP instruction. The segment selector from this pointer identifies the call gate (see Figure 4-8); the offset from the pointer is required, but not used or checked by the processor. (The offset can be set to any value.)

When the processor has accessed the call gate, it uses the segment selector from the call gate to locate the segment descriptor for the destination code segment. (This segment descriptor can be in the GDT or the LDT.) It then combines the base address from the code-segment descriptor with the offset from the call gate to form the linear address of the procedure entry point in the code segment.

As shown in Figure 4-9, four different privilege levels are used to check the validity of a program control transfer through a call gate:

- The CPL (current privilege level).
- The RPL (requestor's privilege level) of the call gate's selector.
- The DPL (descriptor privilege level) of the call gate descriptor.
- The DPL of the segment descriptor of the destination code segment.

The C flag (conforming) in the segment descriptor for the destination code segment is also checked.

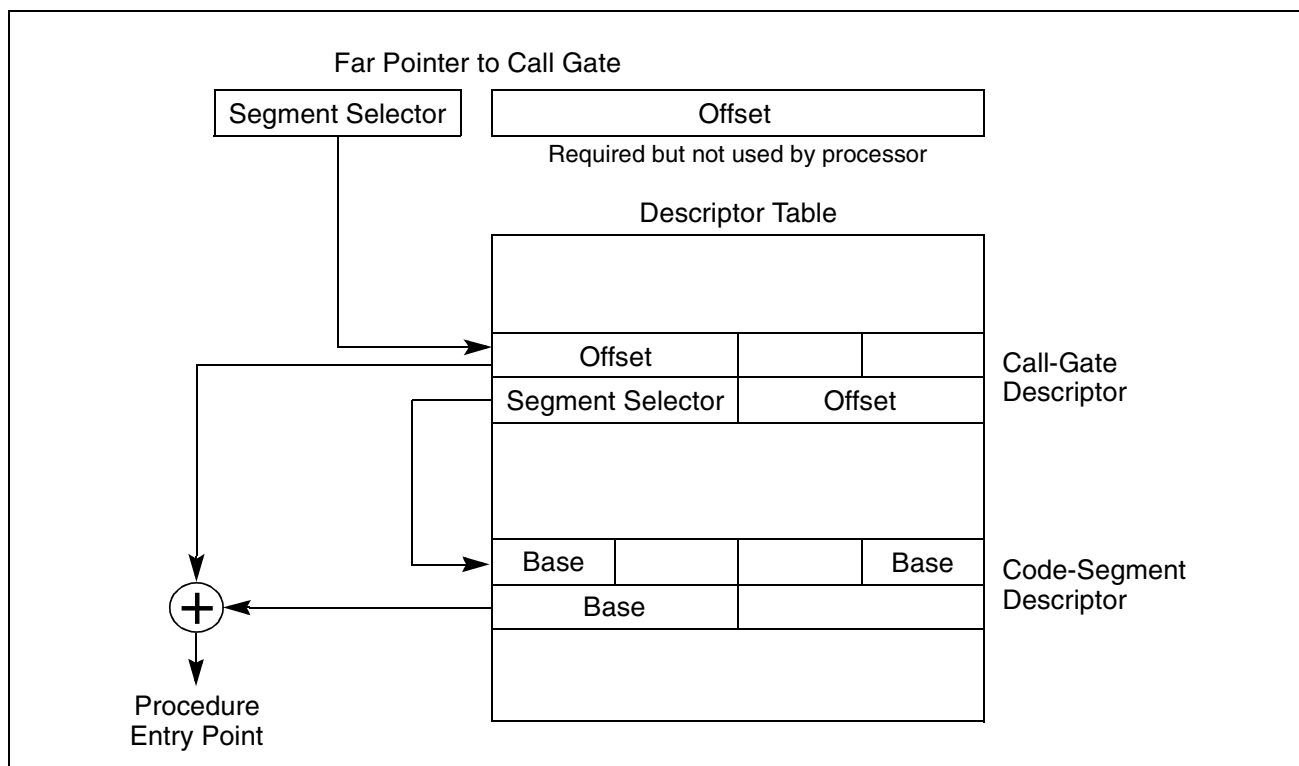


Figure 4-8. Call-Gate Mechanism

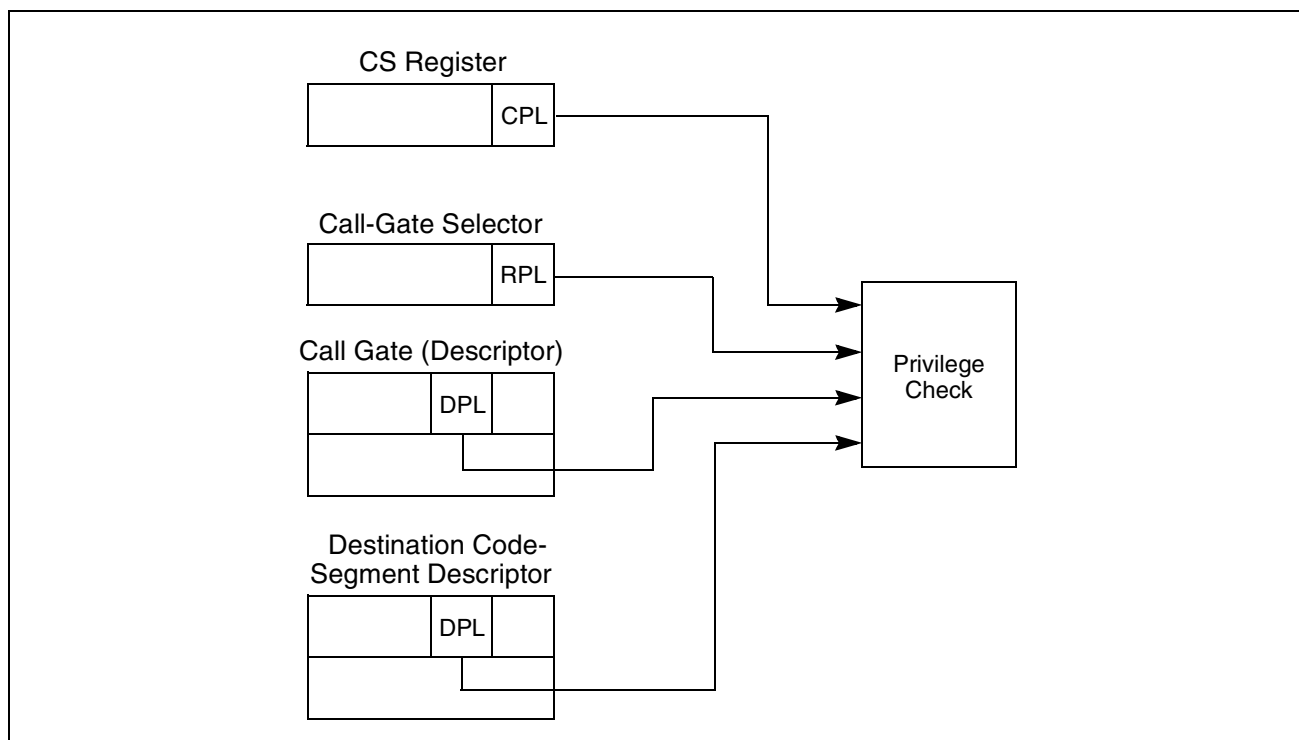


Figure 4-9. Privilege Check for Control Transfer with Call Gate

The privilege checking rules are different depending on whether the control transfer was initiated with a CALL or a JMP instruction, as shown in Table 4-1.

Table 4-1. Privilege Check Rules for Call Gates

Instruction	Privilege Check Rules
CALL	$CPL \leq \text{call gate DPL}; RPL \leq \text{call gate DPL}$ Destination conforming code segment $DPL \leq CPL$ Destination nonconforming code segment $DPL \leq CPL$
JMP	$CPL \leq \text{call gate DPL}; RPL \leq \text{call gate DPL}$ Destination conforming code segment $DPL \leq CPL$ Destination nonconforming code segment $DPL = CPL$

The DPL field of the call-gate descriptor specifies the numerically highest privilege level from which a calling procedure can access the call gate; that is, to access a call gate, the CPL of a calling procedure must be equal to or less than the DPL of the call gate. For example, in Figure 4-12, call gate A has a DPL of 3. So calling procedures at all CPLs (0 through 3) can access this call gate, which includes calling procedures in code segments A, B, and C. Call gate B has a DPL of 2, so only calling procedures at a CPL of 0, 1, or 2 can access call gate B, which includes calling procedures in code segments B and C. The dotted line shows that a calling procedure in code segment A cannot access call gate B.

The RPL of the segment selector to a call gate must satisfy the same test as the CPL of the calling procedure; that is, the RPL must be less than or equal to the DPL of the call gate. In the example in Figure 4-12, a calling procedure in code segment C can access call gate B using gate selector B2 or B1, but it could not use gate selector B3 to access call gate B.

If the privilege checks between the calling procedure and call gate are successful, the processor then checks the DPL of the code-segment descriptor against the CPL of the calling procedure. Here, the privilege check rules vary between CALL and JMP instructions. Only CALL instructions can use call gates to transfer program control to more privileged (numerically lower privilege level) nonconforming code segments; that is, to nonconforming code segments with a DPL less than the CPL. A JMP instruction can use a call gate only to transfer program control to a nonconforming code segment with a DPL equal to the CPL. CALL and JMP instruction can both transfer program control to a more privileged conforming code segment; that is, to a conforming code segment with a DPL less than or equal to the CPL.

If a call is made to a more privileged (numerically lower privilege level) nonconforming destination code segment, the CPL is lowered to the DPL of the destination code segment and a stack switch occurs (see Section 4.8.5., “Stack Switching”). If a call or jump is made to a more privileged conforming destination code segment, the CPL is not changed and no stack switch occurs.

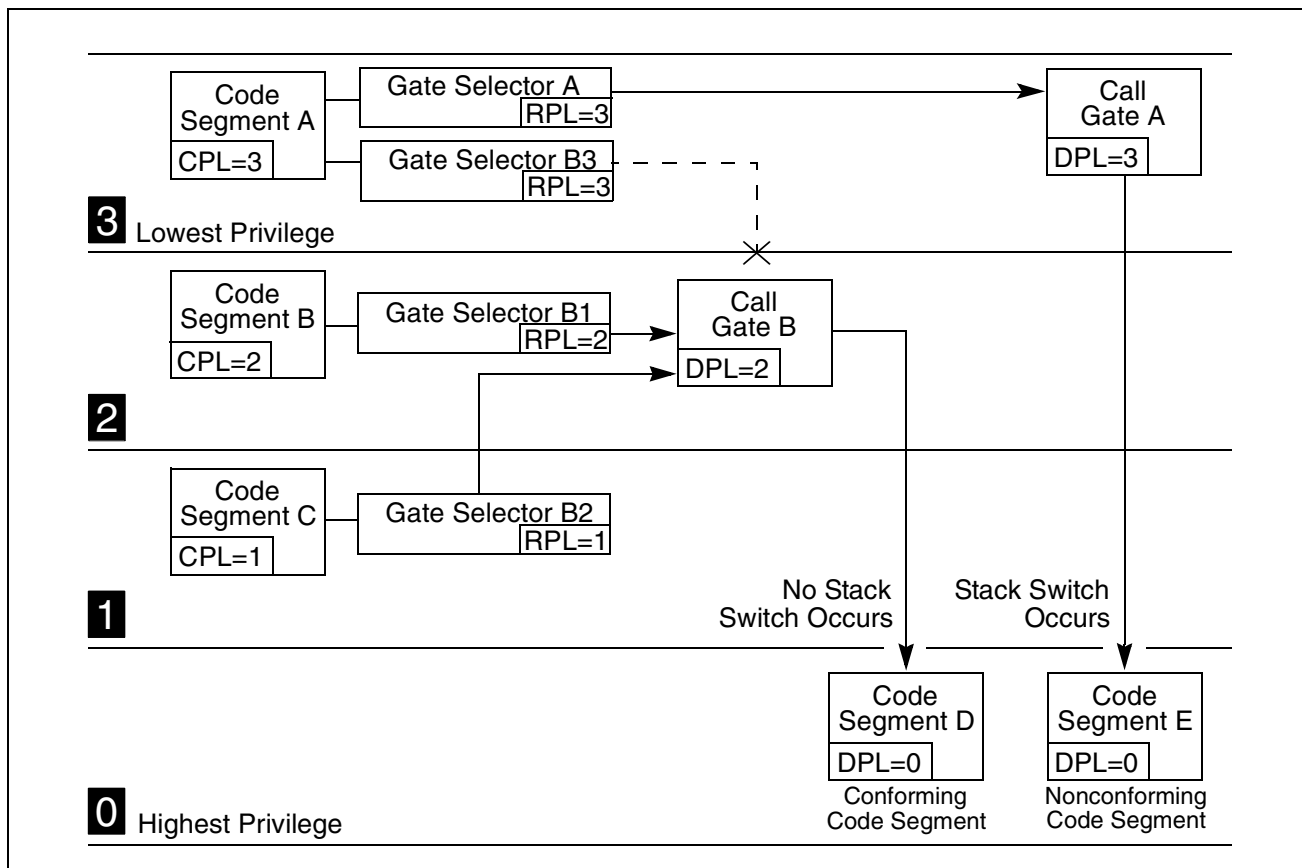


Figure 4-10. Example of Accessing Call Gates At Various Privilege Levels

Call gates allow a single code segment to have procedures that can be accessed at different privilege levels. For example, an operating system located in a code segment may have some services which are intended to be used by both the operating system and application software (such as procedures for handling character I/O). Call gates for these procedures can be set up that allow access at all privilege levels (0 through 3). More privileged call gates (with DPLs of 0 or 1) can then be set up for other operating system services that are intended to be used only by the operating system (such as procedures that initialize device drivers).

4.8.5. Stack Switching

Whenever a call gate is used to transfer program control to a more privileged nonconforming code segment (that is, when the DPL of the nonconforming destination code segment is less than the CPL), the processor automatically switches to the stack for the destination code segment's privilege level. This stack switching is carried out to prevent more privileged procedures from crashing due to insufficient stack space. It also prevents less privileged procedures from interfering (by accident or intent) with more privileged procedures through a shared stack.

Each task must define up to 4 stacks: one for applications code (running at privilege level 3) and one for each of the privilege levels 2, 1, and 0 that are used. (If only two privilege levels are used [3 and 0], then only two stacks must be defined.) Each of these stacks is located in a separate segment and is identified with a segment selector and an offset into the stack segment (a stack pointer).

The segment selector and stack pointer for the privilege level 3 stack is located in the SS and ESP registers, respectively, when privilege-level-3 code is being executed and is automatically stored on the called procedure's stack when a stack switch occurs.

Pointers to the privilege level 0, 1, and 2 stacks are stored in the TSS for the currently running task (see Figure 6-2). Each of these pointers consists of a segment selector and a stack pointer (loaded into the ESP register). These initial pointers are strictly read-only values. The processor does not change them while the task is running. They are used only to create new stacks when calls are made to more privileged levels (numerically lower privilege levels). These stacks are disposed of when a return is made from the called procedure. The next time the procedure is called, a new stack is created using the initial stack pointer. (The TSS does not specify a stack for privilege level 3 because the processor does not allow a transfer of program control from a procedure running at a CPL of 0, 1, or 2 to a procedure running at a CPL of 3, except on a return.)

The operating system is responsible for creating stacks and stack-segment descriptors for all the privilege levels to be used and for loading initial pointers for these stacks into the TSS. Each stack must be read/write accessible (as specified in the type field of its segment descriptor) and must contain enough space (as specified in the limit field) to hold the following items:

- The contents of the SS, ESP, CS, and EIP registers for the calling procedure.
- The parameters and temporary variables required by the called procedure.
- The EFLAGS register and error code, when implicit calls are made to an exception or interrupt handler.

The stack will need to require enough space to contain many frames of these items, because procedures often call other procedures, and an operating system may support nesting of multiple interrupts. Each stack should be large enough to allow for the worst case nesting scenario at its privilege level.

(If the operating system does not use the processor's multitasking mechanism, it still must create at least one TSS for this stack-related purpose.)

When a procedure call through a call gate results in a change in privilege level, the processor performs the following steps to switch stacks and begin execution of the called procedure at a new privilege level:

1. Uses the DPL of the destination code segment (the new CPL) to select a pointer to the new stack (segment selector and stack pointer) from the TSS.
2. Reads the segment selector and stack pointer for the stack to be switched to from the current TSS. Any limit violations detected while reading the stack-segment selector, stack pointer, or stack-segment descriptor cause an invalid TSS (#TS) exception to be generated.
3. Checks the stack-segment descriptor for the proper privileges and type and generates an invalid TSS (#TS) exception if violations are detected.

4. Temporarily saves the current values of the SS and ESP registers.
5. Loads the segment selector and stack pointer for the new stack in the SS and ESP registers.
6. Pushes the temporarily saved values for the SS and ESP registers (for the calling procedure) onto the new stack (see Figure 4-11).
7. Copies the number of parameter specified in the parameter count field of the call gate from the calling procedure's stack to the new stack. If the count is 0, no parameters are copied.
8. Pushes the return instruction pointer (the current contents of the CS and EIP registers) onto the new stack.
9. Loads the segment selector for the new code segment and the new instruction pointer from the call gate into the CS and EIP registers, respectively, and begins execution of the called procedure.

See the description of the CALL instruction in Chapter 3, *Instruction Set Reference*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*, for a detailed description of the privilege level checks and other protection checks that the processor performs on a far call through a call gate.

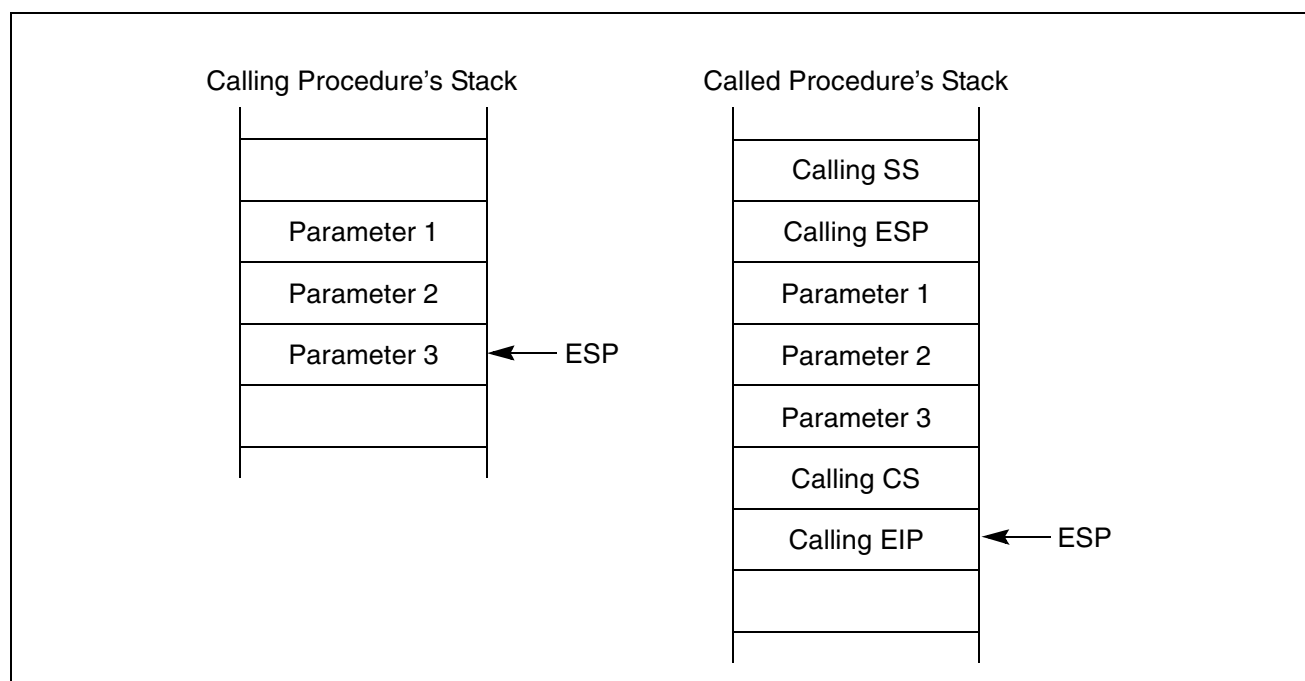


Figure 4-11. Stack Switching During an Interprivilege-Level Call

The parameter count field in a call gate specifies the number of data items (up to 31) that the processor should copy from the calling procedure's stack to the stack of the called procedure. If more than 31 data items need to be passed to the called procedure, one of the parameters can be a pointer to a data structure, or the saved contents of the SS and ESP registers may be used to access parameters in the old stack space. The size of the data items passed to the called procedure depends on the call gate size, as described in Section 4.8.3., "Call Gates".

4.8.6. Returning from a Called Procedure

The RET instruction can be used to perform a near return, a far return at the same privilege level, and a far return to a different privilege level. This instruction is intended to execute returns from procedures that were called with a CALL instruction. It does not support returns from a JMP instruction, because the JMP instruction does not save a return instruction pointer on the stack.

A near return only transfers program control within the current code segment; therefore, the processor performs only a limit check. When the processor pops the return instruction pointer from the stack into the EIP register, it checks that the pointer does not exceed the limit of the current code segment.

On a far return at the same privilege level, the processor pops both a segment selector for the code segment being returned to and a return instruction pointer from the stack. Under normal conditions, these pointers should be valid, because they were pushed on the stack by the CALL instruction. However, the processor performs privilege checks to detect situations where the current procedure might have altered the pointer or failed to maintain the stack properly.

A far return that requires a privilege-level change is only allowed when returning to a less privileged level (that is, the DPL of the return code segment is numerically greater than the CPL). The processor uses the RPL field from the CS register value saved for the calling procedure (see Figure 4-11) to determine if a return to a numerically higher privilege level is required. If the RPL is numerically greater (less privileged) than the CPL, a return across privilege levels occurs.

The processor performs the following steps when performing a far return to a calling procedure (see Figures 6-2 and 6-4 in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for an illustration of the stack contents prior to and after a return):

1. Checks the RPL field of the saved CS register value to determine if a privilege level change is required on the return.
2. Loads the CS and EIP registers with the values on the called procedure's stack. (Type and privilege level checks are performed on the code-segment descriptor and RPL of the code-segment selector.)
3. (If the RET instruction includes a parameter count operand and the return requires a privilege level change.) Adds the parameter count (in bytes obtained from the RET instruction) to the current ESP register value (after popping the CS and EIP values), to step past the parameters on the called procedure's stack. The resulting value in the ESP register points to the saved SS and ESP values for the calling procedure's stack. (Note that the byte count in the RET instruction must be chosen to match the parameter count in the call gate that the calling procedure referenced when it made the original call multiplied by the size of the parameters.)
4. (If the return requires a privilege level change.) Loads the SS and ESP registers with the saved SS and ESP values and switches back to the calling procedure's stack. The SS and ESP values for the called procedure's stack are discarded. Any limit violations detected while loading the stack-segment selector or stack pointer cause a general-protection exception (#GP) to be generated. The new stack-segment descriptor is also checked for type and privilege violations.

5. (If the RET instruction includes a parameter count operand.) Adds the parameter count (in bytes obtained from the RET instruction) to the current ESP register value, to step past the parameters on the calling procedure's stack. The resulting ESP value is not checked against the limit of the stack segment. If the ESP value is beyond the limit, that fact is not recognized until the next stack operation.
6. (If the return requires a privilege level change.) Checks the contents of the DS, ES, FS, and GS segment registers. If any of these registers refer to segments whose DPL is less than the new CPL (excluding conforming code segments), the segment register is loaded with a null segment selector.

See the description of the RET instruction in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*, for a detailed description of the privilege level checks and other protection checks that the processor performs on a far return.

4.8.7. Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions

The SYSENTER and SYSEXIT instructions were introduced into the IA-32 architecture in the Pentium II processors for the purpose of providing a fast (low overhead) mechanism for calling operating system or executive procedures. The SYSENTER instruction is intended for use by user code running at privilege level 3 to access operating system or executive procedures running at privilege level 0. The SYSEXIT procedure is intended for use by privilege level 0 operating system or executive procedures for fast returns to privilege level 3 user code. The SYSENTER instruction can be executed from privilege levels 3, 2, or 1; the SYSEXIT instruction can only be executed from privilege level 0.

The SYSENTER and SYSEXIT instructions are companion instructions, but they do not constitute a call/return pair because the SYSENTER instruction does not save any state information for use by the SYSEXIT instruction on a return.

The target instruction and stack pointer for these instructions are not specified through instruction operands. Instead, they are specified through parameters entered in several MSRs and general-purpose registers. For the SYSENTER instruction, the processor gets the privilege level 0 target instruction and stack pointer from the following sources:

- Target code segment—Reads it from the SYSENTER_CS_MSR.
- Target instruction—Reads it from the SYSENTER_EIP_MSR.
- Stack segment—Computes it adding 8 to the value in the SYSENTER_CS_MSR.
- Stack pointer—Reads it from the SYSENTER_ESP_MSR.

For the SYSEXIT instruction, the privilege level 3 target instruction and stack pointer are specified as follows:

- Target code segment—Computes it by adding 16 to the value in the SYSENTER_CS_MSR.
- Target instruction—Reads it from the EDX register.

- Stack segment—Computes it by adding 24 to the value in the SYSENTER_CS_MSR.
- Stack pointer—Reads it from the ECX register.

The SYSENTER and SYSEXIT instructions perform “fast” calls and returns because they force the processor into a predefined privilege level 0 state when a SYSENTER instruction is executed and into a predefined privilege level 3 state when a SYSEXIT instruction is executed. By forcing predefined and consistent processor states, the number of privilege checks ordinarily required to perform a far call to another privilege levels are greatly reduced. Also, by predefining the target context state in MSRs and general-purpose registers eliminates all memory accesses except when fetching the target code.

Any additional state that needs to be saved to allow a return to the calling procedure must be saved explicitly by the calling procedure or be predefined through programming conventions.

4.9. PRIVILEGED INSTRUCTIONS

Some of the system instructions (called “privileged instructions” are protected from use by application programs. The privileged instructions control system functions (such as the loading of system registers). They can be executed only when the CPL is 0 (most privileged). If one of these instructions is executed when the CPL is not 0, a general-protection exception (#GP) is generated. The following system instructions are privileged instructions:

- LGDT—Load GDT register.
- LLDT—Load LDT register.
- LTR—Load task register.
- LIDT—Load IDT register.
- MOV (control registers)—Load and store control registers.
- LMSW—Load machine status word.
- CLTS—Clear task-switched flag in register CR0.
- MOV (debug registers)—Load and store debug registers.
- INVD—Invalidate cache, without writeback.
- WBINVD—Invalidate cache, with writeback.
- INVLPG—Invalidate TLB entry.
- HLT—Halt processor.
- RDMSR—Read Model-Specific Registers.
- WRMSR—Write Model-Specific Registers.
- RDPMC—Read Performance-Monitoring Counter.
- RDTSC—Read Time-Stamp Counter.

Some of the privileged instructions are available only in the more recent families of IA-32 processors (see Section 18.9., “New Instructions In the Pentium and Later IA-32 Processors”).

The PCE and TSD flags in register CR4 (bits 4 and 2, respectively) enable the RDPMC and RDTSC instructions, respectively, to be executed at any CPL.

4.10. POINTER VALIDATION

When operating in protected mode, the processor validates all pointers to enforce protection between segments and maintain isolation between privilege levels. Pointer validation consists of the following checks:

1. Checking access rights to determine if the segment type is compatible with its use.
2. Checking read/write rights
3. Checking if the pointer offset exceeds the segment limit.
4. Checking if the supplier of the pointer is allowed to access the segment.
5. Checking the offset alignment.

The processor automatically performs first, second, and third checks during instruction execution. Software must explicitly request the fourth check by issuing an ARPL instruction. The fifth check (offset alignment) is performed automatically at privilege level 3 if alignment checking is turned on. Offset alignment does not affect isolation of privilege levels.

4.10.1. Checking Access Rights (LAR Instruction)

When the processor accesses a segment using a far pointer, it performs an access rights check on the segment descriptor pointed to by the far pointer. This check is performed to determine if type and privilege level (DPL) of the segment descriptor are compatible with the operation to be performed. For example, when making a far call in protected mode, the segment-descriptor type must be for a conforming or nonconforming code segment, a call gate, a task gate, or a TSS. Then, if the call is to a nonconforming code segment, the DPL of the code segment must be equal to the CPL, and the RPL of the code segment's segment selector must be less than or equal to the DPL. If type or privilege level are found to be incompatible, the appropriate exception is generated.

To prevent type incompatibility exceptions from being generated, software can check the access rights of a segment descriptor using the LAR (load access rights) instruction. The LAR instruction specifies the segment selector for the segment descriptor whose access rights are to be checked and a destination register. The instruction then performs the following operations:

1. Check that the segment selector is not null.
2. Checks that the segment selector points to a segment descriptor that is within the descriptor table limit (GDT or LDT).

3. Checks that the segment descriptor is a code, data, LDT, call gate, task gate, or TSS segment-descriptor type.
4. If the segment is not a conforming code segment, checks if the segment descriptor is visible at the CPL (that is, if the CPL and the RPL of the segment selector are less than or equal to the DPL).
5. If the privilege level and type checks pass, loads the second doubleword of the segment descriptor into the destination register (masked by the value 00FXFF00H, where X indicates that the corresponding 4 bits are undefined) and sets the ZF flag in the EFLAGS register. If the segment selector is not visible at the current privilege level or is an invalid type for the LAR instruction, the instruction does not modify the destination register and clears the ZF flag.

Once loaded in the destination register, software can preform additional checks on the access rights information.

4.10.2. Checking Read/Write Rights (VERR and VERW Instructions)

When the processor accesses any code or data segment it checks the read/write privileges assigned to the segment to verify that the intended read or write operation is allowed. Software can check read/write rights using the VERR (verify for reading) and VERW (verify for writing) instructions. Both these instructions specify the segment selector for the segment being checked. The instructions then perform the following operations:

1. Check that the segment selector is not null.
2. Checks that the segment selector points to a segment descriptor that is within the descriptor table limit (GDT or LDT).
3. Checks that the segment descriptor is a code or data-segment descriptor type.
4. If the segment is not a conforming code segment, checks if the segment descriptor is visible at the CPL (that is, if the CPL and the RPL of the segment selector are less than or equal to the DPL).
5. Checks that the segment is readable (for the VERR instruction) or writable (for the VERW) instruction.

The VERR instruction sets the ZF flag in the EFLAGS register if the segment is visible at the CPL and readable; the VERW sets the ZF flag if the segment is visible and writable. (Code segments are never writable.) The ZF flag is cleared if any of these checks fail.

4.10.3. Checking That the Pointer Offset Is Within Limits (LSL Instruction)

When the processor accesses any segment it performs a limit check to insure that the offset is within the limit of the segment. Software can perform this limit check using the LSL (load segment limit) instruction. Like the LAR instruction, the LSL instruction specifies the segment selector for the segment descriptor whose limit is to be checked and a destination register. The instruction then performs the following operations:

1. Check that the segment selector is not null.
2. Checks that the segment selector points to a segment descriptor that is within the descriptor table limit (GDT or LDT).
3. Checks that the segment descriptor is a code, data, LDT, or TSS segment-descriptor type.
4. If the segment is not a conforming code segment, checks if the segment descriptor is visible at the CPL (that is, if the CPL and the RPL of the segment selector less than or equal to the DPL).
5. If the privilege level and type checks pass, loads the unscrambled limit (the limit scaled according to the setting of the G flag in the segment descriptor) into the destination register and sets the ZF flag in the EFLAGS register. If the segment selector is not visible at the current privilege level or is an invalid type for the LSL instruction, the instruction does not modify the destination register and clears the ZF flag.

Once loaded in the destination register, software can compare the segment limit with the offset of a pointer.

4.10.4. Checking Caller Access Privileges (ARPL Instruction)

The requestor's privilege level (RPL) field of a segment selector is intended to carry the privilege level of a calling procedure (the calling procedure's CPL) to a called procedure. The called procedure then uses the RPL to determine if access to a segment is allowed. The RPL is said to "weaken" the privilege level of the called procedure to that of the RPL.

Operating-system procedures typically use the RPL to prevent less privileged application programs from accessing data located in more privileged segments. When an operating-system procedure (the called procedure) receives a segment selector from an application program (the calling procedure), it sets the segment selector's RPL to the privilege level of the calling procedure. Then, when the operating system uses the segment selector to access its associated segment, the processor performs privilege checks using the calling procedure's privilege level (stored in the RPL) rather than the numerically lower privilege level (the CPL) of the operating-system procedure. The RPL thus insures that the operating system does not access a segment on behalf of an application program unless that program itself has access to the segment.

Figure 4-12 shows an example of how the processor uses the RPL field. In this example, an application program (located in code segment A) possesses a segment selector (segment selector D1) that points to a privileged data structure (that is, a data structure located in a data segment D at privilege level 0).

The application program cannot access data segment D, because it does not have sufficient privilege, but the operating system (located in code segment C) can. So, in an attempt to access data segment D, the application program executes a call to the operating system and passes segment selector D1 to the operating system as a parameter on the stack. Before passing the segment selector, the (well behaved) application program sets the RPL of the segment selector to its current privilege level (which in this example is 3). If the operating system attempts to access data segment D using segment selector D1, the processor compares the CPL (which is now 0 following the call), the RPL of segment selector D1, and the DPL of data segment D (which is 0). Since the RPL is greater than the DPL, access to data segment D is denied. The processor's protection mechanism thus protects data segment D from access by the operating system, because application program's privilege level (represented by the RPL of segment selector B) is greater than the DPL of data segment D.

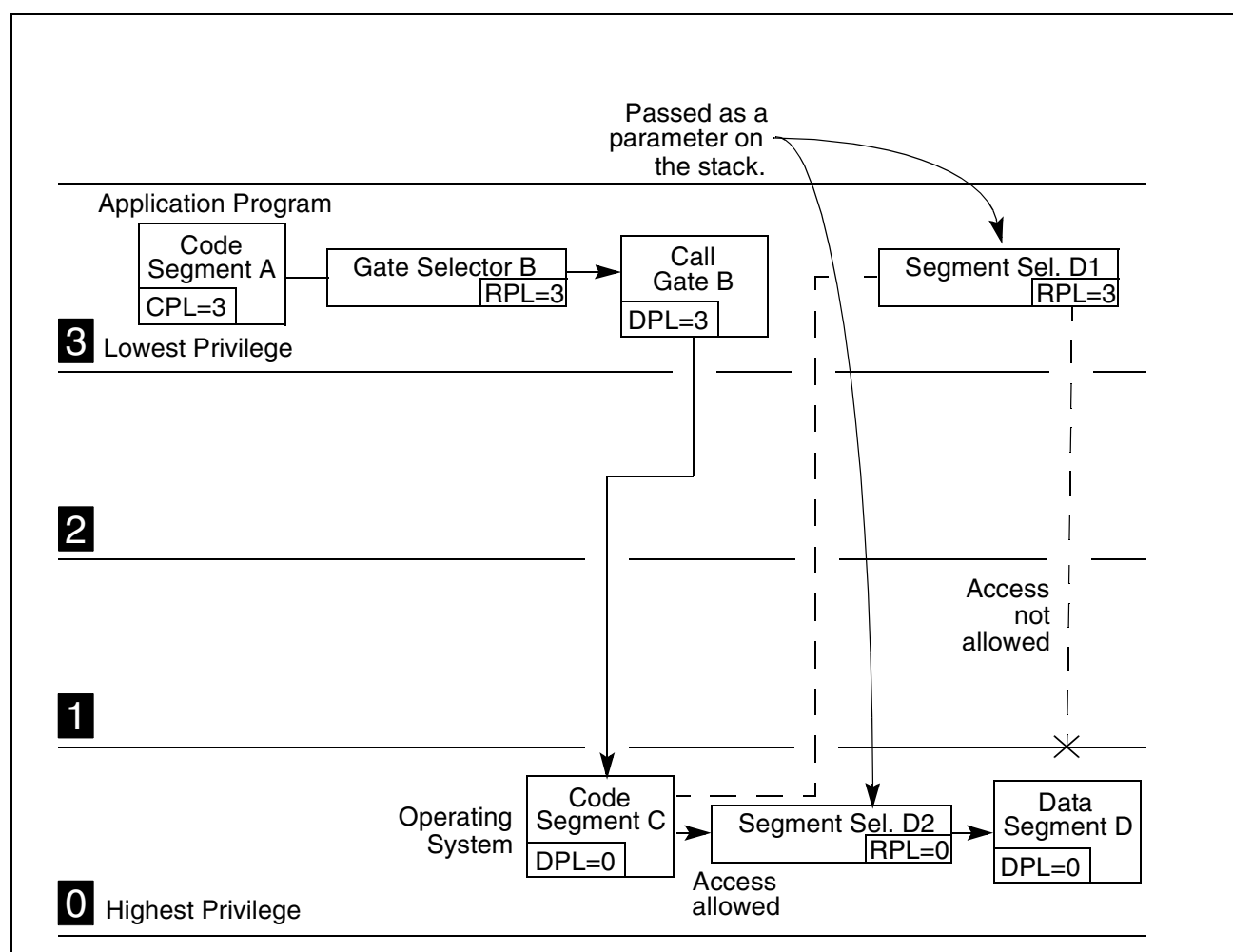


Figure 4-12. Use of RPL to Weaken Privilege Level of Called Procedure

Now assume that instead of setting the RPL of the segment selector to 3, the application program sets the RPL to 0 (segment selector D2). The operating system can now access data segment D, because its CPL and the RPL of segment selector D2 are both equal to the DPL of data segment D.

Because the application program is able to change the RPL of a segment selector to any value, it can potentially use a procedure operating at a numerically lower privilege level to access a protected data structure. This ability to lower the RPL of a segment selector breaches the processor's protection mechanism.

Because a called procedure cannot rely on the calling procedure to set the RPL correctly, operating-system procedures (executing at numerically lower privilege-levels) that receive segment selectors from numerically higher privilege-level procedures need to test the RPL of the segment selector to determine if it is at the appropriate level. The ARPL (adjust requested privilege level) instruction is provided for this purpose. This instruction adjusts the RPL of one segment selector to match that of another segment selector.

The example in Figure 4-12 demonstrates how the ARPL instruction is intended to be used. When the operating-system receives segment selector D2 from the application program, it uses the ARPL instruction to compare the RPL of the segment selector with the privilege level of the application program (represented by the code-segment selector pushed onto the stack). If the RPL is less than application program's privilege level, the ARPL instruction changes the RPL of the segment selector to match the privilege level of the application program (segment selector D1). Using this instruction thus prevents a procedure running at a numerically higher privilege level from accessing numerically lower privilege-level (more privileged) segments by lowering the RPL of a segment selector.

Note that the privilege level of the application program can be determined by reading the RPL field of the segment selector for the application-program's code segment. This segment selector is stored on the stack as part of the call to the operating system. The operating system can copy the segment selector from the stack into a register for use as an operand for the ARPL instruction.

4.10.5. Checking Alignment

When the CPL is 3, alignment of memory references can be checked by setting the AM flag in the CR0 register and the AC flag in the EFLAGS register. Unaligned memory references generate alignment exceptions (#AC). The processor does not generate alignment exceptions when operating at privilege level 0, 1, or 2. See Table 5-6 for a description of the alignment requirements when alignment checking is enabled.

4.11. PAGE-LEVEL PROTECTION

Page-level protection can be used alone or applied to segments. When page-level protection is used with the flat memory model, it allows supervisor code and data (the operating system or executive) to be protected from user code and data (application programs). It also allows pages containing code to be write protected. When the segment- and page-level protection are combined, page-level read/write protection allows more protection granularity within segments.

With page-level protection (as with segment-level protection) each memory reference is checked to verify that protection checks are satisfied. All checks are made before the memory cycle is started, and any violation prevents the cycle from starting and results in a page-fault exception being generated. Because checks are performed in parallel with address translation, there is no performance penalty.

The processor performs two page-level protection checks:

- Restriction of addressable domain (supervisor and user modes).
- Page type (read only or read/write).

Violations of either of these checks results in a page-fault exception being generated. See Chapter 5, “Interrupt 14—Page-Fault Exception (#PF)”, for an explanation of the page-fault exception mechanism. This chapter describes the protection violations which lead to page-fault exceptions.

4.11.1. Page-Protection Flags

Protection information for pages is contained in two flags in a page-directory or page-table entry (see Figure 3-14): the read/write flag (bit 1) and the user/supervisor flag (bit 2). The protection checks are applied to both first- and second-level page tables (that is, page directories and page tables).

4.11.2. Restricting Addressable Domain

The page-level protection mechanism allows restricting access to pages based on two privilege levels:

- Supervisor mode (U/S flag is 0)—(Most privileged) For the operating system or executive, other system software (such as device drivers), and protected system data (such as page tables).
- User mode (U/S flag is 1)—(Least privileged) For application code and data.

The segment privilege levels map to the page privilege levels as follows. If the processor is currently operating at a CPL of 0, 1, or 2, it is in supervisor mode; if it is operating at a CPL of 3, it is in user mode. When the processor is in supervisor mode, it can access all pages; when in user mode, it can access only user-level pages. (Note that the WP flag in control register CR0 modifies the supervisor permissions, as described in Section 4.11.3., “Page Type”.)

Note that to use the page-level protection mechanism, code and data segments must be set up for at least two segment-based privilege levels: level 0 for supervisor code and data segments and level 3 for user code and data segments. (In this model, the stacks are placed in the data segments.) To minimize the use of segments, a flat memory model can be used (see Section 3.2.1., “Basic Flat Model”).

Here, the user and supervisor code and data segments all begin at address zero in the linear address space and overlay each other. With this arrangement, operating-system code (running at the supervisor level) and application code (running at the user level) can execute as if there are no segments. Protection between operating-system and application code and data is provided by the processor's page-level protection mechanism.

4.11.3. Page Type

The page-level protection mechanism recognizes two page types:

- Read-only access (R/W flag is 0).
- Read/write access (R/W flag is 1).

When the processor is in supervisor mode and the WP flag in register CR0 is clear (its state following reset initialization), all pages are both readable and writable (write-protection is ignored). When the processor is in user mode, it can write only to user-mode pages that are read/write accessible. User-mode pages which are read/write or read-only are readable; supervisor-mode pages are neither readable nor writable from user mode. A page-fault exception is generated on any attempt to violate the protection rules.

The P6 family, Pentium, and Intel486 processors allow user-mode pages to be write-protected against supervisor-mode access. Setting the WP flag in register CR0 to 1 enables supervisor-mode sensitivity to user-mode, write-protected pages. This supervisor write-protect feature is useful for implementing a “copy-on-write” strategy used by some operating systems, such as UNIX*, for task creation (also called forking or spawning). When a new task is created, it is possible to copy the entire address space of the parent task. This gives the child task a complete, duplicate set of the parent's segments and pages. An alternative copy-on-write strategy saves memory space and time by mapping the child's segments and pages to the same segments and pages used by the parent task. A private copy of a page gets created only when one of the tasks writes to the page. By using the WP flag and marking the shared pages as read-only, the supervisor can detect an attempt to write to a user-level page, and can copy the page at that time.

4.11.4. Combining Protection of Both Levels of Page Tables

For any one page, the protection attributes of its page-directory entry (first-level page table) may differ from those of its page-table entry (second-level page table). The processor checks the protection for a page in both its page-directory and the page-table entries. Table 4-2 shows the protection provided by the possible combinations of protection attributes when the WP flag is clear.

4.11.5. Overrides to Page Protection

The following types of memory accesses are checked as if they are privilege-level 0 accesses, regardless of the CPL at which the processor is currently operating:

- Access to segment descriptors in the GDT, LDT, or IDT.
- Access to an inner-privilege-level stack during an inter-privilege-level call or a call to an exception or interrupt handler, when a change of privilege level occurs.

4.12. COMBINING PAGE AND SEGMENT PROTECTION

When paging is enabled, the processor evaluates segment protection first, then evaluates page protection. If the processor detects a protection violation at either the segment level or the page level, the memory access is not carried out and an exception is generated. If an exception is generated by segmentation, no paging exception is generated.

Page-level protections cannot be used to override segment-level protection. For example, a code segment is by definition not writable. If a code segment is paged, setting the R/W flag for the pages to read-write does not make the pages writable. Attempts to write into the pages will be blocked by segment-level protection checks.

Page-level protection can be used to enhance segment-level protection. For example, if a large read-write data segment is paged, the page-protection mechanism can be used to write-protect individual pages.

Table 4-2. Combined Page-Directory and Page-Table Protection

Page-Directory Entry		Page-Table Entry		Combined Effect	
Privilege	Access Type	Privilege	Access Type	Privilege	Access Type
User	Read-Only	User	Read-Only	User	Read-Only
User	Read-Only	User	Read-Write	User	Read-Only
User	Read-Write	User	Read-Only	User	Read-Only
User	Read-Write	User	Read-Write	User	Read/Write
User	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
User	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	User	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	User	Read-Write	Supervisor	Read/Write
Supervisor	Read-Only	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Only	Supervisor	Read-Write	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Only	Supervisor	Read/Write*
Supervisor	Read-Write	Supervisor	Read-Write	Supervisor	Read/Write

NOTE:

- * If the WP flag of CR0 is set, the access type is determined by the R/W flags of the page-directory and page-table entries.

5

Interrupt and Exception Handling

CHAPTER 5

INTERRUPT AND EXCEPTION HANDLING

This chapter describes the processor's interrupt and exception-handling mechanism, when operating in protected mode. Most of the information provided here also applies to the interrupt and exception mechanism used in real-address or virtual-8086 mode. See Chapter 15, *Debugging and Performance Monitoring*, for a description of the differences in the interrupt and exception mechanism for real-address and virtual-8086 mode.

5.1. INTERRUPT AND EXCEPTION OVERVIEW

Interrupts and **exceptions** are events that indicate that a condition exists somewhere in the system, the processor, or within the currently executing program or task that requires the attention of a processor. They typically result in a forced transfer of execution from the currently running program or task to a special software routine or task called an **interrupt handler** or an **exception handler**. The action taken by a processor in response to an interrupt or exception is referred to as **servicing** or **handling** the interrupt or exception.

Interrupts typically occur at random times during the execution of a program, in response to signals from hardware. System hardware uses interrupts to handle events external to the processor, such as requests to service peripheral devices. Software can also generate interrupts by executing the `INT n` instruction.

Exceptions occur when the processor detects an error condition while executing an instruction, such as division by zero. The processor detects a variety of error conditions including protection violations, page faults, and internal machine faults. The **machine-check architecture** of the Pentium 4, Intel Xeon, P6 family, and Pentium processors also permits a machine-check exception to be generated when internal hardware errors and bus errors are detected.

The IA-32 architecture's interrupt and exception-handling mechanism allows interrupts and exceptions to be handled transparently to application programs and the operating system or executive. When an interrupt is received or an exception is detected, the currently running procedure or task is automatically suspended while the processor executes an interrupt or exception handler. When execution of the handler is complete, the processor resumes execution of the interrupted procedure or task. The resumption of the interrupted procedure or task happens without loss of program continuity, unless recovery from an exception was not possible or an interrupt caused the currently running program to be terminated.

This chapter describes the processor's interrupt and exception-handling mechanism, when operating in protected mode. A detailed description of the exceptions and the conditions that cause them to be generated is given at the end of this chapter. See Chapter 16, *8086 Emulation*, for a description of the interrupt and exception mechanism for real-address and virtual-8086 mode.

5.2. EXCEPTION AND INTERRUPT VECTORS

To aid in handling exceptions and interrupts, each IA-32 architecture-defined exception and each interrupt condition that requires special handling by the processor is assigned a unique identification number, called a **vector**. The processor uses the vector assigned to an exception or interrupt as an index into its interrupt descriptor table (IDT) to locate the entry point of an exception or interrupt handler (see Section 5.10., “Interrupt Descriptor Table (IDT)”).

The allowable range for vector numbers is 0 to 255. The vectors in the range 0 through 31 are reserved by the IA-32 architecture for architecture-defined exceptions and interrupts. Not all of the vectors in this range have a currently defined function. The unassigned vectors in this range are reserved for future uses. **Do not use the reserved vectors.**

The vectors in the range 32 to 255 are designated as user-defined interrupts and are not reserved by the IA-32 architecture. These interrupts are generally assigned to external I/O devices to enable those devices to send interrupts to the processor through one of the external hardware interrupt mechanisms described in Section 5.3., “Sources of Interrupts”.

Table 5-1 shows the assignments vectors to architecturally defined exceptions and to the NMI interrupt. For each exception, this table gives the exception type (see Section 5.5., “Exception Classifications”) and indicates whether an error code is saved on the stack for the exception. The source of each predefined exception and the NMI interrupt is also given.

5.3. SOURCES OF INTERRUPTS

The processor receives interrupts from two sources:

- External (hardware generated) interrupts.
- Software-generated interrupts.

5.3.1. External Interrupts

External interrupts are received through pins on the processor or through the local APIC. The primary interrupt pins on Pentium 4, Intel Xeon, P6 family, and Pentium processors are the LINT[1:0] pins, which are connected to the local APIC (see Chapter 8, *Advanced Programmable Interrupt Controller (APIC)*). When the local APIC is enabled, the LINT[1:0] pins can be programmed through the APIC’s local vector table (LVT) to be associated with any of the processor’s exception or interrupt vectors.

When the local APIC is disabled, these pins are configured as INTR and NMI pins, respectively. Asserting the INTR pin signals the processor that an external interrupt has occurred, and the processor reads from the system bus the interrupt vector number provided by an external interrupt controller, such as an 8259A (see Section 5.2., “Exception and Interrupt Vectors”). Asserting the NMI pin signals a non-maskable interrupt (NMI), which is assigned to interrupt vector 2.

Table 5-1. Protected-Mode Exceptions and Interrupts

Vector No.	Mnemonic	Description	Type	Error Code	Source
0	#DE	Divide Error	Fault	No	DIV and IDIV instructions.
1	#DB	RESERVED	Fault/Trap	No	For Intel use only.
2	—	NMI Interrupt	Interrupt	No	Nonmaskable external interrupt.
3	#BP	Breakpoint	Trap	No	INT 3 instruction.
4	#OF	Overflow	Trap	No	INTO instruction.
5	#BR	BOUND Range Exceeded	Fault	No	BOUND instruction.
6	#UD	Invalid Opcode (Undefined Opcode)	Fault	No	UD2 instruction or reserved opcode. ¹
7	#NM	Device Not Available (No Math Coprocessor)	Fault	No	Floating-point or WAIT/FWAIT instruction.
8	#DF	Double Fault	Abort	Yes (Zero)	Any instruction that can generate an exception, an NMI, or an INTR.
9		Coprocessor Segment Overrun (reserved)	Fault	No	Floating-point instruction. ²
10	#TS	Invalid TSS	Fault	Yes	Task switch or TSS access.
11	#NP	Segment Not Present	Fault	Yes	Loading segment registers or accessing system segments.
12	#SS	Stack-Segment Fault	Fault	Yes	Stack operations and SS register loads.
13	#GP	General Protection	Fault	Yes	Any memory reference and other protection checks.
14	#PF	Page Fault	Fault	Yes	Any memory reference.
15	—	(Intel reserved. Do not use.)		No	
16	#MF	x87 FPU Floating-Point Error (Math Fault)	Fault	No	x87 FPU floating-point or WAIT/FWAIT instruction.
17	#AC	Alignment Check	Fault	Yes (Zero)	Any data reference in memory. ³
18	#MC	Machine Check	Abort	No	Error codes (if any) and source are model dependent. ⁴
19	#XF	SIMD Floating-Point Exception	Fault	No	SSE and SSE2 floating-point instructions ⁵
20-31	—	Intel reserved. Do not use.			
32-255	—	User Defined (Non-reserved) Interrupts	Interrupt		External interrupt or INT <i>n</i> instruction.

NOTES:

1. The UD2 instruction was introduced in the Pentium Pro processor.
2. IA-32 processors after the Intel386 processor do not generate this exception.
3. This exception was introduced in the Intel486 processor.
4. This exception was introduced in the Pentium processor and enhanced in the P6 family processors.
5. This exception was introduced in the Pentium III processor.

The processor's local APIC is normally connected to a system-based I/O APIC. Here, external interrupts received at the I/O APIC's pins can be directed to the local APIC through the system bus (Pentium 4 and Intel Xeon processors) or the APIC serial bus (P6 family and Pentium processors). The I/O APIC determines the vector number of the interrupt and sends this number to the local APIC. When a system contains multiple processors, processors can also send interrupts to one another by means of the system bus (Pentium 4 and Intel Xeon processors) or the APIC serial bus (P6 family and Pentium processors).

The LINT[1:0] pins are not available on the Intel486 processor and the earlier Pentium processors that do not contain an on-chip local APIC. Instead these processors have dedicated NMI and INTR pins. With these processors, external interrupts are typically generated by a system-based interrupt controller (8259A), with the interrupts being signaled through the INTR pin.

Note that several other pins on the processor cause a processor interrupt to occur; however, these interrupts are not handled by the interrupt and exception mechanism described in this chapter. These pins include the RESET#, FLUSH#, STPCLK#, SMI#, R/S#, and INIT# pins. Which of these pins are included on a particular IA-32 processor is implementation dependent. The functions of these pins are described in the data books for the individual processors. The SMI# pin is also described in Chapter 13, *System Management*.

5.3.2. Maskable Hardware Interrupts

Any external interrupt that is delivered to the processor by means of the INTR pin or through the local APIC is called a **maskable hardware interrupt**. The maskable hardware interrupts that can be delivered through the INTR pin include all IA-32 architecture defined interrupt vectors from 0 through 255; those that can be delivered through the local APIC include interrupt vectors 16 through 255.

The IF flag in the EFLAGS register permits all the maskable hardware interrupts to be masked as a group (see Section 5.8.1., "Masking Maskable Hardware Interrupts"). Note that when interrupts 0 through 15 are delivered through the local APIC, the APIC indicates the receipt of an illegal vector.

5.3.3. Software-Generated Interrupts

The INT *n* instruction permits interrupts to be generated from within software by supplying the interrupt vector number as an operand. For example, the INT 35 instruction forces an implicit call to the interrupt handler for interrupt 35.

Any of the interrupt vectors from 0 to 255 can be used as a parameter in this instruction. If the processor's predefined NMI vector is used, however, the response of the processor will not be the same as it would be from an NMI interrupt generated in the normal manner. If vector number 2 (the NMI vector) is used in this instruction, the NMI interrupt handler is called, but the processor's NMI-handling hardware is not activated.

NOTE

Interrupts generated in software with the INT *n* instruction cannot be masked by the IF flag in the EFLAGS register.

5.4. SOURCES OF EXCEPTIONS

The processor receives exceptions from three sources:

- Processor-detected program-error exceptions.
- Software-generated exceptions.
- Machine-check exceptions.

5.4.1. Program-Error Exceptions

The processor generates one or more exceptions when it detects program errors during the execution in an application program or the operating system or executive. The IA-32 architecture defines a vector number for each processor-detectable exception. The exceptions are further classified as **faults**, **traps**, and **aborts** (see Section 5.5., “Exception Classifications”).

5.4.2. Software-Generated Exceptions

The INTO, INT 3, and BOUND instructions permit exceptions to be generated in software. These instructions allow checks for specific exception conditions to be performed at specific points in the instruction stream. For example, the INT 3 instruction causes a breakpoint exception to be generated.

The INT *n* instruction can be used to emulate a specific exception in software, with one limitation. If the *n* operand in the INT *n* instruction contains a vector for one of the IA-32 architecture exceptions, the processor will generate an interrupt to that vector, which will in turn invoke the exception handler associated with that vector. Because this is actually an interrupt, however, the processor does not push an error code onto the stack, even if a hardware-generated exception for that vector normally produces one. For those exceptions that produce an error code, the exception handler will attempt to pop an error code from the stack while handling the exception. If the INT *n* instruction was used to emulate the generation of an exception, the handler will pop off and discard the EIP (in place of the missing error code), sending the return to the wrong location.

5.4.3. Machine-Check Exceptions

The P6 family and Pentium processors provide both internal and external machine-check mechanisms for checking the operation of the internal chip hardware and bus transactions. These mechanisms constitute extended (implementation dependent) exception mechanisms. When a machine-check error is detected, the processor signals a machine-check exception (vector 18) and returns an error code.

See “Interrupt 18—Machine Check Exception (#MC)” at the end of this chapter and Chapter 14, *Machine-Check Architecture*, for a detailed description of the machine-check mechanism.

5.5. EXCEPTION CLASSIFICATIONS

Exceptions are classified as **faults**, **traps**, or **aborts** depending on the way they are reported and whether the instruction that caused the exception can be restarted with no loss of program or task continuity.

Faults	A fault is an exception that can generally be corrected and that, once corrected, allows the program to be restarted with no loss of continuity. When a fault is reported, the processor restores the machine state to the state prior to the beginning of execution of the faulting instruction. The return address (saved contents of the CS and EIP registers) for the fault handler points to the faulting instruction, rather than the instruction following the faulting instruction.
Traps	A trap is an exception that is reported immediately following the execution of the trapping instruction. Traps allow execution of a program or task to be continued without loss of program continuity. The return address for the trap handler points to the instruction to be executed after the trapping instruction.
Aborts	An abort is an exception that does not always report the precise location of the instruction causing the exception and does not allow restart of the program or task that caused the exception. Aborts are used to report severe errors, such as hardware errors and inconsistent or illegal values in system tables.

NOTE

A small subset of exceptions that are normally reported as faults are not restartable and will result in loss of some processor state. An example, executing a POPAD instruction where the stack frame crosses over the end of the stack segment will cause such a fault to be reported. Here, the exception handler will see that the instruction pointer (CS:EIP) has been restored as if the POPAD instruction had not been executed; however, the internal processor state (particularly, the general-purpose registers) will have been modified. These corner cases are considered programming errors, and an application causing this class of exceptions will likely be terminated by the operating system.

5.6. PROGRAM OR TASK RESTART

To allow restarting of program or task following the handling of an exception or an interrupt, all exceptions except aborts are guaranteed to report the exception on a precise instruction boundary, and all interrupts are guaranteed to be taken on an instruction boundary.

For fault-class exceptions, the return instruction pointer that the processor saves when it generates the exception points to the faulting instruction. So, when a program or task is restarted following the handling of a fault, the faulting instruction is restarted (re-executed). Restarting the faulting instruction is commonly used to handle exceptions that are generated when access to an operand is blocked. The most common example of a fault is a page-fault exception (#PF) that occurs when a program or task references an operand in a page that is not in memory. When a page-fault exception occurs, the exception handler can load the page into memory and resume execution of the program or task by restarting the faulting instruction. To insure that this instruction restart is handled transparently to the currently executing program or task, the processor saves the necessary registers and stack pointers to allow it to restore itself to its state prior to the execution of the faulting instruction.

For trap-class exceptions, the return instruction pointer points to the instruction following the trapping instruction. If a trap is detected during an instruction which transfers execution, the return instruction pointer reflects the transfer. For example, if a trap is detected while executing a JMP instruction, the return instruction pointer points to the destination of the JMP instruction, not to the next address past the JMP instruction. All trap exceptions allow program or task restart with no loss of continuity. For example, the overflow exception is a trapping exception. Here, the return instruction pointer points to the instruction following the INTO instruction that tested the OF (overflow) flag in the EFLAGS register. The trap handler for this exception resolves the overflow condition. Upon return from the trap handler, program or task execution continues at the next instruction following the INTO instruction.

The abort-class exceptions do not support reliable restarting of the program or task. Abort handlers generally are designed to collect diagnostic information about the state of the processor when the abort exception occurred and then shut down the application and system as gracefully as possible.

Interrupts rigorously support restarting of interrupted programs and tasks without loss of continuity. The return instruction pointer saved for an interrupt points to the next instruction to be executed at the instruction boundary where the processor took the interrupt. If the instruction just executed has a repeat prefix, the interrupt is taken at the end of the current iteration with the registers set to execute the next iteration.

The ability of a P6 family processor to speculatively execute instructions does not affect the taking of interrupts by the processor. Interrupts are taken at instruction boundaries located during the retirement phase of instruction execution; so they are always taken in the “in-order” instruction stream. See Chapter 2, *Introduction to the Intel Architecture*, in the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, for more information about the P6 family processors’ microarchitecture and its support for out-of-order instruction execution.

Note that the Pentium processor and earlier IA-32 processors also perform varying amounts of prefetching and preliminary decoding of instructions; however, here also exceptions and interrupts are not signaled until actual “in-order” execution of the instructions. For a given code sample, the signaling of exceptions will occur uniformly when the code is executed on any family of IA-32 processors (except where new exceptions or new opcodes have been defined).

5.7. NONMASKABLE INTERRUPT (NMI)

The nonmaskable interrupt (NMI) can be generated in either of two ways:

- External hardware asserts the NMI pin.
- The processor receives a message on the system bus (Pentium 4 and Intel Xeon processors) or the APIC serial bus (P6 family and Pentium processors) with a delivery mode NMI.

When the processor receives a NMI from either of these sources, the processor handles it immediately by calling the NMI handler pointed to by interrupt vector number 2. The processor also invokes certain hardware conditions to insure that no other interrupts, including NMI interrupts, are received until the NMI handler has completed executing (see Section 5.7.1., “Handling Multiple NMIs”).

Also, when an NMI is received from either of the above sources, it cannot be masked by the IF flag in the EFLAGS register.

It is possible to issue a maskable hardware interrupt (through the INTR pin) to vector 2 to invoke the NMI interrupt handler; however, this interrupt will not truly be an NMI interrupt. A true NMI interrupt that activates the processor’s NMI-handling hardware can only be delivered through one of the mechanisms listed above.

5.7.1. Handling Multiple NMIs

While an NMI interrupt handler is executing, the processor disables additional calls to the NMI handler until the next IRET instruction is executed. This blocking of subsequent NMIs prevents stacking up calls to the NMI handler. It is recommended that the NMI interrupt handler be accessed through an interrupt gate to disable maskable hardware interrupts (see Section 5.8.1., “Masking Maskable Hardware Interrupts”). If the NMI handler is a virtual-8086 task with an IOPL of less than 3, an IRET instruction issued from the handler generates a general-protection exception (see Section 16.2.7., “Sensitive Instructions”). In this case, the NMI is unmasked before the general-protection exception handler is invoked.

5.8. ENABLING AND DISABLING INTERRUPTS

The processor inhibits the generation of some interrupts, depending on the state of the processor and of the IF and RF flags in the EFLAGS register, as described in the following sections.

5.8.1. Masking Maskable Hardware Interrupts

The IF flag can disable the servicing of maskable hardware interrupts received on the processor’s INTR pin or through the local APIC (see Section 5.3.2., “Maskable Hardware Interrupts”). When the IF flag is clear, the processor inhibits interrupts delivered to the INTR pin or through the local APIC from generating an internal interrupt request; when the IF flag is set, interrupts delivered to the INTR or through the local APIC pin are processed as normal external interrupts.

The IF flag does not affect non-maskable interrupts (NMIs) delivered to the NMI pin or delivery mode NMI messages delivered through the local APIC, nor does it affect processor generated exceptions. As with the other flags in the EFLAGS register, the processor clears the IF flag in response to a hardware reset.

The fact that the group of maskable hardware interrupts includes the reserved interrupt and exception vectors 0 through 32 can potentially cause confusion. Architecturally, when the IF flag is set, an interrupt for any of the vectors from 0 through 32 can be delivered to the processor through the INTR pin and any of the vectors from 16 through 32 can be delivered through the local APIC. The processor will then generate an interrupt and call the interrupt or exception handler pointed to by the vector number. So for example, it is possible to invoke the page-fault handler through the INTR pin (by means of vector 14); however, this is not a true page-fault exception. It is an interrupt. As with the *INT n* instruction (see Section 5.4.2., “Software-Generated Exceptions”), when an interrupt is generated through the INTR pin to an exception vector, the processor does not push an error code on the stack, so the exception handler may not operate correctly.

The IF flag can be set or cleared with the STI (set interrupt-enable flag) and CLI (clear interrupt-enable flag) instructions, respectively. These instructions may be executed only if the CPL is equal to or less than the IOPL. A general-protection exception (#GP) is generated if they are executed when the CPL is greater than the IOPL. (The effect of the IOPL on these instructions is modified slightly when the virtual mode extension is enabled by setting the VME flag in control register CR4: see Section 16.3., “Interrupt and Exception Handling in Virtual-8086 Mode”. Behavior is also impacted by the PVI flag: see Section 16.4., “Protected-Mode Virtual Interrupts”.)

The IF flag is also affected by the following operations:

- The PUSHF instruction stores all flags on the stack, where they can be examined and modified. The POPF instruction can be used to load the modified flags back into the EFLAGS register.
- Task switches and the POPF and IRET instructions load the EFLAGS register; therefore, they can be used to modify the setting of the IF flag.
- When an interrupt is handled through an interrupt gate, the IF flag is automatically cleared, which disables maskable hardware interrupts. (If an interrupt is handled through a trap gate, the IF flag is not cleared.)

See the descriptions of the CLI, STI, PUSHF, POPF, and IRET instructions in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*, for a detailed description of the operations these instructions are allowed to perform on the IF flag.

5.8.2. Masking Instruction Breakpoints

The RF (resume) flag in the EFLAGS register controls the response of the processor to instruction-breakpoint conditions (see the description of the RF flag in Section 2.3., “System Flags and Fields in the EFLAGS Register”).

When set, it prevents an instruction breakpoint from generating a debug exception (#DB); when clear, instruction breakpoints will generate debug exceptions. The primary function of the RF flag is to prevent the processor from going into a debug exception loop on an instruction-breakpoint. See Section 15.3.1.1., “Instruction-Breakpoint Exception Condition”, for more information on the use of this flag.

5.8.3. Masking Exceptions and Interrupts When Switching Stacks

To switch to a different stack segment, software often uses a pair of instructions, for example:

```
MOV SS, AX
MOV ESP, StackTop
```

If an interrupt or exception occurs after the segment selector has been loaded into the SS register but before the ESP register has been loaded, these two parts of the logical address into the stack space are inconsistent for the duration of the interrupt or exception handler.

To prevent this situation, the processor inhibits interrupts, debug exceptions, and single-step trap exceptions after either a MOV to SS instruction or a POP to SS instruction, until the instruction boundary following the next instruction is reached. All other faults may still be generated. If the LSS instruction is used to modify the contents of the SS register (which is the recommended method of modifying this register), this problem does not occur.

5.9. PRIORITY AMONG SIMULTANEOUS EXCEPTIONS AND INTERRUPTS

If more than one exception or interrupt is pending at an instruction boundary, the processor services them in a predictable order. Table 5-2 shows the priority among classes of exception and interrupt sources.

Table 5-2. Priority Among Simultaneous Exceptions and Interrupts

Priority	Descriptions
1 (Highest)	Hardware Reset and Machine Checks - RESET - Machine Check
2	Trap on Task Switch - T flag in TSS is set
3	External Hardware Interventions - FLUSH - STOPCLK - SMI - INIT
4	Traps on the Previous Instruction - Breakpoints - Debug Trap Exceptions (TF flag set or data/I-O breakpoint)

Table 5-2. Priority Among Simultaneous Exceptions and Interrupts (Contd.)

Priority	Descriptions
5	External Interrupts - NMI Interrupts - Maskable Hardware Interrupts
6	Faults from Fetching Next Instruction - Code Breakpoint Fault - Code-Segment Limit Violation* - Code Page Fault*
7	Faults from Decoding the Next Instruction - Instruction length > 15 bytes - Invalid Opcode - Coprocessor Not Available
8 (Lowest)	Faults on Executing an Instruction - Overflow - Bound error - Invalid TSS - Segment Not Present - Stack fault - General Protection - Data Page Fault - Alignment Check - x87 FPU Floating-point exception - SIMD floating-point exception

NOTE:

* For the Pentium and Intel486 processors, the Code Segment Limit Violation and the Code Page Fault exceptions are assigned to the priority 7.

While priority among these classes listed in Table 5-2 is consistent throughout the architecture, exceptions within each class are implementation-dependent and may vary from processor to processor. The processor first services a pending exception or interrupt from the class which has the highest priority, transferring execution to the first instruction of the handler. Lower priority exceptions are discarded; lower priority interrupts are held pending. Discarded exceptions are re-generated when the interrupt handler returns execution to the point in the program or task where the exceptions and/or interrupts occurred.

5.10. INTERRUPT DESCRIPTOR TABLE (IDT)

The interrupt descriptor table (IDT) associates each exception or interrupt vector with a gate descriptor for the procedure or task used to service the associated exception or interrupt. Like the GDT and LDTs, the IDT is an array of 8-byte descriptors (in protected mode). Unlike the GDT, the first entry of the IDT may contain a descriptor. To form an index into the IDT, the processor scales the exception or interrupt vector by eight (the number of bytes in a gate descriptor). Because there are only 256 interrupt or exception vectors, the IDT need not contain more than 256 descriptors. It can contain fewer than 256 descriptors, because descriptors are required only for the interrupt and exception vectors that may occur. All empty descriptor slots in the IDT should have the present flag for the descriptor set to 0.

The base addresses of the IDT should be aligned on an 8-byte boundary to maximize performance of cache line fills. The limit value is expressed in bytes and is added to the base address to get the address of the last valid byte. A limit value of 0 results in exactly 1 valid byte. Because IDT entries are always eight bytes long, the limit should always be one less than an integral multiple of eight (that is, $8N - 1$).

The IDT may reside anywhere in the linear address space. As shown in Figure 5-1, the processor locates the IDT using the IDTR register. This register holds both a 32-bit base address and 16-bit limit for the IDT.

The LIDT (load IDT register) and SIDT (store IDT register) instructions load and store the contents of the IDTR register, respectively. The LIDT instruction loads the IDTR register with the base address and limit held in a memory operand. This instruction can be executed only when the CPL is 0. It normally is used by the initialization code of an operating system when creating an IDT. An operating system also may use it to change from one IDT to another. The SIDT instruction copies the base and limit value stored in IDTR to memory. This instruction can be executed at any privilege level.

If a vector references a descriptor beyond the limit of the IDT, a general-protection exception (#GP) is generated.

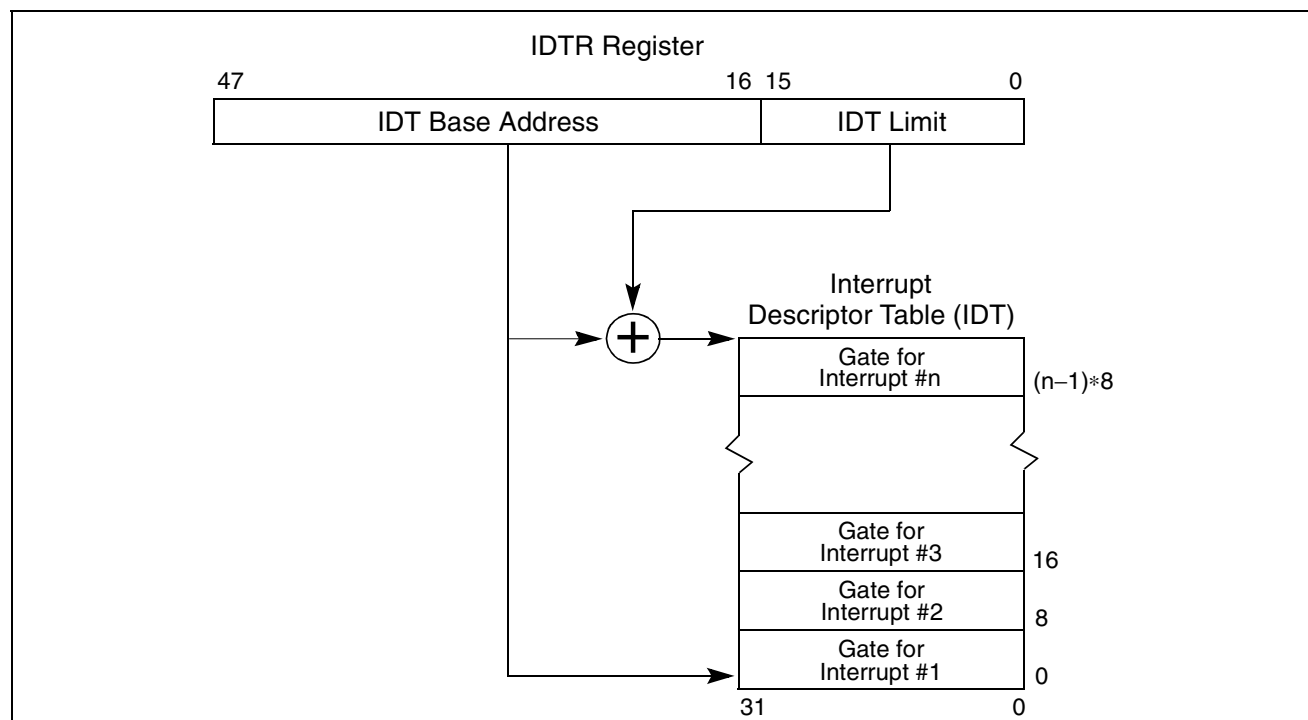


Figure 5-1. Relationship of the IDTR and IDT

5.11. IDT DESCRIPTORS

The IDT may contain any of three kinds of gate descriptors:

- Task-gate descriptor
- Interrupt-gate descriptor
- Trap-gate descriptor

Figure 5-2 shows the formats for the task-gate, interrupt-gate, and trap-gate descriptors. The format of a task gate used in an IDT is the same as that of a task gate used in the GDT or an LDT (see Section 6.2.4., “Task-Gate Descriptor”). The task gate contains the segment selector for a TSS for an exception and/or interrupt handler task.

Interrupt and trap gates are very similar to call gates (see Section 4.8.3., “Call Gates”). They contain a far pointer (segment selector and offset) that the processor uses to transfer program execution to a handler procedure in an exception- or interrupt-handler code segment. These gates differ in the way the processor handles the IF flag in the EFLAGS register (see Section 5.12.1.2., “Flag Usage By Exception- or Interrupt-Handler Procedure”).

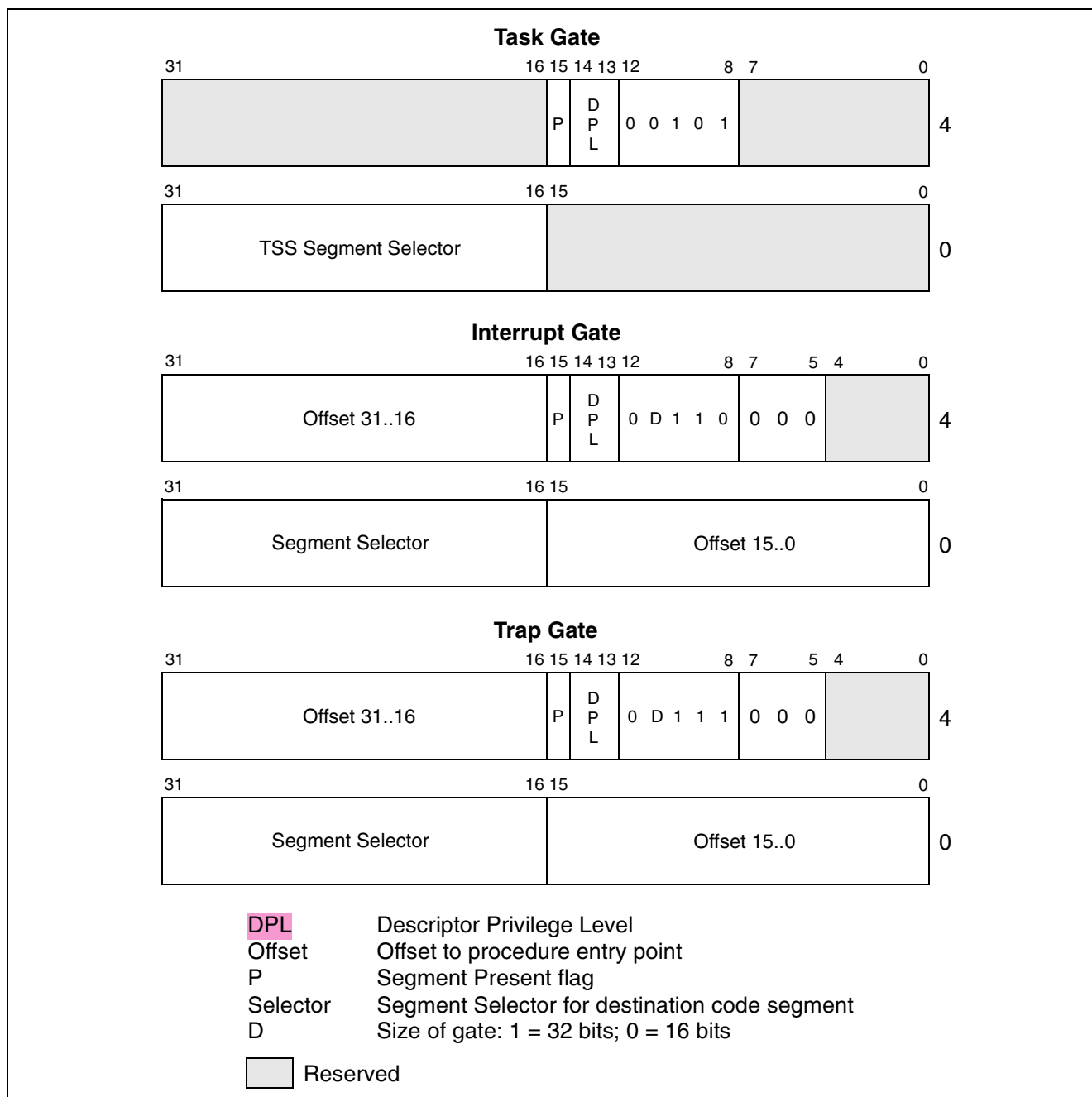


Figure 5-2. IDT Gate Descriptors

5.12. EXCEPTION AND INTERRUPT HANDLING

The processor handles calls to exception- and interrupt-handlers similar to the way it handles calls with a CALL instruction to a procedure or a task. When responding to an exception or interrupt, the processor uses the exception or interrupt vector as an index to a descriptor in the IDT. If the index points to an interrupt gate or trap gate, the processor calls the exception or interrupt handler in a manner similar to a CALL to a call gate (see Section 4.8.2., “Gate Descriptors” through Section 4.8.6., “Returning from a Called Procedure”). If index points to a task gate, the processor executes a task switch to the exception- or interrupt-handler task in a manner similar to a CALL to a task gate (see Section 6.3., “Task Switching”).

5.12.1. Exception- or Interrupt-Handler Procedures

An interrupt gate or trap gate references an exception- or interrupt-handler procedure that runs in the context of the currently executing task (see Figure 5-3). The segment selector for the gate points to a segment descriptor for an executable code segment in either the GDT or the current LDT. The offset field of the gate descriptor points to the beginning of the exception- or interrupt-handling procedure.

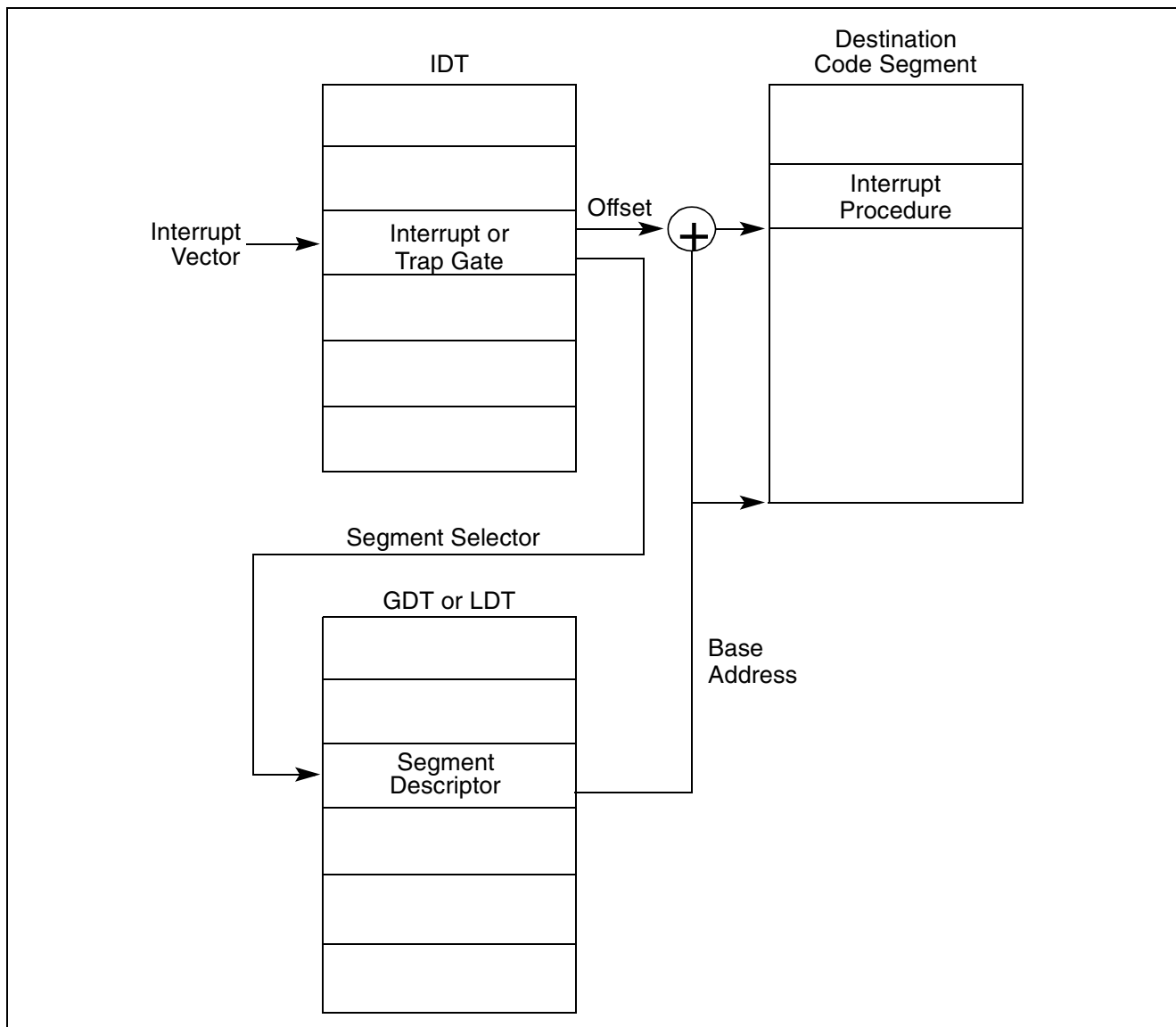


Figure 5-3. Interrupt Procedure Call

When the processor performs a call to the exception- or interrupt-handler procedure, it saves the current states of the EFLAGS register, CS register, and EIP register on the stack (see Figure 5-4). (The CS and EIP registers provide a return instruction pointer for the handler.) If an exception causes an error code to be saved, it is pushed on the stack after the EIP value.

If the handler procedure is going to be executed at the same privilege level as the interrupted procedure, the handler uses the current stack.

If the handler procedure is going to be executed at a numerically lower privilege level, a stack switch occurs. When a stack switch occurs, a stack pointer for the stack to be returned to is also saved on the stack. (The SS and ESP registers provide a return stack pointer for the handler.) The segment selector and stack pointer for the stack to be used by the handler is obtained from the TSS for the currently executing task. The processor copies the EFLAGS, SS, ESP, CS, EIP, and error code information from the interrupted procedure's stack to the handler's stack.

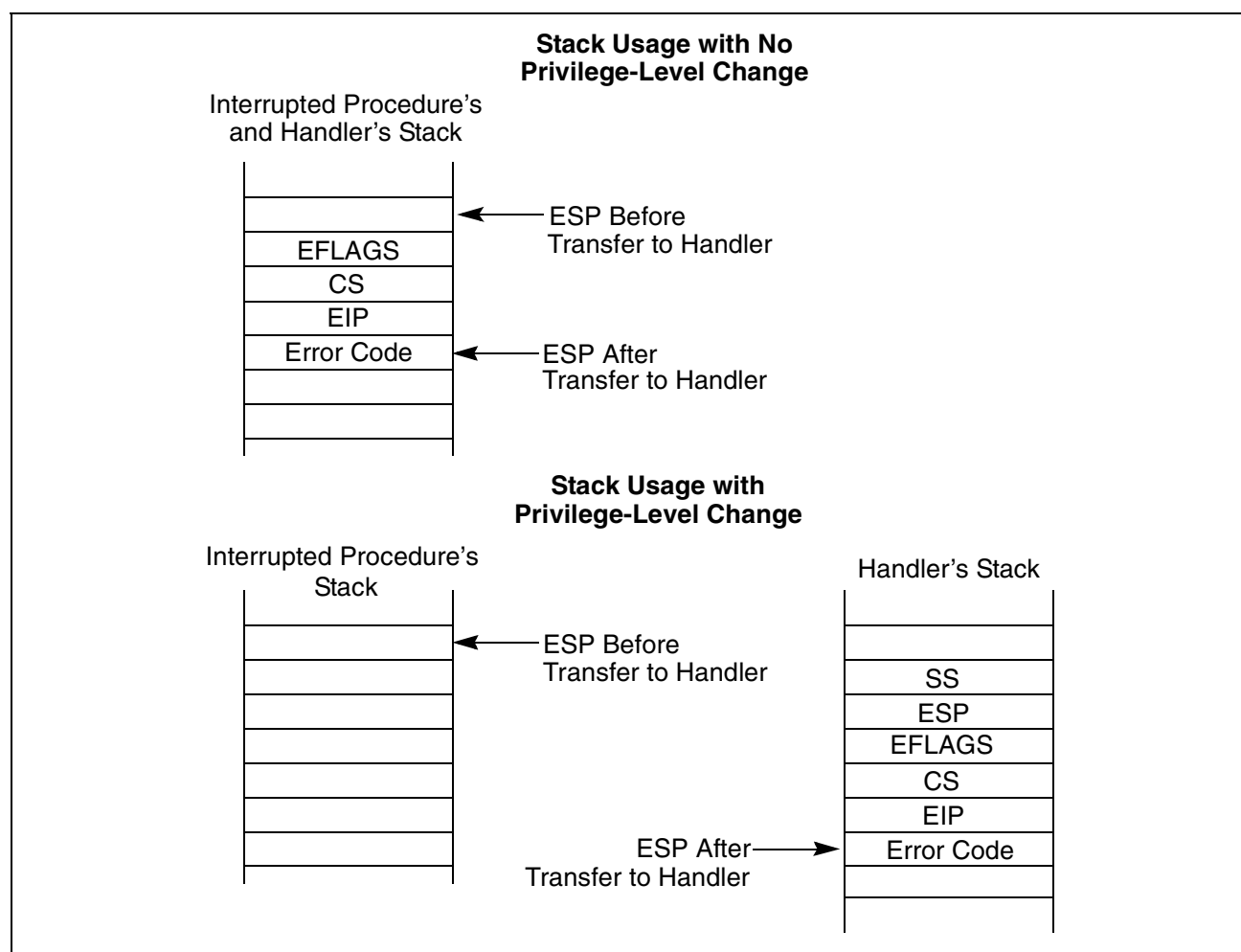


Figure 5-4. Stack Usage on Transfers to Interrupt and Exception-Handling Routines

To return from an exception- or interrupt-handler procedure, the handler must use the IRET (or IRETD) instruction. The IRET instruction is similar to the RET instruction except that it restores the saved flags into the EFLAGS register. The IOPL field of the EFLAGS register is restored only if the CPL is 0. The IF flag is changed only if the CPL is less than or equal to the IOPL. See “IRET/IRETD—Interrupt Return” in Chapter 3 of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*, for the complete operation performed by the IRET instruction.

If a stack switch occurred when calling the handler procedure, the IRET instruction switches back to the interrupted procedure's stack on the return.

5.12.1.1. PROTECTION OF EXCEPTION- AND INTERRUPT-HANDLER PROCEDURES

The privilege-level protection for exception- and interrupt-handler procedures is similar to that used for ordinary procedure calls when called through a call gate (see Section 4.8.4., “Accessing a Code Segment Through a Call Gate”). The processor does not permit transfer of execution to an exception- or interrupt-handler procedure in a less privileged code segment (numerically greater privilege level) than the CPL.

An attempt to violate this rule results in a general-protection exception (#GP). The protection mechanism for exception- and interrupt-handler procedures is different in the following ways:

- Because interrupt and exception vectors have no RPL, the RPL is not checked on implicit calls to exception and interrupt handlers.
- The processor checks the DPL of the interrupt or trap gate only if an exception or interrupt is generated with an INT n , INT 3, or INTO instruction. Here, the CPL must be less than or equal to the DPL of the gate. This restriction prevents application programs or procedures running at privilege level 3 from using a software interrupt to access critical exception handlers, such as the page-fault handler, providing that those handlers are placed in more privileged code segments (numerically lower privilege level). For hardware-generated interrupts and processor-detected exceptions, the processor ignores the DPL of interrupt and trap gates.

Because exceptions and interrupts generally do not occur at predictable times, these privilege rules effectively impose restrictions on the privilege levels at which exception and interrupt-handling procedures can run. Either of the following techniques can be used to avoid privilege-level violations.

- The exception or interrupt handler can be placed in a conforming code segment. This technique can be used for handlers that only need to access data available on the stack (for example, divide error exceptions). If the handler needs data from a data segment, the data segment needs to be accessible from privilege level 3, which would make it unprotected.
- The handler can be placed in a nonconforming code segment with privilege level 0. This handler would always run, regardless of the CPL that the interrupted program or task is running at.

5.12.1.2. FLAG USAGE BY EXCEPTION- OR INTERRUPT-HANDLER PROCEDURE

When accessing an exception or interrupt handler through either an interrupt gate or a trap gate, the processor clears the TF flag in the EFLAGS register after it saves the contents of the EFLAGS register on the stack. (On calls to exception and interrupt handlers, the processor also clears the VM, RF, and NT flags in the EFLAGS register, after they are saved on the stack.) Clearing the TF flag prevents instruction tracing from affecting interrupt response. A subsequent IRET instruction restores the TF (and VM, RF, and NT) flags to the values in the saved contents of the EFLAGS register on the stack.

The only difference between an interrupt gate and a trap gate is the way the processor handles the IF flag in the EFLAGS register. When accessing an exception- or interrupt-handling procedure through an interrupt gate, the processor clears the IF flag to prevent other interrupts from interfering with the current interrupt handler. A subsequent IRET instruction restores the IF flag to its value in the saved contents of the EFLAGS register on the stack. Accessing a handler procedure through a trap gate does not affect the IF flag.

5.12.2. Interrupt Tasks

When an exception or interrupt handler is accessed through a task gate in the IDT, a task switch results. Handling an exception or interrupt with a separate task offers several advantages:

- The entire context of the interrupted program or task is saved automatically.
- A new TSS permits the handler to use a new privilege level 0 stack when handling the exception or interrupt. If an exception or interrupt occurs when the current privilege level 0 stack is corrupted, accessing the handler through a task gate can prevent a system crash by providing the handler with a new privilege level 0 stack.
- The handler can be further isolated from other tasks by giving it a separate address space. This is done by giving it a separate LDT.

The disadvantage of handling an interrupt with a separate task is that the amount of machine state that must be saved on a task switch makes it slower than using an interrupt gate, resulting in increased interrupt latency.

A task gate in the IDT references a TSS descriptor in the GDT (see Figure 5-5). A switch to the handler task is handled in the same manner as an ordinary task switch (see Section 6.3., “Task Switching”). The link back to the interrupted task is stored in the previous task link field of the handler task’s TSS. If an exception caused an error code to be generated, this error code is copied to the stack of the new task.

When exception- or interrupt-handler tasks are used in an operating system, there are actually two mechanisms that can be used to dispatch tasks: the software scheduler (part of the operating system) and the hardware scheduler (part of the processor's interrupt mechanism). The software scheduler needs to accommodate interrupt tasks that may be dispatched when interrupts are enabled.

NOTE

Because IA-32 architecture tasks are not re-entrant, an interrupt-handler task must disable interrupts between the time it completes handling the interrupt and the time it executes the IRET instruction. This action prevents another interrupt from occurring while the interrupt task’s TSS is still marked busy, which would cause a general-protection (#GP) exception.

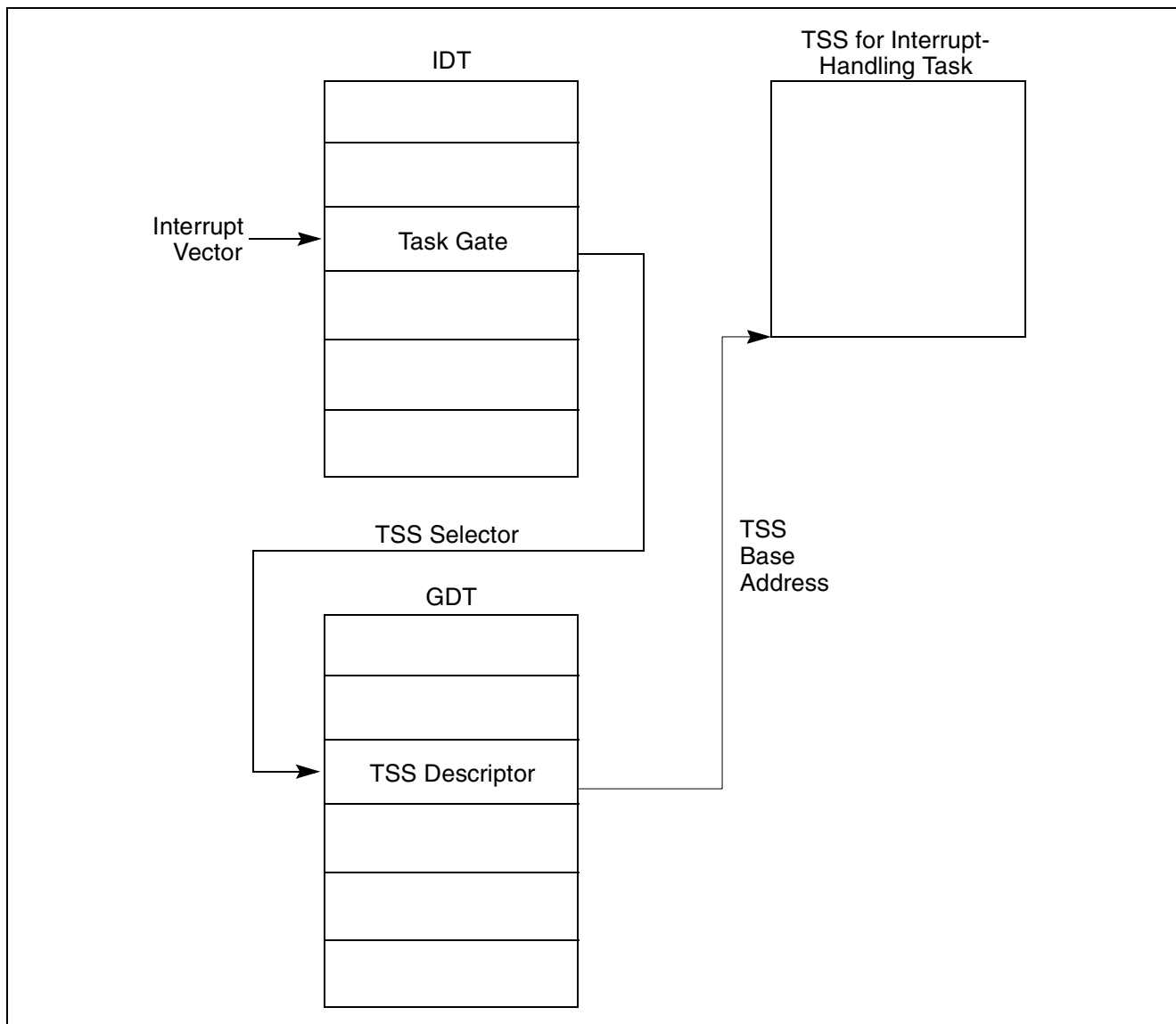


Figure 5-5. Interrupt Task Switch

5.13. ERROR CODE

When an exception condition is related to a specific segment, the processor pushes an error code onto the stack of the exception handler (whether it is a procedure or task). The error code has the format shown in Figure 5-6. The error code resembles a segment selector; however, instead of a TI flag and RPL field, the error code contains 3 flags:

- EXT** **External event (bit 0).** When set, indicates that an event external to the program, such as a hardware interrupt, caused the exception.
- IDT** **Descriptor location (bit 1).** When set, indicates that the index portion of the error code refers to a gate descriptor in the IDT; when clear, indicates that the index refers to a descriptor in the GDT or the current LDT.
- TI** **GDT/LDT (bit 2).** Only used when the IDT flag is clear. When set, the TI flag indicates that the index portion of the error code refers to a segment or gate descriptor in the LDT; when clear, it indicates that the index refers to a descriptor in the current GDT.

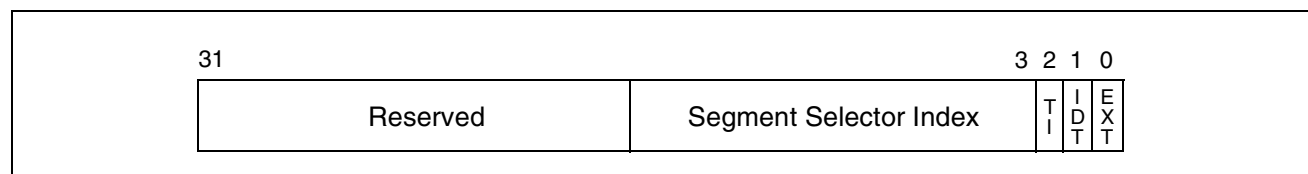


Figure 5-6. Error Code

The segment selector index field provides an index into the IDT, GDT, or current LDT to the segment or gate selector being referenced by the error code. In some cases the error code is null (that is, all bits in the lower word are clear). A null error code indicates that the error was not caused by a reference to a specific segment or that a null segment descriptor was referenced in an operation.

The format of the error code is different for page-fault exceptions (#PF), see “Interrupt 14—Page-Fault Exception (#PF)” in this chapter.

The error code is pushed on the stack as a doubleword or word (depending on the default interrupt, trap, or task gate size). To keep the stack aligned for doubleword pushes, the upper half of the error code is reserved. Note that the error code is not popped when the IRET instruction is executed to return from an exception handler, so the handler must remove the error code before executing a return.

Error codes are not pushed on the stack for exceptions that are generated externally (with the INTR or LINT[1:0] pins) or the INT *n* instruction, even if an error code is normally produced for those exceptions.

5.14. EXCEPTION AND INTERRUPT REFERENCE

The following sections describe conditions which generate exceptions and interrupts. They are arranged in the order of vector numbers. The information contained in these sections are as follows:

Exception Class	Indicates whether the exception class is a fault, trap, or abort type. Some exceptions can be either a fault or trap type, depending on when the error condition is detected. (This section is not applicable to interrupts.)
Description	Gives a general description of the purpose of the exception or interrupt type. It also describes how the processor handles the exception or interrupt.
Exception Error Code	Indicates whether an error code is saved for the exception. If one is saved, the contents of the error code are described. (This section is not applicable to interrupts.)
Saved Instruction Pointer	Describes which instruction the saved (or return) instruction pointer points to. It also indicates whether the pointer can be used to restart a faulting instruction.
Program State Change	Describes the effects of the exception or interrupt on the state of the currently running program or task and the possibilities of restarting the program or task without loss of continuity.

Interrupt 0—Divide Error Exception (#DE)

Exception Class Fault.

Description

Indicates the divisor operand for a DIV or IDIV instruction is 0 or that the result cannot be represented in the number of bits specified for the destination operand.

Exception Error Code

None.

Saved Instruction Pointer

Saved contents of CS and EIP registers point to the instruction that generated the exception.

Program State Change

A program-state change does not accompany the divide error, because the exception occurs before the faulting instruction is executed.

Interrupt 1—Debug Exception (#DB)

Reserved for Intel use only.

Interrupt 2—NMI Interrupt

Exception Class Not applicable.

Description

The nonmaskable interrupt (NMI) is generated externally by asserting the processor's NMI pin or through an NMI request set by the I/O APIC to the local APIC. This interrupt causes the NMI interrupt handler to be called.

Exception Error Code

Not applicable.

Saved Instruction Pointer

The processor always takes an NMI interrupt on an instruction boundary. The saved contents of CS and EIP registers point to the next instruction to be executed at the point the interrupt is taken. See Section 5.5., “Exception Classifications”, for more information about when the processor takes NMI interrupts.

Program State Change

The instruction executing when an NMI interrupt is received is completed before the NMI is generated. A program or task can thus be restarted upon returning from an interrupt handler without loss of continuity, provided the interrupt handler saves the state of the processor before handling the interrupt and restores the processor's state prior to a return.

Interrupt 3—Breakpoint Exception (#BP)

Exception Class Trap.

Description

Indicates that a breakpoint instruction (INT 3) was executed, causing a breakpoint trap to be generated. Typically, a debugger sets a breakpoint by replacing the first opcode byte of an instruction with the opcode for the INT 3 instruction. (The INT 3 instruction is one byte long, which makes it easy to replace an opcode in a code segment in RAM with the breakpoint opcode.) The operating system or a debugging tool can use a data segment mapped to the same physical address space as the code segment to place an INT 3 instruction in places where it is desired to call the debugger.

With the P6 family, Pentium, Intel486, and Intel386 processors, it is more convenient to set breakpoints with the debug registers. (See Section 15.3.2., “Breakpoint Exception (#BP)—Interrupt Vector 3”, for information about the breakpoint exception.) If more breakpoints are needed beyond what the debug registers allow, the INT 3 instruction can be used.

The breakpoint (#BP) exception can also be generated by executing the INT *n* instruction with an operand of 3. The action of this instruction (INT 3) is slightly different than that of the INT 3 instruction (see “INTn/INTO/INT3—Call to Interrupt Procedure” in Chapter 3 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 2*).

Exception Error Code

None.

Saved Instruction Pointer

Saved contents of CS and EIP registers point to the instruction following the INT 3 instruction.

Program State Change

Even though the EIP points to the instruction following the breakpoint instruction, the state of the program is essentially unchanged because the INT 3 instruction does not affect any register or memory locations. The debugger can thus resume the suspended program by replacing the INT 3 instruction that caused the breakpoint with the original opcode and decrementing the saved contents of the EIP register. Upon returning from the debugger, program execution resumes with the replaced instruction.

Interrupt 4—Overflow Exception (#OF)

Exception Class Trap.

Description

Indicates that an overflow trap occurred when an INTO instruction was executed. The INTO instruction checks the state of the OF flag in the EFLAGS register. If the OF flag is set, an overflow trap is generated.

Some arithmetic instructions (such as the ADD and SUB) perform both signed and unsigned arithmetic. These instructions set the OF and CF flags in the EFLAGS register to indicate signed overflow and unsigned overflow, respectively. When performing arithmetic on signed operands, the OF flag can be tested directly or the INTO instruction can be used. The benefit of using the INTO instruction is that if the overflow exception is detected, an exception handler can be called automatically to handle the overflow condition.

Exception Error Code

None.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction following the INTO instruction.

Program State Change

Even though the EIP points to the instruction following the INTO instruction, the state of the program is essentially unchanged because the INTO instruction does not affect any register or memory locations. The program can thus resume normal execution upon returning from the overflow exception handler.

Interrupt 5—BOUND Range Exceeded Exception (#BR)

Exception Class Fault.

Description

Indicates that a BOUND-range-exceeded fault occurred when a BOUND instruction was executed. The BOUND instruction checks that a signed array index is within the upper and lower bounds of an array located in memory. If the array index is not within the bounds of the array, a BOUND-range-exceeded fault is generated.

Exception Error Code

None.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the BOUND instruction that generated the exception.

Program State Change

A program-state change does not accompany the bounds-check fault, because the operands for the BOUND instruction are not modified. Returning from the BOUND-range-exceeded exception handler causes the BOUND instruction to be restarted.

Interrupt 6—Invalid Opcode Exception (#UD)

Exception Class Fault.

Description

Indicates that the processor did one of the following things:

- Attempted to execute an invalid or reserved opcode.
- Attempted to execute an instruction with an operand type that is invalid for its accompanying opcode; for example, the source operand for a LES instruction is not a memory location.
- Attempted to execute an MMX, SSE, or SSE2 instruction on an IA-32 processor that does not support the MMX technology, SSE, or SSE2 extensions, respectively. CPUID feature flags MMX (bit 23), SSE (bit 25), and SSE2 (bit 26) indicate support for these extensions.
- Attempted to execute an MMX instruction or an SSE or SSE2 SIMD instruction (with the exception of the MOVNTI, PAUSE, PREFETCH h , SFENCE, LFENCE, MFENCE, and CLFLUSH instructions) when the EM flag in control register CR0 is set (1).
- Attempted to execute an SSE or SSE2 instruction when the OSFXSR bit in control register CR4 is clear (0). Note this does not include the following SSE and SSE2 instructions: MASKMOVQ, MOVNTQ, MOVNTI, PREFETCH h , SFENCE, LFENCE, MFENCE, and CLFLUSH, or the 64-bit versions of the PAVGB, PAVGW, PEXTRW, PINSRW, PMAXSW, PMAXUB, PMINSW, PMINUB, PMOVMASKB, PMULHUW, PSADBW, PSHUFW, PADDQ, and PSUBQ instructions.
- Attempted to execute an SSE or SSE2 instruction on an IA-32 processor that causes a SIMD floating-point exception when the OSXMMEXCPT bit in control register CR4 is clear (0).
- Executed a UD2 instruction. Note that even though it is the execution of the UD2 instruction that causes the invalid opcode exception, the saved instruction pointer still points at the UD2 instruction.
- Detected a LOCK prefix that precedes an instruction that may not be locked or one that may be locked but the destination operand is not a memory location.
- Attempted to execute an LLDT, SLDT, LTR, STR, LSL, LAR, VERR, VERW, or ARPL instruction while in real-address or virtual-8086 mode.
- Attempted to execute the RSM instruction when not in SMM mode.

In the Pentium 4, Intel Xeon, and P6 family processors, this exception is not generated until an attempt is made to retire the result of executing an invalid instruction; that is, decoding and speculatively attempting to execute an invalid opcode does not generate this exception. Likewise, in the Pentium processor and earlier IA-32 processors, this exception is not generated as the result of prefetching and preliminary decoding of an invalid instruction. (See Section 5.5., “Exception Classifications”, for general rules for taking of interrupts and exceptions.)

Interrupt 6—Invalid Opcode Exception (#UD) (Continued)

The opcodes D6 and F1 are undefined opcodes that are reserved by the IA-32 architecture. These opcodes, even though undefined, do not generate an invalid opcode exception.

The UD2 instruction is guaranteed to generate an invalid opcode exception.

Exception Error Code

None.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction that generated the exception.

Program State Change

A program-state change does not accompany an invalid-opcode fault, because the invalid instruction is not executed.

Interrupt 7—Device Not Available Exception (#NM)

Exception Class Fault.

Description

Indicates one of the following things:

The device-not-available exception is generated by either of three conditions:

- The processor executed an x87 FPU floating-point instruction while the EM flag in control register CR0 was set (1). (See the paragraph below for the special case of the WAIT/FWAIT instruction.)
- The processor executed a WAIT/FWAIT instruction while the MP and TS flags of register CR0 were set, regardless of the setting of the EM flag.
- The processor executed an x87 FPU, MMX, SSE, or SSE2 instruction (with the exception of the MOVNTI, PAUSE, PREFETCH/h, SFENCE, LFENCE, MFENCE, and CLFLUSH instructions) while the TS flag in control register CR0 was set and the EM flag is clear.

The EM flag is set when the processor does not have an internal x87 FPU floating-point unit. A device-not-available exception is then generated each time an x87 FPU floating-point instruction is encountered, allowing an exception handler to call floating-point instruction emulation routines.

The TS flag indicates that a context switch (task switch) has occurred since the last time an x87 floating-point, MMX, SSE, or SSE2 instruction was executed, but that the context of the x87 FPU, XMM, and MXCSR registers were not saved. When the TS flag is set and the EM flag is clear, the processor generates a device-not-available exception each time an x87 floating-point, MMX, SSE, or SSE2 instruction is encountered (with the exception of the instructions listed above). The exception handler can then save the context of the x87 FPU, XMM, and MXCSR registers before it executes the instruction. See Section 2.5., “Control Registers”, for more information about the TS flag.

The MP flag in control register CR0 is used along with the TS flag to determine if WAIT or FWAIT instructions should generate a device-not-available exception. It extends the function of the TS flag to the WAIT and FWAIT instructions, giving the exception handler an opportunity to save the context of the x87 FPU before the WAIT or FWAIT instruction is executed. The MP flag is provided primarily for use with the Intel 286 and Intel386 DX processors. For programs running on the Pentium 4, Intel Xeon, P6 family, Pentium, or Intel486 DX processors, or the Intel 487 SX coprocessors, the MP flag should always be set; for programs running on the Intel486 SX processor, the MP flag should be clear.

Exception Error Code

None.

Interrupt 7—Device Not Available Exception (#NM) (Continued)

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the floating-point instruction or the WAIT/FWAIT instruction that generated the exception.

Program State Change

A program-state change does not accompany a device-not-available fault, because the instruction that generated the exception is not executed.

If the EM flag is set, the exception handler can then read the floating-point instruction pointed to by the EIP and call the appropriate emulation routine.

If the MP and TS flags are set or the TS flag alone is set, the exception handler can save the context of the x87 FPU, clear the TS flag, and continue execution at the interrupted floating-point or WAIT/FWAIT instruction.

Interrupt 8—Double Fault Exception (#DF)

Exception Class Abort.

Description

Indicates that the processor detected a second exception while calling an exception handler for a prior exception. Normally, when the processor detects another exception while trying to call an exception handler, the two exceptions can be handled serially. If, however, the processor cannot handle them serially, it signals the double-fault exception. To determine when two faults need to be signalled as a double fault, the processor divides the exceptions into three classes: benign exceptions, contributory exceptions, and page faults (see Table 5-3).

Table 5-3. Interrupt and Exception Classes

Class	Vector Number	Description
Benign Exceptions and Interrupts	1	Debug
	2	NMI Interrupt
	3	Breakpoint
	4	Overflow
	5	BOUND Range Exceeded
	6	Invalid Opcode
	7	Device Not Available
	9	Coprocessor Segment Overrun
	16	Floating-Point Error
	17	Alignment Check
	18	Machine Check
	19	SIMD floating-point
	All	INT <i>n</i>
	All	INTR
Contributory Exceptions	0	Divide Error
	10	Invalid TSS
	11	Segment Not Present
	12	Stack Fault
	13	General Protection
Page Faults	14	Page Fault

Table 5-4 shows the various combinations of exception classes that cause a double fault to be generated. A double-fault exception falls in the abort class of exceptions. The program or task cannot be restarted or resumed. The double-fault handler can be used to collect diagnostic information about the state of the machine and/or, when possible, to shut the application and/or system down gracefully or restart the system.

A segment or page fault may be encountered while prefetching instructions; however, this behavior is outside the domain of Table 5-4. Any further faults generated while the processor is attempting to transfer control to the appropriate fault handler could still lead to a double-fault sequence.

Interrupt 8—Double Fault Exception (#DF) (Continued)

Table 5-4. Conditions for Generating a Double Fault

First Exception	Second Exception		
	Benign	Contributory	Page Fault
Benign	Handle Exceptions Serially	Handle Exceptions Serially	Handle Exceptions Serially
Contributory	Handle Exceptions Serially	Generate a Double Fault	Handle Exceptions Serially
Page Fault	Handle Exceptions Serially	Generate a Double Fault	Generate a Double Fault

If another exception occurs while attempting to call the double-fault handler, the processor enters shutdown mode. This mode is similar to the state following execution of an HLT instruction. In this mode, the processor stops executing instructions until an NMI interrupt, SMI interrupt, hardware reset, or INIT# is received. The processor generates a special bus cycle to indicate that it has entered shutdown mode. Software designers may need to be aware of the response of hardware when it goes into shutdown mode. For example, hardware may turn on an indicator light on the front panel, generate an NMI interrupt to record diagnostic information, invoke reset initialization, generate an INIT initialization, or generate an SMI. If any events are pending during shutdown, they will be handled after an wake event from shutdown is processed (for example, A20M# interrupts).

If a shutdown occurs while the processor is executing an NMI interrupt handler, then only a hardware reset can restart the processor. Likewise, if the shutdown occurs while executing in SMM, a hardware reset must be used to restart the processor

Exception Error Code

Zero. The processor always pushes an error code of 0 onto the stack of the double-fault handler.

Saved Instruction Pointer

The saved contents of CS and EIP registers are undefined.

Program State Change

A program-state following a double-fault exception is undefined. The program or task cannot be resumed or restarted. The only available action of the double-fault exception handler is to collect all possible context information for use in diagnostics and then close the application and/or shut down or reset the processor.

If the double fault occurs when any portion of the exception handling machine state is corrupted, the handler cannot be invoked and the processor must be reset.

Interrupt 9—Coprocesor Segment Overrun

Exception Class Abort. (Intel reserved; do not use. Recent IA-32 processors do not generate this exception.)

Description

Indicates that an Intel386 CPU-based systems with an Intel 387 math coprocessor detected a page or segment violation while transferring the middle portion of an Intel 387 math coprocessor operand. The P6 family, Pentium, and Intel486 processors do not generate this exception; instead, this condition is detected with a general protection exception (#GP), interrupt 13.

Exception Error Code

None.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction that generated the exception.

Program State Change

A program-state following a coprocessor segment-overrun exception is undefined. The program or task cannot be resumed or restarted. The only available action of the exception handler is to save the instruction pointer and reinitialize the x87 FPU using the FNINIT instruction.

Interrupt 10—Invalid TSS Exception (#TS)

Exception Class Fault.

Description

Indicates that a task switch was attempted and that invalid information was detected in the TSS for the target task. Table 5-5 shows the conditions that will cause an invalid-TSS exception to be generated. In general, these invalid conditions result from protection violations for the TSS descriptor; the LDT pointed to by the TSS; or the stack, code, or data segments referenced by the TSS.

Table 5-5. Invalid TSS Conditions

Error Code Index	Invalid Condition
TSS segment selector index	TSS segment limit less than 67H for 32-bit TSS or less than 2CH for 16-bit TSS
TSS segment selector index	During an IRET task switch, the TI flag in the TSS segment selector indicates the LDT
TSS segment selector index	During an IRET task switch, the TSS segment selector exceeds descriptor table limit
TSS segment selector index	During an IRET task switch, the busy flag in the TSS descriptor indicates an inactive task
LDT segment selector index	Invalid LDT or LDT not present
Stack-segment selector index	Stack-segment selector exceeds descriptor table limit
Stack-segment selector index	Stack segment is not writable
Stack-segment selector index	Stack segment DPL $\neq$ CPL
Stack-segment selector index	Stack-segment selector RPL $\neq$ CPL
Code-segment selector index	Code-segment selector exceeds descriptor table limit
Code-segment selector index	Code segment is not executable
Code-segment selector index	Nonconforming code segment DPL $\neq$ CPL
Code-segment selector index	Conforming code segment DPL greater than CPL
Data-segment selector index	Data-segment selector exceeds descriptor table limit
Data-segment selector index	Data segment not readable

This exception can generated either in the context of the original task or in the context of the new task (see Section 6.3., “Task Switching”). Until the processor has completely verified the presence of the new TSS, the exception is generated in the context of the original task. Once the existence of the new TSS is verified, the task switch is considered complete. Any invalid-TSS conditions detected after this point are handled in the context of the new task. (A task switch is considered complete when the task register is loaded with the segment selector for the new TSS and, if the switch is due to a procedure call or interrupt, the previous task link field of the new TSS references the old TSS.)

Interrupt 10—Invalid TSS Exception (#TS) (Continued)

The invalid-TSS handler must be a task called using a task gate. Handling this exception inside the faulting TSS context is not recommended because the processor state may not be consistent.

Exception Error Code

An error code containing the segment selector index for the segment descriptor that caused the violation is pushed onto the stack of the exception handler. If the EXT flag is set, it indicates that the exception was caused by an event external to the currently running program (for example, if an external interrupt handler using a task gate attempted a task switch to an invalid TSS).

Saved Instruction Pointer

If the exception condition was detected before the task switch was carried out, the saved contents of CS and EIP registers point to the instruction that invoked the task switch. If the exception condition was detected after the task switch was carried out, the saved contents of CS and EIP registers point to the first instruction of the new task.

Program State Change

The ability of the invalid-TSS handler to recover from the fault depends on the error condition that causes the fault. See Section 6.3., “Task Switching”, for more information on the task switch process and the possible recovery actions that can be taken.

If an invalid TSS exception occurs during a task switch, it can occur before or after the commit-to-new-task point. If it occurs before the commit point, no program state change occurs. If it occurs after the commit point (when the segment descriptor information for the new segment selectors have been loaded in the segment registers), the processor will load all the state information from the new TSS before it generates the exception. During a task switch, the processor first loads all the segment registers with segment selectors from the TSS, then checks their contents for validity. If an invalid TSS exception is discovered, the remaining segment registers are loaded but not checked for validity and therefore may not be usable for referencing memory. The invalid TSS handler should not rely on being able to use the segment selectors found in the CS, SS, DS, ES, FS, and GS registers without causing another exception. The exception handler should load all segment registers before trying to resume the new task; otherwise, general-protection exceptions (#GP) may result later under conditions that make diagnosis more difficult. The Intel recommended way of dealing with this situation is to use a task for the invalid TSS exception handler. The task switch back to the interrupted task from the invalid-TSS exception-handler task will then cause the processor to check the registers as it loads them from the TSS.

Interrupt 11—Segment Not Present (#NP)

Exception Class Fault.

Description

Indicates that the present flag of a segment or gate descriptor is clear. The processor can generate this exception during any of the following operations:

- While attempting to load CS, DS, ES, FS, or GS registers. [Detection of a not-present segment while loading the SS register causes a stack fault exception (#SS) to be generated.] This situation can occur while performing a task switch.
- While attempting to load the LDTR using an LLDT instruction. Detection of a not-present LDT while loading the LDTR during a task switch operation causes an invalid-TSS exception (#TS) to be generated.
- When executing the LTR instruction and the TSS is marked not present.
- While attempting to use a gate descriptor or TSS that is marked segment-not-present, but is otherwise valid.

An operating system typically uses the segment-not-present exception to implement virtual memory at the segment level. If the exception handler loads the segment and returns, the interrupted program or task resumes execution.

A not-present indication in a gate descriptor, however, does not indicate that a segment is not present (because gates do not correspond to segments). The operating system may use the present flag for gate descriptors to trigger exceptions of special significance to the operating system.

Exception Error Code

An error code containing the segment selector index for the segment descriptor that caused the violation is pushed onto the stack of the exception handler. If the EXT flag is set, it indicates that the exception resulted from an external event (NMI or INTR) that caused an interrupt, which subsequently referenced a not-present segment. The IDT flag is set if the error code refers to an IDT entry (e.g., an INT instruction referencing a not-present gate).

Saved Instruction Pointer

The saved contents of CS and EIP registers normally point to the instruction that generated the exception. If the exception occurred while loading segment descriptors for the segment selectors in a new TSS, the CS and EIP registers point to the first instruction in the new task. If the exception occurred while accessing a gate descriptor, the CS and EIP registers point to the instruction that invoked the access (for example a CALL instruction that references a call gate).

Interrupt 11—Segment Not Present (#NP) (Continued)

Program State Change

If the segment-not-present exception occurs as the result of loading a register (CS, DS, SS, ES, FS, GS, or LDTR), a program-state change does accompany the exception, because the register is not loaded. Recovery from this exception is possible by simply loading the missing segment into memory and setting the present flag in the segment descriptor.

If the segment-not-present exception occurs while accessing a gate descriptor, a program-state change does not accompany the exception. Recovery from this exception is possible merely by setting the present flag in the gate descriptor.

If a segment-not-present exception occurs during a task switch, it can occur before or after the commit-to-new-task point (see Section 6.3., “Task Switching”). If it occurs before the commit point, no program state change occurs. If it occurs after the commit point, the processor will load all the state information from the new TSS (without performing any additional limit, present, or type checks) before it generates the exception. The segment-not-present exception handler should thus not rely on being able to use the segment selectors found in the CS, SS, DS, ES, FS, and GS registers without causing another exception. (See the Program State Change description for “Interrupt 10—Invalid TSS Exception (#TS)” in this chapter for additional information on how to handle this situation.)

Interrupt 12—Stack Fault Exception (#SS)

Exception Class Fault.

Description

Indicates that one of the following stack related conditions was detected:

- A limit violation is detected during an operation that refers to the SS register. Operations that can cause a limit violation include stack-oriented instructions such as POP, PUSH, CALL, RET, IRET, ENTER, and LEAVE, as well as other memory references which implicitly or explicitly use the SS register (for example, MOV AX, [BP+6] or MOV AX, SS:[EAX+6]). The ENTER instruction generates this exception when there is not enough stack space for allocating local variables.
- A not-present stack segment is detected when attempting to load the SS register. This violation can occur during the execution of a task switch, a CALL instruction to a different privilege level, a return to a different privilege level, an LSS instruction, or a MOV or POP instruction to the SS register.

Recovery from this fault is possible by either extending the limit of the stack segment (in the case of a limit violation) or loading the missing stack segment into memory (in the case of a not-present violation).

Exception Error Code

If the exception is caused by a not-present stack segment or by overflow of the new stack during an inter-privilege-level call, the error code contains a segment selector for the segment that caused the exception. Here, the exception handler can test the present flag in the segment descriptor pointed to by the segment selector to determine the cause of the exception. For a normal limit violation (on a stack segment already in use) the error code is set to 0.

Saved Instruction Pointer

The saved contents of CS and EIP registers generally point to the instruction that generated the exception. However, when the exception results from attempting to load a not-present stack segment during a task switch, the CS and EIP registers point to the first instruction of the new task.

Program State Change

A program-state change does not generally accompany a stack-fault exception, because the instruction that generated the fault is not executed. Here, the instruction can be restarted after the exception handler has corrected the stack fault condition.

Interrupt 12—Stack Fault Exception (#SS) (Continued)

If a stack fault occurs during a task switch, it occurs after the commit-to-new-task point (see Section 6.3., “Task Switching”). Here, the processor loads all the state information from the new TSS (without performing any additional limit, present, or type checks) before it generates the exception. The stack fault handler should thus not rely on being able to use the segment selectors found in the CS, SS, DS, ES, FS, and GS registers without causing another exception. The exception handler should check all segment registers before trying to resume the new task; otherwise, general protection faults may result later under conditions that are more difficult to diagnose. (See the Program State Change description for “Interrupt 10—Invalid TSS Exception (#TS)” in this chapter for additional information on how to handle this situation.)

Interrupt 13—General Protection Exception (#GP)

Exception Class Fault.

Description

Indicates that the processor detected one of a class of protection violations called “general-protection violations.” The conditions that cause this exception to be generated comprise all the protection violations that do not cause other exceptions to be generated (such as, invalid-TSS, segment-not-present, stack-fault, or page-fault exceptions). The following conditions cause general-protection exceptions to be generated:

- Exceeding the segment limit when accessing the CS, DS, ES, FS, or GS segments.
- Exceeding the segment limit when referencing a descriptor table (except during a task switch or a stack switch).
- Transferring execution to a segment that is not executable.
- Writing to a code segment or a read-only data segment.
- Reading from an execute-only code segment.
- Loading the SS register with a segment selector for a read-only segment (unless the selector comes from a TSS during a task switch, in which case an invalid-TSS exception occurs).
- Loading the SS, DS, ES, FS, or GS register with a segment selector for a system segment.
- Loading the DS, ES, FS, or GS register with a segment selector for an execute-only code segment.
- Loading the SS register with the segment selector of an executable segment or a null segment selector.
- Loading the CS register with a segment selector for a data segment or a null segment selector.
- Accessing memory using the DS, ES, FS, or GS register when it contains a null segment selector.
- Switching to a busy task during a call or jump to a TSS.
- Using a segment selector on a non-IRET task switch that points to a TSS descriptor in the current LDT. TSS descriptors can only reside in the GDT. This condition causes a #TS exception during an IRET task switch.
- Violating any of the privilege rules described in Chapter 4, *Protection*.
- Exceeding the instruction length limit of 15 bytes (this only can occur when redundant prefixes are placed before an instruction).

Interrupt 13—General Protection Exception (#GP) (Continued)

- Loading the CR0 register with a set PG flag (paging enabled) and a clear PE flag (protection disabled).
- Loading the CR0 register with a set NW flag and a clear CD flag.
- Referencing an entry in the IDT (following an interrupt or exception) that is not an interrupt, trap, or task gate.
- Attempting to access an interrupt or exception handler through an interrupt or trap gate from virtual-8086 mode when the handler's code segment DPL is greater than 0.
- Attempting to write a 1 into a reserved bit of CR4.
- Attempting to execute a privileged instruction when the CPL is not equal to 0 (see Section 4.9., "Privileged Instructions", for a list of privileged instructions).
- Writing to a reserved bit in an MSR.
- Accessing a gate that contains a null segment selector.
- Executing the INT *n* instruction when the CPL is greater than the DPL of the referenced interrupt, trap, or task gate.
- The segment selector in a call, interrupt, or trap gate does not point to a code segment.
- The segment selector operand in the LLDT instruction is a local type (TI flag is set) or does not point to a segment descriptor of the LDT type.
- The segment selector operand in the LTR instruction is local or points to a TSS that is not available.
- The target code-segment selector for a call, jump, or return is null.
- If the PAE and/or PSE flag in control register CR4 is set and the processor detects any reserved bits in a page-directory-pointer-table entry set to 1. These bits are checked during a write to control registers CR0, CR3, or CR4 that causes a reloading of the page-directory-pointer-table entry.
- Attempting to write a non-zero value into the reserved bits of the MXCSR register.
- Executing an SSE or SSE2 instruction that attempts to access a 128-bit memory location that is not aligned on a 16-byte boundary when the instruction requires 16-byte alignment. This condition also applies to the stack segment.

A program or task can be restarted following any general-protection exception. If the exception occurs while attempting to call an interrupt handler, the interrupted program can be restartable, but the interrupt may be lost.

Interrupt 13—General Protection Exception (#GP) (Continued)

Exception Error Code

The processor pushes an error code onto the exception handler's stack. If the fault condition was detected while loading a segment descriptor, the error code contains a segment selector to or IDT vector number for the descriptor; otherwise, the error code is 0. The source of the selector in an error code may be any of the following:

- An operand of the instruction.
- A selector from a gate which is the operand of the instruction.
- A selector from a TSS involved in a task switch.
- IDT vector number.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction that generated the exception.

Program State Change

In general, a program-state change does not accompany a general-protection exception, because the invalid instruction or operation is not executed. An exception handler can be designed to correct all of the conditions that cause general-protection exceptions and restart the program or task without any loss of program continuity.

If a general-protection exception occurs during a task switch, it can occur before or after the commit-to-new-task point (see Section 6.3., “Task Switching”). If it occurs before the commit point, no program state change occurs. If it occurs after the commit point, the processor will load all the state information from the new TSS (without performing any additional limit, present, or type checks) before it generates the exception. The general-protection exception handler should thus not rely on being able to use the segment selectors found in the CS, SS, DS, ES, FS, and GS registers without causing another exception. (See the Program State Change description for “Interrupt 10—Invalid TSS Exception (#TS)” in this chapter for additional information on how to handle this situation.)

Interrupt 14—Page-Fault Exception (#PF)

Exception Class Fault.

Description

Indicates that, with paging enabled (the PG flag in the CR0 register is set), the processor detected one of the following conditions while using the page-translation mechanism to translate a linear address to a physical address:

- The P (present) flag in a page-directory or page-table entry needed for the address translation is clear, indicating that a page table or the page containing the operand is not present in physical memory.
- The procedure does not have sufficient privilege to access the indicated page (that is, a procedure running in user mode attempts to access a supervisor-mode page).
- Code running in user mode attempts to write to a read-only page. In the Intel486 and later processors, if the WP flag is set in CR0, the page fault will also be triggered by code running in supervisor mode that tries to write to a read-only user-mode page.
- One or more reserved bits in page directory entry are set to 1. See description below of RSVD error code flag

The exception handler can recover from page-not-present conditions and restart the program or task without any loss of program continuity. It can also restart the program or task after a privilege violation, but the problem that caused the privilege violation may be uncorrectable.

Exception Error Code

Yes (special format). The processor provides the page-fault handler with two items of information to aid in diagnosing the exception and recovering from it:

- An error code on the stack. The error code for a page fault has a format different from that for other exceptions (see Figure 5-7). The error code tells the exception handler four things:
 - The P flag indicates whether the exception was due to a not-present page (0) or to either an access rights violation or the use of a reserved bit (1).
 - The W/R flag indicates whether the memory access that caused the exception was a read (0) or write (1).
 - The U/S flag indicates whether the processor was executing at user mode (1) or supervisor mode (0) at the time of the exception.
 - The RSVD flag indicates that the processor detected 1s in reserved bits of the page directory, when the PSE or PAE flags in control register CR4 are set to 1. (The PSE flag is only available in the Pentium 4, Intel Xeon, P6 family, and Pentium processors, and the PAE flag is only available on the Pentium 4, Intel Xeon, and P6 family processors. In earlier IA-32 processor, the bit position of the RSVD flag is reserved.)

Interrupt 14—Page-Fault Exception (#PF) (Continued)

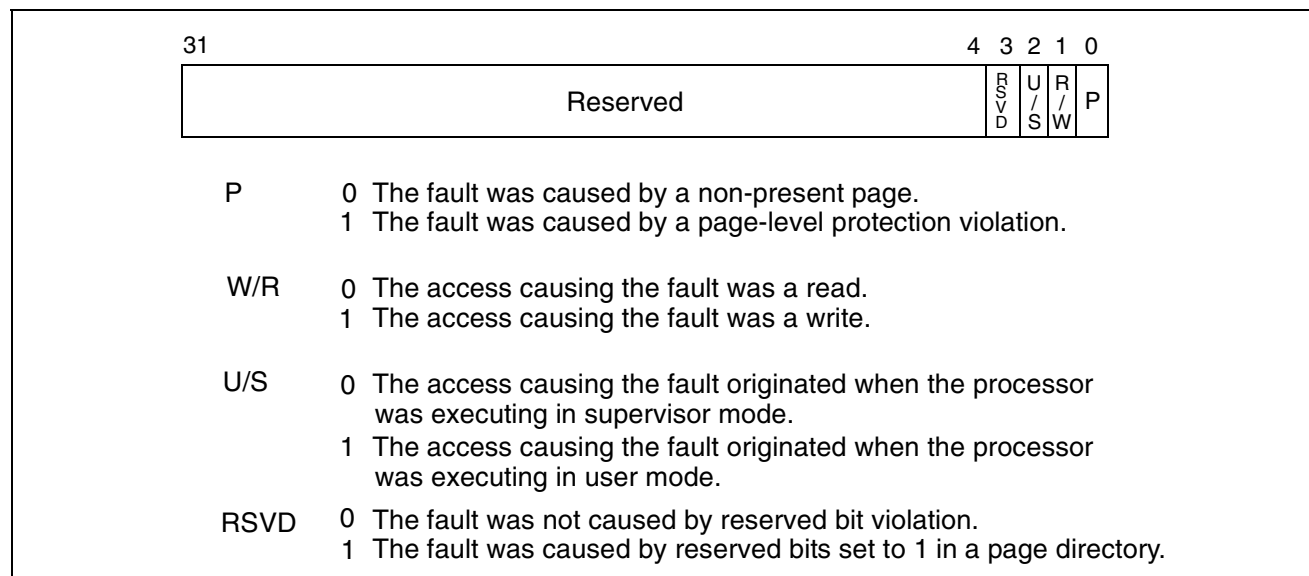


Figure 5-7. Page-Fault Error Code

- The contents of the CR2 register. The processor loads the CR2 register with the 32-bit linear address that generated the exception. The page-fault handler can use this address to locate the corresponding page directory and page-table entries. Another page fault can potentially occur during execution of the page-fault handler; the handler should save the contents of the CR2 register before a second page fault can occur.¹ If a page fault is caused by a page-level protection violation, the access flag in the page-directory entry is set when the fault occurs. The behavior of IA-32 processors regarding the access flag in the corresponding page-table entry is model specific and not architecturally defined.

Saved Instruction Pointer

The saved contents of CS and EIP registers generally point to the instruction that generated the exception. If the page-fault exception occurred during a task switch, the CS and EIP registers may point to the first instruction of the new task (as described in the following “Program State Change” section).

Program State Change

A program-state change does not normally accompany a page-fault exception, because the instruction that causes the exception to be generated is not executed. After the page-fault exception handler has corrected the violation (for example, loaded the missing page into memory), execution of the program or task can be resumed.

1. Processors update CR2 whenever a page fault is detected. If a second page fault occurs while an earlier page fault is being delivered, the faulting linear address of the second fault will overwrite the contents of CR2 (replacing the previous address). These updates to CR2 occur even if the page fault results in a double fault or occurs during the delivery of a double fault.

Interrupt 14—Page-Fault Exception (#PF) (Continued)

When a page-fault exception is generated during a task switch, the program-state may change, as follows. During a task switch, a page-fault exception can occur during any of following operations:

- While writing the state of the original task into the TSS of that task.
- While reading the GDT to locate the TSS descriptor of the new task.
- While reading the TSS of the new task.
- While reading segment descriptors associated with segment selectors from the new task.
- While reading the LDT of the new task to verify the segment registers stored in the new TSS.

In the last two cases the exception occurs in the context of the new task. The instruction pointer refers to the first instruction of the new task, not to the instruction which caused the task switch (or the last instruction to be executed, in the case of an interrupt). If the design of the operating system permits page faults to occur during task-switches, the page-fault handler should be called through a task gate.

If a page fault occurs during a task switch, the processor will load all the state information from the new TSS (without performing any additional limit, present, or type checks) before it generates the exception. The page-fault handler should thus not rely on being able to use the segment selectors found in the CS, SS, DS, ES, FS, and GS registers without causing another exception. (See the Program State Change description for “Interrupt 10—Invalid TSS Exception (#TS)” in this chapter for additional information on how to handle this situation.)

Additional Exception-Handling Information

Special care should be taken to ensure that an exception that occurs during an explicit stack switch does not cause the processor to use an invalid stack pointer (SS:ESP). Software written for 16-bit IA-32 processors often use a pair of instructions to change to a new stack, for example:

```
MOV SS, AX
MOV SP, StackTop
```

When executing this code on one of the 32-bit IA-32 processors, it is possible to get a page fault, general-protection fault (#GP), or alignment check fault (#AC) after the segment selector has been loaded into the SS register but before the ESP register has been loaded. At this point, the two parts of the stack pointer (SS and ESP) are inconsistent. The new stack segment is being used with the old stack pointer.

The processor does not use the inconsistent stack pointer if the exception handler switches to a well defined stack (that is, the handler is a task or a more privileged procedure). However, if the exception handler is called at the same privilege level and from the same task, the processor will attempt to use the inconsistent stack pointer.

Interrupt 14—Page-Fault Exception (#PF) (Continued)

In systems that handle page-fault, general-protection, or alignment check exceptions within the faulting task (with trap or interrupt gates), software executing at the same privilege level as the exception handler should initialize a new stack by using the LSS instruction rather than a pair of MOV instructions, as described earlier in this note. When the exception handler is running at privilege level 0 (the normal case), the problem is limited to procedures or tasks that run at privilege level 0, typically the kernel of the operating system.

Interrupt 16—x87 FPU Floating-Point Error (#MF)

Exception Class Fault.

Description

Indicates that the x87 FPU has detected a floating-point error. The NE flag in the register CR0 must be set for an interrupt 16 (floating-point error exception) to be generated. (See Section 2.5., “Control Registers”, for a detailed description of the NE flag.)

NOTE

SIMD floating-point exceptions (#XF) are signaled through interrupt 19.

While executing x87 FPU instructions, the x87 FPU detects and reports six types of floating-point error conditions:

- Invalid operation (#I)
 - Stack overflow or underflow (#IS)
 - Invalid arithmetic operation (#IA)
- Divide-by-zero (#Z)
- Denormalized operand (#D)
- Numeric overflow (#O)
- Numeric underflow (#U)
- Inexact result (precision) (#P)

Each of these error conditions represents an x87 FPU exception type, and for each of exception type, the x87 FPU provides a flag in the x87 FPU status register and a mask bit in the x87 FPU control register. If the x87 FPU detects a floating-point error and the mask bit for the exception type is set, the x87 FPU handles the exception automatically by generating a predefined (default) response and continuing program execution. The default responses have been designed to provide a reasonable result for most floating-point applications.

If the mask for the exception is clear and the NE flag in register CR0 is set, the x87 FPU does the following:

1. Sets the necessary flag in the FPU status register.
2. Waits until the next “waiting” x87 FPU instruction or WAIT/FWAIT instruction is encountered in the program’s instruction stream.
3. Generates an internal error signal that cause the processor to generate a floating-point exception (#MF).

Interrupt 16—x87 FPU Floating-Point Error (#MF) (Continued)

Prior to executing a waiting x87 FPU instruction or the WAIT/FWAIT instruction, the x87 FPU checks for pending x87 FPU floating-point exceptions (as described in step 2 above). Pending x87 FPU floating-point exceptions are ignored for “non-waiting” x87 FPU instructions, which include the FNINIT, FNCLEX, FNSTSW, FNSTSW AX, FNSTCW, FNSTENV, and FNSAVE instructions. Pending x87 FPU exceptions are also ignored when executing the state management instructions FXSAVE and FXRSTOR.

All of the x87 FPU floating-point error conditions can be recovered from. The x87 FPU floating-point-error exception handler can determine the error condition that caused the exception from the settings of the flags in the x87 FPU status word. See “Software Exception Handling” in Chapter 8 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, for more information on handling x87 FPU floating-point exceptions.

Exception Error Code

None. The x87 FPU provides its own error information.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the floating-point or WAIT/FWAIT instruction that was about to be executed when the floating-point-error exception was generated. This is not the faulting instruction in which the error condition was detected. The address of the faulting instruction is contained in the x87 FPU instruction pointer register. See “x87 FPU Instruction and Operand (Data) Pointers” in Chapter 8 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, for more information about information the FPU saves for use in handling floating-point-error exceptions.

Program State Change

A program-state change generally accompanies an x87 FPU floating-point exception because the handling of the exception is delayed until the next waiting x87 FPU floating-point or WAIT/FWAIT instruction following the faulting instruction. The x87 FPU, however, saves sufficient information about the error condition to allow recovery from the error and re-execution of the faulting instruction if needed.

In situations where non- x87 FPU floating-point instructions depend on the results of an x87 FPU floating-point instruction, a WAIT or FWAIT instruction can be inserted in front of a dependent instruction to force a pending x87 FPU floating-point exception to be handled before the dependent instruction is executed. See “x87 FPU Exception Synchronization” in Chapter 8 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, for more information about synchronization of x87 floating-point-error exceptions.

Interrupt 17—Alignment Check Exception (#AC)

Exception Class Fault.

Description

Indicates that the processor detected an unaligned memory operand when alignment checking was enabled. Alignment checks are only carried out in data (or stack) segments (not in code or system segments). An example of an alignment-check violation is a word stored at an odd byte address, or a doubleword stored at an address that is not an integer multiple of 4. Table 5-6 lists the alignment requirements various data types recognized by the processor.

Table 5-6. Alignment Requirements by Data Type

Data Type	Address Must Be Divisible By
Word	2
Doubleword	4
Single-precision floating-point (32-bits)	4
Double-precision floating-point (64-bits)	8
Double extended-precision floating-point (80-bits)	8
Quadword	8
Double quadword	16
Segment Selector	2
32-bit Far Pointer	2
48-bit Far Pointer	4
32-bit Pointer	4
GDTR, IDTR, LDTR, or Task Register Contents	4
FSTENV/FLDENV Save Area	4 or 2, depending on operand size
FSAVE/FRSTOR Save Area	4 or 2, depending on operand size
Bit String	2 or 4 depending on the operand-size attribute.

Note that the alignment check exception (#AC) is generated only for data types that must be aligned on word, doubleword, and quadword boundaries. A general-protection exception (#GP) is generated 128-bit data types that are not aligned on a 16-byte boundary.

To enable alignment checking, the following conditions must be true:

- AM flag in CR0 register is set.
- AC flag in the EFLAGS register is set.
- The CPL is 3 (protected mode or virtual-8086 mode).

Interrupt 17—Alignment Check Exception (#AC) (Continued)

Alignment-check exceptions (#AC) are generated only when operating at privilege level 3 (user mode). Memory references that default to privilege level 0, such as segment descriptor loads, do not generate alignment-check exceptions, even when caused by a memory reference made from privilege level 3.

Storing the contents of the GDTR, IDTR, LDTR, or task register in memory while at privilege level 3 can generate an alignment-check exception. Although application programs do not normally store these registers, the fault can be avoided by aligning the information stored on an even word-address.

The FXSAVE and FXRSTOR instructions save and restore a 512-byte data structure, the first byte of which must be aligned on a 16-byte boundary. If the alignment-check exception (#AC) is enabled when executing these instructions (and CPL is 3), a misaligned memory operand can cause either an alignment-check exception or a general-protection exception (#GP) depending on the IA-32 processor implementation (see “FXSAVE-Save x87 FPU, MMX, SSE, and SSE2 State” and “FXRSTOR-Restore x87 FPU, MMX, SSE, and SSE2 State” in Chapter 3 of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*).

The MOVUPS and MOVUPD instructions, which perform a 128-bit unaligned load or store do not generate general-protection exceptions (#GP) when an operand is not aligned on a 16-byte boundary. However, if alignment checking is enabled (as described above), 2-, 4-, and 8-byte misalignments will be detected and cause an alignment-check exception to be generated.

FSAVE and FRSTOR instructions generate unaligned references, which can cause alignment-check faults. These instructions are rarely needed by application programs.

Exception Error Code

Yes (always zero).

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction that generated the exception.

Program State Change

A program-state change does not accompany an alignment-check fault, because the instruction is not executed.

Interrupt 18—Machine-Check Exception (#MC)

Exception Class Abort.

Description

Indicates that the processor detected an internal machine error or a bus error, or that an external agent detected a bus error. The machine-check exception is model-specific, available only on the Pentium 4, Intel Xeon, P6 family, and Pentium processors. The implementation of the machine-check exception is different between the Pentium 4, Intel Xeon, P6 family, and Pentium processors, and these implementations may not be compatible with future IA-32 processors. (Use the CPUID instruction to determine whether this feature is present.)

Bus errors detected by external agents are signaled to the processor on dedicated pins: the BINIT# and MCERR# pins on the Pentium 4, Intel Xeon, and P6 family processors and the BUSCHK# pin on the Pentium processor. When one of these pins is enabled, asserting the pin causes error information to be loaded into machine-check registers and a machine-check exception is generated.

The machine-check exception and machine-check architecture are discussed in detail in Chapter 14, *Machine-Check Architecture*. Also, see the data books for the individual processors for processor-specific hardware information.

Exception Error Code

None. Error information is provide by machine-check MSRs.

Saved Instruction Pointer

For the Pentium 4 and Intel Xeon processors, the saved contents of extended machine-check state registers are directly associated with the error that caused the machine-check exception to be generated (see Section 14.3.1.3., “IA32_MCG_STATUS MSR” and Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”).

For the P6 family processors, if the EIPV flag in the MCG_STATUS MSR is set, the saved contents of CS and EIP registers are directly associated with the error that caused the machine-check exception to be generated; if the flag is clear, the saved instruction pointer may not be associated with the error (see Section 14.3.1.3., “IA32_MCG_STATUS MSR”).

For the Pentium processor, contents of the CS and EIP registers may not be associated with the error.

Program State Change

The machine-check mechanism is enabled by setting the MCE flag in control register CR4.

Interrupt 18—Machine-Check Exception (#MC) (Continued)

For the Pentium 4, Intel Xeon, P6 family, and Pentium processors, a program-state change always accompanies a machine-check exception, and an abort class exception is generated. For abort exceptions, information about the exception can be collected from the machine-check MSRs, but the program cannot generally be restarted.

If the machine-check mechanism is not enabled (the MCE flag in control register CR4 is clear), a machine-check exception causes the processor to enter the shutdown state.

Interrupt 19—SIMD Floating-Point Exception (#XF)

Exception Class Fault.

Description

Indicates the processor has detected a SSE or SSE2 SIMD floating-point exception. The appropriate status flag in the MXCSR register must be set and the particular exception unmasked for this interrupt to be generated.

There are six classes of numeric exception conditions that can occur while executing a SSE or SSE2 SIMD floating-point instruction:

- Invalid operation (#I)
- Divide-by-zero (#Z)
- Denormal operand (#D)
- Numeric overflow (#O)
- Numeric underflow (#U)
- Inexact result (Precision) (#P)

The invalid operation, divide-by-zero, and denormal-operand exceptions are pre-computation exceptions; that is, they are detected before any arithmetic operation occurs. The numeric underflow, numeric overflow, and inexact result exceptions are post-computational exceptions.

See "SIMD Floating-Point Exceptions", in Chapter 11 of the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for additional information about the SIMD floating-point exception classes.

When a SIMD floating-point exception occurs, the processor does either of the following things:

- It handles the exception automatically by producing the most reasonable result and allowing program execution to continue undisturbed. This is the response to masked exceptions.
- It generates a SIMD floating-point exception, which in turn invokes a software exception handler. This is the response to unmasked exceptions.

Each of the six SIMD floating-point exception conditions has a corresponding flag bit and mask bit in the MXCSR register. If an exception is masked (the corresponding mask bit in the MXCSR register is set), the processor takes an appropriate automatic default action and continues with the computation. If the exception is unmasked (the corresponding mask bit is clear) and the operating system supports SIMD floating-point exceptions (the OSXMMEXCPT flag in control register CR4 is set), a software exception handler is invoked through a SIMD floating-point exception. If the exception is unmasked and the OSXMMEXCPT bit is clear (indicating that the operating system does not support unmasked SIMD floating-point exceptions), an invalid opcode exception (#UD) is signaled instead of a SIMD floating-point exception.

Interrupt 19—SIMD Floating-Point Exception (#XF) (Continued)

Note that because SIMD floating-point exceptions are precise and occur immediately, the situation does not arise where an x87 FPU instruction, a WAIT/FWAIT instruction, or another SSE or SSE2 instruction will catch a pending unmasked SIMD floating-point exception.

In situations where a SIMD floating-point exception occurred while the SIMD floating-point exceptions were masked (causing the corresponding exception flag to be set) and the SIMD floating-point exception was subsequently unmasked, then no exception is generated when the exception is unmasked.

When the SSE and SSE2 SIMD floating-point instructions operate on packed operands (made up of two or four sub-operands), multiple SIMD floating-point exception conditions may be detected. If no more than one exception condition is detected for one or more sets of sub-operands, the exception flags are set for each exception condition detected. For example, an invalid exception detected for one sub-operand will not prevent the reporting of a divide-by-zero exception for another sub-operand. However, when two or more exceptions conditions are generated for one sub-operand, only one exception condition is reported, according to the precedences shown in Table 5-7. This exception precedence sometimes results in the higher priority exception condition being reported and the lower priority exception conditions being ignored.

Table 5-7. SIMD Floating-Point Exceptions Priority

Priority	Description
1 (Highest)	Invalid operation exception due to SNaN operand (or any NaN operand for maximum, minimum, or certain compare and convert operations).
2	QNaN operand ¹ .
3	Any other invalid operation exception not mentioned above or a divide-by-zero exception ² .
4	Denormal operand exception ² .
5	Numeric overflow and underflow exceptions possibly in conjunction with the inexact result exception ² .
6 (Lowest)	Inexact result exception.

Notes:

1. Though a QNaN this is not an exception, the handling of a QNaN operand has precedence over lower priority exceptions. For example, a QNaN divided by zero results in a QNaN, not a divide-by-zero-exception.
2. If masked, then instruction execution continues, and a lower priority exception can occur as well.

Exception Error Code

None.

Interrupt 19—SIMD Floating-Point Exception (#XF) (Continued)

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the SSE or SSE2 instruction that was executed when the SIMD floating-point exception was generated. This is the faulting instruction in which the error condition was detected.

Program State Change

A program-state change does not accompany a SIMD floating-point exception because the handling of the exception is immediate unless the particular exception is masked. The available state information is often sufficient to allow recovery from the error and re-execution of the faulting instruction if needed.

Interrupts 32 to 255—User Defined Interrupts

Exception Class Not applicable.

Description

Indicates that the processor did one of the following things:

- Executed an INT *n* instruction where the instruction operand is one of the vector numbers from 32 through 255.
- Responded to an interrupt request at the INTR pin or from the local APIC when the interrupt vector number associated with the request is from 32 through 255.

Exception Error Code

Not applicable.

Saved Instruction Pointer

The saved contents of CS and EIP registers point to the instruction that follows the INT *n* instruction or instruction following the instruction on which the INTR signal occurred.

Program State Change

A program-state change does not accompany interrupts generated by the INT *n* instruction or the INTR signal. The INT *n* instruction generates the interrupt within the instruction stream. When the processor receives an INTR signal, it commits all state changes for all previous instructions before it responds to the interrupt; so, program execution can resume upon returning from the interrupt handler.

6

Task Management

CHAPTER 6

TASK MANAGEMENT

This chapter describes the IA-32 architecture's task management facilities. These facilities are only available when the processor is running in protected mode.

6.1. TASK MANAGEMENT OVERVIEW

A task is a unit of work that a processor can dispatch, execute, and suspend. It can be used to execute a program, a task or process, an operating-system service utility, an interrupt or exception handler, or a kernel or executive utility.

The IA-32 architecture provides a mechanism for saving the state of a task, for dispatching tasks for execution, and for switching from one task to another. When operating in protected mode, all processor execution takes place from within a task. Even simple systems must define at least one task. More complex systems can use the processor's task management facilities to support multitasking applications.

6.1.1. Task Structure

A task is made up of two parts: a task execution space and a task-state segment (TSS). The task execution space consists of a code segment, a stack segment, and one or more data segments (see Figure 6-1). If an operating system or executive uses the processor's privilege-level protection mechanism, the task execution space also provides a separate stack for each privilege level.

The TSS specifies the segments that make up the task execution space and provides a storage place for task state information. In multitasking systems, the TSS also provides a mechanism for linking tasks.

NOTE

This chapter describes primarily 32-bit tasks and the 32-bit TSS structure. For information on 16-bit tasks and the 16-bit TSS structure, see Section 6.6., "16-Bit Task-State Segment (TSS)".

A task is identified by the segment selector for its TSS. When a task is loaded into the processor for execution, the segment selector, base address, limit, and segment descriptor attributes for the TSS are loaded into the task register (see Section 2.4.4., "Task Register (TR)").

If paging is implemented for the task, the base address of the page directory used by the task is loaded into control register CR3.

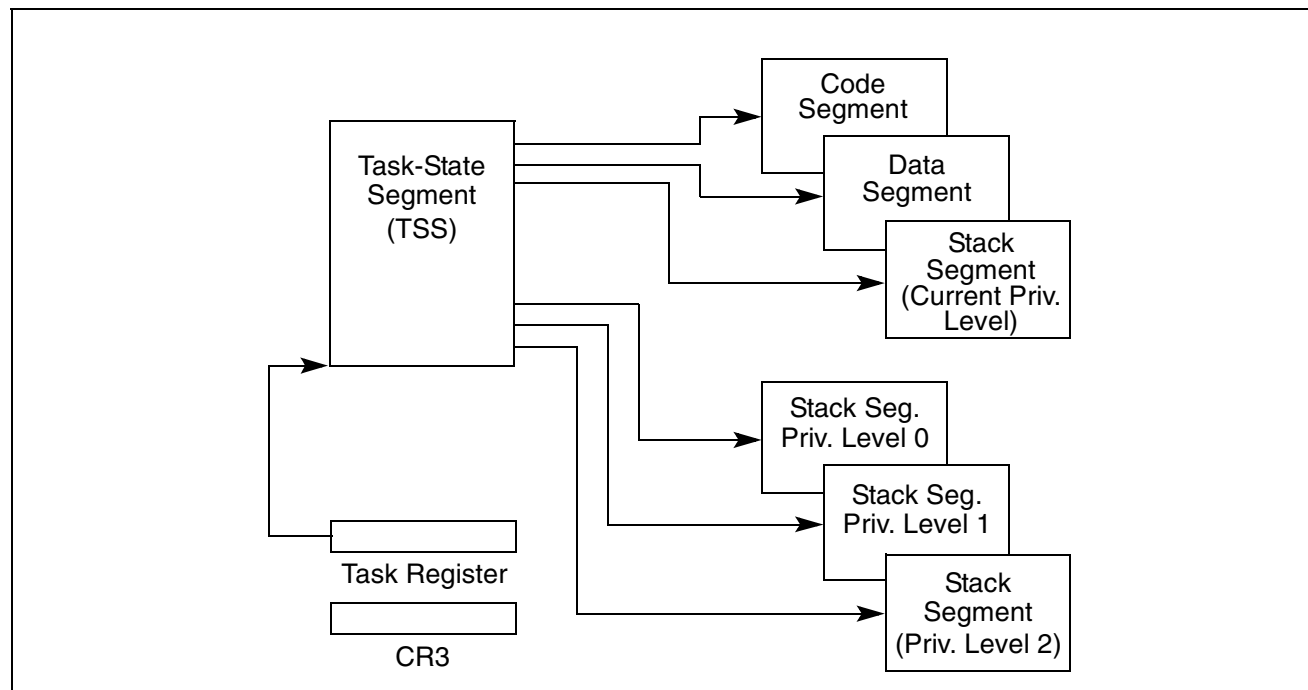


Figure 6-1. Structure of a Task

6.1.2. Task State

The following items define the state of the currently executing task:

- The task's current execution space, defined by the segment selectors in the segment registers (CS, DS, SS, ES, FS, and GS).
- The state of the general-purpose registers.
- The state of the EFLAGS register.
- The state of the EIP register.
- The state of control register CR3.
- The state of the task register.
- The state of the LDTR register.
- The I/O map base address and I/O map (contained in the TSS).
- Stack pointers to the privilege 0, 1, and 2 stacks (contained in the TSS).
- Link to previously executed task (contained in the TSS).

Prior to dispatching a task, all of these items are contained in the task's TSS, except the state of the task register. Also, the complete contents of the LDTR register are not contained in the TSS, only the segment selector for the LDT.

6.1.3. Executing a Task

Software or the processor can dispatch a task for execution in one of the following ways:

- A explicit call to a task with the CALL instruction.
- A explicit jump to a task with the JMP instruction.
- An implicit call (by the processor) to an interrupt-handler task.
- An implicit call to an exception-handler task.
- A return (initiated with an IRET instruction) when the NT flag in the EFLAGS register is set.

All of these methods of dispatching a task identify the task to be dispatched with a segment selector that points either to a task gate or the TSS for the task. When dispatching a task with a CALL or JMP instruction, the selector in the instruction may select either the TSS directly or a task gate that holds the selector for the TSS. When dispatching a task to handle an interrupt or exception, the IDT entry for the interrupt or exception must contain a task gate that holds the selector for the interrupt- or exception-handler TSS.

When a task is dispatched for execution, a task switch automatically occurs between the currently running task and the dispatched task. During a task switch, the execution environment of the currently executing task (called the task's state or **context**) is saved in its TSS and execution of the task is suspended. The context for the dispatched task is then loaded into the processor and execution of that task begins with the instruction pointed to by the newly loaded EIP register. If the task has not been run since the system was last initialized, the EIP will point to the first instruction of the task's code; otherwise, it will point to the next instruction after the last instruction that the task executed when it was last active.

If the currently executing task (the calling task) called the task being dispatched (the called task), the TSS segment selector for the calling task is stored in the TSS of the called task to provide a link back to the calling task.

For all IA-32 processors, tasks are not recursive. A task cannot call or jump to itself.

Interrupts and exceptions can be handled with a task switch to a handler task. Here, the processor not only can perform a task switch to handle the interrupt or exception, but it can automatically switch back to the interrupted task upon returning from the interrupt- or exception-handler task. This mechanism can handle interrupts that occur during interrupt tasks.

As part of a task switch, the processor can also switch to another LDT, allowing each task to have a different logical-to-physical address mapping for LDT-based segments. The page-directory base register (CR3) also is reloaded on a task switch, allowing each task to have its own set of page tables. These protection facilities help isolate tasks and prevent them from interfering with one another. If one or both of these protection mechanisms are not used, the processor provides no protection between tasks. This is true even with operating systems that use multiple privilege levels for protection. Here, a task running at privilege level 3 that uses the same LDT and page tables as other privilege-level-3 tasks can access code and corrupt data and the stack of other tasks.

Use of task management facilities for handling multitasking applications is optional. Multitasking can be handled in software, with each software defined task executed in the context of a single IA-32 architecture task.

6.2. TASK MANAGEMENT DATA STRUCTURES

The processor defines five data structures for handling task-related activities:

- Task-state segment (TSS).
- Task-gate descriptor.
- TSS descriptor.
- Task register.
- NT flag in the EFLAGS register.

When operating in protected mode, a TSS and TSS descriptor must be created for at least one task, and the segment selector for the TSS must be loaded into the task register (using the LTR instruction).

6.2.1. Task-State Segment (TSS)

The processor state information needed to restore a task is saved in a system segment called the task-state segment (TSS). Figure 6-2 shows the format of a TSS for tasks designed for 32-bit CPUs. (Compatibility with 16-bit Intel 286 processor tasks is provided by a different kind of TSS, see Figure 6-9.) The fields of a TSS are divided into two main categories: dynamic fields and static fields.

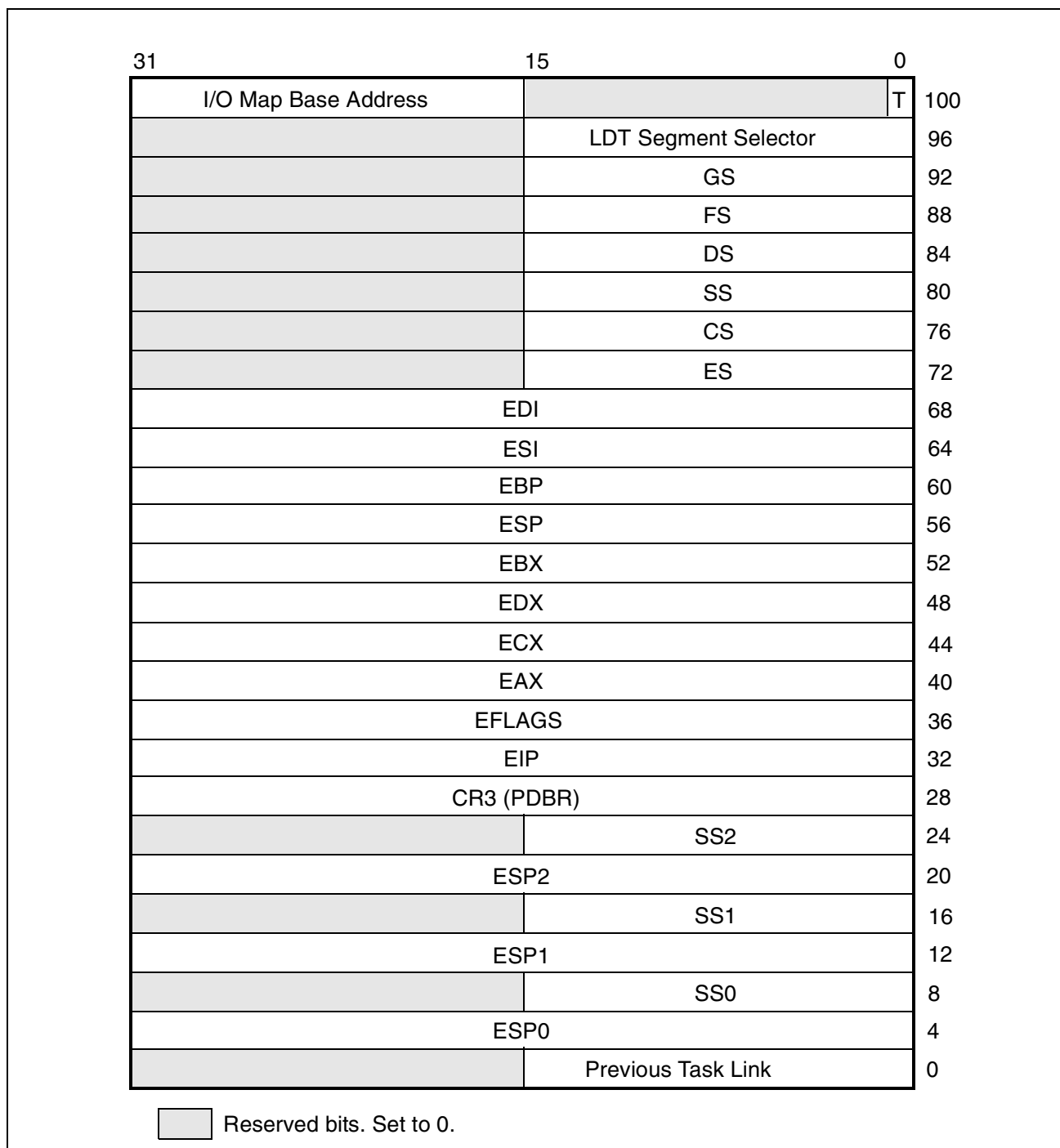


Figure 6-2. 32-Bit Task-State Segment (TSS)

The processor updates the dynamic fields when a task is suspended during a task switch. The following are dynamic fields:

General-purpose register fields

State of the EAX, ECX, EDX, EBX, ESP, EBP, ESI, and EDI registers prior to the task switch.

Segment selector fields

Segment selectors stored in the ES, CS, SS, DS, FS, and GS registers prior to the task switch.

EFLAGS register field

State of the EFLAGS register prior to the task switch.

EIP (instruction pointer) field

State of the EIP register prior to the task switch.

Previous task link field

Contains the segment selector for the TSS of the previous task (updated on a task switch that was initiated by a call, interrupt, or exception). This field (which is sometimes called the back link field) permits a task switch back to the previous task to be initiated with an IRET instruction.

The processor reads the static fields, but does not normally change them. These fields are set up when a task is created. The following are static fields:

LDT segment selector field

Contains the segment selector for the task's LDT.

CR3 control register field

Contains the base physical address of the page directory to be used by the task. Control register CR3 is also known as the page-directory base register (PDBR).

Privilege level-0, -1, and -2 stack pointer fields

These stack pointers consist of a logical address made up of the segment selector for the stack segment (SS0, SS1, and SS2) and an offset into the stack (ESP0, ESP1, and ESP2). Note that the values in these fields are static for a particular task; whereas, the SS and ESP values will change if stack switching occurs within the task.

T (debug trap) flag (byte 100, bit 0)

When set, the T flag causes the processor to raise a debug exception when a task switch to this task occurs (see Section 15.3.1.5., "Task-Switch Exception Condition").

I/O map base address field

Contains a 16-bit offset from the base of the TSS to the I/O permission bit map and interrupt redirection bitmap. When present, these maps are stored in the TSS at higher addresses. The I/O map base address points to the beginning of the I/O permission bit map and the end of the interrupt redirection bit map. See Chapter 12, *Input/Output*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for more information about the I/O permission bit map. See Section 16.3., "Interrupt and Exception Handling in Virtual-8086 Mode", for a detailed description of the interrupt redirection bit map.

If paging is used, care should be taken to avoid placing a page boundary within the part of the TSS that the processor reads during a task switch (the first 104 bytes). If a page boundary is placed within this part of the TSS, the pages on either side of the boundary must be present at the same time and contiguous in physical memory.

The reason for this restriction is that when accessing a TSS during a task switch, the processor reads and writes into the first 104 bytes of each TSS from contiguous physical addresses beginning with the physical address of the first byte of the TSS. It may not perform address translations at a page boundary if one occurs within this area. So, after the TSS access begins, if a part of the 104 bytes is not both present and physically contiguous, the processor will access incorrect TSS information, without generating a page-fault exception. The reading of this incorrect information will generally lead to an unrecoverable exception later in the task switch process.

Also, if paging is used, the pages corresponding to the previous task's TSS, the current task's TSS, and the descriptor table entries for each should be marked as read/write. The task switch will be carried out faster if the pages containing these structures are also present in memory before the task switch is initiated.

6.2.2. TSS Descriptor

The TSS, like all other segments, is defined by a segment descriptor. Figure 6-3 shows the format of a TSS descriptor. TSS descriptors may only be placed in the GDT; they cannot be placed in an LDT or the IDT.

An attempt to access a TSS using a segment selector with its TI flag set (which indicates the current LDT) causes a general-protection exception (#GP) to be generated during CALLs and JMPs; it causes an invalid TSS exception (#TS) during IRETs. A general-protection exception is also generated if an attempt is made to load a segment selector for a TSS into a segment register.

The busy flag (B) in the type field indicates whether the task is busy. A busy task is currently running or is suspended. A type field with a value of 1001B indicates an inactive task; a value of 1011B indicates a busy task. Tasks are not recursive. The processor uses the busy flag to detect an attempt to call a task whose execution has been interrupted. To insure that there is only one busy flag is associated with a task, each TSS should have only one TSS descriptor that points to it.

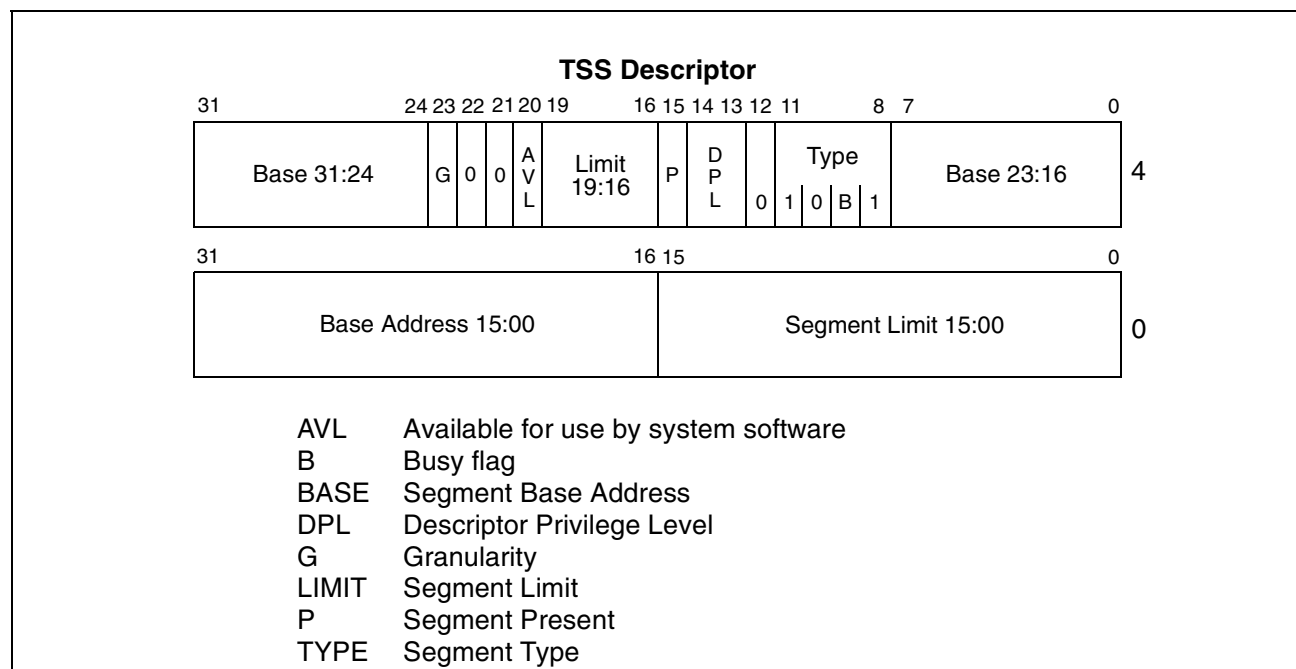


Figure 6-3. TSS Descriptor

The base, limit, and DPL fields and the granularity and present flags have functions similar to their use in data-segment descriptors (see Section 3.4.3., “Segment Descriptors”). When the G flag is 0 in a TSS descriptor for a 32-bit TSS, the limit field must have a value equal to or greater than 67H, one byte less than the minimum size of a TSS. Attempting to switch to a task whose TSS descriptor has a limit less than 67H generates an invalid-TSS exception (#TS). A larger limit is required if an I/O permission bit map is included in the TSS. An even larger limit would be required if the operating system stores additional data in the TSS. The processor does not check for a limit greater than 67H on a task switch; however, it does when accessing the I/O permission bit map or interrupt redirection bit map.

Any program or procedure with access to a TSS descriptor (that is, whose CPL is numerically equal to or less than the DPL of the TSS descriptor) can dispatch the task with a call or a jump.

In most systems, the DPLs of TSS descriptors should be set to values less than 3, so that only privileged software can perform task switching. However, in multitasking applications, DPLs for some TSS descriptors can be set to 3 to allow task switching at the application (or user) privilege level.

6.2.3. Task Register

The task register holds the 16-bit segment selector and the entire segment descriptor (32-bit base address, 16-bit segment limit, and descriptor attributes) for the TSS of the current task (see Figure 2-4). This information is copied from the TSS descriptor in the GDT for the current task. Figure 6-4 shows the path the processor uses to access the TSS, using the information in the task register.

The task register has both a visible part (that can be read and changed by software) and an invisible part (that is maintained by the processor and is inaccessible by software). The segment selector in the visible portion points to a TSS descriptor in the GDT. The processor uses the invisible portion of the task register to cache the segment descriptor for the TSS. Caching these values in a register makes execution of the task more efficient, because the processor does not need to fetch these values from memory to reference the TSS of the current task.

The LTR (load task register) and STR (store task register) instructions load and read the visible portion of the task register. The LTR instruction loads a segment selector (source operand) into the task register that points to a TSS descriptor in the GDT, and then loads the invisible portion of the task register with information from the TSS descriptor. This instruction is a privileged instruction that may be executed only when the CPL is 0. The LTR instruction generally is used during system initialization to put an initial value in the task register. Afterwards, the contents of the task register are changed implicitly when a task switch occurs.

The STR (store task register) instruction stores the visible portion of the task register in a general-purpose register or memory. This instruction can be executed by code running at any privilege level, to identify the currently running task; however, it is normally used only by operating system software.

On power up or reset of the processor, the segment selector and base address are set to the default value of 0 and the limit is set to FFFFH.

6.2.4. Task-Gate Descriptor

A task-gate descriptor provides an indirect, protected reference to a task. Figure 6-5 shows the format of a task-gate descriptor. A task-gate descriptor can be placed in the GDT, an LDT, or the IDT.

The TSS segment selector field in a task-gate descriptor points to a TSS descriptor in the GDT. The RPL in this segment selector is not used.

The DPL of a task-gate descriptor controls access to the TSS descriptor during a task switch. When a program or procedure makes a call or jump to a task through a task gate, the CPL and the RPL field of the gate selector pointing to the task gate must be less than or equal to the DPL of the task-gate descriptor. (Note that when a task gate is used, the DPL of the destination TSS descriptor is not used.)

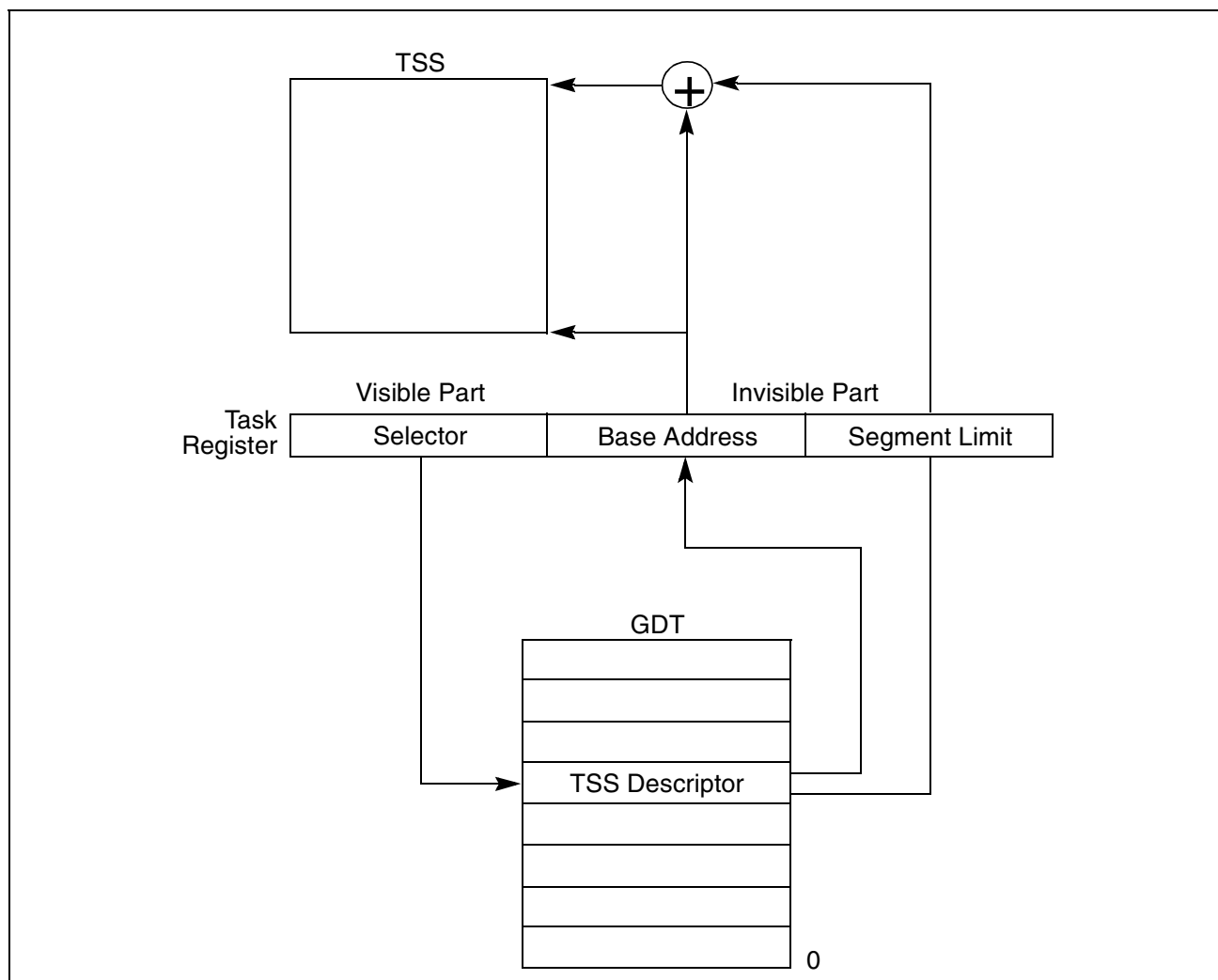


Figure 6-4. Task Register

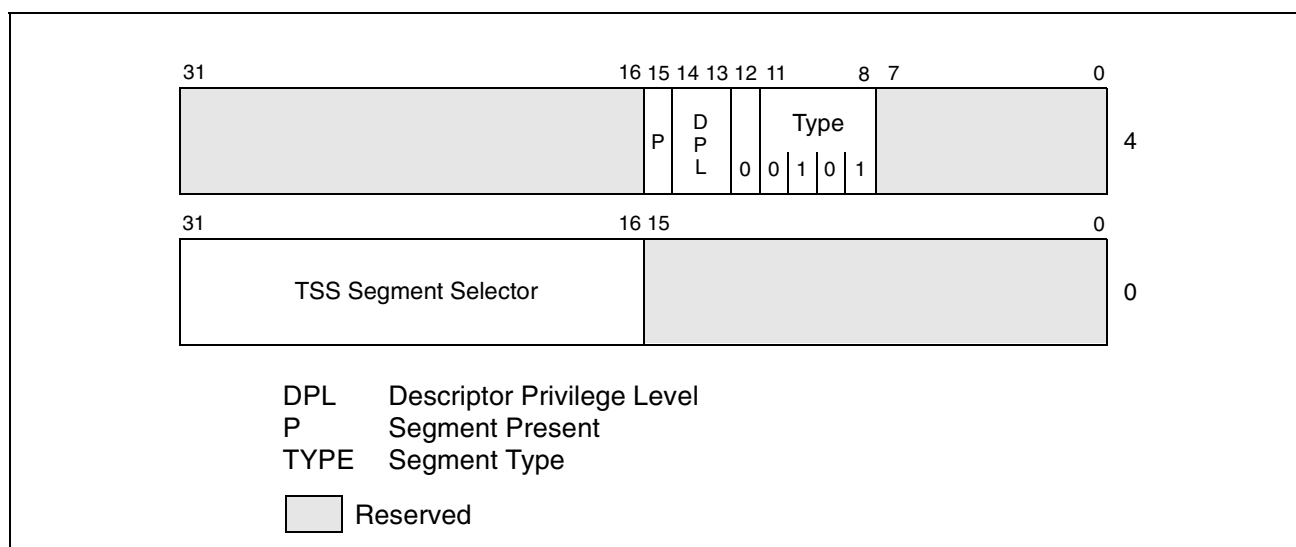


Figure 6-5. Task-Gate Descriptor

A task can be accessed either through a task-gate descriptor or a TSS descriptor. Both of these structures are provided to satisfy the following needs:

- The need for a task to have only one busy flag. Because the busy flag for a task is stored in the TSS descriptor, each task should have only one TSS descriptor. There may, however, be several task gates that reference the same TSS descriptor.
- The need to provide selective access to tasks. Task gates fill this need, because they can reside in an LDT and can have a DPL that is different from the TSS descriptor's DPL. A program or procedure that does not have sufficient privilege to access the TSS descriptor for a task in the GDT (which usually has a DPL of 0) may be allowed access to the task through a task gate with a higher DPL. Task gates give the operating system greater latitude for limiting access to specific tasks.
- The need for an interrupt or exception to be handled by an independent task. Task gates may also reside in the IDT, which allows interrupts and exceptions to be handled by handler tasks. When an interrupt or exception vector points to a task gate, the processor switches to the specified task.

Figure 6-6 illustrates how a task gate in an LDT, a task gate in the GDT, and a task gate in the IDT can all point to the same task.

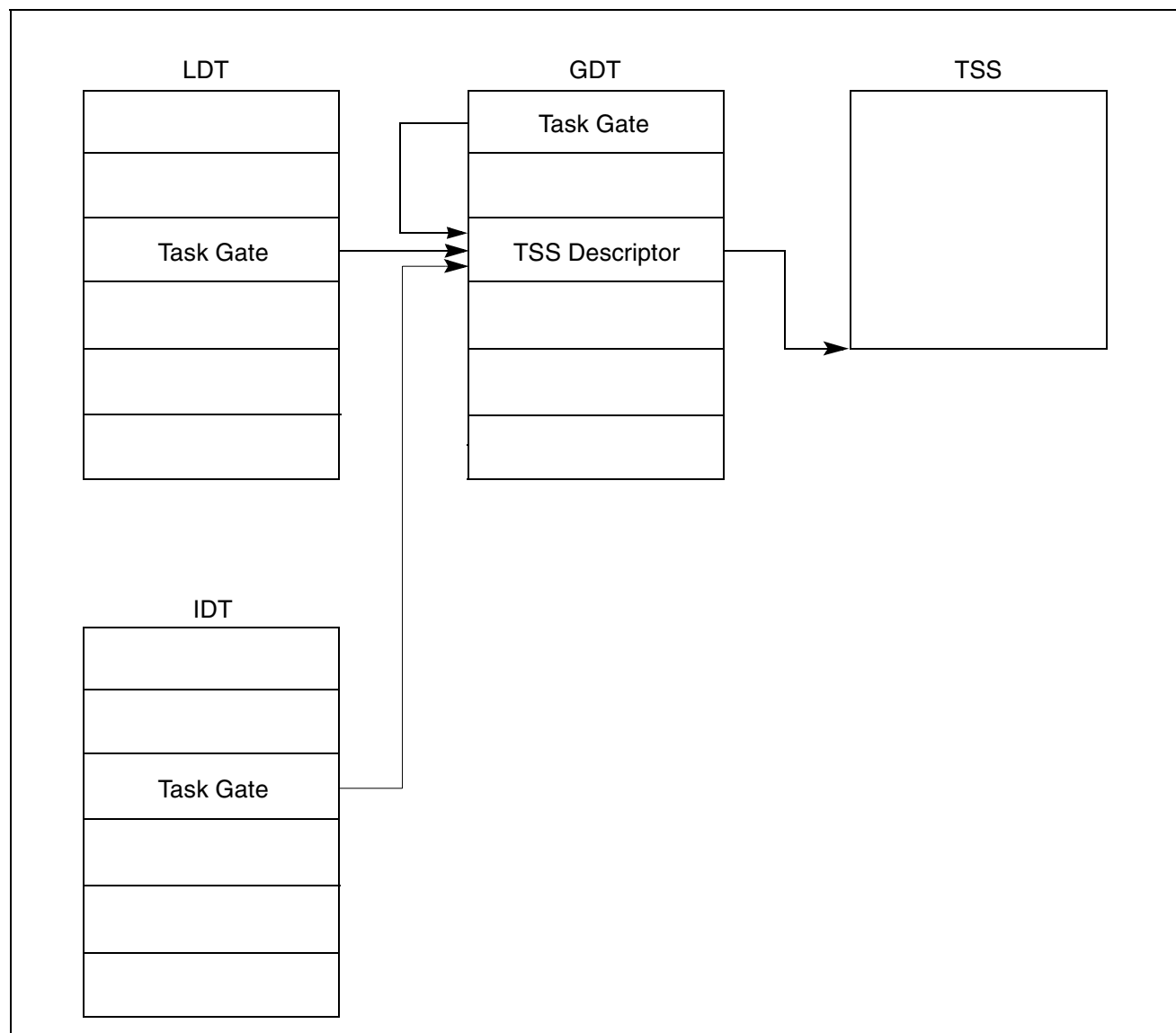


Figure 6-6. Task Gates Referencing the Same Task

6.3. TASK SWITCHING

The processor transfers execution to another task in any of four cases:

- The current program, task, or procedure executes a JMP or CALL instruction to a TSS descriptor in the GDT.
- The current program, task, or procedure executes a JMP or CALL instruction to a task-gate descriptor in the GDT or the current LDT.
- An interrupt or exception vector points to a task-gate descriptor in the IDT.
- The current task executes an IRET when the NT flag in the EFLAGS register is set.

The JMP, CALL, and IRET instructions, as well as interrupts and exceptions, are all generalized mechanisms for redirecting a program. The referencing of a TSS descriptor or a task gate (when calling or jumping to a task) or the state of the NT flag (when executing an IRET instruction) determines whether a task switch occurs.

The processor performs the following operations when switching to a new task:

1. Obtains the TSS segment selector for the new task as the operand of the JMP or CALL instruction, from a task gate, or from the previous task link field (for a task switch initiated with an IRET instruction).
2. Checks that the current (old) task is allowed to switch to the new task. Data-access privilege rules apply to JMP and CALL instructions. The CPL of the current (old) task and the RPL of the segment selector for the new task must be less than or equal to the DPL of the TSS descriptor or task gate being referenced. Exceptions, interrupts (except for interrupts generated by the INT *n* instruction), and the IRET instruction are permitted to switch tasks regardless of the DPL of the destination task-gate or TSS descriptor. For interrupts generated by the INT *n* instruction, the DPL is checked.
3. Checks that the TSS descriptor of the new task is marked present and has a valid limit (greater than or equal to 67H).
4. Checks that the new task is available (call, jump, exception, or interrupt) or busy (IRET return).
5. Checks that the current (old) TSS, new TSS, and all segment descriptors used in the task switch are paged into system memory.
6. If the task switch was initiated with a JMP or IRET instruction, the processor clears the busy (B) flag in the current (old) task's TSS descriptor; if initiated with a CALL instruction, an exception, or an interrupt, the busy (B) flag is left set. (See Table 6-2.)
7. If the task switch was initiated with an IRET instruction, the processor clears the NT flag in a temporarily saved image of the EFLAGS register; if initiated with a CALL or JMP instruction, an exception, or an interrupt, the NT flag is left unchanged in the saved EFLAGS image.
8. Saves the state of the current (old) task in the current task's TSS. The processor finds the base address of the current TSS in the task register and then copies the states of the following registers into the current TSS: all the general-purpose registers, segment selectors from the segment registers, the temporarily saved image of the EFLAGS register, and the instruction pointer register (EIP).
9. If the task switch was initiated with a CALL instruction, an exception, or an interrupt, the processor will set the NT flag in the EFLAGS loaded from the new task. If initiated with an IRET instruction or JMP instruction, the NT flag will reflect the state of NT in the EFLAGS loaded from the new task (see Table 6-2).

10. If the task switch was initiated with a CALL instruction, JMP instruction, an exception, or an interrupt, the processor sets the busy (B) flag in the new task's TSS descriptor; if initiated with an IRET instruction, the busy (B) flag is left set.
11. Loads the task register with the segment selector and descriptor for the new task's TSS.
12. The TSS state is loaded into the processor. This includes the LDTR register, the PDBR (control register CR3), the EFLAGS registers, the EIP register, the general-purpose registers, and the segment selectors. Note that a fault during the load of this state may corrupt architectural state.
13. The descriptors associated with the segment selectors are loaded and qualified. Any errors associated with this loading and qualification occur in the context of the new task.

NOTE

At this point, if all checks and saves have been carried out successfully, the processor commits to the task switch. If an unrecoverable error occurs in steps 1 through 11, the processor does not complete the task switch and insures that the processor is returned to its state prior to the execution of the instruction that initiated the task switch. If an unrecoverable error occurs in step 12, architectural state may be corrupted, but an attempt will be made to handle the error in the prior execution environment. If an unrecoverable error occurs after the commit point (in step 13), the processor completes the task switch (without performing additional access and segment availability checks) and generates the appropriate exception prior to beginning execution of the new task. If exceptions occur after the commit point, the exception handler must finish the task switch itself before allowing the processor to begin executing the new task. See Chapter 5, "Interrupt 10—Invalid TSS Exception (#TS)", for more information about the affect of exceptions on a task when they occur after the commit point of a task switch.

14. Begins executing the new task. (To an exception handler, the first instruction of the new task appears not to have been executed.)

The state of the currently executing task is always saved when a successful task switch occurs. If the task is resumed, execution starts with the instruction pointed to by the saved EIP value, and the registers are restored to the values they held when the task was suspended.

When switching tasks, the privilege level of the new task does not inherit its privilege level from the suspended task. The new task begins executing at the privilege level specified in the CPL field of the CS register, which is loaded from the TSS. Because tasks are isolated by their separate address spaces and TSSs and because privilege rules control access to a TSS, software does not need to perform explicit privilege checks on a task switch.

Table 6-1 shows the exception conditions that the processor checks for when switching tasks. It also shows the exception that is generated for each check if an error is detected and the segment that the error code references. (The order of the checks in the table is the order used in the P6 family processors. The exact order is model specific and may be different for other IA-32 processors.) Exception handlers designed to handle these exceptions may be subject to recursive

calls if they attempt to reload the segment selector that generated the exception. The cause of the exception (or the first of multiple causes) should be fixed before reloading the selector.

Table 6-1. Exception Conditions Checked During a Task Switch

Condition Checked	Exception ¹	Error Code Reference ²
Segment selector for a TSS descriptor references the GDT and is within the limits of the table.	#GP #TS (for IRET)	New Task's TSS
TSS descriptor is present in memory.	#NP	New Task's TSS
TSS descriptor is not busy (for task switch initiated by a call, interrupt, or exception).	#GP (for JMP, CALL, INT)	Task's back-link TSS
TSS descriptor is not busy (for task switch initiated by an IRET instruction).	#TS (for IRET)	New Task's TSS
TSS segment limit greater than or equal to 108 (for 32-bit TSS) or 44 (for 16-bit TSS).	#TS	New Task's TSS
Registers are loaded from the values in the TSS.		
LDT segment selector of new task is valid ³ .	#TS	New Task's LDT
Code segment DPL matches segment selector RPL.	#TS	New Code Segment
SS segment selector is valid ² .	#TS	New Stack Segment
Stack segment is present in memory.	#SF	New Stack Segment
Stack segment DPL matches CPL.	#TS	New stack segment
LDT of new task is present in memory.	#TS	New Task's LDT
CS segment selector is valid ³ .	#TS	New Code Segment
Code segment is present in memory.	#NP	New Code Segment
Stack segment DPL matches selector RPL.	#TS	New Stack Segment
DS, ES, FS, and GS segment selectors are valid ³ .	#TS	New Data Segment
DS, ES, FS, and GS segments are readable.	#TS	New Data Segment
DS, ES, FS, and GS segments are present in memory.	#NP	New Data Segment
DS, ES, FS, and GS segment DPL greater than or equal to CPL (unless these are conforming segments).	#TS	New Data Segment

NOTES:

1. #NP is segment-not-present exception, #GP is general-protection exception, #TS is invalid-TSS exception, and #SF is stack-fault exception.
2. The error code contains an index to the segment descriptor referenced in this column.
3. A segment selector is valid if it is in a compatible type of table (GDT or LDT), occupies an address within the table's segment limit, and refers to a compatible type of descriptor (for example, a segment selector in the CS register only is valid when it points to a code-segment descriptor).

The TS (task switched) flag in the control register CR0 is set every time a task switch occurs. System software uses the TS flag to coordinate the actions of floating-point unit when gener-

ating floating-point exceptions with the rest of the processor. The TS flag indicates that the context of the floating-point unit may be different from that of the current task. See Section 2.5., “Control Registers”, for a detailed description of the function and use of the TS flag.

6.4. TASK LINKING

The previous task link field of the TSS (sometimes called the “backlink”) and the NT flag in the EFLAGS register are used to return execution to the previous task. The NT flag indicates whether the currently executing task is nested within the execution of another task, and the previous task link field of the current task's TSS holds the TSS selector for the higher-level task in the nesting hierarchy, if there is one (see Figure 6-7).

When a CALL instruction, an interrupt, or an exception causes a task switch, the processor copies the segment selector for the current TSS into the previous task link field of the TSS for the new task, and then sets the NT flag in the EFLAGS register. The NT flag indicates that the previous task link field of the TSS has been loaded with a saved TSS segment selector. If software uses an IRET instruction to suspend the new task, the processor uses the value in the previous task link field and the NT flag to return to the previous task; that is, if the NT flag is set, the processor performs a task switch to the task specified in the previous task link field.

NOTE

When a JMP instruction causes a task switch, the new task is not nested; that is, the NT flag is set to 0 and the previous task link field is not used. A JMP instruction is used to dispatch a new task when nesting is not desired.

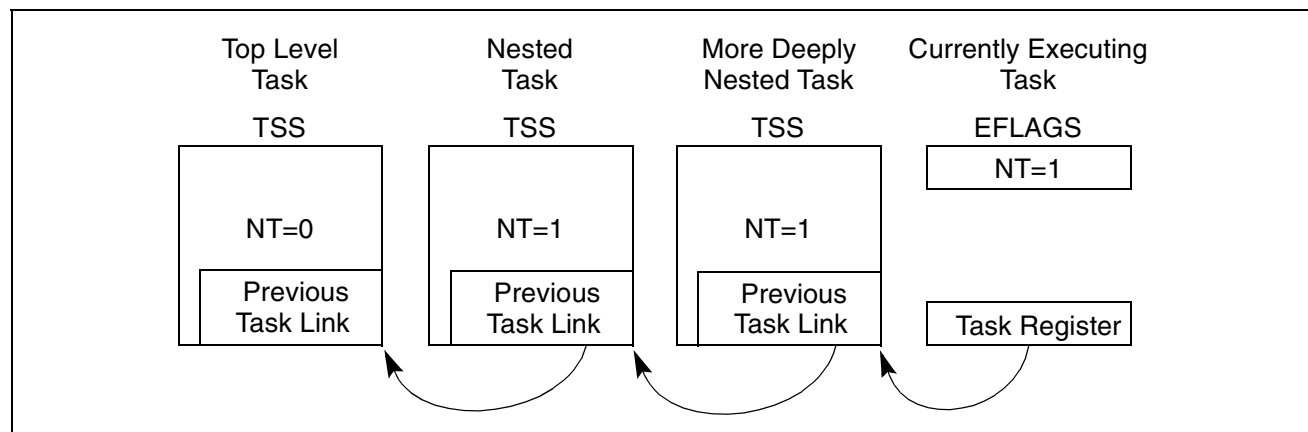


Figure 6-7. Nested Tasks

Table 6-2 summarizes the uses of the busy flag (in the TSS segment descriptor), the NT flag, the previous task link field, and TS flag (in control register CR0) during a task switch. Note that the NT flag may be modified by software executing at any privilege level. It is possible for a program to set its NT flag and execute an IRET instruction, which would have the effect of invoking the task specified in the previous link field of the current task's TSS. To keep spurious

task switches from succeeding, the operating system should initialize the previous task link field for every TSS it creates to 0.

Table 6-2. Effect of a Task Switch on Busy Flag, NT Flag, Previous Task Link Field, and TS Flag

Flag or Field	Effect of JMP instruction	Effect of CALL Instruction or Interrupt	Effect of IRET Instruction
Busy (B) flag of new task.	Flag is set. Must have been clear before.	Flag is set. Must have been clear before.	No change. Must have been set.
Busy flag of old task.	Flag is cleared.	No change. Flag is currently set.	Flag is cleared.
NT flag of new task.	Set to value from TSS of new task.	Flag is set.	Set to value from TSS of new task.
NT flag of old task.	No change.	No change.	Flag is cleared.
Previous task link field of new task.	No change.	Loaded with selector for old task's TSS.	No change.
Previous task link field of old task.	No change.	No change.	No change.
TS flag in control register CR0.	Flag is set.	Flag is set.	Flag is set.

6.4.1. Use of Busy Flag To Prevent Recursive Task Switching

A TSS allows only one context to be saved for a task; therefore, once a task is called (dispatched), a recursive (or re-entrant) call to the task would cause the current state of the task to be lost. The busy flag in the TSS segment descriptor is provided to prevent re-entrant task switching and subsequent loss of task state information. The processor manages the busy flag as follows:

1. When dispatching a task, the processor sets the busy flag of the new task.
2. If during a task switch, the current task is placed in a nested chain (the task switch is being generated by a CALL instruction, an interrupt, or an exception), the busy flag for the current task remains set.
3. When switching to the new task (initiated by a CALL instruction, interrupt, or exception), the processor generates a general-protection exception (#GP) if the busy flag of the new task is already set. (If the task switch is initiated with an IRET instruction, the exception is not raised because the processor expects the busy flag to be set.)
4. When a task is terminated by a jump to a new task (initiated with a JMP instruction in the task code) or by an IRET instruction in the task code, the processor clears the busy flag, returning the task to the “not busy” state.

In this manner the processor prevents recursive task switching by preventing a task from switching to itself or to any task in a nested chain of tasks. The chain of nested suspended tasks

may grow to any length, due to multiple calls, interrupts, or exceptions. The busy flag prevents a task from being invoked if it is in this chain.

The busy flag may be used in multiprocessor configurations, because the processor follows a LOCK protocol (on the bus or in the cache) when it sets or clears the busy flag. This lock keeps two processors from invoking the same task at the same time. (See Section 7.1.2.1., “Automatic Locking”, for more information about setting the busy flag in a multiprocessor applications.)

6.4.2. Modifying Task Linkages

In a uniprocessor system, in situations where it is necessary to remove a task from a chain of linked tasks, use the following procedure to remove the task:

1. Disable interrupts.
2. Change the previous task link field in the TSS of the pre-empting task (the task that suspended the task to be removed). It is assumed that the pre-empting task is the next task (newer task) in the chain from the task to be removed. Change the previous task link field to point to the TSS of the next oldest task in the chain or to an even older task in the chain.
3. Clear the busy (B) flag in the TSS segment descriptor for the task being removed from the chain. If more than one task is being removed from the chain, the busy flag for each task being remove must be cleared.
4. Enable interrupts.

In a multiprocessing system, additional synchronization and serialization operations must be added to this procedure to insure that the TSS and its segment descriptor are both locked when the previous task link field is changed and the busy flag is cleared.

6.5. TASK ADDRESS SPACE

The address space for a task consists of the segments that the task can access. These segments include the code, data, stack, and system segments referenced in the TSS and any other segments accessed by the task code. These segments are mapped into the processor’s linear address space, which is in turn mapped into the processor’s physical address space (either directly or through paging).

The LDT segment field in the TSS can be used to give each task its own LDT. Giving a task its own LDT allows the task address space to be isolated from other tasks by placing the segment descriptors for all the segments associated with the task in the task’s LDT.

It also is possible for several tasks to use the same LDT. This is a simple and memory-efficient way to allow some tasks to communicate with or control each other, without dropping the protection barriers for the entire system.

Because all tasks have access to the GDT, it also is possible to create shared segments accessed through segment descriptors in this table.

If paging is enabled, the CR3 register (PDBR) field in the TSS allows each task can also have its own set of page tables for mapping linear addresses to physical addresses. Or, several tasks can share the same set of page tables.

6.5.1. Mapping Tasks to the Linear and Physical Address Spaces

Tasks can be mapped to the linear address space and physical address space in either of two ways:

- One linear-to-physical address space mapping is shared among all tasks. When paging is not enabled, this is the only choice. Without paging, all linear addresses map to the same physical addresses. When paging is enabled, this form of linear-to-physical address space mapping is obtained by using one page directory for all tasks. The linear address space may exceed the available physical space if demand-paged virtual memory is supported.
- Each task has its own linear address space that is mapped to the physical address space. This form of mapping is accomplished by using a different page directory for each task. Because the PDBR (control register CR3) is loaded on each task switch, each task may have a different page directory.

The linear address spaces of different tasks may map to completely distinct physical addresses. If the entries of different page directories point to different page tables and the page tables point to different pages of physical memory, then the tasks do not share any physical addresses.

With either method of mapping task linear address spaces, the TSSs for all tasks must lie in a shared area of the physical space, which is accessible to all tasks. This mapping is required so that the mapping of TSS addresses does not change while the processor is reading and updating the TSSs during a task switch. The linear address space mapped by the GDT also should be mapped to a shared area of the physical space; otherwise, the purpose of the GDT is defeated. Figure 6-8 shows how the linear address spaces of two tasks can overlap in the physical space by sharing page tables.

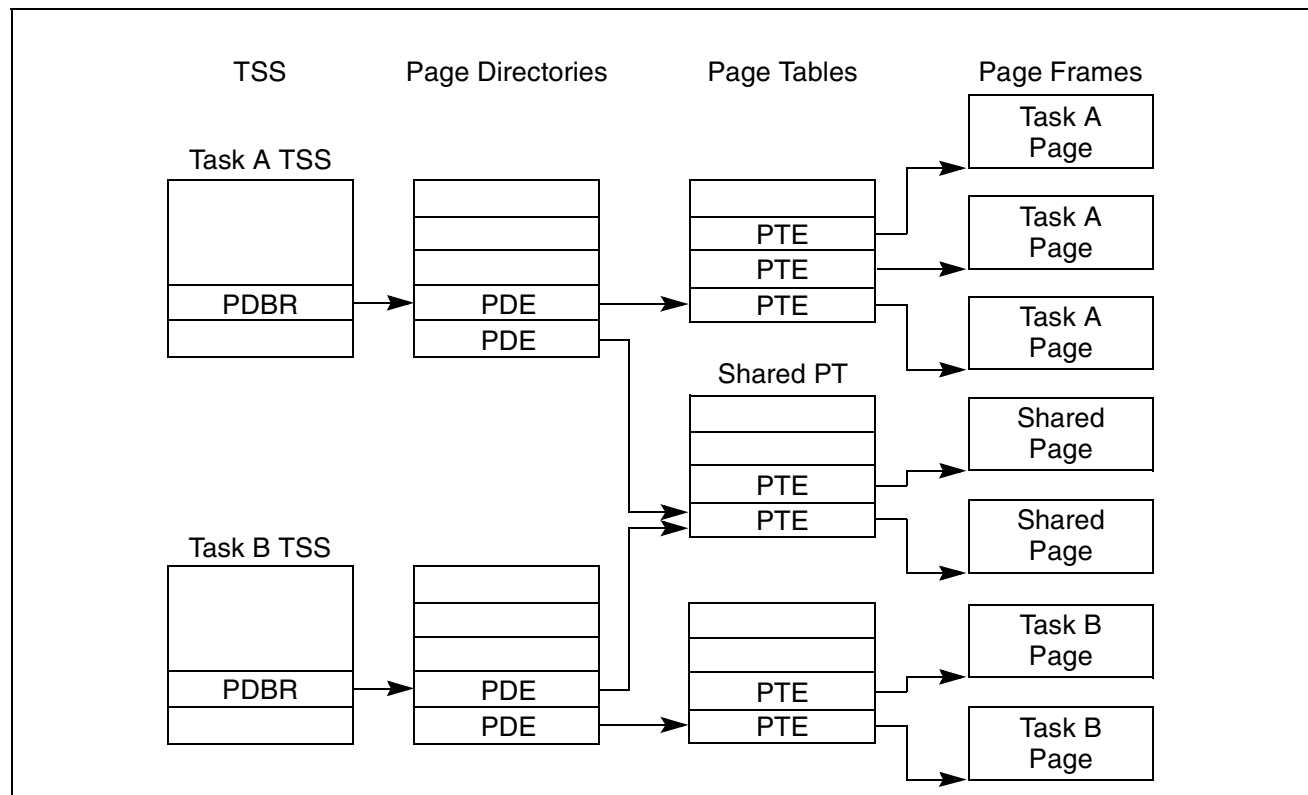


Figure 6-8. Overlapping Linear-to-Physical Mappings

6.5.2. Task Logical Address Space

To allow the sharing of data among tasks, use any of the following techniques to create shared logical-to-physical address-space mappings for data segments:

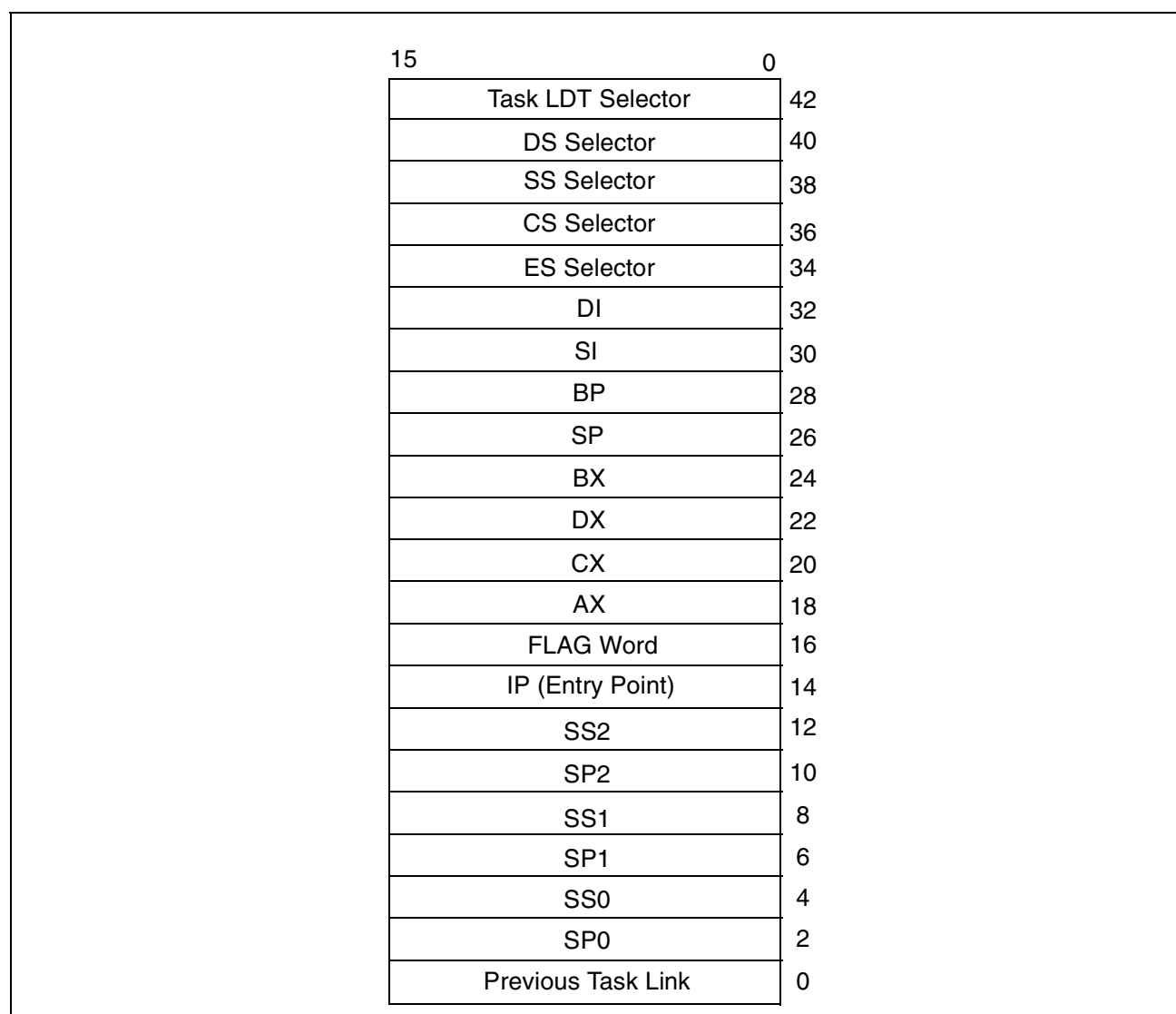
- Through the segment descriptors in the GDT. All tasks must have access to the segment descriptors in the GDT. If some segment descriptors in the GDT point to segments in the linear-address space that are mapped into an area of the physical-address space common to all tasks, then all tasks can share the data and code in those segments.
- Through a shared LDT. Two or more tasks can use the same LDT if the LDT fields in their TSSs point to the same LDT. If some segment descriptors in a shared LDT point to segments that are mapped to a common area of the physical address space, the data and code in those segments can be shared among the tasks that share the LDT. This method of sharing is more selective than sharing through the GDT, because the sharing can be limited to specific tasks. Other tasks in the system may have different LDTs that do not give them access to the shared segments.
- Through segment descriptors in distinct LDTs that are mapped to common addresses in the linear address space. If this common area of the linear address space is mapped to the same area of the physical address space for each task, these segment descriptors permit the tasks to share segments. Such segment descriptors are commonly called aliases. This method of sharing is even more selective than those listed above, because, other segment descriptors in the LDTs may point to independent linear addresses which are not shared.

6.6. 16-BIT TASK-STATE SEGMENT (TSS)

The 32-bit IA-32 processors also recognize a 16-bit TSS format like the one used in Intel 286 processors (see Figure 6-9). It is supported for compatibility with software written to run on these earlier IA-32 processors.

The following additional information is important to know about the 16-bit TSS.

- Do not use a 16-bit TSS to implement a virtual-8086 task.
- The valid segment limit for a 16-bit TSS is 2CH.
- The 16-bit TSS does not contain a field for the base address of the page directory, which is loaded into control register CR3. Therefore, a separate set of page tables for each task is not supported for 16-bit tasks. If a 16-bit task is dispatched, the page-table structure for the previous task is used.
- The I/O base address is not included in the 16-bit TSS, so none of the functions of the I/O map are supported.
- When task state is saved in a 16-bit TSS, the upper 16 bits of the EFLAGS register and the EIP register are lost.
- When the general-purpose registers are loaded or saved from a 16-bit TSS, the upper 16 bits of the registers are modified and not maintained.

**Figure 6-9. 16-Bit TSS Format**

Multiple-Processor Management

CHAPTER 7

MULTIPLE-PROCESSOR MANAGEMENT

The IA-32 architecture provides several mechanisms for managing and improving the performance of multiple processors connected to the same system bus. These mechanisms include:

- Bus locking and/or cache coherency management for performing atomic operations on system memory.
- Serializing instructions. (These instructions apply only to the Pentium 4, Intel Xeon, P6 family, and Pentium processors.)
- Advance programmable interrupt controller (APIC) located on the processor chip (see Chapter 8, *Advanced Programmable Interrupt Controller (APIC)*). The APIC architecture was introduced into the IA-32 processors with the Pentium processor.
- A second-level cache (level 2, L2). For the Pentium 4, Intel Xeon, and P6 family processors, the L2 cache is included in the processor package and is tightly coupled to the processor. For the Pentium and Intel486 processors, pins are provided to support an external L2 cache.
- A third-level cache (level 3, L3). For the Intel Xeon processors, the L3 cache is included in the processor package and is tightly coupled to the processor.
- Hyper-Threading Technology, an extension to the IA-32 architecture that enables a single processor core to execute two or more threads of execution concurrently (see Section 7.6., “Hyper-Threading Technology”).

These mechanisms are particularly useful in symmetric-multiprocessing (SMP) systems; however, they can also be used in applications where a IA-32 processor and a special-purpose processor (such as a communications, graphics, or video processor) share the system bus.

The main goals of these multiprocessing mechanisms are as follows:

- To maintain system memory coherency—When two or more processors are attempting simultaneously to access the same address in system memory, some communication mechanism or memory access protocol must be available to promote data coherency and, in some instances, to allow one processor to temporarily lock a memory location.
- To maintain cache consistency—When one processor accesses data cached in another processor, it must not receive incorrect data. If it modifies data, all other processors that access that data must receive the modified data.
- To allow predictable ordering of writes to memory—In some circumstances, it is important that memory writes be observed externally in precisely the same order as programmed.
- To distribute interrupt handling among a group of processors—When several processors are operating in a system in parallel, it is useful to have a centralized mechanism for receiving interrupts and distributing them to available processors for servicing.

- To increase system performance by exploiting the multi-threaded and multi-process nature of contemporary operating systems and applications.

The IA-32 architecture's caching mechanism and cache consistency are discussed in Chapter 10, *Memory Cache Control*. The APIC architecture is described in Chapter 8, *Advanced Programmable Interrupt Controller (APIC)*. Bus and memory locking, serializing instructions, memory ordering, and Hyper-Threading Technology are discussed in the following sections.

7.1. LOCKED ATOMIC OPERATIONS

The 32-bit IA-32 processors support locked atomic operations on locations in system memory. These operations are typically used to manage shared data structures (such as semaphores, segment descriptors, system segments, or page tables) in which two or more processors may try simultaneously to modify the same field or flag. The processor uses three interdependent mechanisms for carrying out locked atomic operations:

- Guaranteed atomic operations.
- Bus locking, using the LOCK# signal and the LOCK instruction prefix.
- Cache coherency protocols that insure that atomic operations can be carried out on cached data structures (cache lock). This mechanism is present in the Pentium 4, Intel Xeon, and P6 family processors.

These mechanisms are interdependent in the following ways. Certain basic memory transactions (such as reading or writing a byte in system memory) are always guaranteed to be handled atomically. That is, once started, the processor guarantees that the operation will be completed before another processor or bus agent is allowed access to the memory location. The processor also supports bus locking for performing selected memory operations (such as a read-modify-write operation in a shared area of memory) that typically need to be handled atomically, but are not automatically handled this way. Because frequently used memory locations are often cached in a processor's L1 or L2 caches, atomic operations can often be carried out inside a processor's caches without asserting the bus lock. Here the processor's cache coherency protocols insure that other processors that are caching the same memory locations are managed properly while atomic operations are performed on cached memory locations.

Note that the mechanisms for handling locked atomic operations have evolved as the complexity of IA-32 processors has evolved. As such, more recent IA-32 processors (such as the Pentium 4, Intel Xeon, and P6 family processors) provide a more refined locking mechanism than earlier IA-32 processors, as is described in the following sections.

7.1.1. Guaranteed Atomic Operations

The Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors guarantee that the following basic memory operations will always be carried out atomically:

- Reading or writing a byte.
- Reading or writing a word aligned on a 16-bit boundary.
- Reading or writing a doubleword aligned on a 32-bit boundary.

The Pentium 4, Intel Xeon, and P6 family, and Pentium processors guarantee that the following additional memory operations will always be carried out atomically:

- Reading or writing a quadword aligned on a 64-bit boundary.
- 16-bit accesses to uncached memory locations that fit within a 32-bit data bus.

The P6 family processors guarantee that the following additional memory operation will always be carried out atomically:

- Unaligned 16-, 32-, and 64-bit accesses to cached memory that fit within a 32-byte cache line.

Accesses to cacheable memory that are split across bus widths, cache lines, and page boundaries are not guaranteed to be atomic by the Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors. The Pentium 4, Intel Xeon, and P6 family processors provide bus control signals that permit external memory subsystems to make split accesses atomic; however, nonaligned data accesses will seriously impact the performance of the processor and should be avoided.

7.1.2. Bus Locking

IA-32 processors provide a LOCK# signal that is asserted automatically during certain critical memory operations to lock the system bus. While this output signal is asserted, requests from other processors or bus agents for control of the bus are blocked. Software can specify other occasions when the LOCK semantics are to be followed by prepending the LOCK prefix to an instruction.

In the case of the Intel386, Intel486, and Pentium processors, explicitly locked instructions will result in the assertion of the LOCK# signal. It is the responsibility of the hardware designer to make the LOCK# signal available in system hardware to control memory accesses among processors.

For the Pentium 4, Intel Xeon, and P6 family processors, if the memory area being accessed is cached internally in the processor, the LOCK# signal is generally not asserted; instead, locking is only applied to the processor's caches (see Section 7.1.4., "Effects of a LOCK Operation on Internal Processor Caches").

7.1.2.1. AUTOMATIC LOCKING

The operations on which the processor automatically follows the LOCK semantics are as follows:

- **When executing an XCHG instruction that references memory.**
- **When setting the B (busy) flag of a TSS descriptor.** The processor tests and sets the busy flag in the type field of the TSS descriptor when switching to a task. To insure that two processors do not switch to the same task simultaneously, the processor follows the LOCK semantics while testing and setting this flag.
- **When updating segment descriptors.** When loading a segment descriptor, the processor will set the accessed flag in the segment descriptor if the flag is clear. During this operation, the processor follows the LOCK semantics so that the descriptor will not be modified by another processor while it is being updated. For this action to be effective, operating-system procedures that update descriptors should use the following steps:
 - Use a locked operation to modify the access-rights byte to indicate that the segment descriptor is not-present, and specify a value for the type field that indicates that the descriptor is being updated.
 - Update the fields of the segment descriptor. (This operation may require several memory accesses; therefore, locked operations cannot be used.)
 - Use a locked operation to modify the access-rights byte to indicate that the segment descriptor is valid and present.

Note that the Intel386 processor always updates the accessed flag in the segment descriptor, whether it is clear or not. The Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors only update this flag if it is not already set.

- **When updating page-directory and page-table entries.** When updating page-directory and page-table entries, the processor uses locked cycles to set the accessed and dirty flag in the page-directory and page-table entries.
- **Acknowledging interrupts.** After an interrupt request, an interrupt controller may use the data bus to send the interrupt vector for the interrupt to the processor. The processor follows the LOCK semantics during this time to ensure that no other data appears on the data bus when the interrupt vector is being transmitted.

7.1.2.2. SOFTWARE CONTROLLED BUS LOCKING

To explicitly force the LOCK semantics, software can use the LOCK prefix with the following instructions when they are used to modify a memory location. An invalid-opcode exception (#UD) is generated when the LOCK prefix is used with any other instruction or when no write operation is made to memory (that is, when the destination operand is in a register).

- The bit test and modify instructions (BTS, BTR, and BTC).
- The exchange instructions (XADD, CMPXCHG, and CMPXCHG8B).
- The LOCK prefix is automatically assumed for XCHG instruction.

- The following single-operand arithmetic and logical instructions: INC, DEC, NOT, and NEG.
- The following two-operand arithmetic and logical instructions: ADD, ADC, SUB, SBB, AND, OR, and XOR.

A locked instruction is guaranteed to lock only the area of memory defined by the destination operand, but may be interpreted by the system as a lock for a larger memory area.

Software should access semaphores (shared memory used for signalling between multiple processors) using identical addresses and operand lengths. For example, if one processor accesses a semaphore using a word access, other processors should not access the semaphore using a byte access.

The integrity of a bus lock is not affected by the alignment of the memory field. The LOCK semantics are followed for as many bus cycles as necessary to update the entire operand. However, it is recommended that locked accesses be aligned on their natural boundaries for better system performance:

- Any boundary for an 8-bit access (locked or otherwise).
- 16-bit boundary for locked word accesses.
- 32-bit boundary for locked doubleword access.
- 64-bit boundary for locked quadword access.

Locked operations are atomic with respect to all other memory operations and all externally visible events. Only instruction fetch and page table accesses can pass locked instructions. Locked instructions can be used to synchronize data written by one processor and read by another processor.

For the P6 family processors, locked operations serialize all outstanding load and store operations (that is, wait for them to complete). This rule is also true for the Pentium 4 and Intel Xeon processors, with one exception: load operations that reference weakly ordered memory types (such as the WC memory type) may not be serialized.

Locked instructions should not be used to insure that data written can be fetched as instructions.

NOTE

The locked instructions for the current versions of the Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors allow data written to be fetched as instructions. However, Intel recommends that developers who require the use of self-modifying code use a different synchronizing mechanism, described in the following sections.

7.1.3. Handling Self- and Cross-Modifying Code

The act of a processor writing data into a currently executing code segment with the intent of executing that data as code is called **self-modifying code**. IA-32 processors exhibit model-specific behavior when executing self-modified code, depending upon how far ahead of the current execution pointer the code has been modified. As processor architectures become more complex and start to speculatively execute code ahead of the retirement point (as in the Pentium 4, Intel Xeon, and P6 family processors), the rules regarding which code should execute, pre- or post-modification, become blurred. To write self-modifying code and ensure that it is compliant with current and future versions of the IA-32 architecture, one of the following two coding options must be chosen.

(* OPTION 1 *)

Store modified code (as data) into code segment;
Jump to new code or an intermediate location;
Execute new code;

(* OPTION 2 *)

Store modified code (as data) into code segment;
Execute a serializing instruction; (* For example, CPUID instruction *)
Execute new code;

(The use of one of these options is not required for programs intended to run on the Pentium or Intel486 processors, but are recommended to insure compatibility with the Pentium 4, Intel Xeon, and P6 family processors.)

It should be noted that self-modifying code will execute at a lower level of performance than non-self-modifying or normal code. The degree of the performance deterioration will depend upon the frequency of modification and specific characteristics of the code.

The act of one processor writing data into the currently executing code segment of a second processor with the intent of having the second processor execute that data as code is called **cross-modifying code**. As with self-modifying code, IA-32 processors exhibit model-specific behavior when executing cross-modifying code, depending upon how far ahead of the executing processors current execution pointer the code has been modified. To write cross-modifying code and insure that it is compliant with current and future versions of the IA-32 architecture, the following processor synchronization algorithm must be implemented.

; Action of Modifying Processor

Memory_Flag $\leftarrow$ 0; (* Set Memory_Flag to value other than 1 *)

Store modified code (as data) into code segment;

Memory_Flag $\leftarrow$ 1;

; Action of Executing Processor

WHILE (Memory_Flag $\neq$ 1)

 Wait for code to update;

ELIHW;

Execute serializing instruction; (* For example, CPUID instruction *)

Begin executing modified code;

(The use of this option is not required for programs intended to run on the Intel486 processor, but is recommended to insure compatibility with the Pentium 4, Intel Xeon, P6 family, and Pentium processors.)

Like self-modifying code, cross-modifying code will execute at a lower level of performance than non-cross-modifying (normal) code, depending upon the frequency of modification and specific characteristics of the code.

7.1.4. Effects of a LOCK Operation on Internal Processor Caches

For the Intel486 and Pentium processors, the LOCK# signal is always asserted on the bus during a LOCK operation, even if the area of memory being locked is cached in the processor.

For the Pentium 4, Intel Xeon, and P6 family processors, if the area of memory being locked during a LOCK operation is cached in the processor that is performing the LOCK operation as write-back memory and is completely contained in a cache line, the processor may not assert the LOCK# signal on the bus. Instead, it will modify the memory location internally and allow its cache coherency mechanism to insure that the operation is carried out atomically. This operation is called “cache locking.” The cache coherency mechanism automatically prevents two or more processors that have cached the same area of memory from simultaneously modifying data in that area.

7.2. MEMORY ORDERING

The term **memory ordering** refers to the order in which the processor issues reads (loads) and writes (stores) through the system bus to system memory. The IA-32 architecture supports several memory ordering models depending on the implementation of the architecture. For example, the Intel386 processor enforces **program ordering** (generally referred to as **strong ordering**), where reads and writes are issued on the system bus in the order they occur in the instruction stream under all circumstances.

To allow optimizing of instruction execution, the IA-32 architecture allows departures from strong-ordering model called **processor ordering** in Pentium 4, Intel Xeon, and P6 family processors. These **processor-ordering** variations allow performance enhancing operations such as allowing reads to go ahead of buffered writes. The goal of any of these variations is to increase instruction execution speeds, while maintaining memory coherency, even in multiple-processor systems.

The following sections describe the memory ordering models used by the Intel486 and Pentium processors, and by the Pentium 4, Intel Xeon, and P6 family processors.

7.2.1. Memory Ordering in the Pentium® and Intel486™ Processors

The Pentium and Intel486 processors follow the processor-ordered memory model; however, they operate as strongly-ordered processors under most circumstances. Reads and writes always appear in programmed order at the system bus—except for the following situation where processor ordering is exhibited. Read misses are permitted to go ahead of buffered writes on the system bus when all the buffered writes are cache hits and, therefore, are not directed to the same address being accessed by the read miss.

In the case of I/O operations, both reads and writes always appear in programmed order.

Software intended to operate correctly in processor-ordered processors (such as the Pentium 4, Intel Xeon, and P6 family processors) should not depend on the relatively strong ordering of the Pentium or Intel486 processors. Instead, it should insure that accesses to shared variables that are intended to control concurrent execution among processors are explicitly required to obey program ordering through the use of appropriate locking or serializing operations (see Section 7.2.4., “Strengthening or Weakening the Memory Ordering Model”).

7.2.2. Memory Ordering Pentium 4, Intel® Xeon™, and P6 Family Processors

The Pentium 4, Intel Xeon, and P6 family processors also use a processor-ordered memory ordering model that can be further defined as “write ordered with store-buffer forwarding.” This model can be characterized as follows.

In a single-processor system for memory regions defined as write-back cacheable, the following ordering rules apply:

1. Reads can be carried out speculatively and in any order.
2. Reads can pass buffered writes, but the processor is self-consistent.
3. Writes to memory are always carried out in program order, with the exception of writes executed with the CLFLUSH instruction and streaming stores (writes) executed with the non-temporal move instructions (MOVNTI, MOVNTQ, MOVNTDQ, MOVNTPS, and MOVNTPD).
4. Writes can be buffered.
5. Writes are not performed speculatively; they are only performed for instructions that have actually been retired.
6. Data from buffered writes can be forwarded to waiting reads within the processor.
7. Reads or writes cannot pass (be carried out ahead of) I/O instructions, locked instructions, or serializing instructions.
8. Reads cannot pass LFENCE and MFENCE instructions.
9. Writes cannot pass SFENCE and MFENCE instructions.

The second rule allows a read to pass a write. However, if the write is to the same memory location as the read, the processor's internal "snooping" mechanism will detect the conflict and update the already cached read before the processor executes the instruction that uses the value.

The sixth rule constitutes an exception to an otherwise write ordered model.

Note that the term "write ordered with store-buffer forwarding" (introduced at the beginning of this section) refers to the combined effects of rules 2 and 6.

In a multiple-processor system, the following ordering rules apply:

- Individual processors use the same ordering rules as in a single-processor system.
- Writes by a single processor are observed in the same order by all processors.
- Writes from the individual processors on the system bus are NOT ordered with respect to each other.

The latter rule can be clarified by the example in Figure 7-1. Consider three processors in a system and each processor performs three writes, one to each of three defined locations (A, B, and C). Individually, the processors perform the writes in the same program order, but because of bus arbitration and other memory access mechanisms, the order that the three processors write the individual memory locations can differ each time the respective code sequences are executed on the processors. The final values in location A, B, and C would possibly vary on each execution of the write sequence.

The processor-ordering model described in this section is virtually identical to that used by the Pentium and Intel486 processors. The only enhancements in the Pentium 4, Intel Xeon, and P6 family processors are:

- Added support for speculative reads.
- Store-buffer forwarding, when a read passes a write to the same memory location.
- Out of order store from long string store and string move operations (see Section 7.2.3., "Out-of-Order Stores For String Operations in Pentium 4, Intel Xeon, and P6 Family Processors", below).

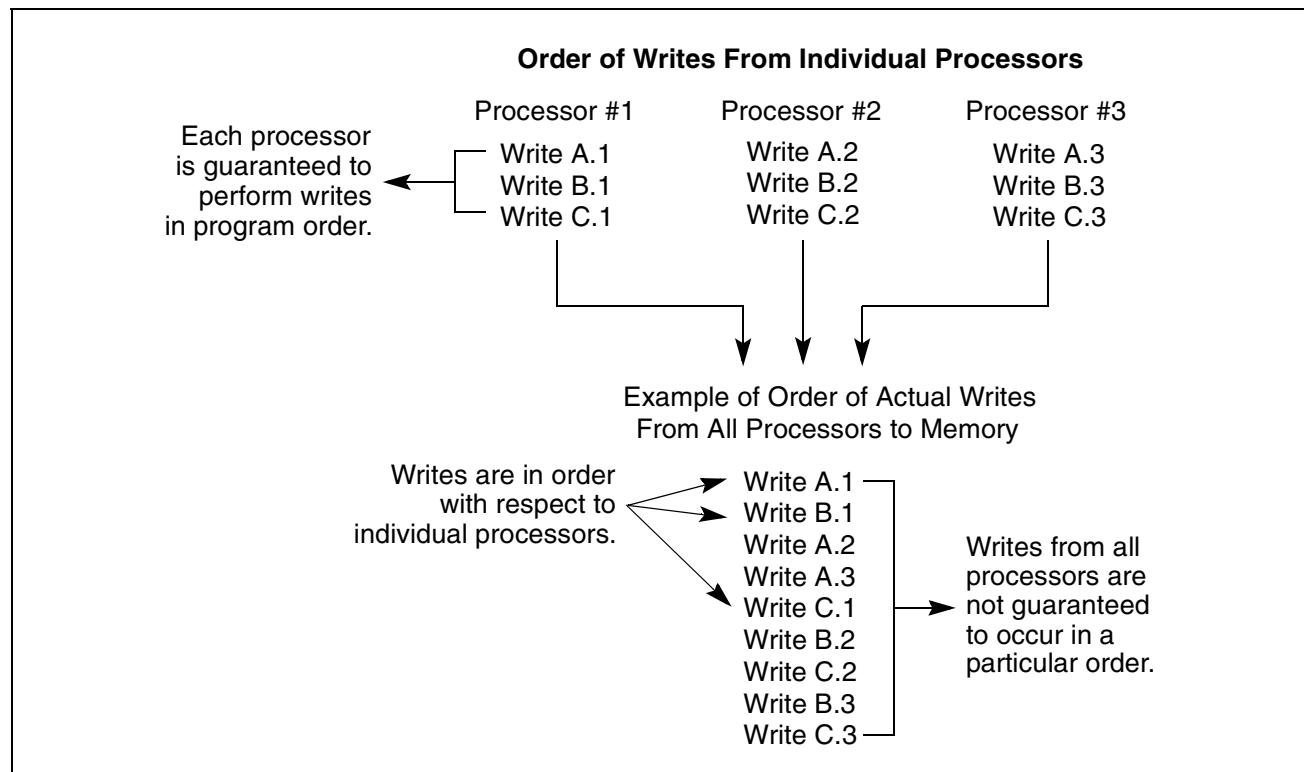


Figure 7-1. Example of Write Ordering in Multiple-Processor Systems

7.2.3. Out-of-Order Stores For String Operations in Pentium 4, Intel Xeon, and P6 Family Processors

The Pentium 4, Intel Xeon, and P6 family processors modify the processors operation during the string store operations (initiated with the MOVS and STOS instructions) to maximize performance. Once the “fast string” operations initial conditions are met (as described below), the processor will essentially operate on, from an external perspective, the string in a cache line by cache line mode. This results in the processor looping on issuing a cache-line read for the source address and an invalidation on the external bus for the destination address, knowing that all bytes in the destination cache line will be modified, for the length of the string. In this mode interrupts will only be accepted by the processor on cache line boundaries. It is possible in this mode that the destination line invalidations, and therefore stores, will be issued on the external bus out of order.

Code dependent upon sequential store ordering should not use the string operations for the entire data structure to be stored. Data and semaphores should be separated. Order dependent code should use a discrete semaphore uniquely stored to after any string operations to allow correctly ordered data to be seen by all processors.

Initial conditions for “fast string” operations:

- EDI and ESI must be 8-byte aligned for the Pentium III processor. EDI must be 8-byte aligned for the Pentium 4 processor.
- String operation must be performed in ascending address order.

- The initial operation counter (ECX) must be equal to or greater than 64.
- Source and destination must not overlap by less than a cache line (64 bytes, Pentium 4 and Intel Xeon processors; 32 bytes P6 family and Pentium processors).
- The memory type for both source and destination addresses must be either WB or WC.

7.2.4. Strengthening or Weakening the Memory Ordering Model

The IA-32 architecture provides several mechanisms for strengthening or weakening the memory ordering model to handle special programming situations. These mechanisms include:

- The I/O instructions, locking instructions, the LOCK prefix, and serializing instructions force stronger ordering on the processor.
- The SFENCE instruction (introduced to the IA-32 architecture in the Pentium III processor) and the LFENCE and MFENCE instructions (introduced in the Pentium 4 and Intel Xeon processors) provide memory ordering and serialization capability for specific types of memory operations.
- The memory type range registers (MTRRs) can be used to strengthen or weaken memory ordering for specific area of physical memory (see Section 10.11., “Memory Type Range Registers (MTRRs)”). MTRRs are available only in the Pentium 4, Intel Xeon, and P6 family processors.
- The page attribute table (PAT) can be used to strengthen memory ordering for a specific page or group of pages (see Section 10.12., “Page Attribute Table (PAT)”). The PAT is available only in the Pentium 4, Intel Xeon, and Pentium III processors.

These mechanisms can be used as follows.

Memory mapped devices and other I/O devices on the bus are often sensitive to the order of writes to their I/O buffers. I/O instructions can be used to (the IN and OUT instructions) impose strong write ordering on such accesses as follows. Prior to executing an I/O instruction, the processor waits for all previous instructions in the program to complete and for all buffered writes to drain to memory. Only instruction fetch and page tables walks can pass I/O instructions. Execution of subsequent instructions do not begin until the processor determines that the I/O instruction has been completed.

Synchronization mechanisms in multiple-processor systems may depend upon a strong memory-ordering model. Here, a program can use a locking instruction such as the XCHG instruction or the LOCK prefix to insure that a read-modify-write operation on memory is carried out atomically. Locking operations typically operate like I/O operations in that they wait for all previous instructions to complete and for all buffered writes to drain to memory (see Section 7.1.2., “Bus Locking”).

Program synchronization can also be carried out with serializing instructions (see Section 7.4., “Serializing Instructions”). These instructions are typically used at critical procedure or task boundaries to force completion of all previous instructions before a jump to a new section of code or a context switch occurs. Like the I/O and locking instructions, the processor waits until all previous instructions have been completed and all buffered writes have been drained to memory before executing the serializing instruction.

The SFENCE, LFENCE, and MFENCE instructions provide a performance-efficient way of insuring load and store memory ordering between routines that produce weakly-ordered results and routines that consume that data. The functions of these instructions are as follows:

- SFENCE—Serializes all store (write) operations that occurred prior to the SFENCE instruction in the program instruction stream, but does not affect load operations.
- LFENCE—Serializes all load (read) operations that occurred prior to the LFENCE instruction in the program instruction stream, but does not affect store operations.
- MFENCE—Serializes all store and load operations that occurred prior to the MFENCE instruction in the program instruction stream.

Note that the SFENCE, LFENCE, and MFENCE instructions provide a more efficient method of controlling memory ordering than the CPUID instruction.

The MTRRs were introduced in the P6 family processors to define the cache characteristics for specified areas of physical memory. The following are two examples of how memory types set up with MTRRs can be used strengthen or weaken memory ordering for the Pentium 4, Intel Xeon, and P6 family processors:

- The strong uncached (UC) memory type forces a strong-ordering model on memory accesses. Here, all reads and writes to the UC memory region appear on the bus and out-of-order or speculative accesses are not performed. This memory type can be applied to an address range dedicated to memory mapped I/O devices to force strong memory ordering.
- For areas of memory where weak ordering is acceptable, the write back (WB) memory type can be chosen. Here, reads can be performed speculatively and writes can be buffered and combined. For this type of memory, cache locking is performed on atomic (locked) operations that do not split across cache lines, which helps to reduce the performance penalty associated with the use of the typical synchronization instructions, such as XCHG, that lock the bus during the entire read-modify-write operation. With the WB memory type, the XCHG instruction locks the cache instead of the bus if the memory access is contained within a cache line.

The PAT was introduced in the Pentium III processor to enhance the caching characteristics that can be assigned to pages or groups of pages. The PAT mechanism typically used to strengthen caching characteristics at the page level with respect to the caching characteristics established by the MTRRs. Table 10-7 shows the interaction of the PAT with the MTRRs.

It is recommended that software written to run on Pentium 4, Intel Xeon, and P6 family processors assume the processor-ordering model or a weaker memory-ordering model. The Pentium 4, Intel Xeon, and P6 family processors do not implement a strong memory-ordering model, except when using the UC memory type. Despite the fact that Pentium 4, Intel Xeon, and P6 family processors support processor ordering, Intel does not guarantee that future processors will support this model. To make software portable to future processors, it is recommended that operating systems provide critical region and resource control constructs and API's (application program interfaces) based on I/O, locking, and/or serializing instructions be used to synchronize access to shared areas of memory in multiple-processor systems. Also, software should not depend on processor ordering in situations where the system hardware does not support this memory-ordering model.

7.3. PROPAGATION OF PAGE TABLE AND PAGE DIRECTORY ENTRY CHANGES TO MULTIPLE PROCESSORS

In a multiprocessor system, when one processor changes a page table or page directory entry, the changes must also be propagated to all the other processors. This process is commonly referred to as “TLB shutdown.” The propagation of changes to page table or page directory entries can be done using memory-based semaphores and/or interprocessor interrupts (IPI) between processors. For example, a simple but algorithmic correct TLB shutdown sequence for a IA-32 processor is as follows:

1. Begin barrier—Stop all but one processor; that is, cause all but one to HALT or stop in a spin loop.
2. Let the active processor change the necessary PTEs and/or PDEs.
3. Let all processors invalidate the PTEs and PDEs modified in their TLBs.
4. End barrier—Resume all processors; resume general processing.

Alternate, performance-optimized, TLB shutdown algorithms may be developed; however, care must be taken by the developers to ensure that either of the following conditions are met:

- Different TLB mappings are not used on different processors during the update process.
- The operating system is prepared to deal with the case where processors are using the stale mapping during the update process.

7.4. SERIALIZING INSTRUCTIONS

The IA-32 architecture defines several **serializing instructions**. These instructions force the processor to complete all modifications to flags, registers, and memory by previous instructions and to drain all buffered writes to memory before the next instruction is fetched and executed. For example, when a MOV to control register instruction is used to load a new value into control register CR0 to enable protected mode, the processor must perform a serializing operation before it enters protected mode. This serializing operation insures that all operations that were started while the processor was in real-address mode are completed before the switch to protected mode is made.

The concept of serializing instructions was introduced into the IA-32 architecture with the Pentium processor to support parallel instruction execution. Serializing instructions have no meaning for the Intel486 and earlier processors that do not implement parallel instruction execution.

It is important to note that executing of serializing instructions on Pentium 4, Intel Xeon, and P6 family processors constrain speculative execution, because the results of speculatively executed instructions are discarded.

The following instructions are serializing instructions:

- Privileged serializing instructions—MOV (to control register), MOV (to debug register), WRMSR, INVD, INVLPG, WBINVD, LGDT, LLDT, LIDT, and LTR.
- Non-privileged serializing instructions—CUID, IRET, and RSM.
- Non-privileged memory ordering instructions—SFENCE, LFENCE, and MFENCE.

When the processor serializes instruction execution, it ensures that all pending memory transactions are completed, including writes stored in its store buffer, before it executes the next instruction. Nothing can pass a serializing instruction, and serializing instructions cannot pass any other instruction (read, write, instruction fetch, or I/O).

The CUID instruction can be executed at any privilege level to serialize instruction execution with no effect on program flow, except that the EAX, EBX, ECX, and EDX registers are modified.

The SFENCE, LFENCE, and MFENCE instructions provide more granularity in controlling the serialization of memory loads and stores (see Section 7.2.4., “Strengthening or Weakening the Memory Ordering Model”).

The following additional information is worth noting regarding serializing instructions:

- The processor does not writeback the contents of modified data in its data cache to external memory when it serializes instruction execution. Software can force modified data to be written back by executing the WBINVD instruction, which is a serializing instruction. It should be noted that frequent use of the WBINVD instruction will seriously reduce system performance.

- When an instruction is executed that enables or disables paging (that is, changes the PG flag in control register CR0), the instruction should be followed by a jump instruction. The target instruction of the jump instruction is fetched with the new setting of the PG flag (that is, paging is enabled or disabled), but the jump instruction itself is fetched with the previous setting. The Pentium 4, Intel Xeon, and P6 family processors do not require the jump operation following the move to register CR0 (because any use of the MOV instruction in a Pentium 4, Intel Xeon, or P6 family processor to write to CR0 is completely serializing). However, to maintain backwards and forward compatibility with code written to run on other IA-32 processors, it is recommended that the jump operation be performed.
- Whenever an instruction is executed to change the contents of CR3 while paging is enabled, the next instruction is fetched using the translation tables that correspond to the new value of CR3. Therefore the next instruction and the sequentially following instructions should have a mapping based upon the new value of CR3. (Global entries in the TLBs are not invalidated, see Section 10.9., “Invalidating the Translation Lookaside Buffers (TLBs)”.)
- The Pentium 4, Intel Xeon, P6 family, and Pentium processors use branch-prediction techniques to improve performance by prefetching the destination of a branch instruction before the branch instruction is executed. Consequently, instruction execution is not deterministically serialized when a branch instruction is executed.

7.5. MULTIPLE-PROCESSOR (MP) INITIALIZATION

The IA-32 architecture (beginning with the P6 family processors) defines a multiple-processor (MP) initialization protocol called the *Multiprocessor Specification Version 1.4*. This specification defines the boot protocol to be used by IA-32 processors in multiple-processor systems. (Here, **multiple processors** is defined as two or more processors.) The MP initialization protocol has the following important features:

- It supports controlled booting of multiple processors without requiring dedicated system hardware.
- It allows hardware to initiate the booting of a system without the need for a dedicated signal or a predefined boot processor.
- It allows all IA-32 processors to be booted in the same manner, including those with Hyper-Threading Technology.

The mechanism for carrying out the MP initialization protocol differs depending on the IA-32 processor family, as follows:

- For P6 family processors—The selection of the BSP and APs (see Section 7.5.1., “BSP and AP Processors”) is handled through arbitration on the APIC bus, using BIPI and FIPI messages. See Appendix C for a complete discussion of MP initialization for P6 family processors.

- Intel Xeon processors with family, model, and stepping IDs up to F09H—The selection of the BSP and APs (see Section 7.5.1., “BSP and AP Processors”) is handled through arbitration on the system bus, using BIPI and FIPI messages. See Section 7.5.3., “MP Initialization Protocol Algorithm for the Intel Xeon Processors” for a complete discussion of MP initialization for Intel Xeon processors.
- Intel Xeon processors with family, model, and stepping IDs of F0AH and beyond—The selection of the BSP and APs is handled through a special system bus cycle, without using BIPI and FIPI message arbitration. This method of selection is also described in Section 7.5.3., “MP Initialization Protocol Algorithm for the Intel Xeon Processors”.

The family, model, and stepping ID for a processor is given in the EAX register when the CPUID instruction is executed with a value of 1 in the EAX register.

7.5.1. BSP and AP Processors

The MP initialization protocol defines two classes of processors: the bootstrap processor (BSP) and the application processors (APs). Following a power-up or RESET of an MP system, system hardware dynamically selects one of the processors on the system bus as the BSP. The remaining processors are designated as APs.

As part of the BSP selection mechanism, the BSP flag is set in the IA32_APIC_BASE MSR (see Figure 8-5) of the BSP, indicating that it is the BSP. This flag is cleared for all other processors.

The BSP executes the BIOS’s boot-strap code to configure the APIC environment, sets up system-wide data structures, and starts and initializes the APs. When the BSP and APs are initialized, the BSP then begins executing the operating-system initialization code.

Following a power-up or reset, the APs complete a minimal self-configuration, then wait for a startup signal (a SIPI message) from the BSP processor. Upon receiving a SIPI message, an AP executes the BIOS AP configuration code, which ends with the AP being placed in halt state.

In IA-32 processors with Hyper-Threading Technology, the MP initialization protocol treats each of the logical processors on the system bus as a separate processor (with a unique APIC ID). During boot-up, one of the logical processors is selected as the BSP and the remainder of the logical processors are designated as APs.

7.5.2. MP Initialization Protocol Requirements and Restrictions for Intel Xeon Processors

The MP initialization protocol imposes the following requirements and restrictions on the system:

- The MP protocol is executed only after a power-up or RESET. If the MP protocol has completed and a BSP is chosen, subsequent INITs (either to a specific processor or system wide) do not cause the MP protocol to be repeated. Instead, each processor examines its BSP flag (in the IA32_APIC_BASE MSR) to determine whether it should execute the BIOS boot-strap code (if it is the BSP) or enter a wait-for-SIPI state (if it is an AP).

- All devices in the system that are capable of delivering interrupts to the processors must be inhibited from doing so for the duration of the MP initialization protocol. The time during which interrupts must be inhibited includes the window between when the BSP issues an INIT-SIPI-SIPI sequence to an AP and when the AP responds to the last SIPI in the sequence.

7.5.3. MP Initialization Protocol Algorithm for the Intel Xeon Processors

Following a power-up or RESET of an MP system, the Intel Xeon processors in the system execute the MP initialization protocol algorithm to initialize each of the processors on the system bus. In the course of executing this algorithm, the following boot-up and initialization operations are carried out:

1. Each processor on the system bus is assigned a unique 8-bit APIC ID, based on system topology (see Section 7.5.5., “Identifying the Processors in an MP System”). This ID is written into the local APIC ID register for each processor.
2. Each processor is assigned a unique arbitration priority based on its APIC ID.
3. Each processor executes its internal BIST simultaneously with the other processors on the system bus.
4. Upon completion of the BIST, the processors use a hardware-defined selection mechanism to select the BSP and the APs from the available processors on the system bus. The BSP selection mechanism differs depending on the family, model, and stepping IDs of the processors, as follows:
 - Family, model, and stepping IDs of F0AH and onwards:
 - The processors begin monitoring the BNR# signal, which is toggling. When the BNR# pin stops toggling, each processor attempts to issue a NOP special cycle on the system bus.
 - The processor with the highest arbitration priority succeeds in issuing a NOP special cycle and is nominated the BSP. This processor sets the BSP flag in its IA32_APIC_BASE MSR, then fetches and begins executing BIOS boot-strap code, beginning at the reset vector (physical address FFFF FFF0H).
 - The remaining processors (that failed in issuing a NOP special cycle) are designated as APs. They leave their BSP flags in the clear state and enter a “wait-for-SIPI state.”
 - Family, model, and stepping IDs up to F09H:
 - Each processor broadcasts a BIPI to “all including self.” The first processor that broadcasts a BIPI (and thus receives its own BIPI vector), selects itself as the BSP and sets the BSP flag in its IA32_APIC_BASE MSR. (See Section C.1., “Overview of the MP Initialization Process For P6 Family Processors”, for a description of the BIPI, FIPI, and SIPI messages.)

- The remainder of the processors (which were not selected as the BSP) are designated as APs. They leave their BSP flags in the clear state and enter a “wait-for-SIPI state.”
 - The newly established BSP broadcasts an FIPI message to “all including self,” which the BSP and APs treat as an end of MP initialization signal. Only the processor with its BSP flag set responds to the FIPI message. It responds by fetching and executing the BIOS boot-strap code, beginning at the reset vector (physical address FFFF FFF0H).
5. As part of the boot-strap code, the BSP creates an ACPI table and an MP table and adds its initial APIC ID to these tables as appropriate.
 6. At the end of the boot-strap procedure, the BSP sets a processor counter to 1, then broadcasts a SIPI message to all the APs in the system. Here, the SIPI message contains a vector to the BIOS AP initialization code (at 000VV000H, where VV is the vector contained in the SIPI message).
 7. The first action of the AP initialization code is to set up a race (among the APs) to a BIOS initialization semaphore. The first AP to the semaphore begins executing the initialization code. (See Section 7.5.4., “MP Initialization Example”, for semaphore implementation details.) As part of the AP initialization procedure, the AP adds its APIC ID number to the ACPI and MP tables as appropriate and increments the processor counter by 1. At the completion of the initialization procedure, the AP executes a CLI instruction and halts itself.
 8. When each of the APs has gained access to the semaphore and executed the AP initialization code, the BSP establishes a count for the number of processors connected to the system bus, completes executing the BIOS boot-strap code, and then begins executing operating-system boot-strap and start-up code.
 9. While the BSP is executing operating-system boot-strap and start-up code, the APs remain in the halted state. In this state they will respond only to INITs, NMIs, and SMIs. They will also respond to snoops and to assertions of the STPCLK# pin.

The following section gives an example (with code) of the MP initialization protocol for multiple Intel Xeon processors operating in an MP configuration.

Appendix D, *Programming the LINT0 and LINT1 Inputs*, describes how to program the LINT[0:1] pins of the processor’s local APICs after an MP configuration has been completed.

7.5.4. MP Initialization Example

The following example illustrates the use of the MP initialization protocol to initialize IA-32 processors in an MP system after the BSP and APs have been established. This code runs successfully on any IA-32 processor that uses the MP initialization protocol, which include the P6 family processors and the Intel Xeon processors (with and without Hyper-Threading Technology).

The following constants and data definitions are used in the accompanying code examples. They are based on the addresses of the APIC registers as defined in Table 8-1.

ICR_LOW	EQU 0FEE00300H
SVR	EQU 0FEE000F0H
APIC_ID	EQU 0FEE00020H
LVT3	EQU 0FEE00370H
APIC_ENABLED	EQU 0100H
BOOT_ID	DD ?
COUNT	EQU 00H
VACANT	EQU 00H

7.5.4.1. TYPICAL BSP INITIALIZATION SEQUENCE

After the BSP and APs have been selected (by means of a hardware protocol, see Section 7.5.3., “MP Initialization Protocol Algorithm for the Intel Xeon Processors”), the BSP begins executing BIOS boot-strap code (POST) at the normal IA-32 architecture starting address (FFFF FFF0H). The boot-strap code typically performs the following operations:

1. Initializes memory.
2. Loads the microcode update into the processor.
3. Initializes the MTRRs.
4. Enables the caches.
5. Executes the CUID instruction with a value of 0H in the EAX register, then reads the EBX, ECX, and EDX registers to determine if the BSP is “GenuineIntel.”
6. Executes the CUID instruction with a value of 1H in the EAX register, then saves the values in the EAX, ECX, and EDX registers in a system configuration space in RAM for use later.
7. Loads start-up code for the AP to execute into a 4-KByte page in the lower 1 MByte of memory.
8. Switches to protected mode and insures that the APIC address space is mapped to the strong uncacheable (UC) memory type.
9. Determine the BSP’s APIC ID from the local APIC ID register (default is 0):

```

MOV ESI, APIC_ID      ; address of local APIC ID register
MOV EAX, [ESI]
AND EAX, 0FF000000H   ; zero out all other bits except APIC ID
MOV BOOT_ID, EAX      ; save in memory

```

Saves the APIC ID in the ACPI and MP tables and optionally in the system configuration space in RAM.

10. Converts the base address of the 4-KByte page for the AP's bootup code into 8-bit vector. The 8-bit vector defines the address of a 4-KByte page in the real-address mode address space (1-MByte space). For example, a vector of 0BDH specifies a start-up memory address of 000BD000H.
11. Enables the local APIC by setting bit 8 of the APIC spurious vector register (SVR).

```
MOV ESI, SVR          ; address of SVR
MOV EAX, [ESI]
OR  EAX, APIC_ENABLED ; set bit 8 to enable (0 on reset)
MOV [ESI], EAX
```

12. Sets up the LVT error handling entry by establishing an 8-bit vector for the APIC error handler.

```
MOV ESI, LVT3
MOV EAX, [ESI]
AND EAX, FFFFFFF00H; clear out previous vector
OR  EAX, 000000xxH; xx is the 8-bit vector the APIC error
                  ; handler.
MOV [ESI], EAX
```

13. Initializes the Lock Semaphore variable VACANT to 00H. The APs use this semaphore to determine the order in which they execute BIOS AP initialization code.
14. Performs the following operation to set up the BSP to detect the presence of APs in the system and the number of processors:
 - Sets the value of the COUNT variable to 1.
 - Starts a timer (set for an approximate interval of 100 milliseconds). In the AP BIOS initialization code, the AP will increment the COUNT variable to indicate its presence. When the timer expires, the BSP checks the value of the COUNT variable. If the timer expires and the COUNT variable has not been incremented, no APs are present or some error has occurred.

15. Broadcasts an INIT-SIPI-SIPI IPI sequence to the APs to wake them up and initialize them:

```

MOV ESI, ICR_LOW; load address of ICR low dword into ESI
MOV EAX, 000C4500H; load ICR encoding for broadcast INIT IPI
                    ; to all APs into EAX
MOV [ESI], EAX ; broadcast INIT IPI to all APs
                    ; 10-millisecond delay loop
MOV EAX, 000C46XXH; load ICR encoding for broadcast SIPI IPI
                    ; to all APs into EAX, where xx is the
                    ; vector value computed in step 8.
MOV [ESI], EAX ; broadcast SIPI IPI to all APs
                    ; 200-microsecond delay loop
MOV [ESI], EAX ; broadcast second SIPI IPI to all APs
                    ; 200-microsecond delay loop

```

16. Waits for the timer interrupt.
17. Reads and evaluates the COUNT variable and establishes a processor count.
18. If necessary, reconfigures the APIC and continues with the remaining system diagnostics as appropriate.

7.5.4.2. TYPICAL AP INITIALIZATION SEQUENCE

When an AP receives the SIPI, it begins executing BIOS AP initialization code at the vector encoded in the SIPI. The AP initialization code typically performs the following operations:

1. Waits on the BIOS initialization Lock Semaphore. When control of the semaphore is attained, initialization continues.
2. Loads the microcode update into the processor.
3. Initializes the MTRRs (using the same mapping that was used for the BSP).
4. Enables the cache.
5. Executes the CUID instruction with a value of 0H in the EAX register, then reads the EBX, ECX, and EDX registers to determine if the AP is “GenuineIntel.”
6. Executes the CUID instruction with a value of 1H in the EAX register, then saves the values in the EAX, ECX, and EDX registers in a system configuration space in RAM for use later.
7. Switches to protected mode and insures that the APIC address space is mapped to the strong uncacheable (UC) memory type.
8. Determines the AP’s APIC ID from the local APIC ID register, and adds it to the MP and ACPI tables and optionally to the system configuration space in RAM.

9. Initializes and configures the local APIC by setting bit 8 in the SVR register and setting up the LVT3 (error LVT) for error handling (as described in steps 9 and 10 in Section 7.5.4.1., “Typical BSP Initialization Sequence”).
10. Configures the APs SMI execution environment. (Each AP and the BSP must have a different SMBASE address.)
11. Increments the COUNT variable by 1.
12. Releases the semaphore.
13. Executes the CLI and HLT instructions.
14. Waits for an INIT IPI.

7.5.5. Identifying the Processors in an MP System

After the BIOS has completed the MP initialization protocol, each processor can be uniquely identified by its local APIC ID. Software can access these APIC IDs in either of the following ways:

- **Read the APIC ID for a local APIC.** Code running on a processor can execute a MOV instruction to read the contents of the processor’s local APIC ID register (see Section 8.4.6., “Local APIC ID”).
- **Read the ACPI or MP table.** As part of the MP initialization protocol, the BIOS creates an ACPI table and an MP table. These tables are defined in the Multiprocessor Specification Version 1.4 and provide software with a list of the processors in the system and their local APIC IDs. The format of the ACPI table is derived from the ACPI specification, which is an industry standard power management and platform configuration specification for MP systems.

For Intel Xeon processors, the APIC ID assigned to a processor during power-up and initialization is 8 bits (see Figure 7-2). Here, bits 1 and 2 form a 2-bit processor identifier (which can also be thought of as a socket identifier). In systems that configure processors in clusters, bits 3 and 4 form a 2-bit cluster ID. Bit 0 is used in the Intel Xeon processor MP to identify the two logical processors within the package (see Section 7.6.8., “Identifying Logical Processors in an MP System”). For an Intel Xeon processor without Hyper-Threading Technology, bit 0 is always set to 0; for an Intel Xeon processor with Hyper-Threading Technology, bit 0 performs the same function as it does in the Intel Xeon processor MP.

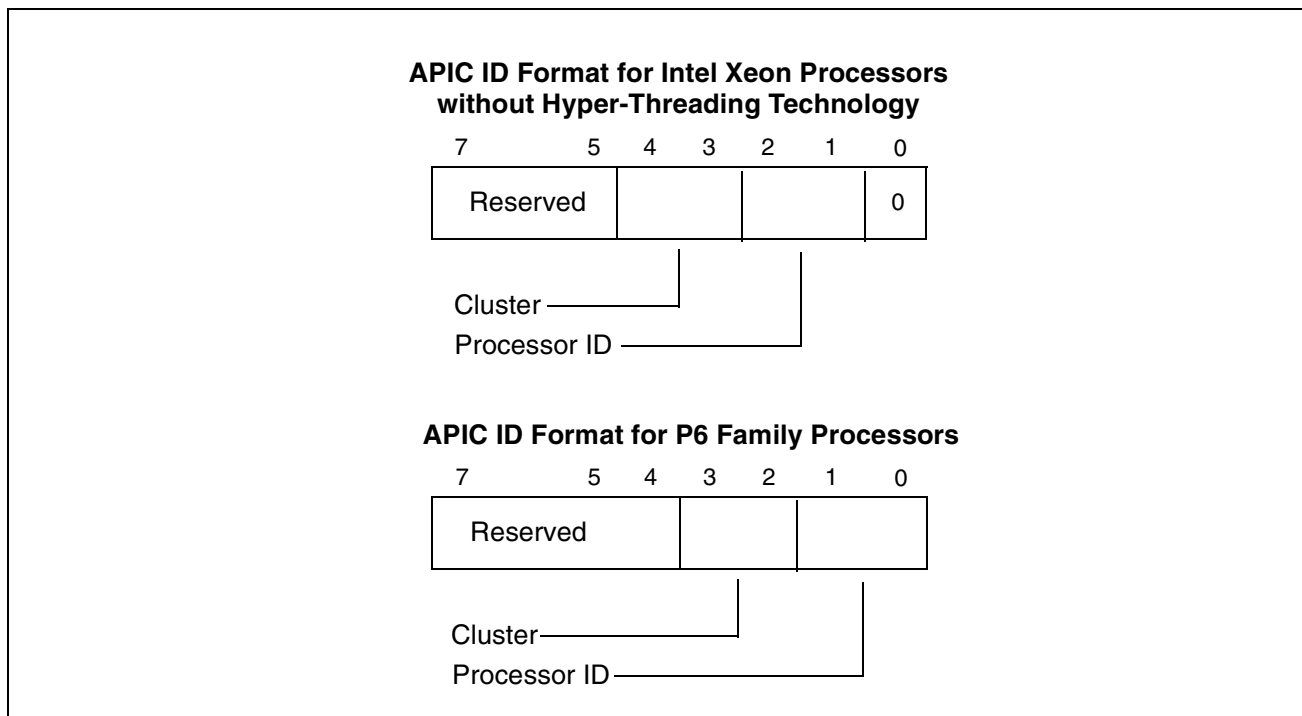


Figure 7-2. Interpretation of APIC ID in MP Systems

For P6 family processors, the APIC ID that is assigned to a processor during power-up and initialization is 4 bits (see Figure 7-2). Here, bits 0 and 1 form a 2-bit processor (or socket) identifier and bits 2 and 3 form a 2-bit cluster ID.

7.6. HYPER-THREADING TECHNOLOGY

Hyper-Threading (HT) Technology was introduced into the IA-32 architecture in the Intel Xeon processor MP and in later steppings of the Intel Xeon processor. It is also supported by the Intel Pentium 4 processor at 3.06 GHz or higher. All HT Technology configurations require a chipset and BIOS that utilize the technology, and an operating system that includes optimizations for HT technology. See www.intel.com/info/hyperthreading for more information. See also: Volume 1, Chapter 2.2.4., *Hyper-Threading Technology*.

Intel recommends that software not rely on IA-32 processor names to determine whether a processor supports HT Technology. Software should use the CUID instruction as described in Section 7.6.3., “Detecting Hyper-Threading Technology”.

HT Technology is an extension to the IA-32 architecture that enables a single physical processor to execute two or more separate code streams (called *threads*) concurrently. The following sections describe how the feature is implemented in IA-32 processors.

7.6.1. Intel Hyper-Threading Technology Architecture

Figure 7-3 shows a generalized view of an IA-32 processor with HT Technology, using the Intel Xeon processor MP as an example. This implementation of the HT Technology consists of two logical processors (each represented by a separate IA-32 architectural state) which share the processor's execution engine and the bus interface. Each logical processor also has its own advanced programmable interrupt controller (APIC).

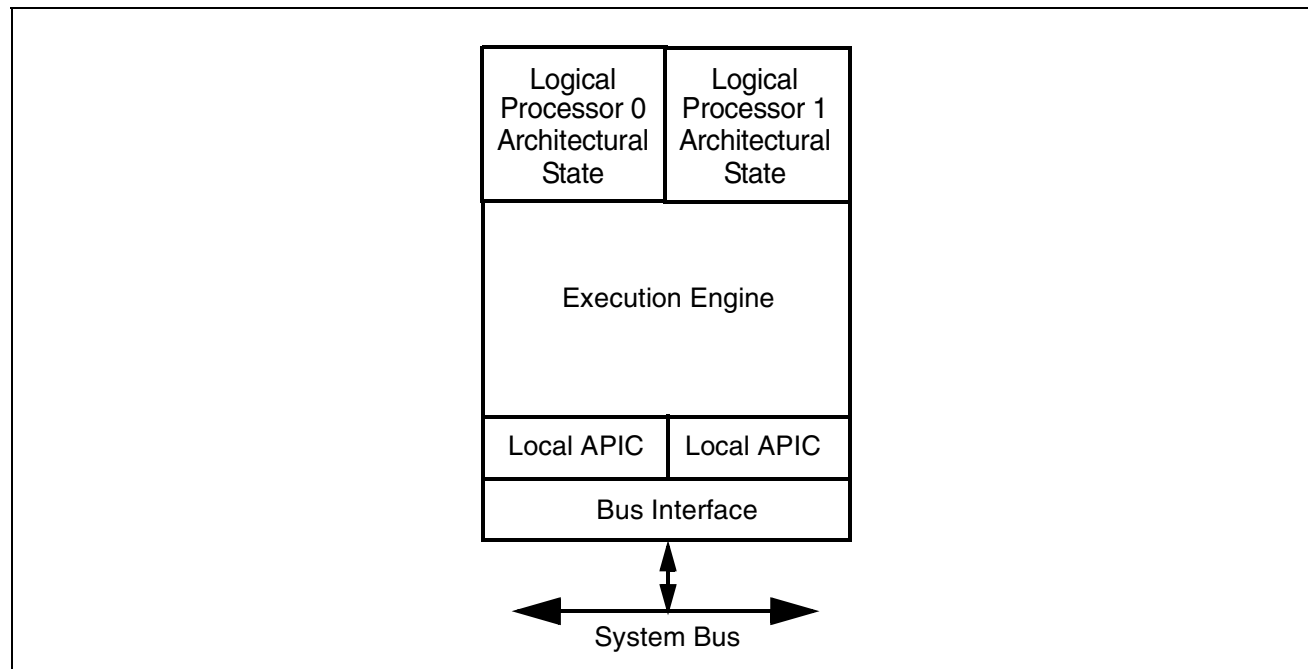


Figure 7-3. IA-32 Processor with Intel Hyper-Threading Technology using Two Logical Processors

7.6.1.1. STATE OF THE LOGICAL PROCESSORS

The following features are considered part of the architectural state of a logical processor with HT Technology. The features can be subdivided into three groups:

- Duplicated for each logical processor
- Shared by logical processors in a physical processor
- Shared or duplicated depending on the implementation

The following features are duplicated for each logical processor:

- General purpose registers (EAX, EBX, ECX, EDX, ESI, EDI, ESP, and EBP)
- Segment registers (CS, DS, SS, ES, FS, and GS)
- EFLAGS and EIP registers. Note that the CS and EIP registers for each logical processor point to the instruction stream for the thread being executed by the logical processor.

- x87 FPU registers (ST0 through ST7, status word, control word, tag word, data operand pointer, and instruction pointer)
- MMX registers (MM0 through MM7)
- XMM registers (XMM0 through XMM7) and the MXCSR register
- Control registers (CR0, CR2, CR3, CR4) and system table pointer registers (GDTR, LDTR, IDTR, task register)
- Debug registers (DR0, DR1, DR2, DR3, DR6, DR7) and the debug control MSR (IA32_DEBUGCTL)
- Machine check global status (IA32_MCG_STATUS) and machine check capability (IA32_MCG_CAP) MSRs
- Thermal clock modulation and ACPI Power management control MSRs
- Time stamp counter MSRs
- Most of the other MSR registers, including the page attribute table (PAT). See the exceptions below.
- Local APIC registers.

The following features are shared by logical processors:

- IA32_MISC_ENABLE MSR (MSR address 1A0H)
- Memory type range registers (MTRRs)

Whether the following features are shared or duplicated is implementation-specific:

- Machine check architecture (MCA) MSRs (except for the IA32_MCG_STATUS and IA32_MCG_CAP MSRs)
- Performance monitoring control and counter MSRs

7.6.1.2. APIC FUNCTIONALITY

When a processor with HT Technology support is initialized, each logical processor is assigned a local APIC ID (see Table 8-1). The local APIC ID serves as an ID for the logical processor and stored in the logical processor's APIC ID register. If two or more IA-32 processors with HT Technology are present in a dual processor (DP) or MP system, each logical processor on the system bus is assigned a unique local APIC ID (see Section 7.6.8., "Identifying Logical Processors in an MP System").

Software communicates with local processors using the APIC's interprocessor interrupt (IPI) messaging facility. The setup and programming of local APICs in processors with HT Technology is identical to that of IA-32 processors without HT Technology. See Chapter 8, *Advanced Programmable Interrupt Controller (APIC)* for a detailed discussion.

7.6.1.3. MEMORY TYPE RANGE REGISTERS (MTRR)

MTRRs in a processor with HT Technology are shared by logical processors. When one logical processor updates the setting of the MTRRs, settings are automatically shared with the other logical processors in the same physical package.

IA-32 architecture requires that all MP systems based on IA-32 processors (this includes logical processors) **MUST** use an identical MTRR memory map. This gives software a consistent view of memory, independent of the processor on which it is running. See Section 10.11., “Memory Type Range Registers (MTRRs)” for information on setting up MTRRs.

7.6.1.4. PAGE ATTRIBUTE TABLE (PAT)

Each logical processor has its own PAT MSR (IA32_CR_PAT). However, as described in Section 10.12., “Page Attribute Table (PAT)”, the PAT MSR settings must be the same for all processors in a system, including the logical processors.

7.6.1.5. MACHINE CHECK ARCHITECTURE

In the HT Technology context, all of the machine check architecture (MCA) MSRs (except for the IA32_MCG_STATUS and IA32_MCG_CAP MSRs) are duplicated for each logical processor. This permits logical processors to initialize, configure, query, and handle machine-check exceptions simultaneously within the same physical processor. The design is compatible with machine check exception handlers that follow the guidelines given in Chapter 14, *Machine-Check Architecture*.

The IA32_MCG_STATUS MSR is duplicated for each logical processor so that its machine check in progress bit field (MCIP) can be used to detect recursion on the part of MCA handlers. In addition, the MSR allows each logical processor to determine that a machine-check exception is in progress independent of the actions of another logical processor in the same physical package.

Because the logical processors within a physical package are tightly coupled with respect to shared hardware resources, both logical processors are notified of machine check errors that occur within a given physical processor. If machine-check exceptions are enabled when a fatal error is reported, all the logical processors within a physical package are dispatched to the machine-check exception handler. If machine-check exceptions are disabled, the logical processors enter the shutdown state and assert the IERR# signal.

When enabling machine-check exceptions, the MCE flag in control register CR4 should be set for each logical processor.

7.6.1.6. DEBUG REGISTERS AND EXTENSIONS

Each logical processor has its own set of debug registers (DR0, DR1, DR2, DR3, DR6, DR7) and its own debug control MSR (IA32_DEBUGCTL). These can be set to control and record debug information for each logical processor independently. Each logical processor also has its own last branch records (LBR) stack.

7.6.1.7. PERFORMANCE MONITORING COUNTERS

Performance counters their companion control MSRs are shared between the logical processors within the physical processor. As a result, software must manage the use of these resources. The performance counter interrupts, events, and precise event monitoring support can be set up and allocated on a per thread (per logical processor) basis.

See Section 15.10., “Performance Monitoring and Hyper-Threading Technology”, for a discussion of performance monitoring in the Intel Xeon processor MP.

7.6.1.8. IA32_MISC_ENABLE MSR

The IA32_MISC_ENABLE MSR (MSR address 1A0H) is shared between the logical processors in an IA-32 processor with HT Technology. Thus the architectural features that this register controls are set the same for all the logical processors in the same physical package.

7.6.1.9. MEMORY ORDERING

The logical processors in an IA-32 processor with HT Technology obey the same rules for memory ordering as IA-32 processors without HT Technology (see Section 7.2., “Memory Ordering”). Each logical processor uses a processor-ordered memory model that can be further defined as “write-ordered with store buffer forwarding.” All mechanisms for strengthening or weakening the memory ordering model to handle special programming situations apply to each logical processor.

7.6.1.10. SERIALIZING INSTRUCTIONS

As a general rule, when a logical processor in an IA-32 processor with HT Technology executes a serializing instruction, only that logical processor is affected by the operation. An exception to this rule is the execution of the WBINVD, INVD, and WRMSR instructions; and the MOV CR instruction when the state of the CD flag in control register CR0 is modified. Here, both logical processors are serialized.

7.6.1.11. MICROCODE UPDATE RESOURCES

In an IA-32 processor with HT Technology, the microcode update facilities are shared between the logical processors; either logical processor can initiate an update. Each logical processor has its own BIOS signature MSR (IA32_BIOS_SIGN_ID at MSR address 8BH). When a logical processor performs an update for the physical processor, the IA32_BIOS_SIGN_ID MSRs for resident logical processors are updated with identical information. If logical processors initiate an update simultaneously, the processor core provides the necessary synchronization needed to insure that only one update is performed at a time.

Operating system microcode update drivers that adhere to Intel’s guidelines do not need to be modified to run on an IA-32 processor with HT Technology.

7.6.1.12. SELF MODIFYING CODE

IA-32 processors with HT Technology support self-modifying code, where data writes modify instructions cached or currently in flight. They also support cross-modifying code, where on an MP system writes generated by one processor modify instructions cached or currently in flight on another. See Section 7.1.3., “Handling Self- and Cross-Modifying Code” for a description of the requirements for self- and cross-modifying code in an IA-32 processor.

7.6.2. Implementation-Specific HT Technology Facilities for the Intel Xeon Processor MP

The following non-architectural facilities are implementation-specific in IA-32 processors with HT Technology:

- Caches
- Translation lookaside buffers (TLBs)
- Thermal monitoring facilities

The Intel Xeon processor MP implementation is described in the following sections.

7.6.2.1. PROCESSOR CACHES

For the Intel Xeon processor MP, the caches are shared. Any cache manipulation instruction that is executed on one logical processor has a global effect on the cache hierarchy of the physical processor. Note the following:

- **WBINVD instruction.** The entire cache hierarchy is invalidated after modified data is written back to memory. All logical processors are stopped from executing until after the write-back and invalidate operation is completed. A special bus cycle is sent to all caching agents.
- **INVD instruction.** The entire cache hierarchy is invalidated without writing back modified data to memory. All logical processors are stopped from executing until after the invalidate operation is completed. A special bus cycle is sent to all caching agents.
- **CLFLUSH instruction.** The specified cache line is invalidated from the cache hierarchy after any modified data is written back to memory and a bus cycle is sent to all caching agents, regardless of which logical processor caused the cache line to be filled.
- **CD flag in control register CR0.** Each logical processor has its own CR0 control register, and thus its own CD flag in CR0. The CD flags for the two logical processors are OREd together, such that when any logical processor sets its CD flag, the entire cache is nominally disabled.

7.6.2.2. PROCESSOR TRANSLATION LOOKASIDE BUFFERS (TLBS)

In an Intel Xeon processor MP, data cache TLBs are shared. The instruction cache TLB is duplicated in each logical processor.

Entries in the TLBs are tagged with an ID that indicates the logical processor that initiated the translation. This tag applies even for translations that are marked global using the page global feature for memory paging.

When a logical processor performs a TLB invalidation operation, only the TLB entries that are tagged for that logical processor are flushed. This protocol applies to all TLB invalidation operations, including writes to control registers CR3 and CR4 and uses of the INVLPG instruction.

7.6.2.3. THERMAL MONITOR

In an Intel Xeon processor MP, logical processors share the catastrophic shutdown detector and the automatic thermal monitoring mechanism (see Section 13.15., “Thermal Monitoring and Protection”). Sharing results in the following behavior:

- If the processor’s core temperature rises above the preset catastrophic shutdown temperature, the processor core halts execution, which causes both logical processors to stop execution.
- When the processor’s core temperature rises above the preset automatic thermal monitor trip temperature, the clock speed of the processor core is automatically modulated, which effects the execution speed of both logical processors.

For software controlled clock modulation, each logical processor has its own IA32_THERM_CONTROL MSR, allowing clock modulation to be enabled or disabled on a per logical processor basis. Typically, if software controlled clock modulation is going to be used, it must be enabled for all the logical processors within a physical processor, and the modulation duty cycle must be set to the same value for each logical processor. If the duty cycle values differ between the logical processors, then the processor clock will be modulated at the highest duty cycle selected.

7.6.2.4. EXTERNAL SIGNAL COMPATIBILITY

This section describes the constraints on external signals received through the pins of an Intel Xeon processor MP and how these signals are shared between its logical processors.

- **STPCLK#.** A single STPCLK# pin is provided on the physical package of the Intel Xeon processor MP. External control logic uses this pin for power management within the system. When the STPCLK# signal is asserted, the processor core transitions to the stop-grant state, where instruction execution is halted but the processor core continues to respond to snoop transactions. Regardless of whether the logical processors are active or halted when the STPCLK# signal is asserted, execution is stopped on both logical processors and neither will respond to interrupts.

In MP systems, the STPCLK# pins on all physical processors are generally tied together. As a result this signal affects all the logical processors within the system simultaneously.

- **LINT0 and LINT1 Pins.** An Intel Xeon processor MP has only one set of LINT0 and LINT1 pins, which are shared between the logical processors. When one of these pins is asserted, both logical processors respond unless the pin has been masked in the APIC local vector tables for one or both of the logical processors.

Typically in MP systems, the LINT0 and LINT1 pins are not used to deliver interrupts to the logical processors. Instead all interrupts are delivered to the local processors through the I/O APIC.

- **A20M# Pin.** On an IA-32 processor, the A20M# pin is typically provided for compatibility with the Intel 286 processor. Asserting this pin causes bit 20 of the physical address to be masked (forced to zero) for all external bus memory accesses. The Intel Xeon processor MP provides one A20M# pin, which affects the operation of both logical processors within the physical processor. This configuration is compatible with the IA-32 architecture.

7.6.3. Detecting Hyper-Threading Technology

Software can use the CUID instruction to detect the presence of HT Technology and its configuration in an IA-32 processor. When the CUID instruction is executed with an input value of 1 in the EAX register, the following two items must be checked to determine HT Technology availability:

- The HT Technology feature flag (bit 28 in the EDX register) indicates (when set) that the processor supports HT Technology.
- Bits 16 through 23 in the EBX register indicate the number of logical processors supported within the physical package.

It is possible to have the CUID HT Technology feature flag set, but have only one logical processor available in the package. In this case, bits 16 through 23 in the EBX register will have a value of 1.

7.6.4. Initializing IA-32 Processors With Hyper-Threading Technology

The initialization process for an MP system that contains IA-32 processors with HT Technology is the same as for a conventional MP system (see Section 7.5., “Multiple-Processor (MP) Initialization”). One of the logical processors in the system is selected as the BSP and the other processors (or logical processors) are designated as APs. The initialization process is identical to that described in Section 7.5.3., “MP Initialization Protocol Algorithm for the Intel Xeon Processors” and Section 7.5.4., “MP Initialization Example”.

As part of the initialization procedure, each logical processor is automatically assigned an APIC ID, which is stored in the local APIC ID register for each logical processor. If two or more processors with HT Technology are present in a system, each logical processor on the system bus is assigned a unique ID (see Section 7.6.8., “Identifying Logical Processors in an MP System”).

Once logical processors have APIC IDs, software can communicate with them by sending APIC IPI messages.

7.6.5. Executing Multiple Threads on an IA-32 Processor With Hyper-Threading Technology

Upon completion of the operating system boot-up procedure, the bootstrap processor (BSP) continues to execute operating system code, while the other logical processors in the system are placed in the halt state. To execute a code stream (thread) on one of the halted logical processors, the operating system must issue an interprocessor interrupt (IPI) addressed to the logical processor. In response to the IPI, the halted logical processor wakes up and begins executing the thread identified by the interrupt vector it received as part of the IPI. When all the logical processors in an IA-32 processor with HT Technology are executing threads, the core execution engine executes the instruction streams for the active threads concurrently, and the shared execution resources are allocated to the active logical processors on an “as needed basis.”

To manage the execution of multiple threads on the logical processors, an operating system can use conventional symmetric multiprocessing (SMP) techniques. For example, the operating system can use a time-slice or other load balancing mechanism to periodically interrupt each of the active logical processors. Upon interrupting a logical processor, the operating system then checks its run queue for a thread waiting to be executed and dispatches the thread to the interrupted logical processor. In this way, an MP-capable operating system can schedule threads for execution on logical processors in the same way that it does on the processors in a conventional MP system.

7.6.6. Handling Interrupts on an IA-32 Processor With Hyper-Threading Technology

Interrupts are handled in the same way in an IA-32 processors with HT Technology as they are in a conventional MP system. External interrupts are received by the I/O APIC, which distributes them as interrupt messages to specific logical processors (see Figure 7-4). Each logical processor can also send IPIs to other logical processors by writing to the ICR register of its local APIC (see Section 8.6., “Issuing Interprocessor Interrupts”).

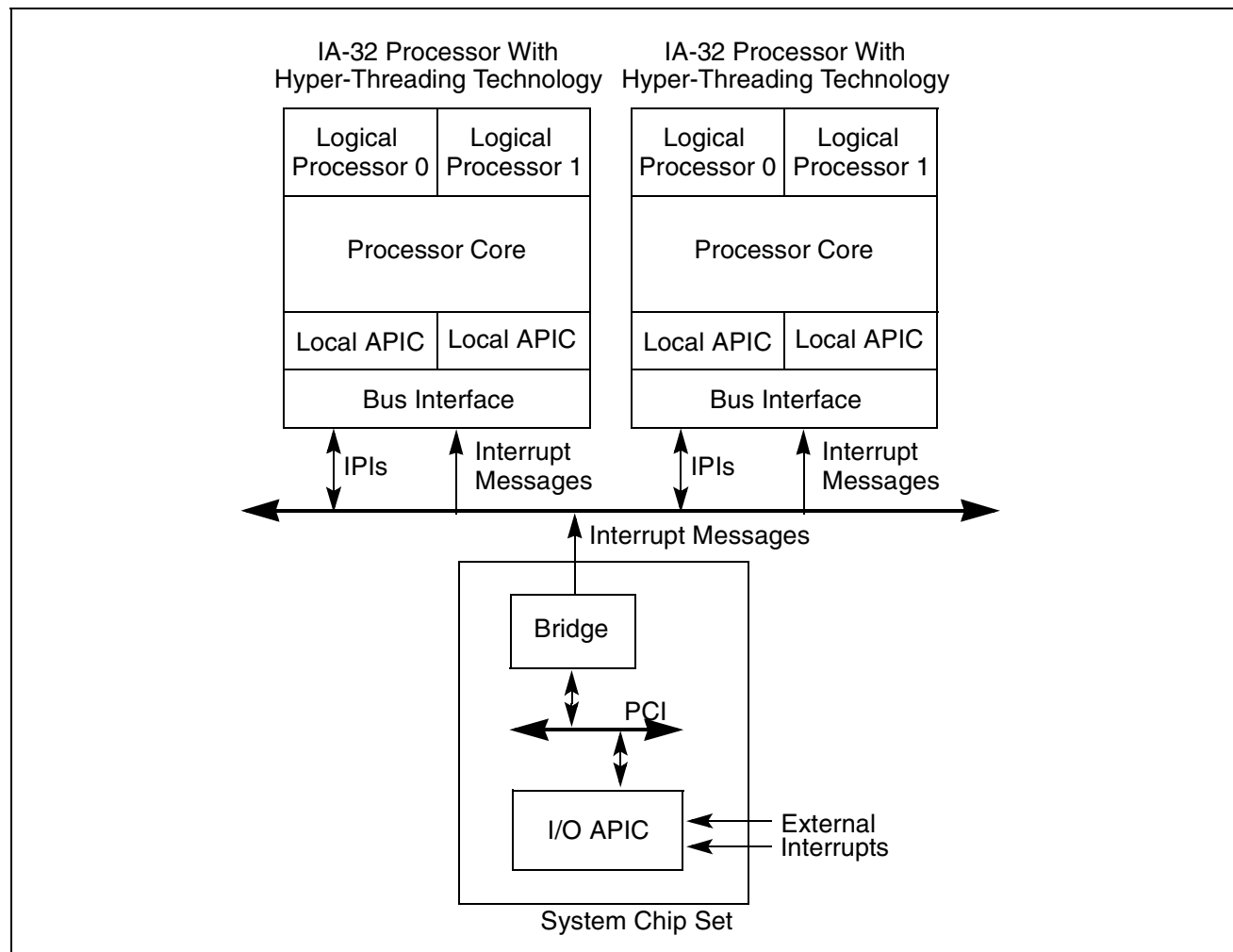


Figure 7-4. Local APICs and I/O APIC When IA-32 Processors with Hyper-Threading Technology Are Used in MP Systems

7.6.7. Management of Idle and Blocked Conditions

During normal execution of an IA-32 processor with HT Technology, with each logical processor actively executing a thread, logical processors use the shared processor resources (such as cache lines, TLB entries, and bus accesses) on an as-needed basis. When one of the logical processors is either idle (software has no work for it to do) or blocked (spinning on a lock or semaphore) additional management of the core execution engine resource by using the HLT (halt) and PAUSE instructions is recommended for efficient operation.

7.6.7.1. HLT INSTRUCTION

The HLT instruction stops the execution of the logical processor on which it is executed and places it in a halted state until further notice (see the description of the HLT instruction in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*). When a logical processor is halted, active logical processors continue to have full access to the shared resources within the physical package. Here shared resources that were being used by the halted logical processor become available to active logical processors, allowing them to execute at greater efficiency. When the halted logical processor resumes execution, shared resources are again shared among all active logical processors. (See Section 7.6.9.2., “Halt Idle Logical Processors”, for more information about using the HLT instruction with IA-32 processors with Hyper-Threading Technology.)

7.6.7.2. PAUSE INSTRUCTION

The PAUSE instruction improves the performance of IA-32 processors with HT Technology when executing “spin-wait loops” and other routines where one thread is accessing a shared lock or semaphore in a tight polling loop. When executing a spin-wait loop, the processor can suffer a severe performance penalty when exiting the loop because it detects a possible memory order violation and flushes the core processor's pipeline. The PAUSE instruction provides a hint to the processor that the code sequence is a spin-wait loop. The processor uses this hint to avoid the memory order violation and prevent the pipeline flush. In addition, the PAUSE instruction depipelined the spin-wait loop to prevent it from consuming execution resources excessively. (See Section 7.6.9.1., “Use the PAUSE Instruction in Spin-Wait Loops”, for more information about using the PAUSE instruction with IA-32 processors with Hyper-Threading Technology.)

7.6.8. Identifying Logical Processors in an MP System

For any IA-32 processor, the system hardware establishes an initial APIC ID for the processor during power-up or RESET (see Section 7.6.4., “Initializing IA-32 Processors With Hyper-Threading Technology”). For an IA-32 processor with HT Technology, system hardware assigns a unique APIC ID to each logical processors on the system bus.

The APIC ID for a logical processor is made up of three fields: logical processor ID, physical package ID, and cluster ID. Figure 7-5 shows the layout of these fields. Here, bit 0 forms a 1-bit logical processor ID, bits 1 and 2 form a 2-bit package ID, and bits 3 and 4 form a 2-bit cluster ID. Bit 0 is used to identify the two logical processor within the package.

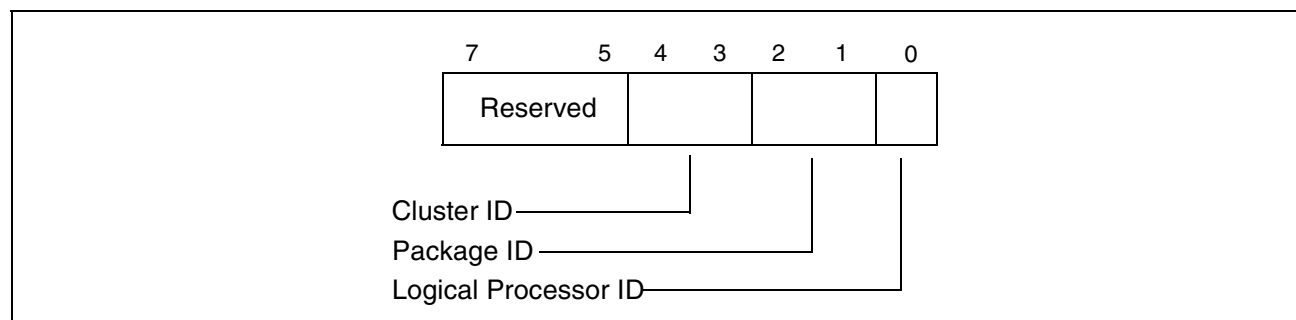


Figure 7-5. Interpretation of the APIC ID

Table 7-1 shows the APIC IDs that are generated for the logical processors in a system with four MP-type Intel Xeon processors (a total of 8 logical processors). Of the two logical processors within a Intel Xeon processor MP, logical processor 0 is also referred to as the “primary logical processor” and logical processor 1 is referred to as the “secondary logical processor.”

Table 7-1. Initial APIC IDs for the Logical Processors in a System that has Four MP-Type Intel Xeon Processors with Hyper-Threading Technology

Logical Processor Initial APIC ID	Physical Processor ID	Logical Processor ID
0H	0H	0H
1H	0H	1H
2H	1H	0H
3H	1H	1H
4H	2H	0H
5H	2H	1H
6H	3H	0H
7H	3H	1H

Software can determine the APIC IDs of the logical processor in the system in either of the two ways described in Section 7.5.5., “Identifying the Processors in an MP System”. Note that only the APIC IDs of the primary logical processors in each physical package are included in the MP table. All the logical processors in the system are included in the ACPI table, with the primary logical processors at the top of the table followed by the secondary logical processors.

If future IA-32 processors with HT Technology that implement more than two logical processors per physical processor, the logical processor bit shown in Figure 7-5 will be expanded to a 2- or 3-bit field to allow each of the logical processors to be identified. The package ID and cluster ID fields will be shifted to the left accordingly. Also, the package ID may be expanded to more than 2 bits, requiring the cluster ID field to be shifted to the left.

Operating system and application software can determine the layout of an APIC ID for a particular processor by interpreting the number of logical processors field and the local APIC physical ID field that are returned to the EBX register when the CPUID instruction is executed with a parameter of 1 in the EAX registers.

As with IA-32 processors without HT Technology, software can assign a different APIC ID to a logical processor by writing the value into the local APIC ID register; however, the CPUID instruction will still report the processor's initial APIC ID (the value assigned during power-up or RESET).

Figure 7-5 depicts the layout of cluster ID, package ID and logical processor ID bit fields of an APIC ID for current implementations of HT Technology (two logical processors per package). In general, the content of an APIC ID (excluding cluster ID) for a logical processor in a package with a finite number of logical processors per package is given by:

$$((\text{Package ID} \ll (1 + ((\text{int})(\log(2)(\max(\text{Logical_Per_Package} - 1, 1)))))) \parallel \text{Logical Processor ID})$$

Use this formula to determine the association between logical processors and their physical packages for future implementations of HT Technology. The pseudo-code below (Examples 7-1 and 7-2) shows an algorithm to determine the relationship between logical and physical processors. This algorithm supports any number of logical processors per package. The algorithm is run on each logical processor in the system using an operating system specific affinity to accomplish binding. After running the algorithm, logical processors that have the same Processor ID exist within the same physical package. All processors present in the system must support the same number of logical processors per physical processor.

The algorithm for detecting support for HT Technology and identifying the relationships between a logical processor to the corresponding physical processor ID consists of five steps:

1. Detect support for HT Technology in the processor.
2. Identify the number of logical processors available in a physical processor package.
3. Extract the initial APIC ID for this processor.
4. Compute a mask value and bit-shift value.
5. Compute a logical processor ID and physical processor package ID.

Example 7-1. Generalized Algorithm to Extract Physical Processor IDs for Hyper-Threading Technology

1. Pseudo-code to detect support for Hyper-Threading Technology in a processor.

```
// Returns non-zero if Hyper-Threading Technology is supported on
// the processors and zero if not. This does not mean that
// Hyper-Threading Technology is necessarily enabled.

unsigned int HTSupported(void)
{
    try { // verify cpuid instruction is supported
        execute cpuid with eax = 0 to get vendor string
        execute cpuid with eax = 1 to get feature flag and signature
    }
    except (EXCEPTION_EXECUTE_HANDLER) {
        return 0 ; // CPUID is not supported and so Hyper-Threading
        // Technology is not supported
    }

    // Check to see if this a a Genuine Intel Processor
    // a member of the Pentium 4 processor family
    // and supporting Hyper-Threading Technology

    if (vendor string NEQ GenuineIntel)
        if (family signature NEQ Pentium4Family)
            return (feature_flag_edx & HTT_BIT);
    return 0;
}
```

2. Pseudo-code to identify the number of logical processors per physical processor package.

```
#define NUM_LOGICAL_BITS 0x00FF0000 // EBX[23:16] indicate number of
// logical processor per package

// Returns the number of logical processors per physical processor.

unsigned char LogicalProcessorsPerPackage(void)
{
    if (!HTSupported()) return (unsigned char) 1;
    execute cpuid with eax = 1
    store returned value of ebx
    return (unsigned char) ((reg_ebx & NUM_LOGICAL_BITS) >> 16);
}
```

Example 7-2. Streamlined Determination of Mask to get the Logical Processor Number

3. Pseudo-code to extract the initial APIC ID of a processor

```
#define INITIAL_APIC_ID_BITS 0xFF000000 // EBX[31:24] initial APIC ID

// Returns the 8-bit unique initial APIC ID for the processor this
// code is actually running on. The default value returned is 0xFF if
// Hyper-Threading Technology is not supported.

unsigned char GetAPIC_ID (void)
{
    unsigned int reg_ebx = 0;
    if (!HTSupported()) return (unsigned char) -1;
    execute cpuid with eax = 1
    store returned value of ebx
    return (unsigned char) ((reg_ebx & INITIAL_APEIC_ID_BITS) >> 24;
}
```

4. Sample code to compute a mask value and a bit-shift value, the logical processor ID and physical processor package ID.

```
unsigned char i = 1;
unsigned char PHY_ID_MASK = 0xFF;
unsigned char PHY_ID_SHIFT = 0;
unsigned char APIC_ID;
unsigned char LOG_ID, PHY_ID;

Logical_Per_Package = LogicalProcessorsPerPackage();
While (i < Logical_Per_Package){
    i *= 2;
    PHY_ID_MASK <= 1;
    PHY_ID_SHIFT++;
}
// Assume this thread is running on the logical processor from
// which we extract the logical processor ID and its physical
// processor package ID. If not, use the OS-specific affinity
// service (See example 7-3) to bind this thread to the target
// logical processor
APIC_ID = GetAPIC_ID();
LOT_ID = APIC_ID & ~PHY_ID_MASK;
PHY_ID = APIC_ID >> PHY_ID_SHIFT;
```

Example 7-3. Using an OS-specific Affinity Service to Identify the Logical Processor IDs in an MP System

5. Compute the logical processor ID and physical processor package ID.

```
// The OS may limit the processor that this process may run on.

hCurrentProcessHandle = GetCurrentProcess();
GetProcessAffinityMask(hCurreentPorcessHandle,
    &dwProcessAffinity, &dwSystemAffinity);

// If the available process affinity mask does not equal the
// available system affinity mask, then determining if
// Hyper-Threading Technology is enabled may not be possible.

if (dwProcessAffinity != dwSystemAffinity)
    printf ("This process can not utilize all processors. \n"),

dwAffinityMask = 1;
while (dwAffinityMask != 0 &&
    dwAffinityMask <= dwProcessAffinity) {
    // Check to make sure we can utilize this processor first.
    if (dwAffinityMask & dwProcessAffinity){
        if (SetProcessAffinityMask(hCurrentProcessHandle,
            dwAffinityMask)) {

            Sleep(0);    // May not be running on the logical processor
                        // on the affinity just set. Sleep gives the
                        // OS a chance to switch to the desired
                        // logical processor.

            // Retrieve APIC_ID for this logical processor
            // Extract logical processor ID and physical processor
            // package ID

        }
    }
}
```

7.6.9. Required Operating System Support

This section describes changes that must be made to an operating system to run on IA-32 processors with HT Technology. It also describes optimizations that can help an operating system make more efficient use of the logical processors within a physical package. The required changes and suggested optimizations are representative of the types of modifications that appear in the Windows XP and the Linux kernel 2.4.0 operating systems to support IA-32 processors with HT Technology. Additional optimizations IA-32 processors with HT Technology are described in the *Pentium 4 and Intel Xeon Processor Optimization Reference Manual* (see Section 1.4., “Related Literature” for an order number).

7.6.9.1. USE THE PAUSE INSTRUCTION IN SPIN-WAIT LOOPS

Intel recommends that a PAUSE instruction be placed in all spin-wait loops that run on Intel Xeon and/or Pentium 4 processors.

Software routines that use spin-wait loops include multiprocessor synchronization primitives (spin-locks, semaphores, and mutex variables) and idle loops. Such routines keep the processor core busy executing a load-compare-branch loop while a thread waits for a resource to become available. Including a PAUSE instruction in such a loop greatly improves efficiency (see Section 7.6.7.2., “PAUSE Instruction”). The following routine gives an example of a spin-wait loop that uses a PAUSE instruction:

```
Spin_Lock:
    CMP lockvar, 0; Check if lock is free
    JE Get_Lock
    PAUSE ; Short delay
    JMP Spin_Lock
Get_Lock:
    MOV EAX, 1
    XCHG EAX, lockvar ; Try to get lock
    CMP EAX, 0 ; Test if successful
    JNE Spin_Lock
Critical_Section:
    <critical section code>
    MOV lockvar, 0
    ...
Continue:
```

The spin-wait loop above uses a “test, test-and-set” technique for determining the availability of the synchronization variable. This technique is recommended when writing spin-wait loops.

In IA-32 processor generations earlier than the Pentium 4 processor, the PAUSE instruction is treated as a NOP instruction.

7.6.9.2. HALT IDLE LOGICAL PROCESSORS

If one of two logical processors is idle or in a spin-wait loop of long duration, explicitly halt that processor by means of a HLT instruction.

In an MP system, operating systems can place idle processors into a loop that continuously checks the run queue for runnable software tasks. Logical processors that execute idle loops consume a significant amount of core's execution resources that might otherwise be used by the other logical processors in the physical package. For this reason, halting idle logical processors optimizes the performance.¹ If all logical processors within a physical package are halted, the processor will enter a power-saving state.

7.6.9.3. GUIDELINES FOR SCHEDULING THREADS ON MULTIPLE LOGICAL PROCESSORS

Because the logical processors, the order in which threads are dispatched to logical processors for execution can affect the overall efficiency of a system. The following guidelines are recommended for scheduling threads for execution.

- Dispatch threads to one logical processor per physical package before dispatching threads to the remaining logical processors in available physical packages. In an MP system with two or more IA-32 processors with HT Technology, distribute threads out over all the physical packages rather than concentrate them in one or two physical processors.
- Use processor affinity to assign a thread to a specific physical processor. The practice increases the chance that the processor's caches will contain some of the thread's code and data when it is dispatched for execution after being suspended. The thread can be dispatched to any logical processor within a physical package because logical processors share the physical processor's caches.

7.6.9.4. ELIMINATE EXECUTION-BASED TIMING LOOPS

Intel discourages the use of timing loops that depend on a processor's execution speed to measure time. There are several reasons:

- Timing loops cause problems when they are calibrated on a IA-32 processor running at one clock speed and then executed on a processor running at another clock speed.
- Routines for calibrating execution-based timing loops produce unpredictable results when run on an IA-32 processor with HT Technology. This is due to the sharing of execution resources between the logical processors within a physical package.

To avoid the problems described, timing loop routines must use a timing mechanism for the loop that does not depend on the execution speed of the logical processors in the system. The following sources are generally available:

- A high resolution system timer (for example, an Intel 8254).

1. Excessive transitions into and out of the HALT state could also incur performance penalties. Operating systems should evaluate the performance trade-offs for their operating system.

- A high resolution timer within the processor (such as, the local APIC timer or the time-stamp counter).

For additional information, see the *Pentium 4 and Intel Xeon Processor Optimization Reference Manual* (see Section 1.4., “Related Literature” for an order number).

7.6.9.5. PLACE LOCKS AND SEMAPHORES IN ALIGNED, 128-BYTE BLOCKS OF MEMORY

When software uses locks or semaphores to synchronize processes, threads, or other code sections; Intel recommends that only one lock or semaphore be present within a cache line. In an Intel Xeon processor MP (which have 128-byte wide cache lines), following this recommendation means that each lock or semaphore should be contained in a 128-byte block of memory that begins on a 128-byte boundary. The practice minimizes the bus traffic required to service locks.

8

Advanced Programmable Interrupt Controller (APIC)

CHAPTER 8

ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

The Advanced Programmable Interrupt Controller (APIC), referred to in the following sections as the local APIC, was introduced into the IA-32 processors with the Pentium processor (see Section 18.23., “Advanced Programmable Interrupt Controller (APIC)”) and is included in the Pentium 4, Intel Xeon, and P6 family processors (see Section 8.4.2., “Presence of the Local APIC”). The local APIC performs two primary functions for the processor:

- It receives interrupts from the processor’s interrupt pins, from internal sources, and/or from an external I/O APIC (or other external interrupt controller) and sends them to the processor core for handling.
- In multiple processor (MP) systems, it sends and receives interprocessor interrupt (IPI) messages to and from other IA-32 processors on the system bus. These IPI messages can be used to distribute interrupts among the processors in the system or to execute system wide functions (such as, booting up processors or distributing work among a group of processors).

The external **I/O APIC** is part of Intel’s system chip set. Its primary function is to receive external interrupt events from the system and its associated I/O devices and relay them to the local APIC as interrupt messages. In MP systems, the I/O APIC also provides a mechanism for distributing external interrupts to the local APICs of selected processors or groups of processors on the system bus.

This chapter provides a detailed description of the local APIC and its programming interface. It also provides an overview of the interface between the local APIC and the I/O APIC. Contact Intel for detailed information about the I/O APIC.

When a local APIC has sent an interrupt to its associated processor core for handling, the processor uses the interrupt and exception handling mechanism described in Chapter 5, *Interrupt and Exception Handling*, to service the interrupt. Section 5.1., “Interrupt and Exception Overview”, gives an introduction to interrupt and exception handling in the IA-32 architecture. It is recommended that this section be read in addition to the following sections to aid in understanding the IA-32 APIC architecture and its functions.

8.1. LOCAL AND I/O APIC OVERVIEW

Each local APIC consists of a set of APIC registers (see Table 8-1) and associated hardware that control the delivery of interrupts to the processor core and the generation of IPI messages. The APIC registers are memory mapped and can be read and written to using the MOV instruction.

The local APIC can receive interrupts from the following sources:

- **Locally connected I/O devices.** These interrupts originate as an edge or level asserted by an I/O device that is connected directly to the processor's local interrupt pins (LINT0 and LINT1). The I/O devices may also be connected to an 8259-type interrupt controller that is in turn connected to the processor through one of the local interrupt pins.
- **Externally connected I/O devices.** These interrupts originate as an edge or level asserted by an I/O device that is connected to the interrupt input pins of an I/O APIC. These interrupts are sent as I/O interrupt messages from the I/O APIC to one or more the IA-32 processors in the system.
- **Inter-processor interrupts (IPIs).** An IA-32 processor can use the IPI mechanism to interrupt another processor or group of processors on the system bus. IPIs are used for such things as software self-interrupts, interrupt forwarding, or preemptive scheduling.
- **APIC timer generated interrupts.** The local APIC timer can be programmed to send a local interrupt to its associated processor when a programmed count is reached (see Section 8.5.4., "APIC Timer").
- **Performance monitoring counter interrupts.** The Pentium 4, Intel Xeon, and P6 family processors provide the ability to send a interrupt to its associated processor when a performance-monitoring counter overflows (see Section 15.9.6.9., "Generating an Interrupt on Overflow").
- **Thermal Sensor interrupts.** The Pentium 4 and Intel Xeon processors provide the ability to send an interrupt to themselves when the internal thermal sensor has been tripped (see Section 13.15.2., "Thermal Monitor").
- **APIC internal error interrupts.** When an error condition is recognized within the local APIC (such as an attempt to access an unimplemented register), the APIC can be programmed to send an interrupt to its associated processor (see Section 8.5.3., "Error Handling").

Of these interrupt sources, the processor's LINT0 and LINT1 pins, the APIC timer, the performance-monitoring counters, the thermal sensor, and the internal APIC error detector are referred to as **local interrupt sources**. Upon receiving a signal from a local interrupt source, the local APIC delivers the interrupt to the processor core using an interrupt delivery protocol that has been set up through a group of APIC registers called the **local vector table** or **LVT** (see Section 8.5.1., "Local Vector Table"). A separate entry is provided in the local vector table for each local interrupt source, which allows a specific interrupt delivery protocol to be set up for each source. For example, if the LINT1 pin is going to be used as an NMI pin, the LINT1 entry in the local vector table can be set up to deliver an interrupt with vector number 2 (NMI interrupt) to the processor core.

The local APIC handles interrupts from the other two interrupt sources (externally connected I/O devices and IPIs) through its IPI message handling facilities.

A processor can generate IPIs by programming the interrupt command register (ICR) in its local APIC (see Section 8.6.1., "Interrupt Command Register (ICR)"). The act of writing to the ICR causes an IPI message to be generated and issued on the system bus (for Pentium 4 and Intel Xeon processors) or on the APIC bus (for Pentium and P6 family processors).

(See Section 8.2., “System Bus Vs. APIC Bus”.) IPIs can be sent to other IA-32 processors in the system or to the originating processor (self-interrupts). When the target processor receives an IPI message, its local APIC handles the message automatically (using information include in the message such as vector number and trigger mode) and delivers it to the processor core for servicing. See Section 8.6., “Issuing Interprocessor Interrupts” for a detailed explanation of the local APIC’s IPI message delivery and acceptance mechanism.

The local APIC can also receive interrupts from externally connected devices through the I/O APIC (see Figure 8-1). The I/O APIC is responsible for receiving interrupts generated by system hardware and I/O devices and forwarding them to the local APIC as interrupt messages.

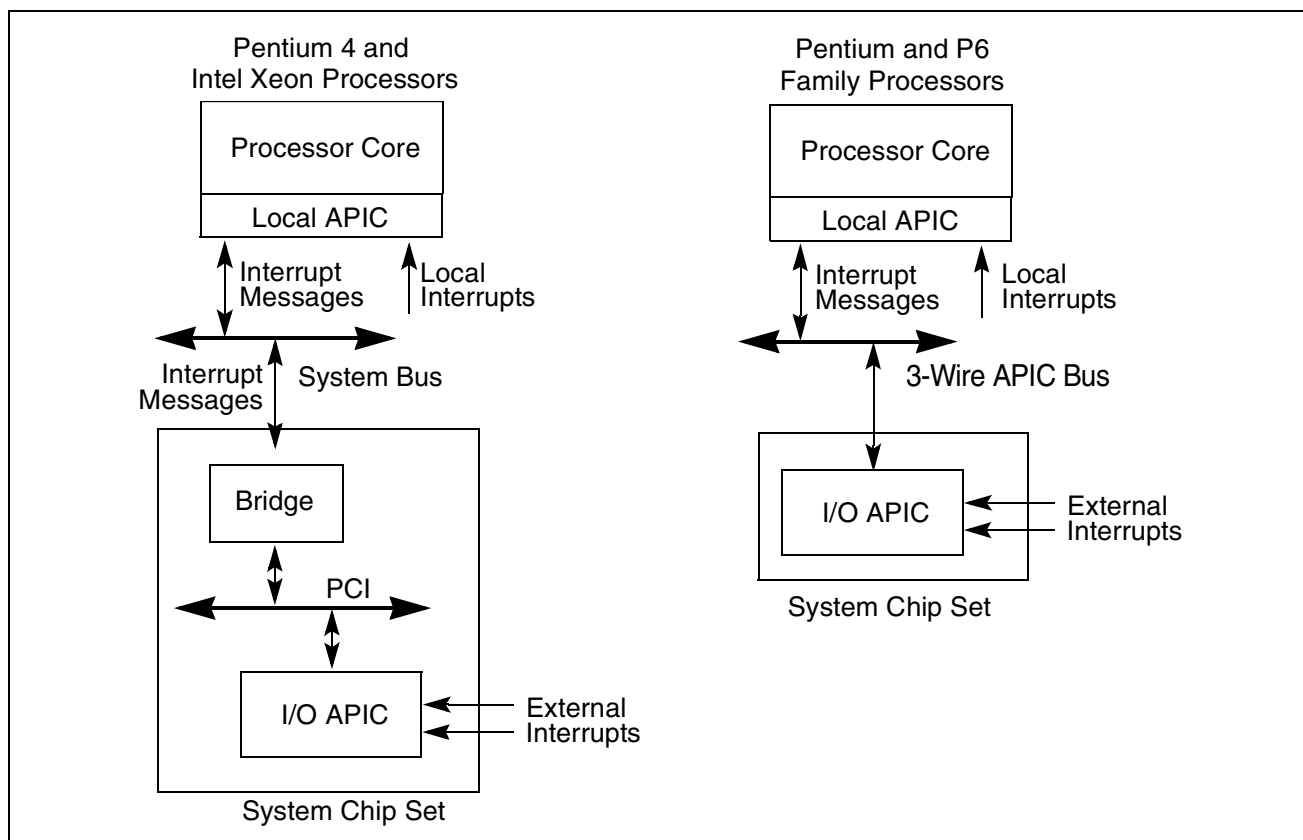


Figure 8-1. Relationship of Local APIC and I/O APIC In Single-Processor Systems

Individual pins on the I/O APIC can be programmed to generate a specific interrupt vector when asserted. The I/O APIC also has a “virtual wire mode” that allows it to communicate with a standard 8259A-style external interrupt controller.

Note that the local APIC can be disabled (see Section 8.4.3., “Enabling or Disabling the Local APIC”), allowing its associated processor core to receive interrupts directly from an 8259A interrupt controller.

Both the local APIC and the I/O APIC are designed to operate in MP systems (see Figures 8-2 and 8-3). Here each local APIC handles both externally generated interrupts that it receives as interrupt messages from the I/O APIC and IPIs from other processors on the system bus, and

from itself. (Interrupts can also be delivered to the individual processors through the local interrupt pins; however, this mechanism is commonly not used in MP systems.)

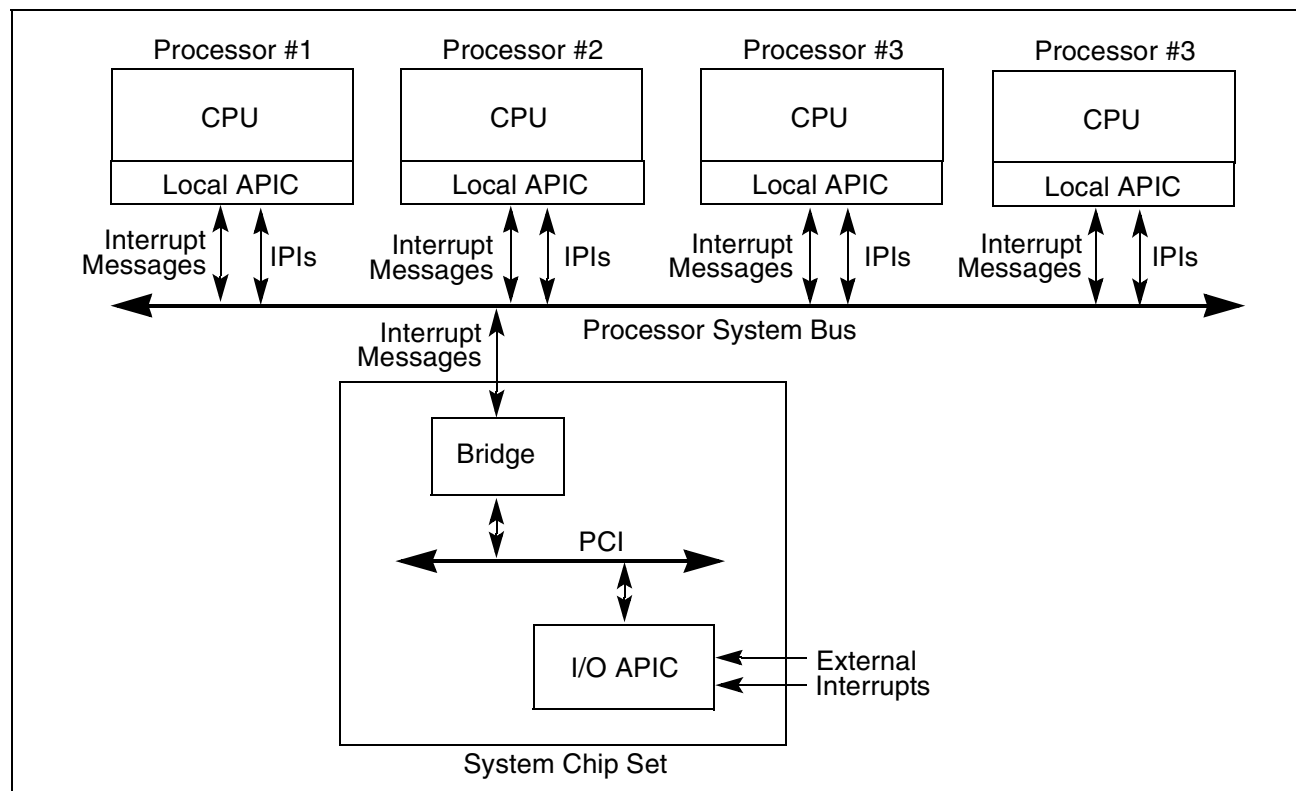


Figure 8-2. Local APICs and I/O APIC When Intel Xeon Processors Are Used in Multiple-Processor Systems

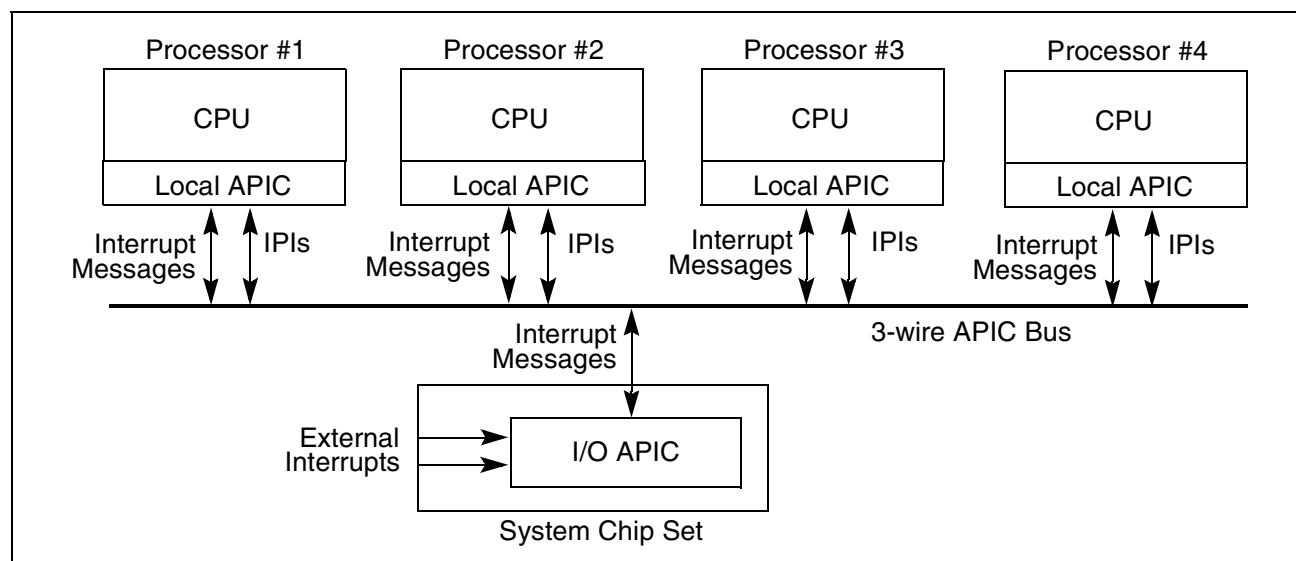


Figure 8-3. Local APICs and I/O APIC When P6 Family Processors Are Used in Multiple-Processor Systems

The IPI mechanism is typically used in MP systems to send fixed interrupts (interrupts for a specific vector number) and special-purpose interrupts to other processors on the system bus or to itself. For example, one local APIC can use an IPI to forward a fixed interrupt to another processor for servicing. Special-purpose IPIs, including NMI, INIT, SMI and SIPI IPIs, allow one or more processors in the system bus to perform system-wide boot-up and control functions.

The following sections focus on the local APIC, and its implementation in the Pentium 4, Intel Xeon, and P6 family processors. In the descriptions in these sections, the generic terms “local APIC” and “I/O APIC” refer to the local and I/O APICs used with the P6 family processors and to the local and I/O xAPICs used with the Pentium 4 and Intel Xeon processors (see Section 8.3., “Relationship Between the Intel 82489DX External APIC, the APIC, and the xAPIC”).

8.2. SYSTEM BUS VS. APIC BUS

For the P6 family and Pentium processors, the I/O APIC and local APICs communicate through the 3-wire inter-APIC bus (see Figure 8-3). The local APICs also use the APIC bus to send and receive IPIs. The APIC bus and its messages are invisible to software and are not classed as architectural.

Beginning with the Pentium 4 and Intel Xeon processors, the I/O APIC and local APICs (using the xAPIC architecture) communicate through the system bus (see Figure 8-2). Here the I/O APIC sends interrupt requests to the processors on the system bus through bridge hardware that is part of the Intel chip set. This bridge hardware generates the actual interrupt messages that go to the local APICs. IPIs between local APICs are transmitted directly on the system bus.

8.3. RELATIONSHIP BETWEEN THE INTEL 82489DX EXTERNAL APIC, THE APIC, AND THE XAPIC

The local APIC in the P6 family and Pentium processors is an architectural subset of the Intel 82489DX external APIC. The differences are described in Section 18.23.1., “Software Visible Differences Between the Local APIC and the 82489DX”.

The APIC architecture used in the Pentium 4 and Intel Xeon processors (called the xAPIC architecture) is an extension of the APIC architecture found in the P6 family processors. The primary difference between the APIC and xAPIC architectures is that with the xAPIC architecture, the local APICs and the I/O APIC communicate with one another through the system bus; whereas, with the APIC architecture, they communicate through the APIC bus (see Section 8.2., “System Bus Vs. APIC Bus”). Also, some of the APIC architectural features have been extended and/or modified in the xAPIC architecture. These extensions and modifications are noted in the following sections.

8.4. LOCAL APIC

The following sections describe the architecture of the local APIC and how to detect it, identify it, and determine its status. Descriptions of how to program the local APIC are given in Section 8.5.1., “Local Vector Table” and Section 8.6.1., “Interrupt Command Register (ICR)”.

8.4.1. The Local APIC Block Diagram

Figure 8-4 gives a functional block diagram for the local APIC. Software interacts with the local APIC by reading and writing its registers. The APIC registers are memory-mapped to a 4-KByte region of the processor’s physical address space with an initial starting address of FEE00000H. For correct APIC operation, this address space must be mapped to an area of memory that has been designated as strong uncacheable (UC). See Section 10.3., “Methods of Caching Available”.

In MP system configurations, the APIC registers for all the IA-32 processors on the system bus are initially mapped to the same 4-KByte region of the physical address space. Software has the option of changing this initial mapping to a different 4-KByte region for all the local APICs or of mapping the APIC registers for each local APIC to its own 4-KByte region. Section 8.4.5., “Relocating the Local APIC Registers” describes how to relocate the base address for the APIC registers for a specific processor.

NOTE

For Pentium 4, Intel Xeon, and P6 family processors, the APIC handles all memory accesses to addresses within the 4-KByte APIC register space internally and no external bus cycles are produced. For the Pentium processors with an on-chip APIC, bus cycles are produced for accesses to the APIC register space. Thus, for software intended to run on Pentium processors, system software should explicitly not map the APIC register space to regular system memory. Doing so can result in an invalid opcode exception (#UD) being generated or unpredictable execution.

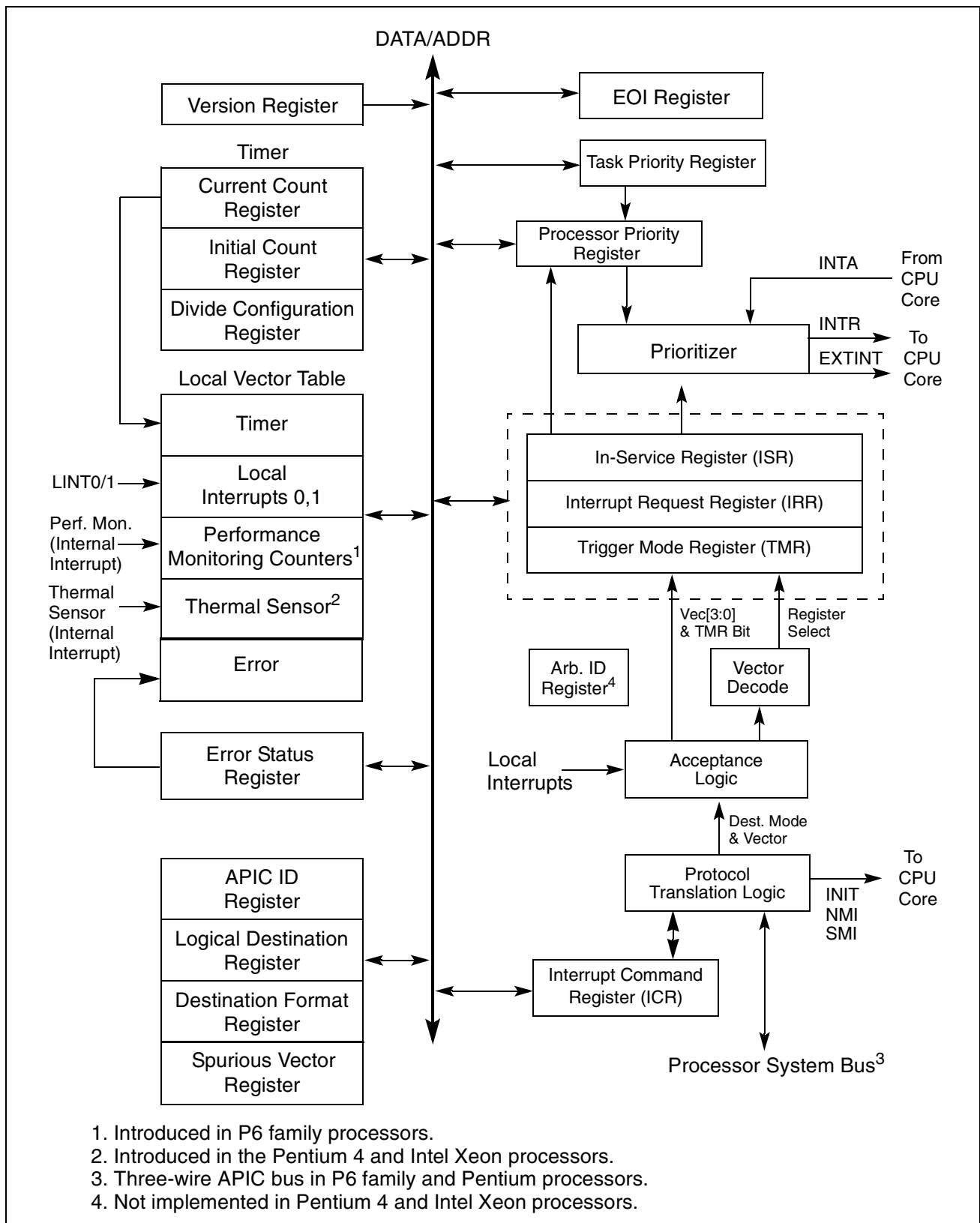


Figure 8-4. Local APIC Structure

Table 8-1 shows how the APIC registers are mapped into the 4-KByte APIC register space. All registers are 32 bits, 64 bits, or 256 bits in width, and all are aligned on 128-bit boundaries. All 32-bit registers must be accessed using 128-bit aligned 32-bit loads or stores. The wider registers (64-bit or 256-bit) must be accessed using multiple 32-bit loads or stores, with the first access being 128-bit aligned. If a LOCK prefix is used with a MOV instruction that accesses the APIC address space, the prefix is ignored; that is, a locking operation does not take place. All the registers listed in Table 8-1 are described in the following sections of this chapter.

Table 8-1. Local APIC Register Address Map

Address	Register Name	Software Read/Write
FEE0 0000H	Reserved	
FEE0 0010H	Reserved	
FEE0 0020H	Local APIC ID Register	Read/Write.
FEE0 0030H	Local APIC Version Register	Read Only.
FEE0 0040H	Reserved	
FEE0 0050H	Reserved	
FEE0 0060H	Reserved	
FEE0 0070H	Reserved	
FEE0 0080H	Task Priority Register (TPR)	Read/Write.
FEE0 0090H	Arbitration Priority Register ¹ (APR)	Read Only.
FEE0 00A0H	Processor Priority Register (PPR)	Read Only.
FEE0 00B0H	EOI Register	Write Only.
FEE0 00C0H	Reserved	
FEE0 00D0H	Logical Destination Register	Read/Write.
FEE0 00E0H	Destination Format Register	Bits 0-27 Read only; bits 28-31 Read/Write.
FEE0 00F0H	Spurious Interrupt Vector Register	Bits 0-8 Read/Write; bits 9-31 Read Only.
FEE0 0100H through FEE0 0170H	In-Service Register (ISR)	Read Only.
FEE0 0180H through FEE0 01F0H	Trigger Mode Register (TMR)	Read Only.
FEE0 0200H through FEE0 0270H	Interrupt Request Register (IRR)	Read Only.
FEE0 0280H	Error Status Register	Read Only.
FEE0 0290H through FEE0 02F0H	Reserved	
FEE0 0300H	Interrupt Command Register (ICR) [0-31]	Read/Write.
FEE0 0310H	Interrupt Command Register (ICR) [32-63]	Read/Write.

Table 8-1. Local APIC Register Address Map (Contd.)

Address	Register Name	Software Read/Write
FEE0 0320H	LVT Timer Register	Read/Write.
FEE0 0330H	LVT Thermal Sensor Register ²	Read/Write.
FEE0 0340H	LVT Performance Monitoring Counters Register ³	Read/Write.
FEE0 0350H	LVT LINT0 Register	Read/Write.
FEE0 0360H	LVT LINT1 Register	Read/Write.
FEE0 0370H	LVT Error Register	Read/Write.
FEE0 0380H	Initial Count Register (for Timer)	Read/Write.
FEE0 0390H	Current Count Register (for Timer)	Read Only.
FEE0 03A0H through FEE0 03D0H	Reserved	
FEE0 03E0H	Divide Configuration Register (for Timer)	Read/Write.
FEE0 03F0H	Reserved	

NOTES:

1. Not supported in the Pentium 4 and Intel Xeon processors.
2. Introduced in the Pentium 4 and Intel Xeon processors. This APIC register and its associated function are implementation dependent and may not be present in future IA-32 processors.
3. Introduced in the Pentium Pro processor. This APIC register and its associated function are implementation dependent and may not be present in future IA-32 processors.

NOTE

The local APIC registers listed in Table 8-1 are not MSRs. The only MSR associated with the programming of the local APIC is the IA32_APIC_BASE MSR (see Section 8.4.3., “Enabling or Disabling the Local APIC”).

8.4.2. Presence of the Local APIC

Beginning with the P6 family processors, the presence or absence of an on-chip local APIC can be detected using the CUID instruction. When the CUID instruction is executed with a source operand of 1 in the EAX register, bit 9 of the CUID feature flags returned in the EDX register indicates the presence (set) or absence (clear) of a local APIC.

8.4.3. Enabling or Disabling the Local APIC

The local APIC can be enabled or disabled in either of two ways:

- APIC global enable/disable flag in the IA32_APIC_BASE MSR (see Figure 8-5).
- APIC software enable/disable flag in the spurious-interrupt vector register (see Figure 8-22)

The APIC global enable/disable flag in the IA32_APIC_BASE MSR permits the local APIC to be permanently disabled. Following a power-up or reset, this flag is set, enabling the local APIC. To permanently disable the local APIC until the next power-up or reset, software can clear this flag. When this flag is clear, the processor is functionally equivalent to an IA-32 processor without an on-chip APIC (for example, an Intel486 processor). In this state, CPUID feature flag for the APIC (bit 9 for the EDX register [see Section 8.4.2., “Presence of the Local APIC”]) is set to 0. Also, when the APIC global enable/disable flag in the IA32_APIC_BASE MSR flag has been cleared, it can only be reset by a power-up or RESET action.

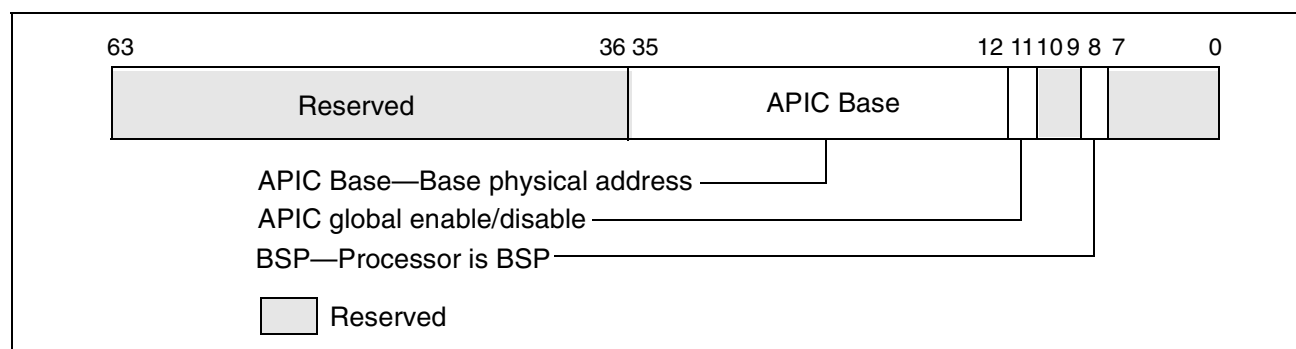


Figure 8-5. IA32_APIC_BASE MSR

For the Pentium processor, the APICEN pin (which is shared with the PICD1 pin) is used during power-up or RESET to disable the local APIC.

If the APIC global enable/disable flag in the IA32_APIC_BASE MSR has not been cleared, software can temporarily disable a local APIC at any time by clearing the APIC software enable/disable flag in the spurious-interrupt vector register (see Figure 8-22). The state of the local APIC when in this software-disabled state is described in Section 8.4.7.2., “Local APIC State After It Has Been Software Disabled”. When the local APIC is in the software-disabled state, it can be re-enabled at any time by setting the APIC software enable/disable flag to 1.

Note that each entry in the LVT has a mask bit that can be used to inhibit interrupts from being delivered to the processor from selected local interrupt sources (the LINT0 and LINT1 pins, the APIC timer, the performance-monitoring counters, the thermal sensor, and/or the internal APIC error detector).

8.4.4. Local APIC Status and Location

The status and location of the local APIC are contained in the IA32_APIC_BASE MSR (called APIC_BASE_MSR in the P6 family processors). This MSR is located at MSR address 27 (1BH). Figure 8-5 shows the encoding of the bits in this MSR. The functions of these bits are as follows:

BSP flag, bit 8 Indicates if the processor is the bootstrap processor (BSP) (see Section 7.5., “Multiple-Processor (MP) Initialization”). Following a power-up or RESET, this flag is set to 1 for the processor that was selected as the BSP and set to 0 for each of the remaining application processors (APs).

APIC Global Enable flag, bit 11

Enables (1) or disables (0) the local APIC (see Section 8.4.3., “Enabling or Disabling the Local APIC”). This flag is available in the Pentium 4, Intel Xeon, and P6 family processors. It is not guaranteed to be available or available at the same location in future IA-32 processors.

APIC Base field, bits 12 through 35

Specifies the base address of the APIC registers. This 24-bit value is extended by 12 bits at the low end to form the base address, which automatically aligns the address on a 4-KByte boundary. Following a power-up or RESET, this field is set to FEE00000H.

Bits 0 through 7, bits 9 and 10, and bits 36 through 63 in the IA32_APIC_BASE MSR are reserved.

8.4.5. Relocating the Local APIC Registers

The Pentium 4, Intel Xeon, and P6 family processors permit the starting address of the APIC registers to be relocated from FEE00000H to another physical address by modifying the value in the 24-bit base address field of the IA32_APIC_BASE MSR. This extension of the APIC architecture is provided to help resolve conflicts with memory maps of existing systems and to allow individual processors in an MP system to map their APIC registers to different locations in physical memory.

8.4.6. Local APIC ID

At power up, system hardware assigns a unique APIC ID to each local APIC on the system bus (for Pentium 4 and Intel Xeon processors) or on the APIC bus (for P6 family and Pentium processors). The hardware assigned APIC ID is based on system topology and includes encoding for socket position and cluster information (see Figure 7-2).

In MP systems, the local APIC ID is also used as a processor ID by the BIOS and the operating system.

The processor receives the hardware assigned APIC ID by sampling pins A11# and A12# and pins BR0# through BR3# (for the Pentium 4, Intel Xeon, and P6 family processors) and pins BE0# through BE3# (for the Pentium processor). The APIC ID latched from these pins is stored in the APIC ID field of the local APIC ID register (see Figure 8-6), and is used as the initial APIC ID for the processor. It is also the value returned to the EBX register, when the CUID instruction is executed with a source operand value of 1 in the EAX register.

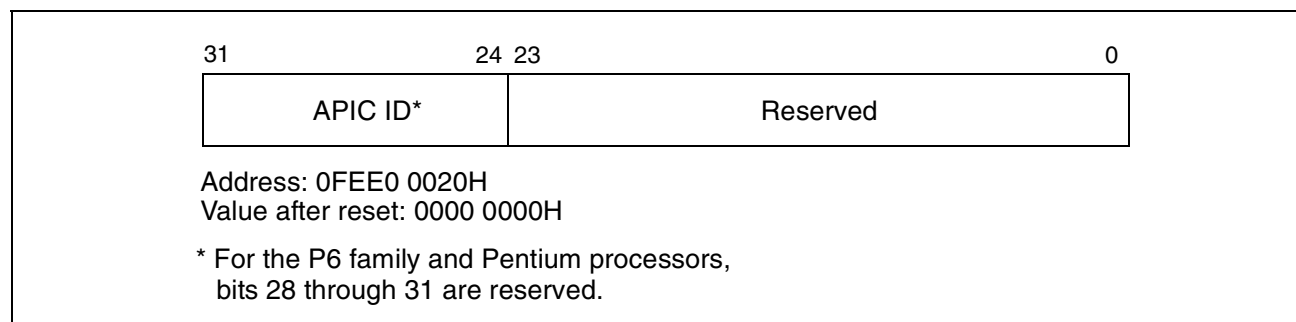


Figure 8-6. Local APIC ID Register

For the P6 family and Pentium processors, the local APIC ID field in the local APIC ID register is 4 bits, and encodings 0H through EH can be used to uniquely identify 15 different processors connected to the APIC bus. For the Pentium 4 and Intel Xeon processors, the xAPIC specification extends the local APIC ID field to 8 bits which can be used to identify up to 255 processors in the system.

Following power up or a hardware reset, software (typically the BIOS software) can modify the APIC ID field in the local APIC ID register for each processor in the system. When changing APIC IDs, software must insure that each APIC ID for each local APIC is unique throughout the system.

8.4.7. Local APIC State

The following sections describe the state of the local APIC and its registers following a power-up or RESET, after it has been software disabled, following an INIT reset, and following an INIT-deassert message.

8.4.7.1. LOCAL APIC STATE AFTER POWER-UP OR RESET

Following a power-up or RESET of the processor, the state of local APIC and its registers are as follows:

- The following registers are reset to all 0s: the IRR, ISR, TMR, ICR, LDR, and TPR registers; the timer initial count and timer current count registers; and the divide configuration register.
- The DFR register is reset to all 1s.
- The LVT register entries are reset to all 0s except for the mask bits, which are set to 1s.

- The local APIC version register is not affected.
- The local APIC ID register is set to a unique APIC ID. (Pentium and P6 family processors only) The Arb ID register is set to the value in the APIC ID register.
- The spurious-interrupt vector register is initialized to 0000 00FFH. The setting of bit 8 to 0 software disables the local APIC.
- If the processor is the only processor in the system or it is in an MP system and has been designated the BSP (see Section 7.5.1., “BSP and AP Processors”), the local APIC will respond normally to INIT and NMI messages, and to INIT# and STPCLK# signals; if it is in an MP system and has been designated as an AP, the local APIC will respond the same as for the BSP and in addition it will respond to a SIPI message. For P6 family processors only, an AP will not respond to a STPCLK# signal.

8.4.7.2. LOCAL APIC STATE AFTER IT HAS BEEN SOFTWARE DISABLED

When the APIC software enable/disable flag in the spurious interrupt vector register has been explicitly cleared (as opposed to being cleared during a power up or RESET), the local APIC is temporarily disabled (see Section 8.4.3., “Enabling or Disabling the Local APIC”). The operation and response of a local APIC while in this software-disabled state is as follows:

- The local APIC will respond normally to INIT, NMI, SMI, and SIPI messages.
- Pending interrupts in the IRR and ISR registers are held and require masking or handling by the CPU.
- A local APIC can still issue IPIs. It is software’s responsibility to avoid issuing IPIs through the IPI mechanism and the ICR register if sending interrupts through this mechanism is not desired.
- The reception or transmission of any IPIs that are in progress when the local APIC is disabled are completed before the local APIC enters the software-disabled state.
- The mask bits for all the LVT entries are set. Attempts to reset these bits will be ignored.
- (Pentium and P6 family processors) The local APIC continues to listen to all bus messages in order to keep its arbitration ID synchronized with the rest of the system.

8.4.7.3. LOCAL APIC STATE AFTER AN INIT RESET (“WAIT-FOR-SIPI” STATE)

An INIT reset of the processor can be initiated in either of two ways:

- By asserting the processor’s INIT# pin.
- By sending the processor an INIT IPI (sending it an IPI with the delivery mode set to INIT).

Upon receiving an INIT through either of these two mechanisms, the processor responds by beginning the initialization process of the processor core and the local APIC. The state of the local APIC following an INIT reset is the same as it is after a power-up or hardware RESET,

except that the APIC ID and arbitration ID registers are not affected. This state is also referred to as the “wait-for-SIPI” state. See Section 7.5.2., “MP Initialization Protocol Requirements and Restrictions for Intel Xeon Processors”, for a discussion of the effect of an INIT that follows a power-up or RESET in an MP system.

8.4.7.4. LOCAL APIC STATE AFTER IT RECEIVES AN INIT-DEASSERT IPI

(Only the Pentium and P6 family processors support the INIT-deassert IPI.) An INIT-disassert IPI has no effect on the state of the APIC, other than to reload the arbitration ID register with the value in the APIC ID register.

8.4.8. Local APIC Version Register

The local APIC contains a hardwired version register, which software can use to identify the APIC version (see Figure 8-7). In addition, this register specifies the number of entries in the local vector table (LVT) for the specific implementation. The fields in the local APIC version register are as follows:

Version	The version numbers of the local APIC:	
	1XH	Local APIC. For Pentium 4 and Intel Xeon processors, 14H is returned.
	0XH	82489DX external APIC.
	20H through FFH	Reserved.
Max LVT Entry	Shows the number of LVT entries minus 1. For the Pentium 4 and Intel Xeon processors (which have 6 LVT entries), the value returned in the Max LVT field is 5; for the P6 family processors (which have 5 LVT entries), the value returned is 4; for the Pentium processor (which has 4 LVT entries), the value returned is 3.	

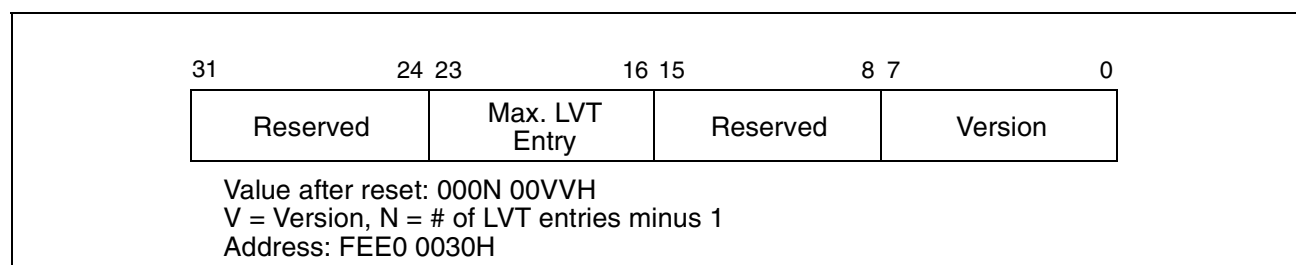


Figure 8-7. Local APIC Version Register

8.5. HANDLING LOCAL INTERRUPTS

The following sections describe the facilities that are provided in the local APIC for handling local interrupts, which include the processor's LINT0 and LINT1 pins, the APIC timer, the performance-monitoring counters, the thermal sensor, and the internal APIC error detector. The local interrupt handling facilities include the LVT, the error status register (ESR), the divide configuration register (DCR), and the initial count and current count registers.

8.5.1. Local Vector Table

The local vector table (LVT) allows software to specify the manner in which the local interrupts are delivered to the processor core. It consists of the following five 32-bit APIC registers (see Figure 8-8), one for each local interrupt:

- LVT Timer Register (FEE0 0320H)—Specifies interrupt delivery when the APIC timer signals an interrupt (see Section 8.5.4., “APIC Timer”).
- LVT Thermal Monitor Register (FEE0 0330H)—Specifies interrupt delivery when the thermal sensor generates an interrupt (see Section 13.15.2., “Thermal Monitor”). This LVT entry is implementation specific, not architectural. If implemented, it will always be at base address FEE0 0330H.
- LVT Performance Counter Register (FEE0 0340H)—Specifies interrupt delivery when a performance counter generates an interrupt on overflow (see Section 15.9.6.9., “Generating an Interrupt on Overflow”). This LVT entry is implementation specific, not architectural. If implemented, it is not guaranteed to be at base address FEE0 0340H.
- LVT LINT0 Register (FEE0 0350H)—Specifies interrupt delivery when an interrupt is signaled at the LINT0 pin.
- LVT LINT1 Register (FEE0 0360H)—Specifies interrupt delivery when an interrupt is signaled at the LINT1 pin.
- LVT Error Register (FEE0 0370H)—Specifies interrupt delivery when the APIC detects an internal error (see Section 8.5.3., “Error Handling”).

NOTE

The LVT performance counter register and its associated interrupt were introduced in the P6 processors and are also present in the Pentium 4 and Intel Xeon processors. The LVT thermal monitor register and its associated interrupt were introduced in the Pentium 4 and Intel Xeon processors

Note that as shown in Figures 8-8, some of these fields and flags are not available (and reserved) for some entries.

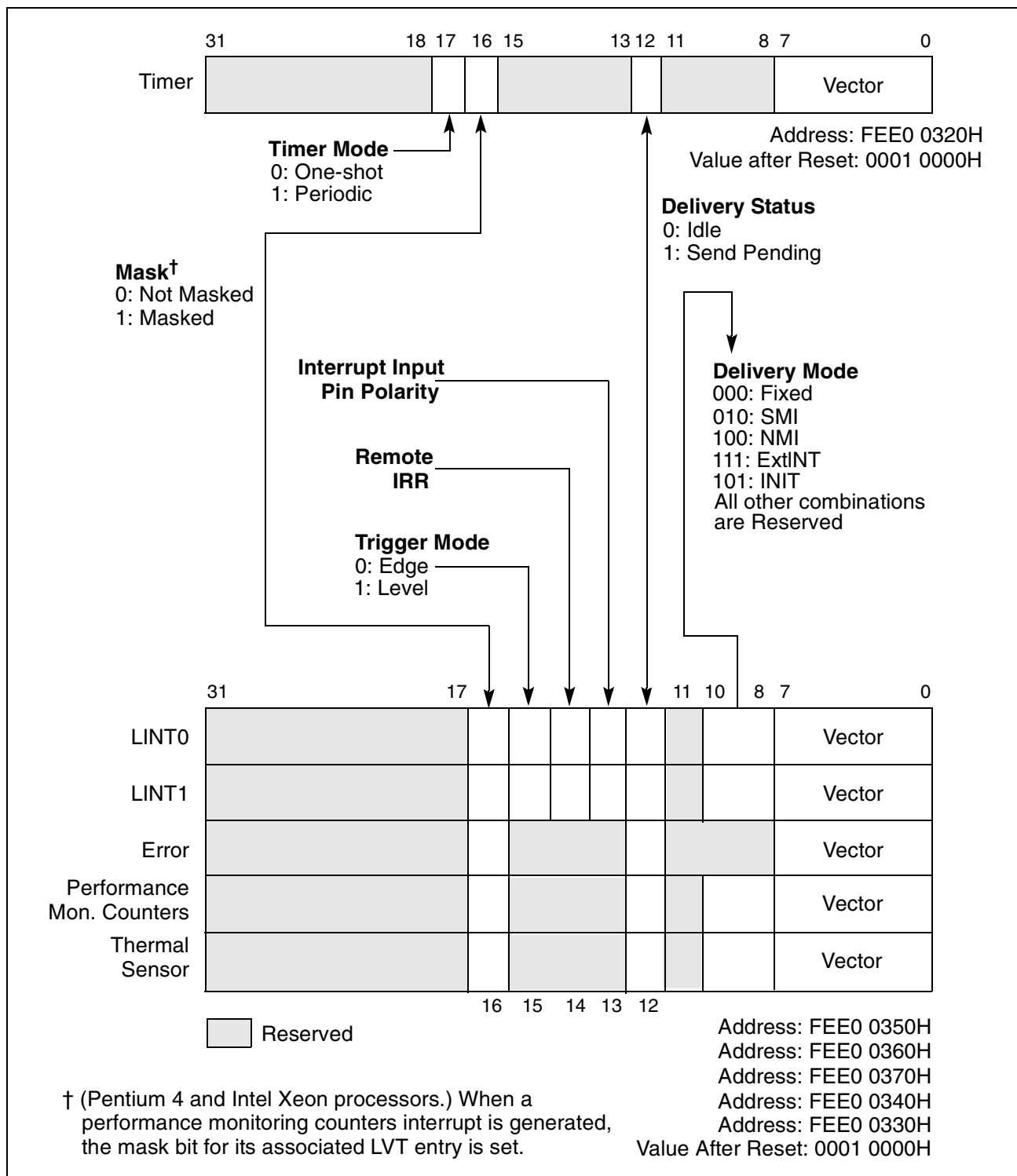


Figure 8-8. Local Vector Table (LVT)

The setup information that can be specified in the registers of the LVT table is as follows:

Vector	Interrupt vector number.
Delivery Mode	Specifies the type of interrupt to be sent to the processor. Note that some delivery modes will only operate as intended when used in conjunction with a specific trigger mode. The allowable delivery modes are as follows:
000 (Fixed)	Delivers the interrupt specified in the vector field.
010 (SMI)	Delivers an SMI interrupt to the processor core through the processor's local SMI signal path. When using this delivery mode, the vector field should be set to 00H for future compatibility.
100 (NMI)	Delivers an NMI interrupt to the processor. The vector information is ignored.
101 (INIT)	Delivers an INIT request to the processor core, which causes the processor to perform an INIT. When using this delivery mode, the vector field should be set to 00H for future compatibility.
111 (ExtINT)	Causes the processor to respond to the interrupt as if the interrupt originated in an externally connected (8259A-compatible) interrupt controller. A special INTA bus cycle corresponding to ExtINT, is routed to the external controller. The external controller is expected to supply the vector information. The APIC architecture supports only one ExtINT source in a system, usually contained in the compatibility bridge.

Delivery Status (Read Only)

Indicates the interrupt delivery status, as follows:

0 (Idle)	There is currently no activity for this interrupt source, or the previous interrupt from this source was delivered to the processor core and accepted.
1 (Send Pending)	Indicates that an interrupt from this source has been delivered to the processor core, but has not yet been accepted (see Section 8.5.5., "Local Interrupt Acceptance").

Interrupt Input Pin Polarity

Specifies the polarity of the corresponding interrupt pin: (0) active high or (1) active low.

Remote IRR Flag (Read Only)

For fixed mode, level-triggered interrupts, this flag is set when the local APIC accepts the interrupt for servicing and is reset when an EOI command is received from the processor. The meaning of this flag is undefined for edge-triggered interrupts and other delivery modes.

Trigger Mode

Selects the trigger mode for the local LINT0 and LINT1 pins: (0) edge sensitive and (1) level sensitive. This flag is only used when the delivery mode is Fixed. When the delivery mode is NMI, SMI, or INIT, the trigger mode is always edge sensitive; when the delivery mode is ExtINT, the trigger mode is always level sensitive. The timer and error interrupts are always treated as edge sensitive.

If the local APIC is not used in conjunction with an I/O APIC and fixed delivery mode is selected, the Pentium 4, Intel Xeon, and P6 family processors will always use level-sensitive triggering, regardless if edge-sensitive triggering is selected.

Mask

Interrupt mask: (0) enables reception of the interrupt and (1) inhibits reception of the interrupt. When the local APIC handles a performance-monitoring counters interrupt, it automatically sets the mask flag in the corresponding LVT entry. This flag will remain set until software clears it.

Timer Mode

Selects the timer mode: (0) one-shot and (1) periodic (see Section 8.5.4., “APIC Timer”).

8.5.2. Valid Interrupt Vectors

The IA-32 architecture defines 256 vector numbers, ranging from 0 through 255 (see Section 5.2., “Exception and Interrupt Vectors”). The local and I/O APICs support 240 of these vectors (in the range of 16 to 255) as valid interrupts.

When an interrupt vector in the range of 0 to 15 is sent or received through the local APIC, the APIC indicates an illegal vector in its Error Status Register [see Section 8.5.3., “Error Handling”]. The IA-32 architecture reserves vectors 16 through 31 for predefined interrupts, exceptions, and Intel-reserved encodings (see Table 5-1); however, the local APIC does not treat vectors in this range as illegal.

When an illegal vector value (0 to 15) is written to an LVT entry and the delivery mode is Fixed (bits 8-11 equal 0), the APIC may signal an illegal vector error, without regard to whether the mask bit is set or whether an interrupt is actually seen on the input.

8.5.3. Error Handling

The local APIC provides an error status register (ESR) that it uses to record errors that it detects when handling interrupts (see Figure 8-9). An APIC error interrupt is generated when the local APIC sets one of the error bits in the ESR. The LVT error register allows selection of the inter-

rupt vector to be delivered to the processor core when APIC error is detected. The LVT error register also provides a means of masking the APIC error interrupt.

The functions of the ESR flags are as follows:

Send Checksum Error	(P6 family and Pentium processors only.) Set when the local APIC detects a checksum error for a message that it sent on the APIC bus.
Receive Checksum Error	(P6 family and Pentium processors only.) Set when the local APIC detects a checksum error for a message that it received on the APIC bus.
Send Accept Error	(P6 family and Pentium processors only.) Set when the local APIC detects that a message it sent was not accepted by any APIC on the APIC bus.
Receive Accept Error	(P6 family and Pentium processors only.) Set when the local APIC detects that the message it received was not accepted by any APIC on the APIC bus, including itself.
Send Illegal Vector	Set when the local APIC detects an illegal vector in the message that it is sending.
Receive Illegal Vector	Set when the local APIC detects an illegal vector in the message it received, including an illegal vector code in the local vector table interrupts or in a self-interrupt.
Illegal Reg. Address	(Pentium 4, Intel Xeon, and P6 family processors only.) Set when the processor is trying to access a register that is not implemented in the processors' local APIC register address space; that is, within the address range of the APIC register base address (specified in the IA32_APIC_BASE MSR) plus 4K Bytes.

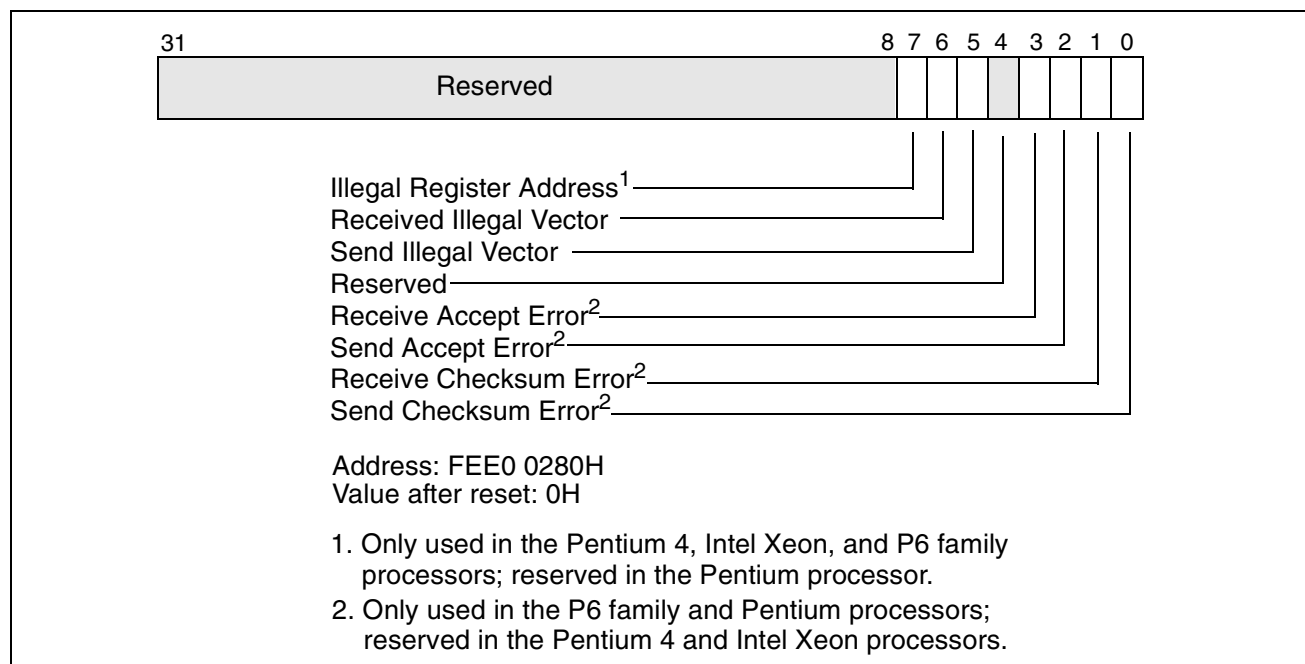


Figure 8-9. Error Status Register (ESR)

The ESR is a write/read register. A write (of any value) to the ESR must be done just prior to reading the ESR to update the register. This initial write causes the ESR contents to be updated with the latest error status. Back-to-back writes clear the ESR register.

After an error bit is set in the register, it remains set until the register is cleared. Setting the mask bit for the LVT error register prevents errors from being recorded in the ESR; however, the state of the ESR before the mask bit was set is maintained.

8.5.4. APIC Timer

The local APIC unit contains a 32-bit programmable timer that is available to software to time events or operations. This timer is set up by programming four registers: the divide configuration register (see Figure 8-10), the initial-count and current-count registers (see Figure 8-11), and the LVT timer register (see Figure 8-8).

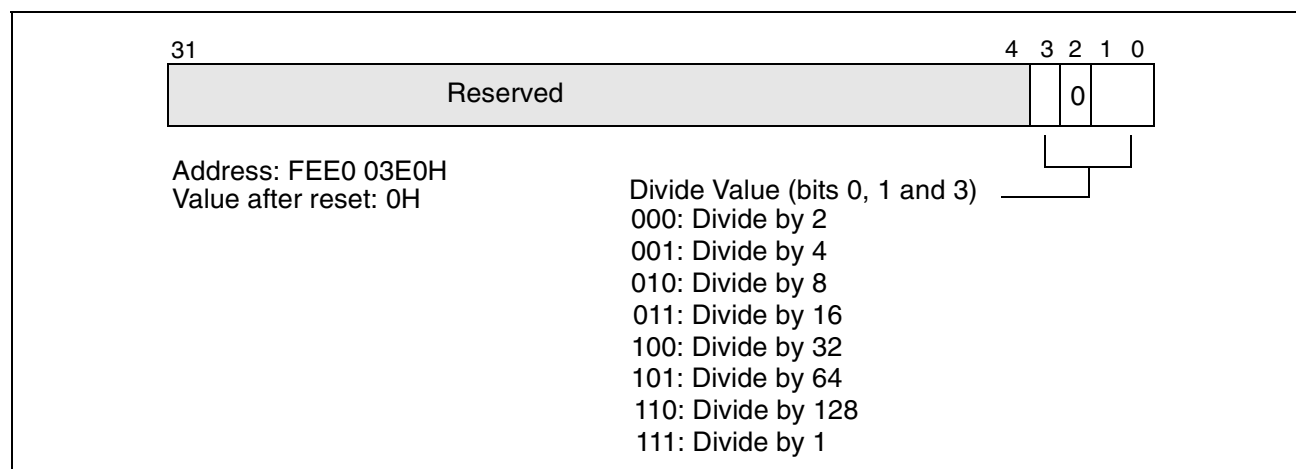


Figure 8-10. Divide Configuration Register

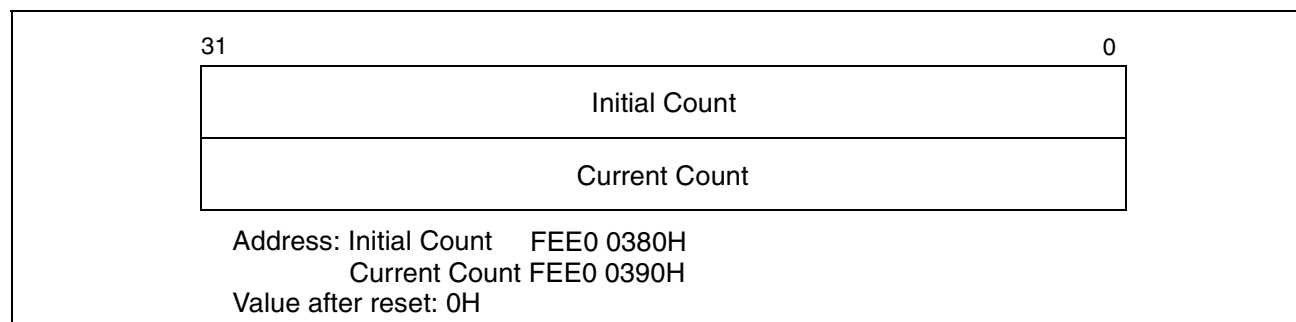


Figure 8-11. Initial Count and Current Count Registers

The time base for the timer is derived from the processor's bus clock, divided by the value specified in the divide configuration register.

The timer can be configured through the timer LVT entry for one-shot or periodic operation. In one-shot mode, the timer is started by programming its initial-count register. The initial count

value is then copied into the current-count register and count-down begins. After the timer reaches zero, an timer interrupt is generated and the timer remains at its 0 value until reprogrammed.

In periodic mode, the current-count register is automatically reloaded from the initial-count register when the count reaches 0 and a timer interrupt is generated, and the count-down is repeated. If during the count-down process the initial-count register is set, counting will restart, using the new initial-count value. The initial-count register is a read-write register; the current-count register is read only.

The LVT timer register determines the vector number that is delivered to the processor with the timer interrupt that is generated when the timer count reaches zero. The mask flag in the LVT timer register can be used to mask the timer interrupt.

8.5.5. Local Interrupt Acceptance

When a local interrupt is sent to the processor core, it is subject to the acceptance criteria specified in the interrupt acceptance flow chart in Figure 8-17. If the interrupt is accepted, it is logged into the IRR register and handled by the processor according to its priority (see Section 8.8.4., “Interrupt Acceptance for Fixed Interrupts”). If the interrupt is not accepted, it is sent back to the local APIC and retried.

8.6. ISSUING INTERPROCESSOR INTERRUPTS

The following sections describe the local APIC facilities that are provided for issuing interprocessor interrupts (IPIs) from software. The primary local APIC facility for issuing IPIs is the interrupt command register (ICR). The ICR can be used for the following functions:

- To send an interrupt to another processor.
- To allow a processor to forward an interrupt that it received but did not service to another processor for servicing.
- To direct the processor to interrupt itself (perform a self interrupt).
- To deliver special IPIs, such as the start-up IPI (SIPI) message, to other processors or to itself.

Interrupts generated with this facility are delivered to the other processors in the system through the system bus (for Pentium 4 and Intel Xeon processors) or the APIC bus (for P6 family and Pentium processors).

8.6.1. Interrupt Command Register (ICR)

The interrupt command register (ICR) is a 64-bit local APIC register (see Figure 8-12) that allows software running on the processor to specify and send interprocessor interrupts (IPIs) to other IA-32 processors in the system.

To send an IPI, software must set up the ICR to indicate the type of IPI message to be sent and the destination processor or processors. (All fields of the ICR are read-write by software with the exception of the delivery status field, which is read-only.) The act of writing to the low doubleword of the ICR causes the IPI to be sent.

The ICR consists of the following fields.

Vector The vector number of the interrupt being sent.

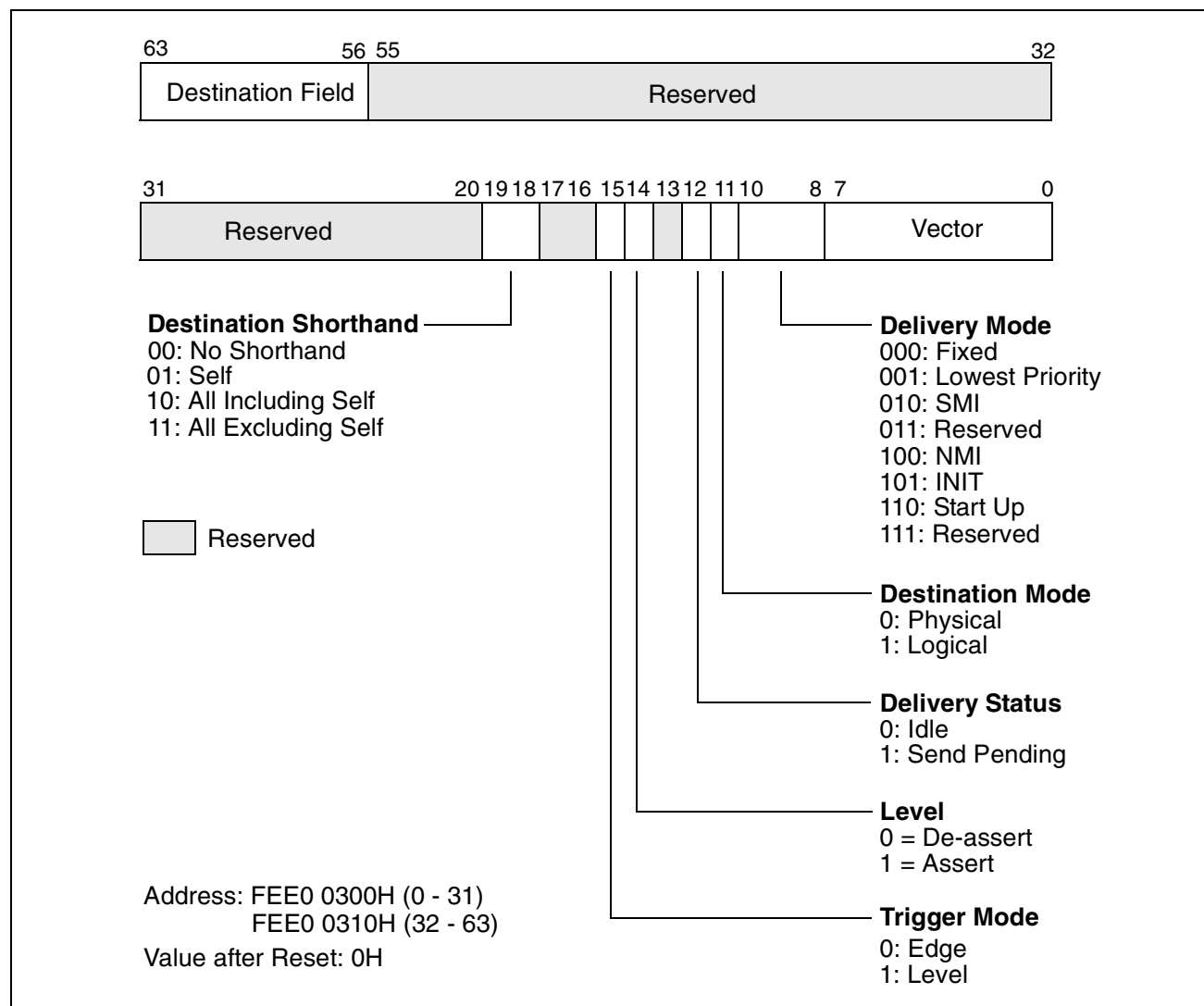


Figure 8-12. Interrupt Command Register (ICR)

Delivery Mode Specifies the type of IPI to be sent. This field is also know as the IPI message type field.

000 (Fixed) Delivers the interrupt specified in the vector field to the target processor or processors.

001 (Lowest Priority)

Same as fixed mode, except that the interrupt is delivered to the processor executing at the lowest priority among the set of processors specified in the destination field. (For Pentium 4 and Intel Xeon processors, use of this delivery mode is not recommended because it may cause multiple IPIs to be sent, which degrades performance.)

010 (SMI)

Delivers an SMI interrupt to the target processor or processors. The vector field must be programmed to 00H for future compatibility.

011 (Reserved)

100 (NMI)

Delivers an NMI interrupt to the target processor or processors. The vector information is ignored.

101 (INIT)

Delivers an INIT request to the target processor or processors, which causes them to perform an INIT. As a result of this IPI message, all the target processors perform an INIT. The vector field must be programmed to 00H for future compatibility.

101 (INIT Level De-assert)

(Not supported in the Pentium 4 and Intel Xeon processors.) Sends a synchronization message to all the local APICs in the system to set their arbitration IDs (stored in their Arb ID registers) to the values of their APIC IDs (see Section 8.7., “System and APIC Bus Arbitration”). For this delivery mode, the level flag must be set to 0 and trigger mode flag to 1. This IPI is sent to all processors, regardless of the value in the destination field or the destination shorthand field; however, software should specify the “all including self” shorthand.

110 (Start-Up)

Sends a special “start-up” IPI (called a SIPI) to the target processor or processors. The vector typically points to a start-up routine that is part of the BIOS boot-strap code (see Section 7.5., “Multiple-Processor (MP) Initialization”). Note that IPIs sent with this delivery mode are not automatically re-tried if the source APIC is unable to deliver it. It is up to the software to determine if the SIPI was not successfully delivered and to reissue the SIPI if necessary.

Destination Mode	Selects either physical (0) or logical (1) destination mode (see Section 8.6.2., “Determining IPI Destination”).				
Delivery Status (Read Only)	Indicates the IPI delivery status, as follows: <table> <tr> <td>0 (Idle)</td><td>There is currently no IPI activity for this local APIC, or the previous IPI sent from this local APIC was delivered and accepted by the target processor or processors.</td></tr> <tr> <td>1 (Send Pending)</td><td>Indicates that the last IPI sent from this local APIC has not yet been accepted by the target processor or processors.</td></tr> </table>	0 (Idle)	There is currently no IPI activity for this local APIC, or the previous IPI sent from this local APIC was delivered and accepted by the target processor or processors.	1 (Send Pending)	Indicates that the last IPI sent from this local APIC has not yet been accepted by the target processor or processors.
0 (Idle)	There is currently no IPI activity for this local APIC, or the previous IPI sent from this local APIC was delivered and accepted by the target processor or processors.				
1 (Send Pending)	Indicates that the last IPI sent from this local APIC has not yet been accepted by the target processor or processors.				
Level	For the INIT level de-assert delivery mode this flag must be set to 0; for all other delivery modes it must be set to 1. (This flag has no meaning in Pentium 4 and Intel Xeon processors, and will always be issued as a 1.)				
Trigger Mode	Selects the trigger mode when using the INIT level de-assert delivery mode: edge (0) or level (1). It is ignored for all other delivery modes. (This flag has no meaning in Pentium 4 and Intel Xeon processors, and will always be issued as a 0.)				
Destination Shorthand	Indicates whether a shorthand notation is used to specify the destination of the interrupt and, if so, which shorthand is used. Destination shorthands are used in place of the 8-bit destination field, and can be sent by software using a single write to the low doubleword of the ICR. Shorthands are defined for the following cases: software self interrupt, IPIs to all processors in the system including the sender, IPIs to all processors in the system excluding the sender. <table> <tr> <td>00: (No Shorthand)</td><td>The destination is specified in the destination field.</td></tr> <tr> <td>01: (Self)</td><td>The issuing APIC is the one and only destination of the IPI. This destination shorthand allows software to interrupt the processor on which it is executing. An APIC implementation is free to deliver the self-interrupt message internally or to issue the message to the bus and “snoop” it as with any other IPI message.</td></tr> </table>	00: (No Shorthand)	The destination is specified in the destination field.	01: (Self)	The issuing APIC is the one and only destination of the IPI. This destination shorthand allows software to interrupt the processor on which it is executing. An APIC implementation is free to deliver the self-interrupt message internally or to issue the message to the bus and “snoop” it as with any other IPI message.
00: (No Shorthand)	The destination is specified in the destination field.				
01: (Self)	The issuing APIC is the one and only destination of the IPI. This destination shorthand allows software to interrupt the processor on which it is executing. An APIC implementation is free to deliver the self-interrupt message internally or to issue the message to the bus and “snoop” it as with any other IPI message.				

10: (All Including Self)

The IPI is sent to all processors in the system including the processor sending the IPI. The APIC will broadcast an IPI message with the destination field set to FH for Pentium and P6 family processors and to FFH for Pentium 4 and Intel Xeon processors.

11: (All Excluding Self)

The IPI is sent to all processors in the system with the exception of the processor sending the IPI. The APIC will broadcast a message using the physical destination mode and destination field set to FH for Pentium and P6 family processors and to FFH for Pentium 4 and Intel Xeon processors. (For Pentium 4 and Intel Xeon processors, when this destination shorthand is used in conjunction with the lowest-priority delivery mode, the IPI may be redirected back to the issuing processor.)

Destination

Specifies the target processor or processors. This field is only used when the destination shorthand field is set to 00B. If the destination mode is set to physical, then bits 56 through 59 contain the APIC ID of the target processor for Pentium and P6 family processors and bits 56 through 63 contain the APIC ID of the target processor the for Pentium 4 and Intel Xeon processors. If the destination mode is set to logical, the interpretation of the 8-bit destination field depends on the settings of the DFR and LDR registers of the local APICs in all the processors in the system (see Section 8.6.2., “Determining IPI Destination”).

Note that not all the combinations of options for the ICR are valid. Table 8-2 shows the valid combinations for the fields in the ICR for the Pentium 4 and Intel Xeon processors; Table 8-3 shows the valid combinations for the fields in the ICR for the P6 family processors.

Table 8-2. Valid Combinations for the Pentium 4 and Intel Xeon Processors’ Local xAPIC Interrupt Command Register

Destination Shorthand	Valid/Invalid	Trigger Mode	Delivery Mode	Destination Mode
No Shorthand	Valid	Edge	All Modes	Physical or Logical
No Shorthand	Invalid ¹	Level	All Modes	Physical or Logical
Self	Valid	Edge	Fixed	X ²
Self	Invalid ¹	Level	Fixed	X
Self	Invalid	X	Lowest Priority, NMI, INIT, SMI, Start-Up	X
All Including Self	Valid	Edge	Fixed	X
All Including Self	Invalid ¹	Level	Fixed	X

Table 8-2. Valid Combinations for the Pentium 4 and Intel Xeon Processors' Local xAPIC Interrupt Command Register (Contd.)

Destination Shorthand	Valid/Invalid	Trigger Mode	Delivery Mode	Destination Mode
All Including Self	Invalid	X	Lowest Priority, NMI, INIT, SMI, Start-Up	X
All Excluding Self	Valid	Edge	Fixed, Lowest Priority ³ , NMI, INIT, SMI, Start-Up	X
All Excluding Self	Invalid ¹	Level	Fixed, Lowest Priority ³ , NMI, INIT, SMI, Start-Up	X

NOTES:

1. For these interrupts, if the trigger mode bit is 1 (Level), the local xAPIC will override the bit setting and issue the interrupt as an edge triggered interrupt.
2. X—don't care.
3. When using the “lowest priority” delivery mode and the “all excluding self” destination, the IPI can be redirected back to the issuing APIC, which is essentially the same as the “all including self” destination mode.

Table 8-3. Valid Combinations for the P6 Family Processors' Local APIC Interrupt Command Register

Destination Shorthand	Valid/Invalid	Trigger Mode	Delivery Mode	Destination Mode
No Shorthand	Valid	Edge	All Modes	Physical or Logical
No Shorthand	Valid ¹	Level	Fixed, Lowest Priority, NMI	Physical or Logical
No Shorthand	Valid ²	Level	INIT	Physical or Logical
Self	Valid	Edge	Fixed	X ³
Self	1	Level	Fixed	X
Self	Invalid ⁴	X	Lowest Priority, NMI, INIT, SMI, Start-Up	X
All including Self	Valid	Edge	Fixed	X
All including Self	Valid ¹	Level	Fixed	X
All including Self	Invalid ⁴	X	Lowest Priority, NMI, INIT, SMI, Start-Up	X
All excluding Self	Valid	Edge	All Modes	X
All excluding Self	Valid ¹	Level	Fixed, Lowest Priority, NMI	X
All excluding Self	Invalid ⁴	Level	SMI, Start-Up	X
All excluding Self	Valid ²	Level	INIT	X
X	Invalid ⁴	Level	SMI, Start-Up	X

NOTES:

1. Treated as edge triggered if level bit is set to 1, otherwise ignored.
2. Treated as edge triggered when Level bit is set to 1; treated as “INIT Level Deassert” message when level bit is set to 0 (deassert). Only INIT level deassert messages are allowed to have the level bit set to 0. For all other messages the level bit must be set to 1.
3. X—Don't care.
4. The behavior of the APIC is undefined.

8.6.2. Determining IPI Destination

The destination of an IPI can be one, all, or a subset (group) of the processors on the system bus. The sender of the IPI specifies the destination of an IPI with the following APIC registers and fields within the registers:

- The ICR register—The following fields in the ICR register are used to specify the destination of an IPI:
 - Destination Mode—selects one of two destination modes (physical or logical).
 - Destination field—In physical destination mode, used to specify the APIC ID of the destination processor; in logical destination mode, used to specify a message destination address (MDA) that can be used to select specific processors in clusters.
 - Destination Shorthand—A quick method of specifying all processors, all excluding self, or self as the destination.
 - Delivery mode, Lowest Priority—Specifies that a lowest-priority arbitration mechanism be used to select a destination processor from a specified group of processors.
- Local destination register (LDR)—Used in conjunction with the logical destination mode and MDAs to select the destination processors.
- Destination format register (DFR)—Used in conjunction with the logical destination mode and MDAs to select the destination processors.

How the ICR, LDR, and DFR are used to select an IPI destination depends on the destination mode used: physical, logical, broadcast/self, or lowest-priority delivery mode. These destination modes are described in the following sections.

8.6.2.1. PHYSICAL DESTINATION MODE

In physical destination mode, the destination processor is specified by its local APIC ID (see Section 8.4.6., “Local APIC ID”). For Pentium 4 and Intel Xeon processors, either a single destination (the local APIC ID is 00H through FEH) or a broadcast to all APICs (the APIC ID is FFH) can be specified in physical destination mode. This APIC ID mechanism allows up to 255 local APICs can be individually addressed on a single system bus.

For the P6 family and Pentium processors, a single destination is specified in physical destination mode with a local APIC ID of 0H through 0EH, allowing up to 15 local APICs to be addressed on the APIC bus. A broadcast to all local APICs is specified with 0FH.

NOTE

The actual number of local APICs that can be addressed on the system bus may be restricted by hardware.

8.6.2.2. LOGICAL DESTINATION MODE

In logical destination mode, IPI destination is specified using an 8-bit message destination address (MDA), which is entered in the destination field of the ICR. Upon receiving an IPI message that was sent using logical destination mode, a local APIC compares the MDA in the message with the values in its LDR and DFR to determine if it should accept and handle the IPI.

Figure 8-13 shows the layout of the logical destination register (LDR). The 8-bit logical APIC ID field in this register is used to create an identifier that can be compared with the MDA.

NOTE

The logical APIC ID should not be confused with the local APIC ID that is contained in the local APIC ID register.

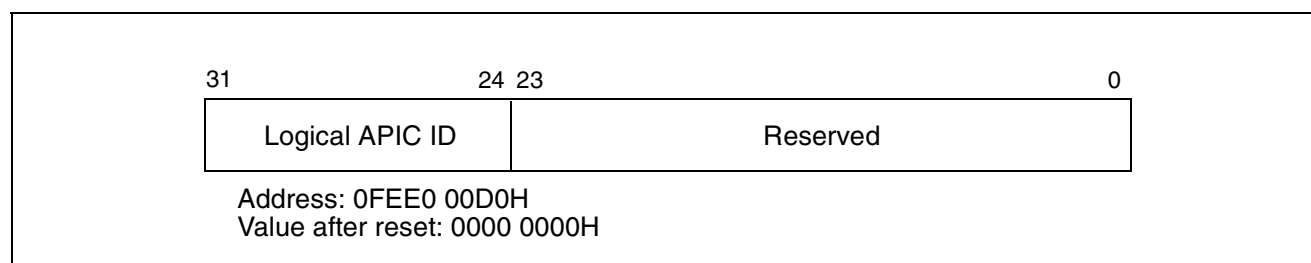


Figure 8-13. Logical Destination Register (LDR)

Figure 8-14 shows the layout of the destination format register (DFR). The 4-bit model field in this register selects one of two models (flat or cluster) that can be used to interpret the MDA when using logical destination mode.

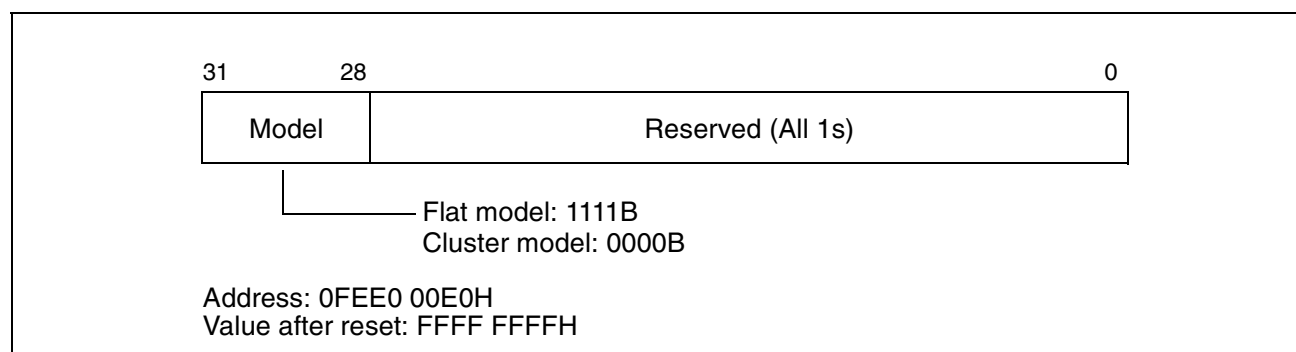


Figure 8-14. Destination Format Register (DFR)

The interpretation of MDA for the two models is described in the following paragraphs.

Flat Model. This model is selected by programming DFR bits 28 through 31 to 1111. Here, a unique logical APIC ID can be established for up to 8 local APICs by setting a different bit in the logical APIC ID field of the LDR for each local APIC. An group of local APICs can then be selected by setting one or more bits in the MDA.

Each local APIC performs a bit-wise AND of the MDA and its logical APIC ID. If a true condition is detected, the local APIC accepts the IPI message. A broadcast to all APICs is achieved by setting the MDA to all 1s.

Cluster Model. This model is selected by programming DFR bits 28 through 31 to 0000. This model supports two basic destination schemes: flat cluster and hierarchical cluster.

The flat cluster destination model is only supported for P6 family and Pentium processors. Using this model, all APICs are assumed to be connected through the APIC bus. Bits 28 through 31 of the MDA contains the encoded address of the destination cluster, and bits 24 through 27 identify up to four local APICs within the cluster (each bit is assigned to one local APIC in the cluster, as in the flat connection model). To identify one or more local APICs, bits 28 through 31 of the MDA are compared with bits 28 through 31 of the LDR to determine if a local APIC is part of the cluster. Bits 24 through 27 of the MDA are compared with Bits 24 through 27 of the LDR to identify a local APICs within the cluster.

Sets of processors within a cluster can be specified by writing the target cluster address in bits 28 through 31 of the MDA and setting selected bits in bits 24 through 27 of the MDA, corresponding to the chosen members of the cluster. In this mode, 15 clusters (with cluster addresses of 0 through 14) each having 4 local APICs can be specified in the message. For the P6 and Pentium processor's local APICs, however, the APIC arbitration ID supports only 15 APIC agents, and hence the total number of processors and their local APICs supported in this mode is limited to 15. Broadcast to all local APICs is achieved by setting all destination bits to one. This guarantees a match on all clusters, and selects all APICs in each cluster.

The hierarchical cluster destination model can be used with Pentium 4, Intel Xeon, P6 family, or Pentium processors. With this model, a hierarchical network can be created by connecting different flat clusters via independent system or APIC buses. This scheme requires a cluster manager within each cluster, which is responsible for handling message passing between system or APIC buses. One cluster contains up to 4 agents. Thus 15 cluster managers, each with 4 agents, can form a network of up to 60 APIC agents. Note that hierarchical APIC networks requires a special cluster manager device, which is not part of the local or the I/O APIC units.

8.6.2.3. BROADCAST/SELF DELIVERY MODE

The destination shorthand field of the ICR allows the delivery mode to be by-passed in favor of broadcasting the IPI to all the processors on the system bus and/or back to itself (see Section 8.6.1., "Interrupt Command Register (ICR)"). Three destination shorthands are supported: self, all excluding self, and all including self. The destination mode is ignored when a destination shorthand is used.

8.6.2.4. LOWEST PRIORITY DELIVERY MODE

With lowest priority delivery mode, the ICR is programmed to send an IPI to several processors on the system bus, using the logical or shorthand destination mechanism for selecting the processor. The selected processors then arbitrate with one another over the system bus or the APIC bus, with the lowest-priority processor accepting the IPI.

For systems based on the Intel Xeon processor, the chipset bus controller accepts messages from the I/O APIC agents in the system and directs interrupts to the processors on the system bus. When using the lowest priority delivery mode, the chipset chooses a target processor to receive the interrupt out of the set of possible targets. The Pentium 4 processor provides a special bus cycle on the system bus that informs the chipset of the current task priority for each logical processor in the system. The chipset saves this information and uses it to choose the lowest priority processor when an interrupt is received.

For systems based on P6 family processors, the processor priority used in lowest-priority arbitration is contained in the arbitration priority register (APR) in each local APIC. Figure 8-15 shows the layout of the APR.

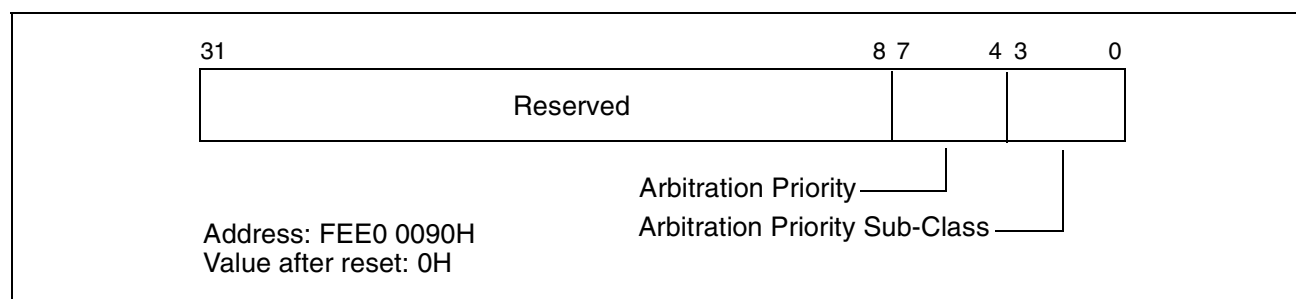


Figure 8-15. Arbitration Priority Register (APR)

The APR value is computed as follows:

```

IF (TPR[7:4] ≥ IRRV[7:4]) AND (TPR[7:4] > ISRV[7:4])
  THEN
    APR[7:0] ← TPR[7:0]
  ELSE
    APR[7:4] ← max(TPR[7:4] AND ISRV[7:4], IRRV[7:4])
    APR[3:0] ← 0.

```

Here, the TPR value is the task priority value in the TPR (see Figure 8-18), the IRRV value is the vector number for the highest priority bit that is set in the IRR (see Figure 8-20) or 00H (if no IRR bit is set), and the ISRV value is the vector number for the highest priority bit that is set in the ISR (see Figure 8-20). Following arbitration among the destination processors, the processor with the lowest value in its APR handles the IPI and the other processors ignore it.

(P6 family and Pentium processors.) For these processors, if a **focus processor** exists, it may accept the interrupt, regardless of its priority. A processor is said to be the focus of an interrupt if it is currently servicing that interrupt or if it has a pending request for that interrupt. For Intel Xeon processors, the concept of a focus processor is not supported.

In operating systems that use the lowest priority delivery mode but do not update the TPR, the TPR information saved in the chipset will potentially cause the interrupt to be always delivered to the same processor from the logical set. This behavior is functionally backward compatible with the P6 family processor but may result in unexpected performance implications.

8.6.3. IPI Delivery and Acceptance

When the low double-word of the ICR is written to, the local APIC creates an IPI message from the information contained in the ICR and sends the message out on the system bus (Pentium 4 and Intel Xeon processors) or the APIC bus (P6 family and Pentium processors). The manner in which these IPIs are handled after being issued is described in Section 8.8., “Handling Interrupts”.

8.7. SYSTEM AND APIC BUS ARBITRATION

When several local APICs and the I/O APIC are sending IPI and interrupt messages on the system bus (or APIC bus), the order in which the messages are sent and handled is determined through bus arbitration.

For the Pentium 4 and Intel Xeon processors, the local and I/O APICs use the arbitration mechanism defined for the system bus to determine the order in which IPIs are handled. This mechanism is non-architectural and cannot be controlled by software.

For the P6 family and Pentium processors, the local and I/O APICs use an APIC-based arbitration mechanism to determine the order in which IPIs are handled. Here, each local APIC is given an arbitration priority of from 0 to 15, which the I/O APIC uses during arbitration to determine which local APIC should be given access to the APIC bus. The local APIC with the highest arbitration priority always wins bus access. Upon completion of an arbitration round, the winning local APIC lowers its arbitration priority to 0 and the losing local APICs each raise theirs by 1.

The current arbitration priority for a local APIC is stored in a 4-bit, software-transparent arbitration ID (Arb ID) register. During reset, this register is initialized to the APIC ID number (stored in the local APIC ID register). The INIT level-deassert IPI, which is issued with an ICR command, can be used to resynchronize the arbitration priorities of the local APICs by resetting Arb ID register of each agent to its current APIC ID value. (The Pentium 4 and Intel Xeon processors do not implement the Arb ID register.)

Section 8.10., “APIC Bus Message Passing Mechanism and Protocol (P6 Family and Pentium Processors Only)”, describes the APIC bus arbitration protocols and bus message formats, while Section 8.6.1., “Interrupt Command Register (ICR)”, describes the INIT level de-assert IPI message.

Note that except for the SIPI IPI (see Section 8.6.1., “Interrupt Command Register (ICR)”), all bus messages that fail to be delivered to their specified destination or destinations are automatically retried. Software should avoid situations in which IPIs are sent to disabled or nonexistent local APICs, causing the messages to be resent repeatedly.

8.8. HANDLING INTERRUPTS

When a local APIC receives an interrupt from a local source, an interrupt message from an I/O APIC, or an IPI, the manner in which it handles the message depends on processor implementation, as described in the following sections.

8.8.1. Interrupt Handling with the Pentium 4 and Intel Xeon Processors

With the Pentium 4 and Intel Xeon processors, the local APIC handles the local interrupts, interrupt messages, and IPIs it receives as follows:

1. It determines if it is the specified destination or not (see Figure 8-16). If it is the specified destination, it accepts the message; if it is not, it discards the message.

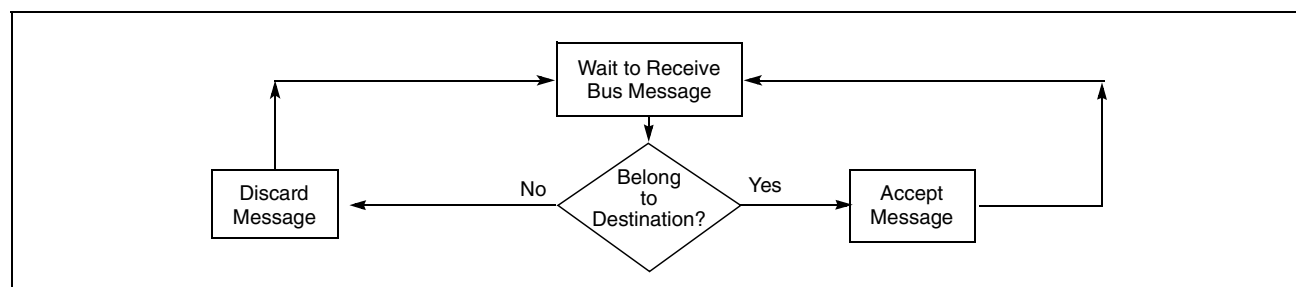


Figure 8-16. Interrupt Acceptance Flow Chart for the Local APIC (Pentium 4 and Intel Xeon Processors)

2. If the local APIC determines that it is the designated destination for the interrupt and if the interrupt request is an NMI, SMI, INIT, ExtINT, or SIPI, the interrupt is sent directly to the processor core for handling.
3. If the local APIC determines that it is the designated destination for the interrupt but the interrupt request is not one of the interrupts given in step 2, the local APIC sets the appropriate bit in the IRR.
4. When interrupts are pending in the IRR and ISR register, the local APIC dispatches them to the processor one at a time, based on their priority and the current task and processor priorities in the TPR and PPR (see Section 8.8.3.1., “Task and Processor Priorities”).
5. When a fixed interrupt has been dispatched to the processor core for handling, the completion of the handler routine is indicated with an instruction in the instruction handler code that writes to the end-of-interrupt (EOI) register in the local APIC (see Section 8.8.5., “Signaling Interrupt Servicing Completion”). The act of writing to the EOI register causes the local APIC to delete the interrupt from its ISR queue and (for level-triggered interrupts) send a message on the bus indicating that the interrupt handling has been completed. (A write to the EOI register must not be included in the handler routine for an NMI, SMI, INIT, ExtINT, or SIPI.)

8.8.2. Interrupt Handling with the P6 Family and Pentium Processors

With the P6 family and Pentium processors, the local APIC handles the local interrupts, interrupt messages, and IPIs it receives as follows (see Figure 8-17).

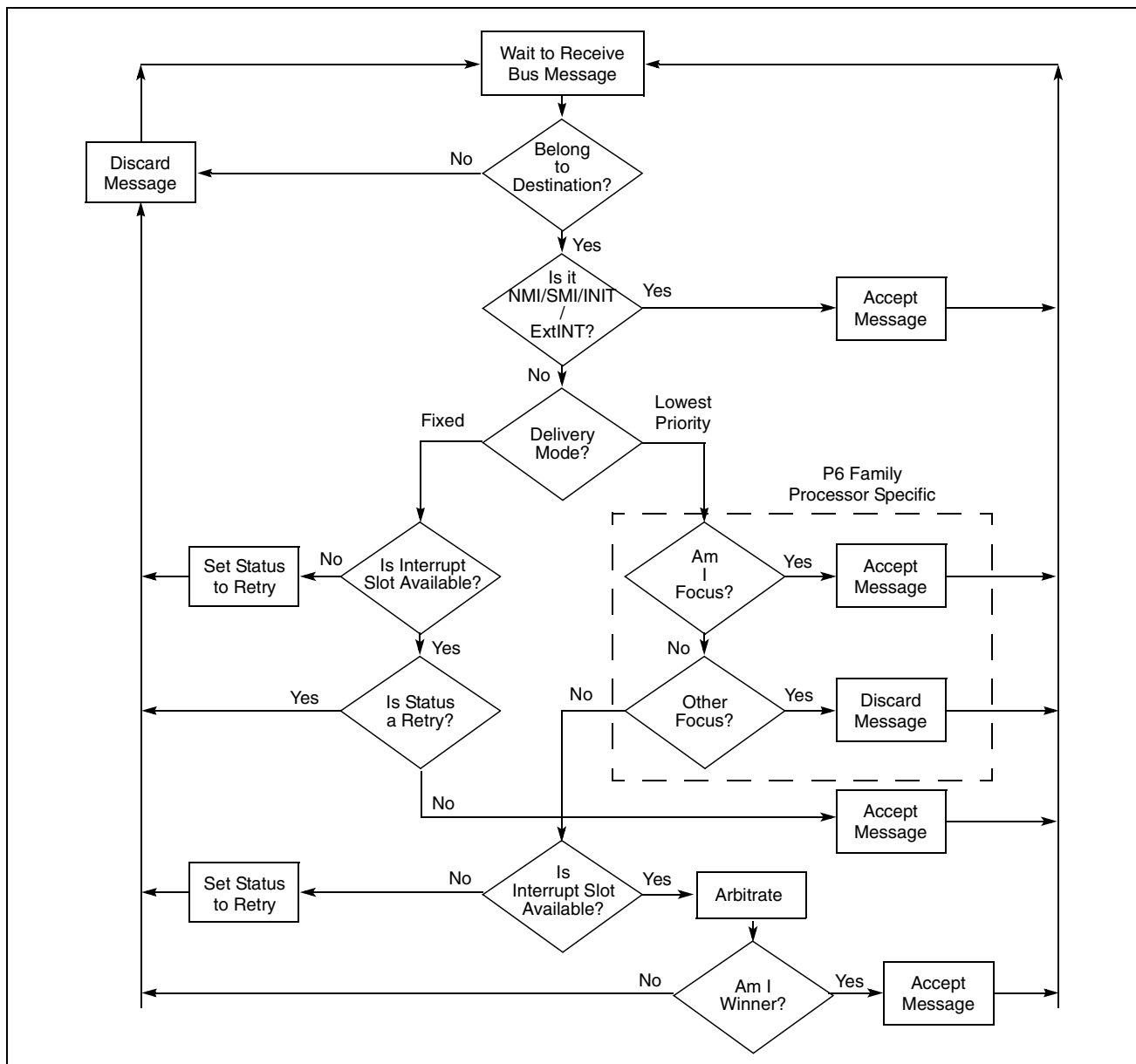


Figure 8-17. Interrupt Acceptance Flow Chart for the Local APIC (P6 Family and Pentium Processors)

1. (IPIs only.) It examines the IPI message to determine if it is the specified destination for the IPI as described in Section 8.6.2., “Determining IPI Destination”. If it is the specified destination, it continues its acceptance procedure; if it is not the destination, it discards the IPI message. When the message specifies lowest-priority delivery mode, the local APIC will arbitrate with the other processors that were designated on recipients of the IPI message (see Section 8.6.2.4., “Lowest Priority Delivery Mode”).
2. If the local APIC determines that it is the designated destination for the interrupt and if the interrupt request is an NMI, SMI, INIT, ExtINT, or INIT-deassert interrupt, or one of the MP protocol IPI messages (BIPI, FIPI, and SIPI), the interrupt is sent directly to the processor core for handling.

3. If the local APIC determines that it is the designated destination for the interrupt but the interrupt request is not one of the interrupts given in step 2, the local APIC looks for an open slot in one of its two pending interrupt queues contained in the IRR and ISR registers (see Figure 8-20). If a slot is available (see Section 8.8.4., “Interrupt Acceptance for Fixed Interrupts”), places the interrupt in the slot. If a slot is not available, it rejects the interrupt request and sends it back to the sender with a retry message.
4. When interrupts are pending in the IRR and ISR register, the local APIC dispatches them to the processor one at a time, based on their priority and the current task and processor priorities in the TPR and PPR (see Section 8.8.3.1., “Task and Processor Priorities”).
5. When a fixed interrupt has been dispatched to the processor core for handling, the completion of the handler routine is indicated with an instruction in the instruction handler code that writes to the end-of-interrupt (EOI) register in the local APIC (see Section 8.8.5., “Signaling Interrupt Servicing Completion”). The act of writing to the EOI register causes the local APIC to delete the interrupt from its queue and (for level-triggered interrupts) send a message on the bus indicating that the interrupt handling has been completed. (A write to the EOI register must not be included in the handler routine for an NMI, SMI, INIT, ExtINT, or SIPI.)

The following sections describe the acceptance of interrupts and their handling by the local APIC and processor in greater detail.

8.8.3. Interrupt, Task, and Processor Priority

For interrupts that are delivered to the processor through the local APIC, each interrupt has an implied priority based on its vector number. The local APIC uses this priority to determine when to service the interrupt relative to the other activities of the processor, including the servicing of other interrupts.

For interrupts vectors in the range of 16 to 255, the interrupt priority is determined using the following relationship:

$$\text{priority} = \text{vector} / 16$$

Here the quotient is rounded down to the nearest integer value to determine the priority, with 1 being the lowest priority and 15 is the highest. Because vectors 0 through 31 are reserved for dedicated uses by the IA-32 architecture, the priorities of user defined interrupts range from 2 to 15.

Each interrupt priority level (sometimes interpreted by software as an interrupt priority class) encompasses 16 vectors. Prioritizing interrupts within a priority level is determined by the vector number. The higher the vector number, the higher the priority within that priority level. In determining the priority of a vector and ranking of vectors within a priority group, the vector number is often divided into two parts, with the high 4 bits of the vector indicating its priority and the low 4 bit indicating its ranking within the priority group.

8.8.3.1. TASK AND PROCESSOR PRIORITIES

The local APIC also defines a task priority and a processor priority that is uses in determining the order in which interrupts should be handled. The task priority is a software selected value between 0 and 15 (see Figure 8-18) that is written into the task priority register (TPR). The TPR is a read/write register.

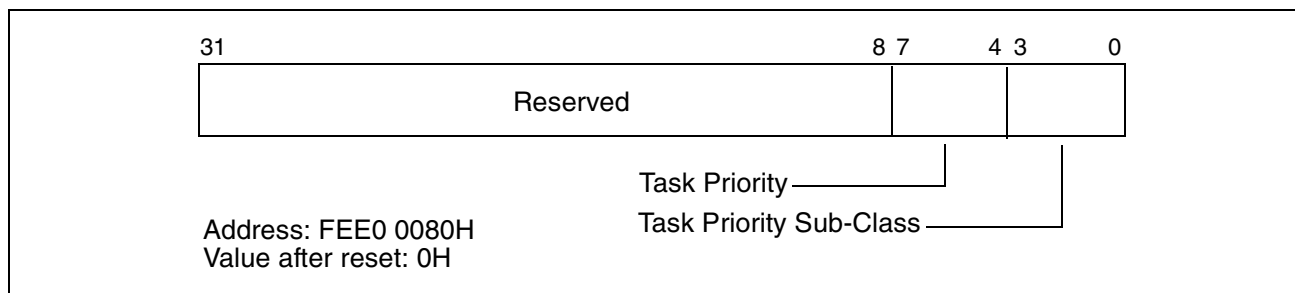


Figure 8-18. Task Priority Register (TPR)

NOTE

In this discussion, the term “task” refers to a software defined task, process, thread, program, or routine that is dispatched to run on the processor by the operating system. It does not refer an IA-32 architecture defined task as described in Chapter 6, *Task Management*.

The task priority allows software to set a **priority threshold** for interrupting the processor. The processor will service only those interrupts that have a priority higher than that specified in the TPR. If software sets the task priority in the TPR to 0, the processor will handle all interrupts; it is it set to 15, all interrupts are inhibited from being handled, except those delivered with the NMI, SMI, INIT, ExtINT, INIT-deassert, and start-up delivery mode. This mechanism enables the operating system to temporarily block specific interrupts (generally low priority interrupts) from disturbing high-priority work that the processor is doing.

Note that the task priority is also used to determine the arbitration priority of the local processor (see Section 8.6.2.4., “Lowest Priority Delivery Mode”).

The processor priority is set by the processor, also to value between 0 and 15 (see Figure 8-19) that is written into the processor priority register (PPR). The PPR is a read only register. The processor priority represents the current priority at which the processor is executing. It is used to determine whether a pending interrupt can be dispensed to the processor.

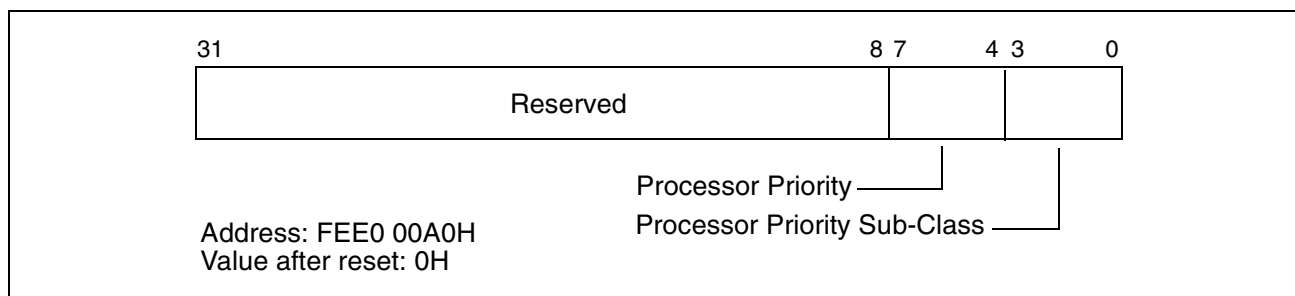


Figure 8-19. Processor Priority Register (PPR)

Its value in the PPR is computed as follows:

```

IF TPR[7:4] ≥ ISRV[7:4]
    THEN
        PPR[7:0] ← TPR[7:0]
    ELSE
        PPR[7:4] ← ISRV[7:4]
        PPR[3:0] ← 0
    
```

Here, the ISRV value is the vector number of the highest priority ISR bit that is set, or 00H if no ISR bit is set. Essentially, the processor priority is set to either to the highest priority pending interrupt in the ISR or to the current task priority, whichever is higher.

8.8.4. Interrupt Acceptance for Fixed Interrupts

The local APIC queues the fixed interrupts that it accepts in one of two interrupt pending registers: the interrupt request register (IRR) or in-service register (ISR). These two 256-bit read-only registers are shown in Figure 8-20). The 256 bits in these registers represent the 256 possible vectors, with vectors 0 through 15 are reserved.

NOTE

All interrupts with an NMI, SMI, INIT, ExtINT, start-up, or INIT-deassert delivery mode bypass the IRR and ISR registers and are sent directly to the processor core for servicing.

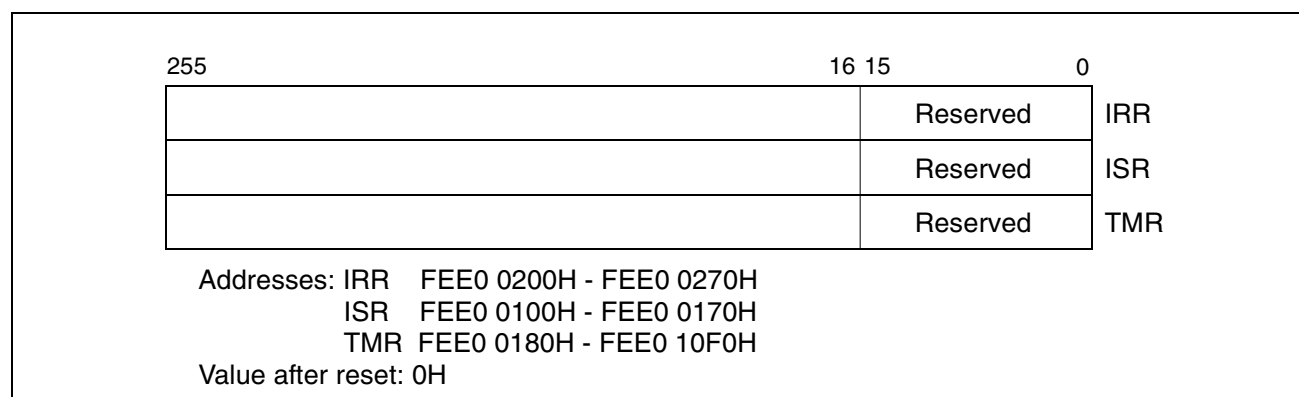


Figure 8-20. IRR, ISR and TMR Registers

The IRR contains the active interrupt requests that have been accepted, but not yet dispatched to the processor for servicing. When the local APIC accepts an interrupt, it sets the bit in the IRR that corresponds the vector of the accepted interrupt. When the processor core is ready to handle the next interrupt, the local APIC clears the highest priority IRR bit that is set and sets the corresponding ISR bit. The vector for the highest priority bit set in the ISR is then dispatched to the processor core for servicing.

While the processor is servicing the highest priority interrupt, the local APIC can send additional fixed interrupts by setting bits in the IRR. When the interrupt service routine issues a write to the EOI register (see Section 8.8.5., “Signaling Interrupt Servicing Completion”), the local APIC responds by clearing the highest priority ISR bit that is set. It then repeats the process of clearing the highest priority bit in the IRR and setting the corresponding bit in the ISR. The processor core then begins executing the service routing for the highest priority bit set in the ISR.

If more than one interrupt is generated with the same vector number, the local APIC can set the bit for the vector both in the IRR and the ISR. This means that for the Pentium 4 and Intel Xeon processors, the IRR and ISR can queue two interrupts for each interrupt vector: one in the IRR and one in the ISR. Any additional interrupts issued for the same interrupt vector are collapsed into the single bit in the IRR.

For the P6 family and Pentium processors, the IRR and ISR registers can queue no more than two interrupts per priority level, and will reject other interrupts that are received within the same priority level.

If the local APIC receives an interrupt with a priority higher than that of the interrupt currently in serviced, and interrupts are enabled in the processor core, the local APIC dispatches the higher priority interrupt to the processor immediately (without waiting for a write to the EOI register). The currently executing interrupt handler is then interrupted so the higher-priority interrupt can be handled. When the handling of the higher-priority interrupt has been completed, the servicing of the interrupted interrupt is resumed.

The trigger mode register (TMR) indicates the trigger mode of the interrupt (see Figure 8-20). Upon acceptance of an interrupt into the IRR, the corresponding TMR bit is cleared for edge-triggered interrupts and set for level-triggered interrupts. If a TMR bit is set when an EOI cycle for its corresponding interrupt vector is generated, an EOI message is sent to all I/O APICs.

8.8.5. Signaling Interrupt Servicing Completion

For all interrupts except those delivered with the NMI, SMI, INIT, ExtINT, the start-up, or INIT-Deassert delivery mode, the interrupt handler must include a write to the end-of-interrupt (EOI) register (see Figure 8-21). This write must occur at the end of the handler routine, sometime before the IRET instruction. This action indicates that the servicing of the current interrupt is complete and the local APIC can issue the next interrupt from the ISR.

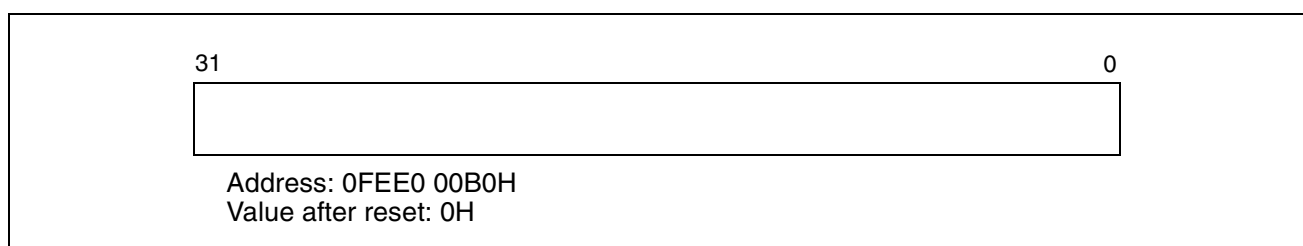


Figure 8-21. EOI Register

Upon receiving and EOI, the APIC clears the highest priority bit in the ISR and dispatches the next highest priority interrupt to the processor. If the terminated interrupt was a level-triggered interrupt, the local APIC also sends an end-of-interrupt message to all I/O APICs.

For future compatibility, the software is requested to issue the end-of-interrupt command by writing a value of 0H into the EOI register.

8.9. SPURIOUS INTERRUPT

A special situation may occur when a processor raises its task priority to be greater than or equal to the level of the interrupt for which the processor INTR signal is currently being asserted. If at the time the INTA cycle is issued, the interrupt that was to be dispensed has become masked (programmed by software), the local APIC will deliver a spurious-interrupt vector. Dispensing the spurious-interrupt vector does not affect the ISR, so the handler for this vector should return without an EOI.

The vector number for the spurious-interrupt vector is specified in the spurious-interrupt vector register (see Figure 8-22). The functions of the fields in this register are as follows:

Spurious Vector	Determines the vector number to be delivered to the processor when the local APIC generates a spurious vector.
	(Pentium 4 and Intel Xeon processors.) Bits 0 through 7 of the this field are programmable by software.
	(P6 family and Pentium processors). Bits 4 through 7 of the this field are programmable by software, and bits 0 through 3 are hardwired to logical ones. Software writes to bits 0 through 3 have no effect.
APIC Software Enable/Disable	Allows software to temporarily enable (1) or disable (0) the local APIC (see Section 8.4.3., “Enabling or Disabling the Local APIC”).
Focus Processor Checking	Determines if focus processor checking is enabled (0) or disabled (1) when using the lowest-priority delivery mode. In Pentium 4 and Intel Xeon processors, this bit is reserved and should be cleared to 0.

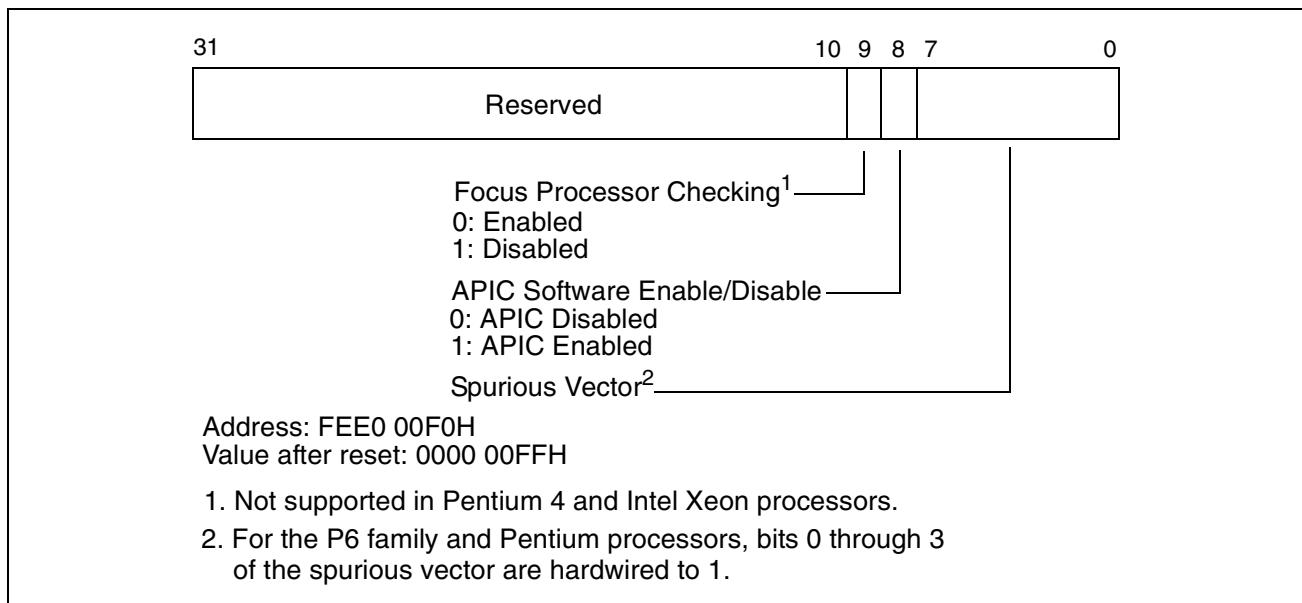


Figure 8-22. Spurious-Interrupt Vector Register (SVR)

8.10. APIC BUS MESSAGE PASSING MECHANISM AND PROTOCOL (P6 FAMILY AND PENTIUM PROCESSORS ONLY)

The Pentium 4 and Intel Xeon processors pass messages among the local and I/O APICs on the system bus, using the system bus message passing mechanism and protocol.

The P6 family and Pentium processors, pass messages among the local and I/O APICs on the serial APIC bus, as follows. Because only one message can be sent at a time on the APIC bus, the I/O APIC and local APICs employ a “rotating priority” arbitration protocol to gain permission to send a message on the APIC bus. One or more APICs may start sending their messages simultaneously. At the beginning of every message, each APIC presents the type of the message it is sending and its current arbitration priority on the APIC bus. This information is used for arbitration. After each arbitration cycle (within an arbitration round), only the potential winners keep driving the bus. By the time all arbitration cycles are completed, there will be only one APIC left driving the bus. Once a winner is selected, it is granted exclusive use of the bus, and will continue driving the bus to send its actual message.

After each successfully transmitted message, all APICs increase their arbitration priority by 1. The previous winner (that is, the one that has just successfully transmitted its message) assumes a priority of 0 (lowest). An agent whose arbitration priority was 15 (highest) during arbitration, but did not send a message, adopts the previous winner’s arbitration priority, increments by 1.

Note that the arbitration protocol described above is slightly different if one of the APICs issues a special End-Of-Interrupt (EOI). This high-priority message is granted the bus regardless of its sender’s arbitration priority, unless more than one APIC issues an EOI message simultaneously. In the latter case, the APICs sending the EOI messages arbitrate using their arbitration priorities.

If the APICs are set up to use “lowest priority” arbitration (see Section 8.6.2.4., “Lowest Priority Delivery Mode”) and multiple APICs are currently executing at the lowest priority (the value in the APR register), the arbitration priorities (unique values in the Arb ID register) are used to break ties. All 8 bits of the APR are used for the lowest priority arbitration.

8.10.1. Bus Message Formats

See Appendix F, *APIC Bus Message Formats*, for a description of bus message formats used to transmit messages on the serial APIC bus.

8.11. MESSAGE SIGNALLED INTERRUPTS

The *PCI Local Bus Specification, Rev 2.2* (www.pcisig.com) introduces the concept of message signalled interrupts. Intel processors and chipsets with this capability currently include the Pentium 4 and Intel Xeon processors. As the specification indicates:

“Message signalled interrupts (MSI) is an optional feature that enables PCI devices to request service by writing a system-specified message to a system-specified address (PCI DWORD memory write transaction). The transaction address specifies the message destination while the transaction data specifies the message. System software is expected to initialize the message destination and message during device configuration, allocating one or more non-shared messages to each MSI capable function.”

The capabilities mechanism provided by the *PCI Local Bus Specification* is used to identify and configure MSI capable PCI devices. Among other fields, this structure contains a Message Data Register and a Message Address Register. To request service, the PCI device function writes the contents of the Message Data Register to the address contained in the Message Address Register (and the Message Upper Address register for 64-bit message addresses).

Section 8.11.1. and Section 8.11.2. provide layout details for the Message Address Register and the Message Data Register. The operation issued by the device is a PCI write command to the Message Address Register with the Message Data Register contents. The operation follows semantic rules as defined for PCI write operations and is a DWORD operation.

8.11.1. Message Address Register Format

The format of the Message Address Register (lower 32-bits) is shown in Figure 8-23.

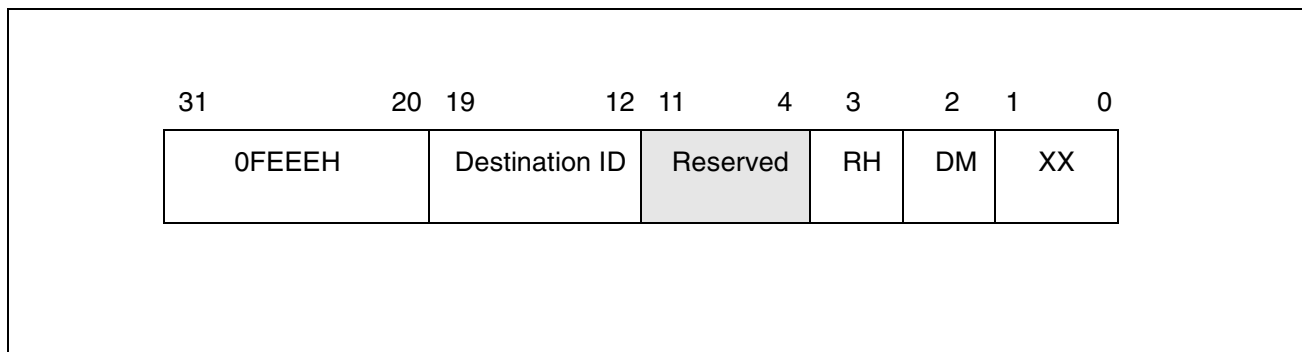


Figure 8-23. Layout of the MSI Message Address Register

Fields in the Message Address Register are as follows:

1. Bits 31-20: These bits contain a fixed value for interrupt messages (0FEEH). This value locates interrupts at the 1MB area with a base address of 4G – 18M. All accesses to this region are directed as interrupt messages. Care must be taken to ensure that no other device claims the region as I/O space.
2. Destination ID: This field contains an 8-bit destination ID. It identifies the message's target processor(s). The destination ID corresponds to bits 63:56 of the I/O APIC Redirection Table Entry if the IOAPIC is used to dispatch the interrupt to the processor(s).
3. Redirection Hint Indication (RH): This bit indicates whether the message should be directed to the processor with the lowest interrupt priority among processors that can receive the interrupt. When the bit is 0, the interrupt is directed to the processor listed in the Destination ID field. When the bit is 1, the interrupt is directed to the processor with the lowest priority of the processors indicated in the Destination ID field. Interpreting the Destination ID field for lowest priority delivery takes the DM bit into account.
4. Destination Mode (DM): This bit indicates whether the Destination ID field should be interpreted as logical or physical APIC ID for delivery of the lowest priority interrupt. If RH is 1 and DM is 0, the Destination ID field is in physical destination mode and only the processor in the system that has the matching APIC ID is considered for delivery of that interrupt (this means no re-direction). If RH is 1 and DM is 1, the Destination ID Field is interpreted as in logical destination mode and the redirection is limited to only those processors that are part of the logical group of processors based on the processor's logical APIC ID and the Destination ID field in the message. The logical group of processors consists of those identified by matching the 8-bit Destination ID with the logical destination identified by the Destination Format Register and the Logical Destination Register in each local APIC. The details are similar to those described in Section 8.6.2., "Determining IPI Destination". If RH is 0, then the DM bit is ignored and the message is sent ahead independent of whether the physical or logical destination mode is used.

8.11.2. Message Data Register Format

The layout of the Message Data Register is shown in Figure 8-24.

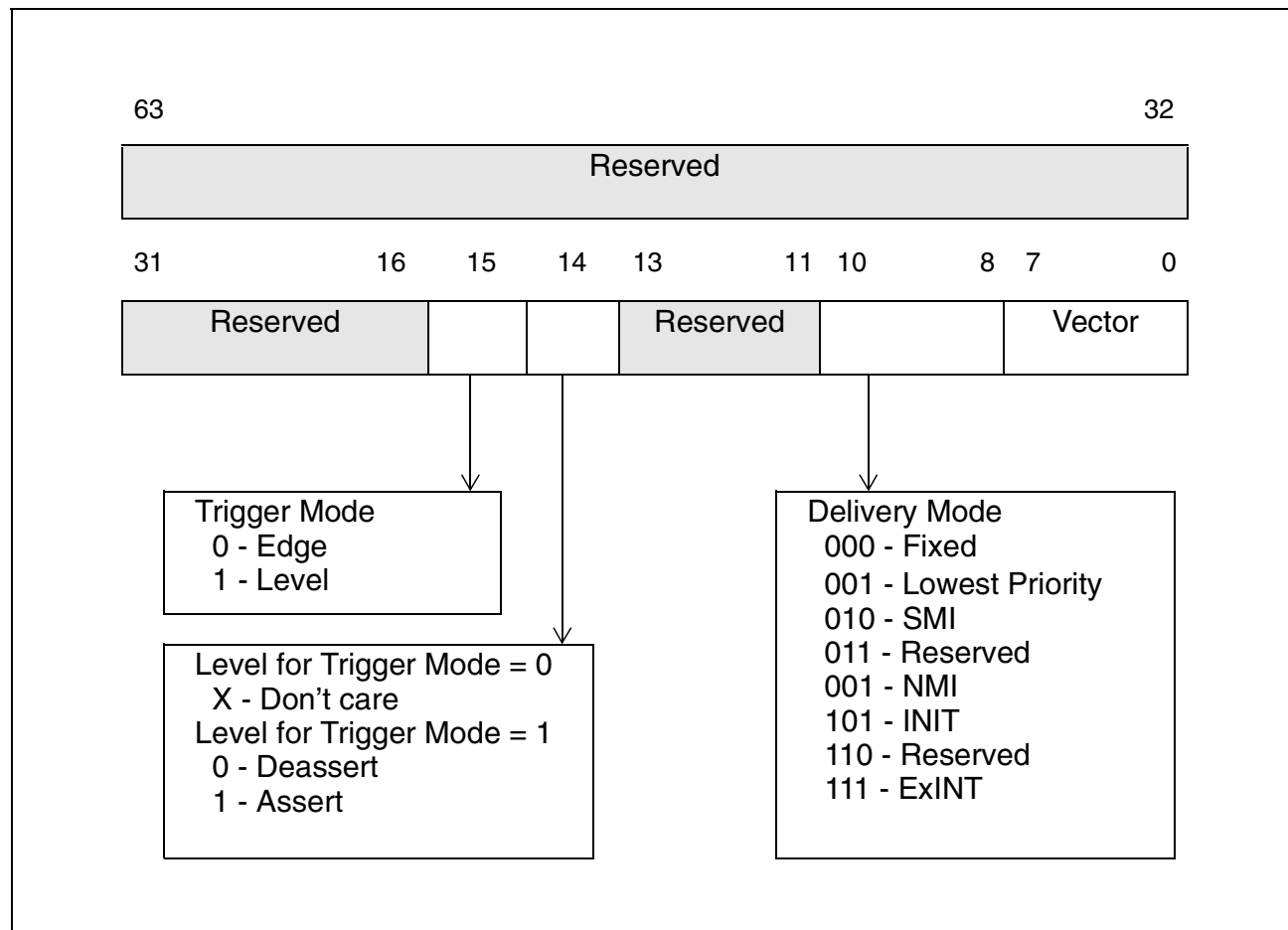


Figure 8-24. Layout of the MSI Message Data Register

Reserved fields are not assumed to be any value. Software must preserve their contents on writes. Other fields in the Message Data Register are described below.

1. **Vector:** This 8-bit field contains the interrupt vector associated with the message. Values range from 010H to 0FEH. Software must guarantee that the field is not programmed with vector 00H to 0FH.
2. **Delivery Mode:** This 3-bit field specifies how the interrupt receipt is handled. Delivery Modes operate only in conjunction with specified Trigger Modes. Correct Trigger Modes must be guaranteed by software. Restrictions are indicated below:
 - a. **000B (Fixed Mode)** – Deliver the signal to all the agents listed in the destination. The Trigger Mode for fixed delivery mode can be edge or level.
 - b. **001B (Lowest Priority)** – Deliver the signal to the agent that is executing at the lowest priority of all agents listed in the destination field. The trigger mode can be edge or level.

- c. 010B (System Management Interrupt or SMI) – The delivery mode is edge only. For systems that rely on SMI semantics, the vector field is ignored but must be programmed to all zeroes for future compatibility.
 - d. 100B (NMI) – Deliver the signal to all the agents listed in the destination field. The vector information is ignored. NMI is an edge triggered interrupt regardless of the Trigger Mode Setting.
 - e. 101B (INIT) – Deliver this signal to all the agents listed in the destination field. The vector information is ignored. INIT is an edge triggered interrupt regardless of the Trigger Mode Setting.
 - f. 111B (ExtINT) – Deliver the signal to the INTR signal of all agents in the destination field (as an interrupt that originated from an 8259A compatible interrupt controller). The vector is supplied by the INTA cycle issued by the activation of the ExtINT. ExtINT is an edge triggered interrupt.
- 3. Level: Edge triggered interrupt messages are always interpreted as assert messages. For edge triggered interrupts this field is not used. For level triggered interrupts, this bit reflects the state of the interrupt input.
 - 4. Trigger Mode: This field indicates the signal type that will trigger a message.
 - a. 0 – Indicates edge sensitive.
 - b. 1 – Indicates level sensitive.

9

Processor Management and Initialization

CHAPTER 9

PROCESSOR MANAGEMENT AND INITIALIZATION

This chapter describes the facilities provided for managing processor wide functions and for initializing the processor. The subjects covered include: processor initialization, x87 FPU initialization, processor configuration, feature determination, mode switching, the MSRs (in the Pentium, P6 family, Pentium 4, and Intel Xeon processors), and the MTRRs (in the P6 family, Pentium 4, and Intel Xeon processors).

9.1. INITIALIZATION OVERVIEW

Following power-up or an assertion of the RESET# pin, each processor on the system bus performs a hardware initialization of the processor (known as a hardware reset) and an optional built-in self-test (BIST). A hardware reset sets each processor's registers to a known state and places the processor in real-address mode. It also invalidates the internal caches, translation lookaside buffers (TLBs) and the branch target buffer (BTB). At this point, the action taken depends on the processor family:

- Pentium 4 and Intel Xeon processors—All the processors on the system bus (including a single processor in a uniprocessor system) execute the multiple processor (MP) initialization protocol. The processor that is selected through this protocol as the bootstrap processor (BSP) then immediately starts executing software-initialization code in the current code segment beginning at the offset in the EIP register. The application (non-BSP) processors (APs) go into a Wait For Startup IPI (SIPI) state while the BSP is executing initialization code. See Section 7.5., “Multiple-Processor (MP) Initialization”, for more details. Note that in a uniprocessor system, the single Pentium 4 or Intel Xeon processor automatically becomes the BSP.
- P6 family processors—The action taken is the same as for the Pentium 4 and Intel Xeon processors (as described in the previous paragraph).
- Pentium processors—In either a single- or dual- processor system, a single Pentium processor is always pre-designated as the primary processor. Following a reset, the primary processor behaves as follows in both single- and dual-processor systems. Using the dual-processor (DP) ready initialization protocol, the primary processor immediately starts executing software-initialization code in the current code segment beginning at the offset in the EIP register. The secondary processor (if there is one) goes into a halt state.
- Intel486 processor—The primary processor (or single processor in a uniprocessor system) immediately starts executing software-initialization code in the current code segment beginning at the offset in the EIP register. (The Intel486 does not automatically execute a DP or MP initialization protocol to determine which processor is the primary processor.)

The software-initialization code performs all system-specific initialization of the BSP or primary processor and the system logic.

At this point, for MP (or DP) systems, the BSP (or primary) processor wakes up each AP (or secondary) processor to enable those processors to execute self-configuration code.

When all processors are initialized, configured, and synchronized, the BSP or primary processor begins executing an initial operating-system or executive task.

The x87 FPU is also initialized to a known state during hardware reset. x87 FPU software initialization code can then be executed to perform operations such as setting the precision of the x87 FPU and the exception masks. No special initialization of the x87 FPU is required to switch operating modes.

Asserting the INIT# pin on the processor invokes a similar response to a hardware reset. The major difference is that during an INIT, the internal caches, MSRs, MTRRs, and x87 FPU state are left unchanged (although, the TLBs and BTB are invalidated as with a hardware reset). An INIT provides a method for switching from protected to real-address mode while maintaining the contents of the internal caches.

9.1.1. Processor State After Reset

Table 9-1 shows the state of the flags and other registers following power-up for the Pentium 4, Intel Xeon, P6 family, and Pentium processors. The state of control register CR0 is 60000010H (see Figure 9-1), which places the processor in real-address mode with paging disabled.

9.1.2. Processor Built-In Self-Test (BIST)

Hardware may request that the BIST be performed at power-up. The EAX register is cleared (0H) if the processor passes the BIST. A nonzero value in the EAX register after the BIST indicates that a processor fault was detected. If the BIST is not requested, the contents of the EAX register after a hardware reset is 0H.

The overhead for performing a BIST varies between processor families. For example, the BIST takes approximately 30 million processor clock periods to execute on the Pentium 4 processor. (This clock count is model-specific, and Intel reserves the right to change the exact number of periods, for any of the IA-32 processors, without notification.)

Table 9-1. 32-Bit IA-32 Processor States Following Power-up, Reset, or INIT

Register	Pentium 4 and Intel Xeon Processor	P6 Family Processor	Pentium Processor
EFLAGS ¹	00000002H	00000002H	00000002H
EIP	0000FFF0H	0000FFF0H	0000FFF0H
CR0	60000010H ²	60000010H ²	60000010H ²
CR2, CR3, CR4	00000000H	00000000H	00000000H
CS	Selector = F000H Base = FFFF0000H Limit = FFFFH AR = Present, R/W, Accessed	Selector = F000H Base = FFFF0000H Limit = FFFFH AR = Present, R/W, Accessed	Selector = F000H Base = FFFF0000H Limit = FFFFH AR = Present, R/W, Accessed
SS, DS, ES, FS, GS	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W, Accessed	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W, Accessed	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W, Accessed
EDX	00000FxxH	000006xxH	000005xxH
EAX	0 ³	0 ³	0 ³
EBX, ECX, ESI, EDI, EBP, ESP	00000000H	00000000H	00000000H
ST0 through ST7 ⁴	Pwr up or Reset: +0.0 FINIT/FNINIT: Unchanged	Pwr up or Reset: +0.0 FINIT/FNINIT: Unchanged	Pwr up or Reset: +0.0 FINIT/FNINIT: Unchanged
x87 FPU Control Word ⁴	Pwr up or Reset: 0040H FINIT/FNINIT: 037FH	Pwr up or Reset: 0040H FINIT/FNINIT: 037FH	Pwr up or Reset: 0040H FINIT/FNINIT: 037FH
x87 FPU Status Word ⁴	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H
x87 FPU Tag Word ⁴	Pwr up or Reset: 5555H FINIT/FNINIT: FFFFH	Pwr up or Reset: 5555H FINIT/FNINIT: FFFFH	Pwr up or Reset: 5555H FINIT/FNINIT: FFFFH
x87 FPU Data Operand and CS Seg. Selectors ⁴	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H	Pwr up or Reset: 0000H FINIT/FNINIT: 0000H
x87 FPU Data Operand and Inst. Pointers ⁴	Pwr up or Reset: 00000000H FINIT/FNINIT: 00000000H	Pwr up or Reset: 00000000H FINIT/FNINIT: 00000000H	Pwr up or Reset: 00000000H FINIT/FNINIT: 00000000H
MM0 through MM7 ⁴	Pwr up or Reset: 0000000000000000H INIT or FINIT/FNINIT: Unchanged	Pentium II and Pentium III Processors Only— Pwr up or Reset: 0000000000000000H INIT or FINIT/FNINIT: Unchanged	Pentium with MMX Technology Only— Pwr up or Reset: 0000000000000000H INIT or FINIT/FNINIT: Unchanged
XMM0 through XMM7	Pwr up or Reset: 0000000000000000H INIT: Unchanged	Pentium III processor Only— Pwr up or Reset: 0000000000000000H INIT: Unchanged	NA

Table 9-1. 32-Bit IA-32 Processor States Following Power-up, Reset, or INIT (Contd.)

Register	Pentium 4 and Intel Xeon Processor	P6 Family Processor	Pentium Processor
MXCSR	Pwr up or Reset: 1F80H INIT: Unchanged	Pentium III processor only- Pwr up or Reset: 1F80H INIT: Unchanged	NA
GDTR, IDTR	Base = 00000000H Limit = FFFFH AR = Present, R/W	Base = 00000000H Limit = FFFFH AR = Present, R/W	Base = 00000000H Limit = FFFFH AR = Present, R/W
LDTR, Task Register	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W	Selector = 0000H Base = 00000000H Limit = FFFFH AR = Present, R/W
DR0, DR1, DR2, DR3	00000000H	00000000H	00000000H
DR6	FFFF0FF0H	FFFF0FF0H	FFFF0FF0H
DR7	00000400H	00000400H	00000400H
Time-Stamp Counter	Power up or Reset: 0H INIT: Unchanged	Power up or Reset: 0H INIT: Unchanged	Power up or Reset: 0H INIT: Unchanged
Perf. Counters and Event Select	Power up or Reset: 0H INIT: Unchanged	Power up or Reset: 0H INIT: Unchanged	Power up or Reset: 0H INIT: Unchanged
All Other MSRs	Pwr up or Reset: Undefined INIT: Unchanged	Pwr up or Reset: Undefined INIT: Unchanged	Pwr up or Reset: Undefined INIT: Unchanged
Data and Code Cache, TLBs	Invalid	Invalid	Invalid
Fixed MTRRs	Pwr up or Reset: Disabled INIT: Unchanged	Pwr up or Reset: Disabled INIT: Unchanged	Not Implemented
Variable MTRRs	Pwr up or Reset: Disabled INIT: Unchanged	Pwr up or Reset: Disabled INIT: Unchanged	Not Implemented
Machine-Check Architecture	Pwr up or Reset: Undefined INIT: Unchanged	Pwr up or Reset: Undefined INIT: Unchanged	Not Implemented
APIC	Pwr up or Reset: Enabled INIT: Unchanged	Pwr up or Reset: Enabled INIT: Unchanged	Pwr up or Reset: Enabled INIT: Unchanged

NOTES:

1. The 10 most-significant bits of the EFLAGS register are undefined following a reset. Software should not depend on the states of any of these bits.
2. The CD and NW flags are unchanged, bit 4 is set to 1, all other bits are cleared.
3. If Built-In Self-Test (BIST) is invoked on power up or reset, EAX is 0 only if all tests passed. (BIST cannot be invoked during an INIT.)
4. The state of the x87 FPU and MMX registers is not changed by the execution of an INIT.

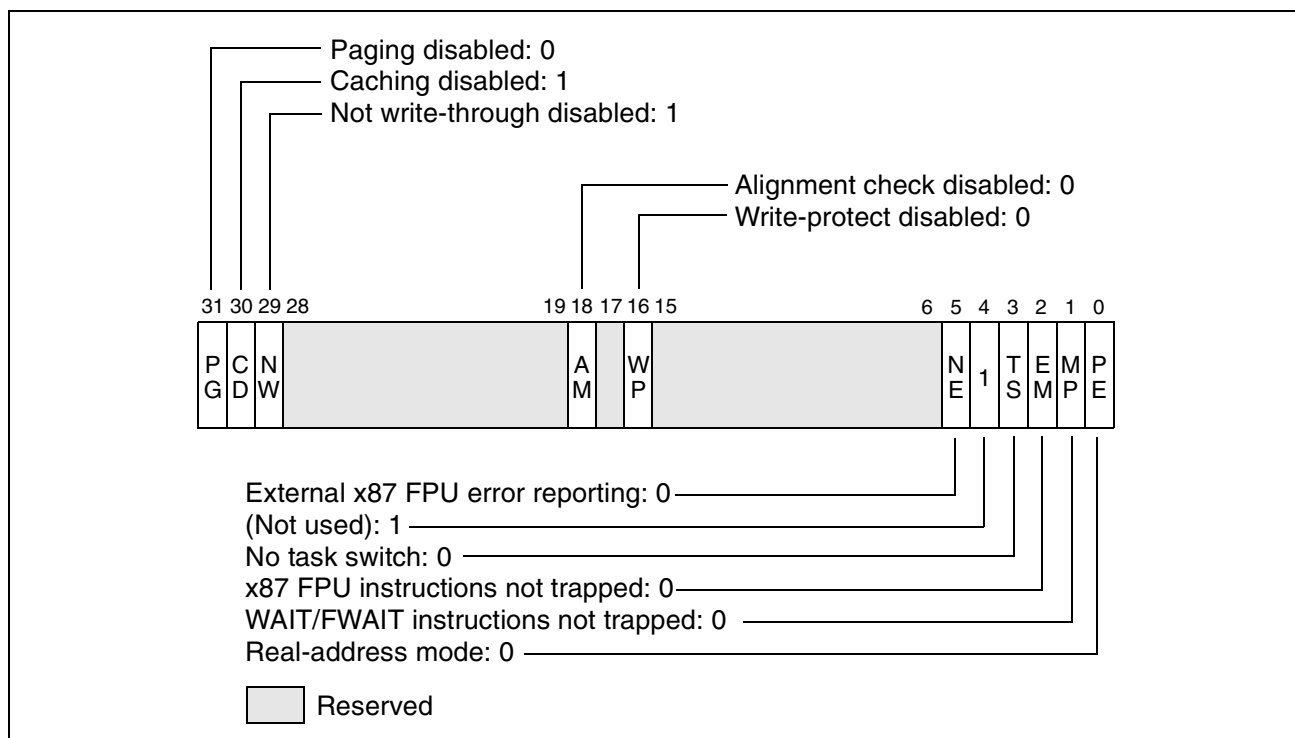


Figure 9-1. Contents of CR0 Register after Reset

9.1.3. Model and Stepping Information

Following a hardware reset, the EDX register contains component identification and revision information (see Figure 9-2). For example, the model, family, and processor type returned for the first processor in the Intel Pentium 4 family is as follows: model (0000B), family (1111B), and processor type (00B).

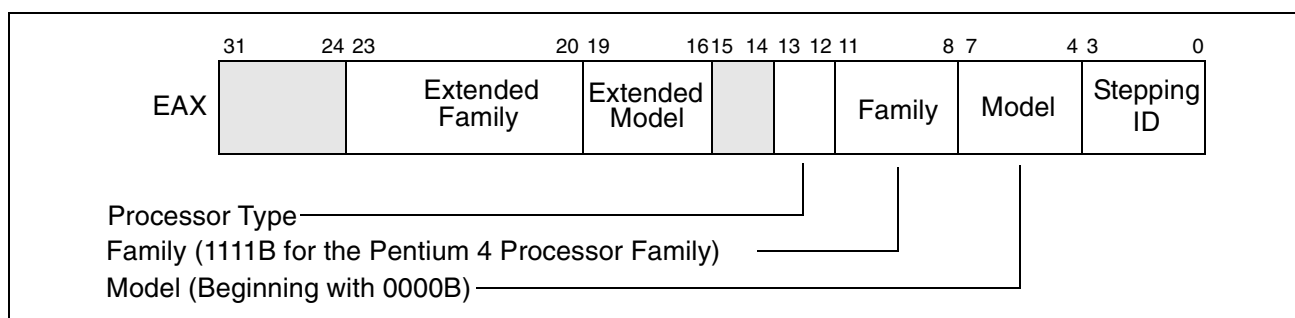


Figure 9-2. Version Information in the EDX Register after Reset

The stepping ID field contains a unique identifier for the processor's stepping ID or revision level. The extended family and extended model fields were added to the IA-32 architecture in the Pentium 4 processors.

9.1.4. First Instruction Executed

The first instruction that is fetched and executed following a hardware reset is located at physical address FFFFFFF0H. This address is 16 bytes below the processor's uppermost physical address. The EPROM containing the software-initialization code must be located at this address.

The address FFFFFFF0H is beyond the 1-MByte addressable range of the processor while in real-address mode. The processor is initialized to this starting address as follows. The CS register has two parts: the visible segment selector part and the hidden base address part. In real-address mode, the base address is normally formed by shifting the 16-bit segment selector value 4 bits to the left to produce a 20-bit base address. However, during a hardware reset, the segment selector in the CS register is loaded with F000H and the base address is loaded with FFFF0000H. The starting address is thus formed by adding the base address to the value in the EIP register (that is, FFFF0000 + FFF0H = FFFFFFF0H).

The first time the CS register is loaded with a new value after a hardware reset, the processor will follow the normal rule for address translation in real-address mode (that is, [CS base address = CS segment selector * 16]). To insure that the base address in the CS register remains unchanged until the EPROM based software-initialization code is completed, the code must not contain a far jump or far call or allow an interrupt to occur (which would cause the CS selector value to be changed).

9.2. X87 FPU INITIALIZATION

Software-initialization code can determine the whether the processor contains an x87 FPU by using the CUID instruction. The code must then initialize the x87 FPU and set flags in control register CR0 to reflect the state of the x87 FPU environment.

A hardware reset places the x87 FPU in the state shown in Table 9-1. This state is different from the state the x87 FPU is placed in following the execution of an FINIT or FNINIT instruction (also shown in Table 9-1). If the x87 FPU is to be used, the software-initialization code should execute an FINIT/FNINIT instruction following a hardware reset. These instructions, tag all data registers as empty, clear all the exception masks, set the TOP-of-stack value to 0, and select the default rounding and precision controls setting (round to nearest and 64-bit precision).

If the processor is reset by asserting the INIT# pin, the x87 FPU state is not changed.

9.2.1. Configuring the x87 FPU Environment

Initialization code must load the appropriate values into the MP, EM, and NE flags of control register CR0. These bits are cleared on hardware reset of the processor. Figure 9-2 shows the suggested settings for these flags, depending on the IA-32 processor being initialized. Initialization code can test for the type of processor present before setting or clearing these flags.

Table 9-2. Recommended Settings of EM and MP Flags on IA-32 processors

EM	MP	NE	IA-32 processor
1	0	1	Intel486™ SX, Intel386™ DX, and Intel386™ SX processors only, without the presence of a math coprocessor.
0	1	1 or 0*	Pentium 4, Intel Xeon, P6 family, Pentium, Intel486™ DX, and Intel 487 SX processors, and Intel386 DX and Intel386 SX processors when a companion math coprocessor is present.

NOTE:

* The setting of the NE flag depends on the operating system being used.

The EM flag determines whether floating-point instructions are executed by the x87 FPU (EM is cleared) or a device-not-available exception (#NM) is generated for all floating-point instructions so that an exception handler can emulate the floating-point operation (EM = 1). Ordinarily, the EM flag is cleared when an x87 FPU or math coprocessor is present and set if they are not present. If the EM flag is set and no x87 FPU, math coprocessor, or floating-point emulator is present, the processor will hang when a floating-point instruction is executed.

The MP flag determines whether WAIT/FWAIT instructions react to the setting of the TS flag. If the MP flag is clear, WAIT/FWAIT instructions ignore the setting of the TS flag; if the MP flag is set, they will generate a device-not-available exception (#NM) if the TS flag is set. Generally, the MP flag should be set for processors with an integrated x87 FPU and clear for processors without an integrated x87 FPU and without a math coprocessor present. However, an operating system can choose to save the floating-point context at every context switch, in which case there would be no need to set the MP bit.

Table 2-1 shows the actions taken for floating-point and WAIT/FWAIT instructions based on the settings of the EM, MP, and TS flags.

The NE flag determines whether unmasked floating-point exceptions are handled by generating a floating-point error exception internally (NE is set, native mode) or through an external interrupt (NE is cleared). In systems where an external interrupt controller is used to invoke numeric exception handlers (such as MS-DOS-based systems), the NE bit should be cleared.

9.2.2. Setting the Processor for x87 FPU Software Emulation

Setting the EM flag causes the processor to generate a device-not-available exception (#NM) and trap to a software exception handler whenever it encounters a floating-point instruction. (Table 9-2 shows when it is appropriate to use this flag.) Setting this flag has two functions:

- It allows x87 FPU code to run on an IA-32 processor that has neither an integrated x87 FPU nor is connected to an external math coprocessor, by using a floating-point emulator.
- It allows floating-point code to be executed using a special or nonstandard floating-point emulator, selected for a particular application, regardless of whether an x87 FPU or math coprocessor is present.

To emulate floating-point instructions, the EM, MP, and NE flag in control register CR0 should be set as shown in Table 9-3.

Table 9-3. Software Emulation Settings of EM, MP, and NE Flags

CR0 Bit	Value
EM	1
MP	0
NE	1

Regardless of the value of the EM bit, the Intel486 SX processor generates a device-not-available exception (#NM) upon encountering any floating-point instruction.

9.3. CACHE ENABLING

The IA-32 processors (beginning with the Intel486 processor) contain internal instruction and data caches. These caches are enabled by clearing the CD and NW flags in control register CR0. (They are set during a hardware reset.) Because all internal cache lines are invalid following reset initialization, it is not necessary to invalidate the cache before enabling caching. Any external caches may require initialization and invalidation using a system-specific initialization and invalidation code sequence.

Depending on the hardware and operating system or executive requirements, additional configuration of the processor's caching facilities will probably be required. Beginning with the Intel486 processor, page-level caching can be controlled with the PCD and PWT flags in page-directory and page-table entries. Beginning with the P6 family processors, the memory type range registers (MTRRs) control the caching characteristics of the regions of physical memory. (For the Intel486 and Pentium processors, external hardware can be used to control the caching characteristics of regions of physical memory.) See Chapter 10, *Memory Cache Control*, for detailed information on configuration of the caching facilities in the Pentium 4, Intel Xeon, and P6 family processors and system memory.

9.4. MODEL-SPECIFIC REGISTERS (MSRS)

The Pentium 4, Intel Xeon, P6 family, and Pentium processors contain a model-specific registers (MSRs). These registers are by definition implementation specific; that is, they are not guaranteed to be supported on future IA-32 processors and/or to have the same functions. The MSRs are provided to control a variety of hardware- and software-related features, including:

- The performance-monitoring counters (see Section 15.8., “Performance Monitoring Overview”).
- (Pentium 4, Intel Xeon, and P6 family processors only.) Debug extensions (see Section 15.4., “Last Branch Recording Overview”).
- (Pentium 4, Intel Xeon, and P6 family processors only.) The machine-check exception capability and its accompanying machine-check architecture (see Chapter 14, *Machine-Check Architecture*).

- (Pentium 4, Intel Xeon, and P6 family processors only.) The MTRRs (see Section 10.11., “Memory Type Range Registers (MTRRs)”).

The MSRs can be read and written to using the RDMSR and WRMSR instructions, respectively.

When performing software initialization of a Pentium 4, Intel Xeon, P6 family, or Pentium processor, many of the MSRs will need to be initialized to set up things like performance-monitoring events, run-time machine checks, and memory types for physical memory.

The list of available performance-monitoring counters for the Pentium 4, Intel Xeon, P6 family, and Pentium processors is given in Appendix A, *Performance-Monitoring Events*, and the list of available MSRs for the Pentium 4, Intel Xeon, P6 family, and Pentium processors is given in Appendix B, *Model-Specific Registers (MSRs)*. The references earlier in this section show where the functions of the various groups of MSRs are described in this manual.

9.5. MEMORY TYPE RANGE REGISTERS (MTRRS)

Memory type range registers (MTRRs) were introduced into the IA-32 architecture with the Pentium Pro processor. They allow the type of caching (or no caching) to be specified in system memory for selected physical address ranges. They allow memory accesses to be optimized for various types of memory such as RAM, ROM, frame buffer memory, and memory-mapped I/O devices.

In general, initializing the MTRRs is normally handled by the software initialization code or BIOS and is not an operating system or executive function. At the very least, all the MTRRs must be cleared to 0, which selects the uncached (UC) memory type. See Section 10.11., “Memory Type Range Registers (MTRRs)”, for detailed information on the MTRRs.

9.6. SSE AND SSE2 EXTENSIONS INITIALIZATION

For processors that contain the SSE extensions (Pentium 4, Intel Xeon, and Pentium III processors) and the SSE2 extensions (Pentium 4 and Intel Xeon processors), several steps must be taken when initializing the processor to allow execution of SSE and SSE2 instructions.

- Check the CPUID feature flags for the presence of the SSE and SSE2 extensions (bits 25 and 26, respectively) and support for the FXSAVE and FXRSTOR instructions (bit 24). Also check for support for the CLFLUSH instruction (bit 19). The CPUID feature flags are loaded in the EDX register when the CPUID instruction is executed with a 1 in the EAX register.
- Set the OSFXSR flag (bit 9 in control register CR4) to indicate that the operating system supports saving and restoring the SSE and SSE2 execution environment (XXM and MXCSR registers) with the FXSAVE and FXRSTOR instructions, respectively. See Section 2.5., “Control Registers”, for a description of the OSFXSR flag.
- Set the OSXMMEXCPT flag (bit 10 in control register CR4) to indicate that the operating system supports the handling of SSE and SSE2 SIMD floating-point exceptions (#XF). See Section 2.5., “Control Registers”, for a description of the OSXMMEXCPT flag.

- Set the mask bits and flags in the MXCSR register according to the mode of operation desired for SSE and SSE2 SIMD floating-point instructions. See “MXCSR Control and Status Register” in Chapter 10 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1* for a detailed description of the bits and flags in the MXCSR register.

9.7. SOFTWARE INITIALIZATION FOR REAL-ADDRESS MODE OPERATION

Following a hardware reset (either through a power-up or the assertion of the RESET# pin) the processor is placed in real-address mode and begins executing software initialization code from physical address FFFFFFF0H. Software initialization code must first set up the necessary data structures for handling basic system functions, such as a real-mode IDT for handling interrupts and exceptions. If the processor is to remain in real-address mode, software must then load additional operating-system or executive code modules and data structures to allow reliable execution of application programs in real-address mode.

If the processor is going to operate in protected mode, software must load the necessary data structures to operate in protected mode and then switch to protected mode. The protected-mode data structures that must be loaded are described in Section 9.8., “Software Initialization for Protected-Mode Operation”.

9.7.1. Real-Address Mode IDT

In real-address mode, the only system data structure that must be loaded into memory is the IDT (also called the “interrupt vector table”). By default, the address of the base of the IDT is physical address 0H. This address can be changed by using the LIDT instruction to change the base address value in the IDTR. Software initialization code needs to load interrupt- and exception-handler pointers into the IDT before interrupts can be enabled.

The actual interrupt- and exception-handler code can be contained either in EPROM or RAM; however, the code must be located within the 1-MByte addressable range of the processor in real-address mode. If the handler code is to be stored in RAM, it must be loaded along with the IDT.

9.7.2. NMI Interrupt Handling

The NMI interrupt is always enabled (except when multiple NMIs are nested). If the IDT and the NMI interrupt handler need to be loaded into RAM, there will be a period of time following hardware reset when an NMI interrupt cannot be handled. During this time, hardware must provide a mechanism to prevent an NMI interrupt from halting code execution until the IDT and the necessary NMI handler software is loaded. Here are two examples of how NMIs can be handled during the initial states of processor initialization:

- A simple IDT and NMI interrupt handler can be provided in EPROM. This allows an NMI interrupt to be handled immediately after reset initialization.

- The system hardware can provide a mechanism to enable and disable NMIs by passing the NMI# signal through an AND gate controlled by a flag in an I/O port. Hardware can clear the flag when the processor is reset, and software can set the flag when it is ready to handle NMI interrupts.

9.8. SOFTWARE INITIALIZATION FOR PROTECTED-MODE OPERATION

The processor is placed in real-address mode following a hardware reset. At this point in the initialization process, some basic data structures and code modules must be loaded into physical memory to support further initialization of the processor, as described in Section 9.7., “Software Initialization for Real-Address Mode Operation”. Before the processor can be switched to protected mode, the software initialization code must load a minimum number of protected mode data structures and code modules into memory to support reliable operation of the processor in protected mode. These data structures include the following:

- A protected-mode IDT.
- A GDT.
- A TSS.
- (Optional.) An LDT.
- If paging is to be used, at least one page directory and one page table.
- A code segment that contains the code to be executed when the processor switches to protected mode.
- One or more code modules that contain the necessary interrupt and exception handlers.

Software initialization code must also initialize the following system registers before the processor can be switched to protected mode:

- The GDTR.
- (Optional.) The IDTR. This register can also be initialized immediately after switching to protected mode, prior to enabling interrupts.
- Control registers CR1 through CR4.
- (Pentium 4, Intel Xeon, and P6 family processors only.) The memory type range registers (MTRRs).

With these data structures, code modules, and system registers initialized, the processor can be switched to protected mode by loading control register CR0 with a value that sets the PE flag (bit 0).

9.8.1. Protected-Mode System Data Structures

The contents of the protected-mode system data structures loaded into memory during software initialization, depend largely on the type of memory management the protected-mode operating-system or executive is going to support: flat, flat with paging, segmented, or segmented with paging.

To implement a flat memory model without paging, software initialization code must at a minimum load a GDT with one code and one data-segment descriptor. A null descriptor in the first GDT entry is also required. The stack can be placed in a normal read/write data segment, so no dedicated descriptor for the stack is required. A flat memory model with paging also requires a page directory and at least one page table (unless all pages are 4 MBytes in which case only a page directory is required). See Section 9.8.3., “Initializing Paging”.

Before the GDT can be used, the base address and limit for the GDT must be loaded into the GDTR register using an LGDT instruction.

A multi-segmented model may require additional segments for the operating system, as well as segments and LDTs for each application program. LDTs require segment descriptors in the GDT. Some operating systems allocate new segments and LDTs as they are needed. This provides maximum flexibility for handling a dynamic programming environment. However, many operating systems use a single LDT for all tasks, allocating GDT entries in advance. An embedded system, such as a process controller, might pre-allocate a fixed number of segments and LDTs for a fixed number of application programs. This would be a simple and efficient way to structure the software environment of a real-time system.

9.8.2. Initializing Protected-Mode Exceptions and Interrupts

Software initialization code must at a minimum load a protected-mode IDT with gate descriptor for each exception vector that the processor can generate. If interrupt or trap gates are used, the gate descriptors can all point to the same code segment, which contains the necessary exception handlers. If task gates are used, one TSS and accompanying code, data, and task segments are required for each exception handler called with a task gate.

If hardware allows interrupts to be generated, gate descriptors must be provided in the IDT for one or more interrupt handlers.

Before the IDT can be used, the base address and limit for the IDT must be loaded into the IDTR register using an LIDT instruction. This operation is typically carried out immediately after switching to protected mode.

9.8.3. Initializing Paging

Paging is controlled by the PG flag in control register CR0. When this flag is clear (its state following a hardware reset), the paging mechanism is turned off; when it is set, paging is enabled. Before setting the PG flag, the following data structures and registers must be initialized:

- Software must load at least one page directory and one page table into physical memory. The page table can be eliminated if the page directory contains a directory entry pointing to itself (here, the page directory and page table reside in the same page), or if only 4-MByte pages are used.
- Control register CR3 (also called the PDBR register) is loaded with the physical base address of the page directory.
- (Optional) Software may provide one set of code and data descriptors in the GDT or in an LDT for supervisor mode and another set for user mode.

With this paging initialization complete, paging is enabled and the processor is switched to protected mode at the same time by loading control register CR0 with an image in which the PG and PE flags are set. (Paging cannot be enabled before the processor is switched to protected mode.)

9.8.4. Initializing Multitasking

If the multitasking mechanism is not going to be used and changes between privilege levels are not allowed, it is not necessary load a TSS into memory or to initialize the task register.

If the multitasking mechanism is going to be used and/or changes between privilege levels are allowed, software initialization code must load at least one TSS and an accompanying TSS descriptor. (A TSS is required to change privilege levels because pointers to the privileged-level 0, 1, and 2 stack segments and the stack pointers for these stacks are obtained from the TSS.) TSS descriptors must not be marked as busy when they are created; they should be marked busy by the processor only as a side-effect of performing a task switch. As with descriptors for LDTs, TSS descriptors reside in the GDT.

After the processor has switched to protected mode, the LTR instruction can be used to load a segment selector for a TSS descriptor into the task register. This instruction marks the TSS descriptor as busy, but does not perform a task switch. The processor can, however, use the TSS to locate pointers to privilege-level 0, 1, and 2 stacks. The segment selector for the TSS must be loaded before software performs its first task switch in protected mode, because a task switch copies the current task state into the TSS.

After the LTR instruction has been executed, further operations on the task register are performed by task switching. As with other segments and LDTs, TSSs and TSS descriptors can be either pre-allocated or allocated as needed.

9.9. MODE SWITCHING

To use the processor in protected mode, a mode switch must be performed from real-address mode. Once in protected mode, software generally does not need to return to real-address mode. To run software written to run in real-address mode (8086 mode), it is generally more convenient to run the software in virtual-8086 mode, than to switch back to real-address mode.

9.9.1. Switching to Protected Mode

Before switching to protected mode, a minimum set of system data structures and code modules must be loaded into memory, as described in Section 9.8., “Software Initialization for Protected-Mode Operation”. Once these tables are created, software initialization code can switch into protected mode.

Protected mode is entered by executing a MOV CR0 instruction that sets the PE flag in the CR0 register. (In the same instruction, the PG flag in register CR0 can be set to enable paging.) Execution in protected mode begins with a CPL of 0.

The 32-bit IA-32 processors have slightly different requirements for switching to protected mode. To insure upwards and downwards code compatibility with all 32-bit IA-32 processors, it is recommended that the following steps be performed:

1. Disable interrupts. A CLI instruction disables maskable hardware interrupts. NMI interrupts can be disabled with external circuitry. (Software must guarantee that no exceptions or interrupts are generated during the mode switching operation.)
2. Execute the LGDT instruction to load the GDTR register with the base address of the GDT.
3. Execute a MOV CR0 instruction that sets the PE flag (and optionally the PG flag) in control register CR0.
4. Immediately following the MOV CR0 instruction, execute a far JMP or far CALL instruction. (This operation is typically a far jump or call to the next instruction in the instruction stream.)

The JMP or CALL instruction immediately after the MOV CR0 instruction changes the flow of execution and serializes the processor.

If paging is enabled, the code for the MOV CR0 instruction and the JMP or CALL instruction must come from a page that is identity mapped (that is, the linear address before the jump is the same as the physical address after paging and protected mode is enabled). The target instruction for the JMP or CALL instruction does not need to be identity mapped.

5. If a local descriptor table is going to be used, execute the LLDT instruction to load the segment selector for the LDT in the LDTR register.
6. Execute the LTR instruction to load the task register with a segment selector to the initial protected-mode task or to a writable area of memory that can be used to store TSS information on a task switch.

7. After entering protected mode, the segment registers continue to hold the contents they had in real-address mode. The JMP or CALL instruction in step 4 resets the CS register. Perform one of the following operations to update the contents of the remaining segment registers.
 - Reload segment registers DS, SS, ES, FS, and GS. If the ES, FS, and/or GS registers are not going to be used, load them with a null selector.
 - Perform a JMP or CALL instruction to a new task, which automatically resets the values of the segment registers and branches to a new code segment.
8. Execute the LIDT instruction to load the IDTR register with the address and limit of the protected-mode IDT.
9. Execute the STI instruction to enable maskable hardware interrupts and perform the necessary hardware operation to enable NMI interrupts.

Random failures can occur if other instructions exist between steps 3 and 4 above. Failures will be readily seen in some situations, such as when instructions that reference memory are inserted between steps 3 and 4 while in system management mode.

9.9.2. Switching Back to Real-Address Mode

The processor switches back to real-address mode if software clears the PE bit in the CR0 register with a MOV CR0 instruction. A procedure that re-enters real-address mode should perform the following steps:

1. Disable interrupts. A CLI instruction disables maskable hardware interrupts. NMI interrupts can be disabled with external circuitry.
2. If paging is enabled, perform the following operations:
 - Transfer program control to linear addresses that are identity mapped to physical addresses (that is, linear addresses equal physical addresses).
 - Insure that the GDT and IDT are in identity mapped pages.
 - Clear the PG bit in the CR0 register.
 - Move 0H into the CR3 register to flush the TLB.
3. Transfer program control to a readable segment that has a limit of 64 KBytes (FFFFH). This operation loads the CS register with the segment limit required in real-address mode.
4. Load segment registers SS, DS, ES, FS, and GS with a selector for a descriptor containing the following values, which are appropriate for real-address mode:
 - Limit = 64 KBytes (0FFFFH)
 - Byte granular (G = 0)
 - Expand up (E = 0)
 - Writable (W = 1)

- Present (P = 1)
- Base = any value

The segment registers must be loaded with non-null segment selectors or the segment registers will be unusable in real-address mode. Note that if the segment registers are not reloaded, execution continues using the descriptor attributes loaded during protected mode.

5. Execute an LIDT instruction to point to a real-address mode interrupt table that is within the 1-MByte real-address mode address range.
6. Clear the PE flag in the CR0 register to switch to real-address mode.
7. Execute a far JMP instruction to jump to a real-address mode program. This operation flushes the instruction queue and loads the appropriate base and access rights values in the CS register.
8. Load the SS, DS, ES, FS, and GS registers as needed by the real-address mode code. If any of the registers are not going to be used in real-address mode, write 0s to them.
9. Execute the STI instruction to enable maskable hardware interrupts and perform the necessary hardware operation to enable NMI interrupts.

NOTE

All the code that is executed in steps 1 through 9 must be in a single page and the linear addresses in that page must be identity mapped to physical addresses.

9.10. INITIALIZATION AND MODE SWITCHING EXAMPLE

This section provides an initialization and mode switching example that can be incorporated into an application. This code was originally written to initialize the Intel386 processor, but it will execute successfully on the Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors. The code in this example is intended to reside in EPROM and to run following a hardware reset of the processor. The function of the code is to do the following:

- Establish a basic real-address mode operating environment.
- Load the necessary protected-mode system data structures into RAM.
- Load the system registers with the necessary pointers to the data structures and the appropriate flag settings for protected-mode operation.
- Switch the processor to protected mode.

Figure 9-3 shows the physical memory layout for the processor following a hardware reset and the starting point of this example. The EPROM that contains the initialization code resides at the upper end of the processor's physical memory address range, starting at address FFFFFFFFH and going down from there. The address of the first instruction to be executed is at FFFFFFF0H, the default starting address for the processor following a hardware reset.

The main steps carried out in this example are summarized in Table 9-4. The source listing for the example (with the filename `STARTUP.ASM`) is given in Example 9-1. The line numbers given in Table 9-4 refer to the source listing.

The following are some additional notes concerning this example:

- When the processor is switched into protected mode, the original code segment base-address value of `FFFF0000H` (located in the hidden part of the CS register) is retained and execution continues from the current offset in the EIP register. The processor will thus continue to execute code in the EPROM until a far jump or call is made to a new code segment, at which time, the base address in the CS register will be changed.
- Maskable hardware interrupts are disabled after a hardware reset and should remain disabled until the necessary interrupt handlers have been installed. The NMI interrupt is not disabled following a reset. The NMI# pin must thus be inhibited from being asserted until an NMI handler has been loaded and made available to the processor.
- The use of a temporary GDT allows simple transfer of tables from the EPROM to anywhere in the RAM area. A GDT entry is constructed with its base pointing to address 0 and a limit of 4 GBytes. When the DS and ES registers are loaded with this descriptor, the temporary GDT is no longer needed and can be replaced by the application GDT.
- This code loads one TSS and no LDTs. If more TSSs exist in the application, they must be loaded into RAM. If there are LDTs they may be loaded as well.

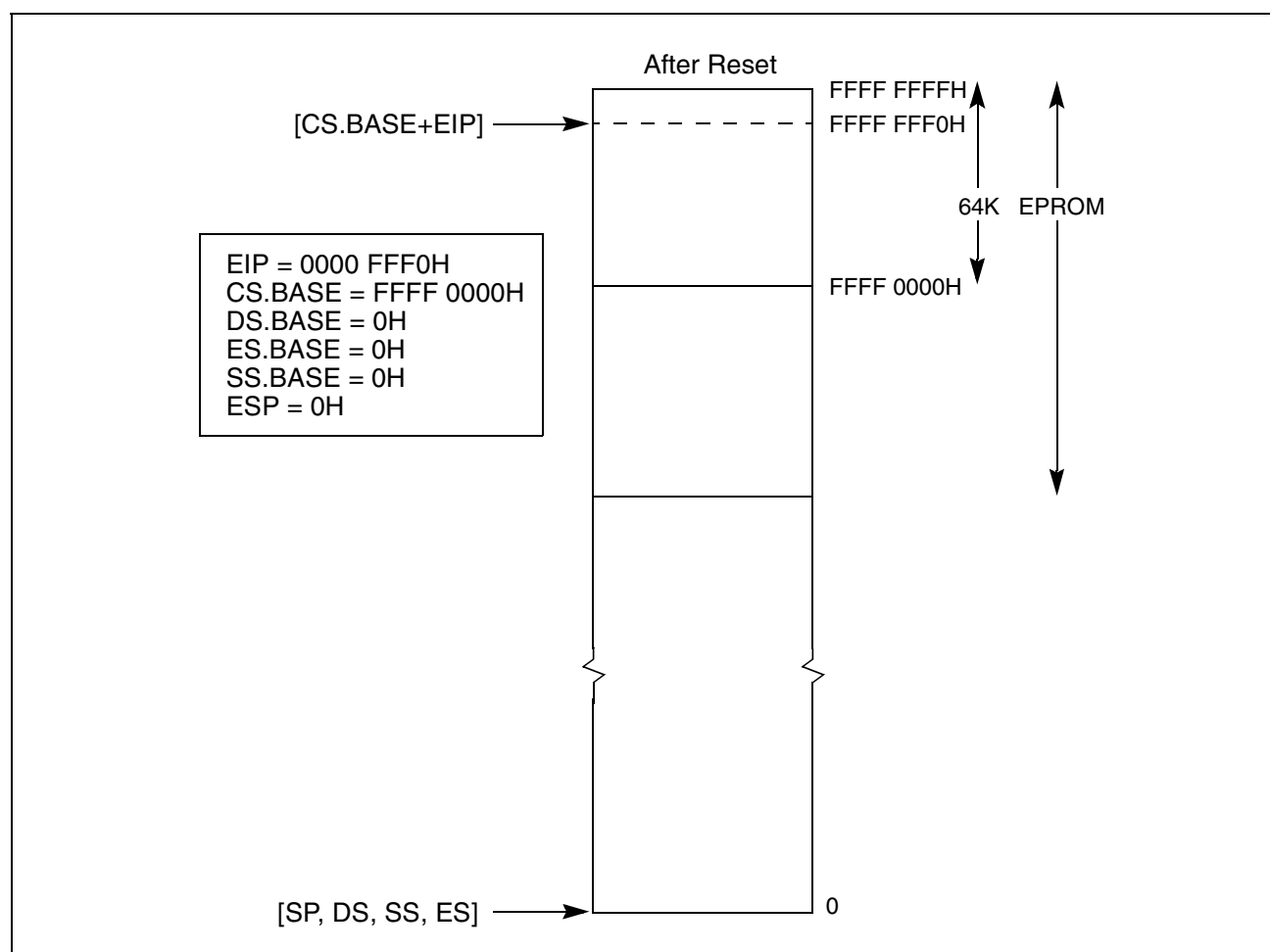


Figure 9-3. Processor State After Reset

Table 9-4. Main Initialization Steps in STARTUP.ASM Source Listing

STARTUP.ASM Line Numbers		Description
From	To	
157	157	Jump (short) to the entry code in the EPROM
162	169	Construct a temporary GDT in RAM with one entry: 0 - null 1 - R/W data segment, base = 0, limit = 4 GBytes
171	172	Load the GDTR to point to the temporary GDT
174	177	Load CR0 with PE flag set to switch to protected mode
179	181	Jump near to clear real mode instruction queue
184	186	Load DS, ES registers with GDT[1] descriptor, so both point to the entire physical memory space
188	195	Perform specific board initialization that is imposed by the new protected mode

Table 9-4. Main Initialization Steps in STARTUP.ASM Source Listing (Contd.)

STARTUP.ASM Line Numbers		Description
From	To	
196	218	Copy the application's GDT from ROM into RAM
220	238	Copy the application's IDT from ROM into RAM
241	243	Load application's GDTR
244	245	Load application's IDTR
247	261	Copy the application's TSS from ROM into RAM
263	267	Update TSS descriptor and other aliases in GDT (GDT alias or IDT alias)
277	277	Load the task register (without task switch) using LTR instruction
282	286	Load SS, ESP with the value found in the application's TSS
287	287	Push EFLAGS value found in the application's TSS
288	288	Push CS value found in the application's TSS
289	289	Push EIP value found in the application's TSS
290	293	Load DS, ES with the value found in the application's TSS
296	296	Perform IRET; pop the above values and enter the application code

9.10.1. Assembler Usage

In this example, the Intel assembler ASM386 and build tools BLD386 are used to assemble and build the initialization code module. The following assumptions are used when using the Intel ASM386 and BLD386 tools.

- The ASM386 will generate the right operand size opcodes according to the code-segment attribute. The attribute is assigned either by the ASM386 invocation controls or in the code-segment definition.
- If a code segment that is going to run in real-address mode is defined, it must be set to a USE 16 attribute. If a 32-bit operand is used in an instruction in this code segment (for example, MOV EAX, EBX), the assembler automatically generates an operand prefix for the instruction that forces the processor to execute a 32-bit operation, even though its default code-segment attribute is 16-bit.
- Intel's ASM386 assembler allows specific use of the 16- or 32-bit instructions, for example, LGDTW, LGDTD, IRETD. If the generic instruction LGDT is used, the default-segment attribute will be used to generate the right opcode.

9.10.2. STARTUP.ASM Listing

Example 9-1 provides high-level sample code designed to move the processor into protected mode. This listing does not include any opcode and offset information.

Example 9-1. STARTUP.ASM

```
MS-DOS* 5.0(045-N) 386(TM) MACRO ASSEMBLER STARTUP 09:44:51 08/19/92
PAGE 1
```

```
MS-DOS 5.0(045-N) 386(TM) MACRO ASSEMBLER V4.0, ASSEMBLY OF MODULE
STARTUP
```

```
OBJECT MODULE PLACED IN startup.obj
```

```
ASSEMBLER INVOKED BY: f:\386tools\ASM386.EXE startup.a58 pw (132 )
```

```
LINE      SOURCE

1          NAME      STARTUP
2
3          ;;;;;;;;;;;;;;
4          ;
5          ; ASSUMPTIONS:
6          ;
7          ; 1. The bottom 64K of memory is ram, and can be used for
8          ;    scratch space by this module.
9          ;
10         ; 2. The system has sufficient free usable ram to copy the
11         ;    initial GDT, IDT, and TSS
12         ;
13         ;;;;;;;;;;;;;;
14
15         ; configuration data - must match with build definition
16
17         CS_BASE      EQU      0FFFF0000H
18
19         ; CS_BASE is the linear address of the segment STARTUP_CODE
20         ; - this is specified in the build language file
21
22         RAM_START    EQU      400H
23
24         ; RAM_START is the start of free, usable ram in the linear
25         ; memory space. The GDT, IDT, and initial TSS will be
26         ; copied above this space, and a small data segment will be
27         ; discarded at this linear address. The 32-bit word at
28         ; RAM_START will contain the linear address of the first
29         ; free byte above the copied tables - this may be useful if
30         ; a memory manager is used.
```

```

31
32 TSS_INDEX      EQU      10
33
34 ; TSS_INDEX is the index of the TSS of the first task to
35 ; run after startup
36
37
38 ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;
39
40 ; ----- STRUCTURES and EQU -----
41 ; structures for system data
42
43 ; TSS structure
44 TASK_STATE      STRUC
45     link          DW ?
46     link_h        DW ?
47     ESP0          DD ?
48     SS0           DW ?
49     SS0_h         DW ?
50     ESP1          DD ?
51     SS1           DW ?
52     SS1_h         DW ?
53     ESP2          DD ?
54     SS2           DW ?
55     SS2_h         DW ?
56     CR3_reg       DD ?
57     EIP_reg       DD ?
58     EFLAGS_reg    DD ?
59     EAX_reg       DD ?
60     ECX_reg       DD ?
61     EDX_reg       DD ?
62     EBX_reg       DD ?
63     ESP_reg       DD ?
64     EBP_reg       DD ?
65     ESI_reg       DD ?
66     EDI_reg       DD ?
67     ES_reg        DW ?
68     ES_h          DW ?
69     CS_reg        DW ?
70     CS_h          DW ?
71     SS_reg        DW ?
72     SS_h          DW ?
73     DS_reg        DW ?
74     DS_h          DW ?
75     FS_reg        DW ?
76     FS_h          DW ?
77     GS_reg        DW ?

```

```

78      GS_h                      DW ?
79      LDT_reg                   DW ?
80      LDT_h                     DW ?
81      TRAP_reg                  DW ?
82      IO_map_base               DW ?
83  TASK_STATE  ENDS
84
85  ; basic structure of a descriptor
86  DESC      STRUC
87      lim_0_15                   DW ?
88      bas_0_15                   DW ?
89      bas_16_23                  DB ?
90      access                     DB ?
91      gran                       DB ?
92      bas_24_31                  DB ?
93  DESC      ENDS
94
95  ; structure for use with LGDT and LIDT instructions
96  TABLE_REG  STRUC
97      table_lim                   DW ?
98      table_linear                 DD ?
99  TABLE_REG  ENDS
100
101  ; offset of GDT and IDT descriptors in builder generated GDT
102  GDT_DESC_OFF  EQU 1*SIZE(DESC)
103  IDT_DESC_OFF  EQU 2*SIZE(DESC)
104
105  ; equates for building temporary GDT in RAM
106  LINEAR_SEL      EQU      1*SIZE (DESC)
107  LINEAR_PROTO_LO  EQU      00000FFFFH  ; LINEAR_ALIAS
108  LINEAR_PROTO_HI  EQU      000CF9200H
109
110  ; Protection Enable Bit in CR0
111  PE_BIT  EQU 1B
112
113  ; -----
114
115  ; ----- DATA SEGMENT-----
116
117  ; Initially, this data segment starts at linear 0, according
118  ; to the processor's power-up state.
119
120  STARTUP_DATA      SEGMENT RW
121
122  free_mem_linear_base  LABEL  DWORD
123  TEMP_GDT              LABEL  BYTE  ; must be first in segment
124  TEMP_GDT_NULL_DESC    DESC      <>

```

```

125 TEMP_GDT_LINEAR_DESC DESC    <>
126
127 ; scratch areas for LGDT and LIDT instructions
128 TEMP_GDT_SCRATCH TABLE_REG  <>
129 APP_GDT_RAM      TABLE_REG  <>
130 APP_IDT_RAM      TABLE_REG  <>
131          ; align end_data
132 fill      DW      ?
133
134 ; last thing in this segment - should be on a dword boundary
135 end_data   LABEL   BYTE
136
137 STARTUP_DATA      ENDS
138 ; -----
139
140
141 ; ----- CODE SEGMENT-----
142 STARTUP_CODE SEGMENT ER PUBLIC USE16
143
144 ; filled in by builder
145     PUBLIC  GDT_EPROM
146 GDT_EPROM  TABLE_REG  <>
147
148 ; filled in by builder
149     PUBLIC  IDT_EPROM
150 IDT_EPROM  TABLE_REG  <>
151
152 ; entry point into startup code - the bootstrap will vector
153 ; here with a near JMP generated by the builder.  This
154 ; label must be in the top 64K of linear memory.
155
156     PUBLIC  STARTUP
157 STARTUP:
158
159 ; DS,ES address the bottom 64K of flat linear memory
160     ASSUME  DS:STARTUP_DATA, ES:STARTUP_DATA
161 ; See Figure 9-4
162 ; load GDTR with temporary GDT
163     LEA     EBX,TEMP_GDT  ; build the TEMP_GDT in low ram,
164     MOV     DWORD PTR [EBX],0    ; where we can address
165     MOV     DWORD PTR [EBX]+4,0
166     MOV     DWORD PTR [EBX]+8, LINEAR_PROTO_LO
167     MOV     DWORD PTR [EBX]+12, LINEAR_PROTO_HI
168     MOV     TEMP_GDT_scratch.table_linear,EBX
169     MOV     TEMP_GDT_scratch.table_lim,15
170
171     DB      66H          ; execute a 32 bit LGDT

```

```

172          LGDT      TEMP_GDT_scratch
173
174      ; enter protected mode
175          MOV        EBX,CR0
176          OR         EBX,PE_BIT
177          MOV        CR0,EBX
178
179      ; clear prefetch queue
180          JMP        CLEAR_LABEL
181  CLEAR_LABEL:
182
183      ; make DS and ES address 4G of linear memory
184          MOV        CX,LINEAR_SEL
185          MOV        DS,CX
186          MOV        ES,CX
187
188      ; do board specific initialization
189      ;
190          ;
191          ; .....
192          ;
193
194
195          ; See Figure 9-5
196          ; copy EPROM GDT to ram at:
197          ;          RAM_START + size (STARTUP_DATA)
198          MOV        EAX,RAM_START
199          ADD        EAX,OFFSET (end_data)
200          MOV        EBX,RAM_START
201          MOV        ECX, CS_BASE
202          ADD        ECX, OFFSET (GDT_EPROM)
203          MOV        ESI, [ECX].table_linear
204          MOV        EDI,EAX
205          MOVZX      ECX, [ECX].table_lim
206          MOV        APP_GDT_ram[EBX].table_lim,CX
207          INC        ECX
208          MOV        EDX,EAX
209          MOV        APP_GDT_ram[EBX].table_linear,EAX
210          ADD        EAX,ECX
211      REP MOVSB      BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
212
213          ; fixup GDT base in descriptor
214          MOV        ECX,EDX
215          MOV        [EDX].bas_0_15+GDT_DESC_OFF,CX
216          ROR        ECX,16
217          MOV        [EDX].bas_16_23+GDT_DESC_OFF,CL
218          MOV        [EDX].bas_24_31+GDT_DESC_OFF,CH

```



```

219
220         ; copy EPROM IDT to ram at:
221         ; RAM_START+size(STARTUP_DATA)+SIZE (EPROM GDT)
222         MOV     ECX, CS_BASE
223         ADD     ECX, OFFSET (IDT_EPROM)
224         MOV     ESI, [ECX].table_linear
225         MOV     EDI,EAX
226         MOVZX   ECX, [ECX].table_lim
227         MOV     APP_IDT_ram[EBX].table_lim,CX
228         INC     ECX
229         MOV     APP_IDT_ram[EBX].table_linear,EAX
230         MOV     EBX,EAX
231         ADD     EAX,ECX
232     REP MOVSB   BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
233
234         ; fixup IDT pointer in GDT
235         MOV     [EDX].bas_0_15+IDT_DESC_OFF,BX
236         ROR     EBX,16
237         MOV     [EDX].bas_16_23+IDT_DESC_OFF,BL
238         MOV     [EDX].bas_24_31+IDT_DESC_OFF,BH
239
240         ; load GDTR and IDTR
241         MOV     EBX,RAM_START
242         DB      66H                ; execute a 32 bit LGDT
243         LGDT    APP_GDT_ram[EBX]
244         DB      66H                ; execute a 32 bit LIDT
245         LIDT    APP_IDT_ram[EBX]
246
247         ; move the TSS
248         MOV     EDI,EAX
249         MOV     EBX,TSS_INDEX*SIZE(DESC)
250         MOV     ECX,GDT_DESC_OFF ;build linear address for TSS
251         MOV     GS,CX
252         MOV     DH,GS:[EBX].bas_24_31
253         MOV     DL,GS:[EBX].bas_16_23
254         ROL     EDX,16
255         MOV     DX,GS:[EBX].bas_0_15
256         MOV     ESI,EDX
257         LSL     ECX,EBX
258         INC     ECX
259         MOV     EDX,EAX
260         ADD     EAX,ECX
261     REP MOVSB   BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
262
263         ; fixup TSS pointer
264         MOV     GS:[EBX].bas_0_15,DX
265         ROL     EDX,16

```

```

266         MOV     GS:[EBX].bas_24_31,DH
267         MOV     GS:[EBX].bas_16_23,DL
268         ROL     EDX,16
269         ;save start of free ram at linear location RAMSTART
270         MOV     free_mem_linear_base+RAM_START,EAX
271
272         ;assume no LDT used in the initial task - if necessary,
273         ;code to move the LDT could be added, and should resemble
274         ;that used to move the TSS
275
276         ; load task register
277         LTR     BX    ; No task switch, only descriptor loading
278         ; See Figure 9-6
279         ; load minimal set of registers necessary to simulate task
280         ; switch
281
282
283         MOV     AX,[EDX].SS_reg    ; start loading registers
284         MOV     EDI,[EDX].ESP_reg
285         MOV     SS,AX
286         MOV     ESP,EDI            ; stack now valid
287         PUSH    DWORD PTR [EDX].EFLAGS_reg
288         PUSH    DWORD PTR [EDX].CS_reg
289         PUSH    DWORD PTR [EDX].EIP_reg
290         MOV     AX,[EDX].DS_reg
291         MOV     BX,[EDX].ES_reg
292         MOV     DS,AX    ; DS and ES no longer linear memory
293         MOV     ES,BX
294
295         ; simulate far jump to initial task
296         IRETD
297
298  STARTUP_CODE  ENDS
*** WARNING #377 IN 298, (PASS 2) SEGMENT CONTAINS PRIVILEGED
INSTRUCTION(S)
299
300  END STARTUP, DS:STARTUP_DATA, SS:STARTUP_DATA
301
302

```

ASSEMBLY COMPLETE, 1 WARNING, NO ERRORS.

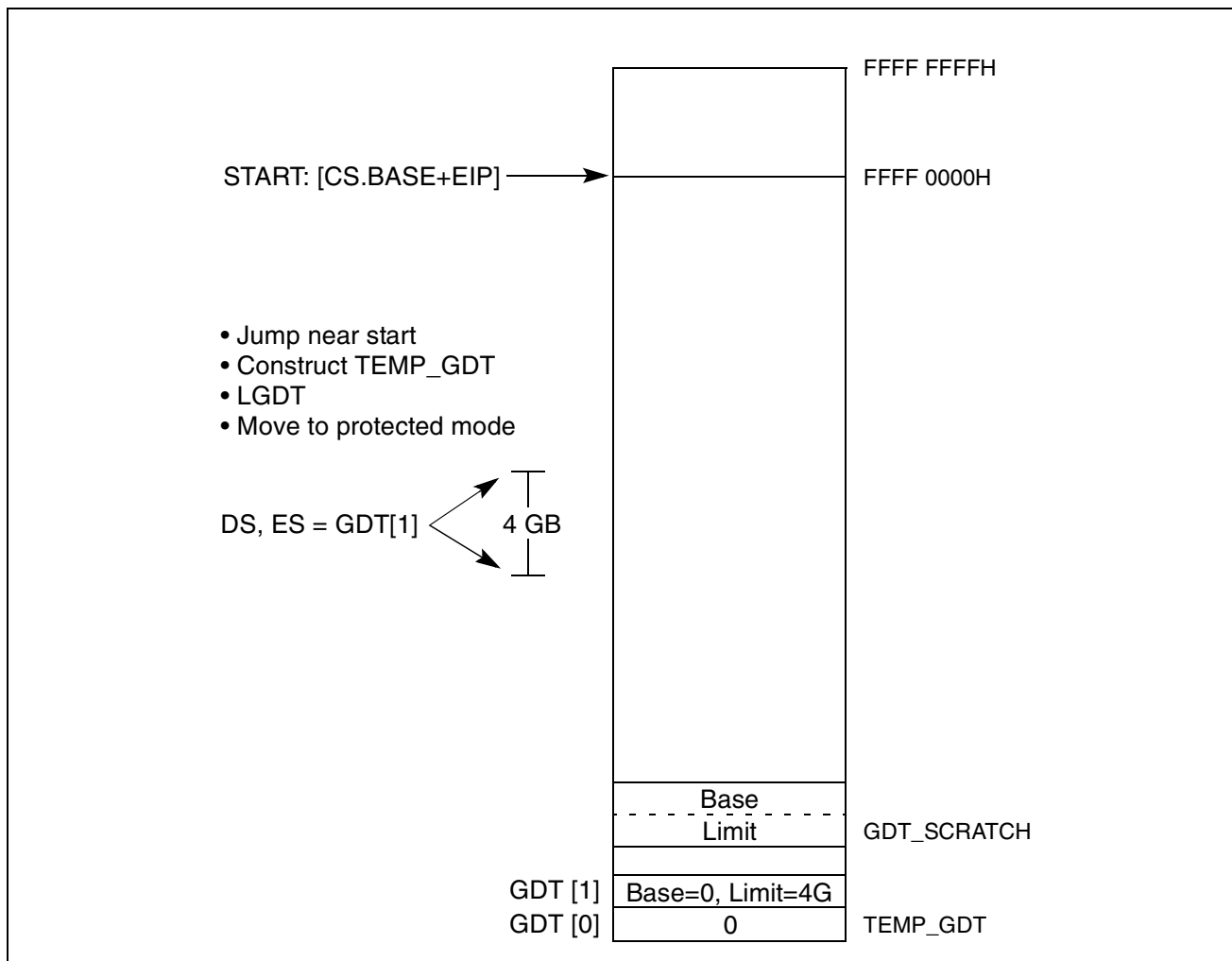


Figure 9-4. Constructing Temporary GDT and Switching to Protected Mode (Lines 162-172 of List File)

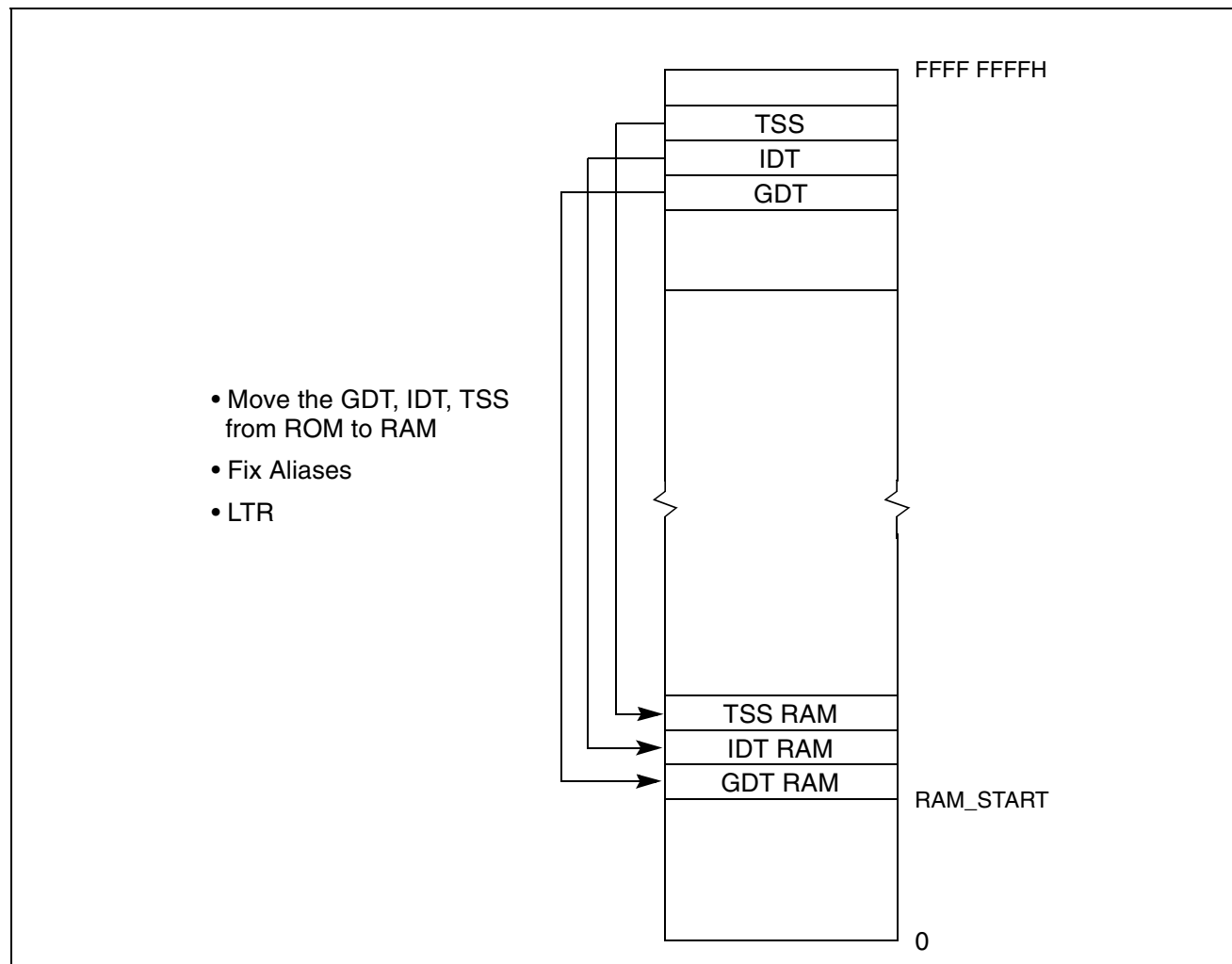


Figure 9-5. Moving the GDT, IDT and TSS from ROM to RAM (Lines 196-261 of List File)

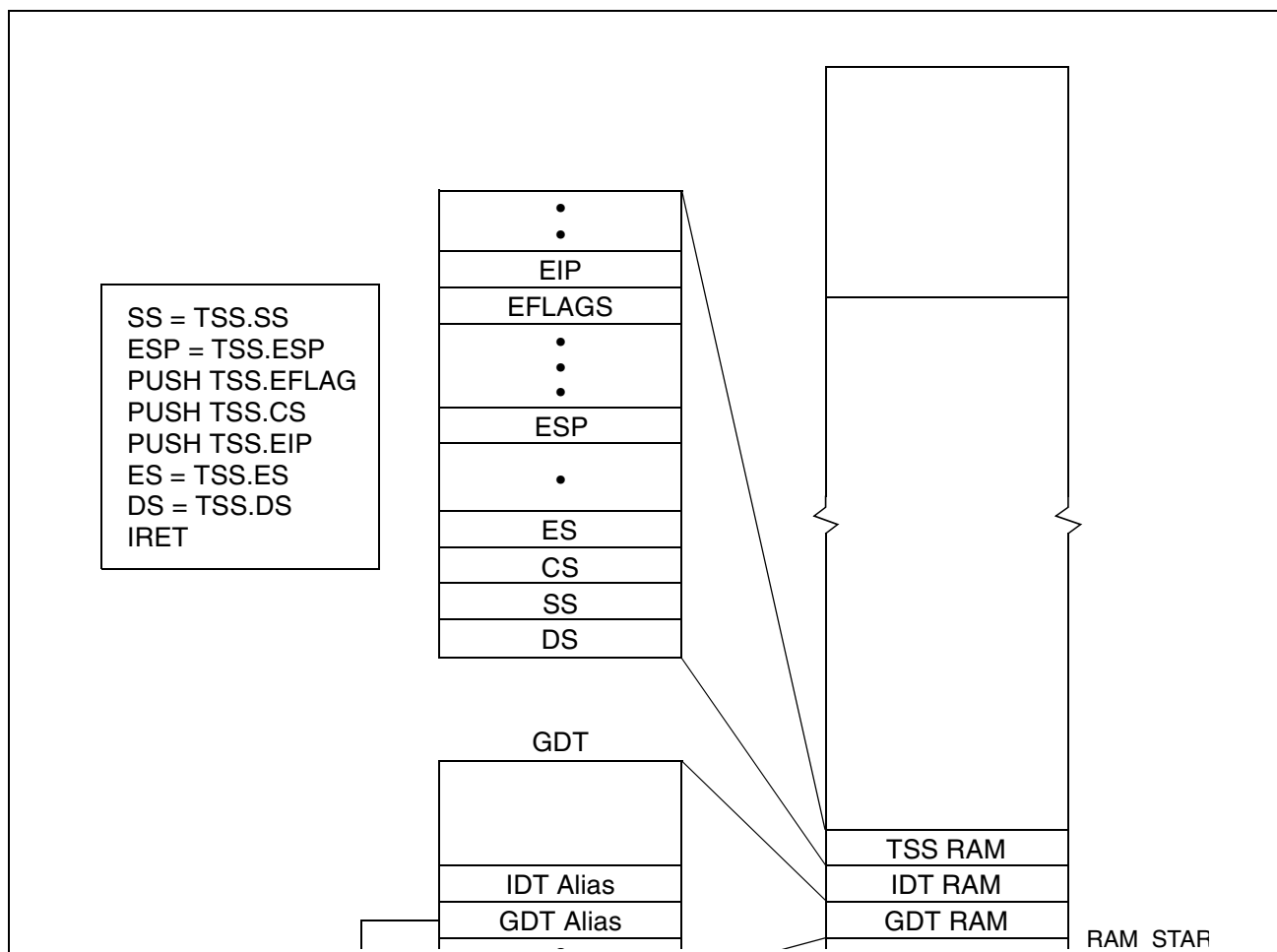


Figure 9-6. Task Switching (Lines 282-296 of List File)

9.10.3. MAIN.ASM Source Code

The file MAIN.ASM shown in Example 9-2 defines the data and stack segments for this application and can be substituted with the main module task written in a high-level language that is invoked by the IRET instruction executed by STARTUP.ASM.

Example 9-2. MAIN.ASM

```
NAME    main_module
data    SEGMENT RW
        dw 1000 dup(?)
DATA    ENDS
stack   stackseg 800
CODE SEGMENT ER use32 PUBLIC
main_start:
        nop
        nop
        nop
CODE    ENDS
END main_start, ds:data, ss:stack
```

9.10.4. Supporting Files

The batch file shown in Example 9-3 can be used to assemble the source code files STARTUP.ASM and MAIN.ASM and build the final application.

Example 9-3. Batch File to Assemble and Build the Application

```
ASM386 STARTUP.ASM
ASM386 MAIN.ASM
BLD386 STARTUP.OBJ, MAIN.OBJ buildfile(EPROM.BLD) bootstrap(STARTUP)
Bootload
```

BLD386 performs several operations in this example:

- It allocates physical memory location to segments and tables.
- It generates tables using the build file and the input files.
- It links object files and resolves references.
- It generates a boot-loadable file to be programmed into the EPROM.

Example 9-4 shows the build file used as an input to BLD386 to perform the above functions.

Example 9-4. Build File

```
INIT_BLD_EXAMPLE;

SEGMENT
    *SEGMENTS(DPL = 0)
    , startup.startup_code(BASE = 0FFFF0000H)
    ;

TASK
    BOOT_TASK(OBJECT = startup, INITIAL,DPL = 0,
               NOT INTENABLED)
    , PROTECTED_MODE_TASK(OBJECT = main_module,DPL = 0,
                           NOT INTENABLED)
    ;

TABLE
    GDT (
        LOCATION = GDT_EPROM
        , ENTRY = (
            10: PROTECTED_MODE_TASK
            , startup.startup_code
            , startup.startup_data
            , main_module.data
            , main_module.code
            , main_module.stack
        )
    )
```

```

    )
),

IDT (
    LOCATION = IDT_EEPROM
);

MEMORY
(
    RESERVE = (0..3FFFH
                -- Area for the GDT, IDT, TSS copied from
ROM
    ,
    , 60000H..0FFFFFFFFFFH)
    , RANGE = (ROM_AREA = ROM (0FFFF0000H..0FFFFFFFFFH))
    , -- Eeprom size 64K
    , RANGE = (RAM_AREA = RAM (4000H..05FFFFFH))
);

END

```

Table 9-5 shows the relationship of each build item with an ASM source file.

Table 9-5. Relationship Between BLD Item and ASM Source File

Item	ASM386 and Startup.A58	BLD386 Controls and BLD file	Effect
Bootstrap	public startup startup:	bootstrap start(startup)	Near jump at 0FFFFFFF0H to start
GDT location	public GDT_EEPROM GDT_EEPROM TABLE_REG <>	TABLE GDT(location = GDT_EEPROM)	The location of the GDT will be programmed into the GDT_EEPROM location
IDT location	public IDT_EEPROM IDT_EEPROM TABLE_REG <>	TABLE IDT(location = IDT_EEPROM)	The location of the IDT will be programmed into the IDT_EEPROM location
RAM start	RAM_START equ 400H	memory (reserve = (0..3FFFH))	RAM_START is used as the ram destination for moving the tables. It must be excluded from the application's segment area.
Location of the application TSS in the GDT	TSS_INDEX EQU 10	TABLE GDT(ENTRY=(10: PROTECTED_MODE_TASK))	Put the descriptor of the application TSS in GDT entry 10

Table 9-5. Relationship Between BLD Item and ASM Source File (Contd.)

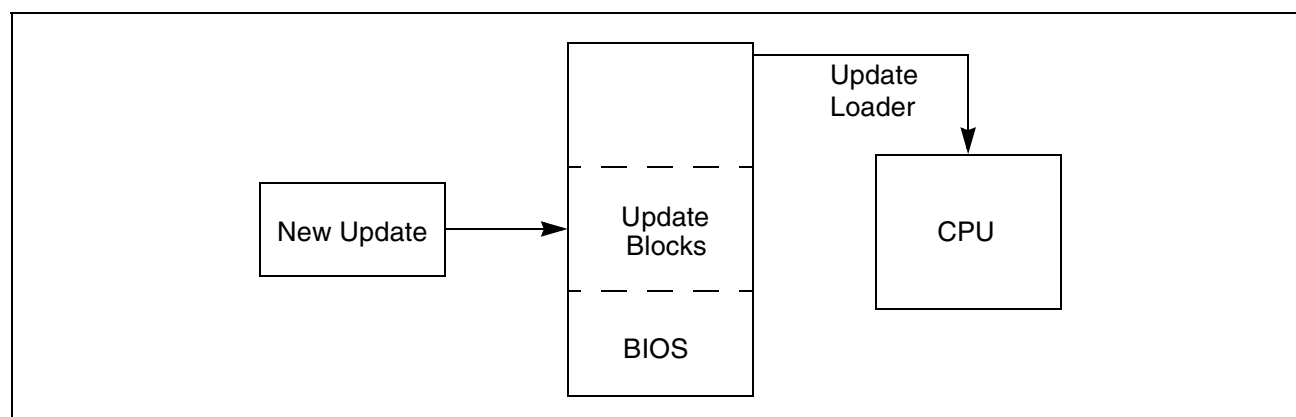
Item	ASM386 and Startup.A58	BLD386 Controls and BLD file	Effect
EPROM size and location	size and location of the initialization code	SEGMENT startup.code (base= 0FFFF0000H) ...memory (RANGE(ROM_AREA = ROM(x..y))	Initialization code size must be less than 64K and resides at upper most 64K of the 4 GB memory space.

9.11. MICROCODE UPDATE FACILITIES

The Pentium 4, Intel Xeon, and P6 family processors have the capability to correct errata by loading an Intel-supplied data block into the processor. The data block is called a *microcode update*. This section describes the mechanisms the BIOS needs to provide in order to use this feature during system initialization. It also describes a specification that permits the incorporation of future updates into a system BIOS.

Intel considers the release of a microcode update for a silicon revision to be the equivalent of a processor stepping. Intel completes full-stepping level validation for releases of microcode updates.

A microcode update is used to correct errata in the processor. The BIOS, which has an update loader, is responsible for loading the update on processors during system initialization (refer to Figure 9-7). There are two steps to this process: the first is to incorporate the necessary update data blocks into the BIOS; the second is to load update data blocks into the processor.

**Figure 9-7. Integrating Processor Specific Updates**

9.11.1. Microcode Update

A microcode update consists of an Intel-supplied binary that contains a descriptive header and data. No executable code resides within the update. Each microcode update is tailored for a particular stepping of a Pentium 4, Intel Xeon, or P6 family processor. A mismatch between the stepping of the processor and the update results in a failure to load.

The processor platform type must be determined to properly target the microcode update. It is determined by reading a model-specific register MSR (17H). Read the 64-bit register by using the RDMSR instruction. See Table 9-6. The three *platform ID bits*, when read as a binary coded decimal (BCD) number, indicate the bit position in the microcode update header's *Processor Flags* field that is associated with the installed processor.

Register Name: IA32_PLATFORM_ID

MSR Address: 017H

Access: Read Only

IA32_PLATFORM_ID is a 64-bit MSR accessed only when referenced as a quadword using the RDMSR instruction.

Table 9-6. Processor MSR Register Components

Bit	Descriptions
63:53	Reserved
52:50	Platform ID bits (RO). The field gives information concerning the intended platform for the processor. 525150 0000100110111 Processor Flag 0 (See <i>Processor Flags</i> in Microcode Update Header) Processor Flag 1 Processor Flag 2 Processor Flag 3 Processor Flag 4 Processor Flag 5 Processor Flag 6 Processor Flag 7
49:0	Reserved

The microcode update is a data block that is exactly 2048 bytes in length. The initial 48 bytes of the update contain a header with information used to identify the update. The update header and its reserved fields are interpreted by software based upon the header version. The initial version of the header is 00000001H. An encoding scheme guards against tampering of the update data and provides a means for determining the authenticity of any given update.

Table 9-7 defines each of the fields and Figure 9-8 shows the format of the microcode update data block.

Table 9-7. Microcode Update Encoding Format

Field Name	Offset (in bytes)	Length (in bytes)	Description
Header Version	0	4	Version number of the update header.
Update Revision	4	4	Unique version number for the update, the basis for the update signature provided by the processor to indicate the current update functioning within the processor. Used by the BIOS to authenticate the update and verify that it is loaded successfully by the processor. The value in this field cannot be used for processor stepping identification alone.
Date	8	4	Date of the update creation in binary format: mmddyyyy (e.g. 07/18/98 is 07181998h).
Processor	12	4	<i>Processor type, family, model, and stepping</i> of processor that requires this particular update revision (e.g., 00000650h). Each microcode update is designed specifically for a given <i>processor type, family, model, and stepping</i> of processor. The BIOS uses the Processor field in conjunction with the CPUID instruction to determine whether or not an update is appropriate to load on a processor. The information encoded within this field exactly corresponds to the bit representations returned by the CPUID instruction.
Checksum	16	4	Checksum of update data and header. Used to verify the integrity of the update header and data. Checksum is correct when the summation of the 512 double words of the update result in the value zero.
Loader Revision	20	4	Version number of the loader program needed to correctly load this update. The initial version is 00000001h.
Processor Flags	24	4	Platform type information is encoded in the lower 8 bits of this 4-byte field. Each bit represents a particular platform type for a given CPUID. The BIOS uses the Processor Flags field in conjunction with the platform ID bits in MSR (17h) to determine whether or not an update is appropriate to load on a processor.
Reserved	28	20	Reserved Fields for future expansion.
Update Data	48	2000	Update data.

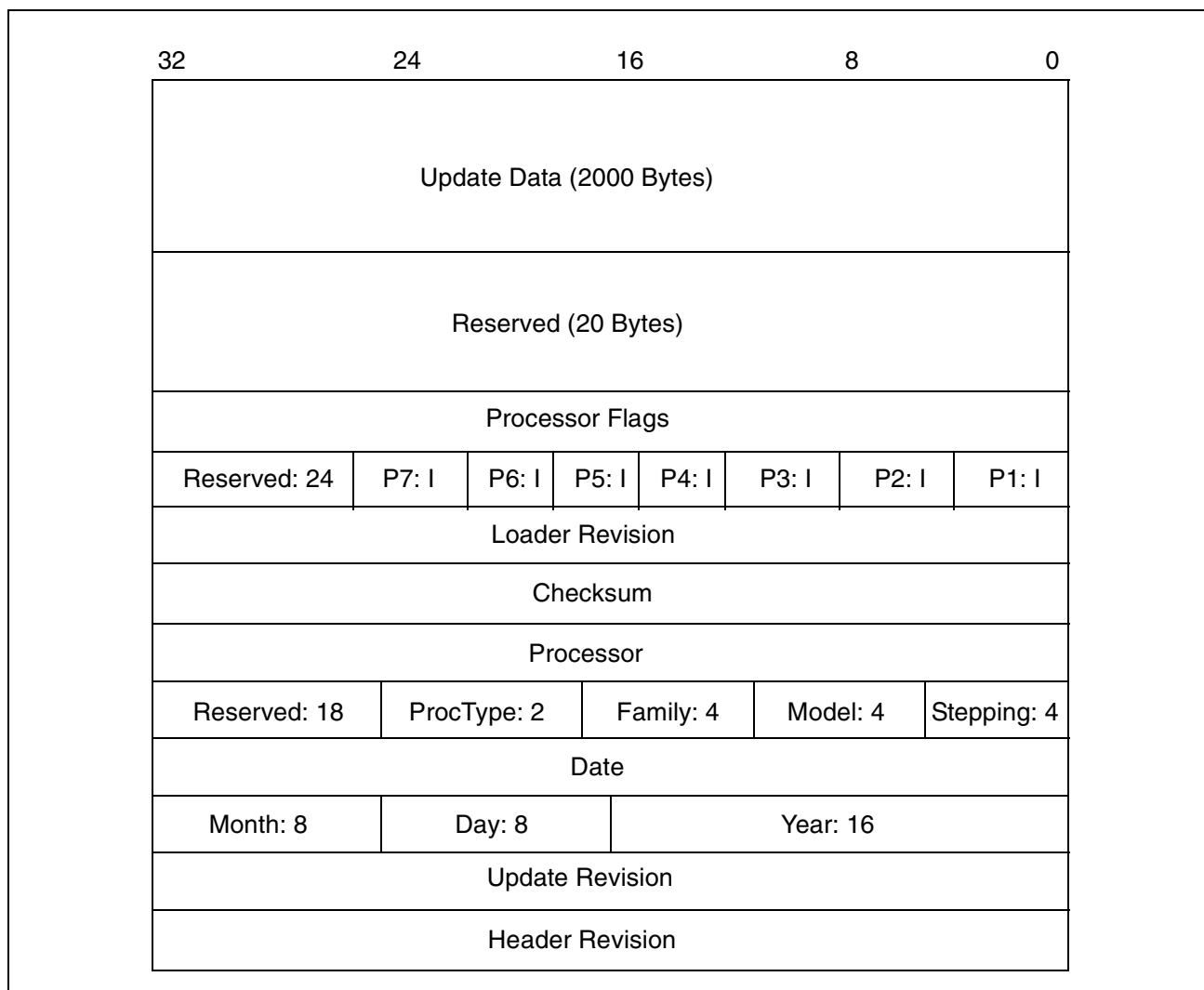


Figure 9-8. Format of the Microcode Update Data Block

9.11.2. Microcode Update Loader

This section describes the update loader used to load a microcode update into a Pentium 4, Intel Xeon, or P6 family processor. It also discusses the requirements placed upon the BIOS to ensure proper loading of an update.

The update loader contains the minimal instructions needed to load an update. The specific instruction sequence that is required to load an update is dependent upon the loader revision field contained within the update header. The revision of the update loader is expected to change very infrequently, potentially only when new processor models are introduced.

The code below represents the update loader with a loader revision of 00000001H:

```
mov    ecx,79h          ; MSR to read in ECX
xor    eax,eax          ; clear EAX
xor    ebx,ebx          ; clear EBX
mov    ax,cs            ; Segment of microcode update
shl    eax,4
mov    bx,offset Update; Offset of microcode update
add    eax,ebx          ; Linear Address of Update in EAX
add    eax,48d          ; Offset of the Update Data within the Update
xor    edx,edx          ; Zero in EDX
WRMSR                     ; microcode update trigger
```

9.11.2.1. UPDATE LOADING PROCEDURE

The simple loader previously described assumes that Update is the address of a microcode update (header and data) embedded within the code segment of the BIOS. It also assumes that the processor is operating in real mode. The data may reside anywhere in memory that is accessible by the processor within its current operating mode (real, protected).

Before the BIOS executes the microcode update trigger (WRMSR) instruction the following must be true:

- EAX contains the linear address of the start of the update data
- EDX contains zero
- ECX contains 79H

The value in the ECX register is the address of the IA32_BIOS_UPDT_TRIG MSR. Writing to this MSR causes the microcode update located at the linear address in the EAX register to be loaded into the processor.

Other requirements to keep in mind are:

- The microcode update must be loaded to the processor early on in the POST, and always prior to the initialization of the processors L2 cache controller.
- If the update is loaded while the processor is in real mode, then the update data may not cross a segment boundary.
- If the update is loaded while the processor is in real mode, then the update data may not exceed a segment limit.
- If paging is enabled, pages that are currently present and accessed must map the update data.
- The microcode update data does not require any particular byte or word boundary alignment.

9.11.2.2. HARD RESETS IN UPDATE LOADING

The effects of a loaded update are cleared from the processor upon a hard reset. Therefore, each time a hard reset is asserted during the BIOS POST, the update must be reloaded on all processors that observed the reset. The effects of a loaded update are, however, maintained across a processor INIT. There are no side effects caused by loading an update into a processor multiple times.

9.11.2.3. UPDATE IN A MULTIPROCESSOR SYSTEM

A multiprocessor (MP) system requires loading each processor with update data appropriate for its CPUID and platform ID bits. The BIOS is responsible for ensuring that this requirement is met, and that the loader is located in a module that is executed by all processors in the system. If a system design permits multiple steppings of Pentium 4, Intel Xeon, and P6 family processors to exist concurrently, then the BIOS must verify each individual processor against the update header information to ensure appropriate loading. Given these considerations, it is most practical to load the update during MP initialization.

9.11.2.4. UPDATE LOADER ENHANCEMENTS

The update loader presented in Section 9.11.2.1., “Update Loading Procedure” is a minimal implementation that can be enhanced to provide additional functionality and features. Some potential enhancements are described below:

- The BIOS can incorporate multiple updates to support multiple steppings of the Pentium 4, Intel Xeon, and P6 family processors. This feature provides for operating in a mixed stepping environment on an MP system and enables a user to upgrade to a later version of the processor. In this case, modify the loader to check the CPUID and platform ID bits of the processor that it is running on against the available headers before loading a particular update. The number of updates is only limited by the available space in the BIOS.
- A loader can load the update and test the processor to determine if the update was loaded correctly. This can be done as described in the Section 9.11.3., “Update Signature and Verification”.
- A loader can verify the integrity of the update data by performing a checksum on the double words of the update summing to zero, and can reject the update.
- A loader can provide power-on messages indicating successful loading of an update.

9.11.3. Update Signature and Verification

The Pentium 4, Intel Xeon, and P6 family processors provides capabilities to verify the authenticity of a particular update and to identify the current update revision. This section describes the model-specific extensions of the processor that support this feature. The update verification method below assumes that the BIOS will only verify an update that is more recent than the revision currently loaded into the processor.

The CUID instruction returns a value in a model specific register in addition to its usual register return values. The semantics of the CUID instruction cause it to deposit an update ID value in the 64-bit model-specific register (MSR) at address 08BH. If no update is present in the processor, the value in the MSR remains unmodified. Normally a zero value is preloaded into the MSR by software before executing the CUID instruction. If the MSR still contains zero after executing CUID, this indicates that no update is present.

The update ID value returned in the EDX register after a RDMSR instruction indicates the revision of the update loaded in the processor. This value, in combination with the normal CUID value returned in the EAX register, uniquely identifies a particular update. The signature ID can be directly compared with the update revision field in the microcode update header for verification of a correct update load. No consecutive updates released for a given stepping of a Pentium 4, Intel Xeon, or P6 family processor may share the same signature. Updates for different steppings are differentiated by the CUID value.

9.11.3.1. DETERMINING THE SIGNATURE

An update that is successfully loaded into the processor provides a signature that matches the update revision of the currently functioning revision. This signature is available any time after the actual update has been loaded, and requesting this signature does not have any negative impact upon any currently loaded update. The procedure for determining this signature is:

```

mov ecx, 08Bh    ;Model Specific Register to Read in ECX
xor eax,eax      ;clear EAX
xor edx,edx      ;clear EDX
WRMSR           ;Load 0 to MSR at 8Bh
mov eax,1
CUID
mov ecx, 08BH    ;Model Specific Register to Read
RDMSR           ;Read Model Specific Register

```

If there is an update currently active in the processor, its update revision is returned in the EDX register after the RDMSR instruction has completed.

9.11.3.2. AUTHENTICATING THE UPDATE

An update may be authenticated by the BIOS using the signature primitive, described above, with the following algorithm:

```

Z = Update revision from the update header to be authenticated;
X = Current Update Signature from MSR 8Bh;
If (Z > X) Then
    Load Update that is to be authenticated;
    Y = New Signature from MSR 8Bh;
    If (Z == Y) then Success
    Else Fail
Else Fail

```

The algorithm requires that the BIOS only authenticate updates that contain a numerically larger revision than the currently loaded revision, where Current Signature (X) < New Update Revision (Z). A processor with no update loaded should be considered to have a revision equal to zero. This authentication procedure relies upon the decoding provided by the processor to verify an update from a potentially hostile source. As an example, this mechanism in conjunction with other safeguards provides security for dynamically incorporating field updates into the BIOS.

9.11.4. Pentium 4, Intel Xeon, and P6 Family Processor Microcode Update Specifications

This section describes the interface that an application can use to dynamically integrate processor-specific updates into the system BIOS. In this discussion, the application is referred to as the *calling program* or *caller*.

The real mode INT15 call specification described here is an Intel extension to an OEM BIOS. This extension allows an application to read and modify the contents of the microcode update data in NVRAM. The update loader, which is part of the system BIOS, cannot be updated by the interface. All of the functions defined in the specification must be implemented for a system to be considered compliant with the specification. The INT15 functions are accessible only from real mode.

9.11.4.1. RESPONSIBILITIES OF THE BIOS

If a BIOS passes the presence test (INT 15H, AX=0D042H, BL=0H) it must implement all of the sub-functions defined in the INT 15H, AX= 0D042H specification. There are no optional functions. The BIOS must load the appropriate update for each processor during system initialization.

A header version of an update block containing the value 0FFFFFFFFH indicates that the update block is unused and available for storing a new update.

The BIOS is responsible for providing a 2048 byte region of non-volatile storage (NVRAM) for each potential processor stepping within a system. This storage unit is referred to as an *update block*. The BIOS for a single processor system need only provide one update block to store the microcode update data. The BIOS for a multiple processor capable system needs to provide one update block for each unique processor stepping supported by the OEM's system. The BIOS is responsible for managing the NVRAM update blocks. This includes garbage collection, such as removing update blocks that exist in NVRAM for which a corresponding processor does not exist in the system. This specification only provides the mechanism for ensuring security, the uniqueness of an entry, and that stale entries are not loaded. The actual update block management is implementation specific on a per-BIOS basis. As an example, the BIOS may use update blocks sequentially in ascending order with CPU signatures sorted versus the first available block. In addition, garbage collection may be implemented as a setup option to clear all NVRAM slots or as BIOS code that searches and eliminates unused entries during boot.

The following algorithm describes the steps performed during BIOS initialization used to load the updates into the processor(s). It assumes that the BIOS ensures that no update contained within NVRAM has a header version or loader version that does not match one currently supported by the BIOS and that the update block contains a correct checksum. It also assumes that the BIOS ensures that at most one update exists for each processor stepping and that older update revisions are not allowed to overwrite more recent ones. These requirements are checked by the BIOS during the execution of the write update function of this interface. The BIOS sequentially scans through all of the update blocks in NVRAM starting with index 0. The BIOS scans until it finds an update where the processor fields in the header match the family, model, and stepping as well as the platform ID bits of the current processor.

```

For each processor in the system {
  Determine the ProcType, Family, Model and Stepping via CPUID;
  Determine the Platform ID Bits by reading the IA32_PLATFORM_ID[52:50] MSR;
  for (I = UpdateBlock 0, I < NumOfUpdates; I++) {
    If ((UpdateHeader.Processor ==
        ProcType, Family, Model and Stepping) &&
        (UpdateHeader.ProcessorFlags == Platform ID Bits)) {
      Load UpdateHeader.UpdateData into the Processor;
      Verify that update was correctly loaded into the processor
      Go on to next processor
      Break;
    }
  }
}

```

NOTE

The platform ID bits in the IA32_PLATFORM_ID MSR are encoded as a three-bit binary coded decimal field. The platform ID bits in the microcode update header are individually bit encoded. The algorithm must do a translation from one format to the other prior to doing the comparison.

When performing the INT 15H, 0D042H functions, the BIOS must assume that the caller has no knowledge about platform specific requirements. It is the responsibility of the BIOS calls to manage all chipset and platform specific prerequisites for managing the NVRAM device. When writing the update data via the write update sub-function, the BIOS must maintain implementation specific data requirements, such as the update of NVRAM checksum. The BIOS should also attempt to verify the success of write operations on the storage device used to record the update.

9.11.4.2. RESPONSIBILITIES OF THE CALLING PROGRAM

This section of the document lists the responsibilities of the calling program using the interface specifications to load microcode update(s) into BIOS NVRAM.

The calling program should call the INT 15H, 0D042H functions from a pure real mode program and should be executing on a system that is running in pure real mode. The caller should issue the presence test function (sub function 0) and verify the signature and return codes of that function. It is important that the calling program provides the required scratch RAM buffers for the BIOS and the proper stack size as specified in the interface definition.

The calling program should read any update data that already exists in the BIOS in order to make decisions about the appropriateness of loading the update. The BIOS refuses to overwrite a newer update with an older version. The update header contains information about version and processor specifics for the calling program to make an intelligent decision about loading.

There can be no ambiguous updates. The BIOS refuses to allow multiple updates for the same CPUID to exist at the same time. The BIOS also refuses to load an update for a processor that does not exist in the system.

The calling application should implement a verify function that is run after the update write function successfully completes. This function reads back the update and verifies that the BIOS returned an image identical to the one that was written. The following pseudo-code represents a calling program.

INT 15 D042 Calling Program Pseudo-code

```
//
// We must be in real mode
//
If the system is not in Real mode
then Exit
//
// Detect the presence of Genuine Intel processor(s) that can be updated (CPUID)
//
If no Intel processors exist that can be updated
then Exit
//
// Detect the presence of the Intel microcode update extensions
//
If the BIOS fails the PresenceTest
then Exit
//
// If the APIC is enabled, see if any other processors are out there
//
Read APICBaseMSR
If APIC enabled {
    Send Broadcast Message to all processors except self via APIC;
    Have all processors execute CPUID and record Type, Family, Model, Stepping
    Have all processors read IA32_PLATFORM_ID[52:50] and record platform ID bits
    If current processor is not updatable
        then Exit
}
//
// Determine the number of unique update slots needed for this system
//
NumSlots = 0;
For each processor {
    If ((this is a unique processor stepping) and
        (we have an update in the database for this processor)) {
        Checksum the update from the database;
```

```
        If Checksum fails
            then Exit;
        Increment NumSlots;
    }
}
//
// Do we have enough update slots for all CPUs?
//
If there are more unique processor steppings than update slots provided by the BIOS
    then Exit

//
// Do we need any update slots at all? If not, then we're all done
//
If (NumSlots == 0)
    then Exit

//
// Record updates for processors in NVRAM.
//
For (I=0; I<NumSlots; I++) {
    //
    // Load each Update
    //
    Issue the WriteUpdate function

    If (STORAGE_FULL) returned {
        Display Error -- BIOS is not managing NVRAM appropriately
        exit
    }
    If (INVALID_REVISION) returned {
        Display Message: More recent update already loaded in NVRAM for this stepping
        continue;
    }

    If any other error returned {
        Display Diagnostic
        exit
    }
    //
    // Verify the update was loaded correctly
    //
    Issue the ReadUpdate function

    If an error occurred {
        Display Diagnostic
        exit
    }
}
```

```

    }
    //
    //   Compare the Update read to that written
    //
    if (Update read != Update written) {
        Display Diagnostic
        exit
    }
}
//
//   Enable Update Loading, and inform user
//
Issue the ControlUpdate function with Task=Enable.

```

9.11.4.3. MICROCODE UPDATE FUNCTIONS

Table 9-8 defines the current Pentium 4, Intel Xeon, and P6 family processor microcode update functions.

Table 9-8. Microcode Update Functions

Microcode Update Function	Function Number	Description	Required/Optional
Presence test	00H	Returns information about the supported functions.	Required
Write update data	01H	Writes one of the update data areas (slots).	Required
Update control	02H	Globally controls the loading of updates.	Required
Read update data	03H	Reads one of the update data areas (slots).	Required

9.11.4.4. INT 15H-BASED INTERFACE

Intel recommends that a BIOS interface be provided that allows additional microcode updates to be added to the system flash. The INT15H interface is an Intel-defined method for doing this.

The program that calls this interface is responsible for providing three 64-kilobyte RAM areas for BIOS use during calls to the read and write functions. These RAM scratch pads can be used by the BIOS for any purpose, but only for the duration of the function call. The calling routine places real mode segments pointing to the RAM blocks in the CX, DX and SI registers. Calls to functions in this interface must be made with a minimum of 32 kilobytes of stack available to the BIOS.

In general, each function returns with CF cleared and AH contains the returned status. The general return codes and other constant definitions are listed in Section 9.11.4.9., “Return Codes”.

The OEM Error (AL) is provided for the OEM to return additional error information specific to the platform. If the BIOS provides no additional information about the error, the OEM Error must be set to SUCCESS.

The OEM Error field is undefined if AH contains either SUCCESS (00) or NOT_IMPLEMENTED (86h). In all other cases it must be set with either SUCCESS or a value meaningful to the OEM.

The following text details the functions provided by the INT15H-based interface.

9.11.4.5. FUNCTION 00H—PRESENCE TEST

This function verifies that the BIOS has implemented the required microcode update functions. Table 9-9 lists the parameters and return codes for the function.

Table 9-9. Parameters for the Presence Test

Input		
AX	Function Code	0D042h
BL	Sub-function	00h - Presence Test
Output		
CF	Carry Flag	Carry Set - Failure - AH Contains Status. Carry Clear - All return values are valid.
AH	Return Code	
AL	OEM Error	Additional OEM Information.
EBX	Signature Part 1	'INTE' - Part one of the signature.
ECX	Signature Part 2	'LPEP' - Part two of the signature.
EDX	Loader Version	Version number of the microcode update loader.
SI	Update Count	Number of update blocks the system can record in NVRAM.
Return Codes (See Table 8-8 for code definitions)		
SUCCESS		Function completed successfully.
NOT_IMPLEMENTED		Function not implemented.

In order to assure that the BIOS function is present, the caller must verify the Carry Flag, the Return Code, and the 64-bit signature. Each update block is exactly 2048 bytes in length. The update count reflects the number of update blocks available for storage within non-volatile RAM. The update count must return with a value greater than or equal to the number of unique processor steppings currently installed within the system.

The loader version number refers to the revision of the update loader program that is included in the system BIOS image.

9.11.4.6. FUNCTION 01H—WRITE MICROCODE UPDATE DATA

This function integrates a new microcode update into the BIOS storage device. Table 9-10 lists the parameters and return codes for the function.

Table 9-10. Parameters for the Write Update Data Function

Input		
AX	Function Code	0D042H
BL	Sub-function	01H - Write Update
ED:DI	Update Address	Real Mode pointer to the Intel Update structure. This buffer is 2048 bytes in length
CX	Scratch Pad1	Real Mode Segment address of 64 kilobytes of RAM Block.
DX	Scratch Pad2	Real Mode Segment address of 64 kilobytes of RAM Block.
SI	Scratch Pad3	Real Mode Segment address of 64 kilobytes of RAM Block.
SS:SP	Stack pointer	32 kilobytes of Stack Minimum.
Output		
CF	Carry Flag	Carry Set - Failure - AH Contains Status. Carry Clear - All return values are valid.
AH	Return Code	Status of the Call
AL	OEM Error	Additional OEM Information.
Return Codes (See Table 8-8 for code definitions)		
SUCCESS	Function completed successfully.	
WRITE_FAILURE	A failure because of the inability to write the storage device.	
ERASE_FAILURE	A failure because of the inability to erase the storage device.	
READ_FAILURE	A failure because of the inability to read the storage device.	
STORAGE_FULL	The BIOS non-volatile storage area is unable to accommodate the update because all available update blocks are filled with updates that are needed for processors in the system.	
CPU_NOT_PRESENT	The processor stepping does not currently exist in the system.	
INVALID_HEADER	The update header contains a header or loader version that is not recognized by the BIOS.	
INVALID_HEADER_CS	The update does not checksum correctly.	
SECURITY_FAILURE	The processor rejected the update.	
INVALID_REVISION	The same or more recent revision of the update exists in the storage device.	

The BIOS is responsible for selecting an appropriate update block in the non-volatile storage for storing the new update. This BIOS is also responsible for ensuring the integrity of the information provided by the caller, including authenticating the proposed update before incorporating it into storage.

Before writing the update block into NVRAM, the BIOS should ensure that the update structure meets the following criteria in the following order:

1. The update header version should be equal to an update header version recognized by the BIOS.
2. The update loader version in the update header should be equal to the update loader version contained within the BIOS image.
3. The update block should checksum to zero. This checksum is computed as a 32-bit summation of all 512 double words in the structure, including the header.

The BIOS selects an update block in non-volatile storage for storing the candidate update. The BIOS can select any available update block as long as it guarantees that only a single update exists for any given processor stepping in non-volatile storage. If the update block selected already contains an update, the following additional criteria apply to overwrite it:

- The processor signature in the proposed update should be equal to the processor signature in the header of the current update in NVRAM (CPUID + platform ID bits).
- The update revision in the proposed update should be greater than the update revision in the header of the current update in NVRAM.

If no unused update blocks are available and the above criteria are not met, the BIOS can overwrite an update block for a processor stepping that is no longer present in the system. This can be done by scanning the update blocks and comparing the processor steppings, identified in the MP Specification table, to the processor steppings that currently exist in the system.

Finally, before storing the proposed update into NVRAM, the BIOS should verify the authenticity of the update via the mechanism described in Section 9.11.2., “Microcode Update Loader”. This includes loading the update into the current processor, executing the CPUID instruction, reading MSR 08Bh, and comparing a calculated value with the update revision in the proposed update header for equality.

When performing the write update function, the BIOS should record the entire update, including the header and the update data. When writing an update, the original contents may be overwritten, assuming the above criteria have been met. It is the responsibility of the BIOS to ensure that more recent updates are not overwritten through the use of this BIOS call, and that only a single update exists within the NVRAM for any processor stepping.

Figure 9-9 shows the process the BIOS follows to choose an update block and ensure the integrity of the data when it stores the new microcode update.

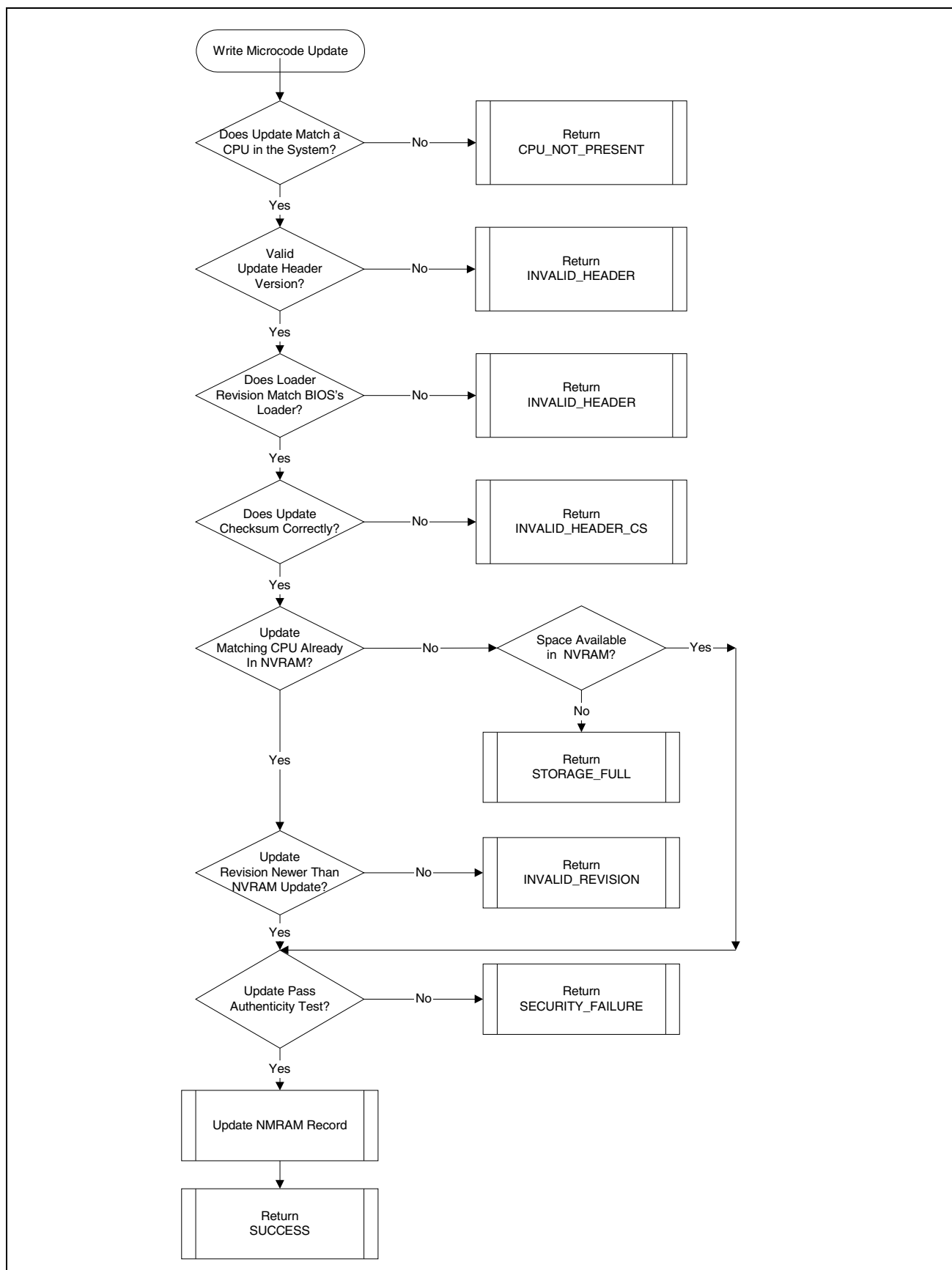


Figure 9-9. Write Operation Flow Chart

9.11.4.7. FUNCTION 02H—MICROCODE UPDATE CONTROL

This function enables loading of binary updates into the processor. Table 9-11 lists the parameters and return codes for the function.

Table 9-11. Parameters for the Control Update Sub-function

Input		
AX	Function Code	0D042H
BL	Sub-function	02H - Control Update
BH	Task	See Description.
CX	Scratch Pad1	Real Mode Segment of 64 kilobytes of RAM Block.
DX	Scratch Pad2	Real Mode Segment of 64 kilobytes of RAM Block.
SI	Scratch Pad3	Real Mode Segment of 64 kilobytes of RAM Block.
SS:SP	Stack pointer	32 kilobytes of Stack Minimum.
Output		
CF	Carry Flag	Carry Set - Failure - AH contains Status. Carry Clear - All return values are valid.
AH	Return Code	Status of the Call.
AL	OEM Error	Additional OEM Information.
BL	Update Status	Either Enable or Disable indicator.
Return Codes (See Table 8-8 for code definitions)		
SUCCESS		Function completed successfully.
READ_FAILURE		A failure because of the inability to read the storage device.

This control is provided on a global basis for all updates and processors. The caller can determine the current status of update loading (enabled or disabled) without changing the state. The function does not allow the caller to disable loading of binary updates, as this poses a security risk.

The caller specifies the requested operation by placing one of the values from Table 9-12 in the BH register. After successfully completing this function the BL register contains either the enable or the disable designator. Note that if the function fails, the update status return value is undefined.

Table 9-12. Mnemonic Values

Mnemonic	Value	Meaning
Enable	1	Enable the Update loading at initialization time
Query	2	Determine the current state of the update control without changing its status.

The READ_FAILURE error code returned by this function has meaning only if the control function is implemented in the BIOS NVRAM. The state of this feature (enabled/disabled) can also be implemented using CMOS RAM bits where READ failure errors cannot occur.

9.11.4.8. FUNCTION 03H—READ MICROCODE UPDATE DATA

This function reads a currently installed microcode update from the BIOS storage into a caller-provided RAM buffer. Table 9-13 lists the parameters and return codes for the function.

Table 9-13. Parameters for the Read Microcode Update Data Function

Input		
AX	Function Code	0D042H
BL	Sub-function	03H - Read Update
ES:DI	Buffer Address	Real Mode pointer to the Intel Update structure that will be written with the binary data.
ECX	Scratch Pad1	Real Mode Segment address of 64 kilobytes of RAM Block (lower 16 bits).
ECX	Scratch Pad2	Real Mode Segment address of 64 kilobytes of RAM Block (upper 16 bits).
DX	Scratch Pad3	Real Mode Segment address of 64 kilobytes of RAM Block.
SS:SP	Stack pointer	32 kilobytes of Stack Minimum.
SI	Update Number	The index number of the update block to be read. This value is zero based and must be less than the update count returned from the presence test function.
Output		
CF	Carry Flag	Carry Set - Failure - AH contains Status.
Carry Clear - All return values are valid.		
AH	Return Code	Status of the Call.
AL	OEM Error	Additional OEM Information.
Return Codes (See Table 8-8 for code definitions)		
SUCCESS		Function completed successfully.
READ_FAILURE		A failure because of the inability to read the storage device.
UPDATE_NUM_INVALID		Update number exceeds the maximum number of update blocks implemented by the BIOS.

The read function enables the caller to read any update data that already exists in a BIOS and make decisions about the addition of new updates. As a result of a successful call, the BIOS copies exactly 2048 bytes into the location pointed to by ES:DI, with the contents of the update block represented by update number.

An update block is considered unused and available for storing a new update if its header version contains the value 0FFFFFFFFH after return from this function call. The actual implementation of NVRAM storage management is not specified here and is BIOS dependent. As an example, the actual data value used to represent an empty block by the BIOS may be zero, rather than 0FFFFFFFFH. The BIOS is responsible for translating this information into the header provided by this function.

9.11.4.9. RETURN CODES

After the call has been made, the return codes listed in Table 9-14 are available in the AH register.

Table 9-14. Return Code Definitions

Return Code	Value	Description
SUCCESS	00H	Function completed successfully
NOT_IMPLEMENTED	86H	Function not implemented
ERASE_FAILURE	90H	A failure because of the inability to erase the storage device
WRITE_FAILURE	91H	A failure because of the inability to write the storage device
READ_FAILURE	92H	A failure because of the inability to read the storage device
STORAGE_FULL	93H	The BIOS non-volatile storage area is unable to accommodate the update because all available update blocks are filled with updates that are needed for processors in the system
CPU_NOT_PRESENT	94H	The processor stepping does not currently exist in the system
INVALID_HEADER	95H	The update header contains a header or loader version that is not recognized by the BIOS
INVALID_HEADER_CS	96h	The update does not checksum correctly
SECURITY_FAILURE	97H	The update was rejected by the processor
INVALID_REVISION	98H	The same or more recent revision of the update exists in the storage device
UPDATE_NUM_INVALID	99H	The update number exceeds the maximum number of update blocks implemented by the BIOS

10

Memory Cache Control

CHAPTER 10

MEMORY CACHE CONTROL

This chapter describes the IA-32 architecture's memory cache and cache control mechanisms, the TLBs, and the store buffer. It also describes the memory type range registers (MTRRs) found in the P6 family processors and how they are used to control caching of physical memory locations.

10.1. INTERNAL CACHES, TLBS, AND BUFFERS

The IA-32 architecture supports caches, translation look aside buffers (TLBs), and a store buffer for temporary on-chip (and external) storage of instructions and data. (Figure 10-1 shows the arrangement of caches, TLBs, and the store buffer for the Pentium 4 and Intel Xeon processors.) Table 10-1 shows the characteristics of these caches and buffers for the Pentium 4, Intel Xeon, P6 family, and Pentium processors. **The sizes and characteristics of these units are machine specific and may change in future versions of the processor.** The CUID instruction returns the sizes and characteristics of the caches and buffers for the processor on which the instruction is executed (see “CUID—CPU Identification” in Chapter 3 of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*).

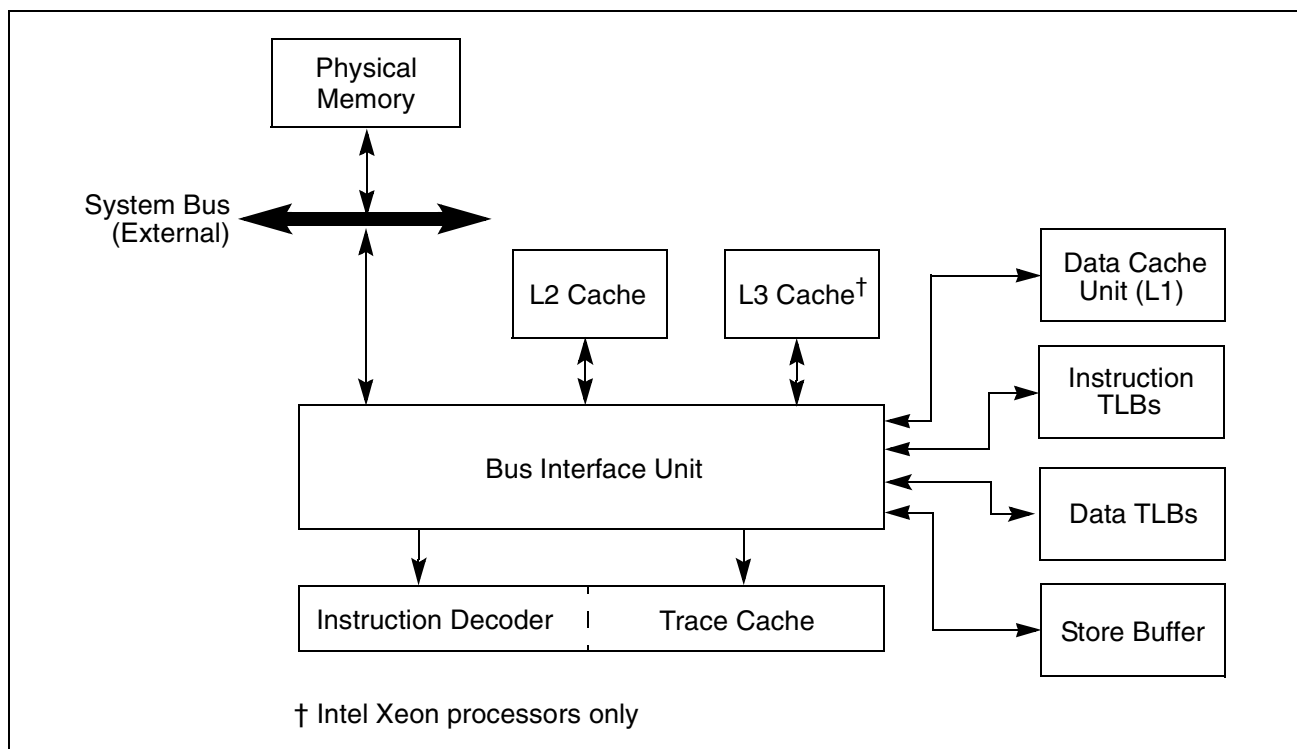


Figure 10-1. Cache Structure of the Pentium 4 and Intel Xeon Processors

Table 10-1. Characteristics of the Caches, TLBs, Store Buffer, and Write Combining Buffer in IA-32 processors

Cache or Buffer	Characteristics
Trace Cache [†]	- Pentium 4 and Intel Xeon processors: 12 Kμops, 8-way set associative. - Pentium M processor: not implemented. - P6 family and Pentium processors: not implemented.
L1 Instruction Cache	- Pentium 4 and Intel Xeon processors: not implemented. - Pentium M processor: 32-KByte, 8-way set associative. - P6 family and Pentium processors: 8- or 16-KByte, 4-way set associative, 32-byte cache line size; 2-way set associative for earlier Pentium processors.
L1 Data Cache	- Pentium 4 and Intel Xeon processors: 8-KByte, 4-way set associative, 64-byte cache line size. - Pentium M processor: 32-KByte, 8-way set associative, 64-byte cache line size. - P6 family processors: 16-KByte, 4-way set associative, 32-byte cache line size; 8-KBytes, 2-way set associative for earlier P6 family processors. - Pentium processors: 16-KByte, 4-way set associative, 32-byte cache line size; 8-KByte, 2-way set associative for earlier Pentium processors.
L2 Unified Cache	- Pentium 4 and Intel Xeon processors: 256-or 512-KByte, 8-way set associative, 64-byte cache line size, 128-byte sector size. - Pentium M processor: 1-MByte, 8-way set associative, 64-byte cache line size. - P6 family processors: 128-KByte, 256-KByte, 512-KByte, 1-MByte, or 2-MByte, 4-way set associative, 32-byte cache line size. - Pentium processor (external optional): System specific, typically 256- or 512-KByte, 4-way set associative, 32-byte cache line size.
L3 Unified Cache	- Intel Xeon processors: 512-KByte or 1-MByte, 8-way set associative, 64-byte cache line size, 128-byte sector size.
Instruction TLB (4-KByte Pages)	- Pentium 4 and Intel Xeon processors: 128 entries, 4-way set associative. - Pentium M processor: 128 entries, 4-way set associative. - P6 family processors: 32 entries, 4-way set associative. - Pentium processor: 32 entries, 4-way set associative; fully set associative for Pentium processors with MMX technology.
Data TLB (4-KByte Pages)	- Pentium 4 and Intel Xeon processors: 64 entries, fully set associative; shared with large page data TLBs. - Pentium M processor: 128 entries, 4-way set associative. - Pentium and P6 family processors: 64 entries, 4-way set associative; fully set. associative for Pentium processors with MMX technology.
Instruction TLB (Large Pages)	- Pentium 4 and Intel Xeon processors: large pages are fragmented. - Pentium M processor: 2 entries, fully associative. - P6 family processors: 2 entries, fully associative. - Pentium processor: Uses same TLB as used for 4-KByte pages.
Data TLB (Large Pages)	- Pentium 4 and Intel Xeon processors: 64 entries, fully set associative; shared with small page data TLBs. - Pentium M processor: 8 entries, fully associative. - P6 family processors: 8 entries, 4-way set associative. - Pentium processor: 8 entries, 4-way set associative; uses same TLB as used for 4-KByte pages in Pentium processors with MMX technology.

Table 10-1. Characteristics of the Caches, TLBs, Store Buffer, and Write Combining Buffer in IA-32 processors (Contd.)

Cache or Buffer	Characteristics
Store Buffer	- Pentium 4 and Intel Xeon processors: 24 entries. - Pentium M processor: 16 entries. - P6 family processors: 12 entries. - Pentium processor: 2 buffers, 1 entry each (Pentium processors with MMX technology have 4 buffers for 4 entries).
Write Combining (WC) Buffer	- Pentium 4 and Intel Xeon processors: 6 entries. - Pentium M processor: 6 entries. - P6 family processors: 4 entries.

NOTES:

† Introduced to the IA-32 architecture in the Pentium 4 and Intel Xeon processors.

The IA-32 processors implement four types of caches: the trace cache, the level 1 (L1) cache, the level 2 (L2) cache, and the level 3 (L3) cache (see Figure 10-1). The uses of these caches differs from the Pentium 4, Intel Xeon, and P6 family processors, as follows:

- Pentium 4 and Intel Xeon processors—The trace cache caches decoded instructions (μ ops) from the instruction decoder, and the L1 cache contains only data. The L2 and L3 caches are unified data and instruction caches that are located on the processor chip. (The L3 cache is only implemented on Intel Xeon processors.)
- P6 family processors—The L1 cache is divided into two sections: one dedicated to caching IA-32 architecture instructions (pre-decoded instructions) and one to caching data. The L2 cache is a unified data and instruction cache that is located on the processor chip. The P6 family processors do not implement a trace cache.
- Pentium processors—The L1 cache has the same structure as on the P6 family processors (and a trace cache is not implemented). The L2 cache is a unified data and instruction cache that is external to the processor chip on earlier Pentium processors and implemented on the processor chip in later Pentium processors. For Pentium processors where the L2 cache is external to the processor, access to the cache is through the system bus.

The cache lines for the L1 and L2 caches in the Pentium 4 and the L1, L2, and L3 caches in the Intel Xeon processors are 64 bytes wide. The processor always reads a cache line from system memory beginning on a 64-byte boundary. (A 64-byte aligned cache line begins at an address with its 6 least-significant bits clear.) A cache line can be filled from memory with a 8-transfer burst transaction. The caches do not support partially-filled cache lines, so caching even a single doubleword requires caching an entire line.

The L1 and L2 cache lines in the P6 family and Pentium processors are 32 bytes wide, with cache line reads from system memory beginning on a 32-byte boundary (5 least-significant bits of a memory address clear.) A cache line can be filled from memory with a 4-transfer burst transaction. Partially-filled cache lines are not supported.

The trace cache in the Pentium 4 and Intel Xeon processors is an integral part of the Intel NetBurst micro-architecture and is available in all execution modes: protected mode, system

management mode (SMM), and real-address mode. The L1, L2, and L3 caches are also available in all execution modes; however, use of them must be handled carefully in SMM (see Section 13.4.2., “SMRAM Caching”).

The TLBs store the most recently used page-directory and page-table entries. They speed up memory accesses when paging is enabled by reducing the number of memory accesses that are required to read the page tables stored in system memory. The TLBs are divided into four groups: instruction TLBs for 4-KByte pages, data TLBs for 4-KByte pages; instruction TLBs for large pages (2-MByte or 4-MByte pages), and data TLBs for large pages. The TLBs are normally active only in protected mode with paging enabled. When paging is disabled or the processor is in real-address mode, the TLBs maintain their contents until explicitly or implicitly flushed (see Section 10.9., “Invalidating the Translation Lookaside Buffers (TLBs)”).

The store buffer is associated with the processors instruction execution units. It allows writes to system memory and/or the internal caches to be saved and in some cases combined to optimize the processor’s bus accesses. The store buffer is always enabled in all execution modes.

The processor’s caches are for the most part transparent to software. When enabled, instructions and data flow through these caches without the need for explicit software control. However, knowledge of the behavior of these caches may be useful in optimizing software performance. For example, knowledge of cache dimensions and replacement algorithms gives an indication of how large of a data structure can be operated on at once without causing cache thrashing.

In multiprocessor systems, maintenance of cache consistency may, in rare circumstances, require intervention by system software. For these rare cases, the processor provides privileged cache control instructions for use in flushing caches and forcing memory ordering.

The Pentium III, Pentium 4, and Intel Xeon processors introduced several instructions that software can use to improve the performance of the L1, L2, and L3 caches, including the PREFETCHh and CLFLUSH instructions and the non-temporal move instructions (MOVNTI, MOVNTQ, MOVNTDQ, MOVNTPS, and MOVNTPD). The use of these instructions are discussed in Section 10.5.5., “Cache Management Instructions”.

10.2. CACHING TERMINOLOGY

The IA-32 architecture (beginning with the Pentium processor) uses the MESI (modified, exclusive, shared, invalid) cache protocol to maintain consistency with internal caches and caches in other processors (see Section 10.4., “Cache Control Protocol”).

When the processor recognizes that an operand being read from memory is cacheable, the processor reads an entire cache line into the appropriate cache (L1, L2, L3, or all). This operation is called a **cache line fill**. If the memory location containing that operand is still cached the next time the processor attempts to access the operand, the processor can read the operand from the cache instead of going back to memory. This operation is called a **cache hit**.

When the processor attempts to write an operand to a cacheable area of memory, it first checks if a cache line for that memory location exists in the cache. If a valid cache line does exist, the processor (depending on the write policy currently in force) can write the operand into the cache instead of writing it out to system memory. This operation is called a **write hit**. If a write misses

the cache (that is, a valid cache line is not present for area of memory being written to), the processor performs a cache line fill, write allocation. Then it writes the operand into the cache line and (depending on the write policy currently in force) can also write it out to memory. If the operand is to be written out to memory, it is written first into the store buffer, and then written from the store buffer to memory when the system bus is available. (Note that for the Pentium processor, write misses do not result in a cache line fill; they always result in a write to memory. For this processor, only read misses result in cache line fills.)

When operating in an MP system, IA-32 processors (beginning with the Intel486 processor) have the ability to **snoop** other processor's accesses to system memory and to their internal caches. They use this snooping ability to keep their internal caches consistent both with system memory and with the caches in other processors on the bus. For example, in the Pentium and P6 family processors, if through snooping one processor detects that another processor intends to write to a memory location that it currently has cached in **shared state**, the snooping processor will invalidate its cache line forcing it to perform a cache line fill the next time it accesses the same memory location.

Beginning with the P6 family processors, if a processor detects (through snooping) that another processor is trying to access a memory location that it has modified in its cache, but has not yet written back to system memory, the snooping processor will signal the other processor (by means of the HITM# signal) that the cache line is held in modified state and will preform an implicit write-back of the modified data. The implicit write-back is transferred directly to the initial requesting processor and snooped by the memory controller to assure that system memory has been updated. Here, the processor with the valid data may pass the data to the other processors without actually writing it to system memory; however, it is the responsibility of the memory controller to snoop this operation and update memory.

10.3. METHODS OF CACHING AVAILABLE

The processor allows any area of system memory to be cached in the L1, L2, and L3 caches. Within individual pages or regions of system memory, it also allows the type of caching (also called **memory type**) to be specified, using a variety of system flags and registers (see Section 10.5., "Cache Control"). The memory types currently defined for the IA-32 architecture are as follows. (Table 10-2 summarizes the memory types and gives their basic characteristics.)

- **Strong Uncacheable (UC)**—System memory locations are not cached. All reads and writes appear on the system bus and are executed in program order without reordering. No speculative memory accesses, page-table walks, or prefetches of speculated branch targets are made. This type of cache-control is useful for memory-mapped I/O devices. When used with normal RAM, it greatly reduces processor performance.

Table 10-2. Memory Types and Their Properties

Memory Type and Mnemonic	Cacheable	Writeback Cacheable	Allows Speculative Reads	Memory Ordering Model
Strong Uncacheable (UC)	No	No	No	Strong Ordering
Uncacheable (UC-)	No	No	No	Strong Ordering. Can only be selected through the PAT. Can be overridden by WC in MTRRs.
Write Combining (WC)	No	No	Yes	Weak Ordering. Available by programming MTRRs or by selecting it through the PAT.
Write Through (WT)	Yes	No	Yes	Speculative Processor Ordering
Write Back (WB)	Yes	Yes	Yes	Speculative Processor Ordering.
Write Protected (WP)	Yes for reads; no for writes	No	Yes	Speculative Processor Ordering. Available by programming MTRRs.

- **Uncacheable (UC-)**—Has same characteristics as the strong uncacheable (UC) memory type, except that this memory type can be overridden by programming the MTRRs for the WC memory type. This memory type is available in the Pentium 4, Intel Xeon, and Pentium III processors and can only be selected through the PAT.
- **Write Combining (WC)**—System memory locations are not cached (as with uncacheable memory) and coherency is not enforced by the processor's bus coherency protocol. Speculative reads are allowed. Writes may be delayed and combined in the write combining buffer (WC buffer) to reduce memory accesses. If the WC buffer is partially filled, the writes may be delayed until the next occurrence of a serializing event; such as, an SFENCE or MFENCE instruction, CUID execution, a read or write to uncached memory, an interrupt occurrence, or a LOCK instruction execution. This type of cache-control is appropriate for video frame buffers, where the order of writes is unimportant as long as the writes update memory so they can be seen on the graphics display. See Section 10.3.1., "Buffering of Write Combining Memory Locations", for more information about caching the WC memory type. This memory type is available in the Pentium Pro and Pentium II processors by programming the MTRRs or in the Pentium III, Pentium 4, and Intel Xeon processors by programming the MTRRs or by selecting it through the PAT.
- **Write-through (WT)**—Writes and reads to and from system memory are cached. Reads come from cache lines on cache hits; read misses cause cache fills. Speculative reads are allowed. All writes are written to a cache line (when possible) and through to system memory. When writing through to memory, invalid cache lines are never filled, and valid cache lines are either filled or invalidated. Write combining is allowed. This type of cache-control is appropriate for frame buffers or when there are devices on the system bus that access system memory, but do not perform snooping of memory accesses. It enforces coherency between caches in the processors and system memory.
- **Write-back (WB)**—Writes and reads to and from system memory are cached. Reads come from cache lines on cache hits; read misses cause cache fills. Speculative reads are

allowed. Write misses cause cache line fills (in the Pentium 4, Intel Xeon, and P6 family processors), and writes are performed entirely in the cache, when possible. Write combining is allowed. The write-back memory type reduces bus traffic by eliminating many unnecessary writes to system memory. Writes to a cache line are not immediately forwarded to system memory; instead, they are accumulated in the cache. The modified cache lines are written to system memory later, when a write-back operation is performed. Write-back operations are triggered when cache lines need to be deallocated, such as when new cache lines are being allocated in a cache that is already full. They also are triggered by the mechanisms used to maintain cache consistency. This type of cache-control provides the best performance, but it requires that all devices that access system memory on the system bus be able to snoop memory accesses to insure system memory and cache coherency.

- **Write protected (WP)**—Reads come from cache lines when possible, and read misses cause cache fills. Writes are propagated to the system bus and cause corresponding cache lines on all processors on the bus to be invalidated. Speculative reads are allowed. This memory type is available in the Pentium 4, Intel Xeon, and P6 family processors by programming the MTRRs (see Table 10-6).

Table 10-3 shows which of these caching methods are available in the Pentium, P6 Family, Pentium 4, and Intel Xeon processors.

Table 10-3. Methods of Caching Available in Pentium 4, Intel Xeon, P6 Family, and Pentium Processors

Memory Type	Pentium 4 and Intel Xeon Processors	P6 Family Processors	Pentium Processor
Strong Uncacheable (UC)	Yes	Yes	Yes
Uncacheable (UC-)	Yes	Yes*	No
Write Combining (WC)	Yes	Yes	No
Write Through (WT)	Yes	Yes	Yes
Write Back (WB)	Yes	Yes	Yes
Write Protected (WP)	Yes	Yes	No

NOTES:

* Introduced in the Pentium III processor; not available in the Pentium Pro or Pentium II processors

10.3.1. Buffering of Write Combining Memory Locations

Writes to the WC memory type are not cached in the typical sense of the word cached. They are retained in an internal write combining buffer (WC buffer) that is separate from the internal L1, L2, and L3 caches and the store buffer. The WC buffer is not snooped and thus does not provide data coherency. Buffering of writes to WC memory is done to allow software a small window of time to supply more modified data to the WC buffer while remaining as non-intrusive to software as possible. The buffering of writes to WC memory also causes data to be collapsed; that

is, multiple writes to the same memory location will leave the last data written in the location and the other writes will be lost.

The size and structure of the WC buffer is not architecturally defined. For the Pentium 4 and Intel Xeon processors, the WC buffer is made up of several 64-byte WC buffers. For the P6 family processors, the WC buffer is made up of several 32-byte WC buffers.

When software begins writing to WC memory, the processor begins filling the WC buffers one at a time. When one or more WC buffers has been filled, the processor has the option of evicting the buffers to system memory. The protocol for evicting the WC buffers is implementation dependent and should not be relied on by software for system memory coherency. When using the WC memory type, software **must** be sensitive to the fact that the writing of data to system memory is being delayed and **must** deliberately empty the WC buffers when system memory coherency is required.

Once the processor has started to evict data from the WC buffer into system memory, it will make a bus-transaction style decision based on how much of the buffer contains valid data. If the buffer is full (for example, all bytes are valid) the processor will execute a burst-write transaction on the bus that will result in all 32 bytes (P6 family processors) or 64 bytes (Pentium 4 and Intel Xeon processor) being transmitted on the data bus in a single burst transaction. If one or more of the WC buffer's bytes are invalid (for example, have not been written by software) then the processor will transmit the data to memory using "partial write" transactions (one chunk at a time, where a "chunk" is 8 bytes).

This will result in a maximum of 4 partial write transactions (for P6 family processors) or 8 partial write transactions (for the Pentium 4 and Intel Xeon processors) for one WC buffer of data sent to memory.

The WC memory type is weakly ordered by definition. Once the eviction of a WC buffer has started, the data is subject to the weak ordering semantics of its definition. Ordering is not maintained between the successive allocation/deallocation of WC buffers (for example, writes to WC buffer 1 followed by writes to WC buffer 2 may appear as buffer 2 followed by buffer 1 on the system bus). When a WC buffer is evicted to memory as partial writes there is no guaranteed ordering between successive partial writes (for example, a partial write for chunk 2 may appear on the bus before the partial write for chunk 1 or vice versa). The only elements of WC propagation to the system bus that are guaranteed are those provided by transaction atomicity. For example, with a P6 family processor, a completely full WC buffer will always be propagated as a single 32-bit burst transaction using any chunk order. In a WC buffer eviction where the data will be evicted as partials, all data contained in the same chunk (0 mod 8 aligned) will be propagated simultaneously. Likewise, with a Pentium 4 or Intel Xeon processor, a full WC buffer will always be propagated as a single burst transactions, using any chunk order within a transaction. For partial buffer propagations, all data contained in the same chunk will be propagated simultaneously.

10.3.2. Choosing a Memory Type

The simplest system memory model does not use memory-mapped I/O with read or write side effects, does not include a frame buffer, and uses the write-back memory type for all memory. An I/O agent can perform direct memory access (DMA) to write-back memory and the cache protocol maintains cache coherency.

A system can use strong uncacheable memory for other memory-mapped I/O, and should always use strong uncacheable memory for memory-mapped I/O with read side effects.

Dual-ported memory can be considered a write side effect, making relatively prompt writes desirable, because those writes cannot be observed at the other port until they reach the memory agent. A system can use strong uncacheable, uncacheable, write-through, or write-combining memory for frame buffers or dual-ported memory that contains pixel values displayed on a screen. Frame buffer memory is typically large (a few megabytes) and is usually written more than it is read by the processor. Using strong uncacheable memory for a frame buffer generates very large amounts of bus traffic, because operations on the entire buffer are implemented using partial writes rather than line writes. Using write-through memory for a frame buffer can displace almost all other useful cached lines in the processor's L2 and L3 caches and L1 data cache. Therefore, systems should use write-combining memory for frame buffers whenever possible.

Software can use page-level cache control, to assign appropriate effective memory types when software will not access data structures in ways that benefit from write-back caching. For example, software may read a large data structure once and not access the structure again until the structure is rewritten by another agent. Such a large data structure should be marked as uncacheable, or reading it will evict cached lines that the processor will be referencing again.

A similar example would be a write-only data structure that is written to (to export the data to another agent), but never read by software. Such a structure can be marked as uncacheable, because software never reads the values that it writes (though as uncacheable memory, it will be written using partial writes, while as write-back memory, it will be written using line writes, which may not occur until the other agent reads the structure and triggers implicit write-backs).

On the Pentium III, Pentium 4, and Intel Xeon processors, new instructions are provided that give software greater control over the caching, prefetching, and the write-back characteristics of data. These instructions allow software to use weakly ordered or processor ordered memory types to improve processor performance, but when necessary to force strong ordering on memory reads and/or writes. They also allow software greater control over the caching of data. (For a description of these instructions and their intended use, see Section 10.5.5., "Cache Management Instructions").

10.4. CACHE CONTROL PROTOCOL

The following section describes the cache control protocol currently defined for the IA-32 architecture. This protocol is used by the Pentium 4, Intel Xeon, P6 family, and Pentium processors.

In the L1 data cache and in the L2 and L3 unified caches, the MESI (modified, exclusive, shared, invalid) cache protocol maintains consistency with caches of other processors. The L1 data

cache and the L2 and L3 unified caches have two MESI status flags per cache line. Each line can thus be marked as being in one of the states defined in Table 10-4. In general, the operation of the MESI protocol is transparent to programs.

Table 10-4. MESI Cache Line States

Cache Line State	M (Modified)	E (Exclusive)	S (Shared)	I (Invalid)
This cache line is valid?	Yes	Yes	Yes	No
The memory copy is...	...out of date	...valid	...valid	—
Copies exist in caches of other processors?	No	No	Maybe	Maybe
A write to this line ...	...does not go to the system bus.	...does not go to the system bus.	...causes the processor to gain exclusive ownership of the line.	...goes directly to the system bus.

The L1 instruction cache in P6 family processors implements only the “SI” part of the MESI protocol, because the instruction cache is not writable. The instruction cache monitors changes in the data cache to maintain consistency between the caches when instructions are modified. See Section 10.6., “Self-Modifying Code”, for more information on the implications of caching instructions.

10.5. CACHE CONTROL

The IA-32 architecture provides a variety of mechanisms for controlling the caching of data and instructions and for controlling the ordering of reads and writes between the processor, the caches, and memory. These mechanisms can be divided into two groups:

- **Cache control registers and bits.** The IA-32 architecture defines several dedicated registers and various bits within control registers and page- and directory-table entries that control the caching system memory locations in the L1, L2, and L3 caches. These mechanisms control the caching of virtual memory pages and of regions of physical memory.
- **Cache Control and Memory Ordering Instructions.** The IA-32 architecture provides several instructions that control the caching of data, the ordering of memory reads and writes, and the prefetching of data. These instructions allow software to control the caching of specific data structures, to control memory coherency for specific locations in memory, and to force strong memory ordering at specific locations in a program.

The following sections describe these two groups of cache control mechanisms.

10.5.1. Cache Control Registers and Bits

The current IA-32 architecture provides the following cache-control registers and bits for use in enabling and/or restricting caching to various pages or regions in memory (see Figure 10-2):

- CD flag, bit 30 of control register CR0—Controls caching of system memory locations (see Section 2.5., “Control Registers”). If the CD flag is clear, caching is enabled for the whole of system memory, but may be restricted for individual pages or regions of memory by other cache-control mechanisms. When the CD flag is set, caching is restricted in the processor’s caches (cache hierarchy) for the Pentium 4, Intel Xeon, and P6 family processors and prevented for the Pentium processor (see note below). With the CD flag set, however, the caches will still respond to snoop traffic. Caches should be explicitly flushed to insure memory coherency. For highest processor performance, both the CD and the NW flags in control register CR0 should be cleared. Table 10-5 shows the interaction of the CD and NW flags.

NOTE

The effect of setting the CD flag is somewhat different for the Pentium 4, Intel Xeon, and P6 family processors than for the Pentium processor (see Table 10-5). To insure memory coherency after the CD flag is set, the caches should be explicitly flushed (see Section 10.5.3., “Preventing Caching”). Setting the CD flag for the Pentium 4, Intel Xeon, and P6 family processors modifies cache line fill and update behaviour. Also for the Pentium 4, Intel Xeon, and P6 family processors, setting the CD flag does not force strict ordering of memory accesses unless the MTRRs are disabled and/or all memory is referenced as uncached (see Section 7.2.4., “Strengthening or Weakening the Memory Ordering Model”).

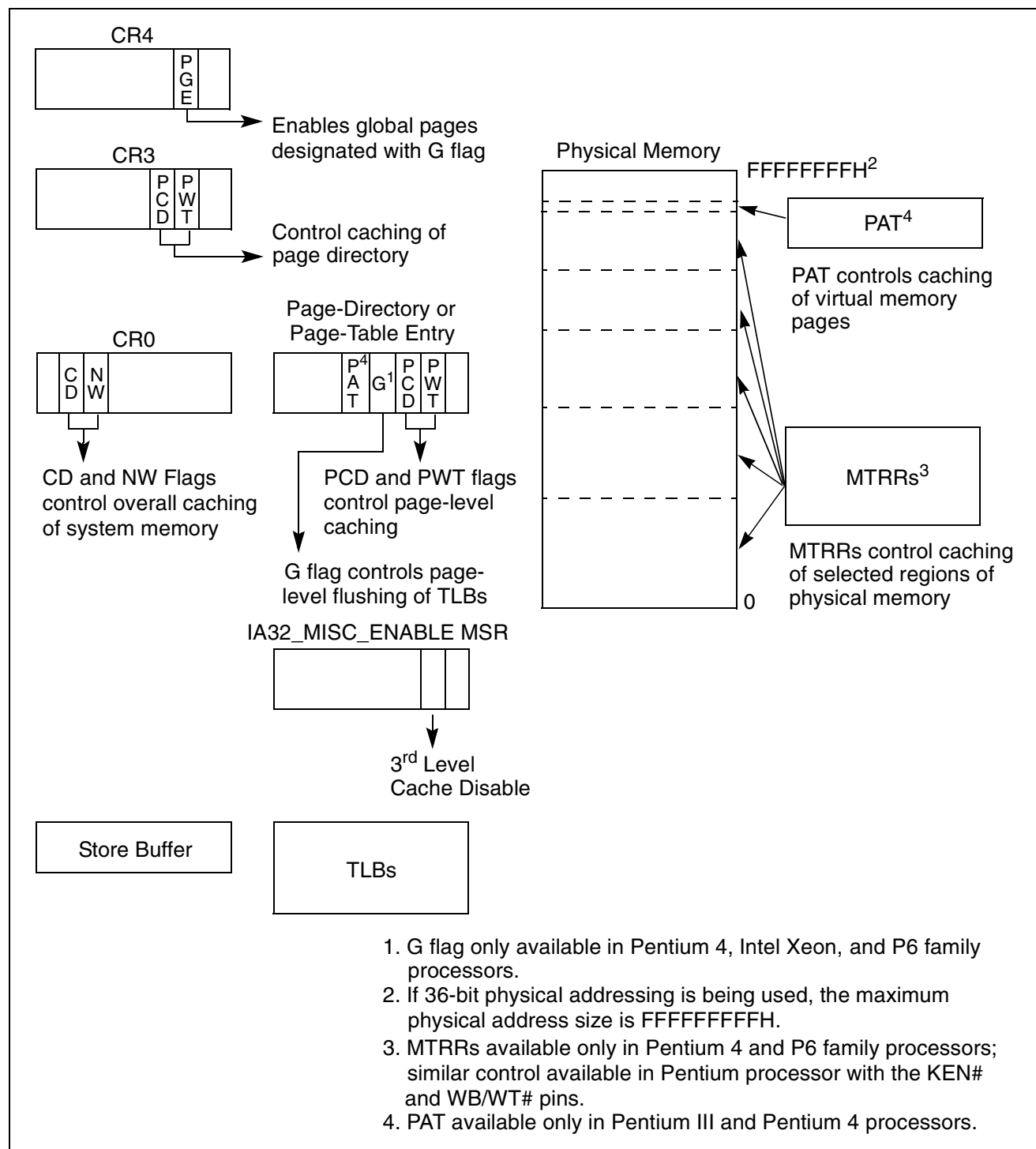


Figure 10-2. Cache-Control Registers and Bits Available in IA-32 Processors

Table 10-5. Cache Operating Modes

CD	NW	Caching and Read/Write Policy	L1	L2/L3 ¹
0	0	Normal Cache Mode. Highest performance cache operation. - Read hits access the cache; read misses may cause replacement. - Write hits update the cache. - Only writes to shared lines and write misses update system memory. - Write misses cause cache line fills. - Write hits can change shared lines to modified under control of the MTRRs and with associated read invalidation cycle. - (Pentium processor only.) Write misses do not cause cache line fills. - (Pentium processor only.) Write hits can change shared lines to exclusive under control of WB/WT#. - Invalidation is allowed. - External snoop traffic is supported.	Yes Yes Yes Yes Yes Yes Yes Yes	Yes Yes Yes Yes Yes Yes Yes Yes
0	1	Invalid setting. Generates a general-protection exception (#GP) with an error code of 0.	NA	NA
1	0	No-fill Cache Mode. Memory coherency is maintained. - (Pentium 4 and Intel Xeon processors.) State of processor after a power up or reset. - Read hits access the cache; read misses do not cause replacement (see Pentium 4 and Intel Xeon processors reference below). - Write hits update the cache. - Only writes to shared lines and write misses update system memory. - Write misses access memory. - Write hits can change shared lines to exclusive under control of the MTRRs and with associated read invalidation cycle. - (Pentium processor only.) Write hits can change shared lines to exclusive under control of the WB/WT#. - (Pentium 4, Intel Xeon, and P6 family processors only.) Strict memory ordering is not enforced unless the MTRRs are disabled and/or all memory is referenced as uncached (see Section 7.2.4., "Strengthening or Weakening the Memory Ordering Model"). - Invalidation is allowed. - External snoop traffic is supported. - (Pentium 4 and Intel Xeon processors) Allows cache line fills and replacements unless the accessed memory is mapped as uncached.	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes	Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes Yes
1	1	Memory coherency is not maintained.² - (P6 family and Pentium processors.) State of the processor after a power up or reset. - Read hits access the cache; read misses do not cause replacement. - Write hits update the cache and change exclusive lines to modified. - Shared lines remain shared after write hit. - Write misses access memory. - Invalidation is inhibited when snooping; but is allowed with INVD and WBINVD instructions. - External snoop traffic is supported.	Yes Yes Yes Yes Yes Yes Yes No	Yes Yes Yes Yes Yes Yes Yes Yes

NOTE:

1. The L2/L3 column in this table is definitive for the Pentium 4, Intel Xeon, and P6 family processors. It is intended to represent what could be implemented in a system based on a Pentium processor with an external, platform specific, write-back L2 cache.
2. The Pentium 4 and Intel Xeon processors do not support this mode; setting the CD and NW bits to 1 selects the no-fill cache mode.

- NW flag, bit 29 of control register CR0—Controls the write policy for system memory locations (see Section 2.5., “Control Registers”). If the NW and CD flags are clear, write-back is enabled for the whole of system memory, but may be restricted for individual pages or regions of memory by other cache-control mechanisms. Table 10-5 shows how the other combinations of CD and NW flags affects caching.

NOTE

For the Pentium 4 and Intel Xeon processors, the NW flag is a don't care flag; that is, when the CD flag is set, the processor uses the no-fill cache mode, regardless of the setting of the NW flag.

For the Pentium processor, when the L1 cache is disabled (the CD and NW flags in control register CR0 are set), external snoops are accepted in DP (dual-processor) systems and inhibited in uniprocessor systems. When snoops are inhibited, address parity is not checked and APCHK# is not asserted for a corrupt address; however, when snoops are accepted, address parity is checked and APCHK# is asserted for corrupt addresses.

- PCD flag in the page-directory and page-table entries—Controls caching for individual page tables and pages, respectively (see Section 3.7.6., “Page-Directory and Page-Table Entries”). This flag only has effect when paging is enabled and the CD flag in control register CR0 is clear. The PCD flag enables caching of the page table or page when clear and prevents caching when set.
- PWT flag in the page-directory and page-table entries—Controls the write policy for individual page tables and pages, respectively (see Section 3.7.6., “Page-Directory and Page-Table Entries”). This flag only has effect when paging is enabled and the NW flag in control register CR0 is clear. The PWT flag enables write-back caching of the page table or page when clear and write-through caching when set.
- PCD and PWT flags in control register CR3. Control the global caching and write policy for the page directory (see Section 2.5., “Control Registers”). The PCD flag enables caching of the page directory when clear and prevents caching when set. The PWT flag enables write-back caching of the page directory when clear and write-through caching when set. These flags do not affect the caching and write policy for individual page tables. These flags only have effect when paging is enabled and the CD flag in control register CR0 is clear.
- G (global) flag in the page-directory and page-table entries (introduced to the IA-32 architecture in the P6 family processors)—Controls the flushing of TLB entries for individual pages. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for more information about this flag.
- PGE (page global enable) flag in control register CR4—Enables the establishment of global pages with the G flag. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for more information about this flag.

- Memory type range registers (MTRRs) (introduced in the P6 family processors)—Control the type of caching used in specific regions of physical memory. Any of the caching types described in Section 10.3., “Methods of Caching Available”, can be selected. See Section 10.11., “Memory Type Range Registers (MTRRs)”, for a detailed description of the MTRRs.
- Page Attribute Table (PAT) MSR (Introduced in the Pentium III processor)—Extends the memory typing capabilities of the processor to permit memory types to be assigned on a page-by-page basis (see Section 10.12., “Page Attribute Table (PAT)”).
- Third-Level Cache Disable flag, bit 6 of the IA32_MISC_ENABLE MSR (Introduced in the Intel Xeon processors)—Allows the L3 cache to be disabled and enabled, independently of the L1 and L2 caches.
- KEN# and WB/WT# pins (Pentium processor)—Allow external hardware to control the caching method used for specific areas of memory. They perform similar (but not identical) functions to the MTRRs in the P6 family processors.
- PCD and PWT pins (Pentium processor)—These pins (which are associated with the PCD and PWT flags in control register CR3 and in the page-directory and page-table entries) permit caching in an external L2 cache to be controlled on a page-by-page basis, consistent with the control exercised on the L1 cache of these processors. The Pentium 4, Intel Xeon, and P6 family processors do not provide these pins because the L2 cache is internal to the chip package.

10.5.2. Precedence of Cache Controls

For the cache control flags and MTRRs operate hierarchically for restricting caching. That is, if the CD flag is set, caching is prevented globally (see Table 10-5). If the CD flag is clear, the page-level cache control flags and/or the MTRRs can be used to restrict caching. If there is an overlap of page-level and MTRR caching controls, the mechanism that prevents caching has precedence. For example, if an MTRR makes a region of system memory uncachable, a page-level caching control cannot be used to enable caching for a page in that region. The converse is also true; that is, if a page-level caching control designates a page as uncachable, an MTRR cannot be used to make the page cacheable.

In cases where there is an overlap in the assignment of the write-back and write-through caching policies to a page and a region of memory, the write-through policy takes precedence. The write-combining policy (which can only be assigned through an MTRR or the PAT) takes precedence over either write-through or write-back.

The selection of memory types at the page level varies depending on whether PAT is being used to select memory types for pages, as described in the following sections.

Third-level cache disable flag (bit 6 of the IA32_MISC_ENABLE MSR) takes precedence over the CD flag, MTRRs, and PAT for the L3 cache. That is, when the third-level cache disable flag is set (cache disabled), the other cache controls have no effect on the L3 cache; when the flag is clear (enabled), the cache controls have the same effect on the L3 cache as they have on the L1 and L2 caches.

10.5.2.1. SELECTING MEMORY TYPES FOR PENTIUM PRO AND PENTIUM II PROCESSORS

The Pentium Pro and Pentium II processors do not support the PAT. Here, the effective memory type for a page is selected with the MTRRs and the PCD and PWT bits in the page-table or page-directory entry for the page. Table 10-6 describes the mapping of MTRR memory types and page-level caching attributes to effective memory types, when normal caching is in effect (the CD and NW flags in control register CR0 are clear). Combinations that appear in gray are implementation-defined for the Pentium Pro and Pentium II processors. System designers are encouraged to avoid these implementation-defined combinations.

Table 10-6. Effective Page-Level Memory Type for Pentium Pro and Pentium II Processors*

MTRR Memory Type	PCD Value	PWT Value	Effective Memory Type
UC	X	X	UC
WC	0	0	WC
	0	1	WC
	1	0	WC
	1	1	UC
WT	0	X	WT
	1	X	UC
WP	0	0	WP
	0	1	WP
	1	0	WC
	1	1	UC
WB	0	0	WB
	0	1	WT
	1	X	UC

Note:

* These effective memory types also apply to the Pentium 4, Intel Xeon, and Pentium III processors when the PAT bit is not used (set to 0) in page-table and page-directory entries.

When normal caching is in effect, the effective memory type shown in Table 10-6 is determined using the following rules:

1. If the PCD and PWT attributes for the page are both 0, then the effective memory type is identical to the MTRR-defined memory type.
2. If the PCD flag is set, then the effective memory type is UC.
3. If the PCD flag is clear and the PWT flag is set, the effective memory type is WT for the WB memory type and the MTRR-defined memory type for all other memory types.

4. Setting the PCD and PWT flags to opposite values is considered model-specific for the WP and WC memory types and architecturally-defined for the WB, WT, and UC memory types.

10.5.2.2. SELECTING MEMORY TYPES FOR PENTIUM 4, INTEL XEON, AND PENTIUM III PROCESSORS

The Pentium 4, Intel Xeon, and Pentium III processors use the PAT to select effective page-level memory types. Here, a memory type for a page is selected by the MTRRs and the value in a PAT entry that is selected with the PAT, PCD and PWT bits in a page-table or page-directory entry (see Section 10.12.3., “Selecting a Memory Type from the PAT”). Table 10-7 describes the mapping of MTRR memory types and PAT entry types to effective memory types, when normal caching is in effect (the CD and NW flags in control register CR0 are clear). The combinations shown in gray are implementation-defined for the Pentium 4, Intel Xeon, and Pentium III processors. System designers are encouraged to avoid the implementation-defined combinations.

Table 10-7. Effective Page-Level Memory Types for Pentium III, Pentium 4, and Intel Xeon Processors

MTRR Memory Type	PAT Entry Value	Effective Memory Type
UC	UC	UC ¹
	UC-	UC ¹
	WC	WC
	WT	UC ¹
	WB	UC ¹
	WP	UC ¹
WC	UC	UC ²
	UC-	WC
	WC	WC
	WT	Undefined
	WB	WC
	WP	Undefined
WT	UC	UC ²
	UC-	UC ²
	WC	WC
	WT	WT
	WB	WT
	WP	Undefined

Table 10-7. Effective Page-Level Memory Types for Pentium III, Pentium 4, and Intel Xeon Processors (Contd.)

WB	UC	UC ²
	UC-	UC ²
	WC	WC
	WT	WT
	WB	WB
	WP	WP
WP	UC	UC ²
	UC-	Undefined
	WC	WC
	WT	Undefined
	WB	WP
	WP	WP

NOTES:

1. The UC attribute comes from the MTRRs and the processors are not required to snoop their caches since the data could never have been cached. This attribute is preferred for performance reasons.
2. The UC attribute came from the page-table or page-directory entry and processors are required to check their caches because the data may be cached due to page aliasing, which is not recommended.

10.5.2.3. WRITING VALUES ACROSS PAGES WITH DIFFERENT MEMORY TYPES

If two adjoining pages in memory have different memory types, and a word or longer operand is written to a memory location that crosses the page boundary between those two pages, the operand might be written to memory twice. This action does not present a problem for writes to actual memory; however, if a device is mapped the memory space assigned to the pages, the device might malfunction.

10.5.3. Preventing Caching

To disable the L1, L2, and L3 caches after they have been enabled and have received cache fills, perform the following steps:

1. Enter the no-fill cache mode. (Set the CD flag in control register CR0 to 1 and the NW flag to 0.
2. Flush all caches using the WBINVD instruction.
3. Disable the MTRRs and set the default memory type to uncached or set all MTRRs for the uncached memory type (see the discussion of the discussion of the TYPE field and the E flag in Section 10.11.2.1., “IA32_MTRR_DEF_TYPE MSR”).

The caches must be flushed (step 2) after the CD flag is set to insure system memory coherency. If the caches are not flushed, cache hits on reads will still occur and data will be read from valid cache lines.

NOTE

Setting the CD flag in control register CR0 modifies the processor's caching behaviour as indicated in Table 10-5, but it does not force the effective memory type for all physical memory to be UC nor does it force strict memory ordering. To force the UC memory type and strict memory ordering on all of physical memory, either the MTRRs must all be programmed for the UC memory type or they must be disabled.

For the Pentium 4 and Intel Xeon processors, after the sequence of steps given above has been executed, the cache lines containing the code between the end of the WBINVD instruction and before the MTRRS have actually been disabled may be retained in the cache hierarchy. Here, to remove code from the cache completely, a second WBINVD instruction must be executed after the MTRRs have been disabled.

10.5.4. Disabling and Enabling the L3 Cache

Third-level cache disable flag (bit 6 of the IA32_MISC_ENABLE MSR) allows the L3 cache to be disabled and enabled, independently of the L1 and L2 caches. Prior to using this control to disable or enable the L3 cache, software should disable and flush all the processor caches, as described earlier in Section 10.5.3., "Preventing Caching", to prevent loss of information stored in the L3 cache. After the L3 cache has been disabled or enabled, caching for the whole processor can be restored.

10.5.5. Cache Management Instructions

The IA-32 architecture provides several instructions for managing the L1, L2, and L3 caches. The INVD, WBINVD, and PREFETCH instructions are system instructions that operate on the L1, L2, and L3 caches as a whole. The PREFETCH and CLFLUSH instructions and the non-temporal move instructions (MOVNTI, MOVNTQ, MOVNTDQ, MOVNTPS, and MOVNTPD), which were introduced in the SSE and SSE2 extensions, offer more granular control over caching.

The INVD and WBINVD instructions are used to invalidate the contents of the L1, L2, and L3 caches. The INVD instruction invalidates all internal cache entries, then generates a special-function bus cycle that indicates that external caches also should be invalidated. The INVD instruction should be used with care. It does not force a write-back of modified cache lines; therefore, data stored in the caches and not written back to system memory will be lost. Unless there is a specific requirement or benefit to invalidating the caches without writing back the modified lines (such as, during testing or fault recovery where cache coherency with main memory is not a concern), software should use the WBINVD instruction.

The WBINVD instruction first writes back any modified lines in all the internal caches, then invalidates the contents of both the L1, L2, and L3 caches. It ensures that cache coherency with main memory is maintained regardless of the write policy in effect (that is, write-through or write-back). Following this operation, the WBINVD instruction generates one (P6 family processors) or two (Pentium and Intel486 processors) special-function bus cycles to indicate to external cache controllers that write-back of modified data followed by invalidation of external caches should occur.

The PREFETCH h instructions allow a program to suggest to the processor that a cache line from a specified location in system memory be prefetched into the cache hierarchy (see Section 10.8., “Explicit Caching”).

The CLFLUSH instruction allow selected cache lines to be flushed from memory. This instruction give a program the ability to explicitly free up cache space, when it is known that cached section of system memory will not be accessed in the near future.

The non-temporal move instructions (MOVNTI, MOVNTQ, MOVNTDQ, MOVNTPS, and MOVNTPD) allow data to be moved from the processor’s registers directly into system memory without being also written into the L1, L2, and/or L3 caches. These instructions can be used to prevent cache pollution when operating on data that is going to be modified only once before being stored back into system memory. These instructions operate on data in the general-purpose, MMX, and XMM registers.

10.5.6. L1 Data Cache Context Mode

First-level data cache context mode is a feature of Intel Pentium 4 processor with Hyper-Threading Technology. When the Context ID feature flag (ECX[10]) is set after executing the CPUID instruction with EAX = 1, then the processor supports setting of the L1 Data Cache Context Mode using IA32_MISC_ENABLE MSR. The selectable modes are Adaptive Mode (default) and Shared Mode.

The BIOS is responsible for configuring the L1 data cache context mode.

10.5.6.1. ADAPTIVE MODE

In adaptive mode, memory accessed using the page directory is mapped identically across logical processors sharing an L1 data cache. Since mapping is identical, the targeted cache appears as full size to each logical processor (instead of being competitively shared).

If the CR3 register is configured the same for logical processors that share an L1 data cache, the cache will take advantage of the adaptive mode feature. If the L1 data cache is configured for adaptive mode, but CR3 registers are not programmed identical across the logical processors that share the same L1 data cache, then each logical processor will compete for L1 data cache resources. In this case, the cache does not look full size to any of the logical processors.

10.5.6.2. SHARED MODE

In shared mode, the L1 data cache is competitively shared. This is true even when CR3 registers are configured identically across logical processors that share the same L1 data cache.

10.6. SELF-MODIFYING CODE

A write to a memory location in a code segment that is currently cached in the processor causes the associated cache line (or lines) to be invalidated. This check is based on the physical address of the instruction. In addition, the P6 family and Pentium processors check whether a write to a code segment may modify an instruction that has been prefetched for execution. If the write affects a prefetched instruction, the prefetch queue is invalidated. This latter check is based on the linear address of the instruction. For the Pentium 4 and Intel Xeon processors, a write or a snoop of an instruction in a code segment, where the target instruction is already decoded and resident in the trace cache, invalidates the entire trace cache. The latter behavior means that programs that self-modify code can cause severe degradation of performance when run on the Pentium 4 and Intel Xeon processors.

In practice, the check on linear addresses should not create compatibility problems among IA-32 processors. Applications that include self-modifying code use the same linear address for modifying and fetching the instruction. Systems software, such as a debugger, that might possibly modify an instruction using a different linear address than that used to fetch the instruction, will execute a serializing operation, such as a CPUID instruction, before the modified instruction is executed, which will automatically resynchronize the instruction cache and prefetch queue. (See Section 7.1.3., “Handling Self- and Cross-Modifying Code”, for more information about the use of self-modifying code.)

For Intel486 processors, a write to an instruction in the cache will modify it in both the cache and memory, but if the instruction was prefetched before the write, the old version of the instruction could be the one executed. To prevent the old instruction from being executed, flush the instruction prefetch unit by coding a jump instruction immediately after any write that modifies an instruction.

10.7. IMPLICIT CACHING (PENTIUM 4, INTEL XEON, AND P6 FAMILY PROCESSORS)

Implicit caching occurs when a memory element is made potentially cacheable, although the element may never have been accessed in the normal von Neumann sequence. Implicit caching occurs on the Pentium 4, Intel Xeon, and P6 family processors due to aggressive prefetching, branch prediction, and TLB miss handling. Implicit caching is an extension of the behavior of existing Intel386, Intel486, and Pentium processor systems, since software running on these processor families also has not been able to deterministically predict the behavior of instruction prefetch.

To avoid problems related to implicit caching, the operating system must explicitly invalidate the cache when changes are made to cacheable data that the cache coherency mechanism does not automatically handle. This includes writes to dual-ported or physically aliased memory

boards that are not detected by the snooping mechanisms of the processor, and changes to page-table entries in memory.

The code in Example 10-1 shows the effect of implicit caching on page-table entries. The linear address F000H points to physical location B000H (the page-table entry for F000H contains the value B000H), and the page-table entry for linear address F000 is PTE_F000.

Example 10-1. Effect of Implicit Caching on Page-Table Entries

```
mov EAX, CR3          ; Invalidate the TLB
mov CR3, EAX          ; by copying CR3 to itself
mov PTE_F000, A000H; Change F000H to point to A000H
mov EBX, [F000H];
```

Because of speculative execution in the Pentium 4, Intel Xeon, and P6 family processors, the last MOV instruction performed would place the value at physical location B000H into EBX, rather than the value at the new physical address A000H. This situation is remedied by placing a TLB invalidation between the load and the store.

10.8. EXPLICIT CACHING

The Pentium III processor introduced four new instructions, the PREFETCHh instructions, that provide software with explicit control over the caching of data. These instructions provide “hints” to the processor that the data requested by a PREFETCHh instruction should be read into cache hierarchy now or as soon as possible, in anticipation of its use. The instructions provide different variations of the hint that allow selection of the cache level into which data will be read.

The PREFETCHh instructions can help reduce the long latency typically associated with reading data from memory and thus help prevent processor “stalls.” However, these instructions should be used judiciously. Overuse can lead to resource conflicts and hence reduce the performance of an application. Also, these instructions should only be used to prefetch data from memory; they should not be used to prefetch instructions. For more detailed information on the proper use of the prefetch instruction, refer to Chapter 6, “*Optimizing Cache Usage for the Intel Pentium 4 Processors*”, in the *Pentium 4 Processor Optimization Reference Manual* (see Section 1.4., “Related Literature”, for the document order number).

10.9. INVALIDATING THE TRANSLATION LOOKASIDE BUFFERS (TLBS)

The processor updates its address translation caches (TLBs) transparently to software. Several mechanisms are available, however, that allow software and hardware to invalidate the TLBs either explicitly or as a side effect of another operation.

The INVLPG instruction invalidates the TLB for a specific page. This instruction is the most efficient in cases where software only needs to invalidate a specific page, because it improves

performance over invalidating the whole TLB. This instruction is not affected by the state of the G flag in a page-directory or page-table entry.

The following operations invalidate all TLB entries except global entries. (A global entry is one for which the G (global) flag is set in its corresponding page-directory or page-table entry. The global flag was introduced into the IA-32 architecture in the P6 family processors, see Section 10.5., “Cache Control”.)

- Writing to control register CR3.
- A task switch that changes control register CR3.

The following operations invalidate all TLB entries, irrespective of the setting of the G flag:

- Asserting or de-asserting the FLUSH# pin.
- (Pentium 4, Intel Xeon, and P6 family processors only.) Writing to an MTRR (with a WRMSR instruction).
- Writing to control register CR0 to modify the PG or PE flag.
- (Pentium 4, Intel Xeon, and P6 family processors only.) Writing to control register CR4 to modify the PSE, PGE, or PAE flag.

See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for additional information about the TLBs.

10.10. STORE BUFFER

IA-32 processors temporarily store each write (store) to memory in a store buffer. The store buffer improves processor performance by allowing the processor to continue executing instructions without having to wait until a write to memory and/or to a cache is complete. It also allows writes to be delayed for more efficient use of memory-access bus cycles.

In general, the existence of the store buffer is transparent to software, even in systems that use multiple processors. The processor ensures that write operations are always carried out in program order. It also insures that the contents of the store buffer are always drained to memory in the following situations:

- When an exception or interrupt is generated.
- (Pentium 4, Intel Xeon, and P6 family processors only.) When a serializing instruction is executed.
- When an I/O instruction is executed.
- When a LOCK operation is performed.
- (Pentium 4, Intel Xeon, and P6 family processors only.) When a BINIT operation is performed.

- (Pentium III, Pentium 4, and Intel Xeon processors only.) When using an SFENCE instruction to order stores.
- (Pentium 4 and Intel Xeon processors only.) When using an MFENCE instruction to order stores.

The discussion of write ordering in Section 7.2., “Memory Ordering”, gives a detailed description of the operation of the store buffer.

10.11. MEMORY TYPE RANGE REGISTERS (MTRRS)

The following section pertains only to the Pentium 4, Intel Xeon, and P6 family processors.

The memory type range registers (MTRRs) provide a mechanism for associating the memory types (see Section 10.3., “Methods of Caching Available”) with physical-address ranges in system memory. They allow the processor to optimize operations for different types of memory such as RAM, ROM, frame-buffer memory, and memory-mapped I/O devices. They also simplify system hardware design by eliminating the memory control pins used for this function on earlier IA-32 processors and the external logic needed to drive them.

The MTRR mechanism allows up to 96 memory ranges to be defined in physical memory, and it defines a set of model-specific registers (MSRs) for specifying the type of memory that is contained in each range. Table 10-8 shows the memory types that can be specified and their properties; Figure 10-3 shows the mapping of physical memory with MTRRs. See Section 10.3., “Methods of Caching Available”, for a more detailed description of each memory type.

Following a hardware reset, a Pentium 4, Intel Xeon, or P6 family processor disables all the fixed and variable MTRRs, which in effect makes all of physical memory uncachable. Initialization software should then set the MTRRs to a specific, system-defined memory map. Typically, the BIOS (basic input/output system) software configures the MTRRs. The operating system or executive is then free to modify the memory map using the normal page-level cacheability attributes.

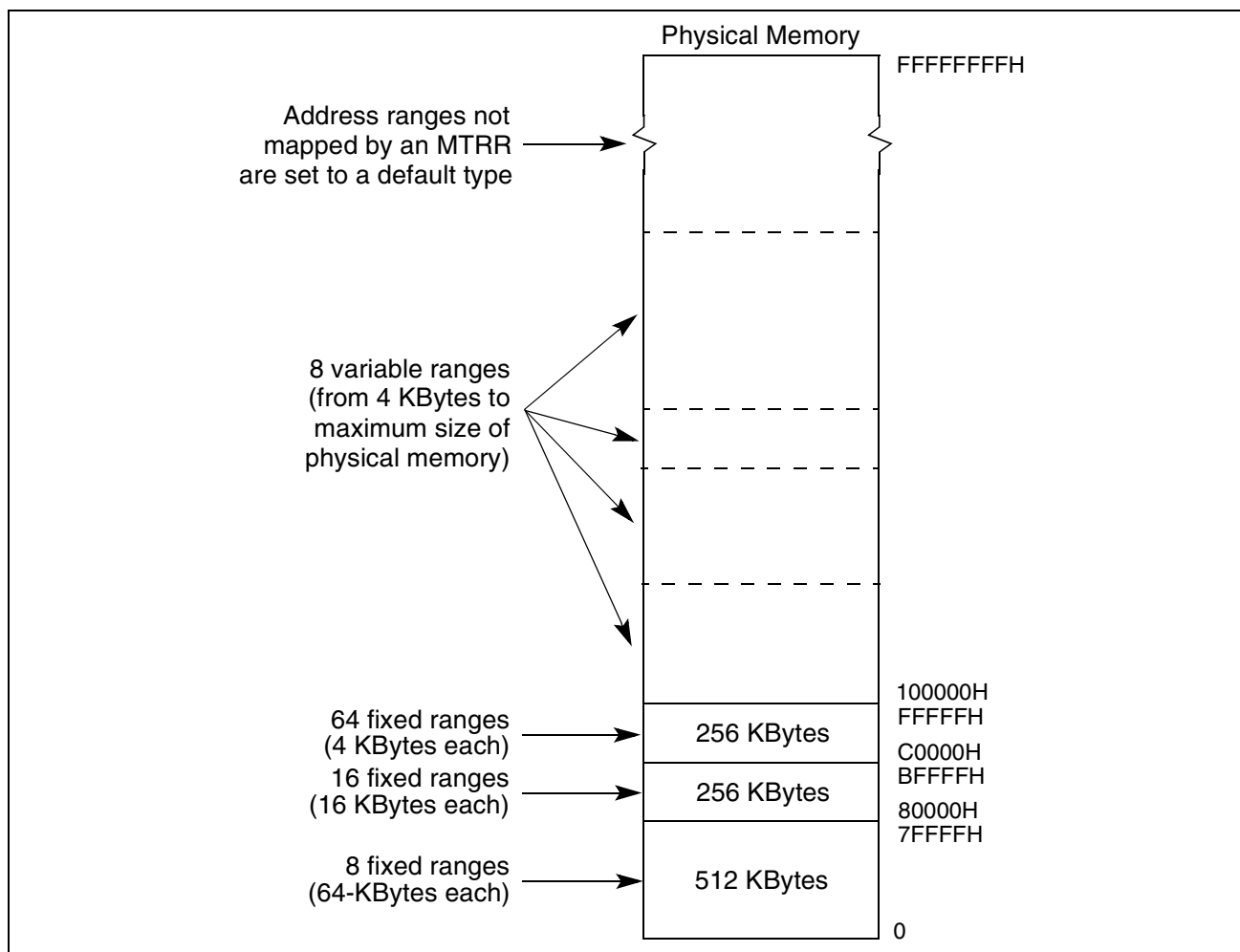
In a multiprocessor system, different Pentium 4, Intel Xeon, or P6 family processors **MUST** use the identical MTRR memory map so that software has a consistent view of memory, independent of the processor executing a program.

Table 10-8. Memory Types That Can Be Encoded in MTRRs

Memory Type and Mnemonic	Encoding in MTRR
Uncacheable (UC)	00H
Write Combining (WC)	01H
Reserved*	02H
Reserved*	03H
Write-through (WT)	04H
Write-protected (WP)	05H
Writeback (WB)	06H
Reserved*	7H through FFH

NOTE:

* Using these encoding result in a general-protection exception (#GP) being generated.


Figure 10-3. Mapping Physical Memory With MTRRs

10.11.1. MTRR Feature Identification

The availability of the MTRR feature is model-specific. Software can determine if MTRRs are supported on a processor by executing the CUID instruction and reading the state of the MTRR flag (bit 12) in the feature information register (EDX).

If the MTRR flag is set (indicating that the processor implements MTRRs), additional information about MTRRs can be obtained from the 64-bit IA32_MTRRCAP MSR (named MTRRcap MSR for the P6 family processors). The IA32_MTRRCAP MSR is a read-only MSR that can be read with the RDMSR instruction. Figure 10-4 shows the contents of the IA32_MTRRCAP MSR. The functions of the flags and field in this register are as follows:

VCNT (variable range registers count) field, bits 0 through 7

Indicates the number of variable ranges implemented on the processor. The Pentium 4, Intel Xeon, and P6 family processors have eight pairs of MTRRs for setting up eight variable ranges.

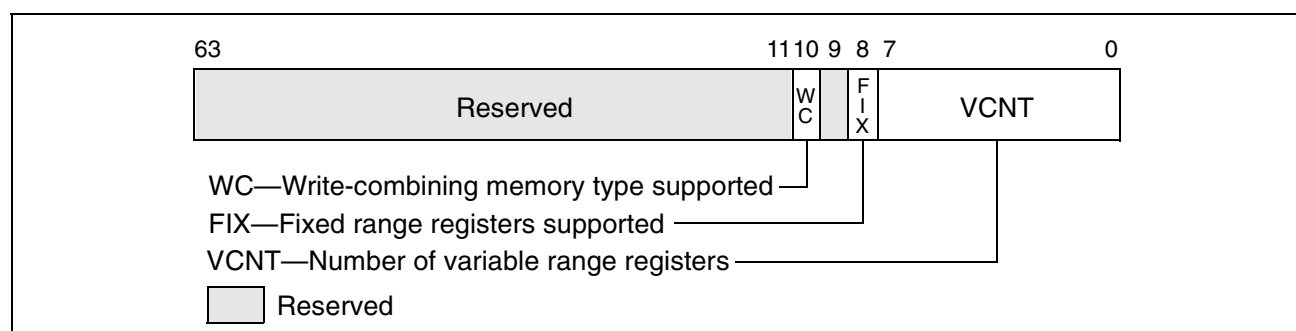


Figure 10-4. IA32_MTRRCAP Register

FIX (fixed range registers supported) flag, bit 8

Fixed range MTRRs (IA32_MTRR_FIX64K_00000 through IA32_MTRR_FIX4K_0F8000) are supported when set; no fixed range registers are supported when clear.

WC (write combining) flag, bit 10

The write-combining (WC) memory type is supported when set; the WC type is not supported when clear.

Bit 9 and bits 11 through 63 in the IA32_MTRRCAP MSR are reserved. If software attempts to write to the IA32_MTRRCAP MSR, a general-protection exception (#GP) is generated.

For the Pentium 4, Intel Xeon, and P6 family processors, the IA32_MTRRCAP MSR always contains the value 508H.

10.11.2. Setting Memory Ranges with MTRRs

The memory ranges and the types of memory specified in each range are set by three groups of registers: the IA32_MTRR_DEF_TYPE MSR, the fixed-range MTRRs, and the variable range MTRRs. These registers can be read and written to using the RDMSR and WRMSR instructions, respectively. The IA32_MTRRCAP MSR indicates the availability of these registers on the processor (see Section 10.11.1., “MTRR Feature Identification”).

10.11.2.1. IA32_MTRR_DEF_TYPE MSR

The IA32_MTRR_DEF_TYPE MSR (named MTRRdefType MSR for the P6 family processors) sets the default properties of the regions of physical memory that are not encompassed by MTRRs (see Figure 10-4). The functions of the flags and field in this register are as follows:

Type field, bits 0 through 7

Indicates the default memory type used for those physical memory address ranges that do not have a memory type specified for them by an MTRR. (See Table 10-8 for the encoding of this field.) If the MTRRs are disabled, this field defines the memory type for all of physical memory. The legal values for this field are 0, 1, 4, 5, and 6. All other values result in a general-protection exception (#GP) being generated.

Intel recommends the use of the UC (uncached) memory type for all physical memory addresses where memory does not exist. To assign the UC type to nonexistent memory locations, it can either be specified as the default type in the Type field or be explicitly assigned with the fixed and variable MTRRs.

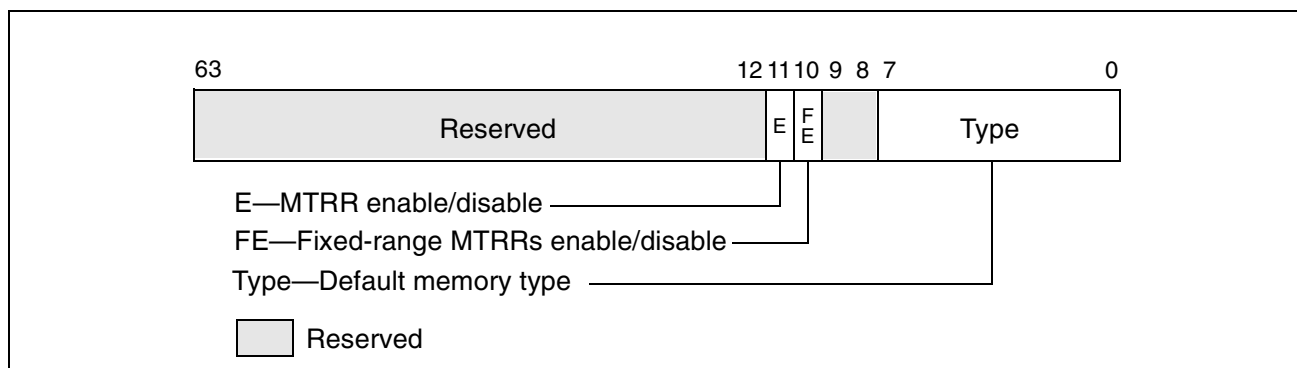


Figure 10-5. IA32_MTRR_DEF_TYPE MSR

FE (fixed MTRRs enabled) flag, bit 10

Fixed-range MTRRs are enabled when set; fixed-range MTRRs are disabled when clear. When the fixed-range MTRRs are enabled, they take priority over the variable-range MTRRs when overlaps in ranges occur. If the fixed-range MTRRs are disabled, the variable-range MTRRs can still be used and can map the range ordinarily covered by the fixed-range MTRRs.

E (MTRRs enabled) flag, bit 11

MTRRs are enabled when set; all MTRRs are disabled when clear, and the UC memory type is applied to all of physical memory. When this flag is set, the FE flag can disable the fixed-range MTRRs; when the flag is clear, the FE flag has no affect. When the E flag is set, the type specified in the default memory type field is used for areas of memory not already mapped by either a fixed or variable MTRR.

Bits 8 and 9, and bits 12 through 63, in the IA32_MTRR_DEF_TYPE MSR are reserved; the processor generates a general-protection exception (#GP) if software attempts to write nonzero values to them.

10.11.2.2. FIXED RANGE MTRRS

The fixed memory ranges are mapped with 11 fixed-range registers of 64 bits each. Each of these registers is divided into 8-bit fields that are used to specify the memory type for each of the sub-ranges the register controls:

- **Register IA32_MTRR_FIX64K_00000.** Maps the 512-KByte address range from 0H to 7FFFFH. This range is divided into eight 64-KByte sub-ranges.
- **Registers IA32_MTRR_FIX16K_80000 and IA32_MTRR_FIX16K_A0000.** Maps the two 128-KByte address ranges from 80000H to BFFFFH. This range is divided into sixteen 16-KByte sub-ranges, 8 ranges per register.
- **Registers IA32_MTRR_FIX4K_C0000 through IA32_MTRR_FIX4K_F8000.** Maps eight 32-KByte address ranges from C0000H to FFFFFH. This range is divided into sixty-four 4-KByte sub-ranges, 8 ranges per register.

Table 10-9 shows the relationship between the fixed physical-address ranges and the corresponding fields of the fixed-range MTRRs; Table 10-8 shows the possible encoding of these fields.

Note that for the P6 family processors, the prefix for the fixed range MTRRs is MTRRfix.

10.11.2.3. VARIABLE RANGE MTRRS

The Pentium 4, Intel Xeon, and P6 family processors permit software to specify the memory type for eight variable-size address ranges, using a pair of MTRRs for each range. The first of each pair (IA32_MTRR_PHYSBASE n) defines the base address and memory type for the range, and the second (IA32_MTRR_PHYSMASK n) contains a mask that is used to determine the address range. The “ n ” suffix indicates registers pairs 0 through 7.

Note that for the P6 family processors, the prefixes for the variable range MTRRs are MTRRphysBase and MTRRphysMask.

Table 10-9. Address Mapping for Fixed-Range MTRRs

Address Range (hexadecimal)								MTRR
63 56	55 48	47 40	39 32	31 24	23 16	15 8	7 0	
7000-7FFFF	6000-6FFFF	5000-5FFFF	4000-4FFFF	3000-3FFFF	2000-2FFFF	1000-1FFFF	0000-0FFFF	IA32_MTRR_FIX64K_00000
9C000-9FFFF	98000-98FFF	94000-97FFF	90000-93FFF	8C000-8FFFF	88000-8BFFF	84000-87FFF	80000-83FFF	IA32_MTRR_FIX16K_80000
BC000-BFFFF	B8000-BBFFF	B4000-B7FFF	B0000-B3FFF	AC000-AFFFF	A8000-ABFFF	A4000-A7FFF	A0000-A3FFF	IA32_MTRR_FIX16K_A0000
C7000-C7FFF	C6000-C6FFF	C5000-C5FFF	C4000-C4FFF	C3000-C3FFF	C2000-C2FFF	C1000-C1FFF	C0000-C0FFF	IA32_MTRR_FIX4K_C0000
CF000-CFFFF	CE000-CEFFF	CD000-CDFFF	CC000-CCFFF	CB000-CBFFF	CA000-CAFFF	C9000-C9FFF	C8000-C8FFF	IA32_MTRR_FIX4K_C8000
D7000-D7FFF	D6000-D6FFF	D5000-D5FFF	D4000-D4FFF	D3000-D3FFF	D2000-D2FFF	D1000-D1FFF	D0000-D0FFF	IA32_MTRR_FIX4K_D0000
DF000-DFFFF	DE000-DEFFF	DD000-DDFFF	DC000-DCFFF	DB000-DBFFF	DA000-DAFFF	D9000-D9FFF	D8000-D8FFF	IA32_MTRR_FIX4K_D8000
E7000-E7FFF	E6000-E6FFF	E5000-E5FFF	E4000-E4FFF	E3000-E3FFF	E2000-E2FFF	E1000-E1FFF	E0000-E0FFF	IA32_MTRR_FIX4K_E0000
EF000-EFFFF	EE000-EEFFF	ED000-EDFFF	EC000-ECFFF	EB000-EBFFF	EA000-EAFFF	E9000-E9FFF	E8000-E8FFF	IA32_MTRR_FIX4K_E8000
F7000-F7FFF	F6000-F6FFF	F5000-F5FFF	F4000-F4FFF	F3000-F3FFF	F2000-F2FFF	F1000-F1FFF	F0000-F0FFF	IA32_MTRR_FIX4K_F0000
FF000-FFFFFF	FE000-FEFFF	FD000-FDFFF	FC000-FCFFF	FB000-FBFFF	FA000-FAFFF	F9000-F9FFF	F8000-F8FFF	IA32_MTRR_FIX4K_F8000

Figure 10-6 shows flags and fields in these registers. The functions of the flags and fields in these registers are as follows:

Type field, bits 0 through 7

Specifies the memory type for the range (see Table 10-8 for the encoding of this field).

PhysBase field, bits 12 through 35

Specifies the base address of the address range. This 24-bit value is extended by 12 bits at the low end to form the base address, which automatically aligns the address on a 4-KByte boundary.

PhysMask field, bits 12 through 35

Specifies a 24-bit mask that determines the range of the region being mapped, according to the following relationship:

$$\text{Address_Within_Range AND PhysMask} = \text{PhysBase AND PhysMask}$$

This 24-bit value is extended by 12 bits at the low end to form the mask value. See Section 10.11.3., “Example Base and Mask Calculations”, for more information and some examples of base address and mask computations.

V (valid) flag, bit 11

Enables the register pair when set; disables register pair when clear.

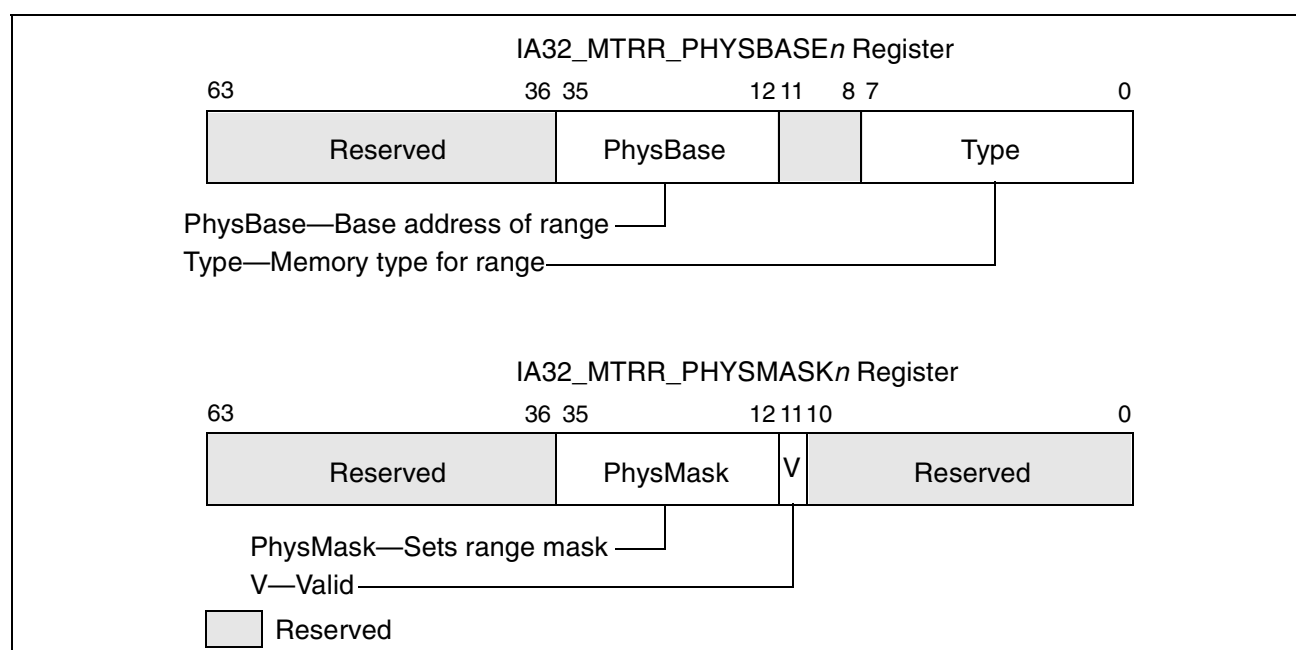


Figure 10-6. IA32_MTRR_PHYSBASE_n and IA32_MTRR_PHYSMASK_n Variable-Range Register Pair

All other bits in the IA32_MTRR_PHYSBASE_n and IA32_MTRR_PHYSMASK_n registers are reserved; the processor generates a general-protection exception (#GP) if software attempts to write to them.

Overlapping variable MTRR ranges are not supported generically. However, two variable ranges are allowed to overlap, if the following conditions are present:

- If both of them are UC (uncached).
- If one range is of type UC and the other is of type WB (write back).

In both cases above, the effective type for the overlapping region is UC. The processor's behavior is undefined for all other cases of overlapping variable ranges.

A variable range can overlap a fixed range (provided the fixed range MTRR's are enabled). Here, the memory type specified in the fixed range register overrides the one specified in variable-range register pair.

NOTE

Some mask values can result in discontinuous ranges. In a discontinuous range, the area not mapped by the mask value is set to the default memory type. Intel does not encourage the use of discontinuous ranges, because they could require physical memory to be present throughout the entire 4-GByte physical memory map. If memory is not provided for the complete memory map, the behaviour of the processor is undefined.

10.11.3. Example Base and Mask Calculations

The base and mask values entered into the variable-range MTRR pairs are 24-bit values that the processor extends to 36-bits. For example, to enter a base address of 2 MBytes (200000H) to the IA32_MTRR_PHYSBASE3 register, the 12 least-significant bits are truncated and the value 000200H is entered into the PhysBase field. The same operation must be performed on mask values. For instance, to map the address range from 200000H to 3FFFFFFH (2 MBytes to 4 MBytes), a mask value of FFFE0000H is required. Here again, the 12 least-significant bits of this mask value are truncated, so that the value entered in the PhysMask field of the IA32_MTRR_PHYSMASK3 register is FFFE00H. This mask is chosen so that when any address in the 200000H to 3FFFFFFH range is ANDed with the mask value it will return the same value as when the base address is ANDed with the mask value (which is 200000H).

To map the address range from 400000H 7FFFFFFH (4 MBytes to 8 MBytes), a base value of 000400H is entered in the PhysBase field and a mask value of FFFC00H is entered in the PhysMask field.

Here is a real-life example of setting up the MTRRs for an entire system. Assume that the system has the following characteristics:

- 96 MBytes of system memory is mapped as write-back memory (WB) for highest system performance.
- A custom 4-MByte I/O card is mapped to uncached memory (UC) at a base address of 64 MBytes. This restriction forces the 96 MBytes of system memory to be addressed from 0 to 64 MBytes and from 68 MBytes to 100 MBytes, leaving a 4-MByte hole for the I/O card.
- An 8-MByte graphics card is mapped to write-combining memory (WC) beginning at address A0000000H.
- The BIOS area from 15 MBytes to 16 MBytes is mapped to UC memory.

The following settings for the MTRRs will yield the proper mapping of the physical address space for this system configuration.

```
IA32_MTRR_PHYSBASE0 = 0000 0000 0000 0006H
IA32_MTRR_PHYSMASK0 = 0000 000F FC00 0800H
Caches 0-64 MB as WB cache type.
IA32_MTRR_PHYSBASE1 = 0000 0000 0400 0006H
IA32_MTRR_PHYSMASK1 = 0000 000F FE00 0800H
Caches 64-96 MB as WB cache type.
IA32_MTRR_PHYSBASE2 = 0000 0000 0600 0006H
IA32_MTRR_PHYSMASK2 = 0000 000F FFC0 0800H
Caches 96-100 MB as WB cache type.
IA32_MTRR_PHYSBASE3 = 0000 0000 0400 0000H
IA32_MTRR_PHYSMASK3 = 0000 000F FFC0 0800H
Caches 64-68 MB as UC cache type.
IA32_MTRR_PHYSBASE4 = 0000 0000 00F0 0000H
IA32_MTRR_PHYSMASK4 = 0000 000F FFF0 0800H
Caches 15-16 MB as UC cache type
IA32_MTRR_PHYSBASE5 = 0000 0000 A000 0001H
IA32_MTRR_PHYSMASK5 = 0000 000F FF80 0800H
Caches A0000000-A0800000 as WC type.
```

This MTRR setup uses the ability to overlap any two memory ranges (as long as the ranges are mapped to WB and UC memory types) to minimize the number of MTRR registers that are required to configure the memory environment. This setup also fulfills the requirement that two register pairs are left for operating system usage.

10.11.4. Range Size and Alignment Requirement

The range that is to be mapped to a variable-range MTRR must meet the following “power of 2” size and alignment rules:

1. The minimum range size is 4 KBytes, and the base address of this range must be on at least a 4-KByte boundary.
2. For ranges greater than 4 KBytes, each range must be of length 2^n and its base address must be aligned on a 2^n boundary, where n is a value equal to or greater than 12. The base-address alignment value cannot be less than its length. For example, an 8-KByte range cannot be aligned on a 4-KByte boundary. It must be aligned on at least an 8-KByte boundary.

10.11.4.1. MTRR PRECEDENCES

If the MTRRs are not enabled (by setting the E flag in the IA32_MTRR_DEF_TYPE MSR), then all memory accesses are of the UC memory type. If the MTRRs are enabled, then the memory type used for a memory access is determined as follows:

1. If the physical address falls within the first 1 MByte of physical memory and fixed MTRRs are enabled, the processor uses the memory type stored for the appropriate fixed-range MTRR.
2. Otherwise, the processor attempts to match the physical address with a memory type range set with a pair of variable-range MTRRs:
 - a. If one variable memory range matches, the processor uses the memory type stored in the IA32_MTRR_PHYSBASE n register for that range.
 - b. If two or more variable memory ranges match and the memory types are identical, then that memory type is used.
 - c. If two or more variable memory ranges match and one of the memory types is UC, the UC memory type is used.
 - d. If two or more variable memory ranges match and the memory types are WT and WB, the WT memory type is used.
 - e. If two or more variable memory ranges match and the memory types are other than UC and WB, the behaviour of the processor is undefined.
3. If no fixed or variable memory range matches, the processor uses the default memory type.

10.11.5. MTRR Initialization

On a hardware reset, a Pentium 4, Intel Xeon, or P6 family processor clears the valid flags in the variable-range MTRRs and clears the E flag in the IA32_MTRR_DEF_TYPE MSR to disable all MTRRs. All other bits in the MTRRs are undefined. Prior to initializing the MTRRs, software (normally the system BIOS) must initialize all fixed-range and variable-range MTRR registers fields to 0. Software can then initialize the MTRRs according to the types of memory known to it, including memory on devices that it auto-configures. This initialization is expected to occur prior to booting the operating system.

See Section 10.11.8., “MTRR Considerations in MP Systems”, for information on initializing MTRRs in MP (multiple-processor) systems.

10.11.6. Remapping Memory Types

A system designer may re-map memory types to tune performance or because a future processor may not implement all memory types supported by the Pentium 4, Intel Xeon, and P6 family processors. The following rules support coherent memory-type re-mappings:

1. A memory type should not be mapped into another memory type that has a weaker memory ordering model. For example, the uncacheable type cannot be mapped into any other type, and the write-back, write-through, and write-protected types cannot be mapped into the weakly ordered write-combining type.
2. A memory type that does not delay writes should not be mapped into a memory type that does delay writes, because applications of such a memory type may rely on its write-through behavior. Accordingly, the write-back type cannot be mapped into the write-through type.
3. A memory type that views write data as not necessarily stored and read back by a subsequent read, such as the write-protected type, can only be mapped to another type with the same behaviour (and there are no others for the Pentium 4, Intel Xeon, and P6 family processors) or to the uncacheable type.

In many specific cases, a system designer can have additional information about how a memory type is used, allowing additional mappings. For example, write-through memory with no associated write side effects can be mapped into write-back memory.

10.11.7. MTRR Maintenance Programming Interface

The operating system maintains the MTRRs after booting and sets up or changes the memory types for memory-mapped devices. The operating system should provide a driver and application programming interface (API) to access and set the MTRRs. The function calls MemTypeGet() and MemTypeSet() define this interface.

10.11.7.1. MEMTYPEGET() FUNCTION

The MemTypeGet() function returns the memory type of the physical memory range specified by the parameters base and size. The base address is the starting physical address and the size is the number of bytes for the memory range. The function automatically aligns the base address and size to 4-KByte boundaries. Pseudocode for the MemTypeGet() function is given in Example 10-2.

Example 10-2. MemTypeGet() Pseudocode

```
#define MIXED_TYPES -1    /* 0 < MIXED_TYPES || MIXED_TYPES > 256 */

IF CPU_FEATURES.MTRR /* processor supports MTRRs */
  THEN
    Align BASE and SIZE to 4-KByte boundary;
    IF (BASE + SIZE) wrap 4-GByte address space
```

```

        THEN return INVALID;
    FI;
    IF MTRRdefType.E = 0
        THEN return UC;
    FI;
    FirstType ← Get4KMemType (BASE);
    /* Obtains memory type for first 4-KByte range */
    /* See Get4KMemType (4KByteRange) in Example 10-3 */
    FOR each additional 4-KByte range specified in SIZE
        NextType ← Get4KMemType (4KByteRange);
        IF NextType ≠ FirstType
            THEN return MixedTypes;
    FI;
    ROF;
    return FirstType;
ELSE return UNSUPPORTED;
FI;

```

If the processor does not support MTRRs, the function returns **UNSUPPORTED**. If the MTRRs are not enabled, then the UC memory type is returned. If more than one memory type corresponds to the specified range, a status of **MIXED_TYPES** is returned. Otherwise, the memory type defined for the range (UC, WC, WT, WB, or WP) is returned.

The pseudocode for the **Get4KMemType()** function in Example 10-3 obtains the memory type for a single 4-KByte range at a given physical address. The sample code determines whether an **PHY_ADDRESS** falls within a fixed range by comparing the address with the known fixed ranges: 0 to 7FFFFH (64-KByte regions), 80000H to BFFFFH (16-KByte regions), and C0000H to FFFFFH (4-KByte regions). If an address falls within one of these ranges, the appropriate bits within one of its MTRRs determine the memory type.

Example 10-3. Get4KMemType() Pseudocode

```

IF IA32_MTRRCAP.FIX AND MTRRdefType.FE /* fixed registers enabled */
    THEN IF PHY_ADDRESS is within a fixed range
        return IA32_MTRR_FIX.Type;
FI;
FOR each variable-range MTRR in IA32_MTRRCAP.VCNT
    IF IA32_MTRR_PHYSMASK.V = 0
        THEN continue;
    FI;
    IF (PHY_ADDRESS AND IA32_MTRR_PHYSMASK.Mask) =
        (IA32_MTRR_PHYSBASE.Base
        AND IA32_MTRR_PHYSMASK.Mask)
        THEN
            return IA32_MTRR_PHYSBASE.Type;
    FI;
ROF;
return MTRRdefType.Type;

```

10.11.7.2. MEMTYPESET() FUNCTION

The MemTypeSet() function in Example 10-4 sets a MTRR for the physical memory range specified by the parameters base and size to the type specified by type. The base address and size are multiples of 4 KBytes and the size is not 0.

Example 10-4. MemTypeSet Pseudocode

```

IF CPU_FEATURES.MTRR (* processor supports MTRRs *)
  THEN
    IF BASE and SIZE are not 4-KByte aligned or size is 0
      THEN return INVALID;
    FI;
    IF (BASE + SIZE) wrap 4-GByte address space
      THEN return INVALID;
    FI;
    IF TYPE is invalid for Pentium 4, Intel Xeon, and P6 family processors
      THEN return UNSUPPORTED;
    FI;
    IF TYPE is WC and not supported
      THEN return UNSUPPORTED;
    FI;
    IF IA32_MTRRCAP.FIX is set AND range can be mapped using a fixed-range MTRR
      THEN
        pre_mtrr_change();
        update affected MTRR;
        post_mtrr_change();
      FI;

  ELSE (* try to map using a variable MTRR pair *)
    IF IA32_MTRRCAP.VCNT = 0
      THEN return UNSUPPORTED;
    FI;
    IF conflicts with current variable ranges
      THEN return RANGE_OVERLAP;
    FI;
    IF no MTRRs available
      THEN return VAR_NOT_AVAILABLE;
    FI;
    IF BASE and SIZE do not meet the power of 2 requirements for variable MTRRs
      THEN return INVALID_VAR_REQUEST;
    FI;
    pre_mtrr_change();
    Update affected MTRRs;
    post_mtrr_change();
  FI;

pre_mtrr_change()

```



```
BEGIN
    disable interrupts;
    Save current value of CR4;
    disable and flush caches;
    flush TLBs;
    disable MTRRs;
    IF multiprocessing
        THEN maintain consistency through IPIs;
    FI;
END
post_mtrr_change()
BEGIN
    flush caches and TLBs;
    enable MTRRs;
    enable caches;
    restore value of CR4;
    enable interrupts;
END
```

The physical address to variable range mapping algorithm in the MemTypeSet function detects conflicts with current variable range registers by cycling through them and determining whether the physical address in question matches any of the current ranges. During this scan, the algorithm can detect whether any current variable ranges overlap and can be concatenated into a single range.

The pre_mtrr_change() function disables interrupts prior to changing the MTRRs, to avoid executing code with a partially valid MTRR setup. The algorithm disables caching by setting the CD flag and clearing the NW flag in control register CR0. The caches are invalidated using the WBINVD instruction. The algorithm disables the page global flag (PGE) in control register CR4, if necessary, then flushes all TLB entries by updating control register CR3. Finally, it disables MTRRs by clearing the E flag in the IA32_MTRR_DEF_TYPE MSR.

After the memory type is updated, the post_mtrr_change() function re-enables the MTRRs and again invalidates the caches and TLBs. This second invalidation is required because of the processor's aggressive prefetch of both instructions and data. The algorithm restores interrupts and re-enables caching by setting the CD flag.

An operating system can batch multiple MTRR updates so that only a single pair of cache invalidations occur.

10.11.8. MTRR Considerations in MP Systems

In MP (multiple-processor) systems, the operating systems must maintain MTRR consistency between all the processors in the system. The Pentium 4, Intel Xeon, and P6 family processors provide no hardware support to maintain this consistency. In general, all processors must have the same MTRR values.

This requirement implies that when the operating system initializes an MP system, it must load the MTRRs of the boot processor while the E flag in register MTRRdefType is 0. The operating system then directs other processors to load their MTRRs with the same memory map. After all the processors have loaded their MTRRs, the operating system signals them to enable their MTRRs. Barrier synchronization is used to prevent further memory accesses until all processors indicate that the MTRRs are enabled. This synchronization is likely to be a shoot-down style algorithm, with shared variables and interprocessor interrupts.

Any change to the value of the MTRRs in an MP system requires the operating system to repeat the loading and enabling process to maintain consistency, using the following procedure:

1. Broadcast to all processors to execute the following code sequence.
2. Disable interrupts.
3. Wait for all processors to reach this point.
4. Enter the no-fill cache mode. (Set the CD flag in control register CR0 to 1 and the NW flag to 0.)
5. Flush all caches using the WBINVD instructions. Note on a processor that supports self-snooping, CPUID feature flag bit 27, this step is unnecessary.
6. Clear the PGE flag in control register CR4 (if set).
7. Flush all TLBs. (Execute a MOV from control register CR3 to another register and then a MOV from that register back to CR3.)
8. Disable all range registers (by clearing the E flag in register MTRRdefType). If only variable ranges are being modified, software may clear the valid bits for the affected register pairs instead.
9. Update the MTRRs.
10. Enable all range registers (by setting the E flag in register MTRRdefType). If only variable-range registers were modified and their individual valid bits were cleared, then set the valid bits for the affected ranges instead.
11. Flush all caches and all TLBs a second time. (The TLB flush is required for Pentium 4, Intel Xeon, and P6 family processors. Executing the WBINVD instruction is not needed when using Pentium 4, Intel Xeon, and P6 family processors, but it may be needed in future systems.)
12. Enter the normal cache mode to re-enable caching. (Set the CD and NW flags in control register CR0 to 0.)
13. Set PGE flag in control register CR4, if previously cleared.
14. Wait for all processors to reach this point.
15. Enable interrupts.

10.11.9. Large Page Size Considerations

The MTRRs provide memory typing for a limited number of regions that have a 4 KByte granularity (the same granularity as 4-KByte pages). The memory type for a given page is cached in the processor's TLBs. When using large pages (2 or 4 MBytes), a single page-table entry covers multiple 4-KByte granules, each with a single memory type. Because the memory type for a large page is cached in the TLB, the processor can behave in an undefined manner if a large page is mapped to a region of memory that MTRRs have mapped with multiple memory types.

Undefined behavior can be avoided by insuring that all MTRR memory-type ranges within a large page are of the same type. If a large page maps to a region of memory containing different MTRR-defined memory types, the PCD and PWT flags in the page-table entry should be set for the most conservative memory type for that range. For example, a large page used for memory mapped I/O and regular memory is mapped as UC memory. Alternatively, the operating system can map the region using multiple 4-KByte pages each with its own memory type.

The requirement that all 4-KByte ranges in a large page are of the same memory type implies that large pages with different memory types may suffer a performance penalty, since they must be marked with the lowest common denominator memory type.

The Pentium 4, Intel Xeon, and P6 family processors provide special support for the physical memory range from 0 to 4 MBytes, which is potentially mapped by both the fixed and variable MTRRs. This support is invoked when a Pentium 4, Intel Xeon, or P6 family processor detects a large page overlapping the first 1 MByte of this memory range with a memory type that conflicts with the fixed MTRRs. Here, the processor maps the memory range as multiple 4-KByte pages within the TLB. This operation insures correct behavior at the cost of performance. To avoid this performance penalty, operating-system software should reserve the large page option for regions of memory at addresses greater than or equal to 4 MBytes.

10.12. PAGE ATTRIBUTE TABLE (PAT)

The Page Attribute Table (PAT) extends the IA-32 architecture's page-table format to allow memory types to be assigned to regions of physical memory based on linear address mappings. The PAT is a companion feature to the MTRRs; that is, the MTRRs allow mapping of memory types to regions of the physical address space, where the PAT allows mapping of memory types to pages within the linear address space. The MTRRs are useful for statically describing memory types for physical ranges, and are typically set up by the system BIOS. The PAT extends the functions of the PCD and PWT bits in page tables to allow all five of the memory types that can be assigned with the MTRRs (plus one additional memory type) to also be assigned dynamically to pages of the linear address space.

The PAT was introduced into the IA-32 architecture in the Pentium III processor and is also available in the Pentium 4 and Intel Xeon processors.

NOTE

In multiple processor systems, the operating system must maintain MTRR consistency between all the processors in the system (that is, all processors must use the same MTRR values). The Pentium 4, Intel Xeon, and P6 family processors provide no hardware support for maintaining this consistency.

10.12.1. Detecting Support for the PAT Feature

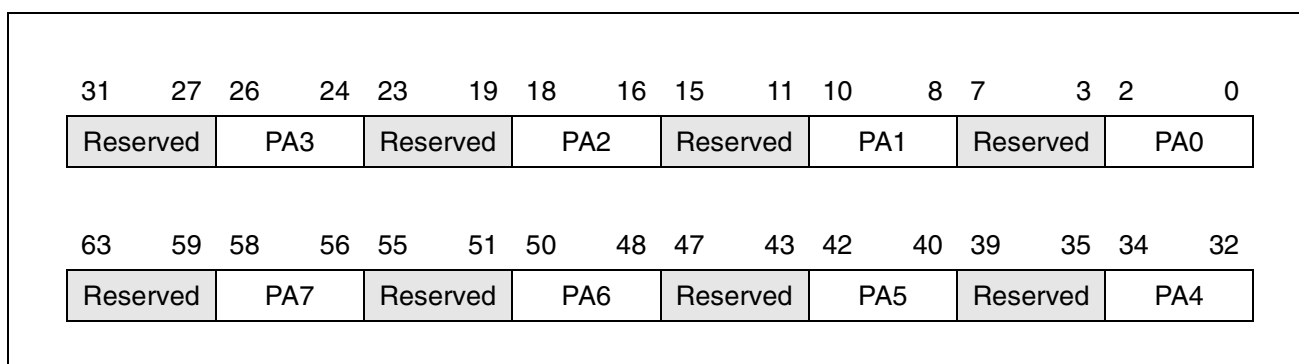
An operating system or executive can detect the availability of the PAT by executing the CUID instruction with a value of 1 in the EAX register. Support for the PAT is indicated by the PAT flag (bit 16 of the values returned to EDX register). If the PAT is supported, the operating system or executive can use the IA32_CR_PAT MSR to program the PAT. When memory types have been assigned to entries in the PAT, software can then use of the PAT-index bit (PAT) in the page-table and page-directory entries along with the PCD and PWT bits to assign memory types from the PAT to individual pages.

Note that there is no separate flag or control bit in any of the control registers that enables the PAT. The PAT is always enabled on all processors that support it, and the table lookup always occurs whenever paging is enabled, in all paging modes.

10.12.2. IA32_CR_PAT MSR

The IA32_CR_PAT MSR is located at MSR address 277H (see to Appendix B, *Model-Specific Registers (MSRs)*), and this address will remain at the same address on future IA-32 processors that support the PAT feature. Figure 10-7 shows the format of the 64-bit IA32_CR_PAT MSR.

The IA32_CR_PAT MSR contains eight page attribute fields: PA0 through PA7. The three low-order bits of each field are used to specify a memory type. The five high-order bits of each field are reserved, and must be set to all 0s. Each of the eight page attribute fields can contain any of the memory type encodings specified in Table 10-10.

**Figure 10-7. IA32_CR_PAT MSR**

Note that for the P6 family processors, the IA32_CR_PAT MSR is named the PAT MSR.

Table 10-10. Memory Types That Can Be Encoded With PAT

Encoding	Mnemonic
00H	Uncacheable (UC)
01H	Write Combining (WC)
02H	Reserved*
03H	Reserved*
04H	Write Through (WT)
05H	Write Protected (WP)
06H	Write Back (WB)
07H	Uncached (UC-)
08H - FFH	Reserved*

Note:

* Using these encoding result in a general-protection exception (#GP) being generated.

10.12.3. Selecting a Memory Type from the PAT

To select a memory type for a page from the PAT, a 3-bit index made up of the PAT, PCD, and PWT bits must be encoded in the page-table or page-directory entry for the page. Table 10-11 shows the possible encodings of the PAT, PCD, and PWT bits and the PAT entry selected with each encoding. The PAT bit is bit 7 in page-table entries that point to 4-KByte pages (see Figures 3-14 and 3-20) and bit 12 in page-directory entries that point to 2-MByte or 4-MByte pages (see Figures 3-15, 3-21, and 3-23). The PCD and PWT bits are always bits 4 and 3, respectively, in page-table and page-directory entries.

The PAT entry selected for a page is used in conjunction with the MTRR setting for the region of physical memory in which the page is mapped to determine the effective memory type for the page, as shown in Table 10-7.

Table 10-11. Selection of PAT Entries with PAT, PCD, and PWT Flags

PAT	PCD	PWT	PAT Entry
0	0	0	PAT0
0	0	1	PAT1
0	1	0	PAT2
0	1	1	PAT3
1	0	0	PAT4
1	0	1	PAT5
1	1	0	PAT6
1	1	1	PAT7

10.12.4. Programming the PAT

Table 10-12 shows the default setting for each PAT entry following a power up or reset of the processor. The setting remain unchanged following a soft reset (INIT reset).

Table 10-12. Memory Type Setting of PAT Entries Following a Power-up or Reset

PAT Entry	Memory Type Following Power-up or Reset
PAT0	WB
PAT1	WT
PAT2	UC-
PAT3	UC
PAT4	WB
PAT5	WT
PAT6	UC-
PAT7	UC

The values in all the entries of the PAT can be changed by writing to the IA32_CR_PAT MSR using the WRMSR instruction. The IA32_CR_PAT MSR is read and write accessible (use of the RDMSR and WRMSR instructions, respectively) to software operating at a CPL of 0. Table 10-10 shows the allowable encoding of the entries in the PAT. Attempting to write an undefined memory type encoding into the PAT causes a general-protection (#GP) exception to be generated.

NOTE

In a multiple processor system, the PATs of all processors must contain the same values.

The operating system is responsible for insuring that changes to a PAT entry occur in a manner that maintains the consistency of the processor caches and translation lookaside buffers (TLB). This is accomplished by following the procedure as specified in Section 10.11.8., “MTRR Considerations in MP Systems” for changing the value of an MTRR in a multiple processor system. It requires a specific sequence of operations that includes flushing the processors caches and TLBs.

The PAT allows any memory type to be specified in the page tables, and therefore it is possible to have a single physical page mapped to two or more different linear addresses, each with different memory types. Intel does not support this practice because it may lead to undefined operations that can result in a system failure. In particular, a WC page must never be aliased to a cacheable page because WC writes may not check the processor caches. When remapping a page that was previously mapped as a cacheable memory type to a WC page, an operating system can avoid this type of aliasing by doing the following:

1. Remove the previous mapping to a cacheable memory type in the page tables; that is, make them not present.
2. Flush the TLBs of processors that may have used the mapping, even speculatively.

3. Create a new mapping to the same physical address with a new memory type, for instance, WC.
4. Flush the caches on all processors that may have used the mapping previously. Note on processors that support self-snooping, CPUID feature flag bit 27, this step is unnecessary.

Operating systems that use a page directory as a page table (to map large pages) and enable page size extensions must carefully scrutinize the use of the PAT index bit for the 4-KByte page-table entries. The PAT index bit for a page-table entry (bit 7) corresponds to the page size bit in a page-directory entry. Therefore, the operating system can only use PAT entries PA0 through PA3 when setting the caching type for a page table that is also used as a page directory. If the operating system attempts to use PAT entries PA4 through PA7 when using this memory as a page table, it effectively sets the PS bit for the access to this memory as a page directory.

NOTE

For compatibility with earlier IA-32 processors that do not support the PAT, care should be taken in selecting the encodings for entries in the PAT (see Section 10.12.5., “PAT Compatibility with Earlier IA-32 Processors”).

10.12.5. PAT Compatibility with Earlier IA-32 Processors

For IA-32 processors that support the PAT, the IA32_CR_PAT MSR is always active. That is, the PCD and PWT bits in page-table entries and in page-directory entries (that point to pages) are always select a memory type for a page indirectly by selecting an entry in the PAT. They never select the memory type for a page directly as they do in earlier IA-32 processors that do not implement the PAT (see Table 10-6).

To allow compatibility for code written to run on earlier IA-32 processor that do not support the PAT, the PAT mechanism has been designed to allow backward compatibility to earlier processors. This compatibility is provided through the ordering of the PAT, PCD, and PWT bits in the 3-bit PAT entry index. For processors that do not implement the PAT, the PAT index bit (bit 7 in the page-table entries and bit 12 in the page-directory entries) is reserved and set to 0. With the PAT bit reserved, only the first four entries of the PAT can be selected with the PCD and PWT bits. At power-up or reset (see Table 10-12), these first four entries are encoded to select the same memory types as the PCD and PWT bits would normally select directly in an IA-32 processor that does not implement the PAT. So, if encodings of the first four entries in the PAT are left unchanged following a power-up or reset, code written to run on earlier IA-32 processors that do not implement the PAT will run correctly on IA-32 processors that do implement the PAT.

11

Intel MMX Technology System Programming

CHAPTER 11

INTEL[®] MMX[™] TECHNOLOGY SYSTEM PROGRAMMING

This chapter describes those features of the Intel[®] MMX[™] technology that must be considered when designing or enhancing an operating system to support MMX technology. It covers MMX instruction set emulation, the MMX state, aliasing of MMX registers, saving MMX state, task and context switching considerations, exception handling, and debugging.

11.1. EMULATION OF THE MMX INSTRUCTION SET

The IA-32 architecture does not support emulation of the MMX instructions, as it does for x87 FPU instructions. The EM flag in control register CR0 (provided to invoke emulation of x87 FPU instructions) cannot be used for MMX instruction emulation. If an MMX instruction is executed when the EM flag is set, an invalid opcode exception (UD#) is generated. Table 11-1 shows the interaction of the EM, MP, and TS flags in control register CR0 when executing MMX instructions.

Table 11-1. Action Taken By MMX Instructions for Different Combinations of EM, MP and TS

CR0 Flags			Action
EM	MP*	TS	
0	1	0	Execute.
0	1	1	#NM exception.
1	1	0	#UD exception.
1	1	1	#UD exception.

Note:

* For processors that support the MMX instructions, the MP flag should be set.

11.2. THE MMX STATE AND MMX REGISTER ALIASING

The MMX state consists of eight 64-bit registers (MM0 through MM7). These registers are aliased to the low 64-bits (bits 0 through 63) of floating-point registers R0 through R7 (see Figure 11-1). Note that the MMX registers are mapped to the physical locations of the floating-point registers (R0 through R7), not to the relative locations of the registers in the floating-point register stack (ST0 through ST7). As a result, the MMX register mapping is fixed and is not affected by value in the Top Of Stack (TOS) field in the floating-point status word (bits 11 through 13).

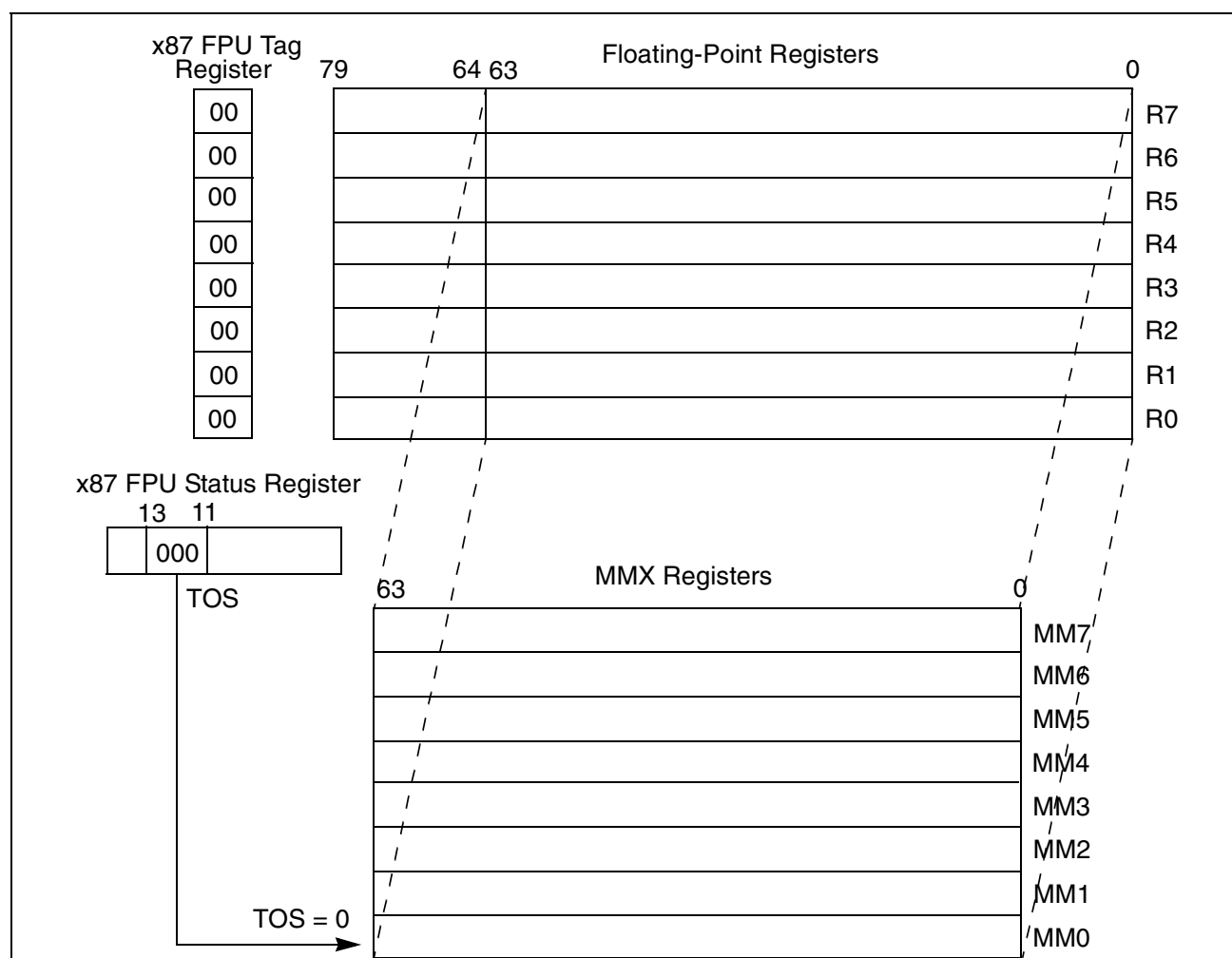


Figure 11-1. Mapping of MMX Registers to Floating-Point Registers

When a value is written into an MMX register using an MMX instruction, the value also appears in the corresponding floating-point register in bits 0 through 63. Likewise, when a floating-point value written into a floating-point register by a x87 FPU, the low 64 bits of that value also appears in the corresponding MMX register.

The execution of MMX instructions have several side effects on the x87 FPU state contained in the floating-point registers, the x87 FPU tag word, and the x87 FPU status word. These side effects are as follows:

- When an MMX instruction writes a value into an MMX register, at the same time, bits 64 through 79 of the corresponding floating-point register are set to all 1s.
- When an MMX instruction (other than the EMMS instruction) is executed, each of the tag fields in the x87 FPU tag word is set to 00B (valid). (See also Section 11.2.1., “Effect of MMX, x87 FPU, FXSAVE, and FXRSTOR Instructions on the x87 FPU Tag Word”.)
- When the EMMS instruction is executed, each tag field in the x87 FPU tag word is set to 11B (empty).
- Each time an MMX instruction is executed, the TOS value is set to 000B.

Execution of MMX instructions does not affect the other bits in the x87 FPU status word (bits 0 through 10 and bits 14 and 15) or the contents of the other x87 FPU registers that comprise the x87 FPU state (the x87 FPU control word, instruction pointer, data pointer, or opcode registers).

Table 11-2 summarizes the effects of the MMX instructions on the x87 FPU state.

Table 11-2. Effects of MMX Instructions on x87 FPU State

MMX Instruction Type	x87 FPU Tag Word	TOS Field of x87 FPU Status Word	Other x87 FPU Registers	Bits 64 Through 79 of x87 FPU Data Registers	Bits 0 Through 63 of x87 FPU Data Registers
Read from MMX register	All tags set to 00B (Valid)	000B	Unchanged	Unchanged	Unchanged
Write to MMX register	All tags set to 00B (Valid)	000B	Unchanged	Set to all 1s	Overwritten with MMX data
EMMS	All fields set to 11B (Empty)	000B	Unchanged	Unchanged	Unchanged

11.2.1. Effect of MMX, x87 FPU, FXSAVE, and FXRSTOR Instructions on the x87 FPU Tag Word

Table 11-3 summarizes the effect of MMX and x87 FPU instructions and the FXSAVE and FXRSTOR instructions on the tags in the x87 FPU tag word and the corresponding tags in an image of the tag word stored in memory.

The values in the fields of the x87 FPU tag word do not affect the contents of the MMX registers or the execution of MMX instructions. However, the MMX instructions do modify the contents of the x87 FPU tag word, as is described in Section 11.2., “The MMX State and MMX Register Aliasing”. These modifications may affect the operation of the x87 FPU when executing x87 FPU instructions, if the x87 FPU state is not initialized or restored prior to beginning x87 FPU instruction execution.

Note that the FSAVE, FXSAVE, and FSTENV instructions (which save x87 FPU state information) read the x87 FPU tag register and contents of each of the floating-point registers, determine the actual tag values for each register (empty, nonzero, zero, or special), and store the updated tag word in memory. After executing these instructions, all the tags in the x87 FPU tag word are set to empty (11B). Likewise, the EMMS instruction clears MMX state from the MMX/floating-point registers by setting all the tags in the x87 FPU tag word to 11B.

Table 11-3. Effect of the MMX, x87 FPU, and FXSAVE/FXRSTOR Instructions on the x87 FPU Tag Word

Instruction Type	Instruction	x87 FPU Tag Word	Image of x87 FPU Tag Word Stored in Memory
MMX	All (except EMMS)	All tags are set to 00B (valid).	Not affected.
MMX	EMMS	All tags are set to 11B (empty).	Not affected.
x87 FPU	All (except FSAVE, FSTENV, FRSTOR, FLDENV)	Tag for modified floating-point register is set to 00B or 11B.	Not affected.
x87 FPU and FXSAVE	FSAVE, FSTENV, FXSAVE	Tags and register values are read and interpreted; then all tags are set to 11B.	Tags are set according to the actual values in the floating-point registers; that is, empty registers are marked 11B and valid registers are marked 00B (nonzero), 01B (zero), or 10B (special).
x87 FPU and FXRSTOR	FRSTOR, FLDENV, FXRSTOR	All tags marked 11B in memory are set to 11B; all other tags are set according to the value in the corresponding floating-point register: 00B (nonzero), 01B (zero), or 10B (special).	Tags are read and interpreted, but not modified.

11.3. SAVING AND RESTORING THE MMX STATE AND REGISTERS

Because the MMX registers are aliased to the x87 FPU data registers, the MMX state can be saved to memory and restored from memory as follows:

- Execute an FSAVE, FNSAVE, or FXSAVE instruction to save the MMX state to memory. (The FXSAVE instruction also saves the state of the XMM and MXCSR registers.)
- Execute an FRSTOR or FXRSTOR instruction to restore the MMX state from memory. (The FXRSTOR instruction also restores the state of the XMM and MXCSR registers.)

The save and restore methods described above are required for operating systems (see Section 11.4., “Saving MMX State on Task or Context Switches”). Applications can in some cases save and restore only the MMX registers in the following way:

- Execute eight MOVQ instructions to save the contents of the MMX0 through MMX7 registers to memory. An EMMS instruction may then (optionally) be executed to clear the MMX state in the x87 FPU.
- Execute eight MOVQ instructions to read the saved contents of MMX registers from memory into the MMX0 through MMX7 registers.

NOTE

The IA-32 architecture does not support scanning the x87 FPU tag word and then only saving valid entries.

11.4. SAVING MMX STATE ON TASK OR CONTEXT SWITCHES

When switching from one task or context to another, it is often necessary to save the MMX state. As a general rule, if the existing task switching code for an operating system includes facilities for saving the state of the x87 FPU, these facilities can also be relied upon to save the MMX state, without rewriting the task switch code. This reliance is possible because the MMX state is aliased to the x87 FPU state (see Section 11.2., “The MMX State and MMX Register Aliasing”).

With the introduction of the FXSAVE and FXRSTOR instructions and of the SSE and SSE2 extensions to the IA-32 architecture, it is possible (and more efficient) to create state saving facilities in the operating system or executive that save the x87 FPU, MMX, SSE, and SSE2 state, all in one operation. Section 12.5., “Designing Operating System Facilities for Automatically Saving x87 FPU, MMX, SSE, and SSE2 state on Task or Context Switches” describes how to design such facilities. The techniques describes in Section 12.5. can be adapted to saving only the MMX and x87 FPU state if needed.

11.5. EXCEPTIONS THAT CAN OCCUR WHEN EXECUTING MMX INSTRUCTIONS

MMX instructions do not generate x87 FPU floating-point exceptions, nor do they affect the processor’s status flags in the EFLAGS register or the x87 FPU status word. The following exceptions can be generated during the execution of an MMX instruction:

- Exceptions during memory accesses:
 - Stack-segment fault (#SS).
 - General protection (#GP).
 - Page fault (#PF).
 - Alignment check (#AC), if alignment checking is enabled.
- System exceptions:
 - Invalid Opcode (#UD), if the EM flag in control register CR0 is set when an MMX instruction is executed (see Section 11.1., “Emulation of the MMX Instruction Set”).
 - Device not available (#NM), if an MMX instruction is executed when the TS flag in control register CR0 is set. (See Section 12.5.1., “Using the TS Flag to Control the Saving of the x87 FPU, MMX, SSE, and SSE2 State”).)
- Floating-point error (#MF). (See Section 11.5.1., “Effect of MMX Instructions on Pending x87 Floating-Point Exceptions”).)

- Other exceptions can occur indirectly due to the faulty execution of the exception handlers for the above exceptions.

11.5.1. Effect of MMX Instructions on Pending x87 Floating-Point Exceptions

If an x87 FPU floating-point exception is pending and the processor encounters an MMX instruction, the processor generates a x87 FPU floating-point error (#MF) prior to executing the MMX instruction, to allow the pending exception to be handled by the x87 FPU floating-point error exception handler. While this exception handler is executing, the x87 FPU state is maintained and is visible to the handler. Upon returning from the exception handler, the MMX instruction is executed, which will alter the x87 FPU state, as described in Section 11.2., “The MMX State and MMX Register Aliasing”.

11.6. DEBUGGING MMX CODE

The debug facilities of the IA-32 architecture operate in the same manner when executing MMX instructions as when executing other IA-32 architecture instructions.

To correctly interpret the contents of the MMX or x87 FPU registers from the FSAVE/FNSAVE or FXSAVE image in memory, a debugger needs to take account of the relationship between the x87 FPU register's logical locations relative to TOS and the MMX register's physical locations.

In the x87 FPU context, ST n refers to an x87 FPU register at location n relative to the TOS. However, the tags in the x87 FPU tag word are associated with the physical locations of the x87 FPU registers (R0 through R7). The MMX registers always refer to the physical locations of the registers (with MM0 through MM7 being mapped to R0 through R7). Figure 11-2 shows this relationship. Here, the inner circle refers to the physical location of the x87 FPU and MMX registers. The outer circle refers to the x87 FPU registers's relative location to the current TOS.

When the TOS equals 0 (case A in Figure 11-2), ST0 points to the physical location R0 on the floating-point stack. MM0 maps to ST0, MM1 maps to ST1, and so on.

When the TOS equals 2 (case B in Figure 11-2), ST0 points to the physical location R2. MM0 maps to ST6, MM1 maps to ST7, MM2 maps to ST0, and so on.

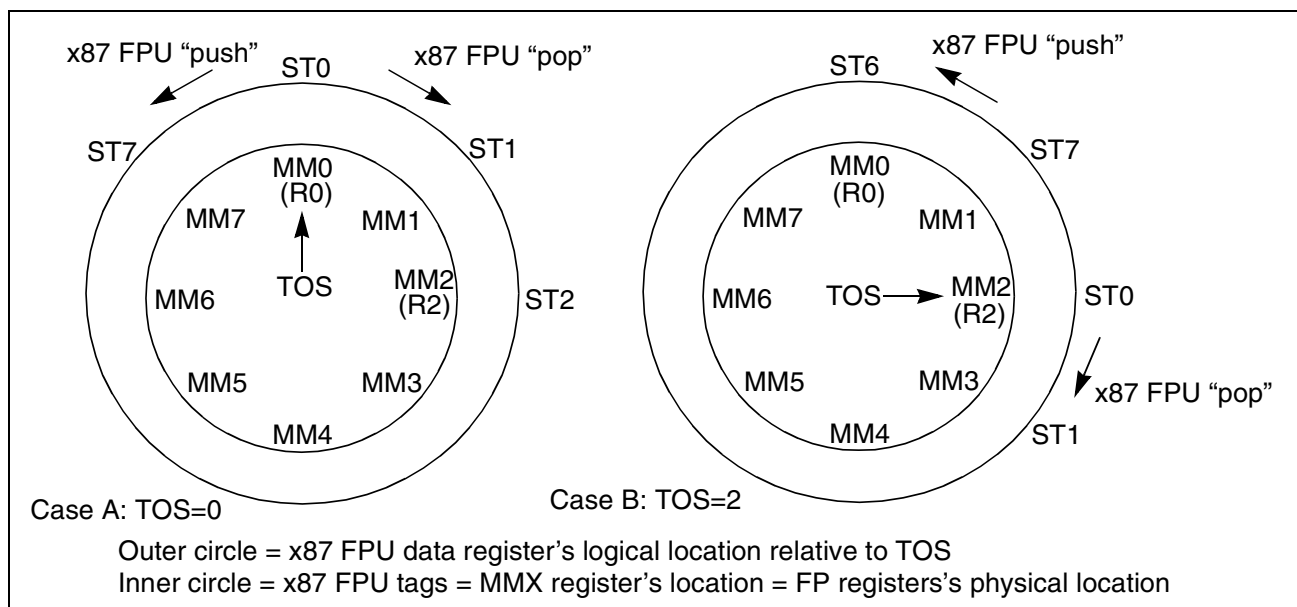


Figure 11-2. Mapping of MMX Registers to x87 FPU Data Register Stack

12

SSE and SSE2 System Programming

CHAPTER 12

SSE AND SSE2 SYSTEM PROGRAMMING

This chapter describes those features of the streaming SIMD extensions (SSE) and streaming SIMD extensions 2 (SSE2) that must be considered when designing or enhancing an operating system to support the Pentium III, Pentium 4, and Intel Xeon processors. It covers enabling the SSE and SSE2 extensions, providing operating system or executive support for the SSE and SSE2 extensions, SIMD floating-point exceptions, exception handling, and task (context) switching considerations.

12.1. PROVIDING OPERATING SYSTEM SUPPORT FOR THE SSE AND SSE2 EXTENSIONS

To the SSE and SSE2 extensions, the operating system or executive must provide support for initializing the processor to use the extensions, for handling the FXSAVE and FXRSTOR state saving instructions, and for handling SIMD floating-point exceptions. The following sections give some guidelines for providing this support in an operating-system or executive. Because SSE and SSE2 extensions share the same state and perform companion operations, these guidelines apply to both sets of extensions.

Chapter 11, Programming with the Streaming SIMD Extensions 2 (SSE2), in the IA-32 Intel Architecture Software Developer's Manual, Volume 1 discusses support for SSE and SSE2 extensions from the point of view of an applications program.

12.1.1. General Guidelines for Adding Support to an Operating System for the SSE and SSE2 Extensions

The following guidelines describe operations that an operating system or executive must perform to support the SSE and SSE2 extensions:

- Check that the processor supports the SSE and SSE2 extensions.
- Check that the processor supports the FXSAVE and FXRSTOR instructions.
- Provide an initialization procedure that initializes the SSE and SSE2 state.
- Provide support for the FXSAVE and FXRSTOR instructions.
- Provide support (if necessary) in non-numeric exception handlers for exceptions generated by the SSE and SSE2 instructions.
- Provide an exception handler for the SIMD floating-point exception (#XF).

The following sections describe how to implement each of these guidelines.

12.1.2. Checking for SSE and SSE2 Support

Before an operating system or executive attempts to use the SSE and/or SSE2 extensions, it should check that they are present on the processor. To make this check, execute the CUID instruction with an argument of 1 in the EAX register, and check that bit 25 (SSE) and/or bit 26 (SSE2) are set to 1.

NOTE

If the processor attempts to execute an unsupported SSE or a SSE2 instruction, the processor will generate an invalid-opcode exception (#UD).

12.1.3. Checking for Support for the FXSAVE and FXRSTOR Instructions

The FXSAVE and FXRSTOR instructions are not part of the SSE or SSE2 extensions, so a separate check must be made to insure that the processor supports them. To make this check, execute the CUID instruction with an argument of 1 in the EAX register, and check that bit 24 (FXSR) is set to 1.

12.1.4. Initialization of the SSE and SSE2 Extensions

The operating system or executive should carry out the following steps to set up the SSE and SSE2 extensions for use by applications programs.

1. Set bit 9 of CR4 (the OSFXSR bit) to 1. Setting this flag assumes that the operating system provides facilities for saving and restoring the SSE and SSE2 state using the FXSAVE and FXRSTOR instructions, respectively. These instructions are commonly used to save the SSE and SSE2 state during task switches and when invoking the SIMD floating-point exception (#XF) handler (see Section 12.4., “Saving SSE and SSE2 State on Task or Context Switches” and Section 12.1.6., “Providing an Handler for the SIMD Floating-Point Exception (#XF)”, respectively). If the processor does not support the FXSAVE and FXRSTOR instructions, attempting to set the OSFXSR flag will cause an invalid operand exception (#UD) to be generated.
2. Set bit 10 of CR4 (the OSXMMEXCPT bit) to 1. Setting this flag assumes that the operating system provides a SIMD floating-point exception (#XF) handler (see Section 12.1.6., “Providing an Handler for the SIMD Floating-Point Exception (#XF)”).

NOTE

The OSFXSR and OSXMMEXCPT bits in control register CR4 must be set by the operating system. The processor has no other way of detecting operating-system support for the FXSAVE and FXRSTOR instructions or for handling SIMD floating-point exceptions.

3. Clear the EM flag (bit 2) of control register CR0. This action disables emulation of the x87 FPU, which is required when executing SSE and SSE2 instructions (see Section 2.5., “Control Registers”).
4. Clear the MP flag (bit 1) of control register CR0. This setting is the required setting for all IA-32 processors that support the SSE and SSE2 extensions (see Section 9.2.1., “Configuring the x87 FPU Environment”).

Table 12-1 shows the actions of the processor when an SSE or SSE2 instruction is executed, depending on the settings of the OSFXSR and OSXMMEXCPT flags in control register CR4, the SSE and SSE2 feature flags returned with the CPUID instructions, and the EM, MP, and TS flags in control register CR0.

Table 12-1. Action Taken for Combinations of OSFXSR, OSXMMEXCPT, SSE, SSE2, EM, MP, and TS¹

CR4		CPUID		CR0 Flags			Action
OSFXSR	OSXMMEXCPT	SSE	SSE2	EM	MP ₂	TS	
0	X ³	X	X	X	1	X	#UD exception.
1	X	0	0	X	1	X	#UD exception.
1	X	1	1	1	1	X	#UD exception.
1	0	1	1	0	1	0	Execute instruction; #UD exception if unmasked SIMD floating-point exception is detected.
1	1	1	1	0	1	0	Execute instruction; #XF exception if unmasked SIMD floating-point exception is detected.
1	X	1	1	0	1	1	#NM exception.

Note:

1. For execution of any SSE or SSE2 instructions except the PAUSE, PREFETCH_h, SFENCE, LFENCE, MFENCE, MOVNTI, and CLFLUSH instructions.
2. For processors that support the MMX instructions, the MP flag should be set.
3. X—Don't care.

The SIMD floating-point exception mask bits (bits 7 through 12), the flush-to-zero flag (bit 15), the denormals-are-zero flag (bit 6), and the rounding control field (bits 13 and 14) in the MXCSR register should be left in their default values of 0. This permits the application to determine how these features are to be used.

12.1.5. Providing Non-Numeric Exception Handlers for Exceptions Generated by the SSE and SSE2 Instructions

The SSE and SSE2 instructions can generate the same type of memory access exceptions (such as, page fault, segment not present, and limit violations) and other non-numeric exceptions as other IA-32 architecture instructions can.

Ordinarily, existing exception handlers can handle these and other non-numeric exceptions without any code modification. However, depending on the mechanisms used in existing exception handlers, some modifications might need to be made.

The SSE and SSE2 extensions can generate the non-numeric exceptions listed below:

- Memory Access Exceptions.
 - Invalid opcode (#UD).
 - Stack-segment fault (#SS).
 - General protection (#GP). Executing most SSE and SSE2 instruction with an unaligned 128-bit memory reference generates a general-protection exception. (The MOVUPS and MOVUPD instructions allow unaligned loads or stores of 128-bit memory locations, without generating a general-protection exception.) A 128-bit reference within the stack segment that is not aligned to a 16-byte boundary will also generate a general-protection exception, instead a stack-segment fault exception (#SS).
 - Page fault (#PF).
 - Alignment check (#AC). When enabled, this type of alignment check operates on operands that are less than 128-bits in size: 16-bit, 32-bit, and 64-bit. To enable the generation of alignment check exceptions, the following things must be done:
 - The AM flag (bit 18 of control register CR0) must be set
 - The AC flag (bit 18 of the EFLAGS register) must be set
 - The CPL must be 3.If alignment check exceptions are enabled, 16-bit, 32-bit, and 64-bit misalignment will be detected for the MOVUPD and MOVUPS instructions, but detection of 128-bit misalignment is not guaranteed and may vary with implementation.
- System Exceptions:
 - Invalid-opcode exception (#UD). This exception is generated when executing SSE and SSE2 instructions under the following conditions:
 - The SSE and/or SSE2 feature flags returned by the CPUID instruction are set to 0. These flags are located in bits 25 and 26, respectively, of the EAX register. (This condition does not affect the CLFLUSH instruction.)
 - The CLFSH feature flag returned by the CPUID instruction are set to 0. This flag is located in bit 19 of the EAX register. This exception condition only pertains to the execution of the CLFLUSH instruction.
 - The EM flag (bit 2) in control register CR0 is set to 1, regardless of the value of TS flag (bit 3) of CR0. (This condition does not affect the PAUSE, PREFETCHh, MOVNTI, SFENCE, LFENCE, MFENSE, and CLFLUSH instructions.)

- The OSFXSR flag (bit 9) in control register CR4 is set to 0. (This condition does not affect the PAVGB, PAVGW, PEXTRW, PINSRW, PMAWSW, PMAWUB, PMINSW, PMINUB, PMOVMSKB, PMULHUW, PSADBW, PSHUFW, MASKMOVQ, MOVNTQ, MOVNTI, PAUSE, PREFETCHh, SFENCE, LFENCE, MFENCE, and CLFLUSH instructions.)
 - Executing a instruction that causes a SIMD floating-point exception when the OSXMMEXCPT flag (bit 10) in control register CR4 is set to 0 (see Section 12.5.1., “Using the TS Flag to Control the Saving of the x87 FPU, MMX, SSE, and SSE2 State”).
- Device not available (#NM). This exception is generated when executing SSE and SSE2 instruction when the TS flag (bit 3) of CR0 is set to 1.

Other exceptions can occur indirectly due to faulty execution of the above exceptions.

12.1.6. Providing an Handler for the SIMD Floating-Point Exception (#XF)

The SSE and SSE2 instructions do not generate any numeric exceptions on packed integer operations; however, they can generate the following numeric (SIMD floating-point) exceptions on packed and scalar single-precision and double-precision floating-point operations.

- Invalid operation (#I)
- Divide-by-zero (#Z)
- Denormal operand (#D)
- Numeric overflow (#O)
- Numeric underflow (#U)
- Inexact result (Precision) (#P)

These SIMD floating-point exceptions (with the exception of the denormal operand exception) are defined in the IEEE Standard 754 for Binary Floating-Point Arithmetic and represent the same conditions that cause x87 FPU floating-point error exceptions (#MF) to be generated for x87 FPU instructions.

Each of these SIMD floating-point exceptions can be masked, in which case the processor returns a reasonable result to the destination operand, without invoking an exception handler. However, if any of these exceptions are left unmasked, detection of the exception condition results in a SIMD floating-point exception (#XF) being generated (see Chapter 5, “Interrupt 19—SIMD Floating-Point Exception (#XF)”).

To handle unmasked SIMD floating-point exceptions, the operating system or executive must provide an exception handler. The section titled “SSE and SSE2 SIMD Floating-Point Exceptions” in Chapter 11 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, describes the SIMD floating-point exception classes and gives suggestions for writing an exception handler to handle them.

To indicate that the operating system provides a handler for SIMD floating-point exceptions (#XF), the OSXMMEXCPT flag (bit 10) must be set in control register CR0.

12.1.6.1. NUMERIC ERROR FLAG AND IGNNE#

SSE and SSE2 extensions ignore the NE flag in control register CR0 (that is, treats it as if it were always set) and the IGNNE# pin. When an unmasked SIMD floating-point exception is detected, it is always reported by generating a SIMD floating-point exception (#XF).

12.2. EMULATION OF THE SSE AND SSE2 EXTENSIONS

The IA-32 architecture does not support emulation of the SSE and SSE2 instructions, as it does the x87 FPU instructions. The EM flag in control register CR0 (provided to invoke emulation of x87 FPU instructions) cannot be used to invoke emulation of SSE and SSE2 instructions. If an SSE or SSE2 instruction is executed when the EM flag is set, an invalid opcode exception (#UD) is generated (see Table 12-1).

12.3. SAVING AND RESTORING THE SSE AND SSE2 STATE

The SSE, and SSE2 state consists of the state of the XMM and MXCSR registers. The recommended method of saving and restoring this state is as follows:

- Execute an FXSAVE instruction to save the state of the XMM and MXCSR registers to memory.
- Execute an FXRSTOR instruction to restore the state of the XMM and MXCSR registers from the image saved in memory by the FXSAVE instruction.

This save and restore method is required for operating systems (see Section 12.5., “Designing Operating System Facilities for Automatically Saving x87 FPU, MMX, SSE, and SSE2 state on Task or Context Switches”). Applications can in some cases save only the XMM and MXCSR registers in the following way:

- Execute eight MOVDQ instructions to save the contents of the XMM0 through XMM7 registers to memory.
- Execute a STMXCSR instruction to save the state of the MXCSR register to memory.

Applications can restore only the XMM and MXCSR registers in the following way:

- Execute eight MOVDQ instructions to read the saved contents of XMM registers from memory into the XMM0 through XMM7 registers.
- Execute a LDMXCSR instruction to restore the state of the MXCSR register from memory.

12.4. SAVING SSE AND SSE2 STATE ON TASK OR CONTEXT SWITCHES

When switching from one task or context to another, it is often necessary to save the SSE and SSE2 state. The FXSAVE and FXRSTOR instructions provide a simple method of saving and restoring this state (as described in Section 12.3., “Saving and Restoring the SSE and SSE2 State”). These instructions offer the added benefit of saving the x87 FPU and MMX state as well, which provides operating system or executive procedures with a convenient method of saving and restoring the complete SSE, SSE2, MMX, and x87 FPU state on task or context switches. Guidelines for writing such procedures are given in the following section, Section 12.5., “Designing Operating System Facilities for Automatically Saving x87 FPU, MMX, SSE, and SSE2 state on Task or Context Switches”.

12.5. DESIGNING OPERATING SYSTEM FACILITIES FOR AUTOMATICALLY SAVING X87 FPU, MMX, SSE, AND SSE2 STATE ON TASK OR CONTEXT SWITCHES

The x87 FPU, MMX, SSE, and SSE2 state consists of the state of the x87 FPU, MMX, XMM, and MXCSR registers. The FXSAVE and FXRSTOR instructions provide a simple and fast method of saving and restoring this entire state. If task or context switching facilities are already implemented in an operating system or executive that uses the FSAVE/FNSAVE and FRSTOR instructions to save the x87 FPU and MMX state, these facilities can often be extended to also save and restore the SSE and SSE2 state by substituting the FXSAVE and FXRSTOR instructions for the FSAVE/FNSAVE and FRSTOR instructions.

In cases where task or context switching facilities must be written from scratch, several approaches can be taken for using the FXSAVE and FXRSTOR instructions to save and restore the x87 FPU, MMX, SSE, and SSE2 state:

- The operating system can require that applications that are intended to be run as tasks take responsibility for saving the state of the x87 FPU, MMX, XMM, and MXCSR registers prior to a task suspension during a task switch and for restoring the registers when the task is resumed. This approach is appropriate for cooperative multitasking operating systems, where the application has control over (or is able to determine) when a task switch is about to occur and can save state prior to the task switch.
- The operating system can take the responsibility for automatically saving the x87 FPU, MMX, XMM, and MXCSR registers as part of the task switch process (using an FXSAVE instruction) and automatically restoring the state of the registers when a suspended task is resumed (using an FXRSTOR instruction). Here, the x87 FPU, MMX, SSE, and SSE2 state must be saved as part of the task state. This approach is appropriate for preemptive multitasking operating systems, where the application cannot know when it is going to be preempted and cannot prepare in advance for task switching. Here, the operating system is responsible for saving and restoring the task and the x87 FPU, MMX, SSE, and SSE2 state when necessary.

- The operating system can take the responsibility for saving the x87 FPU, MMX, XMM, and MXCSR registers as part of the task switch process, but delay the saving of the MMX and x87 FPU state until an x87 FPU, MMX, SSE, or SSE2 instruction is actually executed by the new task. Using this approach, the x87 FPU, MMX, SSE, and SSE2 state is saved only if an x87 FPU, MMX, SSE, or SSE2 instruction needs to be executed in the new task. (See Section 12.5.1., “Using the TS Flag to Control the Saving of the x87 FPU, MMX, SSE, and SSE2 State”, for more information on this technique for saving the x87 FPU, MMX, SSE, and SSE2 state.)

12.5.1. Using the TS Flag to Control the Saving of the x87 FPU, MMX, SSE, and SSE2 State

Saving the x87 FPU, MMX, SSE, and SSE2 state using an FXSAVE instruction requires some processor overhead. If a task being switched to will not access the x87 FPU, MMX, XMM, and MXCSR registers, this overhead can be avoided by not automatically saving the state of these registers on a task switch.

The TS flag in control register CR0 is provided to allow the operating system to delay saving the x87 FPU, MMX, SSE, and SSE2 state until an instruction that would actually accessed this state is encountered in the new task. When the TS flag is set, the processor monitors the instruction stream for x87 FPU, MMX, SSE, and SSE2 instructions. When the processor detects one of these instruction, it raises a device-not-available exception (#NM) prior to executing the instruction. The device-not-available exception handler can then be used to save the x87 FPU, MMX, SSE, and SSE2 state for the previous task (using an FXSAVE instruction) and load the x87 FPU, MMX, SSE, and SSE2 state for the current task (using an FXRSTOR instruction). If the task never encounters an x87 FPU, MMX, SSE, or SSE2 instruction, the device-not-available exception will not be raised and the x87 FPU, MMX, SSE, and SSE2 state will not be saved unnecessarily.

The TS flag can be set either explicitly (by executing a MOV instruction to control register CR0) or implicitly (using the IA-32 architecture’s native task switching mechanism). When the native task switching mechanism is used, the processor automatically sets the TS flag on a task switch. After the device-not-available handler has saved the x87 FPU, MMX, SSE, and SSE2 state, it should execute the CLTS instruction to clear the TS flag in CR0.

Figure 12-1 gives an example of an operating system that implements x87 FPU, MMX, SSE, and SSE2 state saving using the TS flag. In this example, task A is the currently running task and task B is the task being switched to.

The operating system maintains a save area for the x87 FPU, MMX, SSE, and SSE2 state for each task and defines a variable (x87_MMX_SSE_SSE2_StateOwner) that indicates which task “owns” the state. In this example, task A is the current x87 FPU, MMX, SSE, and SSE2 state owner.

On a task switch, the operating system task switching code must execute the following pseudo-code to set the TS flag according to who is the current owner of the x87 FPU, MMX, SSE, and SSE2 state. If the new task (task B in this example) is not the current owner of this state, the TS flag is set to 1; otherwise, it is set to 0.

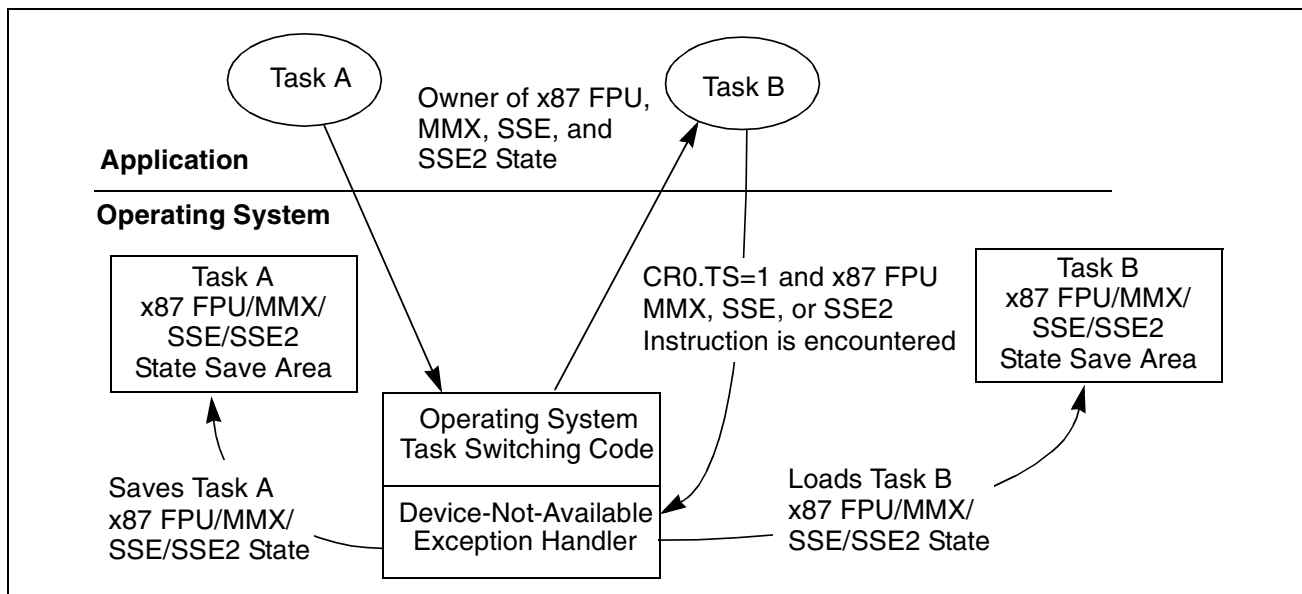


Figure 12-1. Example of Saving the x87 FPU, MMX, SSE, and SSE2 State During an Operating-System Controlled Task Switch

```

IF Task_Being_Switched_To ≠ x87FPU_MMX_SSE_SSE2_StateOwner
THEN
    CR0.TS ← 1;
ELSE
    CR0.TS ← 0;
FI;

```

If a new task attempts to access an x87 FPU, MMX, XMM, or MXCSR registers while the TS flag is set to 1, a device-not-available exception (#NM) is generated and the device-not-available exception handler executes the following pseudo-code.

```

FSAVE "To x87FPU/MMX/SSE/SSE2 State Save Area for Current
      x87FPU_MMX_SSE_SSE2_StateOwner";
FRSTOR "x87FPU/MMX/SSE/SSE2 State From Current Task's x87FPU/MMX/SSE/SSE2
      State Save Area";
x87FPU_MMX_SSE_SSE2_StateOwner ← Current_Task;
CR0.TS ← 0;

```

This exception handler code performs the following tasks:

- Saves the x87 FPU, MMX, XMM, or MXCSR registers in the state save area for the current owner of the x87 FPU, MMX, SSE, and SSE2 state.
- Restores the x87 FPU, MMX, XMM, or MXCSR registers from the new task's save area for the x87 FPU, MMX, SSE, and SSE2 state.
- Updates the current x87 FPU, MMX, SSE, and SSE2 state owner to be the current task.
- Clears the TS flag.

13

System Management

CHAPTER 13

SYSTEM MANAGEMENT

This chapter describes the two aspects of IA-32 architecture used to manage system resources: system management mode (SMM) and the thermal monitoring facilities.

SMM provides an alternate operating environment that can be used to monitor and manage various system resources for more efficient energy usage, to control system hardware, and/or to run proprietary code. It was introduced into the IA-32 architecture in the Intel386 SL processor (a mobile specialized version of the Intel386 processor). It is also available in the Pentium 4, Intel Xeon, P6 family, and Pentium and Intel486 processors (beginning with the enhanced versions of the Intel486 SL and Intel486 processors). For a detailed description of the hardware that supports SMM, see the developer's manuals for each of the IA-32 processors.

The thermal monitoring facilities enable monitoring and controlling the core temperature of an IA-32 processor. These facilities were introduced in the P6 family processors and extended in the Pentium 4, Intel Xeon and Pentium M processors.

13.1. SYSTEM MANAGEMENT MODE OVERVIEW

SMM is a special-purpose operating mode provided for handling system-wide functions like power management, system hardware control, or proprietary OEM-designed code. It is intended for use only by system firmware, not by applications software or general-purpose systems software. The main benefit of SMM is that it offers a distinct and easily isolated processor environment that operates transparently to the operating system or executive and software applications.

When SMM is invoked through a system management interrupt (SMI), the processor saves the current state of the processor (the processor's context), then switches to a separate operating environment contained in system management RAM (SMRAM). While in SMM, the processor executes SMI handler code to perform operations such as powering down unused disk drives or monitors, executing proprietary code, or placing the whole system in a suspended state. When the SMI handler has completed its operations, it executes a resume (RSM) instruction. This instruction causes the processor to reload the saved context of the processor, switch back to protected or real mode, and resume executing the interrupted application or operating-system program or task.

The following SMM mechanisms make it transparent to applications programs and operating systems:

- The only way to enter SMM is by means of an SMI.
- The processor executes SMM code in a separate address space (SMRAM) that can be made inaccessible from the other operating modes.
- Upon entering SMM, the processor saves the context of the interrupted program or task.

- All interrupts normally handled by the operating system are disabled upon entry into SMM.
- The RSM instruction can be executed only in SMM.

SMM is similar to real-address mode in that there are no privilege levels or address mapping. An SMM program can address up to 4 GBytes of memory and can execute all I/O and applicable system instructions. See Section 13.5., “SMI Handler Execution Environment”, for more information about the SMM execution environment.

NOTE

The physical address extension (PAE) mechanism available in the P6 family processors is not supported when a processor is in SMM.

13.2. SYSTEM MANAGEMENT INTERRUPT (SMI)

The only way to enter SMM is by signaling an SMI through the SMI# pin on the processor or through an SMI message received through the APIC bus. The SMI is a nonmaskable external interrupt that operates independently from the processor’s interrupt- and exception-handling mechanism and the local APIC. The SMI takes precedence over an NMI and a maskable interrupt. SMM is non-reentrant; that is, the SMI is disabled while the processor is in SMM.

NOTE

In the Pentium 4, Intel Xeon, and P6 family processors, when a processor that is designated as an application processor during an MP initialization sequence is waiting for a startup IPI (SIPI), it is in a mode where SMIs are masked. However if a SMI is received while an application processor is in the wait for SIPI mode, the SMI will be pended. The processor then responds on receipt of a SIPI by immediately servicing the pended SMI and going into SMM before handling the SIPI.

13.3. SWITCHING BETWEEN SMM AND THE OTHER PROCESSOR OPERATING MODES

Figure 2-2 shows how the processor moves between SMM and the other processor operating modes (protected, real-address, and virtual-8086). Signaling an SMI while the processor is in real-address, protected, or virtual-8086 modes always causes the processor to switch to SMM. Upon execution of the RSM instruction, the processor always returns to the mode it was in when the SMI occurred.

13.3.1. Entering SMM

The processor always handles an SMI on an architecturally defined “interruptible” point in program execution (which is commonly at an IA-32 architecture instruction boundary). When the processor receives an SMI, it waits for all instructions to retire and for all stores to complete. The processor then saves its current context in SMRAM (see Section 13.4., “SMRAM”), enters SMM, and begins to execute the SMI handler.

Upon entering SMM, the processor signals external hardware that SMM handling has begun. The signaling mechanism used is implementation dependent. For the P6 family processors, an SMI acknowledge transaction is generated on the system bus and the multiplexed status signal EXF4 is asserted each time a bus transaction is generated while the processor is in SMM. For the Pentium and Intel486 processors, the SMIACK# pin is asserted.

An SMI has a greater priority than debug exceptions and external interrupts. Thus, if an NMI, maskable hardware interrupt, or a debug exception occurs at an instruction boundary along with an SMI, only the SMI is handled. Subsequent SMI requests are not acknowledged while the processor is in SMM. The first SMI interrupt request that occurs while the processor is in SMM (that is, after SMM has been acknowledged to external hardware) is latched and serviced when the processor exits SMM with the RSM instruction. The processor will latch only one SMI while in SMM.

See Section 13.5., “SMI Handler Execution Environment”, for a detailed description of the execution environment when in SMM.

13.3.2. Exiting From SMM

The only way to exit SMM is to execute the RSM instruction. The RSM instruction is only available to the SMI handler; if the processor is not in SMM, attempts to execute the RSM instruction result in an invalid-opcode exception (#UD) being generated.

The RSM instruction restores the processor’s context by loading the state save image from SMRAM back into the processor’s registers. The processor then returns an SMIACK transaction on the system bus and returns program control back to the interrupted program.

Upon successful completion of the RSM instruction, the processor signals external hardware that SMM has been exited. For the P6 family processors, an SMI acknowledge transaction is generated on the system bus and the multiplexed status signal EXF4 is no longer generated on bus cycles. For the Pentium and Intel486 processors, the SMIACK# pin is desasserted.

If the processor detects invalid state information saved in the SMRAM, it enters the shutdown state and generates a special bus cycle to indicate it has entered shutdown state. Shutdown happens only in the following situations:

- A reserved bit in control register CR4 is set to 1 on a write to CR4. This error should not happen unless SMI handler code modifies reserved areas of the SMRAM saved state map (see Section 13.4.1., “SMRAM State Save Map”). Note that CR4 is saved in the state map in a reserved location and cannot be read or modified in its saved state.

- An illegal combination of bits is written to control register CR0, in particular PG set to 1 and PE set to 0, or NW set to 1 and CD set to 0.
- (For the Pentium and Intel486 processors only.) If the address stored in the SMBASE register when an RSM instruction is executed is not aligned on a 32-KByte boundary. This restriction does not apply to the P6 family processors.

In the shutdown state, Intel processors stop executing instructions until a RESET#, INIT# or NMI# is asserted. Processors do recognize the FLUSH# signal in the shutdown state. While Pentium family processors recognize the SMI# signal in shutdown state, P6 family and Intel486 processors do not. Intel does not support using SMI# to recover from shutdown states for any processor family; the response of processors in this circumstance is not well defined.

If the processor is in the HALT state when the SMI is received, the processor handles the return from SMM slightly differently (see Section 13.10., “Auto HALT Restart”). Also, the SMBASE address can be changed on a return from SMM (see Section 13.11., “SMBASE Relocation”).

13.4. SMRAM

While in SMM, the processor executes code and stores data in the SMRAM space. The SMRAM space is mapped to the physical address space of the processor and can be up to 4 GBytes in size. The processor uses this space to save the context of the processor and to store the SMI handler code, data and stack. It can also be used to store system management information (such as the system configuration and specific information about powered-down devices) and OEM-specific information.

The default SMRAM size is 64 KBytes beginning at a base physical address in physical memory called the SMBASE (see Figure 13-1). The SMBASE default value following a hardware reset is 30000H. The processor looks for the first instruction of the SMI handler at the address [SMBASE + 8000H]. It stores the processor's state in the area from [SMBASE + FE00H] to [SMBASE + FFFFH]. See Section 13.4.1., “SMRAM State Save Map”, for a description of the mapping of the state save area.

The system logic is minimally required to decode the physical address range for the SMRAM from [SMBASE + 8000H] to [SMBASE + FFFFH]. A larger area can be decoded if needed. The size of this SMRAM can be between 32 KBytes and 4 GBytes.

The location of the SMRAM can be changed by changing the SMBASE value (see Section 13.11., “SMBASE Relocation”). It should be noted that all processors in a multiple-processor system are initialized with the same SMBASE value (30000H). Initialization software must sequentially place each processor in SMM and change its SMBASE so that it does not overlap those of other processors.

The actual physical location of the SMRAM can be in system memory or in a separate RAM memory. The processor generates an SMI acknowledge transaction (P6 family processors) or asserts the SMIACK# pin (Pentium and Intel486 processors) when the processor receives an SMI (see Section 13.3.1., “Entering SMM”).

System logic can use the SMI acknowledge transaction or the assertion of the SMIACK# pin to decode accesses to the SMRAM and redirect them (if desired) to specific SMRAM memory. If

a separate RAM memory is used for SMRAM, system logic should provide a programmable method of mapping the SMRAM into system memory space when the processor is not in SMM. This mechanism will enable start-up procedures to initialize the SMRAM space (that is, load the SMI handler) before executing the SMI handler during SMM.

13.4.1. SMRAM State Save Map

When the processor initially enters SMM, it writes its state to the state save area of the SMRAM. The state save area begins at [SMBASE + 8000H + 7FFFH] and extends down to [SMBASE + 8000H + 7E00H]. Table 13-1 shows the state save map. The offset in column 1 is relative to the SMBASE value plus 8000H. Reserved spaces should not be used by software.

Some of the registers in the SMRAM state save area (marked YES in column 3) may be read and changed by the SMI handler, with the changed values restored to the processor registers by the RSM instruction. Some register images are read-only, and must not be modified (modifying these registers will result in unpredictable behavior). An SMI handler should not rely on any values stored in an area that is marked as reserved.

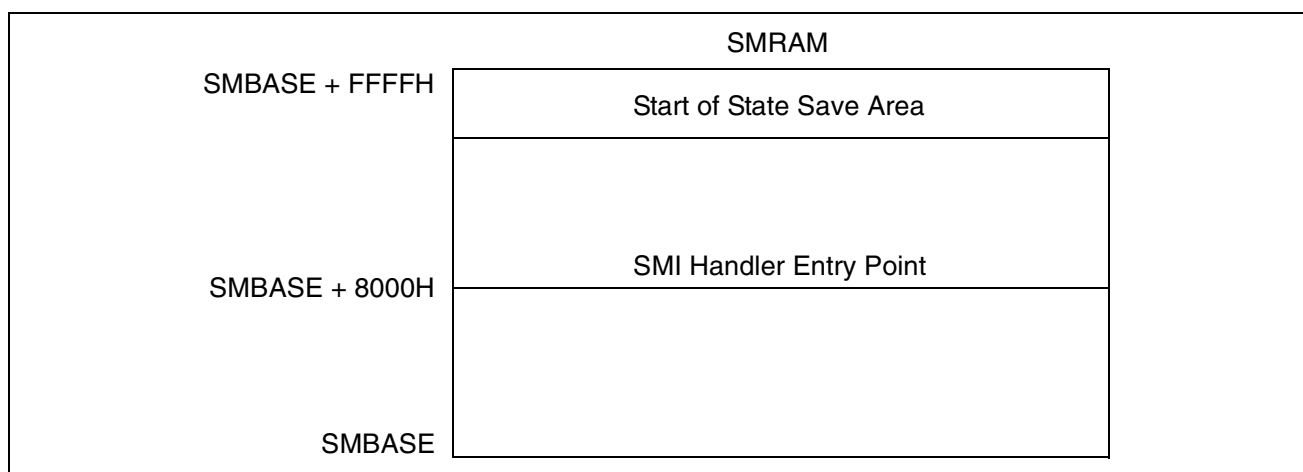


Figure 13-1. SMRAM Usage

Table 13-1. SMRAM State Save Map

Offset (Added to SMBASE + 8000H)	Register	Writable?
7FFCH	CR0	No
7FF8H	CR3	No
7FF4H	EFLAGS	Yes
7FF0H	EIP	Yes
7FECH	EDI	Yes
7FE8H	ESI	Yes
7FE4H	EBP	Yes

Table 13-1. SMRAM State Save Map (Contd.)

Offset (Added to SMBASE + 8000H)	Register	Writable?
7FE0H	ESP	Yes
7FDCH	EBX	Yes
7FD8H	EDX	Yes
7FD4H	ECX	Yes
7FD0H	EAX	Yes
7FCCH	DR6	No
7FC8H	DR7	No
7FC4H	TR*	No
7FC0H	Reserved	No
7FBCH	GS*	No
7FB8H	FS*	No
7FB4H	DS*	No
7FB0H	SS*	No
7FACH	CS*	No
7FA8H	ES*	No
7FA7H - 7F98H	Reserved	No
7F94H	Reserved	No
7F93H - 7F8CH	Reserved	No
7F88H	Reserved	No
7F87H - 7F04H	Reserved	No
7F02H	Auto HALT Restart Field (Word)	Yes
7F00H	I/O Instruction Restart Field (Word)	Yes
7EFCH	SMM Revision Identifier Field (Doubleword)	No
7EF8H	SMBASE Field (Doubleword)	Yes
7EF7H - 7E00H	Reserved	No

NOTE:

* Upper two bytes are reserved.

The following registers are saved (but not readable) and restored upon exiting SMM:

- Control register CR4. (This register is cleared to all 0s while in SMM).
- The hidden segment descriptor information stored in segment registers CS, DS, ES, FS, GS, and SS.

If an SMI request is issued for the purpose of powering down the processor, the values of all reserved locations in the SMM state save must be saved to nonvolatile memory.

The following state is not automatically saved and restored following an SMI and the RSM instruction, respectively:

- Debug registers DR0 through DR3.
- The x87 FPU registers.
- The MTRRs.
- Control register CR2.
- The model-specific registers (for the P6 family and Pentium processors) or test registers TR3 through TR7 (for the Pentium and Intel486 processors).
- The state of the trap controller.
- The machine-check architecture registers.
- The APIC internal interrupt state (ISR, IRR, etc.).
- The microcode update state.

If an SMI is used to power down the processor, a power-on reset will be required before returning to SMM, which will reset much of this state back to its default values. So an SMI handler that is going to trigger power down should first read these registers listed above directly, and save them (along with the rest of RAM) to nonvolatile storage. After the power-on reset, the continuation of the SMI handler should restore these values, along with the rest of the system's state. Anytime the SMI handler changes these registers in the processor, it must also save and restore them.

NOTE

A small subset of the MSRs (such as, the time-stamp counter and performance-monitoring counters) are not arbitrarily writable and therefore cannot be saved and restored. SMM-based power-down and restoration should only be performed with operating systems that do not use or rely on the values of these registers. Operating system developers should be aware of this fact and insure that their operating-system assisted power-down and restoration software is immune to unexpected changes in these register values.

13.4.2. SMRAM Caching

An IA-32 processor does not automatically write back and invalidate its caches before entering SMM or before exiting SMM. Because of this behavior, care must be taken in the placement of the SMRAM in system memory and in the caching of the SMRAM to prevent cache incoherence when switching back and forth between SMM and protected mode operation. Either of the following three methods of locating the SMRAM in system memory will guarantee cache coherency:

- Place the SRAM in a dedicated section of system memory that the operating system and applications are prevented from accessing. Here, the SRAM can be designated as cacheable (WB, WT, or WC) for optimum processor performance, without risking cache incoherence when entering or exiting SMM.
- Place the SRAM in a section of memory that overlaps an area used by the operating system (such as the video memory), but designate the SMRAM as uncacheable (UC). This method prevents cache access when in SMM to maintain cache coherency, but the use of uncacheable memory reduces the performance of SMM code.
- Place the SRAM in a section of system memory that overlaps an area used by the operating system and/or application code, but explicitly flush (write back and invalidate) the caches upon entering and exiting SMM mode. This method maintains cache coherency, but the incurs the overhead of two complete cache flushes.

For Pentium 4, Intel Xeon, and P6 family processors, a combination of the first two methods of locating the SMRAM is recommended. Here the SMRAM is split between an overlapping and a dedicated region of memory. Upon entering SMM, the SMRAM space that is accessed overlaps video memory (typically located in low memory). This SMRAM section is designated as UC memory. The initial SMM code then jumps to a second SMRAM section that is located in a dedicated region of system memory (typically in high memory). This SMRAM section can be cached for optimum processor performance.

For systems that explicitly flush the caches upon entering SMM (the third method described above), the cache flush can be accomplished by asserting the FLUSH# pin at the same time as the request to enter SMM (generally initiated by asserting the SMI# pin). The priorities of the FLUSH# and SMI# pins are such that the FLUSH# is serviced first. To guarantee this behavior, the processor requires that the following constraints on the interaction of FLUSH# and SMI# be met. In a system where the FLUSH# and SMI# pins are synchronous and the set up and hold times are met, then the FLUSH# and SMI# pins may be asserted in the same clock. In asynchronous systems, the FLUSH# pin must be asserted at least one clock before the SMI# pin to guarantee that the FLUSH# pin is serviced first.

Upon leaving SMM (for systems that explicitly flush the caches), the WBINVD instruction should be executed prior to leaving SMM to flush the caches.

NOTE

In systems based on the Pentium processor that use the FLUSH# pin to write back and invalidate cache contents before entering SMM, the processor will prefetch at least one cache line in between when the Flush Acknowledge cycle is run, and the subsequent recognition of SMI# and the assertion of SMIACK#. It is the obligation of the system to ensure that these lines are not cached by returning KEN# inactive to the Pentium processor.

13.5. SMI HANDLER EXECUTION ENVIRONMENT

After saving the current context of the processor, the processor initializes its core registers to the values shown in Table 13-2. Upon entering SMM, the PE and PG flags in control register CR0 are cleared, which places the processor in an environment similar to real-address mode. The differences between the SMM execution environment and the real-address mode execution environment are as follows:

- The addressable SMRAM address space ranges from 0 to FFFFFFFFH (4 GBytes). (The physical address extension (enabled with the PAE flag in control register CR4) is not supported in SMM.)
- The normal 64-KByte segment limit for real-address mode is increased to 4 GBytes.
- The default operand and address sizes are set to 16 bits, which restricts the addressable SMRAM address space to the 1-MByte real-address mode limit for native real-address-mode code. However, operand-size and address-size override prefixes can be used to access the address space beyond the 1-MByte.

Table 13-2. Processor Register Initialization in SMM

Register	Contents
General-purpose registers	Undefined
EFLAGS	00000002H
EIP	00008000H
CS selector	SMM Base shifted right 4 bits (default 3000H)
CS base	SMM Base (default 30000H)
DS, ES, FS, GS, SS Selectors	0000H
DS, ES, FS, GS, SS Bases	000000000H
DS, ES, FS, GS, SS Limits	0FFFFFFFFH
CR0	PE, EM, TS and PG flags set to 0; others unmodified
CR4	Cleared to zero
DR6	Undefined
DR7	00000400H

- Near jumps and calls can be made to anywhere in the 4-GByte address space if a 32-bit operand-size override prefix is used. Due to the real-address-mode style of base-address formation, a far call or jump cannot transfer control to a segment with a base address of more than 20 bits (1 MByte). However, since the segment limit in SMM is 4 GBytes, offsets into a segment that go beyond the 1-MByte limit are allowed when using 32-bit operand-size override prefixes. Any program control transfer that does not have a 32-bit operand-size override prefix truncates the EIP value to the 16 low-order bits.
- Data and the stack can be located anywhere in the 4-GByte address space, but can be accessed only with a 32-bit address-size override if they are located above 1 MByte. As with the code segment, the base address for a data or stack segment cannot be more than 20 bits.

The value in segment register CS is automatically set to the default of 30000H for the SMBASE shifted 4 bits to the right; that is, 3000H. The EIP register is set to 8000H. When the EIP value is added to shifted CS value (the SMBASE), the resulting linear address points to the first instruction of the SMI handler.

The other segment registers (DS, SS, ES, FS, and GS) are cleared to 0 and their segment limits are set to 4 GBytes. In this state, the SMRAM address space may be treated as a single flat 4-Gbyte linear address space. If a segment register is loaded with a 16-bit value, that value is then shifted left by 4 bits and loaded into the segment base (hidden part of the segment register). The limits and attributes are not modified.

Maskable hardware interrupts, exceptions, NMI interrupts, SMI interrupts, A20M interrupts, single-step traps, breakpoint traps, and INIT operations are inhibited when the processor enters SMM. Maskable hardware interrupts, exceptions, single-step traps, and breakpoint traps can be enabled in SMM if the SMM execution environment provides and initializes an interrupt table and the necessary interrupt and exception handlers (see Section 13.6., “Exceptions and Interrupts Within SMM”).

13.6. EXCEPTIONS AND INTERRUPTS WITHIN SMM

When the processor enters SMM, all hardware interrupts are disabled in the following manner:

- The IF flag in the EFLAGS register is cleared, which inhibits maskable hardware interrupts from being generated.
- The TF flag in the EFLAGS register is cleared, which disables single-step traps.
- Debug register DR7 is cleared, which disables breakpoint traps. (This action prevents a debugger from accidentally breaking into an SMM handler if a debug breakpoint is set in normal address space that overlays code or data in SMRAM.)
- NMI, SMI, and A20M interrupts are blocked by internal SMM logic. (See Section 13.7., “NMI Handling While in SMM”, for further information about how NMIs are handled in SMM.)

Software-invoked interrupts and exceptions can still occur, and maskable hardware interrupts can be enabled by setting the IF flag. Intel recommends that SMM code be written in so that it does not invoke software interrupts (with the INT *n*, INTO, INT 3, or BOUND instructions) or generate exceptions.

If the SMM handler requires interrupt and exception handling, an SMM interrupt table and the necessary exception and interrupt handlers must be created and initialized from within SMM. Until the interrupt table is correctly initialized (using the LIDT instruction), exceptions and software interrupts will result in unpredictable processor behavior.

The following restrictions apply when designing SMM interrupt and exception-handling facilities:

- The interrupt table should be located at linear address 0 and must contain real-address mode style interrupt vectors (4 bytes containing CS and IP).
- Due to the real-address mode style of base address formation, an interrupt or exception cannot transfer control to a segment with a base address of more than 20 bits.
- An interrupt or exception cannot transfer control to a segment offset of more than 16 bits (64 KBytes).
- When an exception or interrupt occurs, only the 16 least-significant bits of the return address (EIP) are pushed onto the stack. If the offset of the interrupted procedure is greater than 64 KBytes, it is not possible for the interrupt/exception handler to return control to that procedure. (One solution to this problem is for a handler to adjust the return address on the stack.)
- The SMBASE relocation feature affects the way the processor will return from an interrupt or exception generated while the SMI handler is executing. For example, if the SMBASE is relocated to above 1 MByte, but the exception handlers are below 1 MByte, a normal return to the SMI handler is not possible. One solution is to provide the exception handler with a mechanism for calculating a return address above 1 MByte from the 16-bit return address on the stack, then use a 32-bit far call to return to the interrupted procedure.
- If an SMI handler needs access to the debug trap facilities, it must insure that an SMM accessible debug handler is available and save the current contents of debug registers DR0 through DR3 (for later restoration). Debug registers DR0 through DR3 and DR7 must then be initialized with the appropriate values.
- If an SMI handler needs access to the single-step mechanism, it must insure that an SMM accessible single-step handler is available, and then set the TF flag in the EFLAGS register.
- If the SMI design requires the processor to respond to maskable hardware interrupts or software-generated interrupts while in SMM, it must ensure that SMM accessible interrupt handlers are available and then set the IF flag in the EFLAGS register (using the STI instruction). Software interrupts are not blocked upon entry to SMM, so they do not need to be enabled.

13.7. NMI HANDLING WHILE IN SMM

NMI interrupts are blocked upon entry to the SMI handler. If an NMI request occurs during the SMI handler, it is latched and serviced after the processor exits SMM. Only one NMI request will be latched during the SMI handler. If an NMI request is pending when the processor executes the RSM instruction, the NMI is serviced before the next instruction of the interrupted code sequenc. This assumes that NMIs were not blocked before the SMI occurred. If NMIs were blocked before the SMI occurred, they are blocked after execution of RSM.

Although NMI requests are blocked when the processor enters SMM, they may be enabled through software by executing an IRET/IRETD instruction. If the SMM handler requires the use of NMI interrupts, it should invoke a dummy interrupt service routine for the purpose of executing an IRET/IRETD instruction. Once an IRET/IRETD instruction is executed, NMI interrupt requests are serviced in the same “real mode” manner in which they are handled outside of SMM.

A special case can occur if an SMI handler nests inside an NMI handler and then another NMI occurs. During NMI interrupt handling, NMI interrupts are disabled, so normally NMI interrupts are serviced and completed with an IRET instruction one at a time. When the processor enters SMM while executing an NMI handler, the processor saves the SMRAM state save map but does not save the attribute to keep NMI interrupts disabled. Potentially, an NMI could be latched (while in SMM or upon exit) and serviced upon exit of SMM even though the previous NMI handler has still not completed. One or more NMIs could thus be nested inside the first NMI handler. The NMI interrupt handler should take this possibility into consideration.

Also, for the Pentium processor, exceptions that invoke a trap or fault handler will enable NMI interrupts from inside of SMM. This behavior is implementation specific for the Pentium processor and is not part the IA-32 architecture.

13.8. SAVING THE X87 FPU STATE WHILE IN SMM

In some instances (for example prior to powering down system memory when entering a 0-volt suspend state), it is necessary to save the state of the x87 FPU while in SMM. Care should be taken when performing this operation to insure that relevant x87 FPU state information is not lost. The safest way to perform this task is to place the processor in 32-bit protected mode before saving the x87 FPU state. The reason for this is as follows.

The FSAVE instruction saves the x87 FPU context in any of four different formats, depending on which mode the processor is in when FSAVE is executed (see Figures 8-9 through 8-12 in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*). When in SMM, by default, the 16-bit real-address mode format is used (shown in Figure 8-12). If an SMI interrupt occurs while the processor is in a mode other than 16-bit real-address mode, FSAVE and FRSTOR will be unable to save and restore all the relevant x87 FPU information, and this situation may result in a malfunction when the interrupted program is resumed. To avoid this problem, the processor should be in 32-bit protected mode when executing the FSAVE and FRSTOR instructions.

The following guidelines should be used when going into protected mode from an SMI handler to save and restore the x87 FPU state:

- Use the CUID instruction to insure that the processor contains an x87 FPU.
- Create a 32-bit code segment in SMRAM space that contains procedures or routines to save and restore the x87 FPU using the FSAVE and FRSTOR instructions, respectively. A GDT with an appropriate code-segment descriptor (D bit is set to 1) for the 32-bit code segment must also be placed in SMRAM.
- Write a procedure or routine that can be called by the SMI handler to save and restore the x87 FPU state. This procedure should do the following:
 - Place the processor in 32-bit protected mode as describe in Section 9.9.1., “Switching to Protected Mode”.
 - Execute a far JMP to the 32-bit code segment that contains the x87 FPU save and restore procedures.
 - Place the processor back in 16-bit real-address mode before returning to the SMI handler (see Section 9.9.2., “Switching Back to Real-Address Mode”).

The SMI handler may continue to execute in protected mode after the x87 FPU state has been saved and return safely to the interrupted program from protected mode. However, it is recommended that the handler execute primarily in 16- or 32-bit real-address mode.

13.9. SMM REVISION IDENTIFIER

The SMM revision identifier field is used to indicate the version of SMM and the SMM extensions that are supported by the processor (see Figure 13-2). The SMM revision identifier is written during SMM entry and can be examined in SMRAM space at offset 7EFCH. The lower word of the SMM revision identifier refers to the version of the base SMM architecture.

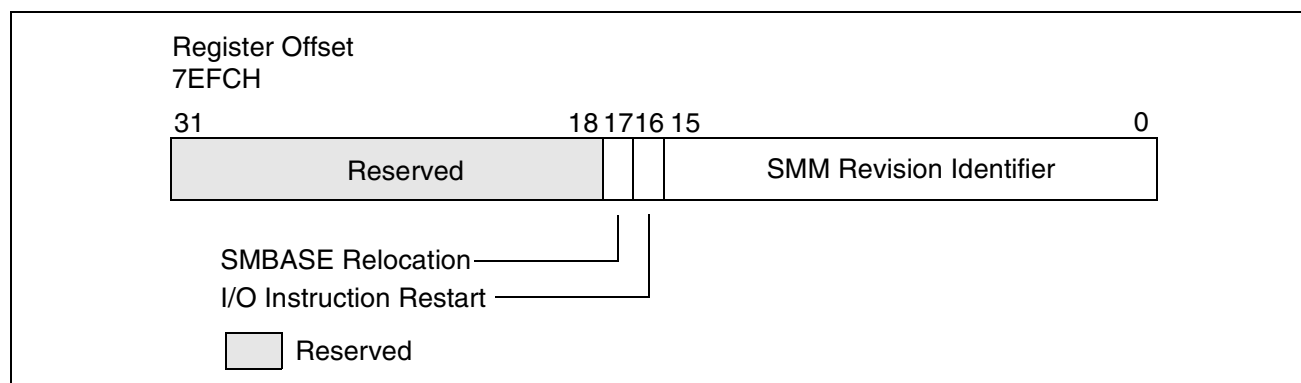


Figure 13-2. SMM Revision Identifier

The upper word of the SMM revision identifier refers to the extensions available. If the I/O instruction restart flag (bit 16) is set, the processor supports the I/O instruction restart (see Section 13.12., “I/O Instruction Restart”); if the SMBASE relocation flag (bit 17) is set, SMRAM base address relocation is supported (see Section 13.11., “SMBASE Relocation”).

13.10. AUTO HALT RESTART

If the processor is in a HALT state (due to the prior execution of a HLT instruction) when it receives an SMI, the processor records the fact in the auto HALT restart flag in the saved processor state (see Figure 13-3). (This flag is located at offset 7F02H and bit 0 in the state save area of the SMRAM.)

If the processor sets the auto HALT restart flag upon entering SMM (indicating that the SMI occurred when the processor was in the HALT state), the SMI handler has two options:

- It can leave the auto HALT restart flag set, which instructs the RSM instruction to return program control to the HLT instruction. This option in effect causes the processor to re-enter the HALT state after handling the SMI. (This is the default operation.)
- It can clear the auto HALT restart flag, which instructs the RSM instruction to return program control to the instruction following the HLT instruction.

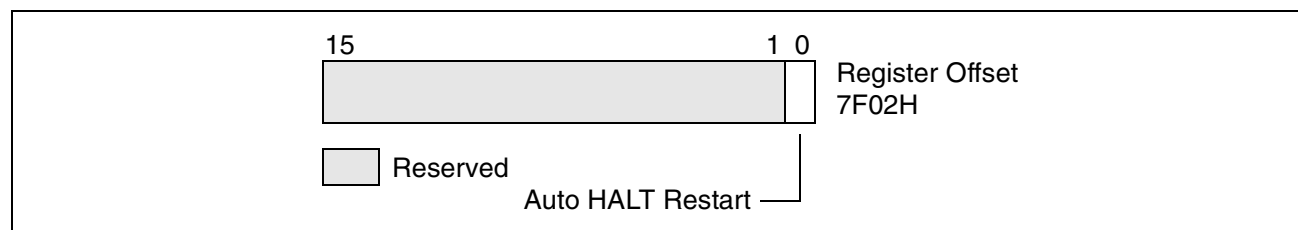


Figure 13-3. Auto HALT Restart Field

These options are summarized in Table 13-3. Note that if the processor was not in a HALT state when the SMI was received (the auto HALT restart flag is cleared), setting the flag to 1 will cause unpredictable behavior when the RSM instruction is executed.

Table 13-3. Auto HALT Restart Flag Values

Value of Flag After Entry to SMM	Value of Flag When Exiting SMM	Action of Processor When Exiting SMM
0	0	Returns to next instruction in interrupted program or task
0	1	Unpredictable
1	0	Returns to next instruction after HLT instruction
1	1	Returns to HALT state

If the HLT instruction is restarted, the processor will generate a memory access to fetch the HLT instruction (if it is not in the internal cache), and execute a HLT bus transaction. This behavior results in multiple HLT bus transactions for the same HLT instruction.

13.10.1. Executing the HLT Instruction in SMM

The HLT instruction should not be executed during SMM, unless interrupts have been enabled by setting the IF flag in the EFLAGS register. If the processor is halted in SMM, the only event that can remove the processor from this state is a maskable hardware interrupt or a hardware reset.

13.11. SMBASE RELOCATION

The default base address for the SMRAM is 30000H. This value is contained in an internal processor register called the SMBASE register. The operating system or executive can relocate the SMRAM by setting the SMBASE field in the saved state map (at offset 7EF8H) to a new value (see Figure 13-4). The RSM instruction reloads the internal SMBASE register with the value in the SMBASE field each time it exits SMM. All subsequent SMI requests will use the new SMBASE value to find the starting address for the SMI handler (at SMBASE + 8000H) and the SMRAM state save area (from SMBASE + FE00H to SMBASE + FFFFH). (The processor resets the value in its internal SMBASE register to 30000H on a RESET, but does not change it on an INIT.)

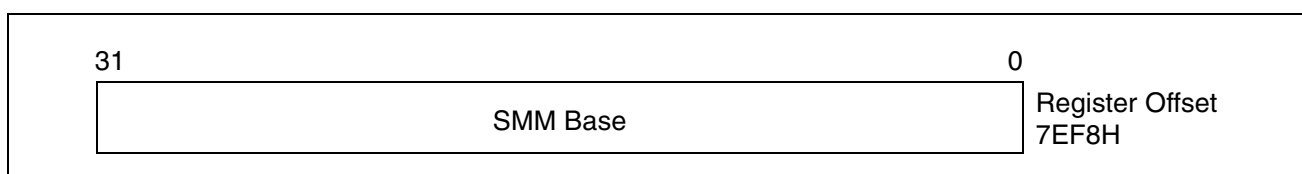


Figure 13-4. SMBASE Relocation Field

In multiple-processor systems, initialization software must adjust the SMBASE value for each processor so that the SMRAM state save areas for each processor do not overlap. (For Pentium and Intel486 processors, the SMBASE values must be aligned on a 32-KByte boundary or the processor will enter shutdown state during the execution of a RSM instruction.)

If the SMBASE relocation flag in the SMM revision identifier field is set, it indicates the ability to relocate the SMBASE (see Section 13.9., “SMM Revision Identifier”).

13.11.1. Relocating SMRAM to an Address Above 1 MByte

In SMM, the segment base registers can only be updated by changing the value in the segment registers. The segment registers contain only 16 bits, which allows only 20 bits to be used for a segment base address (the segment register is shifted left 4 bits to determine the segment base address). If SMRAM is relocated to an address above 1 MByte, software operating in real-address mode can no longer initialize the segment registers to point to the SMRAM base address (SMBASE).

The SMRAM can still be accessed by using 32-bit address-size override prefixes to generate an offset to the correct address. For example, if the SMBASE has been relocated to FFFFFFFH (immediately below the 16-MByte boundary) and the DS, ES, FS, and GS registers are still initialized to 0H, data in SMRAM can be accessed by using 32-bit displacement registers, as in the following example:

```
mov     esi,00FFxxxxH; 64K segment immediately below 16M
mov     ax,ds:[esi]
```

A stack located above the 1-MByte boundary can be accessed in the same manner.

13.12. I/O INSTRUCTION RESTART

If the I/O instruction restart flag in the SMM revision identifier field is set (see Section 13.9., “SMM Revision Identifier”), the I/O instruction restart mechanism is present on the processor. This mechanism allows an interrupted I/O instruction to be re-executed upon returning from SMM mode. For example, if an I/O instruction is used to access a powered-down I/O device, a chip set supporting this device can intercept the access and respond by asserting SMI#. This action invokes the SMI handler to power-up the device. Upon returning from the SMI handler, the I/O instruction restart mechanism can be used to re-execute the I/O instruction that caused the SMI.

The I/O instruction restart field (at offset 7F00H in the SMM state-save area, see Figure 13-5) controls I/O instruction restart. When an RSM instruction is executed, if this field contains the value FFH, then the EIP register is modified to point to the I/O instruction that received the SMI request. The processor will then automatically re-execute the I/O instruction that the SMI trapped. (The processor saves the necessary machine state to insure that re-execution of the instruction is handled coherently.)

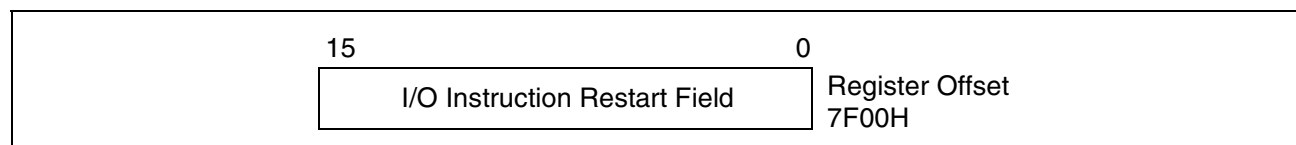


Figure 13-5. I/O Instruction Restart Field

If the I/O instruction restart field contains the value 00H when the RSM instruction is executed, then the processor begins program execution with the instruction following the I/O instruction. (When a repeat prefix is being used, the next instruction may be the next I/O instruction in the repeat loop.) Not re-executing the interrupted I/O instruction is the default behavior; the processor automatically initializes the I/O instruction restart field to 00H upon entering SMM. Table 13-4 summarizes the states of the I/O instruction restart field.

Table 13-4. I/O Instruction Restart Field Values

Value of Flag After Entry to SMM	Value of Flag When Exiting SMM	Action of Processor When Exiting SMM
00H 00H	00H FFH	Does not re-execute trapped I/O instruction. Re-executes trapped I/O instruction.

Note that the I/O instruction restart mechanism does not indicate the cause of the SMI. It is the responsibility of the SMI handler to examine the state of the processor to determine the cause of the SMI and to determine if an I/O instruction was interrupted and should be restarted upon exiting SMM. If an SMI interrupt is signaled on a non-I/O instruction boundary, setting the I/O instruction restart field to FFH prior to executing the RSM instruction will likely result in a program error.

13.12.1. Back-to-Back SMI Interrupts When I/O Instruction Restart Is Being Used

If an SMI interrupt is signaled while the processor is servicing an SMI interrupt that occurred on an I/O instruction boundary, the processor will service the new SMI request before restarting the originally interrupted I/O instruction. If the I/O instruction restart field is set to FFH prior to returning from the second SMI handler, the EIP will point to an address different from the originally interrupted I/O instruction, which will likely lead to a program error. To avoid this situation, the SMI handler must be able to recognize the occurrence of back-to-back SMI interrupts when I/O instruction restart is being used and insure that the handler sets the I/O instruction restart field to 00H prior to returning from the second invocation of the SMI handler.

13.13. SMM MULTIPLE-PROCESSOR CONSIDERATIONS

The following should be noted when designing multiple-processor systems:

- Any processor in a multiprocessor system can respond to an SMM.
- Each processor needs its own SMRAM space. This space can be in system memory or in a separate RAM.
- The SMRAMs for different processors can be overlapped in the same memory space. The only stipulation is that each processor needs its own state save area and its own dynamic data storage area. (Also, for the Pentium and Intel486 processors, the SMBASE address must be located on a 32-KByte boundary.) Code and static data can be shared among processors. Overlapping SMRAM spaces can be done more efficiently with the P6 family processors because they do not require that the SMBASE address be on a 32-KByte boundary.
- The SMI handler will need to initialize the SMBASE for each processor.
- Processors can respond to local SMIs through their SMI# pins or to SMIs received through the APIC interface. The APIC interface can distribute SMIs to different processors.

- Two or more processors can be executing in SMM at the same time.
- When operating Pentium processors in dual processing (DP) mode, the SMI $\overline{ACT}$ # pin is driven only by the MRM processor and should be sampled with ADS#. For additional details, see Chapter 14 of the *Pentium Processor Family User's Manual, Volume 1*.

SMM is not re-entrant, because the SMRAM State Save Map is fixed relative to the SMBASE. If there is a need to support two or more processors in SMM mode at the same time then each processor should have dedicated SMRAM spaces. This can be done by using the SMBASE Relocation feature (see Section 13.11., “SMBASE Relocation”).

13.14. ENHANCED INTEL SPEEDSTEP[®] TECHNOLOGY

Enhanced Intel SpeedStep[®] Technology on the Pentium M processor efficiently manages processor power consumption via performance state transitions. Processor performance states are defined as discrete operating points associated with different frequencies.

Enhanced Intel SpeedStep Technology on the Pentium M processor differs from previous generations of Intel SpeedStep Technology in two basic ways:

- Centralization of the control mechanism and software interface in the processor by using model-specific registers.
- Reduced hardware overhead; this permits more frequent performance state transitions.

Previous generations of the Intel SpeedStep Technology require processors to be a deep sleep state, holding off bus master transfers for the duration of a performance state transition. Performance state transitions under the Enhanced Intel SpeedStep Technology are discrete transitions to a new target frequency.

Support is indicated by CPUID, using ECX feature bit 07. Enhanced Intel SpeedStep Technology is enabled by setting IA32_MISC_ENABLE MSR, bit 16. On reset, bit 16 of IA32_MISC_ENABLE MSR is cleared.

13.14.1. Software Interface For Initiating Performance State Transitions

State transitions are initiated by writing a 16-bit value to the MSR_PERF_CTL register. If a transition is already in progress, transition to a new value will take effect subsequently.

Reads of MSR_PERF_CTL determine the last targeted operating point. The current operating point can be read from MSR_PERF_STATUS. MSR_PERF_STATUS is updated dynamically.

The 16-bit encoding that defines valid operating points is model-specific. Applications and performance tools are not expected to use either MSR_PERF_CTL or MSR_PERF_STATUS and should treat both as reserved. Performance monitoring tools can access model-specific events and report the occurrences of state transitions.

13.15. THERMAL MONITORING AND PROTECTION

The IA-32 architecture provides three mechanisms for monitoring temperature and controlling power consumption of an IA-32 processor:

1. A **catastrophic shutdown detector** that forces processor execution to stop if the processor's core temperature rises above a preset limit.
2. An **automatic thermal monitoring mechanism** that forces the processor to reduce its power consumption in order to maintain a predetermined temperature limit.
3. A **software controlled clock modulation mechanism** that permits operating system to implement a power management policy to reduce the power consumption of an IA-32 processor; this is in addition to the reduction offered by the automatic thermal monitoring mechanism.

The first mechanism is not visible to software. The other two mechanisms are visible to software using processor feature information returned by executing CPUID with EAX = 1.

The second mechanism, automatic thermal monitoring, provides two modes of operation. One mode modulates the clock duty cycle; the second mode changes the processor's frequency. Both modes are used to control the core temperature of the processor.

The third mechanism modulates the clock duty cycle of the processor. As shown in Figure 13-6, the phrase 'duty cycle' does not refer to the actual duty cycle of the clock signal. Instead it refers to the time period during which the clock signal is allowed to drive the processor chip. By using the stop clock mechanism to control how often the processor is clocked, processor power consumption can be modulated.

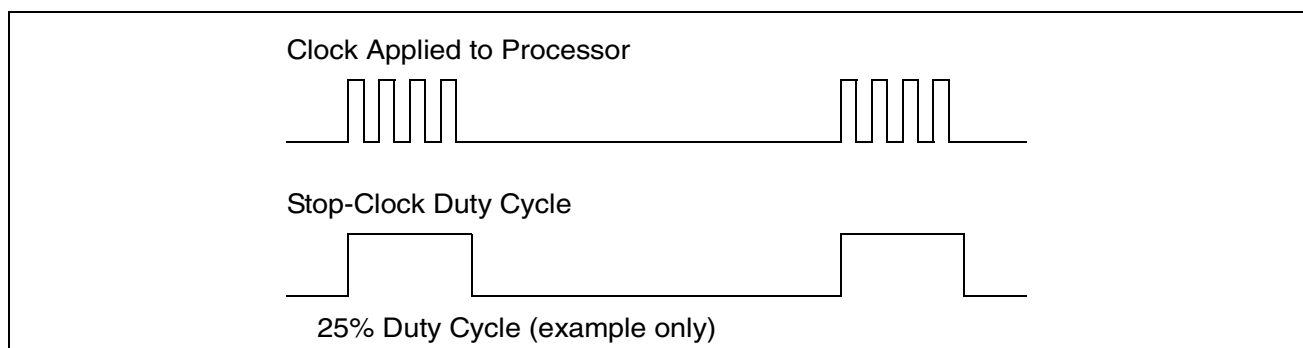


Figure 13-6. Processor Modulation Through Stop-Clock Mechanism

13.15.1. Catastrophic Shutdown Detector

P6 family processors introduced a thermal sensor that acts as a catastrophic shutdown detector. When processor core temperature reaches a factory preset level, the thermal sensor trips and processor execution is halted until after the next reset cycle. This mechanism provides catastrophic over-temperature protection for the processor.

The catastrophic shutdown detector is also implemented in Pentium 4, Intel Xeon and Pentium M processors. It is always enabled.

13.15.2. Thermal Monitor

Pentium 4, Intel Xeon and Pentium M processors include a second temperature sensor that is factory-calibrated to trip when the processor's core temperature crosses a level corresponding to the recommended thermal design envelop. The trip-temperature of the second sensor is calibrated below the temperature assigned to the catastrophic shutdown detector.

13.15.2.1. THERMAL MONITOR 1

The Pentium 4 processor uses the second temperature sensor in conjunction with a mechanism called TM1 (Thermal Monitor 1) to control the core temperature of the processor. TM1 controls the processor's temperature by modulating the duty cycle of the processor clock. Modulation of duty cycles is processor model specific. Note that the processors STPCLK# pin is not used here; the stop-clock circuitry is controlled internally.

Support for TM1 is indicated by CPUID EDX feature bit 29.

TM1 is enabled by setting the thermal-monitor enable flag (bit 3) in IA32_MISC_ENABLE [see Appendix B, *Model-Specific Registers (MSRs)*]. Following a power-up or reset, the flag is cleared, disabling TM1. The basic input/output system (BIOS) is required to enable only one automatic thermal monitoring modes. Operating systems and applications must not disable the operation of these mechanisms.

13.15.2.2. THERMAL MONITOR 2

The Intel Pentium M processor provides an additional automatic mechanism called TM2 (Thermal Monitor 2) to control the core temperature of the processor. TM2 controls temperature by reducing the operating frequency and voltage of the processor. TM2 offers a higher performance level for a given level of power reduction than TM1.

Note that TM2 is triggered by the same temperature sensor in the processor as TM1. Support for TM2 is indicated by CPUID ECX feature bit 8.

NOTE

The mechanism to enable TM2 may be implemented differently in future IA-32 processors.

On Pentium M processors, TM2 is enabled if the TM_SELECT flag (bit 16) of the MSR_THERM2_CTL register is set to 1 and bit 3 of the IA32_MISC_ENABLE register is set to 1.

Following a power-up or reset, the TM_SELECT flag is cleared. BIOS is required to enable either TM1 or TM2. Operating systems and applications must not disable the mechanisms that enable TM1 or TM2. On Pentium M processors, if bit 3 of the IA32_MISC_ENABLE register is set and TM_SELECT flag of the MSR_THERM2_CTL register is cleared, TM1 is enabled.

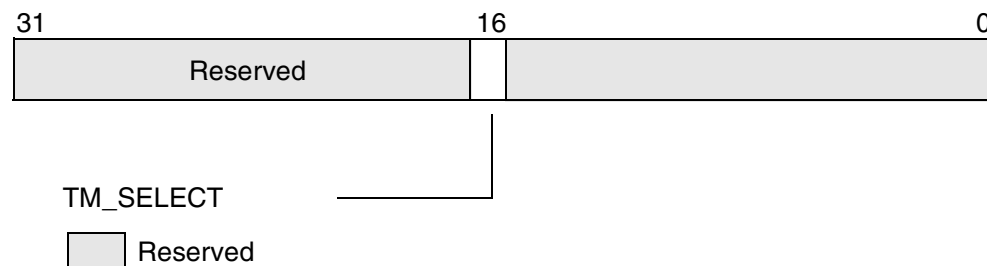


Figure 13-7. MSR_THERM2_CTL Register

13.15.2.3. PERFORMANCE STATE TRANSITIONS AND THERMAL MONITORING

If the thermal control circuitry (TCC) for thermal monitor (TM1/TM2) is active, writes to the MSR_PERF_CTL will effect a new target operating point specified in the MSR_PERF_CTL register.

If TM1 is enabled and the TCC is engaged, the performance state transition can commence before the TCC is disengaged. If TM2 is enabled and the TCC is engaged, the performance state transition specified by a write to the MSR_PERF_CTL will commence after the TCC has disengaged.

13.15.2.4. THERMAL STATUS INFORMATION

The status of the temperature sensor that triggers the thermal monitor (TM1/TM2) is indicated through the thermal status flag (bit 0) and thermal status log flag (bit 1) in the IA32_THERM_STATUS MSR (see Figure 13-8).

The functions of these flags are:

Thermal Status flag, bit 0

When set, indicates that the processor core temperature is currently at the trip temperature of the thermal monitor and that the processor power consumption is being reduced via either TM1 or TM2, depending on which is enabled. When clear, the flag indicates that the core temperature is below the thermal monitor trip temperature. This flag is read only.

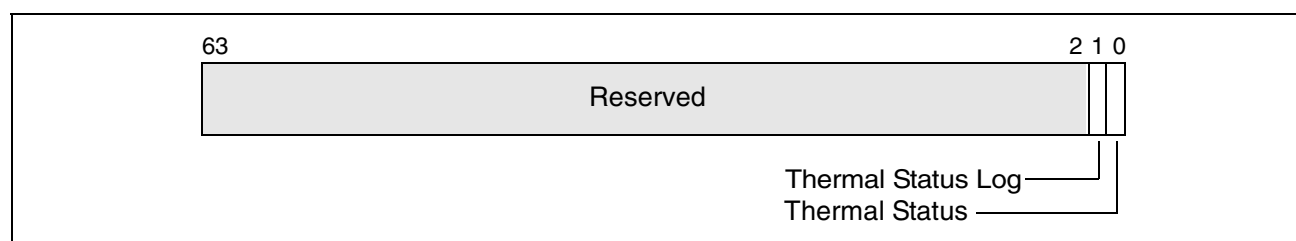


Figure 13-8. IA32_THERM_STATUS MSR

Thermal Status Log flag, bit 1

When set, indicates that the thermal sensor has tripped since the last power-up or reset or since the last time that software cleared this flag. This flag is a sticky bit; once set it remains set until cleared by software or until a power-up or reset of the processor. The default state is clear.

After the second temperature sensor has been tripped, the thermal monitor (TM1/TM2) will remain engaged for at least 1 ms or until the processor core temperature drops below the preset trip temperature of the temperature sensor, taking hysteresis into account.

While the processor is in a stop-clock state, interrupts will be blocked from interrupting the processor. This holding off of interrupts increases the interrupt latency, but does not cause interrupts to be lost. Outstanding interrupts remain pending until clock modulation is complete.

The thermal monitor can be programmed to generate an interrupt to the processor when the thermal sensor is tripped. The delivery mode, mask and vector for this interrupt can be programmed through the thermal entry in the local APIC's LVT (see Section 8.5.1., "Local Vector Table"). The low-temperature interrupt enable and high-temperature interrupt enable flags (bits 0 and 1, respectively) in the IA32_THERM_INTERRUPT MSR (see Figure 13-9) control when the interrupt is generated; that is, on a transition from a temperature below the trip point to above and/or vice-versa.

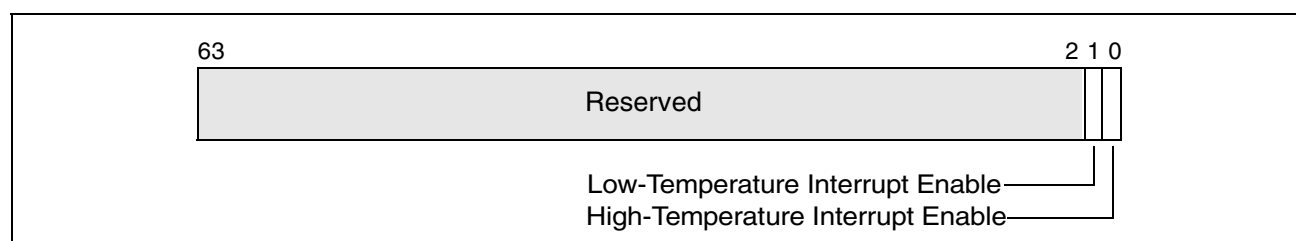


Figure 13-9. IA32_THERM_INTERRUPT MSR

Low-Temperature Interrupt Enable flag, bit 1

Enables an interrupt to be generated on the transition from a high-temperature to a low-temperature when set; disables the interrupt when clear.

High-Temperature Interrupt Enable flag, bit 0

Enables an interrupt to be generated on the transition from a low-temperature to a high-temperature when set; disables the interrupt when clear.(R/W).

The thermal monitor interrupt can be masked by the thermal LVT entry. After a power-up or reset, the low-temperature interrupt enable and high-temperature interrupt enable flags in the IA32_THERM_INTERRUPT MSR are cleared (interrupts are disabled) and the thermal LVT entry is set to mask interrupts. This interrupt should be handled either by the operating system or system management mode (SMM) code.

Note that the operation of the thermal monitoring mechanism has no effect upon the clock rate of the processor's internal high-resolution timer (time stamp counter).

13.15.3. Software Controlled Clock Modulation

Pentium 4, Intel Xeon and Pentium M processors also support software-controlled clock modulation. This provides a means for operating systems to implement a power management policy to reduce the power consumption of the processor. Here, the stop-clock duty cycle is controlled by software through the IA32_THERM_CONTROL MSR (see Figure 13-10).

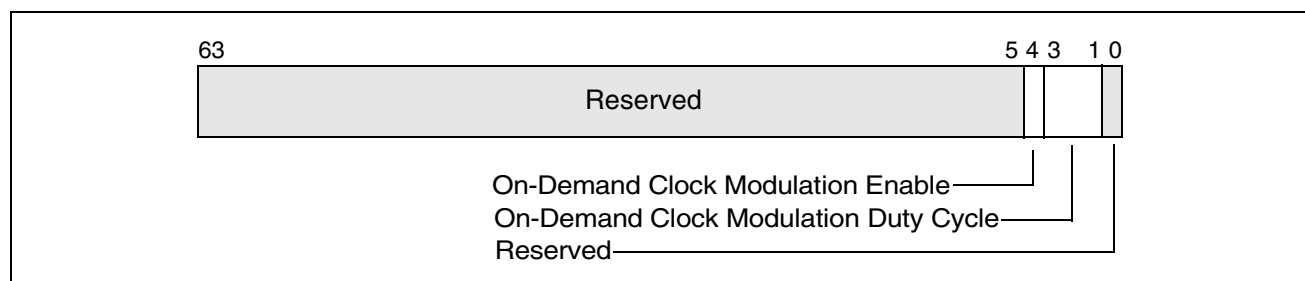


Figure 13-10. IA32_THERM_CONTROL MSR

The IA32_THERM_CONTROL MSR contains the following flag and field used to enable software-controlled clock modulation and to select the clock modulation duty cycle.

On-Demand Clock Modulation Enable, bit 4

Enables on-demand software controlled clock modulation when set; disables software-controlled clock modulation when clear.

On-Demand Clock Modulation Duty Cycle, bits 1 through 3

Selects the on-demand clock modulation duty cycle (see Table 13-5). This field is only active when the on-demand clock modulation enable flag is set.

Note that the on-demand clock modulation mechanism (like the thermal monitor) controls the processor's stop-clock circuitry internally to modulate the clock signal. The STPCLK# pin is not used.

Table 13-5. On-Demand Clock Modulation Duty Cycle Field Encoding

Duty Cycle Field Encoding	Duty Cycle
000B	Reserved
001B	12.5% (Default)
010B	25.0%
011B	37.5%
100B	50.0%
101B	63.5%
110B	75%
111B	87.5%

The on-demand clock modulation mechanism can be used to control processor power consumption. Power management software can write to the IA32_THERM_CONTROL MSR to enable clock modulation and to select a modulation duty cycle. If on-demand clock modulation and TM1 are both enabled and the thermal status of the processor is hot (bit 0 of the IA32_THERM_STATUS MSR is set), clock modulation at the duty cycle specified by TM1 takes precedence, regardless of the setting of the on-demand clock modulation duty cycle.

For Hyper-Threading Technology enabled processors, the IA32_THERM_CONTROL register is duplicated for each logical processor. In order for the On-demand clock modulation feature to work properly, the IA32_THERM_CONTROL register must be programmed identically on all logical processors in the same physical processor.

For the P6 family processors, on-demand clock modulation was implemented through the chipset, which controlled clock modulation through the processor's STPCLK# pin.

13.15.4. Detection of Thermal Monitor and Software Controlled Clock Modulation Facilities

The ACPI flag (bit 22) of the CPUID feature flags indicates the presence of the IA32_THERM_STATUS, IA32_THERM_INTERRUPT, and IA32_THERM_CONTROL MSRs, and the xAPIC thermal LVT entry.

The TM1 flag (bit 29) of the CPUID feature flags indicates the presence of the automatic thermal monitoring facilities that modulate clock duty cycles.

Machine-Check Architecture

CHAPTER 14

MACHINE-CHECK ARCHITECTURE

This chapter describes the machine-check architecture and machine-check exception mechanism found in the Pentium 4, Intel Xeon, and P6 family processors. See Chapter 5, “Interrupt 18—Machine-Check Exception (#MC)”, for more information on the machine-check exception. A brief description of the Pentium processor’s machine check capability is also given.

14.1. MACHINE-CHECK EXCEPTIONS AND ARCHITECTURE

The Pentium 4, Intel Xeon, and P6 family processors implement a machine-check architecture that provides a mechanism for detecting and reporting hardware (machine) errors, such as system bus errors, ECC errors, parity errors, cache errors, and TLB errors. It consists of a set of model-specific registers (MSRs) that are used to set up machine checking and additional banks of MSRs for recording the errors that are detected. The processor signals the detection of a machine-check error by generating a machine-check exception (#MC), which is an abort class exception. The implementation of the machine-check architecture, does not ordinarily permit the processor to be restarted reliably after generating a machine-check exception; however, the machine-check-exception handler can collect information about the machine-check error from the machine-check MSRs.

14.2. COMPATIBILITY WITH PENTIUM PROCESSOR

The Pentium 4, Intel Xeon, and P6 family processors support and extend the machine-check exception mechanism used in the Pentium processor. The Pentium processor reports the following machine-check errors:

- Data parity errors during read cycles.
- Unsuccessful completion of a bus cycle.

These errors are reported through the P5_MC_TYPE and P5_MC_ADDR MSRs, which are implementation specific for the Pentium processor. These MSRs can be read with the RDMSR instruction. See Table B-4 for the register addresses for these MSRs.

The machine-check error reporting mechanism that the Pentium processors use is similar to that used in the Pentium 4, Intel Xeon, and P6 family processors. That is, when an error is detected, it is recorded in the P5_MC_TYPE and P5_MC_ADDR MSRs and then the processor generates a machine-check exception (#MC).

See Section 14.3.3., “Mapping of the Pentium Processor Machine-Check Errors to the Machine-Check Architecture”, and Section 14.7.2., “Pentium Processor Machine-Check Exception Handling”, for information on compatibility between machine-check code written to run on the Pentium processors and code written to run on P6 family processors.

14.3. MACHINE-CHECK MSRS

The machine check MSRs in the Pentium 4, Intel Xeon, and P6 family processors consist of a set of global control and status registers and several error-reporting register banks (see Figure 14-1). Each error-reporting bank is associated with a specific hardware unit (or group of hardware units) within the processor. The RDMSR and WRMSR instructions are used to read and write these registers.

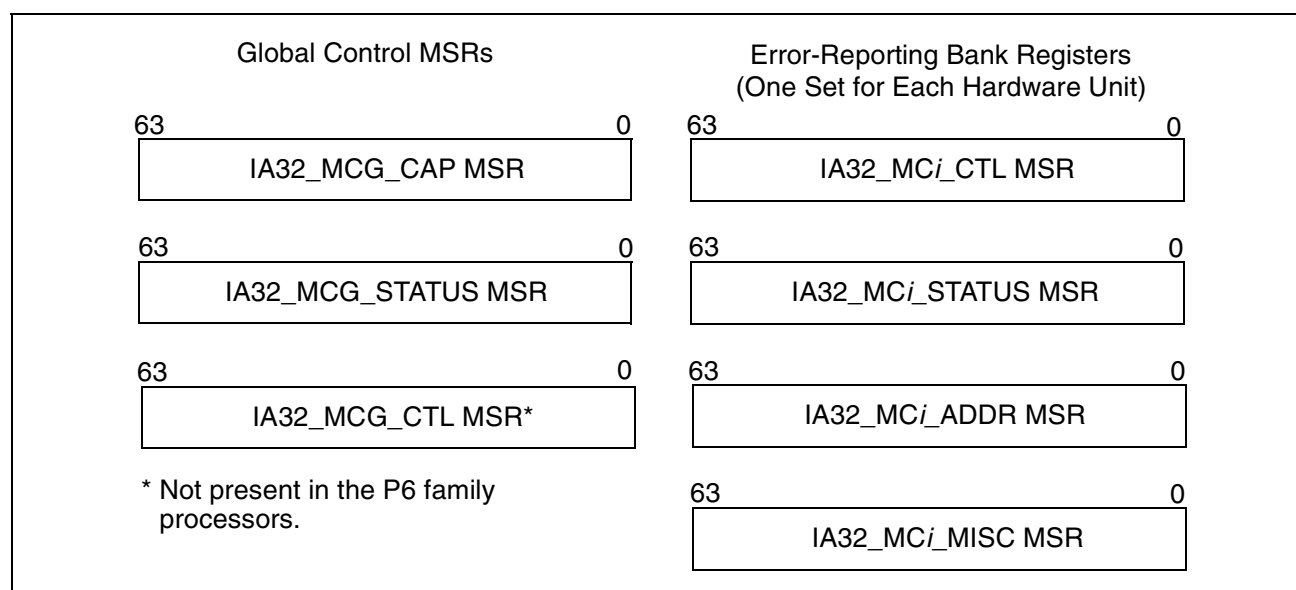


Figure 14-1. Machine-Check MSRs

14.3.1. Machine-Check Global Control MSRs

The machine-check global control MSRs include the IA32_MCG_CAP, IA32_MCG_STATUS, and IA32_MCG_CTL MSRs. See Appendix B, *Model-Specific Registers (MSRs)*, for the addresses of these registers. The structure of the IA32_MCG_CAP MSR is implemented differently in the Pentium 4 and Intel Xeon processors and in the P6 family processors. Also note that the register names used for the P6 family processors do not have the “IA32” prefix.

14.3.1.1. IA32_MCG_CAP MSR (PENTIUM 4 AND INTEL XEON PROCESSORS)

The IA32_MCG_CAP MSR is a read-only register that provides information about the machine-check architecture implementation in the Pentium 4 and Intel Xeon processors (see Figure 14-2). It contains the following field and flag:

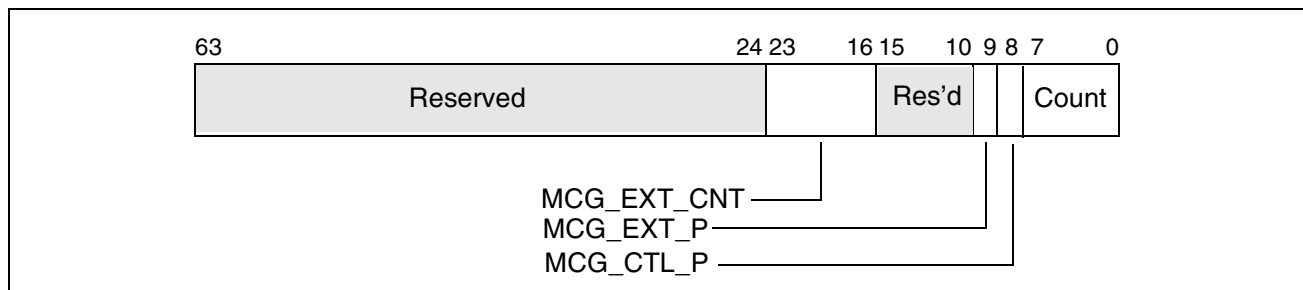


Figure 14-2. IA32_MCG_CAP Register

Count field, bits 0 through 7

Indicates the number of hardware unit error-reporting banks available in a particular processor implementation.

MCG_CTL_P (control MSR present) flag, bit 8

Indicates that the processor implements the IA32_MCG_CTL MSR when set; these registers are absent when clear.

MCG_EXT_P (extended MSRs present) flag, bit 9

Indicates that the processor implements the extended machine-check state registers found starting at MSR address 180H; these registers are absent when clear. This is a feature was introduced in the Pentium 4 and Intel Xeon processors.

MCG_EXT_CNT, bits 16 through 23

Indicates the number of extended machine-check state registers present. This field is meaningful only when the MCG_EXT_P flag is set.

Bits 10 through 15 and bits 24 through 63 are reserved. The effect of writing to the IA32_MCG_CAP register is undefined.

14.3.1.2. MCG_CAP MSR (P6 FAMILY PROCESSORS)

The MCG_CAP MSR is a read-only register that provides information about the machine-check architecture implementation in the P6 family processors (see Figure 14-3). It contains the following field and flag:

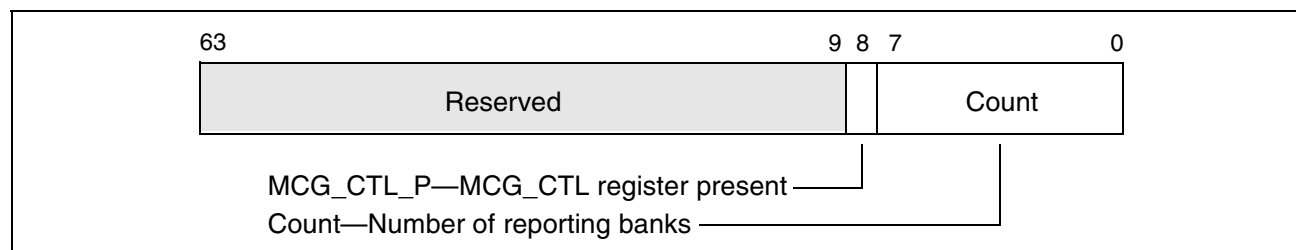


Figure 14-3. MCG_CAP Register

Count field, bits 0 through 7

Indicates the number of hardware unit error-reporting banks available in a particular processor implementation.

MCG_CTL_P (register present) flag, bit 8

Indicates that the MCG_CTL register is present when set, and absent when clear.

Bits 9 through 63 are reserved. The effect of writing to the MCG_CAP register is undefined.

14.3.1.3. IA32_MCG_STATUS MSR

The IA32_MCG_STATUS MSR (called the MCG_STATUS MSR for the P6 family processors) describes the current state of the processor after a machine-check exception has occurred (see Figure 14-4).

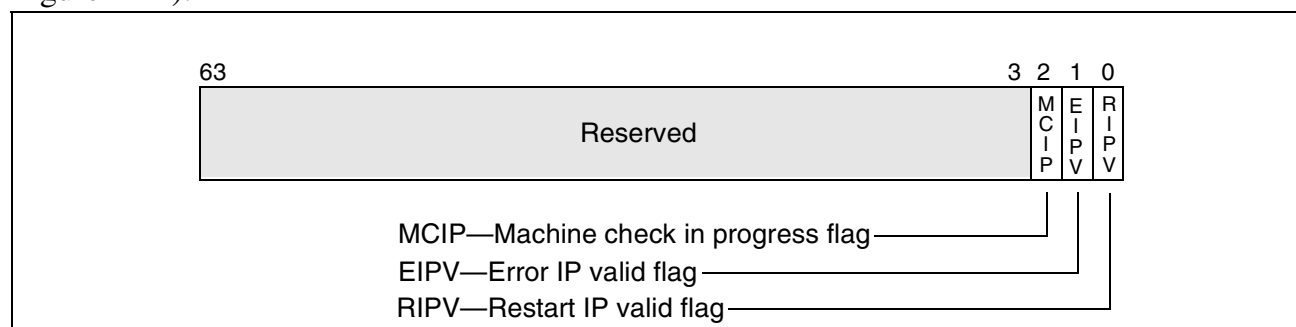


Figure 14-4. IA32_MCG_STATUS Register

This register contains the following flags:

RIPV (restart IP valid) flag, bit 0

Indicates (when set) that program execution can be restarted reliably at the instruction pointed to by the instruction pointer pushed on the stack when the machine-check exception is generated. When clear, the program cannot be reliably restarted at the pushed instruction pointer.

EIPV (error IP valid) flag, bit 1

Indicates (when set) that the instruction pointed to by the instruction pointer pushed onto the stack when the machine-check exception is generated is directly associated with the error. When this flag is cleared, the instruction pointed to may not be associated with the error.

MCIP (machine check in progress) flag, bit 2

Indicates (when set) that a machine-check exception was generated. Software can set or clear this flag. The occurrence of a second Machine-Check Event while MCIP is set will cause the processor to enter a shutdown state.

Bits 3 through 63 in the MCG_STATUS register are reserved.

14.3.1.4. IA32_MCG_CTL MSR

The IA32_MCG_CTL MSR (called the MCG_CTL MSR for the P6 family processors) is present if the capability flag MCG_CTL_P is set in the IA32_MCG_CAP MSR (or MCG_CAP MSR). The IA32_MCG_CTL register controls the reporting of machine-check exceptions. If present, writing all 1s to this register enables all machine-check features and writing all 0s disables all machine-check features. All other values are undefined and/or implementation specific.

14.3.1.5. IA32_MCG_MISC MSR

The IA32_MCG_MISC MSR (called the MCG_MISC MSR for the P6 family processors) is an architecturally defined register, provided to contain miscellaneous machine check control and status information. See Table B-1 for a description of the bits in this register. This register is not implemented in the P6 family processors.

14.3.2. Error-Reporting Register Banks

Each error-reporting register bank can contain an IA32_MCi_CTL, IA32_MCi_STATUS, IA32_MCi_ADDR, and IA32_MCi_MISC MSR (called MCi_CTL, MCi_STATUS, MCi_ADDR, and MCi_MISC in the P6 family processors). The Pentium 4 and Intel Xeon processors provide four banks of error-reporting registers; the P6 family processors provide five banks of error-reporting registers. The first error-reporting register (IA32_MC0_CTL) always starts at address 400H. See Table B-1 for the addresses of the error-reporting registers in the Pentium 4 and Intel Xeon processors; see Table B-3 for the addresses of the error-reporting registers P6 family processors.

14.3.2.1. IA32_MCI_CTL MSRS

The IA32_MCi_CTL MSR (called MCi_CTL in the P6 family processors) controls error reporting for specific errors produced by a particular hardware unit (or group of hardware units). Each of the 64 flags (EE_j) represents a potential error. Setting an EE_j flag enables reporting of the associated error and clearing it disables reporting of the error. Writing the 64-bit value

FFFFFFFFFFFFFFFH to an MCi_CTL register enables logging of all errors. The processor does not write changes to bits that are not implemented. Figure 14-5 shows the bit fields of IA32 MCi CTL.

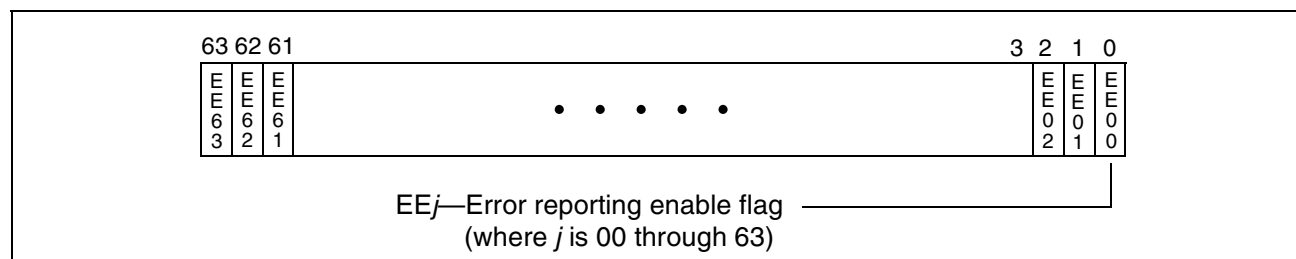


Figure 14-5. IA32_MCi_CTL Register

NOTE

(P6 family processors only.) Operating system or executive software must not modify the contents of the MC0_CTL MSR. This MSR is internally aliased to the EBL_CR_POWERON MSR and as such controls system-specific error handling features. These features are platform specific. System specific firmware (the BIOS) is responsible for the appropriate initialization of the MC0_CTL MSR. The P6 family processors only allow the writing of all 1s or all 0s to the MC_i CTL MSR.

14.3.2.2. IA32 MC_i STATUS MSRS

Each IA32_MCi_STATUS MSR (called MCi_STATUS in the P6 family processors) contains information related to a machine-check error if its VAL (valid) flag is set (see Figure 14-6). Software is responsible for clearing the IA32_MCi_STATUS MSRs by explicitly writing all 0s to them; writing 1s to them will cause a general-protection exception to be generated. The flags and fields in these MSRs are as follows:

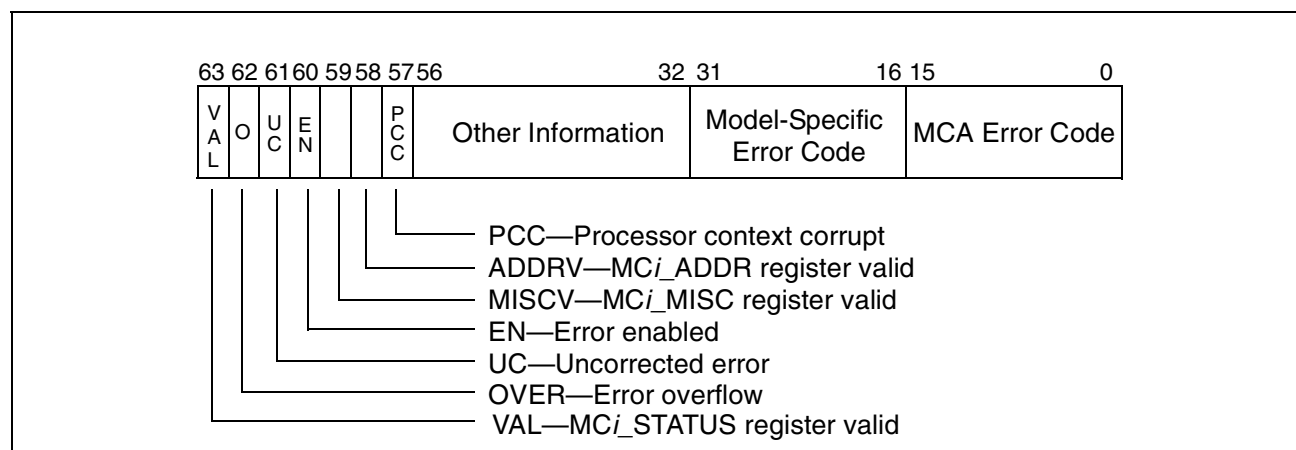


Figure 14-6. IA32 MCi STATUS Register

MCA (machine-check architecture) error code field, bits 0 through 15

Specifies the machine-check architecture-defined error code for the machine-check error condition detected. The machine-check architecture-defined error codes are guaranteed to be the same for all IA-32 processors that implement the machine-check architecture. See Section 14.6., “Interpreting the MCA Error Codes”, for information on machine-check error codes.

Model-specific error code field, bits 16 through 31

Specifies the model-specific error code that uniquely identifies the machine-check error condition detected. The model-specific error codes may differ among IA-32 processors for the same machine-check error condition.

Other information field, bits 32 through 56

The functions of the bits in this field are implementation specific and are not part of the machine-check architecture. Software that is intended to be portable among IA-32 processors should not rely on the values in this field.

PCC (processor context corrupt) flag, bit 57

Indicates (when set) that the state of the processor might have been corrupted by the error condition detected and that reliable restarting of the processor may not be possible. When clear, this flag indicates that the error did not affect the processor’s state.

ADDRV (MCi_ADDR register valid) flag, bit 58

Indicates (when set) that the MCi_ADDR register contains the address where the error occurred (see Section 14.3.2.3., “IA32_MCi_ADDR MSRs”). When clear, this flag indicates that the MCi_ADDR register is either not implemented or does not contain the address where the error occurred. Do not read these registers if they are not implemented in the processor.

MISCV (MCi_MISC register valid) flag, bit 59

Indicates (when set) that the MCi_MISC register contains additional information regarding the error. When clear, this flag indicates that the MCi_MISC register is either not implemented or does not contain additional information regarding the error. Do not read these registers if they are not implemented in the processor.

EN (error enabled) flag, bit 60

Indicates (when set) that the error was enabled by the associated EEj bit of the MCi_CTL register.

UC (error uncorrected) flag, bit 61

Indicates (when set) that the processor did not or was not able to correct the error condition. When clear, this flag indicates that the processor was able to correct the error condition.

OVER (machine check overflow) flag, bit 62

Indicates (when set) that a machine-check error occurred while the results of a previous error were still in the error-reporting register bank (that is, the VAL bit was already set in the MCi_STATUS register). The processor sets the OVER flag and software is responsible for clearing it. Enabled errors are written over disabled errors, and uncor-

rected errors are written over corrected errors. Uncorrected errors are not written over previous valid uncorrected errors.

VAL (MCi_STATUS register valid) flag, bit 63

Indicates (when set) that the information within the MCi_STATUS register is valid. When this flag is set, the processor follows the rules given for the OVER flag in the MCi_STATUS register when overwriting previously valid entries. The processor sets the VAL flag and software is responsible for clearing it.

14.3.2.3. IA32_MCi_ADDR MSRS

The IA32_MCi_ADDR MSR (called MCi_ADDR in the P6 family processors) contains the address of the code or data memory location that produced the machine-check error if the ADDR_V flag in the IA32_MCi_STATUS register is set (see Section 14-7, “IA32_MCi_ADDR MSR”). The IA32_MCi_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MCi_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general protection exception.

The address returned is either 32-bit offset into a segment, 32-bit linear address, or 36-bit physical address, depending upon the type of error encountered.

Bits 36 through 63 of this register are reserved for future address expansion and are always read as zeros. These registers can be cleared by explicitly writing all 0s to them; writing 1s to them will cause a general-protection exception to be generated.

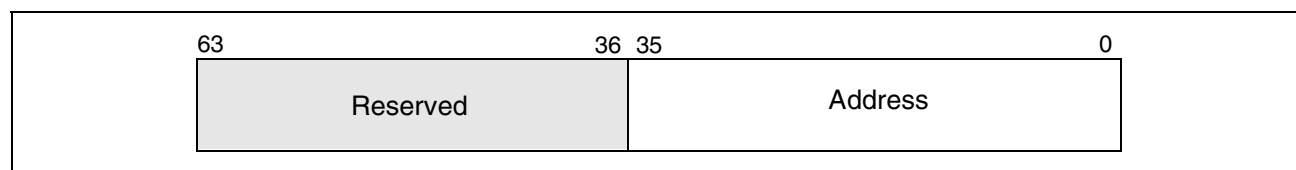


Figure 14-7. IA32_MCi_ADDR MSR

14.3.2.4. IA32_MCi_MISC MSRS

The IA32_MCi_MISC MSR (called MCi_MISC in the P6 family processors) contains additional information describing the machine-check error if the MISCV flag in the IA32_MCi_STATUS register is set. The IA32_MCi_MISC_MASR is either not implemented or does not contain additional information if the MISCV flag in the IA32_MCi_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general protection exception. When implemented in a processor, these registers can be cleared by explicitly writing all 0s to them; writing 1s to them causes a general-protection exception to be generated. This register is not implemented in any of the error-reporting register banks for the P6 family processors.

14.3.2.5. IA32_MCG Extended Machine Check State MSRs

The Pentium 4 and Intel Xeon processors implement a variable number of extended machine-check state MSRs (the architectural entries are documented in Table 14-1). The MCG_EXT_P flag in the IA32_MCG_CAP MSR indicates the presence of these extended registers, and the MCG_EXT_CNT field indicates the number of these registers actually implemented (see Section 14.3.1.1., “IA32_MCG_CAP MSR (Pentium 4 and Intel Xeon Processors)”)

There may be registers available beyond the IA32_MCG_MISC register. These registers should be referred to as IA32_MCG_RESERVED1 to IA32_MCG_RESERVEDn depending on the actual number.

Table 14-1. Extended Machine-Check State MSRs

MSR	Address	Description
IA32_MCG_EAX	180H	State of the EAX register at the time of the machine-check error.
IA32_MCG_EBX	181H	State of the EBX register at the time of the machine-check error.
IA32_MCG_ECX	182H	State of the ECX register at the time of the machine-check error.
IA32_MCG_EDX	183H	State of the EDX register at the time of the machine-check error.
IA32_MCG_ESI	184H	State of the ESI register at the time of the machine-check error.
IA32_MCG_EDI	185H	State of the EDI register at the time of the machine-check error.
IA32_MCG_EBP	186H	State of the EBP register at the time of the machine-check error.
IA32_MCG_ESP	187H	State of the ESP register at the time of the machine-check error.
IA32_MCG_EFLAGS	188H	State of the EFLAGS register at the time of the machine-check error.
IA32_MCG_EIP	189H	State of the EIP register at the time of the machine-check error.
IA32_MCG_RESERVED	18AH	Reserved.

When a machine-check error is detected on a Pentium 4 or Intel Xeon processor, the processor saves the state of the general-purpose registers, the EFLAGS register, and the EIP in these extended machine-check state MSRs. This information can be used by a debugger to analyze the error.

These registers are read/write to zero registers. This means software can read them, but if software writes to them, only all zeros is allowed. If software attempts to write a non-zero value into one of these registers, a general-protection (#GP) exception is generated. These registers are cleared on a hardware reset (power-up or RESET), but maintains their contents following a soft reset (INIT reset).

14.3.3. Mapping of the Pentium Processor Machine-Check Errors to the Machine-Check Architecture

The Pentium processor reports machine-check errors using two registers: P5_MC_TYPE and P5_MC_ADDR. The Pentium 4, Intel Xeon, and P6 family processors map these registers into the IA32_MCi_STATUS and IA32_MCi_ADDR registers of the error-reporting register bank that reports on the type of external bus errors reported in the P5_MC_TYPE and P5_MC_ADDR registers. The information in these registers can then be accessed in either of two ways:

- By reading the IA32_MCi_STATUS and IA32_MCi_ADDR registers as part of a generalized machine-check exception handler written for a Pentium 4 and P6 family processors.
- By reading the P5_MC_TYPE and P5_MC_ADDR registers with the RDMSR instruction.

The second access capability permits a machine-check exception handler written to run on a Pentium processor to be run on a Pentium 4, Intel Xeon, or P6 family processor. There is a limitation in that information returned by the Pentium 4, Intel Xeon, and P6 family processors will be encoded differently than it is for the Pentium processor. To run the Pentium processor machine-check exception handler on a Pentium 4, Intel Xeon, or P6 family processor, it must be rewritten to interpret the P5_MC_TYPE register encodings correctly.

14.4. MACHINE-CHECK AVAILABILITY

The machine-check architecture and machine-check exception (#MC) are model-specific features. Software can execute the CUID instruction to determine whether a processor implements these features. Following the execution of the CUID instruction, the settings of the MCA flag (bit 14) and MCE flag (bit 7) in the EDX register indicate whether the processor implements the machine-check architecture and machine-check exception, respectively.

14.5. MACHINE-CHECK INITIALIZATION

To use the processors machine-check architecture, software must initialize the processor to activate the machine-check exception and the error-reporting mechanism.

Example 14-1 gives pseudocode for performing this initialization. This pseudocode checks for the existence of the machine-check architecture and exception on the processor, then enables the machine-check exception and the error-reporting register banks. The pseudocode shown is compatible with the Pentium 4, Intel Xeon, P6 family, and Pentium processors.

Following power up or power cycling, the IA32_MCi_STATUS registers are not guaranteed to have valid data until after the registers are initially cleared to all 0s by software, as shown in the initialization pseudocode in Example 14-1. In addition, when using P6 family processors, the software must set MCi_STATUS registers to 0 when doing a soft-reset.

Example 14-1. Machine-Check Initialization Pseudocode

```

Check CPUID Feature Flags for MCE and MCA support
IF CPU supports MCE
THEN
    IF CPU supports MCA
    THEN
        IF (IA32_MCG_CAP.MCG_CTL_P = 1)
        (* IA32_MCG_CTL register is present *)
        THEN
            IA32_MCG_CTL ← FFFFFFFFFFFFFFFFH;
            (* enables all MCA features *)
        FI

        (* Determine number of error-reporting banks supported *)
        COUNT ← IA32_MCG_CAP.Count;
        MAX_BANK_NUMBER ← COUNT - 1;

        IF (Processor Family is 6H)
        THEN
            (* Enable logging of all errors except for MC0_CTL register *)
            FOR error-reporting banks (1 through MAX_BANK_NUMBER)
            DO
                IA32_MCi_CTL ← 0FFFFFFFFFFFFFFFH;
            OD

            (* Clear all errors *)
            FOR error-reporting banks (0 through MAX_BANK_NUMBER)
            DO
                IA32_MCi_STATUS ← 0;
            OD

        ELSE IF (Processor Family is 0FH) (*any Processor Extended Family *)
        THEN
            (* Enable logging of all errors including MC0_CTL register *)
            FOR error-reporting banks (0 through MAX_BANK_NUMBER)
            DO
                IA32_MCi_CTL ← 0FFFFFFFFFFFFFFFH;
            OD

            (* BIOS clears all errors only on power-on reset *)
            IF (BIOS detects Power-on reset)
            THEN
                FOR error-reporting banks (0 through MAX_BANK_NUMBER)
                DO
                    IA32_MCi_STATUS ← 0;
                OD
            ELSE

```

```

        FOR error-reporting banks (0 through MAX_BANK_NUMBER)
        DO
            (Optional for BIOS and OS) Log valid errors
            (OS only) IA32_MCi_STATUS ← 0;
        OD

    FI

FI

Setup the Machine Check Exception (#MC) handler for vector 18 in IDT

Set the MCE bit (bit 6) in CR4 register to enable Machine-Check Exceptions
FI

```

14.6. INTERPRETING THE MCA ERROR CODES

When the processor detects a machine-check error condition, it writes a 16-bit error code in the MCA error code field of one of the IA32_MCi_STATUS registers and sets the VAL (valid) flag in that register. The processor may also write a 16-bit model-specific error code in the IA32_MCi_STATUS register depending on the implementation of the machine-check architecture of the processor.

The MCA error codes are architecturally defined for IA-32 processors; however, the specific IA32_MCi_STATUS register that a code is written into is model specific. To determine the cause of a machine-check exception, the machine-check exception handler must read the VAL flag for each IA32_MCi_STATUS register, and, if the flag is set, then read the MCA error code field of the register. It is the encoding of the MCACOD value that determines the type of error being reported and not the register bank reporting it.

There are two types of MCA error codes: simple error codes and compound error codes.

14.6.1. Simple Error Codes

Table 14-2 shows the simple error codes. These unique codes indicate global error information.

Table 14-2. Simple Error Codes

Error Code	Binary Encoding	Meaning
No Error	0000 0000 0000 0000	No error has been reported to this bank of error-reporting registers.
Unclassified	0000 0000 0000 0001	This error has not been classified into the MCA error classes.
Microcode ROM Parity Error	0000 0000 0000 0010	Parity error in internal microcode ROM
External Error	0000 0000 0000 0011	The BINIT# from another processor caused this processor to enter machine check.
FRC Error	0000 0000 0000 0100	FRC (functional redundancy check) master/slave error
Internal Unclassified	0000 01xx xxxx xxxx	Internal unclassified errors

14.6.2. Compound Error Codes

The compound error codes describe errors related to the TLBs, memory, caches, bus and interconnect logic. A set of sub-fields is common to all of the compound error encodings. These sub-fields describe the type of access, level in the memory hierarchy, and type of request. Table 14-4 shows the general form of the compound error codes. The interpretation column indicates the name of a compound error. The name is constructed by substituting mnemonics from Tables 14-4 through 14-7 for the sub-field names given within curly braces.

For example, the error code ICACHEL1_RD_ERR is constructed from the form:

{TT}CACHE{LL}_{RRRR}_ERR

where {TT} is replaced by I, {LL} is replaced by L1, and {RRRR} is replaced by RD.

Table 14-3. General Forms of Compound Error Codes

Type	Form	Interpretation
TLB Errors	0000 0000 0001 TTLL	{TT}TLB{LL}_ERR
Memory Hierarchy Errors	0000 0001 RRRR TTLL	{TT}CACHE{LL}_{RRRR}_ERR
Bus and Interconnect Errors	0000 1PPT RRRR IILL	BUS{LL}_{PP}_{RRRR}_{II}_{T}_ERR

The 2-bit TT sub-field (see Table 14-4) indicates the type of transaction (data, instruction, or generic). It applies to the TLB, cache, and interconnect error conditions. The generic type is reported when the processor cannot determine the transaction type.

Table 14-4. Encoding for TT (Transaction Type) Sub-Field

Transaction Type	Mnemonic	Binary Encoding
Instruction	I	00
Data	D	01
Generic	G	10

The 2-bit LL sub-field (see Table 14-5) indicates the level in the memory hierarchy where the error occurred (level 0, level 1, level 2, or generic). The LL sub-field also applies to the TLB, cache, and interconnect error conditions. The Pentium 4, Intel Xeon, and P6 family processors support two levels in the cache hierarchy and one level in the TLBs. Again, the generic type is reported when the processor cannot determine the hierarchy level.

Table 14-5. Level Encoding for LL (Memory Hierarchy Level) Sub-Field

Hierarchy Level	Mnemonic	Binary Encoding
Level 0	L0	00
Level 1	L1	01
Level 2	L2	10
Generic	LG	11

The 4-bit RRRR sub-field (see Table 14-6) indicates the type of action associated with the error. Actions include read and write operations, prefetches, cache evictions, and snoops. Generic error is returned when the type of error cannot be determined. Generic read and generic write are returned when the processor cannot determine the type of instruction or data request that caused the error. Eviction and snoop requests apply only to the caches. All of the other requests apply to TLBs, caches and interconnects.

Table 14-6. Encoding of Request (RRRR) Sub-Field

Request Type	Mnemonic	Binary Encoding
Generic Error	ERR	0000
Generic Read	RD	0001
Generic Write	WR	0010
Data Read	DRD	0011
Data Write	DWR	0100
Instruction Fetch	IRD	0101
Prefetch	PREFETCH	0110
Eviction	EVICT	0111
Snoop	SNOOP	1000

The bus and interconnect errors are defined with the 2-bit PP (participation), 1-bit T (time-out), and 2-bit II (memory or I/O) sub-fields, in addition to the LL and RRRR sub-fields (see Table 14-7). The bus error conditions are implementation dependent and related to the type of bus implemented by the processor. Likewise, the interconnect error conditions are predicated on a specific implementation-dependent interconnect model that describes the connections between the different levels of the storage hierarchy. The type of bus is implementation dependent, and as such is not specified in this document. A bus or interconnect transaction consists of a request involving an address and a response.

Table 14-7. Encodings of PP, T, and II Sub-Fields

Sub-Field	Transaction	Mnemonic	Binary Encoding
PP (Participation)	Local processor originated request	SRC	00
	Local processor responded to request	RES	01
	Local processor observed error as third party	OBS	10
	Generic		11
T (Time-out)	Request timed out	TIMEOUT	1
	Request did not time out	NOTIMEOUT	0
II (Memory or I/O)	Memory Access	M	00
	Reserved		01
	I/O	IO	10
	Other transaction		11

14.6.3. Example Machine-Check Error Codes Interpretation

Appendix E, *Interpreting Machine-Check Error Codes*, gives an example of a machine-check external bus error and shows how to interpret the MCA error code, model-specific error code, and other information error code fields. This example only pertains to the P6 family processors.

14.7. GUIDELINES FOR WRITING MACHINE-CHECK SOFTWARE

The machine-check architecture and error logging can be used in two different ways:

- To detect machine errors during normal instruction execution, using the machine-check exception (#MC).
- To periodically check and log machine errors.

To use the machine-check exception, the operating system or executive software must provide a machine-check exception handler. This handler can be designed specifically for Pentium 4 and Intel Xeon processors, for P6 family processors, or be a portable handler that also handles processor machine-check errors from several generations of IA-32 processors.

A special program or utility is required to log machine errors.

Guidelines for writing a machine-check exception handler or a machine-error logging utility are given in the following sections.

14.7.1. Machine-Check Exception Handler

The machine-check exception (#MC) corresponds to vector 18. To service machine-check exceptions, a trap gate must be added to the IDT, and the pointer in the trap gate must point to a machine-check exception handler. Two approaches can be taken to designing the exception handler:

- The handler can merely log all the machine status and error information, then call a debugger or shut down the system.
- The handler can analyze the reported error information and, in some cases, attempt to correct the error and restart the processor.

For Pentium 4, Intel Xeon, P6 family, and Pentium processors, virtually all the machine-check conditions detected cannot be recovered from (they result in abort-type exceptions). The logging of status and error information is therefore a baseline implementation. See Section 14.7., “Guidelines for Writing Machine-Check Software”, for more information on logging errors.

When recovery from a machine-check error may be possible, the following things should be considered when writing a machine-check exception handler:

- To determine the nature of the error, the handler must read each of the error-reporting register banks. The count field in the IA32_MCG_CAP register gives number of register banks. The first register of register bank 0 is at address 400H.
- The VAL (valid) flag in each IA32_MCi_STATUS register indicates whether the error information in the register is valid. If this flag is clear, the registers in that bank do not contain valid error information and do not need to be checked.
- To write a portable exception handler, only the MCA error code field in the IA32_MCi_STATUS register should be checked. See Section 14.6., “Interpreting the MCA Error Codes”, for information that can be used to write an algorithm to interpret this field.
- The RIPV, PCC, and OVER flags in each IA32_MCi_STATUS register indicate whether recovery from the error is possible. If either of these fields is set, recovery is not possible. The OVER field indicates that two or more machine-check error occurred. When recovery is not possible, the handler typically records the error information and signals an abort to the operating system.
- Corrected errors will have been corrected automatically by the processor. The UC flag in each IA32_MCi_STATUS register indicates whether the processor automatically corrected the error.
- The RIPV flag in the IA32_MCG_STATUS register indicates whether the program can be restarted at the instruction pointed to by the instruction pointer pushed on the stack when the exception was generated. If this flag is clear, the processor may still be able to be restarted (for debugging purposes), but not without loss of program continuity.
- For unrecoverable errors, the EIPV flag in the IA32_MCG_STATUS register indicates whether the instruction pointed to by the instruction pointer pushed on the stack when the

exception was generated is related to the error. If this flag is clear, the pushed instruction may not be related to the error.

- The MCIP flag in the IA32_MCG_STATUS register indicates whether a machine-check exception was generated. Before returning from the machine-check exception handler, software should clear this flag so that it can be used reliably by an error logging utility. The MCIP flag also detects recursion. The machine-check architecture does not support recursion. When the processor detects machine-check recursion, it enters the shutdown state.

NOTE

For complete operation of the processors machine check capabilities it is essential that the system BIOS enable BINIT# drive and BINIT# observation. This allows the processor to use BINIT# to clear internal and potentially external blocking states and correctly report a wider range of machine check exceptions. For Example, on a Intel(R) Pentium (R) III processor that is executing a locked CMPXCHG8B instruction and a machine check exception is seen on the initial data read, but the comparison operation fails, the processor unlocks the bus after completion of the locked sequence by asserting a BINIT# signal. Without BINIT# drive (UP environment) or BINIT# drive and observation (MP environment) enabled, the machine check error is logged, but the machine check exception is not taken (if MCE's are enabled).

Example 14-2 gives typical steps carried out by a machine-check exception handler.

Example 14-2. Machine-Check Exception Handler Pseudocode

```

IF CPU supports MCE
  THEN
    IF CPU supports MCA
      THEN
        call errorlogging routine; (* returns restartability *)
      FI;
    ELSE (* Pentium(R) processor compatible *)
      READ P5_MC_ADDR
      READ P5_MC_TYPE;
      report RESTARTABILITY to console;
    FI;
  IF error is not restartable
    THEN
      report RESTARTABILITY to console;
      abort system;
    FI;
  CLEAR MCIP flag in IA32_MCG_STATUS;

```

14.7.2. Pentium Processor Machine-Check Exception Handling

To make the machine-check exception handler portable to the Pentium 4, Intel Xeon, P6 family, and Pentium processors, checks can be made (using the CUID instruction) to determine the processor type. Then based on the processor type, machine-check exceptions can be handled specifically for Pentium 4, Intel Xeon, P6 family, or Pentium processors.

When machine-check exceptions are enabled for the Pentium processor (MCE flag is set in control register CR4), the machine-check exception handler uses the RDMSR instruction to read the error type from the P5_MC_TYPE register and the machine check address from the P5_MC_ADDR register. The handler then normally reports these register values to the system console before aborting execution (see Example 14-2).

14.7.3. Logging Correctable Machine-Check Errors

If a machine-check error is correctable, the processor does not generate a machine-check exception for it. To detect correctable machine-check errors, a utility program must be written that reads each of the machine-check error-reporting register banks and logs the results in an accounting file or data structure. This utility can be implemented in either of the following ways:

- A system daemon that polls the register banks on an infrequent basis, such as hourly or daily.
- A user-initiated application that polls the register banks and records the exceptions. Here, the actual polling service is provided by an operating-system driver or through the system call interface.

Example 14-3 gives pseudocode for an error logging utility.

Example 14-3. Machine-Check Error Logging Pseudocode

Assume that execution is restartable;

IF the processor supports MCA

THEN

FOR each bank of machine-check registers

DO

READ IA32_MC_i_STATUS;

IF VAL flag in IA32_MC_i_STATUS = 1

THEN

IF ADDR_V flag in IA32_MC_i_STATUS = 1

THEN READ IA32_MC_i_ADDR;

FI;

IF MISCV flag in IA32_MC_i_STATUS = 1

THEN READ IA32_MC_i_MISC;

FI;

IF MCIP flag in IA32_MCG_STATUS = 1

(* Machine-check exception is in progress *)

AND PCC flag in IA32_MC_i_STATUS = 1

```

        AND RIPV flag in IA32_MCG_STATUS = 0
        (* execution is not restartable *)
        THEN
            RESTARTABILITY = FALSE;
            return RESTARTABILITY to calling procedure;
    FI;
    Save time-stamp counter and processor ID;
    Set IA32_MCi_STATUS to all 0s;
    Execute serializing instruction (i.e., CPUID);
FI;
OD;
FI;
```

If the processor supports the machine-check architecture, the utility reads through the banks of error-reporting registers looking for valid register entries, and then saves the values of the IA32_MCi_STATUS, IA32_MCi_ADDR, IA32_MCi_MISC and IA32_MCG_STATUS registers for each bank that is valid. The routine minimizes processing time by recording the raw data into a system data structure or file, reducing the overhead associated with polling. User utilities analyze the collected data in an off-line environment.

When the MCIP flag is set in the IA32_MCG_STATUS register, a machine-check exception is in progress and the machine-check exception handler has called the exception logging routine.

Once the logging process has been completed the exception-handling routine must determine whether execution can be restarted, which is usually possible when damage has not occurred (The PCC flag is clear, in the IA32_MCi_STATUS register) and when the processor can guarantee that execution is restartable (the RIPV flag is set in the IA32_MCG_STATUS register). If execution cannot be restarted, the system is not recoverable and the exception-handling routine should signal the console appropriately before returning the error status to the Operating System kernel for subsequent shutdown.

The machine-check architecture allows buffering of exceptions from a given error-reporting bank although the Pentium 4, Intel Xeon, and P6 family processors do not implement this feature. The error logging routine should provide compatibility with future processors by reading each hardware error-reporting bank's IA32_MCi_STATUS register and then writing 0s to clear the OVER and VAL flags in this register. The error logging utility should re-read the IA32_MCi_STATUS register for the bank ensuring that the valid bit is clear. The processor will write the next error into the register bank and set the VAL flags.

Additional information that should be stored by the exception-logging routine includes the processor's time-stamp counter value, which provides a mechanism to indicate the frequency of exceptions. A multiprocessing operating system stores the identity of the processor node incurring the exception using a unique identifier, such as the processor's APIC ID (see Section 8.8., "Handling Interrupts").

The basic algorithm given in Example 14-3 can be modified to provide more robust recovery techniques. For example, software has the flexibility to attempt recovery using information unavailable to the hardware. Specifically, the machine-check exception handler can, after logging carefully analyze the error-reporting registers when the error-logging routine reports an error that does not allow execution to be restarted. These recovery techniques can use external bus related model-specific information provided with the error report to localize the source of the error within the system and determine the appropriate recovery strategy.

Debugging and Performance Monitoring

CHAPTER 15

DEBUGGING AND PERFORMANCE MONITORING

The IA-32 architecture provides extensive debugging facilities for use in debugging code and monitoring code execution and processor performance. These facilities are valuable for debugging applications software, system software, and multitasking operating systems.

The debugging support is accessed through the debug registers (DB0 through DB7) and two model-specific registers (MSRs). The debug registers of the IA-32 processors hold the addresses of memory and I/O locations, called breakpoints. Breakpoints are user-selected locations in a program, a data-storage area in memory, or specific I/O ports where a programmer or system designer wishes to halt execution of a program and examine the state of the processor by invoking debugger software. A debug exception (#DB) is generated when a memory or I/O access is made to one of these breakpoint addresses. A breakpoint is specified for a particular form of memory or I/O access, such as a memory read and/or write operation or an I/O read and/or write operation. The debug registers support both instruction breakpoints and data breakpoints. The MSRs (which were introduced into the IA-32 architecture in the P6 family processors) monitor branches, interrupts, and exceptions and record the addresses of the last branch, interrupt or exception taken and the last branch taken before an interrupt or exception.

15.1. OVERVIEW OF THE DEBUGGING SUPPORT FACILITIES

The following processor facilities support debugging and performance monitoring:

- **Debug exception (#DB)**—Transfers program control to a debugger procedure or task when a debug event occurs.
- **Breakpoint exception (#BP)**—Transfers program control to a debugger procedure or task when an INT 3 instruction is executed.
- **Breakpoint-address registers (DR0 through DR3)**—Specifies the addresses of up to 4 breakpoints.
- **Debug status register (DR6)**—Reports the conditions that were in effect when a debug or breakpoint exception was generated.
- **Debug control register (DR7)**—Specifies the forms of memory or I/O access that cause breakpoints to be generated.
- **T (trap) flag, TSS**—Generates a debug exception (#DB) when an attempt is made to switch to a task with the T flag set in its TSS.
- **RF (resume) flag, EFLAGS register**—Suppresses multiple exceptions to the same instruction.
- **TF (trap) flag, EFLAGS register**—Generates a debug exception (#DB) after every execution of an instruction.

- **Breakpoint instruction (INT 3)**—Generates a breakpoint exception (#BP), which transfers program control to the debugger procedure or task. This instruction is an alternative way to set code breakpoints. It is especially useful when more than four breakpoints are desired, or when breakpoints are being placed in the source code.
- **Last branch recording facilities**—See Section 15.5., “Last Branch, Interrupt, and Exception Recording (Pentium 4 and Intel Xeon Processors)” and Section 15.6., “Last Branch, Interrupt, and Exception Recording (P6 Family Processors)”.

These facilities allow a debugger to be called as a separate task or as a procedure in the context of the current program or task. The following conditions can be used to invoke the debugger:

- Task switch to a specific task.
- Execution of the breakpoint instruction.
- Execution of any instruction.
- Execution of an instruction at a specified address.
- Read or write of a byte, word, or doubleword at a specified memory address.
- Write to a byte, word, or doubleword at a specified memory address.
- Input of a byte, word, or doubleword at a specified I/O address.
- Output of a byte, word, or doubleword at a specified I/O address.
- Attempt to change the contents of a debug register.

15.2. DEBUG REGISTERS

The eight debug registers (see Figure 15-1) control the debug operation of the processor. These registers can be written to and read using the move to or from debug register form of the MOV instruction. A debug register may be the source or destination operand for one of these instructions. The debug registers are privileged resources; a MOV instruction that accesses these registers can only be executed in real-address mode, in SMM, or in protected mode at a CPL of 0. An attempt to read or write the debug registers from any other privilege level generates a general-protection exception (#GP).

The primary function of the debug registers is to set up and monitor from 1 to 4 breakpoints, numbered 0 through 3. For each breakpoint, the following information can be specified and detected with the debug registers:

- The linear address where the breakpoint is to occur.
- The length of the breakpoint location (1, 2, or 4 bytes).
- The operation that must be performed at the address for a debug exception to be generated.
- Whether the breakpoint is enabled.
- Whether the breakpoint condition was present when the debug exception was generated.

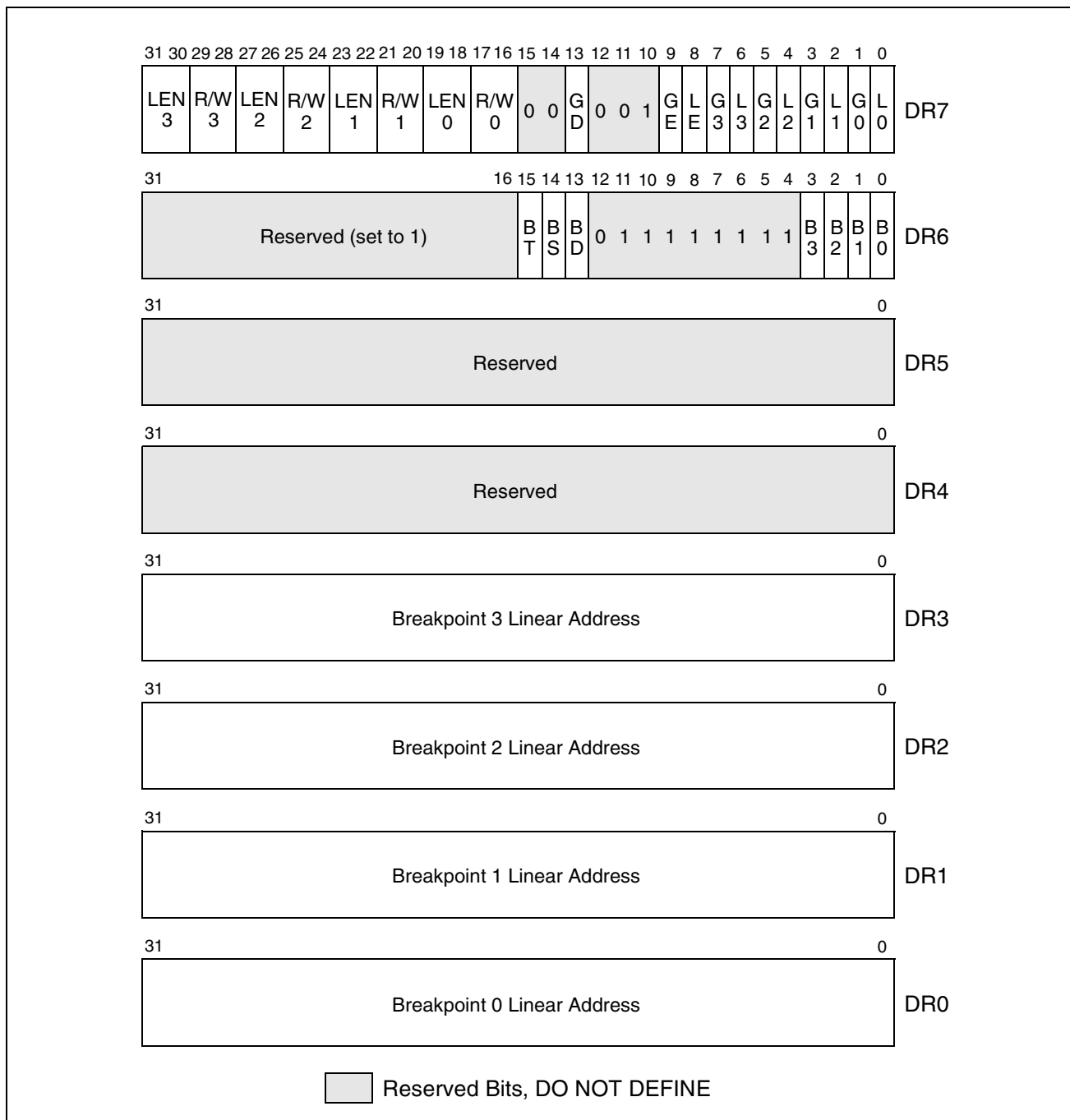


Figure 15-1. Debug Registers

The following paragraphs describe the functions of flags and fields in the debug registers.

15.2.1. Debug Address Registers (DR0-DR3)

Each of the debug-address registers (DR0 through DR3) holds the 32-bit linear address of a breakpoint (see Figure 15-1). Breakpoint comparisons are made before physical address translation occurs. The contents of debug register DR7 further specifies each breakpoint condition.

15.2.2. Debug Registers DR4 and DR5

Debug registers DR4 and DR5 are reserved when debug extensions are enabled (when the DE flag in control register CR4 is set), and attempts to reference the DR4 and DR5 registers cause an invalid-opcode exception (#UD) to be generated. When debug extensions are not enabled (when the DE flag is clear), these registers are aliased to debug registers DR6 and DR7.

15.2.3. Debug Status Register (DR6)

The debug status register (DR6) reports the debug conditions that were sampled at the time the last debug exception was generated (see Figure 15-1). Updates to this register only occur when an exception is generated. The flags in this register show the following information:

B0 through B3 (breakpoint condition detected) flags (bits 0 through 3)

Indicates (when set) that its associated breakpoint condition was met when a debug exception was generated. These flags are set if the condition described for each breakpoint by the LEN_n , and R/W_n flags in debug control register DR7 is true. They are set even if the breakpoint is not enabled by the Ln and Gn flags in register DR7.

BD (debug register access detected) flag (bit 13)

Indicates that the next instruction in the instruction stream will access one of the debug registers (DR0 through DR7). This flag is enabled when the GD (general detect) flag in debug control register DR7 is set. See Section 15.2.4., “Debug Control Register (DR7)”, for further explanation of the purpose of this flag.

BS (single step) flag (bit 14)

Indicates (when set) that the debug exception was triggered by the single-step execution mode (enabled with the TF flag in the EFLAGS register). The single-step mode is the highest-priority debug exception. When the BS flag is set, any of the other debug status bits also may be set.

BT (task switch) flag (bit 15)

Indicates (when set) that the debug exception resulted from a task switch where the T flag (debug trap flag) in the TSS of the target task was set (see Section 6.2.1., “Task-State Segment (TSS)”, for the format of a TSS). There is no flag in debug control register DR7 to enable or disable this exception; the T flag of the TSS is the only enabling flag.

Certain debug exceptions may clear bits 0-3. The remaining contents of the DR6 register are never cleared by the processor. To avoid confusion in identifying debug exceptions, debug handlers should clear the register before returning to the interrupted task.

15.2.4. Debug Control Register (DR7)

The debug control register (DR7) enables or disables breakpoints and sets breakpoint conditions (see Figure 15-1). The flags and fields in this register control the following things:

L0 through L3 (local breakpoint enable) flags (bits 0, 2, 4, and 6)

Enable (when set) the breakpoint condition for the associated breakpoint for the current task. When a breakpoint condition is detected and its associated *L_n* flag is set, a debug exception is generated. The processor automatically clears these flags on every task switch to avoid unwanted breakpoint conditions in the new task.

G0 through G3 (global breakpoint enable) flags (bits 1, 3, 5, and 7)

Enable (when set) the breakpoint condition for the associated breakpoint for all tasks. When a breakpoint condition is detected and its associated *G_n* flag is set, a debug exception is generated. The processor does not clear these flags on a task switch, allowing a breakpoint to be enabled for all tasks.

LE and GE (local and global exact breakpoint enable) flags (bits 8 and 9)

(Not supported in the P6 family processors and later IA-32 processors.) When set, these flags cause the processor to detect the exact instruction that caused a data breakpoint condition. For backward and forward compatibility with other IA-32 processors, Intel recommends that the LE and GE flags be set to 1 if exact breakpoints are required.

GD (general detect enable) flag (bit 13)

Enables (when set) debug-register protection, which causes a debug exception to be generated prior to any MOV instruction that accesses a debug register. When such a condition is detected, the BD flag in debug status register DR6 is set prior to generating the exception. This condition is provided to support in-circuit emulators. (When the emulator needs to access the debug registers, emulator software can set the GD flag to prevent interference from the program currently executing on the processor.) The processor clears the GD flag upon entering to the debug exception handler, to allow the handler access to the debug registers.

R/W0 through R/W3 (read/write) fields (bits 16, 17, 20, 21, 24, 25, 28, and 29)

Specifies the breakpoint condition for the corresponding breakpoint. The DE (debug extensions) flag in control register CR4 determines how the bits in the R/W_{*n*} fields are interpreted. When the DE flag is set, the processor interprets these bits as follows:

- 00—Break on instruction execution only.
- 01—Break on data writes only.
- 10—Break on I/O reads or writes.
- 11—Break on data reads or writes but not instruction fetches.

When the DE flag is clear, the processor interprets the R/W n bits the same as for the Intel386™ and Intel486™ processors, which is as follows:

- 00—Break on instruction execution only.
- 01—Break on data writes only.
- 10—Undefined.
- 11—Break on data reads or writes but not instruction fetches.

LEN0 through LEN3 (Length) fields (bits 18, 19, 22, 23, 26, 27, 30, and 31)

Specify the size of the memory location at the address specified in the corresponding breakpoint address register (DR0 through DR3). These fields are interpreted as follows:

- 00—1-byte length
- 01—2-byte length
- 10—Undefined
- 11—4-byte length

If the corresponding R/W n field in register DR7 is 00 (instruction execution), then the LEN n field should also be 00. The effect of using any other length is undefined. See Section 15.2.5., “Breakpoint Field Recognition”, for further information on the use of these fields.

15.2.5. Breakpoint Field Recognition

The breakpoint address registers (debug registers DR0 through DR3) and the LEN n fields for each breakpoint define a range of sequential byte addresses for a data or I/O breakpoint. The LEN n fields permit specification of a 1-, 2-, or 4-byte range beginning at the linear address specified in the corresponding debug register (DR n). Two-byte ranges must be aligned on word boundaries and 4-byte ranges must be aligned on doubleword boundaries. I/O breakpoint addresses are zero extended from 16 to 32 bits for purposes of comparison with the breakpoint address in the selected debug register. These requirements are enforced by the processor; it uses the LEN n field bits to mask the lower address bits in the debug registers. Unaligned data or I/O breakpoint addresses do not yield the expected results.

A data breakpoint for reading or writing data is triggered if any of the bytes participating in an access is within the range defined by a breakpoint address register and its LEN n field. Table 15-1 gives an example setup of the debug registers and the data accesses that would subsequently trap or not trap on the breakpoints.

A data breakpoint for an unaligned operand can be constructed using two breakpoints, where each breakpoint is byte-aligned, and the two breakpoints together cover the operand. These breakpoints generate exceptions only for the operand, not for any neighboring bytes.

Instruction breakpoint addresses must have a length specification of 1 byte (the LEN n field is set to 00). The behavior of code breakpoints for other operand sizes is undefined. The processor recognizes an instruction breakpoint address only when it points to the first byte of an instruction. If the instruction has any prefixes, the breakpoint address must point to the first prefix.

15.3. DEBUG EXCEPTIONS

The IA-32 processors dedicate two interrupt vectors to handling debug exceptions: vector 1 (debug exception, #DB) and vector 3 (breakpoint exception, #BP). The following sections describe how these exceptions are generated and typical exception handler operations for handling these exceptions.

Table 15-1. Breakpointing Examples

Debug Register Setup			
Debug Register	R/W <i>n</i>	Breakpoint Address	LEN <i>n</i>
DR0	R/W0 = 11 (Read/Write)	A0001H	LEN0 = 00 (1 byte)
DR1	R/W1 = 01 (Write)	A0002H	LEN1 = 00 (1 byte)
DR2	R/W2 = 11 (Read/Write)	B0002H	LEN2 = 01) (2 bytes)
DR3	R/W3 = 01 (Write)	C0000H	LEN3 = 11 (4 bytes)
Data Accesses			
Operation		Address	Access Length (In Bytes)
Data operations that trap			
- Read or write		A0001H	1
- Read or write		A0001H	2
- Write		A0002H	1
- Write		A0002H	2
- Read or write		B0001H	4
- Read or write		B0002H	1
- Read or write		B0002H	2
- Write		C0000H	4
- Write		C0001H	2
- Write		C0003H	1
Data operations that do not trap			
- Read or write		A0000H	1
- Read		A0002H	1
- Read or write		A0003H	4
- Read or write		B0000H	2
- Read		C0000H	2
- Read or write		C0004H	4

15.3.1. Debug Exception (#DB)—Interrupt Vector 1

The debug-exception handler is usually a debugger program or is part of a larger software system. The processor generates a debug exception for any of several conditions. The debugger can check flags in the DR6 and DR7 registers to determine which condition caused the exception and which other conditions might also apply. Table 15-2 shows the states of these flags following the generation of each kind of breakpoint condition.

Instruction-breakpoint and general-detect conditions (see Section 15.3.1.3., “General-Detect Exception Condition”) result in faults; other debug-exception conditions result in traps. The debug exception may report either or both at one time. The following sections describe each class of debug exception. See Chapter 5, “Interrupt 1—Debug Exception (#DB)”, for additional information about this exception.

15.3.1.1. INSTRUCTION-BREAKPOINT EXCEPTION CONDITION

The processor reports an instruction breakpoint when it attempts to execute an instruction at an address specified in a breakpoint-address register (DB0 through DR3) that has been set up to detect instruction execution (R/W flag is set to 0). Upon reporting the instruction breakpoint, the processor generates a fault-class, debug exception (#DB) before it executes the target instruction for the breakpoint. Instruction breakpoints are the highest priority debug exceptions and are guaranteed to be serviced before any other exceptions that may be detected during the decoding or execution of an instruction.

Table 15-2. Debug Exception Conditions

Debug or Breakpoint Condition	DR6 Flags Tested	DR7 Flags Tested	Exception Class
Single-step trap	BS = 1		Trap
Instruction breakpoint, at addresses defined by DR n and LEN n	B n = 1 and (G n or L n = 1)	R/W n = 0	Fault
Data write breakpoint, at addresses defined by DR n and LEN n	B n = 1 and (G n or L n = 1)	R/W n = 1	Trap
I/O read or write breakpoint, at addresses defined by DR n and LEN n	B n = 1 and (G n or L n = 1)	R/W n = 2	Trap
Data read or write (but not instruction fetches), at addresses defined by DR n and LEN n	B n = 1 and (G n or L n = 1)	R/W n = 3	Trap
General detect fault, resulting from an attempt to modify debug registers (usually in conjunction with in-circuit emulation)	BD = 1		Fault
Task switch	BT = 1		Trap

Because the debug exception for an instruction breakpoint is generated before the instruction is executed, if the instruction breakpoint is not removed by the exception handler, the processor will detect the instruction breakpoint again when the instruction is restarted and generate another debug exception. To prevent looping on an instruction breakpoint, the IA-32 architecture provides the RF flag (resume flag) in the EFLAGS register (see Section 2.3., “System Flags and Fields in the EFLAGS Register”). When the RF flag is set, the processor ignores instruction breakpoints.

All IA-32 processors manage the RF flag as follows. The processor sets the RF flag automatically prior to calling an exception handler for any fault-class exception except a debug exception that was generated in response to an instruction breakpoint. For debug exceptions resulting from instruction breakpoints, the processor does not set the RF flag prior to calling the debug exception handler. The debug exception handler then has the option of disabling the instruction breakpoint or setting the RF flag in the EFLAGS image on the stack. If the RF flag in the EFLAGS image is set when the processor returns from the exception handler, it is copied into the RF flag in the EFLAGS register by the IRETD or task switch instruction that causes the return. The processor then ignores instruction breakpoints for the duration of the next instruction. (Note that the POPF, POPFD, and IRET instructions do not transfer the RF image into the EFLAGS

register.) Setting the RF flag does not prevent other types of debug-exception conditions (such as, I/O or data breakpoints) from being detected, nor does it prevent non-debug exceptions from being generated. After the instruction is successfully executed, the processor clears the RF flag in the EFLAGS register, except after an IRETD instruction or after a JMP, CALL, or INT *n* instruction that causes a task switch.

(Note that the processor also does not set the RF flag when calling exception or interrupt handlers for trap-class exceptions, for hardware interrupts, or for software-generated interrupts.)

For the Pentium processor, when an instruction breakpoint coincides with another fault-type exception (such as a page fault), the processor may generate one spurious debug exception after the second exception has been handled, even though the debug exception handler set the RF flag in the EFLAGS image. To prevent this spurious exception with Pentium processors, all fault-class exception handlers should set the RF flag in the EFLAGS image.

15.3.1.2. DATA MEMORY AND I/O BREAKPOINT EXCEPTION CONDITIONS

Data memory and I/O breakpoints are reported when the processor attempts to access a memory or I/O address specified in a breakpoint-address register (DB0 through DR3) that has been set up to detect data or I/O accesses (R/W flag is set to 1, 2, or 3). The processor generates the exception after it executes the instruction that made the access, so these breakpoint condition causes a trap-class exception to be generated.

Because data breakpoints are traps, the original data is overwritten before the trap exception is generated. If a debugger needs to save the contents of a write breakpoint location, it should save the original contents before setting the breakpoint. The handler can report the saved value after the breakpoint is triggered. The address in the debug registers can be used to locate the new value stored by the instruction that triggered the breakpoint.

The Intel486 and later IA-32 processors ignore the GE and LE flags in DR7. In the Intel386 processor, exact data breakpoint matching does not occur unless it is enabled by setting the LE and/or the GE flags.

The P6 family processors, however, are unable to report data breakpoints exactly for the REP MOVS and REP STOS instructions until the completion of the iteration after the iteration in which the breakpoint occurred.

For repeated INS and OUTS instructions that generate an I/O-breakpoint debug exception, the processor generates the exception after the completion of the first iteration. Repeated INS and OUTS instructions generate an I/O-breakpoint debug exception after the iteration in which the memory address breakpoint location is accessed.

15.3.1.3. GENERAL-DETECT EXCEPTION CONDITION

When the GD flag in DR7 is set, the general-detect debug exception occurs when a program attempts to access any of the debug registers (DR0 through DR7) at the same time they are being used by another application, such as an emulator or debugger. This additional protection feature guarantees full control over the debug registers when required. The debug exception handler can detect this condition by checking the state of the BD flag of the DR6 register. The processor

generates the exception before it executes the MOV instruction that accesses a debug register, which causes a fault-class exception to be generated.

15.3.1.4. SINGLE-STEP EXCEPTION CONDITION

The processor generates a single-step debug exception if (while an instruction is being executed) it detects that the TF flag in the EFLAGS register is set. The exception is a trap-class exception, because the exception is generated after the instruction is executed. (Note that the processor does not generate this exception after an instruction that sets the TF flag. For example, if the POPF instruction is used to set the TF flag, a single-step trap does not occur until after the instruction that follows the POPF instruction.)

The processor clears the TF flag before calling the exception handler. If the TF flag was set in a TSS at the time of a task switch, the exception occurs after the first instruction is executed in the new task.

The TF flag normally is not cleared by privilege changes inside a task. The INT *n* and INTO instructions, however, do clear this flag. Therefore, software debuggers that single-step code must recognize and emulate INT *n* or INTO instructions rather than executing them directly. To maintain protection, the operating system should check the CPL after any single-step trap to see if single stepping should continue at the current privilege level.

The interrupt priorities guarantee that, if an external interrupt occurs, single stepping stops. When both an external interrupt and a single-step interrupt occur together, the single-step interrupt is processed first. This operation clears the TF flag. After saving the return address or switching tasks, the external interrupt input is examined before the first instruction of the single-step handler executes. If the external interrupt is still pending, then it is serviced. The external interrupt handler does not run in single-step mode. To single step an interrupt handler, single step an INT *n* instruction that calls the interrupt handler.

15.3.1.5. TASK-SWITCH EXCEPTION CONDITION

The processor generates a debug exception after a task switch if the T flag of the new task's TSS is set. This exception is generated after program control has passed to the new task, and prior to the execution of the first instruction of that task. The exception handler can detect this condition by examining the BT flag of the DR6 register.

Note that, if the debug exception handler is a task, the T bit of its TSS should not be set. Failure to observe this rule will put the processor in a loop.

15.3.2. Breakpoint Exception (#BP)—Interrupt Vector 3

The breakpoint exception (interrupt 3) is caused by execution of an INT 3 instruction (see Chapter 5, “Interrupt 3—Breakpoint Exception (#BP)”). Debuggers use break exceptions in the same way that they use the breakpoint registers; that is, as a mechanism for suspending program execution to examine registers and memory locations. With earlier IA-32 processors, breakpoint exceptions are used extensively for setting instruction breakpoints.

With the Intel386 and later IA-32 processors, it is more convenient to set breakpoints with the breakpoint-address registers (DR0 through DR3). However, the breakpoint exception still is useful for breakpointing debuggers, because the breakpoint exception can call a separate exception handler. The breakpoint exception is also useful when it is necessary to set more breakpoints than there are debug registers or when breakpoints are being placed in the source code of a program under development.

Note that with Pentium M processors, #BPs for fast string operations are reported only on cache line boundaries.

15.4. LAST BRANCH RECORDING OVERVIEW

The P6 family processors introduced the ability to set breakpoints on taken branches, interrupts, and exceptions, and to single-step from one branch to the next. This capability was modified and extended in the Pentium 4 and Intel Xeon processors to allow the logging of branch trace messages in a branch trace store (BTS) buffer in memory. See the following sections for descriptions of the two mechanisms for last branch recording:

- Section 15.5., “Last Branch, Interrupt, and Exception Recording (Pentium 4 and Intel Xeon Processors)”
- Section 15.6., “Last Branch, Interrupt, and Exception Recording (P6 Family Processors)”

The IA-32 branch instructions that are tracked with the last branch recording mechanism are the JMP, Jcc, LOOP, and CALL instructions.

15.5. LAST BRANCH, INTERRUPT, AND EXCEPTION RECORDING (PENTIUM 4 AND INTEL XEON PROCESSORS)

The Pentium 4 and Intel Xeon processors provide the following methods of recording taken branches, interrupts and exceptions:

- Store branch records in the last branch record (LBR) stack MSRs for the most recent taken branches, interrupts, and/or exceptions in MSRs. A branch record consist of a branch-from and a branch-to instruction address.
- Send the branch records out on the system bus as branch trace messages (BTMs).
- Log BTMs in a memory-resident branch trace store (BTS) buffer.

To support these functions, the processor provides the following six MSRs:

- IA32_DEBUGCTL MSR—Enables last branch, interrupt, and exception recording; single-stepping on taken branches; branch trace messages (BTMs); and branch trace store (BTS). This register is named DebugCtlMSR in the P6 family processors.
- Debug store (DS) feature flag (bit 21), returned by the CPUID instruction—Indicates that the processor provides the debug store (DS) mechanism, which allows BTMs to be stored in a memory-resident BTS buffer.

- IA32_MISC_ENABLE MSR—Indicates that the processor provides the BTS facilities.
- Last Branch Record (LBR) Stack—The LBR stack is a circular stack that consists of four MSRs:
 - MSR_LASTBRANCH_0
 - MSR_LASTBRANCH_1
 - MSR_LASTBRANCH_2
 - MSR_LASTBRANCH_3

When enabled, the processor records a branch record in one of these MSRs for each taken branch, interrupt, or exception. When the registers are full, the processor wraps around the register stack and begins overwriting the registers.

- Last Branch Record Top-of-Stack (TOS) Pointer—The MSR_LASTBRANCH_TOS MSR contains a 2-bit pointer (0, 1, 2, or 3) to the MSR in the LBR stack that contains the most recent branch, interrupt, or exception recorded.
- Last Exception Record—See Section 15.5.6., “Last Exception Records (Pentium 4 and Intel Xeon Processors)”.

The following sections describe the IA32_DEBUGCTL MSR and the various last branch recording mechanisms. See Appendix B, *Model-Specific Registers (MSRs)*, for a detailed description of each of the last branch recording MSRs described above.

15.5.1. IA32_DEBUGCTL MSR (Pentium 4 and Intel Xeon Processors)

The IA32_DEBUGCTL MSR enables and disables the various last branch recording mechanisms described in the previous section. This register can be written to using the WRMSR instruction, when operating at privilege level 0 or when in real-address mode. A protected-mode operating system procedure is required to provide user access to this register. Figure 15-2 shows the flags in the IA32_DEBUGCTL MSR. The functions of these flags are as follows:

LBR (last branch/interrupt/exception) flag (bit 0)—When set, the processor records a running trace of the most recent branches, interrupts, and/or exceptions taken by the processor (prior to a debug exception being generated) in the last branch record (LBR) stack. Each branch, interrupt, or exception is recorded as a 64-bit branch record (see Section 15.5.2., “LBR Stack (Pentium 4 and Intel Xeon Processors)”). The processor clears this flag whenever a debug exception is generated (for example, when an instruction or data breakpoint or a single-step trap occurs).

BTF (single-step on branches) flag (bit 1)

When set, the processor treats the TF flag in the EFLAGS register as a “single-step on branches” flag rather than a “single-step on instructions” flag. This mechanism allows single-stepping the processor on taken branches, interrupts, and exceptions. See Section 15.5.4., “Single-Stepping on Branches, Excep-

tions, and Interrupts” for more information about the BTF flag.

TR (trace message enable) flag (bit 2)

When set, branch trace messages are enabled. Thereafter, when the processor detects a taken branch, interrupt, or exception, it sends the branch record out on the system bus as a branch trace message (BTM). See Section 15.5.5., “Branch Trace Messages” for more information about the TR flag.

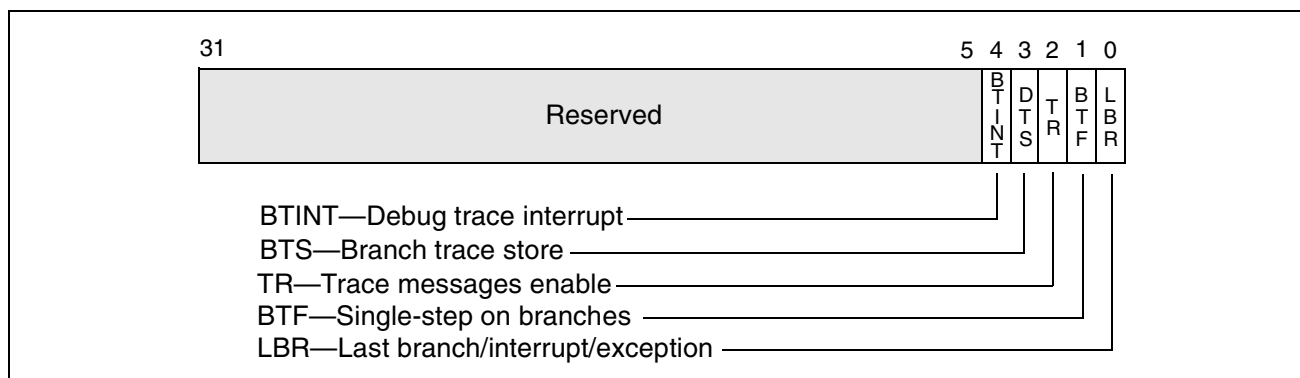


Figure 15-2. IA32_DEBUGCTL MSR (Pentium 4 and Intel Xeon Processors)

BTS (branch trace store) flag (bit 3)

When set, enables the BTS facilities to log BTMs to a memory-resident BTS buffer that is part of the DS save area (see Section 15.9.5., “DS Save Area”).

BTINT (branch trace interrupt) flag (bits 4)

When set, the BTS facilities generate an interrupt when the BTS buffer is full. When clear, BTMs are logged to the BTS buffer in a circular fashion. (See Section 15.5.7., “Branch Trace Store (BTS)” for a description of this mechanism.)

15.5.2. LBR Stack (Pentium 4 and Intel Xeon Processors)

The LBR stack is made up of four LBR MSRs (see Figure 15-3), that are treated by the processor as a circular stack. The TOS pointer (MSR_LASTBRANCH_TOS MSR) points to the LBR MSR that contains the most recent (last) branch record placed on the stack. Prior to placing a new branch record on the stack, the TOS is incremented by 1. When the TOS pointer reaches 3, it wraps around to 0. Figure 15-4 shows the layout of the MSR_LASTBRANCH_TOS MSR.

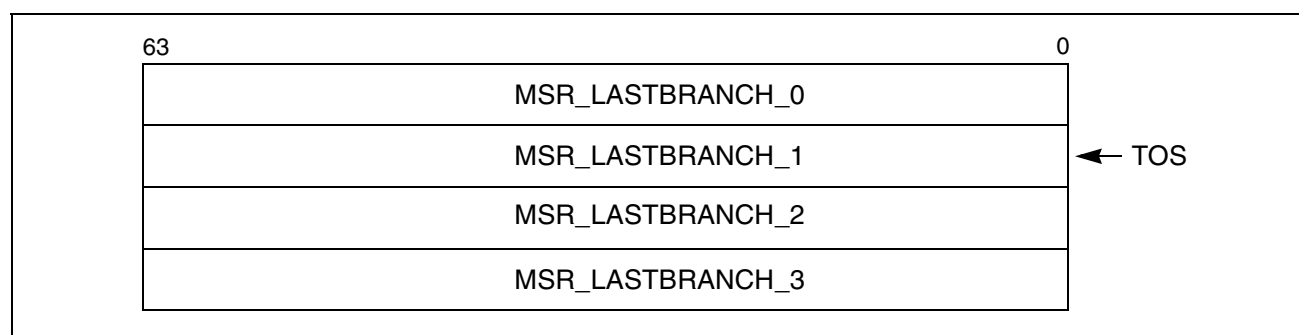


Figure 15-3. LBR MSR Stack Structure

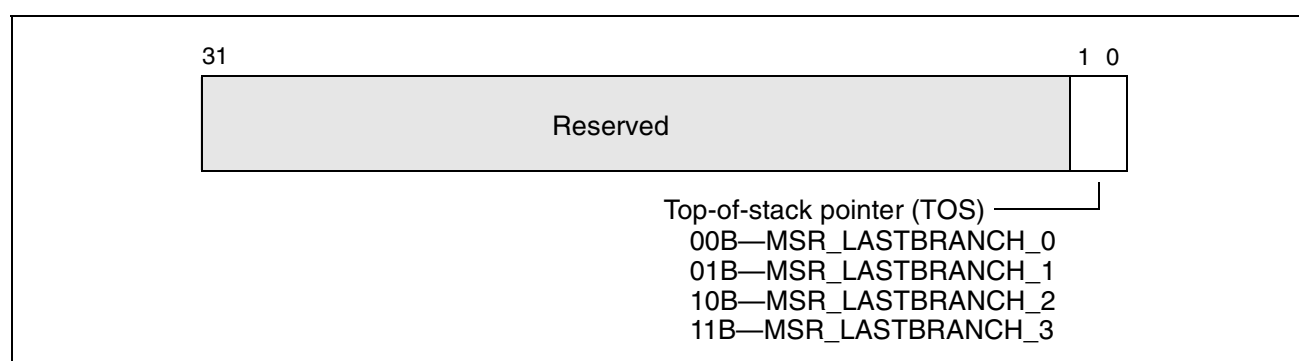


Figure 15-4. MSR_LASTBRANCH_TOS MSR Layout

The registers in the LBR MSR stack and the MSR_LASTBRANCH_TOS MSR are read-only and can be read using the RDMSR instruction.

Figure 15-5 shows the layout of a branch record in an LBR MSR. Each branch record consists of two linear addresses, which represent the “from” and “to” instruction pointers for a branch, interrupt, or exception. The contents of the from and to addresses differ, depending on the source of the branch:

- **Taken Branch**—If the record is for a taken branch, the “from” address is the address of the branch instruction and the “to” address is the target instruction of the branch.
- **Interrupt**—If the record is for an interrupt, the “from” address the return instruction pointer (RIP) saved for the interrupt and the “to” address is the address of the first instruction in the interrupt handler routine. The RIP is the linear address of the next instruction to be executed upon returning from the interrupt handler.
- **Exception**—If the record is for an exception, the “from” address is the linear address of the instruction that caused the exception to be generated and the “to” address is the address of the first instruction in the exception handler routine.

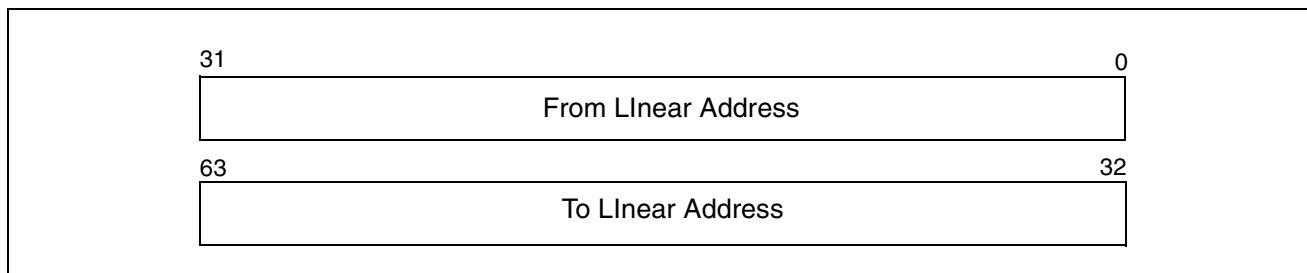


Figure 15-5. LBR MSR Branch Record Layout

Additional information is saved if an exception or interrupt occurs in conjunction with a branch instruction. If a branch instruction generates a trap type exception, two branch records are stored in the LBR stack: a branch record for the branch instruction followed by a branch record for the exception.

If a branch instruction generates a fault type exception, a branch record is stored in the LBR stack for the exception, but not for the branch instruction itself. Here, the location of the branch instruction can be determined from the CS and EIP registers in the exception stack frame that is written by the processor onto the stack.

If a branch instruction is immediately followed by an interrupt, a branch record is stored in the LBR stack for the branch instruction followed by a record for the interrupt.

15.5.3. Monitoring Branches, Exceptions, and Interrupts (Pentium 4 and Intel Xeon Processors)

When the LBR flag in the IA32_DEBUGCTL MSR is set, the processor automatically begins recording branch records for taken branches, interrupts, and exceptions (except for debug exceptions) in the LBR stack MSRs.

When the processor generates a debug exception (#DB), it automatically clears the LBR flag before executing the exception handler. This action does not clear previously stored LBR stack MSRs. The branch record for the last four taken branches, interrupts and/or exceptions are retained for analysis.

A debugger can use the linear addresses in the LBR stack to reset breakpoints in the break-point address registers (DR0 through DR3). This allows a backward trace from the manifestation of a particular bug toward its source.

If the LBR flag is cleared and TR flag in the IA32_DEBUGCTLTR MSR remains set, the processor will continue to update LBR stack MSRs. This is because BTM information must be generated from entries in the LBR stack (see 14.5.5). A #DB does not automatically clear the TR flag.

15.5.4. Single-Stepping on Branches, Exceptions, and Interrupts

When software sets both the BTF flag in the IA32_DEBUGCTL MSR and the TF flag in the EFLAGS register, the processor generates a single-step debug exception the next time it takes a branch, services an interrupt, or generates an exception. This mechanism allows the debugger to single-step on control transfers caused by branches, interrupts, and exceptions. This “control-flow single stepping” helps isolate a bug to a particular block of code before instruction single-stepping further narrows the search. If the BTF flag is set when the processor generates a debug exception, the processor clears the BTF flag along with the TF flag. The debugger must reset the BTF and TF flags before resuming program execution to continue control-flow single stepping.

15.5.5. Branch Trace Messages

Setting The TR flag in the IA32_DEBUGCTL MSR enables branch trace messages (BTMs). Thereafter, when the processor detects a branch, exception, or interrupt, it sends a branch record out on the system bus as a BTM. A debugging device that is monitoring the system bus can read these messages and synchronize operations with taken branch, interrupt, and exception events.

When interrupts or exceptions occur in conjunction with a taken branch, additional BTMs are sent out on the bus, as described in Section 15.5.3., “Monitoring Branches, Exceptions, and Interrupts (Pentium 4 and Intel Xeon Processors)”.

Setting this flag greatly reduces the performance of the processor.

Unlike the P6 family processors, the Pentium 4 and Intel Xeon processors can collect branch records in the LBR stack MSRs while at the same time sending BTMs out on the system bus when both the TR and LBR flags are set in the IA32_DEBUGCTL MSR.

15.5.6. Last Exception Records (Pentium 4 and Intel Xeon Processors)

The Pentium 4 and Intel Xeon processors provide two 32 bit MSRs (the MSR_LER_TO_LIP and the MSR_LER_FROM_LIP MSRs) that duplicate the functions of the LastExceptionToIP and LastExceptionFromIP MSRs found in the P6 family processors. The MSR_LER_TO_LIP and MSR_LER_FROM_LIP MSRs contain a branch record for the last branch that the processor took prior to an exception or interrupt being generated.

15.5.7. Branch Trace Store (BTS)

A trace of taken branches, interrupts, and exceptions is useful for debugging code by providing a method of determining the decision path taken to reach a particular code location. The Pentium 4 and Intel Xeon processors provide a mechanism for capturing records of taken branches, interrupts, and exceptions and saving them in the last branch record (LBR) stack MSRs and/or sending them out onto the system bus as BTMs. The branch trace store (BTS) mechanism provides the additional capability of saving the branch records in a memory-resident BTS buffer, which is part of the DS save area (see Section 15.9.5., “DS Save Area”). The BTS buffer can be

configured to be circular so that the most recent branch records are always available or it can be configured to generate an interrupt when the buffer is nearly full so that all the branch records can be saved.

15.5.7.1. DETECTION OF THE BTS FACILITIES

The DS feature flag (bit 21) returned by the CPUID instruction indicates (when set) the availability of the DS mechanism in the processor, which supports the BTS (and PEBS) facilities. When this bit is set, the following BTS facilities are available:

- The `BTS_UNAVAILABLE` flag in the `IA32_MISC_ENABLE` MSR indicates (when clear) the availability of the BTS facilities, including the ability to set the BTS and BTINT bits in the `IA32_DEBUGCTL` MSR.
- The `IA32_DS_AREA` MSR can be programmed to point to the DS save area.

15.5.7.2. SETTING UP THE DS SAVE AREA

To save branch records with the BTS buffer, the DS save area must first be set up in memory as described in the following procedure. See Section 15.5.7.3., “Setting Up the BTS Buffer” and Section 15.9.8.3., “Setting Up the PEBS Buffer” for instructions for setting up a BTS buffer and/or a PEBS buffer, respectively, in the DS save area:

1. Create the DS buffer management information area in memory (see Section 15.9.5., “DS Save Area”, for layout information). See additional notes in this section.
2. Write the base linear address of the DS buffer management area into the `IA32_DS_AREA` MSR.
3. Set up the performance counter entry in the xAPIC LVT for fixed delivery and edge sensitive. See Section 8.5.1., “Local Vector Table”.
4. Establish an interrupt handler in the IDT for the vector associated with the performance counter entry in the xAPIC LVT.
5. Write an interrupt service routine to handle the interrupt (see Section 15.5.7.4., “Writing the DS Interrupt Service Routine”).

The following restrictions should be applied to the DS save area.

- The three DS save area sections should be allocated from a non-paged pool, and marked accessed and dirty. It is the responsibility of the operating system to keep the pages that contain the buffer present and to mark them accessed and dirty. The implication is that the operating system cannot do “lazy” page-table entry propagation for these pages.
- The DS save area can be larger than a page, but the pages must be mapped to contiguous linear addresses. The buffer may share a page, so it need not be aligned on a 4-KByte boundary. For performance reasons, the base of the buffer must be aligned on a doubleword boundary and should be aligned on a cache line boundary.
- It is recommended that the buffer size for the BTS buffer and the PEBS buffer be an integer multiple of the corresponding record sizes.

- The precise event records buffer should be large enough to hold the number of precise event records that can occur while waiting for the interrupt to be serviced.
- The DS save area should be in kernel space. It must not be on the same page as code, to avoid triggering self-modifying code actions.
- There are no memory type restrictions on the buffers, although it is recommended that the buffers be designated as WB memory type for performance considerations.
- Either the system must be prevented from entering A20M mode while DS save area is active, or bit 20 of all addresses within buffer bounds must be 0.
- Pages that contain buffers must be mapped to the same physical addresses for all processes, such that any change to control register CR3 will not change the DS addresses.
- The DS save area is expected to be used only on systems with an enabled APIC. The LVT Performance Counter entry in the APCI must be initialized to use an interrupt gate instead of the trap gate.

15.5.7.3. SETTING UP THE BTS BUFFER

Three flags in the IA32_DEBUGCTL MSR (see Table 15-3) control the generation of branch records and storing of them in the BTS buffer: TR, BTS, and BTINT. The TR flag enables the generation of BTMs. The BTS flag determines whether the BTMs are sent out on the system bus (clear) or stored in the BTS buffer (set). BTMs cannot be simultaneously sent to the system bus and logged in the BTS buffer. The BTINT flag enables the generation of an interrupt when the BTS buffer is full. When this flag is clear, the BTS buffer is a circular buffer.

Table 15-3. IA32_DEBUGCTL MSR Flag Encodings

TR	BTS	BTINT	Description
0	X	X	Branch trace messages (BTMs) off
1	0	X	Generate BTMs
1	1	0	Store BTMs in the BTS buffer, used here as a circular buffer
1	1	1	Store BTMs in the BTS buffer, and generate an interrupt when the buffer is nearly full

The following procedure describes how to set up a Pentium 4 or Intel Xeon processor to collect branch records in the BTS buffer in the DS save area:

1. Place values in the BTS buffer base, BTS index, BTS absolute maximum, and BTS interrupt threshold fields of the DS buffer management area to set up the BTS buffer in memory.
2. Set the TR and BTS flags in the IA32_DEBUGCTL MSR.
3. Either clear the BTINT flag in the IA32_DEBUGCTL MSR (to set up a circular BTS buffer) or set the BTINT flag (to generate an interrupt when the BTS buffer is nearly full).

15.5.7.4. WRITING THE DS INTERRUPT SERVICE ROUTINE

The BTS, non-precise event-based sampling, and PEBS facilities share the same interrupt vector and interrupt service routine (called the debug store interrupt service routine or DS ISR). To handle BTS, non-precise event-based sampling, and PEBS interrupts, separate handler routines must be included in the DS ISR. Use the following guidelines when writing a DS ISR to handle BTS, non-precise event-based sampling, and/or PEBS interrupts.

- The DS interrupt service routine (ISR) must be part of a kernel driver and operate at a current privilege level of 0 to secure the buffer storage area.
- Because the BTS, non-precise event-based sampling, and PEBS facilities share the same interrupt vector, the DS ISR must check for all the possible causes of interrupts from these facilities and pass control on to the appropriate handler.

BTS and PEBS buffer overflow would be the sources of the interrupt if the buffer index matches/exceeds the interrupt threshold specified. Detection of non-precise event-based sampling as the source of the interrupt is accomplished by checking for counter overflow.

- There must be separate save areas, buffers, and state for each processor in an MP system.
- Upon entering the ISR, branch trace messages and PEBS should be disabled to prevent race conditions during access to the DS save area. This is done by clearing TR flag in the IA32_DEBUGCTL MSR and by clearing the precise event enable flag in the IA32_PEBS_ENABLE MSR. These settings should be restored to their original values when exiting the ISR.
- The processor will not disable the DS save area when the buffer is full and the circular mode has not been selected. The current DS setting must be retained and restored by the ISR on exit.
- After reading the data in the appropriate buffer, up to but not including the current index into the buffer, the ISR must reset the buffer index to the beginning of the buffer. Otherwise, everything up to the index will look like new entries upon the next invocation of the ISR.
- The ISR must clear the mask bit in the performance counter LVT entry.
- The ISR must re-enable the CCCR's ENABLE bit if it is servicing an overflow PMI due to PEBS.

15.6. LAST BRANCH, INTERRUPT, AND EXCEPTION RECORDING (P6 FAMILY PROCESSORS)

The P6 family processors provide five MSRs for recording the last branch, interrupt, or exception taken by the processor: DebugCtlMSR, LastBranchToIP, LastBranchFromIP, LastExceptionToIP, and LastExceptionFromIP. These registers can be used to collect last branch records, to set breakpoints on branches, interrupts, and exceptions, and to single-step from one branch to the next.

See Appendix B, *Model-Specific Registers (MSRs)*, for a detailed description of each of the last branch recording MSRs described above.

15.6.1. DebugCtlMSR Register (P6 Family Processors)

The version of the DebugCtlMSR register found in the P6 family processors enables last branch, interrupt, and exception recording; taken branch breakpoints; the breakpoint reporting pins; and trace messages. This register can be written to using the WRMSR instruction, when operating at privilege level 0 or when in real-address mode. A protected-mode operating system procedure is required to provide user access to this register. Figure 15-6 shows the flags in the DebugCtlMSR register for the P6 family processors. The functions of these flags are as follows:

LBR (last branch/interrupt/exception) flag (bit 0)

When set, the processor records the source and target addresses (in the LastBranchToIP, LastBranchFromIP, LastExceptionToIP, and LastExceptionFromIP MSRs) for the last branch and the last exception or interrupt taken by the processor prior to a debug exception being generated. The processor clears this flag whenever a debug exception, such as an instruction or data breakpoint or single-step trap occurs.

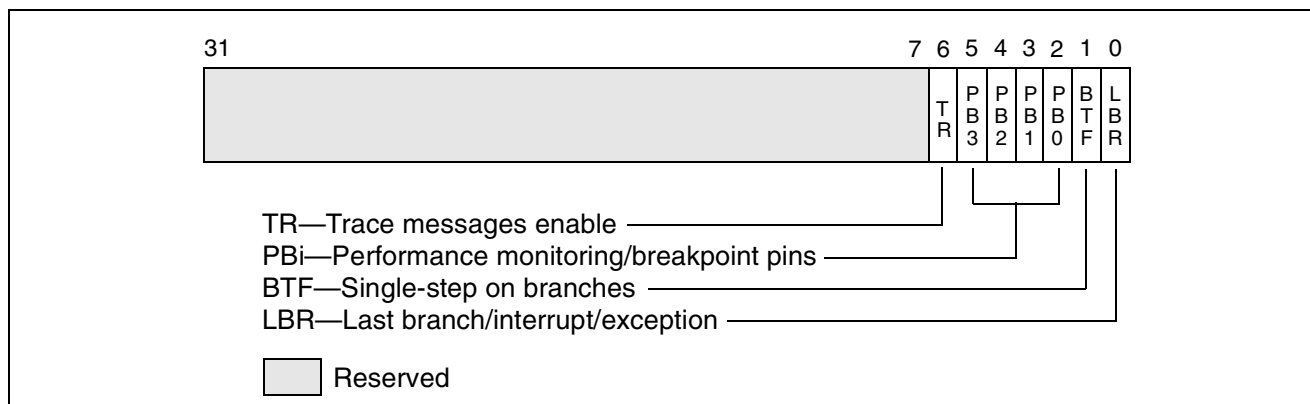


Figure 15-6. DebugCtlMSR Register (P6 Family Processors)

BTF (single-step on branches) flag (bit 1)

When set, the processor treats the TF flag in the EFLAGS register as a “single-step on branches” flag (see Section 15.5.4., “Single-Stepping on Branches, Exceptions, and Interrupts”).

PBi (performance monitoring/breakpoint pins) flags (bits 2 through 5)

When these flags are set, the performance monitoring/breakpoint pins on the processor (BP0#, BP1#, BP2#, and BP3#) report breakpoint matches in the corresponding breakpoint-address registers (DR0 through DR3). The processor asserts then deasserts the corresponding BPi# pin when a breakpoint match occurs. When a PBi flag is clear, the performance monitoring/breakpoint pins report performance events. Processor execution is not affected by reporting performance events.

TR (trace message enable) flag (bit 6)

When set, trace messages are enabled as described in Section 15.5.5., “Branch Trace Messages”. Setting this flag greatly reduces the performance of the processor. When trace messages are enabled, the values stored in the LastBranchToIP, LastBranchFromIP, LastExceptionToIP, and LastExceptionFromIP MSRs are undefined.

15.6.2. Last Branch and Last Exception MSRs (P6 Family Processors)

The LastBranchToIP and LastBranchFromIP MSRs are 32-bit registers for recording the instruction pointers for the last branch, interrupt, or exception that the processor took prior to a debug exception being generated. When a branch occurs, the processor loads the address of the branch instruction into the LastBranchFromIP MSR and loads the target address for the branch into the LastBranchToIP MSR.

When an interrupt or exception occurs (other than a debug exception), the address of the instruction that was interrupted by the exception or interrupt is loaded into the LastBranchFromIP MSR and the address of the exception or interrupt handler that is called is loaded into the LastBranchToIP MSR.

The LastExceptionToIP and LastExceptionFromIP MSRs (also 32-bit registers) record the instruction pointers for the last branch that the processor took prior to an exception or interrupt being generated. When an exception or interrupt occurs, the contents of the LastBranchToIP and LastBranchFromIP MSRs are copied into these registers before the to and from addresses of the exception or interrupt are recorded in the LastBranchToIP and LastBranchFromIP MSRs.

These registers can be read using the RDMSR instruction.

Note that the values stored in the LastBranchToIP, LastBranchFromIP, LastExceptionToIP, and LastExceptionFromIP MSRs are offsets into the current code segment, as opposed to linear addresses, which are saved in last branch records for the Pentium 4 and Intel Xeon processors.

15.6.3. Monitoring Branches, Exceptions, and Interrupts (P6 Family Processors)

When the LBR flag in the DebugCtlMSR register is set, the processor automatically begins recording branches that it takes, exceptions that are generated (except for debug exceptions), and interrupts that are serviced. Each time a branch, exception, or interrupt occurs, the processor records the to and from instruction pointers in the LastBranchToIP and LastBranchFromIP MSRs. In addition, for interrupts and exceptions, the processor copies the contents of the LastBranchToIP and LastBranchFromIP MSRs into the LastExceptionToIP and LastExceptionFromIP MSRs prior to recording the to and from addresses of the interrupt or exception.

When the processor generates a debug exception (#DB), it automatically clears the LBR flag before executing the exception handler, but does not touch the last branch and last exception MSRs. The addresses for the last branch, interrupt, or exception taken are thus retained in the LastBranchToIP and LastBranchFromIP MSRs and the addresses of the last branch prior to an interrupt or exception are retained in the LastExceptionToIP, and LastExceptionFromIP MSRs.

The debugger can use the last branch, interrupt, and/or exception addresses in combination with code-segment selectors retrieved from the stack to reset breakpoints in the breakpoint-address registers (DR0 through DR3), allowing a backward trace from the manifestation of a particular bug toward its source. Because the instruction pointers recorded in the LastBranchToIP, LastBranchFromIP, LastExceptionToIP, and LastExceptionFromIP MSRs are offsets into a code segment, software must determine the segment base address of the code segment associated with

the control transfer to calculate the linear address to be placed in the breakpoint-address registers. The segment base address can be determined by reading the segment selector for the code segment from the stack and using it to locate the segment descriptor for the segment in the GDT or LDT. The segment base address can then be read from the segment descriptor.

Before resuming program execution from a debug-exception handler, the handler must set the LBR flag again to re-enable last branch and last exception/interrupt recording.

15.7. TIME-STAMP COUNTER

The IA-32 architecture (beginning with the Pentium processor) defines a time-stamp counter mechanism that can be used to monitor and identify the relative time of occurrence of processor events. The time-stamp counter architecture includes the time-stamp counter (IA32_TIME_STAMP_COUNTER MSR [called the TSC MSR in the P6 family and Pentium processors]), an instruction for reading the time-stamp counter (RDTSC), a feature bit (TSC flag) that can be read with the CPUID instruction, and a time-stamp counter disable bit (TSD flag) in control register CR4.

Following execution of the CPUID instruction, the TSC flag in register EDX (bit 4) indicates (when set) that the time-stamp counter is present in a particular IA-32 processor implementation. (See “CPUID—CPU Identification” in Chapter 3 of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*.)

The time-stamp counter (as implemented in the Pentium 4, Intel Xeon, P6 family, and Pentium processors) is a 64-bit counter that is set to 0 following the hardware reset of the processor. Following reset, the counter is incremented every processor clock cycle, even when the processor is halted by the HLT instruction or the external STPCLK# pin. However, the assertion of the external DPSLP# pin may cause the time-stamp counter to stop and Intel SpeedStep® technology transitions may cause the frequency at which the time-stamp counter increments to change in accordance with the processor's internal clock frequency.

The RDTSC instruction reads the time-stamp counter and is guaranteed to return a monotonically increasing unique value whenever executed, except for 64-bit counter wraparound. Intel guarantees, architecturally, that the time-stamp counter frequency and configuration will be such that it will not wraparound within 10 years after being reset to 0. The period for counter wrap is several thousands of years in the Pentium 4, Intel Xeon, P6 family, and Pentium processors.

Normally, the RDTSC instruction can be executed by programs and procedures running at any privilege level and in virtual-8086 mode. The TSD flag in control register CR4 (bit 2) allows use of this instruction to be restricted to only programs and procedures running at privilege level 0. A secure operating system would set the TSD flag during system initialization to disable user access to the time-stamp counter. An operating system that disables user access to the time-stamp counter should emulate the instruction through a user-accessible programming interface.

The RDTSC instruction is not serializing or ordered with other instructions. Thus, it does not necessarily wait until all previous instructions have been executed before reading the counter. Similarly, subsequent instructions may begin execution before the RDTSC instruction operation is performed.

The RDMSR and WRMSR instructions can read and write the time-stamp counter, respectively, as an MSR (at MSR address 10H). In the Pentium 4, Intel Xeon, and P6 family processors, all 64-bits of the time-stamp counter can be read with the RDMSR instruction (just as with the RDTSC instruction). However, when the WRMSR instruction is used to write to the time-stamp counter, only the low order 32-bits of the time-stamp counter can be written to, and the high-order 32 bits cleared to all 0s.

15.8. PERFORMANCE MONITORING OVERVIEW

Performance monitoring was introduced to the IA-32 architecture in the Pentium processor with a set of model-specific performance-monitoring counter MSRs. These counters permit a selection of processor performance parameters to be monitored and measured. The information obtained from these counters can then be used for tuning system and compiler performance.

In the Intel P6 family of processors, the performance monitoring mechanism was modified and enhanced to permit a wider selection of events to be monitored and to allow greater control over the choice of the events to be monitored.

The Pentium 4 and Intel Xeon processors introduced a new performance monitoring mechanism and new set of performance events that can be counted.

The performance monitoring mechanisms and performance events defined for the Pentium, P6 family, Pentium 4, and Intel Xeon processors are not architectural. They are all model specific and are not compatible among the three IA-32 processor families.

The following sections describe the performance monitoring mechanisms for the Pentium 4, Intel Xeon, P6 family, and Pentium processors, respectively:

- Section 15.9., “Performance Monitoring (Pentium 4 and Intel Xeon Processors)”
- Section 15.11., “performance Monitoring (P6 Family Processor)”
- Section 15.12., “Performance Monitoring (Pentium Processors)”

15.9. PERFORMANCE MONITORING (PENTIUM 4 AND INTEL XEON PROCESSORS)

The performance monitoring mechanism provided in the Pentium 4 and Intel Xeon processors is considerably different from that provided in the P6 family and Pentium processors. While the general concept of selecting, filtering, counting, and reading performance events through the WRMSR, RDMSR, and RDPMC instructions is unchanged, the setup mechanism and MSR layouts are different and incompatible with the P6 family and Pentium processor mechanisms. Also, the RDPMC instruction has been enhanced to read the additional performance counters provided in the Pentium 4 and Intel Xeon processors and to allow faster reading of the counters.

The event monitoring mechanism provided with the Pentium 4 and Intel Xeon processors consists of the following facilities:

- The IA32_MISC_ENABLE MSR, which indicates the availability in an IA-32 processor of the performance monitoring and precise event-based sampling (PEBS) facilities.
- 45 event selection control (ESCR) MSRs for selecting events to be monitored with specific performance counters.
- 18 performance counter MSRs for counting events.
- 18 counter configuration control (CCCR) MSRs, with one CCCR associated with each performance counter. Each CCCR sets up its associated performance counter for a specific method or style of counting.
- A debug store (DS) save area in memory for storing PEBS records.
- The IA32_DS_AREA MSR, which establishes the location of the DS save area.
- The debug store (DS) feature flag (bit 21) returned by the CPUID instruction, which indicates the availability in an IA-32 processor of the DS mechanism.
- The IA32_PEBS_ENABLE MSR, which enables the PEBS facilities and replay tagging used in at-retirement event counting.
- A set of predefined events and event metrics that simplify the setting up of the performance counters to count specific events.

Table 15-4 lists the performance counters and their associated CCCRs, along with the ESCRs that select events to be counted for each performance counter. The predefined event metrics and events are listed in Table in Appendix A, *Performance-Monitoring Events*.

Table 15-4. Performance Counter MSRs and Associated CCCR and ESCR MSRs (Pentium 4 and Intel Xeon Processors)

Counter			CCCR		ESCR		
Name	No.	Addr	Name	Addr	Name	No.	Addr
MSR_BPU_COUNTER0	0	300H	MSR_BPU_CCCR0	360H	MSR_BSU_ESCR0	7	3A0H
					MSR_FSB_ESCR0	6	3A2H
					MSR_MOB_ESCR0	2	3AAH
					MSR_PMH_ESCR0	4	3ACH
					MSR_BPU_ESCR0	0	3B2H
					MSR_IS_ESCR0	1	3B4H
					MSR_ITLB_ESCR0	3	3B6H
					MSR_IX_ESCR0	5	3C8H
MSR_BPU_COUNTER1	1	301H	MSR_BPU_CCCR1	361H	MSR_BSU_ESCR0	7	3A0H
					MSR_FSB_ESCR0	6	3A2H
					MSR_MOB_ESCR0	2	3AAH
					MSR_PMH_ESCR0	4	3ACH
					MSR_BPU_ESCR0	0	3B2H
					MSR_IS_ESCR0	1	3B4H
					MSR_ITLB_ESCR0	3	3B6H
					MSR_IX_ESCR0	5	3C8H

Table 15-4. Performance Counter MSRs and Associated CCCR and ESCR MSRs (Pentium 4 and Intel Xeon Processors) (Contd.)

Counter			CCCR		ESCR		
Name	No.	Addr	Name	Addr	Name	No.	Addr
MSR_BPU_COUNTER2	2	302H	MSR_BPU_CCCR2	362H	MSR_BSU_ESCR1	7	3A1H
					MSR_FSB_ESCR1	6	3A3H
					MSR_MOB_ESCR1	2	3ABH
					MSR_PMH_ESCR1	4	3ADH
					MSR_BPU_ESCR1	0	3B3H
					MSR_IS_ESCR1	1	3B5H
					MSR_ITLB_ESCR1	3	3B7H
MSR_IX_ESCR1	5	3C9H					
MSR_BPU_COUNTER3	3	303H	MSR_BPU_CCCR3	363H	MSR_BSU_ESCR1	7	3A1H
					MSR_FSB_ESCR1	6	3A3H
					MSR_MOB_ESCR1	2	3ABH
					MSR_PMH_ESCR1	4	3ADH
					MSR_BPU_ESCR1	0	3B3H
					MSR_IS_ESCR1	1	3B5H
					MSR_ITLB_ESCR1	3	3B7H
MSR_IX_ESCR1	5	3C9H					
MSR_MS_COUNTER0	4	304H	MSR_MS_CCCR0	364H	MSR_MS_ESCR0	0	3C0H
					MSR_TBPU_ESCR0	2	3C2H
					MSR_TC_ESCR0	1	3C4H
MSR_MS_COUNTER1	5	305H	MSR_MS_CCCR1	365H	MSR_MS_ESCR0	0	3C0H
					MSR_TBPU_ESCR0	2	3C2H
					MSR_TC_ESCR0	1	3C4H
MSR_MS_COUNTER2	6	306H	MSR_MS_CCCR2	366H	MSR_MS_ESCR1	0	3C1H
					MSR_TBPU_ESCR1	2	3C3H
					MSR_TC_ESCR1	1	3C5H
MSR_MS_COUNTER3	7	307H	MSR_MS_CCCR3	367H	MSR_MS_ESCR1	0	3C1H
					MSR_TBPU_ESCR1	2	3C3H
					MSR_TC_ESCR1	1	3C5H
MSR_FLAME_COUNTER0	8	308H	MSR_FLAME_CCCR0	368H	MSR_FIRM_ESCR0	1	3A4H
					MSR_FLAME_ESCR0	0	3A6H
					MSR_DAC_ESCR0	5	3A8H
					MSR_SAA_T_ESCR0	2	3AEH
MSR_U2L_ESCR0	3	3B0H					
MSR_FLAME_COUNTER1	9	309H	MSR_FLAME_CCCR1	369H	MSR_FIRM_ESCR0	1	3A4H
					MSR_FLAME_ESCR0	0	3A6H
					MSR_DAC_ESCR0	5	3A8H
					MSR_SAA_T_ESCR0	2	3AEH
MSR_U2L_ESCR0	3	3B0H					
MSR_FLAME_COUNTER2	10	30AH	MSR_FLAME_CCCR2	36AH	MSR_FIRM_ESCR1	1	3A5H
					MSR_FLAME_ESCR1	0	3A7H
					MSR_DAC_ESCR1	5	3A9H
					MSR_SAA_T_ESCR1	2	3AFH
MSR_U2L_ESCR1	3	3B1H					
MSR_FLAME_COUNTER3	11	30BH	MSR_FLAME_CCCR3	36BH	MSR_FIRM_ESCR1	1	3A5H
					MSR_FLAME_ESCR1	0	3A7H
					MSR_DAC_ESCR1	5	3A9H
					MSR_SAA_T_ESCR1	2	3AFH
MSR_U2L_ESCR1	3	3B1H					
MSR_IQ_COUNTER0	12	30CH	MSR_IQ_CCCR0	36CH	MSR_CRU_ESCR0	4	3B8H
					MSR_CRU_ESCR2	5	3CCH
					MSR_CRU_ESCR4	6	3E0H
					MSR_IQ_ESCR0	0	3BAH
					MSR_RAT_ESCR0	2	3BCH
					MSR_SSU_ESCR0	3	3BEH
					MSR_ALF_ESCR0	1	3CAH

Table 15-4. Performance Counter MSRs and Associated CCCR and ESCR MSRs (Pentium 4 and Intel Xeon Processors) (Contd.)

Counter			CCCR		ESCR		
Name	No.	Addr	Name	Addr	Name	No.	Addr
MSR_IQ_COUNTER1	13	30DH	MSR_IQ_CCCR1	36DH	MSR_CRU_ESCR0	4	3B8H
					MSR_CRU_ESCR2	5	3CCH
					MSR_CRU_ESCR4	6	3E0H
					MSR_IQ_ESCR0	0	3BAH
					MSR_RAT_ESCR0	2	3BCH
					MSR_SSU_ESCR0	3	3BEH
					MSR_ALF_ESCR0	1	3CAH
MSR_IQ_COUNTER2	14	30EH	MSR_IQ_CCCR2	36EH	MSR_CRU_ESCR1	4	3B9H
					MSR_CRU_ESCR3	5	3CDH
					MSR_CRU_ESCR5	6	3E1H
					MSR_IQ_ESCR1	0	3BBH
					MSR_RAT_ESCR1	2	3BDH
					MSR_ALF_ESCR1	1	3CBH
MSR_IQ_COUNTER3	15	30FH	MSR_IQ_CCCR3	36FH	MSR_CRU_ESCR1	4	3B9H
					MSR_CRU_ESCR3	5	3CDH
					MSR_CRU_ESCR5	6	3E1H
					MSR_IQ_ESCR1	0	3BBH
					MSR_RAT_ESCR1	2	3BDH
					MSR_ALF_ESCR1	1	3CBH
MSR_IQ_COUNTER4	16	310H	MSR_IQ_CCCR4	370H	MSR_CRU_ESCR0	4	3B8H
					MSR_CRU_ESCR2	5	3CCH
					MSR_CRU_ESCR4	6	3E0H
					MSR_IQ_ESCR0	0	3BAH
					MSR_RAT_ESCR0	2	3BCH
					MSR_SSU_ESCR0	3	3BEH
					MSR_ALF_ESCR0	1	3CAH
MSR_IQ_COUNTER5	17	311H	MSR_IQ_CCCR5	371H	MSR_CRU_ESCR1	4	3B9H
					MSR_CRU_ESCR3	5	3CDH
					MSR_CRU_ESCR5	6	3E1H
					MSR_IQ_ESCR1	0	3BBH
					MSR_RAT_ESCR1	2	3BDH
					MSR_ALF_ESCR1	1	3CBH

The types of events that can be counted with these performance monitoring facilities are divided into two classes: non-retirement events and at-retirement events.

- Non-retirement events (see Table A-1) are events that occur any time during instruction execution (such as bus transactions or cache transactions).
- At-retirement events (see Table A-2) are events that are counted at the retirement stage of instruction execution, which allows finer granularity in counting events and capturing machine state. The at-retirement counting mechanism includes facilities for tagging μ ops that have encountered a particular performance event during instruction execution. Tagging allows events to be sorted between those that occurred on an execution path that resulted in architectural state being committed at retirement as well as events that occurred on an execution path where the results were eventually cancelled and never committed to architectural state (such as, the execution of a mispredicted branch).

The Pentium 4 and Intel Xeon processors' performance monitoring facilities support the three usage models described below. The first two models can be used to count both non-retirement and at-retirement events, the third model can be used only to count a subset of at-retirement events:

- **Event counting.** A performance counter is configured to count one or more types of events. While the counter is counting, software reads the counter at selected intervals to determine the number of events that have been counted between the intervals.
- **Non-precise event-based sampling.** A performance counter is configured to count one or more types of events and to generate an interrupt when it overflows. To trigger an overflow, the counter is preset to a modulus value that will cause the counter to overflow after a specific number of events have been counted. When the counter overflows, the processor generates a performance monitoring interrupt (PMI). The interrupt service routine for the PMI then records the return instruction pointer (RIP), resets the modulus, and restarts the counter. Code performance can be analyzed by examining the distribution of RIPs with a tool like the VTune™ Performance Analyzer.
- **Precise event-based sampling (PEBS).** This type of performance monitoring is similar to non-precise event-based sampling, except that a memory buffer is used to save a record of the architectural state of the processor whenever the counter overflows. The records of architectural state provide additional information for use in performance tuning. Precise event-based sampling can be used to count only a subset of at-retirement events.

The following sections describe the MSRs and data structures used for performance monitoring in the Pentium 4 and Intel Xeon processors, then describes how these facilities are used with the three usage models described above.

15.9.1. ESCR MSRs

The 45 ESCR MSRs (see Table 15-4) allow software to select specific events to be countered. Each ESCR is usually associated with a pair of performance counters (see Table 15-4), and each performance counter has several ESCRs associated with it (allowing the events to be counted to be selected from a variety of events).

Figure 15-7 shows the layout of an ESCR MSR. The functions of the flags and fields are as follows:

USR flag, bit 2

When set, events are counted when the processor is operating at a current privilege level (CPL) of 1, 2, or 3. These privilege levels are generally used by application code and unprotected operating system code.

OS flag, bit 3

When set, events are counted when the processor is operating at CPL of 0. This privilege level is generally reserved for protected operating system code. (When both the OS and USR flags are set, events are counted at all privilege levels.)

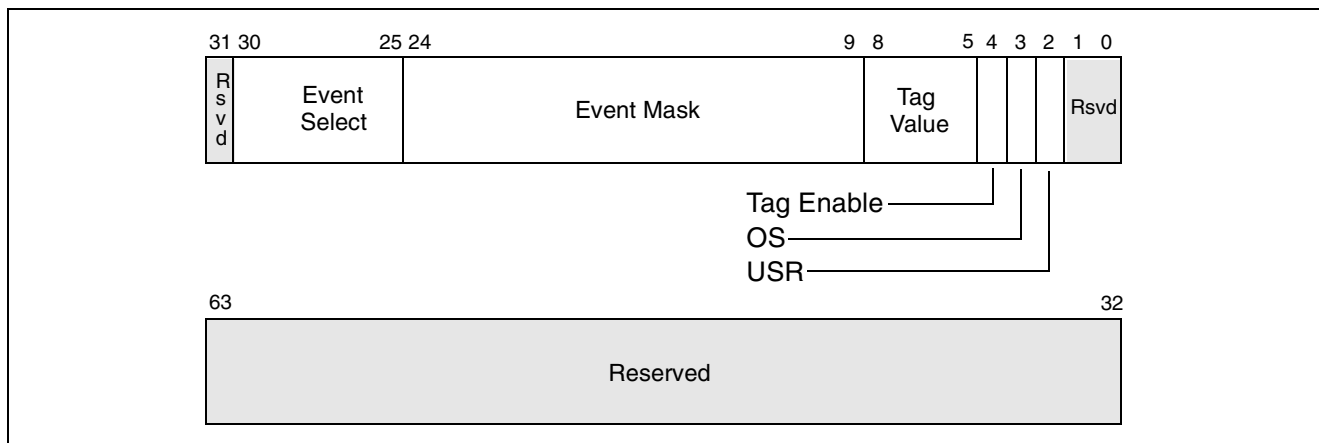


Figure 15-7. Event Selection Control Register (ESCR) for Pentium 4 and Intel Xeon processors without HT Technology support

Tag Enable, bit 4

When set, enables tagging of μ ops to assist in at-retirement event counting; when clear, disables tagging. See Section 15.9.7., “At-Retirement Counting”.

Tag Value field, bits 5 through 8

Selects a tag value to associate with a μ op to assist in at-retirement event counting.

Event Mask field, bits 9 through 24

Selects events to be counted from the event class selected with the event select field.

Event Select field, bits 25 through 30

Selects a class of events to be counted. The events within this class that are counted are selected with the event mask field.

When setting up an ESCR, the event select field is used to select a specific class of events to count, such as retired branches. The event mask field is then used to select one or more of the specific events within the class to be counted. For example, when counting retired branches, four different events can be counted: branch not taken predicted, branch not taken mispredicted, branch taken predicted, and branch taken mispredicted. The OS and USR flags allow counts to be enabled for events that occur when operating system code and/or application code are being executed. If neither the OS nor USR flag is set, no events will be counted.

The ESCRs are initialized to all 0s on reset. The flags and fields of an ESCR are configured by writing to the ESCR using the WRMSR instruction. Table 15-4 gives the addresses of the ESCR MSRs.

NOTE

Writing to an ESCR MSR does not enable counting with its associated performance counter; it only selects the event or events to be counted. The CCCR for the selected performance counter must also be configured. Configuration of the CCCR includes selecting the ESCR and enabling the counter.

15.9.2. Performance Counters

The performance counters in conjunction with the counter configuration control registers (CCCRs) are used for filtering and counting the events selected by the ESCRs. The Pentium 4 and Intel Xeon processors provide 18 performance counters organized into 9 pairs. A pair of performance counters is associated with a particular subset of events and ESCR's (see Table 15-4). The counter pairs are partitioned into four groups:

- The BPU group, includes two performance counter pairs:
 - MSR_BPU_COUNTER0 and MSR_BPU_COUNTER1
 - MSR_BPU_COUNTER2 and MSR_BPU_COUNTER3.
- The MS group, includes two performance counter pairs:
 - MSR_MS_COUNTER0 and MSR_MS_COUNTER1.
 - MSR_MS_COUNTER2 and MSR_MS_COUNTER3.
- The FLAME group, includes two performance counter pairs:
 - MSR_FLAME_COUNTER0 and MSR_FLAME_COUNTER1.
 - MSR_FLAME_COUNTER2 and MSR_FLAME_COUNTER3.
- The IQ group, includes three performance counter pairs:
 - MSR_IQ_COUNTER0 and MSR_IQ_COUNTER1.
 - MSR_IQ_COUNTER2 and MSR_IQ_COUNTER3.
 - MSR_IQ_COUNTER4 and MSR_IQ_COUNTER5.

The MSR_IQ_COUNTER4 counter in the IQ group provides support for the PEBS.

Alternate counters in each group can be cascaded: the first counter in one pair can start the first counter in the second pair and vice versa. A similar cascading is possible for the second counters in each pair. For example, within the BPU group of counters, MSR_BPU_COUNTER0 can start MSR_BPU_COUNTER2 and vice versa, and MSR_BPU_COUNTER1 can start MSR_BPU_COUNTER3 and vice versa (see Section 15.9.6.6., “Cascading Counters”). The cascade flag in the CCCR register for the performance counter enables the cascading of counters.

Each performance counter is 40-bits wide (see Figure 15-8). The RDPMC instruction has been enhanced in the Pentium 4 and Intel Xeon processors to allow reading of either the full counter-width (40-bits) or the low 32-bits of the counter. Reading the low 32-bits is faster than reading the full counter width and is appropriate in situations where the count is small enough to be contained in 32 bits.

The RDPMC instruction can be used by programs or procedures running at any privilege level and in virtual-8086 mode to read these counters. The PCE flag in control register CR4 (bit 8) allows the use of this instruction to be restricted to only programs and procedures running at privilege level 0.

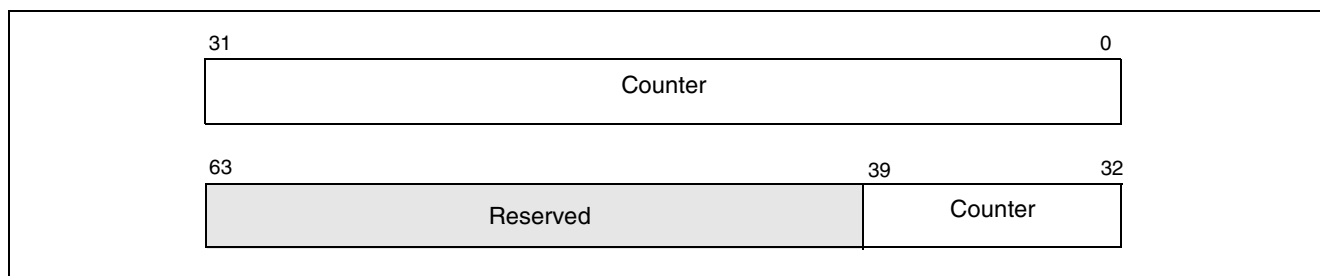


Figure 15-8. Performance Counter (Pentium 4 and Intel Xeon Processors)

The RDPMC instruction is not serializing or ordered with other instructions. Thus, it does not necessarily wait until all previous instructions have been executed before reading the counter. Similarly, subsequent instructions may begin execution before the RDPMC instruction operation is performed.

Only the operating system, executing at privilege level 0, can directly manipulate the performance counters, using the RDMSR and WRMSR instructions. A secure operating system would clear the PCE flag during system initialization to disable direct user access to the performance-monitoring counters, but provide a user-accessible programming interface that emulates the RDPMC instruction.

Some uses of the performance counters require the counters to be preset before counting begins (that is, before the counter is enabled). This can be accomplished by writing to the counter using the WRMSR instruction. To set a counter to a specified number of counts before overflow, enter a 2s complement negative integer in the counter. The counter will then count from the preset value up to -1 and overflow. Writing to a performance counter in a Pentium 4 or Intel Xeon processor with the WRMSR instruction causes all 40 bits of the counter to be written.

15.9.3. CCCR MSRs

Each of the 18 performance counters in a Pentium 4 or Intel Xeon processor has one CCCR MSR associated with it (see Table 15-4). The CCCRs control the filtering and counting of events as well as interrupt generation. Figure 15-9 shows the layout of an CCCR MSR. The functions of the flags and fields are as follows:

Enable flag, bit 12

When set, enables counting; when clear, the counter is disabled. This flag is cleared on reset.

ESCR Select field, bits 13 through 15

Identifies the ESCR to be used to select events to be counted with the counter associated with the CCCR.

Compare flag, bit 18

When set, enables filtering of the event count; when clear, disables filtering. The filtering method is selected with the threshold, complement, and edge flags.

Complement flag, bit 19

Selects how the incoming event count is compared with the threshold value. When set, event counts that are less than or equal to the threshold value result in a single count being delivered to the performance counter; when clear, counts greater than the threshold value result in a count being delivered to the performance counter (see Section 15.9.6.2., “Filtering Events”). The complement flag is not active unless the compare flag is set.

Threshold field, bits 20 through 23

Selects the threshold value to be used for comparisons. The processor examines this field only when the compare flag is set, and uses the complement flag setting to determine the type of threshold comparison to be made. The useful range of values that can be entered in this field depend on the type of event being counted (see Section 15.9.6.2., “Filtering Events”).

Edge flag, bit 24

When set, enables rising edge (false-to-true) edge detection of the threshold comparison output for filtering event counts; when clear, rising edge detection is disabled. This flag is active only when the compare flag is set.

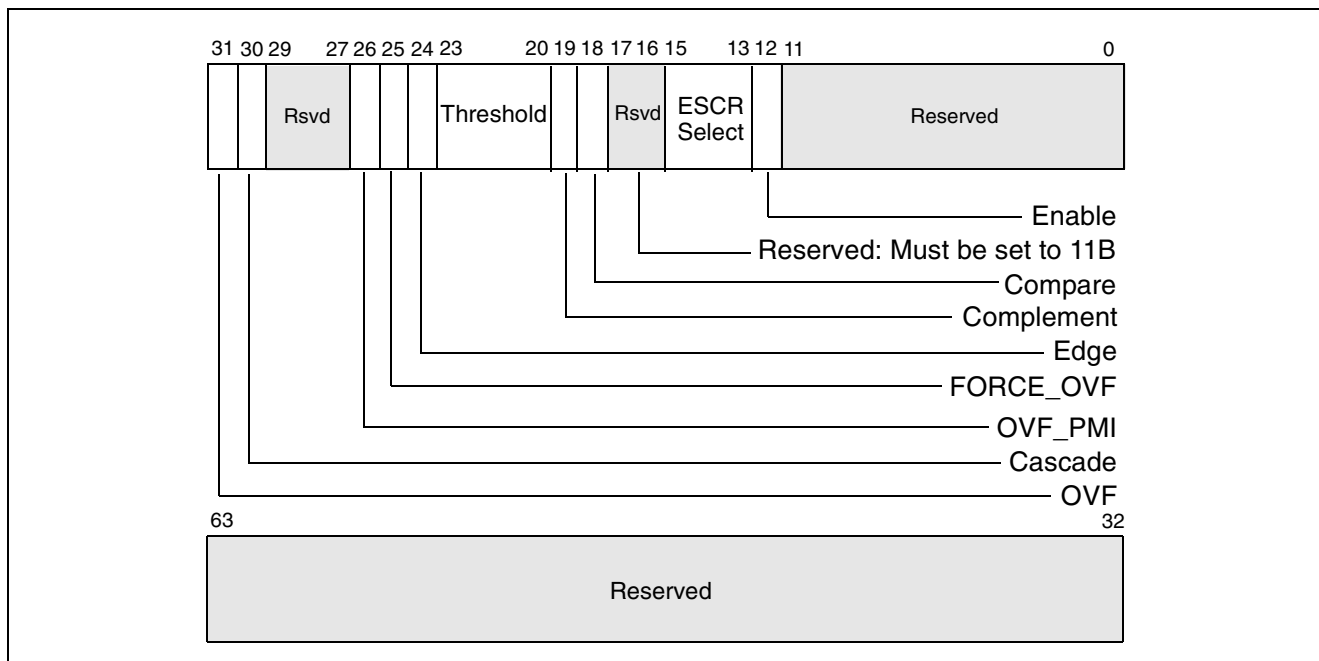


Figure 15-9. Counter Configuration Control Register (CCCR)

FORCE_OVF flag, bit 25

When set, forces a counter overflow on every counter increment; when clear, overflow only occurs when the counter actually overflows.

OVF_PMI flag, bit 26

When set, causes a performance monitor interrupt (PMI) to be generated when the counter overflows occurs; when clear, disables PMI generation. Note that the PMI is generated on the next event count after the counter has overflowed.

Cascade flag, bit 30

When set, enables counting on one counter of a counter pair when its alternate counter in the other the counter pair in the same counter group overflows (see Section 15.9.2., “Performance Counters” for further details); when clear, disables cascading of counters.

OVF flag, bit 31

Indicates that the counter has overflowed when set. This flag is a sticky flag that must be explicitly cleared by software.

The CCCRs are initialized to all 0s on reset.

The events that an enabled performance counter actually counts are selected and filtered by the following flags and fields in the ESCR and CCCR registers and in the qualification order given:

1. The event select and event mask fields in the ESCR select a class of events to be counted and one or more event types within the class, respectively.
2. The OS and USR flags in the ESCR selected the privilege levels at which events will be counted.

3. The ESCR select field of the CCCR selects the ESCR. Since each counter has several ESCRs associated with it, one ESCR must be chosen to select the classes of events that may be counted.
4. The compare and complement flags and the threshold field of the CCCR select an optional threshold to be used in qualifying an event count.
5. The edge flag in the CCCR allows events to be counted only on rising-edge transitions.

The qualification order in the above list implies that the filtered output of one “stage” forms the input for the next. For instance, events filtered using the privilege level flags can be further qualified by the compare and complement flags and the threshold field, and an event that matched the threshold criteria, can be further qualified by edge detection.

The uses of the flags and fields in the CCCRs are discussed in greater detail in Section 15.9.6., “Programming the Performance Counters for Non-Retirement Events”.

15.9.4. Debug Store (DS) Mechanism

The debug store (DS) mechanism was introduced in the Pentium 4 and Intel Xeon processors to allow various types of information to be collected in memory-resident buffers for use in debugging and tuning programs. For the Pentium 4 and Intel Xeon processors, the DS mechanism is used to collect two types of information: branch records and precise event-based sampling (PEBS) records (see Section 15.5.7., “Branch Trace Store (BTS)” and Section 15.9.8., “Precise Event-Based Sampling (PEBS)” for a description of these facilities.) The availability of the DS mechanism in a processor is indicated with the DS feature flag (bit 21) returned by the CPUID instruction.

Records collected with the DS mechanism are saved in the DS save area (see Section 15.9.5., “DS Save Area”).

15.9.5. DS Save Area

The debug store (DS) save area is a software-designated area of memory that is used to collect the following two types of information:

- **Branch Records.** When the BTS flag in the IA32_DEBUGCTL MSR is set, a branch record is stored in the BTS buffer in the DS save area whenever a taken branch, interrupt, or exception is detected.
- **PEBS Records.** When a performance counter is configured for PEBS, a PEBS record is stored in the PEBS buffer in the DS save area whenever a counter overflow occurs. This record contains the architectural state of the processor (state of the 8 general purpose registers, EIP register, and EFLAGS register) at the time of the event that caused the counter to overflow. When the state information has been logged, the counter is automatically reset to a preselected value, and event counting begins again. This feature is available only for a subset of the Pentium 4 and Intel Xeon processors’ performance events.

NOTE

DS save area and recording mechanism is not available in the SMM. The feature is disabled on transition to the SMM mode. Similarly DS recording is disabled on the generation of a machine check exception and is cleared on processor RESET and INIT. DS recording is available in real address mode.

The BTS and PEBS facilities may not be available on all IA-32 processors. The availability of these facilities is indicated with the `BTS_UNAVAILABLE` and `PEBS_UNAVAILABLE` flags, respectively, in the `IA32_MISC_ENABLE` MSR (see Table B-1).

The DS save area is divided into three parts (see Figure 15-10): buffer management area, branch trace store (BTS) buffer, and PEBS buffer. The buffer management area is used to define the location and size of the BTS and PEBS buffers. The processor then uses the buffer management area to keep track of the branch and/or PEBS records in their respective buffers and to record the performance counter reset value. The linear address of the first byte of the DS buffer management area is specified with the `IA32_DS_AREA` MSR.

The fields in the buffer management area are as follows:

BTS buffer base

Linear address of the first byte of the BTS buffer. This address should point to a natural doubleword boundary.

BTS Index

Linear address of the first byte of the next BTS record to be written to. Initially, this address should be the same as the address in the BTS buffer base field.

BTS absolute maximum

Linear address of the next byte past the end of the BTS buffer. This address should be a multiple of the BTS record size (12 bytes) plus 1.

BTS interrupt threshold

Linear address of the BTS record on which an interrupt is to be generated. This address must point to an offset from the BTS buffer base that is a multiple of the BTS record size. Also, it must be several records short of the BTS absolute maximum address to allow a pending interrupt to be handled prior to processor writing the BTS absolute maximum record.

PEBS buffer base

Linear address of the first byte of the PEBS buffer. This address should point to a natural doubleword boundary.

PEBS index

Linear address of the first byte of the next PEBS record to be written to. Initially, this address should be the same as the address in the PEBS buffer base field.

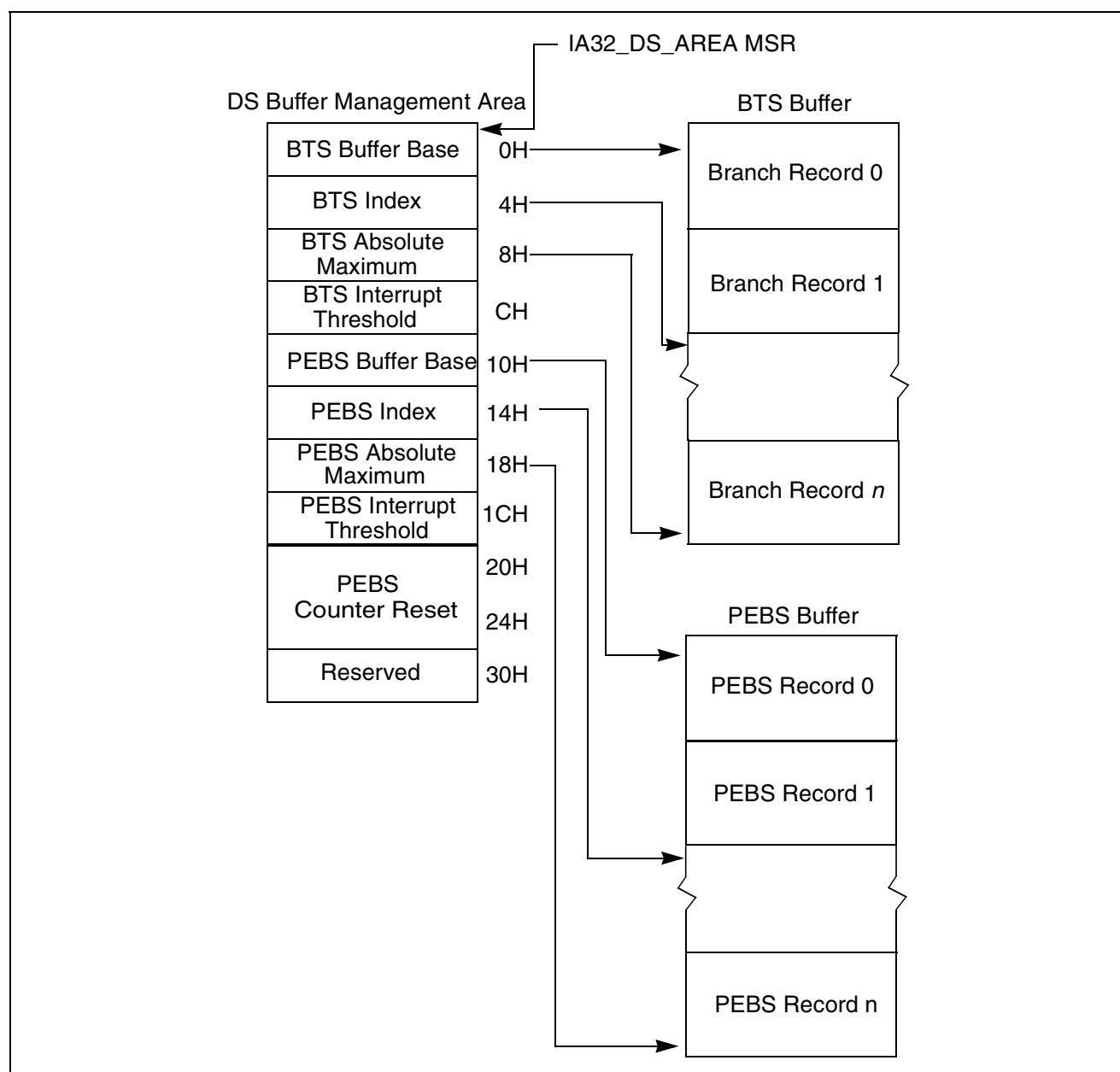


Figure 15-10. DS Save Area

PEBS absolute maximum

Linear address of the next byte past the end of the PEBS buffer. This address should be a multiple of the PEBS record size (40 bytes) plus 1.

PEBS interrupt threshold

Linear address of the PEBS record on which an interrupt is to be generated. This address must point to an offset from the PEBS buffer base that is a multiple of the PEBS record size. Also, it must be several records short of the PEBS absolute maximum address to allow a pending interrupt to be handled prior to processor writing the PEBS absolute maximum record.

PEBS counter reset value

A 40-bit value that the counter is to be reset to after state information has collected following counter overflow. This value allows state information to be collected after a preset number of events have been counted.

Figures 15-11 shows the structure of a 12-byte branch record in the BTS buffer. The fields in each record are as follows:

Last Branch From

Linear address of the instruction from which the branch, interrupt, or exception was taken.

Last Branch To Linear address of the branch target or the first instruction in the interrupt or exception service routine.

Branch Predicted

Bit 4 of field indicates whether the branch that was taken was predicted (set) or not predicted (clear).

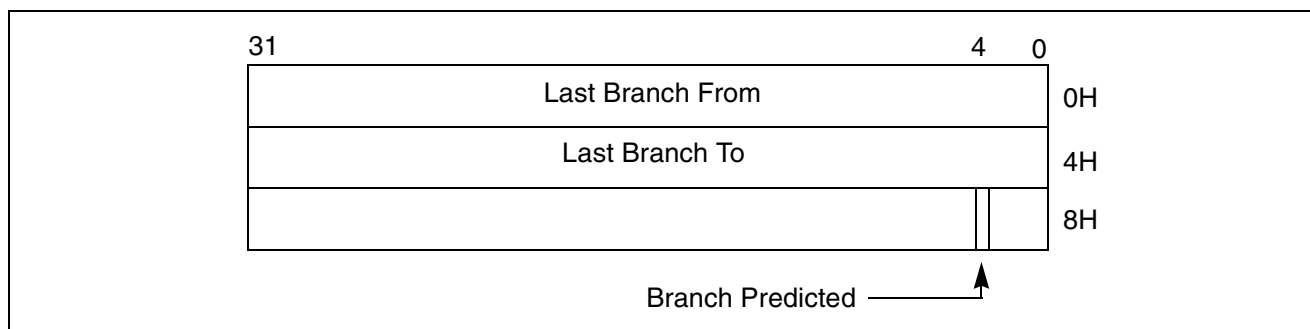


Figure 15-11. Branch Trace Record Format

Figures 15-12 shows the structure of the 40-byte PEBS records. Nominally the register values are those at the beginning of the instruction that caused the event. However, there are cases where the registers may be logged in a partially modified state. The linear IP field shows the value in the EIP register translated from an offset into the current code segment to a linear address.

15.9.6. Programming the Performance Counters for Non-Retirement Events

To program a performance counter and begin counting events, software must perform the following operations.

1. Select the event or events to be counted.
2. For each event, select an ESCR that supports the event using the values in the ESCR Restrictions row in Table A-1.

3. Match the CCCR Select value and ESCR name in Table A-1 to the values listed ESCR Name and ESCR No. columns in Table 15-4, to select a CCCR and performance counter.
4. Set up an ESCR for the specific event or events to be counted and the privilege levels they are to be counted at.
5. Set up the CCCR for the performance counter to be used to count the events, by selecting the chosen the ESCR and selecting the desired event filters.
6. Set up the CCCR for optional cascading of event counts, so that when the selected counter overflows its alternate counter starts counting.
7. Set up the CCCR to generate an optional performance monitor interrupt (PMI) when the counter overflows. (If PMI generation is enabled, the local APIC must be set up to deliver the interrupt to the processor and a handler for the interrupt must be in place.)
8. Enable the counter to begin counting.

31		0
	EFLAGS	0H
	Linear IP	4H
	EAX	8H
	EBX	CH
	ECX	10H
	EDX	14H
	ESI	18H
	EDI	1CH
	EBP	20H
	ESP	24H

Figure 15-12. PEBS Record Format

15.9.6.1. SELECTING EVENTS TO COUNT

Table A-1 lists a set of non-retirement events for the Pentium 4 and Intel Xeon processors. For each event listed in Table A-1, specific setup information is provided. Figure 15-13 gives an example of one of the non-retirement events from Table A-1.

In Tables A-1 and A-2, the name of the event is listed in the Event Name column and various parameters that define the event and other information are listed in the Event Parameters column. The Parameter Value and Description columns give specific parameters for the event and additional description information. The entries in the Event Parameters column are described below.

ESCR Restrictions

Lists the ESCRs that can be used to program the event. Typically only one ESCR is needed to count an event.

Event Name	Event Parameters	Parameter Value	Description
Branch_retired			Counts the retirement of a branch. Specify one or more mask bits to select any combination of branch taken, not-taken, predicted and mispredicted.
	ESCR restrictions	MSR_CRU_ESCR2 MSR_CRU_ESCR3	See Table 15-3 for the addresses of the ESCR MSRs
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	The counter numbers associated with each ESCR are provided. The performance counters and corresponding CCCRs can be obtained from Table 15-3.
	ESCR Event Select	06H	ESCR[31:25]
	ESCR Event Mask	Bit 0: MMNP 1: MMNM 2: MMTP 3: MMTM	ESCR[24:9], Branch Not-taken Predicted, Branch Not-taken Mispredicted, Branch Taken Predicted, Branch Taken Mispredicted.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		P6: EMON_BR_INST_RETIRED
	Can Support PEBS	No	
	Requires Additional MSRs for Tagging	No	

Figure 15-13. Event Example

Counter numbers per ESCR

Lists which performance counters are associated with each ESCR. Table 15-4 gives the name of the counter and CCCR for each counter number. Typically only one counter is needed to count the event.

ESCR Event Select

Gives the value to be placed in the event select field of the ESCR to select the event.

ESCR Event Mask

Gives the value to be placed in the Event Mask field of the ESCR to select sub-events to be counted. The parameter value column defines the documented bits with relative bit position offset starting from 0 (where the absolute bit position of relative offset 0 is bit 9 of the ESCR. All undocumented bits are reserved and should be set to 0.

CCCR Select Gives the value to be placed in the ESCR select field of the CCCR associated with the counter to select the ESCR to be used to define the event. (Note that this value is not the address of the ESCR; instead, it is the number of the ESCR from the Number column in Table 15-4.)

Event Specific Notes

Gives additional information about the event, such as the name of the same or a similar event defined for the P6 family processors.

Can Support PEBS

Indicates if PEBS is supported for the event. (This information is only supplied for at-retirement events listed in Table A-2.)

Requires Additional MSR for Tagging

Indicates which if any additional MSRs must be programmed to count the events. (This information is only supplied for the at-retirement events listed in Table A-2.)

NOTE

The performance-monitoring events listed in Appendix A, *Performance-Monitoring Events* are intended to be used as guides for performance tuning. The counter values reported are not guaranteed to be absolutely accurate and should be used as a relative guide for tuning. Known discrepancies are documented where applicable.

The following procedure shows how to set up a performance counter for basic counting; that is, the counter is set up to count a specified event indefinitely, wrapping around whenever it reaches its maximum count. This procedure is continued through the following four sections.

Using the information given in Table A-1, an event to be counted can be selected as follows:

1. Select the event to be counted.
2. Select the ESCR to be used to select events to be counted from the ESCRs field.
3. Select the number of the counter to be used to count the event from the Counter Numbers Per ESCR field.
4. Determine the name of the counter and the CCCR associated with the counter, and determine the MSR addresses of the counter, CCCR, and ESCR from Table 15-4.
5. Use the WRMSR instruction to write the ESCR Event Select and ESCR Event Mask values from Table A-1 into the appropriate fields in the ESCR. At the same time set or clear the USR and OS flags in the ESCR as desired.
6. Use the WRMSR instruction to write the CCCR Select value from Table A-1 into the appropriate field in the CCCR.

NOTE

Typically all the fields and flags of the CCCR will be written with one WRMSR instruction; however, in this procedure, several WRMSR writes are used to more clearly demonstrate the uses of the various CCCR fields and flags.

This setup procedure is continued in the next section, Section 15.9.6.2., “Filtering Events”.

15.9.6.2. FILTERING EVENTS

Each counter receives up to 4 input lines from the processor hardware from which it is counting events. The counter treats these inputs as binary inputs (input 0 has a value of 1, input 1 has a value of 2, input 2 has a value of 4, and input 3 has a value of 8). When a counter is enabled, it adds this binary input value to the counter value on each clock cycle. For each clock cycle, the value added to the counter can then range from 0 (no event) to 15.

For many events, only the 0 input line is active, so the counter is merely counting the clock cycles during which the 0 input is asserted. However, for some events two or more input lines are used. Here, the counter's threshold setting can be used to filter events. The compare, complement, threshold, and edge fields control the filtering of counter increments by input value.

If the compare flag is set, then a “greater than” or a “less than or equal to” comparison of the input value vs. a threshold value can be made. The complement flag selects “less than or equal to” (flag set) or “greater than” (flag clear). The threshold field selects a threshold value of from 0 to 15. For example, if the complement flag is cleared and the threshold field is set to 6, then any input value of 7 or greater on the 4 inputs to the counter will cause the counter to be incremented by 1, and any value less than 7 will cause an increment of 0 (or no increment) of the counter. Conversely, if the complement flag is set, any value from 0 to 6 will increment the counter and any value from 7 to 15 will not increment the counter. Note that when a threshold condition has been satisfied, the input to the counter is always 1, not the input value that is presented to the threshold filter.

The edge flag provides further filtering of the counter inputs when a threshold comparison is being made. The edge flag is only active when the compare flag is set. When the edge flag is set, the resulting output from the threshold filter (a value of 0 or 1) is used as an input to the edge filter. Each clock cycle, the edge filter examines the last and current input values and sends a count to the counter only when it detects a “rising edge” event; that is, a false-to-true transition. Figure 15-14 illustrates rising edge filtering.

The following procedure shows how to configure a CCCR to filter events using the threshold filter and the edge filter. This procedure is a continuation of the setup procedure introduced in Section 15.9.6.1., “Selecting Events to Count”.

7. (Optional) To set up the counter for threshold filtering, use the WRMSR instruction to write values in the CCCR compare and complement flags and the threshold field:
 - Set the compare flag.
 - Set or clear the complement flag for less than or equal to or greater than comparisons, respectively.
 - Enter a value from 0 to 15 in the threshold field.
8. (Optional) Select rising edge filtering by setting the CCCR edge flag.

This setup procedure is continued in the next section, Section 15.9.6.3., “Starting Event Counting”.

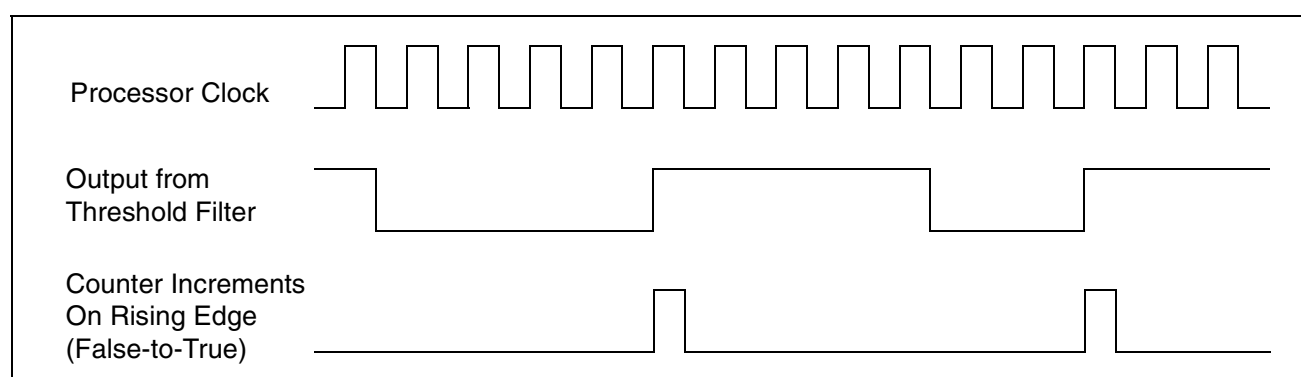


Figure 15-14. Effects of Edge Filtering

15.9.6.3. STARTING EVENT COUNTING

Event counting by a performance counter can be initiated in either of two ways. The typical way is to set the enable flag in the counter’s CCCR. Following the instruction to set the enable flag, event counting begins and continues until it is stopped (see Section 15.9.6.5., “Halting Event Counting”).

The following procedural step shows how to start event counting. This step is a continuation of the setup procedure introduced in Section 15.9.6.2., “Filtering Events”.

9. To start event counting, use the WRMSR instruction to set the CCCR enable flag for the performance counter.

This setup procedure is continued in the next section, Section 15.9.6.4., “Reading a Performance Counter’s Count”.

The second way that a counter can be started by using the cascade feature. Here, the overflow of one counter automatically starts its alternate counter (see Section 15.9.6.6., “Cascading Counters”).

15.9.6.4. READING A PERFORMANCE COUNTER'S COUNT

The Pentium 4 and Intel Xeon processors' performance counters can be read using either the RDPMC or RDMSR instructions. The enhanced functions of the RDPMC instruction (including fast read) are described in Section 15.9.2., "Performance Counters". These instructions can be used to read a performance counter while it is counting or when it is stopped.

The following procedural step shows how to read the event counter. This step is a continuation of the setup procedure introduced in Section 15.9.6.3., "Starting Event Counting".

10. To read a performance counters current event count, execute the RDPMC instruction with the counter number obtained from Table 15-4 used as an operand.

This setup procedure is continued in the next section, Section 15.9.6.5., "Halting Event Counting".

15.9.6.5. HALTING EVENT COUNTING

After a performance counter has been started (enabled), it continues counting indefinitely. If the counter overflows (goes one count past its maximum count), it wraps around and continues counting. When the counter wraps around, it sets its OVF flag to indicate that the counter has overflowed. The OVF flag is a sticky flag that indicates that the counter has overflowed at least once since the OVF bit was last cleared.

To halt counting, the CCCR enable flag for the counter must be cleared.

The following procedural step shows how to stop event counting. This step is a continuation of the setup procedure introduced in Section 15.9.6.4., "Reading a Performance Counter's Count".

11. To stop event counting, execute a WRMSR instruction to clear the CCCR enable flag for the performance counter.

To halt a cascaded counter (a counter that was started when its alternate counter overflowed), either clear the Cascade flag in the cascaded counter's CCCR MSR or clear the OVF flag in the alternate counter's CCCR MSR.

15.9.6.6. CASCADING COUNTERS

As described in Section 15.9.2., "Performance Counters", eighteen performance counters are implemented in pairs. Nine pairs of counters and associated CCCRs are further organized as four blocks: BPU, MS, FLAME, and IQ (see Table 15-4). The first three blocks contain two pairs each. The IQ block contains three pairs of counters (12 through 17) with associated CCCRs (MSR_IQ_CCCR0 through MSR_IQ_CCCR5).

The first 8 counter pairs (0 through 15) can be programmed using ESCRs to detect performance monitoring events. Pairs of ESCRs in each of the four blocks allow many different types of events to be counted. The cascade flag in the CCCR MSR allows nested monitoring of events to be performed by cascading one counter to a second counter located in another pair in the same block (see Figure 15-9 for the location of the flag).

Counters 0 and 1 form the first pair in the BPU block. Either counter 0 or 1 can be programmed to detect an event via MSR_MOB_ESCR0. Counters 0 and 2 can be cascaded in any order, as can counters 1 and 3. It's possible to set up 4 counters in the same block to cascade on two pairs of independent events. The pairing described also applies to subsequent blocks. Since the IQ PUB has two extra counters, cascading operates somewhat differently if 16 and 17 are involved. In the IQ block, counter 16 can only be cascaded from counter 14 (not from 12); counter 14 cannot be cascaded from counter 16 using the CCCR cascade bit mechanism. Similar restrictions apply to counter 17.

Example Scenario

Assume a scenario where counter X is set up to count 200 occurrences of event A; then counter Y is set up to count 400 occurrences of event B. Each counter is set up to count a specific event and overflow to the next counter. In the above example, counter X is preset for a count of -200 and counter Y for a count of -400; this setup causes the counters to overflow on the 200th and 400th counts respectively.

Continuing this scenario, counter X is set up to count indefinitely and wraparound on overflow (as described in the basic performance counter setup procedure that begins in Section 15.9.6.1., "Selecting Events to Count"). Counter Y is set up with the cascade flag in its associated CCCR MSR set to 1 and its enable flag set to 0.

To begin the nested counting, the enable bit for the counter X is set. Once enabled, counter X counts until it overflows. At this point, counter Y is automatically enabled and begins counting. Thus counter X overflows after 200 occurrences of event A. Counter Y then starts, counting 400 occurrences of event B before overflowing. When performance counters are cascaded, the counter Y would typically be set up to generate an interrupt on overflow, as described in Section 15.9.6.9., "Generating an Interrupt on Overflow".

The cascading counters mechanism can be used to count a single event. The counting begins on one counter then continues on the second counter after the first counter overflows. This technique doubles the number of event counts that can be recorded, since the contents of the two counters can be added together.

15.9.6.7. EXTENDED CASCADING

Extended cascading is a model-specific feature in the Intel NetBurst microarchitecture. The feature is available to Pentium 4 and Xeon processor family with family encoding of 15 and model encoding greater than or equal to 2. This feature uses bit 11 in CCCRs associated with the IQ block. See the table below.

Table 15-5. CCR Names and Bit Positions

CCCR Name:Bit Position	Bit Name	Description
MSR_IQ_CCCR1 2:11	Reserved	
MSR_IQ_CCCR0:11	CASCNT4INTO0	Allow counter 4 to cascade into counter 0
MSR_IQ_CCCR3:11	CASCNT5INTO3	Allow counter 5 to cascade into counter 3
MSR_IQ_CCCR4:11	CASCNT5INTO4	Allow counter 5 to cascade into counter 4
MSR_IQ_CCCR5:11	CASCNT4INTO5	Allow counter 4 to cascade into counter 5

The extended cascading feature can be adapted to the sampling usage model for performance monitoring. However, it is known that performance counters do not generate PMI in cascade mode or extended cascade mode due to an erratum. This erratum applies to Pentium 4 and Intel Xeon processors with model encoding of 2. For Pentium 4 and Intel Xeon processors with model encoding of 0 and 1, the erratum applies to processors with stepping encoding greater than 09H.

15.9.6.8. EXTENDED CASCADING

Counters 16 and 17 in the IQ block are frequently used in precise event-based sampling or at-retirement counting of events indicating a stalled condition in the pipeline. Neither counter 16 or 17 can initiate the cascading of counter pairs using the cascade bit in a CCCR.

Extended cascading permits performance monitoring tools to use counters 16 and 17 to initiate cascading of two counters in the IQ block. Extended cascading from counter 16 and 17 is conceptually similar to cascading other counters, but instead of using CASCADE bit of a CCCR, one of the four CASCNTxINTOy bits is used.

Example Scenario

A usage scenario for extended cascading is to sample instructions retired on logical processor 1 after the first 4096 instructions retired on logical processor 0. A procedure to program extended cascading in this scenario is outlined below:

1. Write the value 0 to counter 12.
2. Write the value 04000603H to MSR_CRU_ESCR0 (corresponding to selecting the NBOGNTAG and NBOGTAG event masks with qualification restricted to logical processor 1.).
3. Write the value 04038800H to MSR_IQ_CCCR0 to enable CASCNT4INTO0, and OVF_PMI. An ISR can sample on instruction addresses in this case. (Do not set ENABLE, nor CASCADE).
4. Write the value FFFFF000H into counter 16.

5. Write the value 0400060CH to MSR_CRU_ESCR2 (corresponding to selecting the NBOGNTAG and NBOGTAG event masks with qualification restricted to logical processor 0).
6. Write the value 00039000H to MSR_IQ_CCCR4 (set ENABLE bit, but not OVF_PMI).

Another use for cascading is to locate stalled execution in a multithreaded application. Assume MOB replays in thread B cause thread A to stall. Getting a sample of the stalled execution in this scenario could be accomplished by:

1. Set up counter B to count MOB replays on thread B.
2. Set up counter A to count resource stalls on thread A; set its force overflow bit and the appropriate CASCNTxINTOy bit.
3. Use the performance monitoring interrupt to capture the program execution data of the stalled thread.

15.9.6.9. GENERATING AN INTERRUPT ON OVERFLOW

Any of the performance counters can be configured to generate a performance monitor interrupt (PMI) if the counter overflows. The PMI interrupt service routine can then collect information about the state of the processor or program when overflow occurred. This information can then be used with a tool like the VTune™ Performance Analyzer to analyze and tune program performance.

To enable an interrupt on counter overflow, the OVR_PMI flag in the counter's associated CCCR MSR must be set. When overflow occurs, the PMI is generated through the local APIC. (Here, the performance counter entry in the local vector table [LVT] is set up to deliver the interrupt generated by the PMI to the processor.)

The PMI service routine can use the OVF flag to determine which counter overflowed when multiple counters have been configured generate PMIs.

When generating interrupts on overflow, the performance counter being used should be preset to value that will cause an overflow after a specified number of events are counted plus 1. The simplest way to select the preset value is to write a negative number into the counter, as described in Section 15.9.6.6., “Cascading Counters”. Here, however, if an interrupt is to be generated after 100 event counts, the counter should be preset to minus 100 plus 1 ($-100 + 1$), or -99. The counter will then overflow after it counts 99 events and generate an interrupt on the next (100th) event counted. The difference of 1 for this count enables the interrupt to be generated immediately after the selected event count has been reached, instead of waiting for the overflow to be propagation through the counter.

Because of latency in the micro-architecture between the generation of events and the generation of interrupts on overflow, it is sometimes difficult to generate an interrupt close to an event that caused it. In these situations, the FORCE_OVF flag in the CCCR can be used to improve reporting. Setting this flag causes the counter to overflow on every counter increment, which in turn triggers an interrupt after every counter increment.

15.9.6.10. COUNTER USAGE GUIDELINE

There are some instances where the user must take care to configure counting logic properly, so that it is not powered down. To use any ESCR, even when it is being used just for tagging, (any) one of the counters that the particular ESCR (or its paired ESCR) can be connected to should be enabled. If this is not done, 0 counts may result. Likewise, to use any counter, there must be some event selected in a corresponding ESCR (other than no_event, which generally has a select value of 0).

15.9.7. At-Retirement Counting

At-retirement counting provides a means counting only events that represent work committed to architectural state and ignoring work that was performed speculatively and later discarded.

The Intel NetBurst micro-architecture used in the Pentium 4 and Intel Xeon processors performs many speculative activities in an attempt to increase effective processing speeds. One example of this speculative activity is branch prediction. The Pentium 4 and Intel Xeon processors typically predict the direction of branches and then decode and execute instructions down the predicted path in anticipation of the actual branch decision. When a branch misprediction occurs, the results of instructions that were decoded and executed down the mispredicted path are canceled. If a performance counter was set up to count all executed instructions, the count would include instructions whose results were canceled as well as those whose results committed to architectural state.

To provide finer granularity in event counting in these situations, the performance monitoring facilities provided in the Pentium 4 and Intel Xeon processors provide a mechanism for tagging events and then counting only those tagged events that represent committed results. This mechanism is called “at-retirement counting.”

Tables A-2 through A-5 list predefined at-retirement events and event metrics that can be used to for tagging events when using at retirement counting. The following terminology is used in describing at-retirement counting.

Bogus, Non-Bogus, Retire

In at-retirement event descriptions, the term “bogus” refers to instructions or μ ops that must be canceled because they are on a path taken from a mispredicted branch. The terms “retired” and “non-bogus” refer to instructions or μ ops along the path that results in committed architectural state changes as required by the program being executed. Thus instructions and μ ops are either bogus or non-bogus, but not both. Several of the Pentium 4 and Intel Xeon processors’ performance monitoring events (such as, Instruction_Retired and Uops_Retired in Table A-2) can count instructions or μ ops that are retired based on the characterization of bogus” versus non-bogus.

Tagging

Tagging is a means of marking μ ops that have encountered a particular performance event so they can be counted at retirement. During the course of execution, the same event can happen more than once per μ op and a direct count of the event would not provide an indication of how many μ ops encountered that event.

The tagging mechanisms allow a μ op to be tagged once during its lifetime and thus counted once at retirement. The retired suffix is used for performance metrics that increment a count once per μ op, rather than once per event. For example, a μ op may encounter a cache miss more than once during its life time, but a “Miss Retired” metric (that counts the number of retired μ ops that encountered a cache miss) will increment only once for that μ op. A “Miss Retired” metric would be useful for characterizing the performance of the cache hierarchy for a particular instruction sequence. Details of various performance metrics and how these can be constructed using the Pentium 4 and Intel Xeon processors performance events are provided in the *Intel Pentium 4 Processor Optimization Reference Manual* (see Section 1.4., “Related Literature”).

Replay

To maximize performance for the common case, the Intel NetBurst micro-architecture aggressively schedules μ ops for execution before all the conditions for correct execution are guaranteed to be satisfied. In the event that all of these conditions are not satisfied, μ ops must be reissued. The mechanism that the Pentium 4 and Intel Xeon processors use for this reissuing of μ ops is called replay. Some examples of replay causes are cache misses, dependence violations, and unforeseen resource constraints. In normal operation, some number of replays is common and unavoidable. An excessive number of replays is an indication of a performance problem.

Assist

When the hardware needs the assistance of microcode to deal with some event, the machine takes an assist. One example of this is an underflow condition in the input operands of a floating-point operation. The hardware must internally modify the format of the operands in order to perform the computation. Assists clear the entire machine of μ ops before they begin and are costly.

15.9.7.1. USING AT-RETIREMENT COUNTING

The Pentium 4 and Intel Xeon processors allow counting both events and μ ops that encountered a specified event. For a subset of the at-retirement events listed in Table A-2, a μ op may be tagged when it encounters that event. The tagging mechanisms can be used in non-precise event-based sampling, and a subset of these mechanisms can be used in PEBS. There are four independent tagging mechanisms, and each mechanism uses a different event to count μ ops tagged with that mechanism:

- **Front-end tagging.** This mechanism pertains to the tagging of μ ops that encountered front-end events (for example, trace cache and instruction counts) and are counted with the `Front_end_event` event
- **Execution tagging.** This mechanism pertains to the tagging of μ ops that encountered execution events (for example, instruction types) and are counted with the `Execution_Event` event.
- **Replay tagging.** This mechanism pertains to tagging of μ ops whose retirement is replayed (for example, a cache miss) and are counted with the `Replay_event` event. Branch mispredictions are also tagged with this mechanism.

- **No tags.** This mechanism does not use tags. It uses the Instr_retired and the Uops_retired events.

Each tagging mechanism is independent from all others; that is, a μ op that has been tagged using one mechanism will not be detected with another mechanism's tagged- μ op detector. For example, if μ ops are tagged using the front-end tagging mechanisms, the Replay_event will not count those as tagged μ ops unless they are also tagged using the replay tagging mechanism. However, execution tags allow up to four different types of μ ops to be counted at retirement through execution tagging.

The independence of tagging mechanisms does not hold when using PEBS. When using PEBS, only one tagging mechanism should be used at a time.

Certain kinds of μ ops that cannot be tagged, including I/O, uncacheable and locked accesses, returns, and far transfers.

Table A-2 lists the performance monitoring events that support at-retirement counting: specifically the Front_end_event, Execution_event, Replay_event, Inst_retired and Uops_retired events. The following sections describe the tagging mechanisms for using these events to tag μ op and count tagged μ ops.

15.9.7.2. TAGGING MECHANISM FOR FRONT_END_EVENT

The Front_end_event counts μ ops that have been tagged as encountering any of the following events:

- **μ op decode events.** Tagging μ ops for μ op decode events requires specifying bits in the ESCR associated with the performance-monitoring event, Uop_type.
- **Trace cache events.** Tagging μ ops for trace cache events may require specifying certain bits in the MSR_TC_PRECISE_EVENT MSR (see Table A-3).

Table A-2 describes the Front_end_event and Table A-3 describes metrics that are used to set up a Front_end_event count.

The MSRs specified in the Table A-2 that are supported by the front-end tagging mechanism must be set and one or both of the NBOGUS and BOGUS bits in the Front_end_event event mask must be set to count events. None of the events currently supported requires the use of the MSR_TC_PRECISE_EVENT MSR.

15.9.7.3. TAGGING MECHANISM FOR EXECUTION_EVENT

Table A-2 describes the Execution_event and Table A-4 describes metrics that are used to set up an Execution_event count.

The execution tagging mechanism differs from other tagging mechanisms in how it causes tagging. One *upstream* ESCR is used to specify an event to detect and to specify a tag value (bits 5 through 8) to identify that event. A second *downstream* ESCR is used to detect μ ops that have been tagged with that tag value identifier using Execution_event for the event selection.

The upstream ESCR that counts the event must have its tag enable flag (bit 4) set and must have an appropriate tag value mask entered in its tag value field. The 4-bit tag value mask specifies which of tag bits should be set for a particular μ op. The value selected for the tag value should coincide with the event mask selected in the downstream ESCR. For example, if a tag value of 1 is set, then the event mask of NBOGUS0 should be enabled, correspondingly in the downstream ESCR. The downstream ESCR detects and counts tagged μ ops. The normal (not tag value) mask bits in the downstream ESCR specify which tag bits to count. If any one of the tag bits selected by the mask is set, the related counter is incremented by one. This mechanism is summarized in the Table A-4 metrics that are supported by the execution tagging mechanism. The tag enable and tag value bits are irrelevant for the downstream ESCR used to select the Execution_event.

The four separate tag bits allow the user to simultaneously but distinctly count up to four execution events at retirement (This applies for non-precise event-based sampling. There are additional restrictions for PEBS as noted in Section 15.9.8.3., “Setting Up the PEBS Buffer”). It is also possible to detect or count combinations of events by setting multiple tag value bits in the upstream ESCR or multiple mask bits in the downstream ESCR. For example, use a tag value of 3H in the upstream ESCR and use NBOGUS0/NBOGUS1 in the downstream ESCR event mask.

15.9.7.4. TAGGING MECHANISM FOR REPLAY_EVENT

Table A-2 describes the Replay_event and Table A-5 describes metrics that are used to set up an Replay_event count.

The replay mechanism enables tagging of μ ops for a subset of all replays before retirement. Use of the replay mechanism requires selecting the type of μ op that may experience the replay in the MSR_PEBS_MATRIX_VERT MSR and selecting the type of event in the IA32_PEBS_ENABLE MSR. Replay tagging must also be enabled with the UOP_Tag flag (bit 24) in the IA32_PEBS_ENABLE MSR.

The Table A-5 lists the metrics that are support the replay tagging mechanism and the at-retirement events that use the replay tagging mechanism, and specifies how the appropriate MSRs need to be configured. The replay tags defined in Table A-5 also enable Precise Event-Based Sampling (PEBS, see Section 15.9.8). Each of these replay tags can also be used in normal sampling by not setting Bit 24 nor Bit 25 in IA_32_PEBS_ENABLE_MSR. Each of these metrics requires that the Replay_Event (see Table A-2) be used to count the tagged μ ops.

15.9.8. Precise Event-Based Sampling (PEBS)

The debug store (DS) mechanism in the Pentium 4 and Intel Xeon processors allow two types of information to be collected for use in debugging and tuning programs: PEBS records and BTS records. (See Section 15.5.7., “Branch Trace Store (BTS)” for a description of the BTS mechanism.)

PEBS permits the saving of precise architectural information associated with one or more performance events in the precise event records buffer, which is part of the DS save area (see Section 15.9.5., “DS Save Area”). To use this mechanism, a counter is configured to overflow after it has counted a preset number of events. When the counter overflows, the processor copies the current state of the general-purpose and EFLAGS registers and instruction pointer into a record in the precise event records buffer. The processor then resets the count in the performance counter and restarts the counter. When the precise event records buffer is nearly full, an interrupt is generated, allowing the precise event records to be saved. A circular buffer is not supported for precise event records.

PEBS is supported only for a subset of the at-retirement events: `Execution_event`, `Front_end_event`, and `Replay_event`. Also, PEBS can only be carried out using the one performance counter, the `MSR_IQ_COUNTER4` MSR.

15.9.8.1. DETECTION OF THE AVAILABILITY OF THE PEBS FACILITIES

The DS feature flag (bit 21) returned by the `CPUID` instruction indicates (when set) the availability of the DS mechanism in the processor, which supports the PEBS (and BTS) facilities. When this bit is set, the following PEBS facilities are available:

- The `PEBS_UNAVAILABLE` flag in the `IA32_MISC_ENABLE` MSR indicates (when clear) the availability of the PEBS facilities, including the `IA32_PEBS_ENABLE` MSR.
- The enable PEBS flag (bit 24) in the `IA32_PEBS_ENABLE` MSR allows PEBS to be enabled (set) or disabled (clear).
- The `IA32_DS_AREA` MSR can be programmed to point to the DS save area.

15.9.8.2. SETTING UP THE DS SAVE AREA

Section 15.5.7.2., “Setting Up the DS Save Area” describes how to set up and enable the DS save area. This procedure is common for PEBS and BTS:

15.9.8.3. SETTING UP THE PEBS BUFFER

Only the `MSR_IQ_COUNTER4` performance counter can be used for PEBS. Use the following procedure to set up the processor and this counter for PEBS:

1. Set up the precise event buffering facilities. Place values in the precise event buffer base, precise event index, precise event absolute maximum, and precise event interrupt threshold, and precise event counter reset fields of the DS buffer management area (see Figure 15-10) to set up the precise event records buffer in memory.
2. Enable PEBS. Set the Enable PEBS flag (bit 24) in `IA32_PEBS_ENABLE` MSR.
3. Set up the `MSR_IQ_COUNTER4` performance counter and its associated CCCR and one or more ESCRs for PEBS as described in Tables A-2 through A-5.

15.9.8.4. WRITING A PEBS INTERRUPT SERVICE ROUTINE

The PEBS facilities share the same interrupt vector and interrupt service routine (called the DS ISR) with the non-precise event-based sampling and BTS facilities. To handle PEBS interrupts, PEBS handler code must be included in the DS ISR. See Section 15.5.7.4., “Writing the DS Interrupt Service Routine” for guidelines for writing the DS ISR.

15.9.8.5. OTHER DS MECHANISM IMPLICATIONS

The DS mechanism is not available in the SMM. It is disabled on transition to the SMM mode. Similarly the DS mechanism is disabled on the generation of a machine check exception and is cleared on processor RESET and INIT. The DS mechanism is available in real address mode.

15.9.9. Counting Clocks

The count of cycles, also known as clockticks, forms a the basis for measuring how long a program takes to execute. Clockticks are also used as part of efficiency ratios like cycles per instruction (CPI). Processor clocks may stop ticking under circumstances like the following:

- The processor is halted when there is nothing for the CPU to do. For example, the processor may halt to save power while the computer is servicing an I/O request. When Hyper-Threading Technology is enabled, both logical processors must be halted for performance-monitoring counters to be powered down.
- The processor is asleep as a result of being halted or because of a power-management scheme. There are different levels of sleep. In the some deep sleep levels, the Time Stamp Counter stops counting.

There are three ways to count processor clock cycles to monitor performance. These are:

- Non-Halted Clockticks – Measures clock cycles in which the specified logical processor is not halted and is not in any power-saving state. When Hyper-Threading Technology is enabled, this these ticks can be measured on a per-logical-processor basis.
- Non-Sleep Clockticks – Measures clock cycles in which the specified physical processor is not in a sleep mode or in a power-saving state. These ticks **cannot** be measured on a logical-processor basis.
- Time Stamp Counter – Measures clock cycles in which the physical processor is not in deep sleep. These ticks **cannot** be measured on a logical-processor basis.

The first two methods use performance counters and can be set up to cause an interrupt upon overflow (for sampling). They may also be useful where it is easier for a tool to read a performance counter than to use a time stamp counter (the timestamp counter is accessed via an instruction, RDTSC).

For applications with a significant amount of I/O, there are two ratios of interest:

- **Non-Halted CPI:** Non-halted clockticks/instructions retired measures the CPI for phases where the CPU was being used. This ratio can be measured on a logical-processor basis when Hyper-Threading Technology is enabled.
- **Nominal CPI:** Time stamp counter ticks/instructions retired measures the CPI over the duration of a program, including those periods when the machine halts while waiting for I/O.

Non-Halted Clockticks

Use the following procedure to program ESCRs and CCCRs to obtain non-halted clock ticks:

1. Select an ESCR for the `global_power_events` and specify the `RUNNING` sub-event mask and the desired `T0_OS/T0_USR/T1_OS/T1_USR` bits for the targeted processor.
2. Select an appropriate counter.
3. Enable counting in the CCCR for that counter by setting the enable bit.

Non-Sleep Clockticks

Performance monitoring counters can be configured to count clockticks whenever the performance monitoring hardware is not powered-down. To count Non-sleep Clockticks with a performance-monitoring counter, do the following:

1. Select one of the 18 counters.
2. Select any of the ESCRs whose events the selected counter can count. Set its event select to anything other than `no_event`. This may not seem necessary, but the counter may be disabled if this is not done.
3. Turn threshold comparison on in the CCCR by setting the compare bit to 1.
4. Set the threshold to 15 and the complement to 1 in the CCCR. Since no event can exceed this threshold, the threshold condition is met every cycle and the counter counts every cycle. Note that this overrides any qualification (e.g. by CPL) specified in the ESCR.
5. Enable counting in the CCCR for the counter by setting the enable bit.

In most cases, the counts produced by the non-halted and non-sleep metrics are equivalent if the physical package supports one logical processor and is not placed in a power-saving state. Operating systems may execute an `HLT` instruction and place a physical processor in a power-saving state.

On processors that support Hyper-Threading Technology (HT), each physical package can support two or more logical processors. Current implementation of HT provides two logical processors for each physical processor. While both logical processors can execute two threads simultaneously, one logical processor may halt to allow the other logical processor to execute without sharing execution resources between two logical processors.

Non-halted Clockticks can be set up to count the number of processor clock cycles for each logical processor whenever the logical processor is not halted (the count may include some portion of the clock cycles for that logical processor to complete a transition to a halted state). Physical processors that support HT enter into a power-saving state if all logical processors halt.

The Non-sleep Clockticks mechanism uses a filtering mechanism in CCCRs. The mechanism will continue to increment as long as one logical processor is not halted or in a power-saving state. Applications may cause a processor to enter into a power-saving state by using an OS service that transfers control to an OS's idle loop. The idle loop then may place the processor into a power-saving state after an implementation-dependent period if there is no work for the processor.

Time Stamp Counter

The Time Stamp Counter increments whenever the sleep pin is not asserted or when the clock signal on the system bus is active. The counter can be read with the RDTSC instruction. Computing the difference in values between two reads (modulo 2^{64}) gives the number of processor clocks between reads.

The Time Stamp Counter and the Non-sleep Clockticks count should agree in practically all cases. However, it is possible to have both logical processors in a physical package halted, resulting in most of the chip (including the performance monitoring hardware) being powered down. When this happens, it is possible for the time stamp counter to continue incrementing because the clock signal on the system bus is still active. Non-sleep Clockticks will no longer increment because the performance monitoring hardware is powered down in a power-saving state.

15.9.10. Operating System Implications

The DS mechanism can be used by the operating system as a debugging extension to facilitate failure analysis. When using this facility, a 25 to 30 times slowdown can be expected due to the effects of the trace store occurring on every taken branch.

Depending upon intended usage, the instruction pointers that are part of the branch records or the PEBS records need to have an association with the corresponding process. One solution requires the ability for the DS specific operating system module to be chained to the context switch. A separate buffer can then be maintained for each process of interest and the MSR pointing to the configuration area saved and setup appropriately on each context switch.

If the BTS facility has been enabled, then it must be disabled and state stored on transition of the system to a sleep state in which processor context is lost. The state must be restored on return from the sleep state.

It is required that an interrupt gate be used for the DS interrupt as opposed to a trap gate to prevent the generation of an endless interrupt loop.

Pages that contain buffers must have mappings to the same physical address for all processes/logical processors, such that any change to CR3 will not change DS addresses. If this requirement cannot be satisfied (that is, the feature is enabled on a per thread/process basis), then the operating system must ensure that the feature is enabled/disabled appropriately in the context switch code.

15.10. PERFORMANCE MONITORING AND HYPER-THREADING TECHNOLOGY

The performance monitoring capability of IA-32 processors with Hyper-Threading Technology is similar to that on the Pentium 4 and Intel Xeon processors. However, the performance monitoring capability is extended so that:

- The performance counters can be programmed to select events that are qualified by logical processor IDs.
- Performance monitoring interrupts can be directed to a specific logical processor within the physical processor.

This section describes the programming interfaces with respect to using performance counters, qualifying events by logical processor IDs, additional programmable bits in ESCRs, and CCCRs, as well as the special purpose IA32_PEBS_ENABLE, MSR_PEBS_MATRIX_VERT, and MSR_TC_PRECISE_EVENT MSRs.

In the Intel IA-32 processors with Hyper-Threading Technology, these registers are shared between the two logical processors in the physical processor. To allow these shared registers to be used to monitor performance events on either logical processor or both, additional flags have been added to the ESCR and CCCR MSRs and to the IA32_PEBS_ENABLE MSR. These additional flags and the effect of these flags on event monitoring in while Hyper-Threading Technology is active are described in the following sections.

15.10.1. ESCR MSRs

Figure 15-15 shows the layout of an ESCR MSR in the Intel IA-32 processors with Hyper Threading Technology.

The functions of the flags and fields are as follows:

T1_USR flag, bit 0

When set, events are counted when thread 1 (logical processor 1) is executing at a current privilege level (CPL) of 1, 2, or 3. These privilege levels are generally used by application code and unprotected operating system code.

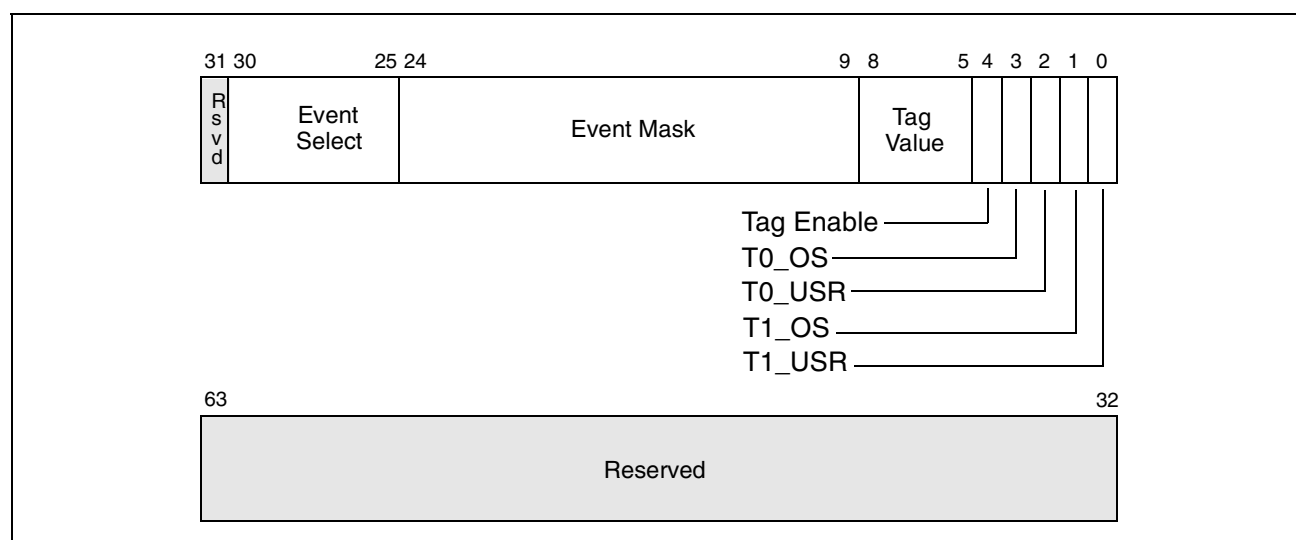


Figure 15-15. Event Selection Control Register (ESCR) for the Pentium 4 processor, Intel Xeon processor and Intel Xeon processor MP with HT Technology support

T1_OS flag, bit 1

When set, events are counted when thread 1 (logical processor 1) is executing at CPL of 0. This privilege level is generally reserved for protected operating system code. (When both the T1_OS and T1_USR flags are set, thread 1 events are counted at all privilege levels.)

T0_USR flag, bit 2

When set, events are counted when thread 0 (logical processor 0) is executing at a CPL of 1, 2, or 3.

T0_OS flag, bit 3

When set, events are counted when thread 0 (logical processor 0) is executing at CPL of 0. (When both the T0_OS and T0_USR flags are set, thread 0 events are counted at all privilege levels.)

Tag Enable, bit 4

When set, enables tagging of μ ops to assist in at-retirement event counting; when clear, disables tagging. See Section 15.9.7., “At-Retirement Counting”.

Tag Value field, bits 5 through 8

Selects a tag value to associate with a μ op to assist in at-retirement event counting.

Event Mask field, bits 9 through 24

Selects events to be counted from the event class selected with the event select field.

Event Select field, bits 25 through 30

Selects a class of events to be counted. The events within this class that are counted are selected with the event mask field.

The T0_OS and T0_USR flags and the T1_OS and T1_USR flags allow event counting and sampling to be specified for a specific logical processor (0 or 1) within an Intel Xeon processor MP. (See Section 7.6.8., “Identifying Logical Processors in an MP System” for information on identifying logical processor in an Xeon processor MP.)

Not all performance monitoring events can be detected within an Intel Xeon processor MP on a per logical processor basis (see Section 15.10.4., “Performance Monitoring Events”). Some sub-events (specified by an event mask bits) are counted or sampled without regard to which logical processor is associated with the detected event.

15.10.2. CCCR MSRs

Figure 15-16 shows the layout of a CCCR MSR in the Intel IA-32 processors with Hyper-Threading Technology.

The functions of the flags and fields are as follows:

Enable flag, bit 12

When set, enables counting; when clear, the counter is disabled. This flag is cleared on reset

ESCR Select field, bits 13 through 15

Identifies the ESCR to be used to select events to be counted with the counter associated with the CCCR.

Active Thread field, bits 16 and 17

Enables counting depending on which logical processors are active (executing a thread). This field enables filtering of events based on the state (active or inactive) of the logical processors. The encodings of this field are as follows:

00—None. Count only when neither logical processor is active.

01—Single. Count only when one logical processor is active (either 0 or 1).

10—Both. Count only when both logical processors are active.

11—Any. Count when either logical processor is active.

Note that a halted logical processor or a logical processor in the “wait for SIPI” state is considered inactive.

Compare flag, bit 18

When set, enables filtering of the event count; when clear, disables filtering. The filtering method is selected with the threshold, complement, and edge flags.

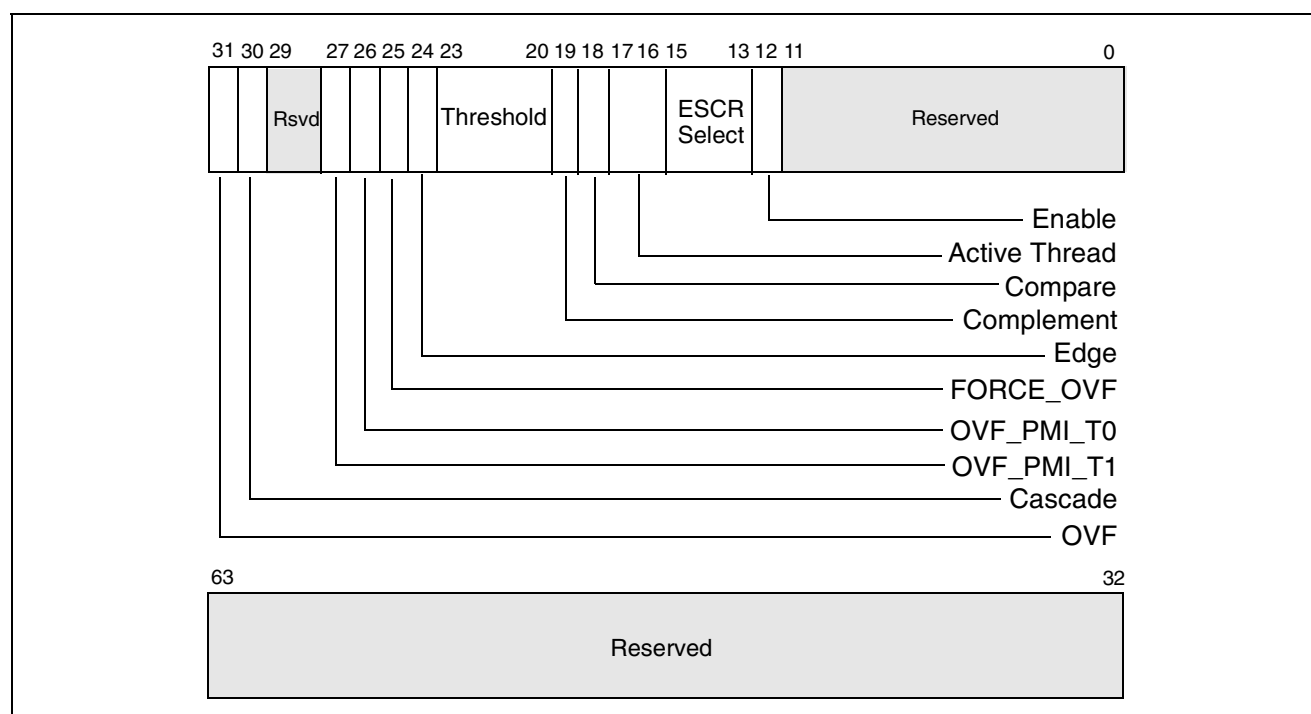


Figure 15-16. Counter Configuration Control Register (CCCR)

Complement flag, bit 19

Selects how the incoming event count is compared with the threshold value. When set, event counts that are less than or equal to the threshold value result in a single count being delivered to the performance counter; when clear, counts greater than the threshold value result in a count being delivered to the performance counter (see Section 15.9.6.2., “Filtering Events”). The compare flag is not active unless the compare flag is set.

Threshold field, bits 20 through 23

Selects the threshold value to be used for comparisons. The processor examines this field only when the compare flag is set, and uses the complement flag setting to determine the type of threshold comparison to be made. The useful range of values that can be entered in this field depend on the type of event being counted (see Section 15.9.6.2., “Filtering Events”).

Edge flag, bit 24

When set, enables rising edge (false-to-true) edge detection of the threshold comparison output for filtering event counts; when clear, rising edge detection is disabled. This flag is active only when the compare flag is set.

FORCE_OVF flag, bit 25

When set, forces a counter overflow on every counter increment; when clear, overflow only occurs when the counter actually overflows.

OVF_PMI_T0 flag, bit 26

When set, causes a performance monitor interrupt (PMI) to be sent to logical processor 0 when the counter overflows occurs; when clear, disables PMI

generation for logical processor 0. Note that the PMI is generate on the next event count after the counter has overflowed.

OVF_PMI_T1 flag, bit 27

When set, causes a performance monitor interrupt (PMI) to be sent to logical processor 1 when the counter overflows occurs; when clear, disables PMI generation for logical processor 1. Note that the PMI is generate on the next event count after the counter has overflowed.

Cascade flag, bit 30

When set, enables counting on one counter of a counter pair when its alternate counter in the other the counter pair in the same counter group overflows (see Section 15.9.2., “Performance Counters” for further details); when clear, disables cascading of counters.

OVF flag, bit 31

Indicates that the counter has overflowed when set. This flag is a sticky flag that must be explicitly cleared by software.

15.10.3. IA32_PEBS_ENABLE MSR

In an IA-32 processor with Hyper-Threading Technology, PEBS is enabled and qualified with two bits in the IA32_PEBS_ENABLE MSR: bit 25 (ENABLE_PEBS_MY_THR) and 26 (ENABLE_PEBS_OTH_THR) respectively. These bits do not explicitly identify a specific logical processor by logic processor ID(T0 or T1); instead, they allow a software agent to enable PEBS for subsequent threads of execution on the same logical processor on which the agent is running (“my thread”) or for the other logical processor in the physical package on which the agent is not running (“other thread”).

PEBS is supported for only a subset of the at-retirement events: Execution_event, Front_end_event, and Replay_event. Also, PEBS can be carried out only with two performance counters: MSR_IQ_CCCR4 (MSR address 370H) for logical processor 0 and MSR_IQ_CCCR5 (MSR address 371H) for logical processor 1.

NOTE

Performance monitoring tools should use a processor affinity mask to bind the kernel mode components that need to modify the ENABLE_PEBS_MY_THR and ENABLE_PEBS_OTH_THR bits in the IA32_PEBS_ENABLE MSR to a specific logical processor to prevent these kernel mode components from migrating between different logical processors due to OS scheduling.

15.10.4. Performance Monitoring Events

All of the events listed in Table A-1 and A-2 are available in an Intel Xeon processor MP. When Hyper-Threading Technology is active, many performance monitoring events can be qualified by the logical processor ID, which corresponds to bit 0 of the initial APIC ID. This allows for counting an event in any or all of the logical processors. However, not all the events have this logic processor specificity, or thread specificity.

Here, each event falls into one of two categories:

- Thread specific (TS). The event can be qualified as occurring on a specific logical processor.
- Thread independent (TI). The event cannot be qualified as being associated with a specific logical processor.

Table A-6 gives logical processor specific information (TS or TI) for each of the events described in Tables A-1 and A-2.

If for example, a TS event occurred in logical processor T0, the counting of the event (as shown in Table 15-6) depends only on the setting of the T0_USR and T0_OS flags in the ESCR being used to set up the event counter. The T1_USR and T1_OS flags have no effect on the count.

Table 15-6. Effect of Logical Processor and CPL Qualification for Logical-Processor-Specific (TS) Events

	T1_OS/T1_USR=00	T1_OS/T1_USR=01	T1_OS/T1_USR=11	T1_OS/T1_USR=10
T0_OS/T0_USR=00	Zero count	Counts while T1 in USR	Counts while T1 in OS or USR	Counts while T1 in OS
T0_OS/T0_USR=01	Counts while T0 in USR	Counts while T0 in USR or T1 in USR	Counts while (a) T0 in USR or (b) T1 in OS or (c) T1 in USR	Counts while (a) T0 in OS or (b) T1 in OS
T0_OS/T0_USR=11	Counts while T0 in OS or USR	Counts while (a) T0 in OS or (b) T0 in USR or (c) T1 in USR	Counts irrespective of CPL, T0, T1	Counts while (a) T0 in OS or (b) T0 in USR or (c) T1 in OS
T0_OS/T0_USR=10	Counts T0 in OS	Counts T0 in OS or T1 in USR	Counts while (a) T0 in OS or (b) T1 in OS or (c) T1 in USR	Counts while (a) T0 in OS or (b) T1 in OS

When a bit in the event mask field is TI, the effect of specifying bit-0-3 of the associated ESCR are described in Table 15-6. For events that are marked as TI in Appendix A, the effect of selectively specifying T0_USR, T0_OS, T1_USR, T1_OS bits is shown in Table 15-6.

Table 15-7. Effect of logical processor and CPL qualification for non-logical-processor-specific (TI) events

	T1_OS/T1_USR=00	T1_OS/T1_USR=01	T1_OS/T1_USR=11	T1_OS/T1_USR=10
T0_OS/T0_USR=00	Zero count	Counts while (a) T0 in USR or (b) T1 in USR	Counts irrespective of CPL, T0, T1	Counts while (a) T0 in OS or (b) T1 in OS
T0_OS/T0_USR=01	Counts while (a) T0 in USR or (b) T1 in USR	Counts while (a) T0 in USR or (b) T1 in USR	Counts irrespective of CPL, T0, T1	Counts irrespective of CPL, T0, T1
T0_OS/T0_USR=11	Counts irrespective of CPL, T0, T1	Counts irrespective of CPL, T0, T1	Counts irrespective of CPL, T0, T1	Counts irrespective of CPL, T0, T1
T0_OS/T0_USR=10	Counts while (a) T0 in OS or (b) T1 in OS	Counts irrespective of CPL, T0, T1	Counts irrespective of CPL, T0, T1	Counts while (a) T0 in OS or (b) T1 in OS

15.11. PERFORMANCE MONITORING (P6 FAMILY PROCESSOR)

The P6 family processors provide two 40-bit performance counters, allowing two types of events to be monitored simultaneously. These counters can either count events or measure duration. When counting events, a counter is incremented each time a specified event takes place or a specified number of events takes place. When measuring duration, a counter counts the number of processor clocks that occur while a specified condition is true. The counters can count events or measure durations that occur at any privilege level. Table A-9 in Appendix A, *Performance-Monitoring Events*, lists the events that can be counted with the P6 family performance monitoring counters.

NOTE

The performance-monitoring event listed in Appendix A, *Performance-Monitoring Events* are intended to be used as guides for performance tuning. The counter values reported are not guaranteed to be absolutely accurate and should be used as a relative guide for tuning. Known discrepancies are documented where applicable.

The performance-monitoring counters are supported by four MSRs: the performance event select MSRs (PerfEvtSel0 and PerfEvtSel1) and the performance counter MSRs (PerfCtr0 and PerfCtr1). These registers can be read from and written to using the RDMSR and WRMSR instructions, respectively. They can be accessed using these instructions only when operating at privilege level 0. The PerfCtr0 and PerfCtr1 MSRs can be read from any privilege level using the RDPMC (read performance-monitoring counters) instruction.

NOTE

The PerfEvtSel0, PerfEvtSel1, PerfCtr0, and PerfCtr1 MSRs and the events listed in Table A-9 are model-specific for P6 family processors. They are not guaranteed to be available in future IA-32 processors.

15.11.1. PerfEvtSel0 and PerfEvtSel1 MSRs

The PerfEvtSel0 and PerfEvtSel1 MSRs control the operation of the performance-monitoring counters, with one register used to set up each counter. They specify the events to be counted, how they should be counted, and the privilege levels at which counting should take place. Figure 15-17 shows the flags and fields in these MSRs.

The functions of the flags and fields in the PerfEvtSel0 and PerfEvtSel1 MSRs are as follows:

Event select field (bits 0 through 7)

Selects the event to be monitored (see Table A-9, for a list of events and their 8-bit codes).

Unit mask (UMASK) field (bits 8 through 15)

Further qualifies the event selected in the event select field. For example, for some cache events, the mask is used as a MESI-protocol qualifier of cache states (see Table A-9).

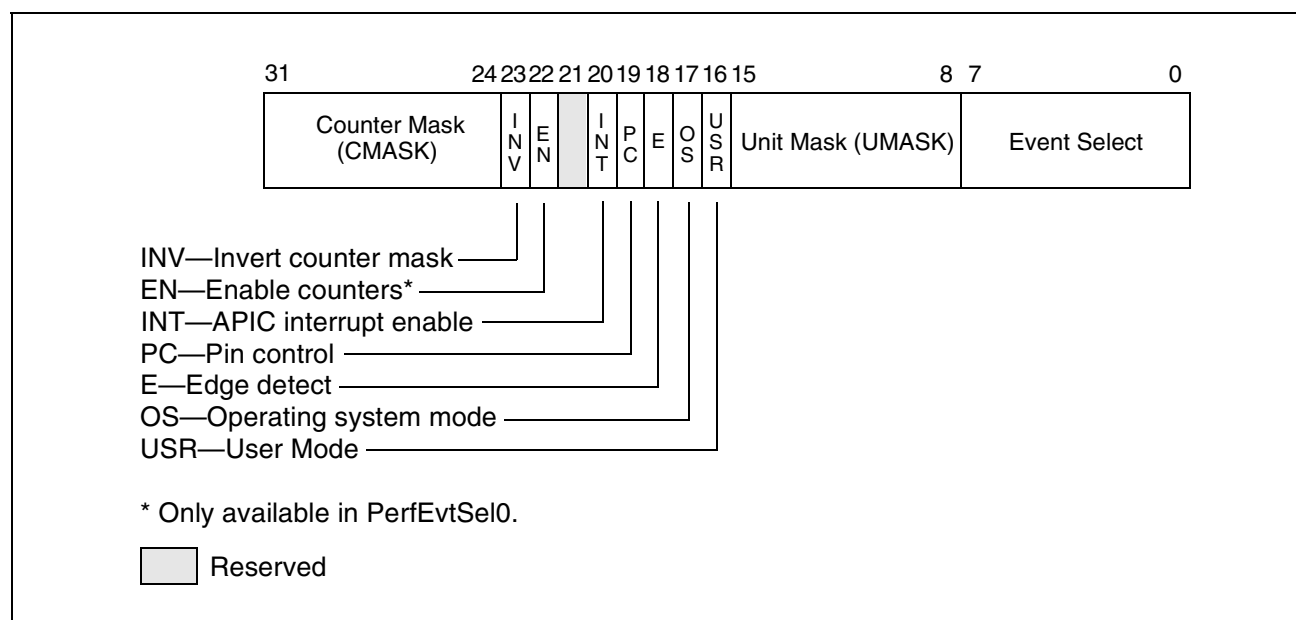


Figure 15-17. PerfEvtSel0 and PerfEvtSel1 MSRs

USR (user mode) flag (bit 16)

Specifies that events are counted only when the processor is operating at privilege levels 1, 2 or 3. This flag can be used in conjunction with the OS flag.

OS (operating system mode) flag (bit 17)

Specifies that events are counted only when the processor is operating at privilege level 0. This flag can be used in conjunction with the USR flag.

E (edge detect) flag (bit 18)

Enables (when set) edge detection of events. The processor counts the number of deasserted to asserted transitions of any condition that can be expressed by the other fields. The mechanism is limited in that it does not permit back-to-

back assertions to be distinguished. This mechanism allows software to measure not only the fraction of time spent in a particular state, but also the average length of time spent in such a state (for example, the time spent waiting for an interrupt to be serviced).

PC (pin control) flag (bit 19)

When set, the processor toggles the PMi pins and increments the counter when performance-monitoring events occur; when clear, the processor toggles the PMi pins when the counter overflows. The toggling of a pin is defined as assertion of the pin for a single bus clock followed by deassertion.

INT (APIC interrupt enable) flag (bit 20)

When set, the processor generates an exception through its local APIC on counter overflow.

EN (Enable Counters) Flag (bit 22)

This flag is only present in the PerfEvtSel0 MSR. When set, performance counting is enabled in both performance-monitoring counters; when clear, both counters are disabled.

INV (invert) flag (bit 23)

Inverts the result of the counter-mask comparison when set, so that both greater than and less than comparisons can be made.

Counter mask (CMASK) field (bits 24 through 31)

When nonzero, the processor compares this mask to the number of events counted during a single cycle. If the event count is greater than or equal to this mask, the counter is incremented by one. Otherwise the counter is not incremented. This mask can be used to count events only if multiple occurrences happen per clock (for example, two or more instructions retired per clock). If the counter-mask field is 0, then the counter is incremented each cycle by the number of events that occurred that cycle.

15.11.2. PerfCtr0 and PerfCtr1 MSRs

The performance-counter MSRs (PerfCtr0 and PerfCtr1) contain the event or duration counts for the selected events being counted. The RDPMC instruction can be used by programs or procedures running at any privilege level and in virtual-8086 mode to read these counters. The PCE flag in control register CR4 (bit 8) allows the use of this instruction to be restricted to only programs and procedures running at privilege level 0.

The RDPMC instruction is not serializing or ordered with other instructions. Thus, it does not necessarily wait until all previous instructions have been executed before reading the counter. Similarly, subsequent instructions may begin execution before the RDPMC instruction operation is performed.

Only the operating system, executing at privilege level 0, can directly manipulate the performance counters, using the RDMSR and WRMSR instructions. A secure operating system would clear the PCE flag during system initialization to disable direct user access to the performance-

monitoring counters, but provide a user-accessible programming interface that emulates the RDPMC instruction.

The WRMSR instruction cannot arbitrarily write to the performance-monitoring counter MSRs (PerfCtr0 and PerfCtr1). Instead, the lower-order 32 bits of each MSR may be written with any value, and the high-order 8 bits are sign-extended according to the value of bit 31. This operation allows writing both positive and negative values to the performance counters.

15.11.3. Starting and Stopping the Performance-Monitoring Counters

The performance-monitoring counters are started by writing valid setup information in the PerfEvtSel0 and/or PerfEvtSel1 MSRs and setting the enable counters flag in the PerfEvtSel0 MSR. If the setup is valid, the counters begin counting following the execution of a WRMSR instruction that sets the enable counter flag. The counters can be stopped by clearing the enable counters flag or by clearing all the bits in the PerfEvtSel0 and PerfEvtSel1 MSRs. Counter 1 alone can be stopped by clearing the PerfEvtSel1 MSR.

15.11.4. Event and Time-Stamp Monitoring Software

To use the performance-monitoring counters and time-stamp counter, the operating system needs to provide an event-monitoring device driver. This driver should include procedures for handling the following operations:

- Feature checking.
- Initialize and start counters.
- Stop counters.
- Read the event counters.
- Read the time-stamp counter.

The event monitor feature determination procedure must determine whether the current processor supports the performance-monitoring counters and time-stamp counter. This procedure compares the family and model of the processor returned by the CPUID instruction with those of processors known to support performance monitoring. (The Pentium and P6 family processors support performance counters.) The procedure also checks the MSR and TSC flags returned to register EDI by the CPUID instruction to determine if the MSRs and the RDTSC instruction are supported.

The initialize and start counters procedure sets the PerfEvtSel0 and/or PerfEvtSel1 MSRs for the events to be counted and the method used to count them and initializes the counter MSRs (PerfCtr0 and PerfCtr1) to starting counts. The stop counters procedure stops the performance counters. (See Section 15.11.3., “Starting and Stopping the Performance-Monitoring Counters”, for more information about starting and stopping the counters.)

The read counters procedure reads the values in the PerfCtr0 and PerfCtr1 MSRs, and a read time-stamp counter procedure reads the time-stamp counter. These procedures would be provided in lieu of enabling the RDTSC and RDPMC instructions that allow application code to read the counters.

15.11.5. Monitoring Counter Overflow

The P6 family processors provide the option of generating a local APIC interrupt when a performance-monitoring counter overflows. This mechanism is enabled by setting the interrupt enable flag in either the PerfEvtSel0 or the PerfEvtSel1 MSR. The primary use of this option is for statistical performance sampling.

To use this option, the operating system should do the following things on the processor for which performance events are required to be monitored:

- Provide an interrupt vector for handling the counter-overflow interrupt.
- Initialize the APIC PERF local vector entry to enable handling of performance-monitor counter overflow events.
- Provide an entry in the IDT that points to a stub exception handler that returns without executing any instructions.
- Provide an event monitor driver that provides the actual interrupt handler and modifies the reserved IDT entry to point to its interrupt routine.

When interrupted by a counter overflow, the interrupt handler needs to perform the following actions:

- Save the instruction pointer (EIP register), code-segment selector, TSS segment selector, counter values and other relevant information at the time of the interrupt.
- Reset the counter to its initial setting and return from the interrupt.

An event monitor application utility or another application program can read the information collected for analysis of the performance of the profiled application.

15.12. PERFORMANCE MONITORING (PENTIUM PROCESSORS)

The Pentium processor provides two 40-bit performance counters, which can be used either to count events or measure duration. The performance-monitoring counters are supported by three MSRs: the control and event select MSR (CESR) and the performance counter MSRs (CTR0 and CTR1). These registers can be read from and written to using the RDMSR and WRMSR instructions, respectively.

They can be accessed using these instructions only when operating at privilege level 0. Each counter has an associated external pin (PM0/BP0 and PM1/BP1), which can be used to indicate the state of the counter to external hardware.

NOTES

The CESR, CTR0, and CTR1 MSRs and the events listed in Table A-9 are model-specific for the Pentium processor.

The performance-monitoring event listed in Appendix B, *Model-Specific Registers (MSRs)* are intended to be used as guides for performance tuning. The counter values reported are not guaranteed to be absolutely accurate and should be used as a relative guide for tuning. Known discrepancies are documented where applicable.

15.12.1. Control and Event Select Register (CESR)

The 32-bit control and event select MSR (CESR) is used to control the operation of performance-monitoring counters CTR0 and CTR1 and their associated pins (see Figure 15-18). To control each counter, the CESR register contains a 6-bit event select field (ES0 and ES1), a pin control flag (PC0 and PC1), and a 3-bit counter control field (CC0 and CC1). The functions of these fields are as follows:

ES0 and ES1 (event select) fields (bits 0 through 5, bits 16 through 21)

Selects (by entering an event code in the field) up to two events to be monitored. See Table A-9 for a list of available event codes

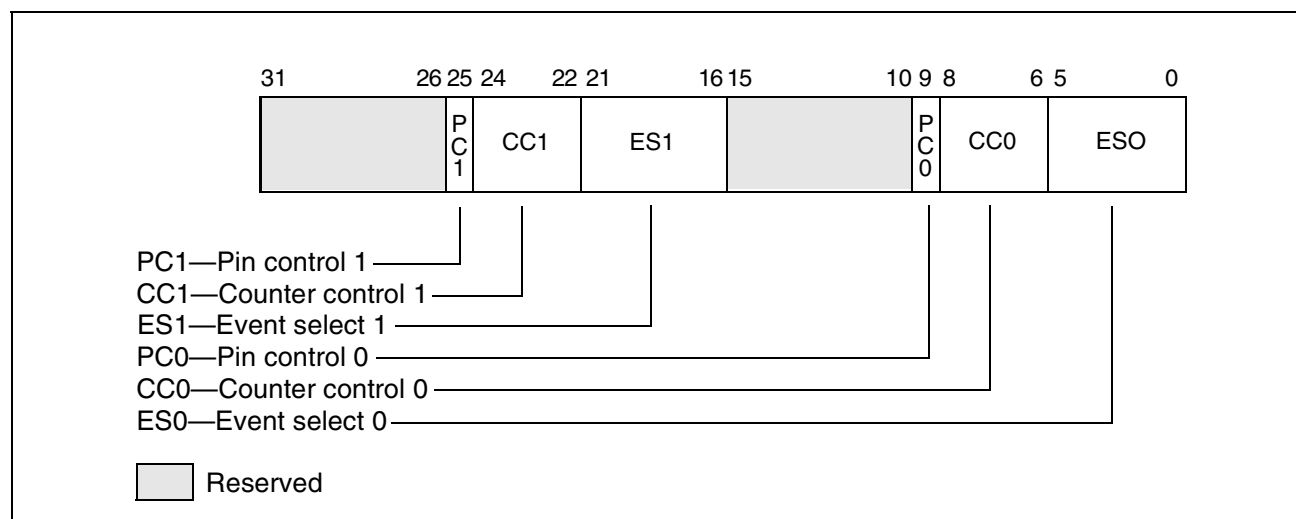


Figure 15-18. CESR MSR (Pentium Processor Only)

CC0 and CC1 (counter control) fields (bits 6 through 8, bits 22 through 24)

Controls the operation of the counter. The possible control codes are as follows:

CCn	Meaning
000	Count nothing (counter disabled)
001	Count the selected event while CPL is 0, 1, or 2
010	Count the selected event while CPL is 3

011	Count the selected event regardless of CPL
100	Count nothing (counter disabled)
101	Count clocks (duration) while CPL is 0, 1, or 2
110	Count clocks (duration) while CPL is 3
111	Count clocks (duration) regardless of CPL

Note that the highest order bit selects between counting events and counting clocks (duration); the middle bit enables counting when the CPL is 3; and the low-order bit enables counting when the CPL is 0, 1, or 2.

PC0 and PC1 (pin control) flags (bit 9, bits 25)

Selects the function of the external performance-monitoring counter pin (PM0/BP0 and PM1/BP1). Setting one of these flags to 1 causes the processor to assert its associated pin when the counter has overflowed; setting the flag to 0 causes the pin to be asserted when the counter has been incremented. These flags permit the pins to be individually programmed to indicate the overflow or incremented condition. Note that the external signalling of the event on the pins will lag the internal event by a few clocks as the signals are latched and buffered.

While a counter need not be stopped to sample its contents, it must be stopped and cleared or preset before switching to a new event. It is not possible to set one counter separately. If only one event needs to be changed, the CESR register must be read, the appropriate bits modified, and all bits must then be written back to CESR. At reset, all bits in the CESR register are cleared.

15.12.2. Use of the Performance-Monitoring Pins

When the performance-monitor pins PM0/BP0 and/or PM1/BP1 are configured to indicate when the performance-monitor counter has incremented and an “occurrence event” is being counted, the associated pin is asserted (high) each time the event occurs. When a “duration event” is being counted the associated PM pin is asserted for the entire duration of the event. When the performance-monitor pins are configured to indicate when the counter has overflowed, the associated PM pin is not asserted until the counter has overflowed.

When the PM0/BP0 and/or PM1/BP1 pins are configured to signal that a counter has incremented, it should be noted that although the counters may increment by 1 or 2 in a single clock, the pins can only indicate that the event occurred. Moreover, since the internal clock frequency may be higher than the external clock frequency, a single external clock may correspond to multiple internal clocks.

A “count up to” function may be provided when the event pin is programmed to signal an overflow of the counter. Because the counters are 40 bits, a carry out of bit 39 indicates an overflow. A counter may be preset to a specific value less than $2^{40} - 1$. After the counter has been enabled and the prescribed number of events has transpired, the counter will overflow.

Approximately 5 clocks later, the overflow is indicated externally and appropriate action, such as signaling an interrupt, may then be taken.

The PM0/BP0 and PM1/BP1 pins also serve to indicate breakpoint matches during in-circuit emulation, during which time the counter increment or overflow function of these pins is not available. After RESET, the PM0/BP0 and PM1/BP1 pins are configured for performance monitoring, however a hardware debugger may reconfigure these pins to indicate breakpoint matches.

15.12.3. Events Counted

The events that the performance-monitoring counters can set to count and record in the CTR0 and CTR1 MSRs are divided into two categories: occurrences and duration. Occurrences events are counted each time the event takes place. If the PM0/BP0 or PM1/BP1 pins are configured to indicate when a counter increments, they are asserted for each clock the counter increments. Note that if an event can happen twice in one clock, the counter increments by 2, however, the pins are asserted only once.

For duration events, the counter counts the total number of clocks that the condition is true. When configured to indicate when a counter increments, the PM0/BP0 and/or PM1/BP1 pins are asserted for the duration of the event.

Table A-9 lists the events that can be counted with the Pentium processor performance-monitoring counters.

16

8086 Emulation

CHAPTER 16

8086 EMULATION

IA-32 processors (beginning with the Intel386 processor) provide two ways to execute new or legacy programs that are assembled and/or compiled to run on an Intel 8086 processor:

- Real-address mode.
- Virtual-8086 mode.

Figure 2-2 shows the relationship of these operating modes to protected mode and system management mode (SMM).

When the processor is powered up or reset, it is placed in the real-address mode. This operating mode almost exactly duplicates the execution environment of the Intel 8086 processor, with some extensions. Virtually any program assembled and/or compiled to run on an Intel 8086 processor will run on an IA-32 processor in this mode.

When running in protected mode, the processor can be switched to virtual-8086 mode to run 8086 programs. This mode also duplicates the execution environment of the Intel 8086 processor, with extensions. In virtual-8086 mode, an 8086 program runs as a separate protected-mode task. Legacy 8086 programs are thus able to run under an operating system (such as Microsoft Windows*) that takes advantage of protected mode and to use protected-mode facilities, such as the protected-mode interrupt- and exception-handling facilities. Protected-mode multitasking permits multiple virtual-8086 mode tasks (with each task running a separate 8086 program) to be run on the processor along with other non-virtual-8086 mode tasks.

This section describes both the basic real-address mode execution environment and the virtual-8086-mode execution environment, available on the IA-32 processors beginning with the Intel386 processor.

16.1. REAL-ADDRESS MODE

The IA-32 architecture's real-address mode runs programs written for the Intel 8086, Intel 8088, Intel 80186, and Intel 80188 processors, or for the real-address mode of the Intel 286, Intel386, Intel486, Pentium, P6 family, Pentium 4, and Intel Xeon processors.

The execution environment of the processor in real-address mode is designed to duplicate the execution environment of the Intel 8086 processor. To an 8086 program, a processor operating in real-address mode behaves like a high-speed 8086 processor. The principal features of this architecture are defined in Chapter 3, *Basic Execution Environment*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*.

The following is a summary of the core features of the real-address mode execution environment as would be seen by a program written for the 8086:

- The processor supports a nominal 1-MByte physical address space (see Section 16.1.1., “Address Translation in Real-Address Mode”, for specific details). This address space is divided into segments, each of which can be up to 64 KBytes in length. The base of a segment is specified with a 16-bit segment selector, which is zero extended to form a 20-bit offset from address 0 in the address space. An operand within a segment is addressed with a 16-bit offset from the base of the segment. A physical address is thus formed by adding the offset to the 20-bit segment base (see Section 16.1.1., “Address Translation in Real-Address Mode”).
- All operands in “native 8086 code” are 8-bit or 16-bit values. (Operand size override prefixes can be used to access 32-bit operands.)
- Eight 16-bit general-purpose registers are provided: AX, BX, CX, DX, SP, BP, SI, and DI. The extended 32 bit registers (EAX, EBX, ECX, EDX, ESP, EBP, ESI, and EDI) are accessible to programs that explicitly perform a size override operation.
- Four segment registers are provided: CS, DS, SS, and ES. (The FS and GS registers are accessible to programs that explicitly access them.) The CS register contains the segment selector for the code segment; the DS and ES registers contain segment selectors for data segments; and the SS register contains the segment selector for the stack segment.
- The 8086 16-bit instruction pointer (IP) is mapped to the lower 16-bits of the EIP register. Note this register is a 32-bit register and unintentional address wrapping may occur.
- The 16-bit FLAGS register contains status and control flags. (This register is mapped to the 16 least significant bits of the 32-bit EFLAGS register.)
- All of the Intel 8086 instructions are supported (see Section 16.1.3., “Instructions Supported in Real-Address Mode”).
- A single, 16-bit-wide stack is provided for handling procedure calls and invocations of interrupt and exception handlers. This stack is contained in the stack segment identified with the SS register. The SP (stack pointer) register contains an offset into the stack segment. The stack grows down (toward lower segment offsets) from the stack pointer. The BP (base pointer) register also contains an offset into the stack segment that can be used as a pointer to a parameter list. When a CALL instruction is executed, the processor pushes the current instruction pointer (the 16 least-significant bits of the EIP register and, on far calls, the current value of the CS register) onto the stack. On a return, initiated with a RET instruction, the processor pops the saved instruction pointer from the stack into the EIP register (and CS register on far returns). When an implicit call to an interrupt or exception handler is executed, the processor pushes the EIP, CS, and EFLAGS (low-order 16-bits only) registers onto the stack. On a return from an interrupt or exception handler, initiated with an IRET instruction, the processor pops the saved instruction pointer and EFLAGS image from the stack into the EIP, CS, and EFLAGS registers.
- A single interrupt table, called the “interrupt vector table” or “interrupt table,” is provided for handling interrupts and exceptions (see Figure 16-2). The interrupt table (which has 4-byte entries) takes the place of the interrupt descriptor table (IDT, with 8-byte entries) used

when handling protected-mode interrupts and exceptions. Interrupt and exception vector numbers provide an index to entries in the interrupt table. Each entry provides a pointer (called a “vector”) to an interrupt- or exception-handling procedure.

See Section 16.1.4., “Interrupt and Exception Handling”, for more details. It is possible for software to relocate the IDT by means of the LIDT instruction on IA-32 processors beginning with the Intel386 processor.

- The x87 FPU is active and available to execute x87 FPU instructions in real-address mode. Programs written to run on the Intel 8087 and Intel 287 math coprocessors can be run in real-address mode without modification.

The following extensions to the Intel 8086 execution environment are available in the IA-32 architecture’s real-address mode. If backwards compatibility to Intel 286 and Intel 8086 processors is required, these features should not be used in new programs written to run in real-address mode.

- Two additional segment registers (FS and GS) are available.
- Many of the integer and system instructions that have been added to later IA-32 processors can be executed in real-address mode (see Section 16.1.3., “Instructions Supported in Real-Address Mode”).
- The 32-bit operand prefix can be used in real-address mode programs to execute the 32-bit forms of instructions. This prefix also allows real-address mode programs to use the processor’s 32-bit general-purpose registers.
- The 32-bit address prefix can be used in real-address mode programs, allowing 32-bit offsets.

The following sections describe address formation, registers, available instructions, and interrupt and exception handling in real-address mode. For information on I/O in real-address mode, see Chapter 12, *Input/Output*, in the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*.

16.1.1. Address Translation in Real-Address Mode

In real-address mode, the processor does not interpret segment selectors as indexes into a descriptor table; instead, it uses them directly to form linear addresses as the 8086 processor does. It shifts the segment selector left by 4 bits to form a 20-bit base address (see Figure 16-1). The offset into a segment is added to the base address to create a linear address that maps directly to the physical address space.

When using 8086-style address translation, it is possible to specify addresses larger than 1 MByte. For example, with a segment selector value of FFFFH and an offset of FFFFH, the linear (and physical) address would be 10FFEFH (1 megabyte plus 64 KBytes). The 8086 processor, which can form addresses only up to 20 bits long, truncates the high-order bit, thereby “wrapping” this address to FFEFH. When operating in real-address mode, however, the processor does not truncate such an address and uses it as a physical address. (Note, however, that for IA-32 processors beginning with the Intel486 processor, the A20M# signal can be used in real-address mode to mask address line A20, thereby mimicking the 20-bit wrap-around behavior of the 8086

processor.) Care should be taken to ensure that A20M# based address wrapping is handled correctly in multiprocessor based system.

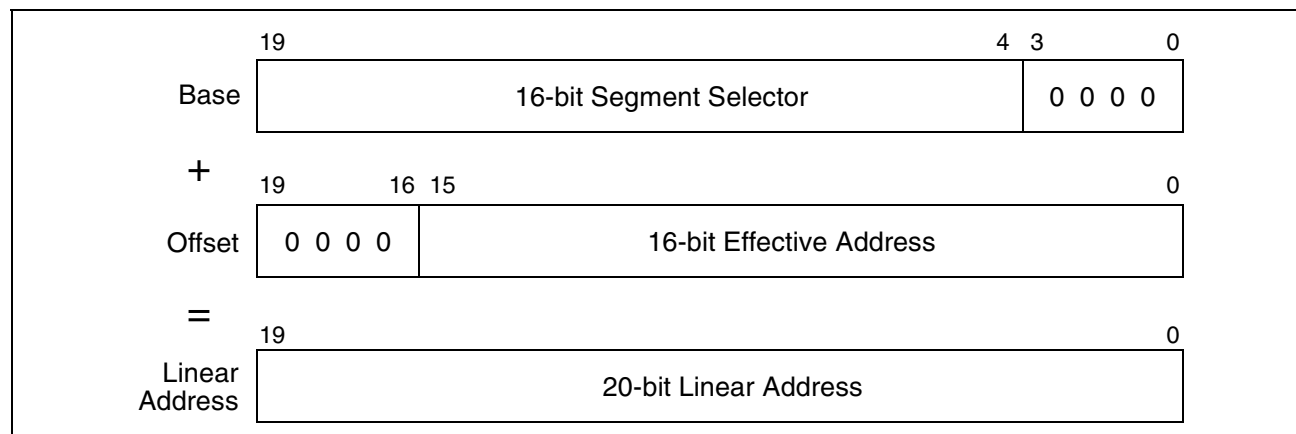


Figure 16-1. Real-Address Mode Address Translation

The IA-32 processors beginning with the Intel386 processor can generate 32-bit offsets using an address override prefix; however, in real-address mode, the value of a 32-bit offset may not exceed FFFFH without causing an exception.

For full compatibility with Intel 286 real-address mode, pseudo-protection faults (interrupt 12 or 13) occur if a 32-bit offset is generated outside the range 0 through FFFFH.

16.1.2. Registers Supported in Real-Address Mode

The register set available in real-address mode includes all the registers defined for the 8086 processor plus the new registers introduced in later IA-32 processors, such as the FS and GS segment registers, the debug registers, the control registers, and the floating-point unit registers. The 32-bit operand prefix allows a real-address mode program to use the 32-bit general-purpose registers (EAX, EBX, ECX, EDX, ESP, EBP, ESI, and EDI).

16.1.3. Instructions Supported in Real-Address Mode

The following instructions make up the core instruction set for the 8086 processor. If backwards compatibility to the Intel 286 and Intel 8086 processors is required, only these instructions should be used in a new program written to run in real-address mode.

- Move (MOV) instructions that move operands between general-purpose registers, segment registers, and between memory and general-purpose registers.
- The exchange (XCHG) instruction.
- Load segment register instructions LDS and LES.
- Arithmetic instructions ADD, ADC, SUB, SBB, MUL, IMUL, DIV, IDIV, INC, DEC, CMP, and NEG.

- Logical instructions AND, OR, XOR, and NOT.
- Decimal instructions DAA, DAS, AAA, AAS, AAM, and AAD.
- Stack instructions PUSH and POP (to general-purpose registers and segment registers).
- Type conversion instructions CWD, CDQ, CBW, and CWDE.
- Shift and rotate instructions SAL, SHL, SHR, SAR, ROL, ROR, RCL, and RCR.
- TEST instruction.
- Control instructions JMP, *Jcc*, CALL, RET, LOOP, LOOPE, and LOOPNE.
- Interrupt instructions INT *n*, INTO, and IRET.
- EFLAGS control instructions STC, CLC, CMC, CLD, STD, LAHF, SAHF, PUSHF, and POPF.
- I/O instructions IN, INS, OUT, and OUTS.
- Load effective address (LEA) instruction, and translate (XLATB) instruction.
- LOCK prefix.
- Repeat prefixes REP, REPE, REPZ, REPNE, and REPNZ.
- Processor halt (HLT) instruction.
- No operation (NOP) instruction.

The following instructions, added to later IA-32 processors (some in the Intel 286 processor and the remainder in the Intel386 processor), can be executed in real-address mode, if backwards compatibility to the Intel 8086 processor is not required.

- Move (MOV) instructions that operate on the control and debug registers.
- Load segment register instructions LSS, LFS, and LGS.
- Generalized multiply instructions and multiply immediate data.
- Shift and rotate by immediate counts.
- Stack instructions PUSHA, PUSHAD, POPA and POPAD, and PUSH immediate data.
- Move with sign extension instructions MOVSX and MOVZX.
- Long-displacement *Jcc* instructions.
- Exchange instructions CMPXCHG, CMPXCHG8B, and XADD.
- String instructions MOVS, CMPS, SCAS, LODS, and STOS.
- Bit test and bit scan instructions BT, BTS, BTR, BTC, BSF, and BSR; the byte-set-on condition instruction SET_{cc}; and the byte swap (BSWAP) instruction.
- Double shift instructions SHLD and SHRD.
- EFLAGS control instructions PUSHF and POPF.

- ENTER and LEAVE control instructions.
- BOUND instruction.
- CPU identification (CUID) instruction.
- System instructions CLTS, INVD, WINVD, INVLPG, LGDT, SGDT, LIDT, SIDT, LMSW, SMSW, RDMSR, WRMSR, RDTSC, and RDPMC.

Execution of any of the other IA-32 architecture instructions (not given in the previous two lists) in real-address mode result in an invalid-opcode exception (#UD) being generated.

16.1.4. Interrupt and Exception Handling

When operating in real-address mode, software must provide interrupt and exception-handling facilities that are separate from those provided in protected mode. Even during the early stages of processor initialization when the processor is still in real-address mode, elementary real-address mode interrupt and exception-handling facilities must be provided to insure reliable operation of the processor, or the initialization code must insure that no interrupts or exceptions will occur.

The IA-32 processors handle interrupts and exceptions in real-address mode similar to the way they handle them in protected mode. When a processor receives an interrupt or generates an exception, it uses the vector number of the interrupt or exception as an index into the interrupt table. (In protected mode, the interrupt table is called the **interrupt descriptor table (IDT)**, but in real-address mode, the table is usually called the **interrupt vector table**, or simply the **interrupt table**.) The entry in the interrupt vector table provides a pointer to an interrupt- or exception-handler procedure. (The pointer consists of a segment selector for a code segment and a 16-bit offset into the segment.) The processor performs the following actions to make an implicit call to the selected handler:

1. Pushes the current values of the CS and EIP registers onto the stack. (Only the 16 least-significant bits of the EIP register are pushed.)
2. Pushes the low-order 16 bits of the EFLAGS register onto the stack.
3. Clears the IF flag in the EFLAGS register to disable interrupts.
4. Clears the TF, RC, and AC flags, in the EFLAGS register.
5. Transfers program control to the location specified in the interrupt vector table.

An IRET instruction at the end of the handler procedure reverses these steps to return program control to the interrupted program. Exceptions do not return error codes in real-address mode.

The interrupt vector table is an array of 4-byte entries (see Figure 16-2). Each entry consists of a far pointer to a handler procedure, made up of a segment selector and an offset. The processor scales the interrupt or exception vector by 4 to obtain an offset into the interrupt table. Following reset, the base of the interrupt vector table is located at physical address 0 and its limit is set to 3FFH. In the Intel 8086 processor, the base address and limit of the interrupt vector table cannot be changed. In the later IA-32 processors, the base address and limit of the interrupt vector table are contained in the IDTR register and can be changed using the LIDT instruction.

(For backward compatibility to Intel 8086 processors, the default base address and limit of the interrupt vector table should not be changed.)

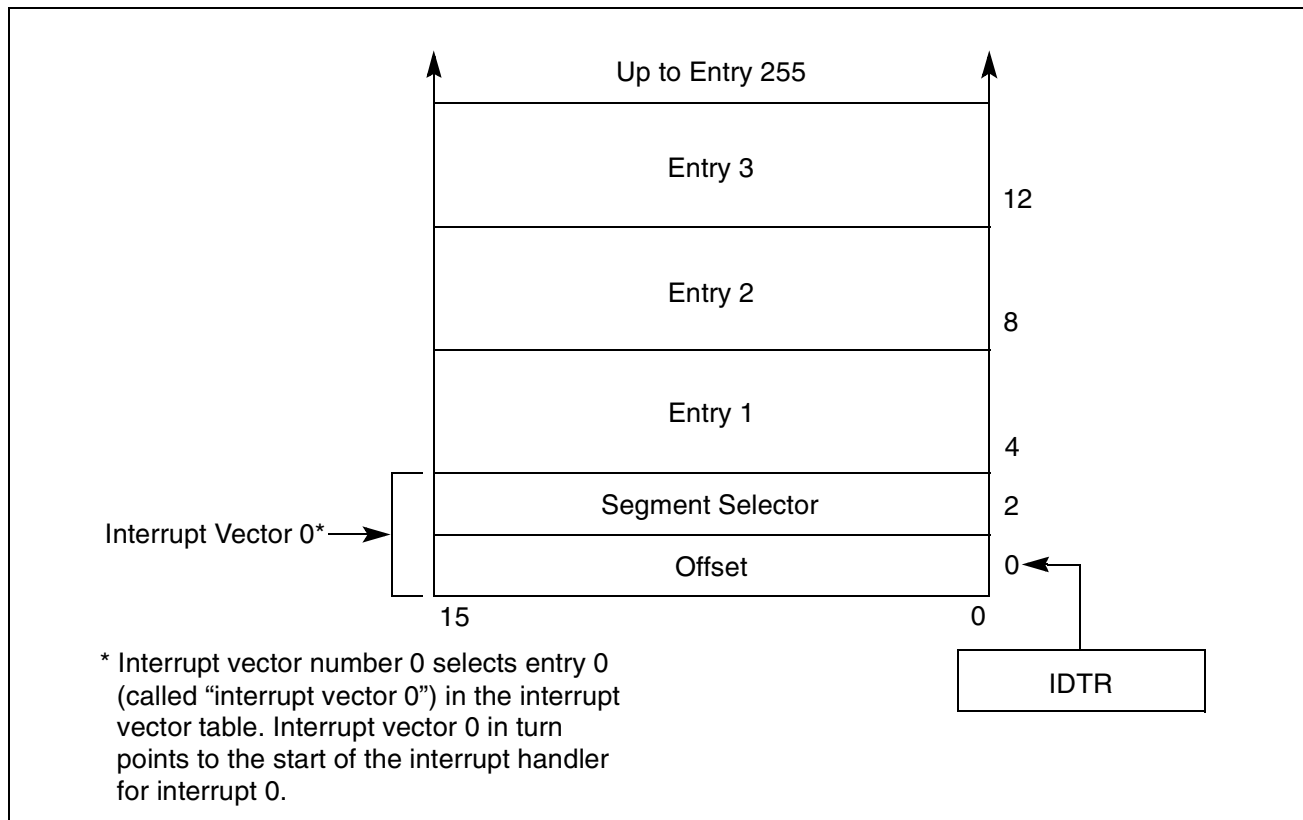


Figure 16-2. Interrupt Vector Table in Real-Address Mode

Table 16-1 shows the interrupt and exception vectors that can be generated in real-address mode and virtual-8086 mode, and in the Intel 8086 processor. See Chapter 5, *Interrupt and Exception Handling*, for a description of the exception conditions.

16.2. VIRTUAL-8086 MODE

Virtual-8086 mode is actually a special type of a task that runs in protected mode. When the operating-system or executive switches to a virtual-8086-mode task, the processor emulates an Intel 8086 processor. The execution environment of the processor while in the 8086-emulation state is the same as is described in Section 16.1., "Real-Address Mode" for real-address mode, including the extensions. The major difference between the two modes is that in virtual-8086 mode the 8086 emulator uses some protected-mode services (such as the protected-mode interrupt and exception-handling and paging facilities).

As in real-address mode, any new or legacy program that has been assembled and/or compiled to run on an Intel 8086 processor will run in a virtual-8086-mode task. And several 8086 programs can be run as virtual-8086-mode tasks concurrently with normal protected-mode tasks, using the processor's multitasking facilities.

Table 16-1. Real-Address Mode Exceptions and Interrupts

Vector No.	Description	Real-Address Mode	Virtual-8086 Mode	Intel 8086 Processor
0	Divide Error (#DE)	Yes	Yes	Yes
1	Debug Exception (#DB)	Yes	Yes	No
2	NMI Interrupt	Yes	Yes	Yes
3	Breakpoint (#BP)	Yes	Yes	Yes
4	Overflow (#OF)	Yes	Yes	Yes
5	BOUND Range Exceeded (#BR)	Yes	Yes	Reserved
6	Invalid Opcode (#UD)	Yes	Yes	Reserved
7	Device Not Available (#NM)	Yes	Yes	Reserved
8	Double Fault (#DF)	Yes	Yes	Reserved
9	(Intel reserved. Do not use.)	Reserved	Reserved	Reserved
10	Invalid TSS (#TS)	Reserved	Yes	Reserved
11	Segment Not Present (#NP)	Reserved	Yes	Reserved
12	Stack Fault (#SS)	Yes	Yes	Reserved
13	General Protection (#GP)*	Yes	Yes	Reserved
14	Page Fault (#PF)	Reserved	Yes	Reserved
15	(Intel reserved. Do not use.)	Reserved	Reserved	Reserved
16	Floating-Point Error (#MF)	Yes	Yes	Reserved
17	Alignment Check (#AC)	Reserved	Yes	Reserved
18	Machine Check (#MC)	Yes	Yes	Reserved
19-31	(Intel reserved. Do not use.)	Reserved	Reserved	Reserved
32-255	User Defined Interrupts	Yes	Yes	Yes

NOTE:

- * In the real-address mode, vector 13 is the segment overrun exception. In protected and virtual-8086 modes, this exception covers all general-protection error conditions, including traps to the virtual-8086 monitor from virtual-8086 mode.

16.2.1. Enabling Virtual-8086 Mode

The processor runs in virtual-8086 mode when the VM (virtual machine) flag in the EFLAGS register is set. This flag can only be set when the processor switches to a new protected-mode task or resumes virtual-8086 mode via an IRET instruction.

System software cannot change the state of the VM flag directly in the EFLAGS register (for example, by using the POPFD instruction). Instead it changes the flag in the image of the EFLAGS register stored in the TSS or on the stack following a call to an interrupt- or exception-handler procedure. For example, software sets the VM flag in the EFLAGS image in the TSS when first creating a virtual-8086 task.

The processor tests the VM flag under three general conditions:

- When loading segment registers, to determine whether to use 8086-style address translation.
- When decoding instructions, to determine which instructions are not supported in virtual-8086 mode and which instructions are sensitive to IOPL.
- When checking privileged instructions, on page accesses, or when performing other permission checks. (Virtual-8086 mode always executes at CPL 3.)

16.2.2. Structure of a Virtual-8086 Task

A virtual-8086-mode task consists of the following items:

- A 32-bit TSS for the task.
- The 8086 program.
- A virtual-8086 monitor.
- 8086 operating-system services.

The TSS of the new task must be a 32-bit TSS, not a 16-bit TSS, because the 16-bit TSS does not load the most-significant word of the EFLAGS register, which contains the VM flag. All TSS's, stacks, data, and code used to handle exceptions when in virtual-8086 mode must also be 32-bit segments.

The processor enters virtual-8086 mode to run the 8086 program and returns to protected mode to run the virtual-8086 monitor.

The virtual-8086 monitor is a 32-bit protected-mode code module that runs at a CPL of 0. The monitor consists of initialization, interrupt- and exception-handling, and I/O emulation procedures that emulate a personal computer or other 8086-based platform. Typically, the monitor is either part of or closely associated with the protected-mode general-protection (#GP) exception handler, which also runs at a CPL of 0. As with any protected-mode code module, code-segment descriptors for the virtual-8086 monitor must exist in the GDT or in the task's LDT. The virtual-8086 monitor also may need data-segment descriptors so it can examine the IDT or other parts of the 8086 program in the first 1 MByte of the address space. The linear addresses above 10FFEFH are available for the monitor, the operating system, and other system software.

The 8086 operating-system services consists of a kernel and/or operating-system procedures that the 8086 program makes calls to. These services can be implemented in either of the following two ways:

- They can be included in the 8086 program. This approach is desirable for either of the following reasons:
 - The 8086 program code modifies the 8086 operating-system services.
 - There is not sufficient development time to merge the 8086 operating-system services into main operating system or executive.
- They can be implemented or emulated in the virtual-8086 monitor. This approach is desirable for any of the following reasons:
 - The 8086 operating-system procedures can be more easily coordinated among several virtual-8086 tasks.
 - Memory can be saved by not duplicating 8086 operating-system procedure code for several virtual-8086 tasks.
 - The 8086 operating-system procedures can be easily emulated by calls to the main operating system or executive.

The approach chosen for implementing the 8086 operating-system services may result in different virtual-8086-mode tasks using different 8086 operating-system services.

16.2.3. Paging of Virtual-8086 Tasks

Even though a program running in virtual-8086 mode can use only 20-bit linear addresses, the processor converts these addresses into 32-bit linear addresses before mapping them to the physical address space. If paging is being used, the 8086 address space for a program running in virtual-8086 mode can be paged and located in a set of pages in physical address space. If paging is used, it is transparent to the program running in virtual-8086 mode just as it is for any task running on the processor.

Paging is not necessary for a single virtual-8086-mode task, but paging is useful or necessary in the following situations:

- When running multiple virtual-8086-mode tasks. Here, paging allows the lower 1 MByte of the linear address space for each virtual-8086-mode task to be mapped to a different physical address location.
- When emulating the 8086 address-wraparound that occurs at 1 MByte. When using 8086-style address translation, it is possible to specify addresses larger than 1 MByte. These addresses automatically wraparound in the Intel 8086 processor (see Section 16.1.1., “Address Translation in Real-Address Mode”). If any 8086 programs depend on address wraparound, the same effect can be achieved in a virtual-8086-mode task by mapping the linear addresses between 100000H and 110000H and linear addresses between 0 and 10000H to the same physical addresses.

- When sharing the 8086 operating-system services or ROM code that is common to several 8086 programs running as different 8086-mode tasks.
- When redirecting or trapping references to memory-mapped I/O devices.

16.2.4. Protection within a Virtual-8086 Task

Protection is not enforced between the segments of an 8086 program. Either of the following techniques can be used to protect the system software running in a virtual-8086-mode task from the 8086 program:

- Reserve the first 1 MByte plus 64 KBytes of each task's linear address space for the 8086 program. An 8086 processor task cannot generate addresses outside this range.
- Use the U/S flag of page-table entries to protect the virtual-8086 monitor and other system software in the virtual-8086 mode task space. When the processor is in virtual-8086 mode, the CPL is 3. Therefore, an 8086 processor program has only user privileges. If the pages of the virtual-8086 monitor have supervisor privilege, they cannot be accessed by the 8086 program.

16.2.5. Entering Virtual-8086 Mode

Figure 16-3 summarizes the methods of entering and leaving virtual-8086 mode. The processor switches to virtual-8086 mode in either of the following situations:

- Task switch when the VM flag is set to 1 in the EFLAGS register image stored in the TSS for the task. Here the task switch can be initiated in either of two ways:
 - A CALL or JMP instruction.
 - An IRET instruction, where the NT flag in the EFLAGS image is set to 1.
- Return from a protected-mode interrupt or exception handler when the VM flag is set to 1 in the EFLAGS register image on the stack.

When a task switch is used to enter virtual-8086 mode, the TSS for the virtual-8086-mode task must be a 32-bit TSS. (If the new TSS is a 16-bit TSS, the upper word of the EFLAGS register is not in the TSS, causing the processor to clear the VM flag when it loads the EFLAGS register.) The processor updates the VM flag prior to loading the segment registers from their images in the new TSS. The new setting of the VM flag determines whether the processor interprets the contents of the segment registers as 8086-style segment selectors or protected-mode segment selectors. When the VM flag is set, the segment registers are loaded from the TSS, using 8086-style address translation to form base addresses.

See Section 16.3., “Interrupt and Exception Handling in Virtual-8086 Mode”, for information on entering virtual-8086 mode on a return from an interrupt or exception handler.

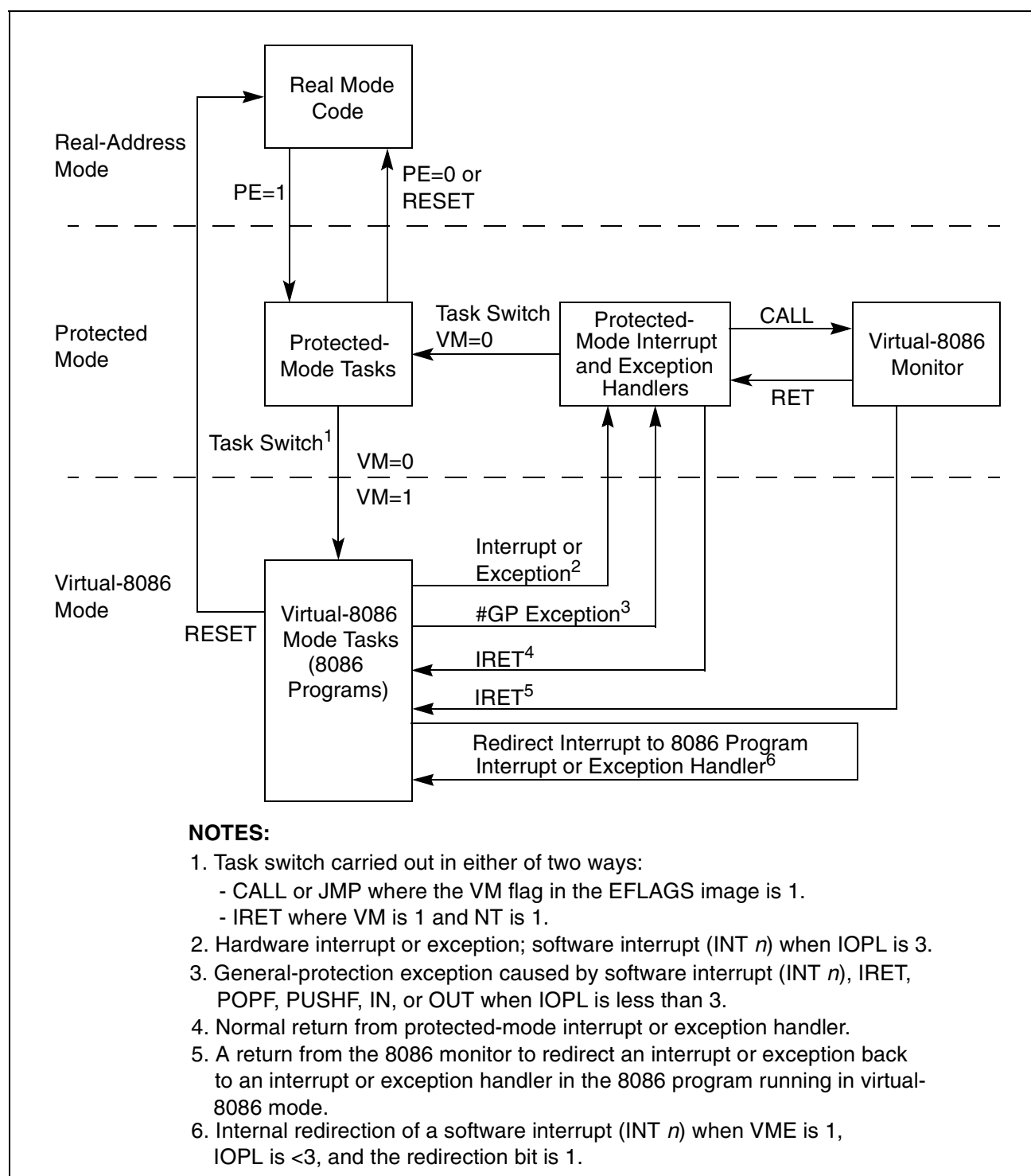


Figure 16-3. Entering and Leaving Virtual-8086 Mode

16.2.6. Leaving Virtual-8086 Mode

The processor can leave the virtual-8086 mode only through an interrupt or exception. The following are situations where an interrupt or exception will lead to the processor leaving virtual-8086 mode (see Figure 16-3):

- The processor services a hardware interrupt generated to signal the suspension of execution of the virtual-8086 application. This hardware interrupt may be generated by a timer or other external mechanism. Upon receiving the hardware interrupt, the processor enters protected mode and switches to a protected-mode (or another virtual-8086 mode) task either through a task gate in the protected-mode IDT or through a trap or interrupt gate that points to a handler that initiates a task switch. A task switch from a virtual-8086 task to another task loads the EFLAGS register from the TSS of the new task. The value of the VM flag in the new EFLAGS determines if the new task executes in virtual-8086 mode or not.
- The processor services an exception caused by code executing the virtual-8086 task or services a hardware interrupt that “belongs to” the virtual-8086 task. Here, the processor enters protected mode and services the exception or hardware interrupt through the protected-mode IDT (normally through an interrupt or trap gate) and the protected-mode exception- and interrupt-handlers. The processor may handle the exception or interrupt within the context of the virtual 8086 task and return to virtual-8086 mode on a return from the handler procedure. The processor may also execute a task switch and handle the exception or interrupt in the context of another task.
- The processor services a software interrupt generated by code executing in the virtual-8086 task (such as a software interrupt to call a MS-DOS* operating system routine). The processor provides several methods of handling these software interrupts, which are discussed in detail in Section 16.3.3., “Class 3—Software Interrupt Handling in Virtual-8086 Mode”. Most of them involve the processor entering protected mode, often by means of a general-protection (#GP) exception. In protected mode, the processor can send the interrupt to the virtual-8086 monitor for handling and/or redirect the interrupt back to the application program running in virtual-8086 mode task for handling.

IA-32 processors that incorporate the virtual mode extension (enabled with the VME flag in control register CR4) are capable of redirecting software-generated interrupts back to the program’s interrupt handlers without leaving virtual-8086 mode. See Section 16.3.3.4., “Method 5: Software Interrupt Handling”, for more information on this mechanism.

- A hardware reset initiated by asserting the RESET or INIT pin is a special kind of interrupt. When a RESET or INIT is signaled while the processor is in virtual-8086 mode, the processor leaves virtual-8086 mode and enters real-address mode.
- Execution of the HLT instruction in virtual-8086 mode will cause a general-protection (GP#) fault, which the protected-mode handler generally sends to the virtual-8086 monitor. The virtual-8086 monitor then determines the correct execution sequence after verifying that it was entered as a result of a HLT execution.

See Section 16.3., “Interrupt and Exception Handling in Virtual-8086 Mode”, for information on leaving virtual-8086 mode to handle an interrupt or exception generated in virtual-8086 mode.

16.2.7. Sensitive Instructions

When an IA-32 processor is running in virtual-8086 mode, the CLI, STI, PUSHF, POPF, INT n , and IRET instructions are sensitive to IOPL. The IN, INS, OUT, and OUTS instructions, which are sensitive to IOPL in protected mode, are not sensitive in virtual-8086 mode.

The CPL is always 3 while running in virtual-8086 mode; if the IOPL is less than 3, an attempt to use the IOPL-sensitive instructions listed above triggers a general-protection exception (#GP). These instructions are sensitive to IOPL to give the virtual-8086 monitor a chance to emulate the facilities they affect.

16.2.8. Virtual-8086 Mode I/O

Many 8086 programs written for non-multitasking systems directly access I/O ports. This practice may cause problems in a multitasking environment. If more than one program accesses the same port, they may interfere with each other. Most multitasking systems require application programs to access I/O ports through the operating system. This results in simplified, centralized control.

The processor provides I/O protection for creating I/O that is compatible with the environment and transparent to 8086 programs. Designers may take any of several possible approaches to protecting I/O ports:

- Protect the I/O address space and generate exceptions for all attempts to perform I/O directly.
- Let the 8086 program perform I/O directly.
- Generate exceptions on attempts to access specific I/O ports.
- Generate exceptions on attempts to access specific memory-mapped I/O ports.

The method of controlling access to I/O ports depends upon whether they are I/O-port mapped or memory mapped.

16.2.8.1. I/O-PORT-MAPPED I/O

The I/O permission bit map in the TSS can be used to generate exceptions on attempts to access specific I/O port addresses. The I/O permission bit map of each virtual-8086-mode task determines which I/O addresses generate exceptions for that task. Because each task may have a different I/O permission bit map, the addresses that generate exceptions for one task may be different from the addresses for another task. This differs from protected mode in which, if the CPL is less than or equal to the IOPL, I/O access is allowed without checking the I/O permission

bit map. See Chapter 12, *Input/Output*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for more information about the I/O permission bit map.

16.2.8.2. MEMORY-MAPPED I/O

In systems which use memory-mapped I/O, the paging facilities of the processor can be used to generate exceptions for attempts to access I/O ports. The virtual-8086 monitor may use paging to control memory-mapped I/O in these ways:

- Map part of the linear address space of each task that needs to perform I/O to the physical address space where I/O ports are placed. By putting the I/O ports at different addresses (in different pages), the paging mechanism can enforce isolation between tasks.
- Map part of the linear address space to pages that are not-present. This generates an exception whenever a task attempts to perform I/O to those pages. System software then can interpret the I/O operation being attempted.

Software emulation of the I/O space may require too much operating system intervention under some conditions. In these cases, it may be possible to generate an exception for only the first attempt to access I/O. The system software then may determine whether a program can be given exclusive control of I/O temporarily, the protection of the I/O space may be lifted, and the program allowed to run at full speed.

16.2.8.3. SPECIAL I/O BUFFERS

Buffers of intelligent controllers (for example, a bit-mapped frame buffer) also can be emulated using page mapping. The linear space for the buffer can be mapped to a different physical space for each virtual-8086-mode task. The virtual-8086 monitor then can control which virtual buffer to copy onto the real buffer in the physical address space.

16.3. INTERRUPT AND EXCEPTION HANDLING IN VIRTUAL-8086 MODE

When the processor receives an interrupt or detects an exception condition while in virtual-8086 mode, it invokes an interrupt or exception handler, just as it does in protected or real-address mode. The interrupt or exception handler that is invoked and the mechanism used to invoke it depends on the class of interrupt or exception that has been detected or generated and the state of various system flags and fields.

In virtual-8086 mode, the interrupts and exceptions are divided into three classes for the purposes of handling:

- Class 1—All processor-generated exceptions and all hardware interrupts, including the NMI interrupt and the hardware interrupts sent to the processor's external interrupt delivery pins. All class 1 exceptions and interrupts are handled by the protected-mode exception and interrupt handlers.

- Class 2—Special case for maskable hardware interrupts (Section 5.3.2., “Maskable Hardware Interrupts”) when the virtual mode extensions are enabled.
- Class 3—All software-generated interrupts, that is interrupts generated with the INT *n* instruction¹.

The method the processor uses to handle class 2 and 3 interrupts depends on the setting of the following flags and fields:

- IOPL field (bits 12 and 13 in the EFLAGS register)—Controls how class 3 software interrupts are handled when the processor is in virtual-8086 mode (see Section 2.3., “System Flags and Fields in the EFLAGS Register”). This field also controls the enabling of the VIF and VIP flags in the EFLAGS register when the VME flag is set. The VIF and VIP flags are provided to assist in the handling of class 2 maskable hardware interrupts.
- VME flag (bit 0 in control register CR4)—Enables the virtual mode extension for the processor when set (see Section 2.5., “Control Registers”).
- Software interrupt redirection bit map (32 bytes in the TSS, see Figure 16-5)—Contains 256 flags that indicates how class 3 software interrupts should be handled when they occur in virtual-8086 mode. A software interrupt can be directed either to the interrupt and exception handlers in the currently running 8086 program or to the protected-mode interrupt and exception handlers.
- The virtual interrupt flag (VIF) and virtual interrupt pending flag (VIP) in the EFLAGS register—Provides **virtual interrupt support** for the handling of class 2 maskable hardware interrupts (see Section 16.3.2., “Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism”).

NOTE

The VME flag, software interrupt redirection bit map, and VIF and VIP flags are only available in IA-32 processors that support the virtual mode extensions. These extensions were introduced in the IA-32 architecture with the Pentium processor.

The following sections describe the actions that processor takes and the possible actions of interrupt and exception handlers for the two classes of interrupts described in the previous paragraphs. These sections describe three possible types of interrupt and exception handlers:

- Protected-mode interrupt and exceptions handlers—These are the standard handlers that the processor calls through the protected-mode IDT.
- Virtual-8086 monitor interrupt and exception handlers—These handlers are resident in the virtual-8086 monitor, and they are commonly accessed through a general-protection exception (#GP, interrupt 13) that is directed to the protected-mode general-protection exception handler.

1. The INT 3 instruction is a special case (see the description of the INT *n* instruction in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*).

- 8086 program interrupt and exception handlers—These handlers are part of the 8086 program that is running in virtual-8086 mode.

The following sections describe how these handlers are used, depending on the selected class and method of interrupt and exception handling.

16.3.1. Class 1—Hardware Interrupt and Exception Handling in Virtual-8086 Mode

In virtual-8086 mode, the Pentium, P6 family, Pentium 4, and Intel Xeon processors handle hardware interrupts and exceptions in the same manner as they are handled by the Intel486 and Intel386 processors. They invoke the protected-mode interrupt or exception handler that the interrupt or exception vector points to in the IDT. Here, the IDT entry must contain either a 32-bit trap or interrupt gate or a task gate. The following sections describe various ways that a virtual-8086 mode interrupt or exception can be handled after the protected-mode handler has been invoked.

See Section 16.3.2., “Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism”, for a description of the virtual interrupt mechanism that is available for handling maskable hardware interrupts while in virtual-8086 mode. When this mechanism is either not available or not enabled, maskable hardware interrupts are handled in the same manner as exceptions, as described in the following sections.

16.3.1.1. HANDLING AN INTERRUPT OR EXCEPTION THROUGH A PROTECTED-MODE TRAP OR INTERRUPT GATE

When an interrupt or exception vector points to a 32-bit trap or interrupt gate in the IDT, the gate must in turn point to a nonconforming, privilege-level 0, code segment. When accessing this code segment, processor performs the following steps.

1. Switches to 32-bit protected mode and privilege level 0.
2. Saves the state of the processor on the privilege-level 0 stack. The states of the EIP, CS, EFLAGS, ESP, SS, ES, DS, FS, and GS registers are saved (see Figure 16-4).
3. Clears the segment registers. Saving the DS, ES, FS, and GS registers on the stack and then clearing the registers lets the interrupt or exception handler safely save and restore these registers regardless of the type segment selectors they contain (protected-mode or 8086-style). The interrupt and exception handlers, which may be called in the context of either a protected-mode task or a virtual-8086-mode task, can use the same code sequences for saving and restoring the registers for any task. Clearing these registers before execution of the IRET instruction does not cause a trap in the interrupt handler. Interrupt procedures that expect values in the segment registers or that return values in the segment registers must use the register images saved on the stack for privilege level 0.
4. Clears VM, NT, RF and TF flags (in the EFLAGS register).
5. Begins executing the selected interrupt or exception handler.

If the trap or interrupt gate references a procedure in a conforming segment or in a segment at a privilege level other than 0, the processor generates a general-protection exception (#GP). Here, the error code is the segment selector of the code segment to which a call was attempted.

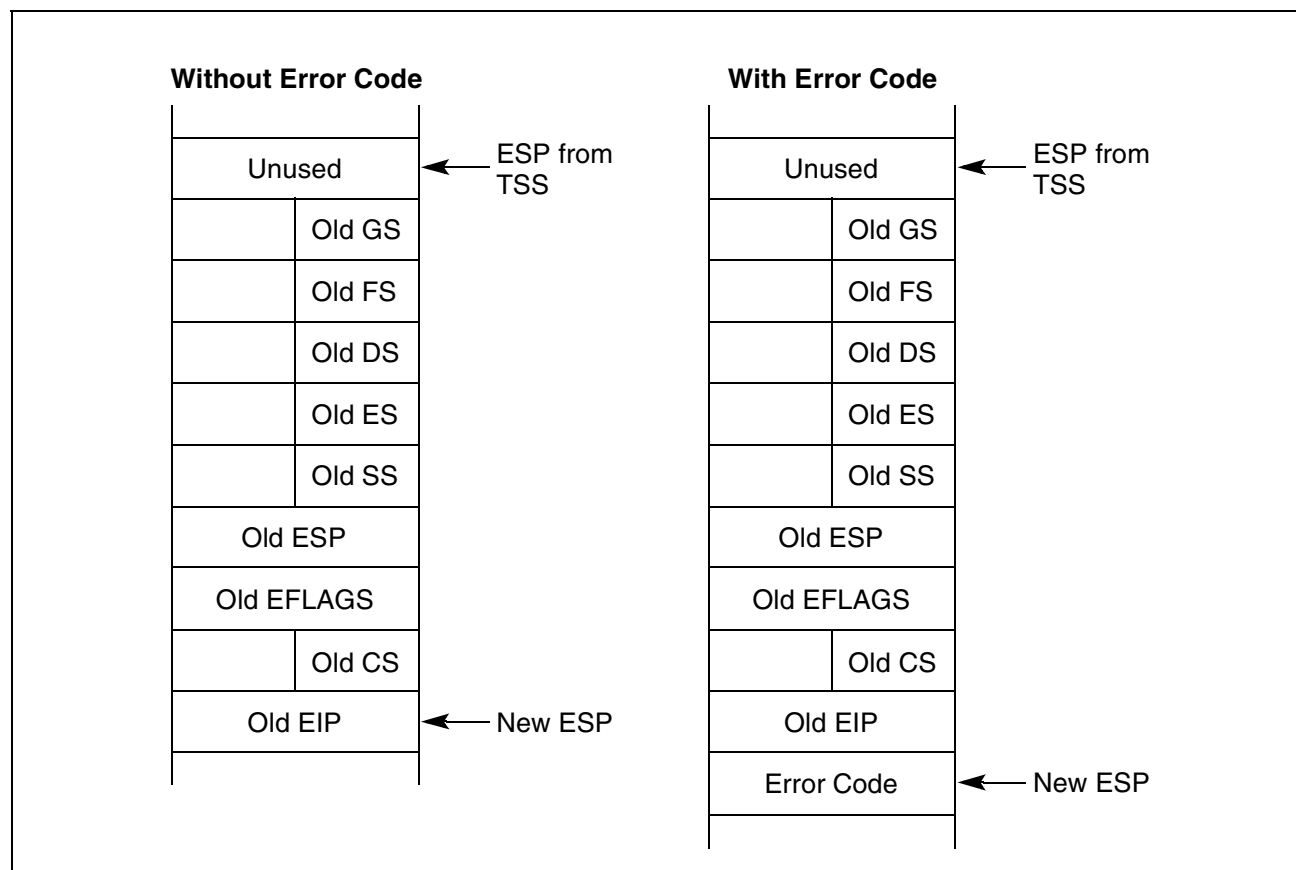


Figure 16-4. Privilege Level 0 Stack After Interrupt or Exception in Virtual-8086 Mode

Interrupt and exception handlers can examine the VM flag on the stack to determine if the interrupted procedure was running in virtual-8086 mode. If so, the interrupt or exception can be handled in one of three ways:

- The protected-mode interrupt or exception handler that was called can handle the interrupt or exception.
- The protected-mode interrupt or exception handler can call the virtual-8086 monitor to handle the interrupt or exception.
- The virtual-8086 monitor (if called) can in turn pass control back to the 8086 program's interrupt and exception handler.

If the interrupt or exception is handled with a protected-mode handler, the handler can return to the interrupted program in virtual-8086 mode by executing an IRET instruction. This instruction loads the EFLAGS and segment registers from the images saved in the privilege level 0 stack (see Figure 16-4). A set VM flag in the EFLAGS image causes the processor to switch back to virtual-8086 mode. The CPL at the time the IRET instruction is executed must be 0, otherwise the processor does not change the state of the VM flag.

The virtual-8086 monitor runs at privilege level 0, like the protected-mode interrupt and exception handlers. It is commonly closely tied to the protected-mode general-protection exception (#GP, vector 13) handler. If the protected-mode interrupt or exception handler calls the virtual-8086 monitor to handle the interrupt or exception, the return from the virtual-8086 monitor to the interrupted virtual-8086 mode program requires two return instructions: a RET instruction to return to the protected-mode handler and an IRET instruction to return to the interrupted program.

The virtual-8086 monitor has the option of directing the interrupt and exception back to an interrupt or exception handler that is part of the interrupted 8086 program, as described in Section 16.3.1.2., “Handling an Interrupt or Exception With an 8086 Program Interrupt or Exception Handler”.

16.3.1.2. HANDLING AN INTERRUPT OR EXCEPTION WITH AN 8086 PROGRAM INTERRUPT OR EXCEPTION HANDLER

Because it was designed to run on an 8086 processor, an 8086 program running in a virtual-8086-mode task contains an 8086-style interrupt vector table, which starts at linear address 0. If the virtual-8086 monitor correctly directs an interrupt or exception vector back to the virtual-8086-mode task it came from, the handlers in the 8086 program can handle the interrupt or exception. The virtual-8086 monitor must carry out the following steps to send an interrupt or exception back to the 8086 program:

1. Use the 8086 interrupt vector to locate the appropriate handler procedure in the 8086 program interrupt table.
2. Store the EFLAGS (low-order 16 bits only), CS and EIP values of the 8086 program on the privilege-level 3 stack. This is the stack that the virtual-8086-mode task is using. (The 8086 handler may use or modify this information.)
3. Change the return link on the privilege-level 0 stack to point to the privilege-level 3 handler procedure.
4. Execute an IRET instruction to pass control to the 8086 program handler.
5. When the IRET instruction from the privilege-level 3 handler triggers a general-protection exception (#GP) and thus effectively again calls the virtual-8086 monitor, restore the return link on the privilege-level 0 stack to point to the original, interrupted, privilege-level 3 procedure.
6. Copy the low order 16 bits of the EFLAGS image from the privilege-level 3 stack to the privilege-level 0 stack (because some 8086 handlers modify these flags to return information to the code that caused the interrupt).
7. Execute an IRET instruction to pass control back to the interrupted 8086 program.

Note that if an operating system intends to support all 8086 MS-DOS-based programs, it is necessary to use the actual 8086 interrupt and exception handlers supplied with the program. The reason for this is that some programs modify their own interrupt vector table to substitute (or hook in series) their own specialized interrupt and exception handlers.

16.3.1.3. HANDLING AN INTERRUPT OR EXCEPTION THROUGH A TASK GATE

When an interrupt or exception vector points to a task gate in the IDT, the processor performs a task switch to the selected interrupt- or exception-handling task. The following actions are carried out as part of this task switch:

1. The EFLAGS register with the VM flag set is saved in the current TSS.
2. The link field in the TSS of the called task is loaded with the segment selector of the TSS for the interrupted virtual-8086-mode task.
3. The EFLAGS register is loaded from the image in the new TSS, which clears the VM flag and causes the processor to switch to protected mode.
4. The NT flag in the EFLAGS register is set.
5. The processor begins executing the selected interrupt- or exception-handler task.

When an IRET instruction is executed in the handler task and the NT flag in the EFLAGS register is set, the processor switches from a protected-mode interrupt- or exception-handler task back to a virtual-8086-mode task. Here, the EFLAGS and segment registers are loaded from images saved in the TSS for the virtual-8086-mode task. If the VM flag is set in the EFLAGS image, the processor switches back to virtual-8086 mode on the task switch. The CPL at the time the IRET instruction is executed must be 0, otherwise the processor does not change the state of the VM flag.

16.3.2. Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism

Maskable hardware interrupts are those interrupts that are delivered through the INTR# pin or through an interrupt request to the local APIC (see Section 5.3.2., “Maskable Hardware Interrupts”). These interrupts can be inhibited (masked) from interrupting an executing program or task by clearing the IF flag in the EFLAGS register.

When the VME flag in control register CR4 is set and the IOPL field in the EFLAGS register is less than 3, two additional flags are activated in the EFLAGS register:

- VIF (virtual interrupt) flag, bit 19 of the EFLAGS register.
- VIP (virtual interrupt pending) flag, bit 20 of the EFLAGS register.

These flags provide the virtual-8086 monitor with more efficient control over handling maskable hardware interrupts that occur during virtual-8086 mode tasks. They also reduce interrupt-handling overhead, by eliminating the need for all IF related operations (such as PUSHF, POPF, CLI, and STI instructions) to trap to the virtual-8086 monitor. The purpose and use of these flags are as follows.

NOTE

The VIF and VIP flags are only available in IA-32 processors that support the virtual mode extensions. These extensions were introduced in the IA-32 architecture with the Pentium processor. When this mechanism is either not available or not enabled, maskable hardware interrupts are handled as class 1 interrupts. Here, if VIF and VIP flags are needed, the virtual-8086 monitor can implement them in software.

Existing 8086 programs commonly set and clear the IF flag in the EFLAGS register to enable and disable maskable hardware interrupts, respectively; for example, to disable interrupts while handling another interrupt or an exception. This practice works well in single task environments, but can cause problems in multitasking and multiple-processor environments, where it is often desirable to prevent an application program from having direct control over the handling of hardware interrupts. When using earlier IA-32 processors, this problem was often solved by creating a virtual IF flag in software. The IA-32 processors (beginning with the Pentium processor) provide hardware support for this virtual IF flag through the VIF and VIP flags.

The VIF flag is a virtualized version of the IF flag, which an application program running from within a virtual-8086 task can use to control the handling of maskable hardware interrupts. When the VIF flag is enabled, the CLI and STI instructions operate on the VIF flag instead of the IF flag. When an 8086 program executes the CLI instruction, the processor clears the VIF flag to request that the virtual-8086 monitor inhibit maskable hardware interrupts from interrupting program execution; when it executes the STI instruction, the processor sets the VIF flag requesting that the virtual-8086 monitor enable maskable hardware interrupts for the 8086 program. But actually the IF flag, managed by the operating system, always controls whether maskable hardware interrupts are enabled. Also, if under these circumstances an 8086 program tries to read or change the IF flag using the PUSHF or POPF instructions, the processor will change the VIF flag instead, leaving IF unchanged.

The VIP flag provides software a means of recording the existence of a deferred (or pending) maskable hardware interrupt. This flag is read by the processor but never explicitly written by the processor; it can only be written by software.

If the IF flag is set and the VIF and VIP flags are enabled, and the processor receives a maskable hardware interrupt (interrupt vector 0 through 255), the processor performs and the interrupt handler software should perform the following operations:

1. The processor invokes the protected-mode interrupt handler for the interrupt received, as described in the following steps. These steps are almost identical to those described for method 1 interrupt and exception handling in Section 16.3.1.1., “Handling an Interrupt or Exception Through a Protected-Mode Trap or Interrupt Gate”:
 - a. Switches to 32-bit protected mode and privilege level 0.
 - b. Saves the state of the processor on the privilege-level 0 stack. The states of the EIP, CS, EFLAGS, ESP, SS, ES, DS, FS, and GS registers are saved (see Figure 16-4). In the EFLAGS image on the stack, the IOPL field is set to 3 and the VIF flag is copied to the IF flag.

- c. Clears the segment registers.
 - d. Clears the VM flag in the EFLAGS register.
 - e. Begins executing the selected protected-mode interrupt handler.
2. The recommended action of the protected-mode interrupt handler is to read the VM flag from the EFLAGS image on the stack. If this flag is set, the handler makes a call to the virtual-8086 monitor.
3. The virtual-8086 monitor should read the VIF flag in the EFLAGS register.
 - If the VIF flag is clear, the virtual-8086 monitor sets the VIP flag in the EFLAGS image on the stack to indicate that there is a deferred interrupt pending and returns to the protected-mode handler.
 - If the VIF flag is set, the virtual-8086 monitor can handle the interrupt if it “belongs” to the 8086 program running in the interrupted virtual-8086 task; otherwise, it can call the protected-mode interrupt handler to handle the interrupt.
4. The protected-mode handler executes a return to the program executing in virtual-8086 mode.
5. Upon returning to virtual-8086 mode, the processor continues execution of the 8086 program.

When the 8086 program is ready to receive maskable hardware interrupts, it executes the STI instruction to set the VIF flag (enabling maskable hardware interrupts). Prior to setting the VIF flag, the processor automatically checks the VIP flag and does one of the following, depending on the state of the flag:

- If the VIP flag is clear (indicating no pending interrupts), the processor sets the VIF flag.
- If the VIP flag is set (indicating a pending interrupt), the processor generates a general-protection exception (#GP).

The recommended action of the protected-mode general-protection exception handler is to then call the virtual-8086 monitor and let it handle the pending interrupt. After handling the pending interrupt, the typical action of the virtual-8086 monitor is to clear the VIP flag and set the VIF flag in the EFLAGS image on the stack, and then execute a return to the virtual-8086 mode. The next time the processor receives a maskable hardware interrupt, it will then handle it as described in steps 1 through 5 earlier in this section.

If the processor finds that both the VIF and VIP flags are set at the beginning of an instruction, it generates a general-protection exception. This action allows the virtual-8086 monitor to handle the pending interrupt for the virtual-8086 mode task for which the VIF flag is enabled. Note that this situation can only occur immediately following execution of a POPF or IRET instruction or upon entering a virtual-8086 mode task through a task switch.

Note that the states of the VIF and VIP flags are not modified in real-address mode or during transitions between real-address and protected modes.

NOTE

The virtual interrupt mechanism described in this section is also available for use in protected mode, see Section 16.4., “Protected-Mode Virtual Interrupts”.

16.3.3. Class 3—Software Interrupt Handling in Virtual-8086 Mode

When the processor receives a software interrupt (an interrupt generated with the `INT n` instruction) while in virtual-8086 mode, it can use any of six different methods to handle the interrupt. The method selected depends on the settings of the VME flag in control register CR4, the IOPL field in the EFLAGS register, and the software interrupt redirection bit map in the TSS. Table 16-2 lists the six methods of handling software interrupts in virtual-8086 mode and the respective settings of the VME flag, IOPL field, and the bits in the interrupt redirection bit map for each method. The table also summarizes the various actions the processor takes for each method.

The VME flag enables the virtual mode extensions for the Pentium and later IA-32 processors. When this flag is clear, the processor responds to interrupts and exceptions in virtual-8086 mode in the same manner as an Intel386 or Intel486 processor does. When this flag is set, the virtual mode extension provides the following enhancements to virtual-8086 mode:

- Speeds up the handling of software-generated interrupts in virtual-8086 mode by allowing the processor to bypass the virtual-8086 monitor and redirect software interrupts back to the interrupt handlers that are part of the currently running 8086 program.
- Supports virtual interrupts for software written to run on the 8086 processor.

The IOPL value interacts with the VME flag and the bits in the interrupt redirection bit map to determine how specific software interrupts should be handled.

The software interrupt redirection bit map (see Figure 16-5) is a 32-byte field in the TSS. This map is located directly below the I/O permission bit map in the TSS. Each bit in the interrupt redirection bit map is mapped to an interrupt vector. Bit 0 in the interrupt redirection bit map (which maps to vector zero in the interrupt table) is located at the I/O base map address in the TSS minus 32 bytes. When a bit in this bit map is set, it indicates that the associated software interrupt (interrupt generated with an `INT n` instruction) should be handled through the protected-mode IDT and interrupt and exception handlers. When a bit in this bit map is clear, the processor redirects the associated software interrupt back to the interrupt table in the 8086 program (located at linear address 0 in the program’s address space).

NOTE

The software interrupt redirection bit map does not affect hardware generated interrupts and exceptions. Hardware generated interrupts and exceptions are always handled by the protected-mode interrupt and exception handlers.

Table 16-2. Software Interrupt Handling Methods While in Virtual-8086 Mode

Method	VME	IOPL	Bit in Redir. Bitmap*	Processor Action
1	0	3	X	Interrupt directed to a protected-mode interrupt handler: - Clears VM and TF flags - If serviced through interrupt gate, clears IF flag - Switches to privilege-level 0 stack - Pushes GS, FS, DS and ES onto privilege-level 0 stack - Clears GS, FS, DS and ES to 0 - Pushes SS, ESP, EFLAGS, CS and EIP of interrupted task onto privilege-level 0 stack - Sets CS and EIP from interrupt gate
2	0	< 3	X	Interrupt directed to protected-mode general-protection exception (#GP) handler.
3	1	< 3	1	Interrupt directed to a protected-mode general-protection exception (#GP) handler; VIF and VIP flag support for handling class 2 maskable hardware interrupts.
4	1	3	1	Interrupt directed to protected-mode interrupt handler: (see method 1 processor action).
5	1	3	0	Interrupt redirected to 8086 program interrupt handler: - Pushes EFLAGS with NT cleared and IOPL set to 0 - Pushes CS and EIP (lower 16 bits only) - Clears IF flag - Clears TF flag - Loads CS and EIP (lower 16 bits only) from selected entry in the interrupt vector table of the current virtual-8086 task
6	1	< 3	0	Interrupt redirected to 8086 program interrupt handler; VIF and VIP flag support for handling class 2 maskable hardware interrupts: - Pushes EFLAGS with IOPL set to 3 and VIF copied to IF - Pushes CS and EIP (lower 16 bits only) - Clears the VIF flag - Clears TF flag - Loads CS and EIP (lower 16 bits only) from selected entry in the interrupt vector table of the current virtual-8086 task

NOTE:

- * When set to 0, software interrupt is redirected back to the 8086 program interrupt handler; when set to 1, interrupt is directed to protected-mode handler.

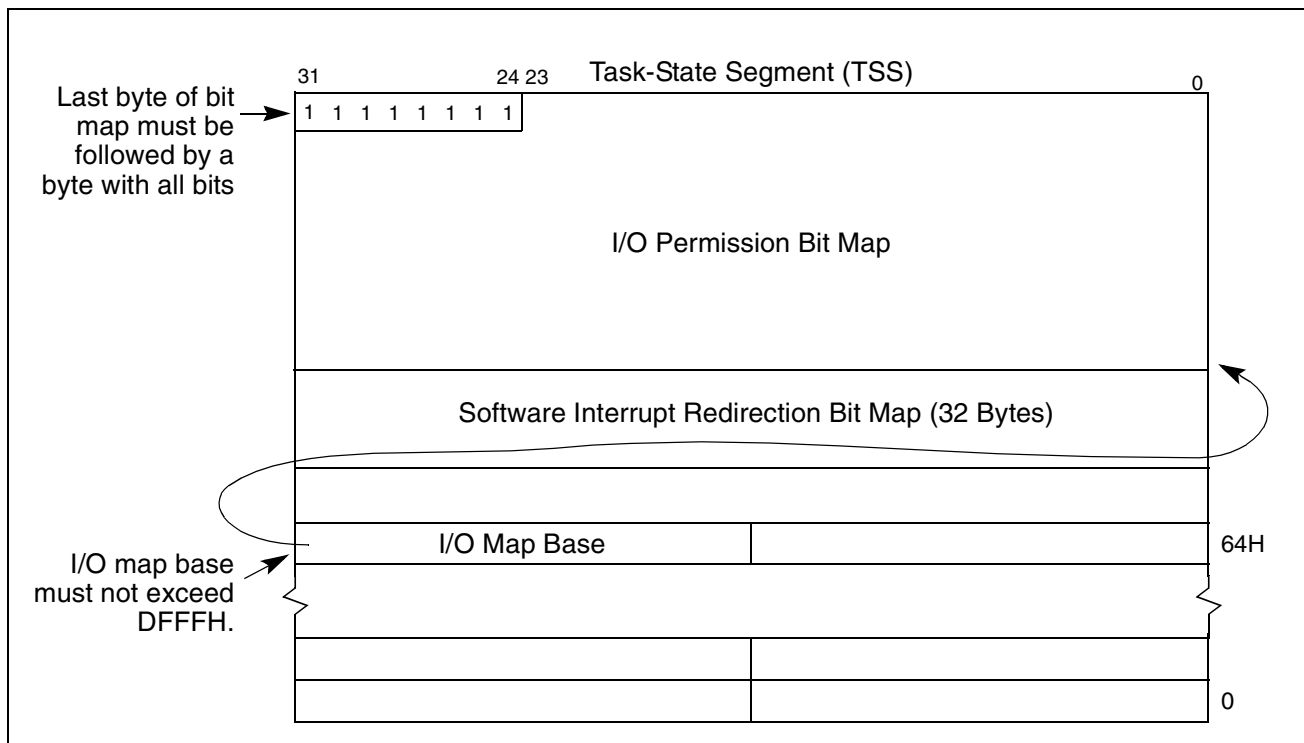


Figure 16-5. Software Interrupt Redirection Bit Map in TSS

Redirecting software interrupts back to the 8086 program potentially speeds up interrupt handling because a switch back and forth between virtual-8086 mode and protected mode is not required. This latter interrupt-handling technique is particularly useful for 8086 operating systems (such as MS-DOS) that use the `INT n` instruction to call operating system procedures.

The `CPUID` instruction can be used to verify that the virtual mode extension is implemented on the processor. Bit 1 of the feature flags register (EDX) indicates the availability of the virtual mode extension (see “`CPUID`—CPU Identification” in Chapter 3 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 2*).

The following sections describe the six methods (or mechanisms) for handling software interrupts in virtual-8086 mode. See Section 16.3.2., “Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism”, for a description of the use of the VIF and VIP flags in the EFLAGS register for handling maskable hardware interrupts.

16.3.3.1. METHOD 1: SOFTWARE INTERRUPT HANDLING

When the VME flag in control register CR4 is clear and the IOPL field is 3, a Pentium or later IA-32 processor handles software interrupts in the same manner as they are handled by an Intel386 or Intel486 processor. It executes an implicit call to the interrupt handler in the protected-mode IDT pointed to by the interrupt vector. See Section 16.3.1., “Class 1—Hardware Interrupt and Exception Handling in Virtual-8086 Mode”, for a complete description of this mechanism and its possible uses.

16.3.3.2. METHODS 2 AND 3: SOFTWARE INTERRUPT HANDLING

When a software interrupt occurs in virtual-8086 mode and the method 2 or 3 conditions are present, the processor generates a general-protection exception (#GP). Method 2 is enabled when the VME flag is set to 0 and the IOPL value is less than 3. Here the IOPL value is used to bypass the protected-mode interrupt handlers and cause any software interrupt that occurs in virtual-8086 mode to be treated as a protected-mode general-protection exception (#GP). The general-protection exception handler calls the virtual-8086 monitor, which can then emulate an 8086-program interrupt handler or pass control back to the 8086 program's handler, as described in Section 16.3.1.2., "Handling an Interrupt or Exception With an 8086 Program Interrupt or Exception Handler".

Method 3 is enabled when the VME flag is set to 1, the IOPL value is less than 3, and the corresponding bit for the software interrupt in the software interrupt redirection bit map is set to 1. Here, the processor performs the same operation as it does for method 2 software interrupt handling. If the corresponding bit for the software interrupt in the software interrupt redirection bit map is set to 0, the interrupt is handled using method 6 (see Section 16.3.3.5., "Method 6: Software Interrupt Handling").

16.3.3.3. METHOD 4: SOFTWARE INTERRUPT HANDLING

Method 4 handling is enabled when the VME flag is set to 1, the IOPL value is 3, and the bit for the interrupt vector in the redirection bit map is set to 1. Method 4 software interrupt handling allows method 1 style handling when the virtual mode extension is enabled; that is, the interrupt is directed to a protected-mode handler (see Section 16.3.3.1., "Method 1: Software Interrupt Handling").

16.3.3.4. METHOD 5: SOFTWARE INTERRUPT HANDLING

Method 5 software interrupt handling provides a streamlined method of redirecting software interrupts (invoked with the INT *n* instruction) that occur in virtual 8086 mode back to the 8086 program's interrupt vector table and its interrupt handlers. Method 5 handling is enabled when the VME flag is set to 1, the IOPL value is 3, and the bit for the interrupt vector in the redirection bit map is set to 0. The processor performs the following actions to make an implicit call to the selected 8086 program interrupt handler:

1. Pushes the low-order 16 bits of the EFLAGS register onto the stack with the NT and IOPL bits cleared.
2. Pushes the current values of the CS and EIP registers onto the current stack. (Only the 16 least-significant bits of the EIP register are pushed and no stack switch occurs.)
3. Clears the IF flag in the EFLAGS register to disable interrupts.
4. Clears the TF flag, in the EFLAGS register.
5. Locates the 8086 program interrupt vector table at linear address 0 for the 8086-mode task.

6. Loads the CS and EIP registers with values from the interrupt vector table entry pointed to by the interrupt vector number. Only the 16 low-order bits of the EIP are loaded and the 16 high-order bits are set to 0. The interrupt vector table is assumed to be at linear address 0 of the current virtual-8086 task.
7. Begins executing the selected interrupt handler.

An IRET instruction at the end of the handler procedure reverses these steps to return program control to the interrupted 8086 program.

Note that with method 5 handling, a mode switch from virtual-8086 mode to protected mode does not occur. The processor remains in virtual-8086 mode throughout the interrupt-handling operation.

The method 5 handling actions are virtually identical to the actions the processor takes when handling software interrupts in real-address mode. The benefit of using method 5 handling to access the 8086 program handlers is that it avoids the overhead of methods 2 and 3 handling, which requires first going to the virtual-8086 monitor, then to the 8086 program handler, then back again to the virtual-8086 monitor, before returning to the interrupted 8086 program (see Section 16.3.1.2., “Handling an Interrupt or Exception With an 8086 Program Interrupt or Exception Handler”).

NOTE

Methods 1 and 4 handling can handle a software interrupt in a virtual-8086 task with a regular protected-mode handler, but this approach requires all virtual-8086 tasks to use the same software interrupt handlers, which generally does not give sufficient latitude to the programs running in the virtual-8086 tasks, particularly MS-DOS programs.

16.3.3.5. METHOD 6: SOFTWARE INTERRUPT HANDLING

Method 6 handling is enabled when the VME flag is set to 1, the IOPL value is less than 3, and the bit for the interrupt or exception vector in the redirection bit map is set to 0. With method 6 interrupt handling, software interrupts are handled in the same manner as was described for method 5 handling (see Section 16.3.3.4., “Method 5: Software Interrupt Handling”).

Method 6 differs from method 5 in that with the IOPL value set to less than 3, the VIF and VIP flags in the EFLAGS register are enabled, providing virtual interrupt support for handling class 2 maskable hardware interrupts (see Section 16.3.2., “Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism”). These flags provide the virtual-8086 monitor with an efficient means of handling maskable hardware interrupts that occur during a virtual-8086 mode task. Also, because the IOPL value is less than 3 and the VIF flag is enabled, the information pushed on the stack by the processor when invoking the interrupt handler is slightly different between methods 5 and 6 (see Table 16-2).

16.4. PROTECTED-MODE VIRTUAL INTERRUPTS

The IA-32 processors (beginning with the Pentium processor) also support the VIF and VIP flags in the EFLAGS register in protected mode by setting the PVI (protected-mode virtual interrupt) flag in the CR4 register. Setting the PVI flag allows applications running at privilege level 3 to execute the CLI and STI instructions without causing a general-protection exception (#GP) or affecting hardware interrupts.

When the PVI flag is set to 1, the CPL is 3, and the IOPL is less than 3, the STI and CLI instructions set and clear the VIF flag in the EFLAGS register, leaving IF unaffected. In this mode of operation, an application running in protected mode and at a CPL of 3 can inhibit interrupts in the same manner as is described in Section 16.3.2., “Class 2—Maskable Hardware Interrupt Handling in Virtual-8086 Mode Using the Virtual Interrupt Mechanism”, for a virtual-8086 mode task. When the application executes the CLI instruction, the processor clears the VIF flag. If the processor receives a maskable hardware interrupt, the processor invokes the protected-mode interrupt handler. This handler checks the state of the VIF flag in the EFLAGS register. If the VIF flag is clear (indicating that the active task does not want to have interrupts handled now), the handler sets the VIP flag in the EFLAGS image on the stack and returns to the privilege-level 3 application, which continues program execution. When the application executes a STI instruction to set the VIF flag, the processor automatically invokes the general-protection exception handler, which can then handle the pending interrupt. After handling the pending interrupt, the handler typically sets the VIF flag and clears the VIP flag in the EFLAGS image on the stack and executes a return to the application program. The next time the processor receives a maskable hardware interrupt, the processor will handle it in the normal manner for interrupts received while the processor is operating at a CPL of 3.

As with the virtual mode extension (enabled with the VME flag in the CR4 register), the protected-mode virtual interrupt extension only affects maskable hardware interrupts (interrupt vectors 32 through 255). NMI interrupts and exceptions are handled in the normal manner.

When protected-mode virtual interrupts are disabled (that is, when the PVI flag in control register CR4 is set to 0, the CPL is less than 3, or the IOPL value is 3), then the CLI and STI instructions execute in a manner compatible with the Intel486 processor. That is, if the CPL is greater (less privileged) than the I/O privilege level (IOPL), a general-protection exception occurs. If the IOPL value is 3, CLI and STI clear or set the IF flag, respectively.

PUSHF, POPF, IRET and INT are executed like in the Intel486 processor, regardless of whether protected-mode virtual interrupts are enabled.

It is only possible to enter virtual-8086 mode through a task switch or the execution of an IRET instruction, and it is only possible to leave virtual-8086 mode by faulting to a protected-mode interrupt handler (typically the general-protection exception handler, which in turn calls the virtual 8086-mode monitor). In both cases, the EFLAGS register is saved and restored. This is not true, however, in protected mode when the PVI flag is set and the processor is not in virtual-8086 mode. Here, it is possible to call a procedure at a different privilege level, in which case the EFLAGS register is not saved or modified. However, the states of VIF and VIP flags are never examined by the processor when the CPL is not 3.

17

Mixing 16-Bit and 32-Bit Code

CHAPTER 17

MIXING 16-BIT AND 32-BIT CODE

Program modules written to run on IA-32 processors can be either 16-bit modules or 32-bit modules. Table 17-1 shows the characteristic of 16-bit and 32-bit modules.

Table 17-1. Characteristics of 16-Bit and 32-Bit Program Modules

Characteristic	16-Bit Program Modules	32-Bit Program Modules
Segment Size	0 to 64 KBytes	0 to 4 GBytes
Operand Sizes	8 bits and 16 bits	8 bits and 32 bits
Pointer Offset Size (Address Size)	16 bits	32 bits
Stack Pointer Size	16 Bits	32 Bits
Control Transfers Allowed to Code Segments of This Size	16 Bits	32 Bits

The IA-32 processors function most efficiently when executing 32-bit program modules. They can, however, also execute 16-bit program modules, in any of the following ways:

- In real-address mode.
- In virtual-8086 mode.
- System management mode (SMM).
- As a protected-mode task, when the code, data, and stack segments for the task are all configured as a 16-bit segments.
- By integrating 16-bit and 32-bit segments into a single protected-mode task.
- By integrating 16-bit operations into 32-bit code segments.

Real-address mode, virtual-8086 mode, and SMM are native 16-bit modes. A legacy program assembled and/or compiled to run on an Intel 8086 or Intel 286 processor should run in real-address mode or virtual-8086 mode without modification. Sixteen-bit program modules can also be written to run in real-address mode for handling system initialization or to run in SMM for handling system management functions. See Chapter 16, *8086 Emulation*, for detailed information on real-address mode and virtual-8086 mode; see Chapter 13, *System Management*, for information on SMM.

This chapter describes how to integrate 16-bit program modules with 32-bit program modules when operating in protected mode and how to mix 16-bit and 32-bit code within 32-bit code segments.

17.1. DEFINING 16-BIT AND 32-BIT PROGRAM MODULES

The following IA-32 architecture mechanisms are used to distinguish between and support 16-bit and 32-bit segments and operations:

- The D (default operand and address size) flag in code-segment descriptors.
- The B (default stack size) flag in stack-segment descriptors.
- 16-bit and 32-bit call gates, interrupt gates, and trap gates.
- Operand-size and address-size instruction prefixes.
- 16-bit and 32-bit general-purpose registers.

The D flag in a code-segment descriptor determines the default operand-size and address-size for the instructions of a code segment. (In real-address mode and virtual-8086 mode, which do not use segment descriptors, the default is 16 bits.) A code segment with its D flag set is a 32-bit segment; a code segment with its D flag clear is a 16-bit segment.

The B flag in the stack-segment descriptor specifies the size of stack pointer (the 32-bit ESP register or the 16-bit SP register) used by the processor for implicit stack references. The B flag for all data descriptors also controls upper address range for expand down segments.

When transferring program control to another code segment through a call gate, interrupt gate, or trap gate, the operand size used during the transfer is determined by the type of gate used (16-bit or 32-bit), (not by the D-flag or prefix of the transfer instruction). The gate type determines how return information is saved on the stack (or stacks).

For most efficient and trouble-free operation of the processor, 32-bit programs or tasks should have the D flag in the code-segment descriptor and the B flag in the stack-segment descriptor set, and 16-bit programs or tasks should have these flags clear. Program control transfers from 16-bit segments to 32-bit segments (and vice versa) are handled most efficiently through call, interrupt, or trap gates.

Instruction prefixes can be used to override the default operand size and address size of a code segment. These prefixes can be used in real-address mode as well as in protected mode and virtual-8086 mode. An operand-size or address-size prefix only changes the size for the duration of the instruction.

17.2. MIXING 16-BIT AND 32-BIT OPERATIONS WITHIN A CODE SEGMENT

The following two instruction prefixes allow mixing of 32-bit and 16-bit operations within one segment:

- The operand-size prefix (66H)
- The address-size prefix (67H)

These prefixes reverse the default size selected by the D flag in the code-segment descriptor. For example, the processor can interpret the (MOV *mem, reg*) instruction in any of four ways:

- In a 32-bit code segment:
 - Moves 32 bits from a 32-bit register to memory using a 32-bit effective address.
 - If preceded by an operand-size prefix, moves 16 bits from a 16-bit register to memory using a 32-bit effective address.
 - If preceded by an address-size prefix, moves 32 bits from a 32-bit register to memory using a 16-bit effective address.
 - If preceded by both an address-size prefix and an operand-size prefix, moves 16 bits from a 16-bit register to memory using a 16-bit effective address.
- In a 16-bit code segment:
 - Moves 16 bits from a 16-bit register to memory using a 16-bit effective address.
 - If preceded by an operand-size prefix, moves 32 bits from a 32-bit register to memory using a 16-bit effective address.
 - If preceded by an address-size prefix, moves 16 bits from a 16-bit register to memory using a 32-bit effective address.
 - If preceded by both an address-size prefix and an operand-size prefix, moves 32 bits from a 32-bit register to memory using a 32-bit effective address.

The previous examples show that any instruction can generate any combination of operand size and address size regardless of whether the instruction is in a 16- or 32-bit segment. The choice of the 16- or 32-bit default for a code segment is normally based on the following criteria:

- **Performance**—Always use 32-bit code segments when possible. They run much faster than 16-bit code segments on P6 family processors, and somewhat faster on earlier IA-32 processors.
- **The operating system the code segment will be running on**—If the operating system is a 16-bit operating system, it may not support 32-bit program modules.
- **Mode of operation**—If the code segment is being designed to run in real-address mode, virtual-8086 mode, or SMM, it must be a 16-bit code segment.
- **Backward compatibility to earlier IA-32 processors**—If a code segment must be able to run on an Intel 8086 or Intel 286 processor, it must be a 16-bit code segment.

17.3. SHARING DATA AMONG MIXED-SIZE CODE SEGMENTS

Data segments can be accessed from both 16-bit and 32-bit code segments. When a data segment that is larger than 64 KBytes is to be shared among 16- and 32-bit code segments, the data that is to be accessed from the 16-bit code segments must be located within the first 64 KBytes of the data segment. The reason for this is that 16-bit pointers by definition can only point to the first 64 KBytes of a segment.

A stack that spans less than 64 KBytes can be shared by both 16- and 32-bit code segments. This class of stacks includes:

- Stacks in expand-up segments with the G (granularity) and B (big) flags in the stack-segment descriptor clear.
- Stacks in expand-down segments with the G and B flags clear.
- Stacks in expand-up segments with the G flag set and the B flag clear and where the stack is contained completely within the lower 64 KBytes. (Offsets greater than FFFFH can be used for data, other than the stack, which is not shared.)

See Section 3.4.3., “Segment Descriptors”, for a description of the G and B flags and the expand-down stack type.

The B flag cannot, in general, be used to change the size of stack used by a 16-bit code segment. This flag controls the size of the stack pointer only for implicit stack references such as those caused by interrupts, exceptions, and the PUSH, POP, CALL, and RET instructions. It does not control explicit stack references, such as accesses to parameters or local variables. A 16-bit code segment can use a 32-bit stack only if the code is modified so that all explicit references to the stack are preceded by the 32-bit address-size prefix, causing those references to use 32-bit addressing and explicit writes to the stack pointer are preceded by a 32-bit operand-size prefix.

In 32-bit, expand-down segments, all offsets may be greater than 64 KBytes; therefore, 16-bit code cannot use this kind of stack segment unless the code segment is modified to use 32-bit addressing.

17.4. TRANSFERRING CONTROL AMONG MIXED-SIZE CODE SEGMENTS

There are three ways for a procedure in a 16-bit code segment to safely make a call to a 32-bit code segment:

- Make the call through a 32-bit call gate.
- Make a 16-bit call to a 32-bit interface procedure. The interface procedure then makes a 32-bit call to the intended destination.
- Modify the 16-bit procedure, inserting an operand-size prefix before the call, to change it to a 32-bit call.

Likewise, there are three ways for procedure in a 32-bit code segment to safely make a call to a 16-bit code segment:

- Make the call through a 16-bit call gate. Here, the EIP value at the CALL instruction cannot exceed FFFFH.
- Make a 32-bit call to a 16-bit interface procedure. The interface procedure then makes a 16-bit call to the intended destination.
- Modify the 32-bit procedure, inserting an operand-size prefix before the call, changing it to a 16-bit call. Be certain that the return offset does not exceed FFFFH.

These methods of transferring program control overcome the following architectural limitations imposed on calls between 16-bit and 32-bit code segments:

- Pointers from 16-bit code segments (which by default can only be 16-bits) cannot be used to address data or code located beyond FFFFH in a 32-bit segment.
- The operand-size attributes for a CALL and its companion RETURN instruction must be the same to maintain stack coherency. This is also true for implicit calls to interrupt and exception handlers and their companion IRET instructions.
- A 32-bit parameters (particularly a pointer parameter) greater than FFFFH cannot be squeezed into a 16-bit parameter location on a stack.
- The size of the stack pointer (SP or ESP) changes when switching between 16-bit and 32-bit code segments.

These limitations are discussed in greater detail in the following sections.

17.4.1. Code-Segment Pointer Size

For control-transfer instructions that use a pointer to identify the next instruction (that is, those that do not use gates), the operand-size attribute determines the size of the offset portion of the pointer. The implications of this rule are as follows:

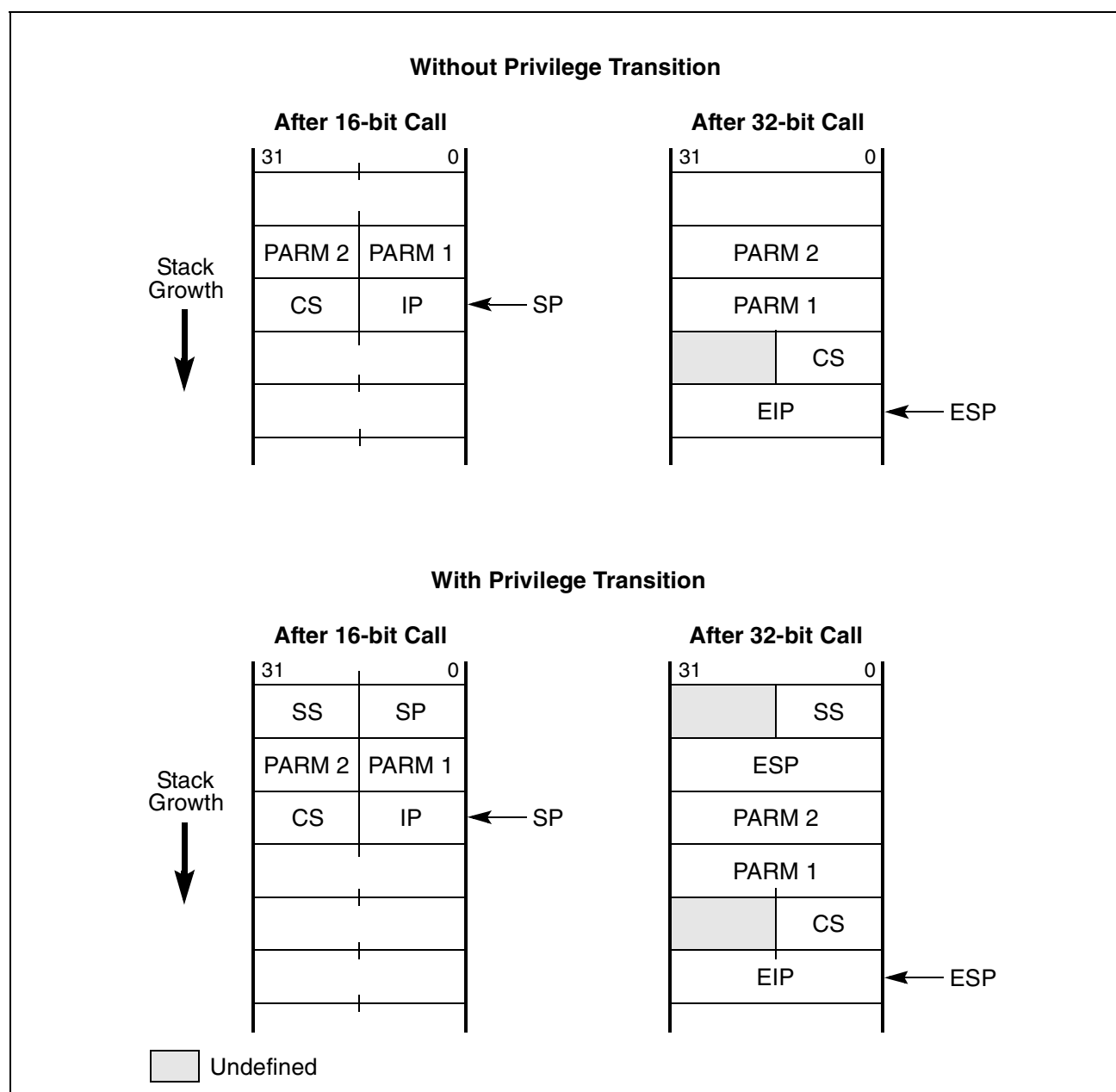
- A JMP, CALL, or RET instruction from a 32-bit segment to a 16-bit segment is always possible using a 32-bit operand size, providing the 32-bit pointer does not exceed FFFFH.
- A JMP, CALL, or RET instruction from a 16-bit segment to a 32-bit segment cannot address a destination greater than FFFFH, unless the instruction is given an operand-size prefix.

See Section 17.4.5., “Writing Interface Procedures”, for an interface procedure that can transfer program control from 16-bit segments to destinations in 32-bit segments beyond FFFFH.

17.4.2. Stack Management for Control Transfer

Because the stack is managed differently for 16-bit procedure calls than for 32-bit calls, the operand-size attribute of the RET instruction must match that of the CALL instruction (see Figure 17-1). On a 16-bit call, the processor pushes the contents of the 16-bit IP register and (for calls between privilege levels) the 16-bit SP register. The matching RET instruction must also use a 16-bit operand size to pop these 16-bit values from the stack into the 16-bit registers.

A 32-bit CALL instruction pushes the contents of the 32-bit EIP register and (for inter-privilege-level calls) the 32-bit ESP register. Here, the matching RET instruction must use a 32-bit operand size to pop these 32-bit values from the stack into the 32-bit registers. If the two parts of a CALL/RET instruction pair do not have matching operand sizes, the stack will not be managed correctly and the values of the instruction pointer and stack pointer will not be restored to correct values.

**Figure 17-1. Stack after Far 16- and 32-Bit Calls**

While executing 32-bit code, if a call is made to a 16-bit code segment which is at the same or a more privileged level (that is, the DPL of the called code segment is less than or equal to the CPL of the calling code segment) through a 16-bit call gate, then the upper 16-bits of the ESP register may be unreliable upon returning to the 32-bit code segment (that is, after executing a RET in the 16-bit code segment).

When the CALL instruction and its matching RET instruction are in code segments that have D flags with the same values (that is, both are 32-bit code segments or both are 16-bit code segments), the default settings may be used. When the CALL instruction and its matching RET instruction are in segments which have different D-flag settings, an operand-size prefix must be used.

17.4.2.1. CONTROLLING THE OPERAND-SIZE ATTRIBUTE FOR A CALL

Three things can determine the operand-size of a call:

- The D flag in the segment descriptor for the calling code segment.
- An operand-size instruction prefix.
- The type of call gate (16-bit or 32-bit), if a call is made through a call gate.

When a call is made with a pointer (rather than a call gate), the D flag for the calling code segment determines the operand-size for the CALL instruction. This operand-size attribute can be overridden by prepending an operand-size prefix to the CALL instruction. So, for example, if the D flag for a code segment is set for 16 bits and the operand-size prefix is used with a CALL instruction, the processor will cause the information stored on the stack to be stored in 32-bit format. If the call is to a 32-bit code segment, the instructions in that code segment will be able to read the stack coherently. Also, a RET instruction from the 32-bit code segment without an operand-size prefix will maintain stack coherency with the 16-bit code segment being returned to.

When a CALL instruction references a call-gate descriptor, the type of call is determined by the type of call gate (16-bit or 32-bit). The offset to the destination in the code segment being called is taken from the gate descriptor; therefore, if a 32-bit call gate is used, a procedure in a 16-bit code segment can call a procedure located more than 64 Kbytes from the base of a 32-bit code segment, because a 32-bit call gate uses a 32-bit offset.

Note that regardless of the operand size of the call and how it is determined, the size of the stack pointer used (SP or ESP) is always controlled by the B flag in the stack-segment descriptor currently in use (that is, when B is clear, SP is used, and when B is set, ESP is used).

An unmodified 16-bit code segment that has run successfully on an 8086 processor or in real-mode on a later IA-32 architecture processor will have its D flag clear and will not use operand-size override prefixes. As a result, all CALL instructions in this code segment will use the 16-bit operand-size attribute. Procedures in these code segments can be modified to safely call procedures to 32-bit code segments in either of two ways:

- Relink the CALL instruction to point to 32-bit call gates (see Section 17.4.2.2., “Passing Parameters With a Gate”).
- Add a 32-bit operand-size prefix to each CALL instruction.

17.4.2.2. PASSING PARAMETERS WITH A GATE

When referencing 32-bit gates with 16-bit procedures, it is important to consider the number of parameters passed in each procedure call. The count field of the gate descriptor specifies the size of the parameter string to copy from the current stack to the stack of a more privileged (numerically lower privilege level) procedure. The count field of a 16-bit gate specifies the number of 16-bit words to be copied, whereas the count field of a 32-bit gate specifies the number of 32-bit doublewords to be copied. The count field for a 32-bit gate must thus be half the size of the number of words being placed on the stack by a 16-bit procedure. Also, the 16-bit procedure must use an even number of words as parameters.

17.4.3. Interrupt Control Transfers

A program-control transfer caused by an exception or interrupt is always carried out through an interrupt or trap gate (located in the IDT). Here, the type of the gate (16-bit or 32-bit) determines the operand-size attribute used in the implicit call to the exception or interrupt handler procedure in another code segment.

A 32-bit interrupt or trap gate provides a safe interface to a 32-bit exception or interrupt handler when the exception or interrupt occurs in either a 32-bit or a 16-bit code segment. It is sometimes impractical, however, to place exception or interrupt handlers in 16-bit code segments, because only 16-bit return addresses are saved on the stack. If an exception or interrupt occurs in a 32-bit code segment when the EIP was greater than FFFFH, the 16-bit handler procedure cannot provide the correct return address.

17.4.4. Parameter Translation

When segment offsets or pointers (which contain segment offsets) are passed as parameters between 16-bit and 32-bit procedures, some translation is required. If a 32-bit procedure passes a pointer to data located beyond 64 KBytes to a 16-bit procedure, the 16-bit procedure cannot use it. Except for this limitation, interface code can perform any format conversion between 32-bit and 16-bit pointers that may be needed.

Parameters passed by value between 32-bit and 16-bit code also may require translation between 32-bit and 16-bit formats. The form of the translation is application-dependent.

17.4.5. Writing Interface Procedures

Placing interface code between 32-bit and 16-bit procedures can be the solution to the following interface problems:

- Allowing procedures in 16-bit code segments to call procedures with offsets greater than FFFFH in 32-bit code segments.
- Matching operand-size attributes between companion CALL and RET instructions.
- Translating parameters (data), including managing parameter strings with a variable count or an odd number of 16-bit words.
- The possible invalidation of the upper bits of the ESP register.

The interface procedure is simplified where these rules are followed.

1. The interface procedure must reside in a 32-bit code segment (the D flag for the code-segment descriptor is set).
2. All procedures that may be called by 16-bit procedures must have offsets not greater than FFFFH.
3. All return addresses saved by 16-bit procedures must have offsets not greater than FFFFH.

The interface procedure becomes more complex if any of these rules are violated. For example, if a 16-bit procedure calls a 32-bit procedure with an entry point beyond FFFFH, the interface procedure will need to provide the offset to the entry point. The mapping between 16- and 32-bit addresses is only performed automatically when a call gate is used, because the gate descriptor for a call gate contains a 32-bit address. When a call gate is not used, the interface code must provide the 32-bit address.

The structure of the interface procedure depends on the types of calls it is going to support, as follows:

- **Calls from 16-bit procedures to 32-bit procedures.** Calls to the interface procedure from a 16-bit code segment are made with 16-bit CALL instructions (by default, because the D flag for the calling code-segment descriptor is clear), and 16-bit operand-size prefixes are used with RET instructions to return from the interface procedure to the calling procedure. Calls from the interface procedure to 32-bit procedures are performed with 32-bit CALL instructions (by default, because the D flag for the interface procedure's code segment is set), and returns from the called procedures to the interface procedure are performed with 32-bit RET instructions (also by default).
- **Calls from 32-bit procedures to 16-bit procedures.** Calls to the interface procedure from a 32-bit code segment are made with 32-bit CALL instructions (by default), and returns to the calling procedure from the interface procedure are made with 32-bit RET instructions (also by default). Calls from the interface procedure to 16-bit procedures require the CALL instructions to have the operand-size prefixes, and returns from the called procedures to the interface procedure are performed with 16-bit RET instructions (by default).

18

Intel Architecture Compatibility

CHAPTER 18

IA-32 COMPATIBILITY

All IA-32 processors are binary compatible. Compatibility means that, within certain limited constraints, programs that execute on previous generations of IA-32 processors will produce identical results when executed on later IA-32 processors. The compatibility constraints and any implementation differences between the IA-32 processors are described in this chapter.

Each new IA-32 processor has enhanced the software visible architecture from that found in earlier IA-32 processors. Those enhancements have been defined with consideration for compatibility with previous and future processors. This chapter also summarizes the compatibility considerations for those extensions.

18.1. IA-32 PROCESSOR FAMILIES AND CATEGORIES

IA-32 processors are referred to in several different ways in this chapter, depending on the type of compatibility information being related, as described in the following:

- **IA-32 Processors**—All the Intel processors based on the Intel IA-32 Architecture, which include the 8086/88, Intel 286, Intel386, Intel486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium 4, and Intel Xeon processors.
- **32-bit Processors**—All the IA-32 processors that use a 32-bit architecture, which include the Intel386, Intel486, Pentium, Pentium Pro, Pentium II, Pentium III, Pentium 4, and Intel Xeon processors.
- **16-bit Processors**—All the IA-32 processors that use a 16-bit architecture, which include the 8086/88 and Intel 286 processors.
- **P6 Family Processors**—All the IA-32 processors that are based on the P6 micro-architecture, which include the Pentium Pro, Pentium II, and Pentium III processors.
- **Pentium 4 Family Processors**—A family of IA-32 processors that is based on the Intel NetBurst micro-architecture.
- **Intel Xeon Family Processors**—A family of IA-32 processors that is based on the Intel NetBurst micro-architecture. This family includes the Intel Xeon processor and the Intel Xeon processor MP.

18.2. RESERVED BITS

Throughout this manual, certain bits are marked as reserved in many register and memory layout descriptions. When bits are marked as undefined or reserved, it is essential for compatibility with future processors that software treat these bits as having a future, though unknown effect. Software should follow these guidelines in dealing with reserved bits:

- Do not depend on the states of any reserved bits when testing the values of registers or memory locations that contain such bits. Mask out the reserved bits before testing.
- Do not depend on the states of any reserved bits when storing them to memory or to a register.
- Do not depend on the ability to retain information written into any reserved bits.
- When loading a register, always load the reserved bits with the values indicated in the documentation, if any, or reload them with values previously read from the same register.

Software written for existing IA-32 processor that handles reserved bits correctly will port to future IA-32 processors without generating protection exceptions.

18.3. ENABLING NEW FUNCTIONS AND MODES

Most of the new control functions defined for the P6 family and Pentium processors are enabled by new mode flags in the control registers (primarily register CR4). This register is undefined for IA-32 processors earlier than the Pentium processor. Attempting to access this register with an Intel486 or earlier IA-32 processor results in an invalid-opcode exception (#UD). Consequently, programs that execute correctly on the Intel486 or earlier IA-32 processor cannot erroneously enable these functions. Attempting to set a reserved bit in register CR4 to a value other than its original value results in a general-protection exception (#GP). So, programs that execute on the P6 family and Pentium processors cannot erroneously enable functions that may be implemented in future IA-32 processors.

The P6 family and Pentium processors do not check for attempts to set reserved bits in model-specific registers. It is the obligation of the software writer to enforce this discipline. These reserved bits may be used in future Intel processors.

18.4. DETECTING THE PRESENCE OF NEW FEATURES THROUGH SOFTWARE

Software can check for the presence of new architectural features and extensions in either of two ways:

- Test for the presence of the feature or extension — Software can test for the presence of new flags in the EFLAGS register and control registers. If these flags are reserved (meaning not present in the processor executing the test), an exception is generated. Likewise, software can attempt to execute a new instruction, which results in an invalid-opcode exception (#UD) being generated if it is not supported.

- Execute the CUID instruction — The CUID instruction (added to the IA-32 in the Pentium processor) indicates the presence of new features directly.

See Chapter 13, *Processor Identification and Feature Determination*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for detailed information on detecting new processor features and extensions.

18.5. INTEL MMX TECHNOLOGY

The Pentium processor with MMX technology introduced the MMX technology and a set of MMX instructions to the IA-32. The MMX instructions are summarized in Chapter 5, *Instruction Set Summary*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1* and are described in detail in Chapter 3 in the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*. The MMX technology and MMX instructions are also included in the Pentium II, Pentium III, Pentium 4, and Intel Xeon processors.

18.6. STREAMING SIMD EXTENSIONS (SSE)

The Streaming SIMD Extensions (SSE) were introduced in the Pentium III processor. The SSE extensions consist of a new set of instructions and a new set of registers. The new register include the eight 128-bit XMM registers and the 32-bit MXCSR control and status register. These instructions and registers are designed to allow SIMD computations to be made on single-precision floating-point numbers. Several of these new instructions also operate in the MMX registers. The SSE instructions and registers are described in Chapter 10, *Programming with the Streaming SIMD Extensions (SSE)*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1* and in Chapter 3, *Instruction Set Reference*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*.

18.7. STREAMING SIMD EXTENSIONS 2 (SSE2)

The Streaming SIMD Extensions 2 (SSE2) were introduced in the Pentium 4 and Intel Xeon processors. They consist of a new set of instructions that operate on the XMM and MXCSR registers and perform SIMD operations on double-precision floating-point values and on integer values. Several of these new instructions also operate in the MMX registers. The SSE instructions and registers are described in Chapter 11, *Programming with the Streaming SIMD Extensions 2 (SSE2)*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1* and in Chapter 3, *Instruction Set Reference*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 2*.

18.8. HYPER-THREADING TECHNOLOGY

Hyper-Threading Technology is an extension to the IA-32 architecture that enables a single physical processor to execute two or more separate code streams (called *threads*) concurrently. This capability was introduced into the IA-32 architecture in the Intel Xeon processor MP and in later steppings of the Intel Xeon processor. Both of these processors implement Hyper-Threading Technology with two logical processors per physical processor. See Section 7.6., “Hyper-Threading Technology”, for a complete description of the Hyper-Threading Technology architecture.

18.9. NEW INSTRUCTIONS IN THE PENTIUM AND LATER IA-32 PROCESSORS

Table 18-1 identifies the instructions introduced into the IA-32 in the Pentium processor and later IA-32 processors.

18.9.1. Instructions Added Prior to the Pentium Processor

The following instructions were added in the Intel486 processor:

- BSWAP (byte swap) instruction.
- XADD (exchange and add) instruction.
- CMPXCHG (compare and exchange) instruction.
- INVD (invalidate cache) instruction.
- WBINVD (write-back and invalidate cache) instruction.
- INVLPG (invalidate TLB entry) instruction.

Table 18-1. New Instruction in the Pentium Processor and Later IA-32 Processors

Instruction	CPUID Identification Bits	Introduced In
CMOV _{cc} (conditional move)	EDX, Bit 15	Pentium Pro processor
FCMOV _{cc} (floating-point conditional move)	EDX, Bits 0 and 15	
FCOMI (floating-point compare and set EFLAGS)	EDX, Bits 0 and 15	
RDPMC (read performance monitoring counters)	EAX, Bits 8-11, set to 6H; see Note 1	
UD2 (undefined)	EAX, Bits 8-11, set to 6H	

Table 18-1. New Instruction in the Pentium Processor and Later IA-32 Processors (Contd.)

Instruction	CPUID Identification Bits	Introduced In
CMPXCHG8B (compare and exchange 8 bytes)	EDX, Bit 8	Pentium processor
CPUID (CPU identification)	None; see Note 2	
RDTSC (read time-stamp counter)	EDX, Bit 4	
RDMSR (read model-specific register)	EDX, Bit 5	
WRMSR (write model-specific register)	EDX, Bit 5	
MMX Instructions	EDX, Bit 23	

Notes:

1. The RDPNC instruction was introduced in the P6 family of processors and added to later model Pentium processors. This instruction is model specific in nature and not architectural.
2. The CPUID instruction is available in all Pentium and P6 family processors and in later models of the Intel486 processors. The ability to set and clear the ID flag (bit 21) in the EFLAGS register indicates the availability of the CPUID instruction.

The following instructions were added in the Intel386 processor:

- LSS, LFS, and LGS (load SS, FS, and GS registers).
- Long-displacement conditional jumps.
- Single-bit instructions.
- Bit scan instructions.
- Double-shift instructions.
- Byte set on condition instruction.
- Move with sign/zero extension.
- Generalized multiply instruction.
- MOV to and from control registers.
- MOV to and from test registers (now obsolete).
- MOV to and from debug registers.
- RSM (resume from SMM). This instruction was introduced in the Intel386 SL and Intel486 SL processors.

The following instructions were added in the Intel 387 math coprocessor:

- FPREM1.
- FUCOM, FUCOMP, and FUCOMPP.

18.10. OBSOLETE INSTRUCTIONS

The MOV to and from test registers instructions were removed from the Pentium processor and future IA-32 processors. Execution of these instructions generates an invalid-opcode exception (#UD).

18.11. UNDEFINED OPCODES

All new instructions defined for IA-32 processors use binary encodings that were reserved on earlier-generation processors. Attempting to execute a reserved opcode always results in an invalid-opcode (#UD) exception being generated. Consequently, programs that execute correctly on earlier-generation processors cannot erroneously execute these instructions and thereby produce unexpected results when executed on later IA-32 processors.

18.12. NEW FLAGS IN THE EFLAGS REGISTER

The section titled “EFLAGS Register” in Chapter 3 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*, shows the configuration of flags in the EFLAGS register for the P6 family processors. No new flags have been added to this register in the P6 family processors. The flags added to this register in the Pentium and Intel486 processors are described in the following sections.

The following flags were added to the EFLAGS register in the Pentium processor:

- VIF (virtual interrupt flag), bit 19.
- VIP (virtual interrupt pending), bit 20.
- ID (identification flag), bit 21.

The AC flag (bit 18) was added to the EFLAGS register in the Intel486 processor.

18.12.1. Using EFLAGS Flags to Distinguish Between 32-Bit IA-32 Processors

The following bits in the EFLAGS register that can be used to differentiate between the 32-bit IA-32 processors:

- Bit 18 (the AC flag) can be used to distinguish an Intel386 processor from the P6 family, Pentium, and Intel486 processors. Since it is not implemented on the Intel386 processor, it will always be clear.
- Bit 21 (the ID flag) indicates whether an application can execute the CPUID instruction. The ability to set and clear this bit indicates that the processor is a P6 family or Pentium processor. The CPUID instruction can then be used to determine which processor.

- Bits 19 (the VIF flag) and 20 (the VIP flag) will always be zero on processors that do not support virtual mode extensions, which includes all 32-bit processors prior to the Pentium processor.

See Chapter 13, *Processor Identification and Feature Determination*, in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for more information on identifying processors.

18.13. STACK OPERATIONS

This section identifies the differences in stack implementation between the various IA-32 processors.

18.13.1. PUSH SP

The P6 family, Pentium, Intel486, Intel386, and Intel 286 processors push a different value on the stack for a PUSH SP instruction than the 8086 processor. The 32-bit processors push the value of the SP register before it is decremented as part of the push operation; the 8086 processor pushes the value of the SP register after it is decremented. If the value pushed is important, replace PUSH SP instructions with the following three instructions:

```
PUSH BP
MOV BP, SP
XCHG BP, [BP]
```

This code functions as the 8086 processor PUSH SP instruction on the P6 family, Pentium, Intel486, Intel386, and Intel 286 processors.

18.13.2. EFLAGS Pushed on the Stack

The setting of the stored values of bits 12 through 15 (which includes the IOPL field and the NT flag) in the EFLAGS register by the PUSHF instruction, by interrupts, and by exceptions is different with the 32-bit IA-32 processors than with the 8086 and Intel 286 processors. The differences are as follows:

- 8086 processor—bits 12 through 15 are always set.
- Intel 286 processor—bits 12 through 15 are always cleared in real-address mode.
- 32-bit processors in real-address mode—bit 15 (reserved) is always cleared, and bits 12 through 14 have the last value loaded into them.

18.14. X87 FPU

This section addresses the issues that must be faced when porting floating-point software designed to run on earlier IA-32 processors and math coprocessors to a Pentium 4, Intel Xeon, P6 family, or Pentium processor with integrated x87 FPU. To software, a Pentium 4, Intel Xeon, or P6 family processor looks very much like a Pentium processor. Floating-point software which runs on a Pentium or Intel486 DX processor, or on an Intel486 SX processor/Intel 487 SX math coprocessor system or an Intel386 processor/Intel 387 math coprocessor system, will run with at most minor modifications on a Pentium 4, Intel Xeon, or P6 family processor. To port code directly from an Intel 286 processor/Intel 287 math coprocessor system or an Intel 8086 processor/8087 math coprocessor system to a Pentium 4, Intel Xeon, P6 family, or Pentium processor, certain additional issues must be addressed.

In the following sections, the term “32-bit x87 FPUs” refers to the P6 family, Pentium, and Intel486 DX processors, and to the Intel 487 SX and Intel 387 math coprocessors; the term “16-bit IA-32 math coprocessors” refers to the Intel 287 and 8087 math coprocessors.

18.14.1. Control Register CR0 Flags

The ET, NE, and MP flags in control register CR0 control the interface between the integer unit of an IA-32 processor and either its internal x87 FPU or an external math coprocessor. The effect of these flags in the various IA-32 processors are described in the following paragraphs.

The ET (extension type) flag (bit 4 of the CR0 register) is used in the Intel386 processor to indicate whether the math coprocessor in the system is an Intel 287 math coprocessor (flag is clear) or an Intel 387 DX math coprocessor (flag is set). This bit is hardwired to 1 in the P6 family, Pentium, and Intel486 processors.

The NE (Numeric Exception) flag (bit 5 of the CR0 register) is used in the P6 family, Pentium, and Intel486 processors to determine whether unmasked floating-point exceptions are reported internally through interrupt vector 16 (flag is set) or externally through an external interrupt (flag is clear). On a hardware reset, the NE flag is initialized to 0, so software using the automatic internal error-reporting mechanism must set this flag to 1. This flag is nonexistent on the Intel386 processor.

As on the Intel 286 and Intel386 processors, the MP (monitor coprocessor) flag (bit 1 of register CR0) determines whether the WAIT/FWAIT instructions or waiting-type floating-point instructions trap when the context of the x87 FPU is different from that of the currently-executing task. If the MP and TS flag are set, then a WAIT/FWAIT instruction and waiting instructions will cause a device-not-available exception (interrupt vector 7). The MP flag is used on the Intel 286 and Intel386 processors to support the use of a WAIT/FWAIT instruction to wait on a device other than a math coprocessor. The device reports its status through the BUSY# pin. Since the P6 family, Pentium, and Intel486 processors do not have such a pin, the MP flag has no relevant use and should be set to 1 for normal operation.

18.14.2. x87 FPU Status Word

This section identifies differences to the x87 FPU status word for the different IA-32 processors and math coprocessors, the reason for the differences, and their impact on software.

18.14.2.1. CONDITION CODE FLAGS (C0 THROUGH C3)

The following information pertains to differences in the use of the condition code flags (C0 through C3) located in bits 8, 9, 10, and 14 of the x87 FPU status word.

After execution of an FINIT instruction or a hardware reset on a 32-bit x87 FPU, the condition code flags are set to 0. The same operations on a 16-bit IA-32 math coprocessor leave these flags intact (they contain their prior value). This difference in operation has no impact on software and provides a consistent state after reset.

Transcendental instruction results in the core range of the P6 family and Pentium processors may differ from the Intel486 DX processor and Intel 487 SX math coprocessor by 2 to 3 units in the last place (ulps)—(see “Transcendental Instruction Accuracy” in Chapter 8 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*). As a result, the value saved in the C1 flag may also differ.

After an incomplete FPREM/FPREM1 instruction, the C0, C1, and C3 flags are set to 0 on the 32-bit x87 FPU. After the same operation on a 16-bit IA-32 math coprocessor, these flags are left intact.

On the 32-bit x87 FPU, the C2 flag serves as an incomplete flag for the FTAN instruction. On the 16-bit IA-32 math coprocessors, the C2 flag is undefined for the FPTAN instruction. This difference has no impact on software, because Intel 287 or 8087 programs do not check C2 after an FPTAN instruction. The use of this flag on later processors allows fast checking of operand range.

18.14.2.2. STACK FAULT FLAG

When unmasked stack overflow or underflow occurs on a 32-bit x87 FPU, the IE flag (bit 0) and the SF flag (bit 6) of the x87 FPU status word are set to indicate a stack fault and condition code flag C1 is set or cleared to indicate overflow or underflow, respectively. When unmasked stack overflow or underflow occurs on a 16-bit IA-32 math coprocessor, only the IE flag is set. Bit 6 is reserved on these processors. The addition of the SF flag on a 32-bit x87 FPU has no impact on software. Existing exception handlers need not change, but may be upgraded to take advantage of the additional information.

18.14.3. x87 FPU Control Word

Only affine closure is supported for infinity control on a 32-bit x87 FPU. The infinity control flag (bit 12 of the x87 FPU control word) remains programmable on these processors, but has no effect. This change was made to conform to the IEEE Standard 754 for Binary Floating-Point Arithmetic. On a 16-bit IA-32 math coprocessor, both affine and projective closures are

supported, as determined by the setting of bit 12. After a hardware reset, the default value of bit 12 is projective. Software that requires projective infinity arithmetic may give different results.

18.14.4. x87 FPU Tag Word

When loading the tag word of a 32-bit x87 FPU, using an `FLDENV`, `FRSTOR`, or `FXRSTOR` (Pentium III processor only) instruction, the processor examines the incoming tag and classifies the location only as empty or non-empty. Thus, tag values of 00, 01, and 10 are interpreted by the processor to indicate a non-empty location. The tag value of 11 is interpreted by the processor to indicate an empty location. Subsequent operations on a non-empty register always examine the value in the register, not the value in its tag. The `FSTENV`, `FSAVE`, and `FXSAVE` (Pentium III processor only) instructions examine the non-empty registers and put the correct values in the tags before storing the tag word.

The corresponding tag for a 16-bit IA-32 math coprocessor is checked before each register access to determine the class of operand in the register; the tag is updated after every change to a register so that the tag always reflects the most recent status of the register. Software can load a tag with a value that disagrees with the contents of a register (for example, the register contains a valid value, but the tag says special). Here, the 16-bit IA-32 math coprocessors honor the tag and do not examine the register.

Software written to run on a 16-bit IA-32 math coprocessor may not operate correctly on a 16-bit x87 FPU, if it uses the `FLDENV`, `FRSTOR`, or `FXRSTOR` instructions to change tags to values (other than to empty) that are different from actual register contents.

The encoding in the tag word for the 32-bit x87 FPUs for unsupported data formats (including pseudo-zero and unnormal) is special (10B), to comply with IEEE Standard 754. The encoding in the 16-bit IA-32 math coprocessors for pseudo-zero and unnormal is valid (00B) and the encoding for other unsupported data formats is special (10B). Code that recognizes the pseudo-zero or unnormal format as valid must therefore be changed if it is ported to a 32-bit x87 FPU.

18.14.5. Data Types

This section discusses the differences of data types for the various x87 FPUs and math coprocessors.

18.14.5.1. NaNs

The 32-bit x87 FPUs distinguish between signaling NaNs (SNaNs) and quiet NaNs (QNaNs). These x87 FPUs only generate QNaNs and normally do not generate an exception upon encountering a QNaN. An invalid-operation exception (`#I`) is generated only upon encountering a SNaN, except for the `FCOM`, `FIST`, and `FBSTP` instructions, which also generates an invalid-operation exceptions for a QNaNs. This behavior matches IEEE Standard 754.

The 16-bit IA-32 math coprocessors only generate one kind of NaN (the equivalent of a QNaN), but the raise an invalid-operation exception upon encountering any kind of NaN.

When porting software written to run on a 16-bit IA-32 math coprocessor to a 32-bit x87 FPU, uninitialized memory locations that contain QNaNs should be changed to SNaNs to cause the x87 FPU or math coprocessor to fault when uninitialized memory locations are referenced.

18.14.5.2. PSEUDO-ZERO, PSEUDO-NaN, PSEUDO-INFINITY, AND UNNORMAL FORMATS

The 32-bit x87 FPUs neither generate nor support the pseudo-zero, pseudo-NaN, pseudo-infinity, and unnormal formats. Whenever they encounter them in an arithmetic operation, they raise an invalid-operation exception. The 16-bit IA-32 math coprocessors define and support special handling for these formats. Support for these formats was dropped to conform with IEEE Standard 754 for Binary Floating-Point Arithmetic.

This change should not impact software ported from 16-bit IA-32 math coprocessors to 32-bit x87 FPUs. The 32-bit x87 FPUs do not generate these formats, and therefore will not encounter them unless software explicitly loads them in the data registers. The only affect may be in how software handles the tags in the tag word (see Section 18.14.4., “x87 FPU Tag Word”).

18.14.6. Floating-Point Exceptions

This section identifies the implementation differences in exception handling for floating-point instructions in the various x87 FPUs and math coprocessors.

18.14.6.1. DENORMAL OPERAND EXCEPTION (#D)

When the denormal operand exception is masked, the 32-bit x87 FPUs automatically normalize denormalized numbers when possible; whereas, the 16-bit IA-32 math coprocessors return a denormal result. A program written to run on a 16-bit IA-32 math coprocessor that uses the denormal exception solely to normalize denormalized operands is redundant when run on the 32-bit x87 FPUs. If such a program is run on 32-bit x87 FPUs, performance can be improved by masking the denormal exception. Floating-point programs run faster when the FPU performs normalization of denormalized operands.

The denormal operand exception is not raised for transcendental instructions and the FXTRACT instruction on the 16-bit IA-32 math coprocessors. This exception is raised for these instructions on the 32-bit x87 FPUs. The exception handlers ported to these latter processors need to be changed only if the handlers gives special treatment to different opcodes.

18.14.6.2. NUMERIC OVERFLOW EXCEPTION (#O)

On the 32-bit x87 FPUs, when the numeric overflow exception is masked and the rounding mode is set to chop (toward 0), the result is the largest positive or smallest negative number. The 16-bit IA-32 math coprocessors do not signal the overflow exception when the masked response is not ∞ ; that is, they signal overflow only when the rounding control is not set to round to 0. If rounding is set to chop (toward 0), the result is positive or negative ∞ . Under the most common rounding modes, this difference has no impact on existing software.

If rounding is toward 0 (chop), a program on a 32-bit x87 FPU produces, under overflow conditions, a result that is different in the least significant bit of the significand, compared to the result on a 16-bit IA-32 math coprocessor. The reason for this difference is IEEE Standard 754 compatibility.

When the overflow exception is not masked, the precision exception is flagged on the 32-bit x87 FPUs. When the result is stored in the stack, the significand is rounded according to the precision control (PC) field of the FPU control word or according to the opcode. On the 16-bit IA-32 math coprocessors, the precision exception is not flagged and the significand is not rounded. The impact on existing software is that if the result is stored on the stack, a program running on a 32-bit x87 FPU produces a different result under overflow conditions than on a 16-bit IA-32 math coprocessor. The difference is apparent only to the exception handler. This difference is for IEEE Standard 754 compatibility.

18.14.6.3. NUMERIC UNDERFLOW EXCEPTION (#U)

When the underflow exception is masked on the 32-bit x87 FPUs, the underflow exception is signaled when both the result is tiny and denormalization results in a loss of accuracy. When the underflow exception is unmasked and the instruction is supposed to store the result on the stack, the significand is rounded to the appropriate precision (according to the PC flag in the FPU control word, for those instructions controlled by PC, otherwise to extended precision), after adjusting the exponent.

When the underflow exception is masked on the 16-bit IA-32 math coprocessors and rounding is toward 0, the underflow exception flag is raised on a tiny result, regardless of loss of accuracy. When the underflow exception is not masked and the destination is the stack, the significand is not rounded, but instead is left as is.

When the underflow exception is masked, this difference has no impact on existing software. The underflow exception occurs less often when rounding is toward 0.

When the underflow exception not masked. A program running on a 32-bit x87 FPU produces a different result during underflow conditions than on a 16-bit IA-32 math coprocessor if the result is stored on the stack. The difference is only in the least significant bit of the significand and is apparent only to the exception handler.

18.14.6.4. EXCEPTION PRECEDENCE

There is no difference in the precedence of the denormal-operand exception on the 32-bit x87 FPUs, whether it be masked or not. When the denormal-operand exception is not masked on the 16-bit IA-32 math coprocessors, it takes precedence over all other exceptions. This difference causes no impact on existing software, but some unneeded normalization of denormalized operands is prevented on the Intel486 processor and Intel 387 math coprocessor.

18.14.6.5. CS AND EIP FOR FPU EXCEPTIONS

On the Intel 32-bit x87 FPUs, the values from the CS and EIP registers saved for floating-point exceptions point to any prefixes that come before the floating-point instruction. On the 8087 math coprocessor, the saved CS and IP registers points to the floating-point instruction.

18.14.6.6. FPU ERROR SIGNALS

The floating-point error signals to the P6 family, Pentium, and Intel486 processors do not pass through an interrupt controller; an INT# signal from an Intel 387, Intel 287 or 8087 math coprocessors does. If an 8086 processor uses another exception for the 8087 interrupt, both exception vectors should call the floating-point-error exception handler. Some instructions in a floating-point-error exception handler may need to be deleted if they use the interrupt controller. The P6 family, Pentium, and Intel486 processors have signals that, with the addition of external logic, support reporting for emulation of the interrupt mechanism used in many personal computers.

On the P6 family, Pentium, and Intel486 processors, an undefined floating-point opcode will cause an invalid-opcode exception (#UD, interrupt vector 6). Undefined floating-point opcodes, like legal floating-point opcodes, cause a device not available exception (#NM, interrupt vector 7) when either the TS or EM flag in control register CR0 is set. The P6 family, Pentium, and Intel486 processors do not check for floating-point error conditions on encountering an undefined floating-point opcode.

18.14.6.7. ASSERTION OF THE FERR# PIN

When using the MS-DOS compatibility mode for handing floating-point exceptions, the FERR# pin must be connected to an input to an external interrupt controller. An external interrupt is then generated when the FERR# output drives the input to the interrupt controller and the interrupt controller in turn drives the INTR pin on the processor.

For the P6 family and Intel386 processors, an unmasked floating-point exception always causes the FERR# pin to be asserted upon completion of the instruction that caused the exception. For the Pentium and Intel486 processors, an unmasked floating-point exception may cause the FERR# pin to be asserted either at the end of the instruction causing the exception or immediately before execution of the next floating-point instruction. (Note that the next floating-point instruction would not be executed until the pending unmasked exception has been handled.) See Appendix D in the *IA-32 Intel Architecture Software Developer's Manual, Volume 1*, for a complete description of the required mechanism for handling floating-point exceptions using the MS-DOS compatibility mode.

18.14.6.8. INVALID OPERATION EXCEPTION ON DENORMALS

An invalid-operation exception is not generated on the 32-bit x87 FPUs upon encountering a denormal value when executing a FSQRT, FDIV, or FPREM instruction or upon conversion to BCD or to integer. The operation proceeds by first normalizing the value. On the 16-bit IA-32 math coprocessors, upon encountering this situation, the invalid-operation exception is generated. This difference has no impact on existing software. Software running on the 32-bit x87

FPU continues to execute in cases where the 16-bit IA-32 math coprocessors trap. The reason for this change was to eliminate an exception from being raised.

18.14.6.9. ALIGNMENT CHECK EXCEPTIONS (#AC)

If alignment checking is enabled, a misaligned data operand on the P6 family, Pentium, and Intel486 processors causes an alignment check exception (#AC) when a program or procedure is running at privilege-level 3, except for the stack portion of the FSAVE/FNSAVE, FXSAVE, FRSTOR, and FXRSTOR instructions.

18.14.6.10. SEGMENT NOT PRESENT EXCEPTION DURING FLDENV

On the Intel486 processor, when a segment not present exception (#NP) occurs in the middle of an FLDENV instruction, it can happen that part of the environment is loaded and part not. In such cases, the FPU control word is left with a value of 007FH. The P6 family and Pentium processors ensure the internal state is correct at all times by attempting to read the first and last bytes of the environment before updating the internal state.

18.14.6.11. DEVICE NOT AVAILABLE EXCEPTION (#NM)

The device-not-available exception (#NM, interrupt 7) will occur in the P6 family, Pentium, and Intel486 processors as described in Section 2.5., “Control Registers”, Table 2-1, and Chapter 5, “Interrupt 7—Device Not Available Exception (#NM)”.

18.14.6.12. COPROCESSOR SEGMENT OVERRUN EXCEPTION

The coprocessor segment overrun exception (interrupt 9) does not occur in the P6 family, Pentium, and Intel486 processors. In situations where the Intel 387 math coprocessor would cause an interrupt 9, the P6 family, Pentium, and Intel486 processors simply abort the instruction. To avoid undetected segment overruns, it is recommended that the floating-point save area be placed in the same page as the TSS. This placement will prevent the FPU environment from being lost if a page fault occurs during the execution of an FLDENV, FRSTOR, or FXRSTOR instruction while the operating system is performing a task switch.

18.14.6.13. GENERAL PROTECTION EXCEPTION (#GP)

A general-protection exception (#GP, interrupt 13) occurs if the starting address of a floating-point operand falls outside a segment's size. An exception handler should be included to report these programming errors.

18.14.6.14. FLOATING-POINT ERROR EXCEPTION (#MF)

In real mode and protected mode (not including virtual-8086 mode), interrupt vector 16 must point to the floating-point exception handler. In virtual 8086 mode, the virtual-8086 monitor can be programmed to accommodate a different location of the interrupt vector for floating-point exceptions.

18.14.7. Changes to Floating-Point Instructions

This section identifies the differences in floating-point instructions for the various Intel FPU and math coprocessor architectures, the reason for the differences, and their impact on software.

18.14.7.1. FDIV, FPREM, AND FSQRT INSTRUCTIONS

The 32-bit x87 FPUs support operations on denormalized operands and, when detected, an underflow exception can occur, for compatibility with the IEEE Standard 754. The 16-bit IA-32 math coprocessors do not operate on denormalized operands or return underflow results. Instead, they generate an invalid-operation exception when they detect an underflow condition. An existing underflow exception handler will require change only if it gives different treatment to different opcodes. Also, it is possible that fewer invalid-operation exceptions will occur.

18.14.7.2. FSCALE INSTRUCTION

With the 32-bit x87 FPUs, the range of the scaling operand is not restricted. If $(0 < |ST(1)| < 1)$, the scaling factor is 0; therefore, $ST(0)$ remains unchanged. If the rounded result is not exact or if there was a loss of accuracy (masked underflow), the precision exception is signaled. With the 16-bit IA-32 math coprocessors, the range of the scaling operand is restricted. If $(0 < |ST(1)| < 1)$, the result is undefined and no exception is signaled. The impact of this difference on existing software is that different results are delivered on the 32-bit and 16-bit FPUs and math coprocessors when $(0 < |ST(1)| < 1)$.

18.14.7.3. FPREM1 INSTRUCTION

The 32-bit x87 FPUs compute a partial remainder according to IEEE Standard 754. This instruction does not exist on the 16-bit IA-32 math coprocessors. The availability of the FPREM1 instruction has no impact on existing software.

18.14.7.4. FPREM INSTRUCTION

On the 32-bit x87 FPUs, the condition code flags C0, C3, C1 in the status word correctly reflect the three low-order bits of the quotient following execution of the FPREM instruction. On the 16-bit IA-32 math coprocessors, the quotient bits are incorrect when performing a reduction of $(64^N + M)$ when $(N \geq 1)$ and M is 1 or 2. This difference does not affect existing software; software that works around the bug should not be affected.

18.14.7.5. FUCOM, FUCOMP, AND FUCOMPP INSTRUCTIONS

When executing the FUCOM, FUCOMP, and FUCOMPP instructions, the 32-bit x87 FPUs perform unordered compare according to IEEE Standard 754. These instructions do not exist on the 16-bit IA-32 math coprocessors. The availability of these new instructions has no impact on existing software.

18.14.7.6. FPTAN INSTRUCTION

On the 32-bit x87 FPU, the range of the operand for the FPTAN instruction is much less restricted ($|ST(0)| < 2^{63}$) than on earlier math coprocessors. The instruction reduces the operand internally using an internal $\pi/4$ constant that is more accurate. The range of the operand is restricted to $(|ST(0)| < \pi/4)$ on the 16-bit IA-32 math coprocessors; the operand must be reduced to this range using FPREM. This change has no impact on existing software.

18.14.7.7. STACK OVERFLOW

On the 32-bit x87 FPU, if an FPU stack overflow occurs when the invalid-operation exception is masked, the FPU returns the real, integer, or BCD-integer indefinite value to the destination operand, depending on the instruction being executed. On the 16-bit IA-32 math coprocessors, the original operand remains unchanged following a stack overflow, but it is loaded into register ST(1). This difference has no impact on existing software.

18.14.7.8. FSIN, FCOS, AND FSINCOS INSTRUCTIONS

On the 32-bit x87 FPU, these instructions perform three common trigonometric functions. These instructions do not exist on the 16-bit IA-32 math coprocessors. The availability of these instructions has no impact on existing software, but using them provides a performance upgrade.

18.14.7.9. FPATAN INSTRUCTION

On the 32-bit x87 FPU, the range of operands for the FPATAN instruction is unrestricted. On the 16-bit IA-32 math coprocessors, the absolute value of the operand in register ST(0) must be smaller than the absolute value of the operand in register ST(1). This difference has impact on existing software.

18.14.7.10. F2XM1 INSTRUCTION

The 32-bit x87 FPU supports a wider range of operands ($-1 < ST(0) < +1$) for the F2XM1 instruction. The supported operand range for the 16-bit IA-32 math coprocessors is $(0 \leq ST(0) \leq 0.5)$. This difference has no impact on existing software.

18.14.7.11. FLD INSTRUCTION

On the 32-bit x87 FPU, when using the FLD instruction to load an extended-real value, a denormal-operand exception is not generated because the instruction is not arithmetic. The 16-bit IA-32 math coprocessors do report a denormal-operand exception in this situation. This difference does not affect existing software.

On the 32-bit x87 FPU, loading a denormal value that is in single- or double-real format causes the value to be converted to extended-real format. Loading a denormal value on the 16-bit IA-32 math coprocessors causes the value to be converted to an unnormal. If the next instruction is FXTRACT or FXAM, the 32-bit x87 FPU will give a different result than the 16-bit IA-32 math coprocessors. This change was made for IEEE Standard 754 compatibility.

On the 32-bit x87 FPUs, loading an SNaN that is in single- or double-real format causes the FPU to generate an invalid-operation exception. The 16-bit IA-32 math coprocessors do not raise an exception when loading a signaling NaN. The invalid-operation exception handler for 16-bit math coprocessor software needs to be updated to handle this condition when porting software to 32-bit FPUs. This change was made for IEEE Standard 754 compatibility.

18.14.7.12. FXTRACT INSTRUCTION

On the 32-bit x87 FPUs, if the operand is 0 for the FXTRACT instruction, the divide-by-zero exception is reported and $-\infty$ is delivered to register ST(1). If the operand is $+\infty$, no exception is reported. If the operand is 0 on the 16-bit IA-32 math coprocessors, 0 is delivered to register ST(1) and no exception is reported. If the operand is $+\infty$, the invalid-operation exception is reported. These differences have no impact on existing software. Software usually bypasses 0 and ∞ . This change is due to the IEEE Standard 754 recommendation to fully support the “logb” function.

18.14.7.13. LOAD CONSTANT INSTRUCTIONS

On 32-bit x87 FPUs, rounding control is in effect for the load constant instructions. Rounding control is not in effect for the 16-bit IA-32 math coprocessors. Results for the FLDPI, FLDLN2, FLDLG2, and FLDDL2E instructions are the same as for the 16-bit IA-32 math coprocessors when rounding control is set to round to nearest or round to $+\infty$. They are the same for the FLDDL2T instruction when rounding control is set to round to nearest, round to $-\infty$, or round to zero. Results are different from the 16-bit IA-32 math coprocessors in the least significant bit of the mantissa if rounding control is set to round to $-\infty$ or round to 0 for the FLDPI, FLDLN2, FLDLG2, and FLDDL2E instructions; they are different for the FLDDL2T instruction if round to $+\infty$ is specified. These changes were implemented for compatibility with IEEE Standard 754 for Floating-Point Arithmetic recommendations.

18.14.7.14. FSETPM INSTRUCTION

With the 32-bit x87 FPUs, the FSETPM instruction is treated as NOP (no operation). This instruction informs the Intel 287 math coprocessor that the processor is in protected mode. This change has no impact on existing software. The 32-bit x87 FPUs handle all addressing and exception-pointer information, whether in protected mode or not.

18.14.7.15. FXAM INSTRUCTION

With the 32-bit x87 FPUs, if the FPU encounters an empty register when executing the FXAM instruction, it not generate combinations of C0 through C3 equal to 1101 or 1111. The 16-bit IA-32 math coprocessors may generate these combinations, among others. This difference has no impact on existing software; it provides a performance upgrade to provide repeatable results.

18.14.7.16. FSAVE AND FSTENV INSTRUCTIONS

With the 32-bit x87 FPUs, the address of a memory operand pointer stored by FSAVE or FSTENV is undefined if the previous floating-point instruction did not refer to memory

18.14.8. Transcendental Instructions

The floating-point results of the P6 family and Pentium processors for transcendental instructions in the core range may differ from the Intel486 processors by about 2 or 3 ulps (see “Transcendental Instruction Accuracy” in Chapter 8 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*). Condition code flag C1 of the status word may differ as a result. The exact threshold for underflow and overflow will vary by a few ulps. The P6 family and Pentium processors’ results will have a worst case error of less than 1 ulp when rounding to the nearest-even and less than 1.5 ulps when rounding in other modes. The transcendental instructions are guaranteed to be monotonic, with respect to the input operands, throughout the domain supported by the instruction.

Transcendental instructions may generate different results in the round-up flag (C1) on the 32-bit x87 FPUs. The round-up flag is undefined for these instructions on the 16-bit IA-32 math coprocessors. This difference has no impact on existing software.

18.14.9. Obsolete Instructions

The 8087 math coprocessor instructions FENI and FDISI and the Intel 287 math coprocessor instruction FSETPM are treated as integer NOP instructions in the 32-bit x87 FPUs. If these opcodes are detected in the instruction stream, no specific operation is performed and no internal states are affected.

18.14.10. WAIT/FWAIT Prefix Differences

On the Intel486 processor, when a WAIT/FWAIT instruction precedes a floating-point instruction (one which itself automatically synchronizes with the previous floating-point instruction), the WAIT/FWAIT instruction is treated as a no-op. Pending floating-point exceptions from a previous floating-point instruction are processed not on the WAIT/FWAIT instruction but on the floating-point instruction following the WAIT/FWAIT instruction. In such a case, the report of a floating-point exception may appear one instruction later on the Intel486 processor than on a P6 family or Pentium FPU, or on Intel 387 math coprocessor.

18.14.11. Operands Split Across Segments and/or Pages

On the P6 family, Pentium, and Intel486 processor FPUs, when the first half of an operand to be written is inside a page or segment and the second half is outside, a memory fault can cause the first half to be stored but not the second half. In this situation, the Intel 387 math coprocessor stores nothing.

18.14.12.FPU Instruction Synchronization

On the 32-bit x87 FPUs, all floating-point instructions are automatically synchronized; that is, the processor automatically waits until the previous floating-point instruction has completed before completing the next floating-point instruction. No explicit WAIT/FWAIT instructions are required to assure this synchronization. For the 8087 math coprocessors, explicit waits are required before each floating-point instruction to ensure synchronization. Although 8087 programs having explicit WAIT instructions execute perfectly on the 32-bit IA-32 processors without reassembly, these WAIT instructions are unnecessary.

18.15. SERIALIZING INSTRUCTIONS

Certain instructions have been defined to serialize instruction execution to ensure that modifications to flags, registers and memory are completed before the next instruction is executed (or in P6 family processor terminology “committed to machine state”). Because the P6 family processors use branch-prediction and out-of-order execution techniques to improve performance, instruction execution is not generally serialized until the results of an executed instruction are committed to machine state (see Chapter 2, *Introduction to the IA-32 Architecture*, in the *IA-32 Intel Architecture Software Developer’s Manual, Volume 1*).

As a result, at places in a program or task where it is critical to have execution completed for all previous instructions before executing the next instruction (for example, at a branch, at the end of a procedure, or in multiprocessor dependent code), it is useful to add a serializing instruction. See Section 7.4., “Serializing Instructions”, for more information on serializing instructions.

18.16. FPU AND MATH COPROCESSOR INITIALIZATION

Table 9-1 shows the states of the FPUs in the P6 family, Pentium, Intel486 processors and of the Intel 387 math coprocessor and Intel 287 coprocessor following a power-up, reset, or INIT, or following the execution of an FINIT/FNINIT instruction. The following is some additional compatibility information concerning the initialization of x87 FPUs and math coprocessors.

18.16.1. Intel 387 and Intel 287 Math Coprocessor Initialization

Following an Intel386 processor reset, the processor identifies its coprocessor type (Intel 287 or Intel 387 DX math coprocessor) by sampling its ERROR# input some time after the falling edge of RESET# signal and before execution of the first floating-point instruction. The Intel 287 coprocessor keeps its ERROR# output in inactive state after hardware reset; the Intel 387 coprocessor keeps its ERROR# output in active state after hardware reset.

Upon hardware reset or execution of the FINIT/FNINIT instruction, the Intel 387 math coprocessor signals an error condition. The P6 family, Pentium, and Intel486 processors, like the Intel 287 coprocessor, do not.

18.16.2. Intel486 SX Processor and Intel 487 SX Math Coprocessor Initialization

When initializing an Intel486 SX processor and an Intel 487 SX math coprocessor, the initialization routine should check the presence of the math coprocessor and should set the FPU related flags (EM, MP, and NE) in control register CR0 accordingly (see Section 2.5., “Control Registers”, for a complete description of these flags). Table 18-2 gives the recommended settings for these flags when the math coprocessor is present. The FSTCW instruction will give a value of FFFFH for the Intel486 SX microprocessor and 037FH for the Intel 487 SX math coprocessor.

Table 18-2. Recommended Values of the EM, MP, and NE Flags for Intel486 SX Microprocessor/Intel 487 SX Math Coprocessor System

CR0 Flags	Intel486 SX Processor Only	Intel 487 SX Math Coprocessor Present
EM	1	0
MP	0	1
NE	1	0, for MS-DOS* systems 1, for user-defined exception handler

The EM and MP flags in register CR0 are interpreted as shown in Table 18-3.

Table 18-3. EM and MP Flag Interpretation

EM	MP	Interpretation
0	0	Floating-point instructions are passed to FPU; WAIT/FWAIT and other waiting-type instructions ignore TS.
0	1	Floating-point instructions are passed to FPU; WAIT/FWAIT and other waiting-type instructions test TS.
1	0	Floating-point instructions trap to emulator; WAIT/FWAIT and other waiting-type instructions ignore TS.
1	1	Floating-point instructions trap to emulator; WAIT/FWAIT and other waiting-type instructions test TS.

Following is an example code sequence to initialize the system and check for the presence of Intel486 SX processor/Intel 487 SX math coprocessor.

```
fninit
fstcw mem_loc
mov ax, mem_loc
cmp ax, 037fh
jz Intel487_SX_Math_CoProcessor_present; ax=037fh
jmp Intel486_SX_microprocessor_present; ax=ffffh
```

If the Intel 487 SX math coprocessor is not present, the following code can be run to set the CR0 register for the Intel486 SX processor.

```
mov eax, cr0
and eax, ffffffffh ;make MP=0
or  eax, 0024h      ;make EM=1, NE=1
mov cr0, eax
```

This initialization will cause any floating-point instruction to generate a device not available exception (#NH), interrupt 7. The software emulation will then take control to execute these instructions. This code is not required if an Intel 487 SX math coprocessor is present in the system. In that case, the typical initialization routine for the Intel486 SX microprocessor will be adequate.

Also, when designing an Intel486 SX processor based system with an Intel 487 SX math coprocessor, timing loops should be independent of clock speed and clocks per instruction. One way to attain this is to implement these loops in hardware and not in software (for example, BIOS).

18.17. CONTROL REGISTERS

The following sections identify the new control registers and control register flags and fields that were introduced to the 32-bit IA-32 in various processor families. See Figure 2-5 for the location of these flags and fields in the control registers.

The Pentium III processor introduced one new control flag in control register CR4:

- OSXMMEXCPT (bit 10)—The OS will set this bit if it supports unmasked SIMD floating-point exceptions.

The Pentium II processor introduced one new control flag in control register CR4:

- OSFXSR (bit 9)—The OS supports saving and restoring the Pentium III processor state during context switches.

The Pentium Pro processor introduced three new control flags in control register CR4:

- PAE (bit 5)—Physical address extension. Enables paging mechanism to reference 36-bit physical addresses when set; restricts physical addresses to 32 bits when clear (see Section 18.18.1.1., “Physical Memory Addressing Extension”).
- PGE (bit 7)—Page global enable. Inhibits flushing of frequently-used or shared pages on task switches (see Section 18.18.1.2., “Global Pages”).
- PCE (bit 8)—Performance-monitoring counter enable. Enables execution of the RDPMC instruction at any protection level.

The content of CR4 is 0H following a hardware reset.

Control register CR4 was introduced in the Pentium processor. This register contains flags that enable certain new extensions provided in the Pentium processor:

- VME—Virtual-8086 mode extensions. Enables support for a virtual interrupt flag in virtual-8086 mode (see Section 16.3., “Interrupt and Exception Handling in Virtual-8086 Mode”).
- PVI—Protected-mode virtual interrupts. Enables support for a virtual interrupt flag in protected mode (see Section 16.4., “Protected-Mode Virtual Interrupts”).
- TSD—Time-stamp disable. Restricts the execution of the RDTSC instruction to procedures running at privileged level 0.
- DE—Debugging extensions. Causes an undefined opcode (#UD) exception to be generated when debug registers DR4 and DR5 are references for improved performance (see Section 15.2.2., “Debug Registers DR4 and DR5”).
- PSE—Page size extensions. Enables 4-MByte pages when set (see Section 3.6.1., “Paging Options”).
- MCE—Machine-check enable. Enables the machine-check exception, allowing exception handling for certain hardware error conditions (see Chapter 14, *Machine-Check Architecture*).

The Intel486 processor introduced five new flags in control register CR0:

- NE—Numeric error. Enables the normal mechanism for reporting floating-point numeric errors.
- WP—Write protect. Write-protects user-level pages against supervisor-mode accesses.
- AM—Alignment mask. Controls whether alignment checking is performed. Operates in conjunction with the AC (Alignment Check) flag.
- NW—Not write-through. Enables write-throughs and cache invalidation cycles when clear and disables invalidation cycles and write-throughs that hit in the cache when set.
- CD—Cache disable. Enables the internal cache when clear and disables the cache when set.

The Intel486 processor introduced two new flags in control register CR3:

- PCD—Page-level cache disable. The state of this flag is driven on the PCD# pin during bus cycles that are not paged, such as interrupt acknowledge cycles, when paging is enabled. The PCD# pin is used to control caching in an external cache on a cycle-by-cycle basis.
- PWT—Page-level write-through. The state of this flag is driven on the PWT# pin during bus cycles that are not paged, such as interrupt acknowledge cycles, when paging is enabled. The PWT# pin is used to control write through in an external cache on a cycle-by-cycle basis.

18.18. MEMORY MANAGEMENT FACILITIES

The following sections describe the new memory management facilities available in the various IA-32 processors and some compatibility differences.

18.18.1. New Memory Management Control Flags

The Pentium Pro processor introduced three new memory management features: physical memory addressing extension, the global bit in page-table entries, and general support for larger page sizes. These features are only available when operating in protected mode.

18.18.1.1. PHYSICAL MEMORY ADDRESSING EXTENSION

The new PAE (physical address extension) flag in control register CR4, bit 5, enables 4 additional address lines on the processor, allowing 36-bit physical addresses. This option can only be used when paging is enabled, using a new page-table mechanism provided to support the larger physical address range (see Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”).

18.18.1.2. GLOBAL PAGES

The new PGE (page global enable) flag in control register CR4, bit 7, provides a mechanism for preventing frequently used pages from being flushed from the translation lookaside buffer (TLB). When this flag is set, frequently used pages (such as pages containing kernel procedures or common data tables) can be marked global by setting the global flag in a page-directory or page-table entry.

On a task switch or a write to control register CR3 (which normally causes the TLBs to be flushed), the entries in the TLB marked global are not flushed. Marking pages global in this manner prevents unnecessary reloading of the TLB due to TLB misses on frequently used pages. See Section 3.11., “Translation Lookaside Buffers (TLBs)”, for a detailed description of this mechanism.

18.18.1.3. LARGER PAGE SIZES

The P6 family processors support large page sizes. This facility is enabled with the PSE (page size extension) flag in control register CR4, bit 4. When this flag is set, the processor supports either 4-KByte or 4-MByte page sizes when normal paging is used and 4-KByte and 2-MByte page sizes when the physical address extension is used. See Section 3.6.1., “Paging Options”, for more information about large page sizes.

18.18.2. CD and NW Cache Control Flags

The CD and NW flags in control register CR0 were introduced in the Intel486 processor. In the P6 family and Pentium processors, these flags are used to implement a writeback strategy for

the data cache; in the Intel486 processor, they implement a write-through strategy. See Table 10-5 for a comparison of these bits on the P6 family, Pentium, and Intel486 processors. For complete information on caching, see Chapter 10, *Memory Cache Control*.

18.18.3. Descriptor Types and Contents

Operating-system code that manages space in descriptor tables often contains an invalid value in the access-rights field of descriptor-table entries to identify unused entries. Access rights values of 80H and 00H remain invalid for the P6 family, Pentium, Intel486, Intel386, and Intel 286 processors. Other values that were invalid on the Intel 286 processor may be valid on the 32-bit processors because uses for these bits have been defined.

18.18.4. Changes in Segment Descriptor Loads

On the Intel386 processor, loading a segment descriptor always causes a locked read and write to set the accessed bit of the descriptor. On the P6 family, Pentium, and Intel486 processors, the locked read and write occur only if the bit is not already set.

18.19. DEBUG FACILITIES

The P6 family and Pentium processors include extensions to the Intel486 processor debugging support for breakpoints. To use the new breakpoint features, it is necessary to set the DE flag in control register CR4.

18.19.1. Differences in Debug Register DR6

It is not possible to write a 1 to reserved bit 12 in debug status register DR6 on the P6 family and Pentium processors; however, it is possible to write a 1 in this bit on the Intel486 processor. See Table 9-1 for the different setting of this register following a power-up or hardware reset.

18.19.2. Differences in Debug Register DR7

The P6 family and Pentium processors determines the type of breakpoint access by the R/W0 through R/W3 fields in debug control register DR7 as follows:

- | | |
|----|--|
| 00 | Break on instruction execution only. |
| 01 | Break on data writes only. |
| 10 | Undefined if the DE flag in control register CR4 is cleared; break on I/O reads or writes but not instruction fetches if the DE flag in control register CR4 is set. |
| 11 | Break on data reads or writes but not instruction fetches. |

On the P6 family and Pentium processors, reserved bits 11, 12, 14 and 15 are hard-wired to 0. On the Intel486 processor, however, bit 12 can be set. See Table 9-1 for the different settings of this register following a power-up or hardware reset.

18.19.3. Debug Registers DR4 and DR5

Although the DR4 and DR5 registers are documented as reserved, previous generations of processors aliased references to these registers to debug registers DR6 and DR7, respectively. When debug extensions are not enabled (the DE flag in control register CR4 is cleared), the P6 family and Pentium processors remain compatible with existing software by allowing these aliased references. When debug extensions are enabled (the DE flag is set), attempts to reference registers DR4 or DR5 will result in an invalid-opcode exception (#UD).

18.19.4. Recognition of Breakpoints

For the Pentium processor, it is recommended that debuggers execute the LGDT instruction before returning to the program being debugged to ensure that breakpoints are detected. This operation does not need to be performed on the P6 family, Intel486, or Intel386 processors.

18.20. TEST REGISTERS

The implementation of test registers on the Intel486 processor used for testing the cache and TLB has been redesigned using MSRs on the P6 family and Pentium processors. (Note that MSRs used for this function are different on the P6 family and Pentium processors.) The MOV to and from test register instructions generate invalid-opcode exceptions (#UD) on the P6 family processors.

18.21. Exceptions and/or Exception Conditions

This section describes the new exceptions and exception conditions added to the 32-bit IA-32 processors and implementation differences in existing exception handling. See Chapter 5, *Interrupt and Exception Handling*, for a detailed description of the IA-32 exceptions.

The Pentium III processor introduced new state with the XMM registers. Computations involving data in these registers can produce exceptions. A new MXCSR control/status register is used to determine which exception or exceptions have occurred. When an exception associated with the XMM registers occurs, an interrupt is generated.

- SIMD floating-point exception (#XF, interrupt 19)—New exceptions associated with the SIMD floating-point registers and resulting computations.

No new exceptions were added with the Pentium Pro and Pentium II processors. The set of available exceptions is the same as for the Pentium processor. However, the following exception condition was added to the IA-32 with the Pentium Pro processor:

- Machine-check exception (#MC, interrupt 18)—New exception conditions. Many exception conditions have been added to the machine-check exception and a new architecture has been added for handling and reporting on hardware errors. See Chapter 14, *Machine-Check Architecture*, for a detailed description of the new conditions.

The following exceptions and/or exception conditions were added to the IA-32 with the Pentium processor:

- Machine-check exception (#MC, interrupt 18)—New exception. This exception reports parity and other hardware errors. It is a model-specific exception and may not be implemented or implemented differently in future processors. The MCE flag in control register CR4 enables the machine-check exception. When this bit is clear (which it is at reset), the processor inhibits generation of the machine-check exception.
- General-protection exception (#GP, interrupt 13)—New exception condition added. An attempt to write a 1 to a reserved bit position of a special register causes a general-protection exception to be generated.
- Page-fault exception (#PF, interrupt 14)—New exception condition added. When a 1 is detected in any of the reserved bit positions of a page-table entry, page-directory entry, or page-directory pointer during address translation, a page-fault exception is generated.

The following exception was added to the Intel486 processor:

- Alignment-check exception (#AC, interrupt 17)—New exception. Reports unaligned memory references when alignment checking is being performed.

The following exceptions and/or exception conditions were added to the Intel386 processor:

- Divide-error exception (#DE, interrupt 0)
 - Change in exception handling. Divide-error exceptions on the Intel386 processors always leave the saved CS:IP value pointing to the instruction that failed. On the 8086 processor, the CS:IP value points to the next instruction.
 - Change in exception handling. The Intel386 processors can generate the largest negative number as a quotient for the IDIV instruction (80H and 8000H). The 8086 processor generates a divide-error exception instead.
- Invalid-opcode exception (#UD, interrupt 6)—New exception condition added. Improper use of the LOCK instruction prefix can generate an invalid-opcode exception.
- Page-fault exception (#PF, interrupt 14)—New exception condition added. If paging is enabled in a 16-bit program, a page-fault exception can be generated as follows. Paging can be used in a system with 16-bit tasks if all tasks use the same page directory. Because there is no place in a 16-bit TSS to store the PDBR register, switching to a 16-bit task does not change the value of the PDBR register. Tasks ported from the Intel 286 processor should be given 32-bit TSSs so they can make full use of paging.

- General-protection exception (#GP, interrupt 13)—New exception condition added. The Intel386 processor sets a limit of 15 bytes on instruction length. The only way to violate this limit is by putting redundant prefixes before an instruction. A general-protection exception is generated if the limit on instruction length is violated. The 8086 processor has no instruction length limit.

18.21.1. Machine-Check Architecture

The Pentium Pro processor introduced a new architecture to the IA-32 for handling and reporting on machine-check exceptions. This machine-check architecture (described in detail in Chapter 14, *Machine-Check Architecture*) greatly expands the ability of the processor to report on internal hardware errors.

18.21.2. Priority OF Exceptions

The priority of exceptions are broken down into several major categories:

1. Traps on the previous instruction
2. External interrupts
3. Faults on fetching the next instruction
4. Faults in decoding the next instruction
5. Faults on executing an instruction

There are no changes in the priority of these major categories between the different processors, however, exceptions within these categories are implementation dependent and may change from processor to processor.

18.22. INTERRUPTS

The following differences in handling interrupts are found among the IA-32 processors.

18.22.1. Interrupt Propagation Delay

External hardware interrupts may be recognized on different instruction boundaries on the P6 family, Pentium, Intel486, and Intel386 processors, due to the superscaler designs of the P6 family and Pentium processors. Therefore, the EIP pushed onto the stack when servicing an interrupt may be different for the P6 family, Pentium, Intel486, and Intel386 processors.

18.22.2. NMI Interrupts

After an NMI interrupt is recognized by the P6 family, Pentium, Intel486, Intel386, and Intel 286 processors, the NMI interrupt is masked until the first IRET instruction is executed, unlike the 8086 processor.

18.22.3. IDT Limit

The LIDT instruction can be used to set a limit on the size of the IDT. A double-fault exception (#DF) is generated if an interrupt or exception attempts to read a vector beyond the limit. Shutdown then occurs on the 32-bit IA-32 processors if the double-fault handler vector is beyond the limit. (The 8086 processor does not have a shutdown mode nor a limit.)

18.23. ADVANCED PROGRAMMABLE INTERRUPT CONTROLLER (APIC)

The Advanced Programmable Interrupt Controller (APIC), referred to in this book as the **local APIC**, was introduced into the IA-32 processors with the Pentium processor (beginning with the 735/90 and 815/100 models) and is included in the Pentium 4, Intel Xeon, and P6 family processors. The features and functions of the local APIC are derived from the Intel 82489DX external APIC, which was used with the Intel486 and early Pentium processors. Additional refinements of the local APIC architecture were incorporated in the Pentium 4 and Intel Xeon processors.

18.23.1. Software Visible Differences Between the Local APIC and the 82489DX

The following features in the local APIC features differ from those found in the 82489DX external APIC:

- When the local APIC is disabled by clearing the APIC software enable/disable flag in the spurious-interrupt vector MSR, the state of its internal registers are unaffected, except that the mask bits in the LVT are all set to block local interrupts to the processor. Also, the local APIC ceases accepting IPIs except for INIT, SMI, NMI, and start-up IPIs. In the 82489DX, when the local unit is disabled, all the internal registers including the IRR, ISR and TMR are cleared and the mask bits in the LVT are set. In this state, the 82489DX local unit will accept only the reset deassert message.
- In the local APIC, NMI and INIT (except for INIT deassert) are always treated as edge triggered interrupts, even if programmed otherwise. In the 82489DX, these interrupts are always level triggered.
- In the local APIC, IPIs generated through the ICR are always treated as edge triggered (except INIT Deassert). In the 82489DX, the ICR can be used to generate either edge or level triggered IPIs.

- In the local APIC, the logical destination register supports 8 bits; in the 82489DX, it supports 32 bits.
- In the local APIC, the APIC ID register is 4 bits wide; in the 82489DX, it is 8 bits wide.
- The remote read delivery mode provided in the 82489DX and local APIC for Pentium processors is not supported in the local APIC in the Pentium 4, Intel Xeon, and P6 family processors.
- For the 82489DX, in the lowest priority delivery mode, all the target local APICs specified by the destination field participate in the lowest priority arbitration. For the local APIC, only those local APICs which have free interrupt slots will participate in the lowest priority arbitration.

18.23.2. New Features Incorporated in the Local APIC for the P6 Family and Pentium Processors

The local APIC in the Pentium and P6 family processors have the following new features not found in the 82489DX external APIC.

- Cluster addressing is supported in logical destination mode.
- Focus processor checking can be enabled/disabled.
- Interrupt input signal polarity can be programmed for the LINT0 and LINT1 pins.
- An SMI IPI is supported through the ICR and I/O redirection table.
- An error status register is incorporated into the LVT to log and report APIC errors.

In the P6 family processors, the local APIC incorporates an additional LVT register to handle performance monitoring counter interrupts.

18.23.3. New Features Incorporated in the Local APIC of the Pentium 4 and Intel Xeon Processors

The local APIC in the Pentium 4 and Intel Xeon processors has the following new features not found in the P6 family and Pentium processors and in the 82489DX.

- The local APIC ID is extended to 8 bits.
- An thermal sensor register is incorporated into the LVT to handle thermal sensor interrupts.
- The the ability to deliver lowest-priority interrupts to a focus processor is no longer supported.
- The flat cluster logical destination mode is not supported.

18.24. TASK SWITCHING AND TSS

This section identifies the implementation differences of task switching, additions to the TSS and the handling of TSSs and TSS segment selectors.

18.24.1. P6 Family and Pentium Processor TSS

When the virtual mode extensions are enabled (by setting the VME flag in control register CR4), the TSS in the P6 family and Pentium processors contain an interrupt redirection bit map, which is used in virtual-8086 mode to redirect interrupts back to an 8086 program.

18.24.2. TSS Selector Writes

During task state saves, the Intel486 processor writes 2-byte segment selectors into a 32-bit TSS, leaving the upper 16 bits undefined. For performance reasons, the P6 family and Pentium processors write 4-byte segment selectors into the TSS, with the upper 2 bytes being 0. For compatibility reasons, code should not depend on the value of the upper 16 bits of the selector in the TSS.

18.24.3. Order of Reads/Writes to the TSS

The order of reads and writes into the TSS is processor dependent. The P6 family and Pentium processors may generate different page-fault addresses in control register CR2 in the same TSS area than the Intel486 and Intel386 processors, if a TSS crosses a page boundary (which is not recommended).

18.24.4. Using A 16-Bit TSS with 32-Bit Constructs

Task switches using 16-bit TSSs should be used only for pure 16-bit code. Any new code written using 32-bit constructs (operands, addressing, or the upper word of the EFLAGS register) should use only 32-bit TSSs. This is due to the fact that the 32-bit processors do not save the upper 16 bits of EFLAGS to a 16-bit TSS. A task switch back to a 16-bit task that was executing in virtual mode will never re-enable the virtual mode, as this flag was not saved in the upper half of the EFLAGS value in the TSS. Therefore, it is strongly recommended that any code using 32-bit constructs use a 32-bit TSS to ensure correct behavior in a multitasking environment.

18.24.5. Differences in I/O Map Base Addresses

The Intel486 processor considers the TSS segment to be a 16-bit segment and wraps around the 64K boundary. Any I/O accesses check for permission to access this I/O address at the I/O base address plus the I/O offset. If the I/O map base address exceeds the specified limit of 0DFFFH, an I/O access will wrap around and obtain the permission for the I/O address at an incorrect

location within the TSS. A TSS limit violation does not occur in this situation on the Intel486 processor. However, the P6 family and Pentium processors consider the TSS to be a 32-bit segment and a limit violation occurs when the I/O base address plus the I/O offset is greater than the TSS limit. By following the recommended specification for the I/O base address to be less than 0DFFFH, the Intel486 processor will not wrap around and access incorrect locations within the TSS for I/O port validation and the P6 family and Pentium processors will not experience general-protection exceptions (#GP). Figure 18-1 demonstrates the different areas accessed by the Intel486 and the P6 family and Pentium processors.

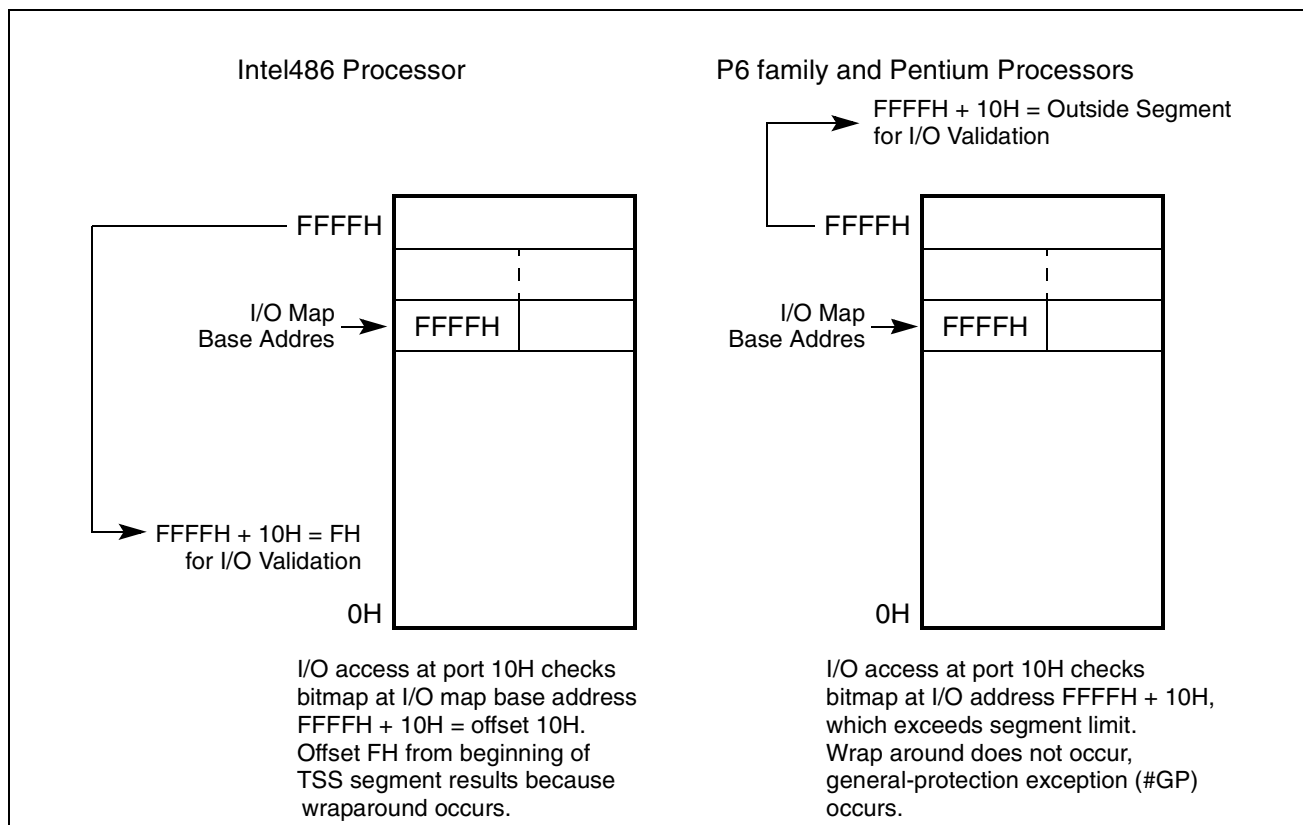


Figure 18-1. I/O Map Base Address Differences

18.25. CACHE MANAGEMENT

The P6 family processors include two levels of internal caches: L1 (level 1) and L2 (level 2). The L1 cache is divided into an instruction cache and a data cache; the L2 cache is a general-purpose cache. See Section 10.1., “Internal Caches, TLBs, and Buffers”, for a description of these caches. (Note that although the Pentium II processor L2 cache is physically located on a separate chip in the cassette, it is considered an internal cache.)

The Pentium processor includes separate level 1 instruction and data caches. The data cache supports a writeback (or alternatively write-through, on a line by line basis) policy for memory updates. Refer to the *Pentium Processor Data Book* for more information about the organization and operation of the Pentium processor caches.

The Intel486 processor includes a single level 1 cache for both instructions and data.

The meaning of the CD and NW flags in control register CR0 have been redefined for the P6 family and Pentium processors. For these processors, the recommended value (00B) enables writeback for the data cache of the Pentium processor and for the L1 data cache and L2 cache of the P6 family processors. In the Intel486 processor, setting these flags to (00B) enables write-through for the cache.

External system hardware can force the Pentium processor to disable caching or to use the write-through cache policy should that be required. Refer to the *Pentium Processor Data Book* for more information about hardware control of the Pentium processor caches. In the P6 family processors, the MTRRs can be used to override the CD and NW flags (see Table 10-6).

The P6 family and Pentium processors support page-level cache management in the same manner as the Intel486 processor by using the PCD and PWT flags in control register CR3, the page-directory entries, and the page-table entries. The Intel486 processor, however, is not affected by the state of the PWT flag since the internal cache of the Intel486 processor is a write-through cache.

18.25.1. Self-Modifying Code with Cache Enabled

On the Intel486 processor, a write to an instruction in the cache will modify it in both the cache and memory. If the instruction was prefetched before the write, however, the old version of the instruction could be the one executed. To prevent this problem, it is necessary to flush the instruction prefetch unit of the Intel486 processor by coding a jump instruction immediately after any write that modifies an instruction. The P6 family and Pentium processors, however, check whether a write may modify an instruction that has been prefetched for execution. This check is based on the linear address of the instruction. If the linear address of an instruction is found to be present in the prefetch queue, the P6 family and Pentium processors flush the prefetch queue, eliminating the need to code a jump instruction after any writes that modify an instruction.

Because the linear address of the write is checked against the linear address of the instructions that have been prefetched, special care must be taken for self-modifying code to work correctly when the physical addresses of the instruction and the written data are the same, but the linear addresses differ. In such cases, it is necessary to execute a serializing operation to flush the prefetch queue after the write and before executing the modified instruction. See Section 7.4., “Serializing Instructions”, for more information on serializing instructions.

NOTE

The check on linear addresses described above is not in practice a concern for compatibility. Applications that include self-modifying code use the same linear address for modifying and fetching the instruction. System software, such as a debugger, that might possibly modify an instruction using a different linear address than that used to fetch the instruction must execute a serializing operation, such as IRET, before the modified instruction is executed.

18.25.2. Disabling the L3 Cache

A unified third-level (L3) cache was introduced in the Pentium 4 and Intel Xeon processors (see Section 10.1., “Internal Caches, TLBs, and Buffers”) along with the third-level cache disable flag, bit 6 of the IA32_MISC_ENABLE MSR. The third-level cache disable flag allows the L3 cache to be disabled and enabled, independently of the L1 and L2 caches (see Section 10.5.4., “Disabling and Enabling the L3 Cache”).

18.26. PAGING

This section identifies enhancements made to the paging mechanism and implementation differences in the paging mechanism for various IA-32 processors.

18.26.1. Large Pages

The Pentium processor extended the memory management/paging facilities of the IA-32 to allow large (4Mbytes) pages sizes (see Section 3.6.1., “Paging Options”). The first P6 family processor (the Pentium Pro processor) added a 2MByte page size to the IA-32 in conjunction with the physical address extension (PAE) feature (see Section 3.8., “36-Bit Physical Addressing Using the PAE Paging Mechanism”).

The availability of large pages on any IA-32 processor can be determined via feature bit 3 (PSE) of register EDX after the CUID instruction has been execution with an argument of 1. Intel processors that do not support the CUID instruction do not support page size enhancements. (See “CUID—CPU Identification” in Chapter 3, *Instruction Set Reference*, of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 2*, and AP-485, *Intel Processor Identification and the CUID Instruction*, for more information on the CUID instruction.)

18.26.2. PCD and PWT Flags

The PCD and PWT flags were introduced to the IA-32 in the Intel486 processor to control the caching of pages:

- PCD (page-level cache disable) flag—Controls caching on a page-by-page basis.
- PWT (page-level write-through) flag—Controls the write-through/writeback caching policy on a page-by-page basis. Since the internal cache of the Intel486 processor is a write-through cache, it is not affected by the state of the PWT flag.

18.26.3. Enabling and Disabling Paging

Paging is enabled and disabled by loading a value into control register CR0 that modifies the PG flag. For backward and forward compatibility with all IA-32 processors, Intel recommends that the following operations be performed when enabling or disabling paging:

1. Execute a MOV CR0, REG instruction to either set (enable paging) or clear (disable paging) the PG flag.
2. Execute a near JMP instruction.

The sequence bounded by the MOV and JMP instructions should be identity mapped (that is, the instructions should reside on a page whose linear and physical addresses are identical).

For the P6 family processors, the MOV CR0, REG instruction is serializing, so the jump operation is not required. However, for backwards compatibility, the JMP instruction should still be included.

18.27. STACK OPERATIONS

This section identifies the differences in the stack mechanism for the various IA-32 processors.

18.27.1. Selector Pushes and Pops

When pushing a segment selector onto the stack, the Pentium 4, Intel Xeon, P6 family, and Intel486 processors decrement the ESP register by the operand size and then write 2 bytes. If the operand size is 32-bits, the upper two bytes of the write are not modified. The Pentium processor decrements the ESP register by the operand size and determines the size of the write by the operand size. If the operand size is 32-bits, the upper two bytes are written as 0s.

When popping a segment selector from the stack, the Pentium 4, Intel Xeon, P6 family, and Intel486 processors read 2 bytes and increment the ESP register by the operand size of the instruction. The Pentium processor determines the size of the read from the operand size and increments the ESP register by the operand size.

It is possible to align a 32-bit selector push or pop such that the operation generates an exception on a Pentium processor and not on an Pentium 4, Intel Xeon, P6 family, or Intel486 processor. This could occur if the third and/or fourth byte of the operation lies beyond the limit of the segment or if the third and/or fourth byte of the operation is located on a non-present or inaccessible page.

For a POP-to-memory instruction that meets the following conditions:

- The stack segment size is 16-bit.
- Any 32-bit addressing form with the SIB byte specifying ESP as the base register.
- The initial stack pointer is FFFCH (32-bit operand) or FFFEh (16-bit operand) and will wrap around to 0H as a result of the POP operation.

The result of the memory write is implementation-specific. For example, in P6 family processors, the result of the memory write is SS:0H plus any scaled index and displacement. In Pentium processors, the result of the memory write may be either a stack fault (real mode or protected mode with stack segment size of 64Kbyte), or write to SS:10000H plus any scaled index and displacement (protected mode and stack segment size exceeds 64Kbyte).

18.27.2. Error Code Pushes

The Intel486 processor implements the error code pushed on the stack as a 16-bit value. When pushed onto a 32-bit stack, the Intel486 processor only pushes 2 bytes and updates ESP by 4. The P6 family and Pentium processors' error code is a full 32 bits with the upper 16 bits set to zero. The P6 family and Pentium processors, therefore, push 4 bytes and update ESP by 4. Any code that relies on the state of the upper 16 bits may produce inconsistent results.

18.27.3. Fault Handling Effects on the Stack

During the handling of certain instructions, such as CALL and PUSHA, faults may occur in different sequences for the different processors. For example, during far calls, the Intel486 processor pushes the old CS and EIP before a possible branch fault is resolved. A branch fault is a fault from a branch instruction occurring from a segment limit or access rights violation. If a branch fault is taken, the Intel486 and P6 family processors will have corrupted memory below the stack pointer. However, the ESP register is backed up to make the instruction restartable. The P6 family processors issue the branch before the pushes. Therefore, if a branch fault does occur, these processors do not corrupt memory below the stack pointer. This implementation difference, however, does not constitute a compatibility problem, as only values at or above the stack pointer are considered to be valid.

18.27.4. Interlevel RET/IRET From a 16-Bit Interrupt or Call Gate

If a call or interrupt is made from a 32-bit stack environment through a 16-bit gate, only 16 bits of the old ESP can be pushed onto the stack. On the subsequent RET/IRET, the 16-bit ESP is popped but the full 32-bit ESP is updated since control is being resumed in a 32-bit stack environment. The Intel486 processor writes the SS selector into the upper 16 bits of ESP. The P6 family and Pentium processors write zeros into the upper 16 bits.

18.28. MIXING 16- AND 32-BIT SEGMENTS

The features of the 16-bit Intel 286 processor are an object-code compatible subset of those of the 32-bit IA-32 processors. The D (default operation size) flag in segment descriptors indicates whether the processor treats a code or data segment as a 16-bit or 32-bit segment; the B (default stack size) flag in segment descriptors indicates whether the processor treats a stack segment as a 16-bit or 32-bit segment.

The segment descriptors used by the Intel 286 processor are supported by the 32-bit IA-32 processors if the Intel-reserved word (highest word) of the descriptor is clear. On the 32-bit IA-32 processors, this word includes the upper bits of the base address and the segment limit.

The segment descriptors for data segments, code segments, local descriptor tables (there are no descriptors for global descriptor tables), and task gates are the same for the 16- and 32-bit processors. Other 16-bit descriptors (TSS segment, call gate, interrupt gate, and trap gate) are supported by the 32-bit processors.

The 32-bit processors also have descriptors for TSS segments, call gates, interrupt gates, and trap gates that support the 32-bit architecture. Both kinds of descriptors can be used in the same system.

For those segment descriptors common to both 16- and 32-bit processors, clear bits in the reserved word cause the 32-bit processors to interpret these descriptors exactly as an Intel 286 processor does, that is:

- **Base Address**—The upper 8 bits of the 32-bit base address are clear, which limits base addresses to 24 bits.
- **Limit**—The upper 4 bits of the limit field are clear, restricting the value of the limit field to 64 Kbytes.
- **Granularity bit**—The G (granularity) flag is clear, indicating the value of the 16-bit limit is interpreted in units of 1 byte.
- **Big bit**—In a data-segment descriptor, the B flag is clear in the segment descriptor used by the 32-bit processors, indicating the segment is no larger than 64 Kbytes.
- **Default bit**—In a code-segment descriptor, the D flag is clear, indicating 16-bit addressing and operands are the default. In a stack-segment descriptor, the D flag is clear, indicating use of the SP register (instead of the ESP register) and a 64-Kbyte maximum segment limit.

For information on mixing 16- and 32-bit code in applications, see Chapter 17, *Mixing 16-Bit and 32-Bit Code*.

18.29. SEGMENT AND ADDRESS WRAPAROUND

This section discusses differences in segment and address wraparound between the P6 family, Pentium, Intel486, Intel386, Intel 286, and 8086 processors.

18.29.1. Segment Wraparound

On the 8086 processor, an attempt to access a memory operand that crosses offset 65,535 or 0FFFFH or offset 0 (for example, moving a word to offset 65,535 or pushing a word when the stack pointer is set to 1) causes the offset to wrap around modulo 65,536 or 010000H. With the Intel 286 processor, any base and offset combination that addresses beyond 16 MBytes wraps

around to the 1 MByte of the address space. The P6 family, Pentium, Intel486, and Intel386 processors in real-address mode generate an exception in these cases:

- A general-protection exception (#GP) if the segment is a data segment (that is, if the CS, DS, ES, FS, or GS register is being used to address the segment).
- A stack-fault exception (#SS) if the segment is a stack segment (that is, if the SS register is being used).

An exception to this behavior occurs when a stack access is data aligned, and the stack pointer is pointing to the last aligned piece of data that size at the top of the stack (ESP is FFFFFFFCH). When this data is popped, no segment limit violation occurs and the stack pointer will wrap around to 0.

The address space of the P6 family, Pentium, and Intel486 processors may wraparound at 1 MByte in real-address mode. An external A20M# pin forces wraparound if enabled. On Intel 8086 processors, it is possible to specify addresses greater than 1 MByte. For example, with a selector value FFFFH and an offset of FFFFH, the effective address would be 10FFEFH (1 MByte plus 65519 bytes). The 8086 processor, which can form addresses up to 20 bits long, truncates the uppermost bit, which “wraps” this address to FFEFH. However, the P6 family, Pentium, and Intel486 processors do not truncate this bit if A20M# is not enabled.

If a stack operation wraps around the address limit, shutdown occurs. (The 8086 processor does not have a shutdown mode or a limit.)

The behavior when executing near the limit of a 4 GB selector (limit=0xFFFFFFFF) is different between the Pentium Pro and the Pentium 4 family of processors. On the Pentium Pro, instructions which cross the limit -- for example, a two byte instruction such as INC EAX that is encoded as 0xFF 0xC0 starting exactly at the limit faults for a segment violation (a one byte instruction at 0xFFFFFFFF does not cause an exception). Using the Pentium 4 microprocessor family, neither of these situations causes a fault.

18.30. STORE BUFFERS AND MEMORY ORDERING

The Pentium 4, Intel Xeon, and P6 family processors provide a store buffer for temporary storage of writes (stores) to memory (see Section 10.10., “Store Buffer”). Writes stored in the store buffer(s) are always written to memory in program order, with the exception of “fast string” store operations (see Section 7.2.3., “Out-of-Order Stores For String Operations in Pentium 4, Intel Xeon, and P6 Family Processors”).

The Pentium processor has two store buffers, one corresponding to each of the pipelines. Writes in these buffers are always written to memory in the order they were generated by the processor core.

It should be noted that only memory writes are buffered and I/O writes are not. The Pentium 4, Intel Xeon, P6 family, Pentium, and Intel486 processors do not synchronize the completion of memory writes on the bus and instruction execution after a write. An I/O, locked, or serializing instruction needs to be executed to synchronize writes with the next instruction (see Section 7.4., “Serializing Instructions”).

The Pentium 4, Intel Xeon, and P6 family processors use processor ordering to maintain consistency in the order that data is read (loaded) and written (stored) in a program and the order the processor actually carries out the reads and writes. With this type of ordering, reads can be carried out speculatively and in any order, reads can pass buffered writes, and writes to memory are always carried out in program order. (See Section 7.2., “Memory Ordering” for more information about processor ordering.) The Pentium III processor introduced a new instruction to serialize writes and make them globally visible. Memory ordering issues can arise between a producer and a consumer of data. The SFENCE instruction provides a performance-efficient way of ensuring ordering between routines that produce weakly-ordered results and routines that consume this data.

No re-ordering of reads occurs on the Pentium processor, except under the condition noted in Section 7.2.1., “Memory Ordering in the Pentium® and Intel486™ Processors”, and in the following paragraph describing the Intel486 processor.

Specifically, the store buffers are flushed before the IN instruction is executed. No reads (as a result of cache miss) are reordered around previously generated writes sitting in the store buffers. The implication of this is that the store buffers will be flushed or emptied before a subsequent bus cycle is run on the external bus.

On both the Intel486 and Pentium processors, under certain conditions, a memory read will go onto the external bus before the pending memory writes in the buffer even though the writes occurred earlier in the program execution. A memory read will only be reordered in front of all writes pending in the buffers if all writes pending in the buffers are cache hits and the read is a cache miss. Under these conditions, the Intel486 and Pentium processors will not read from an external memory location that needs to be updated by one of the pending writes.

During a locked bus cycle, the Intel486 processor will always access external memory, it will never look for the location in the on-chip cache. All data pending in the Intel486 processor's store buffers will be written to memory before a locked cycle is allowed to proceed to the external bus. Thus, the locked bus cycle can be used for eliminating the possibility of reordering read cycles on the Intel486 processor. The Pentium processor does check its cache on a read-modify-write access and, if the cache line has been modified, writes the contents back to memory before locking the bus. The P6 family processors write to their cache on a read-modify-write operation (if the access does not split across a cache line) and does not write back to system memory. If the access does split across a cache line, it locks the bus and accesses system memory.

I/O reads are never reordered in front of buffered memory writes on an IA-32 processor. This ensures an update of all memory locations before reading the status from an I/O device.

18.31. BUS LOCKING

The Intel 286 processor performs the bus locking differently than the Intel P6 family, Pentium, Intel486, and Intel386 processors. Programs that use forms of memory locking specific to the Intel 286 processor may not run properly when run on later processors.

A locked instruction is guaranteed to lock only the area of memory defined by the destination operand, but may lock a larger memory area. For example, typical 8086 and Intel 286 configurations lock the entire physical memory space. Programmers should not depend on this.

On the Intel 286 processor, the LOCK prefix is sensitive to IOPL. If the CPL is greater than the IOPL, a general-protection exception (#GP) is generated. On the Intel386 DX, Intel486, and Pentium, and P6 family processors, no check against IOPL is performed.

The Pentium processor automatically asserts the LOCK# signal when acknowledging external interrupts. After signaling an interrupt request, an external interrupt controller may use the data bus to send the interrupt vector to the processor. After receiving the interrupt request signal, the processor asserts LOCK# to insure that no other data appears on the data bus until the interrupt vector is received. This bus locking does not occur on the P6 family processors.

18.32. BUS HOLD

Unlike the 8086 and Intel 286 processors, but like the Intel386 and Intel486 processors, the P6 family and Pentium processors respond to requests for control of the bus from other potential bus masters, such as DMA controllers, between transfers of parts of an unaligned operand, such as two words which form a doubleword. Unlike the Intel386 processor, the P6 family, Pentium and Intel486 processors respond to bus hold during reset initialization.

18.33. MODEL-SPECIFIC EXTENSIONS TO THE IA-32

Certain extensions to the IA-32 are specific to a processor or family of IA-32 processors and may not be implemented or implemented in the same way in future processors. The following sections describe these model-specific extensions. The CPUID instruction indicates the availability of some of the model-specific features.

18.33.1. Model-Specific Registers

The Pentium processor introduced a set of model-specific registers (MSRs) for use in controlling hardware functions and performance monitoring. To access these MSRs, two new instructions were added to the IA-32 architecture: read MSR (RDMSR) and write MSR (WRMSR). The MSRs in the Pentium processor are not guaranteed to be duplicated or provided in the next generation IA-32 processors.

The P6 family processors greatly increased the number of MSRs available to software. See Appendix B, *Model-Specific Registers (MSRs)*, for a complete list of the available MSRs. The new registers control the debug extensions, the performance counters, the machine-check exception capability, the machine-check architecture, and the MTRRs. These registers are accessible using the RDMSR and WRMSR instructions. Specific information on some of these new MSRs is provided in the following sections. As with the Pentium processor MSR, the P6 family processor MSRs are not guaranteed to be duplicated or provided in the next generation IA-32 processors.

18.33.2. RDMSR and WRMSR Instructions

The RDMSR (read model-specific register) and WRMSR (write model-specific register) instructions recognize a much larger number of model-specific registers in the P6 family processors. (See “RDMSR—Read from Model Specific Register” and “WRMSR—Write to Model Specific Register” in Chapter 3 of the *IA-32 Intel Architecture Software Developer’s Manual, Volume 2*, for more information about these instructions.)

18.33.3. Memory Type Range Registers

Memory type range registers (MTRRs) are a new feature introduced into the IA-32 in the Pentium Pro processor. MTRRs allow the processor to optimize memory operations for different types of memory, such as RAM, ROM, frame buffer memory, and memory-mapped I/O.

MTRRs are MSRs that contain an internal map of how physical address ranges are mapped to various types of memory. The processor uses this internal memory map to determine the cacheability of various physical memory locations and the optimal method of accessing memory locations. For example, if a memory location is specified in an MTRR as write-through memory, the processor handles accesses to this location as follows. It reads data from that location in lines and caches the read data or maps all writes to that location to the bus and updates the cache to maintain cache coherency. In mapping the physical address space with MTRRs, the processor recognizes five types of memory: uncacheable (UC), uncacheable, speculatable, write-combining (USWC), write-through (WT), write-protected (WP), and writeback (WB).

Earlier IA-32 processors (such as the Intel486 and Pentium processors) used the KEN# (cache enable) pin and external logic to maintain an external memory map and signal cacheable accesses to the processor. The MTRR mechanism simplifies hardware designs by eliminating the KEN# pin and the external logic required to drive it.

See Chapter 9, *Processor Management and Initialization*, and Appendix B, *Model-Specific Registers (MSRs)*, for more information on the MTRRs.

18.33.4. Machine-Check Exception and Architecture

The Pentium processor introduced a new exception called the machine-check exception (#MC, interrupt 18). This exception is used to detect hardware-related errors, such as a parity error on a read cycle.

The P6 family processors extend the types of errors that can be detected and that generate a machine-check exception. It also provides a new machine-check architecture for recording information about a machine-check error and provides extended recovery capability.

The machine-check architecture provides several banks of reporting registers for recording machine-check errors. Each bank of registers is associated with a specific hardware unit in the processor. The primary focus of the machine checks is on bus and interconnect operations; however, checks are also made of translation lookaside buffer (TLB) and cache operations.

The machine-check architecture can correct some errors automatically and allow for reliable restart of instruction execution. It also collects sufficient information for software to use in correcting other machine errors not corrected by hardware.

See Chapter 14, *Machine-Check Architecture*, for more information on the machine-check exception and the machine-check architecture.

18.33.5. Performance-Monitoring Counters

The P6 family and Pentium processors provide two performance-monitoring counters for use in monitoring internal hardware operations. These counters are event counters that can be programmed to count a variety of different types of events, such as the number of instructions decoded, number of interrupts received, or number of cache loads. Appendix A, *Performance-Monitoring Events*, lists all the events that can be counted (Table A-9 for the P6 family processors and Table A-10 for the Pentium processors). The counters are set up, started, and stopped using two MSRs and the RDMSR and WRMSR instructions. For the P6 family processors, the current count for a particular counter can be read using the new RDPMC instruction.

The performance-monitoring counters are useful for debugging programs, optimizing code, diagnosing system failures, or refining hardware designs. See Chapter 15, *Debugging and Performance Monitoring*, for more information on these counters.

18.34. TWO WAYS TO RUN INTEL 286 PROCESSOR TASKS

When porting 16-bit programs to run on 32-bit IA-32 processors, there are two approaches to consider:

- Porting an entire 16-bit software system to a 32-bit processor, complete with the old operating system, loader, and system builder. Here, all tasks will have 16-bit TSSs. The 32-bit processor is being used as if it were a faster version of the 16-bit processor.
- Porting selected 16-bit applications to run in a 32-bit processor environment with a 32-bit operating system, loader, and system builder. Here, the TSSs used to represent 286 tasks should be changed to 32-bit TSSs. It is possible to mix 16 and 32-bit TSSs, but the benefits are small and the problems are great. All tasks in a 32-bit software system should have 32-bit TSSs. It is not necessary to change the 16-bit object modules themselves; TSSs are usually constructed by the operating system, by the loader, or by the system builder. See Chapter 17, *Mixing 16-Bit and 32-Bit Code*, for more detailed information about mixing 16-bit and 32-bit code.

Because the 32-bit processors use the contents of the reserved word of 16-bit segment descriptors, 16-bit programs that place values in this word may not run correctly on the 32-bit processors.



Performance- Monitoring Events

APPENDIX A

PERFORMANCE-MONITORING EVENTS

This appendix contains list of the performance-monitoring events that can be monitored with the IA-32 processors. In the IA-32 processors, the ability to monitor performance events and the events that can be monitored are model specific. Section A.1., *Pentium 4 and Intel Xeon Processor Performance-Monitoring Events*, lists and describes the events that can be monitored with the Pentium 4 processors; Section A.3., *P6 Family Processor Performance-Monitoring Events*, lists and describes the events that can be monitored with the P6 family processors; and Section A.4., *Pentium Processor Performance-Monitoring Events*, lists and describes the events that can be monitored with Pentium processors.

NOTE

These performance-monitoring events are intended to be used as guides for performance tuning. The counter values reported by the performance-monitoring events are approximate and believed to be useful as relative guides for tuning software. Known discrepancies are documented where applicable.

A.1. PENTIUM 4 AND INTEL XEON PROCESSOR PERFORMANCE-MONITORING EVENTS

Tables A-1 and A-2 list the Pentium 4 and Intel Xeon processor performance-monitoring events that can be counted or sampled. Table A-1 lists the non-retirement events, and Table A-2 lists the at-retirement events. Tables A-3, A-4, and A-5 describes three sets of parameters that are available for three of the at-retirement counting events defined in Table A-2. Table A-6 shows which of the non-retirement and at retirement events are logical processor specific (TS) (see Section 15.10.4., *Performance Monitoring Events*) and which are non-logical processor specific (TI).

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
TC_deliver_mode			This event counts the duration (in clock cycles) of the operating modes of the trace cache and decode engine in the processor package. The mode is specified by one or more of the event mask bits.
	ESCR restrictions	MSR_TC_ESCR0 MSR_TC_ESCR1	
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	01H	ESCR[31:25]
	ESCR Event Mask	Bit 0: DD 1: DB 2: DI 3: BD 4: BB 5: BI 6: ID 7: IB	ESCR[24:9], Both logical processors are in deliver mode Logical processor 0 is in deliver mode and logical processor 1 is in build mode Logical processor 0 is in deliver mode and logical processor 1 is either halted, under a machine clear condition or transitioning to a long microcode flow Logical processor 0 is in build mode and logical processor 1 is in deliver mode. Both logical processors are in build mode Logical processor 0 is in build mode and logical processor 1 is either halted, under a machine clear condition or transitioning to a long microcode flow Logical processor 0 is either halted, under a machine clear condition or transitioning to a long microcode flow. Logical processor 1 is in deliver mode. Logical processor 0 is either halted, under a machine clear condition or transitioning to a long microcode flow. Logical processor 1 is in build mode.
	CCCR Select	01H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		If only one logical processor is available from a physical processor package, the event mask should be interpreted as logical processor 1 is halted. Event mask bit 2 was previously known as "DELIVER", bit 5 was previously known as "BUILD".
BPU_fetch_request			This event counts instruction fetch requests of specified request type by the Branch Prediction unit. Specify one or more mask bits to qualify the request type(s).
	ESCR restrictions	MSR_BPU_ESCR0 MSR_BPU_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	03H	ESCR[31:25]
	ESCR Event Mask	Bit 0: TCMISS	ESCR[24:9], Trace cache lookup miss.
	CCCR Select	00H	CCCR[15:13]
	Event Specific Notes		
ITLB_reference			This event counts translations using the Instruction Translation Look-aside Buffer (ITLB).
	ESCR restrictions	MSR_ITLB_ESCR0 MSR_ITLB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	18H	ESCR[31:25]
	ESCR Event Mask	Bit 0: HIT 1: MISS 2: HIT_UC	ESCR[24:9], ITLB hit, ITLB miss, Uncacheable ITLB hit.
	CCCR Select	03H	CCCR[15:13]
	Event Specific Notes		All page references regardless of the page size are looked up as actual 4-KByte pages. Use the page_walk_type event with the ITMISS mask for a more conservative count.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
memory_cancel			This event counts the canceling of various type of request in the Data cache Address Control unit (DAC). Specify one or more mask bits to select the type of requests that are canceled.
	ESCR restrictions	MSR_DAC_ESCR0 MSR_DAC_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	02H	ESCR[31:25]
	ESCR Event Mask	Bit 2: ST_RB_FULL 3: 64K_CONF	ESCR[24:9], Replayed because no store request buffer is available. Conflicts due to 64K aliasing.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		Note: All_CACHE_MISS will include uncacheable memory in its count.
memory_complete			This event counts the completion of a load split, store split, uncacheable (UC) split, or UC load. Specify one or more mask bits to select the operations to be counted.
	ESCR restrictions	MSR_SAAT_ESCR0 MSR_SAAT_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	08H	ESCR[31:25]
	ESCR Event Mask	Bit 0: LSC 1: SSC	ESCR[24:9], Load split completed, excluding UC/WC loads Any split stores completed
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		
load_port_replay			This event counts replayed events at the load port. Specify one or more mask bits to select the cause of the replay.
	ESCR restrictions	MSR_SAAT_ESCR0 MSR_SAAT_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Select	04H	ESCR[31:25]
	ESCR Event Mask	Bit 1: SPLIT_LD	ESCR[24:9], Split load.
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		Must use ESCR1 for at-retirement counting.
store_port_replay			This event counts replayed events at the store port. Specify one or more mask bits to select the cause of the replay.
	ESCR restrictions	MSR_SAAT_ESCR0 MSR_SAAT_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	05H	ESCR[31:25]
	ESCR Event Mask	Bit 1: SPLIT_ST	ESCR[24:9], Split store
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		Must use ESCR1 for at-retirement counting.
MOB_load_replay			This event triggers if the memory order buffer (MOB) caused a load operation to be replayed. Specify one or more mask bits to select the cause of the replay.
	ESCR restrictions	MSR_MOB_ESCR0 MSR_MOB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	03H	ESCR[31:25]
	ESCR Event Mask	Bit 1: NO_STA 3: NO_STD 4: PARTIAL_DATA 5: UNALGN_ADDR	ESCR[24:9], Replayed because of unknown store address, Replayed because of unknown store data, Replayed because of partially overlapped data access between the load and store operations, Replayed because the lower 4 bits of the linear address do not match between the load and store operations.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		
page_walk_type			This event counts various types of page walks that the page miss handler (PMH) performs.
	ESCR restrictions	PMH_CR_ESCR0 PMH_CR_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	01H	ESCR[31:25]
	ESCR Event Mask	Bit 0: DTMISS 1: ITMISS	ESCR[24:9], Page walk for a data TLB miss (either load or store). Page walk for an instruction TLB miss.
	CCCR Select	04H	CCCR[15:13]
	Event Specific Notes		
BSQ_cache_reference			This event counts cache references (2nd level cache or 3rd level cache) as seen by the bus unit. Specify one or more mask bit to select an access according to the access type (read type includes both load and RFO, write type includes writebacks and evictions) and the access result (hit, misses)
	ESCR restrictions	BSU_CR_ESCR0 BSU_CR_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	0CH	ESCR[31:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	0: RD_2ndL_HITS 1: RD_2ndL_HITE 2: RD_2ndL_HITM 3: RD_3rdL_HITS 4: RD_3rdL_HITE 5: RD_3rdL_HITM 8: RD_2ndL_MISS 9: RD_3rdL_MISS 10: WR_2ndL_MISS	ESCR[24:9], Read 2nd level cache hit Shared (includes load and RFO) Read 2nd level cache hit Exclusive (includes load and RFO) Read 2nd level cache hit Modified (includes load and RFO) Read 3rd level cache hit Shared (includes load and RFO) Read 3rd level cache hit Exclusive (includes load and RFO) Read 3rd level cache hit Modified (includes load and RFO) Read 2nd level cache miss (includes load and RFO) Read 3rd level cache miss (includes load and RFO) A Writeback lookup from DAC misses the 2nd level cache (unlikely to happen)
	CCCR Select	07H	CCCR[15:13]
	Event Specific Notes		1: The implementation of this event in current Pentium 4 and Xeon processors treats either a load operation or a request for ownership (RFO) request as a "read" type operation. 2: Currently this event causes both over and undercounting by as much as a factor of two due to an erratum. 3: It is possible for a transaction that is started as a prefetch to change the transaction's internal status, making it no longer a prefetch. or change the access result status (hit, miss) as seen by this event.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
IOQ_allocation			This event counts the various types of transactions on the bus. A count is generated each time a transaction is allocated into the IOQ that matches the specified mask bits. An allocated entry can be a sector (64 bytes) or a chunks of 8 bytes. Note that requests are counted once per retry. The event mask bits constitute 4 bit fields. A transaction type is specified by interpreting the values of each bit field. Specify one or more event mask bits in a bit field to select the value of the bit field. Each field (bits 0-4 are one field) are independent of and can be ORed with the others. The request type field is further combined with bit 5 and 6 to form a binary expression. Bits 7 and 8 form a bit field to specify the memory type of the target address. Bits 13 and 14 form a bit field to specify the source agent of the request. Bit 15 affects read operation only. The event is triggered by evaluating the logical expression: (((Request type) OR Bit 5 OR Bit 6) OR (Memory type)) AND (Source agent).
	ESCR restrictions	MSR_FSB_ESCR0, MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1; ESCR1: 2, 3	
	ESCR Event Select	03H	ESCR[31:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bits 0-4 (single field) 5: ALL_READ 6: ALL_WRITE 7: MEM_UC 8: MEM_WC 9: MEM_WT 10: MEM_WP 11: MEM_WB 13: OWN 14: OTHER 15: PREFETCH	ESCR[24:9] Bus request type (use 00001 for invalid or default) Count read entries Count write entries Count UC memory access entries Count WC memory access entries Count write-through (WT) memory access entries Count write-protected (WP) memory access entries Count WB memory access entries Count all store requests driven by processor, as opposed to other processor or DMA Count all requests driven by other processors or DMA Include HW and SW prefetch requests in the count
	CCCR Select	06H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		1: If PREFETCH bit is cleared, sectors fetched via prefetch are excluded in the counts. If PREFETCH bit is set, all sectors or chunks read are counted. 2: Specify edge trigger in CCCR to avoid double counting. 3: The mapping of interpreted bit field values to transaction types may differ across different processor model implementations of the Pentium 4 processor family. Applications that programs performance monitoring events should use the CPUID instruction to detect processor models when using this event. The logical expression that triggers this event as describe below applies to the CPUID feature flag with the MODEL field encoding equal to 0, 1 and 2. 4a: For Pentium 4 and Xeon Processors starting with CPUID MODEL field encoding equal to 2 or greater, this event is triggered by evaluating the logical expression ((Request type) and (Bit 5 or Bit 6) and (Memory type) and (Source agent)). 4b: For Pentium 4 and Xeon Processors with CPUID MODEL field encoding less than 2, this event is triggered by evaluating the logical expression [((Request type) or Bit 5 or Bit 6) or (Memory type)] and (Source agent) 5: Event mask bits for memory type are ignored if either ALL_READ or ALL_WRITE is specified.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes (continued)		6: This event is known to ignore CPL in early implementations of Pentium 4 and Xeon Processors. Both user requests and OS requests are included in the count. This behavior is fixed starting with Pentium 4 and Xeon Processors with CPUID signature 0xf27, i.e. Family 15, Model 2, Stepping 7. 7: For write-through (WT) and write-protected (WP) memory types, this event counts reads as the number of 64-byte sectors. Writes are counted by individual chunks. 8: For uncacheable (UC) memory types, this event counts the number of 8-byte chunks allocated. 9: For Pentium 4 and Xeon Processors with CPUID Signature less than 0xf27, only MSR_FSB_ESCR0 is available.
IOQ_active_entries			This event counts the number of entries (clipped at 15) in the IOQ that are active. An allocated entry can be a sector (64 bytes) or a chunks of 8 bytes. This event must be programmed in conjunction with IOQ_allocation. Specify one or more event mask bits to select the transactions that is counted.
	ESCR restrictions	MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR1: 2, 3	
	ESCR Event Select	01AH	ESCR[30:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bits 0-4 (single field) 5: ALL_READ 6: ALL_WRITE 7: MEM_UC 8: MEM_WC 9: MEM_WT 10: MEM_WP 11: MEM_WB 13: OWN 14: OTHER 15: PREFETCH	ESCR[24:9], Bus request type (use 00001 for invalid or default), Count read entries, Count write entries, Count UC memory access entries, Count WC memory access entries, Count write-through (WT) memory access entries Count write-protected (WP) memory access entries Count WB memory access entries Count all store requests driven by processor, as opposed to other processor or DMA, Count all requests driven by other processors or DMA, Include HW and SW prefetch requests in the count.
	CCCR Select	06H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		1: Specified desired mask bits in ESCR0 and ESCR1. 2: See the ioq_allocation event for descriptions of the mask bits. 3: Edge triggering should not be used when counting cycles. 4: The mapping of interpreted bit field values to transaction types may differ across different processor model implementations of the Pentium 4 processor family. Applications that programs performance monitoring events should use the CPUID instruction to detect processor models when using this event. The logical expression that triggers this event as describe below applies to the CPUID feature flag with the MODEL field encoding equal to 0, 1 and 2 5: For Pentium 4 and Xeon Processors starting with CPUID MODEL field encoding equal to 2 or greater, this event is triggered by evaluating the logical expression ((Request type) and (Bit 5 or Bit 6) and (Memory type) and (Source agent)). 6a: For Pentium 4 and Xeon Processors starting with CPUID MODEL field encoding less than 2, this event is triggered by evaluating the logical expression [((Request type) or Bit 5 or Bit 6) or (Memory type)] and (Source agent). 6b: Event mask bits for memory type are ignored if either ALL_READ or ALL_WRITE is specified. 7: This event is known to ignore CPL in the current implementations of Pentium 4 and Xeon Processors Both user requests and OS requests are included in the count. 8: An allocated entry can be a full line (64 bytes) or in individual chunks of 8 bytes

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
FSB_data_activity			This event increments once for each DRDY or DBSY event that occurs on the front side bus. The event allows selection of a specific DRDY or DBSY event.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	17H	ESCR[31:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bit 0: DRDY_DRV 1: DRDY_OWN 2: DRDY_OTHER 3: DBSY_DRV 4: DBSY_OWN 5:DBSY_OTHER	ESCR[24:9], Count when this processor drives data onto the bus - includes writes and implicit writebacks. Asserted two processor clock cycles for partial writes and 4 processor clocks (usually in consecutive bus clocks) for full line writes. Count when this processor reads data from the bus - includes loads and some PIC transactions. Asserted two processor clock cycles for partial reads and 4 processor clocks (usually in consecutive bus clocks) for full line reads. Count DRDY event that we drive Count DRDY event sampled that we own Count when data is on the bus but not being sampled by the processor. It may or may not be being driven by this processor. Asserted two processor clock cycles for partial transactions and 4 processor clocks (usually in consecutive bus clocks) for full line transactions. Count when this processor reserves the bus for use in the next bus cycle in order to drive data. Asserted for two processor clock cycles for full line writes and not at all for partial line writes. May be asserted multiple times (in consecutive bus clocks) if we stall the bus waiting for a cache lock to complete. Count when some agent reserves the bus for use in the next bus cycle to drive data that this processor will sample. Asserted for two processor clock cycles for full line writes and not at all for partial line writes. May be asserted multiple times (all one bus clock apart) if we stall the bus for some reason. Count when some agent reserves the bus for use in the next bus cycle to drive data that this processor will NOT sample. It may or may not be being driven by this processor. Asserted two processor clock cycles for partial transactions and 4 processor clocks (usually in consecutive bus clocks) for full line transactions.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	CCCR Select	06H	CCCR[15:13]
	Event Specific Notes		1: Specify edge trigger in the CCCR MSR to avoid double counting. 2: DRDY_OWN and DRDY_OTHER are mutually exclusive; similarly for DBSY_OWN and DBSY_OTHER.
BSQ_allocation			This event counts allocations in the Bus Sequence Unit (BSQ) according to the specified mask bit encoding. The event mask bits consist of four sub-groups: Request type, Request length, Memory type, and a sub-group consisting mostly of independent bits (bits 5, 6, 7, 8, 9, and 10). Specify an encoding for each sub-group.
	ESCR restrictions	MSR_BSU_ESCR0	
	Counter numbers per ESCR	ESCR0: 0, 1	
	ESCR Event Select	05H	ESCR[31:25]
	ESCR Event Mask	Bit 0: REQ_TYPE0 1: REQ_TYPE1 2: REQ_LEN0 3: REQ_LEN1 5: REQ_IO_TYPE 6: REQ_LOCK_TYPE 7: REQ_CACHE_TYPE 8: REQ_SPLIT_TYPE 9: REQ_DEM_TYPE 10: REQ_ORD_TYPE 11: MEM_TYPE0 12: MEM_TYPE1 13: MEM_TYPE2	ESCR[24:9], Request type encoding (bit 0 and 1) are: 0 - Read (excludes read invalidate). 1 - Read invalidate. 2 - Write (other than writebacks). 3 - Writeback (evicted from cache). (public) Request length encoding (bit 2, 3) are: 0 - 0 chunks, 1 - 1 chunk, 3 - 8 chunks. Request type is input or output. Request type is bus lock. Request type is cacheable Request type is a bus 8-byte chunk split across 8-byte boundary. Request type is a demand if set Request type is HW.SW prefetch if 0. Request is an ordered type. Memory type encoding (bit 11-13) are: 0 - UC, 1 - USWC, 4 - WT, 5 - WP, 6 - WB
	CCCR Select	07H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		1: Specify edge trigger in CCCR to avoid double counting. 2: A writebacks to 3rd level cache from 2nd level cache counts as a separate entry, this is in additional to the entry allocated for a request to the bus. 3: A read request to WB memory type results in a request to the 64-byte sector, containing the target address, followed by a prefetch request to an adjacent sector. 4: For Pentium 4 and Xeon processors with CPUID model encoding value equals to 0 and 1, an allocated BSQ entry includes both the demand sector and prefetched 2nd sector. 5: An allocated BSQ entry for a data chunk is any request less than 64 bytes. 6a: This event may undercount for requests of split type transactions if the data address straddled across modulo-64 byte boundary. 6b: This event may undercount for requests of read request of 16-byte operands from WC or UC address. 6c: This event may undercount WC partial requests originated from store operands that are dwords.
bsq_active_entries			This event represents the number of BSQ entries (clipped at 15) currently active (valid) which meet the subevent mask criteria during allocation in the BSQ. Active request entries are allocated on the BSQ until de-allocated. De-allocation of an entry does not necessarily imply the request is filled. This event must be programmed in conjunction with BSQ_allocation. Specify one or more event mask bits to select the transactions that is counted.
	ESCR restrictions	ESCR1	
	Counter numbers per ESCR	ESCR1: 2, 3	
	ESCR Event Select	06H	ESCR[30:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bit	ESCR[24:9]
	CCCR Select	07H	CCCR[15:13]
	Event Specific Notes		1: Specified desired mask bits in ESCR0 and ESCR1. 2: See the BSQ_allocation event for descriptions of the mask bits. / 3: Edge triggering should not be used when counting cycles. 4: This event can be used to estimate the latency of a transaction from allocation to de-allocation in the BSQ. The latency observed by BSQ_allocation includes the latency of FSB, plus additional overhead. The additional overhead may include the time it takes to issue two requests (the sector by demand and the adjacent sector via prefetch). Since adjacent sector prefetches have lower priority than demand fetches, on a heavily used system there is a high probability that the adjacent sector prefetch will have to wait until the next bus arbitration.
SSE_input_assist			This event counts the number of times an assist is requested to handle problems with input operands for SSE and SSE2 operations, most notably denormal source operands when the DAZ bit is not set. Set bit 15 of the event mask to use this event.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	34H	ESCR[31:25]
	ESCR Event Mask	15: ALL	ESCR[24:9], Count assists for all SSE and SSE2 μ ops
	CCCR Select	01H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		1: Not all requests for assists are actually taken. This event is known to overcount in that it counts requests for assists from instructions on the non-retired path that do not incur a performance penalty. An assist is actually taken only for non-bogus μops. Any appreciable counts for this event are an indication that the DAZ or FTZ bit should be set and/or the source code should be changed to eliminate the condition. 2: Two common situations for an SSE or SSE2 operation needing an assist are: (1) when a denormal constant is used as an input and the Denormals-Are-Zero (DAZ) mode is not set, (2) when the input operand uses the underflowed result of a previous SSE or SSE2 operation and neither the DAZ nor Flush-To-Zero (FTZ) modes are set. Enabling the DAZ mode prevents SSE and SSE2 operations from needing assists in the first situation. Enabling the FTZ mode prevents SSE and SSE2 operations from needing assists in the second situation.
packed_SP_uop			This event increments for each packed single-precision μ op, specified through the event mask for detection.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	08H	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on packed single-precision operands
	CCCR Select	01H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		If an instruction contains more than one packed SP μ ops, each packed SP μ op that is specified by the event mask will be counted. Note that this metric counts instances of packed memory μ ops in a repeat move string.
packed_DP_uop			This event increments for each packed double-precision μ op, specified through the event mask for detection.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	0CH	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on packed double-precision operands
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		If an instruction contains more than one packed DP μ ops, each packed DP μ op that is specified by the event mask will be counted.
scalar_SP_uop			This event increments for each scalar single-precision μ op, specified through the event mask for detection.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	0AH	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on scalar single-precision operands
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		If an instruction contains more than one scalar SP μ ops, each scalar SP μ op that is specified by the event mask will be counted.

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
scalar_DP_uop			This event increments for each scalar double-precision μ op, specified through the event mask for detection.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	0EH	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on scalar double-precision operands.
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		If an instruction contains more than one scalar DP μ ops, each scalar DP μ op that is specified by the event mask will be counted.
64bit_MMX_uop			This event increments for each MMX instruction, which operate on 64 bit SIMD operands.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	02H	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on 64 bit SIMD integer operands in memory or MMX registers.
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		If an instruction contains more than one 64 bit MMX μ ops, each 64 bit MMX μ op that is specified by the event mask will be counted.
128bit_MMX_uop			This event increments for each integer SIMD SSE2 instructions, which operate on 128 bit SIMD operands.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Select	1AH	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all μ ops operating on 128 bit SIMD integer operands in memory or XMM registers.
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		If an instruction contains more than one 128 bit MMX μ ops, each 128 bit MMX μ op that is specified by the event mask will be counted.
x87_FP_uop			This event increments for each x87 floating-point μ op, specified through the event mask for detection.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	04H	ESCR[31:25]
	ESCR Event Mask	Bit 15: ALL	ESCR[24:9], Count all x87 FP μ ops.
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		1: If an instruction contains more than one x87 FP μ ops, each x87 FP μ op that is specified by the event mask will be counted. 2: This event does not count x87 FP μ op for load, store, move between registers.
x87_SIMD_moves_uop			This event increments for each x87 FPU, MMX, SSE or SSE2 μ op related to load data, store data, or register-to-register moves, and is specified through the event mask for detection. These μ ops are dispatched to port 0 or port 2 at runtime.
	ESCR restrictions	MSR_FIRM_ESCR0 MSR_FIRM_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	2EH	ESCR[31:25]
	CCCR Select	01H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bit 3: ALLP0 4: ALLP2	ESCR[24:9], Count all x87/SIMD store/moves μops. Count all x87/SIMD load μops.
	Event Specific Notes		This event does not count Integer load/store/moves μops. This event does count for a number of flows, including input assists.
global_power_events			This event accumulates the time during which a processor is not stopped.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	013H	ESCR[31:25]
	ESCR Event Mask	Bit 0: Running	ESCR[24:9] The processor is active (includes the handling of HLT STPCLK and throttling.
	CCCR Select	06H	CCCR[15:13]
	Event Specific Notes		
tc_ms_xfer			This event counts the number of times that uop delivery changed from TC to MS ROM.
	ESCR restrictions	MSR_MS_ESCR0 MSR_MS_ESCR1	
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	05H	ESCR[31:25]
	ESCR Event Mask	Bit 0: CISC	ESCR[24:9] A TC to MS transfer occurred.
	CCCR Select	0H	CCCR[15:13]
	Event Specific Notes		
uop_queue_writes			This event counts the number of valid uops written to the uop queue. Specify one or more mask bits to select the source type of writes.
	ESCR restrictions	MSR_MS_ESCR0 MSR_MS_ESCR1	

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	09H	ESCR[31:25]
	ESCR Event Mask	Bit 0: FROM_TC_BUILD 1: FROM_TC_DELIVER 2: FROM_ROM	ESCR[24:9] The uops being written are from TC build mode. The uops being written are from TC deliver mode. The uops being written are from microcode ROM.
	CCCR Select	0H	CCCR[15:13]
	Event Specific Notes		
retired_mispred_branch_type			This event counts retiring mispredicted branches by type.
	ESCR restrictions	MSR_TBPU_ESCR0 MSR_TBPU_ESCR1	
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	05H	ESCR[30:25]
	ESCR Event Mask	Bit 1: CONDITIONAL 2: CALL 3: RETURN 4: INDIRECT	ESCR[24:9], Conditional jumps. Indirect call branches. Return branches. Returns, indirect calls, or indirect jumps.
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		This event may overcount conditional branches if: a: Mispredictions cause the trace cache and delivery engine to build new traces, b: When the processor's pipeline is being cleared.
retired_branch_type			This event counts retiring branches by type. Specify one or more mask bits to qualify the branch by its type
	ESCR restrictions	MSR_TBPU_ESCR0 MSR_TBPU_ESCR1	
	Counter numbers per ESCR	ESCR0: 4, 5 ESCR1: 6, 7	
	ESCR Event Select	04H	ESCR[30:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bit 1: CONDITIONAL 2: CALL 3: RETURN 4: INDIRECT	ESCR[24:9], Conditional jumps. Direct or indirect calls. Return branches. Returns, indirect calls, or indirect jumps.
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		This event may overcount conditional branches if (a) mispredictions cause the trace cache and delivery engine to build new traces, (b) when the processor's pipeline is being cleared.
resource_stall			This event monitors the occurrence or latency of stalls in the Allocator.
	ESCR restrictions	MSR_ALF_ESCR0 MSR_ALF_ESCR1	
	Counter numbers per ESCR	ESCR0: 12, 13, 16 ESCR1: 14, 15, 17	
	ESCR Event Select	01H	ESCR[30:25]
	Event Masks	Bit 5: SBFULL	ESCR[24:9] A Stall due to lack of store buffers
	CCCR Select	01H	CCCR[15:13]
	Event Specific Notes		This event may not be supported in all models of the processor family.
WC_Buffer			This event counts Write Combining Buffer operations that are selected by the event mask.
	ESCR restrictions	MSR_DAC_ESCR0 MSR_DAC_ESCR1	
	Counter numbers per ESCR	ESCR0: 8, 9 ESCR1: 10, 11	
	ESCR Event Select	05H	ESCR[30:25]
	Event Masks	Bit 0: WCB_EVICTS 1: WCB_FULL_EVICT	ESCR[24:9] WC Buffer evictions of all causes WC Buffer eviction: no WC buffer is available
	CCCR Select	05H	CCCR[15:13]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		This event is useful for detecting the subset of 64K aliasing cases that are more costly (i.e. 64K aliasing cases involving stores) as long as there are no significant contributions due to write combining buffer full or hit-modified conditions.
b2b_cycles			This event can be configured to count the number back-to-back bus cycles using sub-event mask bits 1 through 6.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	016H	ESCR[30:25]
	Event Masks	Bit	ESCR[24:9]
	CCCR Select	03H	CCCR[15:13]
	Event Specific Notes		This event may not be supported in all models of the processor family.
bnr			This event can be configured to count bus not ready conditions using sub-event mask bits 0 through 2.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	08H	ESCR[30:25]
	Event Masks	Bit	ESCR[24:9]
	CCCR Select	03H	CCCR[15:13]
	Event Specific Notes		This event may not be supported in all models of the processor family.
snoop			This event can be configured to count snoop hit modified bus traffic using sub-event mask bits 2, 6 and 7.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	06H	ESCR[30:25]

Table A-1. Pentium 4 and Intel Xeon Processor Performance Monitoring Events for Non-Retirement Counting (Contd.)

Event Name	Event Parameters	Parameter Value	Description
response	Event Masks	Bit	ESCR[24:9]
	CCCR Select	03H	CCCR[15:13]
	Event Specific Notes		This event may not be supported in all models of the processor family.
			This event can be configured to count different types of responses using sub-event mask bits 1,2, 8, and 9.
	ESCR restrictions	MSR_FSB_ESCR0 MSR_FSB_ESCR1	
	Counter numbers per ESCR	ESCR0: 0, 1 ESCR1: 2, 3	
	ESCR Event Select	04H	ESCR[30:25]
	Event Masks	Bit	ESCR[24:9]
	CCCR Select	03H	CCCR[15:13]
	Event Specific Notes		This event may not be supported in all models of the processor family.

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
front_end_event			This event counts the retirement of tagged μ ops, which are specified through the front-end tagging mechanism. The event mask specifies bogus or non-bogus μ ops.
	ESCR restrictions	MSR_CRU_ESCR2, MSR_CRU_ESCR3	
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	
	ESCR Event Select	08H	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUS 1: BOGUS	ESCR[24:9], The marked μ ops are not bogus. The marked μ ops are bogus.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		
	Can Support PEBS	Yes	
	Require Additional MSRs for tagging	Selected ESCRs and/or MSR_TC_PRECISE_EVENT	See list of metrics supported by Front_end tagging in Table A-3
execution_event			This event counts the retirement of tagged μ ops, which are specified through the execution tagging mechanism. The event mask allows from one to four types of μ ops to be specified as either bogus or non-bogus μ ops to be tagged.
	ESCR restrictions	MSR_CRU_ESCR2, MSR_CRU_ESCR3	
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	
	ESCR Event Select	0CH	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUS0 1: NBOGUS1 2: NBOGUS2 3: NBOGUS3 4: BOGUS0 5: BOGUS1 6: BOGUS2 7: BOGUS3	ESCR[24:9] The marked μ ops are not bogus. The marked μ ops are not bogus. The marked μ ops are not bogus. The marked μ ops are not bogus. The marked μ ops are bogus. The marked μ ops are bogus. The marked μ ops are bogus. The marked μ ops are bogus.
	CCCR Select	05H	CCCR[15:13]

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
	Event Specific Notes		Each of the 4 slots to specify the bogus/non-bogus μ ops must be coordinated with the 4 TagValue bits in the ESCR., e.g. NBOGUS0 must accompany a '1' in the lowest bit of the TagValue field in ESCR, NBOGUS1 must accompany a '1' in the next but lowest bit of the TagValue field.
	Can Support PEBS	Yes	
	Require Additional MSRs for tagging	An ESCR for an upstream event	See list of metrics supported by execution tagging in Table A-4
replay_event			This event counts the retirement of tagged μ ops, which are specified through the replay tagging mechanism. The event mask specifies bogus or non-bogus μ ops.
	ESCR restrictions	MSR_CRU_ESCR2, MSR_CRU_ESCR3	
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	
	ESCR Event Select	09H	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUS 1: BOGUS	ESCR[24:9], The marked μ ops are not bogus. The marked μ ops are bogus.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		Supports counting tagged μ ops with additional MSRs
	Can Support PEBS	Yes	
	Require Additional MSRs for tagging	IA32_PEBS_ENABLE, MSR_PEBS_MATRIX_VERT, Selected ESCR	See list of metrics supported by replay tagging in Table A-5

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
instr_retired			1: This event counts instructions that are retired during a clock cycle. Mask bits specify bogus or non-bogus (and whether they are tagged via the front-end tagging mechanism). 2: The event count may vary depending on the microarchitecture state of the processor when the event is enabled. 3: The event may count more than once for some IA-32 instructions with complex uop flows and were interrupted before retirement.
	ESCR restrictions	MSR_CRU_ESCR0, MSR_CRU_ESCR1	
	Counter numbers per ESCR	ESCR0: 12, 13, 16 ESCR1: 14, 15, 17	
	ESCR Event Select	02H	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUSNTAG 1: NBOGUSTAG 2: BOGUSNTAG 3: BOGUSTAG	ESCR[24:9], Non-bogus instructions that are not tagged. Non-bogus instructions that are tagged. Bogus instructions that are not tagged. Bogus instructions that are tagged.
	CCCR Select	04H	CCCR[15:13]
	Event Specific Notes		1: The event count may vary depending on the microarchitectural states of the processor when the event detection is enabled. 2: The event may count more than once for some IA-32 instructions with complex uop flows and were interrupted before retirement.
	Can Support PEBS	No	
uops_retired			This event counts μ ops that are retired during a clock cycle. Mask bits specify bogus or non-bogus.
	ESCR restrictions	MSR_CRU_ESCR0, MSR_CRU_ESCR1	

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
	Counter numbers per ESCR	ESCR0: 12, 13, 16 ESCR1: 14, 15, 17	
	ESCR Event Select	01H	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUS 1: BOGUS	ESCR[24:9], The marked μ ops are not bogus. The marked μ ops are bogus.
	CCCR Select	04H	CCCR[15:13]
	Event Specific Notes		P6: EMON_UOPS_RETIRE
	Can Support PEBS	No	
uop_type			This event is used in conjunction with the front-end at-retirement mechanism to tag load and store μ ops.
	ESCR restrictions	MSR_RAT_ESCR0, MSR_RAT_ESCR1	
	Counter numbers per ESCR	ESCR0: 12, 13, 16 ESCR1: 14, 15, 17	
	ESCR Event Select	02H	ESCR[31:25]
	ESCR Event Mask	Bit 1: TAGLOADS 2: TAGSTORES	ESCR[24:9], The μ op is a load operation. The μ op is a store operation.
	CCCR Select	02H	CCCR[15:13]
	Event Specific Notes		Setting the TAGLOADS and TAGSTORES mask bits does not cause a counter to increment. They are only used to tag uops.
	Can Support PEBS	No	
branch_retired			This event counts the retirement of a branch. Specify one or more mask bits to select any combination of taken, not-taken, predicted and mispredicted.
	ESCR restrictions	MSR_CRU_ESCR2 MSR_CRU_ESCR3	See Table 15-4 for the addresses of the ESCR MSRs
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	The counter numbers associated with each ESCR are provided. The performance counters and corresponding CCCRs can be obtained from Table 15-4.
	ESCR Event Select	06H	ESCR[31:25]

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
	ESCR Event Mask	Bit 0: MMNP 1: MMNM 2: MMTP 3: MMTM	ESCR[24:9], Branch Not-taken Predicted, Branch Not-taken Mispredicted, Branch Taken Predicted, Branch Taken Mispredicted.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		P6: EMON_BR_INST_RETIRED
	Can Support PEBS	No	
mispred_branch_ retired			This event represents the retirement of mispredicted IA-32 branch instructions.
	ESCR restrictions	MSR_CRU_ESCR0 MSR_CRU_ESCR1	
	Counter numbers per ESCR	ESCR0: 12, 13, 16 ESCR1: 14, 15, 17	
	ESCR Event Select	03H	ESCR[31:25]
	ESCR Event Mask	Bit 0: NBOGUS	ESCR[24:9] The retired instruction is not bogus
	CCCR Select	04H	CCCR[15:13]
	Event Specific Notes		
	Can Support PEBS	No	
x87_assist			This event counts the retirement of x87 instructions that required special handling. Specifies one or more event mask bits to select the type of assistance.
	ESCR restrictions	MSR_CRU_ESCR2 MSR_CRU_ESCR3	
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	
	ESCR Event Select	03H	ESCR[31:25]
	ESCR Event Mask	Bit 0: FPSU 1: FPSO 2: POAO 3: POAU 4: PREA	ESCR[24:9], Handle FP stack underflow, Handle FP stack overflow, Handle x87 output overflow Handle x87 output underflow Handle x87 input assist
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		
	Can Support PEBS	No	

Table A-2. Pentium 4 and Intel Xeon Processor Performance Monitoring Events For At-Retirement Counting

Event Name	Event Parameters	Parameter Value	Description
machine_clear			This event increments according to the mask bit specified while the entire pipeline of the machine is cleared. Specify one of the mask bit to select the cause.
	ESCR restrictions	MSR_CRU_ESCR2 MSR_CRU_ESCR3	
	Counter numbers per ESCR	ESCR2: 12, 13, 16 ESCR3: 14, 15, 17	
	ESCR Event Select	02H	ESCR[31:25]
	ESCR Event Mask	Bit 0: CLEAR 2: MOCLEAR 3: SMCLEAR	ESCR[24:9] Counts for a portion of the many cycles while the machine is cleared for any cause. Use Edge triggering for this bit only to get a count of occurrence versus a duration. Increments each time the machine is cleared due to memory ordering issues. Increments each time the machine is cleared due to self-modifying code issues.
	CCCR Select	05H	CCCR[15:13]
	Event Specific Notes		
	Can Support PEBS	No	

Table A-3. List of Metrics Available for Front_end Tagging (For Front_end Event Only)

Front-end metric ¹	MSR_ TC_PRECISE_EVENT NT MSR Bit field	Additional MSR	Event mask value for Front_end_event
memory_loads	None	Set TAGLOADS bit in ESCR corresponding to event Uop_Type	NBOGUS
memory_stores	None	Set TAGSTORES bit in the ESCR corresponding to event Uop_Type	NBOGUS

NOTES

1. There may be some undercounting of front end events when there is an overflow or underflow of the floating point stack.

Table A-4. List of Metrics Available for Execution Tagging (For Execution Event only)

Execution metric	Upstream ESCR	TagValue in Upstream ESCR	Event mask value for execution_event
packed_SP_retired	Set ALL bit in event mask, TagUop bit in ESCR of packed_SP_uop,	1	NBOGUS0
packed_DP_retired	Set ALL bit in event mask, TagUop bit in ESCR of packed_DP_uop,	1	NBOGUS0
scalar_SP_retired	Set ALL bit in event mask, TagUop bit in ESCR of scalar_SP_uop,	1	NBOGUS0
scalar_DP_retired	Set ALL bit in event mask, TagUop bit in ESCR of scalar_DP_uop,	1	NBOGUS0
128_bit_MMX_retired	Set ALL bit in event mask, TagUop bit in ESCR of 128_bit_MMX_uop,	1	NBOGUS0
64_bit_MMX_retired	Set ALL bit in event mask, TagUop bit in ESCR of 64_bit_MMX_uop,	1	NBOGUS0
X87_FP_retired	Set ALL bit in event mask, TagUop bit in ESCR of x87_FP_uop,	1	NBOGUS0
X87_SIMD_memory_moves_retired	Set ALLP0, ALLP2 bits in event mask, TagUop bit in ESCR of X87_SIMD_moves_uop,	1	NBOGUS0

Table A-5. List of Metrics Available for ReplayTagging (For Replay Event only)

Replay metric¹	IA32_PEBS_ENABLE Field to Set	MSR_PEBS_MATRIX_VERT Bit Field to Set	Additional MSR/ Event	Event Mask Value for Replay_event
1stL_cache_load_miss_retired	Bit 0, Bit 24, Bit 25	Bit 0	None	NBOGUS
2ndL_cache_load_miss_retired ²	Bit 1, Bit 24, Bit 25	Bit 0	None	NBOGUS
DTLB_load_miss_retired	Bit 2, Bit 24, Bit 25	Bit 0	None	NBOGUS
DTLB_store_miss_retired	Bit 2, Bit 24, Bit 25	Bit 1	None	NBOGUS
DTLB_all_miss_retired	Bit 2, Bit 24, Bit 25	Bit 0, Bit 1	None	NBOGUS
MOB_load_replay_retired ³	Bit 9, Bit 24, Bit 25	Bit 0	Select MOB_load_replay event and set PARTIAL_DATA and UNALGN_ADDR bit	NBOGUS
split_load_retired	Bit 10, Bit 24, Bit 25	Bit 0	Select load_port_replay event with the MSR_SAAT_ESCR1 MSR and set the SPLIT_LD mask bit	NBOGUS
split_store_retired	Bit 10, Bit 24, Bit 25	Bit 1	Select store_port_replay event with the MSR_SAAT_ESCR0 MSR and set the SPLIT_ST mask bit	NBOGUS

NOTES

1. Certain kinds of μ ops cannot be tagged. These include I/O operations, UC and locked accesses, returns, and far transfers.
2. 2nd-level misses retired does not count all 2nd-level misses. It only includes those references that are found to be misses by the fast detection logic and not those that are later found to be misses.
3. While there are several causes for a MOB replay, the event counted with this event mask setting is the case where the data from a load that would otherwise be forwarded is not an aligned subset of the data from a preceding store.

Table A-6. Event Mask Qualification for Logical Processors

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
Non-Retirement	BPU_fetch_request	Bit 0: TCMISS	TS
Non-Retirement	BSQ_allocation	Bit	
		0: REQ_TYPE0	TS
		1: REQ_TYPE1	TS
		2: REQ_LEN0	TS
		3: REQ_LEN1	TS
		5: REQ_IO_TYPE	TS
		6: REQ_LOCK_TYPE	TS
		7: REQ_CACHE_TYPE	TS
		8: REQ_SPLIT_TYPE	TS
		9: REQ_DEM_TYPE	TS
		10: REQ_ORD_TYPE	TS
		11: MEM_TYPE0	TS
		12: MEM_TYPE1	TS
		13: MEM_TYPE2	TS
Non-Retirement	BSQ_cache_reference	Bit	
		0: RD_2ndL_HITS	TS
		1: RD_2ndL_HITE	TS
		2: RD_2ndL_HITM	TS
		3: RD_3rdL_HITS	TS
		4: RD_3rdL_HITE	TS
		5: RD_3rdL_HITM	TS
		6: WR_2ndL_HIT	TS
		7: WR_3rdL_HIT	TS
		8: RD_2ndL_MISS	TS
		9: RD_3rdL_MISS	TS
		10: WR_2ndL_MISS	TS
		11: WR_3rdL_MISS	TS
Non-Retirement	memory_cancel	Bit	
		2: ST_RB_FULL	TS
		3: 64K_CONF	TS
Non-Retirement	SSE_input_assist	Bit 15: ALL	TI
Non-Retirement	64bit_MMX_uop	Bit 15: ALL	TI
Non-Retirement	packed_DP_uop	Bit 15: ALL	TI

Table A-6. Event Mask Qualification for Logical Processors (Contd.)

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
Non-Retirement	packed_SP_uop	Bit 15: ALL	TI
Non-Retirement	scalar_DP_uop	Bit 15: ALL	TI
Non-Retirement	scalar_SP_uop	Bit 15: ALL	TI
Non-Retirement	128bit_MMX_uop	Bit 15: ALL	TI
Non-Retirement	x87_FP_uop	Bit 15: ALL	TI
Non-Retirement	x87_SIMD_moves_uop	Bit	
		3: ALLP0	TI
Non-Retirement	FSB_data_activity	4: ALLP2	TI
		Bit	
		0: DRDY_DRV	TI
		1: DRDY_OWN	TI
		2: DRDY_OTHER	TI
		3: DBSY_DRV	TI
Non-Retirement	IOQ_allocation	4: DBSY_OWN	TI
		5: DBSY_OTHER	TI
		Bit	
		0: ReqA0	TS
		1: ReqA1	TS
		2: ReqA2	TS
		3: ReqA3	TS
		4: ReqA4	TS
		5: ALL_READ	TS
		6: ALL_WRITE	TS
		7: MEM_UC	TS
		8: MEM_WC	TS
		9: MEM_WT	TS
		10: MEM_WP	TS
		11: MEM_WB	TS
		13: OWN	TS
		14: OTHER	TS
		15: PREFETCH	TS

Table A-6. Event Mask Qualification for Logical Processors (Contd.)

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
Non-Retirement	IOQ_active_entries	Bit	TS
		0: ReqA0	
		1: ReqA1	TS
		2: ReqA2	TS
		3: ReqA3	TS
		4: ReqA4	TS
		5: ALL_READ	TS
		6: ALL_WRITE	TS
		7: MEM_UC	TS
		8: MEM_WC	TS
		9: MEM_WT	TS
		10: MEM_WP	TS
		11: MEM_WB	TS
		13: OWN	TS
		14: OTHER	TS
		15: PREFETCH	TS
Non-Retirement	global_power_events	Bit 0: RUNNING	TS
Non-Retirement	ITLB_reference	Bit	TS
		0: HIT	TS
		1: MISS	TS
Non-Retirement	MOB_load_replay	2: HIT_UC	TS
		Bit	TS
		1: NO_STA	TS
		3: NO_STD	TS
Non-Retirement	page_walk_type	4: PARTIAL_DATA	TS
		5: UNALGN_ADDR	TS
		Bit	TI
		0: DTMISS	TI
Non-Retirement	uop_type	1: ITMISS	TI
		Bit	TS
		1: TAGLOADS	TS
		2: TAGSTORES	TS
Non-Retirement	load_port_replay	Bit 1: SPLIT_LD	TS
Non-Retirement	store_port_replay	Bit 1: SPLIT_ST	TS

Table A-6. Event Mask Qualification for Logical Processors (Contd.)

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
Non-Retirement	memory_complete	Bit	
		0: LSC	TS
		1: SSC	TS
		2: USC	TS
		3: ULC	TS
Non-Retirement	retired_mispred_branch_type	Bit	
		0: UNCONDITIONAL	TS
		1: CONDITIONAL	TS
		2: CALL	TS
		3: RETURN	TS
		4: INDIRECT	TS
Non-Retirement	retired_branch_type	Bit	
		0: UNCONDITIONAL	TS
		1: CONDITIONAL	TS
		2: CALL	TS
		3: RETURN	TS
		4: INDIRECT	TS
Non-Retirement	tc_ms_xfer	Bit	
		0: CISC	TS
Non-Retirement	TC_deliver_mode	Bit	
		0: DD	TI
		1: DB	TI
		2: DI	TI
		3: BD	TI
		4: BB	TI
		5: BI	TI
		6: ID	TI
		7: IB	TI
Non-Retirement	uop_queue_writes	Bit	
		0: FROM_TC_BUILD	TS
		1: FROM_TC_DELIVER	TS
		2: FROM_ROM	TS
Non-Retirement	resource_stall	Bit 5: SBFULL	TS

Table A-6. Event Mask Qualification for Logical Processors (Contd.)

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
Non-Retirement	WC_Buffer	Bit	TI
		0: WCB_EVICTS	TI
		1: WCB_FULL_EVICT	TI
		2: WCB_HITM_EVICT	TI
At Retirement	instr_retired	Bit	TS
		0: NBOGUSNTAG	TS
		1: NBOGUSTAG	TS
		2: BOGUSNTAG	TS
At Retirement	machine_clear	3: BOGUSTAG	TS
		Bit	TS
		0: CLEAR	TS
		2: MOCLEAR	TS
At Retirement	front_end_event	6: SMCLEAR	TS
		7: OTCLEAR	TS
		Bit	TS
		0: NBOGUS	TS
At Retirement	replay_event	1: BOGUS	TS
		Bit	TS
At Retirement	execution_event	0: NBOGUS	TS
		1: BOGUS	TS
At Retirement	execution_event	Bit	TS
		0: NONBOGUS0	TS
		1: NONBOGUS1	TS
		2: NONBOGUS2	TS
		3: NONBOGUS3	TS
		4: BOGUS0	TS
		5: BOGUS1	TS
		6: BOGUS2	TS
		7: BOGUS3	TS
At Retirement	x87_assist	Bit	TS
		0: FPSU	TS
		1: FPSO	TS
		2: POAO	TS
		3: POAU	TS
		4: PREA	TS

Table A-6. Event Mask Qualification for Logical Processors (Contd.)

Event Type	Event Name	Event Masks, ESCR[24:9]	TS or TI
At Retirement	branch_retired	Bit 0: MMNP	TS
		1: MMNM	TS
		2: MMTP	TS
		3: MMTM	TS
At Retirement	mispred_branch_retired	Bit 0: NBOGUS	TS
At Retirement	uops_retired	Bit 0: NBOGUS	TS
		1: BOGUS	TS

A.2. PERFORMANCE MONITORING EVENTS FOR INTEL® PENTIUM® M PROCESSORS

The Pentium M processor's performance-monitoring events are based on monitoring events for the P6 family of processors. All of these performance events are model specific for the Pentium M processor and are not available in this form in other processors. Table A-7 lists the Performance-Monitoring events that were added in the Pentium M processor.

Table A-7. Performance Monitoring Events on Intel® Pentium® M Processors

Name	Hex Values	Descriptions
Power Management		
EMON_EST_TRANS	58H	Number of Enhanced Intel SpeedStep technology transitions: Mask = 00H - All transitions Mask = 02H - Only Frequency transitions
EMON_THERMAL_TRIP	59H	Duration/Occurrences in thermal trip; to count number of thermal trips: bit 22 in PerfEvtSel0/1 needs to be set to enable edge detect
BPU		
BR_INST_EXEC	88H	Branch instructions executed (not necessarily retired)
BR_MISSP_EXEC	89H	Branch instructions executed that were mispredicted at execution
BR_BAC_MISSP_EXEC	8AH	Branch instructions executed that were mispredicted at Front End (BAC)
BR_CND_EXEC	8BH	Conditional Branch instructions executed
BR_CND_MISSP_EXEC	8CH	Conditional Branch instructions executed that were mispredicted

Table A-7. Performance Monitoring Events on Intel® Pentium® M Processors (Contd.)

Name	Hex Values	Descriptions
BR_IND_EXEC	8DH	Indirect Branch instructions executed
BR_IND_MISSP_EXEC	8EH	Indirect Branch instructions executed that were mispredicted
BR_RET_EXEC	8FH	Return Branch instructions executed
BR_RET_MISSP_EXEC	90H	Return Branch instructions executed that were mispredicted at Execution
BR_RET_BAC_MISSP_EXEC	91H	Return Branch instructions executed that were mispredicted at Front End (BAC)
BR_CALL_EXEC	92H	CALL instruction executed
BR_CALL_MISSP_EXEC	93H	CALL instruction executed and miss predicted
BR_IND_CALL_EXEC	94H	Indirect CALL instruction executed
Decoder		
EMON_SIMD_INSTR_RETIRED	CEH	Number of retired MMX instructions
EMON_SYNCH_UOPS	D3H	Sync micro-ops
EMON_ESP_UOPS	D7H	Total number of micro-ops
EMON_FUSED_UOPS_RET	DAH	Number of retired fused micro-ops Mask = 0 - All fused micro-ops Mask = 1 - Only load+Op micro-ops Mask = 2 - Only std+sta micro-ops
EMON_UNFUSION	DBH	Number of unfusion events in the ROB, happened on a FP exception to a fused μ Op
Prefetcher		
EMON_PREF_RQSTS_UP	F0H	Number of upward prefetches issued
EMON_PREF_RQSTS_DN	F8H	Number of downward prefetches issued

A number of P6 family processor performance monitoring events are modified for the Pentium M processor. Table A-8 lists the performance monitoring events that were changed in the Pentium M processor, and differ from performance monitoring events for the P6 family of processors.

Table A-8. Performance Monitoring Events Modified on Intel® Pentium® M Processors

Name	Hex Values	Descriptions	
CPU_CLK_UNHALTED	79H	Number of cycles during which the processor is not halted, and not in a thermal trip	
EMON_SSE_SSE2_INST_RETIRE	D8H	Streaming SIMD Extensions Instructions Retired Mask = 0 – SSE Packed Single Mask = 1 – SSE Packed-Single and Scalar-Single Mask = 2 – SSE2 Packed-Double Mask = 3 – SSE2 Scalar-Double	
EMON_SSE_SSE2_COMP_INST_RETIRED	D9H	Computational SSE Instructions Retired Mask = 0 – SSE Packed Single Mask = 1 – SSE Scalar-Single Mask = 2 – SSE2 Packed-Double Mask = 3 – SSE2 Scalar-Double	
L2_LD	29H	L2 data loads	Mask[0] = 1 – count I state lines Mask[1] = 1 – count S state lines Mask[2] = 1 – count E state lines Mask[3] = 1 – count M state lines Mask[5:4] = 00H – Excluding Hardware-Prefetched lines 01H – Hardware-Prefetched lines only 02H/03H – All (HW-prefetched lines and non HW --Prefetched lines)
L2_LINES_IN	24H	L2 lines allocated	
L2_LINES_OUT	26H	L2 lines evicted	
L2_M_LINES_OUT	27H	Lw M-state lines evicted	

A.3. P6 FAMILY PROCESSOR PERFORMANCE-MONITORING EVENTS

Table A-9 lists the events that can be counted with the performance-monitoring counters and read with the RDPMC instruction for the P6 family processors. The unit column gives the microarchitecture or bus unit that produces the event; the event number column gives the hexadecimal number identifying the event; the mnemonic event name column gives the name of the event; the unit mask column gives the unit mask required (if any); the description column describes the event; and the comments column gives additional information about the event.

All of these performance events are model specific for the P6 family processors and are not available in this form in the Pentium 4 processors or the Pentium processors. Some events (such as those added in later generations of the P6 family processors) are only available in specific processors in the P6 family. All performance event encodings not listed in Table A-9 are reserved and their use will result in undefined counter results.

See the end of the table for notes related to certain entries in the table.

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
Data Cache Unit (DCU)	43H	DATA_MEM_REFS	00H	All loads from any memory type. All stores to any memory type. Each part of a split is counted separately. The internal logic counts not only memory loads and stores, but also internal retries. Note: 80-bit floating-point accesses are double counted, since they are decomposed into a 16-bit exponent load and a 64-bit mantissa load. Memory accesses are only counted when they are actually performed (such as a load that gets squashed because a previous cache miss is outstanding to the same address, and which finally gets performed, is only counted once). Does not include I/O accesses, or other nonmemory accesses.	
	45H	DCU_LINES_IN	00H	Total lines allocated in the DCU.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	46H	DCU_M_LINES_IN	00H	Number of M state lines allocated in the DCU.	
	47H	DCU_M_LINES_OUT	00H	Number of M state lines evicted from the DCU. This includes evictions via snoop HITM, intervention or replacement.	
	48H	DCU_MISS_OUTSTANDING	00H	Weighted number of cycles while a DCU miss is outstanding, incremented by the number of outstanding cache misses at any particular time. Cacheable read requests only are considered. Uncacheable requests are excluded. Read-for-ownerships are counted, as well as line fills, invalidates, and stores.	An access that also misses the L2 is short-changed by 2 cycles (i.e., if counts N cycles, should be N+2 cycles). Subsequent loads to the same cache line will not result in any additional counts. Count value not precise, but still useful.
Instruction Fetch Unit (IFU)	80H	IFU_IFETCH	00H	Number of instruction fetches, both cacheable and noncacheable, including UC fetches.	
	81H	IFU_IFETCH_MISSES	00H	Number of instruction fetch misses. All instruction fetches that do not hit the IFU (i.e., that produce memory requests). Includes UC accesses.	
	85H	ITLB_MISS	00H	Number of ITLB misses.	
	86H	IFU_MEM_STALL	00H	Number of cycles instruction fetch is stalled, for any reason. Includes IFU cache misses, ITLB misses, ITLB faults, and other minor stalls.	
	87H	ILD_STALL	00H	Number of cycles that the instruction length decoder is stalled.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
L2 Cache ¹	28H	L2_IFETCH	MESI 0FH	Number of L2 instruction fetches. This event indicates that a normal instruction fetch was received by the L2. The count includes only L2 cacheable instruction fetches; it does not include UC instruction fetches. It does not include ITLB miss accesses.	
	29H	L2_LD	MESI 0FH	Number of L2 data loads. This event indicates that a normal, unlocked, load memory access was received by the L2. It includes only L2 cacheable memory accesses; it does not include I/O accesses, other nonmemory accesses, or memory accesses such as UC/WT memory accesses. It does include L2 cacheable TLB miss memory accesses.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	2AH	L2_ST	MESI 0FH	Number of L2 data stores. This event indicates that a normal, unlocked, store memory access was received by the L2. Specifically, it indicates that the DCU sent a read-for-ownership request to the L2. It also includes Invalid to Modified requests sent by the DCU to the L2. It includes only L2 cacheable memory accesses; it does not include I/O accesses, other nonmemory accesses, or memory accesses such as UC/WT memory accesses. It includes TLB miss memory accesses.	
	24H	L2_LINES_IN	00H	Number of lines allocated in the L2.	
	26H	L2_LINES_OUT	00H	Number of lines removed from the L2 for any reason.	
	25H	L2_M_LINES_INM	00H	Number of modified lines allocated in the L2.	
	27H	L2_M_LINES_OUT M	00H	Number of modified lines removed from the L2 for any reason.	
	2EH	L2_RQSTS	MESI 0FH	Total number of L2 requests.	
	21H	L2_ADS	00H	Number of L2 address strobes.	
	22H	L2_DBUS_BUSY	00H	Number of cycles during which the L2 cache data bus was busy.	
	23H	L2_DBUS_BUSY_ RD	00H	Number of cycles during which the data bus was busy transferring read data from L2 to the processor.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
External Bus Logic (EBL) ²	62H	BUS_DRDY_CLOCKS	00H (Self) 20H (Any)	Number of clocks during which DRDY# is asserted. Utilization of the external system data bus during data transfers.	Unit Mask = 00H counts bus clocks when the processor is driving DRDY#. Unit Mask = 20H counts in processor clocks when any agent is driving DRDY#.
	63H	BUS_LOCK_CLOCKS	00H (Self) 20H (Any)	Number of clocks during which LOCK# is asserted on the external system bus. ³	Always counts in processor clocks.
	60H	BUS_REQ_OUTSTANDING	00H (Self)	Number of bus requests outstanding. This counter is incremented by the number of cacheable read bus requests outstanding in any given cycle.	Counts only DCU full-line cacheable reads, not RFOs, writes, instruction fetches, or anything else. Counts "waiting for bus to complete" (last data chunk received).
	65H	BUS_TRAN_BRD	00H (Self) 20H (Any)	Number of burst read transactions.	
	66H	BUS_TRAN_RFO	00H (Self) 20H (Any)	Number of completed read for ownership transactions.	
	67H	BUS_TRANS_WB	00H (Self) 20H (Any)	Number of completed write back transactions.	
	68H	BUS_TRAN_IFETCH	00H (Self) 20H (Any)	Number of completed instruction fetch transactions.	
	69H	BUS_TRAN_INVALID	00H (Self) 20H (Any)	Number of completed invalidate transactions.	
	6AH	BUS_TRAN_PWR	00H (Self) 20H (Any)	Number of completed partial write transactions.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	6BH	BUS_TRANS_P	00H (Self) 20H (Any)	Number of completed partial transactions.	
	6CH	BUS_TRANS_IO	00H (Self) 20H (Any)	Number of completed I/O transactions.	
	6DH	BUS_TRAN_DEF	00H (Self) 20H (Any)	Number of completed deferred transactions.	
	6EH	BUS_TRAN_BURST	00H (Self) 20H (Any)	Number of completed burst transactions.	
	70H	BUS_TRAN_ANY	00H (Self) 20H (Any)	Number of all completed bus transactions. Address bus utilization can be calculated knowing the minimum address bus occupancy. Includes special cycles, etc.	
	6FH	BUS_TRAN_MEM	00H (Self) 20H (Any)	Number of completed memory transactions.	
	64H	BUS_DATA_RCV	00H (Self)	Number of bus clock cycles during which this processor is receiving data.	
	61H	BUS_BNR_DRV	00H (Self)	Number of bus clock cycles during which this processor is driving the BNR# pin.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	7AH	BUS_HIT_DRV	00H (Self)	Number of bus clock cycles during which this processor is driving the HIT# pin.	Includes cycles due to snoop stalls. The event counts correctly, but the BPM_i pins function as follows based on the setting of the PC bits (bit 19 in the PerfEvtSel0 and PerfEvtSel1 registers): If the core-clock-to-bus-clock ratio is 2:1 or 3:1, and a PC bit is set, the BPM_i pins will be asserted for a single clock when the counters overflow. If the PC bit is clear, the processor toggles the BPM_i pins when the counter overflows. If the clock ratio is not 2:1 or 3:1, the BPM_i pins will not function for these performance-monitoring counter events.

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	7BH	BUS_HITM_DRV	00H (Self)	Number of bus clock cycles during which this processor is driving the HITM# pin.	Includes cycles due to snoop stalls. The event counts correctly, but the BPM_i pins function as follows based on the setting of the PC bits (bit 19 in the PerfEvtSel0 and PerfEvtSel1 registers): If the core-clock-to-bus-clock ratio is 2:1 or 3:1, and a PC bit is set, the BPM_i pins will be asserted for a single clock when the counters overflow. If the PC bit is clear, the processor toggles the BPM_i pins when the counter overflows. If the clock ratio is not 2:1 or 3:1, the BPM_i pins will not function for these performance-monitoring counter events.
	7EH	BUS_SNOOP_STALL	00H (Self)	Number of clock cycles during which the bus is snoop stalled.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
Floating-Point Unit	C1H	FLOPS	00H	Number of computational floating-point operations retired. Excludes floating-point computational operations that cause traps or assists. Includes floating-point computational operations executed by the assist handler. Includes internal sub-operations for complex floating-point instructions like transcendentals. Excludes floating-point loads and stores.	Counter 0 only.
	10H	FP_COMP_OPS_EXE	00H	Number of computational floating-point operations executed. The number of FADD, FSUB, FCOM, FMULs, integer MULs and IMULs, FDIVs, FPREMs, FSQRTS, integer DIVs, and IDIVs. Note not the number of cycles, but the number of operations. This event does not distinguish an FADD used in the middle of a transcendental flow from a separate FADD instruction.	Counter 0 only.
	11H	FP_ASSIST	00H	Number of floating-point exception cases handled by microcode.	Counter 1 only. This event includes counts due to speculative execution.
	12H	MUL	00H	Number of multiplies. Note: Includes integer as well as FP multiplies and is speculative.	Counter 1 only.

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	13H	DIV	00H	Number of divides. Note: Includes integer as well as FP divides and is speculative.	Counter 1 only.
	14H	CYCLES_DIV_BUSY	00H	Number of cycles during which the divider is busy, and cannot accept new divides. Note: Includes integer and FP divides, FPREM, FPSQRT, etc., and is speculative.	Counter 0 only.
Memory Ordering	03H	LD_BLOCKS	00H	Number of load operations delayed due to store buffer blocks. Includes counts caused by preceding stores whose addresses are unknown, preceding stores whose addresses are known but whose data is unknown, and preceding stores that conflicts with the load but which incompletely overlap the load.	
	04H	SB_DRAINS	00H	Number of store buffer drain cycles. Incremented every cycle the store buffer is draining. Draining is caused by serializing operations like CPUID, synchronizing operations like XCHG, interrupt acknowledgment, as well as other conditions (such as cache flushing).	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	05H	MISALIGN_MEM_REF	00H	Number of misaligned data memory references. Incremented by 1 every cycle, during which either the processor's load or store pipeline dispatches a misaligned μop. Counting is performed if it is the first or second half, or if it is blocked, squashed, or missed. Note: In this context, misaligned means crossing a 64-bit boundary.	It should be noted that MISALIGN_MEM_REF is only an approximation to the true number of misaligned memory references. The value returned is roughly proportional to the number of misaligned memory accesses, i.e., the size of the problem.
	07H	EMON_KNI_PREF _DISPATCHED	00H 01H 02H 03H	Number of Streaming SIMD extensions prefetch/weakly-ordered instructions dispatched (speculative prefetches are included in counting) 0: prefetch NTA 1: prefetch T1 2: prefetch T2 3: weakly ordered stores	Counters 0 and 1. Pentium III processor only.
	4BH	EMON_KNI_PREF _MISS	00H 01H 02H 03H	Number of prefetch/weakly-ordered instructions that miss all caches. 0: prefetch NTA 1: prefetch T1 2: prefetch T2 3: weakly ordered stores	Counters 0 and 1. Pentium III processor only.

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
Instruction Decoding and Retirement	C0H	INST_RETIRED	OOH	Number of instructions retired.	A hardware interrupt received during/after the last iteration of the REP STOS flow causes the counter to undercount by 1 instruction. And SMI received while executing a HLT instruction will cause the performance counter to not count the RSM instruction and therefore undercount by 1.
	C2H	UOPS_RETIRED	00H	Number of μ ops retired.	
	D0H	INST_DECODED	00H	Number of instructions decoded.	
	D8H	EMON_KNI_INST_RETIRED	00H 01H	Number of Streaming SIMD extensions retired 0: packed & scalar 1: scalar	Counters 0 and 1. Pentium III processor only.
	D9H	EMON_KNI_COMP_INST_RET	00H 01H	Number of Streaming SIMD extensions computation instructions retired. 0: packed and scalar 1: scalar	Counters 0 and 1. Pentium III processor only.
Interrupts	C8H	HW_INT_RX	00H	Number of hardware interrupts received.	
	C6H	CYCLES_INT_MASKED	00H	Number of processor cycles for which interrupts are disabled.	
	C7H	CYCLES_INT_PENDING_AND_MASKED	00H	Number of processor cycles for which interrupts are disabled and interrupts are pending.	
Branches	C4H	BR_INST_RETIRED	00H	Number of branch instructions retired.	
	C5H	BR_MISS_PRED_RETIRED	00H	Number of mispredicted branches retired.	
	C9H	BR_TAKEN_RETIRED	00H	Number of taken branches retired.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
	CAH	BR_MISS_PRED_TAKEN_RET	00H	Number of taken mispredictions branches retired.	
	E0H	BR_INST_DECODED	00H	Number of branch instructions decoded.	
	E2H	BTB_MISSES	00H	Number of branches for which the BTB did not produce a prediction.	
	E4H	BR_BOGUS	00H	Number of bogus branches.	
	E6H	BACLEARs	00H	Number of times BACLEAR is asserted. This is the number of times that a static branch prediction was made, in which the branch decoder decided to make a branch prediction because the BTB did not.	
Stalls	A2H	RESOURCE_STALLS	00H	Incremented by 1 during every cycle for which there is a resource related stall. Includes register renaming buffer entries, memory buffer entries. Does not include stalls due to bus queue full, too many cache misses, etc. In addition to resource related stalls, this event counts some other events. Includes stalls arising during branch misprediction recovery, such as if retirement of the mispredicted branch is delayed and stalls arising while store buffer is draining from synchronizing operations.	
	D2H	PARTIAL_RAT_STALLS	00H	Number of cycles or events for partial stalls. Note: Includes flag partial stalls.	

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
Segment Register Loads	06H	SEGMENT_REG_LOADS	00H	Number of segment register loads.	
Clocks	79H	CPU_CLK_UNHALTED	00H	Number of cycles during which the processor is not halted.	
MMX Unit	B0H	MMX_INSTR_EXE C	00H	Number of MMX Instructions Executed.	Available in Intel Celeron, Pentium II and Pentium II Xeon processors only. Does not account for MOVQ and MOVD stores from register to memory.
	B1H	MMX_SAT_INSTR_EXEC	00H	Number of MMX Saturating Instructions Executed.	Available in Pentium II and Pentium III processors only.
	B2H	MMX_UOPS_EXE C	0FH	Number of MMX μ ops Executed.	Available in Pentium II and Pentium III processors only.
	B3H	MMX_INSTR_TYPE_EXEC	01H 02H 04H 08H 10H 20H	MMX packed multiply instructions executed. MMX packed shift instructions executed. MMX pack operation instructions executed. MMX unpack operation instructions executed. MMX packed logical instructions executed. MMX packed arithmetic instructions executed.	Available in Pentium II and Pentium III processors only.
	CCH	FP_MMX_TRANS	00H 01H	Transitions from MMX instruction to floating-point instructions. Transitions from floating-point instructions to MMX instructions.	Available in Pentium II and Pentium III processors only.
	CDH	MMX_ASSIST	00H	Number of MMX Assists (that is, the number of EMMS instructions executed).	Available in Pentium II and Pentium III processors only.
	CEH	MMX_INSTR_RET	00H	Number of MMX Instructions Retired.	Available in Pentium II processors only.

Table A-9. Events That Can Be Counted with the P6 Family Performance-Monitoring Counters (Contd.)

Unit	Event Num.	Mnemonic Event Name	Unit Mask	Description	Comments
Segment Register Renaming	D4H	SEG_RENAME_STALLS	01H 02H 04H 08H 0FH	Number of Segment Register Renaming Stalls: Segment register ES Segment register DS Segment register FS Segment register FS Segment registers ES + DS + FS + GS	Available in Pentium II and Pentium III processors only.
	D5H	SEG_REG_RENAMES	01H 02H 04H 08H 0FH	Number of Segment Register Renames: Segment register ES Segment register DS Segment register FS Segment register FS Segment registers ES + DS + FS + GS	Available in Pentium II and Pentium III processors only.
	D6H	RET_SEG_RENAMES	00H	Number of segment register rename events retired.	Available in Pentium II and Pentium III processors only.

NOTES:

- Several L2 cache events, where noted, can be further qualified using the Unit Mask (UMSK) field in the PerfEvtSel0 and PerfEvtSel1 registers. The lower 4 bits of the Unit Mask field are used in conjunction with L2 events to indicate the cache state or cache states involved. The P6 family processors identify cache states using the "MESI" protocol and consequently each bit in the Unit Mask field represents one of the four states: UMSK[3] = M (8H) state, UMSK[2] = E (4H) state, UMSK[1] = S (2H) state, and UMSK[0] = I (1H) state. UMSK[3:0] = MESI" (FH) should be used to collect data for all states; UMSK = 0H, for the applicable events, will result in nothing being counted.
- All of the external bus logic (EBL) events, except where noted, can be further qualified using the Unit Mask (UMSK) field in the PerfEvtSel0 and PerfEvtSel1 registers. Bit 5 of the UMSK field is used in conjunction with the EBL events to indicate whether the processor should count transactions that are self-generated (UMSK[5] = 0) or transactions that result from any processor on the bus (UMSK[5] = 1).
- L2 cache locks, so it is possible to have a zero count.

A.4. PENTIUM PROCESSOR PERFORMANCE-MONITORING EVENTS

Table A-10 lists the events that can be counted with the performance-monitoring counters for the Pentium processor. The Event Number column gives the hexadecimal code that identifies the event and that is entered in the ES0 or ES1 (event select) fields of the CESR MSR. The Mnemonic Event Name column gives the name of the event, and the Description and Comments columns give detailed descriptions of the events. Most events can be counted with either counter 0 or counter 1; however, some events can only be counted with only counter 0 or only counter 1 (as noted).

NOTE

The events in the table that are shaded are implemented only in the Pentium processor with MMX technology.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters

Event Num.	Mnemonic Event Name	Description	Comments
00H	DATA_READ	Number of memory data reads (internal data cache hit and miss combined).	Split cycle reads are counted individually. Data Memory Reads that are part of TLB miss processing are not included. These events may occur at a maximum of two per clock. I/O is not included.
01H	DATA_WRITE	Number of memory data writes (internal data cache hit and miss combined), I/O is not included.	Split cycle writes are counted individually. These events may occur at a maximum of two per clock. I/O is not included.
0H2	DATA_TLB_MISS	Number of misses to the data cache translation look-aside buffer.	
03H	DATA_READ_MISS	Number of memory read accesses that miss the internal data cache whether or not the access is cacheable or noncacheable.	Additional reads to the same cache line after the first BRDY# of the burst line fill is returned but before the final (fourth) BRDY# has been returned, will not cause the counter to be incremented additional times. Data accesses that are part of TLB miss processing are not included. Accesses directed to I/O space are not included.
04H	DATA WRITE MISS	Number of memory write accesses that miss the internal data cache whether or not the access is cacheable or noncacheable.	Data accesses that are part of TLB miss processing are not included. Accesses directed to I/O space are not included.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
05H	WRITE_HIT_TO_M_OR_E-STATE_LINES	Number of write hits to exclusive or modified lines in the data cache.	These are the writes that may be held up if EWBE# is inactive. These events may occur a maximum of two per clock.
06H	DATA_CACHE_LINES_WRITTEN_BACK	Number of dirty lines (all) that are written back, regardless of the cause.	Replacements and internal and external snoops can all cause writeback and are counted.
07H	EXTERNAL_SNOOPS	Number of accepted external snoops whether they hit in the code cache or data cache or neither.	Assertions of EADS# outside of the sampling interval are not counted, and no internal snoops are counted.
08H	EXTERNAL_DATA_CACHE_SNOOP_HITS	Number of external snoops to the data cache.	Snoop hits to a valid line in either the data cache, the data line fill buffer, or one of the write back buffers are all counted as hits.
09H	MEMORY ACCESSES IN BOTH PIPES	Number of data memory reads or writes that are paired in both pipes of the pipeline.	These accesses are not necessarily run in parallel due to cache misses, bank conflicts, etc.
0AH	BANK CONFLICTS	Number of actual bank conflicts.	
0BH	MISALIGNED DATA MEMORY OR I/O REFERENCES	Number of memory or I/O reads or writes that are misaligned.	A 2- or 4-byte access is misaligned when it crosses a 4-byte boundary; an 8-byte access is misaligned when it crosses an 8-byte boundary. Ten byte accesses are treated as two separate accesses of 8 and 2 bytes each.
0CH	CODE READ	Number of instruction reads whether the read is cacheable or noncacheable.	Individual 8-byte noncacheable instruction reads are counted.
0DH	CODE TLB MISS	Number of instruction reads that miss the code TLB whether the read is cacheable or noncacheable.	Individual 8-byte noncacheable instruction reads are counted.
0EH	CODE CACHE MISS	Number of instruction reads that miss the internal code cache whether the read is cacheable or noncacheable.	Individual 8-byte noncacheable instruction reads are counted.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
0FH	ANY SEGMENT REGISTER LOADED	Number of writes into any segment register in real or protected mode including the LDTR, GDTR, IDTR, and TR.	Segment loads are caused by explicit segment register load instructions, far control transfers, and task switches. Far control transfers and task switches causing a privilege level change will signal this event twice. Note that interrupts and exceptions may initiate a far control transfer.
10H	Reserved		
11H	Reserved		
12H	Branches	Number of taken and not taken branches, including conditional branches, jumps, calls, returns, software interrupts, and interrupt returns.	Also counted as taken branches are serializing instructions, VERR and VERW instructions, some segment descriptor loads, hardware interrupts (including FLUSH#), and programmatic exceptions that invoke a trap or fault handler. The pipe is not necessarily flushed. The number of branches actually executed is measured, not the number of predicted branches.
13H	BTB_HITS	Number of BTB hits that occur.	Hits are counted only for those instructions that are actually executed.
14H	TAKEN_BRANCH_OR_BTBT_HIT	Number of taken branches or BTB hits that occur.	This event type is a logical OR of taken branches and BTB hits. It represents an event that may cause a hit in the BTB. Specifically, it is either a candidate for a space in the BTB or it is already in the BTB.
15H	PIPELINE FLUSHES	Number of pipeline flushes that occur. Pipeline flushes are caused by BTB misses on taken branches, mispredictions, exceptions, interrupts, and some segment descriptor loads.	The counter will not be incremented for serializing instructions (serializing instructions cause the prefetch queue to be flushed but will not trigger the Pipeline Flushed event counter) and software interrupts (software interrupts do not flush the pipeline).

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
16H	INSTRUCTIONS_EXECUTED	Number of instructions executed (up to two per clock).	Invocations of a fault handler are considered instructions. All hardware and software interrupts and exceptions will also cause the count to be incremented. Repeat prefixed string instructions will only increment this counter once despite the fact that the repeat loop executes the same instruction multiple times until the loop criteria is satisfied. This applies to all the Repeat string instruction prefixes (i.e., REP, REPE, REPZ, REPNE, and REPNZ). This counter will also only increment once per each HLT instruction executed regardless of how many cycles the processor remains in the HALT state.
17H	INSTRUCTIONS_EXECUTED_V PIPE	Number of instructions executed in the V_pipe. It indicates the number of instructions that were paired.	This event is the same as the 16H event except it only counts the number of instructions actually executed in the V-pipe.
18H	BUS_CYCLE_DURATION	Number of clocks while a bus cycle is in progress. This event measures bus use.	The count includes HLDA, AHOLD, and BOFF# clocks.
19H	WRITE_BUFFER_FULL_STALL_DURATION	Number of clocks while the pipeline is stalled due to full write buffers.	Full write buffers stall data memory read misses, data memory write misses, and data memory write hits to S-state lines. Stalls on I/O accesses are not included.
1AH	WAITING_FOR_DATA_MEMORY_READ_STALL_DURATION	Number of clocks while the pipeline is stalled while waiting for data memory reads.	Data TLB Miss processing is also included in the count. The pipeline stalls while a data memory read is in progress including attempts to read that are not bypassed while a line is being filled.
1BH	STALL ON WRITE TO AN E- OR M-STATE LINE	Number of stalls on writes to E- or M-state lines	
1CH	LOCKED BUS CYCLE	Number of locked bus cycles that occur as the result of the LOCK prefix or LOCK instruction, page-table updates, and descriptor table updates.	Only the read portion of the locked read-modify-write is counted. Split locked cycles (SCYC active) count as two separate accesses. Cycles restarted due to BOFF# are not re-counted.
1DH	I/O READ OR WRITE CYCLE	Number of bus cycles directed to I/O space.	Misaligned I/O accesses will generate two bus cycles. Bus cycles restarted due to BOFF# are not re-counted.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
1EH	NONCACHEABLE_MEMORY_READS	Number of noncacheable instruction or data memory read bus cycles. Count includes read cycles caused by TLB misses, but does not include read cycles to I/O space.	Cycles restarted due to BOFF# are not re-counted.
1FH	PIPELINE_AGI_STALLS	Number of address generation interlock (AGI) stalls. An AGI occurring in both the U- and V-pipelines in the same clock signals this event twice.	An AGI occurs when the instruction in the execute stage of either of U- or V-pipelines is writing to either the index or base address register of an instruction in the D2 (address generation) stage of either the U- or V- pipelines.
20H	Reserved		
21H	Reserved		
22H	FLOPS	Number of floating-point operations that occur.	Number of floating-point adds, subtracts, multiplies, divides, remainders, and square roots are counted. The transcendental instructions consist of multiple adds and multiplies and will signal this event multiple times. Instructions generating the divide-by-zero, negative square root, special operand, or stack exceptions will not be counted. Instructions generating all other floating-point exceptions will be counted. The integer multiply instructions and other instructions which use the x87 FPU will be counted.
23H	BREAKPOINT MATCH ON DR0 REGISTER	Number of matches on register DR0 breakpoint.	The counters is incremented regardless if the breakpoints are enabled or not. However, if breakpoints are not enabled, code breakpoint matches will not be checked for instructions executed in the V-pipe and will not cause this counter to be incremented. (They are checked on instruction executed in the U-pipe only when breakpoints are not enabled.) These events correspond to the signals driven on the BP[3:0] pins. Refer to Chapter 15, <i>Debugging and Performance Monitoring</i> , for more information.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
24H	BREAKPOINT MATCH ON DR1 REGISTER	Number of matches on register DR1 breakpoint.	See comment for 23H event.
25H	BREAKPOINT MATCH ON DR2 REGISTER	Number of matches on register DR2 breakpoint.	See comment for 23H event.
26H	BREAKPOINT MATCH ON DR3 REGISTER	Number of matches on register DR3 breakpoint.	See comment for 23H event.
27H	HARDWARE INTERRUPTS	Number of taken INTR and NMI interrupts.	
28H	DATA_READ_OR_WRITE	Number of memory data reads and/or writes (internal data cache hit and miss combined).	Split cycle reads and writes are counted individually. Data Memory Reads that are part of TLB miss processing are not included. These events may occur at a maximum of two per clock. I/O is not included.
29H	DATA_READ_MISS OR_WRITE MISS	Number of memory read and/or write accesses that miss the internal data cache whether or not the access is cacheable or noncacheable.	Additional reads to the same cache line after the first BRDY# of the burst line fill is returned but before the final (fourth) BRDY# has been returned, will not cause the counter to be incremented additional times. Data accesses that are part of TLB miss processing are not included. Accesses directed to I/O space are not included.
2AH	BUS_OWNERSHIP_LATENCY (Counter 0)	The time from LRM bus ownership request to bus ownership granted (that is, the time from the earlier of a PBREQ (0), PHITM# or HITM# assertion to a PBGNT assertion).	The ratio of the 2AH events counted on counter 0 and counter 1 is the average stall time due to bus ownership conflict.
2AH	BUS_OWNERSHIP_TRANSFERS (Counter 1)	The number of bus ownership transfers (that is, the number of PBREQ (0) assertions).	The ratio of the 2AH events counted on counter 0 and counter 1 is the average stall time due to bus ownership conflict.
2BH	MMX_INSTRUCTIONS_EXECUTED_U-PIPE (Counter 0)	Number of MMX instructions executed in the U-pipe.	
2BH	MMX_INSTRUCTIONS_EXECUTED_V-PIPE (Counter 1)	Number of MMX instructions executed in the V-pipe.	

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
2CH	CACHE_M-STATE_LINE_SHARING (Counter 0)	Number of times a processor identified a hit to a modified line due to a memory access in the other processor (PHITM(O)).	If the average memory latencies of the system are known, this event enables the user to count the Write Backs on PHITM(O) penalty and the Latency on Hit Modified(I) penalty.
2CH	CACHE_LINE_SHARING (Counter 1)	Number of shared data lines in the L1 cache (PHIT(O)).	
2DH	EMMS_INSTRUCTIONS_EXECUTED (Counter 0)	Number of EMMS instructions executed.	
2DH	TRANSITIONS_BETWEEN_MMX_AND_FP_INSTRUCTIONS (Counter 1)	Number of transitions between MMX and floating-point instructions or vice versa. An even count indicates the processor is in MMX state. an odd count indicates it is in FP state.	This event counts the first floating-point instruction following an MMX instruction or first MMX instruction following a floating-point instruction. The count may be used to estimate the penalty in transitions between floating-point state and MMX state.
2EH	BUS_UTILIZATION_DUE_TO_PROCESSOR_ACTIVITY (Counter 0)	Number of clocks the bus is busy due to the processor's own activity, i.e., the bus activity that is caused by the processor.	
2EH	WRITES_TO_NONCACHEABLE_MEMORY (Counter 1)	Number of write accesses to noncacheable memory.	The count includes write cycles caused by TLB misses and I/O write cycles. Cycles restarted due to BOFF# are not re-counted.
2FH	SATURATING_MMX_INSTRUCTIONS_EXECUTED (Counter 0)	Number of saturating MMX instructions executed, independently of whether they actually saturated.	
2FH	SATURATIONS_PERFORMED (Counter 1)	Number of MMX instructions that used saturating arithmetic and that at least one of its results actually saturated.	If an MMX instruction operating on 4 doublewords saturated in three out of the four results, the counter will be incremented by one only.

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
30H	NUMBER_OF_CYCLES_NOT_IN_HALT_STATE (Counter 0)	Number of cycles the processor is not idle due to HLT instruction.	This event will enable the user to calculate "net CPI". Note that during the time that the processor is executing the HLT instruction, the Time-Stamp Counter is not disabled. Since this event is controlled by the Counter Controls CC0, CC1 it can be used to calculate the CPI at CPL=3, which the TSC cannot provide.
30H	DATA_CACHE_TLB_MISS_STALL_DURATION (Counter 1)	Number of clocks the pipeline is stalled due to a data cache translation look-aside buffer (TLB) miss.	
31H	MMX_INSTRUCTION_DATA_READS (Counter 0)	Number of MMX instruction data reads.	
31H	MMX_INSTRUCTION_DATA_READ_MISSES (Counter 1)	Number of MMX instruction data read misses.	
32H	FLOATING_POINT_STALLS_DURATION (Counter 0)	Number of clocks while pipe is stalled due to a floating-point freeze.	
32H	TAKEN_BRANCHES (Counter 1)	Number of taken branches.	
33H	D1_STARVATION_AND_FIFO_IS_EMPTY (Counter 0)	Number of times D1 stage cannot issue ANY instructions since the FIFO buffer is empty.	The D1 stage can issue 0, 1, or 2 instructions per clock if those are available in an instructions FIFO buffer.
33H	D1_STARVATION_AND_ONLY_ONE_INSTRUCTION_IN_FIFO (Counter 1)	Number of times the D1 stage issues just a single instruction since the FIFO buffer had just one instruction ready.	The D1 stage can issue 0, 1, or 2 instructions per clock if those are available in an instructions FIFO buffer. When combined with the previously defined events, Instruction Executed (16H) and Instruction Executed in the V-pipe (17H), this event enables the user to calculate the numbers of time pairing rules prevented issuing of two instructions.
34H	MMX_INSTRUCTION_DATA_WRITES (Counter 0)	Number of data writes caused by MMX instructions.	

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
34H	MMX_ INSTRUCTION_ DATA_WRITE_ MISSES (Counter 1)	Number of data write misses caused by MMX instructions.	
35H	PIPELINE_ FLUSHES_DUE_ TO_WRONG_ BRANCH_ PREDICTIONS (Counter 0)	Number of pipeline flushes due to wrong branch predictions resolved in either the E-stage or the WB-stage.	The count includes any pipeline flush due to a branch that the pipeline did not follow correctly. It includes cases where a branch was not in the BTB, cases where a branch was in the BTB but was mispredicted, and cases where a branch was correctly predicted but to the wrong address. Branches are resolved in either the Execute stage (E-stage) or the Writeback stage (WB-stage). In the later case, the misprediction penalty is larger by one clock. The difference between the 35H event count in counter 0 and counter 1 is the number of E-stage resolved branches.
35H	PIPELINE_ FLUSHES_DUE_ TO_WRONG_ BRANCH_ PREDICTIONS_ RESOLVED_IN_ WB-STAGE (Counter 1)	Number of pipeline flushes due to wrong branch predictions resolved in the WB-stage.	See note for event 35H (Counter 0).
36H	MISALIGNED_ DATA_MEMORY_ REFERENCE_ON_ MMX_ INSTRUCTIONS (Counter 0)	Number of misaligned data memory references when executing MMX instructions.	
36H	PIPELINE_ ISTALL_FOR_MMX_ INSTRUCTION_ DATA_MEMORY_ READS (Counter 1)	Number clocks during pipeline stalls caused by waits form MMX instruction data memory reads.	
37H	MISPREDICTED_ OR_ UNPREDICTED_ RETURNS (Counter 1)	Number of returns predicted incorrectly or not predicted at all.	The count is the difference between the total number of executed returns and the number of returns that were correctly predicted. Only RET instructions are counted (for example, IRET instructions are not counted).

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
37H	PREDICTED_RETURNS (Counter 1)	Number of predicted returns (whether they are predicted correctly and incorrectly.	Only RET instructions are counted (for example, IRET instructions are not counted).
38H	MMX_MULTIPLY_UNIT_INTERLOCK (Counter 0)	Number of clocks the pipe is stalled since the destination of previous MMX multiply instruction is not ready yet.	The counter will not be incremented if there is another cause for a stall. For each occurrence of a multiply interlock this event will be counted twice (if the stalled instruction comes on the next clock after the multiply) or by one (if the stalled instruction comes two clocks after the multiply).
38H	MOVD/MOVQ_STORE_STALL_DUE_TO_PREVIOUS_MMX_OPERATION (Counter 1)	Number of clocks a MOVD/MOVQ instruction store is stalled in D2 stage due to a previous MMX operation with a destination to be used in the store instruction.	
39H	RETURNS (Counter 0)	Number of returns executed.	Only RET instructions are counted; IRET instructions are not counted. Any exception taken on a RET instruction and any interrupt recognized by the processor on the instruction boundary prior to the execution of the RET instruction will also cause this counter to be incremented.
39H	Reserved		
3AH	BTB_FALSE_ENTRIES (Counter 0)	Number of false entries in the Branch Target Buffer.	False entries are causes for misprediction other than a wrong prediction.
3AH	BTB_MISS_PREDICTION_ON_NOT-TAKEN_BRANCH (Counter 1)	Number of times the BTB predicted a not-taken branch as taken.	
3BH	FULL_WRITE_BUFFER_STALL_DURATION_WHILE_EXECUTING_MMX_INSTRUCTIONS (Counter 0)	Number of clocks while the pipeline is stalled due to full write buffers while executing MMX instructions.	

Table A-10. Events That Can Be Counted with the Pentium Processor Performance-Monitoring Counters (Contd.)

Event Num.	Mnemonic Event Name	Description	Comments
3BH	STALL_ON_MMX_INSTRUCTION_WRITE_TO E- OR M-STATE_LINE (Counter 1)	Number of clocks during stalls on MMX instructions writing to E- or M-state lines.	

B

Model-Specific Registers (MSRs)

APPENDIX B

MODEL-SPECIFIC REGISTERS (MSRS)

This appendix lists the MSRs provided in the Pentium 4 and Intel Xeon processors, the P6 family processors, and the Pentium processors in Tables B-1, B-3, and B-4, respectively. All the MSRs listed in these tables can be read with the RDMSR and written with the WRMSR instructions. Register addresses are given in both hexadecimal and decimal; the register name is the mnemonic register name; and the bit description describes individual bits in registers.

Table B-5 lists the architectural MSRs.

B.1. MSRS IN THE PENTIUM 4 AND INTEL XEON PROCESSORS

The following MSRs are defined for the Pentium 4 and Intel Xeon processors. The MSRs with an “IA32_” prefix are designated as “architectural,” meaning that the functions of these MSRs and their addresses remain the same for succeeding families of IA-32 processors. Table B-5 lists the architectural MSRs.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
0H	0	IA32_P5_MC_ADDR	Shared	See Section B.4., “MSRs in Pentium Processors”.
1H	1	IA32_P5_MC_TYPE	Shared	See Section B.4., “MSRs in Pentium Processors”.
10H	16	IA32_TIME_STAMP_COUNTER	Unique	Time Stamp Counter. See Section 15.7., “Time-Stamp Counter”
		63:0		Timestamp Count Value. (R/W) Returns the current time stamp count value. All 64 bits are readable; only the lower 32 bits are writable. On any write to the lower 32 bits, the upper 32 bits are cleared.
17H	23	IA32_PLATFORM_ID	Shared	Platform ID. (R) The operating system can use this MSR to determine “slot” information for the processor and the proper microcode update to load.
		49:0		Reserved.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		52:50		Platform Id. (R) Contains information concerning the intended platform for the processor. 52 51 50 0 0 0 Processor Flag 0 0 0 1 Processor Flag 1 0 1 0 Processor Flag 2 0 1 1 Processor Flag 3 1 0 0 Processor Flag 4 1 0 1 Processor Flag 5 1 1 0 Processor Flag 6 1 1 1 Processor Flag 7
		63:53		Reserved.
1BH	27	IA32_APIC_BASE	Unique	APIC Location and Status. (R/W) Contains location and status information about the APIC (see Section 8.4.4., “Local APIC Status and Location”)
		7:0		Reserved.
		8		Bootstrap Processor (BSP). Set if the processor is the BSP.
		10:9		Reserved.
		11		APIC Global Enable. Set if enabled; cleared if disabled.
		31:12		APIC Base Address. The base address of the xAPIC memory map.
		63:32		Reserved.
2AH	42	MSR_EBC_HARD_POWERON	Shared	Processor Hard Power-On Configuration. (R/W) Enables and disables processor features; (R) indicates current processor configuration.
		0		Output Tri-state Enabled. (R) Indicates whether tri-state output is enabled (1) or disabled (0) as set by the strapping of SMI#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.
		1		Execute BIST. (R) Indicates whether the execution of the BIST is enabled (1) or disabled (0) as set by the strapping of INIT#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		2		In Order Queue Depth. (R) Indicates whether the in order queue depth for the system bus is 1 (1) or up to 12 (0) as set by the strapping of A7#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.
		3		MCERR# Observation Disabled. (R) Indicates whether MCERR# observation is enabled (0) or disabled (1) as set by the strapping of A9#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.
		4		BINIT# Observation Enabled. (R) Indicates whether BINIT# observation is enabled (0) or disabled (1) as set by the strapping of A10#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.
		6:5		APIC Cluster ID. (R) Contains the logical APIC cluster ID value as set by the strapping of A12# and A11#. The logical cluster ID value is written into the field on the deassertion of RESET#; the field is set to 1 when the address bus signal is asserted.
		7		Bus Park Disable. (R) Indicates whether bus park is enabled (0) or disabled (1) as set by the strapping of A15#. The value in this bit is written on the deassertion of RESET#; the bit is set to 1 when the address bus signal is asserted.
		11:8		Reserved.
		13:12		Agent ID. (R) Contains the logical agent ID value as set by the strapping of BR[3:0]. The logical ID value is written into the field on the deassertion of RESET#; the field is set to 1 when the address bus signal is asserted.
		63:14		Reserved.
2BH	43	MSR_EBC_SOFT_POWERON	Shared	Processor Soft Power-On Configuration. (R/W) Enables and disables processor features.
		0		RCNT/SCNT On Request Encoding Enable. (R/W) Controls the driving of RCNT/SCNT on the request encoding. Set to enabled; clear to disabled (default).

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description								
Hex	Dec											
		1		Data Error Checking Disable. (R/W) Set to disable system data bus parity checking (default); clear to enable parity checking.								
		2		Response Error Checking Disable. (R/W) Set to disable (default); clear to enable.								
		3		Address/Request Error Checking Disable. (R/W) Set to disable (default); clear to enable.								
		4		Initiator MCERR# Disable. (R/W) Set to disable MCERR# driving for initiator bus requests (default); clear to enable.								
		5		Internal MCERR# Disable. (R/W) Set to disable MCERR# driving for initiator internal errors (default); clear to enable.								
		6		BINIT# Driver Disable. (R/W) Set to disable BINIT# driver (default); clear to enable driver.								
		63:7		Reserved.								
2CH	44	MSR_EBC_FREQUENCY_ID	Shared	Processor Frequency Configuration. The bit field layout of this MSR varies according to the MODEL value of the CPUID version information. The following bit field layout applies to Pentium 4 and Xeon Processors with MODEL encoding equal or greater than 2. (R) Indicates current processor frequency configuration.								
		15:0		Reserved.								
		18:16		Scalable Bus Speed. (R/W) Indicates the intended scalable bus speed: <table><tr><td><u>Encoding</u></td><td><u>Scalable Bus Speed</u></td></tr><tr><td>000B</td><td>100 MHz</td></tr><tr><td>001B</td><td>133 MHz</td></tr><tr><td>010B</td><td>200 MHz</td></tr></table> 133.33 MHz should be utilized if performing calculation with System Bus Speed when encoding is 001B. All Others Reserved	<u>Encoding</u>	<u>Scalable Bus Speed</u>	000B	100 MHz	001B	133 MHz	010B	200 MHz
		<u>Encoding</u>	<u>Scalable Bus Speed</u>									
000B	100 MHz											
001B	133 MHz											
010B	200 MHz											
63:19		Reserved.										

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description				
Hex	Dec							
2CH	44	MSR_EBC_FREQUENCY_ID	Shared	Processor Frequency Configuration. The bit field layout of this MSR varies according to the MODEL value of the CPUID version information. The following bit field layout applies to Pentium 4 and Xeon Processors with MODEL encoding less than 2. (R) Indicates current processor frequency configuration.				
		20:0		Reserved.				
		23:21		Scalable Bus Speed. (R/W) Indicates the intended scalable bus speed: <table><tr><td><u>Encoding</u></td><td><u>Scalable Bus Speed</u></td></tr><tr><td>000B</td><td>100 MHz</td></tr></table> All Others Reserved	<u>Encoding</u>	<u>Scalable Bus Speed</u>	000B	100 MHz
		<u>Encoding</u>	<u>Scalable Bus Speed</u>					
000B	100 MHz							
63:24		Reserved.						
79H	121	IA32_BIOS_UPDT_TRIG	Shared	BIOS Update Trigger Register. (R/W) Triggers the loading of a microcode update. Executing a WRMSR instruction to this MSR causes a microcode update to be loaded into the processor (see Section 9.11.2.1., “Update Loading Procedure”).				
8BH	139	IA32_BIOS_SIGN_ID	Unique	BIOS Update Signature ID. (R/W) Returns the microcode update signature, following the execution of a CPUID instruction with EAX set to 1.				
		31:0		Reserved.				
		63:32		Microcode Update Signature. (R/W) It is recommended that this field be pre-loaded with a 0 prior to executing the CPUID instruction. If the field remains 0 following the execution of the CPUID instruction, then there is no microcode update loaded. Any other non-0 value is the microcode update signature.				
FEH	254	IA32_MTRRCAP	Unique	MTRR Information. See Section 10.11.1., “MTRR Feature Identification”.				
174H	372	IA32_SYSENTER_CS	Unique	CS register target for CPL 0 code. (R/W) Used by SYSENTER and SYSEXIT instructions (see Section 4.8.7., “Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions”).				

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
175H	373	IA32_SYSENTER_ESP	Unique	Stack pointer for CPL 0 stack. (R/W) Used by SYSENTER and SYSEXIT instructions (see Section 4.8.7., “Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions”).
176H	374	IA32_SYSENTER_EIP	Unique	CPL 0 code entry point. (R/W) Used by SYSENTER and SYSEXIT instructions (see Section 4.8.7., “Performing Fast Calls to System Procedures with the SYSENTER and SYSEXIT Instructions”).
179H	377	IA32_MCG_CAP	Unique	Machine Check Capabilities. (R) Returns the capabilities of the machine check architecture for the processor (see Section 14.3.1.1., “IA32_MCG_CAP MSR (Pentium 4 and Intel Xeon Processors)”).
17AH	378	IA32_STATUS	Unique	Machine Check Status. (R) Returns machine check state following the generation of a machine check exception (see Section 14.3.1.3., “IA32_MCG_STATUS MSR”).
17BH	379	IA32_CTL		Machine Check Feature Enable. (R/W) Enables machine check capability (see Section 14.3.1.4., “IA32_MCG_CTL MSR”).
180H	384	IA32_MCG_EAX	Unique	Machine Check EAX Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EAX Register Contents. (R/W to 0) Contains state of EAX register at the time of the last machine check error.
		63:32		Reserved.
181H	385	IA32_MCG_EBX	Unique	Machine Check EBX Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EBX Register Contents. (R/W to 0) Contains state of EBX register at the time of the last machine check error.
		63:32		Reserved.
182H	386	IA32_MCG_ECX	Unique	Machine Check ECX Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		ECX Register Contents. (R/W to 0) Contains state of ECX register at the time of the last machine check error.
		63:32		Reserved.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
183H	387	IA32_MCG_EDX	Unique	Machine Check EDX Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EDX Register Contents. (R/W to 0) Contains state of EDX register at the time of the last machine check error.
		63:32		Reserved.
184H	388	IA32_MCG_ESI	Unique	Machine Check ESI Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		ESI Register Contents. (R/W to 0) Contains state of EDI register at the time of the last machine check error.
		63:32		Reserved.
185H	389	IA32_MCG_EDI	Unique	Machine Check EDI Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EDI Register Contents. (R/W to 0) Contains state of EDI register at the time of the last machine check error.
		63:32		Reserved.
186H	390	IA32_MCG_EBP	Unique	Machine Check EBP Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EBP Register Contents. (R/W to 0) Contains state of EBP register at the time of the last machine check error.
		63:32		Reserved.
187H	391	IA32_MCG_ESP	Unique	Machine Check ESP Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		ESP Register Contents. (R/W to 0) Contains state of ESP register at the time of the last machine check error.
		63:32		Reserved.
188H	392	IA32_MCG_EFLAGS	Unique	Machine Check EFLAGS Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EFLAGS Register Contents. (R/W to 0) Contains state of EFLAGS register at the time of the last machine check error.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		63:32		Reserved.
189H	393	IA32_MCG_EIP	Unique	Machine Check EIP Save State. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
		31:0		EIP Register Contents. (R/W to 0) Contains state of EIP register at the time of the last machine check error.
		63:32		Reserved.
18AH	394	IA32_MCG_MISC	Unique	Machine Check Miscellaneous. See Section 14.3.2.5., “IA32_MCG Extended Machine Check State MSRs”.
				DS. When set, indicates that a page assist or page fault occurred during DS normal operation. The processor will shutdown under these circumstance. This bit is set as an aid for debugging the DS handling code. It is the responsibility of the user (BIOS or operating system) to clear this bit for normal operation
		63:1		Reserved.
19AH	410	IA32_THERM_CONTROL	Unique	Thermal Monitor Control. (R/W) Enables and disables on-demand clock modulation and allows selection of the on-demand clock modulation duty cycle. (See Section 13.15.3., “Software Controlled Clock Modulation”.)
19BH	411	IA32_THERM_INTERRUPT	Unique	Thermal Interrupt Control. (R/W) Enables and disables the generation of an interrupt on temperature transitions detected with the processor’s thermal sensor and thermal monitor. (See Section 13.15.2., “Thermal Monitor”.)
19CH	412	IA32_THERM_STATUS	Shared	Thermal Monitor Status. (R/W) Contains status information about the processor’s thermal sensor and automatic thermal monitoring facilities. (See Section 13.15.2., “Thermal Monitor”.)
1A0H	416	IA32_MISC_ENABLE	Shared	Enable Miscellaneous Processor Features. (R/W) Allows a variety of processor functions to be enabled and disabled.
		0		Fast-Strings Enable. When set, the fast-strings feature on the Pentium 4 processor is enabled(default); when clear, fast-strings are disabled.
		1		Reserved.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		2		x87 FPU Fopcode Compatibility Mode Enable. When set, fopcode compatibility mode is enabled; when clear (default), mode is disabled. See “Fopcode Compatibility Mode” in Chapter 8 of the <i>IA-32 Intel Architecture Software Developer’s Manual, Volume 1</i> .
		3		Thermal Monitor 1 Enable. When set, clock modulation is controlled by the processor’s internal thermal sensor is enabled; when clear (default), automatic clock modulation is disabled. (See Section 13.15.2., “Thermal Monitor”.)
		4		Split-Lock Disable. This debug feature is specific to the Pentium 4 processor. When set, the bit causes an #AC exception to be issued instead of a split-lock cycle. Operating systems that set this bit must align system structures to avoid split-lock scenarios. When the bit is clear (the default), normal split-locks are issued to the bus.
		5		Reserved.
		6		Third-Level Cache Disable. (R/W) When set, the third-level cache is disabled; when clear (default) the third-level cache is enabled. This flag is reserved for processors that do not have a third-level cache. Note that this bit controls only the third-level cache, and then only if overall caching is enabled through the CD flag of control register CR0, the page-level cache controls, and/or the MTRRs (see Section 10.5.4., “Disabling and Enabling the L3 Cache”).
		7		Performance Monitoring Available. (R) When set, performance monitoring is enabled; when clear, performance monitoring is disabled.
		8		Suppress Lock Enable. When set assert on of lock on the bus is suppressed during a Split Lock access. when clear (default) does not suppress lock.
		9		Prefetch Queue Disable. When set disables the prefetch queue. When clear (default) the prefetch queue is enabled.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		10		FERR# Interrupt Reporting Enable. (R/W) When set, interrupt reporting through the FERR# pin is enabled; when clear, this interrupt reporting function is disabled. When this flag is set and the processor is in the stop-clock state (STPCLK# is asserted), asserting the FERR# pin signals to the processor that an interrupt (such as, INIT#, BINIT#, INTR, NMI, SMI#, or RESET#) is pending and that the processor should return to normal operation to handle the interrupt. This flag does not affect the normal operation of the FERR# pin (to indicate an unmasked floating-point error) when the STPCLK# pin is not asserted.
		11		Branch Trace Storage Unavailable (BTS_UNAVAILABLE). (R) When set, the processor does not support branch trace storage (BTS); when clear, BTS is supported.
		12		Precise Event Based Sampling Unavailable (PEBS_UNAVAILABLE). (R) When set, the processor does not support precise event-based sampling (PEBS); when clear, PEBS is supported.
		18:13		Reserved.
		19		Adjacent Cache Line Prefetch Disable (R/W). When set to 1, the processor fetches the cache line of the 128-byte sector that contains the data currently required by the processor. When set to 0, the processor fetches both cache lines in the sector. Single processor platforms should not set this bit. Server platforms should set or clear this bit based on platform performance observed in validation and testing. BIOS may contain a setup option that controls the setting of this bit.
		23:20		Reserved.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		24		L1 Data Cache Context Mode (R/W). When set to 1, this bit places the L1 Data Cache into shared mode. When set to 0 (the default), this bit places the L1 Data Cache into adaptive mode. In adaptive mode, the Page Directory Base Register contained in CR3 must be identical across all logical processors. Note: If the Context ID feature flag, ECX[10], is not set to 1 after executing CPUID with EAX = 1; the ability to switch modes is not supported and the BIOS must not alter the contents of IA32_MISC_ENABLE[24].
		63:25		Reserved.
1D7H	471	MSR_LER_FROM_LIP	Unique	Last Exception Record From Linear IP. (R) Contains a pointer to the last branch instruction that the processor executed prior to the last exception that was generated or the last interrupt that was handled. (See Section 15.5.6., “Last Exception Records (Pentium 4 and Intel Xeon Processors)”.)
		31:0		From Linear IP: Linear address last branch instruction.
		63:32		Reserved.
1D8H	472	MSR_LER_TO_LIP	Unique	Last Exception Record To Linear IP. (R) Contains a pointer to the target of the last branch instruction that the processor executed prior to the last exception that was generated or the last interrupt that was handled. (See Section 15.5.6., “Last Exception Records (Pentium 4 and Intel Xeon Processors)”.)
		31:0		From Linear IP: Linear address of the target of the last branch instruction.
		63:32		Reserved.
1D9H	473	IA32_DEBUGCTL	Unique	Debug Control. (R/W) Controls how several debug features are used. See Section 15.5.1., “IA32_DEBUGCTL MSR (Pentium 4 and Intel Xeon Processors)”.
1DAH	474	MSR_LASTBRANCH_TOS	Unique	Last Branch Record Stack TOS. (R) Contains an index (0, 1, 2, or 3) that points to the top of the last branch record stack (that is, that points the index of the MSR containing the most recent branch record. See Section 15.5.2., “LBR Stack (Pentium 4 and Intel Xeon Processors)”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
1DBH	475	MSR_LASTBRANCH_0	Unique	Last Branch Record 0. (R/W) One of four last branch record registers on the last branch record stack. It contains pointers to the source and destination instruction for one of the last four branches, exceptions, or interrupts that the processor took. See Section 15.5.2., “LBR Stack (Pentium 4 and Intel Xeon Processors)”.
1DCH	476	MSR_LASTBRANCH_1	Unique	Last Branch Record 1. See description of the MSR_LASTBRANCH_0 MSR.
1DDH	477	MSR_LASTBRANCH_2	Unique	Last Branch Record 1. See description of the MSR_LASTBRANCH_0 MSR.
1DEH	478	MSR_LASTBRANCH_3	Unique	Last Branch Record 1. See description of the MSR_LASTBRANCH_0 MSR.
200H	512	IA32_MTRR_PHYSBASE0	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
201H	513	IA32_MTRR_PHYSMASK0	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
202H	514	IA32_MTRR_PHYSBASE1	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
203H	515	IA32_MTRR_PHYSMASK1	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
204H	516	IA32_MTRR_PHYSBASE2	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
205H	517	IA32_MTRR_PHYSMASK2	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
206H	518	IA32_MTRR_PHYSBASE3	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
207H	519	IA32_MTRR_PHYSMASK3	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
208H	520	IA32_MTRR_PHYSBASE4	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
209H	521	IA32_MTRR_PHYSMASK4	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
20AH	522	IA32_MTRR_PHYSBASE5	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
20BH	523	IA32_MTRR_PHYSMASK5	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
20CH	524	IA32_MTRR_PHYSBASE6	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
20DH	525	IA32_MTRR_PHYSMASK6	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
20EH	526	IA32_MTRR_PHYSBASE7	Shared	Variable Range Base MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
20FH	527	IA32_MTRR_PHYSMASK7	Shared	Variable Range Mask MTRR. See Section 10.11.2.3., “Variable Range MTRRs”.
250H	592	IA32_MTRR_FIX64K_00000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
258H	600	IA32_MTRR_FIX16K_80000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
259H	601	IA32_MTRR_FIX16K_A0000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
268H	616	IA32_MTRR_FIX4K_C0000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
269H	617	IA32_MTRR_FIX4K_C8000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26AH	618	IA32_MTRR_FIX4K_D0000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26BH	619	IA32_MTRR_FIX4K_D8000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26CH	620	IA32_MTRR_FIX4K_E0000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26DH	621	IA32_MTRR_FIX4K_E8000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26EH	622	IA32_MTRR_FIX4K_F0000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
26FH	623	IA32_MTRR_FIX4K_F8000	Shared	Fixed Range MTRR. See Section 10.11.2.2., “Fixed Range MTRRs”.
277H	631	IA32_CR_PAT	Unique	Page Attribute Table. See Section 10.11.2.2., “Fixed Range MTRRs”, for further information about this MSR.
2FFH	767	IA32_MTRR_DEF_TYPE	Shared	Default Memory Types. (R/W) Sets the memory type for the regions of physical memory that are not mapped by the MTRRs. See Section 10.11.2.1., “IA32_MTRR_DEF_TYPE MSR”.
300H	768	MSR_BPU_COUNTER0	Shared	See Section 15.9.2., “Performance Counters”.
301H	769	MSR_BPU_COUNTER1	Shared	See Section 15.9.2., “Performance Counters”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
302H	770	MSR_BPU_COUNTER2	Shared	See Section 15.9.2., “Performance Counters”.
303H	771	MSR_BPU_COUNTER3	Shared	See Section 15.9.2., “Performance Counters”.
304H	772	MSR_MS_COUNTER0	Shared	See Section 15.9.2., “Performance Counters”.
305H	773	MSR_MS_COUNTER1	Shared	See Section 15.9.2., “Performance Counters”.
306H	774	MSR_MS_COUNTER2	Shared	See Section 15.9.2., “Performance Counters”.
307H	775	MSR_MS_COUNTER3	Shared	See Section 15.9.2., “Performance Counters”.
308H	776	MSR_FLAME_COUNTER0	Shared	See Section 15.9.2., “Performance Counters”.
309H	777	MSR_FLAME_COUNTER1	Shared	See Section 15.9.2., “Performance Counters”.
30AH	778	MSR_FLAME_COUNTER2	Shared	See Section 15.9.2., “Performance Counters”.
30BH	779	MSR_FLAME_COUNTER3	Shared	See Section 15.9.2., “Performance Counters”.
30CH	780	MSR_IQ_COUNTER0	Shared	See Section 15.9.2., “Performance Counters”.
30DH	781	MSR_IQ_COUNTER1	Shared	See Section 15.9.2., “Performance Counters”.
30EH	782	MSR_IQ_COUNTER2	Shared	See Section 15.9.2., “Performance Counters”.
30FH	783	MSR_IQ_COUNTER3	Shared	See Section 15.9.2., “Performance Counters”.
310H	784	MSR_IQ_COUNTER4	Shared	See Section 15.9.2., “Performance Counters”.
311H	785	MSR_IQ_COUNTER5	Shared	See Section 15.9.2., “Performance Counters”.
360H	864	MSR_BPU_CCCR0	Shared	See Section 15.9.3., “CCCR MSRs”.
361H	865	MSR_BPU_CCCR1	Shared	See Section 15.9.3., “CCCR MSRs”.
362H	866	MSR_BPU_CCCR2	Shared	See Section 15.9.3., “CCCR MSRs”.
363H	867	MSR_BPU_CCCR3	Shared	See Section 15.9.3., “CCCR MSRs”.
364H	868	MSR_MS_CCCR0	Shared	See Section 15.9.3., “CCCR MSRs”.
365H	869	MSR_MS_CCCR1	Shared	See Section 15.9.3., “CCCR MSRs”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
366H	870	MSR_MS_CCCR2	Shared	See Section 15.9.3., “CCCR MSRs”.
367H	871	MSR_MS_CCCR3	Shared	See Section 15.9.3., “CCCR MSRs”.
368H	872	MSR_FLAME_CCCR0	Shared	See Section 15.9.3., “CCCR MSRs”.
369H	873	MSR_FLAME_CCCR1	Shared	See Section 15.9.3., “CCCR MSRs”.
36AH	874	MSR_FLAME_CCCR2	Shared	See Section 15.9.3., “CCCR MSRs”.
36BH	875	MSR_FLAME_CCCR3	Shared	See Section 15.9.3., “CCCR MSRs”.
36CH	876	MSR_IQ_CCCR0	Shared	See Section 15.9.3., “CCCR MSRs”.
36DH	877	MSR_IQ_CCCR1	Shared	See Section 15.9.3., “CCCR MSRs”.
36EH	878	MSR_IQ_CCCR2	Shared	See Section 15.9.3., “CCCR MSRs”.
36FH	879	MSR_IQ_CCCR3	Shared	See Section 15.9.3., “CCCR MSRs”.
370H	880	MSR_IQ_CCCR4	Shared	See Section 15.9.3., “CCCR MSRs”.
371H	881	MSR_IQ_CCCR5	Shared	See Section 15.9.3., “CCCR MSRs”.
3A0H	928	MSR_BSU_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3A1H	929	MSR_BSU_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3A2H	930	MSR_FSB_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3A3H	931	MSR_FSB_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3A4H	932	MSR_FIRM_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3A5H	933	MSR_FIRM_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3A6H	934	MSR_FLAME_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3A7H	935	MSR_FLAME_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3A8H	936	MSR_DAC_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3A9H	937	MSR_DAC_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3AAH	938	MSR_MOB_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3ABH	939	MSR_MOB_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3ACH	940	MSR_PMH_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3ADH	941	MSR_PMH_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3AEH	942	MSR_SAAT_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3AFH	943	MSR_SAAT_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3B0H	944	MSR_U2L_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3B1H	945	MSR_U2L_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3B2H	946	MSR_BPU_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3B3H	947	MSR_BPU_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
3B4H	948	MSR_IS_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3B5H	949	MSR_IS_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3B6H	950	MSR_ITLB_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3B7H	951	MSR_ITLB_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3B8H	952	MSR_CRU_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3B9H	953	MSR_CRU_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3BAH	954	MSR_IQ_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3BBH	955	MSR_IQ_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3BCH	956	MSR_RAT_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3BDH	957	MSR_RAT_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3BEH	958	MSR_SSU_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3C0H	960	MSR_MS_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3C1H	961	MSR_MS_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3C2H	962	MSR_TBPU_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3C3H	963	MSR_TBPU_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3C4H	964	MSR_TC_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3C5H	965	MSR_TC_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3C8H	968	MSR_IX_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3C9H	969	MSR_IX_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3CAH	970	MSR_ALF_ESCR0	Shared	See Section 15.9.1., “ESCR MSRs”.
3CBH	971	MSR_ALF_ESCR1	Shared	See Section 15.9.1., “ESCR MSRs”.
3CCH	972	MSR_CRU_ESCR2	Shared	See Section 15.9.1., “ESCR MSRs”.
3CDH	973	MSR_CRU_ESCR3	Shared	See Section 15.9.1., “ESCR MSRs”.
3E0H	992	MSR_CRU_ESCR4	Shared	See Section 15.9.1., “ESCR MSRs”.
3E1H	993	MSR_CRU_ESCR5	Shared	See Section 15.9.1., “ESCR MSRs”.
3FOH	1008	MSR_TC_PRECISE _EVENT	Shared	
3F1H	1009	IA32_PEBS_ENABLE	Shared	Precise Event-Based Sampling (PEBS). (R/W) Controls the enabling of precise event sampling and replay tagging.
		12:0		See Table A-5.
		23:13		Reserved.
		24		UOP Tag. Enables replay tagging when set.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
		25		ENABLE_PEBBS_MY_THR. (R/W) Enables PEBS for the target logical processor when set; disables PEBS when clear (default). See Section 15.10.3., “IA32_PEBBS_ENABLE MSR”, for an explanation of the target logical processor. This bit is called ENABLE_PEBBS in IA-32 processors that do not support Hyper-Threading Technology.
		26		ENABLE_PEBBS_OTH_THR. (R/W) Enables PEBS for the target logical processor when set; disables PEBS when clear (default). See Section 15.10.3., “IA32_PEBBS_ENABLE MSR”, for an explanation of the target logical processor. This bit is reserved for IA-32 processors that do not support Hyper-Threading Technology.
		63:27		Reserved.
3F2H	1010	MSR_PEBBS_MATRIX_VERT	Shared	See Table A-5.
400H	1024	IA32_MC0_CTL	Shared	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”.
401H	1025	IA32_MC0_STATUS	Shared	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”.
402H	1026	IA32_MC0_ADDR	Shared	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The IA32_MC0_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC0_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
403H	1027	IA32_MC0_MISC	Shared	See Section 14.3.2.4., “IA32_MCi_MISC MSRs”. The IA32_MC0_MISC MSR is either not implemented or does not contain additional information if the MISC_V flag in the IA32_MC0_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
404H	1028	IA32_MC1_CTL	Shared	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”.
405H	1029	IA32_MC1_STATUS	Shared	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
406H	1030	IA32_MC1_ADDR	Shared	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The IA32_MC1_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC1_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
407H	1031	IA32_MC1_MISC	Shared	See Section 14.3.2.4., “IA32_MCi_MISC MSRs”. The IA32_MC1_MISC MSR is either not implemented or does not contain additional information if the MISCV flag in the IA32_MC1_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
408H	1032	IA32_MC2_CTL	Shared	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”.
409H	1033	IA32_MC2_STATUS	Shared	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”.
40AH	1034	IA32_MC2_ADDR		See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The IA32_MC2_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC2_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
40BH	1035	IA32_MC2_MISC		See Section 14.3.2.4., “IA32_MCi_MISC MSRs”. The IA32_MC2_MISC MSR is either not implemented or does not contain additional information if the MISCV flag in the IA32_MCi_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
40CH	1036	IA32_MC3_CTL	Shared	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”.
40DH	1037	IA32_MC3_STATUS	Shared	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”.

Table B-1. MSRs in the Pentium 4 and Intel Xeon Processors (Contd.)

Register Address		Register Name Fields and Flags	Shared/ Unique ¹	Bit Description
Hex	Dec			
40EH	1038	IA32_MC3_ADDR	Shared	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The IA32_MC3_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC3_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
40FH	1039	IA32_MC3_MISC	Shared	See Section 14.3.2.4., “IA32_MCi_MISC MSRs”. The IA32_MC3_MISC MSR is either not implemented or does not contain additional information if the MISC_V flag in the IA32_MC3_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
600H	1536	IA32_DS_AREA 31:0 63:32	Unique	DS Save Area. (R/W) Points to the DS buffer management area, which is used to manage the BTS and PEBS buffers (see Section 15.9.4., “Debug Store (DS) Mechanism”). DS Buffer Management Area. Linear address of the first byte of the DS buffer management area. Reserved.
¹ For Hyper-Thread enabled (HT) processors, there are two logical processors per physical unit. If an MSR is Shared, this means that one MSR is shared between two logical processors. If an MSR is Unique, this means that each logical processor has its own MSR.				

B.2. MSRS IN THE PENTIUM M PROCESSOR

Model-specific registers (MSRs) for the Pentium M processor are similar to those described in Section B.3. for P6 family processors. The following table describes new MSRs and MSRs whose behavior has changed on the Pentium M processor.

Table B-2. MSRs in Pentium M Processors

Register Address		Register Name	Bit Description																																				
Hex	Dec																																						
0H	0	P5_MC_ADDR	See Section B.4., “MSRs in Pentium Processors”.																																				
1H	1	P5_MC_TYPE	See Section B.4., “MSRs in Pentium Processors”.																																				
10H	16	IA32_TIME_STAMP_COUNTER	See Section 15.7., “Time-Stamp Counter”																																				
17H	23	IA32_PLATFORM_ID	Platform ID. (R) The operating system can use this MSR to determine “slot” information for the processor and the proper microcode update to load.																																				
		49:0	Reserved.																																				
		52:50	Platform Id. (R) Contains information concerning the intended platform for the processor. <table><tr><td><u>52</u></td><td><u>51</u></td><td><u>50</u></td><td></td></tr><tr><td>0</td><td>0</td><td>0</td><td>Processor Flag 0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>Processor Flag 1</td></tr><tr><td>0</td><td>1</td><td>0</td><td>Processor Flag 2</td></tr><tr><td>0</td><td>1</td><td>1</td><td>Processor Flag 3</td></tr><tr><td>1</td><td>0</td><td>0</td><td>Processor Flag 4</td></tr><tr><td>1</td><td>0</td><td>1</td><td>Processor Flag 5</td></tr><tr><td>1</td><td>1</td><td>0</td><td>Processor Flag 6</td></tr><tr><td>1</td><td>1</td><td>1</td><td>Processor Flag 7</td></tr></table>	<u>52</u>	<u>51</u>	<u>50</u>		0	0	0	Processor Flag 0	0	0	1	Processor Flag 1	0	1	0	Processor Flag 2	0	1	1	Processor Flag 3	1	0	0	Processor Flag 4	1	0	1	Processor Flag 5	1	1	0	Processor Flag 6	1	1	1	Processor Flag 7
		<u>52</u>	<u>51</u>	<u>50</u>																																			
0	0	0	Processor Flag 0																																				
0	0	1	Processor Flag 1																																				
0	1	0	Processor Flag 2																																				
0	1	1	Processor Flag 3																																				
1	0	0	Processor Flag 4																																				
1	0	1	Processor Flag 5																																				
1	1	0	Processor Flag 6																																				
1	1	1	Processor Flag 7																																				
63:53	Reserved.																																						
2AH	42	MSR_EBL_CR_POWERON	Processor Hard Power-On Configuration. (R/W) Enables and disables processor features; (R) indicates current processor configuration.																																				
		0	Reserved																																				
		1	Data Error Checking Enable. (R/W) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.																																				
		2	Response Error Checking Enable. (R/W) FRCERR Observation Enable: 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.																																				
		3	AERR# Drive Enable. (R/W) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.																																				

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		4	BERR# Enable for initiator bus requests. (R/W) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.
		5	Reserved
		6	BERR# Driver Enable for initiator internal errors. (R/W) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.
		7	BINIT# Driver Enable. (R/W) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.
		8	Output Tri-state Enabled. (R/O) 1 = Enabled 0 = Disabled
		9	Execute BIST. (R/O) 1 = Enabled 0 = Disabled
		10	AERR# Observation Enabled. (R/O) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.
		11	Reserved
		12	BINIT# Observation Enabled. (R/O) 1 = Enabled 0 = Disabled NOTE: Always 0 on the Pentium M processor.
		13	In Order Queue Depth. (R/O) 1 = 1 0 = 8
		14	1Mbyte Power on Reset Vector. (R/O) 1 = 1 Mbyte 0 = 4 Gbytes NOTE: Always 0 on the Pentium M processor.
		15	Reserved
		17:16	APIC Cluster ID. (R/O) NOTE: Always 00B on the Pentium M processor.
		18	System Bus Frequency. (R/O) 0= 100 MHz 1= Reserved NOTE: Always 0 on the Pentium M processor.
		19	Reserved

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		21: 20	Symmetric Arbitration ID. (R/O) NOTE: Always 00B on the Pentium M processor.
		26:22	Clock Frequency Ratio (R/O)
		63:27	Reserved
119h	281	MSR_BBL_CR_CTL	
		63:0	Reserved
11Eh	281	MSR_BBL_CR_CTL3	
		0	L2 Hardware Enabled. (RO) 1 = If the L2 is hardware-enabled 0 = Indicates if the L2 is hardware-disabled
		4:1	Reserved
		5	ECC Check Enable. (RO) This bit enables ECC checking on the cache data bus. ECC is always generated on write cycles. 0 = Disabled (default) 1 = Enabled NOTE: For the Pentium M processor, ECC checking on the cache data bus is always enabled.
		7:6	Reserved
		8	L2 Enabled. (R/W) 1 = L2 cache has been initialized 0 = Disabled (default) NOTE: Until this bit is set the processor will not respond to the WBINVD instruction or the assertion of the FLUSH# input.
		19:9	Reserved
		22:20	L2 Physical Address Range Supported. (RO) This field indicates the address capability of the cache controller as defined below: 111 = 64Gbyte (A35-A3) 110 = 32Gbyte (A34-A3) 101 = 16Gbyte (A33-A3) 100 = 8Gbyte (A32-A3) 011 = 4Gbyte (A31-A3) 010 = 2Gbyte (A30-A3) 001 = 1Gbyte (A29-A3) 000 = 512Mbyte (A28-A3) The Pentium M processor supports a 4Gbyte physical address range.
		23	L2 Not Present. (RO) 0 = L2 Present 1 = L2 Not Present

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		63:24	Reserved
179H	377	IA32_MCG_CAP	
		7:0	Count. (RO) Indicates the number of hardware unit error reporting banks available in the processor
		8	IA32_MCG_CTL Present. (RO) 1 = Indicates that the processor implements the MSR_MCG_CTL register found at MSR 17BH. 0 = Not supported.
		63:9	Reserved
17AH	378	IA32_MCG_STATUS	
		0	RIPV. When set, this bit indicates that the instruction addressed by the instruction pointer pushed on the stack (when the machine check was generated) can be used to restart the program. If this bit is cleared, the program cannot be reliably restarted
		1	EIPV. When set, this bit indicates that the instruction addressed by the instruction pointer pushed on the stack (when the machine check was generated) is directly associated with the error.
		2	MCIP. When set, this bit indicates that a machine check has been generated. If a second machine check is detected while this bit is still set, the processor enters a shutdown state. Software should write this bit to 0 after processing a machine check exception.
		63:3	Reserved
198H	408	IA32_PERF_STAUS	
		15:0	Current Performance State Value.
		63:16	Reserved
199H	409	IA32_PERF_CTL	
		15:0	Target Performance State Value.
		63:16	Reserved
19AH	410	IA32_THERM_CONTROL	Thermal Monitor Control. (R/W) Enables and disables on-demand clock modulation and allows the selection of the on-demand clock modulation duty cycle. (See Section 13.15.3., “Software Controlled Clock Modulation”.)
19BH	411	IA32_THERM_INTERRUPT	Thermal Interrupt Control. (R/W) Enables and disables the generation of an interrupt on temperature transitions detected with the processor’s thermal sensor and thermal monitor. (See Section 13.15.2., “Thermal Monitor”.)

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
19CH	412	IA32_THERM_STATU S	Thermal Monitor Status. (R/W) Contains status information about the processor's thermal sensor and automatic thermal monitoring facilities. (See Section 13.15.2., "Thermal Monitor".)
19DH	413	MSR_THERM2_CTL	
		15:0	Reserved
		16	TM_SELECT. (R/W) Mode of automatic thermal monitor: 0 = Thermal Monitor 1 (thermally-initiated on-die modulation of the stop-clock duty cycle) 1 = Thermal Monitor 2 (thermally-initiated frequency transitions) If bit 3 of the IA32_MISC_ENABLE register is cleared, TM_SELECT has no effect. Neither TM1 nor TM2 will be enabled.
		63:16	Reserved
1A0	416	IA32_MISC_ENABLE	Enable Miscellaneous Processor Features. (R/W) Allows a variety of processor functions to be enabled and disabled.
		2:0	Reserved.
		3	Automatic Thermal Control Circuit Enable. (R/W) 1 = Setting this bit enables the thermal control circuit (TCC) portion of the Intel Thermal Monitor feature. This allows processor clocks to be automatically modulated based on the processor's thermal sensor operation. 0 = Disabled (default). The automatic thermal control circuit enable bit determines if the thermal control circuit (TCC) will be activated when the processor's internal thermal sensor determines the processor is about to exceed its maximum operating temperature. When the TCC is activated and TM1 is enabled, the processors clocks will be forced to a 50% duty cycle. BIOS must enable this feature. The bit should not be confused with the on-demand thermal control circuit enable bit.
		6:4	Reserved
		7	Performance Monitoring Available. (R) 1 = Performance monitoring enabled 0 = Performance monitoring disabled
		9:8	Reserved

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		10	FERR# Multiplexing Enable. (R/W) 1 = FERR# asserted by the processor to indicate a pending break event within the processor 0 = Indicates compatible FERR# signaling behavior NOTE: This bit must be set to 1 to support XAPIC interrupt model usage.
		11	Branch Trace Storage Unavailable. (RO) 1 = Processor doesn't support branch trace storage (BTS) 0 = BTS is supported
		12	Precise Event Based Sampling Unavailable. (RO) 1 = Processor does not support precise event-based sampling (PEBS); 0 = PEBS is supported. NOTE: The Pentium M processor does not support PEBS.
		15:13	Reserved
		16	Enhanced Intel SpeedStep technology Enable. (R/W) 1 = Enhanced Intel SpeedStep technology enabled NOTE: On the Pentium M processor, this bit may be configured to be read-only.
		63:17	Reserved
2FFH	767	IA32_MTRR_DEF_TY PE	Default Memory Types. (R/W) Sets the memory type for the regions of physical memory that are not mapped by the MTRRs. See Section 10.11.2.1., "IA32_MTRR_DEF_TYPE MSR"
400	1024	IA32_MC0_CTL	See Section 14.3.2.1., "IA32_MCi_CTL MSRs"
401	1025	IA32_MC0_STATUS	See Section 14.3.2.2., "IA32_MCi_STATUS MSRs"
402	1026	IA32_MC0_ADDR	See Section 14.3.2.3., "IA32_MCi_ADDR MSRs". The IA32_MC0_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC0_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
404	1028	IA32_MC1_CTL	See Section 14.3.2.1., "IA32_MCi_CTL MSRs"
405	1029	IA32_MC1_STATUS	See Section 14.3.2.2., "IA32_MCi_STATUS MSRs"
406	1030	IA32_MC1_ADDR	See Section 14.3.2.3., "IA32_MCi_ADDR MSRs". The IA32_MC1_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC1_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
408	1032	IA32_MC2_CTL	See Section 14.3.2.1., "IA32_MCi_CTL MSRs"
409	1033	IA32_MC2_STATUS	See Section 14.3.2.2., "IA32_MCi_STATUS MSRs"

Table B-2. MSRs in Pentium M Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
40A	1034	IA32_MC2_ADDR	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The IA32_MC2_ADDR register is either not implemented or contains no address if the ADDR_V flag in the IA32_MC2_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
40C	1036	MSR_MC4_CTL	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”
40D	1037	MSR_MC4_STATUS	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”
40E	1038	MSR_MC4_ADDR	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The MSR_MC4_ADDR register is either not implemented or contains no address if the ADDR_V flag in the MSR_MC4_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.
410	1040	MSR_MC3_CTL	See Section 14.3.2.1., “IA32_MCi_CTL MSRs”
411	1041	MSR_MC3_STATUS	See Section 14.3.2.2., “IA32_MCi_STATUS MSRs”
412	1042	MSR_MC3_ADDR	See Section 14.3.2.3., “IA32_MCi_ADDR MSRs”. The MSR_MC3_ADDR register is either not implemented or contains no address if the ADDR_V flag in the MSR_MC3_STATUS register is clear. When not implemented in the processor, all reads and writes to this MSR will cause a general-protection exception.

B.3. MSRS IN THE P6 FAMILY PROCESSORS

The following MSRs are defined for the P6 family processors. The MSRs in this table that are shaded are available only in the Pentium II and Pentium III processors. Beginning with the Pentium 4 processor, some of the MSRs in this list have been designated as “architectural” and have had their names changed. See Table B-5 for a list of the architectural MSRs.

Table B-3. MSRs in the P6 Family Processors

Register Address		Register Name	Bit Description
Hex	Dec		
0H	0	P5_MC_ADDR	See Section B.4., “MSRs in Pentium Processors”.
1H	1	P5_MC_TYPE	See Section B.4., “MSRs in Pentium Processors”.
10H	16	TSC	See Section 15.7., “Time-Stamp Counter”
17H	23	IA32_PLATFORM_ID	Platform ID. (R) The operating system can use this MSR to determine “slot” information for the processor and the proper microcode update to load.
		49:0	Reserved.
		52:50	Platform Id. (R) Contains information concerning the intended platform for the processor. 52 51 50 0 0 0 Processor Flag 0 0 0 1 Processor Flag 1 0 1 0 Processor Flag 2 0 1 1 Processor Flag 3 1 0 0 Processor Flag 4 1 0 1 Processor Flag 5 1 1 0 Processor Flag 6 1 1 1 Processor Flag 7
		56:53	L2 Cache Latency Read
		59:57	Reserved
		60	Clock Frequency Ratio Read
		63:61	Reserved.
1BH	27	APIC_BASE	Section 8.4.4., “Local APIC Status and Location”
		7:0	Reserved
		8	Boot Strap Processor indicator Bit. BSP= 1
		10:9	Reserved
		11	APIC Global Enable Bit - Permanent till reset Enabled = 1, Disabled = 0
		31:12	APIC Base Address
		63:32	Reserved

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
2AH	42	EBL_CR_POWERON	Processor Hard Power-On Configuration. (R/W) Enables and disables processor features; (R) indicates current processor configuration.
		0	Reserved ¹
		1	Data Error Checking Enable 1 = Enabled 0 = Disabled Read/Write
		2	Response Error Checking Enable FRCERR Observation Enable 1 = Enabled 0 = Disabled Read/Write
		3	AERR# Drive Enable 1 = Enabled 0 = Disabled Read/Write
		4	BERR# Enable for initiator bus requests 1 = Enabled 0 = Disabled Read/Write
		5	Reserved
		6	BERR# Driver Enable for initiator internal errors 1 = Enabled 0 = Disabled Read/Write
		7	BINIT# Driver Enable 1 = Enabled 0 = Disabled Read/Write
		8	Output Tri-state Enabled 1 = Enabled 0 = Disabled Read
		9	Execute BIST 1 = Enabled 0 = Disabled Read
		10	AERR# Observation Enabled 1 = Enabled 0 = Disabled Read
		11	Reserved

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		12	BINIT# Observation Enabled 1 = Enabled 0 = Disabled Read
		13	In Order Queue Depth 1 = 1 0 = 8 Read
		14	1Mbyte Power on Reset Vector 1 = 1Mbyte 0 = 4Gbytes Read Only
		15	FRC Mode Enable 1 = Enabled 0 = Disabled Read Only
		17:16	APIC Cluster ID Read
		19:18	System Bus Frequency Read 00 = 66MHz 10 = 100Mhz 01 = 133MHz 11 = Reserved
		21: 20	Symmetric Arbitration ID Read
		25:22	Clock Frequency Ratio Read
		26	Low Power Mode Enable Read/Write
		27	Clock Frequency Ratio
		63:28	Reserved ¹
33H	51	TEST_CTL	Test Control Register
		29:0	Reserved
		30	Streaming Buffer Disable
		31	Disable LOCK# assertion for split locked access
79H	121	BIOS_UPDT_TRIG	BIOS Update Trigger Register
88	136	BBL_CR_D0[63:0]	Chunk 0 data register D[63:0]: used to write to and read from the L2
89	137	BBL_CR_D1[63:0]	Chunk 1 data register D[63:0]: used to write to and read from the L2

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
8A	138	BBL_CR_D2[63:0]	Chunk 2 data register D[63:0]: used to write to and read from the L2
8BH	139	BIOS_SIGN/BBL_CR_D3[63:0]	BIOS Update Signature Register or Chunk 3 data register D[63:0]: used to write to and read from the L2 depending on the usage model
C1H	193	PerfCtr0 (PERFCTR0)	
C2H	194	PerfCtr1 (PERFCTR1)	
FEH	254	MTRRcap	
116	278	BBL_CR_ADDR [63:0] BBL_CR_ADDR [63:32] BBL_CR_ADDR [31:3] BBL_CR_ADDR [2:0]	Address register: used to send specified address (A31-A3) to L2 during cache initialization accesses. Reserved, Address bits [35:3] Reserved Set to 0.
118	280	BBL_CR_DECC[63:0]	Data ECC register D[7:0]: used to write ECC and read ECC to/from L2
119	281	BBL_CR_CTL BL_CR_CTL[63:22] BBL_CR_CTL[21] BBL_CR_CTL[20:19] BBL_CR_CTL[18] BBL_CR_CTL[17] BBL_CR_CTL[16] BBL_CR_CTL[15:14] BBL_CR_CTL[13:12] BBL_CR_CTL[11:10] BBL_CR_CTL[9:8] BBL_CR_CTL[7] BBL_CR_CTL[6:5] BBL_CR_CTL[4:0]	Control register: used to program L2 commands to be issued via cache configuration accesses mechanism. Also receives L2 lookup response Reserved Processor number ² Disable = 1 Enable = 0 Reserved User supplied ECC Reserved L2 Hit Reserved State from L2 Modified - 11, Exclusive - 10, Shared - 01, Invalid - 00 Way from L2 Way 0 - 00, Way 1 - 01, Way 2 - 10, Way 3 - 11 Way to L2 Reserved State to L2 L2 Command 01100 Data Read w/ LRU update (RLU) 01110 Tag Read w/ Data Read (TRR) 01111 Tag Inquire (TI) 00010 L2 Control Register Read (CR) 00011 L2 Control Register Write (CW) 010 + MESI encode Tag Write w/ Data Read (TWR) 111 + MESI encode Tag Write w/ Data Write (TWW) 100 + MESI encode Tag Write (TW)
11A	282	BBL_CR_TRIG	Trigger register: used to initiate a cache configuration accesses access, Write only with Data=0.

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
11B	283	BBL_CR_BUSY	Busy register: indicates when a cache configuration accesses L2 command is in progress. D[0] = 1 = BUSY
11E	286	BBL_CR_CTL3	Control register 3: used to configure the L2 Cache
		BBL_CR_CTL3[63:26]	Reserved
		BBL_CR_CTL3[25]	Cache bus fraction (read only)
		BBL_CR_CTL3[24]	Reserved
		BBL_CR_CTL3[23]	L2 Hardware Disable (read only)
		BBL_CR_CTL3[22:20]	L2 Physical Address Range support
		111	64Gbytes
		110	32Gbytes
		101	16Gbytes
		100	8Gbytes
		011	4Gbytes
		010	2Gbytes
		001	1Gbytes
		000	512Mbytes
		BBL_CR_CTL3[19]	Reserved
		BBL_CR_CTL3[18]	Cache State error checking enable (read/write)
		BBL_CR_CTL3[17:13]	Cache size per bank (read/write)
		00001	256Kbytes
		00010	512Kbytes
		00100	1Mbyte
		01000	2Mbyte
		10000	4Mbytes
		BBL_CR_CTL3[12:11]	Number of L2 banks (read only)
		BBL_CR_CTL3[10:9]	L2 Associativity (read only)
		00	Direct Mapped
		01	2 Way
		10	4 Way
		11	Reserved
		BBL_CR_CTL3[8]	L2 Enabled (read/write)
		BBL_CR_CTL3[7]	CRTN Parity Check Enable (read/write)
		BBL_CR_CTL3[6]	Address Parity Check Enable (read/write)
		BBL_CR_CTL3[5]	ECC Check Enable (read/write)
		BBL_CR_CTL3[4:1]	L2 Cache Latency (read/write)
		BBL_CR_CTL3[0]	L2 Configured (read/write)
174H	372	SYSENTER_CS_MSR	CS register target for CPL 0 code
175H	373	SYSENTER_ESP_MSR	Stack pointer for CPL 0 stack
176H	374	SYSENTER_EIP_MSR	CPL 0 code entry point
179H	377	MCG_CAP	
17AH	378	MCG_STATUS	
17BH	379	MCG_CTL	
186H	390	PerfEvtSel0 (EVNTSEL0)	
		7:0	Event Select (Refer to Performance Counter section for a list of event encodings)

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		15:8	UMASK (Unit Mask): Unit mask register set to 0 to enable all count options
		16	USER: Controls the counting of events at Privilege levels of 1, 2, and 3
		17	OS: Controls the counting of events at Privilege level of 0
		18	E: Occurrence/Duration Mode Select 1 = Occurrence 0 = Duration
		19	PC: Enabled the signaling of performance counter overflow via BP0 pin
		20	INT: Enables the signaling of counter overflow via input to APIC 1 = Enable 0 = Disable
		22	ENABLE: Enables the counting of performance events in both counters 1 = Enable 0 = Disable
		23	INV: Inverts the result of the CMASK condition 1 = Inverted 0 = Non-Inverted
		31:24	CMASK (Counter Mask):
187H	391	PerfEvtSel1 (EVNTSEL1)	
		7:0	Event Select (Refer to Performance Counter section for a list of event encodings)
		15:8	UMASK (Unit Mask): Unit mask register set to 0 to enable all count options
		16	USER: Controls the counting of events at Privilege levels of 1, 2, and 3
		17	OS: Controls the counting of events at Privilege level of 0

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		18	E: Occurrence/Duration Mode Select 1 = Occurrence 0 = Duration
		19	PC: Enabled the signaling of performance counter overflow via BP0 pin.
		20	INT: Enables the signaling of counter overflow via input to APIC 1 = Enable 0 = Disable
		23	INV: Inverts the result of the CMASK condition 1 = Inverted 0 = Non-Inverted
		31:24	CMASK (Counter Mask):
1D9H	473	DEBUGCTLMSR	
		0	Enable/Disable Last Branch Records
		1	Branch Trap Flag
		2	Performance Monitoring/Break Point Pins
		3	Performance Monitoring/Break Point Pins
		4	Performance Monitoring/Break Point Pins
		5	Performance Monitoring/Break Point Pins
		6	Enable/Disable Execution Trace Messages
		13:7	Reserved
		14	Enable/Disable Execution Trace Messages
		15	Enable/Disable Execution Trace Messages
1DBH	475	LASTBRANCHFROMIP	
1DCH	476	LASTBRANCHTOIP	
1DDH	477	LASTINTFROMIP	
1DEH	478	LASTINTTOIP	
1E0H	480	ROB_CR_BKUPTMPDR6	
		1:0	Reserved
		2	Fast String Enable bit. Default is enabled
200H	512	MTRRphysBase0	
201H	513	MTRRphysMask0	

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
202H	514	MTRRphysBase1	
203H	515	MTRRphysMask1	
204H	516	MTRRphysBase2	
205H	517	MTRRphysMask2	
206H	518	MTRRphysBase3	
207H	519	MTRRphysMask3	
208H	520	MTRRphysBase4	
209H	521	MTRRphysMask4	
20AH	522	MTRRphysBase5	
20BH	523	MTRRphysMask5	
20CH	524	MTRRphysBase6	
20DH	525	MTRRphysMask6	
20EH	526	MTRRphysBase7	
20FH	527	MTRRphysMask7	
250H	592	MTRRfix64K_00000	
258H	600	MTRRfix16K_80000	
259H	601	MTRRfix16K_A0000	
268H	616	MTRRfix4K_C0000	
269H	617	MTRRfix4K_C8000	
26AH	618	MTRRfix4K_D0000	
26BH	619	MTRRfix4K_D8000	
26CH	620	MTRRfix4K_E0000	
26DH	621	MTRRfix4K_E8000	
26EH	622	MTRRfix4K_F0000	
26FH	623	MTRRfix4K_F8000	
2FFH	767	MTRRdefType	
		2:0	Default memory type
		10	Fixed MTRR enable
		11	MTRR Enable
400H	1024	MC0_CTL	
401H	1025	MC0_STATUS	
		63	MC_STATUS_V

Table B-3. MSRs in the P6 Family Processors (Contd.)

Register Address		Register Name	Bit Description
Hex	Dec		
		62	MC_STATUS_O
		61	MC_STATUS_UC
		60	MC_STATUS_EN. (Note: For MC0_STATUS only, this bit is hardcoded to 1.)
		59	MC_STATUS_MISCV
		58	MC_STATUS_ADDRV
		57	MC_STATUS_DAM
		31:16	MC_STATUS_MCACOD
		15:0	MC_STATUS_MSCOD
402H	1026	MC0_ADDR	
403H	1027	MC0_MISC	Defined in MCA architecture but not implemented in the P6 family processors
404H	1028	MC1_CTL	
405H	1029	MC1_STATUS	Bit definitions same as MC0_STATUS
406H	1030	MC1_ADDR	
407H	1031	MC1_MISC	Defined in MCA architecture but not implemented in the P6 family processors
408H	1032	MC2_CTL	
409H	1033	MC2_STATUS	Bit definitions same as MC0_STATUS
40AH	1034	MC2_ADDR	
40BH	1035	MC2_MISC	Defined in MCA architecture but not implemented in the P6 family processors
40CH	1036	MC4_CTL	
40DH	1037	MC4_STATUS	Bit definitions same as MC0_STATUS, except bits 0, 4, 57, and 61 are hardcoded to 1.
40EH	1038	MC4_ADDR	Defined in MCA architecture but not implemented in P6 Family processors
40FH	1039	MC4_MISC	Defined in MCA architecture but not implemented in the P6 family processors
410H	1040	MC3_CTL	
411H	1041	MC3_STATUS	Bit definitions same as MC0_STATUS
412H	1042	MC3_ADDR	
413H	1043	MC3_MISC	Defined in MCA architecture but not implemented in the P6 family processors

NOTES:

1. Bit 0 of this register has been redefined several times, and is no longer used in P6 family processors.
2. The processor number feature may be disabled by setting bit 21 of the BBL_CR_CTL MSR (model-specific register address 119h) to “1”. Once set, bit 21 of the BBL_CR_CTL may not be cleared. This bit is write-once. The processor number feature will be disabled until the processor is reset.
3. The Pentium III processor will prevent FSB frequency overclocking with a new shutdown mechanism. If the FSB frequency selected is greater than the internal FSB frequency the processor will shutdown. If the FSB selected is less than the internal FSB frequency the BIOS may choose to use bit 11 to implement its own shutdown policy.

B.4. MSRS IN PENTIUM PROCESSORS

The following MSRs are defined for the Pentium processors. The P5_MC_ADDR, P5_MC_TYPE, and TSC MSRs (named IA32_P5_MC_ADDR, IA32_P5_MC_TYPE, and IA32_TIME_STAMP_COUNTER in the Pentium 4 processor) are architectural; that is, code that accesses these registers will run on Pentium 4 and P6 family processors without generating exceptions (see Section B.5., “Architectural MSRs”). The CESR, CTR0, and CTR1 MSRs are unique to Pentium processors; code that accesses these registers will generate exceptions on Pentium 4 and P6 family processors.

Table B-4. MSRs in the Pentium Processor

Register Address		Register Name	Bit Description
Hex	Dec		
0H	0	P5_MC_ADDR	See Section 14.7.2., “Pentium Processor Machine-Check Exception Handling”
1H	1	P5_MC_TYPE	See Section 14.7.2., “Pentium Processor Machine-Check Exception Handling”
10H	16	TSC	See Section 15.7., “Time-Stamp Counter”
11H	17	CESR	See Section 15.12.1., “Control and Event Select Register (CESR)”
12H	18	CTR0	Section 15.12.3., “Events Counted”
13H	19	CTR1	Section 15.12.3., “Events Counted”

B.5. ARCHITECTURAL MSRS

Many of the MSRs shown in Tables B-1, B-3, and B-4 have been carried over from one family of IA-32 processors to the next, and are now considered part of the IA-32 architecture. Beginning with the Pentium 4 processor, these “architectural MSRs” were renamed and given the prefix “IA32_”. Table B-5 lists the architectural MSRs, their addresses, their current names, their names in previous IA-32 processors, and the IA-32 processor family in which they were introduced. Those MSRs that are listed in Tables B-1, B-3, and B-4 but not listed in Table B-5 are considered machine specific (and given the prefix “MSR_” for Pentium 4 processors). Code that accesses a machine specified MSR and that is executed on a processor that does not support that MSR will generate an exception.

Table B-5. IA-32 Architectural MSRs

Register Address		Architectural Name	Former Name	IA-32 Processor Family Introduced In
Hex	Decimal			
0H	0	IA32_P5_MC_ADDR	P5_MC_ADDR	Pentium Processor
1H	1	IA32_P5_MC_TYPE	P5_MC_TYPE	Pentium Processor
10H	16	IA32_TIME_STAMP_COUNTER	TSC	Pentium Processor
17H	23	IA32_PLATFORM_ID	MSR_PLATFORM_ID	P6 Family Processors
1BH	27	IA32_APIC_BASE	APIC_BASE	P6 Family Processors
79H	121	IA32_BIOS_UPDT_TRIG	BIOS_UPDT_TRIG	P6 Family Processors
8BH	139	IA32_BIOS_SIGN_ID	BIOS_SIGN/BBL_CR_D3	P6 Family Processors
FEH	254	IA32_MTRRCAP	MTRRcap	P6 Family Processors
119H	281	IA32_MISC_CTL	MSR_MISC_CTL	P6 Family Processors
174H	372	IA32_SYSENTER_CS	SYSENTER_CS_MSR	P6 Family Processors
175H	373	IA32_SYSENTER_ESP	SYSENTER_ESP_MSR	P6 Family Processors
176H	374	IA32_SYSENTER_EIP	SYSENTER_EIP_MSR	P6 Family Processors
179H	377	IA32_MCG_CAP	MCG_CAP	P6 Family Processors
17AH	378	IA32_MCG_STATUS	MCG_STATUS	P6 Family Processors
17BH	379	IA32_MCG_CTL	MCG_CTL	P6 Family Processors
180H	384	IA32_MCG_EAX		Pentium 4 Processor
181H	385	IA32_MCG_EBX		Pentium 4 Processor
182H	386	IA32_MCG_ECX		Pentium 4 Processor
183H	387	IA32_MCG_EDX		Pentium 4 Processor
184H	388	IA32_MCG_ESI		Pentium 4 Processor
185H	389	IA32_MCG EDI		Pentium 4 Processor
186H	390	IA32_MCG_EBP		Pentium 4 Processor

Table B-5. IA-32 Architectural MSRs (Contd.)

Register Address		Architectural Name	Former Name	IA-32 Processor Family Introduced In
Hex	Decimal			
187H	391	IA32_MCG_ESP		Pentium 4 Processor
188H	392	IA32_MCG_EFLAGS		Pentium 4 Processor
189H	393	IA32_MCG_EIP		Pentium 4 Processor
18AH	394	IA32_MCG_MISC		Pentium 4 Processor
19AH	410	IA32_THERM_CONTROL		Pentium 4 Processor
19BH	411	IA32_THERM_INTERRUPT		Pentium 4 Processor
19CH	412	IA32_THERM_STATUS		Pentium 4 Processor
1A0H	416	IA32_MISC_ENABLE		Pentium 4 Processor
1D9H	473	IA32_DEBUGCTL	DEBUGCTLMSR	P6 Family Processors
200H	512	IA32_MTRR_PHYSBASE0	MTRRphysBase0	P6 Family Processors
201H	513	IA32_MTRR_PHYSMASK0	MTRRphysMask0	P6 Family Processors
202H	514	IA32_MTRR_PHYSBASE1	MTRRphysBase1	P6 Family Processors
203H	515	IA32_MTRR_PHYSMASK1	MTRRphysMask1	P6 Family Processors
204H	516	IA32_MTRR_PHYSBASE2	MTRRphysBase2	P6 Family Processors
205H	517	IA32_MTRR_PHYSMASK2	MTRRphysMask2	P6 Family Processors
206H	518	IA32_MTRR_PHYSBASE3	MTRRphysBase3	P6 Family Processors
207H	519	IA32_MTRR_PHYSMASK3	MTRRphysMask3	P6 Family Processors
208H	520	IA32_MTRR_PHYSBASE4	MTRRphysBase4	P6 Family Processors
209H	521	IA32_MTRR_PHYSMASK4	MTRRphysMask4	P6 Family Processors
20AH	522	IA32_MTRR_PHYSBASE5	MTRRphysBase5	P6 Family Processors
20BH	523	IA32_MTRR_PHYSMASK5	MTRRphysMask5	P6 Family Processors
20CH	524	IA32_MTRR_PHYSBASE6	MTRRphysBase6	P6 Family Processors
20DH	525	IA32_MTRR_PHYSMASK6	MTRRphysMask6	P6 Family Processors
20EH	526	IA32_MTRR_PHYSBASE7	MTRRphysBase7	P6 Family Processors
20FH	527	IA32_MTRR_PHYSMASK7	MTRRphysMask7	P6 Family Processors
250H	592	IA32_MTRR_FIX64K_00000	MTRRfix64K_00000	P6 Family Processors
258H	600	IA32_MTRR_FIX16K_80000	MTRRfix16K_80000	P6 Family Processors
259H	601	IA32_MTRR_FIX16K_A0000	MTRRfix16K_A0000	P6 Family Processors
268H	616	IA32_MTRR_FIX4K_C0000	MTRRfix4K_C0000	P6 Family Processors
269H	617	IA32_MTRR_FIX4K_C8000	MTRRfix4K_C8000	P6 Family Processors
26AH	618	IA32_MTRR_FIX4K_D0000	MTRRfix4K_D0000	P6 Family Processors
26BH	619	IA32_MTRR_FIX4K_D8000	MTRRfix4K_D8000	P6 Family Processors

Table B-5. IA-32 Architectural MSRs (Contd.)

Register Address		Architectural Name	Former Name	IA-32 Processor Family Introduced In
Hex	Decimal			
26CH	620	IA32_MTRR_FIX4K_E0000	MTRRfix4K_E0000	P6 Family Processors
26DH	621	IA32_MTRR_FIX4K_E8000	MTRRfix4K_E8000	P6 Family Processors
26EH	622	IA32_MTRR_FIX4K_F0000	MTRRfix4K_F0000	P6 Family Processors
26FH	623	IA32_MTRR_FIX4K_F8000	MTRRfix4K_F8000	P6 Family Processors
277H	631	IA32_CR_PAT	IA32_CR_PAT	P6 Family Processors
2FFH	767	IA32_MTRR_DEF_TYPE	MTRRdefType	P6 Family Processors
3F1H	1009	IA32_PEBS_ENABLE		Pentium 4 Processor
400H	1024	IA32_MC0_CTL	MC0_CTL	P6 Family Processors
401H	1025	IA32_MC0_STATUS	MC0_STATUS	P6 Family Processors
402H	1026	IA32_MC0_ADDR	MC0_ADDR	P6 Family Processors
403H	1027	IA32_MC0_MISC	MC0_MISC	P6 Family Processors
404H	1028	IA32_MC1_CTL	MC1_CTL	P6 Family Processors
405H	1029	IA32_MC1_STATUS	MC1_STATUS	P6 Family Processors
406H	1030	IA32_MC1_ADDR	MC1_ADDR	P6 Family Processors
407H	1031	IA32_MC1_MISC	MC1_MISC	P6 Family Processors
408H	1032	IA32_MC2_CTL	MC2_CTL	P6 Family Processors
409H	1033	IA32_MC2_STATUS	MC2_STATUS	P6 Family Processors
40AH	1034	IA32_MC2_ADDR	MC2_ADDR	P6 Family Processors
40BH	1035	IA32_MC2_MISC	MC2_MISC	P6 Family Processors
40CH	1036	IA32_MC3_CTL	MC3_CTL	P6 Family Processors
40DH	1037	IA32_MC3_STATUS	MC3_STATUS	P6 Family Processors
40EH	1038	IA32_MC3_ADDR	MC3_ADDR	P6 Family Processors
40FH	1039	IA32_MC3_MISC	MC3_MISC	P6 Family Processors
600H	1536	IA32_DS_AREA		Pentium 4 Processor

C

Multiple-Processor (MP) Bootup Sequence Example (Specific to P6 Family Processors)

APPENDIX C

MP INITIALIZATION FOR P6 FAMILY PROCESSORS

This appendix describes the MP initialization process for systems that use multiple P6 family processors. This process uses the MP initialization protocol that was introduced with the Pentium Pro processor (see Section 7.5., *Multiple-Processor (MP) Initialization*). For P6 family processors, this protocol is typically used to boot 2 or 4 processors that reside on single system bus; however, it can support from 2 to 15 processors in a multi-clustered system when the APIC busses are tied together. Larger systems are not supported.

C.1. OVERVIEW OF THE MP INITIALIZATION PROCESS FOR P6 FAMILY PROCESSORS

During the execution of the MP initialization protocol, one processor is selected as the bootstrap processor (BSP) and the remaining processors are designated as application processors (APs), see Section 7.5.1., *BSP and AP Processors*. Thereafter, the BSP manages the initialization of itself and the APs. This initialization includes executing BIOS initialization code and operating-system initialization code.

The MP protocol imposes the following requirements and restrictions on the system:

- An APIC clock (APICLK) must be provided.
- The MP protocol will be executed only after a power-up or RESET. If the MP protocol has been completed and a BSP has been chosen, subsequent INITs (either to a specific processor or system wide) do not cause the MP protocol to be repeated. Instead, each processor examines its BSP flag (in the APIC_BASE MSR) to determine whether it should execute the BIOS boot-strap code (if it is the BSP) or enter a wait-for-SIPI state (if it is an AP).
- All devices in the system that are capable of delivering interrupts to the processors must be inhibited from doing so for the duration of the MP initialization protocol. The time during which interrupts must be inhibited includes the window between when the BSP issues an INIT-SIPI-SIPI sequence to an AP and when the AP responds to the last SIPI in the sequence.

The following special-purpose interprocessor interrupts (IPIs) are used during the boot phase of the MP initialization protocol. These IPIs are broadcast on the APIC bus.

- Boot IPI (BIPI)—Initiates the arbitration mechanism that selects a BSP from the group of processors on the system bus and designates the remainder of the processors as APs. Each processor on the system bus broadcasts a BIPI to all the processors following a power-up or RESET.

- Final Boot IPI (FIPI)—Initiates the BIOS initialization procedure for the BSP. This IPI is broadcast to all the processors on the system bus, but only the BSP responds to it. The BSP responds by beginning execution of the BIOS initialization code at the reset vector.
- Startup IPI (SIPI)—Initiates the initialization procedure for an AP. The SIPI message contains a vector to the AP initialization code in the BIOS.

Table C-1 describes the various fields of the boot phase IPIs.

Table C-1. Boot Phase IPI Message Format

Type	Destination Field	Destination Shorthand	Trigger Mode	Level	Destination Mode	Delivery Mode	Vector (Hex)
BIPI	Not used	All including self	Edge	Deassert	Don't Care	Fixed (000)	40 to 4E*
FIPI	Not used	All including self	Edge	Deassert	Don't Care	Fixed (000)	10
SIPI	Used	All excluding self	Edge	Assert	Physical	StartUp (110)	00 to FF

NOTE:

* For all P6 family processors.

For BIPI messages, the lower 4 bits of the vector field contain the APIC ID of the processor issuing the message and the upper 4 bits contain the “generation ID” of the message. All P6 family processor will have a generation ID of 4H. BIPIs will therefore use vector values ranging from 40H to 4EH (4FH can not be used because FH is not a valid APIC ID).

C.2. MP INITIALIZATION PROTOCOL ALGORITHM

Following a power-up or RESET of a system, the P6 family processors in the system execute the MP initialization protocol algorithm to initialize each of the processors on the system bus. In the course of executing this algorithm, the following boot-up and initialization operations are carried out:

1. Each processor on the system bus is assigned a unique APIC ID, based on system topology (see Section 7.5.5., *Identifying the Processors in an MP System*). This ID is written into the local APIC ID register for each processor.
2. Each processor executes its internal BIST simultaneously with the other processors on the system bus. Upon completion of the BIST (at T0), each processor broadcasts a BIPI to “all including self” (see Figure C-1).
3. APIC arbitration hardware causes all the APICs to respond to the BIPIs one at a time (at T1, T2, T3, and T4).
4. When the first BIPI is received (at time T1), each APIC compares the four least significant bits of the BIPI’s vector field with its APIC ID. If the vector and APIC ID match, the processor selects itself as the BSP by setting the BSP flag in its IA32_APIC_BASE MSR.

If the vector and APIC ID do not match, the processor selects itself as an AP by entering the “wait for SIPI” state. (Note that in Figure C-1, the BIPI from processor 1 is the first BIPI to be handled, so processor 1 becomes the BSP.)

5. The newly established BSP broadcasts an FIPI message to “all including self.” The FIPI is guaranteed to be handled only after the completion of the BIPIs that were issued by the non-BSP processors.

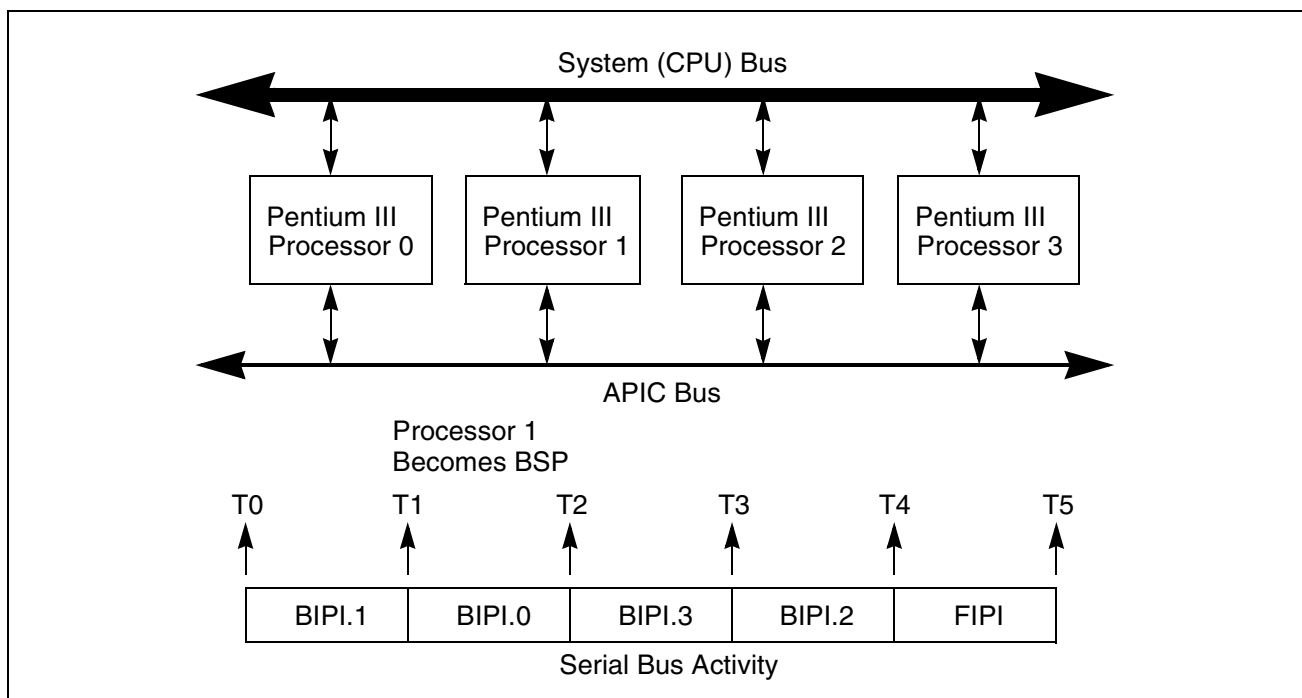


Figure C-1. MP System With Multiple Pentium III Processors

6. After the BSP has been established, the outstanding BIPIs are received one at a time (at T2, T3, and T4) and ignored by all processors.
7. When the FIPI is finally received (at T5), only the BSP responds to it. It responds by fetching and executing BIOS boot-strap code, beginning at the reset vector (physical address FFFF FFF0H).
8. As part of the boot-strap code, the BSP creates an ACPI table and an MP table and adds its initial APIC ID to these tables as appropriate.
9. At the end of the boot-strap procedure, the BSP broadcasts a SIPI message to all the APs in the system. Here, the SIPI message contains a vector to the BIOS AP initialization code (at 000V V000H, where VV is the vector contained in the SIPI message).
10. All APs respond to the SIPI message by racing to a BIOS initialization semaphore. The first one to the semaphore begins executing the initialization code. (See MP init code for semaphore implementation details.) As part of the AP initialization procedure, the AP adds its APIC ID number to the ACPI and MP tables as appropriate. At the completion of the initialization procedure, the AP executes a CLI instruction (to clear the IF flag in the EFLAGS register) and halts itself.

11. When each of the APs has gained access to the semaphore and executed the AP initialization code and all written their APIC IDs into the appropriate places in the ACPI and MP tables, the BSP establishes a count for the number of processors connected to the system bus, completes executing the BIOS boot-strap code, and then begins executing operating-system boot-strap and start-up code.
12. While the BSP is executing operating-system boot-strap and start-up code, the APs remain in the halted state. In this state they will respond only to INITs, NMIs, and SMIs. They will also respond to snoops and to assertions of the STPCLK# pin.

See Section 7.5.4., *MP Initialization Example*, for an annotated example the use of the MP protocol to boot IA-32 processors in an MP. This code should run on any IA-32 processor that used the MP protocol.

C.2.1. Error Detection and Handling During the MP Initialization Protocol

Errors may occur on the APIC bus during the MP initialization phase. These errors may be transient or permanent and can be caused by a variety of failure mechanisms (for example, broken traces, soft errors during bus usage, etc.). All serial bus related errors will result in an APIC checksum or acceptance error.

The MP initialization protocol makes the following assumptions regarding errors that occur during initialization:

- If errors are detected on the APIC bus during execution of the MP initialization protocol, the processors that detect the errors are shut down.
- The MP initialization protocol will be executed by processors even if they fail their BIST sequences.

D

Programming the LINT0 and LINT1 Inputs

APPENDIX D

PROGRAMMING THE LINT0 AND LINT1 INPUTS

The following procedure describes how to program the LINT0 and LINT1 local APIC pins on a processor after multiple processors have been booted and initialized (as described in Appendix C, *MP Initialization For P6 Family Processors* and Appendix D, *Programming the LINT0 and LINT1 Inputs*. In this example, LINT0 is programmed to be the ExtINT pin and LINT1 is programmed to be the NMI pin.

D.1. CONSTANTS

The following constants are defined:

```
LVT1      EQU 0FEE00350H
LVT2      EQU 0FEE00360H
LVT3      EQU 0FEE00370H
SVR       EQU 0FEE000F0H
```

D.2. LINT[0:1] PINS PROGRAMMING PROCEDURE

Use the following to program the LINT[1:0] pins:

1. Mask 8259 interrupts.
2. Enable APIC via SVR (spurious vector register) if not already enabled.
3. Program LVT1 as an ExtINT which delivers the signal to the INTR signal of all processors cores listed in the destination as an interrupt that originated in an externally connected interrupt controller.

```
MOV ESI, SVR      ; address of SVR
MOV EAX, [ESI]
OR  EAX, APIC_ENABLED; set bit 8 to enable (0 on reset)
MOV [ESI], EAX

MOV ESI, LVT1
MOV EAX, [ESI]
AND EAX, 0FFFE58FFH ; mask off bits 8-10, 12, 14 and 16
OR  EAX, 700H       ; Bit 16=0 for not masked, Bit 15=0 for edge
                        ; triggered, Bit 13=0 for high active input
                        ; polarity, Bits 8-10 are 111b for ExtINT
MOV [ESI], EAX      ; Write to LVT1
```

4. Program LVT2 as NMI, which delivers the signal on the NMI signal of all processor cores listed in the destination.

```
MOV ESI, LVT2
MOV EAX, [ESI]
AND EAX, 0FFFE58FFH      ; mask off bits 8-10 and 15
OR  EAX, 000000400H      ; Bit 16=0 for not masked, Bit 15=0 edge
                           ; triggered, Bit 13=0 for high active input
                           ; polarity, Bits 8-10 are 100b for NMI
MOV [ESI], EAX           ; Write to LVT2
;Unmask 8259 interrupts and allow NMI.
```

E

Interpreting Machine-Check Error Codes

APPENDIX E

INTERPRETING MACHINE-CHECK ERROR CODES

Table E-1 gives an example of how to interpret the MCA error code, model-specific error code, and other information error code fields for machine-check errors that occur on the external bus. This information can be used to design a machine-check exception handler for the processor that offers greater granularity for the external bus errors.

NOTE

The information in Table E-1 is implementation-specific for the P6 family processors. The error information returned for a Pentium 4 processor is considerably different.

Table E-1. Encoding of the MC*i*_STATUS Register for External Bus Errors

Bit No.	Bit Function	Bit Description
0-1	MCA Error Code	Undefined.
2-3	MCA Error Code	Bit 2 is set to 1 if the access was a special cycle. Bit 3 is set to 1 if the access was a special cycle OR a I/O cycle.
4-7	MCA Error Code	00WR; W = 1 for writes, R = 1 for reads.
8-9	MCA Error Code	Undefined.
10	MCA Error Code	Set to 0 for all EBL errors. Set to 1 for internal watch-dog timer time-out. For a watch-dog timer time-out, all the MCACOD bits except this bit are set to 0. A watch-dog timer time-out only occurs if the BINIT driver is enabled.
11	MCA Error Code	Set to 1 for EBL errors. Set to 0 for internal watch-dog timer time-out.
12-15	MCA Error Code	Reserved.
16-18	Model-Specific Error Code	Reserved.

Table E-1. Encoding of the MC_i_STATUS Register for External Bus Errors (Contd.)

Bit No.	Bit Function	Bit Description
19-24	Model-Specific Error Code	000000 for BQ_DCU_READ_TYPE error. 000010 for BQ_IFU_DEMAND_TYPE error. 000011 for BQ_IFU_DEMAND_NC_TYPE error. 000100 for BQ_DCU_RFO_TYPE error. 000101 for BQ_DCU_RFO_LOCK_TYPE error. 000110 for BQ_DCU_ITOM_TYPE error. 001000 for BQ_DCU_WB_TYPE error. 001010 for BQ_DCU_WCEVICT_TYPE error. 001011 for BQ_DCU_WCLINE_TYPE error. 001100 for BQ_DCU_BTM_TYPE error. 001101 for BQ_DCU_INTACK_TYPE error. 001110 for BQ_DCU_INVALL2_TYPE error. 001111 for BQ_DCU_FLUSH2_TYPE error. 010000 for BQ_DCU_PART_RD_TYPE error. 010010 for BQ_DCU_PART_WR_TYPE error. 010100 for BQ_DCU_SPEC_CYC_TYPE error. 011000 for BQ_DCU_IO_RD_TYPE error. 011001 for BQ_DCU_IO_WR_TYPE error. 011100 for BQ_DCU_LOCK_RD_TYPE error. 011110 for BQ_DCU_SPLOCK_RD_TYPE error. 011101 for BQ_DCU_LOCK_WR_TYPE error.
27-25	Model-Specific Error Code	000 for BQ_ERR_HARD_TYPE error. 001 for BQ_ERR_DOUBLE_TYPE error. 010 for BQ_ERR_AERR2_TYPE error. 100 for BQ_ERR_SINGLE_TYPE error. 101 for BQ_ERR_AERR1_TYPE error.
28	Model-Specific Error Code	1 if FRC error is active.
29	Model-Specific Error Code	1 if BERR is driven.
30	Model-Specific Error Code	1 if BINIT is driven for this processor.
31	Model-Specific Error Code	Reserved.
32-34	Other Information	Reserved.
35	Other Information BINIT	1 if BINIT is received from external bus.
36	Other Information RESPONSE PARITY ERROR	This bit is asserted in the MC _i _STATUS register if this component has received a parity error on the RS[2:0]# pins for a response transaction. The RS signals are checked by the RSP# external pin.

Table E-1. Encoding of the MC_i_STATUS Register for External Bus Errors (Contd.)

Bit No.	Bit Function	Bit Description
37	Other Information BUS BINIT	This bit is asserted in the MC _i _STATUS register if this component has received a hard error response on a split transaction (one access that has needed to be split across the 64-bit external bus interface into two accesses).
38	Other Information TIMEOUT BINIT	This bit is asserted in the MC_i_STATUS register if this component has experienced a ROB time-out, which indicates that no microinstruction has been retired for a predetermined period of time. A ROB time-out occurs when the 15-bit ROB time-out counter carries a 1 out of its high order bit. The timer is cleared when a microinstruction retires, an exception is detected by the core processor, RESET is asserted, or when a ROB BINIT occurs. The ROB time-out counter is prescaled by the 8-bit PIC timer which is a divide by 128 of the bus clock (the bus clock is 1:2, 1:3, 1:4 the core clock). When a carry out of the 8-bit PIC timer occurs, the ROB counter counts up by one. While this bit is asserted, it cannot be overwritten by another error.
39-41	Other Information	Reserved.
42	Other Information HARD ERROR	This bit is asserted in the MC _i _STATUS register if this component has initiated a bus transactions which has received a hard error response. While this bit is asserted, it cannot be overwritten.
43	Other Information IERR	This bit is asserted in the MC _i _STATUS register if this component has experienced a failure that causes the IERR pin to be asserted. While this bit is asserted, it cannot be overwritten.
44	Other Information AERR	This bit is asserted in the MC _i _STATUS register if this component has initiated 2 failing bus transactions which have failed due to Address Parity Errors (AERR asserted). While this bit is asserted, it cannot be overwritten.
45	Other Information UECC	Uncorrectable ECC error bit is asserted in the MC _i _STATUS register for uncorrected ECC errors. While this bit is asserted, the ECC syndrome field will not be overwritten.
46	Other Information CECC	The correctable ECC error bit is asserted in the MC _i _STATUS register for corrected ECC errors.
47-54	Other Information SYNDROME	The ECC syndrome field in the MC_i_STATUS register contains the 8-bit ECC syndrome only if the error was a correctable/uncorrectable ECC error, and there wasn't a previous valid ECC error syndrome logged in the MC_i_STATUS register. A previous valid ECC error in MC_i_STATUS is indicated by MC_i_STATUS.bit45 (uncorrectable error occurred) being asserted. After processing an ECC error, machine-check handling software should clear MC_i_STATUS.bit45 so that future ECC error syndromes can be logged.
55-56	Other Information	Reserved.

F

APIC Bus Message Formats

APPENDIX F

APIC BUS MESSAGE FORMATS

This appendix describes the message formats used when transmitting messages on the serial APIC bus. The information described here pertains only to the Pentium and P6 family processors.

F.1. BUS MESSAGE FORMATS

The local and I/O APICs transmit three types of messages on the serial APIC bus: EOI message, short message, and non-focused lowest priority message. The purpose of each type of message and its format are described below.

F.2. EOI MESSAGE

Local APICs send 14-cycle EOI messages to the I/O APIC to indicate that a level triggered interrupt has been accepted by the processor. This interrupt, in turn, is a result of software writing into the EOI register of the local APIC. Table F-1 shows the cycles in an EOI message.

Table F-1. EOI Message (14 Cycles)

Cycle	Bit1	Bit0	
1	1	1	11 = EOI
2	ArbID3	0	Arbitration ID bits 3 through 0
3	ArbID2	0	
4	ArbID1	0	
5	ArbID0	0	
6	V7	V6	Interrupt vector V7 - V0
7	V5	V4	
8	V3	V2	
9	V1	V0	
10	C	C	Checksum for cycles 6 - 9
11	0	0	
12	A	A	Status Cycle 0
13	A1	A1	Status Cycle 1
14	0	0	Idle

The checksum is computed for cycles 6 through 9. It is a cumulative sum of the 2-bit (Bit1:Bit0) logical data values. The carry out of all but the last addition is added to the sum. If any APIC computes a different checksum than the one appearing on the bus in cycle 10, it signals an error, driving 11 on the APIC bus during cycle 12. In this case, the APICs disregard the message. The sending APIC will receive an appropriate error indication (see Section 8.5.3., “Error Handling”) and resend the message. The status cycles are defined in Table F-4.

F.2.1. Short Message

Short messages (21-cycles) are used for sending fixed, NMI, SMI, INIT, start-up, ExtINT and lowest-priority-with-focus interrupts. Table F-2 shows the cycles in a short message.

Table F-2. Short Message (21 Cycles)

Cycle	Bit1	Bit0	
1	0	1	0 1 = normal
2	ArbID3	0	Arbitration ID bits 3 through 0
3	ArbID2	0	
4	ArbID1	0	
5	ArbID0	0	
6	DM	M2	DM = Destination Mode
7	M1	M0	M2-M0 = Delivery mode
8	L	TM	L = Level, TM = Trigger Mode
9	V7	V6	V7-V0 = Interrupt Vector
10	V5	V4	
11	V3	V2	
12	V1	V0	
13	D7	D6	D7-D0 = Destination
14	D5	D4	
15	D3	D2	
16	D1	D0	
17	C	C	Checksum for cycles 6-16
18	0	0	
19	A	A	Status cycle 0
20	A1	A1	Status cycle 1
21	0	0	Idle

If the physical delivery mode is being used, then cycles 15 and 16 represent the APIC ID and cycles 13 and 14 are considered don't care by the receiver. If the logical delivery mode is being used, then cycles 13 through 16 are the 8-bit logical destination field.

For shorthands of “all-incl-self” and “all-excl-self,” the physical delivery mode and an arbitration priority of 15 (D0:D3 = 1111) are used. The agent sending the message is the only one required to distinguish between the two cases. It does so using internal information.

When using lowest priority delivery with an existing focus processor, the focus processor identifies itself by driving 10 during cycle 19 and accepts the interrupt. This is an indication to other APICs to terminate arbitration. If the focus processor has not been found, the short message is extended on-the-fly to the non-focused lowest-priority message. Note that except for the EOI message, messages generating a checksum or an acceptance error (see Section 8.5.3., “Error Handling”) terminate after cycle 21.

F.2.2. Non-focused Lowest Priority Message

These 34-cycle messages (see Table F-3) are used in the lowest priority delivery mode when a focus processor is not present. Cycles 1 through 20 are same as for the short message. If during the status cycle (cycle 19) the state of the (A:A) flags is 10B, a focus processor has been identified, and the short message format is used (see Table F-2). If the (A:A) flags are set to 00B, lowest priority arbitration is started and the 34-cycles of the non-focused lowest priority message are competed. For other combinations of status flags, refer to Section F.2.3., “APIC Bus Status Cycles”.

Table F-3. Non-Focused Lowest Priority Message (34 Cycles)

Cycle	Bit0	Bit1	
1	0	1	0 1 = normal
2	ArbID3	0	Arbitration ID bits 3 through 0
3	ArbID2	0	
4	ArbID1	0	
5	ArbID0	0	
6	DM	M2	DM = Destination mode
7	M1	M0	M2-M0 = Delivery mode
8	L	TM	L = Level, TM = Trigger Mode
9	V7	V6	V7-V0 = Interrupt Vector
10	V5	V4	
11	V3	V2	
12	V1	V0	
13	D7	D6	D7-D0 = Destination
14	D5	D4	
15	D3	D2	
16	D1	D0	
17	C	C	Checksum for cycles 6-16

Table F-3. Non-Focused Lowest Priority Message (34 Cycles) (Contd.)

Cycle	Bit0	Bit1	
18	0	0	
19	A	A	Status cycle 0
20	A1	A1	Status cycle 1
21	P7	0	P7 - P0 = Inverted Processor Priority
22	P6	0	
23	P5	0	
24	P4	0	
25	P3	0	
26	P2	0	
27	P1	0	
28	P0	0	
29	ArbID3	0	Arbitration ID 3 -0
30	ArbID2	0	
31	ArbID1	0	
32	ArbID0	0	
33	A2	A2	Status Cycle
34	0	0	Idle

Cycles 21 through 28 are used to arbitrate for the lowest priority processor. The processors participating in the arbitration drive their inverted processor priority on the bus. Only the local APICs having free interrupt slots participate in the lowest priority arbitration. If no such APIC exists, the message will be rejected, requiring it to be tried at a later time.

Cycles 29 through 32 are also used for arbitration in case two or more processors have the same lowest priority. In the lowest priority delivery mode, all combinations of errors in cycle 33 (A2 A2) will set the “accept error” bit in the error status register (see Figure 8-9). Arbitration priority update is performed in cycle 20, and is not affected by errors detected in cycle 33. Only the local APIC that wins in the lowest priority arbitration, drives cycle 33. An error in cycle 33 will force the sender to resend the message.

F.2.3. APIC Bus Status Cycles

Certain cycles within an APIC bus message are status cycles. During these cycles the status flags (A:A) and (A1:A1) are examined. Table F-4 shows how these status flags are interpreted, depending on the current delivery mode and existence of a focus processor.

Table F-4. APIC Bus Status Cycles Interpretation

Delivery Mode	A Status	A1 Status	A2 Status	Update ArbID and Cycle#	Message Length	Retry
EOI	00: CS_OK	10: Accept	XX:	Yes, 13	14 Cycle	No
	00: CS_OK	11: Retry	XX:	Yes, 13	14 Cycle	Yes
	00: CS_OK	0X: Accept Error	XX:	No	14 Cycle	Yes
	11: CS_Error	XX:	XX:	No	14 Cycle	Yes
	10: Error	XX:	XX:	No	14 Cycle	Yes
	01: Error	XX:	XX:	No	14 Cycle	Yes
Fixed	00: CS_OK	10: Accept	XX:	Yes, 20	21 Cycle	No
	00: CS_OK	11: Retry	XX:	Yes, 20	21 Cycle	Yes
	00: CS_OK	0X: Accept Error	XX:	No	21 Cycle	Yes
	11: CS_Error	XX:	XX:	No	21 Cycle	Yes
	10: Error	XX:	XX:	No	21 Cycle	Yes
	01: Error	XX:	XX:	No	21 Cycle	Yes
NMI, SMI, INIT, ExtINT, Start-Up	00: CS_OK	10: Accept	XX:	Yes, 20	21 Cycle	No
	00: CS_OK	11: Retry	XX:	Yes, 20	21 Cycle	Yes
	00: CS_OK	0X: Accept Error	XX:	No	21 Cycle	Yes
	11: CS_Error	XX:	XX:	No	21 Cycle	Yes
	10: Error	XX:	XX:	No	21 Cycle	Yes
	01: Error	XX:	XX:	No	21 Cycle	Yes
Lowest	00: CS_OK, NoFocus	11: Do Lowest	10: Accept	Yes, 20	34 Cycle	No
	00: CS_OK, NoFocus	11: Do Lowest	11: Error	Yes, 20	34 Cycle	Yes
	00: CS_OK, NoFocus	11: Do Lowest	0X: Error	Yes, 20	34 Cycle	Yes
	00: CS_OK, NoFocus	10: End and Retry	XX:	Yes, 20	34 Cycle	Yes
	00: CS_OK, NoFocus	0X: Error	XX:	No	34 Cycle	Yes
	10: CS_OK, Focus	XX:	XX:	Yes, 20	34 Cycle	No
	11: CS_Error	XX:	XX:	No	21 Cycle	Yes
	01: Error	XX:	XX:	No	21 Cycle	Yes



Index

Numerics

- 16-bit code, mixing with 32-bit code 17-1
- 32-bit code, mixing with 16-bit code 17-1
- 32-bit physical addressing
 - description of 3-20
 - overview 3-6
- 36-bit physical addressing
 - overview 3-6
 - using PSE-36 paging mechanism 3-34
 - using the PAE paging mechanism 3-28
- 8086
 - emulation, support for 16-1
 - processor, exceptions and interrupts 16-8
- 8086/8088 processor 18-7
- 8087 math coprocessor 18-8
- 82489DX 18-28
 - relationship with local and I/O APICs 8-5

A

- A (accessed) flag, page-table entries 3-26
- A20M# signal 16-3, 18-37
- Aborts
 - description of 5-6
 - restarting a program or task after 5-7
- AC (alignment check) flag, EFLAGS register . . 2-9, 5-49, 18-6
- Access rights
 - checking 2-21
 - checking caller privileges 4-29
 - description of 4-27
 - invalid values 18-24
- ADC instruction 7-5
- ADD instruction 7-5
- Address
 - size prefix 17-2
 - space, of task 6-18
- Address translation
 - 2-MByte pages, using 36-bit physical addressing 3-30
 - 4-KByte pages, using 32-bit physical addressing 3-20
 - 4-KByte pages, using 36-bit physical addressing 3-29
 - 4-MByte pages, using 32-bit physical addressing 3-21
 - 4-MByte pages, using 36-bit physical addressing 3-34
 - in real-address mode 16-3
 - logical to linear 3-7
 - overview 3-6
- Addressing, segments 1-6
- Advanced programmable interrupt controller (see I/O APIC or Local APIC)

Alignment

- alignment check exception 5-49, 5-50
 - checking 4-31
 - exception 18-14
 - Alignment check exception (#AC) 2-9, 5-49, 5-50, 18-14, 18-26
 - AM (alignment mask) flag, CR0 control register . . 2-9, 2-13, 18-22
 - AND instruction 7-5
 - APIC bus
 - arbitration mechanism and protocol . 8-31, 8-39
 - bus message format 8-40, F-1
 - diagram of 8-3, 8-4
 - EOI message format 8-18, F-1
 - message formats F-1
 - nonfocused lowest priority message F-3
 - short message format F-2
 - SMI message 13-2
 - status cycles F-5
 - structure of 8-5
 - APIC feature flag, CPUID instruction 8-9
 - APIC global enable flag, IA32_APIC_BASE MSR 8-11
 - APIC (see I/O APIC or Local APIC)
 - APIC_BASE_MSR 8-11
 - Arbitration
 - APIC bus 8-39
 - ARPL instruction 2-21, 4-31
 - Atomic operations
 - automatic bus locking 7-4
 - effects of a locked operation on internal processor caches 7-7
 - guaranteed, description of 7-3
 - overview of 7-2, 7-3, 7-4
 - software-controlled bus locking 7-4
 - At-retirement
 - counting 15-47
 - events 15-27, 15-29, 15-47, 15-56, A-28
 - Auto HALT restart
 - field, SMM 13-14
 - SMM 13-14
 - Automatic bus locking 7-4
- ## B
- B (busy) flag, TSS descriptor 6-7, 6-13, 6-14, 6-17, 6-18, 7-4
 - B (default stack size) flag, segment descriptor . . . 17-2, 18-35
 - B0-B3 (breakpoint condition detected) flags, DR6 register 15-4
 - Backlink (see Previous task link)
 - Base address fields, segment descriptor 3-11

BD (debug register access detected) flag, DR6 register 15-4, 15-9

Binary numbers 1-6

BINIT# signal 2-22

Bit order 1-4

BOUND instruction 2-4, 5-5, 5-26

BOUND range exceeded exception (#BR) 5-26

BP0#, BP1#, BP2#, and BP3# pins 15-21

Branch record

- branch trace message 15-16
- saving 15-13
- saving as a branch trace message 15-16
- structure 15-14
- structure of in BTS buffer 15-37

Branch trace message (see BTM)

Branch trace store (see BTS)

Breakpoint exception (#BP) 5-5, 5-24, 15-1, 15-10

Breakpoints

- breakpoint exception (#BP) 15-1
- data breakpoint 15-6
- data breakpoint exception conditions 15-9
- description of 15-1
- DR0-DR3 debug registers 15-3
- example 15-6
- exception 5-24
- field recognition 15-6
- general-detect exception condition 15-9
- instruction breakpoint 15-6
- instruction breakpoint exception condition 15-8
- I/O breakpoint exception conditions 15-9
- LEN0 - LEN3 (Length) fields, DR7 register 15-6
- R/W0-R/W3 (read/write) fields, DR7 register 15-5
- single-step exception condition 15-10
- task-switch exception condition 15-10

BS (single step) flag, DR6 register 15-4

BSP flag, IA32_APIC_BASE MSR 8-11

BSWAP instruction 18-4

BT (task switch) flag, DR6 register 15-4, 15-10

BTC instruction 7-4

BTF (single-step on branches) flag, DebugCtlMSR MSR 15-16, 15-21

BTMs (branch trace messages)

- description of 15-16
- enabling 15-13, 15-18
- TR (trace message enable) flag, IA32_DEBUGCTL MSR 15-13

BTR instruction 7-4

BTS buffer

- description of 15-35
- introduction to 15-11, 15-16
- records in 15-37
- setting up 15-18
- structure of 15-36

BTS instruction 7-4

BTS (branch trace store) facilities

- availability of 15-12

BTS_UNAVAILABLE flag,

- IA32_MISC_ENABLE MSR 15-35, B-10
- detection of 15-17
- introduction to 15-11
- setting up BTS buffer 15-18
- writing an interrupt service routine for 15-19

BTS_UNAVAILABLE flag, IA32_MISC_ENABLE MSR 15-35, B-10

Built-in self-test (BIST)

- description of 9-1
- performing 9-2

Bus

- errors, detected with machine-check architecture 14-15
- hold 18-39
- locking 7-3, 18-38

Byte order 1-4

C

C (conforming) flag, segment descriptor 4-14

C1 flag, x87 FPU status word 18-9, 18-18

C2 flag, x87 FPU status word 18-9

Cache control 10-24

- adaptive mode, L1 Data Cache 10-20
- cache management instructions 10-19, 10-20
- cache mechanisms in IA-32 processors 18-31
- caching terminology 10-4
- CD flag, CR0 control register 10-11, 18-23
- choosing a memory type 10-9
- flags and fields 10-10
- flushing TLBs 10-22

G (global) flag, page-directory entries 10-14, 10-23

G (global) flag, page-table entries 10-14, 10-23

internal caches 10-1

MemTypeGet() function 10-34

MemTypeSet() function 10-36

MESI protocol 10-4, 10-9

methods of caching available 10-5

MTRR initialization 10-33

MTRR precedences 10-32

MTRRs, description of 10-24

multiple-processor considerations 10-37

NW flag, CR0 control register 10-14, 18-23

operating modes 10-13

overview of 10-1

page attribute table (PAT) 10-39

PCD flag, CR3 control register 10-14

PCD flag, page-directory entries 10-14, 10-15, 10-39

PCD flag, page-table entries 10-14, 10-15, 10-39

PGE (page global enable) flag, CR4 control register 10-14

precedence of controls 10-15

preventing caching 10-18

protocol 10-9

PWT flag, CR3 control register	10-14	controlling the operand-size attribute for a call 17-7	
PWT flag, page-directory entries	10-14, 10-39	returning from	4-24
PWT flag, page-table entries	10-14, 10-39	Catastrophic shutdown detector	13-18, 13-20
remapping memory types	10-34	CC0 and CC1 (counter control) fields, CESR MSR (Pentium processor)	15-66
setting up memory ranges with MTRRs	10-27	CD (cache disable) flag, CR0 control register 2-13, 9-8, 10-11, 10-13, 10-15, 10-18, 10-37, 10-38, 18-22, 18-23, 18-32	
shared mode, L1 Data Cache	10-21	CESR (control and event select) MSR (Pentium processor)	15-65, 15-66
variable-range MTRRs	10-28	CLFLSH feature flag, CPUID instruction	9-9
Caches	2-6	CLFLUSH instruction	2-14, 7-8, 9-9, 10-20
cache hit	10-4	CLI instruction	5-9
cache line	10-4	Clocks	
cache line fill	10-4	counting processor clocks	15-52
cache write hit	10-4	Hyper-Threading Technology	15-52
description of	10-1	Nominal CPI	15-52
effects of a locked operation on internal processor caches	7-7	Non-Halted Clockticks	15-52
enabling	9-8	Non-Halted CPI	15-52
management, instructions	2-21, 10-19	Non-Sleep Clockticks	15-52
Caching		Time Stamp Counter	15-52
cache control protocol	10-9	CLTS instruction	2-20, 4-26
cache line	10-4	Cluster model, local APIC	8-29
cache management instructions	10-19	CMOVcc instructions	18-4
cache mechanisms in IA-32 processors	18-31	CMPXCHG instruction	7-4, 18-4
caching terminology	10-4	CMPXCHG8B instruction	7-4, 18-5
choosing a memory type	10-9	Code modules	
flushing TLBs	10-22	16 bit vs. 32 bit	17-2
implicit caching	10-21	mixing 16-bit and 32-bit code	17-1
internal caches	10-1	sharing data among mixed-size code segments 17-3	
L1 (level 1) cache	10-3	transferring control among mixed-size code segments	17-4
L2 (level 2) cache	10-3	Code segments	
L3 (level 3) cache	10-3	accessing data in	4-12
methods of caching available	10-5	accessing through a call gate	4-18
MTRRs, description of	10-24	description of	3-13
operating modes	10-13	descriptor format	4-3
overview of	10-1	descriptor layout	4-3
self-modifying code, effect on	10-21, 18-32	direct calls or jumps to	4-13
snooping	10-5	executable (defined)	3-11
store buffer	10-23	paging of	2-5
TLBs	10-4	pointer size	17-5
UC (strong uncacheable) memory type	10-5	privilege level checking when transferring program control between code segments . 4-12	
UC- (uncacheable) memory type	10-6	size	3-11
WB (write back) memory type	10-6	Compatibility	
WC (write combining) memory type	10-6	IA-32 architecture	18-1
WP (write protected) memory type	10-7	software	1-4
write-back caching	10-5	Condition code flags, x87 FPU status word compatibility information	18-9
WT (write through) memory type	10-6	Conforming code segments	
Call gates		accessing	4-15
16-bit, interlevel return from	18-35	C (conforming) flag	4-14
accessing a code segment through	4-18	description of	3-14
description of	4-17	Context, task (see Task state)	
for 16-bit and 32-bit code modules	17-2		
introduction to	2-3		
mechanism	4-19		
privilege level checking rules	4-20		
CALL instruction. 2-4, 3-9, 4-12, 4-13, 4-18, 4-24, 6-3, 6-12, 6-13, 17-7			
Caller access privileges, checking	4-29		
Calls			
between 16- and 32-bit code segments	17-4		

- Control registers
 - CR0 2-12
 - CR1 (reserved) 2-12
 - CR2 2-12
 - CR3 (PDBR) 2-5, 2-12
 - CR4 2-12
 - description of 2-12
 - introduction to 2-5
 - qualification of flags with CPUID instruction 2-18
 - Coprocessor segment overrun exception . . . 5-33, 18-14
 - Counter mask field, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors) . . . 15-63
 - CPL
 - description of 4-8
 - field, CS segment selector 4-3
 - CPUID instruction. 7-14, 15-23, 15-64, 18-3, 18-5, 18-39
 - qualification of CR4 control register flags . . 2-18
 - CR0 control register 18-8
 - description of 2-12
 - introduction to 2-5
 - state following processor reset 9-2
 - CR1 control register (reserved) 2-12
 - CR2 control register
 - description of 2-12
 - introduction to 2-5
 - CR3 control register (PDBR)
 - associated with a task. 6-1, 6-3
 - changing to access full extended physical address space 3-31
 - description of 2-12, 3-23
 - format with physical address extension enabled 3-29
 - in TSS 6-5, 6-19
 - introduction to 2-5
 - invalidation of non-global TLBs 3-37
 - loading during initialization 9-13
 - memory management 2-5
 - page directory base address 2-5
 - page table base address 2-4
 - CR4 control register
 - description of 2-12
 - enabling control functions 18-2
 - inclusion in IA-32 architecture 18-21
 - introduction to 2-5
 - CS register 18-13
 - saving on call to exception or interrupt handler 5-15
 - state following initialization 9-6
 - CTR0 and CTR1 (performance counters) MSRs (Pentium processor) 15-65, 15-68
 - Current privilege level (see CPL)
- D**
- D (default operation size) flag, segment descriptor 17-2, 18-35
 - D (dirty) flag, page-table entries 3-26
 - Data breakpoint exception conditions 15-9
 - Data segments
 - description of 3-13
 - descriptor layout 4-3
 - expand-down type 3-12
 - paging of 2-5
 - privilege level checking when accessing 4-10
 - DE (debugging extensions) flag, CR4 control register 2-17, 18-22, 18-24, 18-25
 - Debug exception (#DB) 5-9, 5-23, 6-6, 15-1, 15-7, 15-15, 15-22
 - Debug registers
 - description of 15-2
 - introduction to 2-5
 - loading 2-21
 - Debug store (see DS)
 - DebugCtlMSR MSR . . . 15-11, 15-19, 15-20, 15-22
 - Debugging facilities
 - debug registers 15-2
 - exceptions 15-7
 - last branch, interrupt, and exception recording 15-11, 15-19
 - masking debug exceptions 5-9
 - overview of 15-1
 - performance-monitoring counters 15-24
 - see DS (debug store) mechanism
 - DEC instruction 7-5
 - Denormal operand exception (#D) 18-11
 - Denormalized operand 18-15
 - Device-not-available exception (#NM) . . . 2-14, 2-20, 5-29, 5-30, 9-8, 18-13, 18-14
 - DIV instruction 5-22
 - Divide configuration register, local APIC 8-20
 - Divide-error exception (#DE) 5-22, 18-26
 - Double-fault exception (#DF) . . . 5-31, 5-32, 18-28
 - DPL (descriptor privilege level) field, segment descriptor 3-11, 4-2, 4-9
 - DR0-DR3 breakpoint-address registers 15-1, 15-3, 15-21, 15-22
 - DR4-DR5 debug registers 15-4, 18-25
 - DR6 debug status register 15-1, 15-4
 - B0-B3 (breakpoint condition detected) flags 15-4
 - BD (debug register access detected) flag 15-4
 - BS (single step) flag 15-4
 - BT (task switch) flag 15-4
 - reserved bits 18-24
 - DR7 debug control register 15-1, 15-5
 - G0-G3 (global breakpoint enable) flags 15-5
 - GD (general detect enable) flag 15-5
 - GE (global exact breakpoint enable) flag 15-5
 - L0-L3 (local breakpoint enable) flags 15-5
 - LE local exact breakpoint enable) flag 15-5
 - LEN0-LEN3 (Length) fields 15-6

R/W0-R/W3 (read/write) fields 15-5, 18-24
DS feature flag, CPUID instruction. 15-11
DS save area 15-36
DS (debug store) mechanism
availability of 15-34
description of 15-34
DS feature flag, CPUID instruction 15-34
DS save area 15-34
interrupt service routine (DS ISR). 15-19
setting up 15-17
D/B (default operation size/default stack pointer
size and/or upper bound) flag, segment
descriptor 3-11, 4-5

E

E (edge detect) flag, PerfEvtSel0 and PerfEvtSel1
MSRs (P6 family processors) 15-62
E (expansion direction) flag, segment descriptor .
4-2, 4-5
E (MTRRs enabled) flag, IA32_MTRR_DEF_TYPE
MSR. 10-27
EFLAGS register
introduction to 2-5
new flags. 18-6
saved in TSS 6-6
saving on call to exception or interrupt handler
5-15
system flags 2-7
using flags to distinguish between 32-bit IA-32
processors. 18-6
EIP register 18-13
saved in TSS 6-6
saving on call to exception or interrupt handler
5-15
state following initialization 9-6
EM (emulation) flag, CR0 control register . . 2-14,
2-15, 5-29, 9-6, 9-7, 11-1, 12-3
EMMS instruction. 11-3
Error code
exception, description of. 5-18
pushing on stack. 18-35
Error signals. 18-13
ERROR# input 18-19
ERROR# output 18-19
ES0 and ES1 (event select) fields, CESR MSR
(Pentium processor). 15-66
ESP register, saving on call to exception or interrupt
handler. 5-15
ET (extension type) flag, CR0 control register 2-14,
18-8
Event select field, PerfEvtSel0 and PerfEvtSel1
MSRs (P6 family processors) 15-62
Events
at-retirement 15-47
at-retirement (Pentium 4 processor) 15-27, A-28
non-retirement (Pentium 4 processor) . . 15-27,
A-2

P6 family processors. A-45
Pentium processor A-60
Exception handler
calling 5-14
defined 5-1
flag usage by handler procedure. 5-17
machine-check exceptions (#MC). 14-16
procedures 5-14
protection of handler procedures 5-16
task 5-18, 6-3
Exceptions
alignment check 18-14
classifications 5-6
conditions checked during a task switch. . 6-15
coprocessor segment overrun. 18-14
description of. 2-4, 5-1
device not available. 18-14
double fault 5-31, 5-32
error code 5-18
floating-point error. 18-14
general protection 18-14
handler mechanism. 5-14
handler procedures. 5-14
handling. 5-14
handling in real-address mode 16-6
handling in SMM 13-10
handling in virtual-8086 mode. 16-15
handling through a task gate in virtual-8086
mode 16-20
handling through a trap or interrupt gate in
virtual-8086 mode 16-17
IDT 5-11
initializing for protected-mode operation. . 9-12
invalid-opcode. 18-6
masking debug exceptions 5-9
masking when switching stack segments . 5-10
MMX instructions. 11-1
notation 1-6
overview of 5-1
priorities among simultaneous exceptions and
interrupts 5-10
priority of 18-27
priority of, x87 FPU exceptions. 18-12
reference information on all exceptions . . 5-21
restarting a task or program 5-6
segment not present 18-14
sources of 5-5
summary of 5-3
vectors. 5-2
Execution events A-35
Expand-down data segment type. 3-12
External bus errors, detected with machine-check
architecture. 14-15

F

F2XM1 instruction. 18-16
Fast string operations. 7-10

- Faults
 - description of 5-6
 - restarting a program or task after 5-7
 - FCMOVcc instructions 18-4
 - FCOMI instruction 18-4
 - FCOMIP instruction 18-4
 - FCOS instruction 18-16
 - FDISI instruction (obsolete) 18-18
 - FDIV instruction 18-13, 18-15
 - FE (fixed MTRRs enabled) flag,
 - IA32_MTRR_DEF_TYPE MSR . . . 10-27
 - Feature
 - determination, of processor 18-2
 - information, processor 18-2
 - FENI instruction (obsolete) 18-18
 - FINIT/FNINIT instructions 18-9, 18-19
 - FIX (fixed range registers supported) flag,
 - IA32_MTRRCAPMSR 10-26
 - Fixed-range MTRRs
 - description of 10-28
 - mapping to physical memory 10-29
 - Flat segmentation model 3-3
 - FLD instruction 18-16
 - FLDENV instruction 18-14
 - FLDL2E instruction 18-17
 - FLDL2T instruction 18-17
 - FLDLG2 instruction 18-17
 - FLDLN2 instruction 18-17
 - FLDPI instruction 18-17
 - Floating-point error exception (#MF) 18-14
 - Floating-point exceptions
 - denormal operand exception (#D) 18-11
 - invalid operation (#I) 18-17
 - numeric overflow (#O) 18-11
 - numeric underflow (#U) 18-12
 - saved CS and EIP values 18-13
 - FLUSH# pin 5-4
 - FNSAVE instruction 11-4
 - Focus processor, local APIC 8-30
 - FPATAN instruction 18-16
 - FPREM instruction 18-9, 18-13, 18-15
 - FPREM1 instruction 18-9, 18-15
 - FPTAN instruction 18-9, 18-16
 - Front_end events A-34
 - FRSTOR instruction 11-4, 18-14
 - FSAVE instruction 11-3, 11-4
 - FSAVE/FNSAVE instructions 18-14, 18-18
 - FSCALE instruction 18-15
 - FSIN instruction 18-16
 - FSINCOS instruction 18-16
 - FSQRT instruction 18-13, 18-15
 - FSTENV instruction 11-3
 - FSTENV/FNSTENV instructions 18-18
 - FTAN instruction 18-9
 - FUCOM instruction 18-15
 - FUCOMI instruction 18-4
 - FUCOMIP instruction 18-4
 - FUCOMP instruction 18-15
 - FUCOMPP instruction 18-15
 - FWAIT instruction 5-29
 - FXAM instruction 18-16, 18-17
 - FXRSTOR instruction 2-17, 9-9, 11-3, 11-4, 11-5, 12-1, 12-2, 12-6
 - FXSAVE instruction 2-17, 9-9, 11-3, 11-4, 11-5, 12-1, 12-2, 12-6
 - FXSR feature flag, CPUID instruction 9-9
 - FXTRACT instruction 18-11, 18-16, 18-17
- ## G
- G (global) flag
 - page-directory entries 10-14, 10-23
 - page-table entries 3-27, 10-14, 10-23
 - G (granularity) flag, segment descriptor 3-10, 3-12, 4-2, 4-5
 - G0-G3 (global breakpoint enable) flags, DR7
 - register 15-5
 - Gate descriptors
 - call gates 4-17
 - description of 4-16
 - Gates 2-3
 - GD (general detect enable) flag, DR7 register 15-5, 15-9
 - GDT
 - description of 2-3, 3-16
 - index into with index field of segment selector. 3-7
 - initializing 9-12
 - paging of 2-5
 - pointers to exception and interrupt handlers . . 5-14
 - segment descriptors in 3-9
 - selecting with TI (table indicator) flag of segment selector 3-7
 - task switching 6-12
 - task-gate descriptor 6-9
 - TSS descriptors 6-7
 - use in address translation 3-7
 - GDTR register
 - description of 2-3, 2-5, 2-10, 3-16
 - limit 4-5
 - loading during initialization 9-12
 - storing 3-17
 - GE (global exact breakpoint enable) flag, DR7
 - register 15-5, 15-9
 - General-detect exception condition 15-9
 - General-protection exception (#GP) 3-13, 4-7, 4-8, 4-14, 4-15, 5-12, 5-17, 5-40, 5-41, 5-42, 6-7, 15-2, 18-14, 18-26, 18-27, 18-37, 18-39
 - General-purpose registers, saved in TSS 6-5
 - Global descriptor table register (see GDTR)
 - Global descriptor table (see GDT)

H

HALT state, relationship to SMI interrupt . . .	13-4, 13-14
Hardware reset	
description of	9-1
processor state after reset	9-2
state of MTRRs following	10-24
value of SMBASE following	13-4
Hexadecimal numbers	1-6
HITM# line	10-5
HLT instruction	2-22, 4-26, 5-32, 13-14, 13-15, 15-23
Hyper-Threading Technology	
architectural state of a logical processor . .	7-24
architecture description.	7-24
caches	7-28
counting clockticks	15-53
debug registers	7-26
description of	7-23, 18-4
detecting	7-30
executing multiple threads	7-31
execution-based timing loops	7-40
external signal compatibility	7-29
halting logical processors	7-40
handling interrupts	7-31
HLT instruction	7-33
IA32_MISC_ENABLE MSR	7-27
initializing IA-32 processors with.	7-30
introduction of into the IA-32 architecture .	18-4
local APIC, functionality in logical processor .	7-25
logical processors, identifying	7-33
machine check architecture	7-26
managing idle and blocked conditions . . .	7-33
memory ordering.	7-27
microcode update resources	7-27
MP systems	7-32
MTRRs	7-26
PAT.	7-26
PAUSE instruction	7-33
performance monitoring counters.	7-27
proper placement of locks and semaphores . .	7-41
required operating system support.	7-39
scheduling multiple threads	7-40
self modifying code	7-28
serializing instructions.	7-27
spin-wait loops, using PAUSE instructions in .	7-39
thermal monitor.	7-29
TLBs	7-29
Hyper-Threading technology	
performance monitoring	15-55

I

IA-32 Intel architecture	
compatibility	18-1

processors.	18-1
IA32_APIC_BASE MSR	7-16, 7-17, 8-9, 8-10, 8-11, B-2
IA32_BIOS_SIGN_ID MSR	B-5
IA32_BIOS_UPDT_TRIG MSR	9-36, B-5
IA32_CTL MSR	B-6
IA32_DEBUGCTL MSR	15-11, 15-12, 15-15, 15-16, 15-17, 15-18, 15-19, 15-34, B-11
IA32_DS_AREA MSR	15-17, 15-25, 15-35, 15-51, B-19
IA32_MCG_CAP MSR	14-2, 14-3, 14-16, B-6
IA32_MCG_CTL MSR	14-2, 14-5
IA32_MCG_EAX MSR	14-9, B-6
IA32_MCG_EBP MSR	14-9, B-7
IA32_MCG_EBX MSR	14-9, B-6
IA32_MCG_ECX MSR	14-9, B-6
IA32_MCG_EDI MSR.	14-9, B-7
IA32_MCG_EDX MSR	14-9, B-7
IA32_MCG_EFLAGS MSR.	14-9, B-7
IA32_MCG_EIP MSR.	14-9, B-8
IA32_MCG_ESI MSR.	14-9, B-7
IA32_MCG_ESP MSR	14-9, B-7
IA32_MCG_MISC MSR	14-5, B-8
IA32_MCG_STATUS MSR.	14-2, 14-4, 14-17, 14-19
IA32_MCi_ADDR MSR.	14-8, B-17
IA32_MCi_ADDR MSRs.	14-19
IA32_MCi_CTL MSR	14-5, B-17
IA32_MCi_MISC MSR	14-8, B-17
IA32_MCi_MISC MSRs	14-19
IA32_MCi_STATUS MSR.	14-6, B-17
IA32_MCi_STATUS MSRs.	14-16, 14-19
IA32_MISC_ENABLE MSR	13-18, 13-20, 15-12, 15-25, 15-35, B-8
IA32_MTRRCAP MSR	10-26, 10-27, B-5
IA32_MTRR_DEF_TYPE MSR	10-27
IA32_MTRR_FIXn, fixed ranger MTRRs . . .	10-28
IA32_MTRR_PHYSBASEn (variable range)	
MTRRs	10-28
IA32_MTRR_PHYSMASKn (variable range)	
MTRRs	10-28
IA32_P5_MC_ADDR MSR.	B-1
IA32_P5_MC_TYPE MSR	B-1
IA32_PAT_CR MSR.	10-40
IA32_PEBS_ENABLE MSR	A-36
IA32_PEBS_ENABLE MSR	15-25, 15-51, B-16
IA32_PLATFORM_ID.	B-1, B-20, B-27
IA32_STATUS MSR.	B-6
IA32_SYSENTER_CS MSR.	B-5
IA32_SYSENTER_EIP MSR	B-6
IA32_SYSENTER_ESP MSR.	B-6
IA32_THERM_CONTROL MSR.	13-23, 13-24, B-8
IA32_THERM_INTERRUPT MSR	13-22, 13-24, B-8
IA32_THERM_STATUS MSR	13-24, B-8
IA32_TIME_STAMP_COUNTER MSR	B-1
ID (identification) flag, EFLAGS register. .	2-10, 18-6
IDIV instruction	5-22, 18-26

IDT

- calling interrupt- and exception-handlers from . . . 5-14
- changing base and limit in real-address mode . . . 16-6
- description of 5-11
- handling NMI interrupts during initialization . . . 9-10
- initializing, for protected-mode operation . . . 9-12
- initializing, for real-address mode operation . . . 9-10
- introduction to 2-4
- limit 18-28
- paging of 2-5
- structure in real-address mode 16-7
- task switching 6-12
- task-gate descriptor 6-9
- types of descriptors allowed 5-12
- use in real-address mode 16-6

IDTR register

- description of 2-11, 5-12
- introduction to 2-4
- limit 4-5
- loading in real-address mode 16-6
- storing 3-17

IE (invalid operation exception) flag, x87 FPU

- status word 18-9

IEEE Standard 754 for Binary Floating-Point Arithmetic . . . 18-9, 18-10, 18-11, 18-12, 18-15, 18-16, 18-17

IF (interrupt enable) flag, EFLAGS register . . . 2-8, 2-9, 5-8, 5-13, 5-17, 13-10, 16-6, 16-26

IN instruction 7-11, 18-38

INC instruction 7-5

Index field, segment selector 3-7

INIT interrupt 8-5

Initial-count register, local APIC 8-20

Initialization

- built-in self-test (BIST) 9-1, 9-2
- CS register state following 9-6
- EIP register state following 9-6
- example 9-16
- first instruction executed 9-6
- hardware reset 9-1
- IDT, protected mode 9-12
- IDT, real-address mode 9-10
- Intel486 SX processor and Intel 487 SX math coprocessor 18-20
- location of software-initialization code 9-6
- model and stepping information 9-5
- multiple-processor (MP) bootup sequence for P6 family processors C-1
- multitasking environment 9-13
- overview 9-1
- paging 9-13
- processor state after reset 9-2
- protected mode 9-11
- real-address mode 9-10
- RESET# pin 9-1

- setting up exception- and interrupt-handling facilities 9-12
- x87 FPU 9-6

INIT# pin 5-4, 9-2

INIT# signal 2-22

Input/output (see I/O)

INS instruction 15-9

Instruction operands 1-5

Instruction set

- new instructions 18-4
- obsolete instructions 18-6

Instruction-breakpoint exception condition . . . 15-8

Instructions

- privileged 4-26
- serializing 18-19
- supported in real-address mode 16-4
- system 2-6, 2-18

INT 3 instruction 2-4, 5-24, 15-2

INT instruction 2-4, 4-12

INT n instruction 3-9, 5-1, 5-4, 5-5

INT (APIC interrupt enable) flag, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors) 15-63

INT3 instruction 3-9, 5-5

Intel 287 math coprocessor 18-8

Intel 387 math coprocessor system 18-8

Intel 487 SX math coprocessor 18-8, 18-20

Intel 8086 processor 18-8

Intel NetBurst micro-architecture 1-1

Intel Xeon processor 1-1

Intel Xeon processor MP 7-23

Intel286 processor 18-8

Intel386 DX processor 18-8

Intel386 SL processor 2-6

Intel486 DX processor 18-8

Intel486 SX processor 18-8, 18-20

Interprivilege level calls

- call mechanism 4-18
- stack switching 4-21

Interprocessor interrupt (IPI)

- in MP systems 8-1

Inter-processor interrupts (IPIs) 8-2

Interrupt

- description of 5-1

Interrupt command register (ICR), local APIC . . . 8-21

interrupt destination 8-31

Interrupt gates

- 16-bit, interlevel return from 18-35
- clearing IF flag 5-9, 5-17
- difference between interrupt and trap gates . . . 5-17
- for 16-bit and 32-bit code modules 17-2
- handling a virtual-8086 mode interrupt or exception through 16-17
- in IDT 5-12
- introduction to 2-3, 2-4
- layout of 5-12

Interrupt handler

- calling5-14
- defined5-1
- flag usage by handler procedure5-17
- procedures5-14
- protection of handler procedures5-16
- task5-18, 6-3
- Interrupts
 - APIC priority levels8-34
 - automatic bus locking when acknowledging . . 18-39
 - control transfers between 16- and 32-bit code modules17-8
 - description of2-4, 5-1
 - distribution mechanism, local APIC8-29
 - enabling and disabling5-8
 - handling5-14
 - handling in real-address mode16-6
 - handling in SMM13-10
 - handling in virtual-8086 mode16-15
 - handling multiple NMIs5-8
 - handling through a task gate in virtual-8086 mode16-20
 - handling through a trap or interrupt gate in virtual-8086 mode16-17
 - IDT5-11
 - IDTR2-11
 - initializing for protected-mode operation . . 9-12
 - interrupt descriptor table register (see IDTR)
 - interrupt descriptor table (see IDT)
 - list of5-3, 16-8
 - local APIC8-1
 - maskable hardware interrupts2-8
 - masking maskable hardware interrupts . . .5-8
 - masking when switching stack segments . .5-10
 - message signalled interrupts8-40
 - overview of5-1
 - priorities among simultaneous exceptions and interrupts5-10
 - priority8-34
 - propagation delay18-27
 - real-address mode16-8
 - restarting a task or program5-6
 - software5-56
 - sources of8-2
 - summary of5-3
 - user defined5-2, 5-56
 - valid APIC interrupts8-18
 - vectors5-2
 - virtual-8086 mode16-8
- INTn instruction15-10
- INTO instruction2-4, 3-9, 5-5, 5-25, 15-10
- INTR# pin5-2, 5-8
- Invalid opcode exception (#UD) . .2-15, 5-27, 5-28, 5-53, 11-1, 13-3, 15-4, 18-6, 18-13, 18-25, 18-26
- Invalid TSS exception (#TS)5-34, 5-35, 6-8
- Invalid-operation exception, x87 FPU 18-13, 18-17
- INVD instruction2-21, 4-26, 7-14, 10-19, 18-4
- INVLPG instruction2-21, 4-26, 7-14, 18-4
- IOPL (I/O privilege level) field, EFLAGS register
 - description of2-8
 - restoring on return from exception or interrupt handler5-16
 - sensitive instructions in virtual-8086 mode . . .16-14
 - virtual interrupt2-9, 2-10
- IPI (see Interprocessor interrupt)
- IRET instruction . . 3-9, 5-8, 5-9, 5-16, 5-17, 6-12, 6-13, 7-14, 16-6, 16-27
- IRETD instruction2-9, 7-14
- IRR (interrupt request register), local APIC . . 8-36
- I/O
 - breakpoint exception conditions15-9
 - in virtual-8086 mode16-14
 - instruction restart flag, SMM revision identifier field13-16
 - instructions, restarting following an SMI interrupt 13-16
 - I/O permission bit map, TSS6-6
 - map base address field, TSS6-6
- I/O APIC8-31
 - additional information about8-1
 - bus arbitration8-31
 - description of8-1
 - external interrupts5-4
 - interrupt sources8-2
 - overview of8-1
 - relationship of local APIC to I/O APIC .8-3, 8-4
 - valid interrupts8-18
- I/O privilege level (see IOPL)
- J**
- JMP instruction . . . 2-4, 3-9, 4-12, 4-13, 4-18, 6-3, 6-12, 6-13
- K**
- KEN# pin10-15, 18-40
- L**
- L0-L3 (local breakpoint enable) flags, DR7 register 15-5
- L1 (level 1) cache
 - caching methods10-5
 - description of10-3
 - effect of using write-through memory . . .10-9
 - introduction of18-31
 - invalidating and flushing10-19
 - MESI cache protocol10-9
 - shared and adaptive mode10-20
- L2 (level 2) cache
 - caching methods10-5
 - description of10-3
 - disabling10-20
 - effect of using write-through memory . . .10-9

- introduction of 18-31
- invalidating and flushing 10-19
- MESI cache protocol. 10-10
- L3 (level 3) cache
 - caching methods. 10-5
 - description of 10-3
 - disabling and enabling 10-15, 10-19
 - effect of using write-through memory 10-9
 - introduction of 18-33
 - invalidating and flushing 10-19
 - MESI cache protocol. 10-10
- LAR instruction. 2-21, 4-27
- Larger page sizes
 - introduction of 18-33
 - support for. 18-23
- Last branch, interrupt, and exception recording
 - description of 15-11, 15-13, 15-14, 15-19
- LastBranchFromIP MSR 15-22
- LastBranchToIP MSR 15-22
- LastExceptionFromIP MSR 15-16, 15-22
- LastExceptionToIP MSR 15-16, 15-22
- LBR (last branch/interrupt/exception) flag,
 - DebugCtlMSR MSR 15-12, 15-15, 15-21, 15-22
- LDS instruction. 3-9, 4-10
- LDT
 - associated with a task. 6-3
 - description of 2-3, 2-4, 3-17
 - index into with index field of segment selector. 3-7
 - pointer to in TSS. 6-6
 - pointers to exception and interrupt handlers 5-14
 - segment descriptors in 3-9
 - segment selector field, TSS 6-18
 - selecting with TI (table indicator) flag of segment selector 3-7
 - setting up during initialization 9-12
 - task switching 6-12
 - task-gate descriptor 6-9
 - use in address translation. 3-7
- LDTR register
 - description of 2-3, 2-4, 2-5, 2-11, 3-17
 - limit 4-5
 - storing. 3-17
- LE (local exact breakpoint enable) flag, DR7
 - register. 15-5, 15-9
- LEN0-LEN3 (Length) fields, DR7 register 15-6
- LES instruction. 3-9, 4-10, 5-27
- LFENCE instruction 2-14, 7-8, 7-11, 7-12, 7-14
- LFS instruction 3-9, 4-10
- LGDT instruction 2-20, 4-26, 7-14, 9-12, 18-25
- LGS instruction. 3-9, 4-10
- LIDT instruction 2-20, 4-26, 5-12, 7-14, 9-10, 16-6, 18-28
- Limit checking
 - description of 4-5
 - pointer offsets are within limits 4-29
- Limit field, segment descriptor 4-2, 4-5
- Linear address
 - description of. 3-6
 - introduction to 2-5
- Linear address space 3-6
 - defined 3-1
 - of task 6-19
- Link (to previous task) field, TSS 5-18
- Linking tasks
 - mechanism 6-16
 - modifying task linkages. 6-18
- LINT pins
 - function of 5-2
 - programming. D-1
- LLDT instruction 2-20, 4-26, 7-14
- LMSW instruction 2-20, 4-26
- Local APIC
 - APIC_BASE_MSR 8-11
 - arbitration over the APIC bus 8-31
 - arbitration over the system bus. 8-31
 - block diagram 8-6
 - cluster model. 8-29
 - current-count register 8-21
 - description of. 8-1
 - detecting with CPUID instruction. 8-9
 - DFR (destination format register) 8-28
 - differences between local APIC and 82489DX 18-28
 - disabling 8-10
 - divide configuration register 8-20
 - enabling. 8-10
 - external interrupts 5-2
 - focus processor. 8-30
 - IA32_APIC_BASE MSR 8-11
 - indicating performance-monitoring counter overflow 15-65
 - initial-count register 8-20
 - internal error interrupts 8-2
 - interrupt command register (ICR) 8-21
 - interrupt destination. 8-31
 - interrupt distribution mechanism 8-29
 - interrupt sources 8-2
 - IRR (interrupt request register) 8-36
 - local vector table (LVT) 8-15
 - logical destination mode 8-28
 - LVT (local-APIC version register) 8-14
 - MDA (message destination address) 8-28
 - new features incorporated in the Pentium 4 and Intel Xeon processors 18-29
 - new features incorporated in the Pentium and P6 family processors 18-29
 - overview of 8-1
 - physical destination mode. 8-27
 - receiving external interrupts 5-2
 - register address map 8-8
 - relationship of local APIC to I/O APIC. 8-3, 8-4
 - SMI interrupt 13-2
 - spurious interrupt 8-38

state after a software (INIT) reset. 8-13
 state after INIT-deassert message 8-14
 state after power-up reset. 8-12
 state of 8-39
 SVR (spurious-interrupt vector register) . . . 8-10
 timer 8-20
 timer generated interrupts. 8-2
 TMR (trigger mode register) 8-36
 valid interrupts 8-18
 version register 8-14
 Local descriptor table register (see LDTR)
 Local descriptor table (see LDT)
 Local vector table (LVT)
 description of 8-15
 thermal entry. 13-22
 LOCK prefix 2-22, 5-27, 7-2, 7-3, 7-4, 7-11, 18-39
 Locked (atomic) operations
 automatic bus locking 7-4
 bus locking 7-3
 effects of a locked operation on internal
 processor caches 7-7
 loading a segment descriptor 18-24
 on IA-32 processors 18-38
 overview of 7-2
 software-controlled bus locking 7-4
 LOCK# signal. 2-22, 7-2, 7-3, 7-5, 7-7
 Logical address space, of task. 6-20
 Logical address, description of. 3-6
 Logical destination mode, local APIC. 8-28
 LSL instruction 2-21, 4-29
 LSS instruction. 3-9, 4-10
 LTR instruction. 2-20, 4-26, 6-9, 7-14, 9-13
 LVT (see Local vector table)

M

Machine-check architecture
 availability of machine-check architecture and
 exception. 14-10
 compatibility with Pentium processor
 implementation 14-1
 error codes, compound. 14-13
 error codes, interpreting 14-12
 error codes, simple 14-12
 error-reporting MSRs 14-5
 first introduced 18-27
 global MSRs 14-2
 guidelines for writing machine-check software
 14-15
 IA32_MCG extended machine check state
 MSRs. 14-9
 initialization of. 14-10
 interpreting error codes, example (P6 family
 processors) E-1, F-1
 introduction of in IA-32 processors. 18-40
 logging correctable machine-check errors 14-18
 machine-check error codes, external bus errors
 14-15

 machine-check exception handler. 14-16
 MSRs 14-2
 overview 14-1
 Pentium processor machine-check exception
 handling 14-18
 Pentium processor style error reporting . 14-10
 Machine-check exception (#MC) 5-51, 5-52, 14-1,
 14-10, 14-16, 18-26, 18-40
 Maskable hardware interrupts
 description of. 5-4
 handling with virtual interrupt mechanism 16-20
 masking. 2-8, 5-8
 MCA (machine-check architecture) flag, CPUID
 instruction. 14-10
 MCE (machine-check enable) flag, CR4 control
 register 2-17, 18-22
 MCE (machine-check exception) flag, CPUID
 instruction. 14-10
 MCG_CAP MSR. 14-3, 14-4
 MCG_CTL MSR 14-5
 MCG_STATUS MSR 14-4
 MCI_ADDR MSR 14-8
 MCI_CTL MSR 14-5
 MCI_CTL MSRs 14-5
 MCI_MISC MSR 14-8
 MCI_STATUS MSR 14-6
 MDA (message destination address), local APIC .
 8-28
 Memory. 10-1
 Memory management
 introduction to 2-5
 overview 3-1
 paging 3-1, 3-2, 3-17
 registers 2-10
 segments. 3-1, 3-2, 3-3, 3-7
 virtual memory. 3-17
 Memory ordering
 in IA-32 processors 18-37
 out of order stores for string operations . 7-10
 overview 7-7
 processor ordering 7-7
 snooping mechanism 7-9
 strengthening or weakening 7-11
 write forwarding. 7-9
 write ordering 7-7
 Memory type range registers (see MTRRs)
 Memory types
 caching methods, defined 10-5
 choosing 10-9
 MTRR types 10-25
 selecting for Pentium III and Pentium 4
 processors 10-17
 selecting for Pentium Pro and Pentium II
 processors 10-16
 UC (strong uncacheable) 10-5
 UC- (uncacheable) 10-6
 WB (write back). 10-6
 WC (write combining) 10-6

- WP (write protected) 10-7
- writing values across pages with different
 - memory types 10-18
- WT (write through) 10-6
- MemTypeGet() function 10-34
- MemTypeSet() function 10-36
- MESI cache protocol 10-4, 10-9
- Message address register 8-40
- Message data register format 8-42
- Message signalled interrupts
 - message address register 8-40
 - message data register format 8-40
- MFENCE instruction . . . 2-14, 7-8, 7-11, 7-12, 7-14
- Microcode update facilities
 - function 00H presence test 9-44
 - function 01H write microcode update data . . . 9-44
 - function 02H microcode update control . . . 9-48
 - function 03H read microcode update data . . 9-49
 - INT 15H-based interface 9-43
 - microcode update 9-32
 - microcode update loader 9-35
 - microcode update specifications 9-39
 - overview 9-32
 - return codes 9-50
 - update signature and verification 9-37
- Mixing 16-bit and 32-bit code
 - in IA-32 processors 18-35
 - overview 17-1
- MMX technology
 - debugging MMX code 11-6
 - effect of MMX instructions on pending x87
 - floating-point exceptions 11-6
 - emulation of the MMX instruction set . . . 11-1
 - exceptions that can occur when executing MMX
 - instructions 11-1
 - introduction of into the IA-32 architecture . . 18-3
 - MMX register aliasing 11-1
 - MMX state 11-1
 - MMX state, saving and restoring 11-4
 - MMX state, saving on task or context switches
 - 11-5
 - system programming 11-1
 - using TS flag to control saving of MMX state .
 - 12-8
- Mode switching
 - between real-address and protected mode 9-14
 - example 9-16
 - to SMM 13-2
- Model and stepping information, following
 - processor initialization or reset 9-5
- Model-specific registers (see MSRs)
- Modes of operation (see Operating modes)
- MOV instruction 3-9, 4-10
- MOV (control registers) instructions . . . 2-20, 4-26,
 - 7-14, 9-14
- MOV (debug registers) instructions . . . 2-21, 4-26,
 - 7-14, 15-10
- MOVNTDQ instruction 7-8, 10-4, 10-20
- MOVNTI instruction 2-14, 7-8, 10-4, 10-20
- MOVNTPD instruction 7-8, 10-4, 10-20
- MOVNTPS instruction 7-8, 10-4, 10-20
- MOVNTQ instruction 7-8, 10-4, 10-20
- MP (monitor coprocessor) flag, CR0 control register
 - 2-14, 2-16, 5-29, 9-6, 9-7, 11-1, 12-3
- MP (monitor coprocessor) flag, CR0 register . 18-8
- MSRs
 - architectural B-37
 - description of 9-8
 - introduction of in IA-32 processors 18-39
 - introduction to 2-5
 - list of B-1
 - machine-check architecture 14-2
 - P6 family processors B-27
 - Pentium 4 processor B-1, B-20
 - Pentium processors B-36
 - reading and writing 2-23
- MSR_TC_PRECISE_EVENT MSR A-34
- MSR_EBC_FREQUENCY_ID MSR B-4, B-5
- MSR_EBC_HARD_POWERON MSR B-2
- MSR_EBC_SOFT_POWERON MSR B-3
- MSR_LASTBRANCH_0 MSR B-12
- MSR_LASTBRANCH_1 MSR B-12
- MSR_LASTBRANCH_2 MSR B-12
- MSR_LASTBRANCH_3 MSR B-12
- MSR_LASTBRANCH_TOS B-11
- MSR_LASTBRANCH_0 MSR 15-12, 15-13, 15-14
- MSR_LASTBRANCH_1 MSR 15-12, 15-13, 15-14
- MSR_LASTBRANCH_2 MSR 15-12, 15-13, 15-14
- MSR_LASTBRANCH_3 MSR 15-12, 15-13, 15-14
- MSR_LASTBRANCH_TOS MSR . . . 15-12, 15-13
- MSR_LER_FROM_LIP MSR 15-16, B-11
- MSR_LER_TO_LIP MSR 15-16, B-11
- MSR_PEBB_MATRIX_VERT MSR A-36
- MSR_PEBB_MATRIX_VERT MSR B-17
- MTRR feature flag, CPUID instruction 10-26
- MTRRcap MSR 10-26
- MTRRfix MSR 10-28
- MTRRs 7-11
 - address mapping for fixed-range MTRRs 10-29
 - cache control 10-15
 - description of 9-9, 10-24
 - enabling caching 9-8
 - example of base and mask calculations . . 10-31
 - feature identification 10-26
 - fixed-range registers 10-28
 - IA32_MTRRCAP MSR 10-26
 - IA32_MTRR_DEF_TYPE MSR 10-27
 - initialization of 10-33
 - introduction of in IA-32 processors 18-40
 - introduction to 2-5
 - large page size considerations 10-39
 - mapping physical memory with 10-25
 - memory types and their properties 10-25
 - MemTypeGet() function 10-34
 - MemTypeSet() function 10-36
 - multiple-processor considerations 10-37

precedence of cache controls	10-15
precedences	10-32
programming interface	10-34
remapping memory types	10-34
state of following a hardware reset	10-24
variable-range registers	10-28
Multiple-processor initialization	
MP protocol	7-15
procedure	C-2
Multiple-processor management	
bus locking	7-3
guaranteed atomic operations	7-3
local APIC	8-1
memory ordering	7-7
MP protocol	7-15
overview of	7-1
propagation of page table and page directory entry changes	7-13
SMM considerations	13-17
Multiple-processor system	
relationship of local and I/O APICs, P6 family processors	8-4
relationship of local and I/O APICs, Pentium 4 processor	8-4
Multisegment model	3-5
Multitasking	
initialization for	9-13
linking tasks	6-16
mechanism, description of	6-3
overview	6-1
setting up TSS	9-13
setting up TSS descriptor	9-13
MXCSR register	5-53, 9-10, 12-6

N

NaN, compatibility, IA-32 processors	18-10
NE (numeric error) flag, CR0 control register . 2-14, 5-47, 9-6, 9-7, 18-22	
NE (numeric error) flag, CR0 register	18-8
NEG instruction	7-5
NetBurst micro-architecture (see Intel NetBurst micro-architecture)	
NMI interrupt	2-22, 8-5
description of	5-2
handling during initialization	9-10
handling in SMM	13-11
handling multiple NMIs	5-8
masking	18-28
receiving when processor is shutdown	5-32
reference information	5-23
vector	5-2
NMI# pin	5-2, 5-23
Nominal CPI method	15-53
Nonconforming code segments	
accessing	4-14
C (conforming) flag	4-14
description of	3-14

Non-Halted Clockticks	15-52
setting up counters	15-53
Non-Halted CPI method	15-53
Nonmaskable interrupt (see NMI)	
Non-precise event-based sampling	
defined	15-28
writing an interrupt service routine for	15-19
non-precise event-based sampling	
used for at-retirement counting	15-48
Non-retirement events	15-27, A-2
Non-Sleep Clockticks	15-52
setting up counters	15-53
NOT instruction	7-5
Notation	
bit and byte order	1-4
conventions	1-4
exceptions	1-6
hexadecimal and binary numbers	1-6
instruction operands	1-5
reserved bits	1-4
segmented addressing	1-6
NT (nested task) flag, EFLAGS register . 2-9, 6-12, 6-13, 6-16	
Null segment selector, checking for	4-7
Numeric overflow exception (#O)	18-11
Numeric underflow exception (#U)	18-12
NV (invert) flag, PerfEvtSel0 MSR (P6 family processors)	15-63
NW (not write-through) flag, CR0 control register . 2-13, 9-8, 10-13, 10-14, 10-18, 10-37, 10-38, 18-22, 18-23, 18-32	

O

Obsolete instructions	18-6, 18-18
OF flag, EFLAGS register	5-25
Opcodes	
undefined	18-6
Operands	
instruction	1-5
operand-size prefix	17-2
Operating modes	
introduction to	2-6
protected mode	2-6
SMM (system management mode)	2-6
virtual-8086 mode	2-7
OR instruction	7-5
OS (operating system mode) flag, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors)	15-62
OSFXSR (FXSAVE/FXRSTOR support) flag, CR4 control register	2-17, 9-9, 12-2
OSXMMEXCPT (SIMD floating-point exception support) flag, CR4 control register . 2-18, 5-53, 9-9, 12-2	
OUT instruction	7-11
OUTS instruction	15-9
Overflow exception (#OF)	5-25

P

- P (present) flag
 - page-directory entry 5-43
 - page-table entries 3-25
 - page-table entry 5-43
- P (segment-present) flag, segment descriptor . 3-11
- P5_MC_ADDR MSR . . 14-10, 14-18, B-20, B-27, B-36
- P5_MC_TYPE MSR . . 14-10, 14-18, B-20, B-27, B-36
- P6 family processors
 - compatibility with floating-point software . . 18-8
 - description of 1-1
 - list of performance-monitoring events . . . A-45
 - MSR supported by B-27
- PAE feature flag, CPUID instruction 3-28
- PAE (physical address extension) flag, CR4 control register . . . 2-17, 3-6, 3-18, 3-28, 3-34, 18-21, 18-23
- Page attribute table (PAT)
 - compatibility with earlier IA-32 processors . 10-43
 - detecting support for 10-40
 - IA32_CR_PAT MSR 10-40
 - introduction to 10-39
 - memory types that can be encoded with . 10-41
 - MSR 10-15
 - precedence of cache controls 10-15
 - programming 10-42
 - selecting a memory type with 10-41
- Page base address field, page-table entries . 3-23, 3-35
- Page directory
 - base address 3-23
 - base address (PDBR) 6-6
 - description of 3-19
 - introduction to 2-5
 - overview 3-2
 - setting up during initialization 9-13
- Page frame (see Page)
- Page tables
 - description of 3-19
 - introduction to 2-5
 - overview 3-2
 - setting up during initialization 9-13
- Page-directory entries . 3-19, 3-23, 3-24, 3-25, 3-33, 3-35, 7-4, 10-4
- Page-directory-pointer (PDPT) table 3-28
- Page-directory-pointer-table entries 3-33
- Page-fault exception (#PF) . 3-18, 5-43, 5-44, 5-45, 5-46, 18-26
- Pages
 - description of 3-19
 - disabling protection of 4-2
 - enabling protection of 4-2
 - introduction to 2-5
 - overview 3-2
 - PG flag, CR0 control register 4-2
 - sizes 3-20
 - split 18-18
- Page-table base address field, page-directory entries 3-23, 3-35
- Page-table entries . . . 3-19, 3-23, 3-24, 3-33, 7-4, 10-4, 10-21
- Paging
 - 32-bit physical addressing 3-20
 - 36-bit physical addressing, using PAE paging mechanism 3-28
 - 36-bit physical addressing, using PSE-36 paging mechanism 3-34
 - combining segment and page-level protection . 4-34
 - combining with segmentation 3-6
 - defined 3-1
 - initializing 9-13
 - introduction to 2-5
 - large page size MTRR considerations . . 10-39
 - mapping segments to pages 3-36
 - mixing 4-KByte and 4-MByte pages 3-22
 - options 3-18
 - overview 3-17
 - page 3-19
 - page boundaries regarding TSS 6-6
 - page directory 3-19
 - page sizes 3-20
 - page table 3-19
 - page-directory-pointer table 3-19
 - page-fault exception . . . 5-43, 5-44, 5-45, 5-46
 - page-level protection 4-2, 4-31
 - page-level protection flags 4-32
 - physical address sizes 3-20
 - virtual-8086 tasks 16-10
- Parameter
 - passing, between 16- and 32-bit call gates . 17-7
 - translation, between 16- and 32-bit code segments 17-8
- PAUSE instruction 2-14
- PBi (performance monitoring/breakpoint pins) flags, DebugCtIIMSR MSR 15-21
- PC (pin control) flag, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors) 15-63
- PC0 and PC1 (pin control) fields, CESR MSR (Pentium processor) 15-67
- PCD pin (Pentium processor) 10-15
- PCD (page-level cache disable) flag
 - CR3 control register . 2-16, 10-14, 18-22, 18-32
 - page-directory entries . 9-8, 10-14, 10-15, 10-39
 - page-table entries . . . 3-26, 9-8, 10-14, 10-15, 10-39, 18-33
- PCE (performance monitoring counter enable) flag, CR4 control register . 2-17, 4-27, 15-31, 15-63
- PCE (performance-monitoring counter enable) flag, CR4 control register 18-21

- PDBR (see CR3 control register)
- PE (protection enable) flag, CR0 control register .
2-16, 4-2, 9-13, 9-14, 13-9
- PEBS (precise event-based sampling) facilities
 - availability of 15-51
 - description of 15-28, 15-50
 - DS save area 15-34
 - PEBS buffer 15-35, 15-51
 - PEBS records 15-34, 15-37
 - PEBS_UNAVAILABLE flag, IA32_MISC_ENABLE MSR. B-10
 - writing a PEBS interrupt service routine . . 15-19
 - writing a PEBS interrupt service routine . . 15-52
- PEBS_UNAVAILABLE flag, IA32_MISC_ENABLE MSR. 15-35, B-10
- Pentium 4 processor 1-1
 - compatibility with floating-point software . .
18-8
 - list of performance-monitoring events. . . . A-1
 - MSRs supported B-1, B-20
- Pentium II processor 1-1
- Pentium III processor 1-1
- Pentium M processor
 - MSRs supported by B-20
- Pentium Pro processor. 1-1
- Pentium processor 1-1
 - list of performance-monitoring events. . . . A-60
 - MSR supported by B-36
 - performance-monitoring counters. 15-65
- Pentium processors 18-8
- PerfCtr0 and PerfCtr1 MSRs (P6 family processors)
15-61, 15-63
- PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family
processors) 15-61, 15-62
- Performance events
 - at-retirement events (Pentium 4 processor) . .
A-28
 - non-retirement events (Pentium 4 processor) .
A-2
 - P6 family processors A-45
 - Pentium processor A-60
- Performance-monitoring counters
 - description of 15-24
 - events that can be counted (P6 family
processors) A-45
 - events that can be counted (Pentium 4
processor) A-1
 - events that can be counted (Pentium
processors) 15-68, A-60
 - interrupt. 8-2
 - introduction of in IA-32 processors 18-41
 - monitoring counter overflow (P6 family
processors) 15-65
 - overflow, monitoring (P6 family processors) . .
15-65
 - overview of 2-6
 - P6 family processors 15-61
- Pentium II processor 15-61
- Pentium Pro processor 15-61
- Pentium processor 15-65
 - reading 2-22, 15-63
 - setting up (P6 family processors) 15-62
 - software drivers for 15-64
 - starting and stopping. 15-64
- PG (paging) flag, CR0 control register . 2-13, 3-18,
3-26, 3-28, 3-34, 4-2, 9-13, 9-14, 13-9,
18-34
- PGE (page global enable) flag, CR4 control register
2-17, 3-27, 10-14, 18-21, 18-23
- PhysBase field, IA32_MTRR_PHYSBASEn MTRR
10-29
- Physical address extension
 - accessing full extended physical address space
3-31
 - introduction to 3-6
 - page-directory entries 3-31, 3-35
 - page-table entries 3-31
 - using PAE paging mechanism 3-28
 - using PSE-32 paging mechanism 3-34
- Physical address space
 - 4 GBytes 3-6
 - 64 GBytes 3-6
 - defined 3-1
 - description of. 3-6
 - mapped to a task. 6-19
- Physical addressing 2-5
- Physical destination mode, local APIC 8-27
- Physical memory
 - mapping of with fixed-range MTRRs. . . . 10-29
 - mapping of with variable-range MTRRs . 10-28
- PhysMask, IA32_MTRR_PHYSMASKn MTRR . .
10-30
- PM0/BP0 and PM1/BP1 (performance-monitor)
pins (Pentium processor) . 15-65, 15-67,
15-68
- Pointers
 - code-segment pointer size 17-5
 - limit checking. 4-29
 - validation 4-27
- POP instruction. 3-9
- POPF instruction 5-9, 15-10
- Precise event-based sampling (see PEBS)
- PREFETCHH instruction. 2-14, 10-4, 10-20
- Previous task link field, TSS 6-6, 6-16, 6-18
- Priority levels, APIC interrupts 8-34
- Privilege levels
 - checking when accessing data segments. 4-10
 - checking, for call gates 4-18
 - checking, when transferring program control
between code segments 4-12
 - description of. 4-7
 - protection rings 4-9
- Privileged instructions. 4-26
- Processor management
 - initialization 9-1

local APIC 8-1
 overview of 7-1
 snooping mechanism 7-9
 Processor ordering, description of 7-7
 Protected mode
 IDT initialization 9-12
 initialization for 9-11
 mixing 16-bit and 32-bit code modules 17-2
 mode switching 9-14
 PE flag, CR0 register 4-2
 switching to 4-2, 9-14
 system data structures required during
 initialization 9-11, 9-12
 Protection
 combining segment and page-level protection
 4-34
 disabling 4-2
 enabling 4-2
 flags used for page-level protection 4-2
 flags used for segment-level protection . . . 4-2
 of exception- and interrupt-handler procedures
 5-16
 overview of 4-1
 page level 4-2, 4-31, 4-33
 page level, overriding 4-33
 page-level protection flags 4-32
 read/write, page level 4-33
 segment level 4-2
 user/supervisor type 4-32
 Protection rings 4-9
 PS (page size) flag, page-table entries 3-27
 PSE (page size extension) flag, CR4 control
 register . . . 2-17, 3-18, 3-21, 3-22, 3-34,
 10-23, 18-22, 18-23
 PSE-36 feature flag, CPUID instruction . 3-19, 3-34
 PSE-36 page size extension 3-6
 Pseudo-infinity 18-11
 Pseudo-NaN 18-11
 Pseudo-zero 18-11
 PUSH instruction 18-7
 PUSHF instruction 5-9, 18-7
 PVI (protected-mode virtual interrupts) flag, CR4
 control register . . 2-9, 2-10, 2-16, 18-22
 PWT pin (Pentium processor) 10-15
 PWT (page-level write-through) flag
 CR3 control register 2-16, 10-14, 18-22, 18-32
 page-directory entries 9-8, 10-14, 10-39
 page-table entries 3-26, 9-8, 10-14, 10-39,
 18-33

Q

QNaN, compatibility, IA-32 processors 18-10

R

RDMSR instruction 2-23, 4-26, 15-14, 15-22,
 15-24, 15-31, 15-61, 15-63, 15-65, 18-5,
 18-39, 18-40
 RDPIC instruction 2-22, 4-26, 15-31, 15-61,
 15-63, 18-4, 18-21, 18-41
 RDTSC instruction 2-22, 4-26, 15-23, 18-5
 Read/write
 protection, page level 4-33
 rights, checking 4-28
 Real-address mode
 8086 emulation 16-1
 address translation in 16-3
 description of 16-1
 exceptions and interrupts 16-8
 IDT initialization 9-10
 IDT, changing base and limit of 16-6
 IDT, structure of 16-7
 IDT, use of 16-6
 initialization 9-10
 instructions supported 16-4
 interrupt and exception handling 16-6
 interrupts 16-8
 introduction to 2-6
 mode switching 9-14
 native 16-bit mode 17-1
 overview of 16-1
 registers supported 16-4
 switching to 9-15
 Related literature 1-7
 Replay events A-36
 Requested privilege level (see RPL)
 Reserved bits 1-4, 18-2
 RESET# pin 5-4, 18-19
 RESET# signal 2-22
 Restarting program or task, following an exception
 or interrupt 5-6
 Restricting addressable domain 4-32
 RET instruction 4-12, 4-13, 4-24, 17-7
 Returning
 from a called procedure 4-24
 from an interrupt or exception handler . . . 5-16
 RF (resume) flag, EFLAGS register . 2-9, 5-9, 15-1
 RPL
 description of 3-8, 4-9
 field, segment selector 4-2
 RSM instruction 2-22, 7-14, 13-1, 13-2, 13-3,
 13-12, 13-16, 18-5
 R/S# pin 5-4
 R/W (read/write) flag
 page-directory entry 4-2, 4-3, 4-33
 page-table entries 3-25
 page-table entry 4-2, 4-3, 4-33
 R/W0-R/W3 (read/write) fields, DR7 register . 15-5,
 18-24

S

S (descriptor type) flag, segment descriptor . 3-11, 3-13, 4-2, 4-6

SBB instruction 7-5

Segment descriptors

- access rights 4-27
- access rights, invalid values 18-24
- automatic bus locking while updating 7-4
- base address fields 3-11
- code type 4-3
- data type 4-3
- description of 2-3, 3-9
- DPL (descriptor privilege level) field . . 3-11, 4-2
- D/B (default operation size/default stack pointer size and/or upper bound) flag . . . 3-11, 4-5
- E (expansion direction) flag 4-2, 4-5
- G (granularity) flag 3-12, 4-2, 4-5
- limit field 4-2, 4-5
- loading 18-24
- P (segment-present) flag 3-11
- S (descriptor type) flag . . . 3-11, 3-13, 4-2, 4-6
- segment limit field 3-10
- system type 4-3
- tables 3-16
- TSS descriptor 6-7
- type field 3-11, 3-13, 4-2, 4-6
- type field, encoding 3-13, 3-15
- when P (segment-present) flag is clear . . 3-12

Segment limit

- checking 2-21
- field, segment descriptor 3-10

Segment not present exception (#NP) 3-11

Segment registers

- description of 3-8
- saved in TSS 6-6

Segment selectors

- description of 3-7
- index field 3-7
- null 4-7
- RPL field 3-8, 4-2
- TI (table indicator) flag 3-7

Segmented addressing 1-6

Segment-not-present exception (#NP) . . 5-36, 5-37

Segments

- basic flat model 3-3
- code type 3-13
- combining segment and page-level protection 4-34
- combining with paging 3-6
- data type 3-13
- defined 3-1
- disabling protection of 4-2
- enabling protection of 4-2
- mapping to pages 3-36
- multisegment usage model 3-5
- protected flat model 3-3
- segment-level protection 4-2
- segment-not-present exception 5-36, 5-37

- system 2-3
- types, checking access rights 4-27
- typing 4-6
- using 3-3
- wraparound 18-36

Self-modifying code, effect on caches 10-21

Serializing instructions 7-14, 18-19

SF (stack fault) flag, x87 FPU status word . . 18-9

SFENCE instruction . . 2-14, 7-8, 7-11, 7-12, 7-14

SGDT instruction 2-20, 3-17

Shutdown

- resulting from double fault 5-32
- resulting from out of IDT limit condition . . 5-32

SIDT instruction 2-20, 3-17, 5-12

SIMD floating-point exception (#XF) . . 2-18, 5-53, 5-54, 5-55, 9-9

SIMD floating-point exceptions

- description of 5-53, 12-5
- handler 12-2
- support for 2-18

Single-stepping

- breakpoint exception condition 15-10
- on branches 15-16
- on exceptions 15-16
- on interrupts 15-16
- TF (trap) flag, EFLAGS register 15-10

SLDT instruction 2-20

SLTR instruction 3-17

SMBASE

- default value 13-4
- relocation of 13-15

SMI handler

- description of 13-1
- execution environment for 13-9
- exiting from 13-3
- location in SMRAM 13-4

SMI interrupt 2-22, 8-5

- description of 13-1, 13-2
- priority 13-3
- switching to SMM 13-2

SMI# pin 5-4, 13-2, 13-16

SMM

- auto halt restart 13-14
- executing the HLT instruction in 13-15
- exiting from 13-3
- handling exceptions and interrupts 13-10
- introduction to 2-6
- I/O instruction restart 13-16
- native 16-bit mode 17-1
- overview of 13-1
- revision identifier 13-13
- revision identifier field 13-13
- switching to 13-3
- switching to from other operating modes . 13-2
- using x87 FPU in 13-12

SMRAM

- caching 13-8
- description of 13-1

- state save map 13-5
- structure of 13-4
- SMSW instruction. 2-20
- SNaN, compatibility, IA-32 processors 18-10, 18-17
- Snooping mechanism. 7-9, 10-5
- Software interrupts. 5-4
- Software-controlled bus locking. 7-4
- Split pages 18-18
- Spurious interrupt, local APIC 8-38
- SS register, saving on call to exception or interrupt handler. 5-15
- SSE extensions
 - checking for support for FXSAVE and FXRSTOR instructions 12-2
 - checking for with CPUID instruction 12-2
 - designing facilities for automatic saving of SSE state. 12-7
 - EM flag 2-15
 - emulation of 12-6
 - initialization 9-9
 - initializing 12-2
 - introduction of into the IA-32 architecture. 18-3
 - providing exception handlers for. 12-3, 12-5
 - providing operating system support for. 12-1
 - saving and restoring SSE and state 12-6
 - saving SSE state on task or context switches 12-7
 - SIMD Floating-point exception (#XF) 5-53
 - system programming 12-1
 - using TS flag to control saving of SSE state 12-8
- SSE feature flag, CPUID instruction 9-9, 12-2
- SSE2 extensions
 - checking for support for FXSAVE and FXRSTOR instructions 12-2
 - checking for with CPUID instruction 12-2
 - designing facilities for automatic saving of SSE2 state. 12-7
 - EM flag 2-15
 - emulation of 12-6
 - initialization 9-9
 - initializing 12-2
 - introduction of into the IA-32 architecture. 18-3
 - providing exception handlers for. 12-3, 12-5
 - providing operating system support for. 12-1
 - saving and restoring SSE and state 12-6
 - saving SSE2 state on task or context switches 12-7
 - SIMD Floating-point exception (#XF) 5-53
 - system programming 12-1
 - using TS flag to control saving of SSE2 state 12-8
- SSE2 feature flag, CPUID instruction 9-9, 12-2
- Stack fault exception (#SS) 5-38, 5-39
- Stack fault, x87 FPU 18-9, 18-16
- Stack pointers
 - privilege level 0, 1, and 2 stacks. 6-6
 - size of 3-12
- Stack segments
 - paging of 2-5
 - privilege level checks when loading the SS register. 4-12
 - size of stack pointer 3-12
- Stack switching
 - inter-privilege level calls 4-21
 - masking exceptions and interrupts when switching stacks. 5-10
 - on call to exception or interrupt handler 5-15
- Stack-fault exception (#SS) 18-37
- Stacks
 - error code pushes 18-35
 - faults 5-38, 5-39
 - for privilege levels 0, 1, and 2 4-22
 - interlevel RET/IRET from a 16-bit interrupt or call gate 18-35
 - management of control transfers for 16- and 32-bit procedure calls 17-5
 - operation on pushes and pops 18-34
 - pointers to in TSS 6-6
 - stack switching 4-21
 - usage on call to exception or interrupt handler 18-35
- Stepping information, following processor
 - initialization or reset 9-5
- STI instruction. 5-9
- Store buffer
 - caching terminology 10-5
 - characteristics of 10-3
 - description of. 10-4, 10-23
 - in IA-32 processors. 18-37
 - location of 10-1
 - operation of 10-23
- STPCLK# pin 5-4, 15-23
- STR instruction. 2-20, 3-17, 6-9
- Strong uncached (UC) memory type
 - description of. 10-5
 - effect on memory ordering 7-12
 - use of 9-9, 10-9
- SUB instruction. 7-5
- Supervisor mode
 - description of. 4-32
 - U/S (user/supervisor) flag 4-32
- SVR (spurious-interrupt vector register), local APIC 8-10, 18-28
- SYSENTER instruction. 3-9, 4-12, 4-13, 4-25
- SYSENTER_CS_MSR 4-25
- SYSENTER_EIP_MSR 4-25
- SYSENTER_ESP_MSR. 4-25
- SYSEXIT instruction. 3-9, 4-12, 4-13, 4-25
- System
 - architecture 2-1, 2-2
 - data structures 2-2
 - instructions 2-6, 2-18
 - registers, introduction to 2-5
 - segment descriptor, layout of 4-3
 - segments, paging of 2-5

System programming
MMX technology 11-1
SSE and SSE2 extensions 12-1
System-management mode (see SMM)

T

T (debug trap) flag, TSS 6-6, 15-1
Task gates
descriptor 6-9
executing a task 6-3
handling a virtual-8086 mode interrupt or
exception through 16-20
in IDT 5-12
introduction to 2-3, 2-4
layout of 5-12
referencing of TSS descriptor 5-18
Task management 6-1
data structures 6-4
mechanism, description of 6-3
Task register 3-17
description of 2-11, 6-1, 6-8
initializing 9-13
introduction to 2-5
Task state segment (see TSS)
Task switches
saving MMX state on 11-5
Task switching
description of 6-3
exception condition 15-10
operation 6-13
preventing recursive task switching 6-17
saving SSE and SSE2 state on task or context
switches 12-7
T (debug trap) flag 6-6
Tasks
address space 6-18
description of 6-1
exception-handler task 5-14
executing 6-3
Intel 286 processor tasks 18-41
interrupt-handler task 5-14
interrupts and exceptions 5-18
linking 6-16
logical address space 6-20
management 6-1
mapping to linear and physical address spaces
6-19
restart following an exception or interrupt 5-6
state (context) 6-2, 6-3
structure 6-1
switching 6-3
task management data structures 6-4
Test registers 18-25
TF (trap) flag, EFLAGS register 2-8, 5-17, 13-10,
15-1, 15-10, 15-12, 15-16, 15-21, 16-6,
16-26
Thermal monitoring

automatic 13-20
catastrophic shutdown detector 13-18, 13-20
detection of thermal monitor and software
controlled clock modulation facilities 13-24
overview of 13-18, 13-19
software controlled clock modulation 13-23
stop clock mechanism 13-19
Thermal sensor
interrupt 8-2
TI (table indicator) flag, segment selector 3-7
Timer, local APIC 8-20
Timestamp Counter 15-52
Time-stamp counter
description of 15-23
reading 2-22
software drivers for 15-64
TLB shutdown 7-13
TLBs
description of 3-18, 10-1, 10-4
flushing 10-22
invalidating (flushing) 2-21
relationship to PGE flag 3-27, 18-23
relationship to PSE flag 3-22, 10-23
TM flag, CPUID instruction 13-24
TMR (Trigger Mode Register), local APIC 8-36
TR (trace message enable) flag, DebugCtlMSR
MSR 15-13, 15-21
Trace cache 10-3
Transcendental instruction accuracy 18-9, 18-18
Translation lookaside buffer (see TLB)
Trap gates
difference between interrupt and trap gates 5-17
for 16-bit and 32-bit code modules 17-2
handling a virtual-8086 mode interrupt or
exception through 16-17
in IDT 5-12
introduction to 2-3, 2-4
layout of 5-12
Traps
description of 5-6
restarting a program or task after 5-7
TS (task switched) flag, CR0 control register 2-14,
2-20, 5-29, 11-1, 12-3, 12-8
TSD (time-stamp counter disable) flag, CR4 control
register 2-16, 4-27, 15-23, 18-22
TSS
16-bit TSS, structure of 6-21
32-bit TSS, structure of 6-4
CR3 control register (PDBR) 6-5, 6-19
description of 2-3, 2-4, 6-1, 6-4
EFLAGS register 6-6
EIP 6-6
executing a task 6-3
floating-point save area 18-14
general-purpose registers 6-5
initialization for multitasking 9-13
invalid TSS exception 5-34, 5-35

- I/O map base address field. 6-6, 18-30
- I/O permission bit map 6-6
- LDT segment selector field. 6-6, 6-18
- link field. 5-18
- order of reads/writes to. 18-30
- page-directory base address (PDBR). 3-23
- pointed to by task-gate descriptor. 6-9
- previous task link field. 6-6, 6-16, 6-18
- privilege-level 0, 1, and 2 stacks. 4-22
- referenced by task gate 5-18
- segment registers 6-6
- T (debug trap) flag 6-6
- task register 6-8
- using 16-bit TSSs in a 32-bit environment 18-30
- virtual-mode extensions 18-30
- TSS descriptor
 - B (busy) flag 6-7
 - initialization for multitasking 9-13
 - structure of 6-7
- TSS segment selector
 - field, task-gate descriptor 6-9
 - writes 18-30
- Type
 - checking 4-6
 - field, IA32_MTRR_DEF_TYPE MSR 10-27
 - field, IA32_MTRR_PHYSBASEn MTRR. . . 10-29
 - field, segment descriptor. 3-11, 3-13, 3-15, 4-2, 4-6
 - of segment 4-6
- U**
 - UC- (uncacheable) memory type 10-6
 - UD2 instruction. 18-4
 - Uncached (UC-) memory type 10-9
 - Uncached (UC) memory type (see Strong uncached (UC) memory type)
 - Undefined opcodes 18-6
 - Unit mask field, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors) 15-62
 - Un-normal number 18-11
 - User mode
 - description of 4-32
 - U/S (user/supervisor) flag 4-32
 - User-defined interrupts. 5-2, 5-56
 - USR (user mode) flag, PerfEvtSel0 and PerfEvtSel1 MSRs (P6 family processors) 15-62
 - U/S (user/supervisor) flag
 - page-directory entry 4-2, 4-3, 4-32
 - page-table entries. 3-26, 16-11
 - page-table entry 4-2, 4-3, 4-32
- V**
 - V (valid) flag, IA32_MTRR_PHYSMASKn MTRR. 10-30
 - Variable-range MTRRs, description of. 10-28
 - VCNT (variable range registers count) field, IA32_MTRRCAP MSR 10-26
 - Vector (see Interrupt vector)
 - Vectors
 - exceptions 5-2
 - interrupts 5-2
 - reserved 8-34
 - VERR instruction 2-21, 4-28
 - VERW instruction 2-21, 4-28
 - VIF (virtual interrupt) flag, EFLAGS register. . . 2-9, 2-10, 18-6, 18-7
 - VIP (virtual interrupt pending) flag, EFLAGS register 2-9, 2-10, 18-6, 18-7
 - Virtual memory 2-5, 3-1, 3-2, 3-17
 - Virtual-8086 mode
 - 8086 emulation 16-1
 - description of. 16-7
 - emulating 8086 operating system calls. . . 16-25
 - enabling. 16-9
 - entering 16-11
 - exception and interrupt handling, overview. . . 16-15
 - exceptions and interrupts, handling through a task gate 16-19
 - exceptions and interrupts, handling through a trap or interrupt gate 16-17
 - handling exceptions and interrupts through a task gate 16-20
 - interrupts 16-8
 - introduction to 2-7
 - IOPL sensitive instructions 16-14
 - I/O-port-mapped I/O 16-14
 - leaving. 16-13
 - memory mapped I/O 16-15
 - native 16-bit mode. 17-1
 - overview of 16-1
 - paging of virtual-8086 tasks 16-10
 - protection within a virtual-8086 task 16-11
 - special I/O buffers 16-15
 - structure of a virtual-8086 task 16-9
 - virtual I/O. 16-14
 - VM flag, EFLAGS register. 2-9
 - Virtual-8086 tasks
 - paging of 16-10
 - protection within 16-11
 - structure of 16-9
 - VM (virtual-8086 mode) flag, EFLAGS register 2-7, 2-9
 - VME (virtual-8086 mode extensions) flag, CR4 control register . . 2-9, 2-10, 2-16, 18-22
- W**
 - WAIT/FWAIT instructions 5-29, 18-8, 18-18, 18-19
 - WB (write back) memory type 10-6, 10-9
 - WB (write-back) pin (Pentium processor). . . 10-15
 - WBINVD instruction 2-21, 4-26, 7-14, 10-18, 10-19, 10-20, 18-4

WB/WT# pins 10-15
 WC buffer (see Write combining (WC) buffer)
 WC (write combining)
 flag, IA32_MTRRCAP MSR 10-26
 memory type 10-6, 10-9
 WP (write protected) memory type 10-7
 WP (write protect) flag, CR0 control register . 2-13,
 4-33, 18-22
 Write
 forwarding 7-9
 hit 10-4
 Write back (WB) memory type 7-12
 Write combining (WC) buffer 10-3, 10-7
 Write-back caching 10-5
 WRMSR instruction 2-22, 2-23, 4-26, 7-14, 15-12,
 15-20, 15-24, 15-31, 15-61, 15-63,
 15-65, 18-5, 18-39, 18-40
 WT (write through) memory type 10-6, 10-9
 WT# (write-through) pin (Pentium processor) 10-15

X

x87 FPU
 compatibility with IA-32 x87 FPUs and math
 coprocessors 18-8
 configuring the x87 FPU environment 9-6
 device-not-available exception 5-29, 5-30
 effect of MMX instructions on pending x87
 floating-point exceptions 11-6
 effects of MMX instructions on x87 FPU state .
 11-3
 effects of MMX, x87 FPU, FXSAVE, and
 FXRSTOR instructions on x87 FPU tag word
 11-3
 error signals 18-13
 initialization 9-6
 instruction synchronization 18-19
 register stack, aliasing with MMX registers. 11-2
 setting up for software emulation of x87 FPU
 functions 9-7
 using in SMM 13-12
 using TS flag to control saving of x87 FPU state
 12-8
 x87 floating-point error exception (#MF) . . 5-47
 x87 FPU control word
 compatibility, IA-32 processors 18-9
 x87 FPU floating-point error exception (#MF) 5-47,
 5-48
 x87 FPU status word
 condition code flags 18-9
 x87 FPU tag word 18-10
 XADD instruction 7-4, 18-4
 xAPIC
 communication through system bus 8-5
 determining lowest priority processor 8-30
 interrupt control register 8-25
 introduction to 8-5
 message passing protocol on system bus . 8-39

 new features 18-29
 spurious vector 8-38
 XCHG instruction 7-4, 7-11
 XMM registers, saving 12-6
 XOR instruction 7-5

Z

ZF flag, EFLAGS register 4-28



INTEL SALES OFFICES

ASIA PACIFIC

Australia

Intel Corp.
Level 2
448 St Kilda Road
Melbourne VIC
3004
Australia
Fax:613-9862 5599

China

Intel Corp.
Rm 709, Shaanxi
Zhongda Int'l Bldg
No.30 Nandajie Street
Xian AX710002
China
Fax:(86 29) 7203356

Intel Corp.
Rm 2710, Metropolis
Tower
68 Zourong Rd
Chongqing CQ
400015
China

Intel Corp.
C1, 15 Flr, Fujian
Oriental Hotel
No. 96 East Street
Fuzhou FJ
350001
China

Intel Corp.
Rm 5803 CITIC Plaza
233 Tianhe Rd
Guangzhou GD
510613
China

Intel Corp.
Rm 1003, Orient Plaza
No. 235 Huayuan Street
Nangang District
Harbin HL
150001
China

Intel Corp.
Rm 1751 World Trade
Center, No 2
Han Zhong Rd
Nanjing JS
210009
China

Intel Corp.
Hua Xin International
Tower
215 Qing Nian St.
ShenYang LN
110015
China

Intel Corp.
Suite 1128 CITIC Plaza
Jinan
150 Luo Yuan St.
Jinan SN
China

Intel Corp.
Suite 412, Holiday Inn
Crowne Plaza
31, Zong Fu Street
Chengdu SU
610041
China
Fax:86-28-6785965

Intel Corp.
Room 0724, White Rose
Hotel
No 750, MinZhu Road
WuChang District
Wuhan UB
430071
China

India

Intel Corp.
Paharpur Business
Centre
21 Nehru Place
New Delhi DH
110019
India

Intel Corp.
Hotel Rang Sharda, 6th
Floor
Bandra Reclamation
Mumbai MH
400050
India
Fax:91-22-6415578

Intel Corp.
DBS Corporate Club
31A Cathedral Garden
Road
Chennai TD
600034
India

Intel Corp.
DBS Corporate Club
2nd Floor, 8 A.A.C. Bose
Road
Calcutta WB
700017
India

Japan

Intel Corp.
Kokusai Bldg 5F, 3-1-1,
Marunouchi
Chiyoda-Ku, Tokyo
1000005
Japan

Intel Corp.
2-4-1 Terauchi
Toyonaka-Shi
Osaka
5600872
Japan

Malaysia

Intel Corp.
Lot 102 1/F Block A
Wisma Semantan
12 Jalan Gelenggang
Damansara Heights
Kuala Lumpur SL
50490
Malaysia

Thailand

Intel Corp.
87 M. Thai Tower, 9th Fl.
All Seasons Place,
Wireless Road
Lumpini, Patumwan
Bangkok
10330
Thailand

Viet Nam

Intel Corp.
Hanoi Tung Shing
Square, Ste #1106
2 Ngo Quyen St
Hoan Kiem District
Hanoi
Viet Nam

EUROPE & AFRICA

Belgium

Intel Corp.
Woluwelaan 158
Diegem
1831
Belgium

Czech Rep

Intel Corp.
Nahorni 14
Brno
61600
Czech Rep

Denmark

Intel Corp.
Soelodden 13
Maaloev
DK2760
Denmark

Germany

Intel Corp.
Sandstrasse 4
Aichner
86551
Germany

Intel Corp.
Dr Weyerstrasse 2
Juelich
52428
Germany

Intel Corp.
Buchenweg 4
Wildberg
72218
Germany

Intel Corp.
Kemnader Strasse 137
Bochum
44797
Germany

Intel Corp.
Klaus-Schaefer Strasse
16-18
Erfstadt NW
50374
Germany

Intel Corp.
Heldmanskamp 37
Lemgo NW
32657
Germany

Italy

Intel Corp Italia Spa
Milanofiori Palazzo E/4
Assago
Milan
20094
Italy
Fax:39-02-57501221

Netherland

Intel Corp.
Strausslaan 31
Heesch
5384CW
Netherland

Poland

Intel Poland
Developments, Inc
Jerozolimskie Business
Park
Jerozolimskie 146c
Warsaw
2305
Poland
Fax:+48-22-570 81 40

Portugal

Intel Corp.
PO Box 20
Alcabideche
2765
Portugal

Spain

Intel Corp.
Calle Rioja, 9
Bajo F Izquierda
Madrid
28042
Spain

South Africa

Intel SA Corporation
Bldg 14, South Wing,
2nd Floor
Uplands, The Woodlands
Western Services Road
Woodmead
2052
Sth Africa
Fax:+27 11 806 4549

Intel Corp.
19 Summit Place,
Halfway House
Cnr 5th and Harry
Galaun Streets
Midrad
1685
Sth Africa

United Kingdom

Intel Corp.
The Manse
Silver Lane
Needingworth CAMBS
PE274SL
UK

Intel Corp.
2 Cameron Close
Long Melford SUFFK
CO109TS
UK

Israel

Intel Corp.
MTM Industrial Center,
P.O.Box 498
Haifa
31000
Israel
Fax:972-4-8655444

LATIN AMERICA &

CANADA

Argentina

Intel Corp.
Dock IV - Bldg 3 - Floor 3
Olga Cossentini 240
Buenos Aires
C1107BVA
Argentina

Brazil

Intel Corp.
Rua Carlos Gomez
111/403
Porto Alegre
90480-003
Brazil

Intel Corp.
Av. Dr. Chucuri Zaidan
940 - 10th Floor
Sua Paulo
04583-904
Brazil

Intel Corp.
Av. Rio Branco,
1 - Sala 1804
Rio de Janeiro
20090-003
Brazil

Columbia

Intel Corp.
Carrera 7 No. 71021
Torre B, Oficina 603
Santefe de Bogota
Columbia

Mexico

Intel Corp.
Av. Mexico No. 2798-9B,
S.H.
Guadalajara
44680
Mexico

Intel Corp.
Torre Esmeralda II,
7th Floor
Blvd. Manuel Avila
Comacho #36
Mexico Cith DF
11000
Mexico

Intel Corp.
Piso 19, Suite 4
Av. Batallon de San
Patricio No 111
Monterrey, Nuevo le
66269
Mexico

Canada

Intel Corp.
168 Bonis Ave, Suite 202
Scarborough
M1T3V6
Canada
Fax:416-335-7695

Intel Corp.
3901 Highway #7,
Suite 403
Vaughan
L4L 8L5
Canada
Fax:905-856-8868

Intel Corp.
999 CANADA PLACE,
Suite 404, #11
Vancouver BC
V6C 3E2
Canada
Fax:604-844-2813

Intel Corp.
2650 Queensview Drive,
Suite 250
Ottawa ON
K2B 8H6
Canada
Fax:613-820-5936

Intel Corp.
190 Attwell Drive,
Suite 500
Rexdale ON
M9W 6H8
Canada
Fax:416-675-2438

Intel Corp.
171 St. Clair Ave. E,
Suite 6
Toronto ON
Canada

Intel Corp.
1033 Oak Meadow Road
Oakville ON
L6M 1J6
Canada

USA
California
Intel Corp.
551 Lundy Place
Milpitas CA
95035-6833
USA
Fax:408-451-8266

Intel Corp.
1551 N. Tustin Avenue,
Suite 800
Santa Ana CA
92705
USA
Fax:714-541-9157

Intel Corp.
Executive Center del Mar
12230 El Camino Real
Suite 140
San Diego CA
92130
USA
Fax:858-794-5805

Intel Corp.
1960 E. Grand Avenue,
Suite 150
El Segundo CA
90245
USA
Fax:310-640-7133

Intel Corp.
23120 Alicia Parkway,
Suite 215
Mission Viejo CA
92692
USA
Fax:949-586-9499

Intel Corp.
30851 Agoura Road
Suite 202
Agoura Hills CA
91301
USA
Fax:818-874-1166

Intel Corp.
28202 Cabot Road,
Suite #363 & #371
Laguna Niguel CA
92677
USA

Intel Corp.
657 S Cendros Avenue
Solana Beach CA
90075
USA

Intel Corp.
43769 Abeloe Terrace
Fremont CA
94539
USA

Intel Corp.
1721 Warburton, #6
Santa Clara CA
95050
USA

Colorado
Intel Corp.
600 S. Cherry Street,
Suite 700
Denver CO
80222
USA
Fax:303-322-8670

Connecticut
Intel Corp.
Lee Farm Corporate Pk
83 Wooster Heights
Road
Danbury CT
6810
USA
Fax:203-778-2168

Florida
Intel Corp.
7777 Glades Road
Suite 310B
Boca Raton FL
33434
USA
Fax:813-367-5452

Georgia
Intel Corp.
20 Technology Park,
Suite 150
Norcross GA
30092
USA
Fax:770-448-0875

Intel Corp.
Three Northwinds Center
2500 Northwinds
Parkway, 4th Floor
Alpharetta GA
30092
USA
Fax:770-663-6354

Idaho
Intel Corp.
910 W. Main Street, Suite
236
Boise ID
83702
USA
Fax:208-331-2295

Illinois
Intel Corp.
425 N. Martingale Road
Suite 1500
Schaumburg IL
60173
USA
Fax:847-605-9762

Intel Corp.
999 Plaza Drive
Suite 360
Schaumburg IL
60173
USA

Intel Corp.
551 Arlington Lane
South Elgin IL
60177
USA

Indiana
Intel Corp.
9465 Counselors Row,
Suite 200
Indianapolis IN
46240
USA
Fax:317-805-4939

Massachusetts
Intel Corp.
125 Nagog Park
Acton MA
01720
USA
Fax:978-266-3867

Intel Corp.
59 Composit Way
suite 202
Lowell MA
01851
USA

Intel Corp.
800 South Street,
Suite 100
Waltham MA
02154
USA

Maryland
Intel Corp.
131 National Business
Parkway, Suite 200
Annapolis Junction MD
20701
USA
Fax:301-206-3678

Michigan
Intel Corp.
32255 Northwestern
Hwy., Suite 212
Farmington Hills MI
48334
USA
Fax:248-851-8770

Minnesota
Intel Corp.
3600 W 80Th St
Suite 450
Bloomington MN
55431
USA
Fax:952-831-6497

North Carolina
Intel Corp.
2000 CentreGreen Way,
Suite 190
Cary NC
27513
USA
Fax:919-678-2818

New Hampshire
Intel Corp.
7 Suffolk Park
Nashua NH
03063
USA

New Jersey
Intel Corp.
90 Woodbridge Center
Dr, Suite. 240
Woodbridge NJ
07095
USA
Fax:732-602-0096

New York
Intel Corp.
628 Crosskeys Office Pk
Fairport NY
14450
USA
Fax:716-223-2561

Intel Corp.
888 Veterans Memorial
Highway
Suite 530
Hauppauge NY
11788
USA
Fax:516-234-5093

Ohio
Intel Corp.
3401 Park Center Drive
Suite 220
Dayton OH
45414
USA
Fax:937-890-8658

Intel Corp.
56 Milford Drive
Suite 205
Hudson OH
44236
USA
Fax:216-528-1026

Oregon
Intel Corp.
15254 NW Greenbrier
Parkway, Building B
Beaverton OR
97006
USA
Fax:503-645-8181

Pennsylvania
Intel Corp.
925 Harvest Drive
Suite 200
Blue Bell PA
19422
USA
Fax:215-641-0785

Intel Corp.
7500 Brooktree
Suite 213
Wexford PA
15090
USA
Fax:714-541-9157

Texas
Intel Corp.
5000 Quorum Drive,
Suite 750
Dallas TX
75240
USA
Fax:972-233-1325

Intel Corp.
20445 State Highway
249, Suite 300
Houston TX
77070
USA
Fax:281-376-2891

Intel Corp.
8911 Capital of Texas
Hwy, Suite 4230
Austin TX
78759
USA
Fax:512-338-9335

Intel Corp.
7739 La Verdura Drive
Dallas TX
75248
USA

Intel Corp.
77269 La Cabeza Drive
Dallas TX
75249
USA

Intel Corp.
3307 Northland Drive
Austin TX
78731
USA

Intel Corp.
15190 Prestonwood
Blvd. #925
Dallas TX
75248
USA
Intel Corp.

Washington
Intel Corp.
2800 156Th Ave. SE
Suite 105
Bellevue WA
98007
USA
Fax:425-746-4495

Intel Corp.
550 Kirkland Way
Suite 200
Kirkland WA
98033
USA

Wisconsin
Intel Corp.
405 Forest Street
Suites 109/112
Oconomowoc WI
53066
USA